AI 系统性能工程

优化 GPU、CUDA 与 PyTorch 的模型训练与推理工作负载

Chris Fregly 著

中文原生版

NATIVE CHINESE EDITION 2026

AI 系统性能工程

优化 GPU、CUDA 与 PyTorch 的

模型训练与推理工作负载

Chris Fregly 著

中文原生版

据 O'Reilly Media 2025 年英文第一版译出

AI 系统性能工程

优化 GPU、CUDA 与 PyTorch 的模型训练与推理工作负载

Chris Fregly 著

AI Systems Performance Engineering 利用 GPU、CUDA 与 PyTorch 优化模型训练与推理工作负载

979-8-341-62778-9

AI Systems Performance Engineering 作者：Chris Fregly

Copyright © 2026 Flux Capacitor, LLC. 保留所有权利。

出版发行：O'Reilly Media, Inc.，地址：141 Stony Circle, Suite 195, Santa Rosa, CA 95401。

O'Reilly 图书可用于教育、商业或销售促销用途。本书的大多数版本都提供在线版（https://oreilly.com）。了解更多信息，请联系我们的企业/机构销售部：800-998-9938 或 corporate@oreilly.com。

策划编辑：Nicole Butterfield 开发编辑：Angela Rufino 制作编辑：Kristen Brown 文字编辑：nSight, Inc. 校对：Piper Content Partners

索引编纂：Sue Klefstad 封面设计：Susan Brown 封面插画：José Marzan Jr. 内文设计：David Futato 内文插画：Kate Dullea

2025 年 11 月：第一版

第一版修订历史 2025-11-11：首次发布

版本发布详情请参见 http://oreilly.com/catalog/errata.csp?isbn=9798341627789。

O'Reilly 徽标是 O'Reilly Media, Inc. 的注册商标。AI Systems Performance Engineering、封面图像及相关商业装帧设计均为 O'Reilly Media, Inc. 的商标。

本书所表达的观点仅代表作者本人，不代表出版社。出版社与作者已尽最大努力确保本书所载信息与说明的准确，但对任何错误或遗漏概不承担责任，包括但不限于因使用或信赖本书而导致的损害。使用本书所载信息与说明的风险由读者自行承担。如果本书包含或描述的任何代码示例或其他技术受开源许可证或他人知识产权的约束，读者应自行确保其使用符合此类许可证和/或权利。

本书仅供学习与研究使用。

赞誉

AI 系统正随着高强度的研发飞速演进，其中充斥着层层叠叠的复杂性。Chris 将这种复杂性逐一拆解，并以人人都能受益的方式讲透，为行业提供了一部足以立下多年标杆的独特参考书。

——Chris Lattner，Modular 公司 CEO

理解大语言模型的工作原理只是起点。通过模型与硬件优化把它们的价值发挥到极致，才是优秀实践者与卓越实践者之间的分野。Chris 的这本书正是一本你所缺的参考手册，它告诉你如何做到。

——Sebastian Raschka，博士，ML/AI 研究者，畅销书《从零构建大语言模型》（Build a Large Language Model from Scratch，Manning，2024）作者

这本书的覆盖面大得惊人——CUDA 内核、分布式训练、PyTorch 编译器内部机制、分离式推理，全部汇集于一册。Chris 把那些通常散落在成千上万篇博客与论文中的内容整合到了一起。这是一部出色的 ML 系统百科全书。

——Mark Saroufim，Meta PyTorch 工程师，GPU MODE 社区创始人

在大规模场景下驯服 AI 系统，是我们这个时代的巨大挑战之一。Chris Fregly 为这一新前沿写就了权威的实战指南。他巧妙地把从硅片到应用的各个环节串联起来，提供了每一位 AI 工程师把原始算力转化为高效、高性能模型所需的全栈智慧。

——Harsh Banwait，CoreWeave 产品总监

对于任何希望潜心钻研现代 ML 系统工程的人来说，这本书都是一份非凡的资源。它对 vLLM 和 llm-d 这类前沿项目细节的深入剖析，展现了对推理优化技术和开源软件变革力量的深刻理解。

——Michael Goin，Red Hat vLLM 维护者、首席工程师

对于拒绝满足于默认性能的 AI 工程师而言，这是一本权威指南。无论你是在调优 CUDA 内核、扩展 LLM 推理规模，还是在多 GPU 上编排 AI 智能体，这本书都为你提供了精准的“手术刀式”工具和系统级的清晰视角，助你高效运行现代 AI 工作负载。读它、用它，你将重新定义自己构建、扩展和思考现代 AI

系统的方式。AI Systems Performance Engineering 就是开启 AI 性能之门的万能钥匙。

——Arpitha Srinivas，全球领先的高性能与高能效 AI 服务器制造商的 AI 系统性能工程师

AI Systems Performance Engineering 出色地把 GPU 硬件架构与现代 AI 工作负载优化联系在一起，将严谨的底层调优与高层的系统设计融会贯通。本书涵盖 GPU 内存带宽优化、KV 缓存管理、批处理策略与多 GPU 推理，传递出在真实生产环境中淬炼而成的洞见。Chris 把 Nsight 剖析、Transformer 注意力调优、分布式扩展等复杂主题讲得清清楚楚，使本书成为横跨硬件、系统与 AI 三大领域的性能工程师的必读之作。

——Amer Ather，Netflix 云与机器学习性能工程师

这是介绍如何构建当代 AI 系统的最全面、最与时俱进的一本指南。每一位 AI/ML 开发者与从业者都值得一读。

——Chaim Rand，AI/ML 算法工程师

凡与 AI 性能相关的问题，这本参考书都堪称首选。Chris 的这本新书内容充实，在我日常优化与调优 AI 工作负载的工作中帮了大忙。它覆盖了当下所有 AI 系统性能问题——并且给出的解决方案，对每一家想把 AI 投入生产的企业来说都弥足珍贵。

——Antje Barth，Amazon AGI 技术成员

在这个领域，没有哪本书能与 AI Systems Performance Engineering 相提并论。每一章都展现出深厚的专业功底，单独成书也绰绰有余。书中的内容既新鲜又“常青”，读起来毫不费力。

——Suman Debnath，Anyscale 机器学习系统工程师

一部令人叹服的力作，对于从事最新 AI 应用的性能工程师而言，是必不可少的读物。

——Adrian Cockcroft，OrionX.net（曾任 Netflix 云基础设施负责人、AWS 工程副总裁）

这正是我一直在等的那本书。它把 AI 系统性能工程这个散乱、浩瀚、日新月异的领域，汇聚成一份清晰而现代的资源。

——Madison Kanna，Baseten AI 工程师

随着 AI 日益成为现代计算的基石，理解加速器架构及其生态系统已不再是可选项，而是战略上的必然要求。Chris 把极深的技术造诣提炼成清晰易懂的叙述。对于在超大规模环境下从事工程或领导工作的人来说，这本书是当之无愧的起点。

——Omer Zaki，Oracle 公司 AI 基础设施副总裁

今天的 AI 是一道系统问题：必须让软件与硬件结构协同优化，才能达到巅峰性能。Chris 为你揭开帷幕，展示现代 AI 工作负载赖以运行的 AI 软硬件栈的各个层次。

——Abdul Dakkak，Modular 公司 GenAI 负责人

前言

在旧金山充满活力的街头，创新就像美国 101 号公路上川流不息的自动驾驶车流一样司空见惯，我们被一个精彩绝伦的人工智能世界所环绕。AI 的飞速进步正在重新定义我们日常生活的方方面面。过去 20 年里，我们先后经历了推荐引擎（2000 年代）、AI 助手（2010 年代）和全自动驾驶汽车（2020 年代）。2030 年代将更加令人期待——AI 正以前所未有的速度演进，并将产生巨大的社会影响。

我之所以踏入这个瞬息万变的 AI 系统性能工程领域，最初源于一种好奇心：想弄清楚驱动这些复杂系统与重大应用场景的尖端硬件、高度优化的软件和巧妙算法之间那微妙的平衡与协同设计。这一认识促使我深入“全栈”AI 性能工程的天地。我想弄明白处理器、内存架构、网络互连、操作系统和软件框架等众多组件如何和谐地协同工作。这些交互的复杂性恰恰构成了挑战——以及机遇——激发了我深入探索这一独特技术组合的渴望。

这本书是我多年来作为一线 ML 与 AI 性能工程师不断探索的结晶。我把它写给那些渴望在各个层面理解 AI 系统性能内在机制的工程师、研究者、从业者和爱好者。读者可能正在构建 AI 应用、优化神经网络训练策略，或者设计和管理可扩展的推理服务器，也可能单纯是着迷于现代 AI 系统的运作机理。总而言之，本书提供了贯通理论与实践、横跨多个学科的洞见。

阅读本书的读者很可能已经具备神经网络的基础知识，并对 Python 和机器学习有初步了解。不过，即便缺少这些基础，一位怀有好奇心的读者也能顺着本书以第一性原理为根基、贯穿硬件、软件与算法的多维协同设计性能叙事读下去。我保证，书中有适合每一种读者的内容——而且我敢说，每位读者都会在这些书页中学到一些新东西。

在各章中，我们将考察硬件架构的演进，深究软件优化的种种细节，并探索一些真实世界的案例研究，借以凸显构建高性能且高性价比 AI 系统的模式与最佳实践。每一节都循序渐进，从基础概念一路覆盖到高级应用。

本书使用的惯例

本书使用以下排版惯例：

斜体　表示新术语、URL、电子邮件地址、文件名和文件扩展名。

等宽字体　用于程序清单，也在段落中指代程序元素，例如变量名或函数名、数据库、数据类型、环境变量、语句和关键字。

等宽粗体　表示应由用户原样键入的命令或其他文本。

等宽斜体　表示应由用户提供的值或由上下文确定的值来替换的文本。

此元素表示一条提示或建议。

此元素表示一般性说明。

使用代码示例

补充材料（代码示例、练习等）可从 https://github.com/cfregly/ai-performance-engineering 下载。

如果你对代码示例有技术问题或在使用中遇到困难，请发送电子邮件至 support@oreilly.com。

本书旨在帮助你把工作做好。总体而言，如果本书附有示例代码，你可以在自己的程序和文档中使用它。除非你要复制的是代码中的相当大一部分，否则无须联系我们获取许可。例如，编写一个用到了本书若干段代码的程序并不需要许可；销售或分发 O'Reilly 图书中的示例代码则确实需要许可。通过引用本书并引用示例代码来回答问题不需要许可；而把本书的大量示例代码纳入你产品的文档中，则确实需要许可。

我们感谢署名，但通常并不要求。署名通常包括书名、作者、出版社和 ISBN。例如："AI Systems Performance Engineering, by Chris Fregly (O'Reilly). Copyright 2026 Flux Capacitor, LLC, 979-8-341-62778-9."

如果你觉得自己对代码示例的使用超出了合理使用或上述许可的范围，请随时通过 permissions@oreilly.com 与我们联系。

O'Reilly 在线学习

40 多年来，O'Reilly Media 一直提供技术与商业培训、知识与洞见，帮助企业取得成功。

我们独特的专家与创新者网络通过图书、文章和我们的在线学习平台分享知识与专长。O'Reilly 在线学习平台让你可以按需访问实时培训课程、深入的学习路径、交互式编程环境，以及来自 O'Reilly 和 200 多家其他出版商的海量文字与视频资源。欲了解更多信息，请访问 https://oreilly.com。

如何联系我们

请将关于本书的意见和问题致函出版社：

O'Reilly Media, Inc. 141 Stony Circle, Suite 195 Santa Rosa, CA 95401 800-889-8969（美国或加拿大境内）707-827-7019（国际或本地）707-829-0104（传真）support@oreilly.com https://oreilly.com/about/contact.html

我们为本书设有专属网页，其中列出了勘误表以及更多补充信息。访问地址：https://oreil.ly/AI-SysPerfEng。

有关我们图书和课程的新闻与信息，请访问 https://oreilly.com。

在 LinkedIn 上关注我们：https://linkedin.com/company/oreilly-media。

在 YouTube 上观看我们：https://youtube.com/oreillymedia。

致谢

尽管本书由我独著，但它建立在许多毕生致力于这些主题的从业者与学者的工作之上。我深深感激那些以自己的研究和创新为本书所涉领域铺路的了不起的人们，他们的贡献帮助塑造了本书的内容与深度。我要感谢通读全稿的审稿人（Adrian Cockroft、Antje Barth、Arpitha Moorthy、Chaim Rand、Fabrizio Milo、Simon Zamarin、Suneeta Mall 和 Todd Bezenek），以及众多利用业余时间参与审

阅、大幅提升了本书整体质量的兼职审稿人。你们挑战了我，也挑战了我对这些复杂主题的理解。你们的洞见已经融入本书的每一页。

我尤其要感谢 Meta 的 Mark Saroufim。他和他成功运营的 GPU Mode 社区让我认识到，需要有这样深度的技术内容，来涵盖 PyTorch、CUDA 与 GPU 之间的软硬件协同设计。我深知，为这类高阶主题维护并壮大一个大型社区要付出多少心血。

我要向我亲爱的家人们（Ann Marie、Meredith、Tommy 和 Conor）致以由衷的感谢，你们始终如一的支持让我能挑灯夜写、直至凌晨。抱歉过去这一年去探望你们时，我有时未能真正“在场”……那时的我正窝在地下室里搞研究、写代码、写书呢！

另外，我想以一种多少有些特别的方式，感谢我的爱车——“小红帽”（Little Red Whiting Hood，参见图 P-1）。你是一套复杂的多学科精密系统，由精心调校的硬件与软件构成。从 15 年前开启这段旧金山之旅时起，你就一直陪着我。每次驾驶你，你都会提醒我当初为什么选择成为一名工程师。如果换算成 GPU 的世代，你应该算是 NVIDIA Ampere A100——一件由多学科工程师用当时性能最强的部件打造的永恒经典。你原本是为风冷（Luftkühlung）而设计的，但你懂得，要榨取极致性能，就得上水冷（Wasserkühlung）。你的缔造者一代又一代地固执而不知疲倦地挑战你的物理工程极限。你的涡轮增压器就像 Tensor Core，一旦被压得太狠，就会闹脾气、发热。我永远不会把你送走。

正如经典 Phish 乐曲《Contact》的歌词所唱：

某个十一月的清晨，我一觉醒来

忽然发现自己爱上了你　不是因为你的车头有明亮的前灯

也不是你的排气管，或你头顶的天窗

而是当狂风极力想把你推走时

你却紧贴路面不放的那股劲

我绝不会驾车远行　却不带着你一起回家

谢谢你鼓励我拥抱“机械同理心”，小红帽！

在我们不断推进人工智能与超级计算边界的同时，本书中的知识旨在启发、教育并赋能读者。对 AI Systems Performance Engineering 复杂性的探索不只是技

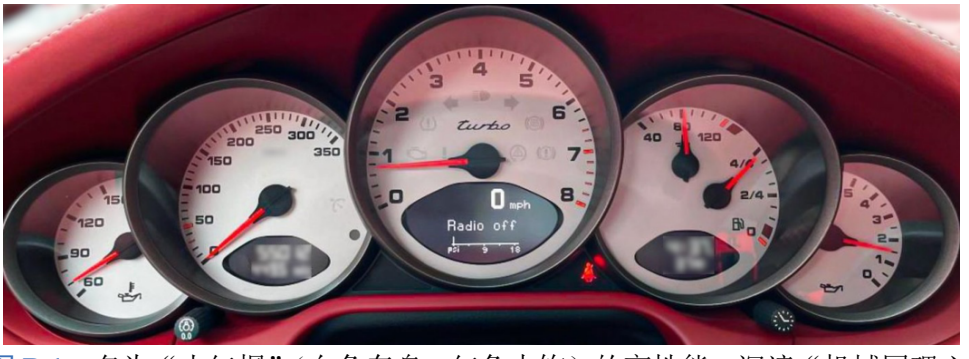

图 P-1　名为“小红帽”（白色车身，红色内饰）的高性能、深谙“机械同理心”的保时捷 911 Turbo 的仪表盘

术层面的探索——它也提醒我们正视人类的聪明才智、理解周遭世界的必要性，以及通过技术与创新不断改进的渴望。在这个世界上，真正理解如何协同设计硬件、软件与算法以实现极致性能与效率的人寥寥无几。读完本书，你将成为他们中的一员！

Chris Fregly 加利福尼亚州旧金山

目录

赞誉 **iii**

前言 **vi**

本书使用的惯例 vii
使用代码示例 vii
O'Reilly 在线学习 viii
如何联系我们 viii
致谢 viii

第 1 章 引言与 AI 系统概览 **1**

1.1 AI 系统性能工程师 3
1.2 基准测试与性能分析 4
1.3 扩展分布式训练与推理 4
1.4 高效管理资源 5
1.5 跨团队协作 5
1.6 透明性与可复现性 6
1.7 美国硬件出口限制下 DeepSeek 扩展至约 6800 亿参数模型 8
1.8 迈向 100 万亿参数模型 11
1.9 NVIDIA 的“机架式 AI 超级计算机” 13
1.10 机械同理心：软硬件协同设计 15
1.11 衡量“有效吞吐量”（Goodput） 16
1.12 本书路线图与方法论 18
1.13 关键要点 20
1.14 结论 21

第 2 章 AI 系统硬件概览 **23**

2.1 CPU 与 GPU 超级芯片 23
2.2 NVIDIA GPU Tensor Core 与 Transformer Engine 28
2.3 流式多处理器、线程与 warp 31

2.4 超大规模网络：让多个 GPU 如同一个整体 33
2.5 NVLink 与 NVSwitch 34
2.6 多 GPU 编程 35
2.7 使用 NVIDIA SHARP 进行网络内聚合 38
2.8 多机架与存储通信 40
2.9 预集成机架设备 41
2.10 共封装光学：网络硬件的未来 42
2.11 计算密度与功耗要求 43
2.12 液冷与风冷 44
2.13 实践中的性能监控与利用率 46
2.14 共享与调度 46
2.15 升级硬件的投资回报率（ROI） 47
2.16 展望未来：NVIDIA 的路线图 49
2.17 要点回顾 52
2.18 结语 54

第 3 章 面向 GPU 环境的操作系统、Docker 与 Kubernetes 调优 56
3.1 操作系统 56
3.2 NVIDIA 软件栈 57
3.3 为 GPU 环境配置 CPU 与 OS 62
3.4 面向性能的 GPU 驱动与 Runtime 设置 79
3.5 GPU 内存超额订阅、碎片化与内存不足处理 88
3.6 面向 GPU 的容器运行时优化 90
3.7 面向拓扑的 Kubernetes 容器编排与网络 94
3.8 优化 Kubernetes 的网络通信 98
3.9 减少 Kubernetes 编排抖动 99
3.10 关键要点 101
3.11 结论 104

第 4 章 分布式网络通信调优 106
4.1 通信与计算重叠（流水线） 107
4.2 NVIDIA Magnum IO 优化栈 116

4.3 多节点通信陷阱 123
4.4 用于分布式多 GPU 通信的 NCCL 130
4.5 分布式数据并行策略 135
4.6 NCCL communicator 生命周期与环境配置陷阱 141
4.7 网络内 SHARP 聚合 148
4.8 持久化 NCCL 用户缓冲区与零拷贝注册 150
4.9 NVIDIA 的 NIXL 与解耦式推理 151
4.10 带回调的 NIXL 异步 API 154
4.11 NIXL 与 NVIDIA Dynamo 等高性能推理系统 161
4.12 NCCL 与 NIXL 对比 161
4.13 关键要点 162
4.14 结论 163

第 5 章 基于 GPU 的存储 I/O 优化 164
5.1 快速存储与数据局部性 164
5.2 顺序读取与随机读取模式 165
5.3 调优 NVMe 和文件系统以提升吞吐量 166
5.4 使用 NVIDIA GDS 168
5.5 使用 cuda-checkpoint 保存 GPU 状态 170
5.6 使用 gdsio 测量 GDS 171
5.7 DeepSeek 的 Fire-Flyer 文件系统 172
5.8 分布式并行文件系统与对象存储 174
5.9 调优、复制和压缩数据 175
5.10 监控存储 I/O 176
5.11 调优数据流水线 178
5.12 高效的数据加载与预处理 178
5.13 随 GPU 数量扩展而扩展 worker 182
5.14 用 NVIDIA DALI 处理多模态数据 182
5.15 用 NVIDIA NeMo Curator 创建高质量 LLM 数据集 183
5.16 持续性能分析与调优工作流 184
5.17 诊断通信受限与计算受限的工作负载 188
5.18 关键要点 189

5.19 结论 190

第 6 章 GPU 架构、CUDA 编程与最大化占用率 192

6.1 理解 GPU 架构 192
6.2 CUDA GPU 的向后兼容与向前兼容模型 203
6.3 CUDA 编程复习 203
6.4 配置启动参数：每个网格的线程块数与每个线程块的线程数 207
6.5 异步内存分配与内存池 211
6.6 理解 GPU 内存层次结构 214
6.7 统一内存 220
6.8 保持高占用率和 GPU 利用率 223
6.9 使用启动边界调节占用率 233
6.10 使用 NVIDIA Compute Sanitizer 调试功能正确性 236
6.11 Roofline 模型：计算受限还是内存受限工作负载 237
6.12 关键要点 242
6.13 结论 243

第 7 章 剖析与调优 GPU 内存访问模式 245

7.1 合并访问与非合并的全局内存访问 245
7.2 向量化内存访问 253
7.3 利用共享内存进行分块与数据复用 264
7.4 避免共享内存 bank 冲突 277
7.5 Warp Shuffle 内建函数：避开共享内存与显式同步 285
7.6 只读数据缓存 287
7.7 异步内存预取与张量内存加速器 292
7.8 本章要点 299
7.9 小结 301

第 8 章 占用率调优、warp 效率与指令级并行 303

8.1 性能分析与 GPU 瓶颈诊断 303
8.2 用 Nsight Compute 分析 warp 停顿原因 309
8.3 查看实际占用率与 GPU 利用率 314
8.4 占用率调优 325

8.5 提升 warp 执行效率（warp 分歧） 333
8.6 暴露指令级并行 345
8.7 本章要点 357
8.8 结语 358

第 9 章 提高 CUDA kernel 的效率与算术强度 359
9.1 多级微块分块与软件预取 361
9.2 利用线程块簇进行分块 363
9.3 kernel 融合 365
9.4 结构化稀疏 369
9.5 重算与内存的权衡 371
9.6 PyTorch 与算术强度 372
9.7 混合精度与利用 Tensor Core 373
9.8 使用 CUTLASS 实现最优算术强度与 Tensor Core 性能 383
9.9 用于微观优化的内联 PTX 与 SASS 调优 388
9.10 DeepSeek 使用内联 PTX 优化内存分配 393
9.11 本章要点 394
9.12 小结 396

第 10 章 kernel 内流水线、warp 专用化与协作线程块簇 397
10.1 kernel 内流水线技术 398
10.2 持久 kernel 与 Megakernel（巨型 kernel） 418
10.3 协作组（Cooperative Groups） 422
10.4 线程块集群与分布式共享内存 429
10.5 利用线程块簇减少全局内存流量 442
10.6 利用线程块簇设计高效算法 445
10.7 基于线程块簇的 Warp 专业化 447
10.8 本章要点 456
10.9 结语 457

第 11 章 内核间流水线、同步和 CUDA 流排序内存分配 459
11.1 使用 CUDA 流重叠内核执行 459
11.2 使用流将计算与数据传输重叠 464

11.3 流排序内存分配器 466
11.4 将 CUDA 流和流排序内存分配器与 LLM 结合使用 470
11.5 旧版默认流 472
11.6 现代每线程默认流 473
11.7 默认流与显式（非默认）流 474
11.8 默认流使用的最佳实践 476
11.9 与事件和回调的细粒度同步 478
11.10 使用 CUDA 事件进行跨流同步 480
11.11 使用 Warp 专业化（内核内）和 CUDA 流（内核间）进行管道化 481
11.12 使用线程块集群和 CUDA 流进行 Warp 专业化 490
11.13 多 GPU 计算和数据传输与 CUDA 流重叠 499
11.14 程序化依赖启动 506
11.15 将 PDL 和线程块集群与 Warp 专业化相结合 509
11.16 要点 517
11.17 结论 519

第 12 章 动态调度、CUDA Graphs 与设备端发起的 Kernel 编排 521
12.1 基于原子工作队列的动态调度 522
12.2 CUDA Graphs 530
12.3 动态并行 547
12.4 跨多 GPU 与集群节点进行编排（NVSHMEM） 554
12.5 由 Roofline 模型指导的调度与编排决策 566
12.6 本章要点 568
12.7 本章小结 569

第 13 章 PyTorch 的性能分析、调优与扩展 571
13.1 NVTX 标记与性能分析工具 571
13.2 对 PyTorch 进行性能分析以定位瓶颈 577
13.3 PyTorch 编译器（`torch.compile`） 592
13.4 PyTorch 优化的注意力机制 605
13.5 PyTorch 架构优化（torchao）：量化、稀疏与剪枝 606
13.6 使用 CUDA 流实现并发 607

13.7 用 CUDA 图降低 kernel 启动开销 614
13.8 在 PyTorch 中进行内存性能分析与调优 622
13.9 PyTorch 对称内存 636
13.10 优化数据输入流水线 637
13.11 用 PyTorch Distributed 进行扩展 640
13.12 持续集成与性能基准测试 647
13.13 要点回顾 653
13.14 结论 658

第 14 章 PyTorch 编译器、OpenAI Triton 与 XLA 后端 659
14.1 PyTorch 编译器深潜 659
14.2 调试编译器阶段、图断裂与性能 690
14.3 PyTorch XLA 后端 718
14.4 关键要点 719
14.5 结论 723

第 15 章 多节点推理、并行、解码与路由优化 725
15.1 分离式预填充与解码架构 725
15.2 服务巨型 MoE 模型的并行策略 731
15.3 推测解码与并行 token 生成技术 742
15.4 约束解码的性能影响 750
15.5 MoE 推理的动态路由策略 752
15.6 本章要点 758
15.7 结论 760

第 16 章 大规模推理的性能分析、调试与调优 761
16.1 推理性能的分析、调试与调优 762
16.2 动态批处理、调度与路由 780
16.3 系统级优化 789
16.4 面向实时推理的量化方法 798
16.5 应用级优化 804
16.6 本章要点 821
16.7 本章小结 823

第 17 章 面向推理的预填充与解码分离扩展 **824**
17.1 为什么要进行预填充-解码分离？ 825
17.2 本章要点 864
17.3 结语 865

第 18 章 高级预填充-解码与 KV 缓存调优 **867**
18.1 优化解码内核 867
18.2 调优 KV 缓存利用与管理 873
18.3 预填充与解码之间的快速 KV 缓存传输 880
18.4 连接器与数据通路设计 885
18.5 预填充与解码的异构硬件和并行策略 888
18.6 GPU-CPU 协同的混合预填充 895
18.7 SLO 感知的请求管理与容错 897
18.8 动态调度与负载均衡 902
18.9 关键要点 908
18.10 小结 909

第 19 章 动态自适应推理引擎优化 **911**
19.1 自适应并行策略（TP、PP 与混合方案对比） 911
19.2 动态精度切换 916
19.3 Transformer 自注意力与 MLP 路径的 kernel 自动调优 924
19.4 动态共享内存分配与占用率感知的 kernel 选择 929
19.5 用于加快 TTFT 的推测式 KV 预取 933
19.6 实时 KV 缓存压缩与策略切换 938
19.7 用于运行时调优 AI 系统的强化学习智能体 948
19.8 动态内存分配切换（slab、缓存与流序） 953
19.9 运行时 kernel 性能改进与可热切换实现 961
19.10 使用时间序列预测持续预热 CUDA Graphs 和缓存 964
19.11 自适应批处理与分块预填充调度 967
19.12 多 GPU 下拥塞感知与拓扑感知的调度 974
19.13 其他自适应与动态优化技术 992
19.14 关键要点 995

19.15 结论 996

第 20 章 AI 辅助的性能优化与迈向数百万 GPU 集群的扩展 998

20.1 AlphaTensor：AI 发现的算法提升 GPU 性能（Google DeepMind） 999
20.2 关键要点 1017
20.3 结论 1019

附录 AI 系统性能检查清单（175+ 项） 1021

20.4 性能调优与成本优化思维方式 1021
20.5 可复现性与文档最佳实践 1024
20.6 系统架构与硬件规划 1025
20.7 统一 CPU-GPU“超级芯片”架构 1026
20.8 多 GPU 扩展与互连优化 1028
20.9 操作系统与驱动程序优化 1029
20.10 GPU 资源管理与调度 1032
20.11 I/O 优化 1034
20.12 数据处理流水线 1036
20.13 性能分析、调试与监控 1038
20.14 GPU 编程与 CUDA 调优优化 1040
20.15 kernel 调度与执行优化 1045
20.16 算术优化与降精度/混合精度 1046
20.17 高级调优策略与算法技巧 1049
20.18 分布式训练与网络优化 1050
20.19 高效推理与服务 1053
20.20 多节点推理与服务 1055
20.21 功耗与热管理 1056
20.22 结论 1058

索引 1059

关于作者 1062

版记 1062

第 1 章 引言与 AI 系统概览

2024 年末，中国一家名为 DeepSeek.AI 的小型初创公司在无法获得当时最新、最先进的 NVIDIA GPU 的情况下，训练出了一个前沿大语言模型（LLM），震动了 AI 社区。由于出口限制，DeepSeek 的工程师无法获得当时的顶级 NVIDIA Blackwell（B200、B300 等）或 Hopper（H100、H200 等）GPU，因此转而采用当时当地可获得且符合出口规定的替代产品，包括 NVIDIA H800 GPU。他们使用自定义 kernel，以及模型蒸馏等先进优化技术，从这些能力较弱的 GPU 中榨取了最大性能。

尽管存在这些限制，DeepSeek.AI 仍训练出了 DeepSeek-R1 模型，其推理能力接近当时使用最强大 NVIDIA 芯片训练的领先前沿模型。这一案例表明，熟悉 AI 系统性能工程的实践者和研究人员能够充分发挥手头硬件的能力——无论约束多么严苛。

例如，DeepSeek 的工程师把通信带宽视为*稀缺资源*，优化在线路上传输的每一个字节，从而在许多人认为不可能的基础设施上实现了这一目标。他们将数千块这类受限 GPU 扩展连接起来；这些 GPU 通过带宽有限的互连相连，并借助新颖的软件和算法优化克服了这些限制。

相比之下，美国和欧洲最大型 AI 前沿实验室采取的是“暴力堆算力”路线。这些实验室仍在追求更大的计算集群和更大的模型。模型规模已经从数百万参数膨胀到数十亿，如今又达到数万亿参数。虽然规模每扩大 10 倍都会解锁质的能力提升，但这也需要巨大的成本和资源。

例如，据报道，训练 OpenAI 的 GPT-4（2023）估计耗资约 $100 million，而训练 Google 的 Gemini Ultra（2023 年末）估计耗资约 $191 million，金额惊人。这说明，随着模型规模和成本不断上升，未来亟需提高资源效率。

DeepSeek 声称，DeepSeek-R1 的计算训练成本低于 $6 million——比 GPT-4 和 Gemini Ultra 等模型低一个数量级。同时，DeepSeek-R1 的性能却达到了那些成本高出几个数量级的竞争模型的水平。

不过，人们对这项 $6 million 的说法及其具体包含的内容仍有疑问（例如是否只包括单次训练运行），也不清楚它排除了哪些内容（例如实验和模型开发流

水线）。这一消息一度震动了美国金融市场，包括 NVIDIA 的股价；股价基于该消息单日下跌约 17%。市场担心 DeepSeek 的效率创新会让未来对 NVIDIA 硬件的需求下降。这一市场反应虽然有些过度——NVIDIA 股价在随后几个交易日中恢复——却说明 AI 效率方面的突破可能对全球金融市场产生重大财务影响。

除了模型训练之外，DeepSeek 还通过针对硬件的新颖算法改进，提高了支撑大多数现代前沿 LLM 的 transformer 架构的推理效率。DeepSeek 清楚地证明，巧妙的 AI 系统性能工程优化能够颠覆超大规模 AI 模型训练和推理的经济性。本书其余部分将全面介绍这些优化。

这里得到的深刻结论是：在这样的规模下，从系统中榨出的每一点性能都可能转化为数百万乃至数十亿美元的节省。每消除一个瓶颈，都可能对训练吞吐量和推理延迟产生非同寻常的影响。反过来，这会降低成本并提升终端用户的总体满意度。简言之，AI 系统性能工程不仅关乎速度，更是要让过去不可能的事情变得既可能又负担得起。

在第 1 章中，我们将深入探讨 AI 系统性能工程师这一角色——在大规模人工智能时代，这一角色已经变得至关重要。本章将全面介绍这一职业的多方面职责，以及它对现代 AI 系统的关键影响。

我们将从追溯 AI 工作负载的演进开始，重点介绍从传统计算范式到当代 AI 应用需求的转变。这一背景为理解 AI 中专门性能工程的必要性奠定基础。

随后，本章将深入介绍 AI 系统性能工程师所需的核心能力。我们将考察这一岗位必需的技术素养，包括对硬件架构、软件优化技术和系统级集成的深入理解。此外，我们还将讨论解决问题、沟通和协作等软技能的重要性；这些技能对于应对 AI 项目的跨学科特性至关重要。

本章有相当篇幅专门讨论这一角色的实践层面。我们将探讨性能工程师如何分析系统瓶颈、实施优化策略，并确保 AI 系统的可扩展性和可靠性。书中还将通过真实世界的场景和案例研究阐释这些概念，为这一领域遇到的挑战与解决方案提供具体实例。

此外，我们还将讨论性能工程师通常使用的工具和方法，介绍性能测试、监控和基准测试实践。这包括行业标准工具的概览，以及如何应用它们来评估和提升系统性能。

读者读完第 1 章后，将全面理解 AI 系统性能工程师的角色、胜任这一岗位所需的技能，以及性能工程对于 AI 系统成功部署和运行的关键重要性。这些基础知识将为后续章节作铺垫；后续章节会深入介绍定义 AI 性能工程卓越实践的具体技术、技术栈和最佳实践。

1.1 AI 系统性能工程师

AI 系统性能工程师是一个专门负责优化 AI 模型及其运行底层系统性能的岗位。这些工程师要确保 AI 的训练和推理流水线运行快速、成本高效且性能良好，同时还要可靠并具备高可用性。随着规模扩大，AI 系统性能工程师的重要性也会进一步提升。

AI 系统性能工程师通常能获得顶尖薪酬，而且理由十分充分。我们的工作会直接影响盈亏。我们融合硬件、软件和算法方面的专业知识，必须理解底层操作系统因素、内存层次结构、网络基础，以及 Python、C++ 等多种语言；还要掌握不同的 AI 框架和库，例如 PyTorch、OpenAI 的 Triton，以及 NVIDIA 的 Compute Unified Device Architecture（CUDA）。

在任何一天，AI 系统性能工程师可能会检查底层 GPU kernel 的效率、优化操作系统线程调度、分析内存访问模式、提高网络吞吐效率，或调试分布式训练算法。AI 系统性能工程师的关键职责包括基准测试、性能分析、调试、优化、扩展和高效管理资源。虽然性能工程师可能专门从事硬件、软件和算法的某种组合，但关键在于必须对这些专业领域进行协同设计（见图 1-1）。因此，理解它们之间的权衡以及彼此的影响十分重要。

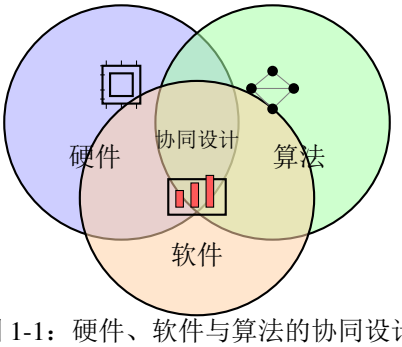

图 1-1：硬件、软件与算法的协同设计

1.2 基准测试与性能分析

基准测试和性能分析包括在各种工作负载下测量 AI 模型的延迟、吞吐量、内存使用量及其他性能指标，其中包括训练和推理。为了识别瓶颈，我们必须反复结合使用 NVIDIA Nsight Systems、NVIDIA Nsight Compute 和 PyTorch profiler。这些工具协同工作，可以在我们持续提升 AI 系统总体性能的过程中，精确定位瓶颈并在不同栈层跟踪性能随时间的变化。

必须在开发周期早期设置自动化性能测试，以捕获回归和性能下降。

调试和优化性能问题要求我们追溯到根本原因：问题可能来自次优的 CUDA kernel、不必要的通信开销，或训练或推理工作负载中的不平衡。

在一种情况下，我们可能希望使用更高效的矩阵运算，利用针对采用 transformer 架构的现代 LLM 优化的最新 NVIDIA Transformer Engine（TE）硬件。在另一种情况下，我们可以通过为“天然可并行”的推理工作负载配置更高程度的并行性来改进软件框架。还有一种情况是，我们可能尝试改进 transformer 的 attention 算法：通过更好的内存管理，减少相对于所需 GPU 计算量而言在 GPU RAM 中移入和移出的内存量。

即使是很小的代码调整，也可能带来巨大的收益。例如，用 Python 编写的数据预处理步骤可能拖慢整个训练流水线。你可以将这段代码用 C++ 重新实现来消除瓶颈，或者使用 NVIDIA cuPyNumeric——一个可直接替换 NumPy 的库——让数组运算工作负载分布到 CPU 和 GPU 上。

1.3 扩展分布式训练与推理

将小型研究工作负载扩展为超大规模集群上的大型生产工作负载，可以确保系统从 8 个 GPU 扩展到 80,000 个 GPU 时仍能以最小的开销和效率损失运行。这需要使用 NVIDIA Collective Communications Library（NCCL，读作“nickel”）优化通信，以支持训练运行中常见的 all-reduce 等分布式 collective。此外，NVIDIA Inference Xfer Library（NIXL）为分布式推理提供跨 GPU 内存和存储层级的高吞吐、低延迟点对点数据移动。这种通信既可以发生在单个节点内的 GPU 之间，

也可以跨越数千个节点。他们还会优化 all-reduce、all-to-all 和 all-gather 等通信与 collective 聚合操作；这些操作在模型训练和推理期间被广泛使用。

你可能希望利用数据并行、张量并行和流水线并行，将数据巧妙地放置在各个节点上。或者，由于模型太大、无法装入单个 GPU，你可能需要重新设计工作负载以使用张量并行或流水线并行。也许你使用的是混合专家（MoE）模型，这时可以利用专家并行。

1.4 高效管理资源

优化模型对 CPU 核心、GPU 内存、互连带宽和存储 I/O 等资源的利用方式十分重要。这可能包括许多工作，例如确保 GPU 始终以满速获得数据、将线程固定到指定 CPU 核心、减少上下文切换开销、编排内存使用，以及在使用大型模型训练和推理时避免 GPU 内存不足（OOM）错误。当不需要完整 GPU 算力执行某项任务时，可以采用 GPU 虚拟化技术（例如 NVIDIA 的 Multi-Instance GPU［MIG］）划分 GPU 资源，以提高整体利用率。

1.5 跨团队协作

跨团队协作对于 AI 系统性能工程师绝对至关重要。与研究人员、数据科学家和应用开发者手拉手合作十分重要；同时也要与基础设施团队协作，包括网络和存储团队。

提升性能可能需要修改模型代码，这就需要与研究人员协调。或者，你可能希望部署新的 GPU 驱动程序来提升效率，这又需要基础设施团队参与。

性能改进往往跨越多个团队。例如，为修复 bug 和提升效率而更新 CUDA 驱动程序或 CUDA 版本，需要与 DevOps、基础设施和支持团队进行周密协调。而要改进模型代码的性能，则需要与研究人员密切合作。性能工程师处在这些多学科领域的交汇处，能够使用 AI、计算机科学和系统工程的共同语言沟通。

1.6 透明性与可复现性

在性能工程中，测量一切并相信数据而不是假设至关重要。通过发表工作成果，其他人可以学习、复现并在你的发现之上继续构建。

DeepSeek 故事的一个显著方面，是他们如何公开分享基础设施优化。在 2025 年 2 月举行的 DeepSeek Open-Source Week 期间，他们发布了一套开源 GitHub 仓库，包括 FlashMLA、DeepGEMM、DeepEP、expert parallelism load balancer（EPLB）、DualPipe 和 Fire-Flyer File System（3FS）。每个项目都经过生产环境测试，目标是从其硬件中榨取最大性能。这些项目在《DeepSeek-V3 Technical Report》中有详细介绍。

FlashMLA 是他们用 CUDA C++ 编写的优化 attention kernel。DeepGEMM 提供了针对 FP8 优化的矩阵乘法库，据报道，其在稠密和稀疏运算上都优于许多厂商 kernel。Deep Experts Parallelism（DeepEP）是他们为混合专家（MoE）模型高度调优的通信库。EPLB 实现了一种冗余专家策略，通过复制负载很重的专家来处理额外负载。DualPipe 是一种双向流水线并行算法，它将前向/反向计算与通信阶段重叠，以减少流水线气泡。3FS 则是他们的高性能分布式文件系统，提醒我们必须优化每一层——包括文件系统——才能获得 AI 系统的最大性能。

DeepSeek 在 2025 年 2 月的“Open-Source Week”期间将这些项目开源到 GitHub，不仅通过允许他人复现结果证明了其主张的可信度，也向社区回馈了成果。这种透明性使其他开发者能够对其方法进行基准测试、复现并学习，包括使用 DeepEP/DualPipe 流水线并行重叠通信，以及利用 3FS 使 NVMe SSD/RDMA 带宽饱和。

DeepSeek Open Infra Index 等开放项目提供了有价值的基线和工具。它们给出了各种 AI 硬件配置上的真实性能测量，并鼓励同口径比较和可复现性。同样，MLPerf 开放基准测试套件也提供了一个标准，用于在不同硬件和软件配置之间可复现地比较训练和推理性能。

MLPerf 等行业基准量化了跨硬件代际的这类协同设计优化。在 MLPerf Training v5.0（2025）中，基于 Blackwell 的 NVIDIA GB200 NVL72 系统，相比等效 Hopper 系统，每 GPU 的训练吞吐量最高提升了 2.6×，如图 1-2 所示。在 MLPerf Inference v5.0（2025）中，Blackwell NVL72 相比等效 Hopper 集群，

每 GPU 的推理吞吐量约提升 3.4×，原因在于更高的单 GPU 性能以及大得多的 NVLink 域。这些结果见图 1-3。

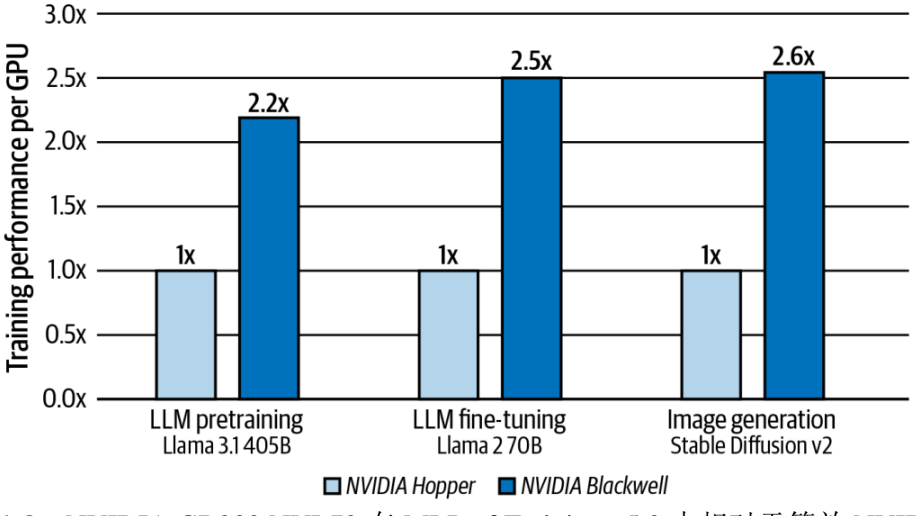

图 1-2　NVIDIA GB200 NVL72 在 MLPerf Training v5.0 中相对于等效 NVIDIA Hopper 集群的每 GPU 吞吐量提升（来源：https://oreil.ly/Ao1l8）。

在后续章节中，我们将引用其中一些开放基准来支撑各种性能调优概念。例如，在讨论 GPU kernel 优化时，我们会引用 DeepSeek 发布的性能分析结果，展示其自定义 kernel 如何在 NVIDIA GPU 上实现接近峰值的内存带宽利用率。

> 引用 MLPerf 结果时，请记住，MLPerf 警告说，跨平台比较时每 GPU 结果并不是主要指标。应以系统级、端到端吞吐量作为正确的比较依据。每 GPU 数值只能作为组件级指标。

实验透明性和可复现性对于推动 AI 性能工程领域向前发展至关重要。我们很容易陷入轶事式的“凭感觉”优化陷阱（“我们做了 X，感觉速度更快了”）。相反，我倡导一种严谨的科学方法：提出假设，使用可复现的基准测试测量结果，根据结果进行调整以改善性能，重新运行基准测试，并在每一步分享全部结果。

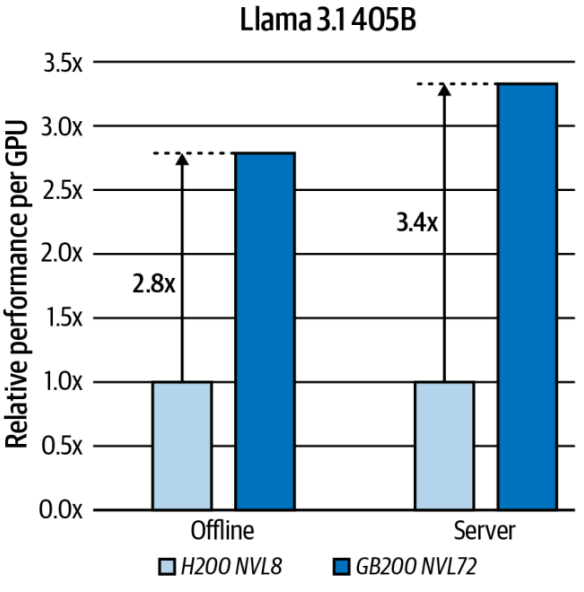

图 1-3　NVIDIA GB200 NVL72 在 MLPerf Inference v5.0 中相对于等效 NVIDIA Hopper 集群的每 GPU 吞吐量提升（来源：https://oreil.ly/V-jze）。

1.7 美国硬件出口限制下 DeepSeek 扩展至约 6800 亿参数模型

有时，系统性能创新源于必需。正如前文所述，DeepSeek 受到限制，只能使用 NVIDIA H800 GPU。H800 是 Hopper GPU 的出口合规版本。与 H100 相比，它降低了 NVLink 互连带宽和 FP64 性能，但 HBM 容量和带宽基本相近。

作为背景参考，NVIDIA H100 每个 GPU 提供约 900 GB/s 的 NVLink 互连带宽，而 H800 每个 GPU 提供约 400 GB/s。这会使 GPU 间数据传输变慢，并最终限制多 GPU 扩展能力。此外，H100 提供 3.35 TB/s 的内存带宽，而 H800 的吞吐受限，意味着数据传输速度慢得多。这会威胁分布式训练作业，造成瓶颈并限制扩展效率。

DeepSeek 着手在这一高度受限的环境中训练一个庞大的、约 6800 亿参数的 MoE 语言模型，名为 DeepSeek-V3。该 MoE 模型对于每个输入 token 只使用约 370 亿个活跃参数，而不是一次性使用约 680 billion 个参数。

借助这一架构，模型在任意时刻只激活其中一部分，从而即使使用紧凑的 H800 配置，也能控制计算负载。具体而言，对于每个 token，DeepSeek-V3 使用 1 个共享专家，再从 256 个专家中使用路由器选择的 8 个专家；每个 token 总共激活 9 个专家，如图 1-4 所示。

我们将在后续章节更详细地介绍 MoE。但现在只需知道，为了应对环境限制，DeepSeek 实现了一种新颖的 DualPipe 并行算法，精心重叠计算与通信，以掩盖 H800 固有的弱点。

通过设计自定义 CUDA kernel，绕过 NCCL 的某些默认通信 collective，DeepSeek 得以让数据传输与正在进行的计算并行协调。这使 GPU 即使在互连带宽降低的情况下也能保持高效利用。这种通信与计算重叠会成为全书反复讨论的主题。

这项创新工程取得了回报：DeepSeek-V3 最终训练完成时所消耗的 GPU 时间和成本，只是 OpenAI、Meta、DeepMind 等机构规模相近的前沿模型所需的一小部分。这些资源远少于许多人原本认为需要用更强集群训练如此规模模型所需的资源。

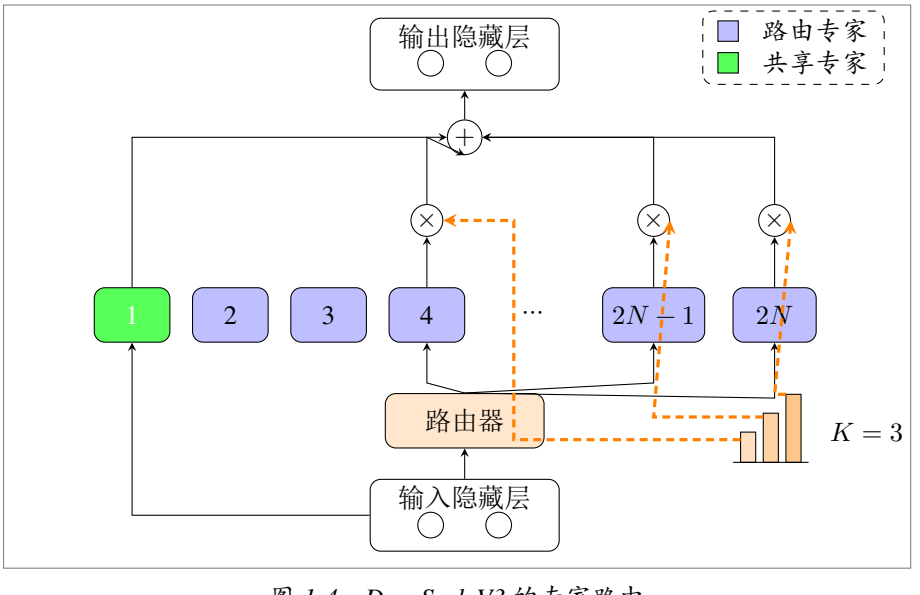

图 *1-4*：*DeepSeek-V3* 的专家路由

DeepSeek 报告称，V3 在多项标准化基准测试上的性能接近 GPT-4，包括语言理解、阅读理解和推理；在某些测试中甚至达到或略高于 GPT-4。这些比较基于业界通用的标准化测试。这意味着，尽管硬件能力较弱，开放式 MoE 模型仍能与最优秀的闭源模型竞争。

随后，团队以 DeepSeek-V3 为基础创建了 DeepSeek-R1；它是专门的推理模型，采用了与 OpenAI 的 o1 和 o3 系列推理模型类似的构建方式。DeepSeek 没有在微调中高度依赖成本高昂的人工反馈循环，而是开创了“冷启动”策略，仅使用少量监督数据，转而强调强化学习，以便将思维链（chain-of-thought）推理直接嵌入 R1。这一方法降低了训练成本和时间，也凸显出聪明的软件与算法设计能够克服硬件瓶颈。

这些经验表明，大型、稀疏激活的 MoE 模型即使在内存和计算预算有限的情况下也能有效扩展。新颖的训练调度，以及底层的通信/计算重叠优化，可以克服硬件限制；DeepSeek 的工作所带来的巨大投资回报率（ROI）证明了这一点。

投资回报率是清晰的：DeepSeek 非传统的方法带来了巨大的效率提升，并以低得多的训练成本和时间创造出一系列强大的模型。团队从 NVIDIA H800 GPU 中榨取了每一分性能——即使通信带宽受限，也交付了与 GPT-4 相当的模型性能，成本却少了数百万美元。此外，DeepSeek 在 R1 的推理微调阶段不需要那么多人工标注数据，又节省了更多资金。

简言之，聪明的软件和算法设计克服了硬件的暴力限制，使 DeepSeek 能够在紧张的成本和硬件预算下开发大规模 AI 模型。

1.8 迈向 100 万亿参数模型

100 万亿参数模型是 AI 的一个理想里程碑，人们常将其与人类新皮层中估计的 100 万亿个突触连接相比较。每个突触连接都相当于一个模型参数。从理论上说，实现这一规模的模型是可能的，但它需要极其庞大的资源和资金。仅靠暴力扩展来达到 100 万亿参数模型的规模，除了最富有的组织之外，对其他所有组织都不切实际。

粗略按数量级估算，一个在约 29 万亿个 token 上训练的稠密 100 万亿参数模型，需要约 1.2×10^{29} 次浮点运算（FLOPS）。更大的 token 数量会使这一数值线性增长，而使用稀疏（MoE）模型可以降低有效计算量。然而，即使考虑稀疏性，这也表明单靠暴力计算不足以满足超大规模模型的计算需求。要使 100 万亿参数模型的训练在合理时间内可行，就需要新的方法。达到这种规模需要硬件、软件和算法协同设计方面的突破性效率提升，而不是继续简单扩展当前方法。

> 虽然本书讨论的优化可以应用于更小的模型和集群规模，但我会继续回到 100 万亿参数模型这个例子，以强化这样一个观点：我们不能只向扩展问题堆砌硬件。我们需要与硬件协同设计的软件和算法创新，包括计算、网络、内存和存储。

训练成本如此爆炸式增长，正在推动人们寻找新的 AI 系统和软件工程技术，以提升性能、降低成本，并让极端规模的 AI 在计算资源有限、电力受限且资金有限的条件下成为可能。研究人员一直在探索降低有效计算需求的新技术。

一个重要思路是利用稀疏性，具体来说就是使用 MoE 模型。MoE 等稀疏模型与传统稠密模型相对，后者包括由 OpenAI 推广的常见 GPT 系列大语言模型（LLM）。事实上，公开讨论推测，OpenAI 的专有 GPT 系列和 o-series 推理模型等一些前沿模型可能采用了 MoE 架构。

MoE 等稀疏模型会针对每个输入 token 只激活模型的一部分。通过让每个输入 token 只经过众多内部“专家”中的一个子集，每个 token 的 FLOPS 即使在总参数量增长时也能大致保持不变。这类稀疏模型证明，扩展到数万亿参数模型并不需要计算成本等比例爆炸。此外，由于这类模型在推理期间使用的活

跃参数更少，相比等规模的稠密模型，MoE 模型的请求—响应延迟通常低得多。这些都是训练和服务 100 万亿参数规模模型的重要启示。

DeepSeek-V3（基础模型）和 -R1（基于强化学习的推理变体）是 MoE 效率的优秀例子。它们包含约 6800 亿个总参数，每个输入 token 激活约 370 亿个参数。DeepSeek 的技术报告和公开文章描述了一种 MoE：每个 token 使用 1 个共享专家，并从 256 个专家中选择 8 个，从而得到约 9 个活跃专家，以及每个 token 约 370 亿个活跃参数。这使 DeepSeek-V3 和 -R1 比规模相近的稠密大语言模型更节省资源。另一个例子是 Google 在 2021 年推出的 Switch Transformer MoE。这一拥有 1.6 万亿个参数的 MoE 模型，仅用一小部分计算量就达到了与稠密模型相同的准确率。它的训练速度比可比的稠密方法快 7×。

除了庞大的计算需求之外，内存也是一个主要瓶颈。例如，如果每个参数都以 16 位（2 字节）精度存储，一个 100 万亿参数模型需要约 182 TB 的 GPU 内存才能装载模型（182 TB = 100 trillion parameters × 16 bits/weight × 8 bits/byte）。这相当于单块 NVIDIA Blackwell B200 GPU 上 192 GB（可用 180 GB）GPU RAM 的 3 个数量级（1,000×）。

仅仅装载 100-trillion 模型的权重，就需要接近 1,000 块 Blackwell B200 GPU（每块 192［可用 180］GB），或者 700 块 Blackwell Ultra B300 GPU（每块约 288 GB）。这个估算只考虑装载模型，不包括激活值内存、优化器状态和推理输入；这些因素还会进一步增加所需的总内存。

作为背景参考，一台典型的 B200 GPU 计算节点只包含 8 块 B200 GPU。要使用这些 GPU，若采用 B200，光是装载模型就需要约 125 个 GPU 节点；若采用 Ultra B300 GPU 计算节点集群，则需要约 86 个 GPU 节点（每个节点配备 8 块 B300 GPU）。

此外，装载训练数据也会变得极其困难，因为要足够快地向这种模型供给数据、让所有 GPU 保持繁忙并非易事。尤其是，当 100 万亿参数模型被划分到 1,000 块 B200 GPU 或 700 块 B300 GPU 上时，GPU 之间的通信开销会显著增长。训练单个模型可能消耗数百万 GPU 小时和兆瓦时的能量。要扩展到足以训练和服务 100 万亿参数模型的规模，这意味着巨大的成本和能源消耗。

100 万亿参数 AI 时代将迫使我们彻底重新思考系统设计，才能让这一规模的训练和部署切实可行。硬件、算法和软件都需要共同演进，以应对这一新的前沿。

1.9 NVIDIA 的“机架式 AI 超级计算机”

为了应对超大规模计算的挑战，NVIDIA 构建了一类新的 AI 超级计算机，专门面向万亿参数规模的工作负载。例如，2024 年的 NVIDIA Grace Blackwell GB200 NVL72，以及 GB300 NVL72 Ultra；后者使用 Blackwell Ultra GPU，每个 GPU 配备 288 GB HBM3e，并保持一个 72-GPU 的 NVLink 域，聚合带宽约为每秒 130 TB。

Vera Rubin VR200（2026）和 Feynman（2028）系统继续沿着这一趋势发展：将百亿亿次级超级计算机压缩进单个数据中心机架。事实上，NVIDIA 把 NVL72 这类机架系统称为“机架式 AI 超级计算机”，而且理由充分。

每个 GB200/GB300 NVL72 机架集成 36 个 Grace Blackwell，通过 NVLink 与 NVSwitch 相连，提供机架级交换结构。每个 Grace Blackwell Superchip 由一颗 NVIDIA Grace CPU（具有 72 个 CPU 核心）和两颗 NVIDIA Blackwell GPU 组成，因此总计有 36 个 Grace CPU 和 72 个 Blackwell GPU——这就是名称“NVL72”中“72”的由来。

> 如果你还没有猜到，NVL72 中的 NVL 代表 *NVLink*。这能提醒你，该系统中的 GPU 使用 NVLink 互连。以防我们忘记！

每块 Grace Blackwell 主板都通过 NVSwitch（机架内的 NVLink 交换网络）连接到其他主板。这样，全部 72 个 GPU 都能以每个 GPU 双向 1.8 TB/s 的完整 NVLink 5 带宽相互通信（18 × 100 GB/s 链路）。在整个机架范围内，NVLink Switch System 在一个 NVL72 域内提供约 130 TB/s 的 GPU 到 GPU 聚合带宽。

实际上，NVL72 的内部互连结构把全部 72 块 Grace Blackwell GPU 统一成一个高速集群。因此，PyTorch（用于训练）和 vLLM（用于推理）等框架可以利用这个单一的 NVLink 域，高效实现数据、张量、流水线和专家并行。CUDA 统一内存可以按需在 NVLink 上迁移页面或远程访问页面。不过，远程内存具

有不同的延迟和带宽，应被视为非一致性访问。当在 Grace 与 Blackwell 之间依赖托管分配（managed allocations）时，应优先使用 cudaMemPrefetchAsync 和 cudaMemAdvise 显式预取，以减少缺页停顿。

后续章节将更详细地介绍这台超级计算机的计算、内存和互连硬件。现在，我们先从现代生成式 AI 模型的角度，分析这台 AI 超级计算机整体的性能规格。

一台完整的 GB200 NVL72 机架，在 FP4 精度、2:1 结构化稀疏条件下，理论上可达约 1.44 exaFLOPS；在 FP8 精度、2:1 结构化稀疏条件下，约为 720 petaFLOPS。它在 72 块 GPU 上提供约 13.5 TB 的 HBM3e 内存（13,824 GB = 192 GB/GPU × 72 块 GPU）；若把同一 NVLink 域中的 Grace CPU 内存也算上，总量约 30 TB。

NVLink 提供跨 72 块 GPU 的池化访问，CUDA 统一内存可以通过 NVLink 迁移页面或远程访问页面；但远程访问的性能不同，整个机架并不会表现为一个均匀一致的内存设备。

简而言之，GB200 NVL72 是一个自包含的系统：72 块 GPU、1.44 exaFLOPS 算力、30 TB 内存，机架功耗约为 120–132 kW（取决于厂商配置和散热方式）。它确实是一台能在单个机架内训练和提供数万亿参数模型服务的 AI 超级计算机。

把这些机架组合成超大规模集群，就能支持庞大的数万亿参数模型。更妙的是，你可以通过常用的云服务商——包括 Amazon Web Services（AWS）、Google Cloud Platform（GCP）、Microsoft Azure、CoreWeave、Lambda Labs 等——点几下鼠标（以及花掉相当多美元！）就完成这些机架和机架集群的部署。

> **注** 本书虽然重点讨论 NVIDIA Grace Blackwell 一代芯片，但所讨论的优化原则源自此前许多代 NVIDIA 硬件。这些优化也将继续适用，并演进到未来许多代 NVIDIA 芯片，包括 Vera Rubin（2026）、Feynman（2028）及更远的世代。这一路线图延续了每一代新 GPU 在性能、内存和集成度上翻倍的模式。

贯穿全书，你将了解每一代在计算、内存、网络和存储方面的创新，如何以超大规模集群、数万亿参数模型、高吞吐训练作业和极低延迟模型推理服务器的形式，推动 AI 进一步扩展。这些创新由硬件感知算法驱动，而这些算法贯彻了下一节讨论的机械同理心和软硬件协同设计原则。

1.10 机械同理心：软硬件协同设计

机械同理心（mechanical sympathy）一词最初由软件工程师 Martin Thompson 提出，他借用了英国一级方程式冠军 Jackie Stewart 的类比——Stewart 对赛车的机械细节了如指掌。在计算领域，机械同理心指编写深刻理解其运行硬件的软件。在 AI 语境中，它意味着让算法与硬件能力携手协同设计，以最大化性能。

现实经验表明，即使对 GPU kernel 或内存访问模式做微小调整，也可能带来超乎比例的收益。一个经典例子是 FlashAttention：这种新算法以硬件感知的方式重新实现了 transformer 注意力机制。

FlashAttention 对 GPU 计算进行“分块”（tiling），从而最大限度减少对 GPU 内存的读写次数。它显著减少了内存搬运，并加速注意力计算。用 FlashAttention 替代默认的 transformer 注意力机制/算法，可在长序列训练和推理中带来 2–4 倍加速，同时降低整体内存占用。

这类改动把曾经的主要瓶颈（注意力）压缩到整体运行时间的一小部分。FlashAttention 几乎在一夜之间成为许多库的默认选择，因为它让模型能够更快、更高效地处理更长序列。自 FlashAttention 之后，又涌现出许多新的注意力算法，包括 DeepSeek 的多头潜在注意力（Multi-Head Latent Attention，MLA）。

DeepSeek 的 MLA 算法以 NVIDIA GPU kernel 实现，并于 2025 年开源，是软硬件协同设计（即机械同理心）的另一个例子。与 FlashAttention 类似，MLA 重构了注意力计算，以更好地利用 NVIDIA 的内存层级和专用 GPU Tensor Core。这些优化使 MLA 能够充分利用受限制的 H800 GPU 架构，以低得多的成本实现更高吞吐量——在同样的 H800 系统上甚至超过了 FlashAttention 的性能。

整本书实际上就是一项关于机械同理心的研究。我们将看到无数案例：新的硬件特性——或像 DeepSeek 面对的硬件约束——催生出新颖的软件和算法技术；反过来，新的软件算法也会推动硬件创新。

例如，transformer 模型和降精度量化（如 FP8/FP4）的兴起，促使 NVIDIA 增加了 Transformer Engine 和专用低精度 Tensor Core 等专用硬件，用于更快的矩阵数学计算单元。这些硬件创新反过来又让研究人员能够探索新颖的数值优化器和神经网络架构，进而推动硬件设计者继续前进，解锁更新颖的算法，如此循环。这是一个良性循环！

现代 GPU 采用支持 FP4 和微缩放（microscaling）的最新一代 Transformer Engine。结合最新一代 NVLink，这些特性主要通过更快的 Tensor Core 数学运算、更高的内存带宽和改进的特殊功能单元（SFU）来提高注意力吞吐量。例如，指数计算单元专门用于加速 transformer 注意力算法大量使用的 softmax 运算——这是当今 LLM 模型的关键部分。随着 transformer 的流行，softmax 延迟在此前的 GPU 上已成为瓶颈，因为它对 transformer 注意力机制至关重要。

> **注** 通过现代 GPU 提升特殊功能吞吐量和快速数学流水线，NVIDIA 以最纯粹的形式展示了机械同理心以及硬件—软件—算法协同设计。他们直接与研究人员和从业者合作，定位并减少现代 LLM 算法（如注意力）在其硬件上运行的瓶颈。

这种紧密的相互作用——GPU 与 AI 算法共同演进——正是 AI 中机械同理心的核心。这些协同设计创新只有在硬件公司（如 NVIDIA 和 Advanced RISC Machine [ARM]）、AI 研究实验室（如 OpenAI 和 Anthropic）以及 AI 系统性能工程师（比如我们！）密切合作时才能发生。

1.11 衡量“有效吞吐量”（Goodput）

在运营数百、数千甚至数百万块 GPU 的集群时，重要的是理解理论硬件能力中有多少真正在做有用功。FLOPS 和设备利用率等传统吞吐量指标会高得具有误导性，因为大量时间可能花在通信停滞、计算空转或作业重启失败上。这正是“有效吞吐量”（goodput）概念的用武之地——Meta 在 2025 年的一篇论文中将其称为“有效训练时间比”（effective training-time ratio）。

> **注** NVIDIA 将理论硬件上限称为“光速”（speed of light），你可能在 NVIDIA 的博客、文档、网络研讨会和会议演讲中见过这一说法。

简单来说，有效吞吐量衡量单位时间内完成的有用功的吞吐量（处理的 token 数或完成的推理请求数），扣除一切不直接贡献于模型训练或推理的部分。它实际上是从有效模型训练或推理角度衡量的系统端到端效率。随后可以用集群的最大可能吞吐量对有效吞吐量做归一化，得到百分比效率值。

例如，假设一个拥有 8 块 GPU 的节点能在 10 秒内处理 100,000 个 token。此时其有效吞吐量为每秒 10,000 个 token。如果该节点每块 GPU 的峰值理论吞吐量为每秒 1,500 个 token，8 块 GPU 合计每秒 12,000 个 token，则节点效率为 83.3%（0.833 = 10,000 实际吞吐量/12,000 峰值吞吐量）。

Meta 的 AI 基础设施团队在《Revisiting Reliability》论文中强调了有效吞吐量的重要性。该论文引入了有效训练时间比指标，并说明即使表面利用率很高，抢占、资源碎片化和故障仍会减少实际训练时间。在这篇论文中，Meta 团队分析了抢占、硬件故障和网络拥塞如何降低实际吞吐量。

换言之，尽管集群看起来利用率达到 100%，却有 70%–75% 的算力因通信延迟、次优并行化、数据延迟或故障恢复等开销而损失。此外，Meta 的分析表明，在大规模下，作业抢占、网络热点和不可恢复故障是有效吞吐量损失的主要来源。

例如，设想一个训练作业在理想硬件上理论上每秒可处理 1,000 个样本，但由于输入流水线不佳和过度同步，实际训练吞吐量只有每秒 300 个样本。我们会说该作业以 30% 的有效吞吐量运行，剩余 70% 的容量基本上被浪费了。

识别这些差距并加以弥补，是我们工作的核心部分。例如，如果 GPU 在等待从存储加载数据，我们可以引入缓存或异步预取；如果 GPU 在模型训练的梯度同步阶段空转，我们通常希望让 GPU 计算（如计算梯度）与 GPU 间通信（如同步梯度）重叠。我们的目标是把浪费的低效周期转化为有用功。

理论性能与实际性能之间的这一差距，正是 AI 系统性能工程师这一角色的价值主张。我们的使命是尽可能提高有效吞吐量——理想情况下使其接近 100%——办法是在技术栈的每一层（包括硬件、软件和算法）攻击低效环节并降低成本。

> **注** 对 AI 系统性能工程师的投入，带来的回报始终远高于成本。设想一位性能工程师帮助集群效率提升 20%，在大型 AI 环境中，这可能减少数百万美元的硬件成本。

通过关注有效吞吐量，我们优化的是真正重要的东西——每美元成本和每焦耳电力所产生的有用训练量。有效吞吐量是衡量成功的终极指标——比原始 FLOPS 或设备利用率更重要——因为它体现了硬件、软件和算法在“更快、更便宜地训练 AI 模型”这一最终目标上的协调程度。

提升有效吞吐量需要深入理解硬件（如 CPU、GPU、网络拓扑、内存层级、存储布局）、软件（如操作系统配置、分页内存、I/O 利用率）和算法（如 transformer 架构变体、注意力机制替代方案，以及不同的缓存和批处理策略）之间的相互作用。

正是这种跨多个学科（包括硬件、软件和算法）的广博而深入的理解，使 AI 系统性能工程师在今天如此稀缺。这也是我写这本书的原因！接下来介绍本书其余部分的路线图和方法论。

1.12 本书路线图与方法论

我们将如何着手优化 100 万亿参数的 AI 系统？本书的组织方式是从硬件基础讲起，逐步上升到软件栈和算法技术，并强调每一层的动手分析。以下是本书其余部分的分章介绍。

第 2 章深入介绍 NVIDIA AI 系统硬件，包括 GB200/GB300 NVL72“机架式 AI 超级计算机”；它把 Grace Blackwell Superchip 设计与 NVLink 网络相结合，形成 AI 超级计算机的性能/功耗特征。

第 3–5 章随后介绍面向 GPU AI 系统的操作系统级、网络和存储优化。这些优化包括 CPU 与内存绑定，以及 Docker 容器和 Kubernetes 编排方面的考虑，包括 GPU 环境的网络 I/O 和存储配置。

第 6–12 章讨论 NVIDIA CUDA 编程基础和 CUDA kernel 优化，这些内容对于开发新颖的硬件感知算法至关重要。此类流行算法包括 FlashAttention 和 DeepSeek 的 MLA；它们针对 transformer 架构中资源密集的注意力机制，而该机制主导着当今的生成式 AI 工作负载。

掌握 NVIDIA 硬件和 CUDA 软件背景后，我们将深入分布式通信优化，包括如何高效训练和提供超大模型服务。我们将考察最小化通信的策略，例如让计算与通信重叠——这一模式适用于 AI 系统栈的许多层面。

第 13 和 14 章讨论 PyTorch 特有的优化，包括 PyTorch 编译器栈，以及 OpenAI 基于 Python 的 Triton 语言和编译器（用于自定义 GPU kernel）。这些编译器降低了开发新颖 CUDA kernel 的门槛，因为开发者不再需要具备开发 CUDA kernel 通常所需的深厚 C++ 功底。这两章还会讨论模型训练的分布式并行技术，包括

数据并行（DP）、完全分片数据并行（FSDP）、张量并行（TP）、流水线并行（PP）、上下文并行（CP）和混合专家（MoE）。我们将展示如何把数万亿参数模型拆分并高效地训练到许多 GPU 上。此外，我们还会讨论超大规模模型训练中的内存优化技术，包括激活检查点、优化器状态分片，以及卸载到更大的 CPU 内存。当模型规模超过 GPU 硬件的物理上限时，这些技术至关重要。

第 15–19 章聚焦高吞吐、低延迟模型推理和智能体式 AI 系统的软件与算法创新。这包括分离式预填充与解码——目前 NVIDIA Dynamo 和社区栈已支持该模式，通过 UCX 配合 GPUDirect RDMA、NCCL 点对点通信或框架自带的传输机制来移动键值（KV）缓存。我们还会讨论广泛使用的模型服务引擎，包括 vLLM、SGLang 和 NVIDIA TensorRT LLM。随后介绍 NVIDIA Dynamo 分布式推理框架；它与这些引擎集成，并包含 NIXL，用于在分离式预填充—解码部署中实现低延迟 KV 缓存传输。

我们还将研究如何利用 NVL72 中的 Grace CPU 进行预处理、并行运行更小的“草稿”模型以支持推测解码等高性能推理算法，以及高效地路由请求和批处理，从而最大化推理系统的整体吞吐量。我们还会探索模型压缩和加速技术，例如 4 位量化、通过知识蒸馏让更小的“学生”模型向更强的“教师”模型学习、稀疏化与剪枝，以及使用专用 TensorRT kernel。本书重点讨论分离式预填充—解码，以及在运行时动态调优系统的自适应技术。

第 20 章介绍利用 AI 辅助工具优化 kernel 和 AI 系统性能的现代实践。该章还包含高效训练和提供数十亿至数万亿参数模型服务的案例研究，并覆盖自我改进的 AI 系统优化与智能体的新兴趋势。这有助于描绘 100 万亿参数规模的 AI 系统将走向何方，以及我们如何为 AI 系统性能工程的未来做好准备。

附录提供了一份常见性能优化和节省成本技巧的清单，可应用于你自己的 AI 系统。它汇总了本书通篇讨论的可操作优化和效率提升方法。

简而言之，本书采用动手实践和基于经验的方法来应用性能优化。我们会经常分析实际运行、案例研究、基准测试结果和性能分析数据，以理解瓶颈并验证改进。到本书结束时，你应当掌握优化超大型 AI 系统的原则，并获得一些工具的实践经验，以便把这些优化应用到超大规模、多 GPU、多节点、多机架的 AI 系统上，例如 NVIDIA GB200/GB300 NVL72 机架式 AI 超级计算机——或现在与未来的类似 AI 系统。

1.13 关键要点

以下品质共同定义了 AI 系统性能工程师这一角色：他们把深厚的技术知识与战略性、由性能分析驱动的优化结合起来，将原始硬件转化为高性价比的高性能 AI 解决方案。

衡量有效吞吐量。不要只看原始 FLOPS 或利用率。相反，要衡量 GPU 花在执行有用工作（如前向/反向传播计算）上的时间，与等待数据或其他开销的时间之比。使用 NVIDIA Nsight Systems/Compute 或 PyTorch profiler 测量该比值。努力提高这一比值，因为有效吞吐量关注的是有效、有用的 GPU 利用率。

优先采用巧妙的工程优化，而不是暴力投入。更多硬件并非万能灵药。当硬件受限时，巧妙的软件和系统优化可以弥合差距，实现原本需要昂贵得多基础设施才能达到的结果。我们在 DeepSeek 的成就中看到了这一点。通过优化通信和硬件使用，他们的工程师在受限的 H800 GPU（互连带宽有限）上以极低的成本训练出一个前沿模型。DeepSeek 模型达到了在强大得多硬件上训练的前沿模型的性能。换言之，巧妙的工程胜过了暴力投入。

通过增量优化追求数量级的影响。在大规模下，即使效率只提升几个百分点，也能节省数百万美元。换句话说，冗余计算和缓慢数据流水线等小低效会随着系统扩展而悄然增加成本。

以性能分析驱动的心态开展性能调优。用数据和性能分析工具指导优化。使用性能分析器找出真正的瓶颈——无论是计算利用率、内存带宽、内存延迟、缓存未命中，还是通信/网络延迟。然后针对该瓶颈应用有针对性的优化。

保持整体视角。提升 AI 系统性能涉及硬件（包括 GPU、CPU、内存和网络）以及软件（如算法和库）。任何一层的短板都可能成为整个系统的瓶颈。优秀的性能工程师会考虑软硬件协同设计：有时算法改动可以缓解硬件限制，有时新的硬件特性可以催生新算法。

持续了解最新硬件、软件和算法。现代 AI 硬件和软件发展迅速。统一 CPU-GPU 内存、更快的互连和新颖的数值精度格式等新能力会改变最优策略。优秀的性能工程师会关注这些变化，并相应更新自己的心智模型，以快速消除瓶颈。此外，MLPerf 基准测试套件是了解各种模型 AI 硬件性能的绝佳资源。

1.14 结论

这一介绍性分析强调，在大规模下优化不是可选项——而是绝对必要的。它决定了一个系统是能够工作，还是完全不可行。传统方法，无论是硬件还是算法，在这种规模下都会失效。要继续前进，我们需要先进硬件，也需要聪明的软件技术。

显然，AI 模型正在逼近物理资源极限。硬件正在竞相跟上新的模型架构和算法。而性能工程师正是掌舵者，确保所有这些昂贵机器真正产出成果。

我们已经证明，AI 系统性能工程师的角色正变得越来越重要。单纯向问题砸钱和堆硬件是不够的。我们需要协同优化一切——模型架构、算法、硬件和系统设计——才能推动 AI 能力实现下一次跃迁。

作为 AI 系统性能工程师，你的工作是多学科、复杂且动态变化的。你可能今天深入研究 GPU 性能分析，明天研究网络拓扑，后天研究算法复杂度。这个角色适合热爱从硬件和软件中榨干每一滴性能的“全栈”性能极客。

本质上，AI 系统性能工程师的信条是“机械同理心”。我们深入理解机器——包括硬件和软件——从而能够量身定制高效解决方案，充分利用整个技术栈的性能能力。

正如我们前面所见，100 万亿参数模型远远超出当今硬件的能力——即使是百亿亿级系统，完成一次训练也需要数千年 GPU 时间。这显然不切实际，也凸显了为何我们需要先进硬件和巧妙的软件优化，才有可能达到这一规模。

在接下来的章节中，我们将演示如何把 AI 系统的各个组成部分——从处理器到内存、互连，再到软件框架——逐一拆解，并学习如何以有原则的方式优化每个组件。我们将研究一些具体案例：小小的改动带来性能和成本的巨大改善。通过这样做，我们将帮助建立一个心智模型，从多个维度推理性能优化。

到这段旅程结束时，作为读者和从业者的你，将掌握当今最佳实践，并具备应对未来挑战的工程思维。你将拥有一套技术武器库，可以在现在和未来把 AI 系统推向极限。对 AI 系统性能工程师而言，使命很明确：我们必须从这些创新中学习，并准备好在技术栈的每一层应用激进的优化。

现在，背景已经建立，让我们深入现代 AI 系统的硬件组件，包括 CPU、GPU、内存技术、网络结构和存储机制。通过研究支撑当代 AI 超级计算机的组件，你将掌握基础知识，为后续章节深入探讨优化技术打下根基。

第 2 章 AI 系统硬件概览

想象一下：把一台超级计算机规模的 AI 硬件压缩进一个机架。NVIDIA 的最新架构正是这样做的。本章将深入了解 NVIDIA 如何把 CPU 和 GPU 融合为强大的超级芯片，再通过超高速互连把几十个超级芯片连接起来，打造一台“盒中 AI 超级计算机”。我们将探讨 Grace CPU 和 Blackwell GPU 这些基础硬件构件，并了解它们的紧密集成和巨大的内存池如何让 AI 工程师的工作更加轻松。

接着，我们会向外扩展到连接 72 个 GPU 的网络 fabric，让它们仿佛是一台机器。在此过程中，我们将重点介绍赋予该系统超能力的计算性能、内存容量和效率方面的飞跃。读完本章，你将理解这种尖端硬件如何支持训练和服务此前看似不可能处理的数万亿参数模型。

2.1 CPU 与 GPU 超级芯片

NVIDIA 扩展 AI 的方法始于一个由 CPU 和 GPU 组合而成的单一超级芯片模块。从 Hopper 这一代开始，NVIDIA 将基于 ARM 的 CPU 与一个或多个 GPU 封装在同一个单元中，并用高速接口将它们紧密连接起来。结果是一个表现得像统一计算引擎的单一模块。

超级芯片的第一个实现是 Grace Hopper（GH200），它将一个 Grace CPU 与一个 Hopper GPU 配对。随后推出了 Grace Blackwell（GB200）超级芯片，在同一封装中将一个 Grace CPU 与两个 Blackwell GPU 配对。Grace CPU 位于模块中央，周围是两个 Blackwell GPU 裸片，如图 2-1 所示。

在传统系统中，CPU 和 GPU 拥有彼此分离的内存池，并通过相对较慢的总线（例如 PCIe）通信，这意味着数据必须来回复制。NVIDIA 的超级芯片通过名为 NVLink-C2C（chip-to-chip，芯片到芯片）的定制高速链路连接 CPU 和 GPU，消除了这一障碍。

在 GB200 超级芯片中，NVLink-C2C 在 Grace CPU 与 Blackwell GPU 之间提供最高约 900 GB/s 的带宽。相比之下，PCIe Gen5 x16（Blackwell B200）每个方向约为 64 GB/s，PCIe Gen6 x16（Blackwell Ultra B300）每个方向约为 128

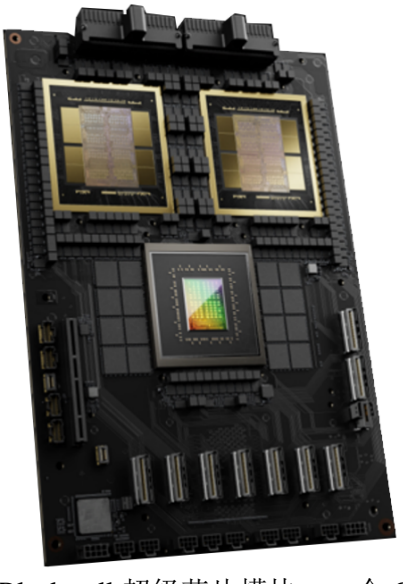

图 2-1　NVIDIA Grace Blackwell 超级芯片模块：一个 Grace CPU（中央）和两个 Blackwell B200 GPU（左上和右上）位于同一模块中，共享统一内存空间，并通过名为 NVLink-C2C（chip-to-chip，芯片到芯片）的定制高速链路连接。

GB/s。NVLink-C2C 的互连速度比典型 PCIe 快一个数量级。更重要的是，它具备缓存一致性。

缓存一致性意味着 CPU 和 GPU 共享一致的统一内存架构，因此它们始终能看到相同的值。实际上，超级芯片上的 Grace CPU 和 Blackwell GPU 都可以直接访问彼此的内存，就像访问一个巨大的内存池一样。GPU 可以读写存储在 CPU 内存中的数据，反之亦然，无需显式复制。NVIDIA 常把这种统一内存架构称为 Unified CPU-GPU Memory 或 Extended GPU Memory（EGM）；它实际上模糊了 CPU 内存与 GPU 内存之间的界限。

每个 Grace Blackwell 超级芯片都配备了海量内存。Grace CPU 连接着数百 GB 的 LPDDR5X DRAM，每个 Blackwell GPU 则拥有自己的高速、高带宽内存（HBM）堆栈。

在 GB200 超级芯片中，Grace CPU 提供最高约 480 GB 的 LPDDR5X，带宽最高约 500 TB/s；两个 Blackwell GPU 合计贡献最高约 384 GB 的 HBM3e 内存（每个 GPU 总计 192 GB）。因此，GB200 超级芯片总共提供约 900 GB 的一致、统一内存，GPU 和 CPU 可以在统一地址空间中访问它。

简单说，每个超级芯片都拥有近 1 TB 的快速统一内存可供使用。这对巨型 AI 模型来说是一次改变游戏规则的升级。在旧系统中，单个 GPU 的内存可能被限制在小于 100 GB，这意味着更大的模型必须分片，或换出到更慢的存储设备。现在，GPU 可以无缝地把 CPU 内存用作扩展。

如果神经网络层或大型嵌入表无法放入 GPU 的本地 HBM，它可以驻留在 CPU 内存中，GPU 仍能通过 NVLink-C2C 使用它。从程序员的角度看，统一虚拟地址空间和一致性简化了正确性问题。不过，从性能角度看，应使用异步预取和分阶段流水线等技术，显式管理数据放置和内存移动。通过 NVLink-C2C 访问 LPDDR5X 的延迟更高，带宽大约比直接访问 HBM 低一个数量级。

GPU 内存仍然比 CPU 内存快得多，也更靠近 GPU 核心；可以把 CPU 内存看作容量更大但略慢的扩展。访问 LPDDR5X 不如访问 GPU 上的 HBM 快：其带宽大约低 10 倍，延迟也更高。智能运行时会把最常用的数据留在 HBM 中，并使用 CPU 的 LPDDR5X 存放溢出数据或对速度不太敏感的数据。关键在于，溢出数据不再需要写入 NVMe SSD 或跨网络传输。

GPU 或许能以约 900 GB/s 的速度从 CPU RAM 获取数据（每个方向 450 GB/s）；虽然这比 HBM 慢，但仍远快于从 NVMe SSD 存储获取数据。这种灵活性至关重要：例如，一个大小为 500 GB 的模型（对单个 GPU 的 HBM 来说太大）仍可完整放入一个超级芯片模块中，使用合计 192 GB（可用 180 GB）的 HBM 和约 500 GB 的 CPU 内存。该模型无需跨多个 GPU 分片即可运行；GPU 只会在需要时透明地从 CPU 内存拉取额外数据。

本质上，只要超大模型的总大小能容纳在超级芯片的 CPU + GPU 组合内存中，内存容量就不再是模型能否装下的硬限制。许多研究人员都遇到过模型无法放入 GPU 时令人头疼的“内存不足”错误；这种架构旨在显著拓宽这一边界。

2.1.1 NVIDIA Grace CPU

Grace CPU 本身也绝非弱者。它是 NVIDIA 为带宽和效率定制设计的 ARM Neoverse V2 CPU。在超级芯片中，它负责处理通用任务、预处理数据并将数据送入 GPU，同时管理与之相连的海量内存。它的时钟频率适中，但以巨大的内存带宽弥补了这一点：连接 LPDDR5X 内存时最高可达约 500 GB/s，并拥有大量缓存，其中包括超过 100 MB 的 L3 缓存。

设计理念是：CPU 在把数据源源不断地送往 GPU 时绝不能成为瓶颈。它可以从存储设备流式读取数据，或执行分词、数据增强等即时数据变换，再通过 NVLink-C2C 高效地向 GPU 供数。如果工作负载中的某一部分更适合 CPU，Grace 核心可以处理它，并让 GPU 立即访问结果。

这是种协调的耦合：CPU 在 GPU 较弱的领域（例如随机内存访问或控制密集型代码）扩展 GPU 的能力，而 GPU 则在 CPU 无法跟上的数值计算中提供加速。

CPU 与 GPU 之间的低延迟链路意味着它们可以交换任务，而不必承受通常的开销。例如，从 CPU 启动 GPU kernel 的速度可以比传统系统快得多，因为命令不必穿过缓慢的 PCIe 总线。CPU 和 GPU 基本位于同一块电路板上。这类似于调用快速的本地函数，而不是较慢的远程函数。下面来看看 Blackwell GPU——超级芯片中的蛮力计算引擎。

2.1.2 NVIDIA Blackwell“双裸片”GPU

Blackwell 是 NVIDIA 对这一代 GPU 的代号；与上一代 Hopper（H100）GPU 相比，它在计算能力和内存方面都实现了显著飞跃。Blackwell B200 和 B300“Ultra”GPU 并不是单个芯片，而是采用多芯片模块（MCM）设计，将两个 GPU 裸片置于一个模块中。因此，Blackwell 被称为双裸片 GPU（见图 2-2）。

> **注** 虽然本节会深入讨论双裸片架构的细节，但本书其余部分会把 Blackwell 的两个组合 GPU 裸片统称为“Blackwell GPU”。

这种 chiplet 方法把通常的一块巨大 GPU 拆分成更小的 GPU 裸片，再用封装内超高速的裸片到裸片互连将它们连接起来。为什么要这样做？因为单片裸片受制造工艺限制，硅片上芯片的尺寸不可能无限增大。通过把两个物理 GPU 裸片组合进一个模块，NVIDIA 可以使该模块的晶体管总预算翻倍。

对于 Blackwell B200 MCM，每个 GPU 裸片约有 1040 亿个晶体管和 96 GB HBM3e 内存。组合后的 GPU 模块约有 2080 亿个晶体管，每个 B200 GPU 的总内存为 192 GB（可用 180 GB）。相比之下，Hopper H100 GPU 约有 800 亿个晶体管和 80 GB HBM3（而 Blackwell 使用 HBM3e）内存。因此，Blackwell B200 的晶体管数量超过两倍，内存容量增加约 2.4 倍。

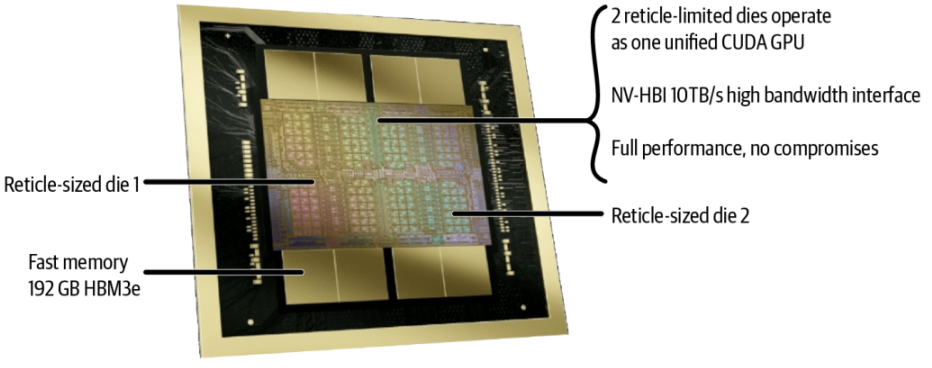

图 2-2　Blackwell 双裸片多芯片模块（MCM）设计。

Blackwell 的两个 GPU 裸片通过一种专用的 10 TB/s 高速裸片到裸片互连 NV-HBI（High-Bandwidth Interface，高带宽接口）进行通信。这使模块中的两个 GPU 裸片能够作为一个统一 GPU 工作。运行在其上的软件层只会看到一个 GPU。

从系统角度看，Blackwell GPU 是一个单一模块或设备，拥有一大块内存（192 GB［可用 180 GB］HBM3e）和大量执行单元；但在内部，它是两块协同工作的芯片。NVIDIA 的软件和调度机制确保工作负载在两个 GPU 裸片之间均衡分配，且内存访问保持一致。这样，开发者基本可以忽略这一复杂性，因为按照 NVIDIA 的设计，它们表现为一个 GPU。

每个 Blackwell B200 GPU 模块在两个 GPU 裸片之间合计拥有 192 GB（可用 180 GB）的 HBM3e 内存（每个裸片 96 GB），并划分为 8-Hi 堆栈。一个 8-Hi HBM3e 堆栈通过垂直堆叠八个 DRAM 裸片构成，每个裸片 3 GB，因此每个堆栈总计 24 GB。

B200 GPU 使用八个这样的堆栈（每个裸片四个），提供 192 GB（可用 180 GB）的封装内存（192 GB = 8 × 24 GB）。与上一代 Hopper GPU 相比，这增加了每个 GPU 的堆栈数量和容量，为模型参数、激活值、梯度和输入数据提供了更大余量。

> **注**　由于纠错码（ECC）、系统固件占用、制造限制以及其他使芯片无法暴露完整 192 GB 的因素，每个 B200 的 192 GB HBM3e 内存中只有 180 GB 可用。因此，后文提到 Blackwell B200 的可用内存时，将使用 180 GB，而不是完整的 192 GB。

内存速度也更快：Blackwell B200 的 HBM3e 每个 GPU 的聚合带宽最高约为 8 TB/s。相比之下，Hopper 使用上一代 HBM3，每个 GPU 可提供约 3.35 TB/s。因此，Blackwell 的内存带宽吞吐量约为 Hopper 的 2.4 倍。

以每秒 8 TB 的速度供数时，Blackwell GPU 核心可以持续处理巨型矩阵，而不必频繁停下来等待数据。NVIDIA 还增强了片上缓存：Blackwell 的 L2 缓存总容量为 126 MB（每个裸片 63 MB）。这种缓存是 GPU 上容量较小但速度极快的内存，用于保存最近使用的数据。

与 Hopper 的 50 MB L2 缓存相比，Blackwell 将 L2 缓存容量提高了 2.5 倍以上，因此可以在芯片上保留更多神经网络权重或中间结果，避免额外访问 HBM。这再次帮助 GPU 确保计算单元很少因缺少数据而停滞。

接下来介绍 Blackwell GPU 如何配备专用的低精度 Tensor Core，以及 NVIDIA 名为 Transformer Engine 的、针对 Transformer 优化的硬件和软件 API。PyTorch 等框架，以及 vLLM 等推理引擎，会通过 CUDA、CUTLASS 和 OpenAI 的 Triton 等库使用这些优化；后续章节将讨论它们。

> **注** 请记住，本书其余部分将 Blackwell 的双裸片 GPU 简称为“Blackwell GPU”。

2.2 NVIDIA GPU Tensor Core 与 Transformer Engine

说到计算单元，Blackwell 引入了专门面向 AI 工作负载的增强功能。其中的核心是 NVIDIA 的 Tensor Core 技术和 Transformer Engine（TE）。Tensor Core 是 GPU 每个流式多处理器（SM）中的专用单元，能够以极高速度执行矩阵乘法运算。

此前各代产品已经配备 Tensor Core，但 Blackwell 的 Tensor Core 支持更多数值格式，包括 8 位和 4 位浮点数等极低精度格式。降低精度的思路很简单：用更少的位表示数字，就能同时执行更多运算；由于表示相同数字所需的位数更少，内存的有效容量也会增加。当然，这要求算法能够容忍少量数值精度损失。如今，许多 AI 算法在设计时就考虑了低精度数值格式。

NVIDIA 首创了 TE，用于在深度学习中自动调整并使用混合精度：关键层使用更高精度（FP16 或 BF16），不太关键的层使用 FP8。TE 会自动优化精度之间的平衡，目标是在较低精度下仍保持模型准确率。

在 Hopper 这一代中，TE 首次引入 FP8 支持，相比 FP16 将吞吐量提高了一倍。Blackwell 更进一步，引入了 NVIDIA FP4（NVFP4），这是一种 4 位浮点格式，使用的位数是 FP8 的一半。FP4 的位宽如此之小，理论上可以将 FP8 的计算吞吐量再提高一倍。图 2-3 展示了 FP8 和 FP4 相对于 FP16 的加速比。

一个完整的 NVL72 机架（72 个 GPU）在 4 位精度下的理论 Tensor Core 吞吐量超过 1.4 exaFLOPS（即 1.4×10^{18} FLOPS）。这是一个令人难以置信的数字，使这个单机架系统达到了世界上最快超级计算机的水平——尽管使用的是低精度 FP4。即使现实工作负载并不总能达到这一峰值，这种能力本身仍然令人惊叹。

现代 GPU 使用支持 NVFP4 的 TE，并改进了缩放和校准。在实践中，可以在 PyTorch 等框架中使用 TE 的 kernel 和模块来采用这些能力；当 FP8 和 NVFP4 能够保持准确率时，就使用它们。不过，在所有框架中，这并不是完全自动的逐层决策。

更先进的技术包括在训练和推理期间动态改变神经网络每一层的精度。目标是为每一层使用仍能保持模型准确率的最低精度。例如，TE 可能让神经网络的前几层保持 FP16，因为早期层可能对噪声敏感；但根据启发式规则，它可以决定对更能容忍误差的后续层，或对高精度并不那么关键的巨型嵌入矩阵使用 FP8 或 FP4。

所有这些都可以在 NVIDIA 库和 PyTorch 等 AI 框架内部完成。作为用户，你只需启用混合精度，就能获得几乎“免费”的巨大加速。第 9 章会讨论混合精度；现在只需知道，许多当代 LLM 正是出于这一原因使用混合精度。这些低精度格式相比 FP16 和 FP32 提高了训练速度，同时减少了准确率损失。Blackwell 的设计目标就是让 FP8 和 FP4 易于使用且高效。

这些低精度格式也会减少内存使用。与 FP8 相比，使用 FP4 会把每个参数所需的内存减半（而 FP8 又会把 FP16 的内存用量减半），这意味着 GPU 内存可以容纳更大的模型。

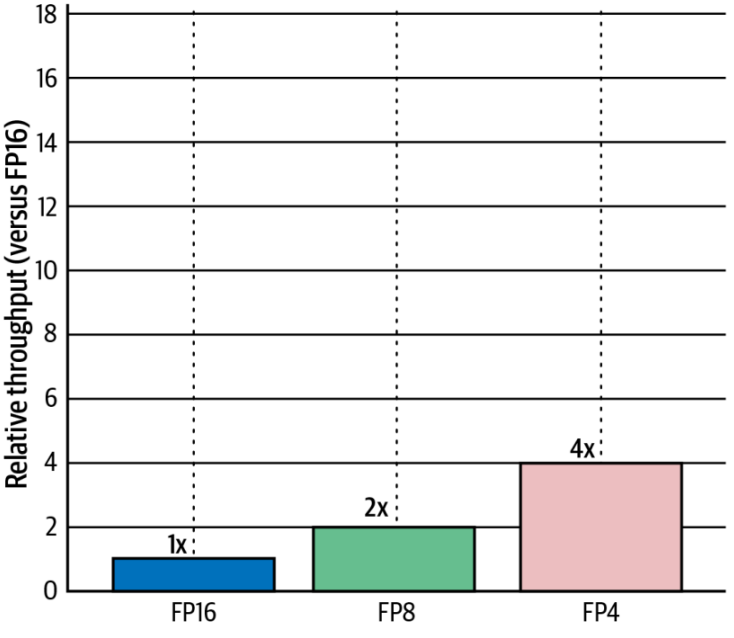

图 2-3　FP8 和 FP4 相对于 FP16 的加速比。

NVIDIA 实际上押注于 AI 的未来将采用更低精度的算术，并赋予 Blackwell 在这方面的卓越能力。这对超大模型的推理服务尤其关键，因为吞吐量（每秒 token 数）和延迟至关重要。

为了说明 Hopper 到 Blackwell 的代际飞跃，NVIDIA 报告称，基于 H100 的系统运行一个包含 1.8 万亿参数的大型 MoE 模型时，每个 GPU 只能生成约 3.4 个 token/s，首 token 延迟超过 5 秒。这对于交互式使用来说太慢了。

基于 Blackwell 的系统（NVL72）运行同一模型时，每个 GPU 达到约 150 token/s，首 token 延迟低至约 50 ms。这相当于相比 Hopper 这一代，实时吞吐量提高了约 30 倍。NVL72 让这个巨型模型能够提供实时响应，从而支持更多低延迟应用场景。

这种加速来自原始 FLOPS、更快的 GPU、低精度 FP4 的使用，以及让 GPU 持续获得数据的 NVLink 互连。这说明，横跨计算和通信的整体设计能够转化为真实世界的性能收益。

本质上，Blackwell GPU 比前代更强大、更智能，也能获得更充足的数据供给。得益于 Tensor Core、TE 和低精度，它们能更快地完成数学运算；同时，系统架构通过巨大的内存带宽、大容量缓存和 NVLink 确保数据能够快速到达。

在继续之前，先快速讨论 GPU 内部的层次结构，因为这有助于理解后续的性能调优。

2.3 流式多处理器、线程与 warp

与前代一样，每个 Blackwell GPU 由许多流式多处理器（SM）组成。可以把它们看作 GPU 的“核心”，如图 2-4 所示。

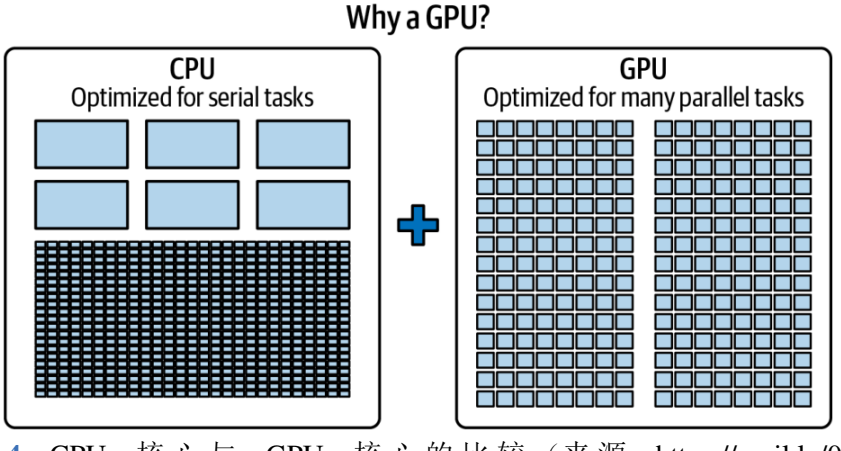

图 2-4 CPU 核心与 GPU 核心的比较（来源：https://oreil.ly/003EH、https://oreil.ly/Z25Tf）。

每个 SM 都包含一组算术单元（用于 FP32、INT32 等）、用于矩阵运算的 Tensor Core、用于内存操作的加载/存储单元，以及用于超越函数等运算的特殊函数单元。GPU 还有自己的少量超高速内存，包括寄存器、共享内存和 L1 缓存。

SM 以称为 warp 的固定大小线程组执行线程；每个 warp 恰好包含 32 个线程，这些线程以锁步方式执行完全相同的指令。这称为单指令、多线程（SIMT）执行模型。

SM 会并行执行许多活跃 warp，以覆盖线程等待从全局内存访问的数据时产生的延迟。设想一个 SM 同时有几十个 warp（数百个线程）处于执行中：如果一个 warp 正在等待内存获取，另一个 warp 就可以运行。这称为延迟隐藏。全书会反复回到延迟隐藏这一主题；它是调优工具箱中非常重要的性能优化手段。

像 Blackwell 这样的高端 GPU 拥有数百个 SM。每个 SM 都能同时运行数千个线程，这就是单个 GPU 上能容纳数万个活跃线程的原因。正如前文所述，所有这些 SM 共享 126 MB 的 L2 缓存，也共享连接 HBM 的内存控制器。内存层次结构为：寄存器（每线程）→ 共享内存（每个 SM 上的线程块）→ L1 缓存（每个 SM）→ L2 缓存（GPU 上所有 SM 共享）→ HBM 内存（片外），如图 2-5 所示。

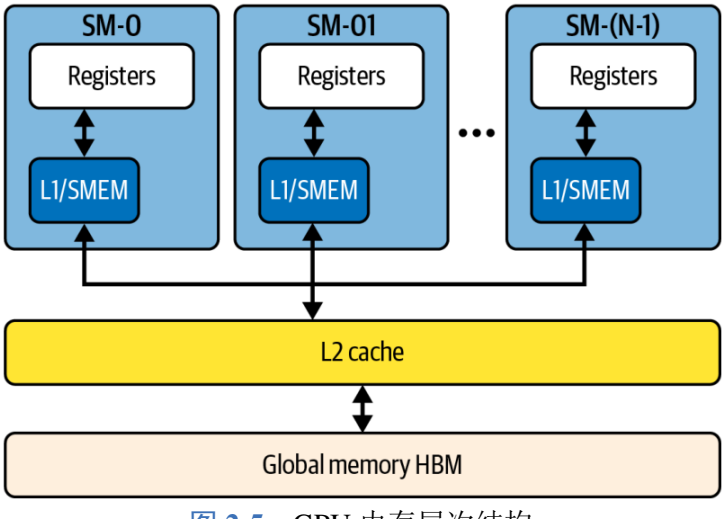

图 2-5　GPU 内存层次结构。

要获得最佳性能，数据应尽可能停留在这一层次结构的高层。如果每次操作都要访问 HBM，即使带宽达到 8 TB/s，GPU 也会因访问片外内存的延迟增加而频繁停滞。将可复用数据保留在 SM 本地内存或 L2 缓存中，GPU 就能实现巨大的吞吐量。Blackwell 架构将缓存和带宽翻倍，正是为了让这头 GPU“野兽”持续获得数据并保持高效运行。

作为性能工程师，我们会看到许多例子：kernel 的性能既受计算限制，也受内存流量和吞吐量限制。NVIDIA 显然将 Blackwell 设计成在许多 AI 工作负载中，让 FLOPS 与内存带宽达到良好匹配。

Blackwell 在计算和内存之间取得平衡，因此对于许多 AI kernel，GPU 可以以极少停滞持续计算。在实践中，经过良好优化的稠密数学运算能够从片上内存复用数据，在不严重受内存限制的情况下接近峰值 FLOPS。

这意味着，只要代码经过良好优化，GPU 往往会忙于计算，而不是等待数据。需要注意的是，巨型归约或随机内存访问等特定操作仍可能受内存限制；不过，更新后的 GPU、内存和互连硬件让这一问题有所缓解。

2.4 超大规模网络：让多个 GPU 如同一个整体

把两个 GPU 和一个 CPU 封装进超级芯片，就得到一个极其强大的节点。下一个挑战是把许多这样的超级芯片连接起来，以扩展到更大规模的模型训练。

NVIDIA 提供了一种使用 GB200/GB300 超级芯片的大型机架配置，称为 NVL72。NVL72 表示一个包含 72 个 Blackwell GPU 和 36 个 Grace CPU、并通过 NVLink 互连的系统。这本质上是一台装在单个机架中的 AI 超级计算机。

GB200/GB300 NVL72 由 18 个计算节点构成，每个节点包含两个 GB200/GB300 超级芯片，因此每个计算节点共有 4 个 Blackwell GPU 和 2 个 Grace CPU，如图 2-6 所示。

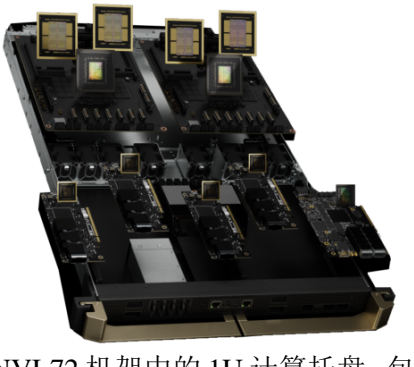

图 2-6　GB200/GB300 NVL72 机架中的 1U 计算托盘，包含两个 Grace Blackwell 超级芯片（来源：developer.nvidia.com）。

这里，每个超级芯片模块包含一个 Grace CPU 和两个 Blackwell GPU（每个 B200 都是双裸片 MCM）。NVL72 将 18 个这样的托盘连接起来。通过连接 18 个计算节点，GB200/GB300 NVL72 将 72 个 Blackwell GPU（18 个节点 × 4 个

GPU）和 36 个 Grace CPU（18 个节点 × 2 个 CPU）连接起来，形成一个强大的统一 CPU-GPU 集群。

NVL72 的有趣之处在于，在单个 NVLink 域内，每个 GPU 都可以通过 NVLink Switch fabric，以极高速度与其他任意 GPU 通信。NVIDIA 通过 GPU 上的 NVLink 5 连接与一种名为 NVSwitch 的专用交换芯片相结合，实现了这一点。

2.5 NVLink 与 NVSwitch

每个 Blackwell GPU 暴露 18 个 NVLink 5 端口。每个 GPU 的 NVLink 聚合双向带宽为 1.8 TB/s（18 条 NVLink 链路 × 100 GB/s 双向），NVL72 会把所有端口连接到 NVLink Switch System。每个 NVLink 交换机托盘提供 144 个 NVLink 端口，每个端口 100 GB/s。在 9 个托盘中，每个 GPU 的 18 条 NVLink 5 链路分别连接到一个 NVSwitch 芯片，因此 72 个 GPU 以全二分带宽完全互连。NVLink 5 的聚合双向带宽为每个 GPU 1.8 TB/s（18 条 NVLink 链路 × 100 GB/s 双向）。

这相当于 Hopper GPU 所使用的上一代方案中每个 GPU NVLink 带宽的两倍。Hopper H100 使用 18 个 NVLink 4 端口，但速度只有 NVLink 5 的一半。通过 NVLink 的 GPU 间延迟处于个位数微秒范围。

GPU 通过 NVSwitch 芯片以网络方式连接。NVSwitch 本质上类似网络交换机中的交换芯片，但它是专门为 NVLink 构建的。这意味着，在 NVLink Switch System 中，任意 GPU 都能通过一级交换阶段，以全二分带宽访问其他任意 GPU。由于每个 GPU 使用 18 条 NVLink 链路连接到 18 个 NVSwitch 芯片，这种单级特性在单个 NVL72 机架内成立，任意 GPU 之间都只需经过一个交换机。图 2-7 展示了 NVL72 中使用的 NVLink 交换机托盘。

每个交换机托盘包含两个 NVSwitch 芯片和多个高速端口。NVL72 机架包含 9 个这样的交换机托盘和 18 个计算托盘，如图 2-8 所示。

由于 9 个交换机托盘各包含两个 NVSwitch 芯片，NVL72 系统总共有 18 个 NVSwitch 芯片。网络采用全交叉结构：每个 GPU 都连接到每个 NVSwitch，每个 NVSwitch 也都连接到每个 GPU。这为任意一对 GPU 之间提供了高带宽路径。

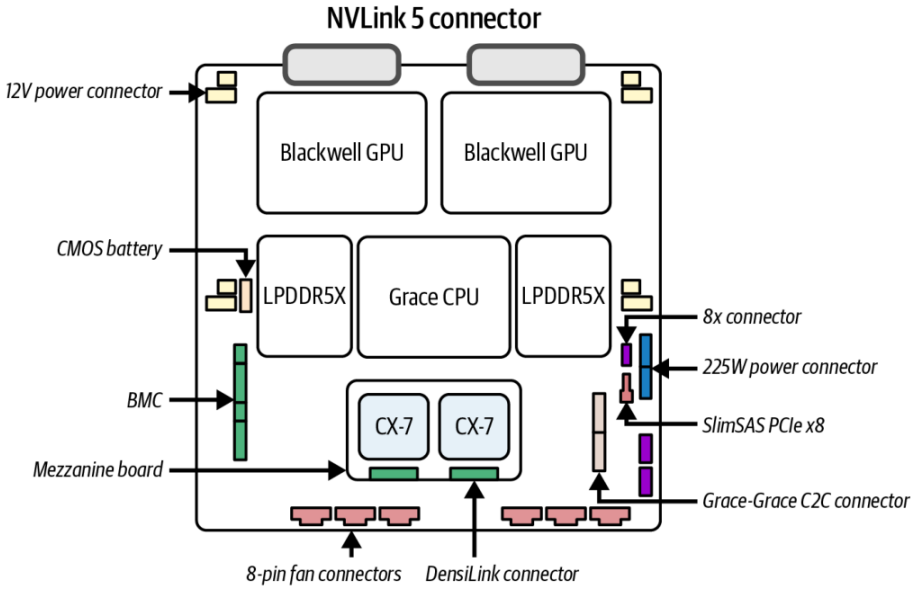

图 2-7　NVL72 内部的一个 NVLink 交换机托盘（来源：https://oreil.ly/h7seG）。

每个交换机托盘暴露 144 个 NVLink 端口，以完整连接每个 GPU 上的 18 条 NVLink 链路。具体来说，每个 GPU 使用 18 条 NVLink 链路分别连接到 18 个 NVSwitch 芯片（每个交换机一条链路）。这意味着任意 GPU 都能通过一跳（GPU → NVSwitch → GPU）访问其他任意 GPU，并在途中获得巨大的带宽。图 2-9 展示了完整的 NVL72 架构：72 个完全互连的 GPU（36 个 GB200 超级芯片）和 18 个 NVSwitch。

整个 72-GPU 网络的聚合二分带宽在 NVL72 机架内约为 130 TB/s。作为参照，这比同等规模的顶级 InfiniBand 集群高出许多倍。该设计提供了一个 GPU 之间完全互连、带宽很高且拥有全局地址空间的 fabric，既支持高效的 collective 和单边操作，又保留了由软件显式控制同步和一致性的能力。

2.6 多 GPU 编程

从编程模型角度看，一个 GPU 可以通过 NVLink，使用点对点和分区全局地址空间（PGAS）模型直接访问另一个 GPU 的内存，例如 NVIDIA SHMEM（NVSHMEM），即 NVIDIA 的 GPU 加速 OpenSHMEM 实现。系统存在全局地址

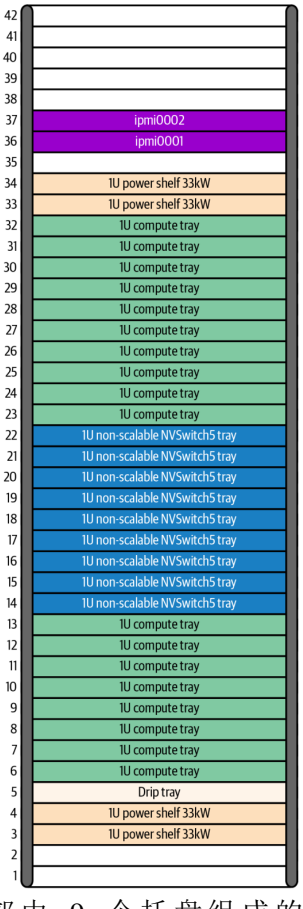

图 2-8 NVL72 机架内部由 9 个托盘组成的 NVSwitch System（来源：https://oreil.ly/h7seG）。

空间，但 GPU 缓存之间并不具有全局一致性；只有经由 NVLink-C2C 的 CPU-GPU 路径具备缓存一致性。NCCL 和 NVSHMEM 等软件栈提供多 GPU 访问所需的同步和排序保证。硬件缓存一致性与软件同步技术结合后，NVL72 基本可以被视为一个大型 GPU。

远程直接内存访问（RDMA）是一种网络技术，使主机之间能够通过 InfiniBand 和 RDMA over Converged Ethernet（RoCE）传输协议直接进行零拷贝内存传输。InfiniBand Trade Association（IBTA）为 InfiniBand 和 RoCE 定义了可选的远程原子操作。

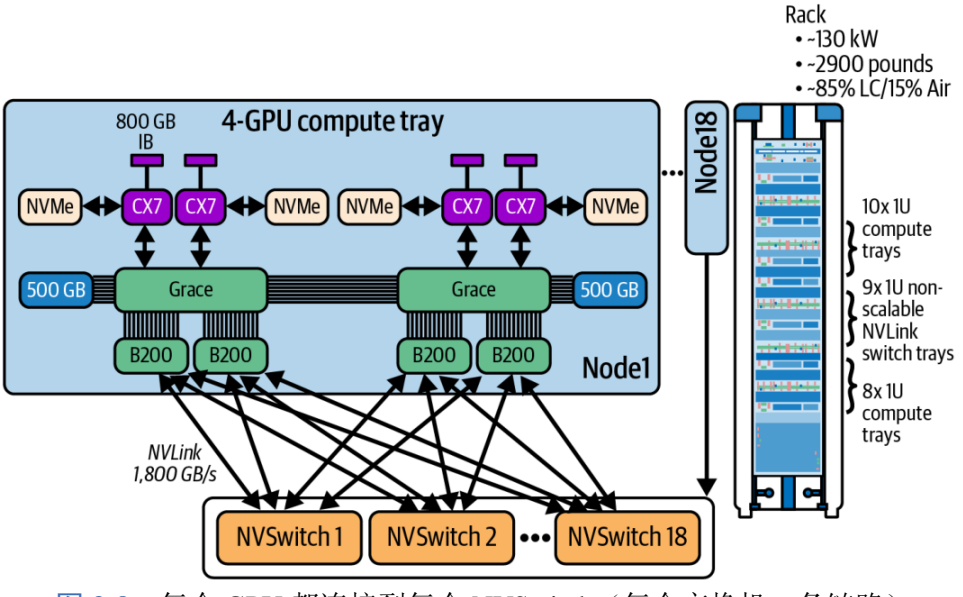

图 2-9　每个 GPU 都连接到每个 NVSwitch（每个交换机一条链路）。

GPUDirect RDMA 是 NVIDIA 对 RDMA 协议的实现，它允许网络接口控制器（NIC）注册 GPU 内存，并通过 nvidia-peermem 驱动直接对 GPU 内存执行 RDMA。这样，GPU 就能跨节点交换数据并执行原子操作，而无需 CPU 参与。NIC 可以直接对 GPU 内存执行 DMA，而不必经过主机 RAM 暂存。

跨节点的远程原子操作和单边操作由 NVSHMEM 等上层库提供；这些库通过 RDMA 传输实现相应语义。需要注意，GPUDirect RDMA 提供的是直接数据路径，而不是原子 API 本身。分布式训练和推理工作负载需要在许多 GPU 之间频繁同步和交换信息。

传统上，GPU 位于不同的计算节点和机架中，因此同步可能要通过 InfiniBand 和以太网等相对较慢的网络链路完成。在扩展到许多 GPU 以支持大型 AI 模型时，这通常会成为瓶颈。

在 NVL72 系统中，这些交换通过 NVLink 和 NVSwitch 以超高速完成。这意味着可以把训练任务或推理集群扩展到 72 个 GPU，同时保持极低的通信开销。由于 GPU 等待彼此数据的时间大幅减少，总体吞吐量在扩展到 72 个 GPU 时接近线性增长。

相比之下，考虑将同一个任务扩展到一个规模相近的 72-GPU H100 集群：它由 9 台独立计算服务器组成，每台配备 8 个 Hopper H100 GPU。这种配置需要使用 InfiniBand，而网络瓶颈会大幅降低集群的扩展效率。

让我们用具体数字分析和比较 NVL72 与 72-GPU H100 集群。在单个 NVL72 机架内，GPU 到 GPU 的带宽最高可达每个 GPU 1.8 TB/s（聚合双向），对于几 KB 量级的小消息，延迟约为 1–2 微秒。大消息耗时更长，通常受带宽限制。在传统 InfiniBand 网络中，每个 GPU 的带宽可能更接近 20–80 GB/s，具体取决于 NIC 数量及其速度；延迟则很可能为 5–10 微秒或更高。

与基于主机 NIC 的 fabric 相比，NVL72 网络提供了显著更高的每 GPU 带宽和更低的延迟。具体而言，NVLink 5 为每个 GPU 提供约 1.8 TB/s 的聚合带宽，而现代主机 NIC 在 400–800 Gb/s 的线路速率下，每端口约提供 50–100 GB/s。所有这些都能把 collective 操作开销从几十个百分点降到几个百分点。

在实践中，与传统节点间 fabric 相比，NVLink 连接的 NVL72 系统内部的 collective 开销显著更低，但迭代时间中具体占比取决于工作负载。例如，NVIDIA 报告称，一个 1.8 万亿参数 MoE 模型在 H100 上每个 GPU 约生成 3.4 token/s，首 token 时间超过 5 秒；而在 GB200 NVL72 上，每个 GPU 约生成 150 token/s，首 token 时间约为 50 ms。这一加速主要来自消除了 NVL72 机架内部的 GPU 间通信瓶颈，同时也受益于 Blackwell 更高的计算吞吐量。

在单个 NVL72 机架内，通信速度快到通信瓶颈几乎被完全消除，因此其优先级很低；而在传统 InfiniBand 和以太网集群中，通信往往是首要瓶颈，需要在软件层仔细优化和调优。

简而言之，应设计和实现能够利用 NVL72 配置的软件，让工作负载尽可能多的通信留在机架内部（“机架内”），从而利用高速 NVLink 和 NVSwitch 硬件。只有在需要超出 NVL72 的计算和内存资源时，才使用较慢的基于 InfiniBand 或以太网的机架间通信。

2.7 使用 NVIDIA SHARP 进行网络内聚合

另一项由硬件支持的优化是 NVIDIA Scalable Hierarchical Aggregation and Reduction Protocol（SHARP，可扩展分层聚合与归约协议）。对于 NVLink Switch

System 机架，网络内归约使用集成在 NVSwitch ASIC 中的 SHARP 引擎，在网络内部卸载归约和其他 collective 操作（见图 2-10）。

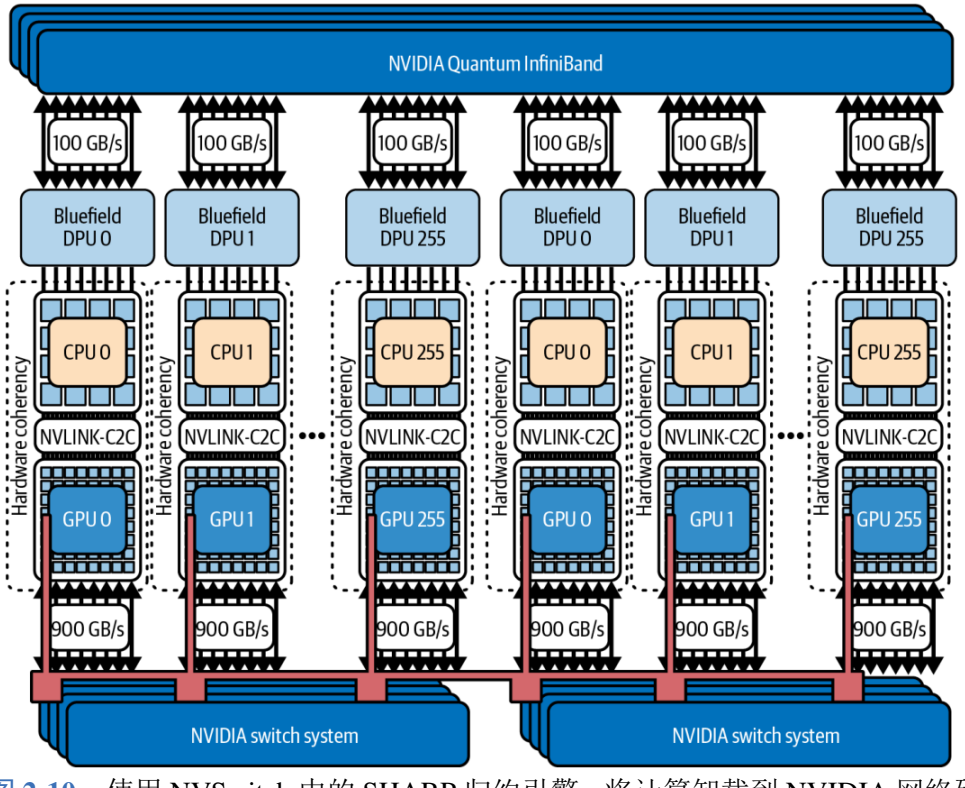

图 2-10　使用 NVSwitch 中的 SHARP 归约引擎，将计算卸载到 NVIDIA 网络硬件。

NVSwitch fabric 会合并部分结果，无需数据汇聚到 GPU。SHARP 将 collective 计算从 GPU 卸载到交换机硬件本身，使 GPU 能专注于更复杂的计算，降低 collective 延迟，减少在网络中传输的数据总量，并提高系统效率。

SHARP 提高的效率意味着，在分布式训练期间，聚合梯度或同步参数等繁重工作由 NVSwitch 的专用 SHARP 引擎处理。结果是在机架内和机架间配置中都能实现更高效的扩展。

借助 SHARP，即使 GPU 数量不断增加，也能看到接近线性的性能提升。这种网络内计算能力对于训练超大模型尤其关键，因为 collective 操作节省的每一微秒都可能转化为整体性能的显著提升。

> **注** SHARP 是 NVIDIA 在 2019–2020 年收购 Mellanox 时获得的最具影响力的创新之一。如果你目前还没有使用 SHARP，应当研究一下。SHARP 可以显著降低 collective 的延迟和流量，并且常常能提高通信受限训练的扩展效率。

2.8 多机架与存储通信

接下来讨论一个 NVL72 机架如何与另一个 NVL72 机架通信，或如何与共享文件系统等外部存储系统通信。正如前文所示，在 NVL72 机架内部，所有 GPU 到 GPU 的流量都由 NVLink 承担；但离开机架后，就要依赖更传统的网络硬件。

NVL72 中的每个计算节点都配备高速网络接口卡和数据处理单元（DPU）。DPU 将网络、存储、安全和管理任务从主机 CPU 卸载出来，进行加速并实现隔离。通过直接在 NIC 上运行这些操作，DPU 降低了 CPU 开销和延迟。

在 NVL72 设计中，BlueField-3 DPU 负责线速数据包处理、RDMA 和 NVMe over Fabrics（NVMe-oF）操作。NVMe-oF 是 NVMe 的一种协议变体，可将存储扩展到网络 fabric 上。因此，DPU 无需 CPU 参与，就能直接在网络、存储和 GPU 内存之间移动数据，从而最大化整体系统吞吐量和效率。

GB200/GB300 NVL72 机架与 Quantum-X800 InfiniBand 或 Spectrum-X800 以太网 fabric 集成。计算托盘通常为每个节点使用四个 ConnectX-8 800 Gb/s NIC，以提供高外部带宽。在需要网络内加速或为存储、安全和控制面任务进行卸载的场景中，会使用 BlueField-3 DPU。

四个 800 Gb/s NIC 意味着每个计算节点达到 3.2 Tbit/s，每个机架约达到 57.6 Tbit/s（57.6 Tbit/s = 3.2 Tbit/s 每节点 ×18 个节点）。虽然这一吞吐量非常惊人，但要记住，离开机架后仍需要超高速网络，才能使多机架扩展不会在机架边界处形成瓶颈。NVIDIA 将这些多机架部署称为 AI 工厂，并确保 NVL72 能通过每节点的四个 NIC 接入更大的网络 fabric。

每个节点中的 BlueField-3 DPU 有助于卸载 RDMA、TCP/IP 和 NVMe SSD 存储访问等网络任务。这样可以避免 Grace CPU 忙于管理网络中断。DPU 本质

上充当智能网络控制器，通过 NVIDIA 的 GPUDirect RDMA 软件在 NIC 与 GPU 内存之间直接移动数据，无需把数据暂存到主机内存，也不消耗 CPU 周期。

BlueField DPU 避免 CPU 参与，这在从存储服务器为大规模训练任务流式读取大型数据集时尤其有用。具体来说，DPU 可以处理传输并将数据直接写入 GPU 内存，而 CPU 则专注于数据预处理等其他任务。

除了提供性能卸载能力外，DPU 还支持安全多租户。它可以隔离不同任务和用户的网络流量，在节点上充当智能防火墙或交换机。

扩展到多个 NVL72 机架时，NVIDIA 使用 Quantum 系列 InfiniBand 交换机。多个 NVL72 机架可以通过这些 InfiniBand 交换机互连，形成一个大型 NVL72 机架集群。

例如，一个总计 576 个 GPU 的 8 机架 NVL72 部署，可通过 NVLink Switch System 连接为一个 NVLink 5 域。随后使用 InfiniBand 或以太网将这一 NVLink 域连接到其他域（例如其他 NVL72 机架）或外部存储；不过，跨机架 InfiniBand 或以太网通信的性能会低于机架内 NVLink/NVSwitch 通信。

简而言之，InfiniBand 和以太网 NIC（例如 NVIDIA 的 ConnectX）以及 BlueField DPU 通常与 NVLink 配合使用。它们为机架之间提供高带宽连接，并通过 DPU 上的网络内计算卸载协议处理工作。

2.9 预集成机架设备

由于 NVL72 是一个极其复杂的系统，NVIDIA 将它作为预集成的机架“设备”交付在一个机柜中。它出厂时已经组装好全部 18 个计算节点、全部 9 个 NVSwitch 单元、内部 NVLink 布线、配电系统和冷却系统。组织可以将其作为一个单元订购，设备到达后即可使用：只需将机架接入设施电源、连接水冷接口、将 InfiniBand 电缆接入网络，然后开机。

系统开箱后基本即可使用，只需极少设置就能开始运行 AI 工作负载。无需再为 72 个 GPU 单独铺设 NVLink，因为 NVIDIA 已经在机架内部完成了布线。甚至液冷系统也是自包含的，后文将对此进行讨论。

这种设备化方式加快了部署，并确保系统由 NVIDIA 正确构建和验证。机架还包含 NVIDIA Base Command 集群管理软件，以及用于集群作业调度和编排的 Simple Linux Utility for Resource Management（SLURM）和 Kubernetes。

简而言之，NVL72 机架的设计目标是直接放入你的环境，开箱即能运行生产级 AI 工作负载。它不需要手动安装或复杂配置。

2.10 共封装光学：网络硬件的未来

随着网络数据吞吐率提升到 800 Gbit/s、1.6 Tbit/s 乃至更高，NVIDIA 已开始把硅光子和共封装光学（CPO）集成到网络硬件中，包括 Quantum-X800 InfiniBand 和 Spectrum-X800 以太网平台。这些平台提供端到端 800 Gb/s 连接和网络内计算功能（例如 SHARP）。借助 CPO，光学发射器被集成在交换芯片旁边。这会大幅缩短电气路径，从而实现机架间更高带宽的链路、降低功耗，并提高整体通信效率。

在实践中，CPO 等技术正在铺平道路，让数百乃至数千个机架（AI 工厂）连接成一个统一 fabric，使机架间带宽不再成为瓶颈。这些光网络进步对于提供超大规模系统所需的高性能机架间带宽至关重要，确保网络能够跟上 GPU 的速度。

总结一下：在 NVL72 机架内部，NVIDIA 使用 NVLink 和 NVSwitch，在 72 个 GPU 之间创建一个极高速、全对全连接的网络。这些互连如此快速且均匀，以至于在许多 collective 操作中，GPU 实际上表现得像一个单元。机架之外，高速 NIC（例如 InfiniBand 或以太网）将机架连接到其他机架或存储，并由 DPU 高效管理数据移动。

NVL72 是一个极其强大的独立系统，也是更大型 AI 超级计算机或 AI 工厂的基础构件。由多个此类机架组成的大型 AI 数据中心——AI 工厂——如今正逐渐成为现实。NVIDIA 与 HPE、Supermicro 等 OEM 和系统厂商合作供应 GB200 NVL72 系统。NVIDIA 的硬件和网络路线图明确瞄准 AI 工厂愿景。简而言之，NVL72 展示了协同设计能够走多远：GPU、网络和物理机架硬件共同构建，以尽可能无缝、高效地扩展到数千乃至数百万个 GPU。

2.11 计算密度与功耗要求

NVL72 机架的计算密度极高，这意味着单个机架的功耗也非常高。满载 NVL72 在最大负载下可消耗约 130 kW 的功率，是 NVIDIA 上一代 AI 机架约 50–60 kW 功耗的两倍以上。把 72 个尖端 GPU 以及所有配套硬件装入一个机架，会把数据中心基础设施的承载能力推向极限。

要向 NVL72 机架提供 130 kW，不能只使用一条标准电源馈线。数据中心通常会配置多条大容量电路来提供这种功率。例如，数据中心可以部署两条完全独立的电源馈线；在这种情况下，每条馈线的容量都足以在另一条发生故障时承载整个机架负载。

如果一条馈线离线，剩余电路仍能支持完整的 130 kW 功耗，从而避免电路过载。这种冗余是重要的保护措施，否则断电可能会让持续数月的训练任务停止。

在机架内部，电力被分配给每个 1U 计算节点的电源。电力从交流电转换为直流电，供本地电子设备使用。NVL72 的每个计算节点包含两个 Grace Blackwell 超级芯片，合计功耗约 6 kW。18 个计算节点总计消耗约 110 kW。NVSwitch 托盘、网络交换机、风冷设备和水冷泵约占 20 kW，因此整个 NVL72 机架总功耗为 130 kW。

典型数据中心使用的电压（例如 415 V 三相交流电）下，电流非常大，因此所有设备都必须按高电流进行工程设计。运维人员必须仔细规划如何容纳这样的机架，这通常需要专用配电单元（PDU）和细致监控。还要考虑功率瞬态：72 个 GPU 从空闲提升到满功率时，可能在几毫秒内迅速拉取数十 kW 的功率。良好的设计会加入电容器或采用分时启动，以避免大幅电压跌落。

系统可能会让 GPU 的 boost 时钟相隔极小时间逐步提升，避免它们在完全相同的微秒级时刻同时出现功率尖峰，从而平滑电涌。这些都是让 130 kW 机架变得可管理所需的电气工程细节。

把这个处于高密度计算前沿的 NVL72 机架称为迷你变电站并不夸张。八个这样的机架组合成 576 个 GPU，将消耗近 1 MW 的功率（8 个机架 × 每机架 130 kW），这相当于一个小型数据中心的全部容量！积极的一面是，虽然单个机架的 130 kW 功耗很高，但它同时也能完成大量工作，因此每瓦性能很高。

如果一个 NVL72 能替代几架旧设备，总体效率会更高。但你确实需要基础设施来承载这种集中的功耗。任何部署 NVL72 机架的设施都必须确保有足够的供电能力和冷却能力，下一节将讨论后者。

2.12 液冷与风冷

在一个机架中冷却 130 kW 的热量，超出了传统风冷的能力。要让气流吹过 72 个、每个可散热约 1,200 W 的 GPU，需要飓风般的气流，声音极大且效率低下；更不用说排出的热空气会非常难以处理。因此，对于以这种功率密度运行的 NVL72 机架，液冷是唯一实用的解决方案。

NVL72 是一个完全液冷的系统。每个 Grace Blackwell 超级芯片模块和每个 NVSwitch 芯片都安装了冷板。冷板是一块内部带管道、直接贴在组件上的金属板。水基冷却液流过管道带走热量。所有冷板都通过软管、歧管和泵连接起来，使冷却液在系统中循环。

通常，机架会为每个节点配备快速断开接头，这样滑入或移出服务器时不会洒出冷却液。机架随后通过供水和回水接口连接到设施的冷冻水系统。通常还会有一个称为 Coolant Distribution Unit（CDU，冷却液分配单元）的热交换器，内置于机架中或紧邻机架。CDU 将机架内部冷却回路的热量传递给数据中心的水回路。

设施提供温度为 20–30°C 的冷冻水，水通过热交换器吸收热量。升温后的水随后被泵回冷水机或冷却塔，再次降温。在现代设计中，甚至可以采用温水冷却：冷却水以 30°C 进入系统，以 45°C 流出。随后可以通过蒸发式冷却塔冷却，而无需主动制冷，从而提高整体效率。关键在于，水或液体冷却剂单位流量所能携带的热量远多于空气，因此在狭小空间内运行高功率设备时，液冷的效果要好得多。

液冷使 GPU 和 CPU 温度远低于风冷条件，从而减少 GPU 的热节流。GPU 可以在不触及温度上限的情况下维持最高时钟频率。芯片在更低温度下运行还会提高可靠性，甚至提高效率，因为温度较低时功率泄漏更小。

在负载下，NVL72 将 GPU 温度保持在 50–70°C 范围内；对于如此耗电的设备而言，这非常出色。冷板和冷却回路经过精心设计，使每个 GPU 能向系统

排出 1,000 W、每个 CPU 能排出 500 W 的热量。此外，冷却液流速必须足够高，才能快速带走这些热量。粗略估计，要在水温升高 10–12°C 的条件下耗散约 130 kW，需要约每分钟 150–200 L 的流量。

系统无疑配备了用于监测冷却液温度、压力和泄漏的传感器与控制装置。如果滴漏传感器或压力损失传感器检测到泄漏，系统可以快速关停或隔离相应部分。建议使用自密封连接器，并考虑增加二次围挡托盘，以尽量降低液体泄漏风险。

这种机架级液冷曾经十分罕见，但如今已成为大型 AI 集群的标准。Meta、xAI 和 Google 等公司都在为 AI 集群采用液冷，因为风冷根本无法支持这些系统所消耗的巨大功率。

因此，虽然 NVL72 对设施提出了更高要求，包括液冷回路，但许多数据中心如今已经在按液冷进行建设。NVL72 机架自带内部液冷，可以直接接入冷却回路。

内部液冷的一个副作用是机架重量。装满硬件和冷却液后，NVL72 机架重量约为 3,000 磅（1.3–1.4 公吨）。对于机架来说，这极其沉重，大约相当于一辆小汽车的重量，却集中在几平方英尺的地面上。采用架空地板的数据中心必须检查地板能否支撑这一负载，通常以每平方英尺磅数计量。高密度机架往往放在加固板上，或由额外支柱支撑。移动这种机架需要叉车等专用设备。这些都是部署时必须考虑的事项，因为你安装的是一台 AI 超级计算机，它带来了独特的物理和物流挑战。

NVIDIA 还通过机架管理控制器集成了管理和安全功能，负责监控冷却泵、阀门位置、功耗以及每个节点的状态。管理员可以通过它更新所有节点的固件，或安全地关闭系统。

所有这些考量都说明，NVL72 是在充分考虑数据中心基础设施的前提下进行协同设计的。NVIDIA 与系统工程师共同设计计算架构，由后者解决供电和冷却问题；同时还与设施工程师协作，确定如何安装和运行这些设备。关键不只是芯片快不快，而是交付一个平衡、可用的系统。

这种复杂性带来的回报非常巨大。通过突破供电和冷却的限制，海量计算能力被集中到单个机架中，并转化为很高的每瓦计算性能。130 kW 的功耗确实

很高，但与把同样的 GPU 分散到多个、冷却效率较低的机架相比，按每个 GPU 或每万亿 FLOP（TFLOP）计算，它实际上更高效。

2.13 实践中的性能监控与利用率

面对如此强大的机器，你当然希望确保它得到充分利用。高效运行 NVL72 需要仔细监控性能、利用率和功耗。NVIDIA 提供了 Data Center GPU Manager（DCGM）等工具，可以跟踪每个 GPU 的指标，例如 GPU 利用率百分比、内存使用量、温度和 NVLink 吞吐量。

作为性能工程师，你会在训练运行和推理工作负载期间关注这些指标。理想情况下，训练任务的大部分时间里 GPU 利用率应接近 100%。如果看到 GPU 利用率为 50%，就意味着某些因素让 GPU 一半时间处于空闲状态，可能是数据加载瓶颈或同步问题。

同样，也可以监控 NVLink 的使用情况。如果 NVLink 链路频繁达到饱和，罪魁祸首很可能是通信。BlueField DPU 和 NIC 也有自己的统计信息，运维人员可以借此确认读取数据时存储链路没有饱和。NVL72 等现代系统会暴露这些遥测数据。

功耗监控同样至关重要。在约 130 kW 的功耗下，即使很小的低效或配置错误，也会浪费大量电力和资金。系统很可能允许你按节点或 GPU 监控功耗。如果不需要满性能，管理员可以限制 GPU 的功率或时钟，以节省能源。

NVIDIA GPU 支持设置功率上限。例如，如果运行的是不需要最后一点性能的小型任务，可以调低 GPU 时钟，以提高以每瓦性能衡量的效率，同时仍满足吞吐量要求。这一操作可能节省数 kW 的功率；经过数周训练后，便能转化为显著的节省和成本效率。

2.14 共享与调度

另一个方面是如何在 NVL72 上共享和调度工作负载。很少有单个任务需要全部 72 个 GPU。你可能有多个团队或多个实验，分别在 GPU 子集上运行。使用 SLURM 或 Kubernetes 等集群调度器及 NVIDIA 插件，可以在同一机架内为

一个用户划分 8 个 GPU，为另一个用户划分 16 个 GPU，再为第三个用户划分 48 个 GPU。

此外，NVIDIA 的 Multi-Instance GPU（MIG）功能可以在硬件层面把一个物理 GPU 拆分成更小的 GPU。例如，一个拥有 180 GB GPU 内存的 Blackwell GPU 可以被分成更小的分区，以便并发运行许多小型推理任务。

每个 Blackwell GPU 最多支持七个完全隔离的 MIG 实例。这允许一个物理 GPU 被划分为最多七个更小的 GPU，每个实例拥有专用内存和 SM。MIG 的大小由产品代际固定。下一章将深入介绍 MIG 分区的细节。

在实践中，对于这样的大型 GPU，MIG 可能用于推理场景：你希望在一个 GPU 上服务多个模型。BlueField DPU 的存在还支持安全多租户，因为 DPU 可以充当防火墙和虚拟交换机，隔离不同任务和用户的网络流量。这意味着组织可以让不同部门甚至外部客户安全地使用系统分区，而互不干扰——类似于云服务提供商将一台大型服务器划分给多个客户，并提供安全的多租户隔离。

从成本角度看，NVL72 这样的系统是一项价值数百万美元的资产，每月可能消耗数万美元的电费。因此，你确实希望尽可能多地完成有用工作，即获得尽可能高的有效吞吐量（goodput）。如果它处于空闲状态，大量资本成本和运营成本就会被浪费。这就是为什么监控一段时间内的利用率很重要：你可以跟踪已使用的 GPU 小时数与可用小时数。

如果发现系统利用率不足，可以考虑整合工作负载，或向更多团队开放系统以承接更多项目。有些组织会实行内部计费模式，由各内部团队从自己的预算中按 GPU 小时付费。这会鼓励高效使用，并计入电费和折旧成本。这样的透明度能确保人们重视这一资源。

2.15 升级硬件的投资回报率（ROI）

有人可能会问，投资这种最前沿的硬件是否值得。在分析投资回报率（ROI）时，答案通常归结为单位美元的性能。如果 NVL72 能完成比如四个旧一代机架的工作，那么从长期看，它可能在硬件和电力两方面都节省资金。本章前面讨论过，按吞吐量计算，一个 Blackwell GPU 可以替代 2–3 个 Hopper GPU。这意味着升级后，为完成同样的工作可能只需要更少的 GPU。

让我们分析一个简短的案例。假设你目前有 100 个 H100 GPU 负责工作负载。由于每个 Blackwell 的速度超过两倍（使用 FP8/FP4 时甚至更高），你可能用 50 个 Blackwell GPU 就能完成同样的工作。因此你只需购买 50 个，而不是 100 个 GPU。即使每个 Blackwell 的成本高于 H100，购买一半数量也可能做到成本持平甚至更好。在功耗方面，100 个 H100 可能消耗 70 kW，而完成同样工作的 50 个 Blackwell 可能消耗 50 kW，这是一项显著的节能。

一年下来，这一功耗差异可以节省数万美元。此外，GPU 数量减少意味着需要维护的服务器更少，服务器所需的 CPU、RAM 和网络资源开销也更低，从而进一步节省成本。总的来说，在某些情况下，升级到新硬件可以在 1–2 年内收回成本，尤其是当你有足够多的工作让它们每天 24 小时保持忙碌时。

具体计算当然取决于准确价格和使用模式，但重点是，对于大规模部署，采用最新 AI 硬件的 ROI 可能非常高。除了有形 ROI 外，使用一套强大的系统而不是许多小系统，还会带来简化系统架构等软性收益。这种简化通过降低功耗和网络复杂性来提高运营效率。

例如，不必因为内存限制而把模型拆分到多个旧 GPU 上，可以简化软件并降低工程复杂性。此外，拥有最新硬件就能利用最新的软件优化，并跟上同样升级的竞争对手。没有人愿意以竞争对手一半的速度训练和服务模型。升级会提升性能，同时支持更大的模型、更快的迭代和更迅速的响应。

高效运行 NVL72 既是软件和管理挑战，也是硬件能力的体现。硬件提供了惊人的潜力，但工程师必须通过监控性能、保持高利用率和智能调度任务，来释放硬件的全部能力。

好消息是，NVIDIA 提供了丰富的软件栈来监控和提升性能，包括驱动、性能分析器、容器运行时以及集群编排工具。本书后续将介绍如何优化软件，以充分利用 GB200/GB300 NVL72 这样的系统。现在要记住的是：当你得到一个盒中具备 exaFLOPS 级性能的 AI 系统时，需要同样先进的策略，让每一次浮点运算和每一个字节都发挥价值。

2.16 展望未来：NVIDIA 的路线图

在本书写作时，Grace Blackwell NVL72 平台代表着 AI 硬件的最先进水平。但 NVIDIA 已经在准备下一轮飞跃。简要了解未来几年的 NVIDIA 硬件路线图是有价值的，因为它展现出清晰的扩展模式：NVIDIA 计划继续加码性能、内存和集成度。

2.16.1 Blackwell Ultra 与 Grace Blackwell Ultra

NVIDIA 的 Blackwell Ultra（B300）及相应的 Grace Blackwell Ultra 超级芯片（GB300）是对 NVL72 架构的即插即用升级。每个 Blackwell Ultra B300 GPU 的内存容量约为 288 GB，比 B200 的 180 GB 增加约 50%；其 AI 计算性能提高 1.5 倍，并配备针对 attention 运算和低精度（例如 NVFP4）专门设计的更大裸片上加速器。这使 Blackwell B300 的推理吞吐量比 B200 高出 45%–50%。

一个由 GB300 组成的 72-GPU 机架包含 36 个 Grace Blackwell Ultra 模块（每个模块含 2 个 GPU + 1 个 CPU）、约 20.7 TB HBM（72 × 288 GB）以及约 18 TB DDR（36 × 500 GB）。合计起来，一个 GB300 NVL72 机架拥有约 38 TB 快速内存。GB300 NVL72 Ultra 的机架内 NVLink 和 NVSwitch 网络与 GB200 NVL72 使用同一代 NVLink 5。

简而言之，GB300 是 GB200 的演进式升级，因为它使用相同的架构；但它在各方面都有更多资源，包括更多 SM、更大的内存和更快的时钟。

2.16.2 Vera Rubin 超级芯片（2026）

Vera Rubin 超级芯片（VR200）以那位为暗物质提供证据的女天文学家命名，是下一步重要的架构。Vera 是基于 ARM 的 CPU，是 Grace CPU 的后继者；Rubin 则是 Blackwell 的后继 GPU 架构。NVIDIA 将继续沿用超级芯片概念，把一个 Vera CPU 与两个 Rubin GPU 组合在同一模块（VR200）中，类似于 Grace Blackwell（GB200/GB300）的配置。

Vera CPU 采用 TSMC 的 3 nm 半导体工艺，拥有更多 CPU 核心，并使用速度约为 1 TB/s 的 LPDDR6 内存。Rubin GPU 支持更高带宽的 GPU 高带宽内存（HBM），带宽约为 13–14 TB/s。

NVLink 预计也会进入第六代，即 NVLink 6，这将使 CPU 到 GPU 以及 GPU 到 GPU 的链路带宽翻倍。还有推测认为，Vera Rubin 可能允许每个机架容纳更多节点，或让每个 NVLink 域包含更多机架，从而突破 8 机架 GB200/GB300 NVL72 集群的 576-GPU 上限。

底线是，Vera Rubin 这一代在大多数指标上又实现了约 2 倍跃升，包括更多核心、更大内存、更高带宽和更多 TFLOPS。Rubin GPU 将每个裸片的 SM 数量提高到约 200 个，这可能进一步带来效率改进。它们还可能集成第二代 FP4 甚至试验性的 2 位精度等新功能，不过目前这仍只是推测。

另一个特别有趣的可能性是：由于 Rubin 的 288 GB HBM 对大型 AI 模型来说仍是瓶颈，NVIDIA 可能会把某种第二级 GPU 内存直接集成到 GPU 模块中。例如，可以将一些 LPDDR 内存直接放在 GPU 模块基板上，作为容量更大但速度更慢的 GPU 内存池，并与 Vera 的 CPU DDR 内存分离。

如果这种设想成为现实，单个 GPU 模块可能拥有约 550 GB 的缓存一致统一内存（288 GB HBM + 256 GB LPDDR）。这会进一步模糊 CPU 内存和 GPU 内存之间的界限，因为 GPU 自身将拥有多层内存层次结构。无论 Rubin GPU 这一代是否实现这一点，这都是值得关注的方向。

总体而言，Vera Rubin 和 Vera Rubin Ultra 机架的性能是 GB200/GB300 NVL72 的 5 倍，功耗也达到 5 倍，即每个机架接近 600 kW。VR200/VR300 NVL 系统在每个机架中、所有 Rubin GPU 之上拥有巨大的 GPU HBM 总量（每个

GPU 288 GB HBM），此外还有数十 TB 的 CPU 内存。机架内部的 NVLink 6 比 NVLink 5 产生更低的通信开销。

2.16.3 Rubin Ultra 与 Vera Rubin Ultra（2027）

按照这一模式，Rubin（R300）和 Vera Rubin 的“Ultra”版本会在原始版本一年后推出。有报道称，届时 NVIDIA 可能会转向四裸片 GPU 模块：把两个双裸片 Rubin 封装组合起来，形成四裸片 Rubin GPU。R300 Rubin Ultra GPU 模块在一个封装中拥有四个 GPU 裸片和 16 个 HBM 堆栈，单个 R300 GPU 模块的 HBM 总容量达到 1 TB。四个裸片组合起来后，核心数是双裸片 B300 模块的两倍。

具体而言，Vera Rubin NVL144 系统在整个机架中拥有 144 个这样的裸片，即 36 个每个含四个裸片的超级芯片模块。此外，还会有 Vera Rubin NVL576 配置，在完整系统中使用多裸片封装，使 GPU 数量增加到 4 倍。

到 2027 年，每个机架的计算性能可能达到 3–4 exaFLOPS，GPU HBM RAM 合计达到 165 TB（每个 Rubin GPU 288 GB HBM × 576 个 GPU）。虽然这些数字仍有些推测性，但朝着拥有海量 exaFLOPS 计算能力和 TB 级 GPU HBM RAM 的超大规模 AI 系统发展的轨迹已经十分清晰。

2.16.4 Feynman GPU（2028）与每年翻倍

NVIDIA 将 Rubin 之后的一代产品命名为 Feynman，计划于 2028 年发布。具体细节还很少，但 Feynman GPU 很可能转向更精细的 2 nm TSMC 工艺节点，采用 HBM5，并在模块内部集成更多 DDR 内存；还可能把裸片数量从四个增加到八个。

到 2028 年，推理需求预计将主导 AI 工作负载，尤其是随着 AI 模型的推理能力不断发展。推理所需的推理时计算量，可能是过去非推理模型的数百或数千倍。因此，芯片设计很可能会针对大规模推理效率进行优化，包括更多新型精度、更大的片上内存以及封装内光学链路，以进一步提高 NVLink 的吞吐量。

NVIDIA 似乎在每一代、甚至可能每一年都让某些东西翻倍：一年翻倍内存，下一年翻倍裸片数量，再下一年翻倍互连带宽，诸如此类。几年下来，这

种翻倍的复合效应非常巨大。NVIDIA 激进的路线从每一代都翻倍某项重要指标中可见一斑。例如，Blackwell 引入了双 GPU 裸片（每个模块两个裸片，而不是一个），NVLink 每条链路的双向带宽从约 900 GB/s 翻倍到约 1.8 TB/s，而每 GPU 内存则从 Blackwell 的 180 GB 增加到 Blackwell Ultra 一代的约 288 GB。Rubin 和 Feynman 还会进一步提升计算、内存和带宽。

NVIDIA 反复谈到 AI 工厂，把机架视为 AI 模型的生产线。NVIDIA 设想通过合作伙伴提供机架即服务，让企业可以租用超级计算机的一部分，而不是自己建造一切。随着尖端硬件以集成 pod 的形式交付、可以直接部署，这一趋势很可能会延续。每一代产品还允许你换入新的 pod，使容量翻倍、性能提高并降低成本。

对我们性能工程师来说，重要的是硬件将持续解锁新的规模水平。今天不可行的模型，几年后可能会变得司空见惯。这也意味着我们必须持续调整软件，以利用新的精度格式、更大的内存池和更先进的互连。这是一个令人兴奋的时代，因为前沿模型的发展与这些硬件创新密切相关。

2.17 要点回顾

以下创新共同使 NVIDIA 硬件能够以前所未有的速度、效率和可扩展性处理超大 AI 模型：

2.17.1 集成式超级芯片架构

NVIDIA 将基于 ARM 的 CPU（Grace）与 GPU（Hopper/Blackwell）融合进一个超级芯片，从而创建统一内存空间。这种设计消除了 CPU 与 GPU 之间手动数据传输的需要，简化了数据管理。

2.17.2 统一内存架构

统一内存架构和一致性互连降低了编程复杂性。开发者可以编写代码，而不必操心显式数据移动，这能加快开发，并让他们专注于改进 AI 算法。

2.17.3 超高速互连

通过使用 NVLink（包括 NVLink-C2C 和 NVLink 5）以及 NVSwitch，系统实现了极高的机架内带宽和极低的延迟。这意味着 GPU 通信时几乎可以像同一个大型处理器的不同部分一样，这对于扩展 AI 训练和推理至关重要。

2.17.4 高密度、超大规模系统（NVL72）

NVL72 机架在一个紧凑系统中集成 72 个 GPU。这种整合式设计将强大的计算性能与巨大的统一内存池结合起来，支持超大模型，完成传统设置难以实现的任务。

2.17.5 先进的冷却与电源管理

NVL72 依靠先进的液冷和稳健的配电系统运行，每个机架的功耗约为 130 kW（130 kW = 18 个节点 ×6 kW/节点+ ∼ 20 kW 的 NVSwitch/冷却/开销）。这种冷却和供电能力对于管理高密度、高性能组件并确保可靠运行必不可少。

2.17.6 显著的性能与效率提升

与 Hopper H100 等前代产品相比，Blackwell GPU 在计算和内存带宽方面大约提升 2–2.5 倍。这带来了显著的训练和推理速度提升：在使用 Blackwell 的 FP4 Tensor Core 和 Transformer Engine 的某些场景中，推理速度最高可快 30 倍；同时，由于所需 GPU 数量减少，还可能节省成本。

2.17.7 现代软件栈支持

NVIDIA 的软件和框架持续演进，以充分利用最新硬件，并支持最新的协同设计系统优化。这包括统一内存管理以及原生 FP8/FP4 精度支持。因此，工程师只需进行极少的代码修改，就能使用系统的全部性能。

2.17.8 面向未来的路线图

NVIDIA 的开发路线图（包括 Blackwell Ultra、Vera Rubin、Vera Rubin Ultra 和 Feynman）承诺持续将计算吞吐量、内存带宽等关键参数翻倍。这一轨迹旨在支持未来越来越大的 AI 模型和越来越复杂的工作负载。

2.18 结语

NVIDIA NVL72 系统凭借 Grace Blackwell 超级芯片、NVLink fabric 和先进冷却，体现了 AI 硬件设计的前沿水平。本章已经看到，每个组件都经过协同设计，服务于加速 AI 工作负载这一共同目标。CPU 和 GPU 融合为一个单元，消除了数据传输瓶颈，并提供巨大的统一内存。

几十个 GPU 通过超高速网络连接起来，表现得像一个通信延迟极低的巨型 GPU。内存子系统得到扩展和加速，以满足 GPU 核心旺盛的数据需求。甚至供电和热管理也被推向新的高度，以支持这种计算密度。

结果是，一个机架就能提供过去只有多机架超级计算机才能达到的性能。NVIDIA 对完整计算栈——芯片、板卡、网络和冷却——进行了端到端优化，使其能够在超大规模下训练和服务大型 AI 模型。

但这些硬件创新也带来了挑战：需要专用设施、周密的供电和冷却规划，以及复杂的软件才能充分利用它们。不过，回报是巨大的。研究人员现在可以使用前所未有规模和复杂度的模型进行实验，而不必等待数周或数月才能得到结果。一个在旧基础设施上可能需要一个月训练的模型，在 NVL72 上或许几天就能训练完成。过去每次查询需要数秒、勉强具有交互性的推理任务，如今可以实时完成，延迟达到毫秒级。这为此前不切实际的 AI 应用打开了大门，例如数万亿参数的交互式 AI 助手和 agent。

NVIDIA 快速的路线图表明，这还只是开始。Grace Blackwell 架构将继续演进为 Vera Rubin、Feynman 以及更后续的架构。正如 NVIDIA CEO 黄仁勋所说：“AI 正以光速发展，各家公司正竞相建设 AI 工厂，以扩展规模，满足推理型 AI 和推理时扩展所需的处理需求。”

NVL72 及其后继产品是 AI 工厂的核心，是处理海量数据、产生惊人 AI 能力的重型机械。作为性能工程师，我们站在这种硬件创新的肩膀上。它赋予我们巨大的原始能力，而我们的职责是开发软件和算法，充分发挥硬件潜力。

下一章我们将从硬件转向软件，探讨如何优化 NVL72 等系统上的操作系统、驱动和库，确保这些强大硬件不会被低效利用。后续章节还会讨论与软件架构相配套的内存管理，以及分布式训练和推理算法。

本书的主题是协同设计。正如硬件是为 AI 协同设计的，我们的软件和方法也必须经过协同设计，才能利用硬件。现在我们已经清楚理解硬件基础，可以进一步研究提升 AI 系统性能的软件策略。AI 超级计算时代已经到来，而充分利用它将是一段激动人心的旅程。

让我们开始吧！

第 3 章 面向 GPU 环境的操作系统、Docker 与 Kubernetes 调优

即使 GPU 代码和库已经过高度优化，系统级瓶颈仍可能限制大规模 AI 训练的性能。GPU 再快，也取决于为它提供数据和指令的环境。本章将探讨如何调优操作系统和容器运行时，让 GPU 发挥全部潜力。

我们先考察基础 GPU 软件栈，然后深入介绍 NUMA 亲和性和大页等关键 CPU 与内存优化。这些优化可确保数据从存储经 CPU 高效流向 GPU。与此同时，我们还会讨论持久化模式、Multi-Process Service（MPS）和 Multi-Instance GPU（MIG）分区等关键 GPU 驱动设置。它们可以减少开销并有效同步资源，从而维持 GPU 的最大利用率。

借助 NVIDIA Container Toolkit、Container Runtime、Kubernetes Topology Manager 和 Kubernetes GPU Operator 等方案，你可以为 GPU 环境构建统一且高度优化的软件栈。这些方案支持在单节点和多节点 GPU 环境中高效分配资源和调度工作负载，并确保充分利用 GPU 能力。

在此过程中，你将形成对这些优化为何重要的直觉。归根结底，它们会降低延迟、提高吞吐量，确保 GPU 始终获得数据并以峰值性能运行。最终，无论是训练还是推理工作负载，都能得到稳健、可扩展的系统，显著提升性能并获得较高的有效吞吐量（goodput）占比。

3.1 操作系统

操作系统（OS）是一切运行的基础。GPU 服务器通常运行 Ubuntu Server LTS 或 Red Hat 等 Linux 发行版，并使用支持最新 GPU 硬件的更新内核。NVIDIA 驱动会安装内核模块，创建诸如 /dev/nvidia0、/dev/nvidia1 和 /dev/nvidia2 的设备文件——每块 GPU 对应一个。驱动还会创建用于驱动控制操作的 /dev/nvidiactl、用于统一虚拟内存的 /dev/nvidia-uvm，以及用于模式设置和缓冲区管理的 /dev/nvidia-modeset。

OS 管理 CPU 调度、内存、网络和存储；为了获得高 GPU 吞吐量，这些部分都应进行调优。因此，应配置 OS，避免干扰 GPU 任务。例如，GPU 节点应禁用交换，或将 vm.swappiness 设为 0，以避免 OS 发起的内存换出干扰 GPU 工作负载。性能工程师的一项工作，就是调整这些 OS 设置，为 GPU 的最大性能做好准备。

面向 GPU 的服务器可能需要运行额外的守护进程或后台进程，例如 NVIDIA Persistence Daemon，以便即使没有 GPU 作业运行，也能让 GPU 驱动和硬件上下文保持加载和就绪。此外，Fabric Manager 负责管理 GPU 互连拓扑，NVIDIA Data Center GPU Manager（DCGM）则监控 GPU 系统健康指标。

3.2 NVIDIA 软件栈

运行一个多千万亿次浮点运算（petaFLOP）级 GPU 集群，不只是编写 PyTorch、TensorFlow 或 JAX 高层代码。GPU 操作之下有一整套软件栈，每一层都可能影响性能。图 3-1 展示了开发和产品化现代 LLM 工作负载时常用的一组框架、库、编译器、运行时和工具，包括 PyTorch、cuDNN、cuBLAS、CUTLASS、CUDA C++、nvcc 以及 CUDA Runtime API（例如 CUDA 工具、驱动等）。

此外，NVIDIA GPU 和 CUDA 生态也广泛采用 Python 库，并支持使用 OpenAI 的 Triton 领域特定语言（DSL）、NVIDIA 的 Warp 框架，以及 NVIDIA 的 CUDA Python、cuTile 和 CUTLASS 库在 Python 中创建 CUDA kernel。

3.2.1 GPU 驱动

底层是 NVIDIA GPU 驱动，它在 Linux OS 与 GPU 硬件之间建立接口。驱动管理 GPU 的底层操作，包括设备上的内存分配、GPU 核心上的任务调度，以及面向多租户使用的 GPU 分区。

GPU 驱动会启用 GPU 的各项功能，并持续为硬件提供工作。保持 NVIDIA 驱动为最新版本非常重要。新的驱动版本通常会解锁性能改进，并支持最新的 GPU 架构和 CUDA 功能。

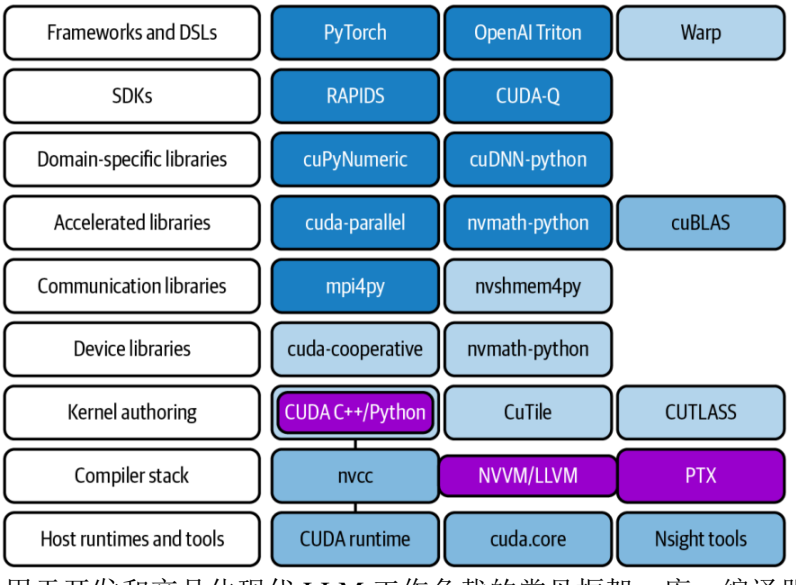

图 3-1　用于开发和产品化现代 LLM 工作负载的常见框架、库、编译器、运行时和工具。

驱动随附的 nvidia-smi 等工具可以监控温度、测量利用率、查询纠错码（ECC）内存状态，以及启用持久化模式等不同 GPU 模式。

3.2.2 CUDA Toolkit 与 Runtime

驱动之上是 CUDA Runtime 和称为 CUDA Toolkit 的库集合。该工具包包含 CUDA 编译器 nvcc，用于编译 CUDA C++ kernel。编译后，CUDA 程序会链接 CUDA runtime（cudart）。CUDA runtime 直接与 NVIDIA 驱动通信，以便在 GPU 上启动工作并分配内存。

此外，CUDA Toolkit 还提供许多经过优化的库：用于神经网络原语的 cuDNN、用于线性代数的 cuBLAS、用于多 GPU 通信的 NCCL 等。因此，务必使用支持你的 GPU 计算能力（CC）的最新 CUDA Toolkit 版本，因为最新工具包包含最新的编译器优化，以及针对你的 GPU 的专用库。我们将在后续章节更详细地介绍 CUDA 编译器和编程模型，以及 CUDA（和 PyTorch）优化。

3.2.3 跨 GPU 硬件代际的 CUDA 前向与后向兼容性

NVIDIA GPU 编程模型的一个重要特性，是它跨硬件代际的兼容性。编译 CUDA 代码时，生成的二进制文件会同时包含虚拟的中间 PTX 代码和物理设备代码（例如 ARM、x86 以及 GPU 指令），如图 3-2 所示。

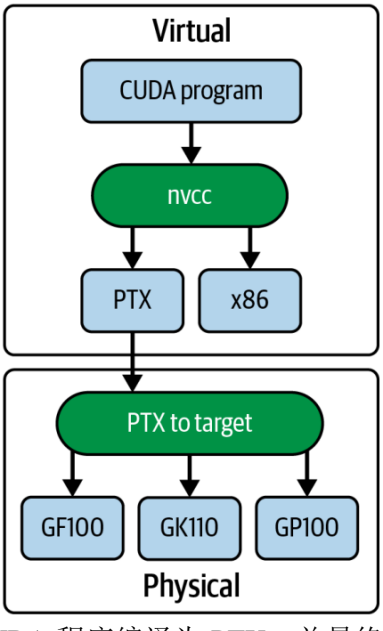

图 3-2　使用 nvcc 将 CUDA 程序编译为 PTX，并最终生成面向目标 GPU 设备的底层指令。

这样，较新的 GPU 就可以对 PTX 执行即时（JIT）编译，使程序能够运行在未来的架构上；同时，较新的 GPU 也能执行面向早期架构的旧二进制代码。这种兼容性通过 NVIDIA 的 fatbinary 模型实现：其中包含用于面向未来的 PTX，以及面向已知架构的 CUBIN（即特定架构的 CUDA 设备代码二进制文件）。

CUBIN 是 nvcc 使用 -cubin 选项生成的二进制文件，其中包含针对某一 NVIDIA 架构编译的 GPU 流式汇编器（SASS）指令。它会被打包进 fatbinary，由 CUDA 驱动在运行时加载。与作为中间且向前兼容的表示形式的 PTX 不同，CUBIN 二进制文件可以在已知的 GPU 架构上直接执行。当 CUBIN 与 PTX 一起包含在 fatbinary 中时，既支持为未来 GPU 对 PTX 进行 JIT 编译，也支持在较新的硬件上运行较旧的 CUBIN 代码。

简言之，嵌入 PTX 后，CUDA 具备前向兼容性，因为驱动可以在运行时为更新的架构 JIT 编译 PTX。CUBIN 对特定架构而言是专用的，并不向前兼容未来的 GPU 架构，因此应包含 PTX，或发布同时包含当前架构 SASS 和用于前向兼容的 PTX 的 fat binary（也称“fatbinary”或简称“fatbin”）。

3.2.4 C++ 与 Python CUDA 库

虽然大多数 CUDA Toolkit 库使用 C++，但 NVIDIA 当前面向 Python 的选项包括 CUDA Python（例如底层驱动和 runtime 访问）；用于数组编程的 cuPyNumeric、CuTe DSL、cuTile 和 CuPy；以及用于在 Python 中编写 GPU kernel 的 NVIDIA Warp。CUTLASS 是一个 C++ 模板库，底层由 cuBLAS 等库使用，而不是 Python 库。

虽然大多数 CUDA Toolkit 库基于 C++，NVIDIA 也在不断推出越来越多基于 Python 的库；它们通常以“Cu”开头，并构建在 C++ 工具包之上。例如，cuTile 和 cuPyNumeric 是 2025 年初推出的 Python 库，旨在降低 Python 开发者使用 CUDA 为 NVIDIA GPU 构建应用的门槛。

cuTile 是一个旨在简化 GPU 大矩阵处理的 Python 库，它将矩阵拆分成称为 tile 的更小、更易管理的子矩阵。它提供基于 tile 的高层抽象，使分块计算、内存访问模式优化以及 GPU kernel 的高效调度更加容易。

通过将大矩阵划分为 tile，cuTile 帮助开发者充分利用 GPU 的并行性，而无需手动管理底层细节。这种方法可以改善缓存使用和整体性能，适用于需要密集矩阵计算的应用。

cuPyNumeric 是常用 Python 库 NumPy 的直接替代品（import cupynumeric as np），并利用 GPU 提供计算能力。它提供与 NumPy 几乎相同的函数、方法和行为，因此开发者通常只需对代码做极少修改即可切换。底层实现中，cuPyNumeric 借助 CUDA 在 GPU 上并行执行操作，从而显著提升大规模数值计算、矩阵运算和数据分析等计算密集型任务的性能。

通过将工作卸载到 GPU，cuPyNumeric 可以加速计算，提高处理海量数据集的应用效率。它的目标是降低 Python 开发者使用 GPU 能力的门槛，无需学习完全不同的接口，因此是 NumPy 的强大高性能直接替代方案。

另一个值得注意的基于 Python 的编程模型，是 OpenAI 的开源 Triton 语言和编译器。Triton 是一种 Python DSL，支持用 Python 编写自定义 GPU kernel。虽然它不是 NVIDIA 库，但 Triton 通过让开发者直接在 Python 中编写高性能 kernel，补充了 CUDA。

我们将在后续章节介绍 Triton 及各种基于 Triton 的优化；现在只需知道，在许多情况下，Triton 减少了手写 CUDA C++ 的需要。它还集成进 PyTorch 编译器后端，可以自动优化并融合 GPU 操作，以获得更佳性能。下面转而讨论 PyTorch。

3.2.5 PyTorch 与高层 AI 框架

一些构建在 CUDA 之上的常用 Python 框架包括 PyTorch、TensorFlow、JAX 和 Keras。这些框架借助 NVIDIA GPU 的能力，为深度学习提供高层接口。本书主要关注 PyTorch 的编译和图优化功能，包括 torch.compile 栈。

PyTorch 编译器栈由 TorchDynamo、AOT Autograd 以及 TorchInductor 或 Accelerated Linear Algebra（XLA）等后端组成，它们会自动捕获并优化模型。TorchInductor 是最常见的后端，底层使用 OpenAI 的 Triton。Triton 会融合 kernel，并针对特定 GPU 和系统环境执行 kernel 自动调优；我们将在第 14 章介绍这些内容。

使用 GPU 对 PyTorch 张量执行操作时，张量看起来会在一次 Python 调用中从 CPU 移到 GPU。然而，这次调用实际上会被转换为一系列利用各种 CUDA 库的 CUDA runtime 调用，如图 3-3 所示。

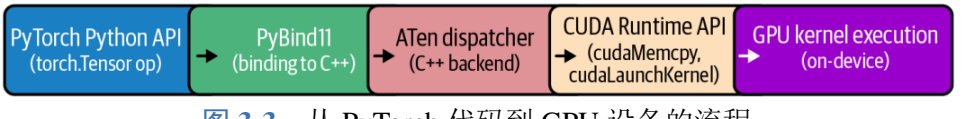

图 3-3　从 PyTorch 代码到 GPU 设备的流程。

例如，执行矩阵乘法时，PyTorch 会将这些任务委托给 cuBLAS 等库。cuBLAS 是 CUDA Toolkit 的一部分，并针对 GPU 执行进行了优化。在后台，PyTorch 确保前向和反向传播等操作使用底层、经过优化的 CUDA 函数和库执行。

简言之，PyTorch 抽象掉了直接进行 CUDA 编程的复杂性，使你能够编写直观的 Python 代码，而这些代码最终会调用高度优化的 CUDA 例程，同时兼具

易开发性和高性能。我们将在第 4 章和第 5 章讨论 CUDA 编程与优化，并在第 9 章讨论 PyTorch 优化。

所有这些组件——OS、GPU Driver、CUDA Toolkit、CUDA 库和 PyTorch——必须协同工作，才能构建理想的 GPU 开发环境。当研究人员提交训练作业时，调度器预留节点；OS 借助 NVIDIA 驱动提供 GPU 设备并分配内存；容器提供正确的软件环境，包括经过优化且感知硬件的 CUDA 库。用户代码（例如 PyTorch、TensorFlow、JAX）使用这些 CUDA 库，而 CUDA 库最终与驱动和硬件通信。

本章介绍的优化旨在让这一栈的每一层都尽可能高效。它们将帮助 GPU 忙于实际有用的训练和推理工作，而不是等待 CPU、内存或磁盘 I/O，或者等待其他 GPU 完成同步。

调优良好的系统可以确保分布在数十块 GPU 上的模型不会被 I/O 或 OS 开销卡住。系统级调优常常被模型优化掩盖，但系统级优化能够带来可观的性能提升。在某些情况下，只需对 OS 级配置做少量调整，就能获得两位数百分比的提升。对于大型 AI 项目，这可能节省数万乃至数十万美元的计算时间成本。

3.3 为 GPU 环境配置 CPU 与 OS

GPU 无法达到满利用率的最常见原因之一，是 CPU 没有持续为 GPU 提供有用的工作。在典型训练循环中，CPU 负责准备下一批数据，包括从磁盘加载数据、对数据进行 tokenization、变换数据等。此外，CPU 还负责分派 GPU kernel，以及协调线程和进程之间的工作。

如果这些主机侧任务速度很慢，或者 OS 调度不当，那么昂贵的 GPU 可能会空闲下来，只能空转并等待下一个任务或数据批次。为避免这种情况，我们需要优化 CPU 和 OS 处理 GPU 工作负载的方式。

这些优化包括设置 CPU 亲和性以避免跨 NUMA 节点流量，让正确的核心处理正确的数据；使用内存分配策略避免 NUMA 惩罚；以及应用 OS 级更改来消除不必要的延迟。这样，GPU 就不会因缺少数据而停转。其中一部分工作是将后台守护进程和 OS 任务隔离到专用核心上，使它们远离为 GPU 提供数据的核心；下一节将讨论这一点。

3.3.1 NUMA 感知与 CPU 绑定

现代服务器 CPU 拥有数十个核心，并且通常分为多个 NUMA 节点。NUMA 节点是 CPU、GPU、网络接口控制器（NIC）和内存在物理位置上彼此接近的一组逻辑资源。了解系统的 NUMA 架构对于性能调优十分重要。在单个 NUMA 节点内访问资源，比访问其他 NUMA 节点的资源更快。

例如，如果运行在 NUMA 节点 0 的 CPU 上的进程需要访问 NUMA 节点 1 上的 GPU，就必须通过节点间链路发送数据，从而产生更高延迟。事实上，跨 NUMA 节点访问时，内存访问延迟可能接近翻倍。

在基于 Grace 的超级芯片（如 GH200 和 GB200）上，CPU 与 GPU 通过 NVLink-C2C 相连，在 Grace 与其配对加速器之间提供最高约 900 GB/s 的一致性 CPU 到 GPU 内存访问。Linux 仍将 CPU DRAM 视为 CPU NUMA 内存，将 GPU HBM 视为设备内存。因此，即使一致性降低了软件开销，你仍应将 CPU 线程绑定到本地 Grace CPU，并遵循数据局部性原则。

在许多双路系统中，远程内存访问延迟可能显著高于本地内存访问。一次实验中，本地 NUMA 节点内存访问延迟约为 80 ns，而远程（跨节点）内存访问延迟约为 139 ns。这意味着延迟约增加 75%，本地和远程 NUMA 节点内存访问的速度差异非常大。

将进程绑定到与其 GPU 位于同一 NUMA 节点的 CPU 上，就可以避免这部分额外开销。例如，可以使用 numactl –cpunodebind=<node> –membind=<node>，将 CPU 线程和内存分配都绑定到 GPU 所在的本地 NUMA 节点。稍后将详细介绍这一点。关键思想是让 CPU 执行和内存访问都局部于它所服务的 GPU。

虽然 Linux 包含基本的 NUMA 均衡功能，但对性能关键型 AI 工作负载而言通常不够。默认情况下，进程可能在 NUMA 节点之间迁移，从而因远程内存访问产生额外延迟。因此，必须显式地将进程和内存绑定到本地 GPU 所在的同一 NUMA 节点。可以使用 numactl、taskset 或 cgroups 来实现，稍后会展示具体方法。

要显式指定 NUMA 亲和性，需要将进程或线程“绑定”到与 GPU 连接在同一 NUMA 节点上的特定 CPU。这种 CPU 亲和性称为 CPU pinning。假设一个

节点有 8 块 GPU，其中 4 块连接到 NUMA 节点 0，另外 4 块连接到 NUMA 节点 1。

如果启动 8 个训练进程，每块 GPU 对应一个进程，就应将每个训练进程绑定到与其 GPU 连接在同一 NUMA 节点的一个或一组 CPU 核心。在此例中，GPU 0–3 连接到 NUMA 节点 0，GPU 4–7 连接到 NUMA 节点 1 的核心，如图 3-4 所示。

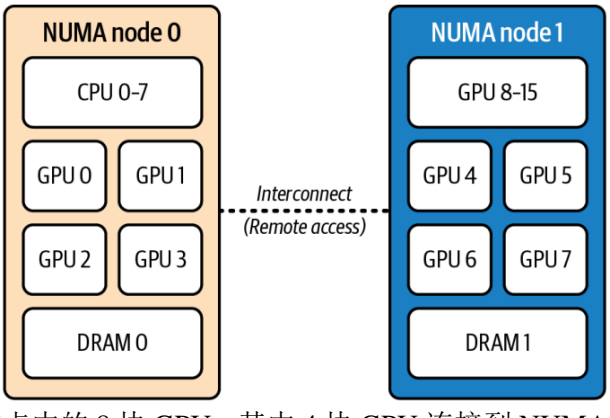

图 3-4 一个节点中的 8 块 GPU，其中 4 块 GPU 连接到 NUMA 节点 0，另 4 块连接到 NUMA 节点 1。

这样，当 CPU 进程想要向 GPU 4 提供数据时，就应运行在连接到 NUMA 节点 1 的 CPU 上，因为 GPU 4 连接到 NUMA 节点 1。Linux 提供了相应工具，包括 numactl –cpunodebind=<node> –membind=<node>；该命令会启动一个绑定到给定 NUMA 节点的进程。

也可以使用 taskset 将进程绑定到特定核心 ID。下面是使用 numactl 将 train.py 脚本绑定到与 GPU 4 位于同一 NUMA 节点 1 的 CPU 上的示例：

```
numactl --cpunodebind=1 --membind=1 \
    python train.py --gpu 4
```

这假定我们知道 NUMA 节点 ID，并且只将脚本绑定到一块 GPU。将 train.py 绑定到多块 GPU 且 NUMA 节点未知时，会复杂一些。下面的脚本通过 nvidia-smi topo 动态查询拓扑，并使用本地 NUMA 节点将脚本绑定到 GPU：

```
#!/bin/bash
for GPU in 0 1 2 3; do
  # Query NUMA node for this GPU
  NODE=$(nvidia-smi topo -m -i $GPU \
         | awk '/NUMA Affinity/ {print $NF}')
  # Launch the training process pinned to that NUMA node
  numactl --cpunodebind=$NODE --membind=$NODE \
    bash -c "CUDA_VISIBLE_DEVICES=$GPU python train.py --gpu $GPU"
done
```

这里使用 topo -m 获取 CPU 和 NUMA 亲和性，然后从 NUMA Affinity 列提取单个节点 ID。最后，将 –cpunodebind 和 –membind 都绑定到该节点，确保进程的线程和内存分配留在 GPU 所属的 NUMA 域内。

许多深度学习框架还允许以编程方式设置线程亲和性。例如，PyTorch 的 DataLoader 暴露了 worker_init_fn，你可以在每个 worker 进程初始化期间设置 CPU 亲和性，如下所示：

```
import os
import re
import glob
import subprocess
import psutil
import ctypes
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data import DataLoader, Dataset
from functools import partial
# Optional: NVML is preferred for GPU<->NUMA mapping
try:
    import pynvml as nvml  # pip install nvidia-ml-py3
    _HAS_NVML = True
except Exception:
    _HAS_NVML = False
# --- libnuma for memory binding
```

```
_libnuma = ctypes.CDLL("libnuma.so")
if _libnuma.numa_available() < 0:
    raise RuntimeError("NUMA not available on this system")
_libnuma.numa_run_on_node.argtypes = [ctypes.c_int]
_libnuma.numa_set_preferred.argtypes = [ctypes.c_int]
def parse_physical_cpu_list(phys_str: str):
    """Parse '0-3,8-11' -> [0,1,2,3,8,9,10,11]."""
    cpus = []
    if not phys_str:
        return cpus
    for part in phys_str.split(','):
        part = part.strip()
        if not part:
            continue
        if '-' in part:
            start, end = map(int, part.split('-'))
            cpus.extend(range(start, end + 1))
        else:
            cpus.append(int(part))
    return cpus
def get_numa_cpus_for_node(node: int):
    """Read /sys/devices/system/node/node{node}/cpulist."""
    path = f"/sys/devices/system/node/node{node}/cpulist"
    with open(path, "r") as f:
        return parse_physical_cpu_list(f.read().strip())
def get_numa_cpus_and_memory():
    """Return (current_cpu_mask, preferred_node) from numactl --show."""
    out = subprocess.run(["numactl", "--show"],
        capture_output=True, text=True).stdout
    phys = re.search(r"physcpubind:\s*([\d,\-\s]+)", out).group(1)
    cpus = parse_physical_cpu_list(phys)
    node = int(re.search(r"preferred node:\s*(-?\d+)", out).group(1))
    return cpus, node

def get_gpu_numa_node(device: int) -> int:
    """
```

```
    Determine NUMA node for a GPU (prefer NVML; fall back to sysfs;
    final fallback to current preferred node).
    """
    # NVML path (preferred)
    if _HAS_NVML:
        try:
            nvml.nvmlInit()
            props = torch.cuda.get_device_properties(device)
            pci = props.pci_bus_id  # '0000:03:00.0' or
             ↪ '00000000:03:00.0'
            # Normalize to 8-hex-digit domain if needed for NVML
            try:
                domain, bus, devfn = pci.split(':')
                if len(domain) < 8:
                    domain = domain.rjust(8, '0')
                pci8 = f"{domain}:{bus}:{devfn}"
            except ValueError:
                pci8 = pci
            try:
                handle = nvml.nvmlDeviceGetHandleByPciBusId_v2(pci8)
            except AttributeError:
                handle = nvml.nvmlDeviceGetHandleByPciBusId(pci8)
            # Direct NUMA ID if driver exposes it
            try:
                numa_id = nvml.nvmlDeviceGetNUMANodeId(handle)
                if isinstance(numa_id, int) and numa_id >= 0:
                    return numa_id
            except Exception:
                pass
            # Derive from NVML CPU affinity
            cpu_count = psutil.cpu_count(logical=True)
            elems = (cpu_count + 63) // 64
            mask = nvml.nvmlDeviceGetCpuAffinity(handle, elems)
            cpus = []
            for i, m in enumerate(mask):
                m = int(m)
```

```
                for b in range(64):
                    if m & (1 << b):
                        cpu_id = i * 64 + b
                        if cpu_id < cpu_count:
                            cpus.append(cpu_id)
            # Build CPU→NUMA map from sysfs and choose majority node
            cpu2node = {}
            for node_path in
             ↪ sorted(glob.glob("/sys/devices/system/node/node*")):
               node_id = int(os.path.basename(node_path).replace("node",
                ↪ ""))
                with open(os.path.join(node_path, "cpulist"), "r") as f:
                    for c in parse_physical_cpu_list(f.read().strip()):
                        cpu2node[c] = node_id
            counts = {}
            for c in cpus:
                n = cpu2node.get(c)
                if n is not None:
                    counts[n] = counts.get(n, 0) + 1
            if counts:
                return max(counts.items(), key=lambda kv: kv[1])[0]
        except Exception:
            pass
    # sysfs fallback
    try:
        props = torch.cuda.get_device_properties(device)
        pci = props.pci_bus_id
        sysfs_path = f"/sys/bus/pci/devices/{pci}/numa_node"
        with open(sysfs_path, "r") as f:
            val = int(f.read().strip())
            return val if val >= 0 else 0
    except Exception:
        pass
    # last resort: current preferred node
    _, node = get_numa_cpus_and_memory()
    return node if node >= 0 else 0
```

```
def set_numa_affinity(node: int):
    """Bind current process to CPUs and memory of the given NUMA node."""
    cpus = get_numa_cpus_for_node(node)  # IMPORTANT: CPUs of target
    ↪ node
    psutil.Process(os.getpid()).cpu_affinity(cpus)
    _libnuma.numa_run_on_node(node)
    _libnuma.numa_set_preferred(node)
    print(f"PID={os.getpid()} bound to NUMA node {node} (CPUs={cpus})")
    return cpus
def _worker_init_fn(worker_id: int, node: int, cpus: list):
    """Reapply binding in each DataLoader worker (no CUDA calls
    ↪ here)."""
    psutil.Process(os.getpid()).cpu_affinity(cpus)
    _libnuma.numa_run_on_node(node)
    _libnuma.numa_set_preferred(node)
    print(f"Worker {worker_id} (PID={os.getpid()}) bound to NUMA node
    ↪ {node}")

 # ----- Example usage below -----
class MyDataset(Dataset):
    def __len__(self): return 1024
    def __getitem__(self, idx): return torch.randn(224*224*3,
    ↪ device='cpu')
def main():
    # DDP setup
    dist.init_process_group(backend="nccl", init_method="env://")
    device = torch.cuda.current_device()
    # Determine GPU's NUMA node and bind this process
    gpu_node = get_gpu_numa_node(device)
    cpus = set_numa_affinity(gpu_node)
    # Build dataloader with closure-based worker_init_fn
    dataset = MyDataset()
    init_fn = partial(_worker_init_fn, node=gpu_node, cpus=cpus)
    dataloader = DataLoader(
        dataset,
        batch_size=32,
```

```
        num_workers=4,
        pin_memory=True,
        persistent_workers=True,  # reduces worker respawn churn
        worker_init_fn=init_fn,
        prefetch_factor=2,
    )
    # Model and DDP
    model = torch.nn.Linear(224*224*3, 10, bias=True).to("cuda")
    ddp_model = DDP(model, device_ids=[device], static_graph=True)
    for batch in dataloader:
        batch = batch.to("cuda", non_blocking=True)
        out = ddp_model(batch)
        # ... loss, backward, optimizer ...
if __name__ == "__main__":
    main()
```

该脚本将主训练进程和每个 DataLoader worker 进程绑定到 GPU 所在的本地 NUMA 节点，以避免跨 NUMA 节点的内存访问。在 DataLoader 中，我们传入基于闭包的 worker_init_fn，在每个 worker 内重新应用预先计算的 NUMA 绑定，而且不会在 worker 中触碰任何 CUDA API。

启动时，进程使用 NVML 将当前 GPU 映射到其 NUMA 节点和 CPU 亲和性掩码。如果可用，就通过 nvmlDeviceGetNUMANodeId 直接读取节点。否则，从 GPU 的 CPU 亲和性掩码（nvmlDeviceGetCpuAffinity）推导节点。如果 NVML 不可用或不暴露该节点，就回退到内核在 /sys/bus/pci/devices/<PCI_ID>/numa_node 中的 sysfs 条目。最后，再回退到进程当前的首选节点。

随后，我们从 /sys/devices/system/node/node<N>/cpulist 计算该节点的 CPU 列表，并使用 psutil 将 CPU 亲和性应用到这些核心。我们还通过 libnuma（numa_run_on_node + numa_set_preferred）将所有未来的内存分配绑定到该节点。

由于某些启动器、容器运行时或内核不能可靠地将 NUMA 策略传播给子进程，我们会在每个 fork 出的 worker 中显式重新应用并验证绑定。仅依赖继承是不安全的。

记得在 H2D 拷贝中设置 pin_memory=True 并使用 non_blocking=True，这样页锁定主机缓冲区就会留在正确的 NUMA 节点上。优先使用 persistent_workers=True，避免重新 fork worker，并避免它们在 epoch 之间丢失亲和性。不要在 worker_init_fn 中调用 torch.cuda.*；应通过闭包或环境变量传入 GPU 索引。

结果是，数据准备和批次加载完全在本地内存中进行。这样，GPU 始终保持繁忙，无需因跨 NUMA 节点跳转而暂停。只要 Linux 服务器安装了 libnuma 和 numactl，这段代码就能提供稳健且感知拓扑的亲和性。

默认情况下，numactl 会将 CPU 和内存策略应用于进程，并且文档说明这些策略会继承给所有 fork 出的子进程。但实际上，在不同内核或 Linux 发行版上，由 Python 框架创建的线程或通过 exec 启动的子进程不一定会采用相同设置。使用框架管理的 worker 进程时，应在每个 worker 内显式重新声明 CPU 和内存策略。

在 Grace Blackwell（以及 Vera Rubin）这类超级芯片架构中，CPU 与 GPU 通过 NVLink-C2C 保持一致性。不过，Linux 仍将 CPU DRAM 和 GPU HBM 建模为独立内存池。将 CPU 线程绑定到本地 CPU NUMA 节点仍有利于保持局部性。

实践中，绑定可以消除不可预测的 CPU 调度行为。它能确保 GPU 的关键线程（例如数据加载线程）不会在训练或推理期间突然被 OS 迁移到另一个 NUMA 节点的核心上。实际中，仅仅消除跨 NUMA 节点流量和 CPU 核心迁移，就可能带来 5%–10% 的训练吞吐量提升。这通常还会减少性能抖动和方差。

许多高性能 AI 系统会评估 CPU 同时多线程（SMT，也常称 hyperthreading），有时为了获得更可预测的单核性能而将其禁用，但收益取决于工作负载。这些系统还可能通过设置 isolcpus 内核参数，专门预留少量核心给 OS 后台任务，使其与通用调度器隔离。也可以使用 Kubernetes CPU 隔离来处理系统守护进程。这样，剩余核心就能完全用于训练和推理线程以及其他有用工作。

需要注意的是，对于 NVIDIA Grace Blackwell 这类 CPU-GPU 集成超级芯片，许多传统的 CPU 到 GPU 数据传输问题已经得到缓解：CPU 和 GPU 通过 NVLink-C2C 暴露一致的共享虚拟地址空间，而 CPU DRAM 与 GPU HBM 仍是

独立的内存池。这意味着跨 NUMA 延迟等问题会减小，数据也能更直接地在 CPU 与 GPU 之间流动。

NVIDIA 通过将 CPU 和 GPU 组合到 Grace Blackwell 这类单一超级芯片中来解决 CPU 到 GPU 瓶颈，并非偶然。在这种设计中，CPU 和 GPU 甚至通过 NVLink-C2C 共享最高 900 GB/s 的统一一致性内存，从而减少数据传输开销。预计 NVIDIA 会继续推出这类与软件和算法需求协同设计的硬件创新，以应对系统瓶颈。

即使 CPU-GPU 超级芯片架构紧密耦合，仍然必须优化软件栈，确保硬件和软件正确配置，使集成系统以峰值效率运行。即使在这种紧密耦合的架构中，也要尽量减少数据处理中的不必要延迟，以保持 GPU 的充分利用。这包括配置大页、使用高效预取以及绑定内存，后续各节会对此展开介绍。

3.3.2 NUMA 友好的内存分配与内存绑定

默认情况下，进程会从当前运行它的 CPU 所在 NUMA 节点分配内存。因此，如果将进程绑定到 NUMA 节点 0，内存自然会来自 NUMA 节点 0 的本地 RAM，这是理想情况。然而，如果 OS 调度器迁移了线程，或者在绑定之前已经分配了一些内存，就可能出现进程运行在 NUMA 节点 0、却使用 NUMA 节点 1 内存的非理想情况。此时，每次内存访问都必须跳转到另一个 NUMA 节点，从而抵消 CPU 绑定带来的收益。

为避免这种情况，numactl –membind 选项会强制从指定 NUMA 节点分配内存，如前一节所述。在代码中，也存在可以影响这一配置的 NUMA API，甚至环境变量。一般规则是让内存靠近 CPU，让 CPU 靠近 GPU。这样，内存到 CPU 再到 GPU 的数据移动链路就全部位于同一 NUMA 节点内。下面是前面示例的同一命令，但加入了 –membind=1，强制从首选的、包含 NUMA 节点 1 的 NUMA 节点分配内存：

```
numactl --cpunodebind=1 --membind=1 python train.py --gpu 5 &
```

需要注意的是，在 numactl 下启动进程时，其 CPU（–cpunodebind）和内存策略（–membind）都会应用于该进程，并继承给所有子进程。因此，训练脚本 fork 出的 worker 子进程会自动使用同样的 NUMA 内存绑定。但前提是它们必

须使用基于 fork 的模型。如果切换到 spawn 启动方式，或以其他方式 exec 一个新程序，这些子进程就不会继承父进程的内存策略。

此外，绑定内存（也称页锁定内存）对于高效、直接访问 GPU 至关重要。内存被绑定后，OS 不会将其换出或移动，从而实现更快的直接内存访问（DMA）传输。从绑定的主机内存向 GPU 拷贝数据，速度可能是从普通可分页内存拷贝的 2–3 倍，因为 GPU 或 NIC 可以直接执行 DMA。

> **注** 可以使用已安装 CUDA 工具中的 bandwidthTest –memory=<pinned or pageable>，测试 CPU 内存与 GPU 内存之间的数据传输带宽。

事实上，这正是 NVIDIA GPUDirect 技术（例如 GPUDirect RDMA）的基础。GPUDirect RDMA 允许 InfiniBand 等 NIC 直接与 GPU 内存交换数据。类似地，GPUDirect Storage（GDS）允许 NVMe 驱动器将数据流式传入 GPU 内存，而无需额外的 CPU 开销。

深度学习框架提供了让数据加载器使用绑定内存的选项。例如，PyTorch 的 DataLoader 有一个 pin_memory=True 标志；启用后，加载的批次会放入绑定 RAM，如图 3-5 所示。

内存绑定可以加快 tensor.to(device) 操作，因为 CUDA 驱动不必在运行时再绑定页面。当批次较大或每次迭代读取大量数据时，收益尤其明显。许多实践者发现，仅在 PyTorch 中开启 pin_memory=True，就可以通过减少数据传输瓶颈并提高主机到设备的传输吞吐量，将性能提升 10%–20%。

简言之，应确保数据加载器使用绑定内存（例如在 PyTorch DataLoader 中设置 pin_memory=True），并在受支持的硬件上启用 GPUDirect RDMA 和 GDS。这样可以降低数据传输延迟。

需要注意，OS 会限制用户能够锁定（绑定）的内存量。该限制通过 ulimit -l <max locked memory> 命令设置。在容器化环境中，可以相应调整容器的安全上下文和 Docker 的 –ulimit memlock 设置。这样，容器就能锁定足够的内存。

如果计划使用大型绑定缓冲区，应确保 ulimit 值足够高，或将其设为 unlimited。否则，分配可能失败。对于大型 AI 工作负载和高性能计算（HPC）应用，通常会将其设为 unlimited。

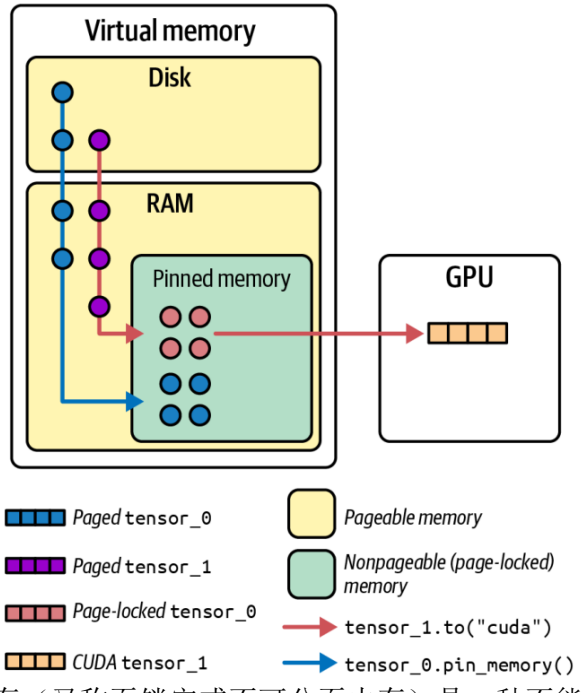

图 3-5　绑定内存（又称页锁定或不可分页内存）是一种不能换出到磁盘的内存。

3.3.3 透明大页

除了绑定内存并将其绑定到 NUMA 节点之外，还应讨论透明大页（THP）。Linux 内存管理通常使用 4 KB 页面，但当进程使用数十或数百 GB 内存时，管理数百万个小页面效率很低；深度学习数据集、预取批次和模型参数都可能出现这种情况。

大页——2 MB 甚至 1 GB 的页面——可以通过增大内存块来减少虚拟内存管理开销。其主要好处是减少缺页次数，并减轻转换后备缓冲区（TLB）的压力。

TLB 是 CPU 用来将虚拟地址映射到物理地址的缓存。页面更大、数量更少，意味着相同数量的 TLB 条目可以覆盖更多内存，从而减少未命中。

大页通常只能带来适度收益——吞吐量提升往往约为 3%–5%。这是因为它们减少了缺页开销和 TLB 压力。在大多数系统上启用 THP 是一种简单有效的优化，因为内核会自动使用 2 MB 页面支持大型分配。在内存池非常大的场

景（例如为 I/O 预先分配的绑定缓冲区），也可以考虑使用 vm.nr_hugepages 或 hugetlbfs 显式配置大页，以获得更确定的性能。

> **注** 记住，使用大型绑定内存区域时，应将 ulimit -l 设置（最大锁定内存）提高到较大值或 unlimited。如果该限制过低，绑定内存的尝试就可能失败，导致回退到可换出的内存，甚至触发内存不足（OOM）错误。

需要注意，THP 的后台压缩可能引入不可预测的暂停，这对延迟敏感型 LLM 推理工作负载是灾难性的。Linux 默认配置为尽可能自动分配 2 MB 页面，使用 THP。这通常已经足够，但仍值得针对你的工作负载进行测试。

可以禁用 THP，但那样就必须手动分配和控制大页。这会增加复杂性，但低延迟推理等工作负载可能确实需要这样做。禁用 THP 后，系统可以避免内核驱动的碎片整理造成的停顿。

目前的共识是：对于重视吞吐量的大多数基于 GPU 的训练工作负载，应启用 THP；对于重视延迟的工作负载（如推理），则应完全禁用 THP（transparent_hugepage=never），或使用 madvise。对于许多 rank（GPU）同时分配内存的分布式训练工作负载，这一结论同样适用。

除了 CPU/内存绑定和大页之外，还有一些值得提及的 OS 级调优，包括线程调度、虚拟内存管理、文件系统缓存和 CPU 频率设置；接下来几节将介绍它们。

3.3.4 调度器与中断亲和性

在繁忙的系统中，应确保数据流水线线程等重要线程不会频繁被中断。Linux 默认使用完全公平调度器（CFS），它在大多数情况下运行良好。

但是，如果有一个为 GPU 提供数据的线程对延迟极其敏感，可以考虑为该线程使用实时先进先出（FIFO）或轮询（RR）优先级调度。这样可以确保高优先级线程不会被普通优先级线程抢占。

不过，使用时要谨慎，因为如果管理不当，实时线程可能会使其他进程得不到运行机会。实际上，如果已经将线程绑定到专用核心，通常不需要调整实时线程优先级，但仍值得留意。

另一种选择是隔离核心或创建独立 CPU 分区，进一步减少对这些专用计算资源的中断。可以使用 cset、isolcpus 和 nohz_full 等内核参数，或使用 cgroup cpuset 隔离。隔离后，OS 调度器会将这些 CPU 核心留给你按需使用。

> **注** 在生产环境中，强烈建议使用 cgroup CPU 和内存亲和性。借助它们，每个 AI 工作负载都能隔离在自己的物理核心和内存区域中，从而避免工作负载之间的争用和 NUMA 惩罚。应使用 cpuset cgroup 或容器运行时（docker –cpuset-cpus）来强制执行这一点。

可以将每个设备的硬件中断分配给同一 NUMA 节点上的核心。这样可以避免跨节点处理中断，否则会产生额外延迟，并驱逐远程节点上的有用缓存行。例如，如果 NUMA 节点 0 上的 GPU 或 NIC 产生中断，就应将它绑定到节点 0 的核心，使其他节点不处理该中断。否则，另一个 NUMA 节点上的 CPU 可能会处理该中断，从而迫使缓存一致性流量和跨节点通信发生。

实际中，性能敏感型系统经常禁用默认的 irqbalance 守护进程，或使用自定义规则运行它。另一种选择是通过 /proc/irq/*/smp_affinity 手动设置每个中断的亲和性掩码。将每个 GPU 和 NIC 中断绑定到最近的核心，可以确保设备中断始终在最佳 NUMA 节点上得到处理。

简言之，专用核心、合适的调度优先级以及感知 NUMA 的硬件中断绑定结合起来，可以帮助减少为 GPU 提供数据的数据加载线程的抖动。

3.3.5 虚拟内存与交换

不用多说，应始终尽量避免内存交换。如果进程的任何部分内存被换出到磁盘，就会出现灾难性的、数量级倍增的减速。GPU 程序往往会为数据缓存分配大量主机内存。如果 OS 决定将部分数据换出到磁盘，当 GPU 需要访问这些数据时就会遭遇巨大延迟。

建议将 vm.swappiness=0，这会告诉 Linux 除非内存压力极端严重，否则避免交换。它实际上会通过 cgroup 限制隔离训练作业的内存，防止发生交换。

> **注** 应通过 Docker 或 Kubernetes 使用 cgroups v2，将内存和 CPU 绑定到 AI 进程。这会在容器化环境中强制执行 NUMA 亲和性和禁用交换的策略。

还可以使用 sudo swapoff -a，临时禁用所有交换设备和交换文件，直到下一次重启。只要确保工作负载拥有足够的 RAM，或设置限制以防止过量提交即可。否则，OOM killer 可能会终止该进程。使用 vmstat 或 free -m 监控交换使用量，确保交换保持为 0。

另一个相关设置是前面提到的 ulimit -l，用于绑定内存。如果希望防止内存交换，应将该限制设得很高，或设为 unlimited，否则可能经历大量内存交换。同样，对于使用大量内存的大型 AI 工作负载，通常会将此限制设为 unlimited。

3.3.6 文件系统缓存与回写

大型训练作业的最佳实践之一，是频繁将检查点写入磁盘，以便在作业失败时从已知的良好检查点重新启动。然而，在保存检查点期间，巨大的数据突发可能填满 OS 页缓存并造成停顿。

对于存储，可以调整 vm.dirty_ratio 和 vm.dirty_background_ratio，以调节用于缓冲写入的页缓存大小。例如，对于多 GB 的检查点，使用较高的 dirty ratio 可以让 OS 在刷写到磁盘之前在 RAM 中批量积累更多数据。这会平滑大型检查点写入，并减少训练循环中的停顿。

另一种选择是在独立线程中执行检查点写入。PyTorch 还有一种较新的选项，可以从集群中的各个节点写入分布式检查点分片。此时，作业失败重启后加载检查点时，会将各分片重新组合。

在延迟敏感的训练工作流中，最好完全绕过页缓存。例如，使用 O_DIRECT 打开检查点文件，或使用 Linux 的 io_uring 执行异步 I/O，以避免页缓存停顿。每次写入检查点后，调用 posix_fadvise(fd, 0, 0, POSIX_FADV_DONTNEED)，立即从缓存中丢弃这些页面，避免后续迭代产生内存压力。

3.3.7 CPU 频率与 C-state

默认情况下，许多计算节点会让 CPU 运行在节能模式：要么降低 CPU 频率，要么在空闲时让 CPU 休眠。这样可以节省能源、减少发热并降低成本。在模型训练期间，GPU 可能正忙于处理数据集的最后几个批次，而 CPU 不一定始

终达到 100% 利用率。然而，当新工作到来、系统重新唤醒 CPU 时，这些电源管理功能可能增加额外延迟。

为了获得最大且稳定的性能，AI 系统通常会将 CPU 频率调节器配置为“performance”模式，使 CPU 始终保持最大频率。可以使用 cpupower frequency-set -g performance，或在基本输入/输出系统（BIOS）中进行设置。

同样，禁用深层 C-state 可以避免核心进入低功耗休眠状态。CPU C-state 是由系统 ACPI 规范定义的省电模式。CPU 核心空闲时，可以进入某个 C-state 以节省能源。C-state 越深，节省的功耗越多，但工作到来时唤醒核心所需时间也越长。禁用更深层 C-state 可以消除过大的延迟尖峰。C0 表示活动状态；高于 C0 的状态都表示更深的休眠状态。

实际中，许多服务器 BIOS/UEFI（统一可扩展固件接口）提供高性能配置文件，可以自动将 CPU 调节器设置为“Performance”，并禁用深层 C-state。

本质上，我们可以用稍高的功耗换取响应更快的 CPU 行为。在训练场景中，GPU 才是主要耗电设备；只要能持续为 GPU 提供数据，CPU 多消耗一些功率通常是可以接受的。例如，数据加载线程如果在等待数据时休眠，而 CPU 进入深层 C6 状态，CPU 的很大一部分就会被关闭，以最大程度节省能源。

如果 CPU 进入更深的休眠状态，可能需要几微秒才能唤醒。虽然这段时间不长，但许多个几微秒累积起来，如果管理不当，就会造成 GPU 气泡。气泡是 GPU 等待 CPU 恢复数据处理的时间段。让 CPU 保持就绪可以减少这类卡顿。许多服务器 BIOS 都有禁用 C-state 或至少限制其深度的设置。

> **注** 应始终关闭系统中可能引入不可预测延迟的因素，例如过多的上下文切换、CPU 频率缩放以及内存到磁盘的交换。理想结果是：CPU 能以 GPU 消耗数据的速度向 GPU 提供数据，OS 不会将任务调度到错误的核心，也不会在错误的时机夺走 CPU 周期。

3.3.8 调优主机 CPU 内存分配器

在调优良好的 GPU 服务器上，由于大部分计算由 GPU 处理，CPU 使用率可能并不高。但 CPU 使用率应当保持稳定，并与 GPU 活动同步。GPU 处理当前批次时，CPU 必须持续忙于准备下一个批次。

正确的 CPU 到 GPU 交接对于维持高 GPU 利用率至关重要。通过调优主机的内存分配器（jemalloc 或 tcmalloc），可以消除数据准备过程中的不可预测暂停。这样，GPU 就能保持峰值运行，只有在有意设置的同步点处才会停顿。

调优后，应看到每块 GPU 的利用率接近 100%，只在必要的同步屏障处下降。GPU 不应因 CPU 侧延迟而等待数据。使用 jemalloc 时，可以将分配分片到每个 CPU 的 arena（narenas），启用 background_thread 执行路径外清理，并延长 dirty_decay_ms/muzzy_decay_ms，使释放的页面不会立即返还给 OS。这些设置可以最大限度减少锁竞争和碎片。

可以使用 MALLOC_CONF 环境变量按如下方式调优 jemalloc：

```
export MALLOC_CONF="narenas:8,dirty_decay_ms:10000,muzzy_decay_ms:10000
,background_thread:true"
```

类似地,tcmalloc 也能从调优 TCMALLOC_MAX_TOTAL_THREAD_CACHE_BYTES 和 TCMALLOC_RELEASE_RATE 环境变量中受益。这些设置会提供更大的线程级缓存，使小型分配避开全局锁和系统调用，以低且可预测的延迟让 CPU 线程持续为 GPU 提供数据。可以按如下方式设置：

```
export TCMALLOC_MAX_TOTAL_THREAD_CACHE_BYTES=$((512*1024*1024))
export TCMALLOC_RELEASE_RATE=16
```

简言之，优化分配器可以减少分配器开销和碎片，使 CPU 线程保持稳定的速度，避免为 GPU 提供数据时出现意外停顿。应尝试这些环境变量，并针对具体工作负载和环境进行调优。

3.4 面向性能的 GPU 驱动与 Runtime 设置

我们已经优化了 CPU 侧，但 GPU 驱动和 runtime 也有会影响性能的重要设置，尤其是在多 GPU 和多用户场景中。NVIDIA GPU 有一些可以调节的旋钮；正确调优后，可以减少开销，改善多个工作负载共享 GPU 的方式。

下面将介绍 GPU 持久化模式、MPS 和 MIG 的分区，以及时钟设置、ECC 内存和内存不足行为等其他注意事项。

3.4.1 GPU 持久化模式

默认情况下，如果没有应用正在使用 GPU，驱动可能会将 GPU 置于较低功耗状态，并卸载部分驱动上下文。下次应用要使用 GPU 时，就需要支付初始化成本；驱动重新启动一切可能需要约一两秒。

GPU 初始化开销会负面影响周期性释放并重新获取 GPU 的工作负载。例如，训练集群中的作业可能频繁启动和停止，或者低流量推理集群可能每次收到新的推理请求都要唤醒 GPU。在这两种情况下，开销都会降低整体工作负载性能。

运行 nvidia-persistenced 守护进程即可启用持久化模式。即使没有活动应用，它也会让 GPU 驱动保持加载、硬件保持就绪。它会请求系统不要在空闲时完全关闭 GPU，从而阻止电源门控。持久化模式让 GPU 保持唤醒，使下一个作业没有启动延迟。这通常推荐用于长时间运行且对延迟敏感的工作负载。可以使用以下命令在启动时启用持久化守护进程：

```
systemctl enable nvidia-persistenced
```

> **注** 在 Kubernetes 环境中，可以配置 NVIDIA GPU Operator，自动为所有 GPU 启用持久化模式。

在 AI 集群中，通常会在服务器启动时直接为所有 GPU 启用持久化模式。这样，作业开始时 GPU 已经完成初始化，可以立即开始处理。它不会让实际计算变快，因为它不会加速数学运算；但它会削减作业启动延迟，避免冷启动延迟。

GPU 持久化模式也有利于交互式使用，因为没有持久化时，空闲一段时间后执行的第一次 CUDA 调用可能会因驱动重新初始化 GPU 而停顿。启用持久化后，该调用会快速返回。

持久化的唯一缺点，是 GPU 在空闲时保持较高就绪状态，会略微增加功耗。但对大多数数据中心 GPU 而言，为了获得更稳定的性能，这种权衡是可以接受的。管理员通过 sudo 权限设置 GPU 持久化模式后，就可以享受其好处，继续处理其他优化。

3.4.2 MPS

通常，当多个进程共享一块 GPU 时，GPU 调度器会在它们之间进行时间切片。例如，如果两个 Python 进程各自有一些 kernel 要在同一块 GPU 上运行，GPU 可能先执行一个进程的 kernel，再执行另一个进程的 kernel，如此交替。如果这些 kernel 很短，且它们之间存在空闲间隔，GPU 就可能因为“乒乓式”的上下文切换而利用率不足，无法重叠工作。

NVIDIA 的 MPS 是一种功能，它建立类似“伞”的环境，让多个进程可以并发运行在 GPU 上，而无需严格的时间切片。只要 GPU 资源（流式多处理器［SM］、Tensor Core 等）可用，MPS 就能同时执行来自不同进程的 kernel。MPS 本质上将各进程的上下文合并为一个调度上下文。这样，就不必为独立进程之间的切换和空转付出全部成本。

什么时候适合使用 MPS？对于模型训练，如果通常每块 GPU 运行一个进程，可能不会使用 MPS。但如果要在一块大型 GPU 上运行许多推理作业，MPS 就会带来巨大改变。想象一下，你拥有一块强大的 GPU 或一个 GPU 集群，但一个推理作业（或一组推理作业）无法充分利用它。

例如，考虑在一块 40 GB GPU 上运行 4 个独立推理作业，每个作业使用 5–10 GB 内存，并且只使用 30% 的 GPU 计算能力。默认情况下，每个推理作业都会获得一个时间片，因此在任一时刻，实际上只有一个作业在 GPU 上运行。这样 GPU 平均有 70% 处于空闲状态。

如果为这些推理作业启用 MPS，GPU 就可以交错执行它们的工作：一个作业等待内存时，另一个作业的 kernel 可能填满 GPU，等等。结果是整体 GPU 利用率更高。实践中，如果两个进程各自使用 GPU 的 40%，启用 MPS 后可能看到 GPU 以 80%–90% 的利用率同时服务两者。

例如，两个训练进程单独运行时各需 1 小时；如果它们在同一块 GPU 上顺序运行，总共需要 2 小时。启用 MPS 后，它们可以同时运行，总时间略多于 1 小时，而不是顺序运行的 2 小时。再举一次同样的例子：两个训练进程单独运行时各需 1 小时，在同一块 GPU 上顺序运行时各自需要 1 小时；它们可以通过 MPS 一起运行。在这种情况下，它们并行完成的总时间略多于 1 小时，而不是顺序运行的 2 小时。当并发客户端的 kernel 和内存带宽彼此互补时，MPS 带来的加速接近翻倍。为了直观理解，设想进程 A 和进程 B 在没有 MPS 时周期性

启动 kernel。GPU 调度可能呈现 A-B-A-B 的交替，中间有两者等待造成的空隙，如图 3-6 所示。

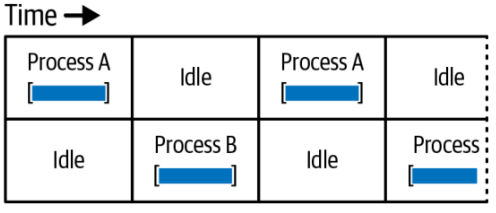

图 3-6　GPU 交替运行进程 A 和进程 B 的 kernel，并形成一个进程等待、另一个进程活动时的空闲间隙。

使用 MPS 后，调度会重叠 A 和 B：每当 A 没有使用 GPU 的某些部分时，B 的工作就可以同时使用这些部分，反之亦然。这种重叠消除了空闲间隙，如图 3-7 所示。

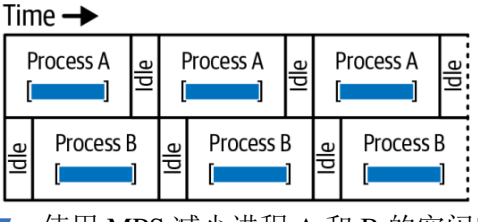

图 3-7　使用 MPS 减少进程 A 和 B 的空闲间隙。

设置 MPS 需要运行 MPS 控制守护进程（nvidia-cuda-mps-control），随后它会启动负责代理 GPU 访问的 MPS 服务器进程。在现代 GPU 上，MPS 更加简洁：客户端（各进程）可以直接与硬件通信，计算节点本身的干预极少。

通常，会在节点上启动 MPS 服务器——每块 GPU 启动一个，或每个用户启动一个——然后让 GPU 作业通过环境变量连接到 MPS。该服务器下的所有作业都会并发共享 GPU。

MPS 的另一个功能，是可以为每个客户端设置活动线程百分比。这会限制客户端能够使用的 SM 数量（本质上就是 GPU 核心数）。如果希望保证服务质量（QoS），这会很有用；例如，可以让两个作业各自最多获得 GPU 执行资源的 50%。此时，可以设置 CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=50，将客户端限制在约 50% 的 SM 执行容量以内。如果不显式设置，作业就会竞争并使用能够获得的 GPU 资源。

注意，MPS 不会分区 GPU 内存，因此所有进程都会共享完整的 GPU 内存空间。MPS 主要处理计算共享和调度。问题在于，一个进程可能请求大量 GPU RAM，导致 GPU OOM 错误，并使 GPU 上运行的其他进程全部终止。这会造成严重干扰。此外，如果某个程序单独就能让 GPU 达到 100% 饱和，MPS 也不会神奇地让它更快，因为利用率不可能超过 100%。只有当单个作业留有可由其他作业填补的余量时，MPS 才有收益。

MPS 的另一个限制是，默认情况下所有 MPS 客户端必须使用同一个 Unix 用户，因为它们共享上下文。在多用户集群中，这意味着 MPS 通常在调度器层面设置，使同一时刻只有一个用户的作业共享 GPU。也可以配置一个由所有用户共享的系统级 MPS，但要明白，从安全角度看，这些作业并未相互隔离。

现代 NVIDIA 驱动支持多用户 MPS，因此不同 Unix 用户的进程可以共享一个 MPS 服务器。这改善了易用性，但并不提供内存隔离。需要强隔离时，应优先使用 MIG。MPS 的一个特定替代方案，是在 Kubernetes 中对 GPU 使用时间切片。Kubernetes 时间切片允许设备插件按时间在同一块 GPU 上调度不同 pod。例如，如果将一块 GPU 配置为时间切片复制因子 4，那么该 GPU 上的 4 个 pod 可以各自获得一部分时间。

Kubernetes 时间切片有点像自动化的时间共享算法，不需要 MPS。但它不会重叠执行，而只是比默认驱动更快地切换。时间切片适合交互式工作负载：可以接受一些空闲时间，以换取隔离。对于高吞吐量作业，使用 MPS 重叠执行，或使用 MIG 切分 GPU，通常优于细粒度时间切片，下一节将讨论 MIG。

3.4.3 MIG

现代 GPU 可以使用 MIG 在硬件层面划分成多个实例。MIG 是一种虚拟化形式，但它在硬件中完成。因此，由于灵活性有所损失，开销非常低，可能只有几个百分点。

如果一个实例空闲，它不能将资源借给另一个实例，因为资源已经被硬切分。MIG 可以将一块 GPU 切分成最多 7 个较小的逻辑 GPU，每个逻辑 GPU 都拥有专用的内存和计算单元（即 SM），如图 3-8 所示。

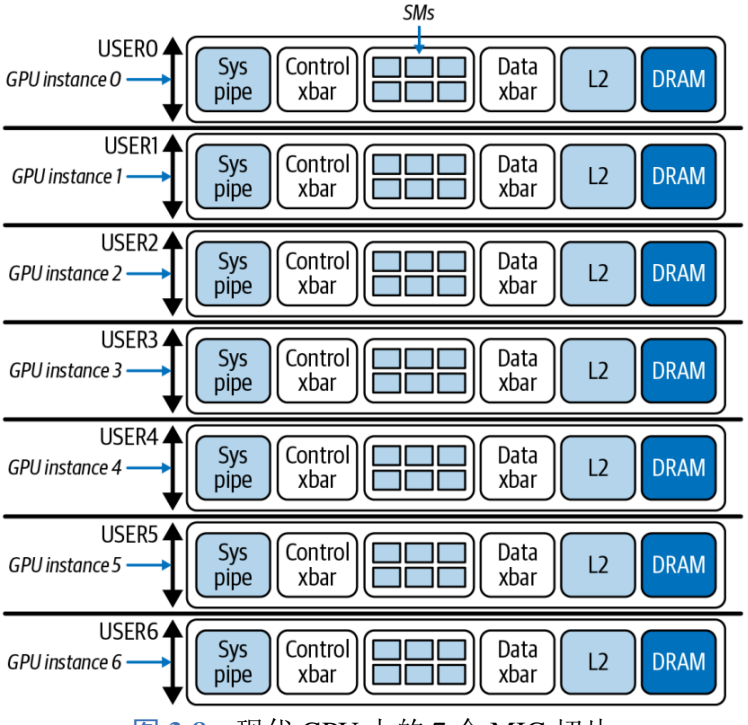

图 3-8　现代 GPU 上的 7 个 MIG 切片。

按照惯例，NVIDIA 的 MIG profile 命名使用前缀 <X>g，表示现代 GPU 上的计算切片数量，范围为 1（最小）到 7（最大）。每个切片编号表示分配给该分区的 SM 组数量。每个 SM 组大约是总 SM 数的 1/7。

如果一块 GPU 有 132 个 SM，那么每个 1/7 切片表示一个包含 132 个 SM × 1/7 = 约 19 个 SM 的组。因此，1g 表示约 19 个 SM，2g 表示约 38 个 SM，依此类推，直到 7g 表示总计约 132 个 SM。

与此相反且有些容易混淆的是，后缀 <Y>gb 指定为该 profile 保留的 HBM GPU RAM 的确切 GB 数。MIG profile 的取值对每一代、每一种 GPU 都是固定的，并列在 NVIDIA 文档中。对于 Blackwell B200，表 3-1 展示了部分 MIG profile 取值。

该表还展示了每个 profile 的硬件单元数、copy engine 数量和 L2 cache 占比。这些固定 profile 与 GPU 的硬件内存控制器对齐，因此每个内存切片都映射到连续的 HBM 通道。

表 3-1 Blackwell B200 的 MIG 配置（来源：https://oreil.ly/FsPEx）

Profile 名称	内存占比	SM 占比	L2 cache 大小	Copy engine 数量	实例数量
MIG 1g.23gb	1/8	1/7	1/8	2	7
MIG 1g.45gb	2/8	1/7	2/8	2	4
MIG 2g.45gb	2/8	2/7	2/8	3	3
MIG 3g.90gb	4/8	3/7	4/8	6	2
MIG 4g.90gb	4/8	4/7	4/8	8	1
MIG 7g.180gb	Full	Full	Full	16	1

这种两部分方案将计算容量（SM 组的数量）与内存容量（总 GB 数）分离。管理员可以选择 MIG profile 的组合。分配的 SM 和 HBM 之和不必精确等于完整 GPU 容量。不过，某些组合会受硬件分区约束；例如，不能凭空创建新的切片尺寸。

管理员只能使用 nvidia-smi -mig 等工具，或使用 NVIDIA Kubernetes GPU Operator 的 nvidia.com/mig.config config map，在每块 GPU 上启用或禁用受支持的 MIG profile（例如 1g.23gb、2g.45gb、4g.90gb 等）。重新配置 MIG 需要先清空工作负载，然后调用 MIG 的动态重新配置能力应用更改。

GPU 处于 MIG 模式后，现代 GPU 可以动态创建和销毁 MIG 分区，而无需重启整个系统。清空现有工作负载后，可以随时调整 MIG 实例；但要在 GPU 上启用或禁用 MIG 模式本身，则需要重置该 GPU。

从软件的角度看，每个 MIG 实例都像一块独立 GPU，因为它拥有自己的内存、自己的 SM，甚至独立的 engine 上下文。MIG 的好处是强隔离，并为每个作业提供有保证的资源。如果有多个用户或服务，每个只需要例如 10 GB 的逻辑 GPU 内存，就可以将它们放在一块物理 GPU 上，而不会相互干扰内存或计算。

作业可以专门请求 MIG 设备，但必须谨慎调度，确保所有切片都得到使用。例如，在一个 7 切片配置中，如果某作业只占用 1 个切片，就应将另外 6 个切片与其他作业打包，否则会浪费大量资源。可以将集群中的某些节点配置为使用 MIG 来运行小型推理作业，将另一些节点配置为非 MIG 工作负载，以运行大型训练作业。

一个重要的运维注意事项是：要使用 MIG，通常需要在系统级或至少节点级进行配置。GPU 必须进入 MIG 模式，创建切片并重置 GPU。完成这些操作后，切片会以独立设备的形式出现在系统中，每个设备都有唯一的设备 ID。

如果由于前期规划不佳而没有使用全部可用 MIG 切片，就会因为切片碎片化且闲置而浪费资源。因此，必须预先规划分区大小，使其匹配工作负载，并在工作负载发生变化时调整分区大小。要让更改生效，需要重置 GPU。

对于大规模模型训练作业和跨越多块 GPU 的推理服务器，MIG 通常没有用，因为我们希望访问完整的 GPU 集合。另一方面，对于可以运行在较小 GPU 分区上的多租户小模型推理服务器，MIG 及其隔离特性可能很有用。

> **注** 截至本文写作时，GPU 处于 MIG 模式时，GPU 到 GPU 的点对点通信（包括 NVLink）会被禁用。这同时适用于 GPU 之间基于 NVLink 和 PCIe 的 P2P。MIG 实例不能与其他 GPU 进行 P2P，MIG 实例之间的 CUDA IPC 也受到限制。这可能降低分布式训练吞吐量。启用 MIG 前，应确认训练或推理拓扑不依赖 GPU 对等路径。大规模训练作业（以及稀疏 MoE 专家推理系统）需要大量 GPU 到 GPU 通信，通常不适合使用 MIG。MIG 实例之间的通信必须经过主机或网络 fabric。

简言之，只有在需要在同一块 GPU 上运行多个相互独立、且需要强隔离的作业时，才应启用 MIG。不要将 MIG 用于跨 GPU 的大规模分布式训练或推理，因为此时需要完整利用 GPU 的能力和高速互连。

在大规模基于 transformer 的模型训练和推理场景中，我们会关闭 MIG。但了解这一功能很有价值。集群或许可以动态切换模式：白天进行大量小型训练或推理实验时运行 MIG，夜间执行使用完整 GPU 的大型训练作业时再关闭 MIG。

Kubernetes device plugin 会将 MIG 设备列为 nvidia.com/mig-1g.23gb 等资源；以 1/7 GPU 切片为例，该切片总计有 23 GB。

3.4.4 GPU 时钟频率与 ECC

NVIDIA GPU 具有称为 GPU Boost 的功能，会在功耗和温度限制内自动调整核心时钟。大多数时候，应让 GPU 自己完成调节。不过，一些用户为了保持

一致性，会锁定时钟，使 GPU 始终以固定的最大频率运行。这样，每次运行的性能就更加稳定，不会受功耗或温度变化影响。

执行基准测试时，固定时钟极其重要，因为后续运行可能因温度过高而受到节流。如果不考虑这一点，就可能错误地解读后续运行的较差结果，因为这些运行中的 GPU 可能已经受到前几次运行产生的高温影响而发生节流。

具体而言，NVIDIA 的 GPU Boost 会上下调整核心时钟，以保持在功耗/温度限制内。使用 nvidia-smi -lgc 锁定核心时钟，使用 -ac 锁定内存时钟，就能将时钟固定在稳定的最大频率，避免 GPU Boost 的默认功能在后续运行中降低频率。

> **注** 这主要与基准测试有关，有助于获得确定且可复现的结果。对于日常训练和推理，建议保留自动 Boost，除非你注意到明显的性能波动和 GPU 节流。

如果追求最后一点确定性和一致性，就应了解锁定时钟这一选项。不过通常让 GPU 保持默认的自动 Boost 模式即可。

一些团队会有意降低 GPU 时钟频率以减少发热，尤其是在运行很长时间的作业且不希望随时间发生热降速时。数据中心 GPU 通常具有足够的温度余量，并配备适当的风冷和液冷，因此通常不需要这样做，但了解这一选项仍有帮助。

另一种方法是使用 nvidia-smi -pl，将功耗上限设置为略低于 GPU 最大热设计功耗（TDP）。TDP 是 GPU 在持续负载下能够产生的最大热量，以瓦特计量。它决定了必须散去多少热量才能避免过热。

如果将功耗上限设在 TDP 以下，GPU Boost 会自动将时钟调整到热节流点以下。这可以降低峰值发热、避免节流，同时只带来很小的性能影响。

GPU 上的 ECC 内存也是一个需要考虑的因素。例如，宇宙射线造成单比特内存错误时，ECC 可以在运行中纠正它；如果发生双比特错误，则会检测到错误，并向调用代码抛出错误。NVIDIA 数据中心 GPU 通常默认启用 ECC。

禁用 ECC 可以释放少量内存，因为 ECC 需要额外比特进行错误检查。由于减少了运行中错误检查的开销，这可能带来边际性能提升，但通常只有几个百分点。然而，关闭 ECC 也会移除关键的内存错误保护，可能导致系统不稳定或数据损坏而未被检测到。

对于包括 Hopper 和 Blackwell 在内的 NVIDIA 数据中心 GPU，ECC 默认启用，其设计目标是保证可靠、经过错误纠正的计算和数据完整性。对于大型模型的长时间训练或推理作业，一次内存错误就可能完全导致作业崩溃，更糟糕的是，还可能在没有警告的情况下悄悄破坏模型。

建议任何严肃的 AI 工作负载都始终保持 ECC 开启。唯一可能考虑关闭 ECC 的情况，是在研究环境中你能够接受风险，而且确实需要那一小部分额外内存，才能让模型装入内存有限的 GPU 集群。

切换 ECC 模式需要重置 GPU，并很可能需要重启当前在该 GPU 上运行的作业。因此，不应频繁切换。为了稳定性和可靠性，应保持 ECC 开启；关闭 ECC 带来的微小加速，不值得牺牲这份安心。

3.5 GPU 内存超额订阅、碎片化与内存不足处理

与 CPU RAM 不同，默认情况下 GPU 没有所谓的“交换”内存。如果尝试分配超过可用量的 GPU 内存，就会得到不友好的 OOM 错误，并伴随更加不友好的进程崩溃。可以通过几种机制缓解这一问题：允许内存动态增长，在 CPU 和 GPU 之间使用统一内存，以及使用内存池和缓存分配器。

默认情况下，一些框架（例如 TensorFlow）会在启动时抓取全部可用 GPU 内存，以避免碎片并改善性能。如果你不知道这一点，在共享 GPU 的场景中可能会造成严重问题。PyTorch 默认只按需分配 GPU 内存。

TensorFlow 提供 TF_FORCE_GPU_ALLOW_GROWTH=true 选项，使其从较小的内存占用开始，按需动态增长 GPU 内存使用量，类似于 PyTorch。然而，PyTorch 和 TensorFlow 都不允许分配超过 GPU 可用量的内存。这种延迟分配在多租户场景中更友好，因为两个进程不会从一开始就同时尝试分配最大可用 GPU 内存。

CUDA 的 Unified Memory 系统允许你分配内存，而不必预先规定它位于 CPU 还是 GPU。CUDA Runtime 会按需处理页面移动。现代 NVIDIA GPU（如 Hopper 和 Blackwell）通过 Page Migration Engine（PME）提供按需分页的硬件支持。

当 GPU 可用内存不足时，PME 会自动在 GPU 内存和主机 CPU RAM 之间迁移内存页面。不过，虽然 PME 提供了灵活性，但与工作负载拥有足够 GPU 内存相比，依赖它可能会带来性能惩罚。

这种 GPU 到 CPU 的内存卸载可能很慢，因为 CPU 内存 I/O 比 GPU 高带宽内存（HBM）I/O 慢，正如我们在第 2 章中学到的那样。对于试图运行无法装入 GPU RAM 的模型的实践者而言，这种机制主要是一种便利功能。

对于性能关键型工作负载，通常应尽可能避免依赖统一内存超额订阅。它是防止脚本直接崩溃的安全网，但 GPU 内存超额订阅后，作业运行会变慢。

PyTorch 等库使用缓存分配器，因此释放 GPU 内存时不会立即将其返还给 OS，而是保留下来供未来分配复用。这可以避免内存碎片和反复请求 OS 分配相同内存块的开销。

可以使用 PYTORCH_ALLOC_CONF（原名 PYTORCH_CUDA_ALLOC_CONF）等环境变量配置 PyTorch 分配器，设置最大内存池大小。后续章节会介绍 PyTorch 内存分配机制的优化。

如果遇到 GPU OOM 错误，你迟早一定会遇到，它很可能由内存碎片或过度缓存造成。可以使用 PyTorch 的 torch.cuda.empty_cache() 尝试清理缓存，但这几乎总是意味着你的工作负载确实需要这么多内存。

PyTorch 还提供 torch.cuda.memory_stats() 和 torch.cuda.memory_summary() 等工具，通过展示已分配与已保留的内存来帮助诊断碎片。NVIDIA 的 Nsight Systems 也能显示 GPU 内存使用模式，帮助识别内存泄漏、与泄漏相关的长期分配、CPU-GPU 互连活动以及 GPUDirect Storage 时间线追踪。除此之外，Nsight Compute profiler 还能提供低层 kernel 分析，包括占用率、吞吐量和 NVLink 使用情况。后续章节会介绍这些工具。

Docker 提供 –gpus 标志，用于选择 GPU 并将其暴露给容器，但它不支持设置 GPU 内存上限。如果需要对 GPU 内存或计算能力进行硬隔离，请使用 MIG 划分设备，或使用带活动线程百分比设置的 Multi-Process Service（MPS）来实现公平共享。需要严格分区时，可在 Kubernetes 中使用 nvidia.com/mig-2g.45gb 等 MIG 资源配置限制。

在多租户节点中，这可能有助于隔离作业。在单作业单 GPU 的场景中，通常不会设置内存上限，因为希望作业尽可能使用可获得的全部 GPU 内存。

一般来说，GPU 内存耗尽可以在应用层管理。例如，如果可行，可以减小数据批次大小、降低模型权重精度，甚至减少模型参数量。

最佳实践是在模型训练和推理期间，使用 nvidia-smi 或 NVML API 监控 GPU 内存使用情况。如果接近内存上限，应考虑减小批次大小、在训练中使用 activation checkpointing，或采用其他降低内存使用的技术。

还应确保 CPU 内存没有发生交换，因为这会间接损害 GPU 利用率和有效吞吐量：每当 GPU 试图从 CPU 主机获取数据，而主机内存页面已经被换到磁盘时，性能就会受到慢得多的磁盘 I/O 限制。因此，应将这些内存缩减最佳实践与前面对绑定内存、提高 ulimit、禁用 swappiness 等建议结合起来。

简言之，建议始终保持 GPU 驱动加载，而不要在作业之间卸载 GPU 驱动。这类似于 GPU 持久化模式，但层次更深。一些集群会在没有作业运行时卸载驱动，以释放 OS 内核内存并满足安全要求。但这样做后，下一个作业必须支付重新加载 GPU 驱动的成本；如果使用 MIG，还必须重新配置 MIG 切片。

> **注** 建议让驱动及任何 MIG 配置在作业之间保持持久。只有在故障排查或升级驱动时才应卸载 GPU 驱动。因此，集群管理员通常会将系统配置为：机器启动后 NVIDIA 驱动模块始终保持加载。

3.6 面向 GPU 的容器运行时优化

许多 AI 系统使用编排工具和容器运行时来管理软件环境。Kubernetes 和 Docker 在 AI 基础设施中很受欢迎。使用容器可以确保包括 CUDA 和库版本在内的所有依赖保持一致，避免“但在我的机器上能运行”的问题。容器会引入一些复杂性和极少量开销，但只要配置得当，GPU 工作负载可以在容器中获得接近裸机的性能。

节点上运行的容器不是传统虚拟机（VM）。与 VM 不同，容器共享主机 OS 内核，因此 CPU 和内存操作可以接近原生速度。借助 NVIDIA Container Toolkit，Docker 容器内部对 GPU 的访问是直接的，不会产生额外开销。

> **注** 对于使用最新 NVIDIA Container Toolkit 运行的现代 GPU，在正确配置的环境中，容器内的 GPU 性能与直接在容器外的裸机主机上运行代码

几乎相同（差异 < 2%）。事实上，Red Hat OpenShift 和 Kubernetes 都用于 MLPerf Inference v5.0 的结果，这说明现代容器和编排配置不会牺牲效率或延迟。

3.6.1 NVIDIA Container Toolkit 与 CUDA 兼容性

使用 GPU 容器时，一个挑战是确保容器内的 CUDA 库与主机上的驱动匹配。NVIDIA 通过 Container Toolkit 和基础 Docker 镜像解决了这一问题。主机提供 NVIDIA 驱动；请记住，驱动与内核和硬件紧密集成。容器内部通常包含某个特定版本的 CUDA runtime 库。

一般规则是：主机的 NVIDIA 驱动版本必须不低于容器内 CUDA 版本所要求的最低驱动版本。对于 CUDA 13.x，所需的 Linux 主机驱动分支至少为 R580；对于 CUDA 12.x，至少为 R525。使用比主机驱动更新的 CUDA runtime，会导致 CUDA 初始化失败。

> **注** 每个新的 CUDA 版本都有最低 NVIDIA 驱动版本要求。更新 CUDA Toolkit 时，务必查阅 NVIDIA 官方兼容性矩阵并升级主机驱动。

对于 Docker 和 Kubernetes 环境，最简单的方法是使用 NVIDIA 在 NVIDIA GPU Cloud（NGC）或 DockerHub 镜像仓库提供的官方基础 Docker 镜像。这些镜像（例如 nvcr.io/nvidia/pytorch 或类似镜像）打包了正确版本的 CUDA runtime、cuDNN、NCCL 等。此外，这些 Docker 镜像还会根据 CUDA 版本列出所需的最低 CUDA 驱动版本。这样可以获得对最新硬件的支持，并避免依赖问题。

3.6.2 NVIDIA Container Runtime

另一种方法是使用 NVIDIA container runtime，在运行时将主机驱动库注入容器，因此不必把 NVIDIA 驱动打包进镜像，而是直接依赖主机驱动。这之所以可行，还是因为容器不像传统 VM 那样完全隔离。Docker 容器可以使用主机设备、卷和库。

在容器内部，应用使用容器镜像中的 CUDA runtime 库，例如 libcudart.so；NVIDIA Container Toolkit 则在容器启动时注入主机驱动库，例如 libcuda.so 和 libnvidia-ml.so。主机驱动库会直接在主机上被调用，因此一切都能正常工作。

只要主机驱动满足镜像中 CUDA Toolkit 所要求的最低版本，CUDA runtime 库（容器）与 NVIDIA Container Toolkit（主机）的这种分工就能正常工作。如果版本不匹配，尝试在容器中使用比主机驱动更新的 CUDA 版本，就可能报错。必须匹配 CUDA 和驱动版本。

关键结论是：用于 GPU 的容器不存在 hypervisor 或虚拟化层。容器直接共享主机内核和驱动，因此 kernel 在 GPU 上启动时，就像从主机上启动一样。

换言之，不会因为基于 Docker 的虚拟化而损失性能——除非使用 VMware 或 Single Root Input/Output Virtualization（SR-IOV）虚拟 GPU 之类的特殊场景，这种场景需要额外调优。Docker 加 NVIDIA 基本等同于裸机性能。

> 注　NVIDIA Container Toolkit 也支持 containerd 和 Podman，不仅支持 Docker。这一点与现代 Kubernetes 环境相关，因为现代 Kubernetes 默认使用 containerd 作为容器运行时。

3.6.3 避免容器 Overlay 文件系统开销

在 Docker 容器中运行与直接在主机上运行的主要差异，可能体现在 I/O 上。容器通常使用联合文件系统，将多个底层文件系统（如主机文件系统和容器文件系统）透明地叠加为一个统一视图。

在 OverlayFS 等联合文件系统中，来自多个源的文件和目录会表现得像属于同一个文件系统。这种机制对容器尤其有用，因为它会将基础镜像层中的只读文件系统与可写容器层组合起来。

不过，使用 overlay 文件系统存在一些开销。额外延迟来自文件系统必须检查多个底层层次（只读层和可写层），以确定应返回哪个文件版本。与从单一、简单文件系统读取相比，额外的元数据查找和合并这些层的逻辑会增加少量开销。

此外，overlay 使用的写时复制（CoW）机制在写入时也有开销。CoW 意味着当你修改只读层中的文件（例如基础镜像）时，必须先将文件复制到可写层。随后，写入操作会针对复制后的可写文件进行，而不是针对原始只读文件。正如前面提到的，读取已修改文件时，也必须同时查看只读层和可写层，以确定应返回哪个版本。

模型训练通常会涉及大量 I/O 操作，例如读取数据集、加载模型和写入模型检查点。为解决这一问题，可以通过 bind mount 将主机目录或网络文件系统挂载到容器中。

Bind mount 会绕过 overlay，因此性能与直接在主机上进行磁盘 I/O 类似。如果主机文件系统是 NVMe SSD 或 NFS 挂载，就能获得底层存储设备的完整性能。我们不会将 TB 级数据集打包进镜像，而是通过挂载将数据带入容器。

例如，如果训练数据位于主机的 /data/dataset，可以使用 -v /data/dataset:/mnt/dataset:ro 运行容器，其中 ro 表示只读挂载。随后训练脚本从 /mnt/dataset 读取数据。这样，读取操作就直接使用主机文件系统。

事实上，最佳实践是避免对容器的可写层执行大量数据读写。相反，应将数据目录和输出目录从主机挂载到容器中，以确保 I/O 不会被容器 CoW 机制的开销限制。

3.6.4 减小镜像大小以加快容器启动

如果镜像很大，需要通过网络拉取，容器启动时间可能会慢很多。但在典型的长时间运行训练循环中，几分钟的启动时间与数小时、数天甚至数月的训练时间相比微不足道。仍然值得让镜像保持合理的精简：不要包含不必要的构建工具或临时构建文件。这样可以节省磁盘空间并改善容器启动时间。

一些 HPC 中心更偏好 Singularity（Apptainer）而不是 Docker，因为它可以在用户空间运行镜像，而不需要 root 守护进程。它还直接使用主机文件系统，除 OS 已有开销外，通常几乎没有额外开销。

无论使用 Docker 还是 Apptainer（原名 Singularity），研究和基准测试都表明，只要配置得当，这些容器方案在容器中运行与直接在主机上运行之间通常只有几个百分点的差异。实质上，如果有人给你一份 GPU 利用率和吞吐量日志，仅凭日志很难判断作业是否运行在容器中。

3.7 面向拓扑的 Kubernetes 容器编排与网络

Kubernetes（也称 K8s）是用于 AI 训练和推理的常用容器编排器。Kubernetes 的 NVIDIA device plugin 是一个轻量组件，会向调度器公布 GPU 硬件（/dev/nvidia0、/dev/nvidiactl 等）。当你在 resources.limits 下请求 nvidia.com/gpu（如果希望同时显式设置两者，也可以在 resources.requests 下请求）时，它会将这些设备节点挂载到 pod 中。这样，在 Kubernetes 上部署容器时，device plugin 会负责让容器获得 GPU。device plugin 还感知拓扑，这意味着它可以优先为某个 pod 分配来自同一 NVLink Switch 或同一 NUMA 节点的多块 GPU。

NVIDIA Kubernetes GPU Operator 会自动安装和管理所有 NVIDIA 软件的生命周期，包括驱动库、前面提到的 NVIDIA Kubernetes device plugin 以及 NVIDIA Container Toolkit。它还负责使用 NVIDIA GPU Feature Discovery 进行节点标记，为每块 GPU 标注其 NUMA 节点和 NVLink/NVSwitch ID。调度器随后可以利用这些标签，智能地将 GPU 分配给作业。GPU Operator 还通过 DCGM 实现 GPU 监控。

使用 Kubernetes 编排基于 GPU 的容器时，希望它能以感知硬件拓扑的方式为容器分配资源，包括 NUMA 节点和网络带宽配置。然而，Kubernetes 默认并不感知拓扑。它将每块 GPU 视为一种资源，但不知道 GPU 0 和 GPU 1 是否位于同一 NUMA 节点，或是否使用同一 NVLink 互连。这可能造成很大差异。

考虑一台 8-GPU 服务器，其中两组各 4 块 GPU，组内通过 NVLink 互连。如果向 Kubernetes 请求 4 块 GPU，理想情况下 K8s 应分配 4 块彼此通过 NVLink 互连的 GPU，因为它们能更快地共享数据。然而，如果 Kubernetes 在整个系统中任意挑选 4 块 GPU，作业可能会被分配到来自一个 NVLink 域的 2 块 GPU，以及来自另一个 NVLink 域的 2 块 GPU。

不感知拓扑地分配 GPU，会将慢速互连（例如 InfiniBand 或 Ethernet）引入 GPU 到 GPU 的路径。这可能使 GPU 间带宽减半。在这种情况下，Kubernetes 理想上应分配 4 块全部通过 NVLink 互连且位于同一 NUMA 节点的 GPU，而不是分配 4 块跨越不同机架和 NUMA 域的 GPU。

例如，考虑建立在 NVLink 5 互连之上的 NVIDIA NVL72 机架，它将 72 块 GPU 连接成一个高带宽域，在机架内提供约 130 TB/s 的合计吞吐量（72 块 GPU

× 每块 GPU 1.8 TB/s）。在这种配置中，如果 Kubernetes 调度器不感知拓扑，就可能将多 GPU 作业放到不同 NVLink 域，甚至放到 NVL72 组之外。不遵循系统拓扑为作业分配 GPU，会抵消 NVL72 巨大的机架内带宽优势。

为了避免资源争用，应尽量预留所需资源，或为作业请求整个节点。对于容器/pod 放置，应使用 Kubernetes Topology Manager 组件，让 pod 的 CPU 亲和性和 NUMA 节点与所分配 GPU 对齐，将容器的 CPU 绑定到 GPU 所在的同一 NUMA 节点。下面讨论这一点。

3.7.1 使用 Kubernetes Topology Manager 编排容器

Kubernetes Topology Manager 可以提供详细的拓扑信息。例如，它可以检测到 GPU 0 连接到 NUMA 节点 0、NVLink 域 A 和 PCIe 总线 Z。随后，Kubernetes 调度器就可以利用这些信息，将容器分配到适合高效处理和通信的 GPU 上。

感知拓扑的 GPU 调度仍在发展。在许多集群中，管理员会使用 Kubernetes label 显式标记节点，以记录 GPU 和系统拓扑。这些 label 可以确保多 GPU pod 被放置在 GPU 共享同一 NVLink 互连，或位于同一 NUMA 域的服务器上。

就本书而言，如果在 Kubernetes 中运行多 GPU 作业，应确保启用感知拓扑的调度。这通常需要将 –topology-manager-policy 配置为 best-effort、restricted，或在某些情况下配置为 single-numa-node。该策略配置通过避免远程内存访问，帮助多 GPU 以及 CPU+GPU 工作负载获得更低延迟，同时补充 OS 级 NUMA 调优。

还应使用最新的 NVIDIA GPU device plugin 和前一节提到的 NVIDIA Kubernetes GPU Operator，因为它们能够感知拓扑，并支持将多 GPU pod 打包到连接在同一 NUMA 节点上的 GPU 上。它们有助于通过减少跨 NUMA 节点通信来优化性能，并降低多节点 GPU 工作负载的延迟。

在 NVLink-5 NVL72 系统上，单个机架级 NVLink 域提供最高 130 TB/s 的 GPU 到 GPU 合计双向带宽，相当于每块 GPU 约 1.8 TB/s。在调度 collective 密集型训练时，应优先选择让流量留在高速 NVLink 域内、再跨越较慢网络 fabric 的放置方案。

3.7.2 使用 Kubernetes 和 SLURM 调度作业

在多节点部署中，作业调度器对于最大化所有节点的资源利用率至关重要。训练集群通常使用 Simple Linux Utility for Resource Management（SLURM），而推理集群通常偏好 Kubernetes。不过，集成 SLURM 与 Kubernetes 的混合方案已经出现。开源 Slinky 项目就是一个例子，可以简化训练和推理工作负载的集群管理。

这些系统负责将 GPU 分配给作业，并协调跨节点启动进程。如果训练作业请求 8 个节点、每个节点 8 块 GPU，调度器就会找到符合条件的节点，并使用 mpirun 或 Docker 等容器运行时启动作业。这样，每个进程都能知道作业中所有可用的 GPU。许多集群还依赖 NVIDIA NGC Docker 仓库等经过验证的 Docker 仓库，以确保所有节点的软件环境一致，包括 GPU 驱动、CUDA Toolkit、PyTorch 库和其他 Python 包。

SLURM 也存在类似问题。SLURM 有“通用资源”（generic resources）概念，可以定义某些 GPU 连接到某些 NUMA 节点或 NVLink/NVSwitch。随后在作业请求中，可以请求例如连接到同一 NUMA 节点的 GPU。

如果配置不当，调度器可能会将所有 GPU 视为相同资源，从而在多 GPU 容器请求中提供不理想的分配。正确配置可以避免不必要的跨 NUMA 节点和跨 NVLink GPU 通信开销。

SLURM 也支持将 MIG 分区作为独立资源进行调度。这对于将多个作业打包到一块 GPU 上很有用，类似于 Kubernetes 通过 device plugin 调度 GPU 切片。下一节将讨论如何在 Kubernetes 中使用 MIG 切片。

3.7.3 使用 MIG 切分 GPU

启用前面介绍的 NVIDIA MIG 模式后，一块物理 GPU 会被切分成更小、固定且在硬件层面隔离的分区，称为 MIG 实例。下面是一个 Kubernetes pod 配置示例，它请求两个 nvidia.com/mig-2g.45gb MIG 切片（该配置假定 NVIDIA Kubernetes device plugin 已配置为识别每个节点上的 MIG 设备）：

```
resources:
  limits:
    nvidia.com/mig-2g.45gb: "2"
```

这里的配置要求在一个节点上运行 pod，并且该节点的一块 GPU 上至少有两个空闲的 2g.45gb 实例；换言之，是 2 个切片，每个切片拥有 2/7 的 SM（2g）。如果一块 GPU 总共有 132 个 SM，则每个切片有 2/7 × 132 个 SM = 约 38 个 SM。将这一数量乘以 2，pod 总共分配约 76 个 SM。GPU RAM 的总内存分配量为 45 GB。

注意，调度器不能将这些资源跨 GPU 或跨节点拆分。只有当单个节点能提供两个分区时，Kubernetes 才会调度该 pod。这是因为 pod 不能跨越多个节点。如果没有单个节点拥有两个空闲的 2g.45gb 切片，无法提供前面计算的总计 76 个 SM 和 45 GB GPU RAM，那么该 pod 会保持在 Kubernetes Pending（未调度）状态，因而不会运行——即使其他节点合起来拥有足够的 MIG 容量。

这一约束凸显了根据典型工作负载规划 MIG 大小的重要性。例如，如果许多作业请求 2g.45gb 切片，可以将每块 GPU 配置为承载 3 个 2g.45gb 实例（在 7 个可能的切片中），这样一个 pod 就能在同一块 GPU 上共置两个此类实例。

> **注** 这种单节点约束可能导致 pod 永远无法运行，即使整个集群的不同节点上合计能够找到所需的 MIG 资源。只有当请求的 MIG 资源在单个节点上可用时，才能满足该请求。

MIG 的一个管理缺点是，在 MIG 模式和普通（非 MIG）模式之间切换 GPU，需要重置 GPU 或重启计算节点。因此，调度器很难针对每个作业动态完成切换。不过，通常会预先创建 MIG 分区，并让该配置运行一段时间。

在 Kubernetes 环境中，NVIDIA Kubernetes GPU Operator 的 MIG Manager 可以自动配置并保留节点上的 MIG 分区。这样，MIG 切片会在重启和驱动重新加载后继续保持活动状态。

可以给一个 K8s 节点打上“mig-enabled”标签，给另一个节点打上“mig-disabled”标签，然后让调度器据此放置作业/pod。这更属于运维细节，但了解这一点很有用：MIG 是静态分区，而不是动态调度器产生的结果。

> **注** 使用 MIG 时建议启用持久化模式，使 MIG 配置即使在没有作业运行时也保持在 GPU 上。这样，GPU 就不必在每个周期性作业运行前反复重建切片。

3.8 优化 Kubernetes 的网络通信

使用 Kubernetes 通过容器运行多节点 GPU 工作负载时，各个 pod 需要彼此通信。在 Kubernetes 中，pod 默认拥有自己的 IP，不同节点上的 pod 之间可能存在 overlay 网络或网络地址转换（NAT）。这会引入复杂性和额外开销。

对于 GPU 集群，最简单的方案通常是为这些性能敏感型作业使用主机网络。这意味着容器网络不再隔离，而是直接使用主机的网络接口。在 Kubernetes 中，可以在 pod spec 中设置 hostNetwork: true。Docker 中则可以使用 –network=host 运行。

使用主机网络后，容器可以像主机一样访问 InfiniBand 互连，不需要额外的转换或防火墙层。这对 MPI 作业尤其有用，因为它消除了为每个 MPI rank 配置端口映射的需要。

不过，如果出于安全策略不能使用主机网络，就必须确保 Kubernetes 容器网络接口（CNI）和任何 overlay 网络能够处理所需流量。在这种情况下，可能需要开放特定端口来支持 NCCL 的握手和数据交换，并使用 NCCL_PORT_RANGE 和 NCCL_SOCKET_IFNAME 等环境变量帮助建立连接。

使用 overlay 网络时，关键是保持低延迟，并让操作运行在内核空间。同时，应确保没有用户空间代理限制节点之间的流量。这些因素都会显著影响性能。

在使用 Kubernetes 环境并希望启用 RDMA 时，可以考虑安装 Mellanox 的 Kubernetes RDMA device plugin。该插件会向 pod 接口暴露 InfiniBand 和 GPUDirect RDMA endpoint，以实现低延迟、零拷贝网络通信。

> **注** 如果使用 InfiniBand 或 RoCE 网络，且 NIC 支持 GPUDirect RDMA，记得在 NVIDIA 驱动中启用 GPUDirect RDMA。这样 GPU 就能直接与 NIC 交换数据，在节点间通信时绕过 CPU。这对于保持多节点环境的高性能至关重要。

3.9 减少 Kubernetes 编排抖动

运行 Kubernetes 这样的编排器，意味着每个节点上都有一些后台进程（例如 Kubernetes 的“kubelet”）、容器运行时守护进程以及（理想情况下）监控代理。这些服务会消耗 CPU 和内存，但通常只相当于单个核心的几个百分点。因此，它们不会从使用这些核心进行数据加载和预处理的 GPU 训练作业中窃取可察觉的时间。

不过，如果训练作业运行在同时承载推理工作负载的节点上，就可能遇到抖动，即执行时序和吞吐量出现不可预测的变化。这在任何多租户场景中都很常见。如果同一台机器上的另一个容器意外使用大量 CPU 或 I/O，它就会与训练或推理容器争用相同资源，进而影响你的容器。

> **注** 从系统角度看，同质工作负载（全部为训练或全部为推理）比训练和推理混合的异质工作负载更容易调试和调优。

3.9.1 改善资源保证

为了防止资源争用，Kubernetes 允许为 pod 定义资源 requests 和 limits。例如，可以指定训练作业需要 16 个 CPU 核心和 64 GB RAM。随后 Kubernetes 会为作业独占地预留这些资源，避免在相同 CPU 上调度其他 pod。

这些限制通过 Linux cgroups 强制执行，因此如果容器超过分配量，就可能被限流，甚至被 OOM killer 终止。常见做法是使用资源 requests，并选择性地启用 CPU Manager 来绑定核心，确保性能关键型作业独占所需 CPU 资源，使其他进程无法从预留核心中窃取 CPU 时间。

正如第 2 章在讨论 IRQ 亲和性时所述，后台内核线程和中断也是抖动的来源。与 Kubernetes 类似，如果其他 pod 使用与你的作业相同的网络或磁盘，它们可能在承载作业的计算节点上造成大量中断和额外内核工作。这会引发抖动并影响作业性能。

理想情况下，GPU 节点应完全专用于你的作业。如果做不到，就应使用 Linux cgroup 的 I/O 和 CPU 控制器仔细划分节点，使其他工作负载不会干扰。

幸运的是，Kubernetes 支持 CPU 隔离，可以确保 pod 获得所请求的专用 CPU 核心和内存，并防止其他 pod 被调度到与你相同的 CPU 核心上。这可以避免上下文切换和资源争用带来的额外开销。

> **注** 实践中，性能敏感型 Kubernetes 作业应请求给定节点的全部 CPU 和 GPU，使没有其他工作负载干扰或争用作业资源。从操作上说这并不容易，但从性能和一致性的角度，这是理想的作业配置。

3.9.2 内存隔离与避免 OOM Killer

如果没有正确限制，也可能发生内存干扰。Kubernetes 提供一流的内存隔离支持（使用 Linux cgroups）。不过，如果不受约束，贪婪的容器可能在主机上分配过多内存，导致主机将部分内存换出到磁盘。

如果不受限制的容器在主机上使用过多内存，臭名昭著的 Linux “OOM killer”就会开始终止进程，甚至可能终止你的 Kubernetes 作业，即使真正使用过多内存的并不是你的作业。

OOM killer 会使用启发式规则决定要终止哪些 pod。有时它会选择终止正在运行的最大 pod，而这很可能正是那个在 CPU RAM 中保存大量数据、用来为 GPU 提供数据的大型训练或推理作业。为避免这种情况，可以有意不为训练或推理容器设置严格的内存限制，使它们在需要时使用全部可用内存。

通过适当的监控和告警，可以确保作业不会超出预期进行过量分配。如果确实设置了内存限制，应确保它高于实际预期使用量，留出一定余量，避免长达 3 天的训练作业在接近结束时被 OOM killer 杀死。

> **注** 在 Kubernetes 中，没有设置 requests/limits 的 Pod 会被视为 BestEffort，最容易被驱逐。要获得 Guaranteed QoS，每个容器都必须同时为 CPU 和内存设置 requests == limits。仅设置一个很高的 limit 会得到 Burstable QoS，而不是 Guaranteed。

3.9.3 处理 I/O 隔离

截至本文写作时，Kubernetes 开箱即用地不提供原生的一流 I/O 隔离。不幸的是，虽然 Linux 支持使用 cgroup 控制器控制 I/O，但 Kubernetes 本身不会像对 CPU 和内存那样自动执行 I/O 限制。

如果需要确保 GPU 节点上的高 I/O 工作负载彼此不干扰，可能需要在节点级手动配置 I/O 控制。这可能包括调整 cgroup v2 I/O 控制器，或使用其他 OS 级配置来划分 I/O 资源。简言之，Kubernetes 可以通过调度和资源 requests 防止 CPU 争用，但 I/O 隔离通常需要对底层 Linux 系统进行额外的手动调优。

需要注意，在容器内部，一些系统设置继承自主机。例如，如果主机将 CPU 频率缩放设置为 performance 模式，容器也会继承该设置。但如果容器运行在云实例等虚拟化环境中，可能无法更改这些设置。

应始终确保主机机器经过调优，因为容器无法更改大页设置或 CPU 调节器限制等内核参数。通常，集群管理员会通过基础 OS 镜像设置这些参数和配置。在 Kubernetes 环境中，他们也可能使用 NVIDIA GPU Operator 之类的工具，在每个节点上设置持久化模式和其他 sysctl 旋钮。

3.10 关键要点

下面列出本章的关键要点，涵盖操作系统、驱动、GPU、CPU 和容器各层的优化：

3.10.1 数据与计算局部性至关重要

确保数据尽可能靠近计算单元存储和处理。使用 NVMe SSD 缓存等本地高速存储，尽量降低延迟，并减少对远程文件系统或网络 I/O 的依赖。

3.10.2 实现感知 NUMA 的配置和 CPU 亲和性

通过让进程和内存分配位于同一 NUMA 节点，优化 CPU 到 GPU 的数据流。使用 numactl 和 taskset 等工具绑定 CPU，可以防止跨节点内存访问，从而降低延迟并提高吞吐量。

3.10.3 最大化 GPU 驱动和 Runtime 效率

细调 GPU 驱动设置，例如启用持久化模式，使 GPU 保持就绪状态。可以考虑使用 Multi-Process Service（MPS）等功能，让单块 GPU 上多个进程的工作发生重叠。在多租户环境中，可以探索 MIG 分区，以有效隔离工作负载。

3.10.4 有效预取和批处理数据

通过提前预取数据并将小型 I/O 操作批处理成更大、更高效的读取操作，让 GPU 始终获得数据。利用 PyTorch DataLoader 的 prefetch_factor（以及 num_workers）等预取机制，提前加载多个批次。

3.10.5 加载数据时绑定内存

结合数据预取和内存绑定，在 PyTorch DataLoader 中使用 pin_memory=True，可以利用绑定的 CPU 内存（页锁定、不可换出到磁盘）执行更快的异步数据传输到 GPU。这样，数据加载和模型执行可以重叠，空闲时间减少，CPU 和 GPU 资源都能持续得到利用。

3.10.6 优化内存传输

利用绑定的页锁定内存和大页等技术，加速主机与 GPU 之间的数据传输。这有助于减少拷贝开销，并让异步传输与计算重叠。

3.10.7 重叠通信与计算

通过将梯度同步和数据暂存等内存操作与正在进行的 GPU 计算重叠，减少等待数据传输的时间。这种重叠有助于保持较高的 GPU 利用率和更好的整体系统效率。

3.10.8 调优并扩展网络栈

在多节点环境中，使用支持 RDMA 的网络（例如 InfiniBand/Ethernet），并调优 TCP 缓冲区、MTU 和中断亲和性等网络设置，以在分布式训练和推理期间保持高吞吐量。

3.10.9 使用容器化和编排保持一致性

使用带 NVIDIA Container Toolkit 的 Docker 等容器运行时，以及带 NVIDIA GPU Operator 和 device plugin 的 Kubernetes 等编排平台，使整个软件栈——包括驱动、CUDA 库和应用代码——在各节点之间保持一致。这些方案有助于根据硬件拓扑对齐 CPU-GPU 亲和性并管理资源分配。

3.10.10 消除容器运行时开销

虽然容器可以提升可复现性并简化部署，但应正确配置 CPU 和 GPU 亲和性、主机网络和资源隔离，以尽量减少容器开销。

3.10.11 采用编排和调度最佳实践

Kubernetes 等健壮的容器编排器是确保高效资源分配的关键组件。Kubernetes Topology Manager 等高级调度技术，有助于确保具有高速互连的 GPU 聚集在一起。

3.10.12 通过动态适应和扩展保持灵活性

编排层会分发工作，并动态管理跨节点的工作负载分段。这种灵活性对于扩展训练任务至关重要，也能确保在数据负载和请求模式变化很大的推理场景中高效运行。

3.10.13 持续、渐进地调优

系统级优化不是一次性工作。应定期监控性能指标；随着工作负载演进，调整 CPU 亲和性、批次大小和预取设置；并将这些小改进累积起来，以获得显著的性能提升。

3.10.14 减少整个栈中的瓶颈

最终目标是确保从 OS 和 CPU 到 GPU 驱动与 runtime 的所有组件协同工作。消除某一层中的瓶颈（例如 CPU 内存分配或驱动初始化）可以释放 GPU 的全部潜力，直接转化为更快的训练、更低的成本以及更高效的资源使用。

这些策略结合起来，可以最大限度减少数据传输摩擦，缩短等待时间，并确保硬件在训练和推理中得到最充分的利用。

3.11 结论

本章说明，即使是最先进的 GPU，也可能受到周围环境低效的阻碍。调优良好的操作系统、容器运行时、集群编排器和软件栈，是高性能 AI 系统的骨干。通过 NUMA 感知的绑定和本地存储方案让数据与计算对齐，通过重叠通信与计算，并细调主机系统和 GPU 驱动，可以降低延迟并提高吞吐量。

可以把整个系统想象成一辆经过精密工程设计的跑车，其中每个组件（CPU、内存、GPU、网络、容器、编排器和编程栈）都必须无缝协作，才能实现最大性能。启用持久化模式或优化 CPU 调度等微小调整，单独看似乎不起眼；但当它们组合起来并扩展到大型 GPU 集群时，就能显著节省时间和成本。这些

优化能确保 GPU 在训练大型 transformer 模型和运行复杂推理流水线时，始终以接近峰值的效率运行。

随着领域发展和模型持续变大，系统级调优的重要性只会增加。本章讨论的技术使性能工程师和系统架构师能够榨取硬件的每一点潜力，从而实现更快的迭代周期和更具成本效益的 AI 部署。归根结底，深度优化的系统会加速研究，并让更广泛的受众更容易使用前沿 AI 应用。

最后要记住，虽然硬件和软件栈看起来像数量难以管理、彼此互联的旋钮和开关，但微小调整可以转化为时间和成本上的显著节省。通过持续监控性能指标，并逐层渐进地改进整个栈，可以将潜在瓶颈转化为效率提升的机会。让数据指导你，就能释放 AI 系统的全部潜力。

第 4 章 分布式网络通信调优

在当今的 AI 格局中，实现 GPU、存储和网络接口之间顺畅且低延迟的数据移动已成为必需。本章将介绍 NVIDIA Magnum IO（例如 NCCL、GPUDirect RDMA、GDS）在训练中的应用，以及用于解耦式推理的 NIXL。我们将结合现代 GPU 和 NVL72 等集群来讨论这些技术。你将了解，这些库及其底层支持硬件如何共同构成超大规模 AI 系统所需的关键互连基础。

在大规模系统中，即使是最快的 GPU，也可能受到低效通信以及从内存和磁盘传输数据的拖累。我们将讨论加速数据传输的策略、恰当的数据分片技术、直接使用高速存储子系统的方法，以及在 GPU 上重叠通信与计算的高级模式。在 AI 系统性能工程的整个探索过程中，通信与计算重叠都是我们会反复回到的常见模式。

我们将使用 NVIDIA I/O 加速平台 Magnum IO 的组件，探讨通信与计算重叠的重要性。Magnum IO 包括 NCCL、GPUDirect RDMA 和 GPUDirect Storage（GDS）。我们将演示如何利用这些库降低通信延迟、减少 CPU 开销，并在多节点、多 GPU AI 系统的各个层面最大化吞吐量。

PyTorch 等高级 AI 框架可以利用这些底层库，将计算与通信重叠起来。把这些技术集成到 AI 系统中，是加速通信和数据流水线的整体性方法，既适用于训练超大规模模型，也适用于扩展分布式推理服务器，从而为数十亿用户提供服务。

所有这些优化都能确保每个组件都针对峰值性能完成调优。性能工程师需要仔细配置和调优网络与存储互连，以维持较高的 GPU 利用率和“有效吞吐量”（goodput）。

4.1 通信与计算重叠（流水线）

在大规模构建高效训练和推理系统时，重叠通信与计算（即流水线）发挥着关键作用。在这些环境中，重要的是让 GPU 保持忙碌，减少等待数据的时间。

核心思想是让数据传输与正在进行的计算并发执行，这样当一个任务完成时，下一阶段所需的结果已经在传输中或已经送达。PyTorch 等现代框架支持异步操作，因此集体通信（例如梯度的 all-reduce）可以与计算任务并行运行。这会减少 GPU 空闲时间（见图 4-1），并提高系统整体吞吐量。

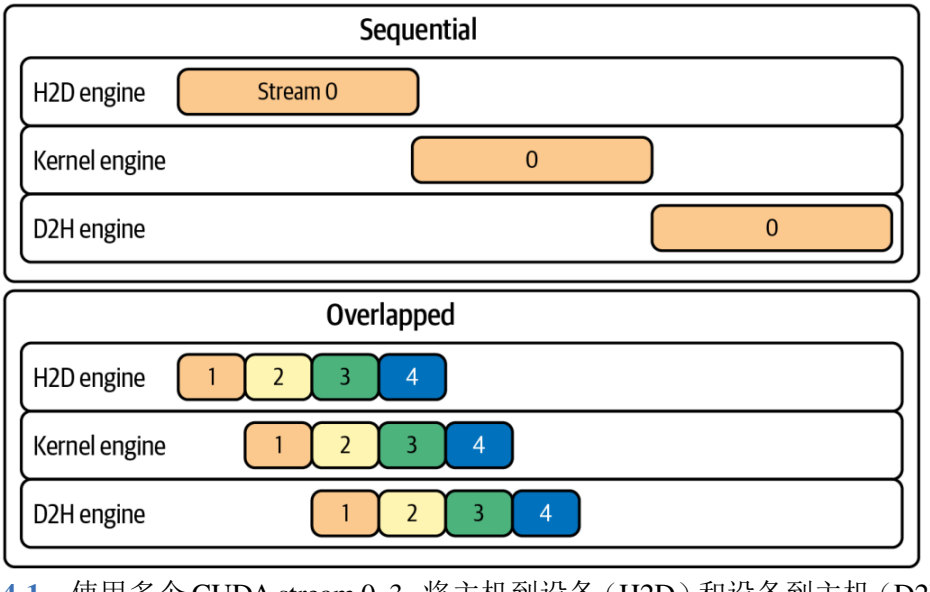

图 4-1　使用多个 CUDA stream 0–3，将主机到设备（H2D）和设备到主机（D2H）的通信与计算重叠。

基于 CUDA 的库会利用多个 CUDA stream 的能力。一个 stream 执行计算密集型矩阵乘法时，另一个 stream 可以处理梯度聚合等通信任务。神经网络的每一层完成计算后，前一层的输出就已经在前往聚合或进一步处理的途中。这种重叠让系统无需不必要的等待即可持续产生结果，并保持稳定的数据流。

增加两次通信事件之间执行的计算量，可以进一步降低通信的相对开销。当系统处理更大的数据批次时，在必须暂停并交换信息之前，它能够执行更多计算。

例如在分布式训练中，这体现为梯度累积：把多个小批次的更新合并到一次同步步骤中。减少通信事件的频率后，每次交换的开销会降低，整体吞吐量也会提高。

另一种支持计算与通信平滑重叠的技术是压缩。压缩会减少需要传输的数据量。例如，模型在发送梯度前先将其压缩，网络就只需移动更少的数据，从而缩短传输时间并缓解拥塞。

传输时间缩短后，通信阶段对计算阶段的干扰也会减小。虽然压缩本身不会直接产生重叠，但它缩短了数据在网络中移动的时间窗口，使计算工作能够更有效地并行持续。

现代深度学习框架还会把大型张量通信拆分成更小的 bucket，以便实现重叠。PyTorch 等框架会自动将大型梯度或激活张量划分为多个 bucket，一旦某个 bucket 可用就立即传输。例如，无需等待整个层的梯度准备完毕，其中一部分就可以立刻开始 all-reduce。

通过调节 bucket 大小并适当安排这些传输，可以实现更高程度的重叠，避免通信延迟阻塞计算流水线。PyTorch profiler 和 NVIDIA Nsight Systems 等工具可以帮助判断计算与通信是否发生重叠，从而让工程师调整这些参数以获得最大效率。

把更大的批次、梯度累积、异步传输、压缩和 bucket 化组合成一个统一策略后，大型分布式 AI 模型就能克服网络限制并减少空闲时间。这种设计既能减少同步事件，又能实现高吞吐量和最佳 GPU 利用率。

这些优化最终形成了一个能够缩短训练和推理总时间、更加高效地利用可用硬件资源的训练系统。工程师不必反复重新实现底层网络例程，而可以专注于创新模型架构和调节更高层的参数，而不是编写复杂的数据传输机制。

4.1.1 使用 stream 的异步执行

实现重叠从根本上依赖异步执行。GPU 支持多个 stream，也就是多个操作队列；如果它们面向不同资源，就可以并发执行或发生重叠。一个 stream 可以处理矩阵乘法等计算 kernel，另一个 stream 则处理数据拷贝和 all-reduce 调用等通信。

将工作分配给不同的 stream 并使用非阻塞操作后，通信就可以在后台进行。例如，可以把 all-reduce 操作启动到独立的 stream 中，而无需等待它完成。

与此同时，默认 stream 可以继续对独立数据执行后续计算。这要求 NCCL 等通信库使用能够立即返回控制权的非阻塞调用。这样，程序员就能在需要时确保正确同步。

在实践中，AI 框架会隐藏大部分复杂性。PyTorch 的 DistributedDataParallel 会在反向传播阶段自动安装 hook，使每个梯度 bucket 在专用通信 CUDA stream 上触发异步 NCCL all-reduce，同时默认 CUDA stream 继续计算后续层的梯度。

> **注** 我们将在后面的章节中更深入地介绍 CUDA stream；现在只需知道，它们有助于重叠通信与计算，并避免代码中不必要的同步点。

通过 CUDA stream 交错执行计算与通信，会形成稳定的工作波浪或级联流水线，隐藏通信延迟并让 GPU 始终保持忙碌。为维持正确的重叠，应避免使用 torch.cuda.synchronize() 等不必要的同步点，也不要因为用 torch.Tensor.item() 将张量移到 CPU 而意外触发整个设备同步。如果确实需要测量一次迭代的总时间，应只在迭代最末尾执行一次同步，等待所有未完成的 GPU 工作结束，同时不打断正在运行的流水线。

4.1.2 减少通信频率和通信量

如前所述，每个通信步骤执行更多工作，可以提高重叠程度和效率。模型训练中的梯度累积就是一种这样的技术。与其对每个小批次的梯度分别执行 all-reduce，不如先累积几个小批次，在本地求和，然后执行一次 all-reduce。这实际上是用保存未归约梯度所需的内存，换取更少的同步点。

假设累积四个小批次，all-reduce 的频率就会降低 4 倍。这样，同步之间可以执行更多计算。代价是有效批次大小会增大，这可能影响模型收敛和内存使用。通常可以找到一个折中点，在通信频率、内存使用和收敛之间取得良好平衡。

减少通信量的另一种方法是压缩或量化交换的数据。梯度压缩等技术可以减少每次通信发送的数据量，同时不会显著影响模型质量。要发送的数据更少，

传输就更快，也有更多机会把传输隐藏在计算之后。极端做法是稀疏化：使用稀疏性时只发送一部分梯度，但通常需要修改算法以保持精度。

PyTorch Distributed Data Parallel（DDP）通信机制中的 bucket 化，还可以通过把许多小张量组合成较大消息来降低单次调用开销。不过，bucket 大小需要权衡。很大的 bucket 能最大化带宽利用率，但在启动 all-reduce 前要等待更多梯度累积，因此会延迟通信开始。很小的 bucket 会更早启动传输，却会因为大量小型 NCCL 调用而产生更多开销。

截至本文撰写时，PyTorch DDP 的默认 bucket 大小为 25 MB。这一设置在大多数情况下能够实现良好重叠。不过，如果模型包含非常大的层，可以增大该值以降低开销；如果模型包含许多小层，减小 bucket 以更早启动通信可能反而有利。最终，要实现最大重叠，可能需要对不同 bucket 大小进行性能分析，找出能带来最佳迭代时间的设置。

4.1.3 在实践中实现最大重叠

为了观察通信与计算重叠的收益，我们通过一个例子比较两种场景：一种在所有计算完成后同步执行梯度通信、完全不重叠；另一种类似 DDP，让通信与计算重叠。我们将模拟一个使用两块 GPU 的简单分布式训练步骤，以说明两者的差异。

假设我们不使用 DDP 内置的重叠机制，而是手动实现分布式训练：每个进程先在本地计算全部梯度，然后在反向传播结束时对这些梯度执行 all-reduce。这会模拟未优化、无重叠的场景，因为通信只在全部计算完成后发生。我们可以使用 PyTorch 的分布式原语来模拟这一过程，禁用 DDP 的 hook，并在 loss.backward() 之后显式调用 dist.all_reduce。

在下面的代码中，我们在单个节点的 gpu/rank 0 和 gpu/rank 1 上启动两个进程，并使用一个简单模型。我们将执行一次前向和反向传播，然后在两个进程之间手动平均梯度：

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.distributed as dist
import torch.multiprocessing as mp
# A simple model with multiple layers to produce multiple gradients
class MultiLayerNet(nn.Module):
    def __init__(self, size):
        super().__init__()
        self.fc1 = nn.Linear(size, size)
        self.fc2 = nn.Linear(size, size)
        self.fc3 = nn.Linear(size, 1)
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)
def train_no_overlap(rank, world_size):
    dist.init_process_group("nccl", init_method="env://",
                            world_size=world_size, rank=rank)
    torch.cuda.set_device(rank)
    # Each rank synthesizes its own data
    # (avoid sending big tensors via spawn)
    batch_size = 256
    data = torch.randn(batch_size, 1024, device=rank)
    target = torch.randn(batch_size, 1, device=rank)
    model = MultiLayerNet(data.size(1)).to(rank)
    optimizer = optim.SGD(model.parameters(), lr=0.01)
    # Forward + backward (manual, no overlap)
    output = model(data)
    loss = nn.functional.mse_loss(output, target)
    loss.backward()
    # Synchronous gradient all-reduce after backward
    for p in model.parameters():
        dist.all_reduce(p.grad, op=dist.ReduceOp.SUM)
        p.grad /= world_size
    optimizer.step()
```

```
    dist.destroy_process_group()
if __name__ == "__main__":
    import torch.multiprocessing as mp
    mp.set_start_method("spawn", force=True)
    world_size = min(2, torch.cuda.device_count() or 1)
    if world_size > 1:
        mp.spawn(train_no_overlap, args=(world_size,),
         ↪ nprocs=world_size,
                 join=True)
    else:
        train_no_overlap(0, 1)
```

在这段代码中，每个进程独立为 MultiLayerNet 计算梯度。loss.backward() 之后，我们显式地对每个参数的梯度执行 all-reduce，并将其平均。这实际上就是 DDP 内部所做的事情，但这里要等整个反向传播结束后才执行，而不是在反向传播期间并发执行 all-reduce。

如果测量这一迭代，all-reduce 操作会直接增加迭代时间，因为它没有与其他步骤重叠。例如，假设前向和反向计算合计耗时 10 ms，梯度 all-reduce 耗时 12 ms，那么这种方法的总迭代时间约为 22 ms。相比之下，完全重叠的实现可能接近这些值的最大值，即本例中的 12 ms。这是因为 all-reduce 通信几乎可以完全隐藏在计算之下。

在实践中，如果使用 Nsight Systems 之类的工具分析这个无重叠案例，会先看到 fc1、fc2 和 fc3 的全部反向计算 kernel 执行，只有在它们完成后才会看到每个梯度对应的 NCCL all-reduce kernel。这里存在明显的分离：先计算，再通信。在通信阶段，GPU 除了执行 NCCL 工作外会处于空闲状态，因为没有进一步的计算。

现在使用 PyTorch DDP 执行同一操作并实现重叠。DDP 会接入反向传播，在反向计算的同时执行梯度归约。代码类似，只需用 DistributedDataParallel 包装模型，并让它处理同步：

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.distributed as dist
import torch.multiprocessing as mp
class MultiLayerNet(nn.Module):
    def __init__(self, size):
        super().__init__()
        self.fc1 = nn.Linear(size, size)
        self.fc2 = nn.Linear(size, size)
        self.fc3 = nn.Linear(size, 1)
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)
def train_ddp(rank, world_size):
    rank = int(os.environ.get("LOCAL_RANK", rank))
    torch.cuda.set_device(rank)
    dist.init_process_group("nccl", init_method="env://",
                            world_size=world_size, rank=rank)
    torch.cuda.set_device(rank)
    model = MultiLayerNet(1024).to(rank)
    ddp_model = nn.parallel.DistributedDataParallel(model,
     ↪ device_ids=[rank])
    optimizer = optim.SGD(ddp_model.parameters(), lr=0.01)
    # Each rank makes its own data
    batch_size = 256
    data = torch.randn(batch_size, 1024, device=rank)
    target = torch.randn(batch_size, 1, device=rank)
    output = ddp_model(data)
    loss = nn.functional.mse_loss(output, target)
    loss.backward()  # DDP overlaps gradient all-reduce on background
     ↪ stream
    optimizer.step()
    dist.destroy_process_group()
def main():
```

```
    world_size = min(2, torch.cuda.device_count() or 1)
    mp.set_start_method("spawn", force=True)
    if world_size > 1:
        mp.spawn(train_ddp, args=(world_size,), nprocs=world_size,
        ↪ join=True)
    else:
       print("Only one GPU present; running DDP demo with world_size=1")
        train_ddp(0, 1)
if __name__ == "__main__":
    main()
```

采用 DDP 重叠方式后，代码更简单，因为我们依赖 DistributedDataParallel 处理梯度同步，而不再自行编写同步逻辑。调用 loss.backward() 时，DDP 的内部 reducer 会将梯度拆分成 bucket，并在每个 bucket 准备好后，立即在独立的 CUDA stream 上启动 NCCL all-reduce。例如，fc3.weight 和 fc3.bias 来自最后一层，而最后一层在反向传播中最后计算，因此它们的梯度可能会在计算完成后立即 all-reduce；与此同时，fc1 和 fc2 的梯度来自更早的层，在到达反向传播末尾时可能已经完成 all-reduce。

来自更早的层，在到达反向传播末尾时可能已经完成 all-reduce。

如果模型非常小，所有梯度都能放进一个 bucket，DDP 可能只在最后执行一次 all-reduce，这样重叠就不明显。但对于更大的模型和更大的批次，会有多个 bucket，并出现显著重叠，从而带来更可观的性能收益。

分析 DDP 案例时，会看到 NCCL all-reduce kernel 与反向计算 kernel 交错出现。因此，时间线不再是清晰的两阶段划分，而会呈现锯齿状模式：执行一部分计算，再执行一部分 NCCL 通信，然后再次计算，如此交替。

> **注** 所有指标表中的数值都只是用于解释概念的示意值。不同 GPU 架构上的实际基准结果请参阅 GitHub 仓库。

在这个示例中，通信与计算重叠让示例工作负载的迭代时间大约改善了 30%。在更大的训练任务中，收益会更加明显，因为更大的模型会产生更多潜在的通信瓶颈。PyTorch DistributedDataParallel 默认使用 25 MiB 的 bucket，因此每

表 4-1　重叠带来的收益。

指标	无重叠（手动同步）	有重叠（DDP）	说明
总反向传播 + 通信时间	100%（基线）	约为基线的 70%	例如，由于重叠，每次迭代快 30%（示意）
通信开始时间	反向传播完成后	反向传播期间	在 DDP 中，通信在反向传播中途开始
通信阶段 GPU 空闲	是——反向传播后，GPU 在 all-reduce 期间等待	极少——通信与其他层的计算并行运行	DDP 隐藏了大部分延迟
SM（GPU）利用率	较低（通信期间有部分周期 SM 空闲）	较高（持续活动）	重叠使 GPU 保持更稳定的忙碌状态
计算覆盖的重叠比例（通信的百分比）	0%（串行执行）	约 50%（或更高）	粗略估计：模型或批次越大，可实现的重叠越多

个 bucket 准备好后就会启动一次 all-reduce。调节 bucket_cap_mb 有助于针对特定模型拓扑提高重叠程度，但更大的 bucket 会增加最后一个 bucket 的延迟。

要点是，调优良好的 DDP 应当让大部分梯度通信与计算重叠。通常唯一无法重叠的通信部分是末尾部分，即最后一个梯度 bucket（如果它在最后的计算结束之后才完成）。研究工作正在尝试进一步让优化器步骤与通信重叠，或使用张量分区等技术实现更多重叠；不过，PyTorch DDP 的默认重叠策略通常被称为无等待反向传播（wait-free backpropagation，WFBP）：它把梯度分成 bucket，并在每个 bucket 准备好后立即启动归约。

值得注意的是，某些编码模式可能会意外消除重叠。例如，如果执行了任何强制在反向传播与下一次迭代之间同步的操作，就会让计算停下来，直到所有通信完成。

> **注**　在确认所有异步 GPU 工作已经完成之前，应避免执行会意外把张量从 GPU 移到 CPU 的操作（例如对张量调用.item()）。否则会强制同步，使计算停顿，并拖慢训练或推理工作负载。这通常发生在为调试而加入 print() 或 log() 语句时，而这类操作可能严重损害性能。

如果可能，应将这类操作移到独立的 stream 中。还应尽量减少手动调用 torch.cuda.synchronize()，只在需要精确基准测量或为了正确性时使用。否则，它

会串行化 GPU 工作，降低性能。DDP 的设计和 PyTorch 的操作本身已经是异步的，并会正确处理依赖；用户代码很少需要显式同步。

总之，重叠计算与通信是分布式性能工程中最有效的技术之一。正确使用后，它可以把通信延迟隐藏在有用工作之下，从而显著缩短训练时间。

在接下来的部分中，我们将探讨实现这种能力的软件和硬件基础设施，以及如何确保现代系统获得最大重叠程度。下一节先概览 NVIDIA Magnum IO 栈，其中包含让计算与通信重叠成为可能的技术。

4.2 NVIDIA Magnum IO 优化栈

Magnum IO 是 NVIDIA 的总体 I/O 加速平台，它汇集了一系列技术，用于加速 GPU、CPU、存储和网络接口之间的数据移动、访问与管理。如图 4-2 所示，Magnum IO 架构横跨存储、网络、网络内计算和 I/O 管理四个关键组件。

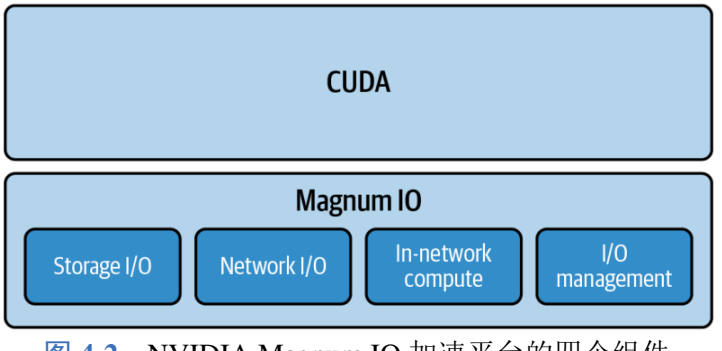

图 4-2　NVIDIA Magnum IO 加速平台的四个组件。

下面介绍图 4-2 中的四个组件：

4.2.1 存储 I/O

这部分由 NVIDIA GPUDirect Storage（GDS）和 BlueField SNAP 等技术实现。它们让 GPU 能够直接访问包括 NVMe SSD 在内的存储，而无需经过主机 CPU 内存进行不必要的拷贝。我们将在第 5 章深入讨论 GDS。

4.2.2 网络 I/O

其中包括 GPUDirect RDMA、NCCL、NVSHMEM、UCX 和 HPC-X（MPI/SHMEM 软件套件）等技术，用于实现节点间 GPU 之间直接、高速的数据传输，并绕过 CPU。

4.2.3 网络内计算

SHARP 在 Quantum 系列 InfiniBand 交换机内部执行网络内归约，归约算术由交换机芯片完成。BlueField DPU 会卸载网络工作，并可以承载 Subnet Manager 和 SHARP Aggregation Manager 等控制服务。当启用 NCCL RDMA SHARP 插件、互连网络具备 SHARP 固件且 Aggregation Manager 处于活动状态时，符合条件的集体操作可以卸载到 IB 交换机，从而减少主机和 GPU 的开销。稍后我们会更详细地介绍 NVIDIA SHARP。

基于以太网的 GPU 集群通常依赖 RoCEv2 等 RDMA 技术，但一般缺少 SHARP 这样的功能。这是许多超大规模 AI 系统使用 InfiniBand 或类似高性能互连，而不是以太网的原因之一。SHARP 能带来显著的性能提升，在可用时应当加以利用。

4.2.4 I/O 管理

NVIDIA NetQ 和 Unified Fabric Manager（UFM）等工具属于这一类别，为数据中心 I/O 互连提供实时遥测、诊断和生命周期管理。

这些组件通过灵活的 API 和 SDK 协同工作，隐藏了大量底层复杂性。Magnum IO 的目标是最大化大规模 AI 和数据分析工作负载的端到端吞吐量。随着新硬件出现，Magnum IO 也在持续演进。

如今 Magnum IO 已集成对 NVLink Switch 网络域的支持，使 GPU 能够在互连规模上进行机架内通信。它还利用 InfiniBand（Quantum-2 和 Quantum-X800 系列）以及以太网（Spectrum-X）的进展，进一步降低通信开销。

在本章中，我们将深入介绍其中的几个组件：用于网络传输的 RDMA、用于集体操作的 NCCL、用于推理数据移动的 NIXL，以及用于高效加载数据的

GDS。此外，我们还将了解如何对这些组件逐一进行性能分析和调优。下面进入正题。

4.2.5 借助 RDMA 实现高速、低开销的数据传输

RDMA 是一种针对低延迟、高吞吐量数据传输优化的技术。它允许设备之间直接进行内存到内存的通信，避免 CPU 承担不必要的数据拷贝操作。简而言之，RDMA 绕过了传统内核网络栈的大部分路径，使 NIC 能够直接读写应用程序内存。这避免了 CPU 参与每个数据包的处理，并减少上下文切换和缓冲区拷贝。只要可用且已验证，就应优先使用 RDMA 路径；并且始终通过日志和微基准测试确认（并持续重新确认）RDMA 数据路径确实处于活动状态。

在 Docker 和 Kubernetes 等容器环境中，要确保容器能够直接访问主机的 InfiniBand 设备（例如 /dev/infiniband）。否则，NCCL 可能会静默地从 GPUDirect RDMA 回退到 TCP socket，而且不会出现明显错误来提示性能退化。吞吐量可能因此从几十 GB/s 降到仅几 Gb/s，同时没有清晰的错误消息。

另一个相关的容器陷阱是容器中的 GID 分配与主机不匹配，某些“rdma-shared”Docker 镜像就可能出现这种情况。这会阻止 GPUDirect 注册，并使用 CPU 驱动的 RDMA 拷贝，而不是真正的 GPU RDMA。

> **注** 务必确认使用的是真正的 GPUDirect RDMA。使用 lsmod | grep nvidia_peermem 确认内核模块已加载，并检查 dmesg 中的初始化信息。端到端检查可以运行带 NCCL_DEBUG=INFO 的 NCCL，以确认 NET/IB 路径；还可以使用带 –use_cuda 的 RDMA perftest 验证 GPU 到 GPU 的传输。

吞吐量下降会在性能分析器中表现为数量级的吞吐量降低。但必须意识到这种可能性，并持续监控系统，以发现这类隐蔽的回退。

NVIDIA 面向 GPU 的 RDMA 实现称为 GPUDirect RDMA。它允许支持 RDMA 的 NIC（例如 InfiniBand 和融合以太网 RDMA，即 RoCE）通过 DMA，在两台服务器之间直接读写 GPU 设备内存，完全绕过主机 CPU 和系统 RAM。图 4-3 展示了使用 RoCE 的数据传输。

通过向 NIC 注册 GPU 缓冲区，GPUDirect RDMA 可以在远端 GPU 之间执行单边 RDMA 读写。这会在多节点训练中同时降低延迟和 CPU 开销。

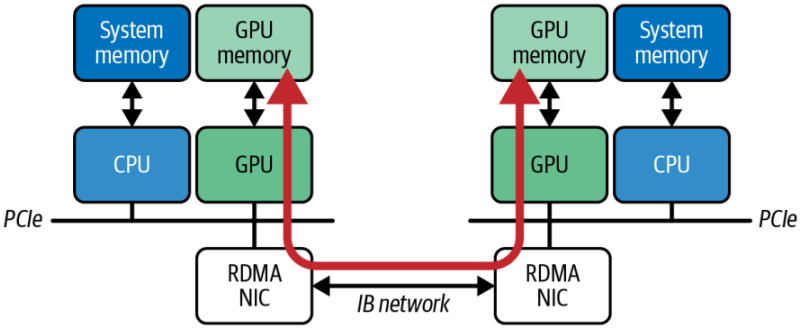

图 4-3　使用 RoCE 的 GPU 到 GPU 直接数据传输。

InfiniBand 原生支持 RDMA，某些高速以太网也可以通过 RoCE 支持 RDMA。使用 RoCE，可以在以太网上获得类似 RDMA 的零拷贝传输，前提是网络设备支持 RDMA 并且完成正确配置。使用 RoCE 的 RDMA 通常需要配置正确的系统和必要的驱动，包括用于 InfiniBand/RoCE 的 NVIDIA OFED。

使用 RDMA 与使用标准 TCP/IP 网络的性能差异可能非常大。例如，现代 InfiniBand 链路对小消息的延迟可能只有几微秒，而标准 TCP over Ethernet 的延迟可能高出 5–10 倍。

对于受网络带宽限制的大型传输，InfiniBand 上的 RDMA 可以维持数百 Gbps 量级的极高吞吐量。相比之下，典型 TCP/IP 网络会受到内核开销和 NIC 速率限制，除非使用具备 RDMA 能力的 200–400 Gbps 以太网，否则通常只有 100 Gbps 或更低。TCP/IP 网络还会产生更多开销。

在分布式深度学习中，大消息吞吐量往往比极小消息的延迟更重要，因为梯度通常很大。高带宽和低延迟都能保持协议高效运行，从而让 GPU 保持忙碌。

如果只能使用以太网，应采用尽可能高带宽、低延迟的配置。例如，使用带 RDMA（RoCE）的 200+ Gbps 以太网处理 all-reduce 流量，性能会远好于基础的 10–25 Gbps TCP 网络。至少应确保使用 MTU 9000 这样的巨型帧。为集群网络启用这一配置后，数据传输会发送更少但更大的数据包，而不是许多小数据包。这会降低 CPU 开销，提高效率，原理类似于更大的磁盘块大小能够提高顺序磁盘吞吐量。

出于类似原因，调优 TCP 栈也很重要。应确认 Linux sysctl 参数 net.core.rmem_max/wmem_max，以及自动调优范围 net.ipv4.tcp_rmem/tcp_wmem 足够高，以充分利用高速链路。

当然，还应使用现代 TCP 拥塞控制算法来提高高延迟链路上的吞吐量。在没有外部互联网流量、设计良好的专用集群网络中，默认的 CUBIC 拥塞控制通常表现良好，因为它本身就是为避免拥塞而设计的。

> **注** 对于任何高延迟、高带宽积的链路，可以考虑使用 Bottleneck Bandwidth and Round-trip propagation time（BBR）等现代 TCP 拥塞算法，并调节缓冲区大小以确保充分利用链路。可用 sysctl net.ipv4.tcp_congestion_control 检查并调节设置。

在云或混合环境中，要谨慎确认自己是否真正拥有受控的高速连接。例如，使用带 Elastic Fabric Adapter（EFA）的 AWS EC2 实例时，如果实例部署在同一个“置放群组（placement group）”中，实例之间可以获得类似 InfiniBand 级别的 RDMA。但如果尝试运行跨越本地数据中心和云端、且没有直接连接的多节点训练或推理任务，流量很可能会经过公共互联网。这会引入不可预测的延迟和拥塞。

> **注** 务必确保多节点环境运行在配置正确、高性能且低拥塞的网络上。应直接与云服务提供商合作，了解网络架构中的每一跳。

即使使用 RDMA，CPU 也并非完全置身事外。主机仍需建立 RDMA 传输并处理通信完成事件。因此，CPU 亲和性很重要。应将网络中断处理线程（或轮询线程）固定到与 NIC、理想情况下也与 GPU 位于同一 NUMA 节点的 CPU 核心上。例如，如果 InfiniBand 主机通道适配器位于 NUMA 节点 0，就应将其中断 CPU 亲和性核心绑定到节点 0。这能减少跨 NUMA 流量以及控制操作的延迟。

4.2.6 调优多节点连接

对于使用 GPU 的分布式多节点训练，确保网络不是瓶颈至关重要。这既包括采用前面介绍的正确通信和网络技术，也包括正确配置这些技术。下面列出一些应采用的建议和需要避免的陷阱：

4.2.7 了解拓扑

使用 nvidia-smi topo -m 可以获得基本的 GPU 互连视图；但对于基于 NVSwitch 和 NVLink 的系统，还建议使用 nvidia-smi nvlink 或 Nsight Systems 来了解多跳交换互连的连接情况。

4.2.8 可用时利用 NVLink Switch 网络域

多节点 NVIDIA GB200 和 GB300 NVL72 机架方案通过 NVLink Switch 将最多 72 个 GPU 连接到同一个 NVLink 域中，并提供几百纳秒量级的极低每跳延迟。GB200 NVL72 架构可在机架内所有 GPU 之间提供最高约 130 TB/s 的全互连带宽，以及亚微秒级延迟。如果集群包含这类基础设施，应确保作业位于同一个 NVLink 域内，以充分利用这种超高速互连。这可以显著减少节点之间对较慢 InfiniBand 和以太网通信的需求。值得庆幸的是，NVIDIA Quantum 系列等现代 InfiniBand 交换机每条链路可提供最高 800 Gb/s，并支持网络内计算功能。不过，NVLink 的巨大机架内带宽和小于 1 μs 的延迟更受青睐。只要可能，就让流量留在 NVLink/NVSwitch 上。

4.2.9 尽可能使用 RDMA

如果运行在 InfiniBand 或支持 RoCE 的硬件上，应确保 NCCL 这样的通信库确实在使用 RDMA。可用时，NCCL 会自动使用 GPUDirect RDMA。但如果 RDMA 配置错误或不受支持，NCCL 可能会静默回退到 TCP。一个警示信号是：在 all-reduce 操作期间 GPU 利用率下降，而 CPU 利用率升高。这表明 CPU 正在为通信拷贝数据。

4.2.10 可用时使用多个 NIC 聚合带宽

一些服务器具有多个网络接口（NIC）。NCCL 可以在多个 NIC 之间条带化流量（称为 multirail）以提高带宽，但可能需要设置 NCCL_NSOCKS_PERTHREAD 和 NCCL_SOCKET_NTHREADS 等环境变量来进行优化。要确保每个 NIC 位于

不同子网，并且 NCCL 能发现两者。在正确配置的情况下，两块 800 Gbps NIC 并行可提供 1.6 Tbps 聚合带宽，四条这样的链路可达到约 3.2 Tbps。

4.2.11 可用时使用优化的“直接 NIC”模式

优先采用高带宽的 multirail NIC 配置，为每个 GPU 或小型 GPU 组提供足够的专用网络带宽。物理上，NIC 通过 PCIe 连接到主机 CPU 或 DPU。在现代 GPU 系统中，NCCL 支持使用 InfiniBand GPUDirect Async（IBGDA）和 direct NIC 路径发起 GPU 网络通信，如图 4-4 所示。这让 GPU 可以在无需 CPU 介入的情况下驱动全带宽 RDMA。

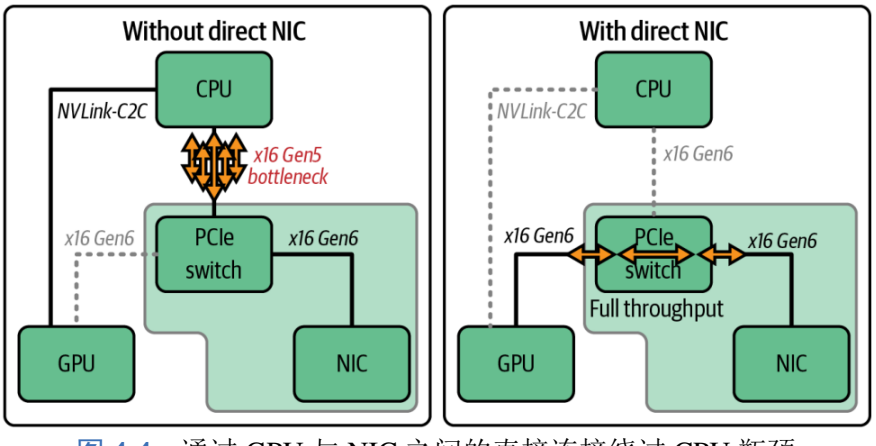

图 4-4　通过 GPU 与 NIC 之间的直接连接绕过 CPU 瓶颈。

4.2.12 检查配置错误

一个常见陷阱是网络配置不匹配，导致回退到较慢的路径。例如，如果 RDMA 因配置错误而无法工作，NCCL 可能会在 100 Gbps 以太网上使用 TCP，但由于内核开销只能达到其中一小部分。更糟糕的是，如果集群的高速网络被错误识别，流量可能会在用户不知情的情况下经过只有 10 Gbps 以太网的较慢管理网络。NCCL 的调试输出以及网络接口计数器（ibstat、ifstat）可以帮助确认实际使用最重的接口。对于具有 200–400 Gbps 大链路的现代系统，降到 10 Gbps 会造成严重瓶颈。

4.3 多节点通信陷阱

在集群中跨多个节点扩展训练会引入一类新的陷阱。下面列出一些常见问题，并通过具体示例说明如何修复。

4.3.1 陷阱 1：使用 CPU 受限的 Gloo 后端而不是 NCCL

PyTorch 分布式框架支持多个通信后端。对于多 GPU 训练，NCCL 是 NVIDIA GPU 的首选后端；但还有一个称为 Gloo 的回退后端，它使用 CPU 和 TCP socket。如果在 GPU 训练中错误地用 Gloo 初始化 ProcessGroup，或者 NCCL 初始化失败并回退到 Gloo，训练仍然可以正确运行，但所有跨 GPU 通信都会经过 CPU 和以太网栈。这会导致极慢的性能。

不幸的是，这种错误配置相当常见，因为代码看起来能正常运行，也不会崩溃，只是慢一个数量级，必须通过性能分析器或仔细分析日志才能发现。总之，多 GPU 训练应始终指定 NCCL，以利用 RDMA 通信。幸运的是，PyTorch 的默认设置就是 NCCL。否则，回退到 Gloo 会静默限制性能（甚至完全报错）。

下面用代码说明这一点。我们将模拟两个进程：gpu/rank 0 和 gpu/rank 1，分别位于两个不同节点上，节点通过 800 Gb/s（100 GB/s）的 InfiniBand 互连连接；然后对一个大型张量执行 all-reduce。首先，我们故意使用 CPU 受限的 Gloo 后端进行 PyTorch 分布式通信：

```
# dist_allreduce.py
#!/usr/bin/env python
import os
import argparse
import torch
import torch.distributed as dist
def main():
    parser = argparse.ArgumentParser(description="Multi-node Gloo
    ↪ all-reduce")
    parser.add_argument(
        "--data-size",
        type=int,
```

```
        default=1024 * 1024 * 100,  # 100M floats ~ 400 MB
        help="Number of elements in the tensor",
    )
    args = parser.parse_args()
    # Initialize the default ProcessGroup over env://
    # (uses MASTER_ADDR, MASTER_PORT, etc.)
    dist.init_process_group(backend="gloo", init_method="env://")
    rank = dist.get_rank()
    world_size = dist.get_world_size()
    # Allocate a large CPU tensor (Gloo is CPU-bound)
    tensor = torch.ones(args.data_size,
       dtype=torch.float32, device="cpu")
    # Warm up and barrier
    dist.barrier()
    # All-reduce (sum) across all ranks
    dist.all_reduce(tensor, op=dist.ReduceOp.SUM)
    # Barrier
    dist.barrier()
    ...
    dist.destroy_process_group()
if __name__ == "__main__":
    main()
```

在这段代码中，我们故意选择 backend=”gloo”。由于 Gloo 受 CPU 限制，张量（400 MB）会在每个 rank 的 CPU 上分配（device=”cpu”）。使用 Gloo 时，集体操作在 CPU 内存上执行，并通过 TCP 通信。如果改用 GPU 张量，PyTorch 会经由 CPU 转发，仍受该路径限制。无论哪种情况，速度都远慢于 GPU 上的 NCCL，因为后者允许 GPU 直接使用 RDMA 通信。

如果对 Gloo process group 调用 dist.all_reduce()，运行要么回退到主机转发路径，要么根据构建方式直接失败。这违背了测量 GPU 集体操作的目的。

在正确计时的情况下，运行这段代码会得到如下结果：

```
Rank0: All-reduce of 400.0 MB took 200.00 ms (2 GB/s)
```

对于 400 MB 数据，200 ms 很慢，因为这相当于 2 GB/s 的聚合吞吐量，远低于本例所用 800 Gb/s InfiniBand 硬件应达到的 100 GB/s。这表明 CPU 路径产生了大量额外开销。通过分析 CPU 利用率可以确认这一点；本例中的 CPU 利用率接近 100%。

现在把后端改成 NCCL，看看差异。只需设置 backend="nccl"，并确保环境已配置为允许 GPU 直接通信。假设 NCCL 配置正确，代码就会使用 GPU 到 GPU 的直接通信。改进非常显著；使用更新后的代码和计时，会看到类似下面的结果：

```
Rank0: All-reduce of 400.0 MB took 4.00 ms (100 GB/s)
```

这里 400 MB 只需 4 ms，即 100 GB/s。这比前面的 2 GB/s 快两个数量级，并达到了本例 800 Gb/s InfiniBand 硬件的链路速率上限。它远胜于 Gloo 的结果。这说明对 GPU 多节点通信而言，使用 NCCL 极其重要。错误的后端会严重降低性能。简而言之，应使用 backend="nccl"，这样集体操作就能在 GPU 上执行，并在可用时使用 GPUDirect RDMA。

> **注** 你可以通过 torch.distributed.get_backend() 调用验证 PyTorch 中的后端。

从 GPU 的角度看，NCCL 的 all-reduce 集体操作由 GPU 执行，通过直接内存访问读取另一块 GPU 的内存。因此，GPU 会在通信期间保持忙碌：要么由 SM 执行某些网络拷贝 kernel，要么由 GPU 的 DMA 引擎处于活动状态。相比之下，使用 Gloo 时，GPU 在通信期间基本处于空闲，因为 CPU 参与其中，GPU 必须等待数据通过 CPU 内存缓冲区和 TCP 传输，而不是通过 InfiniBand。

> **注** 在生产集群环境中，如果存在多个 NIC，应显式设置 NCCL_SOCKET_IFNAME=ib0，让 NCCL 的初始化阶段 TCP 握手经过 InfiniBand 主机通道适配器（HCA），随后切换到最快路径上的 GPUDirect RDMA。务必确保所有节点都能通过所选互连相互到达。

4.3.2 陷阱 2：NCCL 版本不匹配

如果运行 PyTorch 内置的 NCCL（例如 torch.cuda.nccl.version() == ()），同时系统安装的 libnccl 却是另一个版本，系统可能会挂起，甚至静默回退到较慢的实现。这很难检测。应通过匹配 nvidia-nccl-cu* 软件包，或用系统 NCCL 重新构建 PyTorch，确保版本一致，避免这些兼容性陷阱。

4.3.3 陷阱 3：NCCL 引导期间 TCP 端口耗尽

NCCL 使用临时 TCP 端口进行带外设置。如果操作系统的 net.ipv4.ip_local_port_range 过窄，就可能耗尽可用端口，导致握手失败或停滞。建议在 /proc/sys/net/ipv4/ip_local_port_range 中扩大端口范围（例如 50000 51000），以避免隐藏的引导失败。现代 NCCL 已改进引导处理，但大型集群仍应主动设置宽泛端口范围。

4.3.4 陷阱 4：网络带宽不足或 NIC 配置错误

另一个多节点陷阱是同步数据量与网络带宽不匹配，或者没有使用所有可用接口。随着 GPU 集群扩展，这个问题会越来越常见。例如，Blackwell GPU 很容易使每个节点的一条 400 Gbps 链路达到饱和。

在这些不利条件下分析工作负载时，会发现扩展到多个节点后训练明显变慢。换句话说，“每 GPU 吞吐量”会下降。此时应检查网络链路。

例如，可以使用 nvidia-smi dmon 监控网络吞吐量，收集 NVLink/PCIe/网络统计信息。也可以使用 ethtool -S <iface> 或 ip -s link show <iface> 获取字节/数据包计数器，或者启动 iftop、nload 等交互式监视器，观察 NIC 的实时吞吐量。

如果有多个接口，也可以尝试利用它们。例如，如果一条 800 Gbps（100 GB/s）的 InfiniBand 链路已经饱和，而作业需要更高网络吞吐量，可以考虑启用 NCCL 的多 NIC 支持。要调好 NCCL_NSOCKS_PERTHREAD 和 NCCL_SOCKET_NTHREADS，因为它们控制 NCCL 用于网络传输的并行连接数和线程数；提高这些值可能有助于同时利用多块 NIC。

例如，如果有两块 NIC，可以设置 NCCL_NSOCKS_PERTHREAD=2，并保持 NCCL_SOCKET_NTHREADS=2（2 个线程 × 2 个 socket = 每个进程总共 4 个连接）。不要任意增大这些值；根据 NVIDIA 指导，线程数与 socket 数的乘积不应超过 64。

> **注** 应逐步提高这些与线程相关的设置（例如从 2 → 4 → 8），并持续测量吞吐量。线程过多会争用资源，可能使收益递减。

4.3.5 陷阱 5：落后节点或进程

在多节点训练中，最慢的节点或 GPU 会决定整体进度，因为同步必须等待每个节点和 GPU 响应。如果某台机器的网络链路更慢，或被其他任务过度占用，就会拖慢整个作业。

为了避免落后者，应尽可能使用同构硬件，并为每个训练作业提供专用的集群资源。这样环境才具有可预测性。例如在云环境中，混用不同实例类型或不同交换互连可能引入差异。

使用 NVIDIA DCGM 或每个节点上的 InfiniBand 计数器等监控工具，可以发现某个节点是否因 NIC 链路抖动或 GPU 热节流而性能下降。还可以使用 PyTorch 的 torch.distributed.monitored_barrier 等集体性能分析工具，判断某个 rank 是否持续落后，如下所示：

```
# barrier_straggler
import torch
import torch.distributed as dist
import os
import datetime
def run(rank, world_size):
    dist.init_process_group(backend="nccl", init_method="env://")
    local_rank = int(os.environ["LOCAL_RANK"])
    torch.cuda.set_device(local_rank)
    # ... your forward/backward work here ...
    # Before syncing at end of iteration, use a monitored barrier:
    try:
```

```
        # Wait up to 30 seconds for all ranks
        # if one lags, you' ll get a timeout on that rank
        dist.monitored_barrier(timeout=datetime.timedelta(seconds=30))
    except RuntimeError as e:
        print(f"Rank {rank} timed out at barrier: {e}")
    # Now proceed knowing all ranks are roughly in sync
    dist.destroy_process_group()
```

dist.monitored_barrier(timeout=datetime.timedelta(seconds=30)) 会在任何 GPU 未能在 30 秒内到达时引发错误，从而帮助定位落后者。将其与 NCCL_DEBUG=INFO 和 NCCL_ASYNC_ERROR_HANDLING=1 结合，可以同时获得 PyTorch 与 NCCL 的日志，了解哪个 rank 或链路较慢。

应使用 PyTorch 的 torch.distributed.monitored_barrier 和 NCCL 的异步错误处理等现代监控工具，快速发现这类问题。

4.3.6 陷阱 6：UCX/RDMA 下的 GPU 内存碎片

PyTorch 的缓存分配器会在迭代之间保留 GPU 内存。在使用 UCX/RDMA 的分布式环境中，这些长期存在的分配可能耗尽注册池或造成内存碎片，导致偶发的分配失败或性能断崖。监控 torch.cuda.memory_reserved() 与 memory_allocated() 的差值，有助于发现这些边界情况：

```
import torch
import torch.distributed as dist
import os
import time
def log_mem(iteration):
    reserved = torch.cuda.memory_reserved()
    allocated = torch.cuda.memory_allocated()
    print(f"[Iter {iteration:02d}] Reserved: {reserved/1e9:.3f} GB, "
          f"Allocated: {allocated/1e9:.3f} GB")
def run(rank, world_size):
    # Standard DDP / UCX init
    dist.init_process_group(backend="nccl", init_method="env://")
```

```
    rank = int(os.environ["LOCAL_RANK"])
    torch.cuda.set_device(rank)
    # Pre-allocate a big buffer that UCX will register once and hold
    big_buffer = torch.empty(int(2e8), device=rank)  # ~0.8 GB
    log_mem(0)
    for i in range(1, 11):
        # Simulate per-iteration tensor allocations of varying size
        small = torch.randn(int(1e7), device=rank)  # ~40 MB
        medium = torch.randn(int(5e7), device=rank) # ~200 MB
        # Free them explicitly to return to allocator cache
        del small, medium
        torch.cuda.synchronize()
        # Log memory after freeing
        log_mem(i)
        # Barrier so all ranks print in sync
        dist.barrier()
        time.sleep(0.1)
    dist.destroy_process_group()
if __name__ == "__main__":
    run(0, 1)
```

这段代码的输出显示，随着缓存分配器保留已释放的块，reserved memory 会在每次迭代中增长，从而加快后续分配。不过，它不会将这些内存归还给操作系统或 UCX 注册池。每次释放后 allocated memory 会降回零，因为已经没有存活的张量：

```
[Iter 00] Reserved: 0.800 GB, Allocated: 0.800 GB
[Iter 01] Reserved: 1.040 GB, Allocated: 0.000 GB
[Iter 02] Reserved: 1.240 GB, Allocated: 0.000 GB
...
[Iter 10] Reserved: 1.240 GB, Allocated: 0.000 GB
```

为缓解这种情况，应确保升级到最新的 CUDA runtime。也可以把 torch.cuda.empty_cache() 作为最后手段，尝试恢复因碎片造成的问题。但 empty_cache() 不是长期解决方案。长期方案包括调节分配器、追踪问题根源并修复它。

多节点陷阱可以总结如下：GPU 通信始终使用 NCCL；确保 RDMA/高速网络处于活动状态；尽可能利用全部可用带宽（包括多个 NIC）；使用随 CUDA 发布的最新 NCCL 以获得最新修复；并警惕任何会导致回退到较慢通信方式的配置。接下来的部分将聚焦 NCCL 的具体机制，以及如何优化节点内和节点间的 GPU 通信。

4.4 用于分布式多 GPU 通信的 NCCL

NVIDIA NCCL 是一个用于多对多通信操作（称为集体操作）的库，供 GPU 组共享数据使用。NCCL 支撑着 NVIDIA 生态中大多数多 GPU 训练工作负载。

NCCL 为 all-reduce、all-gather、broadcast 和 reduce-scatter 等集体通信操作提供优化实现，规模可以从几块 GPU 扩展到数千块，未来甚至数百万块。在多 GPU 上执行模型训练和推理时，模型权重、梯度和激活等数据必须快速交换，才能让 GPU 保持忙碌。NCCL 就是高效组织这些交换的库。

在分布式训练中，每块 GPU 都会在自己的数据分片上计算梯度。随后使用 NCCL 在所有 GPU 之间对这些梯度执行 all-reduce，使每块 GPU 都能使用平均梯度更新模型权重。

在分布式推理中，GPU 需要交换激活和其他中间结果。

在推理场景中，一些框架会使用 NCCL 的 send() 和 recv()，但许多部署更倾向于使用 UCX 或 NIXL 等专用库提供的传输，以获得更低的尾延迟和更好的重叠。

NCCL 针对 NVIDIA GPU 进行了优化，并支持 PCIe、NVLink、NVSwitch、InfiniBand 和 TCP socket 等多种互连。它会自动选择任意两块 GPU 之间可用的最快路径。

与 NCCL 配套的较新库 NVIDIA Inference Xfer Library（NIXL）针对推理进行了优化，适合 KV 缓存移动等点对点传输。

NIXL 提供可插拔的存储后端，包括 POSIX 文件和 GPUDirect Storage（GDS）。Amazon S3 等对象存储支持由对象存储插件提供，并取决于部署。这

些插件会在适当时在内存层级与存储后端之间移动 KV 缓存片段。我们稍后会深入介绍 NIXL，未来专门讨论推理工作负载调优的章节也会再次涉及它。

> **注** 由于延迟更低，许多推理部署如今更偏好使用 NIXL 进行这些点对点传输。不过，NCCL 的 send() 和 recv() 仍然可用，只是它们在最小延迟方面不如 NIXL 优化。实践中，大规模推理工作负载更倾向于使用 NIXL 执行一对一传输；只有需要自定义集成时，才较少使用 NCCL send/recv。

4.4.1 NCCL 中的拓扑感知

拓扑感知对 NCCL 的性能起着重要作用。NCCL 会检测 GPU 的物理连接方式，并据此优化通信模式。例如，在 GPU 通过 NVLink 和 NVSwitch 完全互连的系统中，每块 GPU 都会使用这些高速互连与其他 GPU 通信。

NCCL 可以使用简单的环形 all-reduce 模式，让每条链路承担相等通信量；但它还会自动使用拓扑感知的分层通信模式，以最大化通信性能。例如，对于具有多个 NUMA 节点域的系统，NCCL 可能先执行节点内归约，再执行跨节点归约，最后执行节点内广播，这实际上是一个分层 all-reduce。目标是让尽可能多的流量走最快的互连。

具体来说，考虑这样一种拓扑：GPU 0–1 共享一条 NVLink 连接，GPU 2–3 共享另一条 NVLink 连接；但 0–1 对与 2–3 对之间的通信必须经过速度较低的 PCIe 互连。在这种情况下，NCCL 的分层算法会先在每个 NVLink 连接的 GPU 对内执行 reduce 集体操作，然后让每对中的一块 GPU 通过 PCIe 链路交换归约结果，最后再次在每对内部分发数据。这样，慢速 PCIe 链路只需处理部分数据。

NCCL 通常会在检测到这类拓扑后选择性能最佳的方案。不过，在某些情况下，自动检测可能不会启用，例如拓扑选择性能相近，或消息很小等。可以通过环境变量 NCCL_ALGO 覆盖算法选择（例如 NCCL_ALGO=NVLS,NVLSTree,Tree,Ring,PAT 等），但通常 NCCL 会自动选出最佳路径。手动覆盖一般只用于故障排查、研究实验等特定场景。

为了说明拓扑的影响，考虑一个具有四块 GPU 和两台 PCIe 交换机的系统（GPU 0–1 位于一台交换机，GPU 2–3 位于另一台）。对四块 GPU 使用单一环进行朴素 all-reduce，会让大量数据经过 PCIe 交换机之间的链路，形成主要通信瓶

颈。相比之下，将 0-1 和 2-3 分成两个独立环，再由每个环/对中的一块 GPU 进行交换，可以降低通信压力。

使用 Nsight Systems 或 NCCL trace 可以快速判断 NCCL 是否针对拓扑进行了优化。分层模式会显示多个 NCCL CUDA kernel，包括组内和组间 all-reduce kernel。未优化算法可能只有每 GPU 数十 GB/s，而拓扑感知算法可以充分利用 NVLink、减少 PCIe 使用，将每 GPU 吞吐量提高到数百 GB/s。

考虑一个朴素方案：GPU 等待 PCIe 互连，SM 利用率只有 60%，且没有重叠时总迭代时间为 100 ms。许多 warp 会因等待慢速 PCIe 互连上的数据传输而停顿。拓扑感知方案中，SM 利用率升至 90%，总迭代时间从 100 ms 下降 30% 变为 70 ms，内存停顿周期明显减少，如表 4-2 所示。

表 4-2 应用拓扑感知 NCCL 以及计算—通信重叠优化前后的关键 GPU 性能指标。

指标	之前（无重叠）	之后（有重叠）
SM 忙碌度	60%	90%
内存停顿 warp	高	低得多
迭代时间	100 ms（无重叠）	70 ms（有重叠）

总之，拓扑感知可能决定多 GPU 系统扩展的成败。应让尽可能多的通信走最快互连——节点内通常是 NVLink/NVSwitch——并尽量减少经过 PCIe 和跨 NUMA 节点链路等慢路径的传输。

如果多 GPU 作业在单节点上无法扩展，首先应确认流量没有被迫走慢路径。GPU 只有固定数量的直接 NVLink 通道；例如 GB200/GB300 NVL72 中每块 Blackwell GPU 支持 18 条约 100 GB/s 的 NVLink 5 链路，GPU 到 GPU 的双向聚合带宽约 1.8 TB/s，是上一代 900 GB/s 的两倍。

没有直接配对的设备之间通信时，可能退回到更少通道甚至 PCIe。若跨越 NUMA 域，吞吐量会显著下降。NVL72 机架内全部 72 块 Blackwell GPU 属于同一 NVLink Switch 域，任意 GPU 都能在单个 NVSwitch 阶段以全二分带宽访问其他 GPU。

NCCL 的分层算法应自动选择高带宽路由，但应通过性能分析确认。如果仍遇到带宽上限，应将作业限制在连接紧密的 GPU 子集上。

例如，同一 NUMA 节点或同一 NVSwitch 岛内的四块 GPU，通常远好于让八块 GPU 跨越较慢链路。有限或间接链路产生的额外同步开销，往往会超过使用更多设备的收益。

NVIDIA superchip 模糊了 CPU 内存和 GPU 内存之间的界限。例如 Grace Blackwell Superchip 通过约 900 GB/s 的 NVLink-C2C 提供极快的 CPU-GPU 互连，使 CPU 内存能够充当 GPU 内存的高速扩展。

这意味着，即使 all-reduce 的一部分涉及 CPU 或系统内存，其速度也可能达到老一代 GPU 到 GPU 链路的水平。关键是通过最快路径优化节点内通信，并尽量减少慢路径。

> 注　NVIDIA Nsight Systems，或者 NCCL trace（设置 NCCL_DEBUG=INFO 和 NCCL_TOPO_DUMP_FILE=<path>），可以显示 NVLink 路径是否得到充分利用。

4.4.2 NCCL 通信算法

NCCL 会根据数据大小、GPU 数量和拓扑采用不同通信算法。主要算法包括 Ring、Tree、CollTree、CollNet 和 Parallel Aggregated Tree（PAT）等。

4.4.3 Ring

在 ring all-reduce 中，GPU 逻辑上排列成环。每块 GPU 以流水线方式向邻居发送数据，并从另一侧邻居接收数据。每个数据块会在环上循环并累积部分和。Ring 能均衡网络负载，每块 GPU 发送和接收相同数据量；每条链路传输 2 ×（data_size ÷ num_gpus）字节，因此带宽最优。缺点是延迟：数据必须经过所有跳数，总时间随 GPU 数量增长。Ring 适合大消息，因为移动字节的时间远超启动传输成本，这是带宽主导型工作负载。

4.4.4 Tree 和 NVLSTree

tree 算法使用生成树结构执行归约和广播。all-reduce 实际上是 reduce-scatter 后接 broadcast。N 块 GPU 可在 O(log N) 步完成 all-reduce，而 ring 需要 O(N)，

因此 tree 对小消息延迟更低。但大消息不一定能充分利用所有链路，因为并非所有 GPU 始终传输。NCCL tree 通常用于传输启动延迟占主导的小消息，即延迟主导型工作负载；NVLSTree 还会启用 NVLink SHARP 卸载。

4.4.5 CollTree（分层树集体操作）

CollTree 构建两级树，在保持本地高带宽的同时降低延迟。本地高速域内先执行 reduce-scatter 和 broadcast，再由每个本地组的 leader 通过 RDMA 参与跨组第二级树。两个层级流水线运行，本地完成的张量块可立即进入跨组阶段。与平坦 ring 相比，它将跨节点步骤降为 O(log N)，并缩短小中消息延迟；NVLink 域还可利用 NVLink SHARP，InfiniBand 上则可在启用 NCCL SHARP 插件时卸载到 SHARP。超大消息可能更适合 Ring 或 PAT，而跨节点延迟占主导时更适合 CollTree。

4.4.6 CollNet（跨节点的分层集体操作）

CollNet 也称树并行，结合两种集体策略优化不同规模通信。它先将共享高速本地互连的 GPU 分组，再在每组内使用 ring 或本地 tree 聚合数据；每组一个 leader 参与跨组第二级树归约。通过在全局树交换之上叠加本地归约，CollNet 同时提供节点间低延迟和节点内高带宽，尤其适合降低超大型多节点 GPU 集群的网络负载。

4.4.7 Parallel Aggregated Tree（PAT）

PAT 是 NCCL 对 ring 和 tree 的流水线混合方案。一个片段完成树归约后，下一片段立即以交错轮询方式开始归约，从而保持链路饱和并达到接近 ring 的带宽，同时将每片段启动延迟限制在 O(log N)。实践中，PAT 把大消息拆块，连续启动各块的 tree reduce-scatter，使始终有工作在途；结果是大传输接近 ring 级吞吐量，较小片段获得 tree 级延迟优势。

NCCL 算法的选择通常取决于消息大小和拓扑。几十 MB 量级的小消息偏好 tree，因为步骤少；大消息偏好 ring，因为带宽利用率更好。

默认情况下，NCCL 会针对集体操作、消息大小和拓扑自动选择最佳算法。它支持对称内存优化和低延迟 kernel，可降低 NVLink 系统中小中消息的 all-reduce 延迟，某些情况下测得降低最高约 7.6 倍；这些改进与 PAT 进一步降低 NVL72 等系统的通信开销。

NCCL 会持续演进拓扑感知策略，例如利用 NVSwitch 硬件多播，在 NVLink 域内执行一跳广播，适合将相同的更新权重同时发送给所有 GPU。

NCCL 还利用 SHARP 和 NVLink SHARP（NVLS）。配置 NCCL-SHARP 插件后，在支持的系统上可加速 all-gather 和 reduce-scatter。

PAT 结合 ring 式吞吐量与 tree 式延迟，将大消息拆块，检查 GPU/交换机拓扑，并在 CUDA stream 上交错 reduce-scatter 与 all-gather，从而兼得两者优势。

初始化 communicator 时，NCCL 会检查消息大小、拓扑和 GPU 代际，选择最快算法与协议组合。若性能分析发现跨节点延迟异常，可设置 NCCL_ALGO=NVLSTree,PAT 覆盖算法；在代码中设置时要先于 ncclCommInitRank()。

4.5 分布式数据并行策略

大规模训练以及多十亿、数万亿参数模型的推理，需要组合数据并行、张量模型并行、流水线并行、专家并行、上下文并行等策略，以实现近似线性扩展并避免浪费 GPU。

关键是在每一层让通信与计算重叠，包括使用 NCCL 执行 all-reduce 和 NIXL 执行一对一传输，从而高效扩展到数千甚至数百万块 GPU。

> **注** 梯度累积和激活检查点等技术对于超大规模内存管理也很关键，能够控制内存占用而不牺牲吞吐量。

单节点扩展到多 GPU 时，PyTorch 提供数据并行（拆分数据）和模型并行（拆分模型）。这里比较两种基本数据并行策略：nn.DataParallel（DP）和 torch.distributed.DistributedDataParallel（DDP）；选错会严重影响性能。

4.5.1 数据并行（DP）

DP 是易用 API，由单进程或单 Python 线程控制多个 GPU。它拆分输入批次，在各 GPU 前向，将输出收回主 GPU 计算聚合损失；反向时再收集梯度到主 GPU、求平均并广播回其他 GPU。

DataParallel 的训练循环是单进程的，集成简单，但启动不同设备操作受 Python GIL（全局解释器锁）限制。因此 DP 在超过 2–4 块 GPU 后扩展性差，单 Python 线程成为瓶颈；梯度收集也是同步的，不与计算重叠，类似前面的无重叠场景。

4.5.2 完全分片数据并行（FSDP）

FSDP 不再保存完整模型副本，而是在 GPU 之间对激活、梯度和参数分片，大幅降低内存开销。对于超大规模模型，FSDP 常与张量并行、流水线并行等策略结合。

> **注** 我们将在第 13 章讨论 FSDP 及专家并行等超大规模训练技术。本节聚焦 DP 和 DDP，为后续复杂性打下基础。

4.5.3 分布式数据并行（DDP）

DDP 为每个 GPU 使用一个进程，并依靠 NCCL 通信梯度。除 FSDP 外的简单数据并行策略通常都让每个进程拥有一份模型副本。

反向传播期间，梯度直接在 GPU 之间交换，即执行 all-reduce；这种通信通常与反向计算重叠。

DDP 用独立进程完全避免 GIL 问题，并由 NCCL 高效的 C++ kernel 处理通信。因此多 GPU 训练中 DDP 几乎总是优于 DP；PyTorch 开发者也建议严肃工作使用 DistributedDataParallel。

回到前面的例子，这次比较单节点上的 DP 和 DDP。我们使用简单模型，测量一次训练步骤。

本场景使用 DataParallel 包装模型，使 PyTorch 拆分输入批次并使用两块 GPU；下面测量一次训练迭代：

```
# before_dataparallel.py
import torch
import torch.nn as nn
import torch.optim as optim
# Dummy model and dataset
class SimpleNet(nn.Module):
    def __init__(self, input_size, hidden_size):
        super(SimpleNet, self).__init__()
        self.linear1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.linear2 = nn.Linear(hidden_size, 1)
    def forward(self, x):
        return self.linear2(self.relu(self.linear1(x)))
# Setup model and data
input_size = 1024
hidden_size = 256
model = SimpleNet(input_size, hidden_size)
model.cuda()  # move model to GPU 0, it will also replicate to GPU 1
model = nn.DataParallel(model)    # utilize 2 GPUs (0 and 1 by default)
optimizer = optim.SGD(model.parameters(), lr=0.01)
data = torch.randn(512, input_size).cuda()   # batch of 512 on GPU0
target = torch.randn(512, 1).cuda()          # target on GPU0
# Run a single training step
output = model(data)                       # forward (DP splits data
↪ internally)
loss = nn.functional.mse_loss(output, target)
loss.backward()                           # backward (DP gathers grads to
↪ GPU0)
optimizer.step()
```

nn.DataParallel 的前向传播会把形状 [512,1024] 的输入拆成两个 [256,1024] 张量，分别发送到 GPU 0 和 GPU 1。前面代码中 GPU 1 和 GPU 2 包含相同模型副本。

> **注** 严格来说模型最初只在 GPU 0；第一次调用 forward 时，DP 会在执行计算前自动将模型复制到 GPU 1 和其他参与 GPU。

随后 DataParallel 使用每 GPU 一个线程并行启动前向传播。但线程共享同一 Python 进程并争用 GIL，kernel 启动调用在 Python 中串行；排入队列的 GPU 工作仍可在设备上并发执行。

前向后，DataParallel 将每 GPU 输出收集到主设备 GPU 0 计算损失；反向时把副本梯度收集并求和到 GPU 0，再广播回其他 GPU，然后执行优化器步骤。

DataParallel 中 GPU 0 承担收集和求和全部梯度的额外负担。每次梯度归约（GPU 1 → GPU 0 再返回）同步并阻塞反向传播。Python 控制线程还必须串行化 kernel 启动，且梯度聚合不与计算重叠，GPU 0 可能成为瓶颈。

切换到 DistributedDataParallel 后，可用 torch.multiprocessing.spawn 为每 GPU 启动一个进程。每个进程持有完整模型副本并处理批次的一部分；两进程各处理 256，因而总批次仍为 512。

```
# after_ddp.py
import os, time
import torch
import torch.nn as nn
import torch.optim as optim
import torch.distributed as dist
import torch.multiprocessing as mp
class SimpleNet(nn.Module):
    def __init__(self, input_size, hidden_size):
        super(SimpleNet, self).__init__()
        self.linear1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.linear2 = nn.Linear(hidden_size, 1)
    def forward(self, x):
        return self.linear2(self.relu(self.linear1(x)))
def train_ddp(rank, world_size):
    rank = int(os.environ.get("LOCAL_RANK", rank))
    torch.cuda.set_device(rank)
    dist.init_process_group("nccl",
```

```
        init_method="env://",
        world_size=world_size, rank=rank)
    model = SimpleNet(input_size=1024,
        hidden_size=256)
    model.cuda(rank)
    ddp_model =
        nn.parallel.DistributedDataParallel(model,
          device_ids=[rank])
    optimizer = optim.SGD(ddp_model.parameters(),
        lr=0.01)
    # Each process gets its own portion of data
    batch_size = 256
    data = torch.randn(batch_size, 1024).cuda(rank)
    target = torch.randn(batch_size, 1).cuda(rank)
    # Run one training iteration
    output = ddp_model(data)
    loss = nn.functional.mse_loss(output, target)
    loss.backward()
    optimizer.step()
    dist.destroy_process_group()
if __name__ == "__main__":
    main()
```

可以使用 torchrun，或集群的 MPI/SLURM/Kubernetes 集成运行代码，并设置 MASTER_ADDR 和 MASTER_PORT。具体结果取决于工作负载和硬件。

```
# Environment (common gotchas)
export NCCL_DEBUG=INFO
export NCCL_ASYNC_ERROR_HANDLING=1
export NCCL_SOCKET_IFNAME=ib0      # use your HCA (e.g., ib0, ib1)
# Optional: for multi-rail IB, set NIC ordering
# so ranks use distinct rails.
# Local bring-up, 2 GPUs
torchrun --standalone --nproc-per-node=2 after_ddp.py
# SLURM (example)
srun --ntasks=$WORLD_SIZE --gpus-per-task=1 --nodes=$NNODES \
     --cpus-per-task=8 python after_ddp.py
### Output: ###
DDP step took 30.00 ms
```

具体数值会有所不同，但 DDP 迭代明显快于 DP。本例中两者处理相同总数据量，DDP 仍比 DP 快 33%。

性能提升来自多个因素：每个进程处理一半批次且无 GIL 争用；梯度用 NCCL all-reduce 并与反向计算重叠；无需把梯度拷贝到单个聚合 GPU 再拷回；每块 GPU 的梯度直接交换并就地求平均。最后，通信工作分摊到参与 all-reduce 的所有 GPU，而不是由一块 GPU 独自承担。

在我们的示例中，DDP 完成耗时 30 ms，而 DP 为 45 ms。对于更大模型，差距会更大，尤其是在扩展到数千块 GPU 时。超过 2–4 块 GPU 后，DP 可能出现超线性退化，因为主线程会不堪重负；而 DDP 往往可以良好扩展到 GPU 数量，主要受通信带宽限制，而不是 CPU 或单 GPU 开销限制。

总而言之，多 GPU 训练应始终优先使用 DistributedDataParallel，而不是 DataParallel。DP 唯一可能可以接受的场景，是在两块 GPU 上快速原型验证、易用性比性能更重要时。

如果关心训练吞吐量，从 Data Parallel 切换到 DistributedDataParallel 只需几行代码，即使在单节点多 GPU 上也可以；每个进程持有自己的模型副本和数据分区。

在底层，DDP 使用异步 NCCL all-reduce，将梯度通信与计算重叠，并完全避免 Python GIL 争用。这带来更高 GPU 利用率和更快端到端迭代时间。

4.6 NCCL communicator 生命周期与环境配置陷阱

虽然 NCCL 抽象了大多数底层细节，但使用方式仍会影响性能。NCCL 的环境变量若配置错误，可能降低性能甚至导致挂起。

本节介绍与 NCCL communicator 和环境设置有关的常见陷阱，并说明如何诊断和避免这些问题。

4.6.1 陷阱 1：过于频繁地创建 NCCL communicator

NCCL communicator 代表一组能够进行集体通信的 GPU 或 rank。无论是 C++ 的 ncclCommInitRank，还是 PyTorch 的 torch.distributed.init_process_group，创建 communicator 都是昂贵操作；初始化器还会交换唯一 ID、网络地址，建立 ring/tree 并分配缓冲区。

如果代码反复初始化 NCCL communicator，每次都会付出很大代价。32 块 GPU 若为每个 rank 创建一个 communicator，可能需要 2–3 分钟而不是 2–3 秒；由于全互连握手和协调，扩展性可能比线性更差。

在 PyTorch DDP 中，只需在程序开始时调用一次 init_process_group，DDP 就会为所有进程创建一个 communicator，并在每次迭代的集体操作中复用。

下面的 PyTorch 示例说明每次迭代创建并销毁 NCCL process group 的代价。

```
import torch
import torch.distributed as dist
import torch.multiprocessing as mp
def run(rank, world_size):
    rank = int(os.environ.get("LOCAL_RANK", rank))
    device = torch.cuda.device(rank)
    for i in range(5):  # simulate 5 iterations
        // This naive approach re-initializes NCCL each
        // iteration. THIS IS EXTREMELY SLOW AND NOT RECOMMENDED!!!
        dist.init_process_group("nccl", init_method="env://",
                                world_size=world_size, rank=rank)
        # do a tiny all-reduce to simulate some work
```

```
        tensor = torch.ones(1).cuda(rank)
        dist.all_reduce(tensor)
        if rank == 0:
            print(f"Iter {i} done")
        dist.destroy_process_group()
```

运行这段代码时，即使 all-reduce 很简单，也会极慢，因为大部分时间花在 init_process_group 和 destroy_process_group 上。实际中 rank 越多，代价越大。

init_process_group 设计为在启动时调用一次，因此应在循环外初始化，如下所示：

```
import torch
import torch.distributed as dist
import torch.multiprocessing as mp
import os
def run(rank, world_size):
    # Pin GPU
    rank = int(os.environ.get("LOCAL_RANK", rank))
    device = torch.cuda.device(rank)
    # Initialize NCCL communicator once
    dist.init_process_group(
        backend="nccl",
        init_method="tcp://127.0.0.1:45678",
        world_size=world_size,
        rank=rank
    )
    # Simulate 5 training iterations
    for i in range(5):
        tensor = torch.ones(1, device=rank)
        dist.all_reduce(tensor)
    # Cleanup once at the end
    dist.destroy_process_group()
if __name__ == "__main__":
    world_size = 2
    mp.spawn(run, args=(world_size,), nprocs=world_size)
```

将 communicator 建立和销毁移到循环外，消除了每次迭代 48 ms 的初始化开销，使总迭代时间降低超过 98%，如表 4-3 所示。

表 4-3　避免重复初始化/销毁 communicator 对每次迭代时间的影响。

指标	之前（每次迭代）	之后（每次迭代）
init_process_group + destroy	48.0 ms	0 ms
dist.all_reduce（1 元素张量）	0.5 ms	0.5 ms
总迭代时间	48.5 ms	0.5 ms

all-reduce 本身仍耗时 0.5 ms，但以前完全被频繁初始化开销掩盖。真实多节点场景中，rank 越多，节省越多。只初始化一次是性能最佳实践。

4.6.2 陷阱 2：不要在每次迭代中创建和销毁 NCCL communicator

定义模型并行和流水线并行的进程子组时，不要意外创建新 communicator。应在开始时使用 torch.distributed.new_group() 创建子 communicator 并复用，绝不要每次迭代创建和销毁。

若确实需要多个 communicator，可使用 ncclGroupStart()、ncclCommInitRank(...) 和 ncclGroupEnd() 一次初始化多个 communicator，以降低开销。

> 注　截至本文撰写时，PyTorch 不支持运行时动态改变成员而不进行完整 communicator 拆除。所有 rank 必须锁步调用创建和销毁，否则会挂起。

4.6.3 陷阱 3：避免使用环境变量过度调优或禁用 NCCL 功能

NCCL 有许多环境变量，如 NCCL_BUFFSIZE、NCCL_NSOCKS_PERTHREAD、NCCL_P2P_LEVEL、NCCL_SHM_DISABLE 等。除非有特定理由，通常应保留默认值；也可显式设置为当前默认值，并查看发行说明。

> 注　增大 NCCL_BUFFSIZE 可提高大型 all-reduce 带宽，但过高会造成 GPU 内存压力或驱逐工作集。应从 4 MB 开始逐步增大并监控 GPU 内存。

常见错误是在调试时禁用功能，却忘记生产环境重新启用。生产环境绝不要让 P2P 或共享内存传输保持关闭。

NCCL_P2P_DISABLE=1 只适合故障排查；若一直启用，会迫使节点内流量经过 CPU 主机转发缓冲区而不是 GPU 直接 NVLink，增加跳数和 CPU 工作，使延迟从几微秒增至几十微秒，并将带宽从数百 GB/s 降至几十 GB/s。

> **注** 调试结束后使用 NCCL_P2P_DISABLE=0 重新启用 P2P，或删除该环境变量。

如果 NCCL_SHM_DISABLE=1，NCCL 不会使用共享内存进行节点内通信，会回退到网络或主机中介拷贝，增加驱动开销和上下文切换，进一步增加延迟并限制吞吐量。

> **注** 只应在调试期间短暂改变性能关键环境变量，恢复运行前要改回生产设置。

NCCL_DEBUG=INFO 或 DEBUG 有助于发现回退到以太网等问题，但额外日志有开销。生产环境不要运行 DEBUG；可降到 WARN（默认）或 VERSION，但后者会使后续调试困难。

NCCL_NSOCKS_PERTHREAD 和 NCCL_SOCKET_NTHREADS：多 NIC 或高带宽时增大可能有帮助。例如两块 NIC 可设置 NCCL_NSOCKS_PERTHREAD=2；关键是两者乘积按 NVIDIA 指导不得超过 64。

> **注** 64 是每个 communicator 允许的 TCP socket 连接上限，定义为 NCCL_SOCKET_NTHREADS × NCCL_NSOCKS_PERTHREAD，用于限制 CPU/网络资源并避免超过系统和硬件限制。

NCCL_MIN_NCHANNELS 和 NCCL_MAX_NCHANNELS 控制子环数量。建议保留默认值；NCCL 会在 NVSwitch 系统中按拓扑和消息大小自动调优。每个通道对应一个通信 CUDA block，通道越多需要的 GPU 资源越多。

NCCL_TOPO_FILE 可设置系统拓扑文件，帮助复杂网络或云环境中的 NCCL 做出决策。NCCL_TOPO_DUMP_FILE 可用于导出运行时检测到的拓扑。

NCCL_MNNVL_ENABLE 启用 NVIDIA 多节点 NVLink（MNNVL），用于支持多节点 NVLink Switch 的系统，例如 GB200/GB300 NVL72。

NCCL_SHARP_DISABLE 控制网络内聚合中 SHARP 的使用。默认情况下，若 SHARP 可用且配置启用，NCCL 会使用它；可设置 NCCL_SHARP_DISABLE=1 进行 A/B 测试和排障。

除非有证据表明某个参数有帮助，否则应使用环境默认值。若改变设置，务必记录并在升级硬件和 NCCL 版本时持续监控效果。

4.6.4 陷阱 4：确认 NCCL 线程的 CPU-GPU NUMA 节点亲和性

NCCL 会启动后台 CPU 线程进行网络轮询和 kernel 分发。用 torch.multiprocessing 或 MPI 启动多个进程时，每个进程都会继承 CPU 亲和性掩码，该掩码可能覆盖所有核心，也可能因 taskset 或 numactl 只覆盖一部分。

NCCL 通常将线程分配到所服务 GPU 附近的核心，但进程若固定在很窄的核心集合上，所有线程可能挤到单个核心。推荐将每个 GPU 进程绑定到其 NUMA 域的 CPU 核心，然后设置 NCCL_IGNORE_CPU_AFFINITY=1，让 NCCL 微调线程放置。到其 NUMA 域对应的 CPU 核心上，然后设置 NCCL_IGNORE_CPU_AFFINITY=1，让 NCCL 在这些核心内微调线程放置。

考虑一个具有两个 NUMA 节点域和八块 GPU 的计算节点。如果 GPU 0–3 连接到第一颗 CPU，设备 4–7 连接到第二颗 CPU，就应将 rank 0–3 绑定到第一组 CPU 核心，将 rank 4–7 绑定到第二组 CPU 核心。然后设置 NCCL_IGNORE_CPU_AFFINITY=1，忽略继承的 CPU 亲和性掩码。

> **注** 实践中，可以使用 numactl，或设置 CUDA_DEVICE_ORDER 和 CUDA_VISIBLE_DEVICES，帮助强制执行这种绑定。PyTorch 的启动工具会自动处理其中大部分工作，但仍应进行验证。

还可以指定显式拓扑文件，以进一步降低延迟、提高吞吐量。如果不想手动固定进程，可以依赖 MPI runtime 绑定，或使用 SLURM –cpu-bind 等作业调度器选项，确保每个 rank 落在正确的核心上。

4.6.5 陷阱 5：不要忽视 NCCL 警告和错误

启用适当日志后，NCCL 会输出许多日志，其中可能包含回退到较慢 PCIe 带宽等警告。这些重要警告需要处理。

如果看到“unable to enable P2P, falling back to copy”之类的日志，不要忽视它们！这通常表明条件不理想。该警告意味着 NCCL 无法在两块 GPU 之间建立直接 GPU P2P，可能是因为它们位于不同的 PCIe 根复合体且不具备相应支持。这意味着数据传输必须经过主机 CPU 内存缓冲区，速度会慢得多。

警告可能促使你重新安排进程使用哪些 GPU。应确保需要互相通信的 GPU 位于同一 NUMA 节点，或采用不同的配对方式。另一个例子是 NCCL INFO NET/Socket: using Ethernet interface eth0 警告，它会告诉你选择了哪个接口。如果这不是性能最高的互连，可能需要显式设置 NCCL_SOCKET_IFNAME，例如 NCCL_SOCKET_IFNAME=ib0，让初始化握手使用预期的互连。

4.6.6 陷阱 6：NCCL communicator 挂起、报错或完全关闭

有时，如果一个进程崩溃或某个 GPU rank 出错，NCCL communicator 可能会让其他 rank 一直挂起，因为集体操作无法完成。不幸的是，考虑到 GPU 在大规模集群中发生故障的频率相对较高，这种情况相当常见；Meta 在表 4-4 中对这一问题进行了描述。

设置 NCCL_ASYNC_ERROR_HANDLING=1 可以通过允许 NCCL 异步中止错误来提高韧性，但可能带来轻微开销。最近版本中，PyTorch 在使用 init_process_group 时会默认设置它；为清晰和可复现，显式设置仍是好主意。

> **注** 绝不要依赖默认值，始终显式设置！默认值有时会随版本变化，在初始化时显式设置可避免版本相关行为差异。

应把 NCCL 视为开箱即用的高性能引擎，但仍需注意初始化和使用方式：只初始化一次，正确绑定 CPU，需要时调节亲和性，并谨慎处理环境变量及其默认值。

表 4-4 Llama 3 405B 预训练 54 天期间意外中断的根因分类（来源：https://oreil.ly/z8QKu）。

组件	类别	中断次数	占中断比例
故障 GPU	GPU	148	30.1%
GPU HBM3 内存	GPU	72	17.2%
软件 bug	依赖项	54	12.9%
网络交换机/电缆	网络	35	8.4%
主机维护	非计划维护	32	7.6%
GPU SRAM 内存	GPU	19	4.5%
GPU 系统处理器	GPU	17	4.1%
NIC	主机	7	1.7%
NCCL watchdog 超时	未知	7	1.7%
静默数据损坏	GPU	6	1.4%
GPU 热接口和传感器	GPU	6	1.4%
SSD	主机	3	0.7%
电源	主机	3	0.7%
服务器机箱	主机	2	0.5%
I/O 扩展板	主机	2	0.5%
依赖项	依赖项	2	0.5%
CPU	主机	2	0.5%
系统内存	主机	2	0.5%

还应在升级后进行测试，因为 NCCL 的默认设置和性能可能发生变化。通常 NCCL 性能会随新版本提升；但默认值有时会改变，如果没有显式固定环境变量，就需要重新调优系统。

4.6.7 NCCL 的性能分析与调试

NCCL 支持异步错误处理和故障转移。要启用它，应设置 NCCL_ASYNC_ERROR_HANDLING=1；调试 NCCL 时还应启用 NCCL_DEBUG=WARN 或 INFO，以检查 rank 不匹配、socket 配置错误等问题。

NCCL profiler plugin API 可以监控 GPU 通信的内部时间线，定位落后设备或系统瓶颈，解决 GPU 集群扩大后越来越难诊断的性能问题。

NCCL profiler plugin 可通过 NCCL_PROFILER_PLUGIN 接口动态加载并由支持它的工具集成；该环境变量控制插件的加载和初始化。

NVIDIA 创建这一灵活 API，以简化第三方性能分析工具（例如 PyTorch Kineto）与 NCCL 的集成，并以清晰、分层、低开销方式捕获通信活动。未启用 NCCL 插件时，Kineto 也可通过 CUPTI 和 NVTX 收集 NCCL 活动。

> 注 NCCL profiler plugin 随 NVIDIA 工具以及 PyTorch/Kineto profiler 等第三方分析器提供，可用于查看所有 all-reduce 操作的时间线。

加载后，插件配置 32 位事件激活掩码，每一位对应组事件、集体事件、点对点事件和 proxy 相关操作等 NCCL 事件，形成自然层级以表示性能信息并快速定位问题。

NCCL profile plugin API 定义五个函数回调。init 设置不透明上下文并确定需分析的事件；startEvent 接收 NCCL 的事件描述符并

分配新的事件对象，返回 NCCL 用于后续操作的不透明句柄。

stopEvent 标记事件完成以便回收资源；recordEventState 允许插件在状态转换时更新事件；finalize 在性能分析结束后释放 profiler 上下文资源。

4.7 网络内 SHARP 聚合

使用支持网络内计算的高级硬件（例如 NVIDIA Scalable Hierarchical Aggregation and Reduction Protocol，即 SHARP）时，可以把部分集体操作卸载到网络本身，获得额外收益。

SHARP 是配合 Quantum 系列 InfiniBand 交换机和 NCCL-SHARP 插件使用的网络内归约技术。NVLink 域中的对应能力是 NVLink SHARP（NVLS），它将集体操作卸载到 NVSwitch 互连；在 NVL72 等现代 NVLink Switch 域中，NVLS 可加速集体操作及域内全互连和广播。

SHARP 让网络互连部分计算 all-reduce 等集体操作。交换机对流入的多 GPU 数据进行归约/聚合并共享部分结果，使 GPU 不必重复传输中间结果，从而减少大型 MPI 和 NCCL 集体操作的延迟。

对于 ring reduce-scatter，每块 GPU 通常要在 n−1 跳中接收 $B(n-1)/n$ 字节。网络内归约后，交换机只返回 B/n 字节；相对于完整 ring，每个端点的接收量降为 $1/(n-1)$。

对于 all-gather，NVLS 硬件多播让每块 GPU 只发送一次自己的 B/n 数据段，网络负责复制；相比完整 ring，发送方数据量降为 $1/(n-1)$。多播 all-gather 与网络内 reduce-scatter 重叠时，带宽受限阶段的端到端时间可能降低约一半，因为聚合和复制由网络执行；分片交换的墙钟时间取两者最大值而非总和。

简而言之，NVLS 让每个端点处理更少数据并减少串行跳数，带来更高有效带宽和更短的分片训练/推理停顿。

> **注** all-gather 没有算术归约，因此 NVLS 主要通过多播复制提供帮助；其加速取决于拓扑和消息大小，通常小于 all-reduce 和 reduce-scatter 的收益。

NCCL 可通过 NCCL RDMA SHARP 插件将 all-reduce 卸载到启用 SHARP 的 InfiniBand 互连；交换机需有 SHARP 固件，管理服务器上运行 SHARP Aggregation Manager 和 Subnet Manager，主机还需加载 GPUDirect RDMA 内核模块。配置后，符合条件的集体操作可卸载到 SHARP；NVIDIA 在某些大规模 AI 系统中报告 all-reduce 可加速 2× 到 5×。

GPU 和计算节点很多时收益更明显，因为网络通常是瓶颈。小集群可能不明显，但 32 节点时 SHARP 可通过减少通信步骤显著降低集体延迟。

> **注** SHARP 默认未启用，需通过插件选择或策略配置。可用 NCCL_SHARP_DISABLE=1 做 A/B 测试；通常建议保持启用。

使用 SHARP 通常不需改代码。查看 NCCL_DEBUG=INFO 日志可确认是否使用 SHARP，也可用 ibv_devinfo 检查设备支持。

总之，SHARP 把归约计算移入网络，进一步模糊计算与通信的边界。高端 InfiniBand 集群值得检查 SHARP 是否启用；它能像“涡轮按钮”一样改善大规模 all-reduce 的扩展效率，并补充 NCCL 的 GPU 中心优化。

截至本文撰写时，SHARP 主要是一项 InfiniBand 技术。Spectrum-X 以太网虽通过拥塞控制和自适应路由改善 all-reduce，但仍未提供类似 SHARP 的交换机驻留归约引擎；公开资料强调端到端遥测和拥塞控制，而非交换机内归约。

SHARP 有时会增加交换机内存开销。交换机归约缓冲区有限，超大集体操作超过硬件限制时可能回退到常规方法。

> **注** 建议持续监控 NCCL 日志，并在因内存压力回退到非 SHARP 聚合时设置告警。

4.8 持久化 NCCL 用户缓冲区与零拷贝注册

NCCL 支持用户缓冲区注册，使集体操作直接在张量缓冲区上运行，无需内部转发，从而减少拷贝和内部通道压力。

> **注** 持久化用户缓冲区对通过 SHARP 获得节点内（NVLS）和节点外（InfiniBand）最佳路径至关重要；零拷贝注册可加速集体操作并减少 SM/通道使用。

可使用 ncclComm Register() 和 ncclCommDeregister() 注册、注销持久化缓冲区。如果任一 rank 使用注册缓冲区，所有 rank 都必须使用；某些算法还要求各 rank 的缓冲区头部偏移一致。

4.9 NVIDIA 的 NIXL 与解耦式推理

NCCL 擅长训练中的多对多组通信，而大规模推理引入了新的通信需求。NVIDIA NIXL 是一个开源、高吞吐、低延迟的点对点通信库，于 2025 年初发布。

NIXL 专门加速大规模 LLM 的分布式和解耦式推理，把不同推理阶段分配给不同 worker，并以极低延迟和开销交换数据。

> 注　解耦式推理和 NIXL 是在节点集群中高效服务巨型模型的最佳实践。

NIXL 是 NVIDIA 开源 Dynamo 推理引擎的核心组件，以极低延迟和开销简化一对一及一对少数传输，例如移动解耦阶段共享的 KV 缓存；它补充主要用于多对多集体操作的 NCCL。

NIXL 提供一致异步 API，在 GPU、CPU、SSD 和共享网络存储之间移动数据，并为每个缓存块选择最快路径。NVIDIA Dynamo 的 KV Cache Manager 使用 NIXL 做出这一选择，如图 4-5 所示。

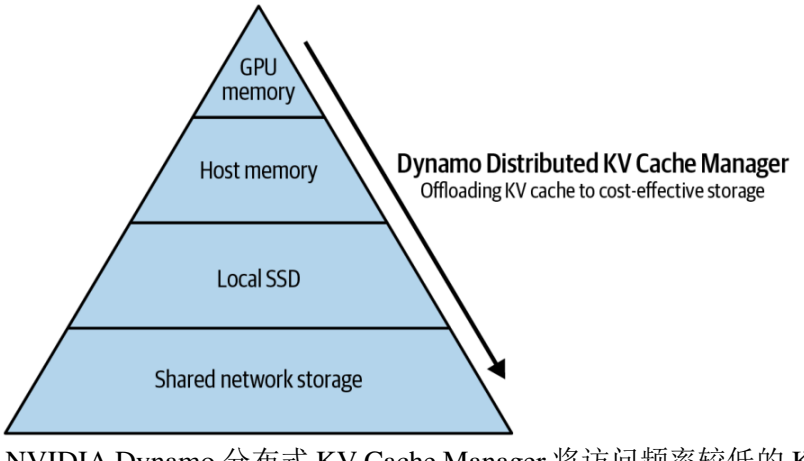

图 4-5　NVIDIA Dynamo 分布式 KV Cache Manager 将访问频率较低的 KV 缓存卸载到更经济的内存层级（来源：https://oreil.ly/nsxdl）。

扩展 LLM 推理时，应高效地在 GPU、CPU、计算节点和机架之间传输大型缓存，如 transformer 注意力 KV 缓存。使用 NIXL，推理引擎可将大型（如 100

GB）KV 缓存通过 NVLink/InfiniBand 低开销地卸载到对等端，释放 GPU 处理新请求。

NIXL 利用 GPUDirect RDMA 在节点间直接移动 GPU 数据，完全绕过主机内存；支持 RDMA 的 NIC（或 DPU）直接在 GPU 内存间传输，CPU 不在数据路径中。

NCCL 仍用于同步集体操作，NIXL 则面向 LLM 推理常见的一对一或一对多传输。NVIDIA Dynamo 广泛使用 NIXL 处理解耦推理中的预填充和解码。

NCCL 是大规模训练多对多集体操作的标准；NIXL 面向大规模推理的一对一或一对少数传输，如移动 KV 缓存。

NIXL 在 NVIDIA Dynamo 中展示了多节点 LLM 服务的吞吐量提升。NIXL 是对 NCCL 的补充，而非替代。

4.9.1 分离预填充与解码推理阶段

基于 transformer 的推理路径分为预填充（prefill）和解码（decode）两个阶段。

预填充通常计算受限，用矩阵乘法根据 prompt 构建 KV 缓存；解码通常受内存吞吐量限制，需要从 GPU HBM 读取模型权重以计算下一组 token（completion 或 response）。

vLLM、SGLang、NVIDIA Dynamo 和 TensorRT-LLM 都实现这一拆分。预填充创建 KV 缓存，解码使用它；NIXL 加速节点间 KV 缓存传输。图 4-6 对比了传统单体服务与让两个阶段运行在不同 GPU 计算节点上的解耦服务。

这里上方是传统设置：每个 GPU 节点同时处理预填充（计算受限）和解码（内存受限、I/O 受限）；下方的解耦服务把预填充 worker 和解码 worker 放在不同 GPU 集群。

预填充集群中的 GPU 为输入序列生成 KV 缓存，并用 NIXL 传输给解码集群中的 GPU。这种专门化带来更高总体吞吐量和更先进的扩展配置。

长 prompt 的 KV 缓存可能达到几十 GB，必须近实时地从一个处理单元平滑移动到另一个，才能让终端用户几乎察觉不到文本生成延迟。

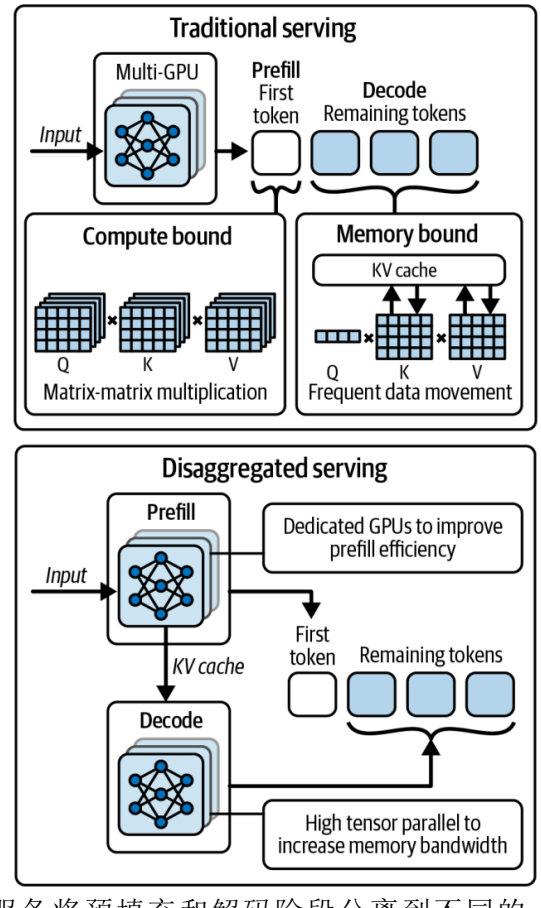

图 4-6 解耦式服务将预填充和解码阶段分离到不同的 GPU 集群（来源：https://oreil.ly/nsxdl）。

通过 CPU 内存甚至存储传递数据的传统方法，无法满足所需速度和低延迟体验。NIXL 让多节点推理能够扩展而不被互连延迟卡住。

我们需要组件之间高带宽的 GPU 到 GPU 直接传输，并让通信与计算重叠，使目标 GPU 在接收下一组输入 token 的 KV 缓存时就开始计算下一个 token。

NIXL 提供直接通道，可在计算节点甚至机架之间把数据从一块 GPU 传给另一块或少量 GPU，并始终选择最快路径。

这种智能路由类似 NCCL 的路径选择，但针对推理和一对一大消息优化。GB200/GB300 NVL72 机架中优先使用 NVSwitch，跨 NVL72 机架则按支持情况切换到 InfiniBand 或以太网 RDMA。

无论是同板卡 NVLink/NVLink-C2C、机架域 NVSwitch、机架间 InfiniBand/RoCE，还是直接 NVMe 访问，NIXL 都会自动选择最快通道。

4.9.2 KV 缓存传输的智能互连路由

经 CPU 在 GPU 间传输太慢；要求源、目标 GPU 位于同一节点又限制扩展。NIXL 因此支持必要时跨 GPU、计算节点和机架直接传输 KV 缓存等大负载。

NIXL 以高带宽运行并尽可能重叠通信与计算，使目标 GPU 接收 KV 缓存时就能生成下一组 token。

NIXL 与互连类型无关：节点内使用 NVLink/NVSwitch，跨节点使用带 RDMA 的 InfiniBand 或以太网，必要时使用 PCIe/NVMe，并统一支持 GPU HBM、CPU DRAM 和 NVMe SSD 等内存层。

4.10 带回调的 NIXL 异步 API

从开发者角度看，NIXL 提供直观 API。提交传输请求时提供数据指针和目标（GPU、CPU 或 Amazon S3 等存储），NIXL 会尽可能快地传输。

请求可以把 KV 缓存片段发送到另一块 GPU、CPU 主机内存或对象存储服务，并通过同一 API 完成，如图 4-7 所示。

模块化设计让 NIXL 能采用未来传输方式，例如新协议或更快存储级内存，而无需改变用户 API；底层由合适后端协调数据移动。

NIXL 能在 GPU HBM、CPU DRAM、NVMe SSD 和对象存储间高效移动数据，提供统一 API 自动选择最快传输（如 NVLink GPU→GPU、GDS GPU→NVMe 或 NVSwitch），获得接近链路速率的 KV 缓存卸载。

在底层，NIXL 隐藏了复杂性：使用 registerMem 注册内存，trim 获取传输描述符，prepXfer 准备非阻塞请求，postXfer 提交请求；NIXL 选择 PCIe/NVLink 拷贝、RDMA 或 GPUDirect Storage 路径。

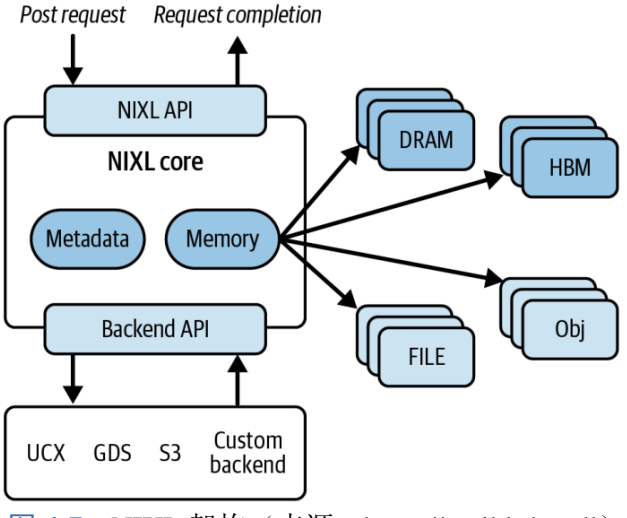

图 4-7　NIXL 架构（来源：https://oreil.ly/nsxdl）。

NIXL 是非阻塞的，会返回请求句柄；用 checkXfer 轮询即可检测完成。这种模式以极低 CPU 开销重叠通信与计算，目标 GPU 可在收到 KV 缓存块后立即消费，即使其他块仍在途。

nixlAgent 是核心传输对象，封装端点配置、内存注册、后端选择，以及元数据、连接信息和异步请求生命周期。

一次传输需要两个 agent；源 agent（agentSrc）封装源端上下文、注册和后端，目标 agent（agentDst）负责接收端。

agentSrc 和 agentDst 各代表一个端点，NIXL 在两者之间协商最佳路径并管理独立资源与请求生命周期。下面代码展示源 GPU 到目标 GPU 的传输：

```
// NIXL 0.5.x style example: nonblocking VRAM->VRAM transfer between
 ↪ two agents
#include <nixl.h>
#include <nixl_types.h>
#include <cuda_runtime.h>
#include <iostream>
#include <thread>
#include <vector>
#include <cassert>
#include <cstdint>
int main() {
    // 1) Configure agents. Prefer UCX for GPU<->GPU,
    // allow GDS if you later target storage.
    nixl_agent_config cfg{};
    cfg.backends = {"UCX"};  // {"UCX","GDS"} if you also plan storage
     ↪ transfers
    cfg.thread_safe = true;  // thread-safe mode added in early 0.2.x
    // 2) Create source and destination agents
    nixlAgent agentSrc("srcAgent", cfg);
    nixlAgent agentDst("dstAgent", cfg);
    // 3) Allocate simple test buffers on the same GPU for illustration
    int deviceId = 0;
    cudaSetDevice(deviceId);
    const size_t bytes = 1 << 20; // 1 MiB
    void* d_src = nullptr;
    void* d_dst = nullptr;
    cudaMalloc(&d_src, bytes);
    cudaMalloc(&d_dst, bytes);
    // 4) Build registration descriptors for VRAM
    //    Each descriptor uses an address, length, and associated device
    nixl_desc_t srcDesc{};
    srcDesc.addr      = reinterpret_cast<uintptr_t>(d_src);
    srcDesc.len       = bytes;
    srcDesc.devId     = deviceId;
    srcDesc.seg       = VRAM_SEG;
    nixl_desc_t dstDesc{};
```

```
dstDesc.addr      = reinterpret_cast<uintptr_t>(d_dst);
dstDesc.len       = bytes;
dstDesc.devId     = deviceId;
dstDesc.seg       = VRAM_SEG;
std::vector<nixl_desc_t> srcList{srcDesc};
std::vector<nixl_desc_t> dstList{dstDesc};
// 5) Register memory with each agent and trim to xfer descriptors
auto srcRegs = agentSrc.registerMem(srcList);
auto dstRegs = agentDst.registerMem(dstList);
auto srcXfer = srcRegs.trim();  // metadata-free descriptors used
 ↪  for xfer
auto dstXfer = dstRegs.trim();
// 6) Prepare a WRITE from srcAgent->dstAgent, then post it
 ↪  (nonblocking)
nixlReqH reqHandle = nullptr;
// prepare + post
if (agentSrc.prepXfer(NIXL_WRITE, srcXfer, dstXfer, "dstAgent",
 ↪  reqHandle)
  != NIXL_SUCCESS) {
    std::cerr << "prepXfer failed\n";
    return 1;
}
if (agentSrc.postXfer(NIXL_WRITE, srcXfer, dstXfer, "dstAgent",
 ↪  reqHandle)
  != NIXL_SUCCESS) {
    std::cerr << "postXfer failed\n";
    return 1;
}
std::cout << "Transfer posted — doing other work...\n";
// 7) Poll for completion (replaces deprecated getNotifs/poll map)
nixl_status_t st;
do {
    st = agentSrc.checkXfer(reqHandle);
    if (st == NIXL_INPROGRESS) std::this_thread::yield();
} while (st == NIXL_INPROGRESS);
if (st != NIXL_SUCCESS) {
```

```
        std::cerr << "Transfer completed with error: " << st << "\n";
        agentSrc.releaseReqH(reqHandle);
        agentSrc.deregisterMem(srcRegs);
        agentDst.deregisterMem(dstRegs);
        cudaFree(d_src);
        cudaFree(d_dst);
        return 1;
    }
    std::cout << "Transfer completed!\n";
    // 8) Cleanup
    agentSrc.releaseReqH(reqHandle);
    agentSrc.deregisterMem(srcRegs);
    agentDst.deregisterMem(dstRegs);
    cudaFree(d_src);
    cudaFree(d_dst);
    return 0;
}
```

代码中，NIXL agent 以名称和配置初始化；registerMem 注册内存，trim 生成传输描述符，prepXfer 准备非阻塞写入，postXfer 提交。程序在传输进行时继续工作，通过 checkXfer 轮询句柄；成功后用 releaseReqH 释放句柄并注销注册。

NIXL 内部使用 UCX，为 InfiniBand、TCP、共享内存等互连提供统一底层传输；它还使用 GPUDirect RDMA 和 IBGDA，让 GPU 无需 CPU 介入发起传输，进一步降低延迟。

NIXL 会避免不必要的拷贝和转发缓冲区：可分页 CPU 内存会被固定，GPU 内存则直接发送。

4.10.1 使用 NIXL 卸载 KV 缓存

若 GPU 无法容纳长序列或多轮对话的完整 KV 缓存，NIXL 可让 NVIDIA Dynamo 等推理服务器将缓存卸载到 CPU 内存甚至 NVMe SSD，并按需取回。

NIXL 与 Dynamo 的 KV Cache Manager 可高效管理这种层级。Grace Hopper 和 Grace Blackwell Superchip 通过高速 NVLink 共享大量统一 CPU/GPU 内存，如图 4-8 所示。

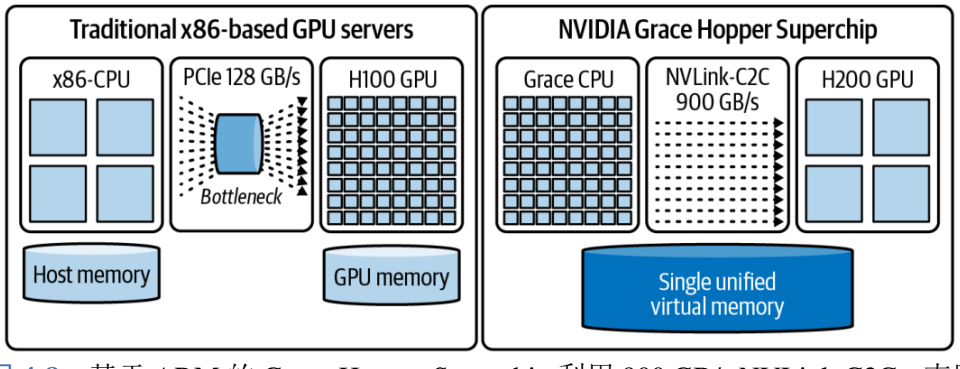

图 4-8　基于 ARM 的 Grace Hopper Superchip 利用 900 GB/s NVLink-C2C，克服传统 PCIe 瓶颈（来源：https://oreil.ly/zf6rF）。

推理服务器可把大型 KV 缓存卸载到大容量 CPU 内存，释放 GPU HBM；基于 PCIe 的 x86+H100 系统在长输入上相较重新计算缓存，TTFT 延迟最高可改善 14 倍，如图 4-9。

ARM 架构 Grace Hopper Superchip 通过 900 GB/s NVLink-C2C，相较非 superchip 的 x86 H100 版本，TTFT 延迟快 2 倍，如图 4-10。

这些收益来自 NIXL 软件与 NVIDIA superchip 硬件的协同设计。NIXL 让 KV 缓存卸载保持低传输成本，成为大规模推理尤其是内存容量受限 LLM 的关键方案。

随着模型变大和工作负载变复杂，使用 NIXL 异步移动大块数据十分关键。若数据在阶段（如流水线并行）及 GPU、存储等组件间移动，应评估 NCCL 是否足够，或 NIXL 是否更适合优化数据流。

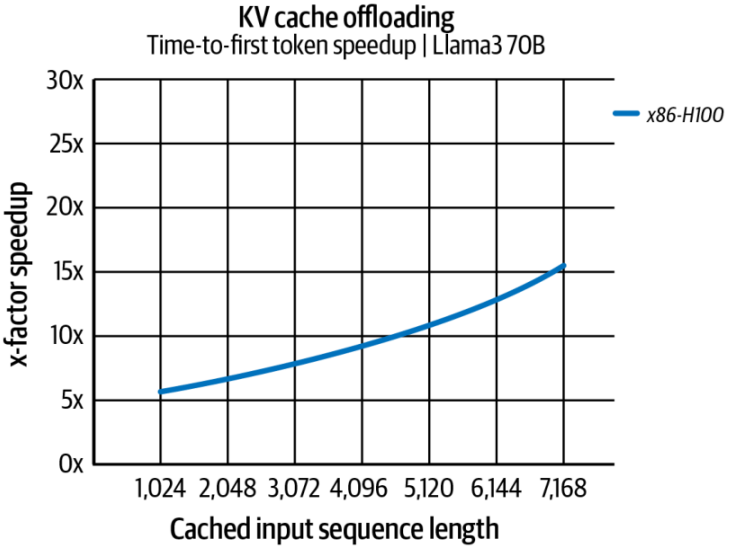

图 4-9　基于 x86 的 NVIDIA H100 GPU 系统在大输入序列上通过 KV 缓存卸载实现 14 倍 TTFT 加速（相较从头重算；来源：https://oreil.ly/zf6rF）。

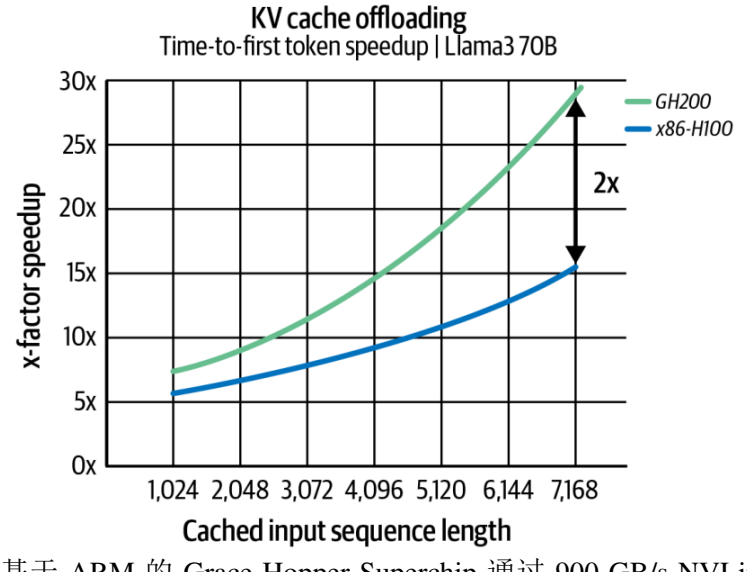

图 4-10　基于 ARM 的 Grace Hopper Superchip 通过 900 GB/s NVLink-C2C 和 KV 缓存卸载，相较基于 x86 的 H100 GPU 系统实现 2 倍 TTFT 加速。

4.11 NIXL 与 NVIDIA Dynamo 等高性能推理系统

NIXL 对 NVIDIA Dynamo 等分布式推理系统影响巨大。NVIDIA 内部测试显示，在 72-GPU Blackwell NVL72 机架上，开源 Dynamo 使用 NIXL 可使约 680B 参数 LLM 的推理吞吐量最高提高 30 倍。

曾经严重限制性能的节点间数 GB 上下文移动，在 NIXL 下变成相对快速的异步操作。后续章节将深入讨论 Dynamo、TensorRT、vLLM 及模型推理优化。

4.12 NCCL 与 NIXL 对比

NIXL 不是 NCCL 的替代品，而是补充。NCCL 负责同一任务/阶段中并行 GPU 的同步集体操作，例如多 GPU all-reduce；NIXL 执行任务/阶段之间或 GPU、CPU、存储等组件之间的异步数据传输。表 4-5 对二者进行了比较。

表 4-5　NCCL 与 NIXL 的比较。

方面	NCCL（集体通信）	NIXL（点对点通信）
主要用例	面向紧密耦合 GPU 组训练的多对多集体操作（如 all-reduce、all-gather）。	面向分布式推理或流水线的一对一或一对少数传输（如大型张量或缓存）。
通信模式	同步集体操作——所有参与者必须到达调用点。	异步发送/接收——一个发起方、一个或多个目标方。
与计算重叠	可通过独立 CUDA stream 部分重叠。	为最大化重叠设计，传输完全并行并轮询完成通知。
拓扑感知	自动检测拓扑并优化 ring/tree、NVLink/NVSwitch。	与互连无关，按源/目标位置使用 NVLink、NVSwitch、PCIe、InfiniBand/RDMA 或 GDS。
数据范围	需在所有 GPU 间聚合的小到中等张量，如梯度。	大型数据块的快速点对点传输，如 LLM KV 缓存或模型分片。
集成	集成于训练框架（PyTorch DDP、Horovod 等调用 NCCL）。	NVIDIA Dynamo 使用的开源库，开发者按需调用 NIXL API。
示例	在 8 块 GPU 间并行 all-reduce 100 MB 梯度。	推理流水线中将 1 GB KV 缓存从 GPU 0 发送到 GPU 1、CPU 内存或 NVMe SSD。

NCCL 虽支持点对点 send()/recv()，但更适合同步训练中的集体操作。NIXL 面向大规模推理和流水线并行的异步点对点传输；例如 NVIDIA Dynamo 用它在 GPU、CPU 和 SSD 等组件间编排 KV 缓存移动。

总之，NCCL 最大化单主机内和跨节点多 GPU 的集体吞吐量，自动选择感知拓扑的 ring/tree 算法并充分占用 PCIe、NVLink、InfiniBand 和以太网链路。NIXL 建立在这些高性能原则上，组织跨 GPU、CPU 和存储设备的异步、硬件无关点对点传输，面向大规模分布式推理及超高速 KV 缓存移动。

4.13 关键要点

通过谨慎工程和本章技术，通常可以达到或接近硬件物理“光速”上限。调优网络层时请记住以下经验：

4.13.1 拓扑很重要

节点间 InfiniBand 和节点内 NVLink/NVSwitch 会影响最佳策略。多节点、多 GPU 配置应考虑分层方案，始终使用最快互连，避免因配置错误或默认值走慢路径；可用时利用 SHARP 进行大规模网络内聚合/归约。

4.13.2 调优环境和系统

单个环境变量或 OS 设置有时就能提高吞吐量，例如增加 NIC 缓冲区、调节 NCCL 功能和日志、正确绑定 CPU；IRQ 亲和性等操作系统和驱动优化也有助于消除瓶颈。

4.13.3 利用最新硬件创新

Grace Hopper、Grace Blackwell Superchip 提供巨大 CPU 内存和高速 CPU-GPU 互连，可承载数据集、分区数据和模型、卸载 KV 缓存。SHARP 等网络内计算可将集体操作加速 2×–5%，尤其在大规模场景中。

目标是让 GPU 100% 时间计算，同时后台通信；让网络链路充满有效数据，让磁盘全速流式传输，所有工作和谐并行。

这需要反复调优、验证，并权衡更多内存和更高代码复杂度。但调优会带来更快的训练和推理，以及对昂贵基础设施更充分的利用。

4.14 结论

高性能、分布式、多 GPU 通信与存储系统的发展构成调优大型复杂 AI 系统的基础。使用 NCCL、NIXL 和 RDMA 等专用库，可显著减少瓶颈并提高性能。

集成 NVSwitch 和支持 SHARP 的 InfiniBand 交换机等智能硬件会直接提升训练和推理性能。保持 CUDA、PyTorch 更新也很关键；NVIDIA Dynamo、vLLM 等服务框架可简化推理部署。

没有单一组件能独自提供峰值性能；只有协调高速通信、高效数据处理和全系统调优并进行协同设计，才能实现可扩展、稳健的 AI 系统。

对性能工程师而言，快速移动数据与原始计算能力同样重要。最快的 GPU 若一直等待来自 CPU 或另一块 GPU 的数据，也几乎没有价值。

下一章将探讨 GPU 存储策略和优化。与 RDMA、NCCL、NIXL 等网络协议和库相辅相成，GDS 和高效输入流水线也是让 GPU 持续获得工作的整体方法的一部分。

第 5 章 基于 GPU 的存储 I/O 优化

对 AI 工作负载来说，向 GPU 提供数据与执行计算同等重要。设想一个拥有 100 万亿参数、在数千块 GPU 上训练的模型。它可能要处理数十亿个训练样本，包括 token、图像、音频和视频等。

这意味着必须从存储中读取海量数据并尽快送入 GPU。存储流水线一旦变慢，GPU 就会因缺少数据而空闲；即使采用复杂的通信优化，利用率仍会很低。

本章讨论存储和输入流水线优化：如何高效地从磁盘或远程存储读取数据、如何预处理数据，以及如何让 I/O 与 GPU 计算重叠。

5.1 快速存储与数据局部性

大型模型训练通常需要读取巨量数据集。大型语言模型的训练样本常达数十亿甚至数万亿，对应 TB 级文本数据；视觉模型则可能需要 PB 级图像数据。

在超大规模场景中，存储系统必须持续提供巨大吞吐量，以跟上可能运行数月的数千乃至数百万块 GPU。将 NVMe SSD 放在同一机架，或采用机架本地交换机拓扑的 NVMe over Fabrics（NVMe-oF），可减少网络跳数并提高性能一致性。

如果数据位于 NFS 服务器等网络附加存储，或 Amazon S3 等云对象存储中，必须确保所有计算节点的聚合读取带宽足够。若每块 GPU 需要 200 MB/s 的训练数据，8 块 GPU 就需要约 1.6 GB/s；Blackwell、Rubin 等现代高端 GPU 需要更多带宽才能保持满载。

> **注** NVIDIA Grace Blackwell GB200/GB300 NVL72 机架包含通过一个 NVLink 域互联的 72 块 Blackwell GPU。若每块 GPU 需要 200 MB/s，则让全部 GPU 保持忙碌可能需要 14–20 GB/s 的聚合存储吞吐量。超大规模工作负载的存储方案必须相应扩展。

> **注** 如果工作负载传输更重的媒体或多模态样本，应根据实测的每样本字节数和每秒样本数校准；聚合需求可能更高。

一种方案是使用同机架 NVMe SSD 或 NVMe-oF 网络拓扑等更快的本地存储；另一种方案是使用 Lustre、通用并行文件系统（GPFS）等并行文件系统，把数据缓存到本地 SSD。只要存储系统跟得上，就应配置多个数据加载线程以保持流水线饱和，并注意 Python GIL。

只要可能，就应让数据物理上尽量靠近计算节点。可以将数据放在同一物理节点的本地 NVMe SSD 上，或至少放在同一机架内，通过 NVMe over Fabric（NVMe-oF）或高级存储加速器访问。

分布式多节点训练通常会跨节点对数据集分片，使每个节点主要从本地磁盘读取自己的子集。例如，100 TB 数据和 10 个节点可预先为每个节点分配 10 TB；各节点的数据加载器只读取本地数据，避免重复读取压满网络，尤其是在数据集无法装入 RAM 时。

PyTorch 的 DistributedSampler 等框架会协调 worker，使每个进程在每个 epoch 获得唯一数据切片，这与跨集群节点分片数据的目标一致。

5.2 顺序读取与随机读取模式

GPU 处理数据很快，但更适合读取大块连续数据。存储设备在大块顺序读取时的吞吐量也远高于小块随机读取，因此准备数据集和存储布局时应尽量安排顺序访问。

图像训练时应避免存储数百万个独立图像文件，因为这会导致磁盘各处大量随机寻道。更好的做法是使用少量大型二进制文件（Arrow、TFRecord 或 Parquet）、数据库文件、WebDataset tar 文件或等价格式，让每个文件包含许多串联样本。

> **注** 在更快 GPU 的今天，把小文件合并成大型分片更加重要，因为过多的小块随机读取会更快成为瓶颈。即使现代并行文件系统和对象存储能处理一定程度的小块随机读取，也应明确验证性能。

从大型文件中读取一个数据块，一次就能得到许多样本。使用 Amazon S3 等对象存储时，也通常会提前把小对象合并成大对象。

还应调节读取大小：由于每次读取的开销更低，以 1 MB 块读取的吞吐量会优于 4 KB 块。许多数据加载库允许调节缓冲区和预取块大小。Python 的 open() 使用操作系统预读缓冲区加速顺序扫描，但随机读取从大缓冲区或缓冲 I/O 库中获益不大。

应将读取批处理成更大的连续块，或使用高级数据集 API，例如 TFRecord-Dataset，或带可调预取大小的 PyTorch IterableDataset 和 DataLoader。即使框架内部已优化大块顺序读取，也需要调节缓冲和预取参数。

如果访问仍必须随机，可通过调用 pread() 的线程或 Linux 异步 I/O 接口 io_uring 并行发起读取。预注册缓冲区和轮询使 io_uring 能以很小的内核开销提交批量 I/O 请求，降低每次系统调用的开销，提高随机读取吞吐量，隐藏延迟并实现高 IOPS。

应使用针对大型并发 I/O 优化的文件系统。XFS 在 Linux NVMe 服务器上很常见，应使用 noatime 挂载以消除每次读取更新访问时间的开销。对 Amazon EFS 等网络存储服务，应使用 Max I/O 性能模式；需要稳定带宽时，可从默认的 Bursting 吞吐模式切换到 Provisioned 吞吐模式。这些设置能确保 I/O 层跟上大规模并行 AI 工作负载。

5.3 调优 NVMe 和文件系统以提升吞吐量

现代 Linux 使用多队列块 I/O 调度器 blk-mq，将 I/O 分散到 CPU 核心。对于高速 NVMe SSD，可能需要调节队列深度和提交队列数量。通常默认设置已经足够；但若已知工作负载以顺序访问为主，可以使用 none I/O 调度器。

传统完全公平排队（CFQ）调度器已经过时。现代内核默认对 NVMe 使用 none 或 mq-deadline 多队列调度器，可通过 /sys/block/<device>/queue/scheduler 检查。none 适用于低延迟工作负载，某些设备还可能使用预算公平排队（BFQ）。

> **注** 高性能 NVMe 仍建议使用 none 或 mq-deadline，以最大化吞吐量。可通过 /sys/block/nvme*/queue/scheduler 验证和设置；虽然通常开箱即用已正确配置，仍值得快速检查。

另一个调优方面是预读。内核检测到顺序读取时会自动预读额外数据，可在 /sys/block/<device>/queue/read_ahead_kb 查看设置。默认值可能为 128 KB；流式读取大文件时可提高到几 MB，以减少系统调用开销并流水化读取，使用 blockdev –setra 即可设置。

使用 NVMe SSD 时，应确保其连接在系统可用的最快接口上，并有足够 PCIe 等通道，避免形成瓶颈。有时可用 RAID 0 对多块 SSD 做条带化，以充分利用设备，尤其是单块磁盘无法让 GPU 饱和时。

Linux 页面缓存会把最近读取的数据自动缓存到 RAM。大型数据集可能超出可用 RAM 并导致缓存抖动，但中等规模数据集的温热缓存可以显著加快训练。

如果数据或其中大部分可以装入 RAM（例如 Grace Blackwell Superchip 上的 CPU + GPU 统一内存），应在启动时完整预加载，形成 GPU 的超高速内存缓存并减少训练期间的磁盘 I/O。PB 级数据集通常无法这样做，此时应使用优化 I/O 流式传输数据。

数据加载务必使用多个 worker，例如 PyTorch 的 DataLoader(num_workers=N)。独立的 CPU 线程或进程会并行获取和预处理数据，为多块 GPU 提供输入；合适的 worker 数量需要实测。

第 13 和 14 章将深入讨论 PyTorch 性能调优；此处值得注意的是，应启用 pin_memory=True 并使用 non_blocking=True，让主机到设备复制重叠；设置 persistent_workers=True 可避免跨 epoch 重新创建 worker 的开销。还应按工作负载调节 prefetch_factor；num_workers 大于 0 时，其默认值为 2。

worker 太少会让 GPU 空闲，太多则会争抢 CPU 核心和 I/O 带宽。应监控 CPU 使用率和磁盘吞吐量，理想情况下接近 100% 利用磁盘吞吐量，同时为 CPU 留出余量。

> **注** 对于 72 核 NVIDIA Grace CPU 这类高核心数 CPU，通常可以使用更多数据加载 worker，但要注意过度 I/O 争用带来的收益递减。

5.4 使用 NVIDIA GDS

GDS 允许 GPU 直接从存储设备或网络存储栈读取数据，不在 CPU 内存中创建额外副本。通常从 NVMe SSD 读取时，数据先进入 CPU 内存，再由 CUDA 调用复制到 GPU 内存。

> **注** GDS 与 GPUDirect RDMA 互补：GDS 加速存储到 GPU 的 DMA，GPUDirect RDMA 加速网络到 GPU 的 DMA。二者都不会消除 CPU 编排，但都会移除主机内存中转缓冲区。

使用 GDS 时，GPU 可直接对 SSD 或 NIC 发起 DMA，将数据移入自己的 HBM，绕过 CPU 路径的额外复制。GDS 支持本地 NVMe 和通过 NVMe-oF 访问的远程存储。

GDS 建立直接 DMA 路径，绕过存储与 GPU 内存之间的主机内存中转缓冲区，因此也适用于集群文件系统和某些对象存储系统。（CPU 仍负责配置和编排 I/O。）

启用 GDS 需要现代 NVIDIA GPU、支持 DMA 的存储栈、正确的 NVIDIA 驱动和 CUDA 工具包。通常使用本地 NVMe SSD 或 RAID 卷。GDS 支持取决于文件系统和支持 RDMA 的栈；截至本文撰写时，支持的栈包括带 O_DIRECT 的 XFS/EXT4 上的本地 NVMe 和 NVMe-oF、基于 RDMA 的 NFS，以及与 nvidia-fs 集成的 BeeGFS、WekaFS、VAST、IBM Storage Scale 等并行文件系统。

应用程序需要使用正确 API。可以用 CUDA 的 cuFile 库通过 GDS 读取文件；cuFile 支持自动缓冲区对齐和与常见文件系统的集成。

如果读取路径使用 cuFileRead，数据就能直接从磁盘流入 GPU 内存，降低 CPU 使用率并可能提高吞吐量，尤其是在 CPU 成为瓶颈时。cuFileRead 与 Linux 文件系统直接集成，也可以使用 cuFileReadAsync、cuFileWriteAsync 等异步 API，将存储 I/O 接入 CUDA 流以实现重叠和流水化。

> **注** 尽可能使用 O_DIRECT，以启用直接 DMA 并绕过操作系统页面缓存。现代 GDS 中 cuFile 也能处理非 O_DIRECT 文件描述符，但未对齐可能导致额外复制或性能下降。

WekaIO、DDN、VAST、Cloudian 等厂商都发布了支持 GDS 的方案或插件，使系统通过 RDMA 将数据直接送入 GPU 内存。这意味着 GDS 可以直接用于企业级 NAS 和并行文件系统。

VAST Data 报告称，某些 AI 工作负载使用 GDS 后读取吞吐量提升 20%。在其测试中，单块 A100 GPU 顺序读取吞吐量提高 20%，在适用时更接近每个 NIC 的 100 Gb/s 链路容量。图 5-1 展示了使用和不使用 GDS 的架构。

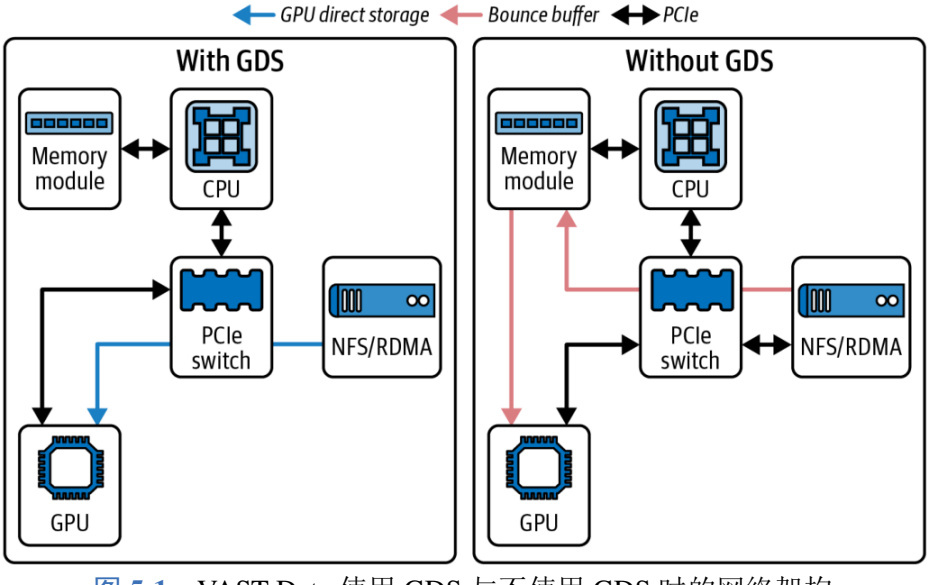

图 5-1　VAST Data 使用 GDS 与不使用 GDS 时的网络架构。

左侧是经过主机内存复制的传统分阶段 DMA；右侧是使用 GDS 的 GPU 直接拉取，绕过主机内存复制并降低 CPU 使用率。VAST 测得 NVIDIA Ampere A100 的读取吞吐量提升 20%，Hopper H100 则提升 30% 以上，原因是其 NIC 带宽更高、CPU 负担更重。

> **注**　应在自己的工作负载和互连网络上验证，因为提升幅度会随 I/O 大小、队列深度、NIC 代际和文件系统实现等因素变化。

GDS 可能需要调优，并非所有工作负载都能获得巨大提升。如果 CPU 原本就能轻松处理数据传输，吞吐量变化可能不大，但 CPU 使用率会下降，从而释放 CPU 做数据处理。若 CPU 被大量 memcpy 操作压满，GDS 会很有帮助。

使用 GDS 时必须正确应用 O_DIRECT 语义和对齐。存储到 GPU 的路径不使用主机固定内存；cuFile 注册 GPU 设备缓冲区，nvidia-fs 内核驱动编排存储设备或 RDMA NIC 与 GPU 内存之间的 DMA。它直接集成 POSIX 文件描述符，因此支持 RDMA 的网络文件系统上的普通文件也能使用 cuFile。

假设训练批很小，每个批为 1 MB，却要每秒向 GPU 提供 1,000 个批，即约 1,000 MB/s。由 CPU 复制很容易耗费几个核心；使用 GDS 时，GPU 可直接从磁盘拉取数据并释放 CPU。速率更高或 GPU 达到数千块时，差异会更加明显。

训练工作负载绝大多数是读密集型，所以 GDS 的收益通常在读取存储数据时评估。不过快速写入检查点同样重要；要实现 RDMA 加速写入，文件系统必须支持 GDS 的 RDMA 写入。

WekaFS 是超大规模 AI 训练中知名的存储提供商，提供带 GDS 感知插件的并行文件系统，支持基于 RDMA 的读取和写入。

5.5 使用 cuda-checkpoint 保存 GPU 状态

在 Linux 上，可以将 NVIDIA 的 cuda-checkpoint 与 CPU 进程检查点工具 Checkpoint/Restore in Userspace（CRIU）结合使用。cuda-checkpoint 会暂停运行进程内的 CUDA，等待已提交工作完成，将设备内存复制到驱动管理的主机分配区并释放 GPU 资源，以便 CPU 侧检查点工具为进程创建快照。

暂停路径会锁定 CUDA 驱动入口、排空未完成工作、把设备内存复制到主机并释放 GPU 资源。估算暂停时间时，应考虑正在使用的设备内存量和暂停期间可用的主机互连带宽。由于驱动在暂停阶段复制设备内存，实际暂停时间受内存映像大小和平台互连限制。应在锁定调用和检查点调用附近使用 Nsight Systems 标记，确认暂停阶段的真实耗时。

恢复进程时，驱动会重新获取 GPU，将设备内存映射回原始地址，恢复 stream、context 等 CUDA 对象，然后解锁驱动和进程，使 CUDA 调用继续。

CUDA Driver API 暴露 cuCheckpointProcessLock、cuCheckpointProcessCheckpoint、cuCheckpointProcessRestore 和 cuCheckpointProcessUnlock。恢复要求启用 persistence mode（或调用 cuInit），并可映射到相同芯片类型的不同物理 GPU。

这条路径与框架级模型检查点（例如 PyTorch checkpoint）正交。CUDA checkpoint 适合长时间训练和推理作业的容错、抢占与迁移。

与 GDS 数据摄取不同，检查点路径不会直接把 GPU 内存 DMA 到存储。暂停时，驱动先把设备内存映像搬到主机内存，再由 CRIU 将进程内存持久化到检查点映像。它应补充而不是替代框架的 state-dict 或分片检查点文件。

5.6 使用 gdsio 测量 GDS

NVIDIA 提供 gdsio 工具，默认安装在 /usr/local/cuda/gds/tools 下，用于基准测试磁盘与 GPU 之间的 GDS 吞吐量。

使用 GDS 时，吞吐量提升 10%–20% 甚至更高并不罕见，特别是在 CPU 受限场景。下面用 NVIDIA gdsio 比较纯 CPU 中介读取（“之前”）和直接 GDS 读取（“之后”），并查看命令行及吞吐量、延迟结果。

```
# Before (Storage → CPU Memory only)
# CPU path, host memory, async copies (-x 2)
$ /usr/local/cuda/gds/tools/gdsio \
    -f /mnt/data/large_file \
    -d 0 -w 4 -s 10G -i 1M -I 0 -x 2


Total Throughput: 8.0 GB/s
Average Latency: 1.25 ms
```

第一次调用使用带固定主机内存和异步复制（-x 2）的 CPU 路径，在读取模式（-I 0）下建立基线。下面的第二次调用在相同配置下启用 GDS 路径（-x 0），仍为读取模式（-I 0）。比较路径时要保持传输选择器一致；gdsio 中 -x 2 表示 CPU 中介传输，-x 0 表示 GDS 路径。

```
# After (Storage → GPU Memory using GPUDirect Storage)
#   - same config, GDS path (-x 0)
$ /usr/local/cuda/gds/tools/gdsio \
    -f /mnt/data/large_file \
    -d 0 -w 4 -s 10G -i 1M -I 0 -x 0


Total Throughput: 9.6 GB/s
Average Latency: 1.00 ms
```

使用 GDS 创建从磁盘到 GPU 内存的直接路径后，吞吐量提高 20%，平均 I/O 延迟相应下降，如表 5-1 所示，同时释放了原本用于通过主机缓冲区移动数据的 CPU 周期。这个简单基准测试展示了如何在自己的系统中验证 GDS 的收益。

表 5-1　GDS 前后的吞吐量与延迟对比。

路径	吞吐量	延迟
存储 → CPU（无 GDS）	8.0 GB/s	1.25 ms
存储 → GPU（有 GDS）	9.6 GB/s（+20%）	1.00 ms（−20%）

在这个例子中，使用 GDS（存储 → GPU）使读取吞吐量从 8.0 GB/s 提高到 9.6 GB/s，延迟从 1.25 ms 降至 1.00 ms，即吞吐量提高、延迟降低约 20%。

5.7 DeepSeek 的 Fire-Flyer 文件系统

DeepSeek 从零创建了定制开源文件系统 Fire-Flyer File System（3FS），源于一个观察：AI 工作负载会执行海量随机读取。

随机读取会使传统读取缓存失效甚至适得其反，尤其对 LLM 训练和推理。3FS 取消缓存并采用直接文件 I/O，使每个请求直接到达 NVMe SSD，避免无谓的缓存管理；这一方法类似于强调直接存储访问的现代 HPC 文件系统，也最大限度减少读取时的内核页面缓存参与和主机内存复制。

> **注**　3FS 体现了专门为 AI 进行存储协同设计的趋势，类似于为高性能并行文件系统设计、以实现直接 GPU 吞吐量的 NVIDIA GDS。

3FS 包含四个关键组件：集群管理器、元数据服务、存储服务和客户端。它们通过支持 RDMA 的 InfiniBand 或 RoCE 等互连网络连接，以减少 CPU 参与和主机侧复制；图 5-2 展示了这些组件及其连接。

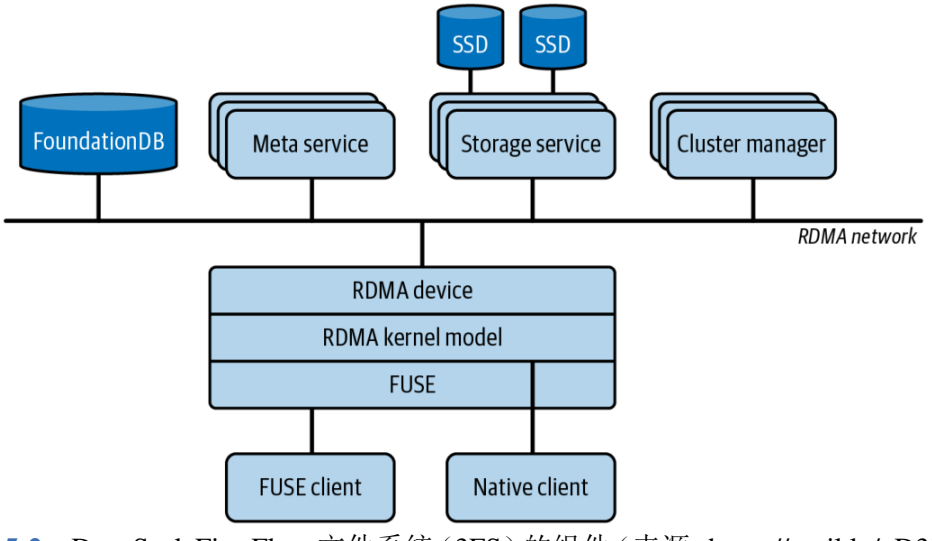

图 5-2　DeepSeek Fire-Flyer 文件系统（3FS）的组件（来源：https://oreil.ly/xD3id）

3FS 是基于 Linux 的文件系统，与现有应用兼容，同时利用 RDMA 读取实现 GPU 可直接访问的数据传输。元数据在多节点间分片和复制以实现扩展，数据路径完全绕过操作系统页面缓存以保持最佳吞吐量。

> 注　用户空间 FUSE 文件系统无法提供 GDS 路径，因为 GDS 要求内核级文件系统集成和 O_DIRECT 语义。只有支持 GDS 的内核客户端或专门集成的并行文件系统，才能把数据直接传入 GPU 内存。

DeepSeek 在 3FS 中集成基于 RDMA 的传输，让数据直接进入 GPU 流水线。需要真正 GDS 路径时，应使用支持 GDS 的内核文件系统客户端，例如 NVMe、NVMe-oF、BeeGFS、WekaFS、IBM Storage Scale 或 VAST，以极低开销异步、零拷贝地把数据移动到 GPU 设备内存。

3FS 让数据预取和传输与 GPU kernel 并发运行，补充了本章关于 I/O 与计算重叠的技术；它把第 4 章讨论的级联流水线/波次概念扩展到存储层。

DeepSeek 公开报告称，3FS 在大型集群上的聚合读取吞吐量达到每秒数 TB，某环境最高 7.3 TB/s。另一项测试中，68 节点 AI-HPC 集群配置 10 × 16 TB NVMe

SSD 和双 100 Gb/s 网络，实现约 6.6 TB/s 聚合读取，同时服务额外 1.4 TB/s 的后台工作负载；6.6 TB/s 远超相似硬件上 Ceph 约 1.1 TB/s 的吞吐量。

3FS 通过协调节点间 I/O 实现这一性能。持续带宽可防止数据暂存阶段成为瓶颈，并保持训练和推理中的高 GPU 利用率。

DeepSeek 创建了针对随机读取的自有文件系统，并将其接入 RDMA 优先的数据路径，展示了端到端全栈性能工程（包括存储设计）对发挥大规模 AI 系统全部性能潜力的重要性。

3FS 表明，重新思考存储层可以消除最后的 I/O 瓶颈。自建文件系统需要大量前期投入和持续维护，更常见的做法是从现有分布式文件系统或对象存储开始。

5.8 分布式并行文件系统与对象存储

多节点训练常用 NFS 服务器等共享文件系统，或 Lustre、GPFS、Ceph 等并行文件系统，让所有节点访问同一数据集；但配置不当时它们可能成为瓶颈。

单个 NFS 服务器虽易设置，但多节点同时读取时很容易成为吞吐量瓶颈。若必须使用 NFS，应配备多个高速 NIC，并考虑用多个 NFS 服务器拆分数据集，让每台服务器负责一个分区。

多 GPU 集群中，NFS 只适合几台节点这样的中小规模；更大训练集群中的单个 NFS 服务器即使是高端实现也很可能成为瓶颈，所以现代 AI 集群更偏好并行文件系统和 Amazon FSx for Lustre 等云存储缓存。

> **注** 对于 Amazon FSx for Lustre 等云存储缓存，重要的是验证性能提升并证明缓存的额外成本合理。如果性能未达预期，应直接与云厂商合作，验证你的架构并确认配置设置。

NFS 还有 rsize/wsize（读写请求大小）等调优参数，建议使用最大值（例如 1 MB）以提高吞吐量。确保底层 NFS 存储使用 NVMe SSD 且速度足够快，必要时采用 RAID 0 配置；同时别忘了检查 NFS 客户端挂载选项，它们也需要调优。

例如，可用 rsize=1048576,wsize=1048576,noatime,async 挂载 NFS 客户端，使用 1 MiB 块并消除访问时间更新（noatime）；还可以添加 actimeo=60,lookupcache=pos，将文件属性和目录项缓存 60 秒。这些简单调整能大幅减少单请求开销，提升大型共享数据集的并行读取吞吐量。

Amazon S3 等对象存储不是典型文件系统，但在 AI 工作负载中非常常见。训练期间朴素访问对象存储可能很慢，解决办法通常是把数据暂存到本地 NVMe SSD 存储，或在对象存储之上使用缓存层（例如 S3 之上的 Amazon FSx for Lustre）；s5cmd 和 aws s3 cp 等工具可以在训练开始前下载数据。

> **注** 应使用高并行、优化的数据传输工具，例如 AWS S3 C++ SDK，以及 s5cmd 等多线程实用程序，以获得最佳性能。

也可以使用通过范围请求从 Amazon S3 读取对象并执行缓存的流式库。直接读取 Amazon S3 时，尽量使用尽可能大的请求，并使用多线程范围 Get 操作。

Lustre、GPFS 等并行文件系统针对高并发和吞吐量设计。Lustre 由多个 Object Storage Target（OST）提供数据服务；跨 OST 对文件条带化可以倍增吞吐量，因此应确保大型数据文件跨多个 OST 条带化。

文件条带化意味着文件块位于不同服务器上，从而可以并行读取。例如，可将 Arrow、TFRecord 或 Parquet 文件跨 4 个 OST 条带化；若每个 OST 提供 500 MB/s，理论峰值读取吞吐量就是 2 GB/s。

5.9 调优、复制和压缩数据

应查阅文件系统文档进行调优。例如，Lustre 用 lfs setstripe 将大型数据集跨 4 或 8 个 OST 条带化，以聚合 OST 带宽。

训练期间可用 Lustre 的 lmt 或厂商监控工具观察文件系统 I/O，检查存储集群中是否有节点过热；若有，应查明原因，通常是分片问题导致更多读写集中在较少节点上。

在某些情况下，可将数据集复制到计算集群的每个节点，以完全消除网络读取。这要求每个节点有足够存储，虽属蛮力方案，却常见且有效，代价是额外存储空间。

另一种方案是在文件系统或对象存储中压缩数据，并即时解压，例如 JPEG 图像及压缩的 Arrow、Parquet 文本。这会节省 I/O 带宽，但消耗额外 CPU 或 GPU 周期；若瓶颈在 I/O 而 CPU、GPU 空闲，这是合理取舍。

许多数据流水线已经这样做，应验证压缩比，并确保流水线每一步都使用压缩；关键是避免解压步骤成为瓶颈。

nvJPEG 等库可在 GPU 上解码图像。现代 GPU 还带有片上解压引擎，支持 LZ4、Snappy、Deflate，以加速将数据移入 GPU 内存并解包。Blackwell GPU 可在流水线中用解压引擎解压磁盘上的压缩批次，释放 SM 去运行更有价值的计算 kernel；I/O 受限时应优先选择这些格式。

这也是把算术操作从 CPU 卸载到 GPU 的方式，并可能让数据解码与训练中的梯度计算重叠。借助带宽最高 900 GB/s 的双向 CPU 到 GPU NVLink-C2C，可避免 CPU 辅助阶段成为瓶颈。

利用软件和硬件 GPU 卸载特性，可进一步把工作负载移出 CPU，保持输入流水线平衡；但要确保解压时间没有取代 I/O 成为瓶颈，否则额外压缩计算可能不值得。

5.10 监控存储 I/O

与任何性能工程任务一样，测量是关键。与监控网络通信类似，应使用所有可用工具监控存储流水线的通信。

这些工具包括 Linux 的 iostat、iotop、nvme-cli、perf 和 eBPF。此外还可以使用厂商专用工具和仪表盘监控队列、延迟、预读效果和缓存命中率，以了解本地 NVMe 设备使用情况，并判断从 NAS 或对象存储读取数据时网络链路是否饱和。

还可以使用 Nsight Systems 等工具跟踪 I/O 等待时间，并可视化与 GPU kernel 的重叠情况。使用 Nsight Systems 的 –trace=gds 选项，可以捕获时间线上的 cuFile API 活动和跟踪；也可通过 /etc/cufile.json 启用 GDS cuFile 静态跟踪点，在 Nsight Systems 中查看 cuFile 事件。NVMe 点对点 DMA 路径的内核态计数器未在 Nsight Systems 中暴露，且并非所有 GDS 栈都可用。

另一个工具是 NVIDIA 数据中心 GPU 管理器（DCGM），可报告有用的 GPU I/O 统计信息。这些 GPU 专用工具与主机操作系统工具互补，能更完整地呈现 I/O 导致的 GPU 饥饿情况。

在 PyTorch 中，调用 next(data_iterator) 可以测得 GPU 等待下一批数据而空闲的总时间；该时间包括后台预取和主机 → 设备复制，而不仅是 Python 数据加载逻辑。

要隔离纯数据加载成本，可临时设置 num_workers=0 关闭预取，只对迭代器拉取计时；也可单独用计时器（或 CUDA 事件）包住.to("cuda") 或固定内存暂存，以捕获主机 → 设备复制开销。

瓶颈可能出在 Python 流水线，也可能出在向 GPU 内存的 memcpy 中；可以用以下方法区分二者，并将计时与整体“GPU 空闲”时间比较：

5.10.1 DataLoader 与 Python 成本

使用 num_workers=0 进行性能分析，观察 Python 循环和变换本身耗时；这会消除所有后台线程调度。

5.10.2 主机 → 设备复制成本

只测量设备传输时间：检查 Nsight Systems 中的“Copy”泳道，量化把数据暂存到 GPU 缓冲区实际阻塞 GPU 的时间；也可以在.to("cuda") 调用外加 torch.cuda.Event。

将这两项计时与整体“GPU 空闲”时间比较，就能判断该加速 Python 流水线（如增加 worker、简化变换），还是优化 H2D 传输路径（如使用固定内存、提高互连带宽或改用 GDS）。

通过监控存储流水线，例如可能发现 GPU 有 30% 的时间在等待数据。此时 GPU 整体吞吐量受 I/O 限制，应实施本章提到的策略以减少 I/O 停顿、提高计算吞吐量。调优后，GPU 等待数据的时间或许只剩 5%，每秒训练步数也会成比例提升——本例中为 6 倍。

存储和 I/O 优化往往在于消除积少成多的小低效，例如这里 5 ms 的延迟、那里小 10 MB 的缓冲区；但在规模上，修复这些问题影响巨大。底线与前面章节一致：让流水线保持满载——不仅是计算流水线，也包括数据流水线。从磁盘到 GPU 内存的每个组件都应监控、分析、剖析和改进，确保数据尽可能连续地流入 GPU。

5.11 调优数据流水线

除原始存储 I/O 外，CPU（或 GPU）上的预处理和数据加载流水线也是 AI 工作负载整体性能的关键部分。调优良好的数据流水线确保 GPU 从不因等待新数据而空闲；同时，还要以适当的并行度执行 CPU 工作，喂饱 GPU。

现代深度学习框架提供了加载和预处理数据的高级 API，可以也应该进行性能调优。下面讨论通用策略，以及 NVIDIA 的 DALI、NeMo 等高级数据流水线管理工具。

5.12 高效的数据加载与预处理

训练中典型的数据加载流程包括：从存储读取数据；解码或反序列化数据（如解析 JSON、解码 JPEG）；应用变换（如文本分词、图像裁剪）；把数据整理成批。这些步骤可能消耗大量 CPU，计算密集时也可以卸载到 GPU。为保持高吞吐量，可采用以下技术：

5.12.1 使用多个 worker 进程/线程

如前所述，PyTorch DataLoader 等框架允许指定 num_workers。每个 worker 并行获取和预处理数据；它们通常是独立进程，以避免 Python GIL 问题。主进程通过队列从 worker 进程异步获取批次。

5.12.2 避免 Python 瓶颈

如果数据加载逻辑在 Python 中，要警惕繁重的 Python 级处理。例如在循环中用纯 Python 代码逐行分词就是危险信号；此时应尽量向量化操作，或使用 C++/C 绑定提高性能。Hugging Face Tokenizers、TorchText 等库处理这类常见任务，虽然提供 Python 绑定，底层却用快速的 Rust/C++ 编写；Python 只是 C/C++ 代码之上的易用接口。

5.12.3 重叠 CPU-GPU

思路是让数据准备与 GPU 处理重叠。理想情况下，GPU 处理批次 N 时，CPU 已经加载并预处理完批次 N+1，把它放入固定内存；GPU 处理完批次 N 后直接 DMA 复制批次 N+1 并立即开始计算，CPU 则继续处理批次 N+2。这种流水化对性能至关重要；使用多个 worker 时大多数框架默认如此，但应监控确认。否则 GPU 可能在每次迭代开始时因等待数据而空闲。

5.12.4 通过整理张量按批执行操作

如果可能，让加载器按批而不是按样本执行操作。例如用向量化操作一次对整批张量应用变换，通过自定义 collate_fn() 整理批次，或在 GPU 上的训练循环中完成。这比逐行处理输入数据好得多；但有些变换必须逐样本执行，因此有时需要先理解工作负载，才能有效地批处理和整理。

5.12.5 数据加载中使用内存固定

在 PyTorch DataLoader 中启用 pin_memory=True 可以加快主机 → GPU（H2D）传输，并且当源内存固定时，.to(..., non_blocking=True) 复制可以真正异步。从固定内存 DMA 避免了额外复制和缺页，因为数据锁定在 RAM 中并可直接传输；向 GPU 传输数据时几乎总是有益的。务必设置较高的 ulimit -l（或容器 –ulimit memlock），避免大型固定缓冲区分配失败。

5.12.6 预取批次

某些框架允许指定预取队列长度，即提前加载多少批。PyTorch DataLoader 默认使用 prefetch_factor=2 等保守值，每个 worker 预取两个批次；内部最多排队 num_workers * prefetch_factor 个批次，因此阻塞前每个 worker 会加载两个批次的数据。如果工作负载 I/O 突发，或 worker 偶尔让 GPU 饥饿，可以把 prefetch_factor 提高到 4 或 8。下面是一个同时使用 pin_memory 和 prefetch_factor=4 的 PyTorch 片段：

```
import torch
from torch.utils.data import Dataset, DataLoader


# Create a Dataset and DataLoader that prefetches 4 batches
# per worker into pinned CPU memory.
class Synthetic(Dataset):
    def __init__(self, n, shape): self.n, self.shape = n, shape
    def __len__(self): return self.n
    def __getitem__(self, i):
        # Cheap CPU-side work; replace with real parse/decode
        return torch.ones(self.shape,  dtype=torch.float32)


B, C, H, W = 32, 3, 224, 224
dataset = Synthetic(n=100_000, shape=(B, C, H, W))


loader = DataLoader(
    dataset,
    batch_size=B,
    num_workers=8,
    pin_memory=True,
    persistent_workers=True,
    prefetch_factor=4,
  )
```

```
copy_stream = torch.cuda.Stream()
compute_stream = torch.cuda.current_stream()

for batch in loader:
    with torch.cuda.stream(copy_stream):
        batch_gpu = batch.to(device, non_blocking=True)
    # ensure pending H2D completes before compute uses it
    with torch.cuda.stream(compute_stream):
        torch.cuda.current_stream().wait_stream(copy_stream)
        outputs = model(batch_gpu)
```

本例中，8 个 worker 进程各将 4 个批次预加载到固定内存；由于主机内存已固定，异步.to(device, non_blocking=True) 传输可以用 DMA 高速复制数据。

因此，GPU 处理当前批次（批次 N）时，DataLoader 已在并行准备和传输下一批次（批次 N+1）。这种重叠至关重要；没有固定内存时，系统要为每次传输即时固定内存，引入不必要延迟。本质上，固定内存让 CPU 到 GPU 的数据传输更快、并与 GPU 计算并发，最大化整体吞吐量。

另一个选项是启用 persistent_workers=True，让 worker 跨 epoch 保持存活并持续填充队列。多次循环同一数据集时效果最好，尤其是迭代（epoch）很短时；worker 启动因导入模块、打开文件等产生显著开销时也很有帮助。使用持久 worker 可避免每个 epoch 边界创建和销毁进程的成本，worker 保持存活，能以最小开销立即为下一 epoch 预取。

常见的陷阱是在流水线中引入隐藏瓶颈，例如添加调试日志或昂贵的 CPU 变换；延迟可能只在负载下显现。要发现它们，先禁用所有下游 GPU 工作，计时 DataLoader 独立产出 100 个批次所需时间；测出基线后，与目标迭代时间和正常训练中测得的总 GPU 空闲时间比较。

如果 DataLoader 单独运行就太慢，可通过删除逐元素日志、简化变换或增加 worker 来优化 Python 流水线。如果隔离加载速度与实际运行加载速度差距很大，很可能受限于主机 → 设备传输或 kernel 启动开销。

> **注** 如果为了分析性能而禁用 GPU kernel 来隔离 DataLoader，也会减少 CPU 侧的 kernel 启动开销，因此“纯”数据加载吞吐量看起来往往低于真实训练运行；该技术仍然有用，但请记住这一点。

5.13 随 GPU 数量扩展而扩展 worker

增加 GPU 时，也必须扩展数据流水线，否则设备会饥饿。实践中意味着增加 DataLoader 的 worker 数量或 I/O 带宽以喂饱每块 GPU；这是提高总批次大小的前提，使每次迭代在更多设备上移动更多样本。

只扩展计算而不扩展摄取流水线资源，会把瓶颈进一步推向数据加载流水线。在多节点数据并行配置中，每个 rank 读取不同的分片，聚合数据加载工作量随集群规模扩展。

> **注** 要始终测量 CPU 使用率，因为随着 GPU 训练加速，数据输入流水线将成为瓶颈。

为维持必要吞吐量，需要如前所述的并行、高带宽分布式存储后端，以支持集群众多节点间的超大规模数据分片。回想前面按节点分片数据集的讨论：增加节点时，要确保每个节点的本地存储能处理自己那份数据集。

5.14 用 NVIDIA DALI 处理多模态数据

对于复杂或繁重的数据预处理，NVIDIA 提供数据加载库 DALI。DALI 把处理移到 GPU，或使用 C++ 编写的高效 CPU 代码来加速数据处理，尤其适合解码和增强能从 GPU 加速中获益的图像和视频数据。

例如，DALI 可以在 GPU 上解码 JPEG 图像，并应用随机裁剪、调整大小、归一化等增强，全都在 GPU 上完成；在 GPU 有空闲周期时通常比 CPU 更快。这卸载了 CPU 处理并减少所需 CPU worker 数量。

DALI 流水线以声明方式定义为算子的静态图：继承 nvidia.dali.pipeline.Pipeline，在 define_graph() 中声明数据源和 CPU/GPU 算子；DALI 内部使用自己的线程池和队列处理执行、预取和线程调度。

如果工作负载受输入限制（如模型训练），集成 DALI 可能显著提高吞吐量；但必须把它整合进训练循环本身，这会增加一些复杂度和学习成本。

对于分类、目标检测、分割等常见工作负载，NVIDIA DALI 提供在 GPU 上解码图像和视频的预构建流水线，充分利用 GPU 的媒体加速硬件。

考虑一个读取图像和视频、执行增强并训练目标检测模型的数据流水线。CPU 使用率可能达到 800%，即 8 个核心 100% 利用，但 GPU 仍偶尔停顿。

使用 DALI 后，CPU 使用率可能降到 200%，即只用两个核心执行文件读取，实际图像和视频解码由 GPU 完成；GPU 还能与计算并发执行读取。

实践中，真正的加速完全取决于在流程中放置 DALI 的位置。如果只用 DALI 解压 JPEG，然后立即把原始像素交回 CPU 进行增强和整理，就会产生额外的主机-设备-主机复制，可能抵消使用 DALI 带来的性能收益。

> **注** 更好的 DALI 用法或许是识别 GPU 友好的预处理操作，直接融合进基于 GPU 的预处理计算图。大多数预处理可以用 TorchVision、TensorRT 等现有 CUDA 库或自定义 CUDA kernel 完成，避免数据在 CPU 和 GPU 之间来回移动；这可能比在流水线中使用 DALI 获得更高的端到端性能，值得探索。

一如既往，应在现实条件下基准测试端到端系统：比较纯 CPU 流水线、启用 DALI 的流水线和完全融合的 GPU 计算图实现，确定哪种能为你的模型和数据集提供 CPU 节省与 GPU 利用率的最佳平衡。

5.15 用 NVIDIA NeMo Curator 创建高质量 LLM 数据集

NVIDIA NeMo 是开发训练语言模型的工具包，其中包含 NeMo 库与框架套件，包括开源的 NeMo Curator 框架。

Curator 帮助为 LLM 训练准备大型多模态数据集，处理来自不同来源的 TB 级数据时很有用；它支持清洗、分词、打乱等数据处理步骤。

NeMo Curator 可以把数据集预处理分布到多块 GPU 或多个节点，利用多个加速器更快地准备数据——组装数 TB 级训练数据集时这一点很重要。

此外，Curator 可以把数据压缩打包成少量大文件，或转换成机器更易消费的二进制格式；还能创建新的合成训练数据集来扩充相对有限且日益稀缺的人类数据集。

Curator 在训练前离线完成繁重预处理后，在线训练数据流水线变得简单得多，只需读取准备好的数据，或许再做些轻量的“最后一公里”打乱。

NeMo Curator 还可以通过去重和移除问题内容来执行数据质量过滤，这对 LLM 训练质量和性能都很重要。提前拥有结构良好、已预处理和清洗的数据集，意味着训练流水线能持续获得结构良好、大小均匀的数据（例如填充到固定长度），无需即时分词，也能避免棘手的字符串处理。

如果有 NeMo Curator 等工具，明智的做法是加以利用，让训练作业主要是 GPU 前向和反向传播，而不是处理文本、读取数百万个大小随机的小文件。基于 NeMo 的训练中，预处理数据集通常存储为可内存映射的.bin 数据文件和.idx 索引文件；NeMo Curator 的 DocumentDataset 读写分片 JSONL 或 Parquet，构建索引数据集时再转换成.bin/.idx。

> **注** 可以考虑以 N 种不同方式打乱并存储 N 份数据副本，避免 N 个 epoch 训练期间的运行时打乱成本；代价显然是磁盘空间和内存，但这是一个值得考虑的取舍。

总的来说，要在训练前准备数据，几乎不应该用原始文本训练。离线预处理可能花费一些时间，但长期回报是更快的训练、更快的迭代和更可预测的扩展。

本章所有技术都旨在让昂贵 GPU 集群绝不因数据加载流水线而闲置。数据流水线供不上输入，性能再高的 GPU 也毫无用处；因此需要在存储、网络、CPU、GPU 等所有层面采取整体性的全栈优化方法。

> **注** NeMo 的数据加载仍在 CPU 上运行。要绕过 CPU I/O，需要与 GDS 等工具集成。

许多情况下，优化数据流水线比任何算法调整带来的提升都大：调优不当的输入流水线可能浪费 50% 的 GPU 时间，而算法优化或许只有几个百分点。

5.16 持续性能分析与调优工作流

性能工程是迭代过程。为确保分布式训练或推理应用在扩展或修改后仍保持高效，应采用持续性能分析和调优工作流：定期收集性能数据、识别瓶颈、应用优化、再次测量。

随着时间推移，硬件和软件更新会改变最优设置，因此需要持续性能分析和调优。为此，注重性能的工程团队通常维护性能仪表盘，长期跟踪 samples/sec 等指标。

可以考虑设置自动化的夜间运行，对训练和推理工作负载进行性能分析，从而捕获性能回退或改进，并追溯到代码变更。

下面看一个典型工作流和一组最佳实践，广泛适用于所有性能分析和调试场景，而不仅限于本章主题：

5.16.1 建立基线

从单块 GPU 或最小配置开始，测量性能，例如以 samples/sec 计的训练吞吐量或（但愿）以毫秒计的推理延迟；然后扩展到单节点多 GPU，再到多节点，每次都通过整体吞吐量等高层指标分析性能扩展情况。理想情况下，简单的数据并行工作负载在 N 块 GPU 上应获得 N 倍吞吐量；若远低于此，说明系统开销过大，下一步是量化瓶颈。例如，8 块 GPU 只带来 5 倍吞吐量提升，说明系统效率仅 62.5%，并不理想。

5.16.2 对多 GPU 运行做性能分析，找出瓶颈

为诊断瓶颈的确切原因，使用 Nsight Systems（nsys）等系统分析工具深入剖析整个多 GPU 作业，获得时间分布概览。第一步是查看 GPU 利用率时间线：GPU 是否频繁停顿？在等什么？还要检查 CPU 时间线：主进程是否落后于其他 worker 进程？是否存在所有线程都在等待的同步点？例如，如果 GPU 在梯度 all-reduce 期间空闲，说明通信是瓶颈；如果在每次迭代开始时空闲，瓶颈可能是数据加载或某个 kernel。

5.16.3 必要时深入具体 kernel

如果发现某个 GPU 操作（网络或计算）比预期慢，可以用 Nsight Compute（ncu）深入分析该 kernel 检查效率。例如前面讨论中，我们查看一个 NCCL kernel：通信走 PCIe 而非 NVLink 时，SM 利用率只有 60%，内存停顿计数器很高；优

化为 NVLink 后，SM 利用率升至 90% 忙碌，内存停顿也减少。这种深入分析可以确认 kernel 是受网络带宽、内存带宽还是计算限制——必居其一。

5.16.4 确定原因

发现瓶颈后，把它映射到一组假设原因，逐一验证或排除。例如：瓶颈在网络，可能没用 RDMA、消息太小或重叠不够；GPU 空闲等待其他 GPU，可能是数据不均衡导致某块 GPU 做更多工作的掉队情况；工作负载受 CPU 限制，可能数据加载器配置不对，或有更适合 GPU 的 CPU 聚合操作；瓶颈在 GPU 内存，可能某些 kernel 在寄存器和 HBM 之间传输过多数据，可以尝试减小批次或引入 kernel 融合。

5.16.5 应用修复或优化

遍历假设找到真正原因后，就该行动了。修复网络和通信瓶颈：确保启用 GPUDirect RDMA；如果有多块 NIC 且仍受网络带宽限制，增大 NCCL_NSOCKS_PERTHREAD；考虑使用梯度压缩等技术压缩数据（后面章节会讲）。如果跨越 NUMA 节点，尝试分层方法，或配置 NCCL 拓扑让每个 NUMA 节点使用更少 GPU，等等。

对于节点内拓扑问题，如果 GPU 分散在多个 PCIe 交换机上，尽量把作业绑定到单个 NUMA 节点的 GPU 以避免慢速互连，或使用更具拓扑感知的算法。对于 CPU 和数据问题，检查数据加载器是否太慢：增加 worker 进程/线程，或用 DALI 把部分预处理移到 GPU，或提前做更多离线预处理。如果某块 GPU 较慢（可能在执行额外验证或日志），尝试减少工作量，或用异步操作把它移出关键路径。

如果同步是问题，删除代码中可能无意间串行化执行的 torch.cuda.synchronize() 调用或屏障（第 13 章详述）。如果环境需要调优，必要时设置 NCCL_IGNORE_CPU_AFFINITY=1，或把 CPU 线程固定到不同的拓扑配置，等等。有时只需几个小改动、花很少力气，就能把很差的资源利用率变成最大利用率（当然说易行难，但保持乐观总是好的！）：

5.16.6 每次改动后重新测量

与任何调试工作一样，重要的是每次只改一两处，然后测量；否则无法知道哪个改动起了作用。在当前配置上获得良好扩展性后，记录良好指标值，并在扩展时努力维持这些值。

5.16.7 保持软件更新，但务必验证

新版本 NCCL 或 CUDA 常常带来性能提升，例如新版 NCCL 可能自动分层或采用通信/计算重叠机制；PyTorch 更新可能降低 DDP 开销或引入更高效的分布式优化器。但每次更新也可能因改变最优设置而带来不稳定，因此要重新运行性能分析工作流，确保维持上次已知良好系统配置的指标值。

5.16.8 利用现代硬件特性

假设突然拿到拥有更多统一内存、更高内存带宽和更快互连的最新硬件。第一步是理解新特性并善加利用：现在可以使用更大的输入批次、把更大的模型装进内存。但要缓慢爬坡并监控资源利用率；扩展过猛会再次占满新的资源，不得不重新开始性能分析和调优工作流！

5.16.9 生产环境自动化监控

如果经常运行大型训练和推理工作负载，最好在生产环境中保持一致的监控设置，持续跟踪 GPU 利用率、网络吞吐量和内存吞吐量。这样，当作业或推理请求因环境问题、内核更新或数据流水线回退而变慢时，能迅速发现。Kubernetes 等作业调度器与监控工具集成良好；例如可以在利用率低于某个阈值时设置告警。

5.16.10 文档化与培训

性能调优常常涉及团队中的隐性知识，例如覆盖了哪些环境变量、哪些库版本有缺陷等。把这些发现直接记录在代码或配置文件中，让其他人或未来的

你每次打开这些文件时都能想起。例如注明“在此集群配置中，我们发现设置 NCCL_SOCKET_NTHREADS=2 使多节点吞吐量提升 10%”。希望这是标准做法。

持续遵循这一工作流，本质上就形成了反馈循环：运行 → 测量 → 调优 → 运行 → 测量 → 调优 → ……这个循环确保在扩展到更多 GPU、使用更好模型时，系统性能和效率得以维持。保持良好性能远比性能退化后再恢复容易得多；让如此多的部件始终发挥最高水平是一场持久战。

总之，把性能当作需要持续测试和验证的功能：像为代码正确性编写测试一样，为性能编写检测测试。例如，GPU 数量翻倍，吞吐量是否大致翻倍？如果没有，就拿起分析器，启动性能分析和调优工作流。建议组合使用 Nsight Systems（高层系统视图）、Nsight Compute（底层 GPU kernel 剖析）以及 NCCL 和 PyTorch 的日志，形成定位问题的完整工具箱。

这个过程结束时，你将拥有一个精细调优的 AI 系统；更新代码或硬件后要再次迭代。在 AI 这样动态快速的环境中，性能调优永无止境；但知道该看什么之后会容易得多——这正是你读这本书的原因！

5.17 诊断通信受限与计算受限的工作负载

例如，要判断模型训练工作负载中计算还是通信是限制因素，可以改变计算与通信的比率，观察 NIC 上实测网络吞吐量（GB/s）的变化。假设剖析训练作业的反向传播，发现梯度 all-reduce 在 100 GB/s 的 NIC 上只利用了 60 GB/s。

要判断是网络堵塞还是 GPU 太慢，可以固定通信量，通过增减批次大小来增减计算量。这一方法很理想，因为增大批次不会影响反向传播梯度 all-reduce 传输的数据量——梯度数量随模型参数数量而非批次大小变化。

在通信量固定的情况下，把批次减半，观察实测网络吞吐量的变化。如果仍保持 60 GB/s，说明 GPU 本可以完成更多工作，是网络不让它们做更多——网络是限制因素。

反之，如果实测网络利用率从 60 GB/s 降到 40 GB/s，说明 GPU 计算完成得不够快，无法让 NIC 保持忙碌，网络因此空闲等待 GPU 提供更多数据——此时计算是限制因素，而非网络。

还可以反过来把批次翻倍进一步验证这一假设。同样，all-reduce 梯度通信量不变：如果通信确实是限制因素，随着批次和计算量增大，NIC 会保持 60 GB/s；但如果计算是限制因素，all-reduce 通信占单次迭代总时间的比例将随计算时间增长而缩小。

观察这两次实验中 NIC 上的绝对 GB/s 以及通信相对计算的时间占比，就能精确定位该调优哪个子系统。更具体地说，可以绘制增减批次大小时 GB/s 与通信占比的关系图，准确看出 Roofline 位置，判断工作负载是通信受限（网络）还是计算受限（GPU SM）。

> **注** 使用 Nsight Systems 获取端到端时间线。如果看到 GPU 空闲、在等待数据——计算 kernel 之间出现对应 NCCL 等待的长间隔——那么很可能是通信受限。如果 GPU 忙碌但未达到预期 FLOPS，则可能是内存受限或计算受限；Nsight Compute 和 PyTorch 分析器可以帮助确定 kernel 的内存和计算效率。

5.18 关键要点

分布式 AI 的峰值性能来自全栈协同优化——从 GPU kernel、网络传输到 CPU 线程和存储，任何一环薄弱都会拖累整个系统。调优存储层时，请记住以下关键经验：

5.18.1 扩展计算的同时扩展输入数据流水线

扩展 GPU 时不要忽视存储和数据加载。确保存储系统提供足够带宽，并充分利用这些带宽；随 GPU 数量增加同步提高数据加载器的并行度。否则总有一天会因输入流水线跟不上，加 GPU 也不再提速。

5.18.2 为工作选用正确的工具

NCCL 面向模型训练中常见的可扩展集合通信（all-reduce 等）；NIXL 面向模型推理中常见的高吞吐点对点和流式传输。token 流式传输占主导的工作负载

应使用 NIXL；大规模集合通信和对称内存模式则优先使用 NCCL/NVSHMEM。GPUDirect RDMA 和 GDS 分别消除网络和存储 I/O 的主机内存中转缓冲区，而 CPU 仍负责调度和控制传输。多 GPU 训练务必使用 DistributedDataParallel 而非 DataParallel。这些专用库和框架经过深度调优，旨在榨干硬件性能；要善用它们而不是重新发明轮子。

5.18.3 端到端性能分析

瓶颈位置并不总是一目了然。使用 Nsight Systems、Nsight Compute 和 PyTorch 分析器等工具查看时间花在哪里；GPU 要么计算受限，要么通信受限，要么 I/O 受限。遵循前面讨论的性能分析示例，可以引导你找到问题根源。例如，可以验证 NCCL kernel 是否正确与计算交织，并在 Nsight Systems 中检查 GPU 空闲时间。

5.19 结论

高性能分布式存储系统是调优大型复杂 AI 系统的基础组件。集成 NVMe SSD 和 GDS 等先进存储技术，可以提升数据加载流水线性能、缩短训练时间、加快实验和迭代速度。

通过离线预处理数据流水线、高效数据缓存和异步通信等技术应对存储和 I/O 挑战，现代 AI 部署即便在模型复杂度和数据集规模不断扩大的情况下，也能维持高吞吐量。

对实践者而言，要点是你无需从零发明定制 I/O 方案。NVIDIA 和开源社区提供了深度调优的专用库和工具，让你专注于模型、数据和应用逻辑，而不是底层管道。

对性能工程师而言，教训是快速的数据移动与原始算力同样关键。世界上最快的 GPU，如果总在等待存储中的数据，也几乎毫无用处。

GDS 和先进输入流水线等技术是保持数据顺畅流动、让 GPU 有活可干的全栈方法的一部分。善用这些技术并持续性能分析和调优，就能在规模上把分布式 AI 系统推向其理论峰值极限。

在接下来的章节中，我们将以此为基础，深入 CUDA 和 PyTorch 优化策略以及一些高级系统调优主题。随着我们继续重叠通信与计算、使用最快的链路并逼近硬件理论最大性能，本章学到的原则将继续适用于栈的每一层。最终，这一切将带来更快的洞察速度、更好的资源利用率和成本节约。

第 6 章 GPU 架构、CUDA 编程与最大化占用率

本章首先回顾单指令、多线程（SIMT）执行模型，以及 warp、线程块和网格如何将基于 GPU 的算法映射到流式多处理器（SM）上。

我们将回顾现代 NVIDIA GPU 上的 SIMT 执行模型，包括 warp、线程块和网格如何映射到 SM。随后深入 CUDA 编程模式，讨论片上内存层次结构（寄存器文件、共享内存/L1、L2、HBM3e），并演示 GPU 的异步数据传输能力，包括张量内存加速器（Tensor Memory Accelerator，TMA）以及作为 Tensor Core 运算累加器的张量内存（Tensor Memory，TMEM）。

我们还将介绍 Roofline 分析，用于识别计算受限与内存受限的 kernel（内核）。这些内容将为推动现代 GPU 系统逼近其理论峰值吞吐量上限奠定基础。

6.1 理解 GPU 架构

CPU 优化的是低延迟的单线程性能，而 GPU 是面向吞吐量优化的处理器，旨在并行运行数千个线程。图 6-1 展示了一个简单的 CUDA 编程流程。

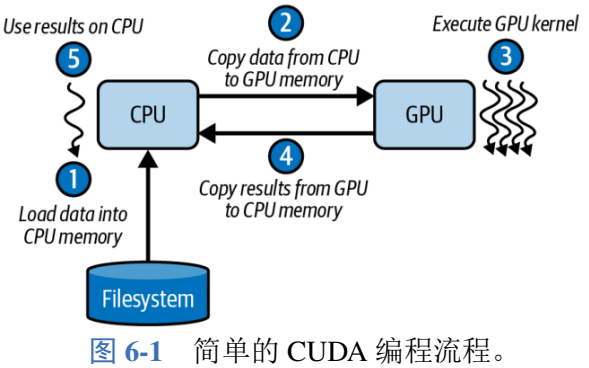

图 6-1　简单的 CUDA 编程流程。

首先，主机将数据加载到 CPU 内存中。随后将数据从 CPU 复制到 GPU 内存。在使用 GPU 内存中的数据调用 GPU kernel 后，CPU 再将结果从 GPU 内存复制回 CPU 内存。此时结果重新位于 CPU 上，可供进一步处理。

GPU 依靠大规模并行性来隐藏数据传输延迟，例如图 6-1 所示的 CPU 到 GPU 数据传输。每个 GPU 都包含许多 SM，它们大致类似于 CPU 核心，但针对并行性进行了精简。在 Blackwell 上，每个 SM 最多可跟踪 64 个 warp（每个 warp 包含 32 个线程）。

每个 GPU 都包含许多 SM，它们类似 CPU 核心，但针对吞吐量进行了优化。在现代 GPU 上，每个 SM 可同时跟踪最多 64 个 warp（2,048 个线程）。Blackwell GPU 的每个 SM 配备 64K 个 32 位寄存器（总计 256 KB），以及每个 SM 组合容量为 256 KB 的 L1 缓存/共享内存。其中最多 228 KB（可用 227 KB）的 SRAM 可配置为每个 SM 上由用户管理的共享内存。单个线程块最多可以申请 227 KB 的动态共享内存（228 KB 中有 1 KB 由 CUDA 保留）。这些资源帮助 SM 支撑 GPU 高度的线程级并行性。

在 Blackwell SM 内部，多个 warp 调度器向可用流水线发射指令；4 个独立的 warp 调度器允许每个周期最多有 4 个 warp 向可用流水线发射指令。此外，每个调度器都支持双发射，可以从同一个 warp 发射两条相互独立的指令（例如一条算术操作和一条内存操作）。注意，双发射必须来自同一个 warp，而不能跨 warp。

在最佳情况下，每个调度器各有一个 warp 可在每个周期并发发射一条指令，从而每个周期并行执行 4 个 warp。当利用指令混合时，吞吐量还会进一步提升，如图 6-2 所示。

这里，每个 SM 被划分为 4 个独立的调度分区，每个分区都有自己的 warp 调度器和派发逻辑。可以把 SM 看成共享片上资源的 4 个“迷你 SM”。这样，硬件就能挑选已经就绪的 warp，并在每个时钟周期从最多 4 个不同的 warp 发射指令。

在 4 个“迷你 SM”分区中的每一个分区内，调度器每周期可从同一个 warp 发射两条指令：一条算术指令（例如 INT32、FP32 或 Tensor Core 指令）和一条内存指令（加载或存储）。这就是调度器被称为“双发射”的原因。表 6-1 总结了这些数值。

> **注** 所有指标表中的数值均为用于解释概念的示例值。关于不同 GPU 架构上的实际基准测试结果，请参见 GitHub 仓库。

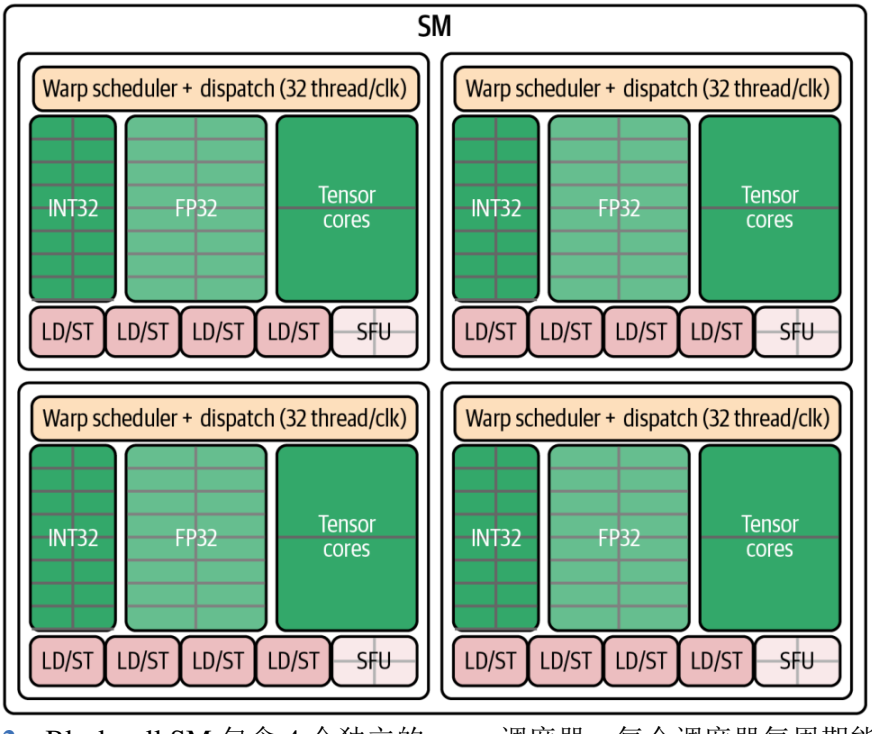

图 6-2　Blackwell SM 包含 4 个独立的 warp 调度器；每个调度器每周期能够发射一条 warp 指令，并可双发射一条数学操作和一条内存操作。

表 6-1　SM 调度器与指令发射的关键限制（每周期）。

指标	数值
调度器数量	4 个
最大已发射 warp 数	4 个（每个调度器 1 个）
最大数学操作数	4 个（每个调度器的算术发射各 1 个）
最大内存操作数	4 个（每个调度器的加载/存储发射各 1 个）

因此，在最佳情况下，每个周期可以跨 4 个 warp 双发射 4 条数学指令和 4 条内存指令。这将同时最大化计算吞吐量和内存吞吐量。这些数值来自 SM 的四路分区，以及其每周期从每个分区选择一个 warp 并发射两条正交指令的能力。

特殊功能单元（Special Function Unit，SFU）与 INT32、FP32 和 Tensor Core 流水线并列。它们处理超越函数运算（例如正弦、余弦、倒数和平方根）。不过，

SFU 并不属于数学与内存双发射组合。SFU 使用专用的 SFU 流水线，独立于主 INT32/FP32 流水线和加载/存储（LD/ST）流水线运行。

由于 SFU 占用独立流水线，并且在需要时可以并行执行，SM 可以继续发射数学和内存指令，而不必等待较慢的函数完成。对于混合操作 kernel，这种分离进一步提高了指令级并行性和总体吞吐量。它们使复杂数学操作不会阻塞核心计算和内存流水线。

由于存在 4 个调度器，且每个调度器通常每周期可以发射一条 warp 指令，因此在有足够独立工作和可配对发射的情况下，每周期最多可以有 4 个 warp 向前推进。例如，内存操作可以流经 SM 合计 16 条加载/存储（LD/ST）流水线（每个调度器 4 条 LD/ST 流水线）。这些流水线将从 L1/共享内存、L2 缓存或全局内存读取数据，或向其中写入数据（下一节将介绍这些内存）。

> **注** 精确的 LD/ST 流水线数量和配对关系并无保证。应依靠性能分析计数器确定 kernel 受内存发射还是计算发射限制，并查阅针对具体架构的 NVIDIA 文档。Blackwell 调优指南是一个很好的起点。

简而言之，GPU 擅长数据并行工作负载，包括大型矩阵乘法、卷积以及其他将同一条指令应用于许多元素的操作。开发者可以直接使用 CUDA C++ 编写 kernel，也可以通过 PyTorch 等高级框架，或 OpenAI 的 Triton 这类基于 Python 的领域专用 GPU 语言间接编写。

在深入 kernel 开发和内存访问优化之前，我们先回顾 CUDA 线程层级，以及支撑所有这些实践的关键术语。

6.1.1 线程、warp、线程块与网格

CUDA 将并行工作组织成三级层级——线程、线程块（也称协作线程阵列，cooperative thread arrays，CTA）和网格——以平衡可编程性与大规模吞吐量。在最低层级，每个线程执行 kernel 代码。在现代 GPU 上，每个线程块最多可包含 1,024 个线程。当启动 kernel 时，线程块会组成一个网格，如图 6-3 所示。

通过适当地设置网格大小，可以扩展到数百万个线程，而无需更改 kernel 逻辑。CUDA 运行时（以及 PyTorch 等框架）负责在所有 SM 之间进行调度和分

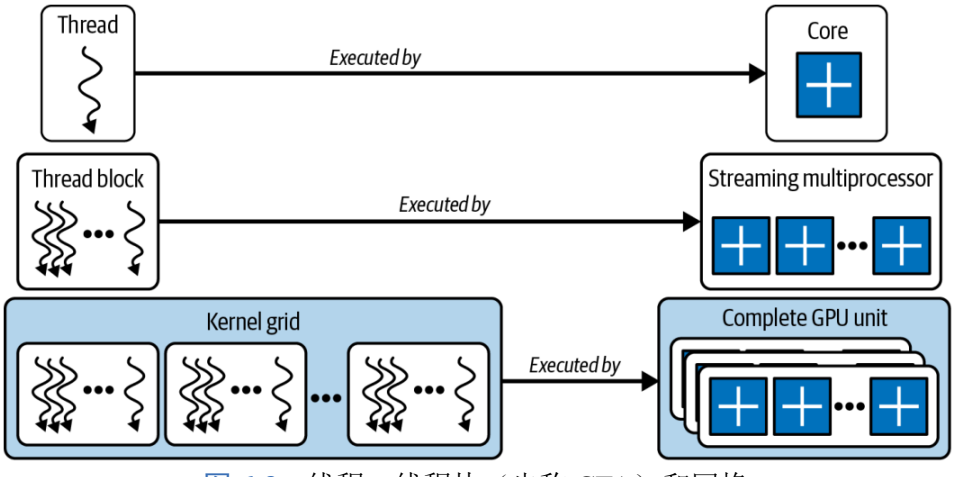

图 6-3　线程、线程块（也称 CTA）和网格。

配。图 6-4 从另一个角度展示了线程层级，其中包括基于 CPU 的主机；主机负责调用运行在 GPU 设备上的 CUDA kernel。

传统上，不同线程块中的线程不能直接协作。不过，现代 GPU 架构和 CUDA 版本支持线程块集群（thread block clusters）。线程块集群是可跨 SM 彼此通信的一组线程块。

具体来说，在一个线程块集群内，不同线程块中的线程可以访问彼此的共享内存，并使用硬件支持的集群作用域屏障。这些机制支持更大规模的计算操作，包括矩阵乘法；矩阵乘法在当今的大型 LLM 工作负载中非常常见。线程块集群在参与集群的 SM 之间共享一个分布式共享内存（distributed shared memory，DSMEM）地址空间，如图 6-5 所示。

DSMEM 是一种硬件特性，它通过高速片上互连，将一个线程块集群中所有 SM 的共享内存 bank 连接起来。借助 DSMEM，多个 SM 共享一个组合的、多 SM 的分布式共享内存池。这种统一使不同线程块中的线程能够以片上速度读取、写入并原子更新彼此的共享缓冲区，而无需占用全局内存带宽。

> 注　第 10 章将介绍线程块集群和 DSMEM 等高级主题。它们是现代 GPU 处理的重要新增能力，也是 AI 系统性能工程师必须理解的内容。本章的重点仍然是块内共享内存优化。

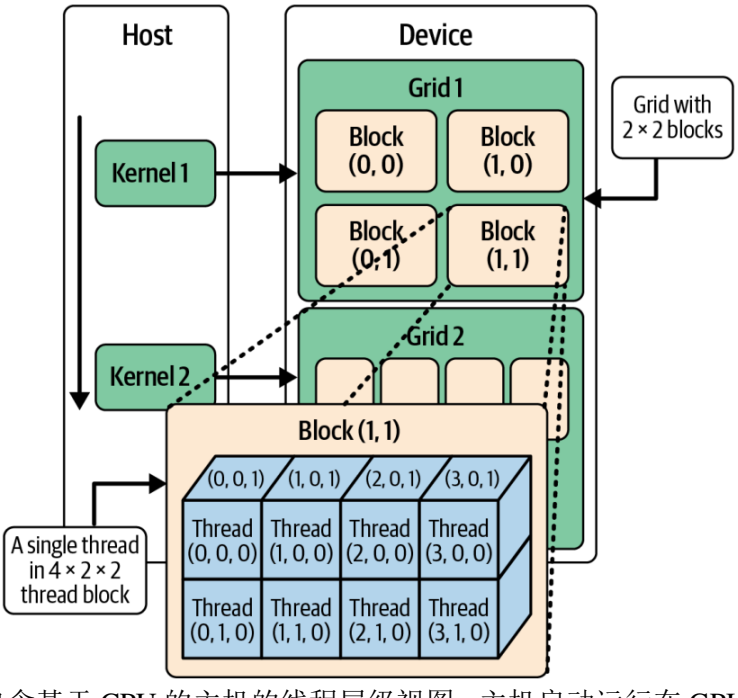

图 6-4　包含基于 CPU 的主机的线程层级视图；主机启动运行在 GPU 设备上的 kernel。

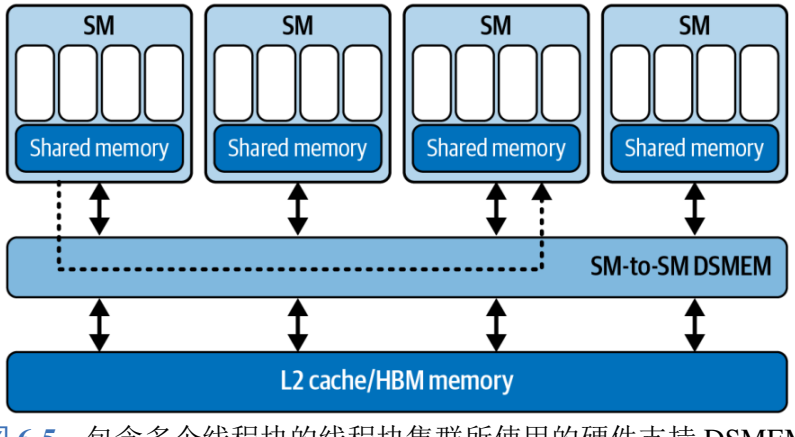

图 6-5　包含多个线程块的线程块集群所使用的硬件支持 DSMEM。

在每个线程块内，线程使用低延迟的片上共享内存共享数据，并使用 `__syncthreads()` 进行同步。由于每个屏障都会产生开销，应尽量减少同步点，如图 6-6 所示。

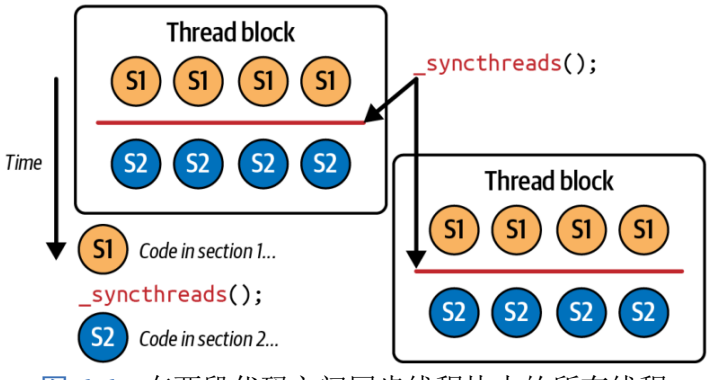

图 6-6　在两段代码之间同步线程块内的所有线程。

目标是尽量减少同步点。不过，GPU 硬件会通过在 warp 之间快速切换，努力隐藏全局内存加载、缓存填充和流水线停顿等长延迟事件。

在线程块内，线程被划分为包含 32 个线程的 warp，并在 SIMT 模型下由 warp 调度器以锁步方式执行，如图 6-7 所示。

让更多 warp 处于飞行中，称为 SM 上的高占用率。当 CUDA 代码能够实现高占用率时，意味着一个 warp 停顿后，另一个 warp 已经准备好运行。这会使 GPU 的计算单元保持繁忙。

不过，高占用率必须与每线程资源限制相平衡，例如寄存器和共享内存。将寄存器溢出到更慢的内存可能造成新的停顿。结合寄存器和共享内存使用量对占用率进行性能分析，有助于选择能够最大化吞吐量、同时又不触发资源竞争的线程块大小。

> **注**　第 8 章将介绍占用率调优，但在理解 SM、warp、线程等概念时，占用率是一个关键概念。

线程块彼此独立执行，且没有保证的执行顺序。这使 GPU 调度器能够将它们分派到所有 SM 上，从而充分利用硬件并行性。这种网格-线程块-warp 层级保证 CUDA kernel 无需修改即可运行在未来拥有更多 SM 和线程的 GPU 架构上。

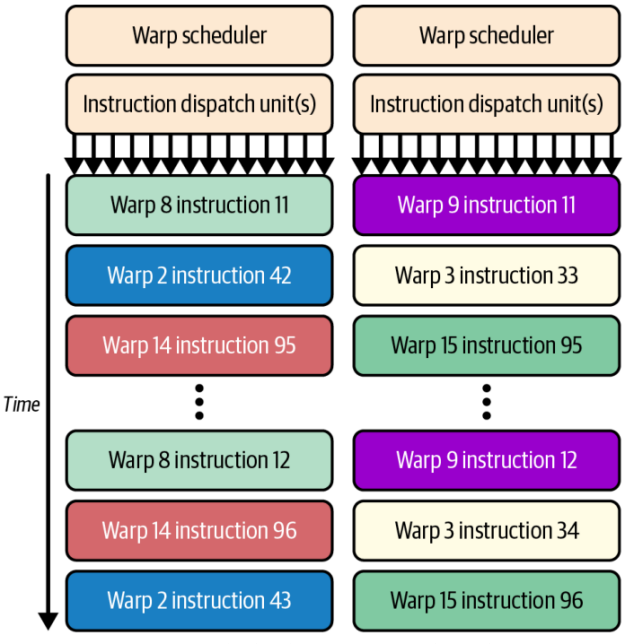

图 6-7　warp（32 个线程）作为整体推进，其指令由 warp 调度器管理。

吞吐量还取决于 warp 执行效率。warp 中的线程必须遵循相同的控制流路径，并执行合并的内存访问。如果部分线程发生分歧，一个分支走 if 路径，其他线程走 else 路径，那么 warp 会将执行串行化，依次处理每条分支路径。这称为 warp 分歧，如图 6-8 所示。

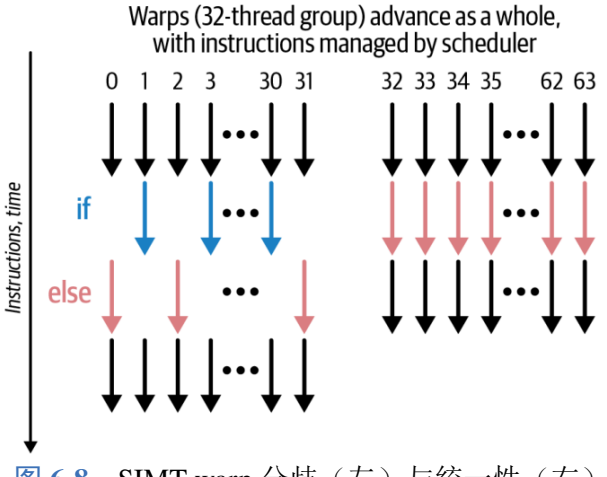

图 6-8　SIMT warp 分歧（左）与统一性（右）

通过屏蔽非活动 lane 并执行额外遍历来覆盖每条分支，warp 分歧会使总执行时间按分支数量成倍增加。第 8 章将更深入地讨论 warp 分歧，以及检测、性能分析和缓解它的方法。

> **注** 分歧只对同一个 warp 内的线程构成问题。不同 warp 可以遵循不同分支，而不会受到性能惩罚。

6.1.2 选择每个线程块的线程数和每个网格的线程块数

GPU 性能的一个关键方面，是选择与硬件 32 线程 warp 大小相匹配的线程块大小。因此，通常应选择 32 的整数倍作为线程块大小。例如，256 线程的线程块（8 个 warp = 256 ÷ 32）可以完整占用每个 warp，而 33 线程的线程块需要两个 warp 槽位，并且第二个 warp 的 lane 只有 1/32 被使用。由于无论 warp 是运行完整的 32 个线程还是只运行 1 个线程，每个 warp 都会占用一个调度器槽位，这会浪费并行机会。

此外，不同 GPU 代际具有不同的硬件限制，包括每个 SM 的最大线程数和每个 SM 的寄存器数量。如果希望保持良好性能，这些限制自然会限制线程块大小。例如，过大的线程块可能需要过多寄存器，从而导致寄存器溢出并降低 kernel 性能。

较大的线程块还可能需要过多共享内存，而 GPU 硬件中的共享内存容量是有限的。具体来说，Blackwell 每个 SM 仅提供 228 KB（可用 227 KB）的共享内存，供运行在该 SM 上的所有驻留线程块寻址。

这些硬件限制会影响一个 SM 同时可以激活多少线程块/warp。这就是我们前面介绍的占用率。较小的线程块如果允许更多并发 warp 在 SM 上运行，可能实现更高占用率。

了解 GPU 代际的相对规模和硬件线程限制非常重要，包括线程数、线程块数、warp 数和 SM 数。图 6-9 展示了这些资源的相对规模及其限制。

表 6-2 总结了 Blackwell B200 GPU 的这些 GPU 限制。其余限制可在 NVIDIA 网站上查到。（其他 GPU 代际的限制会有所不同，因此务必检查系统的确切规格。）

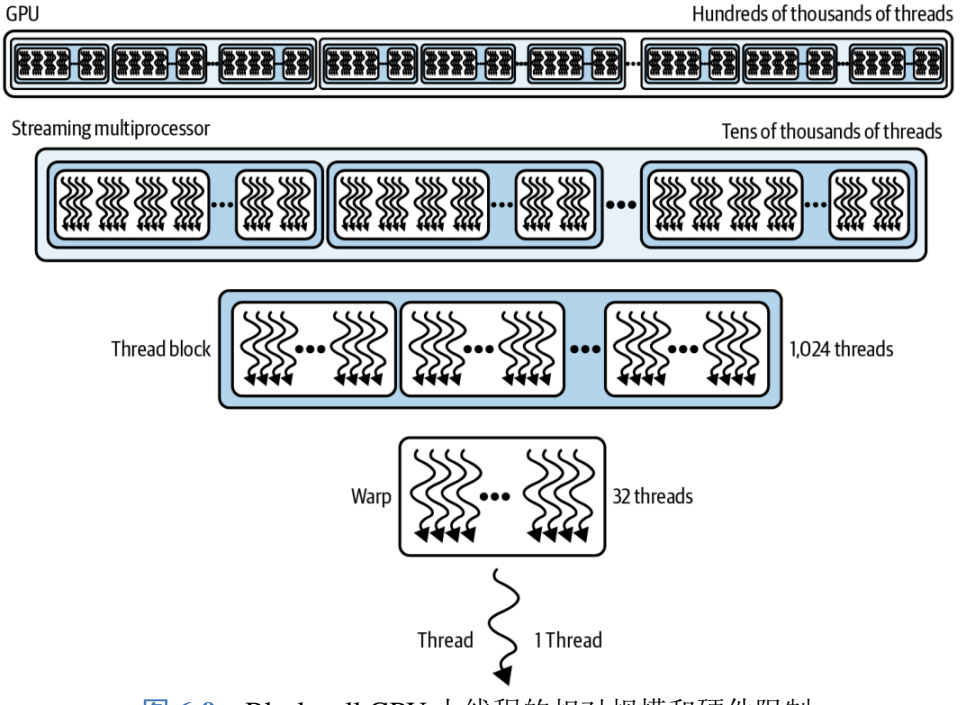

图 6-9　Blackwell GPU 上线程的相对规模和硬件限制。

表 6-2　线程级与线程块级限制（Blackwell B200）。

资源	硬件限制	备注
warp 大小	32 个线程	基本的 SIMT 执行单元是 32 个线程（一个 warp）。始终使用 32 的整数倍，以避免浪费。
每个线程块的最大线程数	1,024 个线程	`blockDim.x` × `blockDim.y` × `blockDim.z` ≤ 1024。
每个线程块的最大 warp 数	32 个 warp	1,024 线程 ÷ 32 线程/warp = 32 个 warp（每块最大值）。

我们已经讨论过 32 个线程的 warp 大小限制；这促使我们选择 32 个线程的整数倍作为线程块维度，以创建“完整 warp”并避免 warp 利用不足。注意，每个线程块最多可包含 1,024 个线程，相应地，一个线程块最多只能包含 32 个 warp。一旦线程块被调度，每个 SM 可以同时容纳的 warp 和线程块数量有限，这些限制会影响占用率。

此外，不同 GPU 代际还存在不同的每 SM 限制，通常称为 SM 驻留限制。表 6-3 总结了 Blackwell 的这些限制。

表 6-3　SM 驻留资源限制（Blackwell B200）。

资源（每个 SM）	硬件限制	备注
每个 SM 的最大驻留 warp 数	64 个 warp	硬件最多可让 64 个 warp 处于飞行中（64 × 32 = 2,048 个线程）。注：这一限制在许多代 GPU 中都保持不变，在 Blackwell 上仍然如此。
每个 SM 的最大驻留线程数	2,048 个线程	等于 64 个 warp × 32 个线程/warp。如果每个线程块使用 1,024 个线程，则同一 SM 上最多可以同时驻留 2 个这样的线程块（64 个 warp）。使用较小线程块（例如 256 个线程）可以让更多线程块驻留在 SM 上（最多 8×256 = 2,048 个线程），这可能提高占用率并帮助隐藏延迟；不过，过多很小的线程块会增加调度开销。
每个 SM 的最大活动线程块数	32 个线程块	一个 SM 上最多可同时驻留 32 个线程块（如果线程块较小，可以在达到此上限前容纳更多线程块）。

这里可以看到，Blackwell 上每个 SM 的最大并发 warp 数为 64。近几代 GPU 没有改变这一数值，因此占用率考量仍然适用。一个 SM 上的最大活动线程块数为 32，相应地，每个 SM 的最大驻留线程数为 2,048。CUDA 网格也有最大维度，如表 6-4 所示。

表 6-4　CUDA 网格限制。

网格维度	限制	备注
X、Y 或 Z 中的最大线程块数	X：2,147,483,647 个线程块；Y：65,535 个线程块；Z：65,535 个线程块	三维网格最大可达 2,147,483,647×65,535×65,535 个线程块。
最大并发网格数（kernel）	128 个网格	一个设备上最多可并发执行 128 个 kernel（即同时驻留 128 个网格）。

了解理论网格限制当然有用，但实际通常会受到前面所示线程/线程块/每 SM 限制的约束。如果某一维需要超过 65,535 个线程块，可以启动二维或三维网格，将工作拆分到多个 kernel 启动中（多次启动）。稍后将给出示例。在实践中，在碰到其他资源限制之前就达到网格大小限制的情况很少见。

6.2 CUDA GPU 的向后兼容与向前兼容模型

CUDA 的核心优势之一是其向前兼容和向后兼容模型。只要在二进制文件中包含 PTX 以支持向前兼容，今天编译的 kernel 通常就能在未来的 GPU 代际上无需修改地运行。如果只为单一架构提供 SASS（例如 Hopper 的 `sm_90` 或 Blackwell 的 `sm_100`），而不包含 PTX，那么该二进制文件不能向前运行在更新的架构上。`sm_100f` 或 `compute_100f` 等特定系列目标会将可移植性限制在同一特性系列的设备上。最佳做法是提供一个 fatbin，其中同时包含所需的通用 cubin/PTX 和系列特定 cubin（例如用于优化等）。

可以在加载时设置 `CUDA_FORCE_PTX_JIT=1`，强制 PTX JIT 编译并缓存结果，从而验证兼容性。如果二进制文件缺少 PTX，kernel 启动将失败。这会迫使你重新构建并启用 PTX 支持。这种兼容模型是庞大 CUDA 生态系统的基础，使你能够用单一代码库同时面向旧硬件和最前沿硬件。

> **注** 若要真正维护当前和未来 GPU 代际之间的向后兼容与向前兼容，应使用通用目标进行编译，或明确包含 PTX。当需要利用较新硬件特性的特定优化时，可以使用代际特定目标。这样做时，务必为其他架构提供回退路径。

6.3 CUDA 编程复习

在 CUDA C++ 中，通过编写 kernel 定义并行工作。kernel 是带有 `__global__` 注解、在 GPU 设备上执行的特殊函数。从 CPU（主机）代码调用 kernel 时，使用 `<<< >>>` “尖括号”语法，通过两个配置参数指定要运行的线程数及其组织方式：`blocksPerGrid` 表示线程块数量，`threadsPerBlock` 表示每个线程块中的线程数量。

下面的简单示例展示了 CUDA kernel 和 kernel 启动的关键组成部分。这个 kernel 原地将输入数组中的每个元素加倍，因此不会创建额外内存，只使用输入数组。CUDA 会在幕后将 `__global__` 函数编译成 GPU 设备代码，使成千上万或数百万个轻量级线程能够并行执行：

```
//-------------------------------------------------------
// Kernel: myKernel running on the device (GPU)
//   - input : device pointer to float array of length N
//   - N   : total number of elements in the input
//-------------------------------------------------------
__global__ void myKernel(float* input, int N) {
    // Compute a unique global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    // Only process valid elements
    if (idx < N) {
        input[idx] *= 2.0f;
    }
}
// This code runs on the host (CPU)
int main() {
    // 1) Problem size: one million floats
    const int N = 1'000'000;
    float *h_input = nullptr;
    float *d_input = nullptr;
    // 1) Allocate input float array of size N on host
    cudaMallocHost(&h_input, N * sizeof(float));
    // 2) Initialize host data (for example, all ones)
    for (int i = 0; i < N; ++i) {
        h_input[i] = 1.0f;
    }
    // 3) Allocate device memory for input on the device
    cudaMalloc(&d_input, N * sizeof(float));
    // 4) Copy data from the host to the device
    cudaMemcpy(d_input, h_input, N * sizeof(float),
      cudaMemcpyHostToDevice);
    // 5) Choose kernel launch parameters

    // Number of threads per block (multiple of 32)
    const int threadsPerBlock = 256;
    // Number of blocks per grid (3,907 for N = 1000000)
    const int blocksPerGrid = (N + threadsPerBlock - 1) /
```

```
      threadsPerBlock;
    // 6) Launch myKernel across blocksPerGrid blocks
    // Each block has threadsPerBlock number of threads
    // Pass a reference to the d_input device array
    myKernel<<<blocksPerGrid, threadsPerBlock>>>(d_input,
      N);
    // 7) Wait for the kernel to finish running on device
    cudaDeviceSynchronize();
    // 8) When finished, copy the results
    //    (stored in d_input) from the device back to
    //     host (stored in h_input)
    cudaMemcpy(h_input, d_input, N * sizeof(float),
               cudaMemcpyDeviceToHost);
    // Cleanup: Free memory on the device and host
    cudaFree(d_input);
    cudaFreeHost(h_input);
    // return 0 for success!
    return 0;
}
```

这段代码尚未完全优化。随着本书继续展开，我们会进一步优化性能。但它提供了一个简单、完整的模板，可作为开始构建自有 CUDA kernel 的起点。

这里传入了 kernel 输入参数 `d_input` 和 `N`，它们可以在 kernel 函数内部访问并用于处理。处理工作以并行方式分摊给许多线程。这正是设计目标。

完整的数据流如下：

1. 在主机上分配内存（`h_input`）。

2. 使用带有 `cudaMemcpyHostToDevice` 的 `cudaMemcpy`，将数据从主机（`h_input`）复制到设备（`d_input`）。

3. 在设备上使用 `d_input` 运行 kernel。

4. 进行同步，确保 kernel 已在设备上执行完毕。

5. 使用带有 `cudaMemcpyDeviceToHost` 的 `cudaMemcpy`，将结果（`d_input`）从设备传输到主机（`h_input`）。

6. 使用 cudaFree 和 cudaFreeHost 清理设备和主机上的内存。

启动时还可以通过 <<< >>> 向 kernel 传入额外的高级 CUDA 特定参数，包括共享内存大小等；但两个核心启动参数 blocksPerGrid 和 threadsPerBlock 是任何 CUDA kernel 调用的基础。下一节将讨论如何选择这些启动参数的值。

你可能会想，为什么必须传入输入数组大小 N？kernel 应该能够检查数组大小，这似乎是多余的。然而，这正是 GPU CUDA kernel 函数与典型 CPU 函数的核心差异：CUDA kernel 函数被设计为在单个线程中运行，与成千上万个其他线程一起处理输入数据的一个分区。因此，N 定义了该特定 kernel 将处理的分区大小。

结合内置 kernel 变量 blockDim（本例中为 1，因为传入的是一维输入数组）、blockIdx 和 threadIdx，kernel 计算输入数组中的具体 idx。这个唯一的 idx 使 kernel 能够在跨越多个不同 SM 同时运行的许多线程中，并行、清晰且唯一地处理输入数组的每个元素。

注意边界检查 if (idx < N)。由于 N 可能不是块大小的整数倍，因此需要它来避免越界访问（边界检查）。例如，假设输入数组大小为 63，即 N = 63。warp 调度器可能会分配两个 warp（每个 32 个线程）来处理输入数组中的 63 个元素。

第一个 warp 会同时运行 kernel 的 32 个实例来处理元素 0-31，绝不会超过 N = 63，这很直接。第二个 warp 与第一个 warp 并行运行，预计处理元素 32-64。不过，当到达 N = 63 时，它会停止。

如果没有 if (idx < N) 边界检查，第二个 warp 会尝试处理 idx = 64，并抛出非法内存访问错误（例如 cudaErrorIllegalAddress）。边界检查确保每个线程要么处理有效输入元素，要么在其 idx 越界时立即退出。

CUDA kernel 在设备上异步执行，不提供每线程异常；相反，任何非法操作（越界访问、未对齐访问等）都会为整个启动设置一个全局故障标志。主机驱动只会在下一次调用同步函数或其他 CUDA API 函数时检查该标志，因此错误会延迟暴露（例如表现为 cudaErrorIllegalAddress 或通用的启动失败）。

这种设计使 GPU 流水线和互连保持充分占用，但要求你在主机端显式同步并轮询错误，通常是在 kernel 启动后立即调用 cudaGetLastError() 和 cudaDeviceSynchronize()。这样可以在故障发生时尽早捕获。

你会在许多 CUDA kernel 中看到边界检查。如果看不到，应理解为什么没有它。它很可能以某种形式存在，或者 CUDA kernel 开发者能够以某种方式保证不会发生非法内存访问错误。

最后是实际的 kernel 逻辑。在输入数组中计算出唯一索引 `idx` 后，这个 kernel（在许多 SM 上由成千上万个线程并行、分别运行）将输入数组中索引为 `idx` 的值乘以 2，然后原地更新输入数组中的该值。在这个特定 kernel 中，除了 `int` 类型的临时变量 `idx` 外，不需要额外内存。

6.4 配置启动参数：每个网格的线程块数与每个线程块的线程数

如前所述，使用 warp 大小（32）的整数倍作为线程块大小至关重要。`threadsPerBlock` 取 256（8 个 warp）是一个常见起点，可平衡占用率和资源使用。这有助于避免 kernel 执行期间出现未填满的 warp、隐藏延迟，并平衡 SM 和其他硬件资源：

6.4.1 32 个线程的整数倍

选择 32 个线程的整数倍作为线程块大小，有助于避免空的 warp 槽位。否则，未填满的 warp 会占用稀缺的调度器资源，却不贡献有用工作。

6.4.2 隐藏延迟

每个 SM 需要数百个线程来隐藏 DRAM 和指令延迟造成的停顿。例如，在容量为 2,048 个线程的 SM 上启动 8 个 256 线程块，可以在不过度超额订阅的情况下保持流水线繁忙。

6.4.3 占用率

例如，使用 256 个 `threadsPerBlock` 时，每个线程块只需要 8 个 warp。这通常能够在不耗尽每个线程块的寄存器或共享内存的情况下实现良好占用率。

注 对于 Blackwell 等现代 GPU，可以考虑每个线程块使用 256–512 个线程，以最大化占用率，同时遵守寄存器和共享内存限制。

6.4.4 资源平衡

256 足够小，几乎不会超过每个线程块 1,024 个线程的限制；同时又足够大，不会因为其他 warp 中的线程停顿而让过多 warp 闲置。

从 `threadsPerBlock=256` 开始，可以根据 kernel 的寄存器和共享内存需求，以及占用率特征，将其向上或向下调节（例如 128、512 等）。

对于 `blocksPerGrid`，可以根据输入元素数量 `N` 和 `threadsPerBlock` 的值来确定。例如，通常将 `blocksPerGrid` 设置为 `(N + threadsPerBlock - 1) / threadsPerBlock`，以向上取整；这样当 `N` 不是 `threadsPerBlock` 的整数倍时，仍可覆盖所有元素。这是一个常见选择，能够保证每个输入元素都由一个线程覆盖。下面的代码展示了这种计算：

```
//------------------------------------------------------
// Kernel: myKernel running on the device (GPU)
//   - input : device pointer to float array of length N
//   - N   : total number of elements in the input
//------------------------------------------------------
__global__ void myKernel(float* input, int N) {
    // Compute a unique global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    // Only process valid elements
    if (idx < N) {
        input[idx] *= 2.0f;
    }
}
// This code runs on the host (CPU)
int main() {
    // 1) Problem size: one million floats
    const int N = 1'000'000;
    float* h_input = nullptr;
    cudaMallocHost(&h_input, N * sizeof(float));
```

```
    // Initialize host data (for example, all ones)
    for (int i = 0; i < N; ++i) {
        h_input[i] = 1.0f;
    }
    // Allocate device memory for input on the device (d_)
    float* d_input = nullptr;
    cudaMalloc(&d_input, N * sizeof(float));
    // Copy data from the host to the device using cudaMemcpyHostToDevice
    cudaMemcpy(d_input, h_input, N * sizeof(float),
      cudaMemcpyHostToDevice);
    // 2) Tune launch parameters
    const int threadsPerBlock = 256; // multiple of 32
    const int blocksPerGrid = (N + threadsPerBlock - 1) /
      threadsPerBlock; // 3,907, in this case
    // Launch myKernel across blocksPerGrid number of blocks
    // Each block has threadsPerBlock number of threads
    // Pass a reference to the d_input device array
    myKernel<<<blocksPerGrid, threadsPerBlock>>>(d_input, N);
    // Wait for the kernel to finish running on the device
    cudaDeviceSynchronize();
    // When finished, copy results (stored in d_input) from device to
     ↪ host
    // (stored in h_input) using cudaMemcpyDeviceToHost
    cudaMemcpy(h_input, d_input, N * sizeof(float),
     ↪ cudaMemcpyDeviceToHost);
    // Cleanup: Free memory on the device and host
    cudaFree(d_input);
    cudaFreeHost(h_input);
    return 0; // return 0 for success!
}
```

这与前面的 kernel 相同，但会根据 N 的大小动态计算 `blocksPerGrid` 和 `threadsPerBlock`。注意其中熟悉的 `if (idx < N)` 边界检查。它确保最后一个线程块中落在 N 之外的任何“多余”线程都什么也不做，从而不会造成非法内存地址错误。接下来，我们探索二维图像和三维体数据等多维输入。

6.4.5 二维和三维 kernel 输入

当输入数据天然存在于二维空间（例如图像）中时，可以启动二维线程块组成的二维网格。例如，下面的 kernel 使用 16 × 16 维线程块（总计 256 个线程），处理一个二维的 1,024 × 1,024 矩阵：

```
// 2d_kernel.cu
#include <cuda_runtime.h>
#include <iostream>
//------------------------------------------------------
// Kernel: my2DKernel running on the device (GPU)
//   - input  : device pointer to float array of size width×height
//   - width  : number of columns
//   - height : number of rows
//------------------------------------------------------
__global__ void my2DKernel(float* input, int width, int height) {
    // Compute 2D thread coordinates
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    // Only process valid pixels
    if (x < width && y < height) {
        int idx = y * width + x;
        input[idx] *= 2.0f;
    }
}
int main() {
    // Image dimensions
    const int width  = 1024;
    const int height = 1024;
    const int N      = width * height;
    // 1) Allocate and initialize host image
    float* h_image = nullptr;
    cudaMallocHost(&h_image, N * sizeof(float));
    for (int i = 0; i < N; ++i) {
        h_image[i] = 1.0f;  // e.g., initialize all pixels to 1.0f
    }
```

```
// 2) Allocate device image and copy data to device
cudaStream_t s; cudaStreamCreateWithFlags(&s,
↪  cudaStreamNonBlocking);
float* d_image = nullptr;
cudaMallocAsync(&d_image, N * sizeof(float), s);
cudaMemcpyAsync(d_image, h_image, N * sizeof(float),
                cudaMemcpyHostToDevice, s);
my2DKernel<<<blocksPerGrid2D, threadsPerBlock2D,
             0, s>>>(d_image, width, height);
cudaMemcpyAsync(h_image, d_image, N * sizeof(float),
                cudaMemcpyDeviceToHost, s);
cudaStreamSynchronize(s);
cudaFreeAsync(d_image, s);
cudaStreamDestroy(s);
```

这里再次给出完整的 kernel（设备）和调用（主机）代码。同一模式可以推广到三维：对 `blocksPerGrid` 和 `threadsPerBlock` 都使用 `dim3(x, y, z)`，即可将体数据直接映射到 GPU 的线程层级。

> **注** 本书大部分内容对 `blocksPerGrid` 和 `threadsPerBlock` 使用一维或二维（分块）值。在一维情况下，可以将它们定义为简单常量，而不是 `dim3`。

6.5 异步内存分配与内存池

如前面示例所示，标准的 `cudaMalloc/cudaFree` 调用是同步的，成本相对较高。它们要求设备进行完整同步（相对较慢），并涉及 `mmap/ioctl` 等操作系统级调用来管理 GPU 内存。

这种操作系统级交互会产生内核态上下文切换和驱动开销，因此与纯设备端操作相比相对缓慢。因此，建议使用异步版本 `cudaMallocAsync` 和 `cudaFreeAsync`，以更高效地在 GPU 上分配内存。

默认情况下，CUDA 运行时维护一个全局 GPU 内存池。异步释放内存时，内存会回到该池中，以便后续分配复用。`cudaMallocAsync` 和 `cudaFreeAsync` 在底层使用 CUDA 内存池。

内存池会回收已释放的内存缓冲区，避免重复调用操作系统来分配新内存。例如，在长时间运行的训练循环中，它可以通过复用先前释放的块，而不是每次迭代都创建新块，减少随时间产生的内存碎片。PyTorch 等高性能库和运行时通常默认启用内存池。

实际上，PyTorch 使用一个自定义的内存缓存分配器，通过 `PYTORCH_ALLOC_CONF`（以前称为 `PYTORCH_CUDA_ALLOC_CONF`）进行配置。PyTorch 内存缓存分配器与 CUDA 内存池的理念类似：它复用 GPU 内存，避免在长时间运行的训练循环的每次迭代中为每个新建的 PyTorch 张量调用同步 `cudaMalloc` 的成本。

在频繁进行细粒度分配的 CUDA 应用中，使用基于异步内存池的例程——`cudaMallocAsync` 和 `cudaFreeAsync`——远比传统的同步 `cudaMalloc/cudaFree` 高效；后者会引起全设备同步，甚至触发操作系统级调用。要使用按流排序的分配，先创建一个非阻塞流：

```
cudaStream_t stream1;
cudaStreamCreateWithFlags(&stream1, cudaStreamNonBlocking);
```

> **注** 使用显式 CUDA 流是重叠传输、kernel 和内存操作的最佳实践。可以把每个流看成一个隔离通道，它保证自身操作之间的顺序。此外，建议使用 `cudaStreamCreateWithFlags(..., cudaStreamNonBlocking)` 创建非阻塞流，以避免传统默认流屏障。第 11 章将更详细地讨论多流重叠技术和最佳实践。

随后，每当需要一个包含 `N` 个 float 的缓冲区时，就在该流上使用 `cudaMallocAsync` 和 `cudaFreeAsync` 分配并释放，如下所示：

```
float* d_buf = nullptr;
cudaMallocAsync(&d_buf, N * sizeof(float), stream1);
// ... launch kernels into stream1 that use d_buf ...
myKernel<<<blocksPerGrid, threadsPerBlock, 0, stream1>>>(d_buf, N);
// Free is deferred until all work in stream1 completes—
cudaFreeAsync(d_buf, stream1);
```

这些 API 从每个设备的内存池分配内存，但会遵守传入流的顺序，因此释放操作会延迟到该流的工作完成之后。由于 cudaFreeAsync 只等待 stream1 完成，不会产生昂贵的全局 cudaDeviceSynchronize，也不会与其他流发生隐式同步。当代码发出数千甚至数百万次分配/释放操作时，这一结果会显著降低分配开销，减少碎片，并平滑延迟尖峰。总体而言，与传统的 cudaMalloc 和 cudaFree 相比，这种模式减少了全局同步和碎片。

你还可以进一步调节设备内存池中按流排序的分配行为，例如设置 cudaMemPoolAttrReleaseThreshold，提示内存池在尝试释放之前应保留多少预留内存。还可以使用 cudaMemPoolTrimTo 主动归还内存。这些设置有助于在 GPU 总内存占用与碎片之间取得平衡。

对于简单的一次性缓冲区，阻塞式 cudaMalloc 和 cudaFree 可能已经足够。然而，在反复分配和释放内存的复杂长时间运行循环中，切换到专用流上的 cudaMallocAsync 和 cudaFreeAsync 并利用其内存池，将带来更一致的性能和更高的吞吐量。

在专用流上切换到 cudaMallocAsync 和 cudaFreeAsync 并利用其内存池，将带来更一致的性能和更高的吞吐量。还可以使用 cudaMemPoolSetAttribute（例如调整 cudaMemPoolAttrReleaseThreshold）进一步调节内存池行为，从而调节释放阈值，在最小内存占用和低碎片之间取得合适的权衡。

6.6 理解 GPU 内存层次结构

到目前为止，我们一直在较高层次上讨论内存分配，并通常将其视为来自全局内存。这些分配来自某个流的内存池，包括默认流 0 的内存池。

不过，实际上 GPU 提供了多级内存层次结构，用于平衡容量与速度。该层次结构包括寄存器、共享内存、缓存、全局内存，以及 Blackwell GPU 及后续 GPU 上的专用 TMEM。稍后将更详细地讨论 TMEM：它是每个 SM 约 256 KB 的专用片上内存，供 Blackwell 第 5 代 Tensor Core 指令（`tcgen05.*`）使用。它不能从 CUDA C++ 直接通过指针寻址。相反，数据移动由 TMA 硬件（全局内存 ↔ SMEM）以及 `tcgen05` Tensor Core 数据移动指令负责（SMEM ↔ TMEM 的移动通过张量描述符隐式完成）。

全局内存（HBM 或 DRAM）容量大、位于片外且相对缓慢。寄存器容量很小、位于片上且极快。L1 缓存、L2 缓存和共享内存的性能处于两者之间。缓存和共享内存的作用，是隐藏访问大型片外内存存储的相对较长延迟。图 6-10 展示了 GPU 内存层次结构（包括 CPU）的高层视图。

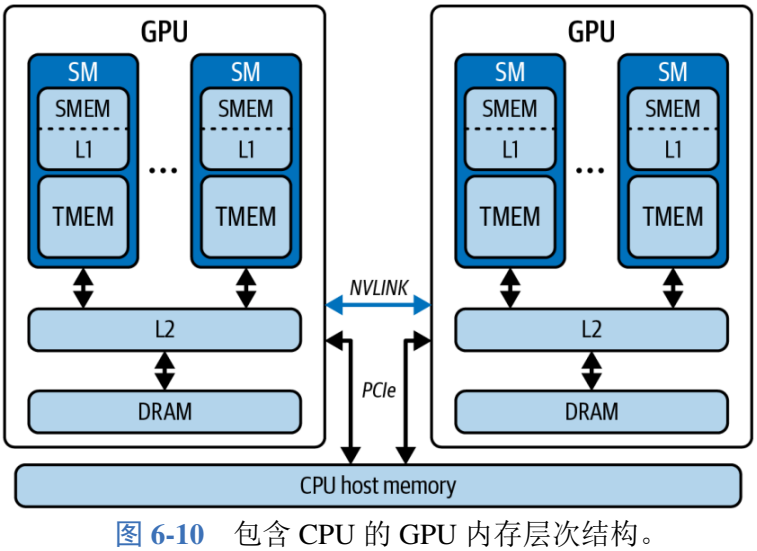

图 6-10　包含 CPU 的 GPU 内存层次结构。

TMEM 是每个 SM 专用的 256 KB 缓冲区，以每秒数十 TB 的带宽透明地与 Tensor Core 通信。这减少了 Tensor Core 对全局内存的依赖。图 6-11 展示了 TMEM 与 SMEM 一起为 Tensor Core 提供服务，以计算 $C = AimesB$ 矩阵乘法。

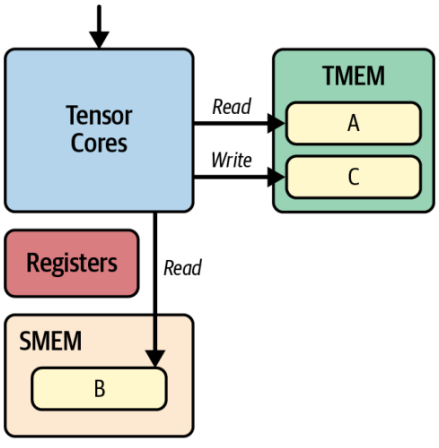

图 6-11　TMEM 和 SMEM 为 Tensor Core 提供服务，以计算 $C = AimesB$ 矩阵乘法。

这里，操作数 B 来自 SMEM。操作数 A 位于 TMEM 中（但也可能位于 SMEM 中）。累加器同样位于 TMEM 中。分块数据通过 TMA（例如 `cuda::memcpy_async`）经由 L2 缓存从全局内存流入 SMEM。

操作数通过 UMMA（统一矩阵乘加，unified matrix-multiply-accumulate）和 `tcgen05.mma` 等 Tensor Core 指令，在 SMEM 与 TMEM 之间隐式移动。

表 6-5 展示了 Blackwell GPU 的不同内存层级及其特征。下面将依次描述内存层次结构的每一级。

表 6-5　Blackwell 内存层级及其特性。

层级	作用域	容量	延迟	带宽（约）
寄存器	每线程（位于 SM 上）	每个 SM 有 64K 个 32 位寄存器（每线程最多 255 个）	接近寄存器的延迟（读/写为单周期，基本免费）	每个 SM 数十 TB/s（寄存器文件端口）
共享内存和 L1 缓存	每个 SM	228 KB（可用 227 KB）共享内存；其余部分作为 L1/数据缓存	约 20–30 个周期（L1/共享内存基准）	每个 SM 达 TB/s（无 bank 冲突时）

TMEM	每个 SM	每个 SM 256 KB SRAM，专用于 Tensor Core	约 10 个周期（SM 上的专用 SRAM）	与 Tensor Core 之间为 TB/s 级通信
常量内存缓存	每个 SM	约 8 KB 缓存，面向 64 KB 的 `__constant_` 空间	约 1 个周期（warp 广播）。当缓存命中且 warp 中所有线程访问相同地址时，由于常量缓存和广播行为，速度与寄存器一样快；分歧或未命中会串行化或带来更高延迟	TB/s 级（广播吞吐量）
L2 缓存	全 GPU（所有 SM）	总计 126 MB	约 200 个周期	聚合多 TB/s
本地内存	每线程（溢出到 DRAM）	近乎无限（由全局内存支持）	数百到 1,000 个周期（类似 DRAM）；HBM3e 约 8 TB/s	约 8 TB/s（HBM3e）
全局内存（HBM 或 DRAM）	设备级（片外 DRAM）	每个 Blackwell B200 GPU 最多 180 GB（每个 Blackwell B300 GPU 最多约 288 GB）	数百到 1,000 个周期（全局内存延迟）	总计约 8 TB/s

由此可以看出，要实现高吞吐量 GPU kernel，必须最大化寄存器、共享内存以及 L1/L2 缓存中的数据复用，并尽量减少对全局内存和本地内存（由全局内存支持）的依赖。下面进一步介绍这些层级：

6.6.1 寄存器

在 Blackwell 上，每个线程都从寄存器文件开始其数据之旅。寄存器文件是 SM 上很小的 SRAM 阵列，用于保存每个线程的局部变量，额外延迟几乎为零。每个 SM 包含 64K 个 32 位寄存器（总计 256 KB），但硬件对每个线程最多暴露 255 个寄存器。

由于读写在一个周期内完成，且几乎不与其他资源争用，寄存器带宽每个 SM 可达数十 TB/s。不过，如果 kernel 需要更多寄存器——无论是由于大量线程局部变量，还是由于编译器临时变量——溢出内容就会进入本地内存；本地内存映射到片外 DRAM，会产生数百到超过 1,000 个周期的延迟。图 6-12 展示了这种本地内存。

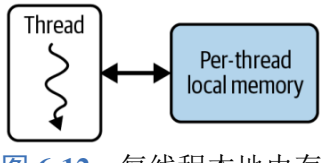

图 6-12　每线程本地内存。

6.6.2 共享内存和 L1 数据缓存

再上一级是统一的 L1/数据缓存和共享内存块。这是每个 SM 上 256 KB 的 SRAM，可以在用户管理的共享内存（每个线程块最多 228 KB）与缓存之间动态划分；在 Blackwell 等采用统一 L1/纹理/共享内存的架构上，可使用 `cudaFuncSetAttribute()` 配合 `cudaFuncAttributePreferredSharedMemoryCarveout` 选择内存划分。每个线程块的最大动态共享内存为 227 KB（CUDA 为每个线程块保留 1 KB），每个 SM 可分配的共享内存总量也受此限制约束。

这里的访问成本约为 20–30 个周期，但如果设计线程块以避免 bank 冲突，就可以达到 TB/s 级吞吐量。图 6-13 展示了线程块共享内存。

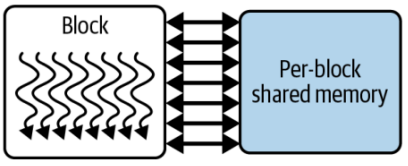

图 6-13　线程块共享内存。

6.6.3 TMEM

TMEM 是每个 SM 上的专用片上内存（Blackwell 上为 256 KB），供 Tensor Core 特定操作和指令使用，包括统一矩阵乘加和 `tcgen05`；第 10 章将讨论这些内容。它不是 CUDA C++ 中可通过普通指针寻址的空间。相反，传输由张量内存加速器（TMA）使用描述符来调度。这使开发者不必手动管理 Tensor Core 的数据流。例如，一些算术操作数位于共享内存中，而累加器位于 TMEM 中。随后，TMA 负责在全局内存、共享内存和 TMEM 内存之间移动数据，以执行计算。

6.6.4 常量内存缓存

对于很小的只读表，Blackwell 在 64 KB 的 `__constant__` 空间前端提供了每个 SM 约 8 KB 的常量内存缓存。当 warp 中全部 32 个线程加载相同地址时，该缓存可以在一个周期内广播该值。

分歧读取会在各 lane 之间串行化。它非常适合共享旋转位置编码、带线性偏置的注意力（Attention with Linear Biases，ALiBi）的斜率、LayerNorm 的 γ/β 向量以及嵌入量化的缩放因子等小型查找表。这些数据可以在所有线程之间共享，而无需产生全局内存流量。

6.6.5 L2 缓存

超出片上 SRAM 后是 L2 缓存，这是一个容量为 126 MB、覆盖整个 GPU 的缓冲区，将所有 SM 与片外 HBM3e 连接起来。其延迟接近 200 个周期，聚合带宽达数十 TB/s；L2 可以吸收来自 L1 的溢出流量。

借助 L2，一个线程块获取的数据可以被其他线程块复用，而无需再次访问 DRAM。为了最大化 L2 的收益，应将全局加载组织为 128 字节、合并的事务，使其整齐地映射到缓存行。稍后将展示如何实现。

> **注** 将全局加载组织为映射到缓存行的 128 字节对齐、合并的片段。这样可以避免拆分事务，并最大化 L2 和 DRAM 带宽的利用。

6.6.6 全局内存（HBM 或 DRAM）

全局内存层、本地溢出空间和 HBM 都位于片外。任何溢出的寄存器或过大的自动数组都会驻留在本地内存中，即使 HBM3e 带宽约为 8 TB/s，也必须承担完整的 DRAM 延迟（数百到超过 1,000 个周期）。

对于 Blackwell，HBM3e 层提供最多 180 GB 的设备级存储，总带宽约为 8 TB/s。不过，由于延迟很高，它是链路中最慢的一环。图 6-14 展示了每设备的全局内存。

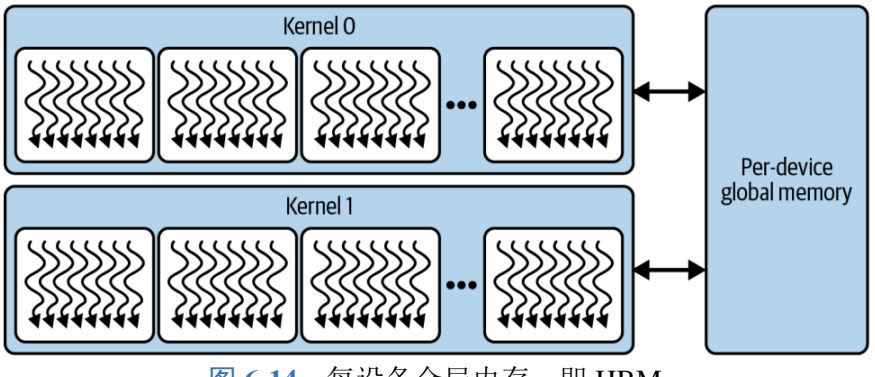

图 6-14 每设备全局内存，即 HBM。

使用 Nsight Compute 等工具跟踪溢出和缓存命中率，可以让 kernel 尽可能接近内存层次结构的片上峰值。这些工具可以帮助你有效地编排寄存器、共享内存/L1、常量缓存和 L2 缓存中的数据。Blackwell 等现代 GPU 允许 kernel 开发者利用 L2 缓存和统一的 L1/共享内存来缓冲并合并访问 HBM，稍后将会看到这一点。

> **注** Blackwell B200 表现为一个具有统一全局地址空间的单一 GPU。不过，它由两个受光罩版图限制的裸片组成，二者通过 10 TB/s 的芯片间互连连

接。每个裸片连接 4 个 HBM3e 堆栈，总计 8 个 HBM3e 堆栈。从开发者角度看，在这个组合地址空间中，HBM 内存访问是一致的，但理解该架构的底层细节仍然很有价值。

不同内存层级的内存一致性点（point of coherency，PoC）取决于你的需求以及线程通信所在的层级。它通常位于以下层级之一：线程、线程块（也称 CTA）、线程块集群（也称 CTA 集群）、设备或系统，如图 6-15 所示。

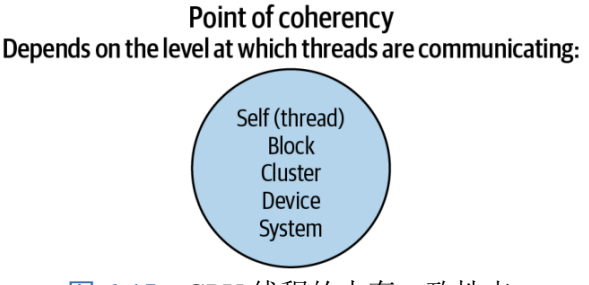

图 6-15　GPU 线程的内存一致性点。

总之，理解 GPU 内存层次结构并恰当地面向每个层级进行优化非常重要。这样可以组织 CUDA kernel 以最大化数据局部性、隐藏内存访问延迟、提高占用率，并充分利用 Blackwell 强大的并行计算能力，稍后我们将对此展开。首先讨论 NVIDIA 的统一内存；鉴于 Grace Hopper 和 Grace Blackwell 的 CPU-GPU 统一超级芯片设计，理解统一内存十分重要。

6.7 统一内存

统一内存（也称 CUDA Managed Memory）提供了一个跨越 CPU 和 GPU 的单一一致地址空间，因此不再需要管理独立的主机和设备缓冲区，也不再需要显式调用 `cudaMemcpy`。在底层，CUDA 运行时会将每次 `cudaMallocManaged()` 分配支持为页面，这些页面可以按需通过连接 CPU 和 GPU 的互连迁移，如图 6-16 所示。

虽然访问统一内存对开发者非常友好，但它可能导致 CPU 与 GPU 之间不希望发生的按需页面迁移，从而引入隐藏延迟和执行停顿。例如，如果 GPU 线程访问当前位于 CPU 内存中的数据，GPU 会发生缺页，并在数据通过 NVLink-C2C 互连传输时等待。统一内存的性能高度依赖底层硬件。

在传统 PCIe 或早期 NVLink 系统上，这些迁移以相对较低的带宽进行，按缺页触发的传输通常比手动 cudaMemcpy 更慢。但在 Grace Hopper 和 Grace Blackwell Superchip 上，NVLink-C2C 结构可在 CPU 的 HBM 与 GPU 的 HBM3e 之间提供最高约 900 GB/s 的带宽。因此，由缺页驱动的迁移速度更接近设备原生速度，尽管仍会产生非零延迟。

也就是说，kernel 启动期间任何意外缺页都会在运行时将所需页面移动到位时使 GPU 停顿。为了避免这些“意外”停顿，可以提前使用 cudaMemPrefetchAsync() 预取内存，如图 6-17 所示。

这会提示驱动在启动 kernel 之前，将指定范围移动到目标 GPU（或 CPU）上，把代价高昂的首次触碰迁移转换为可与其他工作重叠的异步传输。还可以像下面的代码一样提供内存建议：

```
cudaMemAdvise(ptr, size, cudaMemAdviseSetPreferredLocation, gpuId);
cudaMemAdvise(ptr, size, cudaMemAdviseSetReadMostly, gpuId);
```

这里可以使用 PreferredLocation 告诉驱动数据主要在哪里使用，并在数据大多为只读时使用 ReadMostly，如图 6-18 所示。

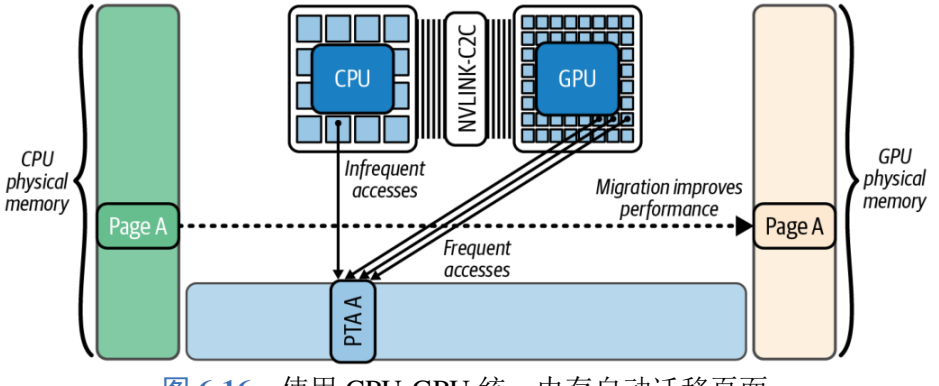

图 6-16　使用 CPU-GPU 统一内存自动迁移页面。

还可以调用下面的函数，让第二个 GPU 映射这些页面，而不会在启动时触发迁移：

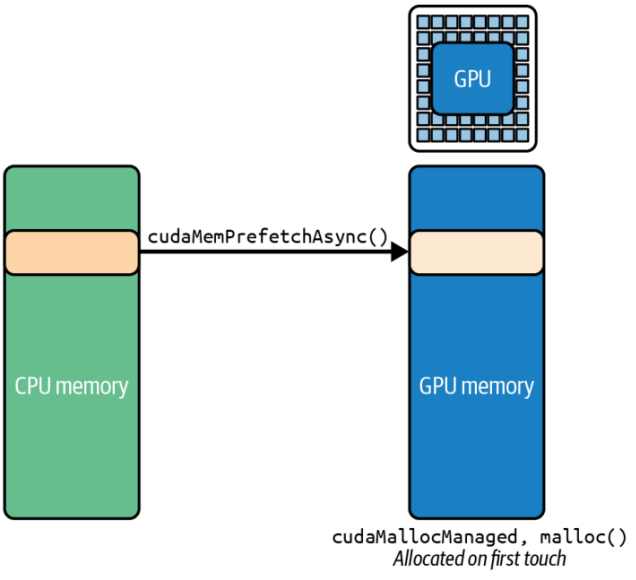

图 6-17　通过 NVLink-C2C 使用 `cudaMemPrefetchAsync()` 将数据从 CPU 流式传输到 GPU。

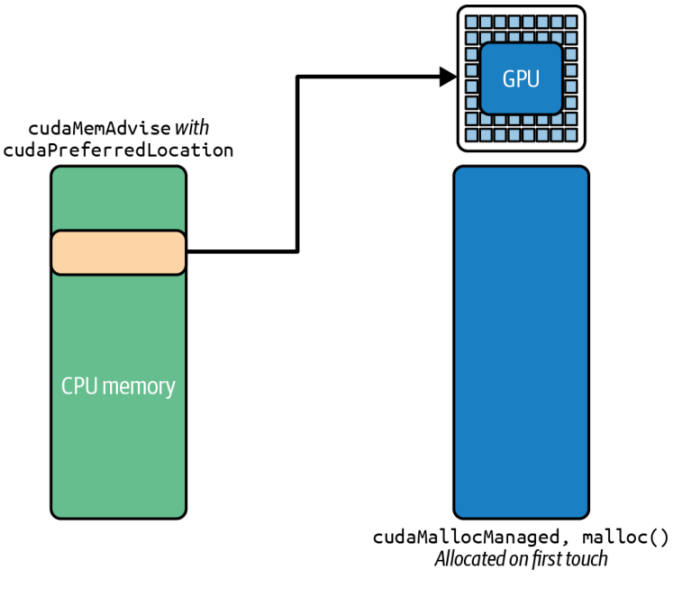

图 6-18　指定“首选位置”，告知 CUDA 驱动数据的主要使用方式（例如对大多只读的工作负载使用 ReadMostly）

```
cudaMemAdvise(ptr, size, cudaMemAdviseSetAccessedBy,
   otherGpuId);
```

默认情况下，任何 CUDA 流或设备 kernel 都可以在托管分配上触发缺页。这可能造成意外迁移和隐式同步。如果知道某个缓冲区一次只会在一个流/GPU 中使用，那么将其附加到该流，可以让迁移与其他流中的操作重叠。调用下面的函数可以将该内存范围绑定到指定流：

```
cudaStreamAttachMemAsync(stream, ptr, 0,
    cudaMemAttachSingle);
```

在这种情况下，只有该流中的操作会使页面发生缺页并迁移。这可以防止其他流意外因其停顿。因此，将一个范围附加到特定流会推迟其迁移，使迁移只与该流的工作重叠，从而避免跨流同步。

> **注** 在没有 NVLink-C2C 的多 GPU 系统中，还可以使用 `cudaMemcpyPeerAsync()`，或预取到特定设备，将数据固定在最近的 NUMA 本地 GPU 内存中，从而避免缓慢的远程访问。

简而言之，显式预取托管内存并提供内存建议，可以消除统一内存的大多数“意外”停顿。GPU 不再需要暂停并按需获取数据；kernel 运行时，数据已经位于所需位置。

借助主动预取、定向内存建议和流附加等技术，统一内存可以在保留统一地址空间简洁性的同时，提供非常接近手动 `cudaMemcpy` 的性能。

6.8 保持高占用率和 GPU 利用率

GPU 通过并发运行许多 warp 来维持性能：当一个 warp 因等待数据而停顿时，另一个 warp 可以运行。快速在 warp 之间切换的能力使 GPU 能够隐藏内存延迟。如前面所述，SM 容量中实际被活动 warp 占据的比例称为占用率。

如果占用率较低（只有少数活动 warp），一个 warp 等待内存时，SM 可能处于空闲状态。这会导致 SM 利用率不佳。在 Blackwell 上，由于其较大的寄存

器文件（每个 SM 64K 个寄存器）可以在不溢出的情况下支持许多 warp，因此实现高占用率稍微容易一些。

> **注** 如前面所见，warp 中的每个线程最多可以使用 255 个寄存器。务必使用性能分析工具检查实际达到的占用率，并相应调节 kernel 的线程块大小和寄存器使用量。

相反，高占用率（每个 SM 有许多活动 warp）会使 GPU 计算单元保持繁忙：当一个 warp 等待内存访问时，其他 warp 会切换到 SM 上执行。这会掩盖长时间的内存访问延迟，通常称为隐藏延迟。

下面展示一个提高占用率，并最终提高 GPU 利用率、吞吐量和 kernel 性能的示例。这是 CUDA 性能优化最基本的规则之一：启动足够多的并行工作，以充分占用 GPU。

如果实际占用率（正在使用的硬件线程槽位所占比例）远低于 GPU 上限且性能不佳，第一种补救措施是增加并行度——使用更多线程块或线程，使占用率在现代 GPU 上接近 80%–100% 的范围。

相反，如果占用率已经中等或较高，但 kernel 受内存吞吐量瓶颈限制，将其推到 100% 可能没有帮助。通常只需要足够多的 warp 来隐藏延迟；超过这个数量后，瓶颈可能位于其他地方（例如内存带宽）。

为了说明占用率的影响，考虑一个非常简单的操作：将长度为 `N` 的两个向量相加（计算 $C = A + B$）。我们考察两个 kernel 实现：`addSequential` 和 `addParallel`。`addSequential` 使用一个线程（或一个 warp）在循环中将全部 `N` 个元素相加；`addParallel` 使用许多线程，使数组中的加法并发执行。

在顺序版本中，一个 GPU 线程串行处理全部工作，如下所示：

```
#include <cuda_runtime.h>
const int N = 1'000'000;
// Single thread does all N additions
__global__ void addSequential(const float* A,
                              const float* B,
                                    float* C,
                              int N)
{
    if (blockIdx.x == 0 && threadIdx.x == 0) {
        for (int i = 0; i < N; ++i) {
            C[i] = A[i] + B[i];
        }
    }
}
int main()
{
    // Allocate and initialize host
    float* h_A = nullptr;
    float* h_B = nullptr;
    float* h_C = nullptr;
    cudaMallocHost(&h_A, N * sizeof(float));
    cudaMallocHost(&h_B, N * sizeof(float));
    cudaMallocHost(&h_C, N * sizeof(float));
    for (int i = 0; i < N; ++i) {
        h_A[i] = float(i);
        h_B[i] = float(i * 2);
    }
    // Allocate device
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, N * sizeof(float));
    cudaMalloc(&d_B, N * sizeof(float));
    cudaMalloc(&d_C, N * sizeof(float));
    // Copy inputs to device
    cudaMemcpy(d_A, h_A, N * sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, N * sizeof(float), cudaMemcpyHostToDevice);
    // Launch: one thread
```

```
    // Note: This kernel assumes <<<1,1>>>
    // (one block, one thread).
    // Do not change the launch config when running this example.
    addSequential<<<1,1>>>(d_A, d_B, d_C, N);
    // Ensure completion before exit
    cudaDeviceSynchronize();
    // Copy d_C => h_C (back to host)
    cudaMemcpy(h_C, d_C, N * sizeof(float), cudaMemcpyDeviceToHost);
    // Cleanup
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    cudaFreeHost(h_A);
    cudaFreeHost(h_B);
    cudaFreeHost(h_C);
    return 0;
}
```

在这个单线程版本中，GPU 的庞大资源大多处于空闲状态。只有一个 warp，甚至只有 warp 内的一个线程在工作，其他线程全部闲置。因此占用率非常低，最终性能也很低。

还必须注意，避免在 PyTorch 等高级库和框架中间接执行低效 GPU 代码。例如，下面的朴素 PyTorch 代码错误地使用 Python `for` 循环执行逐元素操作，依次在 GPU 上发出 `N` 个独立的加法操作：

```
import torch
N = 1_000_000
A = torch.arange(N, dtype=torch.float32, device='cuda')
B = 2 * A
C = torch.empty_like(A)
# Ensure all previous work is done
torch.cuda.synchronize()
# Naive, Sequential GPU operations - DO NOT DO THIS
with torch.inference_mode(): # avoids unnecessary autograd graph
↪   construction
    # This launches N tiny GPU operations serially
    for i in range(N):
        C[i] = A[i] + B[i]
torch.cuda.synchronize()


# Naive, Sequential GPU operations - DO NOT DO THIS
with torch.inference_mode(): # avoids unnecessary autograd graph
↪   construction
    # This launches N tiny GPU operations serially
    for i in range(N):
        C[i] = A[i] + B[i]
torch.cuda.synchronize()
```

这段代码实际上把 GPU 当作标量的非并行处理器来使用，实现的占用率非常低，与前面原生 `addSequential` CUDA C++ 代码类似。

下面优化 CUDA kernel 和 PyTorch 代码，实现向量加法操作的并行版本。图 6-19 展示了向量化加法如何工作。

在下面的 CUDA C++ 代码中，我们启动足够多的线程来覆盖所有元素（`<<<(N+255)/256, 256 >>>`），使每个线程块中的 256 个线程在所需数量的线程块之间并行处理 `N` 个元素：

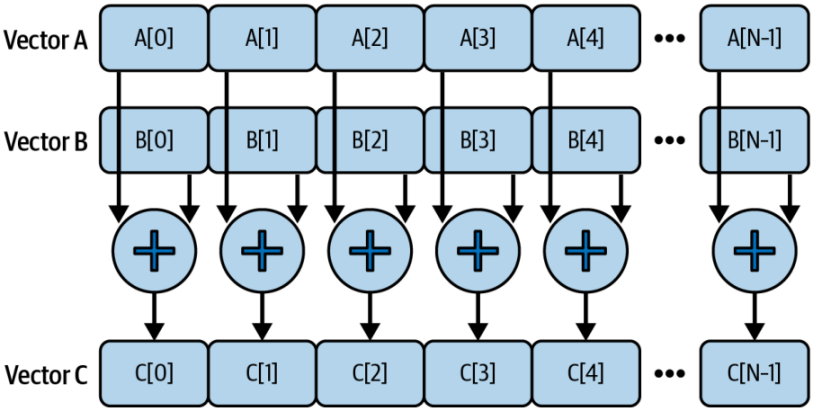

图 6-19　向量化加法在向量元素之间并行执行。

```
#include <cuda_runtime.h>
const int N = 1'000'000;
// One thread per element
__global__ void addParallel(const float* __restrict__ A,
                            const float* __restrict__ B,
                                  float* __restrict__ C,
                            int N)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}
int main()
{
  // Allocate and initialize host (pinned for faster DMA)
  float* h_A = nullptr;
  float* h_B = nullptr;
  float* h_C = nullptr;
  cudaMallocHost(&h_A, N * sizeof(float));
  cudaMallocHost(&h_B, N * sizeof(float));
  cudaMallocHost(&h_C, N * sizeof(float));
  for (int i = 0; i < N; ++i) { h_A[i] = float(i); h_B[i] = float(2*i); }
  // Create a non-blocking stream and allocate device buffers
```

```
  cudaStream_t s; cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking);
  float *d_A = nullptr, *d_B = nullptr, *d_C = nullptr;
  cudaMallocAsync(&d_A, N * sizeof(float), s);
  cudaMallocAsync(&d_B, N * sizeof(float), s);
  cudaMallocAsync(&d_C, N * sizeof(float), s);
  // Async HtoD copies on the same stream
  cudaMemcpyAsync(d_A, h_A, N*sizeof(float), cudaMemcpyHostToDevice, s);
  cudaMemcpyAsync(d_B, h_B, N*sizeof(float), cudaMemcpyHostToDevice, s);
  // Launch (same stream)
  int threads = 256;
  int blocks  = (N + threads - 1) / threads;
  addParallel<<<blocks, threads, 0, s>>>(d_A, d_B, d_C, N);
  // Async DtoH copy and stream sync
  cudaMemcpyAsync(h_C, d_C, N*sizeof(float), cudaMemcpyDeviceToHost, s);
  cudaStreamSynchronize(s);
  // Cleanup (stream-ordered free)
  cudaFreeAsync(d_A, s); cudaFreeAsync(d_B, s); cudaFreeAsync(d_C, s);
  cudaStreamDestroy(s);
  cudaFreeHost(h_A); cudaFreeHost(h_B); cudaFreeHost(h_C);
  return 0;
}
```

当 N 足够大时，GPU 利用率的差异非常显著。现在优化 PyTorch 代码：它启动一个单独的向量化 kernel（A + B），像前面优化的 addParallel CUDA C++ 示例一样，让 GPU 上的许多线程并发工作。下面是 PyTorch 代码的并行版本：

```
# add_parallel.py
import torch
N = 1_000_000
A = torch.arange(N, dtype=torch.float32, device='cuda')
B = 2 * A
torch.cuda.synchronize()
# Proper parallel approach using vectorized operation
# Launches a single GPU kernel that adds all elements in parallel
C = A + B
torch.cuda.synchronize()
```

实践中，当使用向量化张量操作时，PyTorch 等高级框架会做正确的事情。只需注意，在 GPU 操作外层引入 Python 级循环会将工作串行化并损害性能，应尽量避免。除非你正在编写全新的功能，否则几乎总能找到优化过的 PyTorch 原生实现，其中也包括由 PyTorch 编译器生成的代码。

为了量化并行实现与顺序实现之间的性能影响，可以使用 Nsight Systems 和 Nsight Compute 测量两种方法的 kernel 总执行时间、GPU 利用率、占用率和 warp 执行效率指标。下面是 Nsight Systems（`nsys`）和 Nsight Compute（`ncu`）命令：

```
# Sequential add
nsys profile \
  --stats=true \
  -t cuda,nvtx \
  -o sequential_nsys_report \
  ./add_sequential.py
ncu \
 --section SpeedOfLight \
 --metrics
     sm__warps_active.avg.pct_of_peak_sustained_active,gpu__time_durat
      ↪  ion.avg \
 --target-processes all \
 --print-summary per-gpu \
 -o sequential_ncu_report \
 ./add_sequential.py
```

```
# Parallel add
nsys profile \
  --stats=true \
  -t cuda,nvtx \
  -o parallel_nsys_report \
  ./add_parallel.py
ncu \
 --section SpeedOfLight \
 --metrics sm__warps_active.avg.pct_of_peak_sustained_active \
 --target-processes all \
 --print-summary per-gpu \
 -o parallel_ncu_report \
 ./add_parallel.py
```

我们使用 `nsys` 找出时间花在哪里，以及 GPU 是处于饥饿状态还是被阻塞。随后使用 `ncu` 解释 kernel 为什么以当前方式运行——例如是否由于占用率不足等原因。

如果只运行 `nsys`，可能会错过细粒度的 kernel 低效问题；如果只运行 `ncu`，则可能不知道 kernel 是否得到了足够快的数据供给。表 6-6 展示了统一结果。

表 6-6 顺序 CUDA kernel 与并行 CUDA kernel 对比。

指标	add_sequential	add_parallel
kernel 执行时间（ms）	48.21	2.17
GPU 利用率	1.5%	95%
实际达到的占用率	1.3%	38.7%
warp 执行效率	3.1%	100%

> **注** 其他性能分析工具对这些指标的命名可能不同。例如，Nsight Systems 报告总体“GPU 利用率”，而 Nsight Compute 提供每个 kernel 的“SM Active %”指标；但二者都反映 GPU 的 SM 被活动 warp 占用的程度。

正如预期，从单线程、单 warp 实现转为完全并行的多 warp 实现后，平均占用率从 1.3% 提高到约 38.7%。这使运行时间从 48.21 ms 降至 2.17 ms，减少约 22 倍。

在顺序情况下，只有一个 SM 上的一个 warp，且仅其中一个线程在工作。因此 GPU 利用率只有 1.5%；而在并行情况下，许多 SM 正在运行多个活动 warp。由于每条指令期间 warp 中的全部 32 个线程都在执行有用工作，warp 执行效率从 3.1% 提升至 100%。这使 GPU 利用率从 1.5% 提升至 95%。

这个例子展示了为什么 GPU 上充足的并行性至关重要。无论每个线程有多快，都需要大量线程才能发挥 GPU 的吞吐量潜力。

记住，GPU 是面向吞吐量优化的处理器，它与 CPU 交互以启动 CUDA kernel，同时还与内存子系统交互，以从缓存、共享内存和全局内存加载数据。因此，GPU 性能会从隐藏这些延迟中大幅受益。

如果 kernel 编写正确，就会指示 GPU 并行地交错执行来自不同 warp 的内存加载和计算（例如加法）。这有助于在 warp 之间隐藏内存延迟。

在多个 warp 中运行的并行 kernel 尤其受益于 warp 级延迟隐藏。当一个 warp 等待内存加载时，另一个 warp 可以执行加法计算，还有另一个 warp 可以获取下一批数据，依此类推。后续章节将探索多种隐藏内存延迟的技术。

在顺序 kernel 中，一个 warp 等待时没有其他 warp 可运行，因此硬件流水线经常处于空闲状态。时间线是一长串操作，在等待内存期间出现空闲间隔。在并行版本中，这些间隔由其他 warp 的工作填充，因此 GPU 持续繁忙。图 6-20 展示了这种比较。

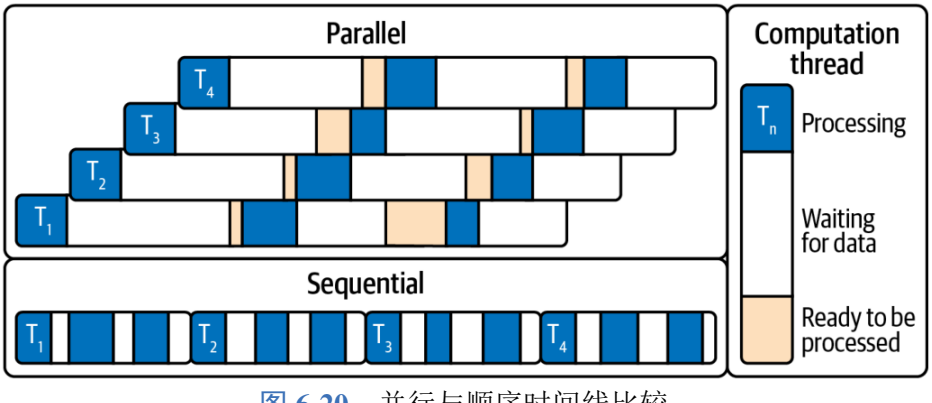

图 6-20　并行与顺序时间线比较。

这里，顺序时间线是一长串操作，在等待内存期间出现空闲间隔。在并行版本中，这些间隔由其他 warp 的工作填充，因此 GPU 持续繁忙。

关键结论是：首先确保有足够的并行工作来充分占用 GPU。高占用率——即有足够 warp 覆盖延迟——可以最大化吞吐量并最小化空闲停顿；在我们的例子中，并行化将 GPU 利用率提高到约 95%。

启动足够多的线程后，下一步是通过指令级并行性和其他每线程改进，提高每个 warp 的执行效率。但要注意，即使占用率达到 100%，如果工作负载是内存受限的——也就是受慢速内存访问而非计算限制——性能仍然可能不佳。

LLM 的“解码”阶段就是内存受限工作负载的一个典型例子。在解码过程中，LLM 需要将大量数据（模型权重或参数）从全局 HBM 内存移动到 GPU 寄存器和共享内存中。

由于现代 LLM 包含数千亿个参数，假设每个参数为 8 bit，即 1 字节，那么模型大小可能达到数百 GB。这么多数据进出 GPU，很容易使内存带宽饱和。

> **注** GPU FLOPS 的增长速度超过内存带宽。例如，Blackwell 的 HBM3e 可提供约 8 TB/s，但计算能力和模型规模增长得更快。因此，优化内存移动对于避免现代 AI 工作负载中的内存受限瓶颈至关重要。

6.9 使用启动边界调节占用率

在某些情况下，仅仅使用更多线程还不够，尤其是当每个线程使用大量寄存器和共享内存等资源时。我们可以使用 CUDA 的 `__launch_bounds__` kernel 注解，引导编译器针对占用率进行优化。

该注解允许我们在编译时为 kernel 指定两个参数：计划启动的每个线程块最大线程数，以及希望在每个 SM 上保持驻留的最小线程块数。这些提示会影响编译器的寄存器分配和函数内联决策。示例如下：

```
__global__ __launch_bounds__(256, 16)
void myKernel(...) { /* ... */ }
```

这里，`__launch_bounds__(256, 16)` 承诺 CUDA kernel 启动时每个线程块不会超过 256 个线程。同时，它请求编译器分配足够的寄存器并内联函数，使

每个 SM 能同时驻留至少 16 个 256 线程块，即 4,096 个线程（16 个线程块 × 每块 256 个线程）。

> **注** 记住，在现代 GPU（例如 Blackwell）上，每个线程块最多只能有 1,024 个线程，每个 SM 最多只能有 2,048 个驻留线程。

实际上，由于当前 NVIDIA GPU 将每个 SM 的总线程数限制为 2,048，将每个线程块限制为 1,024，编译器会把请求降低到硬件最大值——本例中每个 SM 为 2,048 个线程（8 个线程块 × 每块 256 个线程）。同时，它会发出警告，因为 4,096 个线程的请求（16 个线程块 × 每块 256 个线程）超过了 SM 容量。

> **注** 警告大致如下："ptxas warning: Value of threads per SM…is out of range. .minnctapersm will be ignored."

实际上，使用 `__launch_bounds__` 通常会使编译器限制每线程寄存器使用量（有时也限制展开或内联），以避免溢出并允许更高占用率。实质上，这是用一部分每线程性能，以及不使用每个寄存器或不将展开推到最大，换取更多 warp 处于飞行中所带来的更一致的 warp 吞吐量。

提高占用率必须与每线程资源相平衡。要避免寄存器溢出（当强制运行过多线程而寄存器耗尽，并溢出到本地内存时会发生），因为它会造成缓慢的内存访问。

还可以在运行时使用 CUDA 占用率 API 确定最佳启动配置。例如，`cudaOccupancyMaxPotentialBlockSize()` 会考虑给定 kernel 的寄存器和共享内存使用量，计算能够产生最高占用率的线程块大小。实质上，`cudaOccupancyMaxPotentialBlockSize` 可以自动调节线程块大小以获得最佳占用率，如下所示：

```
int minGridSize = 0, bestBlockSize = 0;
// If your kernel uses dynamic shared memory (extern __shared__),
// set this correctly:
size_t dynSmemBytes = /* bytes per block (e.g., tiles * sizeof(T)) */ 0;
cudaOccupancyMaxPotentialBlockSize(
  &minGridSize, &bestBlockSize,
  myKernel,
  dynSmemBytes,      // must match your kernel's dynamic shared memory
   ↪ use
  /* blockSizeLimit = */ 0);
// Compute a grid that covers N, but don't go below the min grid
// that saturates occupancy
int gridSize = std::max(minGridSize, (N + bestBlockSize - 1) /
 ↪ bestBlockSize);
myKernel<<<gridSize, bestBlockSize, dynSmemBytes>>>(...);
```

这个 API 会根据 kernel 的资源使用量，计算可能优化占用率的每块线程数。随后可以使用 `bestBlockSize`（以及建议的网格大小）启动 kernel。需要注意，`minGridSize` 是在该设备上使此 kernel 的占用率达到饱和的最小网格大小，不一定是覆盖长度为 `N` 的输入所需的正确网格大小。应计算 `gridSize = max(minGridSize, ceil_div(N, bestBlockSize))`；如果使用 `extern __shared__`，还要传入 kernel 实际的动态共享内存字节数。

> **注** 通过在候选线程块大小上下各取 1–2 个值，对 kernel 进行计时，以验证占用率 API 的建议。在现代 GPU 上，寄存器压力和 L2 行为实际上可能使略低于最大占用率的配置在实践中更快。

应用这些设置时，编译器的启发式通常已经很好，但在需要时，`__launch_bounds__` 和占用率计算器可以提供显式控制。当你知道 kernel 可以用部分每线程资源换取更多活动 warp 时，应使用它们。这有助于防止繁重线程导致 SM 占用不足。

> **注** 寄存器与占用率之间的权衡非常重要。减少每线程寄存器数量，或使用启动边界限制寄存器数量，可以让更多 warp 驻留，从而改善延迟隐藏。不过，寄存器太少会迫使编译器将数据溢出到本地内存，损害性能。找到

最佳平衡点通常需要实验。Nsight Compute 的“Registers Per Thread”和“Occupancy”指标可以在这里提供指导。

6.10 使用 NVIDIA Compute Sanitizer 调试功能正确性

由于 CUDA 应用每个 kernel 可以生成数千个线程，传统调试可能无法捕获细微的内存错误和竞态条件。NVIDIA Compute Sanitizer 是 CUDA Toolkit 中包含的一套功能正确性工具，通过在运行时插桩代码，在开发早期发现错误，从而应对这些挑战。这可以减少调试交互，并提高整体代码可靠性。

Sanitizer 使用 `compute-sanitizer` CLI 调用，并支持 NVIDIA Tools Extension（NVTX）注解以进行更细粒度的分析。正确性和性能分析都应广泛使用 NVTX。使用 CLI 时，可以通过 `--option value` 指定选项，并加入 `--error-exitcode` 等标志以在发生错误时失败。还可以使用 `--kernel-name` 和 `--kernel-name-exclude` 等选项，仅对特定 kernel 应用过滤。可以使用 `--nvtx yes` 启用 NVTX，以缩小分析范围，并例如减少内存泄漏报告中的误报：

```
compute-sanitizer [--tool toolname] [options] <application> [app_args]
```

注 建议将 Compute Sanitizer 与 `--error-exitcode` 集成到持续集成（CI）流水线中，使用 kernel 过滤器和 NVTX 区域注解捕获正确性回归。

Compute Sanitizer 包含 4 个主要工具：memcheck、racecheck、initcheck 和 synccheck。它们分别帮助检测 CUDA 代码中的越界内存访问、数据竞态、未初始化内存读取和同步问题：

6.10.1 Memcheck

memcheck 工具可以精确检测并归因于全局、局部和共享内存中的越界或未对齐访问；报告 GPU 硬件异常；并识别设备端内存泄漏。它支持通过命令行开关启用的额外检查，例如对堆分配使用 `--check-device-heap`。

6.10.2 Racecheck

racecheck 会报告共享内存数据风险，包括写后写（Write-After-Write）、写后读（Write-After-Read）和读后写（Read-After-Write），这些风险可能导致非确定性行为。racecheck 有助于开发者验证 warp 和线程块内线程到线程通信的正确性。

6.10.3 Initcheck

initcheck 会标记对未初始化设备全局内存的任何访问。这可能由缺少主机到设备的复制，或跳过设备端写入造成。该工具有助于避免由过期数据或垃圾数据引发的隐蔽错误。

6.10.4 Synccheck

synccheck 会检测对同步原语的无效使用，例如不匹配的屏障。它能够识别可能导致死锁和线程间状态不一致的线程排序风险。

简而言之，NVIDIA Compute Sanitizer 提供了一套工具，用于发现并解决 CUDA 应用中的内存、竞态、初始化和同步错误。将这些工具集成到 CI 系统后，开发者可以及早发现正确性问题，从而更有信心地交付可靠、高性能代码。

6.11 Roofline 模型：计算受限还是内存受限工作负载

Roofline 模型是一种有用的可视化方法，用于绘制两个由硬件决定的性能上限：处理器峰值浮点速率对应的一条水平线，以及峰值内存带宽决定的一条斜线。二者共同形成“屋顶线”包络，可以揭示某个 kernel 是受计算限制（计算受限）还是受数据移动限制（内存受限）。

两条线的交点称为脊点（ridge point）。它对应算术强度阈值：kernel 在该阈值左侧时从内存受限转变为右侧的计算受限。算术强度按 GPU 与片外全局内存之间传输的每字节数据所执行的 FLOPS 数量来衡量。

考虑一个简单示例来说明算术强度为何重要。假设一个 kernel 加载两个 32 位 float（总计 8 字节），将它们相加（1 个 FLOP），并写回一个 32 位 float 结果（4 字节）。在这种情况下，算法对 12 字节内存流量执行 1 个 FLOP，算术强度为 0.083 FLOPS/byte（1 $extFLOP/12\ extbytes \approx 0.083\ extFLOPSperbyte$）。

将其与 GPU 的 10 FLOPS/byte 脊点比较（10 $extFLOPS =\sim 80\ extTFLOPS \div 8\ extTB/s$）。这个 float 加法 kernel 的脊点为 0.083，位于 Roofline 左侧的内存受限区域，与脊点相差几个数量级。它比该阈值低 100 多倍，因此无法使算术逻辑单元（ALU）保持繁忙。这个 kernel 位于内存受限区域，性能主要由内存停顿而非计算决定。图 6-21 展示了 Blackwell 的 Roofline 模型示例，其中包括峰值计算性能（约 80 FLOPs/sec 的水平线）和峰值内存带宽（对应 8 TB/s 的斜线）。

这里可以看到，Blackwell GPU 的脊点等于持续 FLOPs/sec 除以持续 HBM 带宽。在本例中，交点为 10 FLOPS/byte。示例 kernel 的算术强度为 0.083 FLOPS/byte，位于斜向的内存带宽对角线左侧。因此，该 kernel 位于 Roofline 的斜向内存带宽上限上，证实它是内存受限的。

要让该 kernel 的内存受限程度降低（从而更加计算受限），可以通过每字节数据执行更多工作来提高其算术强度。这会将 kernel 向右移动，推动性能向计算屋顶线靠近。

降低精度数据是让 kernel 降低内存受限程度的一种简单方法。例如，如果使用 16 位浮点数（FP16）而不是 32 位浮点数（FP32），每次操作传输的字节数会减半，FLOPS/byte 强度会立即翻倍。

现代 GPU 还支持专用的 8 位浮点（FP8）Tensor Core。Blackwell 还为某些 AI 工作负载引入了原生 4 位浮点（FP4）Tensor Core 支持。这些能力进一步减少每次操作的字节数，并使 FLOPS/byte 强度进一步提高。

例如，Blackwell 支持 FP8 Tensor Core（每个值 1 字节），相较 FP16 可使吞吐量翻倍并将内存使用量减半。它还对某些工作负载（如模型推理）支持 FP4（每个值半字节）。

一次 128 字节的内存事务可以携带 32 个 FP32 值、64 个 FP16 值、128 个 FP8 值或 256 个 FP4 值。Blackwell 引入硬件解压缩，以加速压缩模型权重。例如，模型可以以压缩形式存储在 HBM 中，压缩程度甚至可以超过 FP4；硬件能

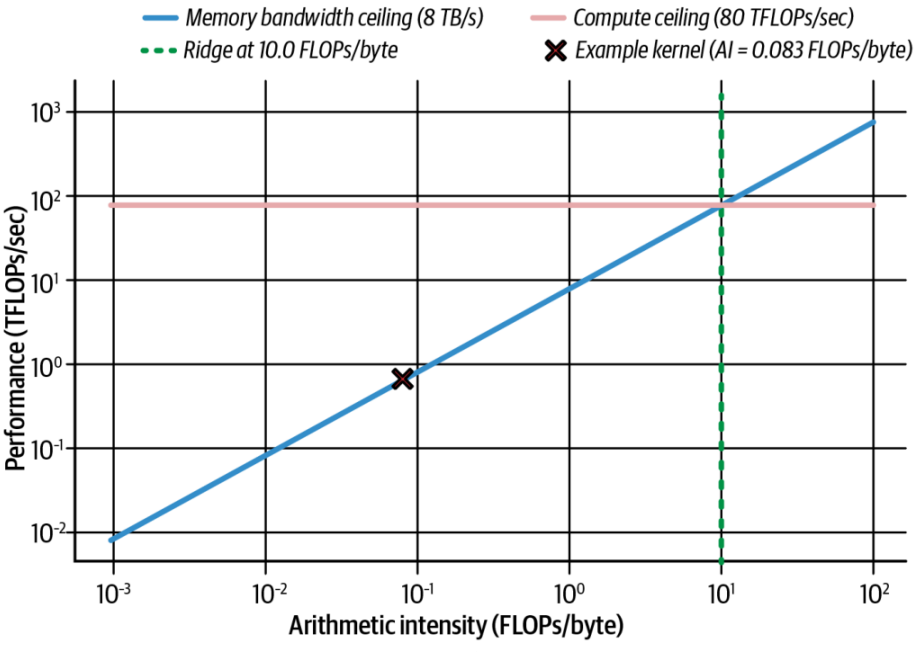

图 6-21　Blackwell 类 GPU 的 Roofline 模型（FP32 约 80 TFLOPs/sec、HBM3e 约 8 TB/s），展示示例 kernel 的点以及约 10 FLOPS/byte 的算术强度脊点。

够在加载时即时解压缩权重。这样，在读取权重时，有效内存带宽也会进一步提高。

因此，Blackwell 对基于 Transformer 的 token 生成等内存受限工作负载具有架构优势。权重以压缩的 4 bit 或 2 bit 方案存储，在加载时由硬件解压缩，然后转换为 FP16/FP32，以进行更高精度的聚合和计算。这说明，较低精度可以减少传输的数据量，提高 kernel 的算术强度，并改善工作负载的总体内存吞吐量。

对于内存受限工作负载，目标是将 kernel 的工作点沿 Roofline 向右推，以提高其算术强度并使其更接近计算受限状态。更接近计算受限区域后，kernel 就能更充分地利用 GPU 的全部浮点计算能力。

> **注**　基于 Transformer 的模型（例如 LLM）在不同阶段可能同时表现为计算受限和内存受限。例如，注意力层（预填充阶段）通常是计算受限的，而矩阵乘法（解码阶段）通常是内存受限的。第 15–18 章深入讨论推理时将进一步介绍。

当 kernel 受内存限制时，Nsight Compute 会报告很高的 DRAM 带宽利用率，同时报告较低的实际计算指标，例如较低的 ALU 利用率。这表示 warp 大部分时间都停顿在内存访问上。

要深入了解发生了什么，最好使用 Nsight Compute 查看每个 kernel 的计数器，包括延迟、缓存命中率和 warp 发射停顿。此外，现代版本的 Nsight Compute 还具有范围重放（带指令级源代码指标）、改进的源代码关联导航和启动栈大小指标。这些特性有助于更快诊断依赖停顿、寄存器压力和启动配置的影响。

随后可以使用 Nsight Systems 查看整体时间线，其中展示 GPU 空闲间隔、与 CPU 工作的重叠以及 PCIe/NVLink 传输。二者结合，可以同时回答“为什么”（哪些停顿、哪些资源）和“什么时候”（这些停顿如何嵌入应用程序的整体执行过程）。

关键是反复进行性能分析，并使用 Nsight Compute 与 Nsight Systems 的指标识别内存热点。应在可疑代码周围添加 NVTX 范围，放大查看时间线行为，并利用反馈进行优化。

例如，可以使用 NVTX 将区域标记为“内存复制”或“kernel 执行”，并在 Nsight Systems 时间线中查看它们。这对于确认主机到设备的传输与计算发生重叠非常有用，前面已经讨论过这一点。

例如，为了验证重叠，可以使用 NVTX 标记数据传输和 kernel 调用的开始/结束。Nsight Systems 会在时间线上显示这些范围。借助异步内存复制（`cudaMemcpyAsync`），数据传输会与 GPU 上的 kernel 执行重叠（见图 6-22），图中比较了同步和异步内存传输。

如果预期会发生重叠，却看到复制和 kernel 是串行运行而非并行运行，那么可能存在不希望出现的默认流同步。否则，很可能是缺少固定内存缓冲区，阻止了真正的重叠。

> **注** 如果不使用固定（页锁定）内存，`cudaMemcpyAsync` 传输无法与 kernel 执行重叠。这是一个常见的性能问题。

当怀疑 kernel 因缺少数据而处于饥饿状态时，先在 Nsight Compute 和 Nsight Systems 下运行它。在 Nsight Compute 中，你会看到全局加载效率指标下降，这

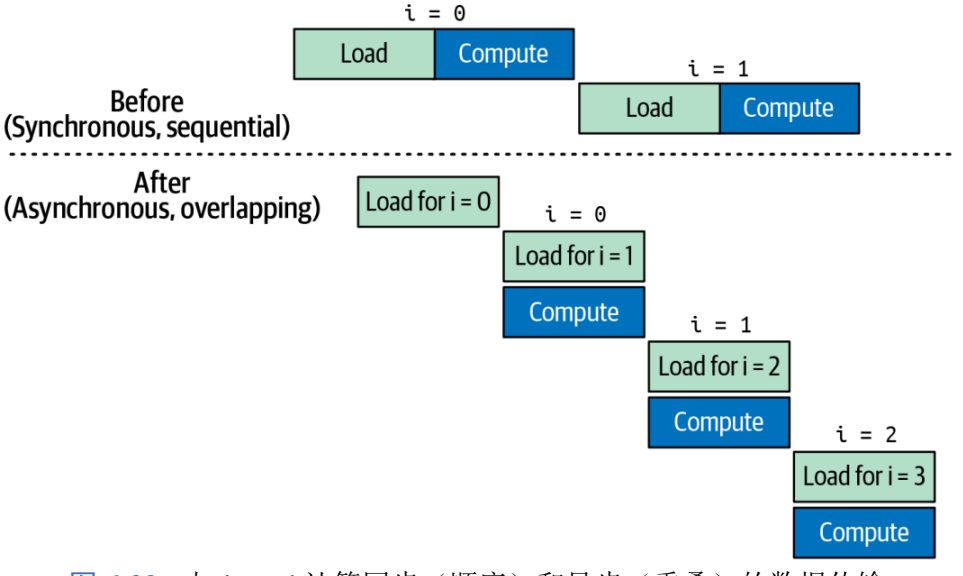

图 6-22　与 kernel 计算同步（顺序）和异步（重叠）的数据传输。

表明 DRAM 请求没有得到足够快的满足。同时，Nsight Systems 时间线会显示 kernel 启动之间的空闲区段，说明 GPU 正在等待数据传输。

应用本章介绍的内存层次结构优化后，这些空闲间隔几乎都会消失，Nsight Compute 会显示内存流水线利用率百分比向峰值攀升。你还会看到端到端 kernel 吞吐量相应提高。

> 注　每次更改后都要进行测量。性能分析工具会确认某项优化是否确实减少了内存停顿。

6.12 关键要点

本章介绍了如何选择能够优化占用率的启动参数、如何异步管理 GPU 内存，以及如何应用 Roofline 分析区分计算受限和内存受限 kernel。下面列出一些值得复习的关键要点：

6.12.1 SIMT 执行模型

GPU 在单指令、多线程（SIMT）模型下，以 warp（32 个线程）为单位执行线程；每个 warp 以锁步方式发射指令。高占用率——让许多 warp 处于飞行中——可以隐藏内存和流水线延迟。

6.12.2 线程层级：线程 → 线程块 → 网格

线程被组织成线程块（最多 1,024 个线程），线程块再组成网格，从而无需修改代码即可扩展到数百万线程。同步（`__syncthreads()` 或协作组）能够启用共享内存中的数据复用，但会产生开销，因此应尽量减少屏障。

6.12.3 占用率与资源限制

选择 32 的整数倍作为线程块大小，以避免未填满的 warp 并最大化调度器利用率。注意每个 SM 的资源限制。对于 Blackwell，每线程最大寄存器数为 255，每个 SM 的最大共享内存为 228 KB，最大驻留 warp 数为 64，最大驻留线程块数为 32。

6.12.4 CUDA kernel 启动参数

从 `threadsPerBlock = 256`（8 个 warp）开始，以平衡占用率和资源使用；计算 `blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock` 以覆盖所有元素。根据性能分析反馈（寄存器/寄存器溢出、共享内存使用量、实际达到的占用率）调节这些值。

6.12.5 异步内存管理

优先在专用流上使用 `cudaMallocAsync/cudaFreeAsync`，并利用 CUDA 内存池，以避免全局同步和操作系统级开销。PyTorch 的缓存分配器遵循类似模式，以高效分配张量，并避免代价高昂的 `cudaMalloc()` 和 `cudaFree()` 调用。

6.12.6 GPU 内存层次结构

寄存器 → L1/共享内存 → L2 → 全局（HBM3e）→ 主机：每一级都在容量与延迟/带宽之间进行权衡。最大化寄存器和共享内存/L1 缓存中的数据复用。

6.12.7 统一内存考量

CUDA 托管内存（统一内存）简化了编程，但可能产生隐式页面迁移；使用 `cudaMemPrefetchAsync` 和内存建议来避免意外停顿。

6.12.8 Roofline 模型分析

算术强度（每字节 FLOPS）决定 kernel 是内存受限还是计算受限。使用低精度（FP16/FP8/FP4 和硬件解压缩）提高 FLOPS/byte 比率，并推动 kernel 向计算屋顶线靠近。使用 Nsight Compute（每 kernel 指标）和 Nsight Systems（时间线）进行性能分析，以识别并消除内存停顿。在结合 FP8 与 FP4 时，使用 Blackwell 的 TMEM 和统一矩阵乘加（UMMA）可以将 kernel 从内存受限推向计算受限。第 10 章将更详细地介绍 UMMA。

6.13 结论

本章通过阐明 GPU 的 SIMT 模型、线程层级和多级内存系统，为高性能 CUDA 开发奠定了基础。请记住，占用率是活动 warp 与 GPU 理论最大值之比，对于隐藏延迟很重要。

不过，最大化占用率并不能保证每种情况下的最佳性能。如果线程具有足够的指令级并行性（ILP），或者其他资源才是瓶颈，GPU 往往可以在中等甚至较低占用率下实现很高的吞吐量。

较高占用率有助于隐藏延迟，但在某些场景中，减少活动线程数可以为其他线程释放寄存器。这样每个线程能够执行更多计算，并最终提高吞吐量。务必对不同占用率水平进行基准测试，以找到适合工作负载和硬件的最优设置。

掌握这些基础知识和性能分析技术后，你已经准备好深入研究有针对性的优化，例如避免 warp 分歧、利用 GPU 内存层次结构以及异步预取内存。我们还将深入介绍 TMA；它负责大批量内存传输，使 GPU 能够专注于有用工作并提高计算有效吞吐量。

第 7 章 剖析与调优 GPU 内存访问模式

随着 AI 模型的规模和复杂度不断攀升，GPU 的内存系统往往成为隔在理论算力与实际性能之间的瓶颈。正如第 6 章所述，现代 NVIDIA GPU 将数千个面向吞吐量优化的简单核心与专用 Tensor Core 相结合，同时配备高带宽内存（HBM）、CPU 与 GPU 一致的统一内存地址空间（例如 Grace Blackwell 超级芯片）、片上共享内存、各级缓存，以及 Tensor Memory Accelerator（TMA）等专用直接内存访问（DMA）引擎。

本章将给出多种 CUDA C++ 与 PyTorch 优化技术：对齐数据结构以实现高效的内存访问、消除冗余的数据加载，并借助硬件把数据传输与计算重叠起来。

通过矩阵乘法、张量运算等具体的“优化前后”对比示例，你将看到内存访问模式、分块（tiling）策略与异步数据传输上的微小改动，如何减少被浪费的带宽、提升算术效率，并把 kernel 从内存受限转变为计算受限。

学完本章后，你将能够编写出更充分利用 GPU 内存层级和硬件优化数据传输引擎的 CUDA kernel。

7.1 合并访问与非合并的全局内存访问

代码的内存访问模式会对性能产生巨大影响。当 warp 内的线程访问的是硬件能够合并为更少、更大事务的连续内存地址时，全局内存访问是最快的。如果线程访问的是零散或未对齐的地址，设备就无法把请求合并成数量最少的缓存行事务——在现代 GPU 上，这种缓存行是 128 字节的，

并由 4 个 32 字节的扇区（sector）组成。这会导致产生多得多的内存事务来取回无用的数据，迅速耗尽内存带宽。

在 Blackwell GPU 上，单卡 HBM3e 带宽最高可达 8 TB/s；在 Grace Blackwell GB200 与 GB300（两块 GPU 组成的超级芯片）中，两块 GPU 合计带宽可达 16 TB/s。非合并内存访存会因多余的内存事务和停滞而闲置其中大部分带宽。

在非合并访问况下，warp 中的每个线程从零散的地址加载数据，从而引发大量相互独立的内存事务。即使 warp 内的线程访问连续地址，只要首地址不是 128 字节对齐，该 warp 的请求也会横跨两个 128 字节缓存行。

举例来说，若 warp 的首个线程起始于一个未按 128 字节对齐的地址，该 warp 的内存请求就会跨越缓存行边界，往往导致两个 128 字节事务而不是一个。此时 warp 可能会在最优的 4 个扇区之外再多取 1 个扇区，即两条缓存行共计 5 个扇区，从而浪费带宽。一条未对齐但连续的 128 字节 warp 加载究竟触及 5 个还是 8 个 32 字节扇区，取决于起始偏移量；对齐访问则始终只需 4 个 32 字节扇区。

而在合并访问况下，线程从连续地址加载数据，这些请求会被合并为一次宽事务。图 7-1 对比了合并与非合并内存访问。

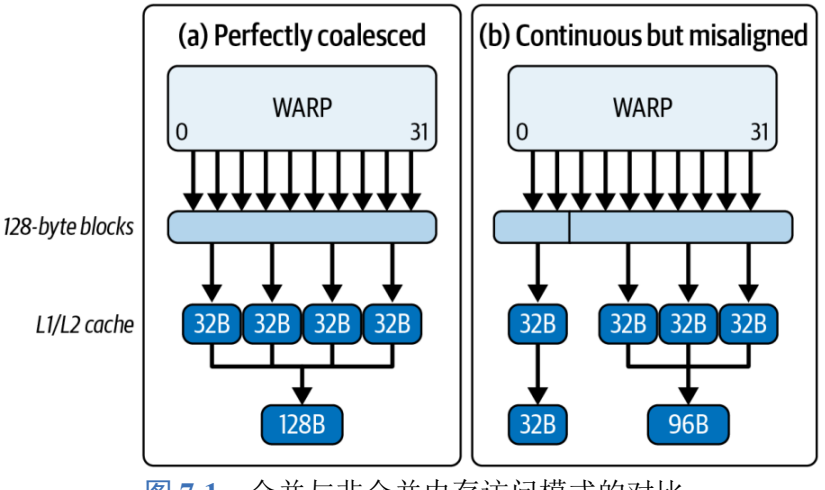

图 7-1　合并与非合并内存访问模式的对比

在 kernel 代码中，这类问题通常表现为跨步或不规则的索引方式，使每个线程触及不同的缓存行。当 kernel 的线程以跨步

或不规则的索引取数时，GPU 会发出许多细小的、非合并的全局内存事务，而不是少数几次全宽度的加载。

在 Nsight Compute 中，一旦出现非合并模式，Memory Workload Analysis 部分就会显示更低的 Global Memory Load Efficiency、更高的 DRAM 扇区读取计数，以及高于 4.0 的每请求平均扇区数。这是因为取回的扇区数量比正确合并的

访问模式更多，说明取回的多数字节并未被使用，带宽正被浪费。同时，DRAM 吞吐量百分比会远低于峰值。这表明确实有 warp 在内存上停滞空耗，而不是驱动 ALU 运算。

要摆脱这种内存受限的状态，可以重新组织数据，使每个 warp 的 32 个线程加载连续元素：要么用 `idx = blockIdx.x * blockDim.x + threadIdx.x` 计算出的索引来访问 `input[idx]`，要么改用数组结构（SoA）布局，使线程 i 总是访问元素 i。结构体数组（AoS）与数组结构（SoA）的区别如图 7-2 所示。

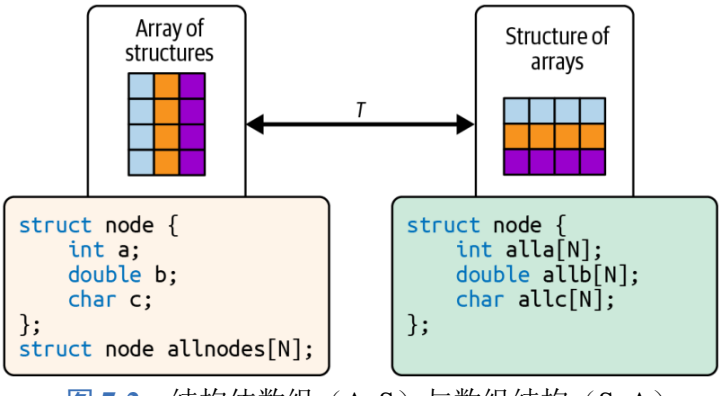

图 7-2　结构体数组（AoS）与数组结构（SoA）

一旦做出这一改动，硬件就会自动把 warp 的全局内存加载合并为更少、更宽的事务，返回的数据中可利用（浪费更少）的比例更高。Nsight Compute 的各项计数器会立即显示改善。

下面用一个例子来说明。优化前后的代码演示了如何把全局内存访问从跨步模式重构为连续模式，从而带来显著的性能提升。

优化前示例（C++）：非合并的跨步访问。在本例中，每个线程以步长 2 从输入数组复制数据，导致未对齐的内存访问：

```
#include <cuda_runtime.h>
#include <iostream>
```

```
__global__ void uncoalescedCopy(const float* __restrict__ in,
float* __restrict__ out,
int N, int stride) {
    // n = 1048576, stride = 2
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        // Loads from in[] with a stride, causing
        // multiple memory segments to be fetched
        out[idx] = in[idx * stride];
    }
}
int main() {
    const int N = 1 << 20;
    const int stride = 2;
    float* h_in = nullptr;
    float* h_out = nullptr;
    cudaMallocHost(&h_in, N * stride * sizeof(float));
    cudaMallocHost(&h_out, N * sizeof(float));
    for (int i = 0; i < N * stride; ++i) {
        h_in[i] = static_cast<float>(i);
    }
    float *d_in, *d_out;
    cudaMalloc(&d_in,  N * stride * sizeof(float));
    cudaMalloc(&d_out, N * sizeof(float));
    cudaMemcpy(d_in, h_in, N * stride * sizeof(float),
      cudaMemcpyHostToDevice);
    // Number of threads per block (multiple of 32)
    const int threadsPerBlock = 256;
    // Number of blocks per grid
    const int blocksPerGrid = (N + threadsPerBlock - 1) /
      threadsPerBlock;
    uncoalescedCopy<<<blocksPerGrid,
      threadsPerBlock>>>(d_in, d_out, N, stride);
    cudaDeviceSynchronize();
    cudaFree(d_in);
    cudaFree(d_out);
```

```
    cudaFreeHost(h_in);
    cudaFreeHost(h_out);
    return 0;
}
```

这个 CUDA C++ kernel 以大于 1 的步长在彼此分隔的地址上发出全局内存加载，按定义就是不连续的。这导致每个 warp 产生多个小事务，而不是一次宽事务。

优化前示例（PyTorch）。在 PyTorch 中，可以用带步长的索引执行 gather 操作，构造出类似的情形：

```
import torch
def uncoalesced_copy(input_tensor, stride):
    # Flatten to 1D so we know exactly
    #   which dimension we're indexing
    flat_tensor = input_tensor.contiguous().view(-1)
    # Generate indices with a fixed stride to gather
    assert flat_tensor.numel() % stride == 0,"stride must divide tensor
     ↪ length"
    idx = torch.arange(0, flat_tensor.numel(), stride,
        device=flat_tensor.device, dtype=torch.long)
    # index_select uses a gather kernel that issues uncoalesced loads
    return torch.index_select(flat_tensor, 0, idx)
# Usage
n, stride = 1 << 20, 2
inp = torch.arange(n * stride, device='cuda',
                   dtype=torch.float32)
out = uncoalesced_copy(inp, stride)
```

这段 PyTorch 代码使用 `torch.index_select`，并采用带步长的索引模式，使底层 GPU 的 gather kernel 执行非合并加载。具体而言，一个 32 线程的 warp 访问的地址彼此相隔 `stride` × 4 字节。

这样无法形成一次宽事务，而是产生 32 次相互独立的加载。每个线程从 `inp` 中加载的值在内存中与下一个线程加载的值相距很远，从而阻碍了合并。图 7-3 展示了合并访问与跨步访问模式——以及随机访问。

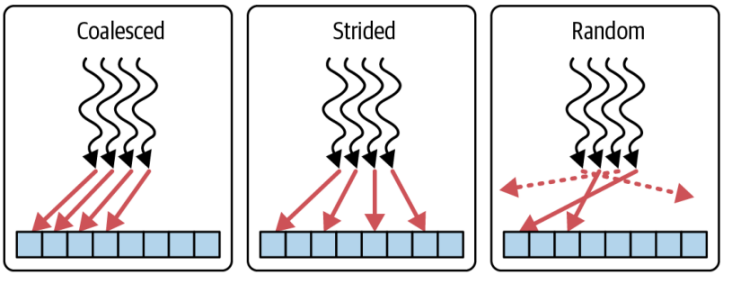

图 7-3　合并、跨步与随机内存访问模式

在 GPU 上运行 C++ 与 PyTorch 代码后，我们测量性能指标。在非合并版本中，每个 warp 的内存请求平均被拆分为 8 个相互独立的 32 字节扇区。由于每次 128 字节缓存行取数会被拆成 4 个独立的 32 字节扇区（这个数字 4 稍后再展开），该访问模式要用两条缓存行才能取回这 8 个独立的 32 字节扇区。

在未优化的版本中，每个 warp 的非合并加载把单个逻辑请求拆成最多 8 个相互独立的 32 字节扇区，使事务数量暴涨、内存管线陷入饥饿。结果，Nsight Compute 报告的动态随机存取内存（DRAM）持续峰值吞吐量只有约 25%。

接下来，我们把内存访问合并起来，让线程读取连续元素，使每个 warp 发出更少、更大的事务，从而完成优化。

优化后示例（C++）：合并访问。只需去掉步长（或将其设为 1），每个线程就会复制一个连续元素。线程访问的这种对齐方式让硬件能把内存请求合并为完整的 128 字节事务：

```
#include <cuda_runtime.h>
#include <iostream>
__global__ void coalescedCopy(const float* __restrict__ in,
                              float* __restrict__ out,
                              int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        // Contiguous load
        // Threads copy neighboring elements
        out[idx] = in[idx];
    }
}
```

```
int main() {
    const int n = 1 << 20;
    const size_t bytes = n * sizeof(float);
    // 1) Allocate pinned host memory
    float *h_in = nullptr, *h_out = nullptr;
    cudaMallocHost(&h_in,  bytes);  // page-locked host alloc
    cudaMallocHost(&h_out, bytes);
    // 2) Initialize input
    for (int i = 0; i < n; ++i) {
        h_in[i] = static_cast<float>(i);
    }
    // 3) Allocate device memory
    float *d_in = nullptr, *d_out = nullptr;
    cudaMalloc(&d_in,  bytes);
    cudaMalloc(&d_out, bytes);
    // 4) Copy to device
    cudaMemcpy(d_in, h_in, bytes,
      cudaMemcpyHostToDevice);
    // 5) Launch kernel
    dim3 block(256);


    dim3 grid((n + block.x - 1) / block.x);
    coalescedCopy<<<grid, block>>>(d_in, d_out, n);
    cudaDeviceSynchronize();
    // 6) Copy back to host
    cudaMemcpy(h_out, d_out, bytes,
      cudaMemcpyDeviceToHost);
    // 7) Clean up
    cudaFree(d_in);
    cudaFree(d_out);
    cudaFreeHost(h_in);
    cudaFreeHost(h_out);
    return 0;
}
```

出于篇幅考虑，我们省略了合并访问的 PyTorch 版本，因为它已经内置于 PyTorch 中。PyTorch 里的合并访问实现只需执行 `out = inp.clone()` 这样的操作，即可高效地复制连续元素。事实上，对连续张量调用 `clone()` 时，底层执行的就是向量化内存拷贝，与我们的合并式 kernel 类似。

> **注** 也可以使用默认 TorchInductor 后端的 `torch.compile` 来减少冗余拷贝，并在安全时融合相邻操作。`clone()` 本身就是一次设备到设备的拷贝，因此通常不会被融合成独立的自定义 kernel。开启自动调优（例如 `mode="max-autotune"`）有助于 TorchInductor 在形状稳定时选择合并且向量化的调度方案。我们将在第 13 章和第 14 章深入讲解 PyTorch 编译器。

当访问保持合并（即无跨步）时，每个 warp 的线程访问的是相邻地址。此时，硬件会把每个 warp 的加载合并成数量最少的 128 字节事务——当首地址按 128 字节对齐时，通常每个 warp 只需一个 128 字节事务；如果访问跨越边界，则需要两个。表 7-1 展示了这项优化带来的显著改进。

表 7-1 合并与非合并内存访问的性能对比。

指标	优化前（未合并）	优化后（合并）
DRAM 吞吐量（占峰值的%）	25%	90%（3.6×）
全局内存加载效率	23%	99%
每请求平均扇区数	8.0	4.0（最优）
SM Active %	62%	99%
kernel 执行时间（ms）	4.8 ms	1.3 ms（3.7×）

各项指标的具体对比为：DRAM 吞吐量（占峰值的%）由 25% 提升至 90%（3.6×）；全局内存加载效率由 23% 提升至 99%；每请求平均扇区数由 8.0 降至 4.0（最优）；SM Active % 由 62% 提升至 99%；kernel 执行时间由 4.8 ms 降至 1.3 ms（3.7×）。

> **注** 表中各项指标的数值仅用于说明概念。不同 GPU 架构上的真实基准测试结果请参见 GitHub 仓库。

修复数据布局并将全局内存访问合并后，DRAM 吞吐量从峰值的 25% 升至 90%，约提高 3.6 倍。kernel 执行时间从 4.8 ms 缩短到 1.3 ms，约改善 3.7 倍——阻塞明显减少，SM 得以持续推进计算。

全局内存加载效率从 23% 升至 99%，意味着取回的每个字节几乎都得到了利用。与此同时，每请求平均扇区数降到了约 4.0。

> **注** 接近 4.0 的取值表明 warp 的加载已完全合并。当首地址按 128 字节对齐时，全部 32 个线程都映射到同一条 128 字节缓存行；这条缓存行包含四个 32 字节扇区，因此该指标报告的每请求平均扇区数为 4.0。

在跨步或分散访问的最坏情况下，该值可接近 32，因为 L2 处的活动以 32 字节扇区为单位上报。取值高于 4.0 就说明访问未合并或未对齐。在未优化的版本中，未合并的加载会把一次逻辑访问拆分为每请求多个扇区，从而逼近最坏情况。

本例中每请求约为 4.0 个扇区，说明请求干净地映射到 128 字节缓存行上，没有留下未使用的扇区。由于内存阻塞减少，SM Active % 从 62% 提升到 99%。

概括来说，让每个 warp 的线程对齐到连续地址上，GPU 的内存控制器就能用少量大事务服务每个 warp，而不必应付几十个小事务。这把全局内存加载效率（每个事务中返回有效数据的比例）推到接近 100%，并让 warp 保持忙碌而非闲置，从而提高了 SM Active %。

至此，你已经看到合并访问如何重塑线程到地址的映射，使整个 warp 的加载都对齐到 GPU 的 128 字节段上。然而，即便是一个完全合并的 warp，在底层仍会发出 32 次独立的 4 字节读取（每线程一次），迫使硬件再把它们拼接回每个 128 字节事务。

为了消除这最后一层低效，我们转向向量化内存访问：让每个线程用单条指令取回更宽的已对齐数据块（例如 16 字节的 `float4`），使合并后的 warp 恰好发出四个 128 字节事务，而不是 32 个。接下来深入探讨如何把每线程的加载打包进 CUDA 内建的向量类型。

7.2 向量化内存访问

内存合并是 NVIDIA GPU 上的运行时硬件优化，而向量化内存访问则是编译期策略：每条加载或存储指令为每个线程显式取回多个连续元素（例如 `float4`，即 16 字节）。这可以减少指令数量，并消除拼接开销。

要在 Blackwell 上实现高效的全局内存访问，关键是让加载与 GPU 原生的 128 字节事务粒度相匹配。当每个线程只读取一个 4 字节 `float` 时，32 线程的 warp 仍然要把 32 个 4 字节请求拼接起来，才能填满一条 128 字节缓存行。

在理想的合并情形下，warp 读取 32 个对齐到 128 字节边界的 4 字节字，恰好映射到四个 32 字节扇区。Nsight Compute 通过每请求平均扇区数这一指标捕捉此类低效：当访问未对齐、跨步或分散时，该值可能远超 4.0；带宽得不到充分利用，全局内存加载效率也随之下降。

解决办法是把每个线程的工作打包成一个更大的自然对齐向量，使其干净地映射到那些 128 字节事务上。CUDA 内建的 `float4` 类型正是为此设计：它把四个 4 字节 `float` 打包成一个 16 字节结构体，编译器保证该结构体按 16 字节对齐。为便于说明，它的形式大致如下面的代码所示：

```
// Example of a custom float4 for illustration purposes.
// Note: it's recommended to use the built-in CUDA float4 type.
// Note: if N is not divisible by 4, handle the last N % 4 elements
// (e.g., a short scalar cleanup) or assert divisibility in host code.
 ↪ The
// input and output pointers must be 16-byte aligned for float4
 ↪ loads/stores.
struct my_float4 {
    float x;  // 4 bytes
    float y;  // 4 bytes
    float z;  // 4 bytes
    float w;  // 4 bytes
};
```

当 warp 中的全部 32 个线程都发出 `float4` 加载时，它们共同取回 32×16 字节 $= 512$ 字节的连续数据，Blackwell 内存控制器随后将其拆分为恰好四个 128 字节事务（512 字节 $\div$ 每个事务 128 字节 $= 4$ 个事务）。这样便保持了前面讨论合并内存访问时提到的理想值——每个 128 字节事务 4.0 个扇区：warp 请求 512 字节，硬件用四个对齐的 128 字节事务服务这一请求，共计 16 个扇区，且每个扇区都得到充分利用。

与可能推高扇区数的未对齐或跨步情形相比，向量化 `float4` 加载把每线程的加载指令减少为原来的四分之一（4×）。当对齐条件得到满足时，这有助于维持在每请求 4.0 扇区的理想水平。

> **注** PyTorch 编译器等高级 GPU 编译器在满足对齐和连续性要求时，往往能自动生成向量化内存操作。让每线程的分块与向量宽度相匹配，可以引导编译器这样做。因此，框架通常也能在底层实现类似的内存事务缩减。不过，从一开始就正确对齐数据仍然是有益的。

得益于这项向量化内存访问优化，全局内存加载效率向 100% 逼近。当每个 128 字节事务都被完全利用时，每事务扇区数保持在 4.0，此时 warp 发出多个对齐事务来服务一个更宽的请求。

向量化加载减少了每搬运一字节所需的内存指令数量，往往还能提高有效带宽。要受益于向量化加载，数据指针必须按向量宽度对齐。CUDA 运行时和驱动层的分配函数（例如 `cudaMalloc`）返回的设备指针至少按 256 字节对齐。

这一基础对齐已足以保证分配边界上的 16 字节和 32 字节向量对齐。如果元素偏移不是向量宽度的整数倍，加上偏移就可能破坏对齐。下面的类型转换把目标类型告知编译器，并假定指针已经正确对齐：

```
auto ptr4 = reinterpret_cast<const float4*>(ptr);
```

转换之前，务必确保指针值是 16 字节（4 个 `float`）的整数倍。原因是 `cudaMalloc()` 返回的指针至少按 256 字节对齐；如果元素偏移不是 4 个 `float` 的整数倍，加上偏移就可能破坏对齐。正确的对齐是向量化加载的前提条件。

当基地址按 32 字节对齐时，每线程每次迭代读取 32 字节，通常编译为每线程两条 16 字节向量加载指令（Hopper），或者一条 32 字节加载指令（Blackwell）。因此，32 线程的 warp 共请求 1024 字节。每条 128 字节缓存行由四个 32 字节扇区组成。于是，在访问正确对齐时，该 warp 生成八个对齐的 128 字节事务，共计 32 个扇区。这样每次访问都落在自然边界上，从而避免事务拆分。

值得注意的是，截至撰写本书时，CUDA 尚未在 `<vector_types.h>` 中提供内建的 8 float 聚合向量类型（例如把八个 `float32` 组合在一起的类型）。你可以自定义一个包含 8 个 `float` 值的结构体，再让每线程将其作为两个 `float4` 加载；并使用 `alignas(32)` 对该类型实施 32 字节对齐，使指针值成为 32 字节的整数倍。现代 GPU 上的全局内存向量指令通常每线程加载 16 字节（Hopper）或 32 字节（Blackwell）。在 Hopper 的每线程 16 字节情形下，8 float 聚合加载会编译成每线程两条 16 字节加载，以覆盖全部 32 字节。

而 Blackwell 在使用恰当的 32 字节对齐数据时，会把每线程编译为单条 32 字节加载。

每线程 32 字节加载（例如 8 个 `float`，每个 4 字节）时，一个 warp 搬运 1024 字节（1024 字节 = 每 warp 32 线程 × 每线程 32 字节）。这通常映射到 8 条 128 字节缓存行，因为每条缓存行由四个 32 字节扇区组成。每线程 16 字节加载（例如 `float4`，即 4 个 `float`）时，一个 warp 搬运 512 字节（512 字节 = 每 warp 32 线程 × 每线程 16 字节），即 4 条 128 字节缓存行。当基地址指针和

每线程步长都自然对齐时，硬件合并单元会把各 lane 的访问合并成完整缓存行，避免扇区拆分。这种与硬件机制协同的设计对达到峰值吞吐量至关重要。因此在 Blackwell 上进行 256 位加载时，应强制 32 字节对齐以维持峰值性能。

接下来，我们在一个简单的向量拷贝 kernel 中看看标量访问与向量化内存访问的对比。

优化前示例（C++）：标量拷贝。每个线程拷贝一个 `float`：

```
#include <cuda_runtime.h>
__global__ void copyScalar(
const float* __restrict__ in,
float* __restrict__ out, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        // Scalar load: 4-byte copy per thread
        out[idx] = in[idx];
    }
}
int main() {
    const int N = 1 << 20;
    float* h_in = nullptr;
    float* h_out = nullptr;
    cudaMallocHost(&h_in, N * sizeof(float));
    cudaMallocHost(&h_out, N * sizeof(float));
    for (int i = 0; i < N; ++i) h_in[i] = float(i);
    float *d_in, *d_out;
    cudaMalloc(&d_in, N * sizeof(float));
    cudaMalloc(&d_out, N * sizeof(float));
    cudaMemcpy(d_in, h_in, N * sizeof(float), cudaMemcpyHostToDevice);
    dim3 block(256), grid((N + 255) / 256);
    copyScalar<<<grid, block>>>(d_in, d_out, N);
    cudaDeviceSynchronize();
    cudaFree(d_in); cudaFree(d_out);
    cudaFreeHost(h_in);
```

```
    cudaFreeHost(h_out);
    return 0;
}
```

优化前示例（PyTorch）。在 PyTorch 中，标量的逐元素拷贝为便于说明可以用 Python 循环实现：

```
import torch
def copy_scalar(inp: torch.Tensor) -> torch.Tensor:
    out = torch.empty_like(inp)
    flat_in = inp.view(-1)
    flat_out = out.view(-1)
    for i in range(flat_in.numel()):
        # Each iteration issues a 4-byte load on the GPU
        # This is extremely slow. DO NOT DO THIS!
        # Use vectorized operations to avoid Python loops
        # on GPU tensors as shown in optimized version
        flat_out[i] = flat_in[i]
    return out
# Usage
N = 1 << 20
inp = torch.arange(N, device='cuda', dtype=torch.float32)
out = copy_scalar(inp)
```

采用标量的 4 字节加载时，只要访问对齐，一个 warp 通常就能用一次 128 字节事务搬运 128 字节；若访问跨越 128 字节边界，则需要两次事务。使用 `float4`（每线程 16 字节）意味着每条内存指令发出的数据量是原来的 4×——每个 warp 发出 512 字节——并被拆分为四次 128 字节事务。这样，传输便能以更少的指令完成。接下来，我们用向量加载来优化。

优化后示例（C++）：向量化拷贝。每个线程拷贝一个 `float4`（16 字节）：

```
#include <cuda_runtime.h>
// 16-byte (128-bit) vector copy: one float4 per thread
static_assert(alignof(float4) == 16, "float4 alignment must be 16
↪ bytes");
__global__ void copyVector16B(const float4* __restrict__ in,
                              float4* __restrict__ out,
                              int N4)  // number of float4 elements
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N4) {
        // Per-thread 16B load+store.
        // On sm_90, NVCC emits ld.global.v4.f32 / st.global.v4.f32.
        // for 16-byte vector loads and stores
        // On sm_90 (Hopper) (Blackwell), NVCC emits
       // ld.global.v8.f32 and st.global.v8.f32 for 32-byte aligned data
        out[idx] = in[idx];
    }


}
int main() {
    const int N  = 1 << 20;   // total number of floats
    const int N4 = N / 4;     // total number of float4s (16B chunks)
    float4 *d_in = nullptr, *d_out = nullptr;
    cudaMalloc(&d_in,  N4 * sizeof(float4));   // >=256B aligned
    cudaMalloc(&d_out, N4 * sizeof(float4));
    dim3 block(256);
    dim3 grid((N4 + block.x - 1) / block.x);
    copyVector16B<<<grid, block>>>(d_in, d_out, N4);
    cudaDeviceSynchronize();
    cudaFree(d_in);
    cudaFree(d_out);
    return 0;
}
```

这里我们使用了 `<cuda_runtime.h>` 中 CUDA 内置的 `float4` 类型。我们启动 `N/4` 个线程，以保持 16 字节对齐并发出真正的向量加载/存储；每个线程

拷贝一个 `float4`（16 字节），也就是说每个线程处理四个 float，每个 warp 总共处理 128 个 float，分四次事务完成。`cudaMalloc` 返回的指针至少按 256 字节对齐，这既满足 `float4`（16 字节）的对齐要求，也有助于在数据起始地址按 32 字节对齐时构造 32 字节对齐的向量。需要强调的是，不对齐的强制类型转换会使向量化失效。下面这个示例利用了 Blackwell 对 32 字节向量化加载的支持：

```
// --- Blackwell-only variant: 32-byte per-thread vector copy
// (PTX: ld.global.v8.f32)
// Requires 32B alignment.
#include <cuda_runtime.h>
// Proof of 32-byte alignment
static_assert(alignof(float8) == 32, "float8 alignment must be 32
 ↪  bytes");
struct alignas(32) float8 { float v[8]; };
__global__ void copyVector32B(const float8* __restrict__ in,
                              float8* __restrict__ out, int N8) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N8) {
        // Per-thread 32B load+store
        // NVCC emits ld.global.v8.f32/st.global.v8.f32 on sm_100
        out[idx] = in[idx];
    }
}
int main() {
    const int N = 1 << 20;
```

```
    const int N8 = N / 8;
    float8 *d_in, *d_out;
    // cudaMalloc returns pointers aligned to >=256 bytes
    // This will allow the compiler to emit
    // optimized ld.global.v8.f32 and st.global.v8.f32 instructions
    cudaMalloc(&d_in,  N8 * sizeof(float8));
    cudaMalloc(&d_out, N8 * sizeof(float8));
    dim3 block(256), grid((N8 + block.x - 1) / block.x);
    copyVector32B<<<grid, block>>>(d_in, d_out, N8);
    cudaDeviceSynchronize();
    cudaFree(d_in); cudaFree(d_out);
    return 0;
}
```

优化后示例（PyTorch）。我们可以借助张量视图与 `clone` 来模拟向量化拷贝：

```
import torch
def copy_vectorized(inp: torch.Tensor) -> torch.Tensor:
    # Reshape into groups of 4 floats for bulk copy
    vec = inp.view(-1, 4)
    # clone() on a contiguous CUDA tensor performs a device-to-device
    ↪ copy
    # using optimized runtime paths such as cudaMemcpyAsync().
    out_vec = vec.clone()
    return out_vec.view(-1)
# Usage
N = 1 << 20
inp = torch.arange(N, device='cuda', dtype=torch.float32)
out = copy_vectorized(inp)
```

对重塑后的张量调用 `clone()`，会让 PyTorch 执行成块连续拷贝，而不是逐个元素地拷贝。借助 `float4` 向量加载，每个线程每条指令可搬运 16 字节——这正是针对 Hopper 优化的情形。（Blackwell 支持每线程 32 字节的向量加载。）该加载会让一个 warp 发出 512 字节，对齐时通常拆分为四次 128 字节事务。其

收益在于：搬运同样字节所需的指令更少、扇区利用率更高，且对齐得当，从而在现代 GPU 上把内存吞吐量推向最大。如表 7-2 所示，这带来了远更好的内存带宽利用率。

> **注** Blackwell 将每线程向量加载/存储的字节数相对 Hopper 翻了一倍。借助 CUDA 13，Blackwell 可以执行 32 字节的向量加载与存储，而不是 Hopper 上的 16 字节。其前提是编译器能够证明 32 字节对齐；否则，编译器可能把每线程的加载/存储拆分成两条 16 字节指令。这会增加加载/存储同样数量数据所需的指令条数，从而对性能产生负面影响。

通过使用对齐得当的 `float4` 向量加载，每个线程每条指令可搬运 16 字节。如表 7-2 中的结果所示，这带来了远更好的内存带宽利用率。

表 7-2 标量与向量化内存访问的 Nsight Compute 指标。

指标	优化前（标量）	优化后（向量化）
全局内存加载效率	28%	97%
每请求平均扇区数	31.8	4.0
DRAM 吞吐量（占峰值的百分比）	25%	90%（3.6×）
kernel 执行时间	4.2 ms	1.2 ms（提升 3.5×）

% 本块为表 7-2 的表体（源版式为表格内容），数据行已并入上表的浮动体。

这些指标证实了上述改进：全局内存加载效率从 28% 跃升至 97%（约 3.5×），DRAM 峰值吞吐量占比从 25% 提高到 90%（约 3.6×），整体 kernel 运行时间缩短约 3.5×。

这一优化把 kernel 的执行时间从 4.2 ms 缩短到 1.2 ms，大约快了 3.5×。全局内存加载效率从 28%（标量）跃升至 97%（向量化），这意味着现在几乎每一次取回的 128 字节事务都得到了利用。

每请求平均扇区数降至接近每线 4.0 个扇区的理想值，说明每个 128 字节的内存行都被完整利用。`warp` 级的 `float4` 指令会发出四次对齐的 128 字节事务来搬运 512 字节，因此每个事务都被充分利用。

内存请求次数的减少，以及用一条向量加载取代四条标量加载，使全局内存加载效率从 28% 提升到 97%，并把持续的 DRAM 吞吐量从约峰值的 25% 提

高到约 90%。在这种情况下，warp 执行的内存操作大幅减少，而每次操作取回的数据更多，因此它们花在等待内存上的时间大幅减少，用于执行有效工作的时间相应增加。最终结果是一个明显更快的 kernel。

注 向量化减少的是指令条数；合并与对齐改善的是每请求扇区数。

下面快速对比一下上一节讨论的合并全局内存访问与本节讨论的向量化内存访问。合并确保一个 warp 中的所有线程都命中同一大块连续区域；向量化则确保每个线程取回的宽块恰好映射到这样的大块上。两者的差异汇总在表 7-3 中。

表 7-3 合并内存访问（上一节）与向量化内存访问的对比。

维度	合并的 warp 加载（线程间）	向量化（线程内）
粒度	线程 ↔ 连续内存地址	每线程 ↔ 向量加载
典型修复方法	连续索引（stride = 1）	使用 `float4`、显式 32 字节对齐的自定义 32 字节结构体，或 32 字节对齐的向量类型（如 Blackwell 与 CUDA 13+）
每 warp 的事务数	32 线程 × 4 字节 → 128 字节，对齐时即 4 个 32 字节扇区（4 个扇区）	32 线程 × 16 字节 → 512 字节，即 4 个 128 字节事务，对应 16 个 32 字节扇区（16 个扇区）
指令数变化	加载数量相同	1 条向量加载代替 4 条标量加载
收益	散落扇区更少	大小恰好匹配，扇区更少

% 本块为表 7-3 的表体（源版式为表格内容），对应行已并入上表的浮动体。

% 本块为表 7-3 的表体（源版式为表格内容），对应行已并入上表的浮动体。

% 本块为表 7-3 的表体（源版式为表格内容），对应行已并入上表的浮动体。

% 本块为表 7-3 的表体（源版式为表格内容），对应行已并入上表的浮动体。

把两种策略结合起来，就能让 warp 命中连续的大块，同时让每个线程取回一条宽度合适、对齐得当的向量。这有助于把 kernel 的内存带宽利用率推到峰值，充分释放 GPU 的全部威力。

值得注意的是，在 Blackwell 与 CUDA 13 之前，每线程的全局内存向量加载最多为 16 字节（128 位）。而 Blackwell 与 CUDA 13 新增了 32 字节（256 位）的加载/存储指令，以及针对 32 字节对齐的特定向量类型的数据类型。

在可用的情况下，对于用户自定义的 8 个 float 的聚合体，应优先选用这些更宽的 32 字节类型和指令。这样可以减少加载和存储 32 字节对齐的宽数据所需的指令条数。

除非显式使用映射到单条 32 字节指令的 32 字节对齐类型，否则自定义的 8 float 聚合体仍会编译成两条 16 字节加载。

> **注** 尽管 Blackwell 的线程可以加载完整的 32 字节向量类型，内存合并器（memory coalescer）仍只能按 128 字节块（即 4 个 32 字节扇区）为单位来服务请求。在 Blackwell 上，一个 32 线程的 warp 每线程移动 32 B，共传输 1024 B（8 × 128 B 行）。Hopper 的每线程 16 B 变体则传输 512 B（4 × 128 B）。注意，事务数量随移动的字节数成比例增加。两者在对齐得当时都能达到完全效率：16 字节（Hopper）或 32 字节（Blackwell 及之后架构）。

7.3 利用共享内存进行分块与数据复用

一个常见的性能陷阱是反复从全局内存读取同一份数据。分块（tiling）正是避免这一问题的技术：把数据成块地加载到更快的片上共享内存中，并在众多线程间复用这些数据块。

例如，对尺寸为 $N \times N$ 的矩阵做朴素的矩阵乘法，矩阵 A 的每个元素可能要从 HBM 中加载 N 次——与 B 的每一行相乘时各加载一次。这意味着每个元素会产生 $N - 1$ 次冗余加载。而在轻松就能达到数十 TFLOPS 的 Blackwell 上，冗余加载会浪费内存带宽，而这些带宽本可以用来为 GPU 的 SM 提供更多数学运算。

分块通过让每个线程块恰好一次地将 A 和 B 的一个小子矩阵（一个 tile）拉取到共享内存中，从而消除这种浪费。随后，所有线程在多次乘累加运算中复用这些缓存值。在接下来的示例中，我们将使用 32×32 的 tile，这是一个常见的选择，能很好地放入共享内存。

块内的线程可以协同地把 tile 加载进共享内存，然后调用 `__syncthreads()` 同步数据。之后，各个线程利用共享内存中的数据并行执行矩阵乘法计算。这样就把全局内存访问的成本摊销到多个线程和多次计算上。值得一提的是，这

些 tile 加载本身也被安排为合并访问：具体来说，每个 warp 从全局内存完整地读取一个 128 字节段写入共享内存——与前面的合并访问示例一致。

通过让每个元素只从 DRAM 读取一次（读入共享内存）并在后续大量计算中反复复用，我们减少了全局内存流量。下面用 $N \times N$ 矩阵乘法示例来说明这一点。先来看一个朴素的实现。

朴素矩阵乘法示例（CUDA C++）：每个线程计算结果矩阵 C 的一个元素，并为每个输出从全局内存读取 A 的整行与 B 的整列：

```
#include <cuda_runtime.h>
#include <iostream>
__global__ void naiveMatMul(const float* A, const float* B, float* C,
↪   int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < N; ++k) {
            // Each thread loads A[row, k] and B[k, col]
            // from global memory for every k.
            // This is very memory heavy.
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
```

```
    }
}
int main() {
    const int N = 1024;
    size_t bytes = N * N * sizeof(float);
    float* h_A = nullptr;
    float* h_B = nullptr;
    float* h_C = nullptr;
    cudaMallocHost(&h_A, N*N * sizeof(float));
    cudaMallocHost(&h_B, N*N * sizeof(float));
    cudaMallocHost(&h_C, N*N * sizeof(float));
    for (int i = 0; i < N*N; ++i) { h_A[i] = 1.0f; h_B[i] = 1.0f; }
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, bytes);
    cudaMalloc(&d_B, bytes);
    cudaMalloc(&d_C, bytes);
    cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);
    dim3 block(32, 32);
    dim3 grid((N + 31) / 32, (N + 31) / 32);
    naiveMatMul<<<grid, block>>>(d_A, d_B, d_C, N);
    cudaDeviceSynchronize();
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    cudaFreeHost(h_A);
    cudaFreeHost(h_B);
    cudaFreeHost(h_C);
    return 0;
}
```

这个 CUDA C++ kernel 在内层循环中为每一次乘法都发起全局内存加载。每个线程对每个 k 都要从 DRAM 读取 `A[row, k]` 与 `B[k, col]`，造成海量的冗余流量。结果就是一个高度内存受限的 kernel：SM 利用率低，并且频繁停顿以等待全局内存。下面是朴素矩阵乘法的 PyTorch 版本：

```
import torch
def naive_matmul(A, B):
    N = A.size(0)
    C = torch.zeros((N, N), device='cuda')
    for i in range(N):
        for j in range(N):
            # Each dot product loads A[i,:] B[:,j] from global memory
            ↪ repeatedly
            C[i, j] = (A[i, :] * B[:, j]).sum()
    return C
```

用法示例：$N = 1024$；`A = torch.ones((N, N), device='cuda', dtype=torch.float32)`；`B = torch.ones((N, N), device='cuda', dtype=torch.float32)`；`C = naive_matmul(A, B)`。这个 PyTorch 实现使用了嵌套的 Python 循环。尽管最内层的操作已作为逐元素乘法与求和卸载到 GPU 上执行，但其底层仍会为每个点积反复触发全局内存加载。这模拟了朴素 CUDA kernel 的内存受限行为，因为 GPU 把大部分周期花在等待内存上，而不是计算乘法。

> **注** 这段 PyTorch 代码刻意写得极慢，用以展示在循环内部反复进行冗余全局内存访问加载的极端情形。在实际中，PyTorch 等框架会对这类操作使用优化过的 kernel。

现在，我们用分块来改进它。把矩阵划分为 32×32 的 tile。32×32 是一个便利的 tile 尺寸：它与 32 的 warp 大小对齐，能很好地放入共享内存，并且映射为一整个 32 线程的 warp 恰好负责读取每一行。这样，每个 warp 可以一次协同地加载并处理（每个 tile 中的）单行。

于是，每个线程块把一个 32×32 的 A tile 和一个 32×32 的 B tile 加载进共享内存，执行 32×32 的矩阵乘法，并累加结果。这样，A 和 B 的每个元素在每个 tile 中只从 HBM 加载一次，而不是朴素版本中的 32 次。利用共享内存缓存 tile 的分块矩阵乘法具体如下：

```
#include <cuda_runtime.h>
#include <iostream>
#define TILE_SIZE 32
__global__ void tiledMatMul(const float* A, const float* B, float* C,
↪  int N) {
    __shared__ float sA[TILE_SIZE][TILE_SIZE];
    __shared__ float sB[TILE_SIZE][TILE_SIZE];
    int row = blockIdx.y * TILE_SIZE + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;
    float sum = 0.0f;
    // compute partial results using the tile
    // in shared memory
    for (int t = 0; t < N; t += TILE_SIZE) {
        // Cooperative load of a tile of A and B into shared memory
        // Load tile A with boundary check
        if (row < N && (t + threadIdx.x) < N) {
            sA[threadIdx.y][threadIdx.x] = A[row * N + t + threadIdx.x];
        } else {


            sA[threadIdx.y][threadIdx.x] = 0.0f;
        }
        // Load tile B with boundary check
        if ((t + threadIdx.y) < N && col < N) {
            sB[threadIdx.y][threadIdx.x] = B[(t + threadIdx.y) * N +
             ↪  col];
        } else {
            sB[threadIdx.y][threadIdx.x] = 0.0f;
        }
        // We will optimize this later to use a
        // thread-block-scoped cooperative-groups barrier
        __syncthreads();
        // Compute using the tile loaded in shared memory
        for (int k = 0; k < TILE_SIZE; ++k) {
            sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
        }
        // We will optimize this later to use a
```

```
        // thread-block-scoped cooperative-groups barrier
        __syncthreads();
    }
    if (row < N && col < N) {
        C[row * N + col] = sum;
    }
}
int main() {
    const int N = 1024;
    size_t bytes = N * N * sizeof(float);
    float* h_A = nullptr;
    float* h_B = nullptr;
    float* h_C = nullptr;
    cudaMallocHost(&h_A, N*N * sizeof(float));
    cudaMallocHost(&h_B, N*N * sizeof(float));
    cudaMallocHost(&h_C, N*N * sizeof(float));
    for (int i = 0; i < N*N; ++i) { h_A[i] = 1.0f; h_B[i] = 1.0f; }
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, bytes);
    cudaMalloc(&d_B, bytes);
    cudaMalloc(&d_C, bytes);
    cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);
    dim3 block(TILE_SIZE, TILE_SIZE);
    dim3 grid((N + TILE_SIZE - 1) / TILE_SIZE, (N + TILE_SIZE - 1) /
    ↪ TILE_SIZE);
    tiledMatMul<<<grid, block>>>(d_A, d_B, d_C, N);
```

```
    // synchronize the kernel with the device
    // for timing accuracy
    cudaDeviceSynchronize();
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    cudaFreeHost(h_A);
    cudaFreeHost(h_B);
    cudaFreeHost(h_C);
    return 0;
}
```

在这个分块 kernel 中，每个块协同地把一个 32×32 的 A tile 加载到 `sA`，把一个 32×32 的 B tile 加载到 `sB`。这些加载发生在循环内的前两行代码中，随后调用 `__syncthreads()`，以确保 tile 在使用前已完全加载。

然后，块利用共享内存中的 tile 执行 32×32 的乘累加运算。这个内层循环以 32 为步长遍历 k，使每个加载值被复用 32 次，从而把这些元素的全局内存读取量减少为原来的 1/32。

在完成所有 tile 迭代之后，每个线程把自己的结果写入 C。其结果是每线程的全局内存访问次数显著减少（约为原来的 1/8），稍后你将看到这一点。

> **注** 这些示例聚焦于内存访问模式，使用的是 FP32 CUDA 核心——而非低精度的 Tensor Core。第 9 章将演示如何利用低精度计算（如 16 位、8 位、4 位）进一步提升性能。

下面的 PyTorch 版本手动对矩阵进行分块，并逐 tile 调用 `torch.mm`。PyTorch 的 `torch.mm` 在 C++ 层内部利用 NVIDIA 的 cuBLAS 与 CUTLASS 实现共享内存分块与复用。下面是分块矩阵乘法的 PyTorch 版本：

```
import torch
def tiled_matmul(A, B, tile_size=32):
    N = A.size(0)
    C = torch.zeros((N, N), device='cuda')
    for i in range(0, N, tile_size):
        for j in range(0, N, tile_size):
            C_block = torch.zeros((tile_size, tile_size), device='cuda')
            for k in range(0, N, tile_size):
                A_block = A[i:i+tile_size, k:k+tile_size]
                B_block = B[k:k+tile_size, j:j+tile_size]
                # torch.mm uses an optimized kernel (likely tiling
                ↪ internally)
                C_block += torch.mm(A_block, B_block)
            C[i:i+tile_size, j:j+tile_size] = C_block


    return C
# Usage
N = 1024
A = torch.ones((N, N), device='cuda', dtype=torch.float32)
B = torch.ones((N, N), device='cuda', dtype=torch.float32)
C = tiled_matmul(A, B)
```

> **注** 使用 32×32 的 tile 时，warp 中的线程在同时访问同一列时会争用同一个共享内存 bank。在实践中，可以避免这一点：把 tile 填充（padding）到 33 列，或使用 swizzling 之类的技术，使每次访问落入不同的 bank。我们将在下一节应用这一优化。目前，先把分块本身当作一项优化来讨论。

分块带来的性能影响十分显著。通过以 tile 为单位组织计算并在共享内存中复用数据，我们降低了 DRAM 流量并提高了算术强度（arithmetic intensity），因为从内存取得的每个字节都被用于更多的浮点运算。除了全局内存事务变少，我们还观察到更高的实际占用率（achieved occupancy）。表 7-4 对比了应用共享内存分块前后的关键指标。

指标	之前（朴素 kernel）

表 7-4　应用共享内存分块带来的性能提升

指标	之前（朴素 kernel）	之后（分块 kernel）	说明
DRAM 吞吐量（占峰值的%）	90%	25%（降低至约 1/3.6）	朴素实现占用更多带宽，但那是浪费；分块后的实现数值更低，因为分块消除了冗余加载，效率更高。
实际占用率（%）	42%	89%	更多驻留 warp 使停顿更少、进展更顺利。
浮点吞吐量（GFLOPS）	15 GFLOPS	170 GFLOPS	矩阵规模小限制了绝对数值。数据复用提高了可持续算力。（绝对 GFLOPS 数值偏低，是因为矩阵规模（$N = 1{,}024$）较小，且 kernel 受内存限制。）
全局内存加载扇区数	9800	1200	每个元素每个 tile 只取回一次，而不是 32 次（每线程一次）。数值为 kernel 发出的 32 字节扇区读取总数。下降幅度反映了共享内存分块消除冗余加载的效果。
共享内存吞吐量（占峰值的%）	52%	99%	分块 kernel 的访问模式避免了 `sA` 和 `sB` 上的内存 bank 冲突。

之后（分块 kernel）

说明

DRAM 吞吐量（占峰值的%）

90%　25%（降低至约 1/3.6）　朴素实现占用更多带宽，但那是浪费；分块后的实现数值更低，因为消除了冗余加载、效率更高。实际占用率（%）

42% 89% 常驻 warp 数量更多，停顿更少，各项工作得以持续推进。

浮点吞吐量（GFLOPS）

15 GFLOPS 170 GFLOPS 矩阵规模较小，限制了绝对数值；数据复用提升了持续计算能力。（绝对 GFLOPS 数值偏低，是因为矩阵规模（N = 1,024）较小，且 kernel 受内存限制。）全局内存加载扇区

9800 1200 现在每个元素每个瓦片只需取回一次，而不是 32 次（每线程一次）。kernel 发出的 32 字节扇区读取总数。这一下降反映了共享内存分块对冗余加载的消除。共享内存吞吐量（占峰值的百分比）

52% 99% 分块 kernel 的访问模式避免了 `sA` 与 `sB` 上的内存 bank 冲突。

把 32 × 32 的瓦片暂存在共享内存中，可以保证每个元素只从 DRAM 取回一次，并在线程块内的所有线程间复用。DRAM 吞吐量从 90% 降至 25%，降幅约 3.6 倍，这正是消除冗余流量之后应有的结果。

DRAM 吞吐量的下降是可取的，因为 kernel 现在每搬运一字节数据能换来更多有效计算。算术强度（arithmetic intensity）随之提高，持续浮点吞吐量从 15 GFLOPS 升至 170 GFLOPS，提升接近 11 倍。

> **注** 优化后的 170 GFLOPS 远低于 Blackwell 约 80 TFLOPS 的理论 FP32 峰值。在小瓦片、频繁访存的场景下，这是正常现象。关键的结果是消除内存瓶颈后取得的 11 倍提升。更大的问题规模，或更多按字节计算的计算工作量，都会使性能更接近峰值。

算术强度的这一变化使 kernel 从内存受限向计算受限移动——这正是理想的结果，因为此时性能受限于充裕的浮点算力，而不是相对稀缺的芯片带宽。全局内存加载扇区数从 9,800 大幅降至 1,200：每个元素只取一次进入共享内存，再供各线程复用，从而消除了冗余加载。分块 kernel 访问 `sA` 与 `sB` 的方式还能避免共享内存 bank 冲突，这正是其共享内存吞吐量接近 100% 的原因。

总而言之，我们把该 kernel 从内存受限转移到了计算受限，而这也正是我们最初的目标。我们成功缓解了内存压力，把内存总线让给其他有效工作，并获得了更高的计算吞吐量。

同时可以看到，实际占用率从 42% 大幅提高到 89%。该指标几乎翻倍，因为分块 kernel 让更多常驻 warp 以更少停顿持续取得进展，SM 因此能更持续地保持忙碌。

通过引入共享内存分块，我们在不使每线程资源用量超出可用寄存器和共享内存的前提下，增加了每线程的工作量。这帮助 kernel 取得更高的占用率与利用率。在 Blackwell 这类 GPU 上这一点尤其重要：它们提供非常庞大的寄存

器文件（每个 SM 65,536 个 32 位寄存器），线程最多可用到每线程 255 个寄存器的上限而不发生寄存器溢出。

> **注** 回顾一下：每个线程最多可使用 255 个寄存器。我们的分块 kernel 实现保持在这一上限之内，避免了寄存器溢出，从而保住了高性能。

考察两个 kernel 的持续 FLOP 速率与平均 DRAM 带宽，即可比较 naive 版本与分块版本的算术强度（arithmetic intensity）。naive 版本运行在 15 GFLOPS，平均每秒搬运 10 GB 数据，即每字节 1.5 FLOPS；相比之下，分块实现达到 170 GFLOPS，平均每秒搬运 21 GB，即每字节 8 FLOPS。

人们可能忍不住想把瓦片进一步加大到 64 × 64 甚至 128 × 128，以进一步降低访存流量、提升数据复用。但请记住：更大的瓦片会消耗更多片上资源，包括寄存器和共享内存，从而压缩每个 SM 上还能容纳的其他线程块数量。

例如，Blackwell GPU 每个 SM 最多提供 228 KB 的可分配共享内存，放下一块 64 × 64 的 4 字节 float 瓦片绰绰有余：A 和 B 两个瓦片各需 16 KB（16,384 字节 = 64 × 64 个 float × 每个 float 4 字节）。不过，除了共享内存的用量，你还必须为每线程新增的寄存器留出预算。

每个 Blackwell SM 总共提供最多 64K 个寄存器（各 32 位），每线程上限为 255 个寄存器。若启动一个 32 × 32 的线程块（1,024 线程）来计算 64 × 64 的瓦片，则每线程负责一个 2 × 2 的子瓦片，产生 4 个输出。

以 64 × 64 的瓦片规模计，每线程需要：4 个累加寄存器（每个输出 1 个）、2 个存放 A 瓦片元素的寄存器（供两个输出复用）、2 个存放 B 瓦片元素的寄存器（同样复用），以及约 4 个用于循环计数、线程索引和地址计算的寄存器，合计约每线程 12 个寄存器。

每线程 12 个寄存器 × 每个线程块 1,024 个线程 = 每个线程块需要 12,288 个寄存器。由于每个 Blackwell SM 拥有 65,536 个 32 位寄存器（每线程上限 255 个），理论上的寄存器容量足可容纳约 5 个线程块（约 61,440 个寄存器）。然而，SM 只能支持 2,048 个并发线程（64 个 warp）。因此，实际占用率由寄存器、共享内存、warp 与线程这几项限制中的较小者决定。就本配置而言，每 SM 两个线程块就会触及 2,048 线程上限。

> **注** 如果你的 kernel 还额外占用寄存器来实现双缓冲、向量化加载等功能，可以使用 `cudaOccupancyMaxPotentialBlockSize` 或 Nsight Compute 的占用率报告重新计算占用率。

以 64 × 64 的瓦片规模计，每线程需要 22 个寄存器，一个 32 × 32 的线程块（1,024 线程）共需 22,528 个寄存器。此时每个 Blackwell SM 至多容纳两个线程块，对应的最大占用率只有 2,048 个线程——仅用两个线程块（每 SM 最多可容纳 16 个并发线程块）就触到了 SM 的线程上限。

如果 tile 尺寸增大导致并发线程块数量减少，一旦超出硬件上限，就会降低占用率并损害性能。实际使用中，tile 的各维尺寸必须落在 GPU 的共享内存与寄存器预算之内，才能维持高占用率和高吞吐量。

以每 SM 约 228 KB 可分配共享内存的 Blackwell GPU 为例，64 × 64 的 tile（每个输入矩阵 tile 约 16 KB）也许仍然放得下，但把 tile 的两个维度都翻倍，会让复用系数变成原来的平方，而共享内存用量变成原来的四倍。此时收益递减，并可能出现权衡取舍。

建议尝试不同的 tile 尺寸，在片上复用与资源限制之间取得平衡。32 × 32 的 tile 是现代 NVIDIA GPU 上一个稳妥的起点，但取决于你的共享内存与寄存器用量，稍小或稍大一点的 tile 可能带来更高的吞吐量。

> **注** 像 CUTLASS 这样的库本身就带有性能分析器（profiler），可以自动完成这种探索，帮助你为你的 kernel 和硬件找到最优的 tile 尺寸。

简而言之，我们把原本只用全局内存的朴素矩阵乘法改造成了一种使用共享内存的分块（tiled）实现。这使得数据可以被协同复用，减少了 DRAM 事务的数量，并提高了算术强度。CUDA C++ 和 PyTorch 两种实现都从这一分块技术中受益。

值得注意的是，我们手动应用的这些分块技术，恰恰就是高性能 GPU 库在底层所做的事情。例如，NVIDIA 的 CUTLASS 库提供了一组模板化组件，用于实现带有多层分块结构的通用矩阵乘法（GEMM）。这些 CUTLASS 组件把矩阵片段加载到寄存器和共享内存中，然后以与我们前面 32 × 32 tile 示例非常相似的方式计算部分结果。

事实上，NVIDIA 优化过的 cuBLAS 和 cuDNN 库在 thread、warp 和 block 三个层级上使用了类似的分块策略，以逼近峰值吞吐量。NVIDIA 甚至在 2025 年初宣布了一个名为 cuTile 的 Python 优先 API，让程序员能用更方便的 Python 风格来描述这些 tile 形状。此外，NVIDIA 还开发了一种基于 tile 的中间表示（IR），称为 TileIR，用以支持 cuTile 并简化自动编译与调优。

其他高性能库同样封装了这些分块模式。例如，NVIDIA 的 CUTLASS C++ 与 Python 库提供了模板化的 tile 迭代器和性能分析器。而基于 PyTorch 的编译器（如使用 OpenAI Triton 库的 TorchInductor）在形状和对齐条件允许时会自动生成分块 kernel。这些库降低了使用分块优化的门槛，也减少了样板代码的数量。

关键思想在于：在回到 DRAM 之前尽可能地在寄存器和共享内存中复用数据，这是根本性的原则，而各类库正是对这一原则的封装。因此，只要可能，就应尽量利用这些高度优化的库（或参考它们的做法）来获得高性能的分块模式。

例如，在 PyTorch 中调用 `torch.mm`，或在代码中使用 `cublasSgemm`，其底层所做的正是这种分块与内存合并。这也解释了为什么我们的 PyTorch 示例自动获得了同样的收益。

在实际生产中，我们会使用 cuBLAS 和 PyTorch 的 `torch.matmul` 这类高性能库，它们早已在 C++ 层面实现了分块及其他优化。在生产代码中，直接使用 `torch.mm` 或 `torch.matmul` 就能带来同样的收益——甚至可能更多，因为它们的 kernel 经过了深度调优。

> **注** 虽然你完全可以复用现有的分块库和框架，但像我们这样深入理解它们的工作原理仍然极有价值：当你需要诊断性能问题，或者需要为这些库和框架未覆盖的特殊场景自行编写定制 kernel 时，这份理解不可或缺。只是别忘了回馈社区——他们给了你很多！

如本节前面所述，使用 32 × 32 tile 时，当 warp 中的线程访问同一列，它们会争用同一个共享内存 bank。我们将在下一节解释这个问题以及相应的优化方法。

7.4 避免共享内存 bank 冲突

在现代 NVIDIA GPU（包括 Blackwell）上，共享内存拥有 32 个 bank，每个 bank 宽度为 4 字节（即地址按 mod 32 映射）。因此，一个以 128 字节（32 个 float × 4 字节）为步长访问的 warp 会把所有线程都映射到同一个 bank。如果 warp 中多个线程访问同一个 bank，就会发生 bank 冲突（bank conflict）。这会迫使内存访问串行化，从而抵消共享内存的速度优势。

在代码中，当线程以某种步长访问共享内存数组，导致它们落入同一个 bank 时，就容易出现 bank 冲突。图 7-4 给出了两个无冲突内存 bank 访问的示例。

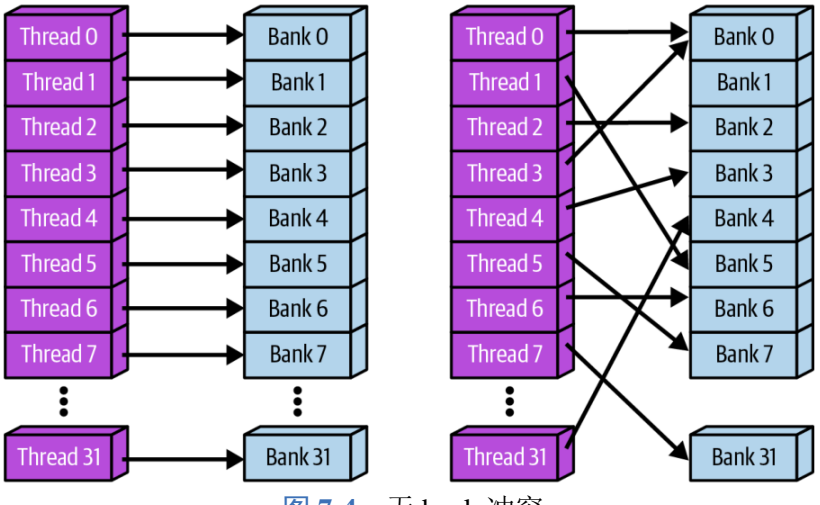

图 7-4　无 bank 冲突

在这个例子中，没有任何两个线程同时访问同一个内存 bank，这是理想情况。图 7-5 展示了 2 路 bank 冲突和 16 路 bank 冲突的示例。

在这个例子中，多个线程正在访问同一个内存 bank，这会引发冲突并影响性能。bank 冲突的一个经典例子是使用 32 × 32 共享内存 tile 的朴素矩阵转置。如果 warp 中 32 个线程各自读取 `tile[i][threadIdx.x]`，使得同一列索引（`threadIdx.x`）跨

越不同的行（`i`）被读取，那么 warp 中全部 32 个线程访问的内存地址都会落在同一个共享内存 bank 中，从而造成 32 路 bank 冲突。

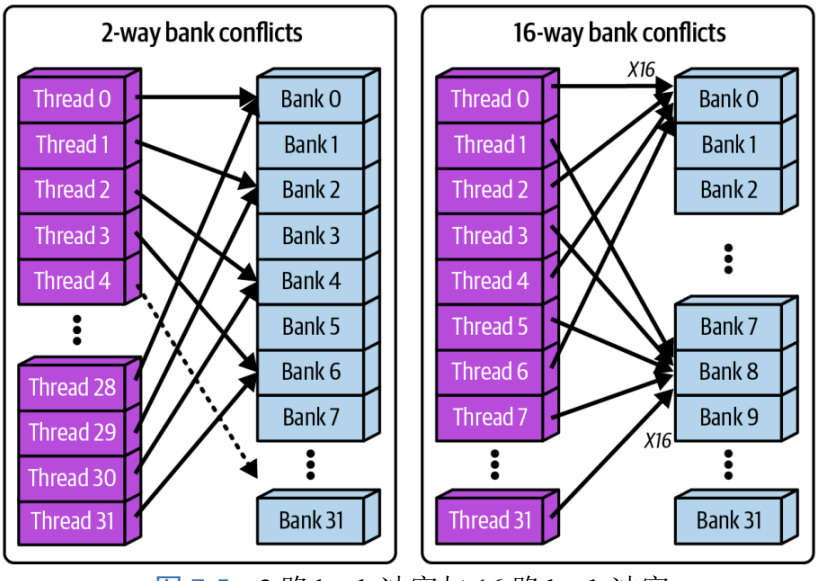

图 7-5　2 路 bank 冲突与 16 路 bank 冲突

具体来说，在矩阵转置过程中，你保持列索引（`threadIdx.x`）不变、让行索引（`i`）在各线程间变化，也就是沿着一个行主序 tile 的同一列自上而下读取。由于每行在内存中相隔 128 字节（32 列 × 每列 4 字节 = 128 字节），各个被访问的内存地址正好相差 128 字节的整数倍。

请记住，地址到 bank 的映射每 128 字节循环一次，因为共有 32 个 bank、每次访问的 bank 宽度为 4 字节。因此，访问相差 128 字节的内存地址，总会落回 bank 0——于是产生了完整的 32 路 bank 冲突。

> **注**　有一个例外值得注意：如果 warp 中全部 32 个线程访问的是同一个内存 bank 中完全相同的地址，硬件会在单个周期内将该值广播给所有线程，从而避免 bank 冲突。除此之外，任何两个或更多线程以不同内存地址访问同一个 bank 的情况都会造成 bank 冲突，并使内存访问串行化。

另一个常见的陷阱是使用与内存 bank 数量相等的步长（本例中为 32）。例如，如果你的索引步长恰好是 32 个 float，每个 4 字节，那么每个线程的地址都会相差 128 字节的整数倍。在这种情况下，所有线程都会映射到 bank 0，如下面的代码所示：

```
// Allocate a shared buffer big enough for several warps (warpCount)
__shared__ float arr[32 * warpCount];
// Each thread reads from arr[threadIdx.x * 32]
float x = arr[threadIdx.x * 32];
```

这里，以 float 计的 `threadIdx.x * 32` 换算成字节就是 `threadIdx.x * 32 * 4`。由于 $32 \times 4 = 128$，每个线程的加载地址都是 `threadIdx.x` $\times 128$ 字节。而对所有线程而言，`threadIdx.x` $\times 128 \bmod 128 = 0$。它们全都同时命中 bank 0，造成 32 路 bank 冲突。

发生这种情况时，硬件必须把本应并行的 32 次读取串行化成一连串的单 bank 访问。在 Nsight Compute（Shared Memory 部分）中，你会看到 bank 冲突计数上升、共享内存效率下降；与此同时，共享内存吞吐量只是其预期带宽的一小部分。在 Nsight Systems 中，你会看到 warp 因长时间的 bank 冲突停顿而等待，而不是在做有效工作。

bank 冲突迫使本应并行的片上共享内存访问逐条重放，抵消了缓冲机制原本期望的加速，而且往往

带来令人失望、低于预期的性能表现。如果你的 kernel 没有获得应有的加速，bank 冲突很可能就是罪魁祸首。

> **注** 务必精心选择步长和数据布局，使同一 warp 中的线程命中不同的 bank，从而避开这种串行化的瓶颈。

解决办法是调整共享内存中的数据布局以避免冲突。一种常用技术是对共享数组进行填充（padding），使每一行（或每种内存访问模式）映射到不同的 bank。例如，如果你有一个 32 × 32 的共享 tile，可以把它声明为 `[32][33]`，即多加一列填充，使每行现在占用 33 个 float。这 1 个元素的额外偏移意味着，当 warp 中第 `k` 个线程访问 `tile[i][k]` 时，相邻各行的起始地址会在共享内存的各个 bank 之间错开。这样就能避免所有线程命中同一个 bank，如图 7-6 所示。

把步长改成 33 之后，当访问某一给定列时，warp 中不会再有任何两个线程争用同一个 bank。这就消除了原本会出现的 32 路 bank 冲突。

这种填充带来的额外开销可以忽略不计：对于宽度为 32 的分块，内存开销只增加约 3%，却能完全消除冲突，从而大幅提升性能。也别忘了，Blackwell 每

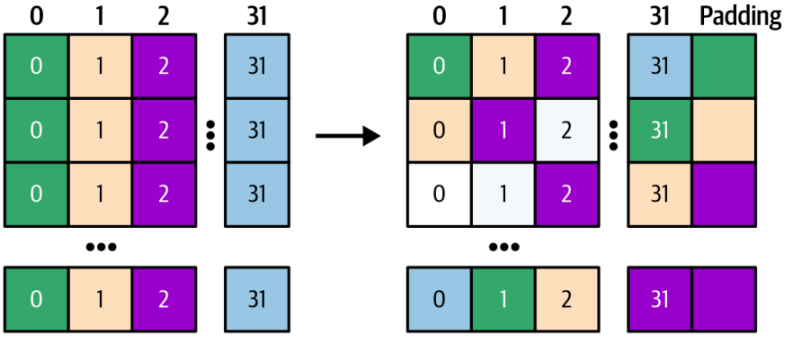

图 7-6　用填充（padding）避免 bank 冲突

个 SM 拥有超过 200 KB 的共享内存，对 32 × 32 的分块而言，3% 的内存开销只有 1 KB。用它换取性能提升非常值得。

下面以一个简单的转置 kernel（内核）为例，演示如何消除共享内存的 bank 冲突。在本例中，每个线程访问的共享内存地址与其他线程落在同一个 bank 中，这会使访存操作串行化、无法并行，性能因而很差。下面就是会引发 bank 冲突的转置 kernel 的原始实现。

改造前示例（C++）：存在 bank 冲突的朴素转置：

```
#include <cuda_runtime.h>
#define TILE_DIM 32
__global__ void transposeNaive(const float *idata, float *odata, int
 ↪  width) {
    __shared__ float tile[TILE_DIM][TILE_DIM];
    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;
    // warp 中的各线程写入同一行
    tile[threadIdx.y][threadIdx.x] = idata[y * width + x];
    __syncthreads();
    // 以转置后的索引读取共享内存
    // 这是同一个 warp 内所有线程
    // 命中同一个 bank 从而引发 bank 冲突的典型情形
    // 从共享内存中按转置读出并写出
    odata[x * width + y] = tile[threadIdx.x][threadIdx.y];
}

int main() {
    const int N = 1024;
    size_t size = N * N * sizeof(float);
    float *h_idata = (float*)malloc(size);
    float *h_odata = (float*)malloc(size);
    // 对输入 h_idata 初始化...
    float *d_idata, *d_odata;
    cudaMalloc(&d_idata, size);
    cudaMalloc(&d_odata, size);
    cudaMemcpy(d_idata, h_idata, size, cudaMemcpyHostToDevice);
    dim3 block(TILE_DIM, TILE_DIM);
    dim3 grid(N / TILE_DIM, N / TILE_DIM);
    transposeNaive<<<grid, block>>>(d_idata, d_odata, N);
    cudaDeviceSynchronize();
    cudaFree(d_idata);
    cudaFree(d_odata);
    free(h_idata);
    free(h_odata);
```

```
    return 0;
}
```

在这个 kernel 中，向共享内存写入 tile 的操作是按行优先（`tile[ty][tx]`）的，因而是合并访存：每个 warp 以连续方式将一整行 32 个浮点数写入共享内存。（注意：若用列优先的 `tile[tx][ty]` 写入 tile，warp 将以 128 字节为步长访问，从而触发 32 路 bank 冲突。）相比之下，读取 tile 的操作是转置的，因为每个线程读取时以 `threadIdx.y` 为行索引、`threadIdx.x` 为列索引。

于是，对于某个 warp 中线程 0–31 上固定的 `threadIdx.y`，所有线程都在访问 `tile[固定行][变化列]`，也就是使用相同的行索引和不同的列索引。这意味着这 32 个地址全部位于 tile 数组的同一行内，而一行恰好落在单个 bank 的存储段上，从而造成 32 路 bank 冲突。结果是，这些读取操作被串行化了 32 倍。

> **注** 请记住，Blackwell 的共享内存有 32 个 bank，恰好与一个 warp 中的线程数 32 相同。因此，当所有线程索引到同一个 bank 时——也就是固定行、变化列的情形——总会造成完整的 32 路冲突。这种一一对应关系意味着，任何在 warp 粒度上错位的访问模式都会迫使每个线程在同一个 bank 上串行排队——无论 GPU 架构如何演进，这一点都不会改变。

此时，32 个线程试图读取 32 个不同的地址，却全部命中 bank 0，导致访存被严重串行化，性能很差。下面我们用填充来消除这些冲突。

改造后示例（C++）：带填充的转置（避免 bank 冲突）。这里我们加入少量填充——一个多余的无用列——使共享内存的每一行从不同的 bank 取模位置开始。这样，bank 索引的碰撞便因这一偏移而被消除：

```
#include <cuda_runtime.h>
#define TILE_DIM 32
#define PAD 1  // 用填充列规避 bank 冲突
__global__ void transposePadded(const float *idata, float *odata, int
 ↪ width) {
    // 每行改为 TILE_DIM+1 个元素，以错开 bank 映射
    __shared__ float tile[TILE_DIM][TILE_DIM + PAD];
    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;
    tile[threadIdx.y][threadIdx.x] = idata[y * width + x];
    // 稍后我们会将其优化为使用线程块作用域的
    // cooperative-groups 屏障
    __syncthreads();
    odata[x * width + y] = tile[threadIdx.x][threadIdx.y];
}
```

（主机端代码与设置与朴素版本相同，为节省篇幅而省略。）

> **注** PyTorch 本身并没有提供任何用于共享内存填充的高级 API，因此你得在 CUDA kernel 中手动实现，再通过 `torch.utils.cpp_extension` 加载。不过在实际应用中，PyTorch 依赖 cuDNN 和 cuBLAS 这类优化库，它们在底层实现了避免 bank 冲突、最大化吞吐量的技术。

为共享内存的每一行额外增加一个元素，每行的长度就变成 33 个浮点数。填充 1 个元素改变了索引计算，使各线程访问 `tile[row][col]` 时地址的低 5 位互不相同，而不是所有线程共享同一个 5 位 bank 索引。这保证了每个线程的索引都映射到不同的 bank。

在带填充的 kernel 中，当 warp 读取 `tile[threadIdx.y][threadIdx.x]` 时，线程 0–31 的地址跨越多个 bank，而不是全部命中 bank 0。既然 warp 中每个线程读取时访问的都是不同的 bank，bank 冲突便被彻底消除，如表 7-5 所示。

% 表 7-5 的扁平化文本内容已并入相邻浮动表格，此处不重复排版。

表 7-5 中的 Nsight Compute 指标印证了这一影响：共享内存加载的 bank 冲突降为 0。据 Nsight Compute 统计，消除冲突后共享内存吞吐率从 52% 升至

表 7-5　消除共享内存 bank 冲突的效果。

指标	优化前（无填充）	优化后（填充）
共享内存加载 bank 冲突	480 万	0
共享内存利用率（占峰值的百分比）	52%	100%
warp 停滞比例（内存）	38%	0.5%
kernel 执行时间（ms）	4 ms	1.3 ms（3 倍提升）

100%。warp 停滞比例——即 warp 在共享内存读取上停滞的周期占比——从约 38% 降到接近 0%。

至此，我们成功消除了 bank 冲突，释放出了片上共享内存的全部性能。由于共享内存访问从串行变成完全并行，这一改动使 kernel 的执行时间从 4 ms 缩短到 1.3 ms，提升约 3 倍。

这为共享内存访问恢复了完全的并行性。填充的代价是每 32 个元素多 1 个浮点数（约 3% 的内存开销），相比性能收益微不足道。

填充之外还有一种同样有效的替代方案：swizzling（索引搅乱）。swizzling 是一种编译期的索引变换，它对共享内存所用的线性索引加以“搅乱”，使顺序编号的线程映射到不同的 bank。例如，可以用位掩码对索引做 XOR，或使用模偏移，从而实现无冲突的访问模式。

swizzling 避免了填充带来的那一点点内存开销，同时仍能保证完美的 bank 并行性。填充实现起来更简单，但 swizzling 能以零内存开销达成同样的目标——而且这个词念起来也挺有趣！

> **注**　NVIDIA 的 CUTLASS 库以及其他高性能 CUDA 库，都在分块迭代器（tile iterator）中使用索引 swizzling，确保各线程映射到不同的 bank、避免 bank 冲突，并优化共享内存的使用。

总之，使用共享内存时，关键在于让 warp 中的线程并行访问不同的 memory bank，而不是在同一个 bank 上排队。填充与 swizzling 之类的技术能够提升共享内存读写的效率，从而在一切使用共享内存的场合带来更高的吞吐量和更好的性能。

接下来，我们探讨一种完全绕开共享内存、让线程之间直接通信的技术。

7.5 Warp Shuffle 内建函数：避开共享内存与显式同步

前面介绍的避免 bank 冲突的技术，都假定用共享内存在线程之间通信。但如果我们可以完全不用共享内存——也就没有 bank 冲突问题了呢？

NVIDIA GPU 支持以 warp 为同步单位的原语（warp-synchronous primitives），使同一个 warp 内的线程通过寄存器而非共享内存交换数据。事实上，这些原语只能在单个 warp 内工作，线程与自己的其余 31 个同伴交换数据。这种 warp 内通信完全不经由任何内存 bank，因此不可能发生 bank 冲突。

最常用的是 `__shfl_sync` 内建函数（即 shuffle）。`__shfl_sync` 可以把某个线程的值广播给 warp 中所有其他线程，也可以在不写共享内存的情况下完成 warp 级归约。还请记住，这些内建函数让线程通过寄存器（而非共享内存）交换值，从根本上消除了共享内存的 bank 冲突。

> **注** 现代 GPU 在所有线程访问完全相同的内存地址时会自动广播，因此这种特殊的单地址情形不会发生 bank 冲突。而 warp shuffle 则把广播用于更一般的情形——任意多值的数据访问模式，同样避开了 bank 问题。

假设需要把一个 warp 内 32 个线程各自的部分结果加起来。朴素的共享内存做法是：每个线程把自己的值写入一个共享数组，调用一次同步屏障，然后读取并累加全部 32 个条目。这样做有 bank 冲突的风险，还会带来额外的同步开销。

借助 `__shfl_sync`，可以完全在寄存器中完成树形归约。例如，使用一个名为 `__shfl_down_sync` 的便捷变体来执行蝶形归约：每个线程读取与自己相距若干 lane 的另一个线程所持有的值，如下所示：

```
unsigned mask = __activemask();
float val = threadVal;  // 每个线程的部分和
// 执行蝶形归约：与距离逐渐拉大的 lane 交换值
for (int offset = 16; offset > 0; offset >>= 1) {
    float other = __shfl_down_sync(mask, val, offset);
    val += other;
}
// 循环结束后，lane 0 持有整个 warp 的总和
```

这里，每次 `__shfl_down_sync` 都直接读取另一个 lane 的寄存器，每一步将活跃线程数减半，直到 lane 0 累加出全部的和。由于所有通信都留在寄存器里，没有任何共享内存流量，因此零 bank 冲突，也不需要额外的同步。

简而言之，`__shfl_sync` 及其变体完全在 warp 的执行 lane 内完成操作。由于不使用共享内存，它避免了 bank 冲突；而且它往往更快，因为用的是寄存器，还减少了共享内存指令的数量。

许多高性能的 warp（线程束）级归约完全绕过共享内存，转而使用 shuffle 内建函数（intrinsic）：它们只需几条指令，就能经由寄存器直接交换数值，且不会产生 bank 冲突。CUDA 的 Cooperative Groups API 正是建立在这些原语之上，提供了诸如 `thread_group.shuffle()` 之类的调用来简化 warp 内通信。在现代 NVIDIA GPU 上，`__reduce_sync()` 等辅助内建函数最终也会编译成这些 shuffle 模式，用来完成 warp 内数据交换。

> **注** 请注意，shuffle 只能在单个 warp 内部使用。对于现代 GPU 架构上的 warp 间数据交换，还应考虑使用 cooperative groups 实现的线程块级归约，以及在硬件支持时的线程块簇（thread block cluster）级通信。我们将在第 10 章讨论这些概念。

不过请记住，shuffle 只能覆盖单个 warp 内的 32 个线程。一旦需要跨 warp 传递数据，仍然必须退回到共享内存或全局内存，并配合正确的同步。

在第 10 章中，我们将探讨 warp 间的多种模式，例如 cooperative group（CG）同步原语、多 warp shuffle 方法以及线程块簇（CTA cluster）。这些技术在底层使用的都是同一批硬件内建函数。同时掌握 warp 内与 warp 间技术至关重要，因为内存相关瓶颈始终是 GPU 性能问题最常见的成因之一。

7.6 只读数据缓存

当所有线程读取相同的值，或者某线程反复读取不会变化的数据时，若不利用 GPU 的缓存机制，性能就可能被卡在瓶颈上。例如，设想自然语言处理（NLP）模型中的嵌入向量（embedding vector）这类只在推理期间读取的只读数据：许多线程可能需要并行访问同一向量，而天真的实现可能每次都从全局内存读取，尽管这些数据不可变、本可以缓存在片上。

需要说明的是，这里所说的只读缓存不同于前面 GPU 内存层级一节讨论过的 64 KB 常量内存缓存。它是一个用于存放不可变数据的大容量缓存；相比之下，常量内存缓存对大型数组来说太小。现代架构依靠更大的 L1/只读缓存来处理这类嵌入向量查找，而不是试图把这些数据挤进小小的 64 KB 常量内存缓存。

在现代 GPU 上，全局内存加载会自动缓存到 L2，通常也会缓存到 L1。你可以把函数参数声明为经 `const __restrict__` 限定的指针，从而将其定义为非一致（non-coherent）、以读为主（相对于只读）的数据。对于只读数据，只要编译器能证明不可变性、无别名与访问安全，就可能把加载路由到这条只读路径。这样，编译器/硬件便能把不变的数据送入只读 L1 缓存——它延迟更低，对广播式访问尤其如此，而且不会挤占其他缓存数据。

> **注** 在现代 CUDA 中，通常不再需要显式调用 `__ldg()`。只要指针带有 `const __restrict__` 限定，且编译器能确认访问安全，它就可能在全局内存加载时使用只读数据缓存。较老的 `__ldg()` 内建函数仍可用于显式控制，但在现代编译器下一般并无必要。

一个常见的性能陷阱是忘记告诉编译器某个缓冲区是真正的只读数据。这样，编译器就不会使用只读（非一致）数据路径，也不会把这些加载送入专用的只读缓存；相反，每一次访问

都会变成普通的全局内存加载，带来冗余的 DRAM 流量和本可避免的缓存缺失。

对这样的 kernel 做性能分析时，你会看到相同的地址被反复地从片外内存读取，指令流中看不到任何 `__ldg()` 操作，该数组的 L2 命中率低得出奇，DRAM 吞吐量居高不下，看起来就像一个完全未经缓存的工作负载。

解决办法是启用只读路径：把数据标记为 `const __restrict__`，或者显式使用 `__ldg()` 内建函数来加载。这等于告诉硬件这些数据不会被修改，于是可以放入专用的只读缓存——它与 L1 并列，对广播式加载具有更低延迟。

当 warp 发出一次常量缓存加载（`__constant__`，或在较老 GPU 上等价的统一 `__ldg()`）时，只要 32 个 lane 命中同一地址，硬件用一次事务就能服务于全部 32 个 lane：它把该值广播给每个线程，只耗用一个周期，而不是各做 32 次独立加载。这种 warp 级广播对查找表、系数等统一数据同时降低了延迟与内存带宽占用，几乎等于免费地让每个 warp 只取一次共享常量，而不是取 32 次（每线程 1 次）。

为了展示标准只读缓存带来的缓存收益，假设有这样一个 kernel：每个线程从大小为 $T = 1,024$ 的表中查出一个值并写入输出。这模拟了嵌入查找（embedding lookup）模式，下面同时给出它的 CUDA C++ 与 PyTorch 版本：

```
#include <cuda_runtime.h>
#define T 1024
__global__ void naiveLookup(const float* table, float* out, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        // __ldg not used here, each access goes to
        // global memory without using read-only cache
        int t = idx % T;
        out[idx] = table[t];
    }
}
int main() {
    const int N = 1 << 20;
    float* h_table = nullptr;
    float* h_out = nullptr;
    cudaMallocHost(&h_table, T * sizeof(float));
    cudaMallocHost(&h_out, N * sizeof(float));
    for (int i = 0; i < T; ++i) h_table[i] = float(i);
```

```
    float *d_table, *d_out;
    cudaMalloc(&d_table, T * sizeof(float));
    cudaMalloc(&d_out, N * sizeof(float));
    cudaMemcpy(d_table, h_table, T * sizeof(float),
        cudaMemcpyHostToDevice);
    dim3 block(256), grid((N + 255) / 256);
    naiveLookup<<<grid, block>>>(d_table, d_out, N);
    cudaDeviceSynchronize();
    cudaMemcpy(h_out, d_out, N * sizeof(float), cudaMemcpyDeviceToHost);
    cudaFree(d_table);
    cudaFree(d_out);
    cudaFreeHost(h_table);
    cudaFreeHost(h_out);
    return 0;
}
```

下面给出与上述 CUDA C++ 天真版本大致对应的 PyTorch 实现：

```
import torch
def vectorized_lookup(table, N):
    flat = table.view(-1)
    T = flat.size(0)
    # build indices [0,1,2,...,N-1] % T all on GPU
    idx = torch.arange(N, device=flat.device) % T
    # one gather kernel does all N loads in parallel
    return flat.index_select(0, idx)
# Usage
T = 1024
N = 1 << 20
table = torch.arange(T, dtype=torch.float32,
                     device='cuda')
out = vectorized_lookup(table, N)
```

在这些天真版本中，首次访问之后，每次查找大概率都能命中 L2（因为 L2 会缓存这些数据），但并没有用到专用的只读缓存。硬件可能仍将其当作普通全

局数据对待，这可能挤占其他有用的缓存数据；而且当 warp 中多个线程读取同一个 table[t] 时，也无法充分享受广播式缓存的好处。

现在我们优化这个 kernel，把表标记为 const __restrict__。这会提示硬件使用只读缓存路径，如下所示：

```
#include <cuda_runtime.h>
#define T 1024
__global__ void lookup(const float* __restrict__ table,
        float* out, int N) {


    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        int t = idx % T;
        // Compiler can turn this into a load from the
        // read-only cache for faster loads
        out[idx] = table[t];
    }
}
int main() {
    const int N = 1 << 20;
    float* h_table = nullptr;
    float* h_out = nullptr;
    cudaMallocHost(&h_table, T * sizeof(float));
    cudaMallocHost(&h_out, N * sizeof(float));
    for (int i = 0; i < T; ++i) h_table[i] = float(i);
    float *d_table, *d_out;
    cudaMalloc(&d_table, T * sizeof(float));
    cudaMalloc(&d_out, N * sizeof(float));
    cudaMemcpy(d_table, h_table, T * sizeof(float),
     ↪ cudaMemcpyHostToDevice);
    dim3 block(256), grid((N + 255) / 256);
    lookup<<<grid, block>>>(d_table, d_out, N);
    cudaDeviceSynchronize();
    cudaMemcpy(h_out, d_out, N * sizeof(float), cudaMemcpyDeviceToHost);
    cudaFree(d_table);
```

```
    cudaFree(d_out);
    cudaFreeHost(h_table);
    cudaFreeHost(h_out);
    return 0;
}
```

> **注** 在 PyTorch 场景下，同样可以用 torch.utils.cpp_extension 编写一个小型 CUDA 扩展，把嵌入表的指针声明为 `const __restrict__`，从而实现同样的技巧。你也可以使用 PyTorch 编译器来优化这段代码。我们将在第 13 章和第 14 章深入讨论 PyTorch 编译器。

简言之，把表标记为 `const __restrict__` 就是告诉编译器这些值不可变且无别名，从而在安全的前提下启用只读数据路径。这会提高缓存命中率并减少片外流量。表 7-6 给出了这一改动前后 Nsight Compute 的测量结果。

表 7-6 只读缓存带来的收益。

指标	改动前（未用 __restrict__）	改动后（使用 __restrict__）
全局内存加载效率	52%	97%
DRAM 吞吐量（GB/s）	600 GB/s	200 GB/s
L2 读吞吐量（GB/s）	1,500 GB/s	1,800 GB/s
SM Active %	45%	93%
kernel 执行时间（ms）	2.5 ms	1.0 ms

%（抽取说明：本条源文本是表 7-6 的扁平化表格正文，内容已完整承载于上方表 7-6 的表格浮动体中，为避免重复排版，此处不再单独呈现。）

加入 `const __restrict__` 后，kernel 耗时提升约 2.5 倍，从 2.5 毫秒降至 1.0 毫秒，因为 warp 在 DRAM 上停顿等待的时间更少了。SM Active 占比从 45% 升至 93%，表明几乎每个周期都有活跃工作，而不是在空等内存。

DRAM 吞吐量从 600 GB/s 降到 200 GB/s，而随着更多请求在缓存内得到满足，L2 读吞吐量从 1,500 GB/s 升到 1,800 GB/s。全局内存加载效率从 52% 提升到 97%，证实绝大多数取回的缓存行都装有有效数据。

> **注** Nsight Systems 提供整体 GPU 活动的时间线视图，而 Nsight Compute 则报告 SM Active 占比等逐 kernel 指标。需要定量的逐 kernel 分析时，请使用 Nsight Compute。

由于更多流量由片上缓存承担，而不必往返 DRAM，计算单元得以持续得到数据供给，算术强度随之提升。正是这种平衡，让 kernel 从内存受限区间走向计算受限区间。对于具有强二维或三维局部性的只读和读写访问模式，还可以使用纹理对象和表面对象：把数组绑定到 `cudaTextureObject_t`，再用 `tex1Dfetch` 或 `tex2D` 访存，硬件便能利用空间局部性获得高缓存命中率，并享有环绕取址、插值等特性。表面对象则支持在类似访问模式下执行写入。

> **注** 虽然纹理引用（texture reference）与表面引用（surface reference）API 已被弃用，但纹理对象与表面对象 API 仍受支持，适合具有二维或三维局部性的访问模式。不过，对大多数涉及一维数据的 AI 工作负载而言，把只读数据缓存与常量内存配合使用要简单得多，也更受推荐。

总而言之，把只读数据标记为 `const __restrict__` 即可利用低延迟的只读缓存，削减 DRAM 流量并提升 SM 活跃度。只要访问模式具有普通缓存

难以妥善处理的 2D/3D 局部性，就应考虑使用纹理或表面内存。这些技术相互配合，消除内存停顿、提升缓存利用率，为内存受限的 kernel 带来可观的性能提升。

7.7 异步内存预取与张量内存加速器

在前面的章节中，我们已经看到，把数十次 4 字节读取合并为一次 128 字节事务，能显著提升全局读取的效率，并减少每次请求中浪费的扇区。然而，即便是合并得近乎完美的读取，仍会让 warp 停顿整整一次 DRAM 往返的时间。

以 Blackwell 为例，在任何计算开始之前，一次完整的 DRAM 往返就要耗费数百个周期。要隐藏这段延迟，就必须让数据传输与计算重叠进行。正是这种重叠隐藏了大部分 DRAM 延迟。

CUDA 的 Pipeline API 与张量内存加速器（TMA）硬件引擎把这一思路推进到线程块级别：不再让每个 warp 动用 SM 的读取/存储（LD/ST）单元从全局

内存取数，而是调用 TMA 引擎，把整个 tile 从全局内存异步取入共享内存，如图 7-7 所示。

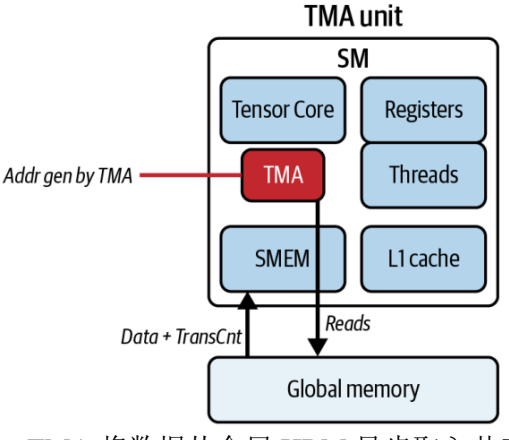

图 7-7　TMA 将数据从全局 HBM 异步取入共享内存

要启动 TMA 传输，只需调用 `cuda::memcpy_async()`。在现代 GPU 架构上，`cuda::memcpy_async()` 与 `cuda::pipeline` 一道，暴露出用于异步全局到共享内存传输的硬件引擎——在可用时会用到 TMA，由其片上 DMA 引擎执行异步的批量数据传输。在 CUDA 中的实现方式如下：

```
cuda::memcpy_async(sharedBuf,
                   globalPtr + offset,
                   cuda::aligned_size_t<16>(bytes),
                   pipe);
pipe.producer_commit();
```

当 TMA 负责批量拷贝——包括合并、跨步传输乃至多维传输——时，kernel 就对上一个 tile 进行计算。这就是所谓的双缓冲（double buffering），亦称乒乓（ping-ponging）。

通过 TMA 实现双缓冲之后，SM 的读取/存储单元被解放出来去干真正的工作，因为 TMA 的 DMA 引擎在后台替我们搬运数据。实际上，数据搬运变成了异步的：当 TMA 把下一个 tile 的数据流式送入时，SM 的 warp 正对上一个 tile 进行计算。正是这种重叠隐藏了 800 个周期的 DRAM 延迟。

具体而言，TMA 能够在全局内存与共享内存之间执行 1D–5D 的批量拷贝以及任意步长的传输，且不会阻塞 SM 的指令流水线。把这些传输从 SM 卸载给 TMA 后，kernel 发出的 LD/ST 指令大幅减少，省去了额外的同步，让 warp 调度器几乎把每个周期都花在有效计算上，而不是干等内存。

下面的代码片段展示了如何使用 CUDA C++ Pipeline API 与 TMA 硬件引擎，发起一次从全局内存到共享内存的异步拷贝：

```
#include <cuda/pipeline>
#include <cuda_runtime.h>
#include <cstdint>
#include <cassert>
#define TILE_SIZE 1024  // example tile size
// User-provided compute function operating on a shared-memory tile
__device__ void processTile(const float* tile);
__global__ void kernelWithTMA(const float* __restrict__ global_ptr,
                              int nTiles) {
    // Two ping-pong buffers in shared memory
    // On Blackwell (CUDA 13+), prefer 32B alignment for 256-bit vectors
    // (v8.f32 / double4).
    __shared__ __align__(VEC_BYTES) float tile0[TILE_SIZE];
    __shared__ __align__(VEC_BYTES) float tile1[TILE_SIZE];
float* tiles[2] = { tile0, tile1 };

    // Alignment / size guards for vectorized copies
    // --- choose vector width
     ↪  ------------------------------------------------
    #ifndef VEC_BYTES
      // Prefer 32B vectors on CUDA 13+/Blackwell.
      // Define -DUSE_256B_VEC to force.
      #if defined(USE_256B_VEC) || (defined(CUDA_VERSION)&&CUDA_VERSION
       ↪  >= 13000)
        #define VEC_BYTES 32      // 256-bit: v8.f32, double4
```

```
  #else
    #define VEC_BYTES 16      // 128-bit: v4.f32, float4
  #endif
#endif
constexpr int VEC_ELEMS = VEC_BYTES / sizeof(float); // 4 (128b) or
 ↪  8 (256b)
constexpr int WARP      = 32;
// Alignment / size guards for vectorized copies/compute
static_assert((TILE_SIZE % (WARP * VEC_ELEMS)) == 0,
              "TILE_SIZE must be a multiple of WARP_SIZE *
               ↪  VEC_ELEMS");
// Also guarantee the tile byte count
//  is a multiple of the vector width
static_assert(((TILE_SIZE * sizeof(float)) % VEC_BYTES) == 0,
              "Tile byte size must be a multiple of VEC_BYTES");
// On Blackwell (CUDA 13+), prefer 32B alignment
// for 256-bit vectors (v8.f32 / double4).
assert((reinterpret_cast<std::uintptr_t>(global_ptr) % VEC_BYTES)
 ↪  == 0);
size_t bytes = TILE_SIZE * sizeof(float);
// Block-scoped pipeline for TMA
__shared__ cuda::pipeline_shared_state<
    cuda::thread_scope_block, 2> state;
auto pipe =
    cuda::make_pipeline(cuda::this_thread_block(),
        &state);
// Prime pipeline with the first async copy into tile0
pipe.producer_acquire();
cuda::memcpy_async(tiles[0],
                   global_ptr + 0 * TILE_SIZE,
                   cuda::aligned_size_t<32>{bytes},
                   pipe);
pipe.producer_commit();
// Loop over the remaining tiles
for (int t = 1; t < nTiles; ++t) {
    // Wait for the previous copy to finish, then compute on it
```

```
        pipe.consumer_wait();
        processTile(tiles[(t - 1) & 1]);
        pipe.consumer_release();
        // Enqueue the next async copy into the alternate buffer
        pipe.producer_acquire();
        cuda::memcpy_async(tiles[t & 1],
                           global_ptr + t * TILE_SIZE,
                           cuda::aligned_size_t<32>{bytes},,
                           pipe);
        pipe.producer_commit();
    }


    // Final wait and compute on the last tile
    pipe.consumer_wait();
    processTile(tiles[(nTiles - 1) & 1]);
    pipe.consumer_release();
}
```

kernel 启动之后，每个线程块立即分配两个共享内存 tile（`tile0`、`tile1`），并构造一个块级作用域的流水线，让线程块中的所有线程协同完成各自的异步 DMA，如下所示：

```
__shared__ cuda::pipeline_shared_state<
    cuda::thread_scope_block, 2> state;
auto pipe =
    cuda::make_pipeline(cuda::this_thread_block(),
        &state);
```

为给流水线灌入首批数据，我们通过 TMA 提交一次异步拷贝：TMA 甚至能合并跨步或多维传输，在后台把字节从全局内存流入 `tiles[0]`，如下所示：

```
pipe.producer_acquire();
cuda::memcpy_async(tiles[0],
                   global_ptr + 0 * TILE_SIZE,
                   cuda::aligned_size_t<32>{bytes},
                   pipe);
pipe.producer_commit();
```

在使用这批数据之前，我们执行以下操作，确保只阻塞到 TMA 恰好完成拷贝为止：

```
pipe.consumer_wait();
processTile(tiles[0]);
pipe.consumer_release();
```

在主循环中，我们交替使用两个缓冲区：先用 `pipe.consumer_wait()` 等待并在上一个 tile 上调用 `processTile()`，接着执行 `pipe.consumer_release()`，然后 `pipe.producer_acquire()` 并向另一个 tile 发起新的 `cuda::memcpy_async()`，最后 `pipe.producer_commit()`。如此往复！

通过在 `tile0` 与 `tile1` 之间乒乓切换，每次新的 `memcpy_async` 都与上一个缓冲区上的 `processTile` 相互重叠。SM 上读取/存储单元的压力随之降低，有了更多指令发射槽可用于计算；与此同时，TMA 在并行搬运数据。这消除了冗余的全局内存读取，降低了同步开销，让 warp 保持忙碌，而不是停等在内存上。

从全局内存到共享内存的异步预取把延迟隐藏在计算背后：线程在处理先前已加载的数据的同时，预先把接下来的数据装入共享内存。

这一模式在内存受限的循环和张量计算中尤其有效。在现代 GPU 上，TMA 可以在对当前 tile 进行处理的同时，为矩阵乘法流式输送下一个 tile。

> **注** 在全局内存与 SMEM/TMEM 之间搬运 2D 及 N 维 tile 的成块拷贝时，TMA 是首选路径。应优先在块级作用域使用 `cuda::memcpy_async` 配合 `cuda::pipeline`；在 Hopper/Blackwell 上，当对齐与传输方向满足条件时（例如全局内存与共享内存之间），实现会自动启用 TMA（`cp.async.bulk.*` 指令族）。

我们用一次合并的整块全局拷贝加上大量快速的共享内存读取，取代了多次分散的全局读取。考虑到 DRAM 与片上 SRAM 之间的巨大差距，这样做非常划算。表 7-7 总结了基于 TMA 的双缓冲实现前后 Nsight Compute 指标的对比。

表 7-7　朴素 kernel（无预取）与 TMA 加速双缓冲的对比。

指标	改进前（无预取）	改进后（异步预取）
全局内存读取效率	23%	99%
平均每次请求扇区数	6.4	4.0
DRAM 吞吐量（占峰值百分比）	25%	90%
SM 活跃占比	62%	100%
kernel 执行时间	18 ms	7 ms

这里可以看到 SM 活跃占比接近 100%，说明 SM 几乎在每个周期都有活跃的 warp。全局内存读取效率从 23% 提升到 99%，意味着取回的每个字节几乎都被有效利用。

平均每次请求扇区数从 6.4 降到约 4.0，表明请求能整齐地映射到缓存中 128 字节的行上。DRAM 吞吐量从峰值的 25% 升至 90%，总耗时从 18 毫秒缩短到 7 毫秒，提速约 2.6 倍。这些结果印证：把批量拷贝卸载给 TMA，并在共享内存缓冲区之间乒乓切换，能让 GPU 保持忙碌，把大部分 DRAM 延迟隐藏在有效工作之后。

NVIDIA 的 CUDA pipeline API 加上 TMA，是软硬件协同设计的教科书式范例。Pipeline API 恰好把 TMA 的能力暴露出来——而 TMA 硬件则精确支持 `cuda::memcpy_async` 所需要的异步、合并、多维拷贝。

这套 API 与 TMA 的 DMA 引擎是携手开发的，因此开发者可以用高层流水线操作，来表达与硬件传输能力紧密对应的工作。

这使得内存搬运与计算得以高效重叠，从而提升性能。

> **注**　几乎在所有情况下，编写涉及全局内存到共享内存分块的 CUDA kernel 时，都应使用所能获得的最高层、最新的 API，其中包括 CUDA Pipeline API（`cuda::memcpy_async`）。这些 API 和相关库在不断演进，会透明地利用 TMA 等最新硬件特性来执行批量、跨步以及 2D/3D 传输；此外，它们还

支持线程块集群上的组播（multicast）等高级性能优化特性。使用这些 API 时，上述一切都能“免费”获得——无需修改任何代码。

总之，当内存访问成为 kernel 性能的瓶颈时，应结合精心的分块设计、双缓冲和 TMA 驱动的异步预取，来卸载并重叠数据搬运。把 tile 分级暂存在共享内存中，并使用 `cuda::memcpy_async` 配合 `pipe.producer_commit` 与 `pipe.consumer_wait`，就能把合并后的多维 DMA 传输交给 TMA，从而卸载全局到共享内存的传输。

利用 TMA 卸载内存搬运，有助于减轻 SM 加载/存储单元的压力，让计算流水线保持满载。这样一来，SM 专注于计算，而共享内存流量则走片上 TMA 通路。在 Blackwell 大带宽的 HBM3e 存储结构上，这些技术对于隐藏 DRAM 延迟、维持峰值吞吐量、把内存受限的 kernel 变成接近计算受限的高效主力而言不可或缺。

7.8 本章要点

通过合并访问、数据复用和异步传输来优化 GPU 上的内存访问模式，可以使 kernel 从内存受限逼近硬件的峰值能力。一些与 GPU 架构更契合的小幅代码改动（例如合理的线程分组、使用共享内存、避免 bank 冲突）就能带来巨大的性能提升。以下是本章的要点：

全局内存合并

当每个 warp 的访问尽量落入尽可能少的 128 字节缓存行时，就实现了合并。合理安排数据布局与线程索引，使每个 warp 的线程读取连续的 4 字节字，让硬件将其融合为少量 128 字节事务。合并的内存加载能最大化有效 DRAM 带宽（即全局内存加载效率，Global Memory Load Efficiency），并把每次请求的平均扇区数降到最优的 4.0。在使用较新版本的 Nsight Compute 时，你还可以使用下面这些计数器来分析和优化内存合并：

```
also use sm__sass_data_bytes_mem_* counters and gpu__dram_throughput
.avg.pct_of_peak_sustained_elapsed to profile and optimize memory
coalescing.
```

向量化加载/存储

使用 `float4` 等内置向量类型来表示 16 字节向量。在 Blackwell 且 CUDA 13 及以上版本上，如果能够证明 32 字节对齐，则优先采用每线程 32 字节向量，例如 `double4` 或自定义结构体 `alignas(32) { float v[8]; }`。这样可以减少每字节的指令数，并在对齐良好时使每次请求的扇区数保持在理想的 4.0。如此一来，每条指令中每个线程就能搬运尽可能多的元素。每个 warp 的 128 字节事务数随请求的总字节数而增加。请注意对齐：确保数组至少以 16 字节对齐（`float4` 所需）；`cudaMalloc` 默认即满足此要求，通常按 256 字节对齐。未对齐的向量访问将失去这些收益。

避免 bank 冲突

为共享内存数组加填充（例如，为适配 32 线程的 warp，把行长设为 33 个浮点数），确保不会有两个线程在同一周期命中同一个 bank。消除 bank 冲突可以恢复共享内存的完整吞吐量。也可以尝试 swizzling（混洗），它比填充方式的内存开销略低。

共享内存分块与数据复用

把工作集暂存在片上共享内存中（例如将矩阵按 32×32 的块分块），使每个元素只从 DRAM 取回一次，却在 SM 上被使用多次。这会提高算术强度，使 kernel 趋向计算受限。

只读数据缓存

将小型静态查找表或系数标记为 `const __restrict__`，让编译器在适用时将加载路由到只读数据通路。统一广播的延迟低于 DRAM，可以避免冗余事务，并能直接由片上缓存提供服务。

用流重叠主机与 GPU 之间的拷贝

将主机缓冲区分配为页锁定（“pinned”）内存，并在多个流上使用 `cudaMemcpyAsync`，使 H2D/D2H 传输与 kernel 执行相重叠。pinned 内存支持异步 DMA 传输，而多个流可以让拷贝与 kernel 执行重叠，从而隐藏 PCIe 或 NVLink 的传输延迟。优先使用带显式流和事件的 `cudaMemcpyAsync` 来重叠 H2D/D2H 传输与 kernel。切记，可分页（非 pinned）内存会使 DMA 重叠失效。你应当核实实

际使用的是可分页内存还是 pinned 内存，因为观测到的传输速率会因这一配置而有很大差异。

用 TMA + Pipeline API 进行异步预取

在满足对齐与作用域要求时，使用 C++ libcu++ 的 barrier 和 pipeline API（`cuda::barrier` 与 `cuda::pipeline`）配合 `cuda::memcpy_async` 驱动 TMA（`cp.async.bulk.tensor`）执行全局内存 → 共享内存的批量拷贝。它将合并的、跨步的（乃至多维的）拷贝卸载到共享内存，并通过双缓冲与计算重叠。这能减轻 SM 的 LD/ST 单元压力，让 SM 专注于计算。

需要注意的是，libcu++ 的 pipeline API 与 TMA 仍在持续演进。对于两阶段的乒乓缓冲，优先使用分阶段的接口（例如 `producer_acquire()`、`producer_commit()`、`consumer_wait()`、`consumer_release()`）。除非确实需要 cluster 作用域或 grid 作用域的拷贝，否则请使用块级管线（例如 `cuda::thread_scope_block`）或块级 barrier（例如 `cuda::barrier<cuda::thread_scope_block>`）。

以性能分析为指导

借助 Nsight Compute 的各项指标，例如全局内存加载效率、每次请求的平均扇区数、共享内存 bank 冲突、SM 活跃率（SM Active %）、warp 停顿原因等。同时查看 Nsight Systems 的时间线，将重叠与停顿可视化，定位瓶颈，并逐一验证每项优化。

7.9 小结

当你的内存访问流水线各环节都火力全开——合并的全局内存加载、无冲突的分块、向量化取数、只读缓存以及 TMA 驱动的预取——最大的数据搬移瓶颈已被消除，SM 得以全速运转。

本章中，我们始终依靠 Nsight Compute 和 Nsight Systems 来准确指出 warp 在哪里因等待数据而停滞，并用它们逐步确认每项优化确实减少了停顿、压缩了无效事务、提升了持续带宽。今后调优任何新 kernel 时，这些工具仍是你的指北星。

下一章将介绍 CUDA 与 GPU 编程中一些基本的延迟隐藏技术，包括调节占用率、提高 warp 效率、避免 warp 分化以及暴露指令级并行。

第 8 章 占用率调优、warp 效率与指令级并行

现代 GPU 加速的工作负载正把硬件推向极限。Blackwell 这类多裸片 GPU 通过 10 TB/s 的 NV-HBI 互连将多个受光刻限制的裸片连接起来，并把 L2 缓存扩大到 126 MB。这些硬件设计选择切实改变了内存与计算之间的权衡以及占用率的最佳区间，这使得性能分析与优化比以往任何时候都更加关键。在内存优化的基础之上，我们将转向更先进的延迟隐藏技术与吞吐量增强手段，以充分释放现代 GPU 的全部算力。

我们将聚焦于识别性能瓶颈，然后应用一套系统化的优化策略来逐一消除它们。本章的关键主题包括占用率调优、warp 效率优化以及提升指令级并行度。

学完本章，你将能够找出 GPU 利用率不足的根本原因，并应用恰当的优化组合。我们还会为后续章节介绍的更高级技术做好准备，例如 kernel 融合，以及利用 CUDA Graphs 和 CUDA 流等原语实现的流水线。

尽管我们的重点是 CUDA C++ 这样的高层语言以及 PyTorch 这样的框架，性能分析与调优的原则却适用于技术栈的所有层次，一直深入到硬件本身。因此，理解底层硬件的性能对于诊断那些高层抽象难以完全定位的瓶颈，仍然至关重要。

8.1 性能分析与 GPU 瓶颈诊断

在优化之前，我们必须先识别代码中的瓶颈，确定究竟是哪一种硬件或软件资源限制了性能。现代 NVIDIA GPU 非常复杂，性能下降可能来自许多方面，包括内存带宽、内存延迟、指令吞吐量、同步开销、并行度不足、主机与设备之间的传输延迟等。

NVIDIA 的性能分析生态包括 Nsight Systems（命令行接口为 `nsys`）和 Nsight Compute（命令行接口为 `ncu`）。Nsight Systems 可捕获系统级时间线，涵盖 CPU 线程、GPU kernel 和内存传输；它还能捕获 Python 回溯和 Python 采样。

将 PyTorch profiler 和各种可视化工具结合起来，Nsight Systems 与 Nsight Compute 可以帮助你诊断 kernel 性能瓶颈、分析 Roofline 图，并衡量迭代优化的成效。

8.1.1 Nsight Systems 时间线视图

Nsight Systems 的时间线视图有助于精确定位并发问题、传输开销和空闲时段。例如，运行以下代码可以生成一份详细的时间线，展示 kernel 启动的重叠情况、CPU 准备间隙、数据传输计时以及由 NVTX 标注的范围：

```
nsys profile \
    --trace=... \
    --capture-range=... \
    --force-overwrite=true \
    <application>
```

此外，Nsight Systems 的 GUI 允许你以交互方式检查时间线。它可以可视化 CPU 线程、GPU kernel 乃至用户自定义的 NVTX 范围，并提供缩放与平移功能，便于进行细致的分析（见图 8-1）。

请记住，对于复杂应用，NVTX 标注是必不可少的。应在代码中使用 NVTX 范围来标记感兴趣的区域，随后用 Nsight Systems 捕获范围配置文件。虽然 CUDA profiler 的 Start 和 Stop API 也支持用于采集控制，但对于框架类工作流，NVTX 范围是推荐的机制。例如，你可以使用 NVTX 的范围 push/pop（在 PyTorch 中通过 `torch.cuda.nvtx` 提供）为“前向传播”“反向传播”等阶段打上标签。这样，Nsight Systems 的时间线就远比之前易于解读，因为性能分析器能够捕获性能关键的迭代，并清晰地划分出关键的计算片段。

8.1.2 数据管线的性能分析与调优

如前所述，Nsight Systems 提供了整个管线的高层视图，其中包括数据加载和处理环节。例如，如果你看到 GPU 空闲而 CPU 正忙于准备数据或执行数据增强（这在训练循环中很常见），那么瓶颈可能不在于 GPU kernel，而在于数据管线本身。

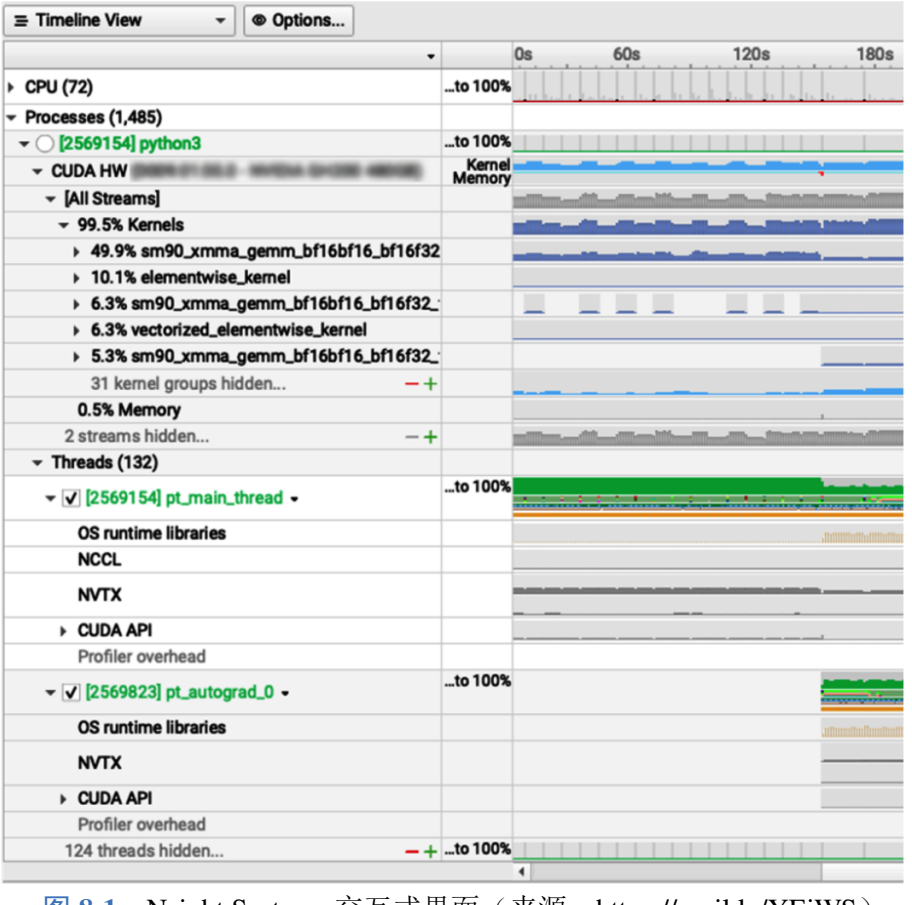

图 8-1　Nsight Systems 交互式界面（来源：https://oreil.ly/YEiWS）

当在真实且具有代表性的数据集上确认瓶颈是数据管线时，你可以调节数据加载线程的数量，利用双缓冲将 CPU 预处理与 GPU 计算重叠起来，或者把更多的预处理工作搬到 GPU 上执行。

> **注**　务必确认你所感知的“GPU 性能问题”并非实际上由 kernel 执行的上游或下游环节（例如数据加载）所造成。

在通过性能分析获得清晰的整体图景之后，我们接下来着手解决具体的瓶颈。接下来的小节将详细讲解核心优化技术。我们先从占用率调优讲起，因为充足的 warp 供给是隐藏延迟并最大化吞吐量的根本。

8.1.3 Nsight Compute 与 Roofline 分析

Nsight Compute（`ncu`）则是一款针对单个 kernel 采集深入指标的性能分析器。例如，它会跟踪实际占用率、每周期发射的 warp 指令数、内存吞吐量（GB/s）、执行单元利用率等众多指标。这些指标合在一起，勾勒出系统性能状况的完整图景。

这些指标按内存、计算、吞吐量等分节组织。Nsight Compute 的自动化分析规则会标记出低内存利用率、分支发散等低效之处，甚至给出优化提示。这些内置规则会随新 GPU 架构不断更新。它们可以依据每字节 FLOPS 之比、停顿原因等指标，快速判断你是内存受限、延迟受限还是其他情况。

Nsight Compute 还包含 Roofline 分析小节和自动化指导。它可以将你的 kernel 实际达到的 FLOPS 与硬件 Roofline 上限对照绘图，甚至在你接近内存带宽或计算极限时给出高亮提示。

请记住，内存与计算的区分是用 Roofline 模型来量化的，该模型将 kernel 的性能对照内存带宽和计算吞吐量的硬件上限进行绘制。Nsight Compute 现在直接提供 Roofline 分析，显示每个 kernel 相对峰值的实际 GFLOPS 及其算术强度（每字节 FLOPS）。低于内存 Roof 线的 kernel 表明存在内存受限行为，而贴近计算 Roof 线的 kernel 则属于 ALU 受限。通过 Nsight Compute 的 Roofline 小节，你可以直接获取算术强度（FLOPs/字节）和按精度划分的 FLOPs。将其与硬件理论峰值 FLOPS/字节相比较，就能看出 kernel 距离 Roofline 有多远。图 8-2 展示了一个 Roofline 图的示例。

通常有效的做法是：先用 Nsight Systems 在时间线上找到热点 kernel 或瓶颈操作，再用 Nsight Compute 对这些 kernel 逐一深入，进行更细粒度的分析与诊断。这种从系统级视图到 kernel 级深入剖析的两步工作流，是处理复杂 GPU 性能调试与调优的常用方法。

8.1.4 PyTorch Profiler 与可视化工具

在使用 PyTorch 这类高层框架时，`torch.profiler` API 可以在模型训练/推理期间采集类似的性能指标。PyTorch profiler 在底层借助 Kineto 库来完成实际的数据采集。

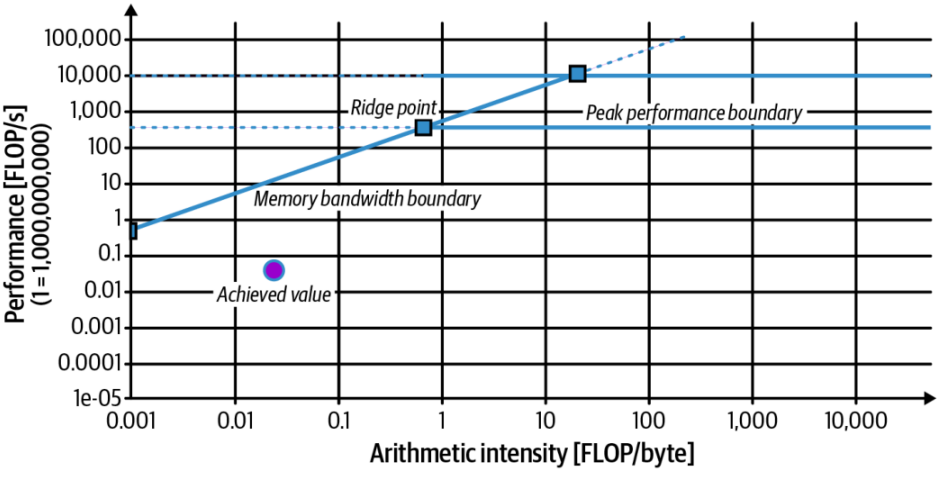

图 8-2　Nsight Compute 界面中显示的 Roofline 图（https://oreil.ly/wUbIz）

Kineto 与 CUDA 的性能工具接口（CUPTI）后端相集成，以捕获算子级执行时间、GPU kernel 启动、内存拷贝和硬件计数器指标；它还与 Linux perf 集成来记录 CPU 事件。Kineto 将所有这些信息合并为一份一致的、按时间排序的跟踪记录，可以借助 PyTorch profiler 界面、Nsight Systems GUI 或直接用 Chrome 浏览器（例如 Chrome tracing 格式）进行可视化。图 8-3 给出了一个 Chrome tracing 可视化的示例。

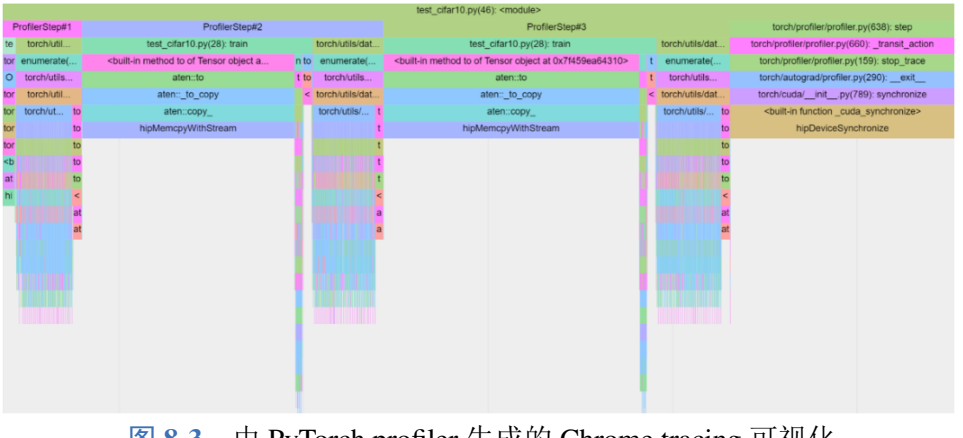

图 8-3　由 PyTorch profiler 生成的 Chrome tracing 可视化

PyTorch profiler 让你可以直接在模型代码中识别瓶颈算子。例如，你可以用 `torch.profiler.profile(..., with_flops=True, profile_memory=True)`

来分析训练循环，以记录内存用量，并为矩阵乘法等受支持的算子估算 FLOPs。（注意：这些是基于公式的算子级估算，而非逐 kernel 的硬件计数器。）这样的集成为 PyTorch 模型代码与底层 CUDA 性能分析之间架起了桥梁。

现代版本的 Nsight Systems 可以采集 Python 回溯采样，还提供面向 PyTorch 的模式，支持 Python 调用栈采样以及一个用于更关联化跟踪的 PyTorch 域。这有助于将框架活动与系统时间线关联起来。Nsight Compute 则可与 CUDA C/C++ 源码以及 PTX 或 SASS 关联：用 `-lineinfo` 编译设备代码即可启用源码行映射；要将模型代码与 kernel 关联起来，可在 Python 中通过 `torch.cuda.nvtx` 使用 NVTX 范围。此外，现代版本的 Nsight Compute 还提供源码视图，其中包含指令构成和吞吐量细分，帮助定位受停顿和吞吐量限制影响的源码行。

> **注** 过去无法采集 Python 信息的局限，曾使 PyTorch 开发者对 Nsight Systems/Compute 望而却步。但如果你使用的是 PyTorch/Python，那么重新审视这些工具是值得的，以便获得与系统其余部分更加整体且相互关联的分析。

本节概述一套诊断 GPU 瓶颈的工作流程，包括如何解读关键性能分析指标，以及它们对瓶颈类型的含义。我们将重点介绍 Nsight Compute ——用于深入分析 kernel（包括 warp 停顿与内存吞吐量）——以及 Nsight Systems ——用于更高层面的应用性能分析（包括并发性与 CPU-GPU 重叠）。

8.1.5 性能分析器引导的分析

关键在于把这些洞见用于指导下一步行动，因为不同的瓶颈需要不同的优化手段。例如，你可以利用 Nsight Compute 的引导式分析规则，它可能会标记“内存受限：每 FLOP 的 L2 事务数偏高”或“计算受限：发射停顿原因表明流水线争用”。

这些规则会针对每一种架构（包括 Blackwell）进行更新。它们能够快速确认瓶颈是否出在内存吞吐量、指令分发、延迟隐藏等方面，从而引导你关注最相关的指标，缩短定位瓶颈所需的时间。

> 注　Nsight Systems 与 Nsight Compute 会持续更新以支持最新的 GPU 架构，帮助性能工程师在把工作负载迁移到更新一代 GPU 时诊断并修复性能问题。

8.2 用 Nsight Compute 分析 warp 停顿原因

如第 6 章所述，NVIDIA GPU 以 SIMT 方式执行 warp（线程束），即按 32 个线程一组来运行。如果 warp 频繁暂停而无法发射指令，性能分析器会对这些原因进行归类。

Nsight Compute 中最具洞察力的视图之一是某个 kernel 的 warp 状态统计（Warp State Statistics）细分，有时也称为 warp 停顿原因（warp stall reasons）。通过检查这些停顿原因，通常可以准确定位限制性能的因素。

warp 停顿有几种不同类型：内存相关型、执行依赖型、执行单元争用型，以及其他一些类型（如纹理缓存相关停顿）。下面分别来看。

8.2.1 内存相关停顿

如果 kernel 在等待全局内存加载，Nsight Compute 会报告较高比例的“Stall: Long Scoreboard”（记分板长时间等待）停顿周期。记分板（scoreboard）记录着每个 warp 未完成的内存请求，因此 Long Scoreboard 表明 warp 频繁地因全局 DRAM 加载的高延迟而等待，如图 8-4 所示。

类似地，Short Scoreboard（记分板短等待）停顿则是由于在共享内存与寄存器之间等待数据传输造成的，如图 8-5 所示。

类似地，标为“Stall: Memory Throttle”的指标意味着加载/存储流水线已经饱和，由于硬件内存队列已满，无法再发出新的内存请求。“Stall: Not Selected”表示 warp 虽然具备发射资格，但必须等待空闲的内存事务槽位才能继续。只要这些内存相关的停顿原因在你的停顿分布中占据主导，就清楚地表明 kernel 是内存受限的。

> 注　利用 Nsight Compute 的源代码视图，可以高亮显示受记分板依赖及其他硬件限制（例如特殊功能单元吞吐量等）影响的代码。

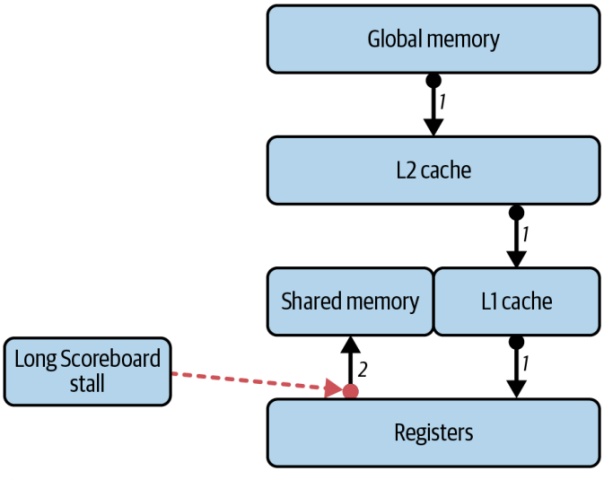

图 8-4　因等待高延迟全局内存访问而导致的 Long Scoreboard 停顿（例如，等待从设备内存取回的数据写入寄存器，或等待溢出的本地内存写入/读出设备内存）

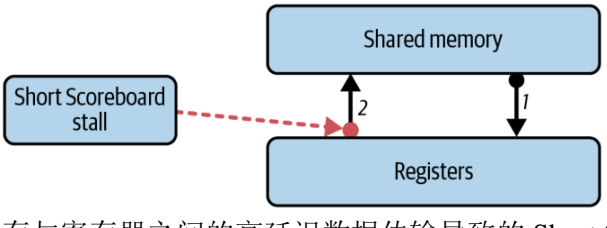

图 8-5　共享内存与寄存器之间的高延迟数据传输导致的 Short Scoreboard 停顿

8.2.2 执行依赖停顿

“Stall: Exec Dependency”占比高，意味着 warp 常常在等待先前指令的结果，比如某条指令依赖一条尚未执行完毕的先前指令的输出。这通常是每个线程内指令级并行不足的迹象——也可能是 ALU 延迟瓶颈。

在一串存在依赖关系的简单算术运算中，靠后的指令必须等待靠前的指令执行完毕，这会使 warp 闲置。若 Exec Dependency 停顿占主导，kernel 很可能正处于等待计算的延迟受限状态。换句话说，由于顺序依赖，每条线程的指令流水线不够繁忙。

8.2.3 执行单元争用

如果你看到明显的“Stall: Compute Unit Busy”，或者 FP32 CUDA 核心或 Tensor Core 的活跃利用率非常高，那么 kernel 很可能是计算受限的。此时，数学单元（FP32/FP64 ALU、Tensor Core 等）已经饱和。具体而言，warp 已准备好执行更多指令，但执行单元无法以更快的速度予以处理。这常常表现为接近峰值的“ALU pipe busy”指标，并可能与高功耗相关。在某些 GPU 上还有一个相关的停顿原因“Stall: Math Pipe Throttle”，它表示由于数学流水线在每一个周期都被完全占用，warp 只得等待。

8.2.4 其他停顿原因

尽管上述停顿原因是最常见的元凶，但还存在许多不太常见的停顿类别，包括取指停顿、纹理单元停顿、同步停顿等。例如，“Stall: Memory Dependency”与 Long Scoreboard 类似，表示某个操作正在等待先前的内存操作。

“Stall: Texture Throttle”表明纹理缓存子系统存在瓶颈。“No Eligible”或“Idle”表示在给定的周期内，warp 调度器没有任何可供发射的就绪 warp。这往往源于先前的同步操作——或者仅仅是因为启动的 warp 数量不够。

实践中，你无需记住每一种停顿原因，只需关注停顿分布中占比最大的部分：如果内存相关停顿占主导，就是内存受限；如果执行依赖占主导，就是 ALU 延迟受限。

如果几乎没有任何停顿，且流水线活跃度接近 100%，则很可能是计算受限。如果 warp 经常“未被选中”或处于空闲状态，则可能意味着并行工作量不足，或存在占用率问题。通过查看自己 kernel 的 warp 停顿分布，你可以把瓶颈缩小到某个类别。

> **注** Nsight Compute 会针对每次 kernel 启动提供这些停顿指标。务必对具有代表性的工作负载进行性能分析：一个很小的测试 kernel 的停顿分布可能与真实应用并不相同。在得出结论之前，务必从真实运行中收集数据。

表 8-1 列出了常见的 warp 停顿原因、其典型含义以及可能的优化方法。

warp 停顿原因

表 8-1　常见的 warp 停顿原因及优化方法。

warp 停顿原因	含义/成因	潜在优化方法
执行依赖	warp 因一条存在依赖且尚未完成的先前指令而停顿。这往往表明线程内指令级并行（ILP）不足。	提升 ILP（在同一线程内执行相互独立的工作）。展开循环或重排指令，使长延迟操作（如乘法或复杂数学运算）有其他工作可以重叠。如果 ILP 已打满而仍在停顿，则依靠更多的 warp（占用率）来隐藏延迟。
内存依赖（Long Scoreboard 停顿）	warp 正在等待内存加载（“长延迟”操作）完成之后才能推进。在数据到达之前，warp 内没有其他工作可以推进。	通过让更多 warp 随时可运行，或提高占用率来隐藏内存延迟，让其他 warp 在该 warp 等待期间继续运行。也可以使用异步内存预取（prefetch），把数据复制到共享内存后继续计算，使内存操作与计算相互重叠。为把内存延迟降到最低，应确保内存访问足够高效：采用合适的合并（coalescing）方式，并充分利用缓存层次。在较新的 GPU 上，可以利用 Tensor Memory Accelerator（TMA）进行批量多维内存拷贝，使内存搬运与计算以更低的线程开销实现重叠。
同步（barrier）	warp 正在 `__syncwarp()` 或其他同步原语处等待，直到所有其他线程都到达该屏障；期间线程处于空转状态。这往往意味着同步过于频繁，或各 warp 到达屏障的时间不均衡（某些 warp 先到达）。	减少不必要的同步。重新审视算法，设法合并各个阶段，或使用更细粒度的同步。确保工作量分配均匀，使各 warp 大致在同一时间到达屏障。某些情况下，可以使用较新的同步原语（例如 warp 级同步或 cluster 同步）来限定同步的作用范围。
取指/发射	warp 因等待取回下一条指令而停顿（可能是指令缓存未命中或流水线发射问题），或因所需的执行流水线繁忙而未能发射。kernel 特别大或某些流水线存在争用时会出现这种情况。	如果问题出在指令缓存未命中，可以考虑减小 kernel 规模（拆分 kernel），或避免过多的循环展开造成代码膨胀；如果是流水线发射问题（某一类指令使功能单元饱和），可以尝试混用不同类型的指令，或再次利用 ILP 去使用不同的流水线。
未被选中（调度器）	warp 已就绪，但在该周期未被选中发射（调度器选择了另一个 warp）。这通常意味着当时还有其他 warp 可用，而这个 warp 只是单纯地等待轮到自己，属于非依赖性停顿。它通常说明该周期选择发射的是其他就绪 warp，在占用率健康的情况下属于预期现象。	通常不是问题——它只是意味着 GPU 在该周期有其他工作要做，因而选择了另一个 warp。如果“Not Selected”（未被选中）占比很高，说明占用率正在（隐藏延迟方面）发挥作用，无需采取行动，除非它反映出不均衡（某个 warp 独占调度器；在极少数情况下，可以使用调度器提示或 yield）。

含义/成因　潜在优化方法

执行依赖

提升 ILP（在同一线程内执行相互独立的工作）。展开循环或重排指令，使长延迟操作（如乘法或复杂数学运算）有其他工作可以重叠。如果 ILP 已打满而仍在停顿，则依靠更多的 warp（占用率）来隐藏延迟。内存依赖（Long Scoreboard 停顿）

warp 因一条存在依赖且尚未完成的先前指令而停顿。这往往表明该线程内指令级并行（ILP）不足。

让更多 warp 处于就绪状态（即提高占用率），使其他 warp 能在该 warp 等待期间运行，从而隐藏内存延迟；或者使用异步内存预取（prefetch）把数据复制到共享内存后继续计算，让内存操作与计算重叠。为了降低延迟，还应确保内存访问高效：采用合适的合并（coalescing），并充分利用缓存层级。在现代 GPU 上，应使用张量内存加速器（TMA）进行批量多维数据拷贝，以极低的线程开销让数据搬运与计算重叠。同步（barrier）

该 warp 正在等待内存加载（“长延迟”操作）完成之后才能继续。在数据到达之前，warp 内无法进行任何其他工作。

减少不必要的同步。重新审视算法，寻找合并各执行阶段或改用细粒度同步的机会。确保工作均匀分布，使各 warp 大致同时到达屏障。某些情况下，可以使用较新的同步原语（如 warp 级同步或集群级同步 cluster sync）来缩小同步的作用范围。指令取指/发射

warp 正停在 `__syncthreads()` 或其他同步点上，在所有线程到达屏障之前保持空闲。这通常意味着同步过于频繁，或者负载不均衡（某些 warp 提前到达并等待）。

warp 因等待取下一条指令而停顿（可能是指令缓存缺失或流水线发射问题），或因所需的执行流水线繁忙而未能发射。这种情况可能出现在非常大的 kernel 中，也可能发生在某些流水线争用场景下。

如果指令缓存缺失是问题所在，可考虑减小 kernel 体积（拆分 kernel，或避免过度展开循环导致代码体积暴涨）。如果是流水线发射问题（某类指令饱和了

某个功能单元），可尝试混合使用不同类型的指令，或再次利用 ILP 来充分利用不同的流水线。

warp 停顿原因

未被选中（调度器）

该 warp 已就绪，但在该周期内未被选中发射（调度器选择了其他 warp）。这通常意味着当时还有其他可用 warp，而这个 warp 只是按顺序等待，并非依赖性停顿。它一般表明该周期调度器选择了其他就绪 warp 发射，在健康的占用率下这是正常现象。

通常这不是问题——它表明 GPU 当时还有其他工作要做，该周期选择了另一个 warp。如果“未被选中”占很高比例，说明高占用率正在发挥作用（隐藏延迟）。除非它表明存在失衡（某个 warp 长期霸占调度器；极少数情况下可以考虑使用调度器提示或让位指令 yield），否则无需采取任何措施。

解读这些停顿原因之后，你就能决定应采用哪种优化。例如，内存停顿时间偏高意味着应尝试用更多 warp、更好的内存访问模式或异步预取来隐藏延迟。

执行依赖性停顿很严重，则提示应提高 ILP 或重排代码。屏障停顿很频繁，则很可能需要重新设计同步方式。这一性能分析步骤会引导你找到最有效的优化手段，而不是盲目地四面出击。

8.3 查看实际占用率与 GPU 利用率

性能分析器报告的另一个关键指标是实际占用率（achieved occupancy），即执行期间每个 SM 上硬件线程槽位（warp）被占用的平均比例。例如，若 GPU 每个流式多处理器（SM）支持 64 个 warp，而实际占用率为 30%，则平均每个 SM 上有 19 个 warp 处于活跃状态。

实际占用率偏低往往意味着利用率不足，因为仅凭少量活跃 warp 无法充分隐藏延迟。另一方面，如果占用率已接近最大值，却仍看不到预期的性能收益，那么瓶颈很可能在其他方面，例如内存带宽受限、指令流效率低下或内存访问模式欠佳。

在这些非理想情形下，单纯增加线程数量无济于事。正确的做法是排查并改进内存合并、提高算术强度、优化 warp 级效率。后续小节将介绍这些技术。

Nsight Compute 还会报告占用率限制因素（occupancy limiter），指出是哪种资源在限制理论占用率。例如，“Limited by max registers per thread”（受限于每线程最大寄存器数）表示 kernel 的寄存器用量正在阻碍每个 SM 调度更多 warp。

而“Limited by shared memory per block”（受限于每 block 共享内存）则表示 kernel 为每个 block 分配的共享内存量是占用率的瓶颈。至于“Limited by thread count”（受限于线程数），

则意味着启动配置本身——网格（grid）或 block 规模——没有请求足够的线程来填满 GPU。

每一种限制因素都对应不同的解决之道，我们将在下一节中详细讨论。例如，如果限制因素是寄存器，可以减少寄存器用量，或使用 `__launch_bounds__` 来容纳更多 block；如果限制因素是共享内存，可以尝试采用更小的分块（tile），减少每个 block 的共享内存用量。

超过某个程度之后，继续提高占用率未必带来加速。极低的占用率（如 10%–20%）会因延迟隐藏不足而损害性能；反过来，如果内存带宽、指令依赖等其他因素成为瓶颈，把占用率推到 100% 也并不总是有益。

因此，除了占用率之外，还应考察硬件利用率。Nsight Compute 会报告实际内存带宽（GB/s）、实际计算吞吐（TFLOPS）、每周期指令数（IPC）、发射槽利用率以及其他资源利用统计。这些数字可以显示你的 kernel 距离 GPU 物理硬件极限还有多远。

例如，如果 kernel 的 ALU 利用率很低，而内存吞吐量达到峰值的 95%，那么几乎可以肯定它受限于内存带宽。反之，如果 ALU 利用率接近最大值而内存吞吐量仍然平平，则 kernel 是计算受限的。在这种情况下，只有提高算术吞吐量才能获得加速——通常做法是改用更低精度的数据类型（FP16、FP8 或 FP4），并把计算迁移到 GPU 上更快的 Tensor Core。

如果 ALU 利用率和内存吞吐量都偏低，那么 kernel 可能正受长延迟操作或同步开销的拖累，或者其并行工作量本身就不充足。这可能表明指令级并行度偏低（将在后续小节讨论），也可能是启动的线程数不足以充分利用 GPU。

你可以借助 Roofline 模型来组织上述分析：该模型将 kernel 的 FLOPS 与其算术强度（每访问 1 字节内存所对应的 FLOPS）绘制在同一张图上。Roofline 定义了两条上限：计算顶（GPU 可持续维持的最大 FLOPS）和内存顶（最大内存带宽）。

如果 kernel 的 FLOP/字节比值低于硬件的计算与内存配比，说明数据供给速度不够快，kernel 属于内存受限；如果该比值很高而实际 FLOPS 仍远低于峰值，则 kernel 可能是延迟受限，或缺少足够的 ILP 来喂饱计算单元。

> **注** 每一代新 GPU 的内存带宽提升幅度有限，而计算能力的增长要快得多。因此，随着时间推移，kernel 往往会越来越偏向内存受限。

在实践中，始终要对比两组关键数字：kernel 的内存吞吐量与硬件的峰值内存带宽，以及 kernel 的计算吞吐量与硬件的峰值 FLOPS。这些对比会告诉你下一轮优化应聚焦于内存访问、计算工作量还是并行度。下面我们逐一讨论。

8.3.1 kernel 内存吞吐量与峰值 HBM 内存带宽的对比

Nsight Compute 会报告 kernel 达到的 GB/s 数值。如果 kernel 显示内存带宽利用率已接近峰值，那么增加计算量也无济于事：必须通过减少每单位操作所需的内存流量来提高算术强度。

可以通过以下手段提高算术强度：配合 Tensor Core 使用降低精度的计算、用并发隐藏更多延迟，以及使用 kernel 融合（稍后介绍）。这些技术通过缩小所传输中间数据的规模来减少全局内存流量。

举例来说，如果 kernel 达到了 GPU 内存带宽的约 80% 或更高，它很可能已受限于内存，因为剩余的余量已经不多。

不过要注意，Blackwell GPU 拥有相对较大的 126 MB L2 缓存，而且其双芯（dual-die）设计采用 10 TB/s 高带宽互连——NVIDIA 高带宽接口（NV-HBI），因此在内存访问上两颗 GPU 芯片表现得如同一体。这样一来，在旧款 GPU 上原本内存受限的许多 kernel，在新一代 GPU 上能更好地利用缓存。

你可以利用 Nsight Compute 的内存图表查看流向 L2 与 DRAM 的流量各占多少。较高的 L2 命中率可以缓解全局内存瓶颈——这意味着即使 kernel 执行了

大量内存访问，它实际上可能仍然是计算受限的，因为片上缓存承担了大部分内存访问。

> **注** 在 Blackwell 上，你可以控制 L2 数据的驻留（persistence），让关键工作集常驻缓存。

8.3.2 kernel 计算吞吐量与 GPU 峰值 FLOPS 的对比

实际达到的 FLOPS 偏低，可能源于占用率过低，也可能是指令级停顿。例如，如果平均只有 30% 的 warp 处于活跃状态，或者流水线经常因等待内存而空闲，那么 kernel 的性能就会远低于计算屋顶线。

你可以使用 Nsight Compute 的 Occupancy 区块和 Source Counters 来精确定位这些问题。确保 kernel 启动了足够多的线程来填满 GPU，直至达到所用设备每个 SM 的常驻 warp 上限（例如，每个 SM 常驻 64 个 warp）。

> **注** 只有同一个 SM 内部的 warp 才需要共享资源，也只有它们能相互隐藏延迟。不同 SM 上的 warp 之间没有任何交互。因此，实际占用率是按 SM 来度量的。我们稍后会讨论占用率。

此外，还要考察指令发射效率与吞吐量指标。Blackwell 单颗设备可扩展到大量 SM，因此 kernel 层面的利用不足会累积成很大的总体损失。（注意：双芯（dual-die）封装本身并不会提高每个 SM 的发射速率。）

如果实际 FLOPS 中等偏高，而内存吞吐量偏低，那么 kernel 可能以计算为主，却受限于指令依赖。你可以在 Nsight Compute 中查看“Exec Dependency”停顿指标以及其他各类停顿原因来加以确认。

如果计算吞吐量已接近峰值，那么你的 kernel 才是真正的计算受限。这种情况下，kernel 的内存访问模式应当已经优化过，或者你已经在使用低精度 Tensor Core 来达到如此高的 FLOPS。

长时间维持接近峰值的计算利用率，有时会触发现代 GPU 的功耗管理计算限流器，以保护硬件。在做性能分析、基准测试和调优时，务必把这一点纳入考虑。判断自己是否受功耗限制，最简单的方法是用下面的命令实时监测强制功耗上限，并同时关注任何“HW Slowdown”标志：

```
nvidia-smi \
  --query-gpu=\
  power.draw,clocks.current.sm,clocks.current.memory,\
  clocks_event_reasons.active \
  --format=csv -l 1
```

该命令每 1 秒打印一行，内容为当前功耗、图形/内存时钟频率以及降频原因。借助这些信息，你可以准确定位 GPU 何时触及功耗上限并开始降频。

你也可以使用 NVIDIA 管理库（NVML）API，它以编程方式提供 CUDA C++ 的 `nvmlDeviceGetPowerUsage()` 与 `nvmlDeviceGetEnforcedPowerLimit()` 接口，以及等价的 NVML Python 接口。它们非常适合用于自定义脚本或与监控系统对接。

简而言之，应当把占用率、warp 停顿原因、内存吞吐量和计算吞吐量等指标结合起来诊断瓶颈。要确保你

看到的是高占用率、高 warp 效率，以及包括 Tensor Core 在内各执行单元的均衡使用。

8.3.3 迭代性能分析与确定 kernel 瓶颈

GPU 停滞不前的根本原因有四类：利用不足（underutilization）、延迟受限（latency bound）、内存受限（memory bound）和计算受限（compute bound）。这些状态彼此关联，但要选出正确的优化方向，必须先逐一独立理解。瓶颈往往在解决一个之后又会暴露出下一个。

利用不足指的是：你根本没有启动足够的线程或工作。此时 FLOPS 和内存带宽都很低，执行时间线上存在空闲空隙。一旦提高并行度，你就会发现 warp 开始因等待内存加载而停顿。

当 GPU 得到较充分的利用之后，就可以进一步区分延迟受限与内存受限。延迟受限的 kernel 每秒发出的字节数远低于硬件能力，因为单个内存加载操作就让 warp 陷入停顿。解决办法是通过提高占用率、使用更多 ILP、预取和流水线化来增强内存与计算的重叠。

相反，内存受限的 kernel 已经把 DRAM 带宽跑满，但 ALU 却处于空闲——这不是因为停顿，而是由于内存通道饱和，每秒已经没有更多数据可取。此时必须通过分块（tiling）、融合（fusing）、利用缓存（L1/纹理）或降低精度来提高算术强度，从而减少内存流量。

如果上述方法都无法带来更高的速度，你就进入了计算受限区间：GPU 的算术流水线（例如 ALU 和 Tensor Core）成了限制因素。此时可以通过循环展开和软件流水线，把相互独立的指令重叠起来，提高每线程的 ILP。在现代 GPU 上，统一型核心无法在同一时钟周期内同时执行一条 INT32 指令和一条 FP32 指令；换言之，混合 INT32 与 FP32 的工作负载不会在同一个 cycle 内从同一核心并行执行这两种指令。因此，可达到的发射速率取决于你的指令构成。

在优化过程中，你往往会依次经历这些状态：利用不足 → 延迟受限 → 内存受限 → 计算受限。每解决一个瓶颈，你多半会撞上下一个瓶颈，并采取相应的策略。优化 GPU 代码时，应当遵循这里描述的结构化方法。

第一步是做性能分析，随后识别瓶颈。kernel 层面的指标（例如 warp 停顿、实际占用率、内存与计算利用率）可用 Nsight Compute 观察；应用层面的时间线（例如并发情况与空闲空隙）则用 Nsight Systems 观察。

确定瓶颈之后，即可判断 kernel 属于内存受限、计算受限、延迟受限，还是单纯未充分利用 GPU。GPU 没有发出足够的工作时，就是利用不足。表 8-2 汇总了这四种状态，包括常见的性能分析指标与应对手段。

% 表 8-2 表头与第 1 行“内存受限”的“限制因素/描述”列，内容已并入上方表格浮动体。

% 表 8-2 第 1 行“内存受限”的“性能分析指标”列，内容已并入上方表格浮动体。

% 表 8-2 第 1 行“内存受限”的“应对手段”列，内容已并入上方表格浮动体。

% 表 8-2 第 2 行“计算受限”的“限制因素/描述”列，内容已并入上方表格浮动体。

% 表 8-2 第 2 行“计算受限”的“应对手段”列与第 3 行“延迟受限”的“限制因素/描述”列，内容已并入上方表格浮动体。

表 8-2　内存受限、延迟受限、计算受限与 GPU 利用不足的对比。

限制因素	描述	性能分析指标	应对手段
内存受限	你已经在以尽可能快的速度搬运数据——接近 DRAM 带宽峰值——但每字节数据上的计算量不足，无法让 ALU 满载。	内存带宽利用率接近峰值，FLOPS 偏低。	提高算术强度（如分块、融合），并改进合并访问与缓存利用。
计算受限	你已经隐藏了内存延迟，内存带宽不再饱和；此时 ALU（如 CUDA core 与 Tensor Core）成为瓶颈。	FLOPS 接近 GPU 峰值，内存利用率偏低。	挖掘更多 ILP（如双发射、循环展开），使用专用单元（如 FP16/FP8/FP4/Tensor Core），减少依赖、融合工作，或利用低精度与稀疏性。
延迟受限	并发执行的工作不足以隐藏单个加载/存储操作的延迟，warp 因等待数据而停顿。	实际带宽远低于峰值，“stall-on-scoreboard”或“not selected”占比很高。	提高占用率，增加 ILP（如循环展开、多个累加器），采用 kernel 内流水线化与软件预取。
利用不足	SM 未被充分占用，或启动的工作量不足——内存与计算资源均处于闲置。	占用率低、实际带宽低、FLOPS 低，时间线上出现空闲空隙或稀疏的 kernel 活动。	增大问题规模或批处理规模，启动更多线程/块，融合任务，或使用常驻 kernel（persistent kernel，见第 10 章）或流（stream，见第 11 章）。

% 表 8-2 第 2 行“计算受限”的“性能分析指标”列，内容已并入上方表格浮动体。

% 表 8-2 第 3 行“延迟受限”的“性能分析指标”列，内容已并入上方表格浮动体。

% 表 8-2 第 3 行“延迟受限”的“应对手段”列，内容已并入上方表格浮动体。

% 表 8-2 第 4 行的限制因素标题“利用不足”，内容已并入上方表格浮动体。

% 表 8-2 第 4 行“利用不足”的描述列，内容已并入上方表格浮动体。

占用率低、实际带宽与 FLOPS 偏低，时间线上出现空隙或 kernel 活动稀疏。

增大问题规模或批量处理工作，启动更多线程/线程块，融合任务，使用常驻 kernel（persistent kernel，见第 10 章）或流（stream，见第 11 章）。

内存受限（memory-bound）指性能受内存吞吐量限制的情况——具体来说，就是 GPU 的全局内存无法以足够快的速度把数据送入计算单元。此时，kernel 的实际 FLOPS 会贴近由内存带宽决定的 Roofline 线。当线程数量充足但数据传输速度无法再提升时，就会出现这种情况。也就是说，你碰上了内存带宽的天花板。

相反，计算受限（compute-bound）的 kernel 会让 GPU 的算术单元（ALU FP32 CUDA 核心或低精度 Tensor Core）达到饱和。当 kernel 接近这些核心的峰值 FLOPS Roofline 线时，这一特征会比较明显。实际占用率、内存利用率、执行依赖停顿等性能分析指标可以帮助确认这一分类。

另一方面，当 kernel 处于延迟受限（latency-bound）状态时，每个线程会把很大一部分时间花在等待单次内存载入上，而不是做有效工作。实际地说，这意味着当 warp 发出一次全局内存载入以取回 `A[idx]` 时，该 warp 中的全部 32 个线程都会停顿，直到这次取数完成。如果

代码紧接着又发出另一条依赖该结果的载入或计算，那么每次载入时 warp 都会空转数百个周期。

当 kernel 延迟受限时，GPU 的 warp 调度器或许可以切换到其他 warp，但如果每个 warp 的结构都一样（例如“一次载入 → 等待 → 计算 → 写入”），就几乎没有什么别的工作可以填补这些空闲周期。这样一来，kernel 中始终没有足够多的独立操作同时在执行，无法掩盖长延迟 DRAM 访问的延迟。

要摆脱延迟受限的局面，就需要让 GPU 有多个操作可以重叠执行。一种方法是启动更多线程和 warp，以提高占用率。这样，当一个 warp 因载入而停顿时，另一个 warp 正准备运行。

同样重要的是提高每个 warp 的 ILP。本章稍后会更详细地介绍 ILP。概括地说，如果每个线程在执行任何算术运算之前，先背靠背地发出两次或更多独立载入（即同时载入 `A[idx]` 与 `B[idx]`），GPU 就能够在硬件上开始重叠这些载入操作。

此时，当第一次载入还在等待 DRAM 时，第二次（以及第三次、第四次等）载入已经在执行途中。只要其中任何一个载入返回，依赖它的算术运算就可以执行，并与其余尚未完成的载入进一步重叠。

> **注** 展开一个小循环也可以提高 ILP，使多个数值在任何计算开始之前就已取回。后文会详述。

通过提高 ILP，每个线程和 warp 总有足够的工作同时在执行，DRAM 延迟便被其他未完成的操作所掩盖。最终效果是把一个延迟受限的 kernel 变成能让 SM 流水线保持繁忙的 kernel，从而提升吞吐量和 kernel 的整体性能。

当 GPU 利用率不足时，说明启动的工作量不足以让 SM 和内存流水线保持繁忙，许多资源处于闲置状态。此时，可以通过增大批大小（batch size）或启动更多线程来增加工作量，从而改善状况。

8.3.4 优化 kernel

在实践中，性能调优应当遵循一条清晰、分步的流程。首先，判断你的 kernel 处于哪一种状态：内存受限、延迟受限、计算受限，还是利用率不足；然后，应用相应的优化。

如果 kernel 是内存受限的，重点应放在减少和掩盖内存流量上。做法包括改进访存合并（coalescing）、提高占用率、增加 ILP，以及通过缓存或分块（tiling）引入数据复用。

当 kernel 是延迟受限的（即单次载入或指令延迟占主导地位）时，要确保有足够多的独立工作同时在执行。做法是：让每个线程发出多条互不依赖的载入/操作，或者提高整体占用率，使调度器始终有就绪的 warp 可以运行。

如果 kernel 是计算受限的（即 ALU 或 Tensor Core 已饱和而内存空闲），改用更低精度的算术（FP16/FP8/FP4）、把工作卸载到 Tensor Core 上，或把更多操作融合在一起，都可以提高算术吞吐量。最后，如果 GPU 表现为利用率不足（占用率低且频繁出现空闲周期），通常只需启动更多线程或线程块（让所有 SM 都有工作可做），就足以让硬件先忙起来，然后再进行更深入的优化。

下面列出了一些高层次优化技术，帮助你把 GPU 性能调优当作一个科学过程来对待。接下来几章会深入讨论这些技术，但每一项都应当包含三个环节：找出限制因素、应用相应优化、度量结果：

将内存受限的工作负载转化为计算受限

如果工作负载是内存受限的（算术强度低），就增加数据复用和每次启动的工作量，做法如下：应用分块（tiling），利用高速共享内存并减少冗余访问；融合 kernel，避免不必要的内存往返；确保访存得到优化（合并访问，并按第 6 章所述避免 bank 冲突）；并考虑使用压缩或更低精度，以减少需要搬运的数据量。

进一步优化计算受限的工作负载

如果工作负载是计算受限的（例如 ALU 利用率很高，但因依赖关系而达不到峰值），就提高有效指令吞吐量，做法如下：使用 ILP 技术（例如展开循环、使用多个累加器以重叠独立操作）；检查分支发散（branch divergence，见第 6 章），并尝试重新组织工作以减少发散，因为发散会浪费 ALU 周期；迁移到 Tensor Core 并降低精度，以提高计算上限。如果 kernel 已到达计算 Roofline 顶点，可以看看降低精度（FP32 → FP16 → FP8 → FP4）能否带来进一步加速。

为延迟受限的工作负载提高并行度

如果工作负载是延迟受限的（例如 warp 频繁停顿、并行度不足以掩盖延迟），就在各个层次上提高并发性，做法如下：尽可能启动更多线程/线程块，直到延迟被掩盖；确保寄存器/共享内存没有过度限制占用率（例如进行占用率调优、平衡资源使用）；在每个线程/warp 内利用异步拷贝（async copy）重叠访存与计算（例如 kernel 内部流水线化）；并使用多条流（stream）让独立任务重叠执行，或让拷贝与计算重叠（例如第 11 章所述的 kernel 间并发）。如果启动开销是个问题（例如大量微型 kernel），可以考虑用协作组（cooperative groups）合并它们，或直接把它们的代码合并（即 kernel 融合）。

提高 GPU 利用率

如果 GPU 利用率不足（例如 SM Active 低、占用率低），要确保启动了足够多的工作来用满所有 SM。具体而言，要让 kernel 以足够大的网格（grid）规模启动，以充分利用 GPU。一个常见错误是启动的线程数正好等于元素个数，却忘了每个线程只做一点点工作。有时需要多遍处理，或者每个元素使用更多线程。

减少同步与主机端（CPU）停顿

应当移除一切不必要的 `cudaDeviceSynchronize()` 或会拖住 GPU 的主机端等待。如果工作负载本身很小，可以考虑将其与其他工作合并成批，或让多个实例并发运行（例如使用 CUDA 流）。也可以使用 CUDA Graph（见第 12 章）预先定义并高效启动一个由许多小 kernel 组成的执行图。

利用专用硬件与低精度/混合精度

Blackwell 的 Tensor Core 在 PTX 中以 `tcgen05.mma` 暴露第五代 MMA 指令，以及配套的载入/存储指令（例如 `tcgen05.ld` 与 `tcgen05.st`）。Tensor Core 可加速多种微缩放（microscaling）格式，包括 MXFP8、MXFP4（OCP MX 格式）以及 NVIDIA 的 NVFP4 格式。它们还支持按块缩放的矩阵乘（block-scaled matmul，K 分组），当缩放元数据存在时，各类库会自动选用该路径。TMEM 与 TMA 支撑着这些高吞吐数据通路。本章稍后与第 9 章会进一步讨论这些技术。

验证并迭代

每次优化之后都要重新进行性能分析，确认针对性的停顿或指标确有改善（例如分块后内存停顿减少、调整线程块大小后实际占用率上升、使用 CUDA 流后“SM Throughput %”上升等）。同时要观察总运行时间的改善。有时一个瓶颈会掩盖另一个瓶颈：你可能解决了内存带宽问题，接着却变成计算受限——这没什么问题，需要时再继续处理新的瓶颈。

保持正确性与可接受的精度

在使用更低精度或新的并行策略时，应当用断言或与参考结果的比对来进行测试。除非应用本身可以接受，否则要确保加速不是以牺牲精度为代价的。通常，像 FP16 甚至 FP8 这类技术都会经过仔细验证，以确保对 AI 模型精度的影响可以忽略不计。

8.4 占用率调优

回顾第 6 章，占用率（occupancy）是 SM 上活跃 warp 数与 SM 上可活跃 warp 最大数之比。低占用率（< 50%）意味着平均只有一半可能活跃的 warp 处于活跃状态。这可能表明你的 kernel 受到寄存器或每线程块共享内存等资源的限制——而不仅仅是受可用并行度的限制。

占用率衡量的是 SM 上活跃的线程或 warp 数相对于硬件最大容量的水平。更高的占用率，即在飞的 warp 更多，可以让 GPU 更好地隐藏延迟。这是因为，当一个 warp 因等待内存加载而停顿时，调度器可以迅速切换去运行另一个 warp。

每个 SM 上的 warp 越多，通常意味着 GPU 的流水线越繁忙，因等待内存而浪费的周期越少。占用率调优是指调整 kernel 启动参数和资源用量（如寄存器和共享内存），以使每个 SM 上的有效并行度最大化，如图 8-6 所示。

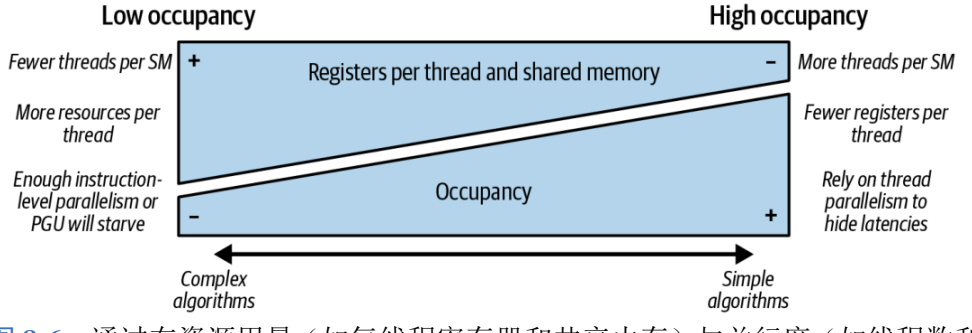

图 8-6　通过在资源用量（如每线程寄存器和共享内存）与并行度（如线程数和 warp 数）之间取得平衡来调优占用率

请记住，占用率调优的目标是保持足够多的活跃 warp，以充分利用 SM 的流水线并隐藏长延迟操作。理想情况下，你可以达到 100% 占用率，填满所有可用的 warp 槽位。例如，Blackwell B200（计算能力 10.0）的每个 SM 提供 64 个 warp 槽位，即 2,048 个线程。

> 注　在 Ampere、Hopper、Blackwell 等现代数据中心 GPU 架构上，“每 SM 64 个 warp”这一上限一直保持不变——即使 GPU 的总核心数在不断增加。硬件性能的提升来自更多的 SM、更大的缓存、多裸片（multidie）等。你可以使用 Nsight Compute 的 Occupancy 分析来确认目标设备上的确切上限。

有意思的是，NVIDIA RTX PRO 6000 和 Spark DGX（GB10 超级芯片）支持更高的计算能力（12.x），但每个 SM 只允许 48 个 warp（1,536 个线程）。

许多实际场景中的 kernel 在较低占用率下依然表现良好——尤其是当它们的内存或算术延迟本来就很小的时候；或者当它们借助 Tensor Core 这类高吞吐单元让流水线保持忙碌，从而不需要那么多活跃 warp 的时候。

例如，计算受限的 kernel 可能只需 50% 的占用率就能达到峰值性能，因为每个 warp 都在持续做功而无须等待。相比之下，内存受限的 kernel 通常受益于高占用率，因为当其他 warp 因等待内存传输而停顿时，仍有一些 warp 可以在 SM 上运行。

8.4.1 为你的工作负载找到合适的占用率

在实践中，有效的占用率调优往往在超过某个点之后开始收益递减。举例来说，如果一个 kernel 严重内存受限，把实际占用率从 10% 提升到 50% 可能带来巨大提升，因为此时你已经有足够的 warp 来覆盖延迟。

但从 50% 提升到 100% 可能只有很小的额外增益，因为其他因素开始占主导，例如缓存未命中、内存带宽饱和等。性能分析有助于确定最优占用率。例如，你可以评估 `eligible warps per cycle`（每周期可发射 warp 数）和 `active warps per scheduler`（每调度器活跃 warp 数）等指标。它们能揭示每周期有多少 warp 已就绪可发射，以及硬件实际能处理多少。

`eligible warps per cycle` 报告的是每个周期处于“可运行”状态、没有未决数据或依赖停顿的 warp 的平均数量。`active warps per scheduler`（通常等于 SM 上的调度器数量）是每周期最多能发射一条指令的 warp 数量上限。

如果 `eligible warps per cycle` 低于 `active warps per scheduler`，GPU 常常会出现就绪 warp 不足的情况：当某个 warp 因内存访问或长延迟指令而停顿时，没有另一个就绪的 warp 可以切换过去。这表明你需要以更高占用率或更高 ILP 的形式增加并发，以隐藏延迟。

相反，如果 `eligible warps per cycle` 已达到或超过调度器上限，而 kernel 仍然运行缓慢，这说明就绪的 warp 数量已经足够，但它们因其他停顿而无法发射，例如内存带宽饱和或执行

依赖。此时，更好的做法是通过更优的合并访问和异步拷贝来隐藏内存延迟，或者通过展开相互独立的工作来提升 ILP。这比单纯增加线程数更为有效。

如果你提高了占用率，并观察到“Stall: Not Selected”或空闲百分比下降、内存流水线更常处于繁忙状态，那么你就成功地改善了占用率。如果占用率已经很高，但 Long Scoreboard 仍占主导，你可能需要借助其他技术，例如改进内存访问模式或让计算重叠执行。

在实践中，一旦达到中等占用率（例如 60%–70%），收益就会开始递减。因此，追求更好的内存局部性、更高的 ILP，以及使用共享内存和寄存器等片上存储来缓存数据，往往更有效。

最大化占用率只是确保有许多 warp 可供运行；但如果这些 warp 正在等待内存，或正在执行发散分支代码，它们可能仍然处于空闲状态。因此，在达到合理占用率（例如 50%–70%）之后，应把注意力转向 warp 效率和延迟隐藏，而不是执迷于 100% 占用率。

> **注** 占用率是达成目的（隐藏延迟）的手段，而不是目的本身。一旦已有足够多的 warp 让 GPU 保持忙碌，其他优化会带来更好的回报。

8.4.2 占用率调优技术

当性能分析表明占用率是限制因素时，有几个直接可行的步骤可以提升占用率：通过启动更多线程来增加并行度、调整块大小、降低每线程资源占用，以及使用 `__launch_bounds__` 或占用率 API。下面逐一讨论：

增加并行度（启动更多线程）

提升占用率最简单的方法，是在当前 kernel 启动尚未用满所有 SM 的情况下启动更多工作。例如，如果你在每个 SM 上只启动了 20 个 warp，而 GPU 能够支持 64 个，那么增大网格规模或每块线程数即可提高占用率（前提是有足够的数据可供处理）。

务必要让所有 SM 都有活干：启动的块数至少要与 SM 数量相等——而且通常要远多于 SM 数，因为各个块会在每个 SM 上并行执行。如果你的 GPU 显示较低的“SM Active Cycles”，说明并非所有 SM 都有工作，此时应扩大工作负

载或批大小。然而，仅使用更多线程是不够的——尤其当每个线程都占用大量资源（如寄存器和共享内存）时更是如此。

调整块大小

有时，每块线程数（即线程块大小）会限制占用率。非常大的线程块（例如1,024 线程）可能占用过多寄存器或共享内存，以至于一个 SM 上一次只能驻留一个块，从而导致占用率偏低。相反，使用中等大小的块（例如 128–256 线程）可以让多个块同时驻留在同一个 SM 上，从而提高可能活跃的 warp 总数。

最优块大小因 kernel 而异。关键在于在块大小与资源占用之间取得平衡。较小的块各自占用的资源更少，因此允许更多块并行运行。

你可以利用内置的 Nsight Compute 占用率计算器（Occupancy Calculator）和 CUDA 占用率 API 试行不同的启动配置，帮助找到一个既使占用率最大化、又不引入其他开销的最佳点。

占用率计算器会针对最大理论占用率给出配置建议。但在实际中，最佳性能可能来自略低于该理论最大值的占用率。

> **注** 务必通过实际的计时实验来验证占用率计算器建议的最佳 `BlockSize`，因为有时占用率略低的配置由于寄存器溢出更少或内存合并访问更好，反而能产生更高的吞吐量。

降低每线程的寄存器和共享内存占用

每个线程都会占用寄存器——并且很可能还占用共享内存。如果 kernel 每线程使用的寄存器或共享内存过多，编译器就必须减少 SM 上可同时活跃的线程或 warp 数量，否则就会超出 SM 的寄存器堆总量或共享内存配额。

针对寄存器用量问题，你可以重构代码，使用更少的活跃变量，从而占用更少的寄存器，释放寄存器容量，让更多 warp 能够同时驻留。类似地，你也可以向编译器传入 `-maxrregcount=<N>`，为每个线程分配的寄存器数量设置上限。

当你限制了每线程的寄存器数量后，编译器将被迫让每个线程塞进更少的寄存器。这样，硬件就能在 SM 上调度更多的 warp，进而提高占用率。

当然，如果对寄存器的限制过于激进，编译器会把多余的变量溢出（spill）到本地内存，反而损害性能。因此，你应当找到最小的寄存器上限：既能最大化占用率，又不造成过度溢出。

同样地，如果每个线程块占用大量共享内存（例如一个大 tile），共享内存就会限制 SM 上能容纳多少个线程块。通过优化共享内存占用——只存储必需的数据，并尽可能利用片上缓存——你可以为更多线程块腾出容量。本质上，让每个线程/线程块使用更精简的资源，就能提高占用率。

但要注意，如果做得过头，减少寄存器或共享内存可能会降低单线程性能。这是一种权衡。性能分析器的 Occupancy limiter 读数会告诉你寄存器或共享内存是否是瓶颈，从而帮助你确定优化的方向。

例如，在 Blackwell 上，如果激进地展开循环，你可能会看到“Limited by max registers per thread”（受每线程最大寄存器数限制）的提示。此时，可以考虑减少展开的迭代次数或拆分工作。这是因为 Blackwell 的每线程寄存器上限是 255。Occupancy 报告有助于量化这些权衡。

`Use __launch_bounds__`

在某些场景下，你可能会有意限制占用率，或者引导编译器针对某个特定占用率进行优化。CUDA 允许在 kernel 代码中通过 `__launch_bounds__` 注解来设置启动边界（launch bounds），向编译器提示该 kernel 预期的使用模式和启动配置。

如果你通过性能分析发现，性能在 50% 左右的占用率达到峰值，而不是 100%，这往往意味着每个线程使用了大量寄存器或共享内存，这会进一步限制占用率。在这种情况下，你可以用 `__launch_bounds__` 来引导编译器。

例如，你可以用 `__launch_bounds__` 限制每线程块的线程数，并指定每 SM 最少的驻留线程块数，从而让更多 warp 驻留在 SM 上。本质上，你是在告诉编译器：用一部分每线程的寄存器/共享内存资源占用，换取更高的 warp 数量。

当你的 kernel 属于延迟受限（latency bound）时，这种做法能提高吞吐量，因为更多的 warp 会隐藏内存延迟和依赖停顿。与之相对，纯粹计算受限（compute-bound）的 kernel 不会从这类优化中获得同样的收益。

使用 CUDA Occupancy API

除了 `__launch_bounds__` 注解之外，CUDA 的 Occupancy API 函数（例如 `cudaOccupancyMaxPotentialBlockSize()` 和 `cudaOccupancyMaxActiveBlocksPerMultiprocessor()`）还可以让你在运行时确定：在 kernel 实际的寄存器与共享内存需求下，哪种线程块大小能达到最高占用率。

这些函数会给出每线程块线程数以及每 SM 线程块数的推荐值。这样，你就无需猜测哪种配置

最为高效。实际操作中，你可以先用 Occupancy API 找到一个候选的线程块大小，再通过 `__launch_bounds__` 提示加以微调，从而锁定某个特定的占用率水平。

> **注** 另一种做法是：确定最优配置后，用 CUDA Graphs 高效地启动预先定义的工作负载。CUDA Graphs 并不直接改变占用率，但能减少每次启动的开销；当你增加线程块数量时，这对占用率是有益的。我们将在第 12 章介绍 CUDA Graphs，并在第 13、14 章介绍 PyTorch 对 CUDA Graphs 的使用，但在此上下文中值得先提一下。简而言之，推荐采用多阶段的方法：先对系统做性能分析，找出理想的占用率区间，再借助编译器与运行时提示达到这一占用率。目标是高效地确定线程块大小，让足够多的 warp 保持“在飞”（in flight），同时不过度占用寄存器和共享内存这类稀缺的片上资源。接下来，让我们更深入地了解如何用 `__launch_bounds__`、Occupancy API 和 PyTorch 来调优占用率。

8.4.3 使用编译器提示来优化占用率

我们可以使用 CUDA 的 `__launch_bounds__` kernel 注解，引导编译器针对占用率进行优化。该注解可以为 kernel 指定两个参数：我们希望启动的每线程块最大线程数（例如 256），以及希望在每个 SM 上保持驻留的最小线程块数（例如 4），如下所示：

```
__global__ __launch_bounds__(256, 8)
void myKernel(...) { /* ... */ }
```

这里我们承诺：绝不会以每线程块超过 256 个线程的方式启动 `myKernel`；同时请求 GPU 尽量让每个 SM 至少有 8 个活跃的线程块。这些提示会影响编译器的寄存器分配与内联决策。

具体来说，`__launch_bounds__` 会限制每个线程的寄存器用量，使得一个线程块最多能容纳 256 个线程，且每个 SM 至少有 8 个线程块处于活跃状态——合计每个 SM 有 2,048 个活跃线程（8 个线程块 × 每线程块 256 线程 = 2,048 线程）。这刚好填满 B200 这类设备的线程槽——它们有每 SM 2,048 线程的上限（例如 Blackwell）。在实践中，`__launch_bounds__` 可能使编译器限制每线程的寄存器用量，并约束展开/内联，从而避免溢出、实现更高的占用率。因此，我们实际上是用一点单线程性能（例如不把寄存器用满、不展开到极限）去交换更平稳的 warp 吞吐量——让更多的 warp 保持“在飞”。

> **注** 提高占用率必须与每线程资源的增加相权衡。要避免因强行塞入过多线程而引发寄存器溢出：一旦寄存器耗尽，数据就会溢出到本地内存（由全局 HBM 支撑），从而造成缓慢的内存访问。找到最佳平衡点通常需要反复实验。Nsight Compute 的“Registers per Thread”和“Occupancy”指标可以为此提供指引。

8.4.4 使用 Occupancy API 确定最优启动配置

你可以在运行时借助 CUDA Occupancy API（包括 `cudaOccupancyMaxActiveBlocksPerMultiprocessor()` 和 `cudaOccupancyMaxPotentialBlockSize()`）确定最优的启动配置。例如，`cudaOccupancyMaxPotentialBlockSize()` 会结合 kernel 的寄存器与共享内存使用情况，计算出让占用率达到最优的块大小，如下所示：

```
int minGridSize = 0, bestBlockSize = 0;
cudaOccupancyMaxPotentialBlockSize(
    &minGridSize, &bestBlockSize,
    myKernel,
    /* dynamicSmemBytes = */ 0,
    /* blockSizeLimit = */ 0 );

// bestBlockSize contains number of threads per block that maximizes
 ↪  occupancy
myKernel<<<minGridSize, bestBlockSize>>>(...);
```

该 API 会根据 kernel 的资源用量，估算每块的线程数取多少才最有可能使占用率达到最大。随后我们就可以在 kernel 启动时使用给出的 `bestBlockSize` 和 `minGridSize`。

实践中，编译器的启发式策略通常已经相当不错。但当你需要显式控制时，`__launch_bounds__` 与 Occupancy API 能提供明确的调节手段。当你确知自己的 kernel 可以用一部分每线程资源换取更多活跃 warp 时，就应使用它们。这有助于避免“过重”的线程导致 SM 占用率不足。

8.4.5 用 PyTorch 调优占用率

对于编写高层代码的 PyTorch 用户来说，通常无需直接管理占用率——PyTorch 的 CUDA kernel 和各种库已在幕后处理了启动参数。矩阵乘法、卷积等运算已经过调优，能够高效地利用 GPU。

不过，理解占用率仍然很有价值。如果你以 PyTorch 扩展的形式编写自定义 CUDA kernel，同样的原则依然适用：启动足够多的线程/

块来充分利用 GPU，选择合理的块大小，并且在每线程过度使用寄存器或共享内存会限制占用率时避免这样做。

即便停留在 Python 层面，也有几件事值得留意。如果你使用 PyTorch 时发现 GPU 利用不足（例如在性能分析轨迹中只有一小部分 SM 处于活跃状态），原因可能是一些张量运算的规模太小、启动的线程数量不多。

极其小的工作负载可能无法在大规模 GPU 上良好扩展。这种情况下，可以尝试分批处理或将多个运算合并执行，让每次启动完成更多工作。PyTorch 的图编译器可以通过 `torch.compile` 融合某些运算，从而增加每次 kernel 启动的工作量——进而提高占用率和效率。

> **注** 在底层，`torch.compile` 通过把许多小运算合并成一个更大的 kernel，生成融合的 GPU kernel。只要可能，就应利用 PyTorch 编译器，因为它在某些场景下可以达到手写 CUDA kernel 的效率。对于形状稳定、长时间运行的训练/推理，建议使用 `torch.compile(mode="max-autotune")`；对于小批量、适合图捕获的循环，建议使用 `mode="reduce-overhead"`。两者都会在有利时启用 CUDA Graphs。如果不希望使用 graphs，也可以使用

`mode="max-autotune-no-cudagraphs"`。我们将在第 13 章和第 14 章深入讨论 PyTorch 编译器及其各种模式选项。

另外值得一提的是，PyTorch 的内置 kernel 通常在内部遵循最佳实践。例如，PyTorch 中的归约运算和逐元素运算都是通过启动配置器实现的，它们会根据设备和张量大小选择最优的块大小与块数量。

如果你深入阅读 PyTorch 的 C++ CUDA 代码，会看到限制块大小、以及针对某些算法在每个 SM 上启动多个块的逻辑——这本质上就是自动化的占用率调优。为了降低优化器开销，应尽可能启用融合实现，例如使用 `torch.optim.AdamW(..., fused=True)`，并搭配自动混合精度（AMP）。

对 PyTorch 用户而言，要点是优先使用已经过优化的大型库和运算，而不是自己编写自定义 kernel。如果确实需要自定义 kernel，就应用我们讨论过的那些占用率原则。

现在我们已经知道如何用足够的线程让 GPU 忙碌起来，接下来要讨论如何让每个 warp 更高效。如果每个 warp 都在浪费时钟周期，那么高占用率也意义不大。这就引出了 warp 级效率优化的讨论，包括 warp 分歧和 warp 内线程协作。

8.5 提升 warp 执行效率（warp 分歧）

即使借助良好的占用率拥有了充足的 warp，每个 warp 自身的执行效率也很重要。warp 执行效率衡量的是平均有多少比例的 warp 通道处于活跃状态。当同一个 warp 内的线程并非都在做相同的工作时，就会产生 warp 级的低效，这称为 warp 分歧；另一个原因是线程在等待数据加载。

具体来说，warp 执行效率偏低通常伴随发散分支数量多、谓词效率低，还可能伴随大量与内存相关的停顿。这意味着 warp 中的线程因 kernel 中 if/else 语句造成的分支分歧或数据依赖而处于空闲状态。

此时应尝试重构代码以消除这些低效。要提高 warp 执行效率，应努力改进内存合并、尽量减少线程分歧，并使用 warp 级内建函数实现最优的 warp 内（线程级）通信。

相对于更高层次的算法改动，这些技术往往只能带来适度的加速（例如 5%-30%），但在每个时钟周期都值得计较的高度优化代码中，它们可能至关重要。它们也与其他优化相辅相成：在优化了占用率、改进了内存访问之后，仍可以通过让每个 warp 尽可能高效地执行来进一步榨取性能。

8.5.1 warp 分歧的成因

如第 7 章所述，当同一 warp 内的线程走上不同的控制流路径时，就会发生 warp 分歧（branch divergence，分支分歧）。例如，考虑 kernel 内的一个 if/else：warp 中一部分线程执行 if 分支，另一部分执行 else 分支。

在 SIMT 执行中，warp 必须串行地执行两条路径：首先执行 if 分支的线程运行，而该 warp 中的其余线程被屏蔽（空闲）；随后执行 else 分支的线程运行，而第一部分线程闲置，如图 8-7 所示。

在发生分歧的代码段内，实际上只有一半或某个比例的线程在做有用的工作，这会降低整体吞吐量。如果 if 与 else 的划分是 50/50，那么在这部分代码上 warp 的活跃利用率会降到 50%；如果划分是 1 个线程对 31 个线程，那么在其中一个子分支中将有 31/32 的线程处于空闲状态。

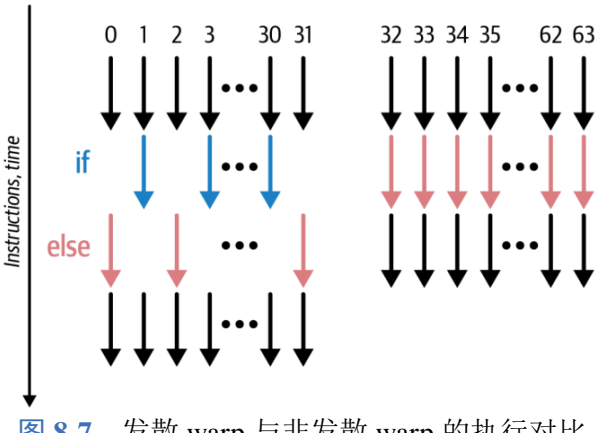

图 8-7　发散 warp 与非发散 warp 的执行对比

注　如果你的 kernel 含有多个分歧点，或者每条分歧分支都承载较重的工作，那么消除这些分支可以让收益叠加。在理想情形下（例如 50/50 的分

支划分），消除这条发散分支可使吞吐量几乎翻倍，达到约 2 倍的加速；消除多条重负载的分歧路径则可将收益进一步叠加。

总体效果是：warp 分歧导致一部分 GPU 核心闲置。由于每条分支路径都由不同的线程子集串行执行，指令总数也随之增加。因此，尽量减少 warp 内分歧是 warp 级效率的关键。

8.5.2 避免 warp 分歧的技巧

减少 warp 分歧有多种最佳实践，包括重构条件、以及将不同分支拆分到多个 kernel 中。

> **注** warp 分歧只影响同一个 warp 内的线程。不同 warp 之间的线程不会互相造成停顿。

此外，你还可以使用 warp 一致分支（warp-unanimous branch）、warp 投票（warp vote）内建函数以及谓词执行。下面逐一讨论：

重构条件

只要可能，就应当组织你的计算，使同一个 warp 内的各线程沿同一条执行路径运行。这可能意味着把产生分支的条件移到内层循环之外的更高层次，或者干脆放到一次单独的 kernel 启动中；也可以对数据排序或分组，让每个 warp 处理的样本更加同质。

举例来说，假设有一个数组，你需要对负数和非负数分别做不同处理。一种朴素做法是在 kernel 内部放置 `if(x<0)`，让每个元素各自产生分支；更聪明的做法是对数据进行划分，让一个 kernel 处理全部负数、另一个 kernel 处理全部非负数。让数据与 warp 对齐，就能减小单个 warp 不得不分裂执行的可能性。

【拆分成多个 kernel】

另一种做法是把工作拆成多次 kernel 启动：一个 kernel 处理 `if` 分支的情形，另一个 kernel 处理 `else` 分支的情形。你可以用前缀和或压缩（compaction）把线程分配到不同的 kernel。这样虽然避免了分支发散，代价却是要启动更多 kernel，并在 kernel 之间增加分发数据的逻辑。如果分支发散问题严重、且发散片段的指令量足够大，这样做就值得。

【把条件改写成 warp 内全体一致】

某些情况下，你可以把条件改写成 warp 全体一致（warp-unanimous）的形式，即要么一个 warp 全部 32 个线程都满足条件，要么全都不满足。一个常用技巧是在条件表达式中使用 warp 编号：先算出 `int warpId = threadIdx.x / 32;`，然后让一半的 warp 做任务 A、另一半做任务 B，写成 `if (warpId % 2 == 0) { /* Task A */ } else { /* Task B */ }`。

在这种情况下，给定 warp 中的所有线程要么一起进入任务 A，要么一起进入任务 B，不会发生 warp 分支发散，因为一个 warp 执行一个分支、另一个 warp 执行另一个分支；而单个 warp 内部所有线程对执行哪个分支总是达成一致。

其代价是：每个 warp 仍然要计算分支条件，但由于条件在 warp 内是一致的，每个 warp 只会执行其中一个分支。这种技术实际上是用灵活性换取 warp 的执行一致性——因为你限制了工作的划分方式。

> **注** 当你能以 32 个线程为一个粗粒度单位划分工作时，可使用 warp 全体一致的分支。

【利用 warp 投票并行算法】

Warp 内建函数（`__ballot_sync`、`__any_sync`、`__all_sync`）、cooperative groups 的 `warp.ballot/any/all`，以及设备端的投票掩码（`%WarpVote`）都能让一个 warp 以集体决定（投票）的方式，确定哪些 lane 需要“特殊”处理。随后 warp 动态地把这些工作委派、重新划分并压缩到一个（或少数几个）lane 上，而不是让全部 32 个线程一起发散。这样避免了逐 lane 的分支发散，但也引入了潜在的负载不均衡权衡，需要用性能分析来评估其影响。我们稍后会更详细地介绍 warp 内建函数。

【为短分支使用谓词执行】

CUDA 编译器有时会对较短的分支代码做谓词化（predication）。所谓谓词化，是指编译器把 `if` 转换为每个线程各自的布尔掩码，并让所有线程执行两条路径，但只提交与各线程自身路径相符的结果。

谓词化避免了分支发散，代价是每个线程要执行额外的无用工作。当分支非常短且发散严重时，这样做很划算。作为程序员，你可以通过编写无分支结

构来促成谓词化，例如 ?: 三元运算符，或针对简单条件的位运算技巧。比如下面这个朴素的 if/else 实现：

```
if (x > 0)
    y = f(x);
else
    y = g(x);
```

可以改写成如下形式：它对每个线程同时计算 f(x) 和 g(x)，再把两个结果分别乘以 cond 与其补数，从而选出与条件相符的那个结果：

```
float cond = x > 0 ? 1.0f : 0.0f;
y = cond * f(x) + (1.0f - cond) * g(x);
```

这里没有分支，因此也没有分支发散，但每个线程都做了额外工作——两个函数都运行了，包括在特定情形下并不需要的那一个。因此，只有当额外开销低于原本分支发散的代价时，谓词化才值得使用。

在实践中，你应当对这些优化的实际效果做性能分析。具体来说，使用谓词化时要查看“predicated-off threads”（被谓词关闭的线程）以及总指令数等指标。像 Nsight Compute 这样的性能分析器能告诉你谓词化究竟是减少了 warp 停顿，还是在执行不必要的多余工作从而降低了有效吞吐量。

一般而言，谓词化适合每个分支只有几条指令、非常短小的简单分支——尤其是当许多 warp 会因这些条件而发散时。反之，如果分支涉及大量工作，或只有极少数线程发散，谓词化反而可能更糟。请谨慎使用这一技术，并且只对小规模的分支工作才用它。

> **注** CUDA 编译器经常会对简单的谓词化做自动处理，但性能关键的 kernel 仍值得专门做性能分析核实。

总之，减少 warp 分支发散需要在算法和数据组织上下功夫。只要可能，就重新组织你的问题，使每个 warp 走统一的执行路径。这可能意味着拆分 kernel、把不同情形放到不同的启动中处理，重排数据让每个 warp 处理相近的条目，或把发散条件从内层循环中移出去。

当分支发散不可避免时（例如树或图这类每线程工作量天然各异的工作负载），应让发散片段尽量短、尽量少出现，使 warp 能迅速重新汇聚（reconverge）。你也可以借助 `__any_sync`、`__ballot_sync` 等 warp 级内建函数和其他投票原语，让线程共同就条件达成一致；还可以把工作压缩到更少的 lane，而不是让全部 32 个线程都去走分支。

8.5.3 分析与检测 warp 分支发散

Nsight Compute 的 Source Counters 和 Warp State 统计能精确定位发散 warp 之类的问题。检测方法是关注过高的 Branch Divergence 指标值，以及合并访存不佳、会触发大量重放（replay）或 L2 缓存缺失事件的加载。

Nsight Compute 还能标记出其他低效来源，包括共享存储 bank 冲突和 Tensor Core 流水线停顿。例如，若你的 kernel 显示 `memory_throttle` 停顿占比很高，可能说明对 GPU 较深的指令流水线而言，内存级并行度不足。此时可以尝试增加在途的独立访存数量，或使用 `cp.async` 之类的异步拷贝指令和 CUDA Pipeline API（见第 9 章与第 10 章）来隐藏延迟。

性能分析器会以低 warp 执行效率（warp execution efficiency）的形式展现 warp 分支发散的症状。该指标衡量一个 warp 中平均处于活跃状态的线程百分比：warp 执行效率为 30% 意味着任意时刻平均只有 30% 的线程在做有用工作，其余线程都因发散而闲置。

对含发散分支的 kernel 做分析时，分析器会标记出高比例的“谓词关闭”（predicated-off）指令——这些指令照样被取指和发射，却因 lane 掩码关闭而不做任何工作。你还会看到与实际处理的数据量相比明显虚高的“动态指令”计数。这两个数字合起来表明：每个 warp 都在把条件语句的各个分支逐一遍历、串行执行。

例如，一个线程子集（即若干 lane）执行路径 A，其余线程闲置；随后闲置线程变为活跃线程，执行路径 B。结果是指令流量翻倍，而 warp 的有效吞吐量（goodput）大幅下降——因为即使线程被掩码屏蔽，闲置线程组也仍然在取指和发射指令。

这在源代码中相对容易发现：任何按线程区分的条件——无论是阈值判断、稀疏数据循环，还是依赖数据的过滤——都会触发这种串行化的重复执行。要恢复性能，就必须消除或抹平这些发散分支。

你可以让条件在整个 warp 内保持一致，把它从紧凑循环中提出，或改用算术运算、查表等技巧，使全部 32 个线程执行相同的指令流并做有用的工作。下面用一个例子把问题讲清楚。

如果观察到 warp 执行效率偏低，就检查 kernel 的分支。把分支逻辑重构到单独的 kernel 中，或使用 warp 级原语（例如 ballot 同步）来更高效地处理发散，可能都会有所帮助，这一点我们马上会讲到。

8.5.4 用谓词执行减少分支发散

下面用一个例子演示如何利用谓词执行来分析并消除 warp 分支发散和分支本身。考虑一个对数组做阈值判断的 kernel，其语句为 `if (x[i] > 0) y[i] = x[i]; else y[i] = 0;`。

如果有一半的值为正、另一半不为正，那么大多数 warp 中都会有部分线程进入 `if` 分支、部分线程进入 `else` 分支。warp 会先为条件为真的线程执行 `if` 分支指令，同时把其他线程屏蔽掉；随后再执行一遍，为其余线程运行 `else` 分支。这样一来，完成同一组逻辑运算却消耗了两套指令，效率因此下降 50%。

改造前示例（CUDA C++）。在这一示例中，如果 warp 内的部分线程满足 `X[i] > threshold`，而另一些线程不满足，warp 就会发生分歧。这是一个明确会引发 warp 分支分歧的 kernel 示例：

```
// threshold_naive.cu
__global__ void threshold_naive(const float* X, float* Y,
                                float threshold, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        if (X[i] > threshold) {
            Y[i] = X[i];    // branch 1
        } else {
            Y[i] = 0.0f;    // branch 2
        }
    }
}
```

这会导致 warp 被串行地执行两次：其中一次执行针对一个线程子集完成赋值 `Y[i]=X[i]`，另一次执行则针对另一个线程子集完成 `Y[i]=0`。warp 执行效率会因此很低——如果两条路径各占一半线程，大约只有 50%。

改造后示例（CUDA C++）。下面是利用谓词化（predication）来减少分歧的写法。这一版本采用三目运算符，编译器很可能会把它翻译成一条带谓词的传送指令（根据条件选择的 SEL/MOV），而不是真正的分支：

```
// threshold_predicated.cu
__global__ void threshold_predicated(const float* X, float* Y,
                                     float threshold, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        float x = X[i];
        // Use a conditional move or multiplication by boolean
        float val = (x > threshold) ? x : 0.0f;
        Y[i] = val;
    }
}
```

在这种情况下，所有线程执行的指令序列完全相同：先计算 `val`，再把它存储起来。由于整个 warp 内的控制流是统一的，这就避免了 warp 分歧。条件为

假的线程只是把 `val` 置为 0。其结果是，所有线程都沿同一条路径执行，warp 执行效率得以保持在高位。

> **注** 在简单情形下，CUDA NVCC 编译器很可能已经按预期为三目运算符生成了带谓词的传送指令。在 PTX/汇编层面，你会看到 PTX 的 `@p` 谓词语法，它以谓词守护写入操作，而无须把 warp 拆分成多条路径。

改造后示例（PyTorch）。在 PyTorch 中，上述阈值操作可以用向量化运算写成如下形式：

```
# threshold_op.py
import torch
X = torch.randn(N, device='cuda')
Y = torch.maximum(X, torch.zeros_like(X))  # equivalent to Y = X > 0 ? X
↪  : 0
torch.cuda.synchronize()
```

与 0 做 `torch.maximum` 运算时不会在 GPU 上产生分支，因为它使用的是逐元素 max，而且以向量化方式实现。PyTorch 这类库在底层通过谓词化或位运算技巧，保证这些逐元素运算在 warp 层面不含分歧：

```
# jit_threshold_op.py
import torch
# In PyTorch, we can compile and fuse this operation
# for even higher throughput
@torch.compile()
def threshold_op(X):
    return torch.maximum(X, torch.zeros_like(X))
X = torch.randn(N, device='cuda')
Y = threshold_op(X)
torch.cuda.synchronize()
```

> **注** 正如第 13 章和第 14 章将要讲到的，`torch.compile` 会用 TorchInductor 把许多逐点运算融合进 kernel，其性能可以接近手写的 CUDA C++ 示例。不过，`torch.compile` 并不保证最优的占用率（occupancy）或指令级并行（ILP），因此你可能仍需要做进一步的性能分析与调优。

在底层，这类逐元素函数在 GPU 上最终会编译成单指令多数据（SIMD）风格的谓词化或位选择操作。这保证了 warp 内的每个线程都沿同一条指令流执行，从而避免了分歧控制流带来的串行化惩罚。

总体而言，减少 warp 分歧能够提升 warp 的执行效率，并在分歧本是主要瓶颈的场景中带来可观的加速。表 8-3 展示了用谓词化运算替换分支、从而消除 warp 分歧后的结果。

表 8-3 消除 warp 分歧后的性能分析结果。

指标	改造前	改造后
kernel 执行时间	30 ms	15 ms（–50%）
动态指令数	600 M 条指令	300 M 条指令（–50%）
warp 分支消解平均停顿延迟	200 周期	100 周期（–50%）
warp 执行效率	50%	99%
被谓词屏蔽的线程比例（%）	50%	0%

% 说明：本块为源 PDF 中表 8-3 表体的扁平化提取文本，其内容已完整并入上方的表 8-3 浮动体，不再重复排版。

> **注** 所有指标表中的数值仅用于解释概念，并非实测数据。不同 GPU 架构上的真实基准测试结果请参见 GitHub 仓库。

用单个带谓词的 `max()` 运算替换双路径分支后，Nsight Compute 的所有关键指标都出现了大幅改善。由 `gpu__time_elapsed.avg` 测得的 kernel 执行时间从 30 ms 降至 15 ms，吞吐量实际上翻了一番。

warp 执行效率攀升到接近 100%，因为每个 warp 中的所有线程始终在忙于有效工作。性能分析器还报告谓词屏蔽比例为 0%，表明在带谓词的 `max()` 方案下没有任何 lane 曾被屏蔽。这证实几乎所有的汇聚（reconvergence）开销都已被消除。

与此同时，动态指令数（`smsp__inst_executed.sum`）从 6 亿条降至 3 亿条，降幅 50%，因为每个 warp 不再花费周期串行地执行分支两侧的指令。warp 分支消解平均停顿延迟（`smsp__average_warp_latency_issue_stalled_branch_resolving`）同样减半，从 200 周期降到 100 周期。

> **注** 如果你的 kernel 含有多个分歧点，或者每条分歧分支承载的工作较重，消除这些分支的收益会相互叠加——每消除一条分支，就有可能带来超过 2 倍的加速。

需要指出，谓词化并非免费。部分线程仍会计算数值（例如 `val = x`），而这些计算结果永远不会被使用。如果每条分支都承载大量工作，那么让每个线程把两个分支都计算一遍，代价可能反而超过轻度分歧的代价。

在我们这个阈值示例这类简单情形下，谓词化占优，但你始终应当进行基准测试。可以在 Nsight Compute 的 warp 执行效率指标下分别测量基于分支的版本与谓词化版本。如果该效率偏低且每条分支都很轻量，谓词化很可能有所帮助。

如果分支很重，容忍一定的分歧——或者改用单独的 kernel——可能是更好的路线。归根结底，最小化分歧对 SIMT 性能至关重要，因此在设计算法时应尽量让 warp 保持在单一路径上，并把残余的分歧逻辑隔离到单独的 kernel 或 warp 规模的区域中。

> **注** CUDA 编译器通常会自动替你应用简单的谓词化，但对于性能关键的 kernel，手工调优与性能分析仍然值得。

8.5.5 利用 warp 内建函数实现高效的 warp 内通信

例如，当 warp 内的线程需要共享数据以计算归约/聚合时，可以使用 warp shuffle 内建函数——包括 `__shfl_sync` 与 `__shfl_down_sync`（第 6 章已介绍）——直接通过寄存器交换数值。这不同于先把数据暂存到共享内存再调用 `__syncthreads()` 的做法，后者会对性能产生负面影响。

共享内存方式会产生额外的 L1/L2 流量，并且需要整个线程块的屏障；而 shuffle 只在寄存器之间移动数据，并在每条指令处隐式同步 warp 内的 32 个 lane，只增加几个时钟周期的延迟。

关于完整讨论——包括性能对比、代码示例，以及如何实现 warp 级归约或前缀和——请参阅第 6 章。如果你的协作跨越多个 warp，仍必须使用块级或网格级同步（例如 `__syncthreads()`），或者使用协作组（cooperative groups，第 10 章介绍）。但对于纯粹的 warp 内通信，shuffle 几乎总是最快的选择。

简言之，如果你的算法要求同一 warp 内的线程共享中间结果或在计算上相互配合，就应选用 `__shfl_sync` 及相关的 warp 级手段；只有当协作确实跨越多个 warp 时，才使用块级共享内存与 `__syncthreads()`。

8.5.6 PyTorch 在 warp 级效率方面的考量

在纯 Python 层面，PyTorch 并未暴露 warp 内建函数，也不允许你直接控制 warp 级的执行。不过，作为 PyTorch 的最终用户，你会间接受益于这些优化，因为 PyTorch 内部的许多 CUDA kernel 都使用了 warp 级技巧。

例如，`torch.sum(x, dim=0)` 用于对张量的某个维度求和，当被归约的维度较小时（例如不超过 32 个元素），其实现往往会采用 warp 级归约。在这种情况下，每个由线程组成的 warp

……借助 shuffle 指令协同工作，在不产生分支发散、也不使用共享内存的情况下算出部分和。这一逻辑实现于库内部，因此你无需自己实现。

如果你以 PyTorch 扩展的形式编写自定义 CUDA kernel，那么在设备代码中，只要条件允许，你就应该使用这些 warp 级优化。例如，如果你的操作需要在 warp 内求和或在线程之间交换数据，应优先使用 `__shfl_sync` 这类内建指令，而不是朴素的共享内存方案，以提高速度并降低开销。同时，在任何自定义 CUDA 代码中都要留意 warp 分支发散。

如果你发现自己在逐元素地写 if 语句，可以改用张量级操作（例如 `torch.where`）把条件表达在整个张量上。这样库就能以更高效的方式处理这个条件——很可能是向量化的、warp 内一致的方式。

第 10 章讨论的 PyTorch JIT 编译器与图执行器可以融合逐元素操作。这让每个 kernel 在一次执行中完成更多工作——也就是让每搬移一字节内存完成更多计算——从而提高 warp 效率和算术强度。

当多个操作被融合进一个 kernel 后，一个 warp 单趟就能处理更多的工作。kernel 融合并不能神奇地消除算法固有的分支发散，但它确实意味着每个 warp 都在做更多有效的工作。因此，分支发散的开销会被摊薄到更多的计算之上。

例如，先做整流线性单元（ReLU）操作、再做 `abs()` 操作，融合后的 kernel 可以在单趟中一并处理两者。这样即便存在分支，也是每个 warp 在单趟中同时处理这两个操作。

作为 CUDA 程序员，你应当避免 warp 内分支发散，并对 warp 范围内的任何集合操作使用 warp 内建指令。这样才能保证 warp 中的全部 32 个线程始终并行地忙于有效工作。

作为 PyTorch 用户，要知道库已经在底层为你执行了这些优化。你的角色是以一种能够利用这些优化的方式来编写计算。例如，用内建的张量操作（已经用 CUDA C++ 优化）来代替带有逐元素条件判断的显式 Python 循环。如果你使用自定义 CUDA 扩展，则应用我们讨论过的同样最佳实践。

理想状态是每个 warp 的 32 条 lane 齐步协作。通过把分支发散降到最低，并对任何必要的通信使用快速内建指令，你可以保证 warp 执行效率。这些技术往往带来适度但可观的加速——大概是 1.1 倍到 1.3 倍的改进。在一个紧凑的 kernel 中，这可能意义十分可观。

这些优化还为指令级并行铺平了道路，后者让每个线程完成更多有效工作。接下来讨论这一主题。

8.6 暴露指令级并行

正如在占用率讨论中所见，并发运行许多 warp 可以让 SM 的调度器在任何因长延迟操作（例如全局内存加载）而停顿的 warp 上切换到其他 warp。除了启动足够多的线程外，我们还可以在每个 warp 内部利用 ILP，使单个 warp 不必等一条指令执行完就能发出下一条指令。

你可以重排或展开代码，使每个线程在消费结果之前先发出多个相互独立的操作（例如内存加载和算术指令）。这样，在前面的指令仍处于执行中时，GPU 的执行单元也能保持繁忙。

利用 ILP 可以让单个 warp 背靠背地发出一些相互独立的指令，从而改善延迟隐藏。例如，线程可以在等待一次加载完成期间，去执行与之无关的算术运算。图 8-8 展示了每个周期内多条指令交叠执行的示例。

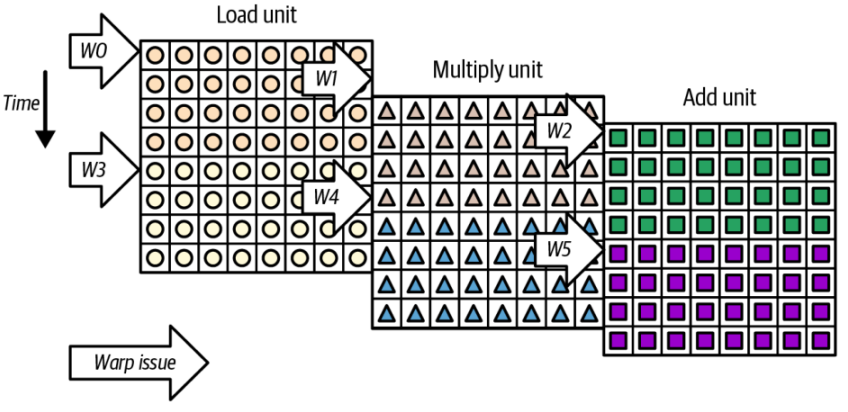

图 8-8　通过 ILP 实现指令交叠

通过展开循环体，你把原本每次迭代的“加载—乘法—存储”序列，改写成在回到循环之前先加载并相乘多个元素的序列。例如，与其这样写：

```
for (int i = 0; i < N; ++i) {
    float ai = a[i];        // load
    float bi = b[i];        // load
    sum += ai * bi;         // multiply after both loads complete
}
```

你可以展开循环，使每次迭代发出两条相互独立的乘法操作，而不是一条，如下一个代码块所示。这给了硬件在

第一条乘法仍在执行时——或仍在等待它的操作数从内存加载时——执行第二条乘法的机会。

这样就恰当地交叠并隐藏了延迟。其结果是更高的每周期指令数（IPC）和更好的延迟隐藏，如下所示并行计算两个乘法 `ai0*bi0` 和 `ai1*bi1`：

```
for (int i = 0; i + 1 < N; i += 2) {
    float ai0 = a[i];        // load a[i]
    float bi0 = b[i];        // load b[i]
    float ai1 = a[i + 1];    // load a[i+1]
    float bi1 = b[i + 1];    // load b[i+1]
    sum += ai0 * bi0 + ai1 * bi1;
}
```

这里，我们在做任何乘法运算之前，先把成对的值（`a[i]`, `b[i]` 与 `a[i+1]`, `b[i+1]`）加载进来。展开使每次循环迭代暴露出两条相互独立的乘法指令。这样 GPU 就能交叠这些算术操作，并隐藏每次操作数加载的延迟。

ILP 并不直接改变算术强度（每字节的 FLOPS）。不过，它通过让计算与内存相互交叠，或让计算与计算之间的依赖相互交叠，提升了整体吞吐量（FLOPS）。我们显式进行展开，是为了增加每个线程的独立工作量，以辅助双发射。换句话说，ILP 是通过对独立的工作做时间复用——而不是靠多做工作——来提升吞吐量的。

> **注** 一个常见的误解是：提高 ILP 会执行更多操作。其实不然——它只是让 GPU 的多个功能单元保持繁忙。只有当程序因延迟而存在空闲发射槽时，ILP 才能发挥作用。如果你的 kernel 已经在每个周期都充分占用了执行单元（例如一个没有停顿的紧凑的计算受限循环），那么提高 ILP 并不会真正提升性能。

为了促使编译器增加（即暴露）ILP，你可以对短循环使用 `#pragma unroll`，或调整 `-maxrregcount` 来允许编译器分配更多寄存器用于保存中间值。这是在明确告诉编译器：你接受它提高寄存器使用量，并可能由此降低占用率。

这样，ILP 与占用率形成互补。高占用率确保当某个 warp 停顿时有许多其他 warp 可以切换，而 ILP 则确保单个 warp 能尽可能多地发出独立指令，以填满流水线并隐藏延迟。这很像 CPU 中的超标量与乱序执行，只不过它是由编译器对 CUDA 代码的调度显式引导的。

8.6.1 warp 调度与双发射指令

每个 SM 包含多个 warp 调度器。例如，Blackwell SM 拥有四个 warp 调度器。如图 8-9 所示，每个调度器每个周期最多可以发射两条相互独立的指令。

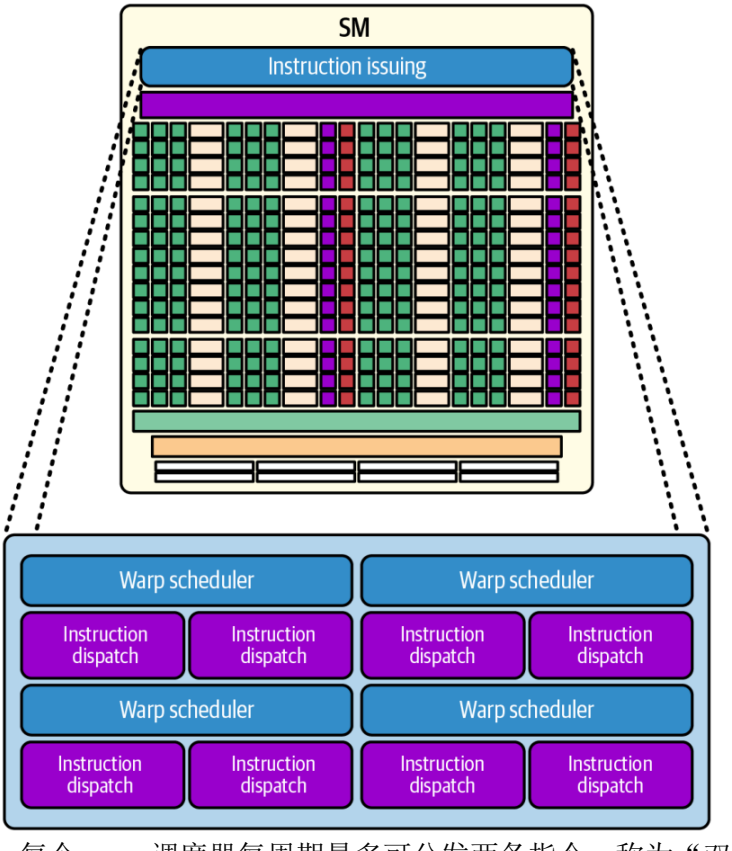

图 8-9　每个 warp 调度器每周期最多可分发两条指令，称为“双发射”

在实践中，这意味着单个 warp 可以让多个相互独立的操作在相继的周期之间交叠执行——有时甚至可以在同一周期执行，只要它们发往不同的流水线，例如一条数学、一条内存或一条特殊函数单元（SFU）流水线。这意味着多条指令可以同时处于在途状态并沿流水线推进。这种交叠就是我们前面讨论的 ILP。

> **注**　在 Blackwell 上，FP32 与 INT32 流水线已合并为一组统一的 CUDA 核心。因此，在给定周期内它们只能执行 FP32 或 INT32 指令其中之一，而不能两者同时执行。每个核心每周期必须选定一种数据类型。结果是，任何依赖

> 在同一周期双发射一条 INT 与一条 FP 指令的 ILP 将不再有效，因为 Blackwell 在每个核心上每周期只能二选一。因此，混合 INT32 与 FP32 的指令流在单个核心上不再从双发射中获益。混合指令流应当转而利用 warp/SM 并发，而不是依赖单核心的双发射。因而，你应当偏向于利用 ILP 让两条相互独立的数学与内存指令（或两条独立的数学指令，例如 FP32 与 FP16）同时在途。这会提高指令发射效率。

ILP 并不要求两条指令在同一个物理周期内发射。它只是确保：当一条指令（例如长延迟的全局内存加载）仍在途中时，warp 能在下一个周期立即发出另一条独立的指令（例如一次算术操作）。

由于这种交叠，当数据从内存返回时，算术工作已经取得了进展。这有效地隐藏了这次加载的延迟。在 Blackwell 上，得益于流水线机制，每个 warp 可以有多条指令同时在途，调度器在相继的周期内向不同的执行单元发射指令。在实践中，ILP 表现为一连串稳定的独立指令发射，而不是每个 warp 固定数量的并发指令。

考虑这样一个场景：kernel 中的每个线程加载两个数组元素——然后将每个元素分别乘以不同的常数，再把结果合并。第一个元素的乘法可以在第二个元素的加载仍在进行时执行。

这种重叠模式让 warp 的执行单元保持忙碌，而不是原地等待，从而在不改变每个元素加载与乘法次数的情况下提升整体吞吐量。下面是一个提高 ILP 的具体例子：

```
// Launch configuration: e.g., 256 threads per block, enough blocks to
 ↪  cover N
__global__ void independentOps(const float *a, const float *b,
                               float *out, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        float x = a[i];
        float y = b[i];
        // Two independent operations (no dependency between u and v):
        float u = x * x;
        float v = y * y;
        // Dependent operation that uses both results:
        float sum = u + v;
        out[i] = sqrtf(sum);
}
}
```

在这个 kernel 中，每个线程从不同的数组加载两个值 `x` 和 `y`，并计算两个结果 `u` 和 `v`。`u = x*x` 与 `v = y*y` 的计算相互独立，因为二者都不使用对方的结果。

以这种方式组织代码，而不是写成 `sum = x*x + y*y`，就给了编译器安排指令序列、进而暴露 ILP 并提升性能的机会。具体来说，它可以先发射 `u = x*x` 的指令，然后在下一个时钟周期、趁 `u` 尚未计算完成时发射 `v = y*y`。

当第一个乘法正在执行——或还在等待流水线槽位——时，第二个乘法可以同时在另一个算术单元中并行运行。等到需要计算 `sum = u + v` 时，两个乘法大概率都已完成。这样一来，warp 中便有多条指令同时执行（in flight），这有助于隐藏这两次乘法运算的延迟。

相反，考虑下面这段代码：`v` 的计算被放在了对 `u` 的使用之后。在这种情况下，我们会制造一条不必要的依赖链，从而限制 ILP：

```
float u = x * x;
float temp = u + 1.0f;    // some dependent use of u
float v = y * y;
...
```

这个 kernel 先计算 `u = x * x`，并立即使用该结果构造 `temp = u + 1.0f`。只有在算出 `temp` 之后，才会发射 `v = y * y`，而它其实既不依赖 `u`，也不依赖 `temp`。因此，把它放在 `temp` 之后，就迫使 GPU 只有等 `u + 1.0f` 这条指令执行完毕，才能发射 `v` 的乘法。

实际上，这样就形成了一条串行依赖链：先 `x * x` → `u`，再 `u + 1.0f` → `temp`，然后才有 `y * y` → `v`。在硬件执行（或等待）`u + 1.0f` 运算的这段时间里，哪怕 `v` 在数学上完全独立，也无法开始 `y * y`。这种顺序安排人为地限制了指令级并行，因为一个独立的乘法也不得不排在一条依赖指令的后面。

相比之下，请看下面这段经过重排与优化的代码：

```
// Optimized ordering with better ILP:
float u = x * x;    // Independent multiply on x
float v = y * y;    // Independent multiply on y (issues immediately
↪  after u)
float temp = u + 1.0f;  // Only now use u
// ... use v or temp as needed ...
```

在这里，我们把代码重排成让 `x * x` 和 `y * y` 背靠背地出现（即 `float u = x * x; float v = y * y; float temp = u + 1.0f;`）。这样，GPU 就能在相邻的时钟周期里连续发射这两个相互独立的乘法，而不必等待 `u + 1.0f` 的加法完成。

也就是说，当 `x * x` 的结果可能还停留在“待定”的流水线槽位中时，硬件已经可以开始执行 `y * y`。只有等这两个乘法都算出结果之后，GPU 才执行 `temp = u + 1.0f`。

换句话说，现在有两次乘法同时在执行中，这既填满了执行单元，也隐藏了延迟。这种改进后的 ILP——尽快发射 `v = y * y`——确保对 `y` 的乘法与对 `x` 的乘法以及后续的加法重叠进行，而不是被迫等待。于是 warp 空闲的周期更少，这直接转化为更高的吞吐量。

GPU 依靠编译器、有时也依靠程序员给出的提示来调度相互独立的指令。在现代 CUDA 编译器上，像前文那样简单的模式通常都能被自动检测出来并得到最优调度。不过，你仍然可以通过恰当地组织代码结构，或使用 pragma 编译指示来促成 ILP。

这个例子很简单，但它说明了一个道理：只要能避免，就不要在线程内部把相互独立的工作串行化。如果一个线程需要加载两个数组并对两者做运算，那么并行处理或交错执行，总好过一段接一段地串行执行。

8.6.2 ILP 与占用率

在现代 GPU 上，每个 SM 最多可驻留 64 个 warp，即 2,048 个线程（2,048 线程 = 32 线程/warp × 64 warp）。只要依赖关系允许，warp 调度器可以在每个时钟周期发射多条指令。INT32 与 FP32 核心是统一的，在给定的时钟周期内要么作为 FP32 运行，要么作为 INT32 运行，而不能两者同时进行。每个 SM 的寄存器堆为 256 KB，即 64K 个 32 位寄存器。正如稍后会看到的，这个大容量寄存器堆对于在不发生溢出的前提下维持 ILP 非常重要。

如果你编写的 kernel 几乎没有 ILP，每个线程每次恰好只发射一条运算，那么通常需要大约 1,536 个活跃线程（约 48 个 warp，占用率 75%）才能打满 SM 的执行单元（这些数字并非硬性上限，而只是近似值）。

相比之下，哪怕只暴露少量 ILP，也能显著降低所需的线程数。表 8-4 归纳了在不同的 ILP 程度下，ILP 如何降低打满一个 Blackwell SM 所需的线程数和占用率。

表 8-4　不同 ILP 程度下打满 Blackwell SM 所需的线程数与占用率。

ILP（每线程独立运算数）	达到约 100% 利用率所需线程数	占用率（占 2,048 线程百分比）
1（无 ILP）	约 1,536 线程（48 warp）	75%
2	约 1,024 线程（32 warp）	50%
3	约 768 线程（24 warp）	37.5%
4	约 512 线程（16 warp）	25%

其中，两路 ILP——每个线程接连发射两个相互独立的运算——通常只需约 1,024 个活跃线程（约 32 个 warp，占用率 50%）即可达到满吞吐量。这是因为只要有一个运算尚未完成，每个 warp 都能让两个运算同时处于执行之中。

而三路 ILP 只需约 768 个线程（约 24 个 warp，占用率 37.5%）即可打满；四路 ILP 可能只需 512 个线程（约 16 个 warp，占用率 25%）就能填满流水线。这是因为每个 warp 自己就承担了四个相互独立运算的工作量。

> **注** 每线程在执行中的运算越多，每个 warp 就越能让数学流水线保持忙碌——即使线程数较少也是如此。

显然，提高 ILP 可以让你用更少的 warp 达到峰值计算吞吐量。当工作负载无法启动大量线程时，或者当你想腾出片上资源供其他任务使用时，这一点尤其有价值。

需要注意，可暴露的 ILP 存在实际的上限。尽管 Blackwell SM 可以让每个 warp 同时保持大量指令在执行，但每个 warp 能容纳的待执行指令数量是有限的。

除此之外，编译器还必须在不超出可用寄存器数量、也不压垮 GPU 指令解码带宽的前提下调度相互独立的指令。所谓解码带宽，是指取指与译码硬件每个时钟周期能推送给执行单元的指令数量的上限。一旦触及这些极限，继续增加相互独立的运算只会带来边际收益递减。

一旦已经有足够的独立工作让发射槽和内存系统保持忙碌，再进一步提高 ILP 的实际收益就微乎其微了。强行做到五路或六路 ILP 几乎没有增益，而且由于额外的寄存器压力和指令开销，甚至可能损害性能。

> **注** 在 Blackwell 上，理想的 ILP 配置通常是每线程 2 到 4 个相互独立的运算。到了这一区间，其调度器和缓存一般已经饱和。不过，确切的临界点取决于 kernel 和工作负载，请使用 Nsight Compute 的发射与停顿指标来确定在哪里收手。

8.6.3 循环展开、交错与编译器提示

要让自己的 kernel 利用 ILP，就要寻找这样的机会：在任何一个结果被需要之前，让每个线程先发射多条相互独立的算术或访存指令。常见的模式包括以下几种：

展开小循环

循环展开允许每个线程使用相互独立的累加寄存器完成 N 次累加（例如 2 倍或 4 倍展开）。例如，考虑下面这个循环：

```
float sum = 0.0f;
for (int k = 0; k < 4; ++k) {
    float a = A[idx * 4 + k];
    sum += a * w[k];  // dependent on load a
}
```

可以改写这段代码以利用 ILP，如下所示。这一改写生成了四对相互独立的“加载-乘法”对——a0..a3 每个对应一对——可以快速连续地发射：

```
float a0 = A[idx * 4 + 0];
float a1 = A[idx * 4 + 1];
float a2 = A[idx * 4 + 2];
float a3 = A[idx * 4 + 3];
float sum0 = a0 * w[0];
float sum1 = a1 * w[1];
float sum2 = a2 * w[2];
float sum3 = a3 * w[3];
float sum = sum0 + sum1 + sum2 + sum3;
```

这里，四条加载指令首先全部发射。每当一次加载完成，其对应的乘法即可执行，让后续的加载与计算相互重叠，从而掩盖内存加载的延迟。换言之，GPU 会让这些操作交错执行：当某一对正在等待内存时，另一对可以正在进行乘法运算。这提高了 ILP 并掩盖了延迟。

交错独立操作

你可以交错使用不同数据元素的操作，如下所示：

```
float x = A[idx];
float y = B[idx];
// If you wrote float u = x * c; then float v = y * d;,
// the multiplies are independent.


float u = x * c;  // can start as soon as x is ready
float v = y * d;  // can start as soon as y is ready, overlapping with u
```

这样，通过不让一条依赖指令阻塞下一条独立指令，执行单元在每个周期都保持忙碌。在这段交错代码中，`u = x*c` 与 `v = y*d` 是相互独立的指令，可以并发执行。编译器会把它们背靠背地调度，使得一个乘法正在进行时，另一个乘法仍在完成。这正是 ILP 的实际体现。

使用编译器提示

对小循环使用 `#pragma unroll` 之类的编译器提示，可以帮助编译器显式地创建多条算术链。如有需要，你还可以调整 `-maxrregcount`，允许编译器为展开后的变量使用更多每线程寄存器。允许每线程使用更多资源，就是以减少线程数为代价来提高 ILP。这自然会以牺牲一些占用率换取更高的 ILP。

简而言之，为了在 Blackwell 的深度流水线上掩盖延迟并最大化吞吐量，应当将高占用率（例如，用足够的 warp/线程来掩盖停顿）与高 ILP（例如，每线程多条独立指令）结合起来。这样，即使某条指令正在等待（例如，内存加载或长延迟算术运算），warp 也能立即发射其他指令。这将显著提高 kernel 实际达到的 FLOPS。

> **注** 在大幅展开时，请留意 Nsight Compute 中的取指与发射停顿。过度展开可能让代码体积膨胀——或使解码与分发饱和——却不带来进一步的吞吐量收益。

8.6.4 性能分析与缓解寄存器压力

然而，要留意寄存器压力，因为 ILP 通常需要更多寄存器来保存多个独立的值和结果。如果展开得过多——或者加入过多并行计算——寄存器使用量可能增长到占用率显著下降的程度。与以往一样，找到恰当的平衡才是关键。

CUDA 编译器在自动展开循环方面做得很好。但如果你通过 `#pragma unroll` 之类的方式手动展开过多迭代，把 ILP 推得太远，就可能注意到占用率下降。

在这种情况下，Nsight Compute 会在其占用率分析中显示“Limited by Registers”。这通常是明确的信号，说明你做得过头了。看到这一信号时，应当回调展开程度、减少独立累加变量的数量，或放宽启动边界（launch bounds）。

具体来说，你可以降低要求的 `MinBlocksPerSM` 或提高 `MaxThreadsPerBlock`，让每个块在不发生溢出的情况下可以使用更多寄存器。换句话说，接受略低一些的占用率，以避免严重的寄存器溢出。这样寄存器压力得到缓解，占用率也随之恢复。

从性能分析的角度看，如果 ILP 确实有效，你会看到“Stall: Exec Dependency”百分比下降，因为 warp 等待前序指令的时间变少了。同时，你还会看到每周期指令数升高。

Nsight Compute 会报告发射效率（Issue Efficiency）或 IPC 指标。随着 ILP 增加，这些指标应当上升。例如，在提高 ILP 时，你会看到发射效率或每周期指令数升高。再举一例：如果展开循环后每个 warp 调度器的 IPC 从 1.0 增至 1.8，则说明 warp 现在平均每周期发射接近两条指令，而不只是一条。不过，实际数值取决于指令组合以及架构的调度约束。

你还可能发现，在比以前更低的占用率下就能取得良好的性能。理想的情况是：你有足够的 ILP 让每个 warp 保持忙碌——同时有足够的 warp 让 SM 保持忙碌。在两者兼备的情况下，warp 内部与 warp 之间的延迟都得到了掩盖。

至此，我们已经讨论了 warp 级并行与指令级并行。这是让 GPU 计算单元保持忙碌的两种途径。接下来，我们转向测量并提升算术强度（arithmetic intensity），也就是最大化每次内存访问所完成的有效工作。

8.7 本章要点

在本章中，你学习了如何通过把工作从缓慢的全局内存转移到更快的片上资源和计算单元，来发现并消除 GPU kernel 的瓶颈。遵循“性能分析—诊断—优化—再分析”的循环，你可以把 kernel 从利用率不足或内存受限的状态，转变为计算饱和、高吞吐量的例程。这些技术将帮助你充分发挥 GPU 的全部能力：

使用 Nsight 与 torch.profiler 进行性能分析

先用 Nsight Systems 可视化端到端时间线，揭示 CPU-GPU 间隙、kernel 重叠以及 NVTX 区间。然后用 Nsight Compute 的停顿指标、Roofline 分析和占用率报告深入剖析各个 kernel。在 PyTorch 代码中，你可以插入分析器插桩（使用构建于 Kineto 库之上的 `torch.profiler` API），把 Python 层级的操作直接映射到 GPU 活动。请记住，PyTorch 分析器的 `with_flops` 估算基于公式，且只覆盖有限的算子集合；它并不读取 GPU 硬件计数器。若要获取硬件计数器数据，应针对关心的 kernel 使用 Nsight Compute。

调优占用率以掩盖延迟

每个 SM 拥有足够的 warp，是覆盖内存与指令延迟的关键。你可以通过代码重构或编译器提示来降低每线程的寄存器/共享内存用量；还应选择与 SM 资源相匹配的块大小（例如，128–256 线程）。这些做法能把 kernel 的实际占用率提升到最优区间——大约 50%–75%——并消除利用率不足的情况（例如，Blackwell 上仅约 32–48 个 warp）。

最小化 warp 分歧以提升 SIMT 效率

由于 warp 按锁步方式执行，任何分支分歧都意味着被掩蔽的 lane 闲置。应当重构 kernel，使一个 warp 内的所有线程走相同的路径；使用三元运算和 `torch.maximum` 这类谓词化操作，或使用 warpvote 内建函数来压缩活跃 lane。这会提升 warp 执行效率并减少串行化执行。

识别性能形态

每个 kernel 都属于以下四种类型之一：利用率不足、延迟受限、内存受限或计算受限。划分依据是 FLOPS、带宽使用情况与停顿原因。弄清哪种形态适

用，有助于确定确切的优化策略。例如，“Long Scoreboard”停顿占比很高，就意味着该工作负载是延迟受限的。

暴露 ILP

通过展开循环、打断依赖链以及预取数据，你可以暴露指令级并行，使每个线程每周期发射多条独立操作。这样，二路或四路 ILP 可以把填满一个 SM 所需的 warp 数减半——减少执行依赖停顿、抬高 IPC，并降低“Stall: Exec Dependency”指标。应始终结合线程数一起分析不同的 ILP 深度，以找到 kernel 的最佳平衡点。

迭代优化与验证

每次修改之后——无论是调整占用率、重构 ILP、分块（tiling）还是缩放数值精度——你都应当重新进行性能分析，以确认内存停顿减少、执行依赖更少、实际占用率或 FLOPS 更高。在降低精度时，务必始终以 FP32 基准为对照，使用断言或数值校验来防范不可接受的精度回退。

8.8 结语

你已经看到，高效地调优 GPU 性能需要一套迭代式的工作流：首先用性能分析器进行测量；然后找出主要瓶颈（计算、内存、互连等）；最后应用相应的优化并重复这一过程。

虽然每一代新的 GPU 架构都会带来计算能力和内存带宽的提升，但同时也引入了更多复杂性。要持续保持峰值性能，你必须不断跟进这些技术创新。

我们介绍了如何调优占用率、避免 warp 发散以及提高 ILP。你还学习了如何利用性能分析器在多块 GPU 之间关联 CPU、kernel 和 NVLink 流量。随后我们深入探讨了 Nsight Compute 中每个细粒度的 kernel 分析。

在接下来的章节中，我们将在此基础上继续深入，聚焦 kernel 级别的效率以提升算术强度。为此，你需要充分利用 GPU 上经过硬件优化的资源：用于计算的 Tensor Core、为 Tensor Core 供数的 TMEM，以及负责数据传输的 TMA。在支持 TMA 的现代 GPU 上，TMA 仍然是从全局内存向共享内存进行批量拷贝的首选方法。让我们继续向硬件的峰值性能极限迈进！

第 9 章 提高 CUDA kernel 的效率与算术强度

即使通过大规模并行与高指令级并行（ILP）完全隐藏了延迟，kernel 的性能仍可能受限于它每次访存所能完成的有用计算量。算术强度（arithmetic intensity），又称操作强度（operational intensity），衡量的是每从内存传输一字节数据所执行的浮点运算量，即每字节的 FLOPS。

较新的 GPU 世代在计算吞吐量上的提升已远超内存带宽的增长。这一不断拉大的差距意味着，提高算术强度比以往任何时候都更加关键。算术强度越高，说明 kernel 每获取一字节数据完成的计算越多，这正是充分发挥 GPU 计算能力的关键所在。

算术强度是 Roofline 性能模型中的关键指标。Roofline 模型是一种直观的可视化工具，它以算术强度（FLOP/字节）为横轴、kernel 性能（FLOP/秒）为纵轴绘图，并绘制内存带宽与计算吞吐量的硬件上限（屋盖），从而让我们看出一个 kernel 是内存受限（性能受内存传输限制）还是计算受限（性能受 ALU 吞吐量限制）。

在实践中，你可以使用 Nsight Compute 等工具生成 Roofline 图，它内置了 Roofline 分析视图。借助这些工具，你可以先确认 kernel 起初是内存受限还是计算受限，然后在做优化的过程中持续进行性能分析并验证改进效果。

目标是把 kernel 推向计算受限区间，从而充分利用 GPU 日益增长的算力。Roofline 性能模型能够恰当地引导你的优化朝这一目标前进。

如前一章所示，Roofline 图用一条水平线表示硬件的峰值计算吞吐量（屋盖），并用一条从原点出发的对角线表示受内存带宽限制所能达到的峰值吞吐量。kernel 的算术强度决定它在横轴上的位置，其性能可与这些上限进行比较，如图 9-1 所示。

算术强度低的 kernel（即每搬运一字节数据只做少量数学运算）将是内存受限的。此时 kernel 的速度受硬件内存带宽的约束，因为 GPU 大部分时间都在等待数据，而不是在埋头计算。

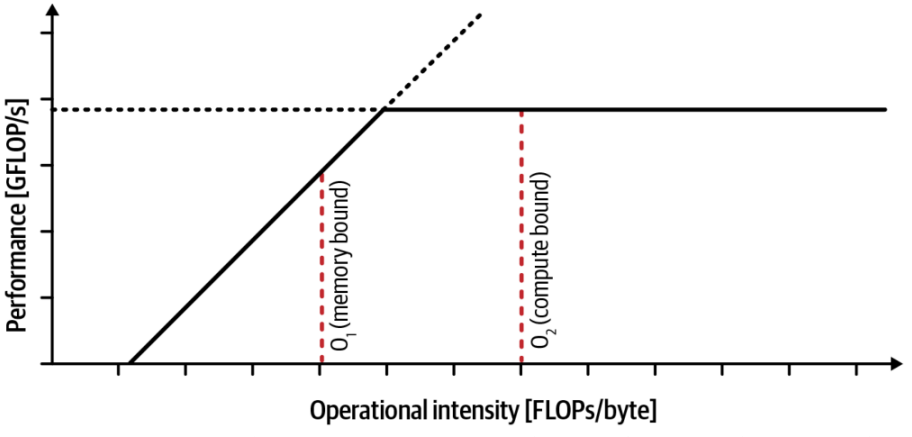

图 9-1　Roofline 模型示例（GFLOP/s 与算术强度 FLOP/字节的关系）

相反，算术强度非常高的 kernel（即每搬运一字节执行大量 FLOP）则是计算受限的，因为它让 ALU 和 Tensor Core 接近峰值能力运转。此时，kernel 对内存带宽的占用只是次要考量。

目标始终是尽可能提高算术强度，即对每字节往返全局内存的数据完成更多的计算工作（FLOP/字节）。可以借助多种技术提高算术强度：例如用循环分块来复用数据、利用片上 L1/共享内存进行复用，以及把多个 kernel 融合为一个，使中间结果不必写回全局内存。

> 注　PyTorch 的 TorchInductor 等现代编译器框架会自动执行其中部分优化，使计算留在 GPU 上、减少片外内存流量，并提高有效算术强度。不过，作为开发者，可能仍需要手动组合这些技术或编写自定义 CUDA kernel，例如确保数据在被逐出缓存之前得到尽可能充分的复用。

你还可以使用低精度数据类型（FP16、FP8、FP4）来减少内存传输量，并利用 Tensor Core 提高每秒完成的 FLOP 数。二者结合能提高每字节的 FLOP 比值，从而提高算术强度。接下来我们讨论其中的一些技术。

要记住，并非每个工作负载都能轻松提高算术强度：它受算法特性的约束。然而，你应当寻找一切机会改进算法、复用数据、融合操作并增大批量大小，在不改变算法结果（例如准确率）的前提下提高算术强度。

9.1 多级微块分块与软件预取

如第 7 章所述，分块（tiling，又称 chunking 或 blocking）与数据复用是提高算术强度的有效途径。在那一章中，我们展示了如何把 A 和 B 的一小块子矩阵（tile）加载到共享内存中，使从全局内存读取的每个字节都能以静态随机存取存储器（SRAM）的速度参与许多次乘累加运算。

每当重构代码使每个元素只加载一次、却被使用几十上百次（分块正是如此）时，就等于把每字节的 FLOP 比值乘上了复用因子。例如，在典型的矩阵乘法中，A 与 B 的一个 32×32 分块可为共享内存中的每个元素产生 1,024（$1024 = 32 \times 32$）次独立的乘法。因此，与每次运算都直接从 DRAM 取数相比，算术强度显著上升。

在简单的共享内存分块之外，还可以通过多级分块进一步提高强度并暴露更多 ILP。采用多级分块时，把一个 tile 暂存到共享内存之后，让每个线程再用 `float4`、`<half2>` 之类的向量化类型将微块（microtile）加载进寄存器。这样，重复的运算就完全在寄存器中完成。图 9-2 展示了多级分块的一个例子。

这种 SM 内部复用路径（寄存器 $\rightarrow$ SMEM $\rightarrow$ DRAM）在每一层级都缩小了工作集，并最大限度减少片外流量。与往常一样，填充共享内存时应确保全局读取合并（coalesced），并对共享数据进行填充（padding）或混洗（swizzle）以避免存储体冲突，如第 7 章所述。

在现代 GPU 上，这些内层循环的分块步骤通常由 MMA fragment API 代为完成。硬件利用 Tensor Core 指令在共享内存与 Tensor Memory（TMEM）之间隐式搬运数据。TMEM 的使用由编译器和库来管理。在现代 GPU 上，`tcgen05` 指令会在共享内存与 TMEM 之间隐式暂存数据，并使用独立的 TMEM 地址空间。不过，在实现某些算法并需要显式控制时，开发者仍可以用 `cp.async` 或 TMA 手动把 tile 移入共享内存。

与多级分块密切相关的一项技术是软件预取，它常以双缓冲（double buffering）的形式实现。例如，不必等到当前 tile 的计算结束，就可以为下一个 tile 发起异步加载，把数据送入共享内存，从而让 DRAM 与共享内存（SMEM）之间的传输和正在进行的算术运算重叠起来。谨慎的预取能显著减少停顿时间并提

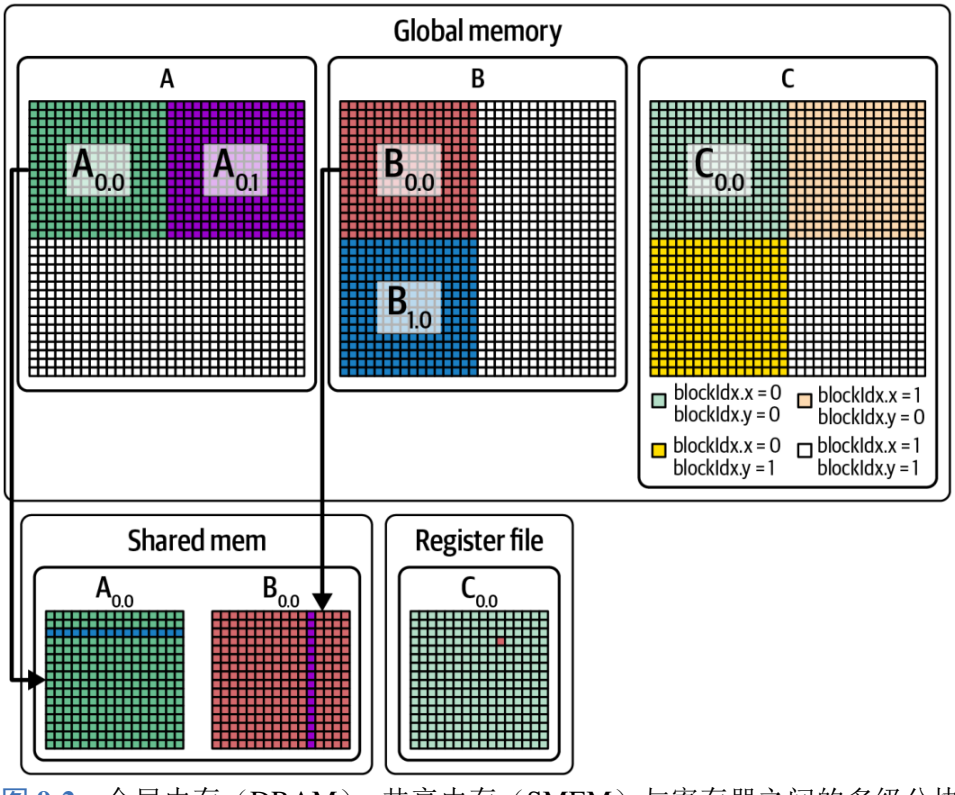

图 9-2　全局内存（DRAM）、共享内存（SMEM）与寄存器之间的多级分块

高吞吐量。其思路是让数据传输与计算重叠，使 ALU 永远不会因等待数据而停顿。

在像 Grace Blackwell 这样的 CPU-GPU 超级芯片上使用统一内存（unified memory）时，可以用 `cudaMemPrefetchAsync()` 提示某个 tile 很快将被用到。这一提示会让运行时通过 NVLink-C2C 迁移相应页面。但预取只是一种提示，并不构成保证。你仍需确保传输与计算充分重叠，并进行恰当的同步，以避免页错误停顿。以这种方式让数据搬运与计算重叠，可在需要新 tile 时保证 ALU 一直有数据可用，进一步隐藏内存延迟并提高实际达到的 FLOPS。

> **注**　统一内存简化了开发，但未必带来最佳性能。经验丰富的用户往往更倾向于显式使用 `cudaMemcpy` 或锁页（pinned）内存分配，以完全避免页面迁移开销。

简言之，来自 DRAM 的每一字节在片上层级（寄存器或共享内存）被复用的次数越多，算术强度就越高；而更高的算术强度会把 kernel 推得离计算受限区间更近。

9.2 利用线程块簇进行分块

在现代 GPU 上，可以借助 Cooperative Groups 提供的 CUDA 线程块簇（thread block cluster，第 10 章讨论）来扩展分块复用的思想。它们让多个线程块通过分布式共享内存（DSMEM）共享数据，如图 9-3 所示。

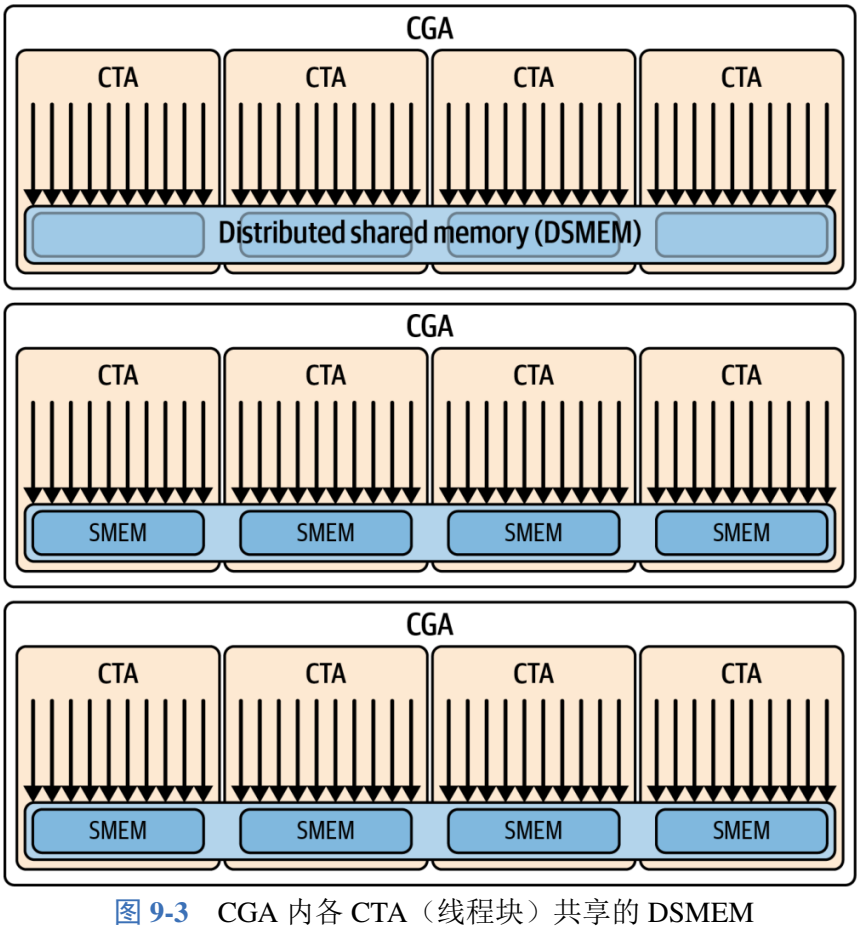

图 9-3　CGA 内各 CTA（线程块）共享的 DSMEM

下一章将详细介绍 CGA 和线程块簇，但它们在这里值得一提，因为它们能够直接提高算术强度。例如，四个线程块组成的簇可以使用 Tensor Memory Accelerator（TMA）的组播（multicast）特性协同加载一个 tile，如图 9-4 所示；该图以四个 CTA 演示了这一机制。

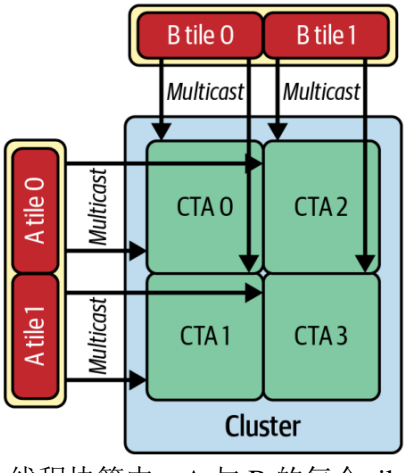

图 9-4 在这四个 2 × 2 线程块簇中，A 与 B 的每个 tile 都通过组播同时加载到四个 CTA（线程块）中（图片来源：https://oreil.ly/EEO_O）

每个 tile 被划分到四个线程块上，使该 tile 的全局内存流量在整个簇上摊销。这些 tile 只取一次，便由全部四个线程块复用。

在 2×2 簇中，当四个 CTA 通过组播复用同一份数据时，线程块簇最多可将全局内存流量减少 4 倍。此外，线程块簇通过降低分母（即从全局内存搬运的字节数）来提高 GPU 整体的算术强度。其中一种特殊形式称为线程块对（thread block pair），稍后在介绍 Tensor Core 分块时会讨论到。

> **注** Blackwell 支持最多 16 个线程块的线程块簇，但需选择启用非可移植的簇大小（超出默认可移植上限 8 个 CTA）。为此，需在 kernel 上设置 `cudaFuncAttributeNonPortableClusterSizeAllowed` 属性。更大的簇能提升复用度，但可能降低占用率，因此在改用 16 之前应先做性能分析。这可以支撑更大的多 SM tile，把数据复用最大化到 16 倍，并以相近的倍数提高算术强度。

9.3 kernel 融合

提升算术强度的另一种方法，是把多个操作——或多个循环迭代——融合为一个操作。通过把多个 kernel 融合在一起，从内存加载的数据可以在写回之前服务于多次计算和多次迭代。

同样，上一节讨论的循环展开能让单个线程对每个加载的数据元素执行更多计算，但代价是寄存器用量增加。过度的融合会加大每线程的寄存器压力并降低占用率，因此这里存在权衡。

务必对融合后的 kernel 进行性能分析。如果寄存器用量过大并开始溢出到局部内存，融合的收益可能被额外的内存流量所抵消。不过，只要找到合适的平衡点，就能提高每字节搬运所对应的 FLOPS，这在内存带宽成为限制因素时非常有利。

现代深度学习框架可以通过其即时编译器和图优化器自动完成融合与展开。例如，PyTorch 的 `torch.compile`，特别是其中的 TorchInductor，能够自动融合逐元素操作序列。我们将在第 13 章和第 14 章讨论 PyTorch 编译器。

> **注** 逐元素操作（elementwise operations）也称为逐点操作（pointwise operations），它对张量的每个元素独立地施加一个简单计算。

融合这些逐元素操作后，中间值得以保留在片上，从而消除了不必要的内存流量。这提高了从全局内存每取回一个字节所能完成的工作量——也就是提升了算术强度。

例如，朴素的实现会启动两个 kernel：第一个 kernel 读取 x 并把 y 写入全局内存；第二个 kernel 读取 y 并写入 z：

```
y = sin(x);
z = sqrt(y);
```

这里每个元素都被访问两次：一次是在 `sin(x)` 之后，一次是在 `sqrt(y)` 之后。因此每个 kernel 的算术强度都非常低，因为每次加载/存储操作只执行一个开销高昂的数学函数（一条多周期 ALU 指令）。相比之下，融合后的 kernel 只需一趟处理就能完成同样的操作序列：

```
z[i] = sqrt(sin(x[i]));
```

这里每个 `x[i]` 只加载一次，`sin` 和 `sqrt` 都在寄存器中完成，也只有最终的 `z[i]` 才写回内存。由于中间量 y 从不进入全局内存，每字节有效 FLOPS 大幅跃升，使该操作更靠近计算一侧的 Roofline 上限。

> **注** 根据经验，如果某段数据会被同一线程块中的线程读取不止一次，通常值得把数据暂存到共享内存中，以消除冗余的全局加载。这有助于把 kernel 从内存受限区间提升到计算受限区间，从而更好地利用 GPU 充足的 FLOPS。

融合减少了全局内存流量并提高了算术强度，因为现在每个元素在每次读写内存操作中都要执行两个数学运算。在我们的例子中，每个元素的 FLOPS 翻了一倍（`sin` + `sqrt`），而由于没有中间写出，内存流量大约减半。这就得到了显著更高的算术强度，即 FLOPS/字节。

为了把这一点讲透，我们用一个具体例子来演示算术强度。假设要对二维张量 x（形状 [batch, hidden]）中每个长度为 hidden 的行做 L2 归一化：对每一行 b，先算出一个范数 norm_b $= \sqrt{\sum_i x[b,i] \cdot x[b,i] + \varepsilon}$，再对所有的 i 写出 $y[b,i] = x[b,i] / \text{norm_b}$。

朴素实现会在第一个 kernel 中求平方，在第二个 kernel 中把每行归约为一个标量，再在第三个 kernel 中做除法。这需要多次 kernel 启动，还要把中间结果写入 HBM。

假设这 4 个 kernel 各需要 1 FLOP 的计算。这样一来，4 个 kernel 中每一个的算术强度都是每 12 字节 1 FLOP（2 次 float 读取、1 次 float 写入），即 0.083 FLOPS/字节。

反过来，我们可以把这 4 个 kernel 融合成一个 kernel，从而提高算术强度。手动融合后的 kernel 代码如下：

```
__global__ void fusedL2Norm(const float* __restrict__ x,
                            float* __restrict__ y,
                            int hidden) {
  extern __shared__ float sdata[];        // reduction buffer
  const int batch   = blockIdx.x;         // one block per batch row
  const int tid     = threadIdx.x;
  const float* batch_ptr = x + size_t(batch) * hidden;
  // 1) Accumulate sum of squares into shared memory
  float local = 0.f;
  for (int i = tid; i < hidden; i += blockDim.x) {
    float v = batch_ptr[i];
    local   = fmaf(v, v, local);          // v*v + local
  }
  sdata[tid] = local;


  __syncthreads();
  // 2) Parallel reduction to sdata[0]
  for (int offset = blockDim.x >> 1; offset > 0; offset >>= 1) {
    if (tid < offset) sdata[tid] += sdata[tid + offset];
    __syncthreads();
  }
  // 3) Normalize (guard tiny norms)
  float norm = sqrtf(sdata[0]);
  float inv = rsqrtf(sdata[0]); // prefer inverse
  float* out_batch = y + size_t(batch) * hidden;
  for (int i = tid; i < hidden; i += blockDim.x) {
    // multiply by inverse (rsqrt) vs. divide by sqrt
    out_batch[i] = batch_ptr[i] * inv;
  }
}
```

在这个融合 kernel 中，每个线程要遍历自己负责的 `x[b,*]` 分片两次——一次用于累加局部平方和，一次用于写出归一化后的输出——因此全局内存流量约为每元素 12 字节（两次读 + 一次写）。每个元素在归约阶段执行约 1 次乘法 + 1 次加法，在归一化阶段执行 1 次乘法。

`sqrt` 与 `rsqrt` 的开销被摊销到了整行之上。就 Roofline 定位而言，保守估计的算术强度约为 3 FLOPs / 12 字节 $\approx$ 0.25 FLOPs/字节（外加每行一次 `sqrt` 带来的微小的 $1/(hidden \cdot 12)$ 贡献）。这样，我们就可以让每个线程处理多个元素来提高 ILP，从而隐藏 `sqrt` 和 `rsqrt` 的延迟。

> **注** 此外，如上面的代码所示，我们计算逆平方根（`rsqrtf`）然后做乘法，而不是直接做除法。这是一种常见的微优化——在热点内层循环中尤其如此。其思路是用高吞吐量的乘法指令流替代缓慢的除法指令流；同时，我们也用开销更低的 `rsqrtf` 近似替换了 `sqrtf`。之所以称其为微优化，是因为总体来看，这条流水线是内存受限而非计算受限的——不过把它点出来还是很有意思的。这里还有一处未展示的优化：让线程块内的单个线程执行一次 `rsqrtf/sqrtf`，再通过共享内存把标量结果广播给其他线程。这种做法对性能的提升更大。关于该优化的更多细节，请参见本书的 GitHub 仓库。

与在 HBM 上做中间写入的朴素三 kernel 流水线（求平方 $\rightarrow$ 归约 $\rightarrow$ 除法）相比，融合版本至少省去了一次全局写入/读取的往返，以及一次启动屏障。因此，其算术强度和运行时间都要好得多

——在实际中尤其如此，即便每元素的 FLOP/字节只有约 0.25。这要归功于延迟的节省和缓存局部性的改善。

在实践中，这个单一的融合 kernel 比一系列分离的 kernel 执行得更快，原因在于更高的算术强度（FLOPS/字节）、更好的缓存局部性，以及把三个独立 kernel 合并为一个后省下的启动开销。

融合不仅提高了算术强度、把 kernel 在 Roofline 上推向更靠近计算受限的一侧，还节省了内存带宽。在朴素的多 kernel 版本中，我们必须把中间结果写入全局内存，再在下一个 kernel 中读回；而在融合版本中，中间结果（例如 `ai*ai`）根本不需要离开线程的寄存器。

在这个代码示例中，加法可以直接利用这些寄存器来计算和值；随后 `sqrt` 再使用这个和——全程无需额外的全局内存流量。只有最终结果才写回全局内存。

这样一来，融合 kernel 从/向全局内存搬运 12 字节数据就能完成约 4 FLOPs 的计算；而朴素的非融合方案在计入中间内存搬运后，要装载和存储 36 字节才能完成 4 FLOPs。这意味着更少的 DRAM 流量和更低的延迟。

这个简单的例子展示了融合如何提高 kernel 的算术强度和整体性能。接下来我们来看另一种提升算术强度的方法：利用 GPU 的 Tensor Core 硬件单元。

> **注** 最先进的 GPU kernel 借助纵向融合（vertical fusion）与横向融合（horizontal fusion）来获得更高的算术强度：纵向融合把对同一数据的顺序操作合并起来，横向融合则把跨数据的并行操作合并起来。NVIDIA 的 CUTLASS 或 OpenAI 的 Triton（已集成到 PyTorch 的编译后端 TorchInductor 中）等库，能帮你利用 Tensor Core、TMEM、TMA 等机制非常高效地实现这些不同类型的融合 kernel。

9.4 结构化稀疏

在现代 GPU 上，2:4 结构化稀疏由 Sparse Tensor Core 和 cuSPARSELt 在硬件层面提供加速。2:4 的含义是：每 4 个连续的权重中恰好有 2 个是非零的。创建这种类型的稀疏有时被称为剪枝（pruning）。

把一半权重剪枝成 2:4 模式后，每次内存加载送来的非零值中，真正参与乘法的数量翻了一倍。换句话说，你不再取回那些实际为零的权重，也就不会把矩阵乘操作浪费在已知为零的数据上，如图 9-5 所示。

结构化稀疏（structured sparsity）是在模型训练完成之后应用的。此时会对模型进行剪枝，并针对推理进行优化。剪枝和格式转换由 cuSPARSELt 等软件栈和框架工具完成。请注意，Transformer Engine 能够加速受支持的稀疏执行，但不会在转换过程中强制施加稀疏性。

剪枝和格式转换由软件处理——通常通过 cuSPARSELt 和框架工具完成。在 PyTorch 中，使用 `to_sparse_semi_structured()` 把训练好的稠密模块转换为 2:4 稀疏格式，然后在 Sparse Tensor Core 上部署稀疏 GEMM。

模型完成转换后，它会调用运行在 Sparse Tensor Core 上的优化稀疏 GEMM kernel，而不是标准 kernel。对于许多推理负载，Sparse Tensor Core 的加速比最

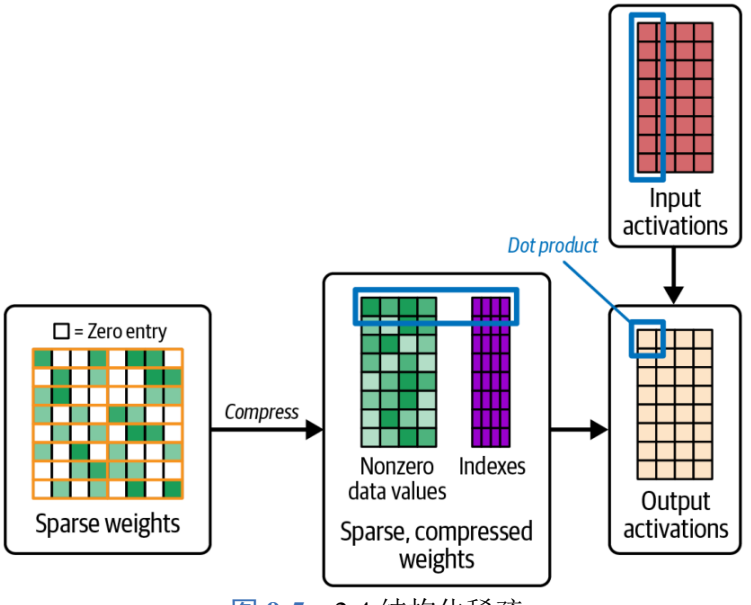

图 9-5　2:4 结构化稀疏

高可接近稠密版本的 2 倍——尤其是在提交大批量输入时，因为大批量可以摊销 kernel 启动开销。

> **注**　批处理（batching）是提高算术强度非常常见且实用的手段。与其一次只处理一个数据项——并承担随之而来的全部内存 I/O 等开销——不如在单次执行中处理多个数据项，使内存访问（例如加载权重等）能摊销到多次计算上。

这使经过稀疏加速的矩阵乘法拥有足够的并行工作量，从而隐藏处理索引或压缩表示所产生的任何开销。在较小批量的情况下，这些开销可能占据主导，并限制你观察到的加速比。

当使用大型矩阵乘法时，2:4 稀疏性带来的收益最大，这在以 LLM 为代表的基于 Transformer 的模型中十分常见。原因在于硬件能够充分发挥专用 Sparse Tensor Core 的能力。这些 Sparse Tensor Core 直接在硬件中对半宽数据进行操作：跳过零值，并在相同的周期预算内对非零元素完成两倍的工作。

由于 Blackwell 的计算能力增长快于 HBM 带宽，结构化稀疏是保持计算受限状态的一个极好手段。即使在 Grace Blackwell 系统上——借助 NVLink-C2C，

GPU 能以极高的吞吐量从 CPU 内存流式读取数据——你仍应最大化每个已加载 tile 的每字节 FLOPS。

例如，按 2:4 模式剪枝 50% 的权重，可以确保一半的内存流量完全不必发生。这会立即减少全局内存读取，并将有效算术强度提高近 2 倍。

NVIDIA GPU 在硬件中实现了这种 2:4 结构化稀疏：每个 16 元素的块可以将其中 8 个元素置零。正是这一模式使稀疏矩阵的 Tensor Core 吞吐量翻倍。截至撰写本书时，尚无其他任意稀疏性模式能在硬件中获得这种特殊加速。

> **注** 稀疏性带来的加速前提是模型精度得到保持。实践中，剪枝之后进行微调或仔细校准非常重要，这样才能把精度损失降到最低。

在应用稀疏性之前，务必先实现前文介绍的基础优化：合并所有全局加载、利用分块（tiling）复用数据、融合逐点运算以消除多余的内存往返。在这些基础措施全部落实并验证之后，结构化稀疏才能为推理带来又一重加速。

> **注** 结构化稀疏通常适用于推理负载。在训练期间，梯度无法从 2:4 稀疏性中获益；此外，在梯度更新中维持稀疏性也十分复杂。因此建议把它用于部署场景，并已事先完成模型的预剪枝和校准。NVIDIA 的 2:4 稀疏 Tensor Core 特性主要用于推理；训练方面的支持有限，且因模型和框架而异。在依赖该特性之前，请先在软件栈中确认其支持情况。

9.5 重算与内存的权衡

另外，当内存带宽成为瓶颈时，与其存储或加载预先计算好的值（例如 x^2），不如按需重新计算它们。例如，在寄存器中反复计算 `x*x` 往往比从全局内存加载先前算好的 x^2 更快。对廉价表达式的持续重算可以提高算术强度，凡在内存紧张之处，这都是一项有用的技术。

许多 LLM 推理引擎利用这一技术来节省内存。它们不把大型激活张量存入 HBM、稍后再读回来，而是即时重算某些层和激活值。这类似于模型训练语境下的激活检查点（activation checkpointing）。

重算能提高有效的每字节 FLOPS，让大型模型装入更小的内存。此外，重算还能为更大的批量腾出内存，用少量额外的 FLOPS 换取内存流量的显著下降。

9.6 PyTorch 与算术强度

在 PyTorch 中，上述许多思路会被自动应用。如前所述，PyTorch compiler（将在第 13 章和第 14 章讨论）能够自动融合逐点运算链条，甚至能融合某些约简操作。它利用执行图级别的优化，让数据尽量留在 GPU 上并被尽可能多地复用。

由于底层使用了 cuDNN 和 cuBLAS 等优化库，PyTorch 在使用 `torch.matmul` 执行矩阵运算时，会替你完成分块并使用共享内存。此外，PyTorch 的 `scaled_dot_product_attention`（SPDA）会根据张量形状和数据类型，选择分发到 FlashAttention、memory-efficient 或 cuDNN 后端。例如，可以通过 `torch.nn.attention.sdpa_kernel(SDPBackend.FLASH_ATTENTION)` 来控制后端选择。作为关注性能的开发者，你应当了解这些优化，并知道如何验证它们何时真正生效。

需要注意的是，虽然 PyTorch 能够识别并编译大多数运算，但某些非标准运算或自定义 CUDA 运算可能无法被融合。在这些情况下，可能仍需手动优化，例如融合、分块等。

> **注** 编写 PyTorch 代码时，应优先选用执行多项计算的融合运算和优化库调用，而不是长串的单独 kernel 启动。在实践中，这意味着使用 `torch.nn.functional` 中的激活函数或 `torch.matmul` 这类高层运算，而不是在 Python 里手写许多小的逐点 kernel。这些库会为这类高层运算调用高效的 kernel，并且编译器也知道如何把它们与周边运算高效地融合起来。

PyTorch 的嵌套张量（nested tensor，也称不规则张量，ragged tensor）让你无需填充即可表示由变长输入组成的批次。每个嵌套张量把变长序列打包进单一的高效底层缓冲区。NestedTensor 暴露常规的张量接口，但消除了不必要的零填充。这样一来，全局内存加载变得更高效，因为读取到的每个字节都对计算有用。

嵌套张量对具有变长序列的 LLM 很有用。使用嵌套张量时，注意力、批量矩阵乘法等运算只从内存取回必要的数据。这使你的 kernel 在 Roofline 模型上更接近计算受限区间，有助于减少内存受限导致的停顿。其结果是更高的持续吞吐量——尤其是在内存敏感的负载上。

> **注** 实践中，嵌套张量需要针对算子覆盖范围和性能特性进行仔细验证。其支持情况取决于具体负载，加速比则取决于张量形状。你可以用具有代表性的序列长度分布和注意力模式来验证端到端的收益。请同时对内存流量和 kernel 时间进行性能分析。

简而言之，PyTorch 提供了多种提高 kernel 算术强度的机制。理解这些选项并为你的负载选出最合适的方案很重要。在现代 GPU 上提高算术强度的另一种有效方法是使用降精度的 Tensor Core。接下来我们逐一介绍这些机制。

9.7 混合精度与利用 Tensor Core

现代 NVIDIA GPU 在 Tensor Core 中实现了 TF32、FP16、FP8、FP4、INT8 等降精度计算。Blackwell 的每个 SM 都有一个 256 KB 的片上 TMEM，专用于存放 Tensor Core 数据。此外，它还有一个专用的 TMA 单元，可在全局内存与共享内存之间异步拷贝 tile。随后，Tensor Core 指令（例如 `tcgen05.mma`）在共享内存与 TMEM 之间隐式搬运操作数和累加器。这一设计以高吞吐量向 Tensor Core 供数，并将停顿降到最少。与直接在寄存器中累加的前几代 GPU 相比，Blackwell 基于 TMEM 的累加器有助于降低寄存器压力。

使用得当时，这些特性能把一个原本内存受限、张量繁重的 kernel 转变为完全计算受限的 kernel，途径是将算术强度（每字节 FLOPS）提高 2 倍、4 倍甚至 8 倍。你可以通过查看 Nsight Compute 中的 Roofline 图和 Stall Stats 来验证其影响。

Nsight Compute 的 Speed of Light 分析会显示“Memory Throttle”之类的内存受限停顿原因以及缓存缺失，这些指标在使用

低精度格式的 Tensor Core 后会显著下降。此外，Nsight Compute 集成了 Roofline 图，可以交叉核对算术强度是否已经提升、把你的 kernel 推向计算屋脊。

当你从 FP32 迁移到 TF32、FP16、FP8 或 FP4 等降精度 Tensor Core kernel 时，Nsight Compute 的 Warp Stall 指标通常显示与内存相关的停顿减少，而依赖或流水线停顿相对增加。这表明算术强度得到提升，执行状态正从内存受限转向计算受限。

9.7.1 用 TMEM 和 TMA 为 Tensor Core 供数

高吞吐量张量计算的核心是 TMEM——每个 SM 上的一个 256 KB SRAM 缓冲区。不过，从宏观层面看，程序员并不显式地分配或管理 TMEM。当你使用 Tensor Core 运算时，TMEM 由硬件或库来处理。TMEM 如图 9-6 所示。

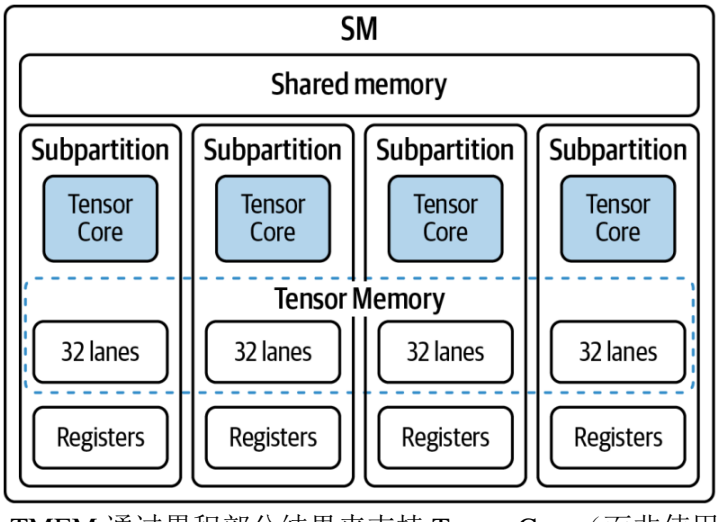

图 9-6　TMEM 通过累积部分结果来支持 Tensor Core（而非使用寄存器）

在底层，Blackwell 使用 `tcgen05.mma` 指令，并借助 TMEM 存放操作数与累加器。CUTLASS 和各类库的 kernel 通过 kernel 配置以及并行线程执行（PTX）汇编来管理所需的分配与使用。因此，Transformer Engine 使用 TMEM 保存中间结果，这降低了 MMA 对寄存器的依赖。

诸如 CUTLASS 这样的高层 API 会自动替你处理上述全部复杂性。应尽量使用 CUTLASS 及其他高层库，因为 CUTLASS 使用实现 Tensor Core 矩阵运算

以及内存加载/存储接口的 `tcgen05.*` PTX 指令。无论何时通过 CUDA MMA 内建函数或 CUTLASS GEMM 启动 Tensor Core MMA 运算，其实现都会经由共享内存和 TMEM 管理操作数的搬运：TMA 在全局内存与共享内存之间流式传输分块（tile），而 Tensor Core 指令则在共享内存与 TMEM 之间隐式移动操作数。

> **注**　Nsight Compute 内置了 roofline 与 speed-of-light 分析，可用于确认 kernel 在采用低精度 Tensor Core 路径之后，是否已从内存受限转向计算受限。

随后，Tensor Core 指令会在执行 MMA 流水线的过程中于共享内存与 TMEM 之间搬运操作数。这一切都在幕后自动完成，无需用户编写显式代码，从而使数据总是被暂存在 Tensor Core 需要的位置。要把数据从全局内存转移进共享内存，可以使用 TMA，也可以配合 CUDA Pipeline API（`<cuda/pipeline>`）使用 `cuda::memcpy_async` 来构建流水线。在代码中，使用 `cuda::memcpy_async` 和 CUDA Pipeline API 实现一个简单的两级流水线如下所示：

```
# two_stage_pipeline.cu
#include <cuda/pipeline>
#include <cooperative_groups.h>
namespace cg = cooperative_groups;
extern "C" __global__
void stage_ab_tiles(const float* __restrict__ globalA,
                    const float* __restrict__ globalB,
                    float* __restrict__ outC,
                    int tile_elems,
                    int num_tiles) {
    // Alignment / size guards for vectorized copies (runtime parameter)
    assert((tile_elems % (32 * 4)) == 0 &&
           "tile_elems must be multiple of 128 for float4
            ↪ vectorization");
    // If you cannot guarantee 16B alignment or sizes, handle
    // the tail/ragged edges with a fallback 4B loop.
  extern __shared__ float smem[];
  auto block = cg::this_thread_block();
  // Shared buffers for double buffering of A and B
  float* A0 = smem + 0 * tile_elems;
  float* A1 = smem + 1 * tile_elems;
  float* B0 = smem + 2 * tile_elems;
  float* B1 = smem + 3 * tile_elems;
  constexpr auto scope = cuda::thread_scope_block;
  constexpr int stages = 2;
  __shared__ cuda::pipeline_shared_state<scope, stages> pstate;
```

```
auto pipe = cuda::make_pipeline(block, &pstate);
// Prime the pipeline with tile 0
pipe.producer_acquire();
cuda::memcpy_async(block, A0, globalA + 0 * tile_elems,
                   cuda::aligned_size_t<32>{tile_elems *
                    ↪ sizeof(float)}, pipe);
cuda::memcpy_async(block, B0, globalB + 0 * tile_elems,
                   cuda::aligned_size_t<32>{tile_elems *
                    ↪ sizeof(float)}, pipe);
pipe.producer_commit();
for (int t = 1; t < num_tiles; ++t) {
  // Stage the next A and B tiles
  pipe.producer_acquire();
  cuda::memcpy_async(block, (t & 1) ? A1 : A0,
                     globalA + t * tile_elems,
                   cuda::aligned_size_t<32>{tile_elems*sizeof(float)},
                     ↪ pipe);
  cuda::memcpy_async(block, (t & 1) ? B1 : B0,
                     globalB + t * tile_elems,
                   cuda::aligned_size_t<32>{tile_elems*sizeof(float)},
                     ↪ pipe);
  pipe.producer_commit();
  // Consume the previously staged tiles
  pipe.consumer_wait();
  float* prevA = (t & 1) ? A0 : A1;
  float* prevB = (t & 1) ? B0 : B1;
  // Perform compute using prevA and prevB
  pipe.consumer_release();
}
// Consume the final staged tiles
pipe.consumer_wait();
int last = (num_tiles - 1) & 1;
float* lastA = last ? A1 : A0;
float* lastB = last ? B1 : B0;
// Perform compute using lastA and lastB
```

```
  pipe.consumer_release();
}
```

启动该 kernel 时，需将动态共享内存大小设为 4 × `tile_elems` × `sizeof(float)`，以便在共享内存中分配 A0、A1、B0 和 B1。这种双缓冲模式确保一旦某个 tile 驻留在共享内存中，Tensor Core 就能立刻开始处理它，与此同时 `cuda::memcpy_async` 会并行地将下一个 tile 取入共享内存。由于 TMEM 为 Tensor Core 指令提供了片上数据缓冲，而共享内存提供了暂存空间，因此 FP16、FP8 或 FP4 tile 的暂存与复用可以完全在片上完成。在流水线经过调优、且 tile 与拷贝大小设置得当的情况下，其结果就是更少的停顿。`cuda::memcpy_async` 能够将 HBM 到共享内存的传输与计算重叠起来，使 kernel 持续保持忙碌，这有助于把内存延迟隐藏在计算背后。

9.7.2 TF32 与自动混合精度（PyTorch）

尽管 Tensor Core 最初是为 FP16 设计的，但它们也支持介于 FP32 与 FP16 之间的 TF32。TF32 采用与 FP32 相同的 8 位指数，以及与 FP16 相同的 10 位尾数。TF32 在 Tensor Core 上的执行吞吐量大幅高于 FP32 在 CUDA core 上的吞吐量，同时保留了 FP32 的指数范围。在 PyTorch 中，启用 TF32 只需在代码中设置如下内容：

```
import torch
torch.set_float32_matmul_precision('high') #
↪ {'highest'|'high'|'medium'}
```

一旦设置了这些标志，`torch.matmul` 和 `torch.nn.Linear` 等高层运算就会自动以 TF32 Tensor Core kernel 的方式执行，而不是以 FP32 在普通 CUDA core 上执行。

除 TF32 之外，PyTorch 的自动混合精度（AMP）可以为每个运算选择最佳精度（FP16 或 BF16），并将结果以 FP32 累加以保证稳定性。BF16 有助于避免 FP16 的溢出问题。默认情况下，CUDA autocast 使用 float-16，只需传入 `dtype=torch.bfloat16` 即可在支持该格式的 GPU 上改用 BF16。例如，可以把模型代码包在一个上下文管理器中，如下所示：

```
with torch.amp.autocast("cuda", dtype=torch.bfloat16):
    output = model(input)
```

在底层，TorchInductor（见第 13 章和第 14 章）会自动融合这些精度转换，从而保证以下几点：大型 GEMM 运算以 FP16 或 TF32 在 Tensor Core 上运行；累加保持为 FP32 以确保数值稳定性；层归一化与 softmax 等小型“敏感”kernel 以 FP32 运行；并且 GradScaler 能在使用 FP16 训练时防止下溢。注意，BF16 具有 FP32 的指数范围，因此使用 BF16 训练时通常不需要 GradScaler。

在 PyTorch 中，这些混合精度决策已被集成进编译器，因此无需人工干预即可获得最优的 dtype 选择（例如计算用 FP16/FP8、累加用 FP32）。图 9-7 展示了这一点，即一次混合精度的矩阵乘累加（MMA）。

这条自动混合精度流水线以极少量的代码改动最大化算术强度。融合后的 Tensor Core kernel 通过在共享内存（如操作数）与 TMEM（如累加器）中暂存并复用数据，最大限度地减少了对 HBM 的往返访问。

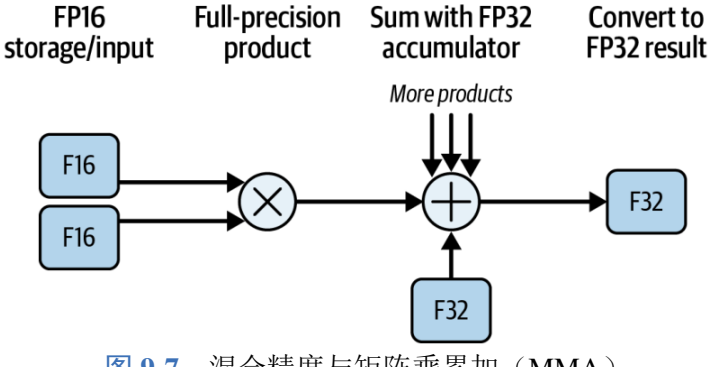

图 9-7　混合精度与矩阵乘累加（MMA）

在使用前文介绍的结构化稀疏或极低精度（FP8/FP4）时，务必保持足够大的批大小或 tile 粒度，使 TMEM 与 Tensor Core 始终得到充分利用。小批量会带来额外开销，包括格式转换、稀疏索引处理、不规则的内存访问模式等，这可能降低实际获得的加速比。

例如，在使用 FP8 或 2:4 稀疏时，批大小为 1 可能收益甚微，因为固定开销无法被摊销；相比之下，128 或 256 的批大小可以充分利用 TMEM 流水线，带来接近峰值的吞吐量。

9.7.3 BF16/FP16、FP8 与 FP4 低精度

BF16/FP16（半精度）已获得多代 GPU 的支持，而现代 GPU 上的 Tensor Core 通常能持续达到 BF16/FP16 峰值吞吐量的 90% 以上，约为 FP32 峰值吞吐量的 4 倍。这是因为硬件在每个周期内并行发射大量 BF16/FP16 FMA 运算。

FP16 训练使用的指数位比 FP32 更窄（仅 5 位），因此除非应用损失缩放（loss scaling），否则非常小的梯度值可能下溢为零。损失缩放可以在反向传播过程中保持数值稳定性，这种缩放既可以是静态的，也可以是动态的。

相比之下，BF16 与 FP32 的 8 位指数范围相同，天然避免了下溢，因而极少（甚至从不）需要损失缩放。这简化了混合精度工作流程，并且在现代 GPU 上往往能提升训练精度。

> **注** 在现代 GPU 上进行训练时通常首选 BF16，因为它无需 FP16 所要求的损失缩放等复杂性，即可保持与 FP32 相当的精度。

若要进一步提升吞吐量，可以使用 FP8。将 16 位权重缩减 50% 降到 8 位，可以把内存流量减半，并将每次 HBM 事务加载的权重数量翻倍。在实际应用中，采用 FP32 或 TF32 累加的 FP8 矩阵乘法可以达到 BF16/FP16 吞吐量的 2–3 倍（TFLOPS）——前提是量化误差带来的轻微精度损失仍可接受。

为了解决极低精度下的精度问题，Transformer Engine 除支持 FP8 外，还支持 NVIDIA 采用微缩放（micro-scaling）的 4 位 NVFP4 格式。NVFP4 采用两级缩放，将逐微块（microblock）缩放与一个更高层级的缩放相结合，使模型在使用 4 位存储权重的同时仍能保持精度。此外，Blackwell B200 的 NVFP4 激进微缩放量化可提供 10 PFLOPS（稠密）算力，而 FP32 峰值约为 80 TFLOPS（稠密），即每权重的理论吞吐量提升约两个数量级。Blackwell B300（Ultra）的 NVFP4 计算能力则比 B200 高出 50%，达到 15 PFLOPS（稠密）。

如果模型在校准之后能够容忍精度下降，NVFP4 kernel 在支持的硬件上可以提供远高于 FP32 的吞吐量，但精度必须针对每个模型逐一验证。

由于精度极低，每个 SM 的 256 KB TMEM 可以容纳较大的 FP4 tile（例如 256×256），从而进一步提高片上复用并改善性能。注意，所有从低精度格式到累加精度的转换都会自动完成：kernel 从 HBM 读取 FP4 输入，Tensor Core 执行 FP4 $\times$ FP4 乘法，MMA API 则将结果累加到 BF16/FP16 或 FP32 累加器中。

精度每下降一级，每字节可执行的运算次数就会翻倍或变为四倍，从而提升算术强度。当 TMEM/TMA 使内存与计算重叠时，这些低精度格式能够把原本内存受限的 kernel 转变为完全计算受限的 kernel，从而充分调动现代 GPU 上每卡数 PFLOPS 级别的 Tensor Core 引擎。

9.7.4 面向推理的 INT8 低精度与 DP4A 指令

LLM 推理场景通常可以容忍低精度的 INT8 量化，现代 GPU 对此的支持方式是：在普通 CUDA core 上使用 DP4A（SIMD 点积）指令，在 Tensor Core 上使用整数矩阵乘累加（MMA）指令。在指令层面，DP4A 每条指令执行 4 个 INT8 乘累加（MAC）运算，而每条指令仅执行 1 个 FP32 融合乘加（FMA）。

由于 INT8 的权重流量仅为每元素 1 字节，而非 FP32 的 4 字节，权重带来的内存流量下降 75%。得益于更高的 INT8 Tensor Core 峰值吞吐量以及更少的内存流量，INT8 推理工作负载的性能可以显著超过 FP32：在使用 INT8 权重时，每块 GPU 每秒

能从内存中处理大约 4 倍的数据量。这得益于 TMEM 和 TMA 让数据与计算保持完美重叠，并以尽可能高的效率向 Tensor Core 持续供给数据。

9.7.5 深入剖析 Transformer Engine 与 TMEM

现代 NVIDIA GPU 包含一个 Transformer Engine，它将 Tensor Core 对低精度格式的硬件支持与负责缩放和类型转换的软件运行时结合在一起。cuBLASLt、cuDNN、CUTLASS 或 OpenAI Triton 中的 kernel 会执行 `cp.async` 指令，或通过 TMA 把数据传输进共享内存。随后，Tensor Core 指令在共享内存与 TMEM 之间隐式地移动操作数。

请记住，TMEM 是每个 SM 256 KB 的 SRAM 缓冲区，供 Transformer Engine 和 Tensor Core 用来存放结果（而不是用寄存器）。在实践中，你从不需要显式分配 TMEM，这全部由硬件处理。例如，在调用 Tensor Core 的 MMA 操作时，硬件会负责所有的内存分配与数据传输。

借助这些 MMA 指令，每个 warp 直接驱动 Tensor Core 执行高吞吐量的混合精度 MMA 操作。这些操作会管理 fragment 加载、寄存器映射与混合精度 MMA 运算。

> **注** 截至撰写本文时，PyTorch 的 INT8 量化支持通过 TorchAO 和各厂商后端提供。量化模块使用专用的 INT8 kernel 运行。若使用 cuBLASLt 或 CUTLASS 执行底层的 INT8 GEMM，可以确保 Tensor Core 得到充分利用。

每当你启动一个基于 Tensor Core 的 kernel 或 GEMM 库函数（例如 CUTLASS）时，其实现都会自动通过共享内存和 TMEM 管理操作数的搬运，使 Tensor Core 始终装满随时待处理的 tile。（注意，应用程序代码并不直接分配 TMEM。）

Transformer Engine 的工作流程十分直观。首先，你的 kernel 发出一次 MMA 调用，或启动一个 CUTLASS GEMM。随后，Transformer Engine 的固件安排 TMA（或 `cuda::memcpy_async`）把权重和激活值从 HBM 复制到共享内存（SMEM）。接着，Tensor Core 指令（例如 `tcgen05.mma`）在 MMA 流水线执行期间于 SMEM 与 TMEM 之间隐式移动操作数。理想情况下，权重采用 FP8 或 FP4 格式，激活值在可行时则被转换为 FP8/FP4——否则，激活值可以保留 FP16/FP32 格式。

Tensor Core 的 MMA 操作以低精度执行，例如 FP8 × FP8 并以更高精度累加，或 FP16 × FP16 并以 FP32 累加。部分和以更高精度（例如 BF16、FP16、FP32）在 TMEM 中累加，至于采用哪种精度

则由 kernel 决定。累加器状态驻留在 TMEM 中，通过 tcgen05 加载和存储接口访问。硬件对这些数据移动进行透明管理。

如果你自行构建 tile 循环，就可以把数据搬运与 Tensor Core 计算重叠起来。为此可以使用 `cuda::memcpy_async` 和 CUDA Pipeline API，如下面的代码所示：

```
#include <cuda/pipeline>
#include <cooperative_groups.h>
namespace cg = cooperative_groups;
extern "C" __global__
void double_buffer_a(const float* __restrict__ globalA,
                     int tile_elems,
                     int numTiles) {
  __shared__ float tileA0[TILE][TILE];
  __shared__ float tileA1[TILE][TILE];
  auto block = cg::this_thread_block();
  constexpr auto scope = cuda::thread_scope_block;
  constexpr int stages = 2;
  __shared__ cuda::pipeline_shared_state<scope, stages> pstate;
  auto pipe = cuda::make_pipeline(block, &pstate);
  // Prime pipeline with tile 0
  pipe.producer_acquire();
  cuda::memcpy_async(block,
                     &tileA0[0][0],
                     globalA + 0 * tile_elems,
                    cuda::aligned_size_t<32>{tile_elems * sizeof(float)}
                     pipe);
  pipe.producer_commit();
  for (int t = 1; t < numTiles; ++t) {
    // Stage next tile into the alternate buffer
    pipe.producer_acquire();
    float* nxtA = (t & 1) ? &tileA1[0][0] : &tileA0[0][0];
    cuda::memcpy_async(block,
                       nxtA,
                       globalA + t * tile_elems,
                       cuda::aligned_size_t<32>{tile_elems *
                        ↪ sizeof(float)}
                       pipe);
    pipe.producer_commit();
    // Consume the previously staged tile
    pipe.consumer_wait();
    float* curA = ((t - 1) & 1) ? &tileA1[0][0] : &tileA0[0][0];
```

```
    pipe.consumer_release();
  }
  // Consume the final staged tile


  pipe.consumer_wait();
  float* lastA = ((numTiles - 1) & 1) ? &tileA1[0][0] : &tileA0[0][0];
  // Use lastA with your compute
  pipe.consumer_release();
}
```

由于 TMEM 是 Tensor Core 指令专用的片上缓冲区，数据因此始终贴近计算单元。当 Tensor Core 在处理当前 tile 时，`cuda::memcpy_async` 正在把下一个 tile 从 HBM 流式传输到共享内存。

这种重叠有助于隐藏内存延迟，并能在流水线调优得当时让 Tensor Core 保持忙碌。Transformer Engine、TMEM 与 TMA 三者的协同可以显著提升算术强度，在优化良好的情形下接近光速极限效率。

> **注** 虽然加载和存储操作相对于发起调用的 warp 是同步的，但计算与数据搬运的重叠应当来自 CUDA Pipeline API。与 `wait`（等待）和 `release`（释放）等流水线原语配合使用时，`cuda::memcpy_async` 会映射到 Tensor Memory Accelerator（TMA），因此对于批量张量传输应始终优先选用它。`cp.async` 只留给 TMA 无法表达的少数特殊场景使用——不过这种情况很少见。此外，务必确认拷贝已经完成后，才使用这些数据。

9.8 使用 CUTLASS 实现最优算术强度与 Tensor Core 性能

自己利用这些优化，最简单的方式之一就是使用 NVIDIA 的 CUTLASS 库。你只需编写一个模板化调用，CUTLASS 就会自动应用许多高级优化。

CUTLASS 应用的优化包括共享内存分块、异步内存传输，以及借助 TMEM 每个 SM 256 KB 缓冲区实现的双缓冲。这样，你的 Tensor Core 无需任何手动 kernel 调优，就能以接近峰值的吞吐量运行。

> **注** CUTLASS 还实现了 warp 特化（warp specialization），这是一种高性能 GPU 优化技术，我们将在下一章讨论。

举例来说，假设你想计算一个 GEMM：$C = A \times B$，输入为半精度，输出为半精度，并视情况以 FP16 或 FP32 累加。你不必手写一个手工调优的 MMA 循环，只需包含 CUTLASS 并实例化一个模板即可，如下面的代码所示：

```
#include <cutlass/numeric_types.h>
#include <cutlass/gemm/device/gemm.h>
using Gemm = cutlass::gemm::device::Gemm<
  cutlass::half_t,  // A (FP16)
  cutlass::layout::RowMajor,
  cutlass::half_t,  // B (FP16)
  cutlass::layout::ColumnMajor,
  cutlass::half_t,  // C / output (FP16)
  cutlass::layout::RowMajor,
  float, // accumulator (FP32 accumulate)
  cutlass::arch::OpClassTensorOp,
  cutlass::arch::Sm100 // e.g., Blackwell B200
>;
// ... (allocate device pointers A_d, B_d, C_d,
// set up dimensions M,N,K, and strides lda, ldb, ldc) ...
Gemm gemm_op;
cutlass::Status status = gemm_op(
    { M, N, K },              // GEMM shape
    float(1.0f)               // alpha
    A_d, lda,                 // A pointer + leading dimension
    B_d, ldb,                 // B pointer + leading dimension
    float(0.0f),              // beta
    C_d, ldc                  // C pointer + leading dimension
);
```

编译并运行这段代码后，CUTLASS 会自动执行几件关键工作。首先，CUTLASS 会选取 tile，以平衡寄存器压力、共享内存容量与 Tensor Core 利用率。在现代 GPU 上，TMEM 与共享内存、L1 并存。CUTLASS 把 tile 先暂存到共享内

存，并使用与 TMEM 交互的 Tensor Core 指令来存放累加器数据。tile 形状是逐 kernel 凭经验选定的。例如，它可能选取 128 × 128 或 256 × 128 这类 tile 尺寸；这些尺寸能装进 TMEM 每个 SM 256 KB 的缓冲区，并在整个 Tensor Core 计算期间一直留在片上。

视精度而定，一个 256 × 512 的 tile 就会把每个 SM 256 KB 的 TMEM 预算用满，因为 256 × 512 个元素 × 每元素 2 字节 = 256 KiB；而 256 × 256 个元素 × 每元素 4 字节 = 256 KB。更大的 tile 能提高单个 tile 的吞吐量，但会减少每个 SM 上可并发运行的 tile 数量，这可能导致较小的 GEMM 利用不足。相反，非常小的 tile 则以牺牲算术强度来换取并行性。

CUTLASS 随后会发出异步内存拷贝（`cp.async` 或 TMA），把每个 tile 从 DRAM 流式传入共享内存。如图 9-8 所示，`cp.async` 指令在不占用每线程寄存器（或可选的 L1 缓存）的情况下，把数据从全局内存暂存到共享内存。缓存行为由 `cp.async` 修饰符控制；对于批量张量传输，则可改用 TMA。

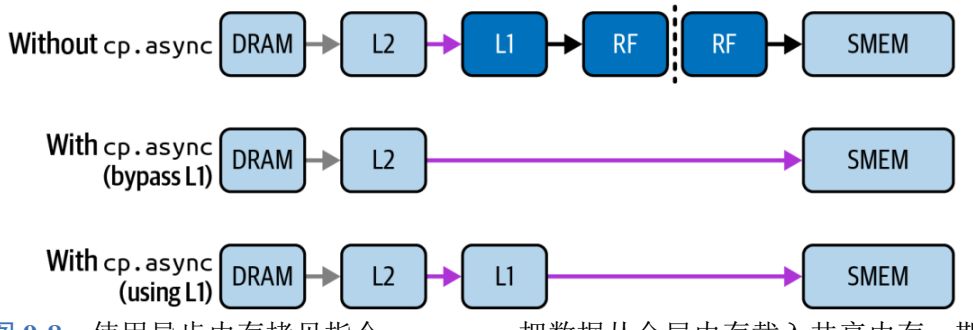

图 9-8 使用异步内存拷贝指令 `cp.async` 把数据从全局内存载入共享内存，期间不经过寄存器堆，（可选地）也不经过 L1 缓存

CUTLASS 使用 `cp.async` 或 TMA（`cp.async.bulk.tensor`）把 tile 从全局 DRAM 暂存进 SMEM。随后，Tensor Core 的 `tcgen05.mma` 指令从 SMEM 读取操作数，并把结果隐式地累加进 TMEM。这就在共享内存中形成了一个由软件管理的暂存区，用于实现双缓冲。这样一来，当 Tensor Core 正在处理当前 tile 时，TMA 已经在把下一个 tile 取入共享内存。

借助 CUDA Pipeline API 和按 warp 特化的计算阶段（将在下一章讨论），CUTLASS 让所有 Tensor Core 流水线保持满负载。它会按你指定的精度累加部

分和（例如输入为 FP16 或 FP8 时使用 FP32），以保证数值保真度——随后以合并访问的方式，把结果从 TMEM 写出到共享内存或全局内存。

> **注** 在有益时，CUTLASS 还会利用线程块集群（thread block cluster），跨多个 SM 进行分块，以获得更大的有效 tile。我们将在下一章讨论线程块集群。

由于所有这些复杂性都被隐藏起来，CUTLASS 为你提供了一个拿来即用的高性能 GEMM kernel，其效果堪比手工调优的 MMA kernel——与手写版本相比，整体的 Tensor Core 利用率和性能通常只差几个百分点，如表 9-1 所示。

表 9-1 手工调优 MMA kernel 与 CUTLASS kernel 的性能与资源用量对比。

指标	手工调优 MMA kernel	CUTLASS GEMM
Tensor Core 占用率	98%	98%
每线程寄存器数	~52	~60（略高）
每线程块（CTA）共享内存	~2 KB	~4 KB
开发工作量	高	低（简单的模板配置）

% （本段为扫描提取的表格内文，内容已完整并入上方表 9-1 的表格浮动体。）

> **注** 所有指标表中的数值仅为解释概念的示意值。不同 GPU 架构上的真实基准测试结果，请参见 GitHub 仓库。

这里，两者都使用 FP16 输入并以 FP32 累加，且都以最大化 Tensor Core 利用率为目标。如表 9-1 所示，CUTLASS 的性能达到甚至超过手工调优的 MMA，差距约在 2% 以内。尽管在本例中 CUTLASS 多用了几个寄存器，并把共享内存用量翻了一倍，但仍远在硬件限制之内，这些微小的增加不会影响占用率。

> **注** 寄存器与共享内存用量的这些细微差异，源于 CUTLASS 出于通用性考虑而对 kernel 做了泛化处理。虽然通过手工调优可以把这些开销优化掉，但在大多数情况下，为这点收益投入额外复杂度很可能并不值得——CUTLASS 的性能与手工调优方案几乎完全相同。

它只需几行模板代码，而免去了数周的低层调优。此外，CUTLASS 模板已经支持 FP4、FP8、FP16 和 TF32 操作数类型，还能把偏置加法（bias-add）与激活等常见的后处理操作融合到同一个 kernel 中。

> **注** 还要记住，如前所述，CUTLASS 模板会透明地使用线程块对、多 SM 分块，以及结合分布式共享内存（DSMEM）的 TMA 多播来最大化数据复用——下一章将对此做更详细的介绍。

与之相对，编写自定义 MMA kernel 则需要手动选择分块大小、编写异步拷贝循环、管理双缓冲、实现 warp 专用化流水线以及线程块集群（thread block cluster）分块——而这一切 CUTLASS 都会自动替你完成。

cuBLAS 等优化库正是基于 CUTLASS 构建的，而 PyTorch 这类高层库的许多 kernel 也会调用这些优化库。在前面的 fused-attention 示例中，我们展示过 TorchInductor 为其分发了一个基于 CUTLASS 的

fused-attention kernel，它使用的恰好是同一套双缓冲 TMEM 流水线。由此实现了 98% 的 Tensor Core 利用率，内存停顿则几乎为零。

> **注** 随着 PyTorch 及其他高层库中有越来越多的算子在底层采用 CUTLASS，你无需亲手编写任何 CUDA C++ 代码，就能坐享这些优化。

当然，仍可能存在需要手动编写 MMA kernel 的场景——例如你需要高度专门化的数据布局，或是 CUTLASS 尚不支持的独特融合模式。

在这种情况下，这些复杂性都得由你自己来实现。你首先要选出一个能装进 TMEM 的分块大小（例如 128×128 的 FP16），然后用 `<cuda/pipeline>` 对每个分块执行异步内存拷贝（`cp.async`）指令。

接着，你需要实现 warp 专用化的 MMA 循环，并为 TMEM 建立双缓冲以隐藏 DRAM 延迟。最后，还要把 softmax、逐元素非线性等自定义后处理步骤交织进来——如果可能的话，全部安排在同一个循环内。

不过，对于几乎所有标准的 GEMM 或 fused-attention 用例，CUTLASS 以及基于它构建的库才是推荐的做法。

它基于模板的设计、面向特定 GPU 的调优，以及对 TMEM 与 TMA 流水线的内建支持，通常能在受支持的形状上取得很高的 Tensor Core 利用率。这让你

在某些情况下只需极少的开发工作量，就能达到 96%–98% 的 Tensor Core 利用率。

依托 CUTLASS 的自动优化，你可以把时间花在模型架构、数值精度策略和端到端性能上。CUTLASS 让你确信：底层张量运算会以接近峰值的算术强度运行，并针对你的特定 GPU 硬件做过优化。

> **注** NVIDIA 会持续更新 CUTLASS 与 cuBLAS 等库，以充分利用 FP8、FP4、线程块集群对、TMEM 等最新硬件特性。使用这些库，你就不必为每一代新 GPU 重写新的 kernel。切换到新的 GPU 架构时，务必检查是否有新版本的 CUTLASS。

9.9 用于微观优化的内联 PTX 与 SASS 调优

对于愿意走出 C++、深入底层微观优化的开发者，CUDA 允许使用内联的并行线程执行（PTX）代码和 SASS（NVIDIA 的汇编语言），把可能被白白丢掉的最后一丝性能也挖掘出来。

这确实属于进阶领域，因为 CUDA 编译器的优化能力已经相当出色。但在某些极端情形下，你可以手动调度汇编指令，或使用专用指令，在非常特定的场景中换取小幅的性能提升。

借助 PTX 和流式汇编器（SASS），你还能启用高层 CUDA 语言尚未暴露的特性。现代 GPU 一般不会引入颠覆性的新汇编指令，但它们确实提供了定制调优的机会。例如，你可以调整 GPU 的缓存策略、修改 CPU-GPU 统一内存访问的协调方式，并实现其他细粒度的微观优化。

> **注** PTX（读音近似“pee-tex”）是一个低层的并行线程执行虚拟机，它把 GPU 呈现为一台并行计算设备，并为 NVIDIA GPU 提供编程模型与指令。高层编译器（例如 CUDA C++）生成 PTX 指令，再由其翻译成目标架构的原生指令。SASS 则是真正在 NVIDIA GPU 硬件上原生执行的低层汇编语言。

举个例子：设想有一段代码，你明确知道某种特定的指令序列才是最优的，但编译器却没能生成这个序列。常见的情形包括：使用没有直接对应 CUDA in-

trinsic 的 GPU 指令；对特定访问施加内存加载修饰符（缓存提示）；在精确的位置插入内存栅栏或屏障；或者手动重排指令以避免流水线停顿。

另一类用途是读取 SM ID、warp lane ID 等特殊寄存器或状态，它们可能没有对应的高层 API。借助 `asm()` 语句，内联 PTX 让你能把汇编直接嵌入 CUDA C++ 代码：为这段汇编指定输入和输出，即可把 C++ 与 PTX 混合使用；编译器随后会把你的 PTX 指令并入最终生成的 SASS。

我们来看一个简单示例：用一条内联 PTX 指令把全局内存地址预取到 L2 缓存中。这里采用 kernel 侧预取，使用的 PTX 指令是 `cp.async.bulk.prefetch.global`：

```
__global__ void PrefetchExample(const float *in, float *out) {
    // ... assume idx is our thread's data index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    // Manually prefetch the next cache line (128B) of in[] into L2:
    // Prefetch 128B from global to L2.
    // Address must be 16B-aligned
    // and size is a 16B multiple.
    asm volatile("cp.async.bulk.prefetch.L2.global [%0], %1;"
                 :: "l"(in + idx + 32), "n"(128));
    float x = in[idx];
    // (do some work here before using in[idx+32] to give time for
    ↪ prefetch)
    out[idx] = x;
}
```

在这段代码中，内联 PTX 指令 `cp.async.bulk.prefetch.L2.global [%0]` 使用我们提供的地址操作数（`in + idx + 32` 字节处，即向前 32 个 float），向 L2 发起一次预取。我们把它标记为 `volatile`，以确保编译器不会将其优化掉。

这些 PTX 指令会被注入到机器码中。像这样使用内联汇编，能带来非常细粒度的控制。例如，我们可以预取到 L2 或 L1（通过使用 `.L1`），也可以自行选择预取距离（本例中是向前 32 个 float）。

这基本上就是 `__prefetch_async` 最终很可能编译成的形态。更一般地，我们可以用内联 PTX 来控制普通加载的缓存行为。例如，可以编写

`asm("ld.global.cg.f32 %0, [%1];": "=f"(val): "l"(ptr))`，以 `.cg`（“cache global”）修饰符加载一个 float。

在某些架构上，这意味着我们希望数据缓存在 L2 中，而绕过 L1 缓存。如果我们知道某个访问会不断颠簸（thrash）L1，并且更倾向于只使用 L2，这种手段就能派上用场。通常编译器的默认选择是缓存在 L1（`.ca`），但我们可以用 PTX 覆盖编译器的决定。

> **注** 在现代架构上做 L2 预取时，在可用的场合请使用 `cp.async.bulk.prefetch.tensor.L2`。这比使用未公开的内建函数更可取。无论如何，了解这一能力的存在总是有益的。

内联 PTX 另一个能派上用场的领域是指令调度。默认情况下，编译器会按它认为最优的顺序发射指令。但你也可能遇到某些情况，希望更有效地交错混合各条操作。

举例来说，假设你有两个相互独立的内存加载，随后是对这两个结果的两次使用。编译器可能按 load1、use1、load2、use2 的顺序发射；但更好的指令调度或许是先背靠背地执行 load1 和 load2，再去使用两个结果。这样可以让内存延迟相互重叠。

通过为这些加载编写内联 PTX，你可以强制它们尽早执行，然后再进行计算。这是前面讨论过的手动提高 ILP 的一种形式。实践中，编译器在这一环节通常已经做得不错，因为现代编译器会尝试用其他独立指令来填满加载延迟；但内联 PTX 和 SASS 汇编能给你确定性。

在共享 CPU 与 GPU 内存的现代 CPU-GPU 超级芯片（superchip）上，这样的细粒度控制可能很有用：比如你管理的负载中，GPU 会轮询一个由 CPU 更新的内存位置。此时，你可以使用合适的内存栅栏（例如 `membar.sys` 或 `__threadfence_system()`），并在加载与存储上施加所需的缓存操作符，以确保在预期范围内的一致性。这类控制是高层 CUDA 可能不会直接暴露的。

> **注** PTX 总体上具有前向兼容性；然而，SASS 汇编会随每一代 GPU 架构而变化。

你还可以用内联 PTX 来利用特殊寄存器。例如，虽然没有读取 SM ID（线程块当前所在的 SM）的 C++ intrinsic，但你可以通过 `asm("mov.u32 %0, %smid;": "=r"(smid))` 拿到 SM ID。这种灵活性对调试和工作划分都很有用。

例如，一些开发者曾在持久 kernel 中使用 `%smid`，让每个 SM 上只有一个线程块去执行某项特定工作。这实际上实现了手动的 SM 分区，而这超出了 CUDA C++ API 所能提供的范畴。

如果代码在算法层面已经得到充分优化，那么在大多数情况下，内联 PTX/SASS 带来的收益往往只是增量式的，量级在百分之几左右。例如，对一个内存受限的 kernel，你可以仔细展开并调度指令，以缩短从加载到使用的延迟空泡，并用 PTX 设立两条相互独立的加载流，从而获得大约 5%–10% 的加速。在这种情形下，编译器的做法可能更为保守。

在计算受限的场景中，你可以借助内联汇编改用更快的数学指令，替换掉更为精确的指令。为此 CUDA 提供了快速数学内建函数，例如 `__sinf()`。

在 C++ 内建函数可用之前，编写矩阵乘 kernel 的开发者有时会嵌入 PTX 指令来使用 Tensor Core。如今，针对这一用途已有更高级别的内建函数。但简而言之，汇编让你在得知某个硬件特性的第一时间就能用上它，而不必等待 CUDA 提供支持。

Nsight Compute 可以为汇编调优提供依据。通过查看“SASS throughput”指标和“Warp Stall Reasons”，你可能会发现大量停顿源于内存依赖。此时可以像前文所说的那样，尝试对加载指令重新排序。

改动之后，你应当希望看到“Stall Memory Dependency”占比下降，同时指令发射率有所提高，即每个周期执行更多指令。请注意，手工调整汇编非常耗费人力，而且会降低代码的可移植性。

通常只有在最热的内层循环、并且已经穷尽了更高层级的优化手段之后，这么做才值得。此外，GPU 代际一旦变化，就可能需要重新调优，并重新验证此前的假设是否仍然成立，包括内存延迟、缓存行为等。

为了说明潜在的微优化，考虑这样一个场景：某个 kernel 已经优化得相当充分，只剩一个瓶颈——一个执行整数下标计算和内存加载的紧凑循环。

你可以用内联 PTX，通过 `mad.wide.u32` 指令（`base + index * stride`）在一条指令内算出字节地址；随后以 `ld.global.cg` 发起加载，针对这种流式访问模式旁路 L1。其结果是循环使用的指令更少，并避免了 L1 逐出。在这个案例中，我们从该 kernel 里挤出约 7% 的加速，把耗时从 1.07 ms 降到 1.0 ms。表 9-2 概括了优化后的 CUDA C++ 与手工调度并附带缓存提示的手写 PTX 之间的假设性前后对比。

表 9-2 优化后的 CUDA C++ 与手工调度并附带缓存提示的手写 PTX 对比。

版本	warp 停顿（内存）	发射 IPC	kernel 耗时	加速比
优化后的 C++（编译器调度）	占周期的 35%	1.5	1.07 ms	1.0× 基准
手工调优的 PTX（手动调度与缓存提示）	占周期的 20%	1.6	1.00 ms	1.07×

由此可见，调优之后，kernel 的内存停顿因加载彼此重叠而减少，缓存提示也降低了一部分延迟。此外，我们提高了 ILP 和每周期指令数（IPC），使 kernel 的整体执行时间净改善 7%。

这些数字与在已经优化过的 kernel 上手工编写汇编的预期相符。某些情况下收益可能更大——例如当编译器做出了欠佳的选择，而你能够实施修正时。反过来，若编译器已经选定了最优实现，收益则基本为零。错误的汇编顺序也可能损害性能，因此必须对每一次改动进行实验和性能分析。

内联 PTX 和 SASS 汇编可以视为最后手段的优化工具：它以更高的复杂度为代价换取极致的控制力。建议只在迫不得已时使用这一最后手段——

即当 CUDA C++ 无法访问某些硬件特性或指令，或者你已经精确定位到一小段可改进编译器调度的代码时。典型例子包括自定义内存访问模式（缓存提示、预取）、细粒度同步或栅栏，以及在官方支持之前使用新指令。

在使用内联汇编时，用性能分析验证其效果尤为重要：你希望看到停顿原因或指令条数如预期般下降。此外，应当把这类代码隔离存放并配以详尽文档；面对新的 GPU 架构它很可能需要更新——尤其是使用 SASS 汇编时，它并不总是向前兼容。

> **注** 尽管 PTX 比 SASS 稳定，某些硬件变更仍可能出于性能原因而要求你更新内联 PTX。

内联 PTX/SASS 调优属于专家级的精雕细琢，用于进一步削减额外延迟或强制特定调度。它能带来温和的加速并实现某些定制行为，但应在穷尽其他所有高层优化之后再考虑。例如，你可能想为执行数百万次的关键循环手工编写汇编。至少，它也是精确了解硬件究竟如何执行你的代码的有效途径。

简言之，请克制地使用内联 PTX/SASS，并勤做性能分析。收益是真实的，但通常只是增量式的，而维护代价则高得多。对大多数用例而言，依赖 CUDA 内置的优化——或 CUB、Thrust 这类高度调优的库——多半已经足够。但心里有数总是好的：必要时，你可以下沉到汇编层，获得对 GPU 的完全控制。

9.10 DeepSeek 使用内联 PTX 优化内存分配

自定义 PTX 的一个著名案例来自 DeepSeek 的 DeepEP 专家并行库。该库使用了一条自行定制的 PTX 指令 `ld.global.nc.l1::no_allocate.l2::256b` 来优化全局内存访问：旁路 L1 缓存分配、保护关键数据，并利用 256 字节的 L2 缓存块。这非常适合将大型数据集直接流入 L2 缓存，而不干扰 L1 缓存中高频访问的内存操作。

这条指令并不属于 NVIDIA 官方 PTX ISA 规范，而是 DeepSeek 工程师以“文档之外”（out-of-doc）的方式摸索发现的，用于在 Hopper GPU 中受美国出口管制约束的 H800 变体上微调缓存行为。

我们来拆解 `ld.global.nc.l1::no_allocate.l2::256b` 这条 PTX 指令。前缀 `ld.global.nc` 发起一次非一致的（nc）全局内存加载（`ld.global`）；修饰符 `l1::no_allocate` 与 `l2::256b` 则指示硬件不要把数据分配进 L1 缓存（`l1::no_allocate`），而是每次取出 256 字节送入 L2（`l2::256b`）。

由于旁路了 L1，加载可以直接把大块数据流入 L2，而不会逐出常驻 L1 的高频使用数据。当你拥有必须留在 L1 以获得低延迟内存访问的热工作集时，这一点至关重要。

在实践中，专家并行的 all-to-all 通信 kernel 这类流式负载正受益于此，因为它们每次分发往往只读一遍大的连续缓冲区。如果这些加载经过 L1，就可能把 SM 仍在活跃使用的先前缓存行逐出。

该指令以 256 字节对齐的块直接取入 L2，减少了不必要的 L1 流量，有助于内存受限的通信与计算操作维持高吞吐量。

不过，以这种方式使用 PTX 存在一定风险：`ld.global.nc.l1::no_allocate.l2::256b` 并不保证在 GPU 各代之间保持稳定。因此，依赖它的代码在未来的架构上可能失效或产生错误结果。

DeepSeek 的 DeepEP 配置甚至提供了编译期开关 `DISABLE_AGGRESSIVE_PTX_INSTRS=1`，一旦出现兼容性问题即可禁用这些激进的指令。DeepEP 的定制 PTX 技巧固然能带来显著加速，但内联 PTX/SASS 仍应谨慎使用，并在每次升级到新的 GPU 架构时反复测试。

9.11 本章要点

你已经看到如何通过把工作从缓慢的全局内存转移到更快的片上资源与计算单元，来发现并消除 GPU kernel 的瓶颈。遵循“性能分析—诊断—优化—再分析”的循环，你就能把 kernel 从利用率不足或内存受限的状态，转变为计算饱和、高吞吐量的程序。以下技巧有助于充分挖掘 GPU 的全部算力：

利用分块与融合提升算术强度

要提高每传输字节的 FLOPS，可以用多级分块把数据暂存在共享内存和寄存器中。例如，把一个 32×32 子矩阵装入 SMEM，让每个元素在多次 FMA 中得到复用。将相邻 kernel 加以融合，通过融合逐元素操作使中间结果留在片上。借助 CUDA Pipeline API 的 `cuda::memcpy_async` 进行异步内存加载，实现软件预取，使数据搬运与计算重叠，从而隐藏 DRAM 延迟。

善用混合精度、Tensor Core 与 Transformer Engine

从 FP32 降到 TF32/BF16/FP16/FP8/FP4，可把权重数据搬运量削减 2× 到 8×，算术强度也随之提升。在 PyTorch 中，可以用 `torch.set_float32_matmul_precision('high')` 和 `torch.cuda.amp` 启

用混合精度。在现代 GPU 上，TMEM 与 TMA 引擎把数据块源源不断地送入 Tensor Core；现代 GPU 还提供 Transformer Engine，专门面向 AI 负载利用这些更低精度，从而进一步提升吞吐量。

利用 CUTLASS 构建高性能 GEMM 与融合 kernel

与其手动编写 MMA 循环，不如实例化 CUTLASS GEMM 模板，让它自动管理双缓冲、TMEM 暂存（staging）和 Tensor Core 流水线。这样得到的 kernel 性能与手工调优的 kernel 相差仅为百分之几。cuBLAS、TorchInductor 等高层框架本身就已经依赖 CUTLASS。只有当你需要非标准布局或独特的融合模式时，才需要进行定制。

针对 PyTorch 的最佳实践

优先使用 `torch.matmul`、融合注意力（fused attention）和嵌套张量（nested tensors）等内置张量操作，因为 PyTorch 通常会随时间推移从 CUDA 继承 Tensor Core 及其他硬件优化。如果你为 PyTorch 编写自定义 kernel 扩展，也应沿用同样的策略：尽量减少寄存器占用、对齐数据以便实现合并内存加载（coalesced memory loads），并面向 Tensor Core。这能确保你的 kernel 与原生 kernel 具有同等性能。

通过遵循“性能分析—诊断—实施针对性优化”的系统化工作流程——从占用率调优、warp 级别的精细打磨，到借助分块（tiling）、融合（fusion）和 Tensor Core 提升 ILP 与高算术强度——你可以将内存受限的 GPU kernel 转变为计算受限的 kernel，从而获得大幅加速；在强内存受限的工作负载中，加速有时甚至可达数量级。

9.12 小结

在本章中，你学习了与 TMA、TMEM、Transformer Engine 和 Tensor Core 等高级内存与计算硬件特性相关的优化技术。这些原则同样适用于从单 GPU 到多 GPU 集群的场景。首先，使用系统级性能分析器（如 NVIDIA Nsight Systems）来关联多 GPU 上的 CPU 活动、GPU kernel 以及 NVLink/NVSwitch 流量；然后使用 Nsight Compute 深入分析单个 kernel 的细节。

通过系统性地性能分析、消除占主导地位的停顿，并掌握高级硬件特性，你可以将内存受限的工作负载转变为计算受限

的工作负载——通常带来可观的加速，在强内存受限路径上甚至可取得数量级的提升。

即便是像 NVIDIA GB200/GB300 NVL72 这样具备高 NVLink/NVSwitch 带宽的超大规模多 GPU 系统，单 GPU 层面的算术强度优化仍然是消除大多数瓶颈的关键。每字节数据所做功太少的 kernel 需要过多的内存搬运，会令互连网络饱和并耗尽内存带宽。

这种互连与内存带宽的饱和，会在我们的 kernel 尚未充分利用众多互连 GPU 的计算能力之前就早早出现（例如 NVL72 系统中共有 72 块互连的 GPU）。因此，提高 kernel 的算术强度是让多 GPU 训练与推理工作负载实现高效扩展的关键。

在下一章中，我们将继续深入探讨持久 kernel（persistent kernels）、megakernel、warp 专业化（warp specialization）、协作组（cooperative groups）和线程块集群（thread block clusters）等技术。这些思想是大多数现代 LLM 运行时优化的基础，因此理解它们的实现方式——以及如何在你自己的底层系统优化工作中加以运用——非常重要。

第 10 章 kernel 内流水线、warp 专用化与协作线程块簇

在前几章中，我们讨论了调优内存访问、最大化并行度、让计算与数据传输重叠、提升占用率以及减少 warp 停顿等基础优化技术。这些技术帮助隐藏延迟、消除瓶颈。不过，现代 GPU 提供了更先进的硬件特性与执行模型，可以把这些基础优化技术进一步推向深入。

本章将介绍一些更高级的 CUDA 技术：warp 专用化流水线、带有 grid 级与簇级同步的 cooperative groups、循环处理动态工作队列的常驻 kernel（persistent kernel），以及利用分布式共享内存（DSMEM，或称 DSM）和张量内存加速器（TMA）多播的线程块簇（thread block cluster，又称协作线程数组簇，CTA）。从高层看，线程块簇是一组保证并发运行的线程块，它们可以通过 DSMEM 相互读写共享内存并执行原子操作。

这些方法让我们无需主机干预即可把内存访问与计算操作重叠起来，还能在线程块之间共享片上数据——并让每个 SM 都保持满负荷运转。

理解了这些现代 GPU 执行模型之后，你就可以进入下一章，通过 CUDA 流探索 kernel 间流水线，把这里的优化再向前推进一步。下一章将在本章所讨论的 kernel 内优化的基础上构建 kernel 间流水线。

10.1 kernel 内流水线技术

kernel 内流水线（intra-kernel pipelining）是指在一单次 kernel 执行内部使内存操作与计算相互重叠的一系列技术。（下一章将讨论 kernel 间流水线，它重叠的是运行在不同流中的多个 kernel 之间的工作。）

其核心思想是把 kernel 组织成若干并发阶段：当一份新数据正在被加载或存储时，先前已加载的数据正在被处理。这些阶段针对不同的 tile 或数据块并行运作，从而提升吞吐量并高效地隐藏延迟。

传统上，GPU 依靠 warp 级多线程来隐藏延迟：当某个 warp 因内存加载而停顿时，其他 warp 继续执行计算。这是执行模型中单指令多线程（SIMT）延迟隐藏机制的基础。

kernel 内流水线把这一机制再推进一步：它让内存操作与计算在同一个 warp 或同一个 kernel 内部相互重叠，利用细粒度的协调来错开内存加载与计算——有时甚至发生在单个 warp 之内。

使用 CUDA Pipeline API 的 kernel 内流水线可以把异步内存传输与计算重叠起来，而且完全不需要任何 `__syncthreads()`。kernel 内流水线的两种常见方法是双缓冲与 warp 专用化。

在双缓冲（两级）流水线方法中，所有线程以统一的方式协作；而在 warp 专用化流水线方法中，各 warp 被赋予不同的角色，如内存加载器、计算与内存存储器。具体选择取决于工作负载和性能需求。表 10-1 总结了这两种 `<cuda/pipeline>` 变体。

表 10-1　在现代 GPU 上使用 CUDA Pipeline API 进行 kernel 内流水线的两种方法。

API 变体	适用场景	主要用途
双缓冲流水线	基于 tile 循环的双缓冲	在同一 warp 或线程块内重叠加载与计算
warp 专用化流水线（例如三级：内存加载、计算、内存存储）	具有多个不同 warp 角色的常驻 kernel（本例中为 3 个）	将 warp 分配到内存加载、计算、内存存储等各自独立的角色/阶段

10.1.1 使用 CUDA Pipeline API 的协作分块与双缓冲

你可以使用 C++ Pipeline API 实现传统的双缓冲分块模式：实例化一个两级流水线，使内存加载与计算相互重叠。具体而言，可以声明一个两级的 `cuda::pipeline_shared_state<cuda::thread_scope_block, 2>` 对象，借助 cooperative groups（稍后讨论）把它限定到某个特定线程块。这本质上是一种生产者—消费者模式，如图 10-1 所示。

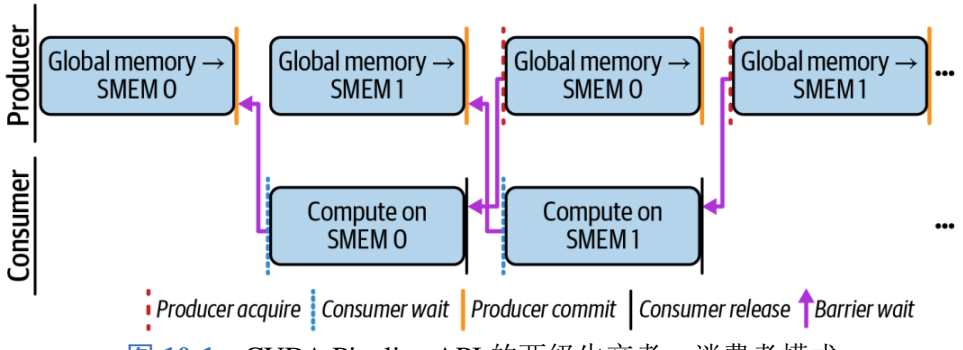

图 10-1　CUDA Pipeline API 的两级生产者—消费者模式

接下来列出 CUDA Pipeline API 的几个关键调用，随后给出一个使用该 API 的双缓冲协作分块 kernel 实现，以演示与现代硬件特性相契合的 CUDA 技术：

```
pipe.producer_acquire()
```

为写入预留下一个流水线阶段。

```
pipe.producer_commit()
```

发出信号，表明此前为该阶段发起的异步操作已可供消费。

```
pipe.consumer_wait()
```

等待该阶段此前已提交的操作完成，以避免循环中出现竞态条件。

```
pipe.consumer_release()
```

释放当前阶段，以便后续复用。

流水线中的两级分别把全局内存加载与计算重叠开来。在第一阶段（Stage 0）中，线程块中的某个 warp 对下一个 tile 向共享内存发起异步预取。该预取会发起协作式的 `cuda::memcpy_async` 拷贝，并最终下变为每线程一条写入共享内存的 `cp.async` 指令。

当 Stage 0 中的那个 warp 正在生产（加载）数据时，线程块中其余的 warp 则在第二阶段（Stage 1）消费（计算）已加载的数据。这一简单的生产者—消费者实现利用持续进行的计算来隐藏 DRAM 延迟，如图 10-2 所示。

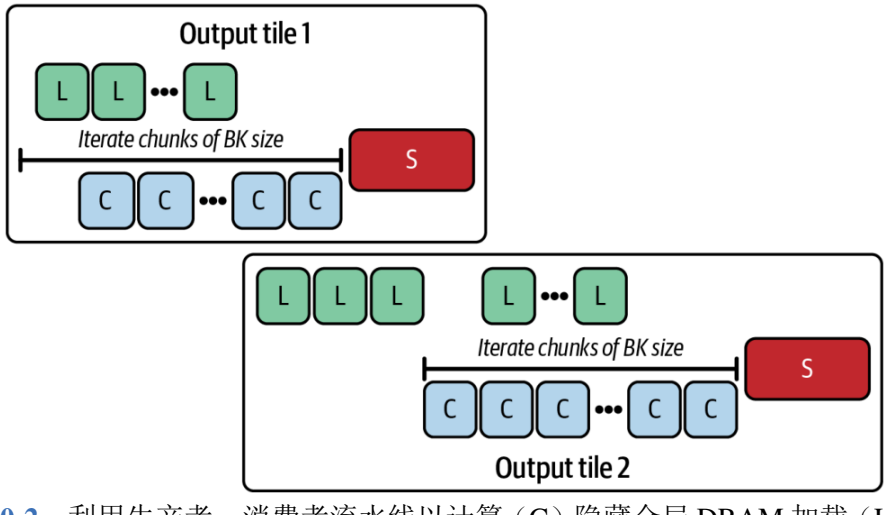

图 10-2 利用生产者—消费者流水线以计算（C）隐藏全局 DRAM 加载（L）延迟

这种模式可以提升 SM 利用率并缩短 kernel 执行时间，但究竟属于计算受限还是内存受限，取决于具体操作、tile 大小与重叠效率。即便是高度优化的 attention kernel（如 FlashAttention-3），由于重叠与数据搬运方面的实际限制，也只能达到峰值 FP16 FLOPs 的 75% 左右。

下面给出一个使用 CUDA C++ Pipeline API 的两级双缓冲示例，该 API 提供了此处用到的细粒度生产者—消费者同步代码：

```
#include <cuda/pipeline>
#include <cooperative_groups.h>
#include <algorithm>
namespace cg = cooperative_groups;
#ifndef TILE_SIZE
#define TILE_SIZE 32
#endif
#ifndef STAGES
#define STAGES 2          // 2 = double buffer, 3 = triple buffer, etc.
#endif
__device__  float computeTile(const float* __restrict__ A_sub,
                              const float* __restrict__ B_sub,
                              int tx, int ty) {


    float s = 0.0f;
    #pragma unroll
    for (int k = 0; k < TILE_SIZE; ++k) {
        s += A_sub[ty * TILE_SIZE + k] * B_sub[k * TILE_SIZE + tx];
    }
    return s;
}
extern "C" __global__
void gemm_tiled_pipeline(const float* __restrict__ A_global, // [M x K]
                         const float* __restrict__ B_global, // [K x N]
                         float* __restrict__ C_global,       // [M x N]
                         int M, int N, int K) {
    cg::thread_block cta = cg::this_thread_block();
    // Shared memory layout: A[STAGES] then B[STAGES]
    extern __shared__ float shared_mem[];
    float* A_buf[STAGES];
    float* B_buf[STAGES];
    {
        float* p = shared_mem;
        for (int s = 0; s < STAGES; ++s) {
            A_buf[s] = p;
            p += TILE_SIZE * TILE_SIZE;
```

```
    }
     for (int s = 0; s < STAGES; ++s) {
         B_buf[s] = p;
         p += TILE_SIZE * TILE_SIZE;
    }
}
__shared__ cuda::pipeline_shared_state<cuda::thread_scope_block,
 ↪  STAGES> state;
auto pipe = cuda::make_pipeline(cta, &state);
int tx = threadIdx.x, ty = threadIdx.y;
int block_row = blockIdx.y * TILE_SIZE;
int block_col = blockIdx.x * TILE_SIZE;
float accum = 0.0f;
int numTiles = (K + TILE_SIZE - 1) / TILE_SIZE;
// Prologue: load first STAGES tiles (or fewer if short)
for (int s = 0; s < std::min(STAGES, numTiles); ++s) {
    int aRow = block_row + ty;
    int aCol = s * TILE_SIZE + tx;
    int bRow = s * TILE_SIZE + ty;
    int bCol = block_col + tx;
    pipe.producer_acquire();
    if (aRow < M && aCol < K) {
        cuda::memcpy_async(cta, A_buf[s] + ty*TILE_SIZE + tx,
                           &A_global[aRow*K + aCol],
                           cuda::aligned_size_t<32>(sizeof(float)),
                            ↪  pipe);
    } else {
        A_buf[s][ty*TILE_SIZE + tx] = 0.0f;
    }
```

```
    if (bRow < K && bCol < N) {
        cuda::memcpy_async(cta, B_buf[s] + ty*TILE_SIZE + tx,
                           &B_global[bRow*N + bCol],
                           cuda::aligned_size_t<32>(sizeof(float)),
                           ↪ pipe);
    } else {
        B_buf[s][ty*TILE_SIZE + tx] = 0.0f;
    }
    pipe.producer_commit();
}
// Steady state
for (int tile = 0; tile < numTiles; ++tile) {
    int s = tile % STAGES;
    // Block-scope wait
    pipe.consumer_wait();
    accum += computeTile(A_buf[s], B_buf[s], tx, ty);
    pipe.consumer_release();
    // Prefetch next tile into the same slot s (ring buffer)
    int nextTile = tile + STAGES;
    if (nextTile < numTiles) {
        int aRow = block_row + ty;
        int aCol = nextTile * TILE_SIZE + tx;
        int bRow = nextTile * TILE_SIZE + ty;
        int bCol = block_col + tx;
        pipe.producer_acquire();
        if (aRow < M && aCol < K) {
            cuda::memcpy_async(cta, A_buf[s] + ty*TILE_SIZE + tx,
                               &A_global[aRow*K + aCol],
                               cuda::aligned_size_t<32>{sizeof(float)},
                               ↪ pipe);
        } else {
            A_buf[s][ty*TILE_SIZE + tx] = 0.0f;
        }
        if (bRow < K && bCol < N) {
            cuda::memcpy_async(cta, B_buf[s] + ty*TILE_SIZE + tx,
                               &B_global[bRow*N + bCol],
```

```
                                cuda::aligned_size_t<32>{sizeof(float)},
                                    ↪ pipe);
            } else {
                B_buf[s][ty*TILE_SIZE + tx] = 0.0f;
            }
            pipe.producer_commit();
        }
    }
    // Epilogue: final store (guard tails)
    int cRow = block_row + ty;
    int cCol = block_col + tx;
    if (cRow < M && cCol < N) {
        C_global[cRow * N + cCol] = accum;
    }
}
```

这段代码首先通过协作组（cooperative groups，CG）获取当前线程块的句柄（CG 将在稍后介绍），随后立即实例化一个绑定到该线程块的两阶段 `cuda::pipeline` 对象。由于流水线在任何异步操作之前就已创建，其内部屏障与内部同步机制会在数据搬运开始之前就位。

接着，kernel 通过定义 `extern __shared__ float shared_mem[]` 为 A、B 两个 tile 分配一块连续的共享内存区域，并用指针运算（`float* A_buf[2]` 与 `float* B_buf[2]`）把该缓冲区划分为四个子区域，A、B 各两个。这样无需额外的动态分配即可实现真正的双缓冲。

在进入主循环之前，kernel 使用绑定到如下流水线的异步拷贝，对前 `STAGES` 个 tile 进行异步预取：`producer_acquire()` → `memcpy_async()` → `producer_commit()`。消费者 warp 则使用 `consumer_wait()` 与 `consumer_release()`，从而在预取的 tile 尚未就绪时恰好开始计算。

这一初始屏障替代了此后对 `__syncthreads()` 的需求，并确保流水线的第 0、1 两个阶段缓冲区已为生产者-消费者序列的首次迭代正确填充。

在每次迭代中，kernel 调用 `pipe.producer_acquire()` 预留出用于加载后续 tile 的下一个缓冲区，然后发起两个 `cuda::memcpy_async` 操作——一个针对

A tile，一个针对 B tile。每次加载都通过 `cuda::memcpy_async(..., pipe)` 绑定到流水线对象，从全局内存向共享内存发起异步拷贝。

当访存合并与 tile 划分做得恰当时，这些异步拷贝可以与计算很好地重叠，使流水线始终有数据可用，从而完成尽可能多的有效工作。在把 `memcpy_async` 调用加入队列之后，kernel 立即通过 `pipe.producer_commit()` 发出完成信号。该提交记录此次拷贝的到达，使消费者 warp 可以只针对该特定阶段进行等待，而不会阻塞整个线程块。

与此同时，线程块中的其他 warp 调用 `pipe.consumer_wait()`。这只会高效地停顿那些依赖 curr 缓冲区数据的线程，直到生产者为该阶段提交为止。等待完成后，每个线程调用设备函数 `computeTile(...)`，执行属于自己的 `TILE_SIZE`× `TILE_SIZE` 点积计算，结果在寄存器中逐步累加。

在当前缓冲区上的计算完成后，这些 warp 调用 `pipe.consumer_release()` 释放该阶段，供后续迭代复用。这种细粒度的释放可避免全线程块范围的停顿，并最大化计算阶段与内存传输阶段之间的重叠。

在每次迭代结束时交换 curr 与 next，让两个双缓冲阶段轮流工作。这样，刚完成计算的内存缓冲区就变成了下一次异步内存操作的目标。

上述生产者与消费者阶段会在 K 维度的所有 tile 上重复执行。处理完最后一个 tile 之后，每个线程的累加器中保存了完整的 `TILE_SIZE` × `TILE_SIZE` 点积结果，随后该结果被写回 `C_global` 中相应的元素。

使用双缓冲时，建议确保每次异步拷贝（`memcpy_async`）之后都跟上配套的 `producer_commit()` 与 `consumer_wait()` 同步调用，这样才能保证计算 kernel 只会在数据正确加载之后才使用它。对于简单循环，现代 CUDA 编译器通常会自动完成这些优化。

> **注** 建议使用 Nsight Compute 的异步拷贝指标来验证流水线是否按预期运行，尤其要关注 warp 占用率与共享内存 bank 冲突。目标是提高 SM Active 百分比并减少停顿周期。DRAM 延迟能否被完全隐藏，取决于占用率、访存模式以及共享内存 bank 的行为。

这种简单的双缓冲方案比朴素分块约快 2 倍。表 10-2 给出了朴素分块实现与优化后的双缓冲流水线实现的性能对比。

表 10-2　朴素分块 kernel 与基于 CUDA Pipeline API 的双缓冲 kernel 的性能对比。

指标	朴素分块	两阶段双缓冲流水线（`double_buffered_pipeline`）
内核执行时间	41.3 ms	20.5 ms（较朴素版约快 2 倍）
SM Active %	68%	92%（较朴素版 +24%）

该表的数据为：内核执行时间——朴素分块 41.3 ms，两阶段双缓冲流水线（`double_buffered_pipeline`）20.5 ms（约为朴素版的 2 倍快）；SM Active %——朴素分块 68%，两阶段双缓冲流水线 92%（较朴素版 +24%）。

> **注**　所有指标表中的数值都只是用于解释概念的示例数据。不同 GPU 架构上的实际基准测试结果请参见 GitHub 仓库。

在本实验中，使用 CUDA C++ Pipeline API 的 `gemm_tiled_pipeline` kernel 相比朴素分块版本取得了 2 倍加速。借助细粒度的 `pipe.producer_commit()` 与 `pipe.consumer_wait()` 原语，流水线始终保持满载，SM Active % 从 68% 跳升 24 个百分点至 92%。

10.1.2 Warp 专门化与生产者-消费者模型

Warp 专门化通过把操作分配给使用不同硬件的 warp 来扩展双缓冲，例如把数据搬运（如 TMA）交给一类 warp，把计算（如 Tensor Core）交给另一类 warp。这与让同一批 warp 既加载数据又进行计算的做法形成对比，如图 10-3 所示。

这种专门化让每组 warp 拥有各自独立的指令序列，指令因此得以持续发射和执行，而不会中断于其他类型的操作。具体而言，warp 专门化允许把一组“生产者”（或“内存”）warp 指定为使用 `cuda::memcpy_async` 异步预取 tile，而让所有其他“消费者”（或“计算”）warp 执行计算，如图 10-4 所示。

这里将四个 warp 指派为生产者角色，其余八个 warp 指派为消费者角色。与大多数生产者-消费者模式一样，你可以为生产者和消费者指派不同数量的 warp。

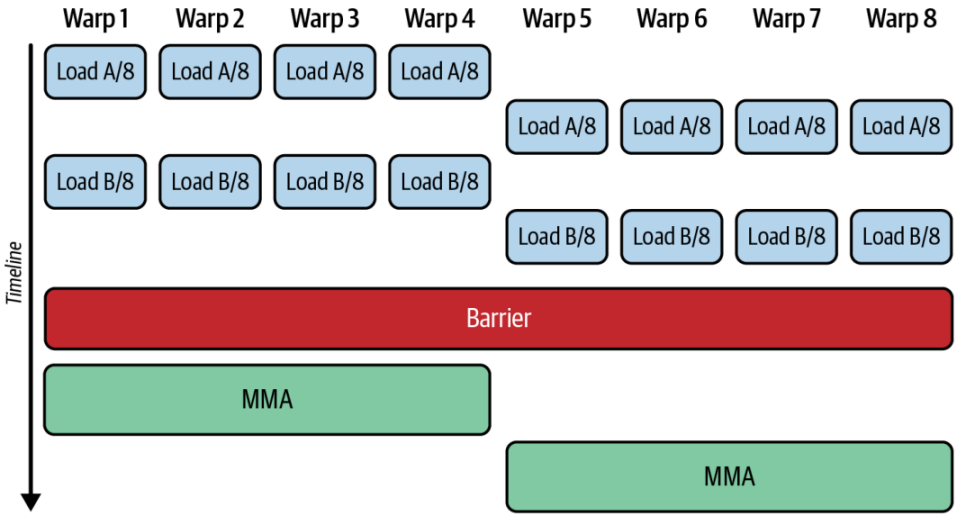

图 10-3　未做 warp 专门化的 kernel：每个 warp 混合执行数据加载与计算（来源：https://oreil.ly/WZDbM）

由于每个 warp 都有自己的调度器，GPU 可以在同一周期内，通过不同的 warp 调度器从不同的 warp 分别发射一条加载指令、一条数学指令和一条写指令。因此，具有多个调度器的 SM 可以在一个周期内发射来自 Warp 0 的一条内存指令、来自 Warp 1 的一条数学指令，依此类推。

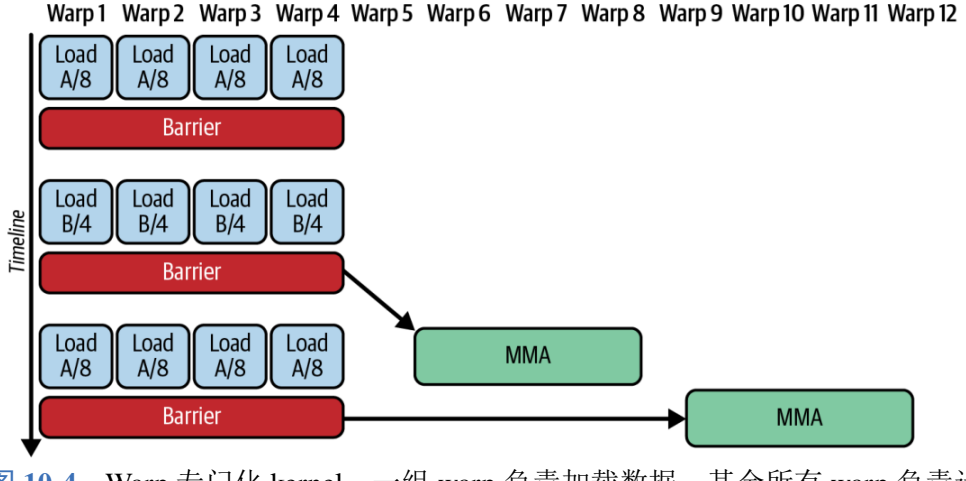

图 10-4　Warp 专门化 kernel：一组 warp 负责加载数据，其余所有 warp 负责计算（来源：https://oreil.ly/WZDbM）

这实际上在 warp 之间构成了线程块级的多发射场景。单 warp 双缓冲无法做到这一点，因为单个 warp 的调度器每个周期只能发射一条指令。

Warp 专门化的一种有趣模式是使用三种不同类型的 warp，例如“加载者”“计算者”“存储者”warp。加载者 warp 把 tile 推入流水线的队列；计算者 warp 对每个 tile 运行计算 kernel；存储者 warp 则写出结果，如图 10-5 所示。

这种 warp 专门化流水线消除了单 warp 双缓冲串行“先加载后计算”循环无法消除的空闲周期。Warp 专门化对数据传输与计算的高效重叠提升了 GPU 利用率——对长时间运行的循环和常驻 kernel 尤其有效。在这些场景下，角色协调与数据交接的开销会在众多迭代中摊薄。

一篇关于 warp 调度的论文表明，warp 专门化可以做到几乎完美的访存与计算重叠。在该研究中，GPU kernel 的访存与计算阶段界限分明，访存和计算轮流充当瓶颈。应用 warp 专门化之后，其工作负载转变为一种新状态：SM 的内存子系统与计算单元几乎在整个时间里同时保持忙碌。

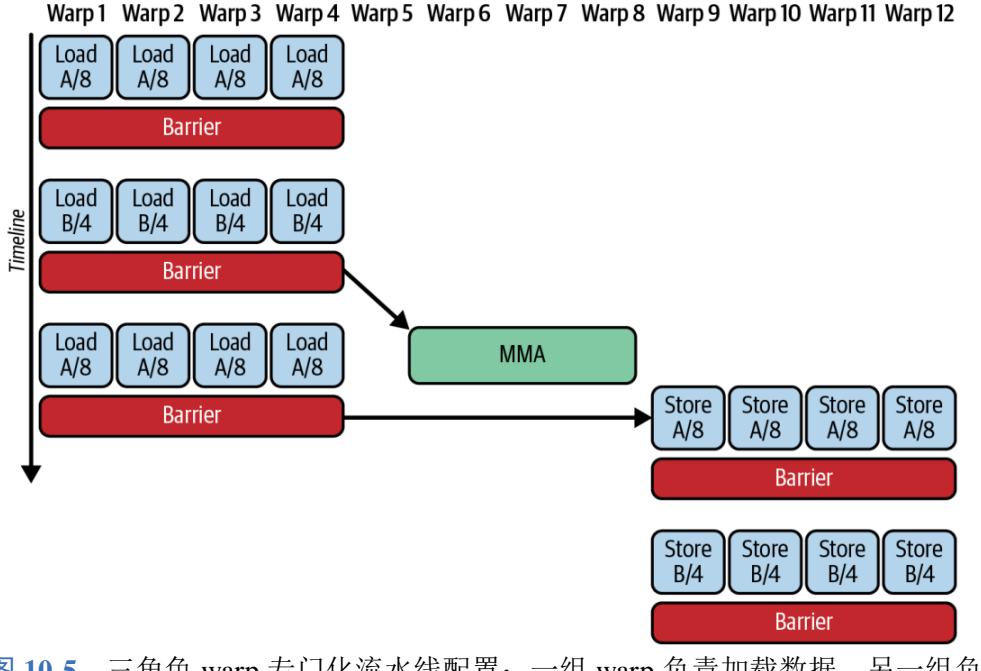

图 10-5　三角色 warp 专门化流水线配置：一组 warp 负责加载数据，另一组负责计算，再一组负责写出数据（来源：https://oreil.ly/xs7YN）

性能分析显示，在使用 warp 专门化之前，其 L2 带宽利用率与 Tensor Core 利用率的相位是错开的；而在 warp 专门化之后，L2 带宽与 Tensor Core 利用率进入同相。这带来了显著更高的有效吞吐量，并表明 warp 专门化能够榨出最后一点空闲时间——即便是调优良好的异步流水线，也可能还有性能被白白留在桌面上。

另一种 warp 专门化模式是对三角色 warp 专门化流水线的改进：它像前面那样给内存加载者指派一组 warp，然后使用两组消费者 warp，在“计算”与“内存写出”两个角色之间“乒乓”切换。这一三角色 warp 专门化架构已在 CUTLASS 中以 `gemm_tma_warpspecialized_pingpong` 的形式对外提供，如图 10-6 所示。

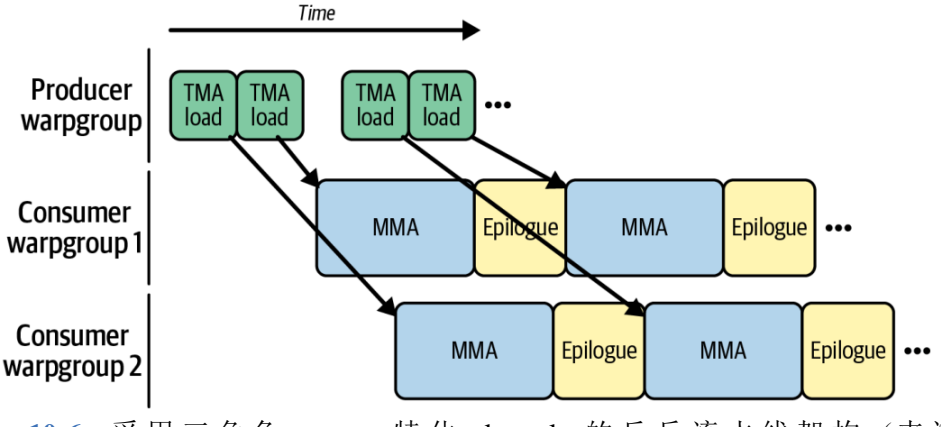

图 10-6 采用三角色 warp 特化 kernel 的乒乓流水线架构（来源：https://oreil.ly/xs7YN）

此处的各个消费者 MMA 相互重叠，并各自包含 MMA 之后的一小段收尾（epilogue）处理。这一逐 MMA 的收尾清理必须在启动下一轮 MMA 之前完成。具体而言，epilogue 可以包括累加、缩放、写回全局内存，或将结果 shuffle 到另一个 warp；此外，它还可执行一些例行的簿记操作，例如推进分块指针、更新循环计数器，以及发出本分块已完成的信号，以便下一次 TMA 加载或 MMA 得以启动。

> **注** 此处虽未画出，但在 MMA 操作之前还存在一个对应的 prologue（前导）处理步骤。就本例而言，prologue 阶段会先向流水线注入若干 TMA 请求，把数据搬运到寄存器中，这样 MMA 消费者随后才能够在 Tensor Core 上开始执行有效计算。

在实践中，warp 特化对榨取最后一点性能极为有效。事实上，FlashAttention v3 将其部分速度提升归功于 warp 特化流水线：它将 GEMM 与 softmax 计算——连同数据传输——重叠起来，让所有硬件单元保持忙碌。正是这种计算与 TMA 驱动数据搬运的激进重叠，帮助注意力计算逼近 FLOPS 峰值。

此外，PyTorch 编译器（第 13 章、第 14 章将介绍）生成的 kernel 也会采用 warp 特化，为数据加载与计算分别调度专职 warp，并使用低开销的屏障来同步这些 warp，类似于下一节详述的 CUDA Pipeline API 实现。PyTorch 编译器系统还与 CUTLASS 的乒乓 GEMM 相集成。torch.compile 与 Triton 都可能为受支持的操作生成 warp 特化 kernel。然而，

它们只会依据启发式规则有选择地应用 warp 特化，并不会为每一个算子都启用该机制。

> **注** 在负载不均衡或需要隐藏延迟的场景中可使用 warp 特化——尤其当单个 warp 的计算量不足以掩盖内存加载延迟时。不过，如果 kernel 本身很小，或极度内存受限，那么坚持采用更简单的双缓冲方案也许能获得相近的收益，同时免去额外的代码复杂度。

10.1.3 使用 CUDA Pipeline API 实现 warp 特化

warp 特化构建在 CUDA Pipeline API 之上，允许各专职 warp 借助细粒度的生产者/消费者原语相互通信。这些调用避免了块级全量屏障，并与 `cuda::memcpy_async` 这类异步拷贝操作自然组合。

使用 CUDA Pipeline API 生产者/消费者调用（例如 `pipe.producer_acquire()`、`pipe.producer_commit()`、`pipe.consumer_wait()` 与 `pipe.consumer_release()`）的关键优势在于：它们只同步真正需要交接数据的特定 warp 或阶段，而不必强迫块内的每个线程都去等待。

块级屏障会使每一个 warp 停滞——包括与该生产者-消费者流水线毫不相干的 warp。屏障之后的一切执行都必须暂停，直到每个线程都到达屏障，如图 10-7 所示。

相比之下，Pipeline API 在内部每个阶段上维护各自的状态。当生产者 warp 完成其异步拷贝并调用 `pipe.producer_commit` 之后，只有那些调用 `pipe.consumer_wait` 的 warp 会阻塞直至数据就绪；块内的其他 warp 可以继续执行不依赖该阶段的任何工作。实践中，CUDA Pipeline API 免除了用屏障暂停整个块的需要，从而减少空闲时间、降低停滞 warp 的数量。借助流水线，你能够在比手写异步拷贝序列（例如 PTX `cp.async` 加 `__syncthreads()`）更细的粒度上协调生产者与消费者的交接。

你可以按三角色模式使用 CUDA Pipeline API 实现 warp 特化：加载 warp 为计算 warp 生产输入，计算 warp 消费这些输入并产出结果，存储 warp 消费这些结果并将它们写出。流水线对象具有块级作用域，其内部自行跟踪阶段顺序。

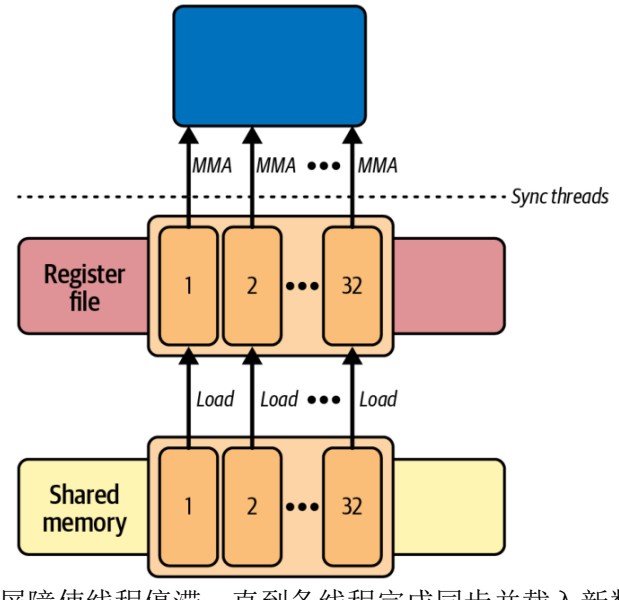

图 10-7　块级屏障使线程停滞，直到各线程完成同步并载入新数据后方能继续

下面给出一个 warp 特化的三角色 kernel 示例，它使用加载 warp（`warp_id` 0）、计算 warp（`warp_id` 1）和存储 warp（`warp_id` 2）来计算数据分块：

```
// warp specialized roles using the CUDA Pipeline API
#include <cuda/pipeline>
#include <cooperative_groups.h>
namespace cg = cooperative_groups;
// smem_bytes = 3×TILE_SIZE^2×sizeof(float) (three roles,
↪  single-pipeline)
// 3 tiles * 112 * 112 * 4 byte per float =
// 150,528 bytes < 227,328 bytes (~227 KB)
// per-block dynamic SMEM limit on Blackwell
// Choose TILE_* so total shared memory per block is <= the device' s
// reported per-block shared memory limit.
// Query cudaDevAttrMaxSharedMemoryPerBlockOption and set
// cudaFuncAttributeMaxDynamicSharedMemorySize accordingly.
#define TILE_SIZE 112
// Example tile compute: C = A x B for one TILE_SIZE by TILE_SIZE tile
__device__ void compute_full_tile(const float* __restrict__ A_tile,
                                  const float* __restrict__ B_tile,
                                  float* __restrict__ C_tile,
                                  int lane_id) {
    for (int idx = lane_id; idx < TILE_SIZE * TILE_SIZE; idx +=
    ↪  warpSize) {

        int row = idx / TILE_SIZE;
        int col = idx % TILE_SIZE;
        float acc = 0.0f;
        #pragma unroll
        for (int k = 0; k < TILE_SIZE; ++k) {
            acc += A_tile[row * TILE_SIZE + k] * B_tile[k * TILE_SIZE +
            ↪  col];
        }
        C_tile[idx] = acc;
    }
}
extern "C"
__global__ void warp_specialized_pipeline_kernel(
        const float* __restrict__ A_global,
```

```
    const float* __restrict__ B_global,
    float* __restrict__ C_global,
    int numTiles) {
thread_block cta = this_thread_block();
// three square tiles in dynamic shared memory: A, B, and C
extern __shared__ float shared_mem[];
float* A_tile = shared_mem;
float* B_tile = A_tile + TILE_SIZE * TILE_SIZE;
float* C_tile = B_tile + TILE_SIZE * TILE_SIZE;
// three stage pipeline shared by the block
__shared__ cuda::pipeline_shared_state<cuda::thread_scope_block, 3>
    pipe_state;
auto pipe = cuda::make_pipeline(cta, &pipe_state);
int warp_id  = threadIdx.x >> 5;
int lane_id  = threadIdx.x & 31;
int warps_per_block = blockDim.x >> 5;
// grid wide warp indexing for persistent tiling
int totalWarps = gridDim.x * warps_per_block;
int global_warp = warp_id + blockIdx.x * warps_per_block;
for (int tile = global_warp; tile < numTiles; tile += totalWarps) {
    size_t offset = static_cast<size_t>(tile) * TILE_SIZE *
    ↪ TILE_SIZE;
    if (warp_id == 0) {
        // loader produces A_tile and B_tile
        pipe.producer_acquire();

        auto bytes = TILE_SIZE * TILE_SIZE * sizeof(float);
        cuda::memcpy_async(cta, A_tile, A_global + offset,
                           cuda::aligned_size_t<32>{bytes}, pipe);
        cuda::memcpy_async(cta, B_tile, B_global + offset,
                           cuda::aligned_size_t<32>{bytes}, pipe);
        pipe.producer_commit();
    }
```

```
        if (warp_id == 1) {
            // wait for A_tile/B_tile
            pipe.consumer_wait();
            // compute C_tile from A_tile, B_tile
            compute_full_tile(A_tile, B_tile, C_tile, lane_id);
            // finished consuming A/B; free that stage
            pipe.consumer_release();
            // publish C_tile for the storer warp
            pipe.producer_acquire();
            pipe.producer_commit();
        }
        if (warp_id == 2) {
            // store consumes computed C_tile
            // Wait for the committed stage before consuming
            pipe.consumer_wait();
            for (int idx=lane_id; idx<TILE_SIZE*TILE_SIZE;
             ↪  idx+=warpSize) {
                C_global[offset + idx] = C_tile[idx];
            }
            pipe.consumer_release();
        }
    }
    // launch with dynamic shared memory size equal to
    // 3 * TILE_SIZE * TILE_SIZE * sizeof(float)
    // When launching with dynamic shared memory >48 KB
    // You need to set cudaFuncAttributeMaxDynamicSharedMemorySize
    // for the kernel
}
```

这里每个 warp 都承担各自的角色。加载 warp 调用 `producer_acquire()`，通过两次 `cuda::memcpy_async` 协同拷贝分别填入 `A_tile` 与 `B_tile`，然后调用 `producer_commit()`。计算 warp 调用 `pipe.consumer_wait()` 等待新提交的数据，并立即调用 `consumer_release()` 释放该阶段以供复用。

随后，计算 warp 通过调用 `producer_acquire()`、计算 `C_tile`、再调用 `producer_commit()`，成为下一段交接中的生产者。存储 warp 调用 con-

`sumer_wait()` 等待计算完成的 `C_tile`，在一个 warp 条带式（warp-striped）循环中将其写入全局内存，然后调用 `consumer_release()`。这一序列只使用一个块级作用域的流水线，无须任何显式的阶段编号，并完全避免了块级 `__syncthreads`。

> **注** 该 kernel 以常驻（persistent）kernel 的形式跨多个分块运行，以摊销启动开销。关于常驻 kernel 的更多内容，稍后详述。

简言之，将 CUDA Pipeline API 与 cooperative groups 结合使用，即可实现细粒度的、SM 级的生产者-消费者交接，而无须任何显式的 `__syncthreads()` 调用。表 10-3 比较了三种实现：朴素的分块 kernel、使用 `double_buffered_pipeline` 的两阶段双缓冲 GEMM，以及我们的 warp 特化流水线 kernel `warp_specialized_pipeline`。

表 10-3 三种实现的比较：朴素分块 kernel、两阶段双缓冲 GEMM 与 warp 特化流水线 kernel。

指标	朴素分块	两阶段双缓冲流水线（`double_buffered_pipeline`）	warp 特化流水线（`warp_specialized_pipeline`）
Kernel 执行时间	41.3 ms	20.5 ms（较朴素 2.01×）	18.4 ms（较两阶段提速 10.2%）
Warp 执行效率	68%	92%（较朴素 +24%）	96%（较两阶段 +4%）
共享内存停顿延迟 / warp 同步停顿	高	低	极低
L2 吞吐量	80 GB/s	155 GB/s（较朴素 +94%）	165 GB/s（较两阶段 +6.45%）
吞吐量可扩展性	最多扩展到每 SM 2–3 个 warp	可良好扩展到每 SM 约 6 个 warp	可近乎线性扩展至 SM 容纳的全部 warp（例如 64 个）

DRAM 读取字节数对比 SM 周期数

重叠较差

重叠良好　重叠极佳

1.7 B　1.05 B（相比 naive 降低 38%）　1.00 B（相比两阶段降低 5%）

指令数

在这里，双缓冲流水线用 20.5 ms 完成 GEMM，而 warp 专门化版本只需 18.4 ms。这一 10.2% 的提升源自 warp 专门化 kernel 只让消费者 warp（例如计算者）发生停顿，而其他 warp（例如加载者和存储者）可独立推进。在前述两阶段双缓冲 kernel 中，每个线程都要参与消费者阶段，因此 `consumer_wait()` 实际上会停顿整个线程块。

这种更细粒度的、按 warp 进行的同步消除了隐式的整块等待，使三个 warp（加载、计算和存储）能够持续重叠执行。于是，平均 SM 利用率从双缓冲设计的约 92% 升至 warp 专门化版本的约 96%，而 warp 停顿周期则降至接近于零。

从可扩展性的角度看，Nsight Compute 显示朴素分块 kernel 在每个 SM 只有 2～3 个活跃 warp 后即趋饱和。这是因为每个 tile 的加载必须全部完成，计算才能开始。

两阶段双缓冲 kernel 在此之上有所改进，它将加载与计算重叠执行。该实现可将每 SM 的 warp 数扩展到 6 个，之后才会触及共享内存或寄存器容量的限制。

相比之下，`warp_specialized_pipeline` 只要还能为加载、计算和存储持续分配新的 warp，便可近乎线性地扩展。例如在 Blackwell 上，每个 SM 的常驻 warp 数可一直提升到架构上限 64 个。（实际常驻数量取决于寄存器、共享内存和块大小。）

如表 10-3 所示，双缓冲与 warp 专门化两种方案都显著优于朴素分块 kernel。`double_buffer_pipeline` 通过重叠 tile 加载与计算将运行时间减半，而 `warp_specialized_pipeline` 由于避免了任何隐式的块级等待，又额外带来 10.2% 的加速——只有专职的“compute”warp 才会停顿。

指令数从朴素版本的 17 亿降至两阶段流水线中的 10.5 亿，减少 38%；在 warp 专门化 kernel 中进一步降至约 10 亿，再减少 4.76%。

L2 加载吞吐量从朴素分块的 80 GB/s 升至两阶段方案的 155 GB/s（+94%），再升至 warp 专门化 kernel 的 165 GB/s（较两阶段 +6.45%）。这是因为该 warp 专门化 kernel 只用一个 warp 把每个 tile 加载进共享内存一次，然后把这一份拷贝多播给所有计算通路，从而消除了残余的冗余 L2 读取。同时，在分块与双缓冲之后，流水线中几乎所有冗余都已被去除。

在实际应用中，两阶段双缓冲流水线适合均匀分块的 GEMM 工作负载，其更简单的生产者/消费者模型能把大部分 DRAM 延迟隐藏在计算之下。而 warp 专门化方案则为不规则或更深的流水线（例如融合 attention kernel）而优化，因为每个 warp 都能持续执行其被指派的角色——加载、计算或存储——而不会迫使块内其余部分停顿。

10.1.4 PyTorch、CUDA Pipeline API 与 warp 专门化

PyTorch 的公开 API 并不直接暴露 `cuda::pipeline`，但当你调用 `torch.compile` 时，编译器会生成（以 OpenAI 的 Triton 语言实现的）kernel，其中使用的优化在功能上等价于 `<cuda/pipeline>` 原语以及 warp 专门化的生产者/消费者。

因此，虽然你不会看到显式生成的 `cuda::pipeline` 调用，但会看到底层的指令与 barrier。这些优化有助于提高占用率，并增大数据传输与计算之间的重叠。换言之，你无需自己编写任何 CUDA 代码，就能获得与手写 `<cuda/pipeline>` 实现相同的低延迟、高吞吐行为。

PyTorch 由 TorchInductor 生成的融合 attention kernel 采用了带生产者与消费者分组的 warp 专门化。例如，考虑 PyTorch 中如下所示的三个独立 GPU kernel：

```
scores = torch.matmul(queries, keys.transpose(-2, -1))
probabilities = F.softmax(scores, dim=-1)
context = torch.matmul(probabilities, values)
```

你可以在 PyTorch 中只用一行代码将其改写，如下所示：

```
context = torch.nn.functional.scaled_dot_product_attention(queries,
   keys, values)
```

在底层，加载 warp 把 tile 取入共享内存，计算 warp 对 tile 进行处理，存储 warp 再将结果写回。这消除了 matmul 与 softmax 阶段之间代价高昂的全局内存往返。

总体而言，`torch.compile` 很适合性能敏感且不规则的工作负载。像 Blackwell 这样的现代 GPU 拥有充裕的 SM 资源和很高的内存带宽。因此，像 warp 专

门化这样的技术能最大限度减少空闲周期，使性能在各 warp 之间接近线性扩展——至少在 SM 或内存带宽完全饱和之前如此。

> **注** 即便是 FlashAttention-3 这样高度优化的 kernel，借助 warp 专门化重叠也只能达到 FP16 峰值 FLOPS 的约 75%。这说明要取得一个显著的优化里程碑，并不需要达到 100% 的计算利用率。

通过把加载、计算与存储解耦为独立阶段，流水线模型最大化了吞吐量与资源利用率，使其成为长时间运行 kernel 的首选方案，包括 transformer attention、融合算子流水线以及自定义任务调度器等。这些 kernel 借助细粒度的 warp 间通信、深度流水线和线性 warp 扩展来提供高性能。

10.2 持久 kernel 与 Megakernel（巨型 kernel）

持久 kernel（persistent kernel），也称持久线程，颠倒了通常“一个任务一个 kernel”的做法。与其启动许多各自带来可观开销的小 kernel，不如只启动一个长期运行的单个 kernel，其线程不断从全局或共享内存中的共享生产者—消费者队列里领取工作。

持久线程循环运行，在数据块到达时（往往借助内存拷贝或主机信号）就地处理它，而不退出 kernel。这从根本上避免了反复启动 kernel 的开销。例如，持久 kernel 可以让某个线程块的 Warp 0 把数据从全局内存（或 CPU 主机内存）复制到共享

内存中；与此同时，Warp 1 正在计算上一批数据。这就是 GPU 上的一种软件流水线形式。

例如，设想有 1,000 个细小且相互独立的任务。传统做法可能要启动 1,000 个独立的 kernel，每个 kernel 只占用少量 SM 运行极短的一段时间便退出。

实践中，GPU 会随着每个细小 kernel 的启动而反复爬升、随后回落，两次启动之间大部分 SM 处于空闲，硬件得不到充分利用。

改用持久 kernel 时，只需启动一个大网格，让它让 GPU 在整个工作负载期间保持忙碌。例如在拥有 132 个 SM 的 GPU 上，这可能意味着每个 SM 启动一

个块、每块 256 个线程，共计 33,792 个线程。随后每个线程执行的代码大致如下：

```
__device__ int g_index;  // global counter for next task; initialize to
↪  0 on host
__global__ void persistentKernel(Task* tasks, int totalTasks) {
    // Every thread loops, atomically grabbing next task index until
    ↪  none remain
    while (true) {
        int idx = atomicAdd(&g_index, 1);
        if (idx >= totalTasks) break;
        processTask(tasks[idx]);
    }
}
```

在启动该 kernel 之前，宿主会把设备内存中的 `g_index` 置为 0，然后按如下方式调用 kernel：

```
cudaMemset(&g_index, 0, sizeof(int));
// one block per SM on a 132 SM GPU
int blocks         = 132;
int threadsPerBlock = 256;
persistentKernel<<<blocks, threadsPerBlock>>>(d_tasks,
    totalTasks);
cudaDeviceSynchronize();
```

此时，你不再为每一个任务单独支付启动开销，而只需在启动 `persistentKernel` 时支付一次。假设启动耗时约 0.02 ms，1,000 个任务各耗时 0.1 ms，那么这些 kernel 将接连运行，总工作时长约 100 ms。

相比之下，接连运行 1,000 个小 kernel，每个启动耗时 0.02 ms，则会额外增加 20 ms 的开销——这还不算 1,000 个任务全部运行所需的 100 ms。简言之，把小任务合并进一个持久循环（persistent loop）就能省下数十毫秒的启动开销。

使用持久内核（persistent kernel）时，GPU 保持在高度利用状态——几乎所有 SM 都在并发处理任务——因为有数以万计的线程可用于处理这约 1,000 个

任务。相比之下，顺序启动 1,000 个小 kernel，GPU 在任意时刻都有相当大的部分处于闲置。在我们的例子中，这相当于平均只利用了 GPU 约 35% 的容量。

在这一持久内核场景中，每个 SM 运行一个包含 256 线程的线程块（SM 最大可容纳 2,048 线程），因此每个 SM 都有工作在做。如此一来，即使每个 SM 自身的占用率相对较低（256 ÷ 2048 线程，即 12%），仍几乎有 100% 的 SM 处于活跃状态。

以这种方式使用持久内核，GPU 可以保持很高的实测 SM Active 百分比，前提是要计入寄存器与共享内存的用量。请务必始终用 Nsight Compute 加以验证。

在现代 GPU 上，持久内核尤为有效，因为更大的共享内存容量和扩展的寄存器堆允许每个线程在片上保存更多中间状态。线程可以使用 TMA 为后续任务预取张量分块（tile），同时其他 warp 进行计算。于是，当一些线程正在处理某个任务时，另一些线程便可用 TMA 预取后续任务所需的数据——而不必让内存传输指令占用 SM 的计算流水线。

10.2.1 持久内核的常见工作负载

当存在许多规模较小或大小不均、且若分别处理启动开销会很高的任务时，持久内核最能发挥作用。它们支持动态负载均衡：较快的线程可以继续循环并领取更多工作。使用持久内核，就不会有任何 SM 过早空闲。

这一模式常见于不规则工作负载，例如图遍历、自定义批量变换，以及 LLM 推理中常见的逐 token 操作。在这些场景下，各任务的耗时可能相差悬殊。

不过，持久内核也有缺点。首先，你必须使用原子操作显式管理任务队列与同步。如果大量线程同时试图递增同一个计数器，就可能导致竞争。

调试单个庞大的持久循环要比调试多个小 kernel 更复杂。这是因为单个发散的线程或一次意外的分支就可能让整个内核挂起。此外，一个持久内核可能无限期地独占 GPU。因此，如果有其他工作负载需要并发运行，就必须仔细分配流（stream）或划分资源。

简言之，与朴素的内核相比，持久内核通过把 GPU 变成一个动态的“工作线程”池来持续领取并处理任务，可显著提升整体吞吐量（例如 2–3 倍）。在现代 GPU 硬件上，这种方法消除了启动开销，最大化 SM 占用率，并且——与协

作组或线程块簇（稍后介绍）相结合时——能让数据在整个多级流水线期间保持驻留片上。

越来越多的框架和库开始使用持久内核与巨型内核（megakernel），以避免容量浪费并提升推理这类延迟敏感型工作负载的性能。关键在于消除重复启动，并借助 GPU 上的设备端任务队列，让 SM 始终满载并执行有效工作。

> **注** 截至撰写本文时，由于调度复杂度，PyTorch 尚不会自动将整个多阶段工作负载融合为一个 kernel。因此，要充分发挥持久内核与巨型内核的全部优势，需要自定义 CUDA 代码或专用编译器。尽管如此，对于多阶段算法，只要正确处理同步并避免死锁，重构为持久的巨型内核仍能带来可观的性能提升。

10.2.2 面向推理的巨型内核

此外，源自大规模推理的一种现代持久内核方法称为巨型内核。巨型内核把跨层——甚至跨 GPU——的整段操作序列融合为一个大型 kernel。如图 10-8 所示，通过消除重复的内核启动开销，持久巨型内核相比传统的逐层启动方式，可获得 1.2 倍至 6.7 倍的延迟改善。

10.2.3 持久内核与 warp 特化

warp 特化通常与持久内核搭配使用，此时线程在较长时间内执行许多次迭代。这样可以构建更深的流水线、获得更好的重叠，并高效利用长生命周期资源。对于运行时间较短的 kernel，持久内核与 warp 特化带来的额外代码复杂度可能得不偿失。

持久内核调度的一个局限在于，很难找到足够多的 SM 供持久内核利用。如果过多的 SM 已被另一个内核占用，持久内核可能没有足够的资源来启动。这使得跨 SM 的调度与负载均衡变得颇具挑战。

为了让持久内核（进而 warp 特化）更容易落地，现代 GPU 支持线程块簇（thread block clusters）——由于线程块也称为协作线程数组（CTA），它们也被称为 CTA 簇。我们将在后文讨论线程块（CTA）簇；简而言之，它允许你把多个线程块组合成占用 GPU 上多个相邻 SM 的“簇”。

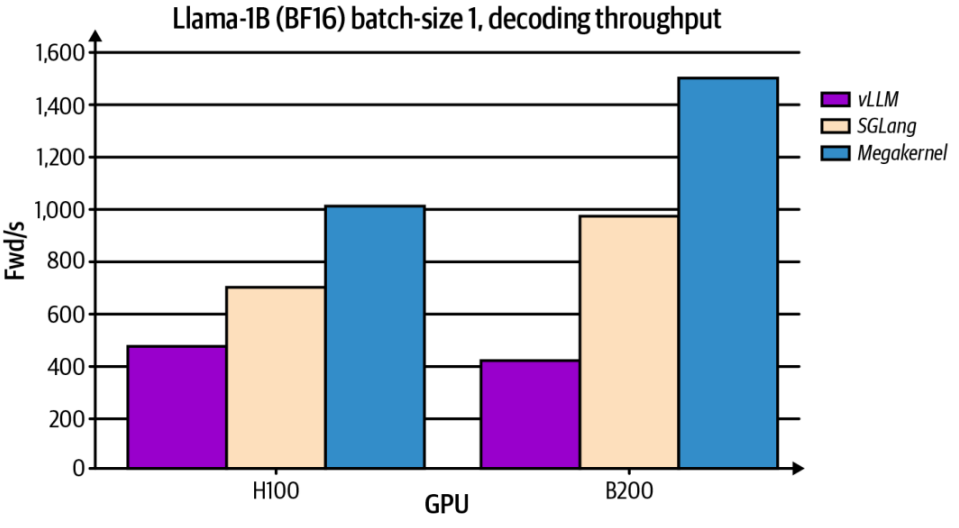

图 10-8 巨型内核相对 vLLM 与 SGLang 的解码吞吐量提升（来源：https://oreil.ly/2aZiF）

10.3 协作组（Cooperative Groups）

协作组让你可以在任意粒度上定义并同步线程组。例如，你可以按单个线程、warp、tile、线程块和簇来创建组，如图 10-9 所示。

协作组提供了安全且可复用的集合操作（collective），如 sync、broadcast 和 reduce，这与自行临时搭建的同步屏障形成对比。通常，线程只能在各自的线程块内通过 `__syncthreads()` 进行同步——而例如整个 grid 并没有内建的全局屏障。

协作组为你在 kernel 内部提供细粒度同步。该 API 非常适合协调跨 warp、线程块和簇的多级流水线。要在你的 kernel 中使用协作组 API，只需包含 `<cooperative_groups.h>`，获取组对象，然后在这些组之间进行同步与协调。

该 API 包括诸如 `cg::this_thread_block()`、`cg::tiled_partition()` 和 `cg::this_cluster()` 等调用。随后你可以调用 `group.sync()` 或类似的集合操作，来协调这些组中的线程。

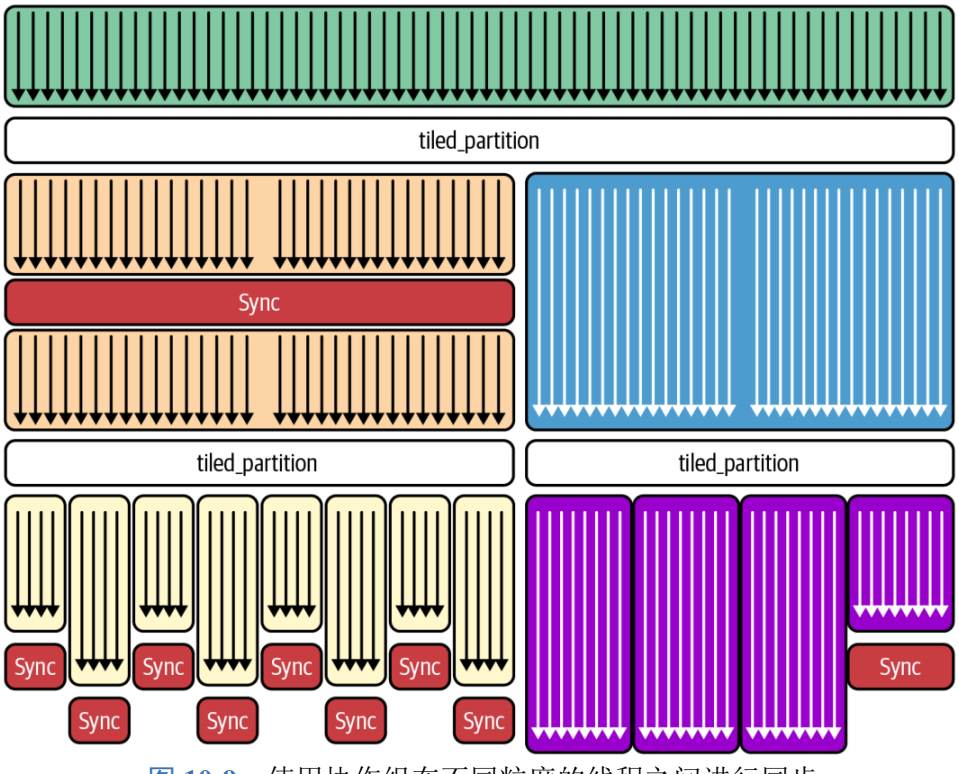

图 10-9　使用协作组在不同粒度的线程之间进行同步

要以协作模式启动 kernel，需使用 `cudaLaunchCooperativeKernel()`。在协作模式下，CUDA 会确保你启动的 grid 能够同时驻留——否则启动就会失败。因此，建议始终基于

`cudaOccupancyMaxActiveBlocksPerMultiprocessor` 来推算协作 grid 的规模，并限制网格大小以避免启动失败。在 kernel 内部，随后可以调用以下代码，实现一个跨所有线程块的屏障：

```
cooperative_groups::this_grid().sync(); // or grid.sync()
```

此时，任何线程块中的任何线程都必须等到每个线程块中的每个线程都到达该点之后才能继续执行。借助这一屏障，你可以在不结束启动、不把控制权交还给主机的情况下，把 kernel 切分为多个顺序阶段。

例如，考虑 LLM 中常见的 softmax 算法，它包含两个阶段：先对整个数组做归约，计算出一个聚合和；随后进行使用该聚合和的逐元素计算。传统做法

是：启动一个 kernel 执行归约，把结果复制回主机内存或全局内存，再启动第二个 kernel 从主机或全局内存读取结果，然后计算 softmax。这需要大量相对缓慢的内存搬运。

使用协作组，这两个阶段可以在一个 kernel 中完成：每个线程块计算各自的部分和，由一个线程块把这些部分和聚合成最终结果，所有线程块调用 `grid.sync()` 等待聚合完成，然后所有线程进入第二阶段。此时每个线程将从寄存器或共享内存——而不是全局内存——读取聚合和。

CUDA 保证协作启动（cooperative launch）中的所有线程块同时驻留在 GPU 上。如果你请求的线程块数量超过了能够并发驻留的数量，启动就会直接失败。

由于协作型 kernel 必须以 GPU 能并发运行的网格规模启动，`grid.sync()` 不会挂起来等待根本不存在的线程块。换言之，CUDA 运行时保证所有线程块都处于活动状态，并且都会到达 `grid.sync()` 屏障。如果 kernel 的启动规模过大而无法一次性运行，启动会直接失败。因此，检查 `cudaLaunchCooperativeKernel()` 的返回状态非常重要。

例如，若某 GPU 拥有 132 个 SM，而你的 kernel 占用的资源恰好允许每个 SM 同时运行 4 个线程块，那么网格不得超过 528 个线程块才能启动成功。一旦超过该上限，协作启动会直接失败。请使用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 为协作型 kernel 确定网格规模，并在假定已有进展之前检查 `cudaLaunchCooperativeKernel` 的返回状态。

在 Blackwell 之前，开发者常常把 `cudaLaunchCooperativeKernel` 与全局内存原子操作或标志位结合起来，以协调不共享片上内存的多个线程块。这种做法可行，但会迫使中间结果被搬运到全局内存，从而引入额外的 HBM 流量。

在 Blackwell 上，线程块集群（thread block cluster）与 DSMEM 提供了高效得多的替代方案。线程块集群可以在片上 SRAM 中共享数据，并且无需经过全局内存往返即可完成同步。本章稍后将讨论线程块集群与 DSMEM。

当你需要在单次 kernel 启动内实现真正覆盖所有线程块的屏障时，应当使用协作型 kernel。此外，当你想把中间结果保留在快速内存（例如寄存器或共享内存）中，而不是反复写入并读回全局内存时，它们也很有用。建议将网格规模限制在 GPU 的最大并发线程块容量之内——或者选用更大的线程块——以减少线程块总数，避免 kernel 启动失败。

协作型 kernel 的缺点在于网格规模受硬件容量的限制。这可能迫使你使用更少但更大的线程块——或者依赖线程块集群（本章稍后介绍）。

即便所有线程块都在并发运行，协作屏障仍要求每个线程块中的每一个线程都调用 `grid.sync()`。只要有一个线程跳过（或永远没到达）`grid.sync()` 调用——例如由于发散的 if 语句——其他所有调用了 `grid.sync()` 的线程都会永远等待下去，从而导致死锁。

简而言之，协作组（cooperative group）kernel 让你把整个 GPU 视为一项单一的协作资源，`grid.sync()` 就是其中的全局屏障。这对需要全局同步与数据共享的多阶段算法非常理想。但请记住，`grid.sync()` 是一种相对重量级的同步，其开销高于块级或集群级屏障，因为它必须协调运行在多个 SM 上的所有线程块。因此，你应当节制地使用 `grid.sync()`，并至少保证 kernel 在两次重量级同步调用之间完成相当多的工作。

> **注** 如果只需要有限的跨线程块协调，使用全局内存原子操作或每线程块标志位可能比依赖全网格屏障更安全、更简单。不过，它们的效率不如线程块集群与 DSMEM，稍后将对此展开讨论。

10.3.1 协作网格同步与常驻 kernel

如果你的工作负载偶尔需要在常驻循环内部借助跨线程块屏障来汇总部分结果，可以通过调用 `grid.sync()` 把常驻 kernel 与协作组结合起来。这样可以在不结束 kernel、也不重新启动的情况下获得网格级屏障，使归约及其他全局步骤构成的多阶段流水线完全留在设备上运行。

例如，考虑这样一个工作负载：跨越 1,000 次迭代反复执行两种不同的计算，且由于存在跨线程块数据依赖，每次计算都需要一次全局屏障。朴素的实现可能每次迭代启动两个独立的 kernel，共产生 2,000 次 kernel 启动。在单一流中，第二个 kernel 会自动等待第一个 kernel——无需显式的主机端同步——但你仍要付出启动开销。更好的做法是把所有工作融合到一个协作型常驻 kernel 中，如下所示：

```
#include <cuda_runtime.h>
#include <cooperative_groups.h>
namespace cg = cooperative_groups;
__device__ inline float someComputationA(float x) { return x * 2.0f; }
__device__ inline float someComputationB(float a, float b) { return a +
 ↪ b; }
__global__ void combinedKernel(float* __restrict__ dataA,
                               float* __restrict__ dataB,
                               int N,
                               int iterations) {
    cg::grid_group grid = cg::this_grid();                // whole-grid
     ↪ group
    const int tid = blockIdx.x * blockDim.x + threadIdx.x; // thread's
     ↪ linear id
    const int stride = gridDim.x * blockDim.x;            // total grid
     ↪ threads
    for (int it = 0; it < iterations; ++it) {
        // Stage 1: update all elements of A using a grid-stride loop
        for (int i = tid; i < N; i += stride) {
            dataA[i] = someComputationA(dataA[i]);
        }
        grid.sync();  // global barrier; also provides memory visibility
        // Stage 2: use finished A to update B (again grid-stride)
        for (int i = tid; i < N; i += stride) {
            const float mid = dataA[i];
            dataB[i] = someComputationB(mid, dataB[i]);
        }
        grid.sync();  // barrier before next iteration
    }
}
```

在主机端，只需按如下方式调用这个合并后的协作型常驻 kernel：

```
// ... allocate/copy dA, dB, set N, iterations, choose blockSize
cudaDeviceProp prop{};
cudaGetDeviceProperties(&prop, 0);
if (!prop.cooperativeLaunch) {
    // Fallback: run two kernels per iteration instead of grid.sync()
    // (see alternative below)
}


// Compute the maximum grid size allowed for a cooperative kernel
int maxBlocksPerSm = 0;
const int blockSize = 256;
cudaOccupancyMaxActiveBlocksPerMultiprocessor(&maxBlocksPerSm,
                                              combinedKernel,
                                              blockSize, 0);
const int maxCoopGrid = maxBlocksPerSm * prop.multiProcessorCount;
// Choose grid size, but clamp to cooperative limit
int gridSize = (N + blockSize - 1) / blockSize;
if (gridSize > maxCoopGrid) gridSize = maxCoopGrid;
// Launch cooperatively
void* args[] = { &dA, &dB, &N, &iterations };
cudaLaunchCooperativeKernel((void*)combinedKernel, gridSize, blockSize,
 ↪  args);
cudaDeviceSynchronize();
```

这个单一 kernel 取代了原本需要 $2 \times 1{,}000 = 2{,}000$ 次 kernel 启动的做法，避免了反复的启动开销。在循环内部，`grid.sync()` 保证正确的执行顺序（所有线程块都完成第 1 阶段之后，任何线程块才开始第 2 阶段，同样，进入下一次迭代之前也是如此），而无需任何主机端同步。

线程私有数据和每线程块数据可以在两次 `grid.sync()` 之间一直保留在寄存器/共享内存中。跨线程块的数据交换则应使用全局内存（或线程块集群与分布式共享内存 DSMEM，稍后将介绍）。同时，由于工作是在 GPU 上循环执行的，首次调用之后便不再有启动开销。

协作型 kernel 需要设备支持（`cooperativeLaunch`），并且必须通过 `cudaLaunchCooperativeKernel` 启动。网格中的所有线程块必须能够同时驻留。请

使用 CUDA 占用率（occupancy）API 确定并钳制网格规模，保证所有 CTA 能够共同驻留，否则启动会失败。例如，当 $((N + \texttt{threads} - 1) \div \texttt{threads})$ 超出协作容量时就会发生这种情况。

注 请使用 CUDA 占用率 API 来确定协作网格的规模。

现代 GPU 可以并发运行数量级在几千左右的线程块（前提是每个线程块的寄存器和共享内存等资源占用都不大），因此仍有相当可观的余量。不过，在采用这种实现之前，仍应核对线程块大小与资源消耗。

在这个 kernel 示例中，kernel 在两次迭代之间从不把控制权交还给主机。因此，对外层循环而言，它的行为如同常驻 kernel，同时在内部各阶段强制执行全局同步点。

这种组合模式在 LLM 推理的多步算法中尤为强大：每一层可能先需要一次归约或归一化（阶段 1），随后再进行逐元素变换（阶段 2）。把所有工作都纳入一个协作式持久内核后，你便消除了所有内核间的往返开销，并最大化了片上数据局部性。

10.3.2 何时组合持久内核与协作组

一种常见的最佳实践是：在资源允许的前提下，以每个 SM 一个线程块的方式启动持久内核。这样，每个 SM 都有一个常驻线程块，从工作队列中迭代地取出任务进行处理，从而最大化占用率并避免 SM 闲置。这些线程块随后可以在各阶段之间使用 `grid.sync()` 进行协调。不过，在协作式与持久式策略之间做取舍——以及是否将二者结合——时，请先回答以下问题：

你是否拥有多个需要全局同步的顺序阶段？

如果有，就使用协作内核（或协作式持久内核），以便在阶段之间调用 `grid.sync()`。

是否存在许多受启动开销拖累的小型或不规则任务？

如果有，就使用持久内核，让线程在共享队列上循环工作，而无需返回主机。

你能否负担得起将整个网格（或一个线程块集群）专用于这项工作？

如果可以，协作式持久内核可能带来最佳性能。

你是否需要让 SM 与其他工作共享？

如果需要，可以考虑使用线程块集群（见下一节），这样持久内核或协作内核就不会去承担其他工作负载。

简而言之，当你使用一个既能在任务上持久循环、又包含 `grid.sync()` 调用以在阶段间同步所有线程块的单一内核时，便消除了原本在多次独立内核启动之间经常出现的停顿和额外的内存传输。

在现代 GPU 上，这意味着数据在整个计算过程中都留在共享内存或寄存器里，而不是在每个阶段之后写回全局内存。因此，GPU 几乎所有时间都在做有用的工作，从而获得接近其峰值硬件极限的性能。

一个重要的注意事项：协作内核会占用网格中的全部 SM，因此其他内核无法在这些 SM 上并发运行。如果你需要协同调度其他工作——例如在单独的流上进行异步数据预取，或运行低优先级的推理内核——就可能需要把 GPU 划分为多个线程块集群，这将在下一节介绍。

10.4 线程块集群与分布式共享内存

协作组是一种软件层抽象，它提供的 API 让你可以把内核线程划分为任意的集合，以进行同步和数据搬运，包括 warp、tile、线程块，甚至整个网格或多设备网格。

相比之下，线程块集群（thread block cluster，或称协作线程阵列 [CTA] 集群）则是一种硬件级层次结构。它把一部分 SM 分配给你的协作网格，其余 SM 留给其他内核使用，从而降低了单个内核独占 GPU 的风险。GPU 保证这些线程块会被同时调度到同一个 GPU 处理集群（GPC）上，如图 10-10 所示。

GPC 是一组相邻的 SM。GPU 将线程块调度到各 GPC 上的方式，与它把一个线程块的线程调度到同一 SM 上的方式类似。

> **注** 在 NVIDIA Blackwell B200/300 和 GB200/300 这类多裸片（multidie）模块上，实际上存在多个 GPC“分区”——每个裸片一个 GPC 分区。Blackwell 是双裸片 GPU，因此有两个 GPC 分区。请记住，Blackwell 的两个 GPU 裸

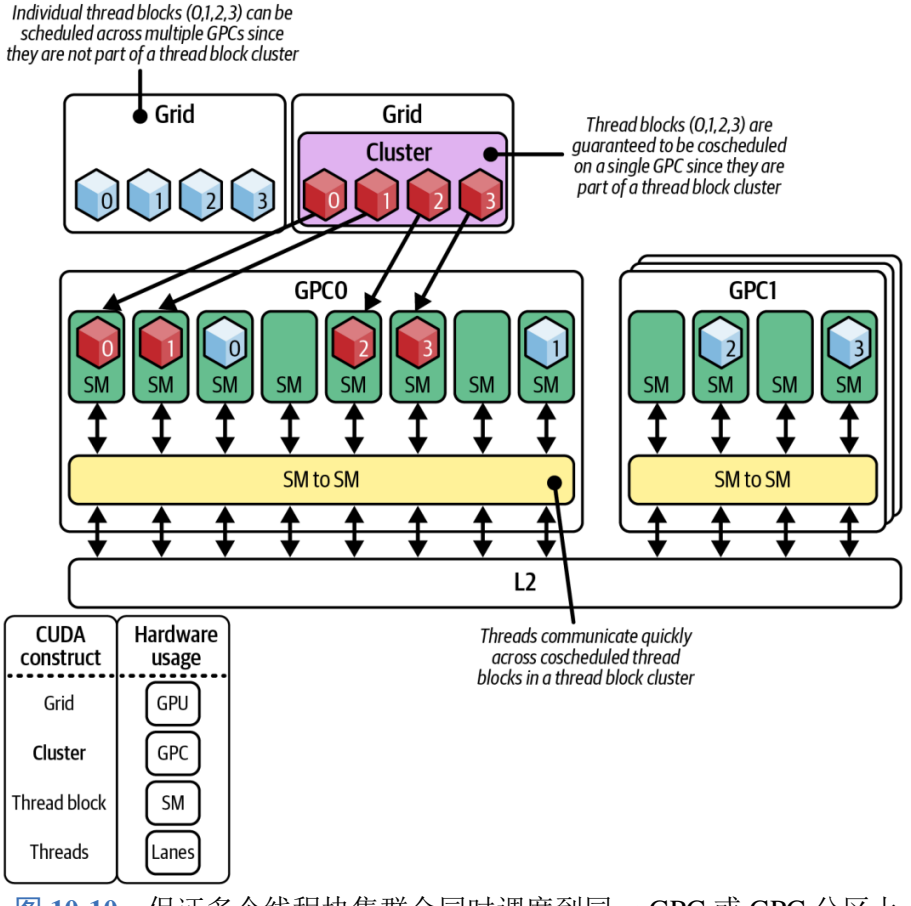

图 10-10　保证多个线程块集群会同时调度到同一 GPC 或 GPC 分区上

片通过 NV-HBI 相连，并对外呈现为一个具有跨裸片完全缓存一致性的单一 CUDA 设备；两颗裸片间的 L2 缓存也是一致的。因此，这些裸片构成了一个合并的逻辑 GPC，架构会替你处理这些独立的 GPC 分区。

GPU 提供分布式共享内存（DSMEM，将在下一节讨论），供线程块集群在其各个线程块之间使用。它还支持通过 Cluster Group API（`cluster.sync()`）实现集群级屏障。

有了这种集群级屏障，你就可以只同步线程块的一个子集，而不阻塞整个 GPU。线程块集群让你能够启动一个协作内核，并把网格细分为更小的线程块组，如图 10-11 所示。

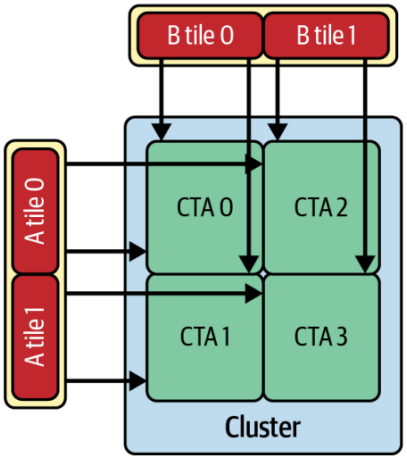

图 10-11 对于这四个（2 × 2）线程集群，A 和 B 的每个 tile 都会同时加载到两个线程块中（来源：https://oreil.ly/kEZsv）

在每个组内调用 `cluster.sync()` 即可获得一个局部屏障。这样，集群内的线程块便可通过专用的片上资源共享数据，而不必独占所有 SM。在现代 GPU 上，你可以使用 DSMEM，它允许线程块集群共享一块连续的片上 SRAM 区域，从而在原生硬件支持下实现同一集群内各线程块之间的低延迟通信。

线程块集群把一部分线程块编为一组，并在每个集群内调用 `cluster.sync()`，只在这些线程块之间进行“局部”同步。传统上，线程块内部的线程可以借助共享内存相互协作；而在现代 GPU 上，线程块之间也能通过线程块集群和 DSMEM 彼此协作。

换言之，在没有线程块集群的情况下，只有同一线程块内的线程才能通过共享内存共享数据；不同线程块必须借助全局内存和全局屏障（例如 `grid.sync()`）来协调，这会阻塞所有线程并限制可扩展性。

不仅如此，此前分布在不同线程块中的线程若想高效共享状态并进行同步，除了使用全局内存之外，就只能像上一节所述那样，借助 `grid.sync()` 进行粗粒度的全网格同步。遗憾的是，全局内存和 `grid.sync()` 都相对较慢，因此都可能成为瓶颈。

线程块集群由 GPU 硬件原生支持，其中包括集群启动控制（cluster launch control）。集群启动控制是一种硬件级机制，负责启动和调度持久的线程块集群。具体来说，它能让持久内核（及其线程块集群）保持工作负载的均衡——即便

部分 SM 已被占用。这为高效的 warp 专门化（warp specialization）实现提供了基础。

借助硬件支持的通信与同步构件，线程块集群可以采用底层 PTX 指令和 CUDA 内建函数形式的集群级屏障来完成同步。由于线程块之间的屏障同步由硬件直接支持，集群内的线程块执行屏障同步的速度远快于使用 `grid.sync()` 的全网格同步。

10.4.1 线程块 Swizzling（重排）

在直接按网格启动时，线程块会严格按行主序或列主序处理 tile。这可能导致先执行的线程块把后续线程块需要的数据逐出缓存，造成复用率低和额外的内存流量。理想的做法是让 A 和 B 的 tile 处于同一波次内，从而能够从 L2 缓存中读出。

要绕开这种低效，可以使用线程块 Swizzling。就像可以用 swizzling 技术优化内存访问、避免共享内存 bank 冲突一样，你也可以通过线程块 Swizzling 避免以低效的行主序或列主序来分配 tile，如图 10-12 所示。

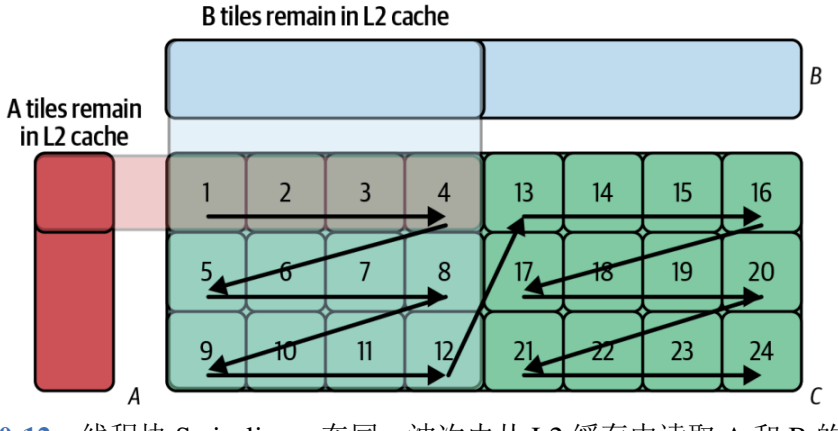

图 10-12　线程块 Swizzling：在同一波次内从 L2 缓存中读取 A 和 B 的 tile

线程块 Swizzling 让同一波次所需的 A、B 两个矩阵的 tile 都留在 L2 中，从而实现最大复用。当应用于持久化和分块的 GEMM 工作负载时，这种 swizzling 通过减少内存未命中和带宽压力，可以带来两位数的性能提升。线程块 Swizzling

是一种简单而强大的模式重排序技术，它使内核启动顺序与缓存局部性保持一致。

凭借对线程块簇（thread block cluster）的硬件支持、较多数量的 SM 以及改进的流水线调度，现代 GPU 可以承载规模更大的持久 kernel，并用精巧的 warp 专业化流水线来优化 kernel 的性能。

简而言之，线程块簇在协作组（cooperative groups）模型的基础上更进一步：它允许一小部分线程块在不“锁死”整个 GPU 的前提下同步并共享状态。这样，你就能在单次 launch 内部构建多阶段、细粒度的流水线，而不排挤设备其余部分；剩余的 SM 依然可以自由地执行各自的独立工作。

在线程块簇内部，有两种用于在线程块之间共享状态的关键机制：DSMEM 和 scratch memory（暂存内存）。下面我们依次介绍它们。

10.4.2 分布式共享内存

分布式共享内存（distributed shared memory，DSMEM）将 `__shared__` 内存的概念从单个线程块扩展到整个线程块簇。在传统 kernel 中，每个线程块都拥有自己私有的 `__shared__` 区域，其他线程块无法访问。

而有了 DSMEM，同一簇内的多个线程块就可以在逻辑上合并了各自本地 `__shared__` 区域的共享内存空间中读写。

从效果上看，整个簇的共享内存被拼接为一块分布式的片上缓冲区。

由于线程块间的数据全部保留在片上内存中，DSMEM 能显著提升有效算术强度：各线程块交换中间结果或执行归约时，无需往返访问全局内存。当数据集大到单个线程块的共享内存装不下时，这一点尤其有价值。

此时不必退回全局内存加载和存储，而是 launch 一个线程块簇，让其中的线程块各自处理数据的一部分，然后通过 DSMEM 进行同步与共享。所有通信都以 SM 到 SM 的速度进行，从而避免了昂贵的 HBM 流量。

简而言之，DSMEM 是一种比通过全局内存通信更快、也更结构化的线程块间通信方案，但它的作用范围仅限于同一簇内的线程块。DSMEM 非常适合那些需要频繁、低延迟线程块间协调的持久 kernel：LLM 中的注意力机制或其

他多阶段算法——每个线程块都必须在继续执行之前交换中间状态——以及一切受益于把中间结果留在片上、而不是写回全局内存再重新加载的工作负载。

> **注** 可移植的最大簇大小为 8 个线程块。某些 GPU 支持更大的非可移植簇大小（例如最多 16 个 CTA），前提是通过 `cudaFuncAttributeNonPortableClusterSizeAllowed` 属性显式启用。这会增大你的 DSMEM 占用，代价是占用簇中更多的 SM。

10.4.3 暂存内存

scratch memory 是支撑 DSMEM 与线程块簇同步的底层硬件基础设施。DSMEM 是对 CUDA 代码隐式可见的共享数据缓冲区，而 scratch memory 则是一块独立的片上 SRAM 区域，由 GPU 用来记录各类协调元数据，例如 barrier 状态、组进度以及 DSMEM 的访问标志。

你不能在 kernel 中直接访问 scratch memory，它由 GPU 自动管理。launch 线程块簇时，硬件会划出一部分 scratch memory，用来维护簇范围的 barrier 计数器（`cluster.sync()` 状态）、跟踪哪些线程块已经到达 DSMEM 操作或同步点，并在多个 SM 之间协调对分布式共享区域的安全访问。

由于 scratch memory 专门针对这些元数据操作做了优化，簇级 barrier 与 DSMEM 访问可以非常快地完成。若簇非常大或同步模式很复杂，导致元数据规模超出了可用的 scratch SRAM，

GPU 就会透明地把一部分状态溢出到由 L1/L2 缓存支撑的本地内存中。这种溢出保证了正确性，同时仍尽可能保留片上执行的效率。

简而言之，DSMEM 是你用来在线程块之间共享数据的抽象，而 scratch memory 则是让这些 DSMEM 操作快速且可扩展的幕后设施。二者共同让一个线程块簇表现得像一个单一逻辑单元，打破了线程块之间的传统壁垒，从而提升那些需要多个线程块紧密协同并行的工作负载的性能。

10.4.4 启动线程块簇

下面通过一个实例来说明如何在实际中使用线程块簇。第一步，我们需要用一个此前未曾用过的簇维度来 launch kernel。

在 CUDA 中，这一步通过一个允许指定簇大小的扩展 launch API 来完成。例如，如果我们想要由两个线程块组成的簇——也叫线程块对（thread block pair）或 CTA 对（CTA pair）——就可以配置一个特殊的 launch 属性，如下所示:

```
// Host code: launch a kernel with thread block clusters of size 2
cudaLaunchConfig_t config{};
config.gridDim = dim3(128, 1, 1);
config.blockDim = dim3(256, 1, 1);
config.dynamicSmemBytes = 0;
cudaLaunchAttribute attr{};
attr.id = cudaLaunchAttributeClusterDimension;
attr.val.clusterDim.x = 2;
attr.val.clusterDim.y = 1;
attr.val.clusterDim.z = 1;
config.attrs = &attr;
config.numAttrs = 1;
// Allow non-portable cluster sizes if you intend to use > 8 later
cudaFuncSetAttribute(MyClusterKernel,
                     cudaFuncAttributeNonPortableClusterSizeAllowed,
                     1);
cudaLaunchKernelEx(&config, MyClusterKernel, args, nullptr);
```

在这份主机代码中，我们用 `cudaLaunchKernelEx` 来 launch `MyClusterKernel`，每个簇包含 2 个线程块（因此共有 64 个簇，因为 128 个线程块除以每簇 2 个）。`clusterDim` 被设为 2，意味着每个簇将包含 2 个线程块。对于需要簇协作 kernel 的情形，这取代了传统的 `<<<gridDim, blockDim>>>` 语法。在底层，CUDA 运行时会确保这些成对的线程块被调度为可以通过 DSMEM 相互通信。

请记住，每个线程块簇都必须事实地上放入你指定的那些 SM 与资源之中。可移植的最大簇维度是 8 个线程块；若想在受支持的 Blackwell 器件上启动 16

个线程块的簇，就必须启用非可移植属性，并把线程块大小调整到簇能够完整驻留在 SM 上。在设定线程块簇大小时，请务必牢记这一点。

10.4.5 用协作组 API 协调线程块簇

要协调簇内多个线程块的工作，先用 `cooperative_groups::this_cluster()` 取得代表这组线程块的句柄。借助这个簇句柄，你可以执行由硬件加速的簇级 barrier，还可以直接访问另一个线程块的共享内存。

这一切都不需要离开 kernel，也不必借助全局内存里的标志。下面这个示例 kernel，会把每个线程块的一个本地值累加进线程块 0 的共享内存中：

```
#include <cuda_runtime.h>
#include <cooperative_groups.h>
namespace cg = cooperative_groups;
__global__ void MyClusterKernel(/* args */) {
    // 1. Form a cluster group for all thread blocks in this thread
     ↪ block cluster
    cg::cluster_group cluster = cg::this_cluster();
    // 2. Allocate the same extern shared buffer in each block
    extern __shared__ int shared_buffer[];
    // 3. Each block learns its rank and the total cluster size
    int clusterRank = cluster.block_rank();
    int clusterSize = cluster.num_blocks();
    // 4. Initialize this block's portion of shared memory (a simple
     ↪ local sum)
    int localSum = threadIdx.x;
    shared_buffer[threadIdx.x] = localSum;
    // 5. Barrier across all blocks in the cluster; no block proceeds
     ↪ until
    //    every block reaches this point and has written its
     ↪ shared_buffer.
    cluster.sync();
    // 6. Map a pointer to block 0's shared_buffer so block can write
     ↪ into it
    // Pointers returned by cluster.map_shared_rank
```

```
    // refer to the remote CTA's shared memory
    // and support remote atomics
    // and memory operations within
    // the cluster
    // Pair updates with cluster.sync at well defined point
    int* remote_buffer = cluster.map_shared_rank(shared_buffer, 0);


    if (clusterRank != 0) {
        // 7. Nonzero blocks atomically add their local shared_buffer[0]
         ↪ into
        //    rank 0's buffer. This atomicAdd is routed on-chip over
         ↪ DSMEM
        //    (not through DRAM.)
        atomicAdd(&remote_buffer[0], shared_buffer[0]);
    }
    // 8. Another cluster-wide barrier ensures all atomic adds have
     ↪ completed
    cluster.sync();
    // 9. Finally, block 0 (rank 0, thread 0) can read the combined
     ↪ result
    if (clusterRank == 0 && threadIdx.x == 0) {
        printf("Combined sum in cluster[0]: %d\n", shared_buffer[0]);
    }
}
```

这里，我们通过调用 `cg::this_cluster()` 取得簇句柄。它返回一个 `cluster_group` 对象，代表以“同一线程块簇”为单位一起 launch 的那些线程块。运行时保证该簇组内的每个线程块都同时驻留——否则，launch 就会失败。

隐式的统一共享内存分配是默认提供的：簇中的每个线程块都必须为 `extern __shared__ int shared_buffer[]` 声明同样的大小。此后，DSMEM 硬件会把各线程块的共享内存合并为一个虚拟地址空间。并且，因为所有线程块都预留了同一大小的 `shared_buffer`，指向线程块 0（rank 0）缓冲区的指针就能在各 SM 之间正确地协调。

> **注** 使用线程块簇时，请像对待全局内存的访存合并那样来组织每个线程块的 DSMEM 访问。也就是说，让每个 warp 读写连续且 32 字节对齐的扇区。这样一来，硬件就能在簇范围内路由 DSMEM 传输，而不出现 bank 冲突或意外的串行化。实践中，应当这样布置分块数据：让线程块 j 中的第 i 个 warp 总是访问一段唯一的、对齐的区间。这能避免内存 bank 争用，并让 DSMEM 数据传输始终以全速运行。

在前面的示例代码中还可以看到，`cluster.sync()` 被用作跨越簇内所有线程块的簇级 barrier。与只同步单个线程块内线程的 `__syncthreads()` 不同，`cluster.sync()` 同步的是本簇中每个线程块内的所有线程。

这个 barrier 由硬件实现，延迟通常低于网格级同步，因为它只协调簇内的线程块。这意味着，只要线程块位于同一个簇内，你就可以频繁地进行同步，而几乎不会造成性能影响。

此外，由于 CUDA 强制要求整个网格正确 launch 并放得进 GPU，因此不会出现因某些线程块缺失而导致死锁的情况。不存在这样的场景：一部分线程块到达 barrier 后，永远悬等那些从未启动的“缺失”线程块。所有线程块都已就位，一旦每个线程块都调用了这个 barrier，它就会顺利完成。

在上面的代码块中可以看到，跨块的共享内存访问是通过 `map_shared_rank()` 完成的。第一次 barrier 之后，每个块的 `shared_buffer[]` 都已初始化完毕。要获得指向块 0 共享内存的指针，其他块调用 `int* remote_buffer = cluster.map_shared_rank(shared_buffer, 0)`，调用中指明了 cluster 内的线程块 id（0）。

GPU 硬件会自动翻译该指针，使任何加载或存储都经由片上 DSMEM 网络完成，而不会落到全局 DRAM。因此，对 `remote_buffer` 的任何写操作——无论是原子写还是普通写——都会直接在片上更新块 0 的共享内存。值得强调的是，远程 DSMEM 访问的性能特征与本地共享内存不同：远程加载与存储同样受益于合并（coalesced）的对齐 32 字节段。

举例来说，在上面的代码块中，当块 1……n 需要把各自本地值累加进块 0 的共享缓冲区时，它们调用 `atomicAdd(&remote_buffer[0], shared_buffer[0])`。由于 `remote_buffer` 是通过 `cluster.map_shared_rank()` 获得的，这次 `atomicAdd` 会经片上 DSMEM 网

络直接写入块 0 的 SMEM。换言之，整个过程不需要往返 DRAM 或 L2 缓存，每一次写入都以片上速度完成。

在上面的代码示例中可以看到，当所有块都完成各自的 `atomicAdd` 后，第二次 `cluster.sync()` 保证块 0 在继续执行之前，能在自己的 `shared_buffer[0]` 中看到完整的求和结果。

简而言之，`cooperative_groups::this_cluster()` 加上 `cluster.sync()` 与 `cluster.map_shared_rank()`，为同步线程块簇内的多个线程块并共享数据提供了一种简单高效的方式。这些操作全部发生在片上，避免了全局内存往返，并支持块间细粒度协作。线程块簇与 DSMEM 的这一组合，其性能远高于任何基于全局内存的退路方案或手工原子标志方案。

10.4.6 线程块对

较新的 NVIDIA GPU 允许把恰好两个线程块组成一个 cluster 共同调度，即线程块对（thread block pair，又称 CTA pair，即 CTA 对），落在同一 GPC 内的多个 SM 上。把线程块编入 cluster（例如 2 块 cluster，如图 10-13 所示）后，需要共享数据的 kernel 便可用 TMA 把 tile 搬进各块的共享内存。

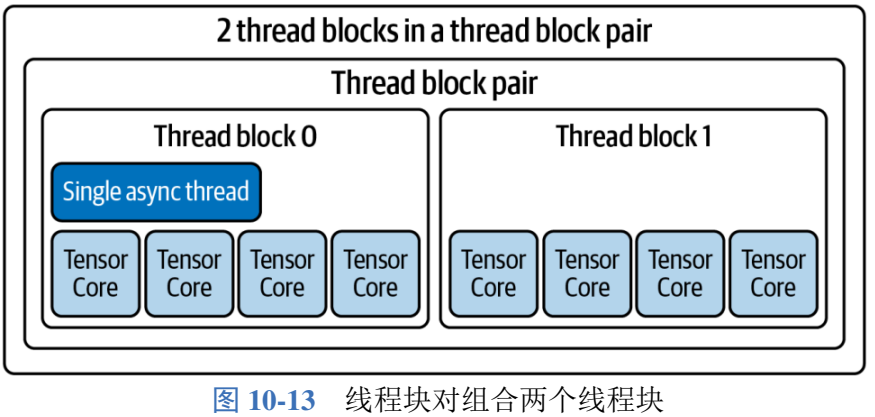

图 10-13　线程块对组合两个线程块

单个线程块可能缺乏独自处理超大 tile（例如 256 × 256 矩阵子分块）所需的寄存器或共享内存容量。通过把同一 GPC 内相邻 SM 上的两个线程块配对，并借助 DSMEM，这两个块可以分担同一个大 tile 的工作，同时经统一的共享内存区域共享数据，如图 10-14 所示。

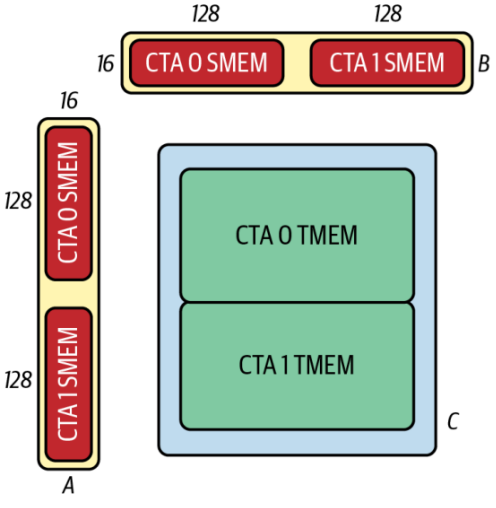

图 10-14 线程块对（CTA pair）把 tile 载入为矩阵乘法 A × B 的操作数（来源：https://oreil.ly/kEZsv）

此时，线程块对中的每个线程块都可以把操作数 tile 的一部分（例如 128 × 16）载入自己的片上 SMEM 用于矩阵乘法。此外，每个线程块还在 Tensor Memory（TMEM）中保存累加器的一部分（例如 128 × 256）。这样，

CTA 对中的两个线程块便能协同处理同一个 tile，如图 10-15 所示。

线程块对可以支持跨越两个物理 SM 的更大规模矩阵乘法。使用一对线程块相当于把 tile 规模扩大一倍，因为每个 SM 只处理 tile 数据的一半。

SM 硬件利用 DSMEM 在 SM 之间共享操作数数据。DSMEM 能减少重复加载、提升数据复用并提高算术强度。图 10-16 展示了线程块簇内 SM 之间基于 DSMEM 的这种数据共享。

针对这类多 SM 操作，CUDA 提供了增强的广播 barrier 与同步原语。线程块对之间使用轻量的 `cluster.sync()` 调用相互同步。这样一来，当一个 SM 用完某个 tile 后，CTA 对中相邻的 SM 就能接着消费它。

每个 tile 送入线程块对时不再产生冗余的全局内存事务，从而更充分地占用原本空闲的寄存器、共享内存 bank 和 Tensor Core。而且，线程块对把可用于同一个 tile 的线程数和共享内存翻了一番，从而获得更高的 Tensor Core 利用率。

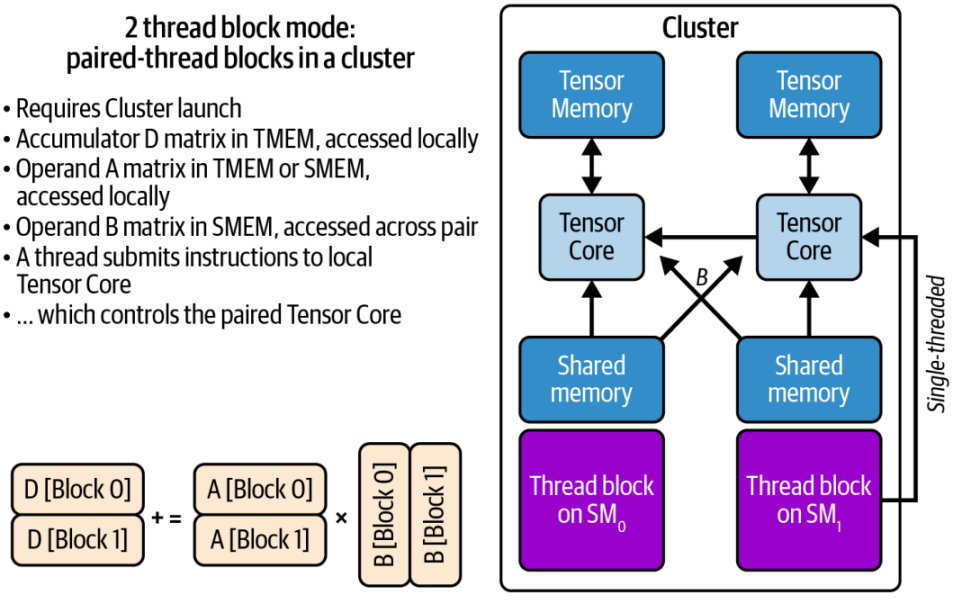

图 10-15　带 Tensor Core 与 TMEM 的线程块对

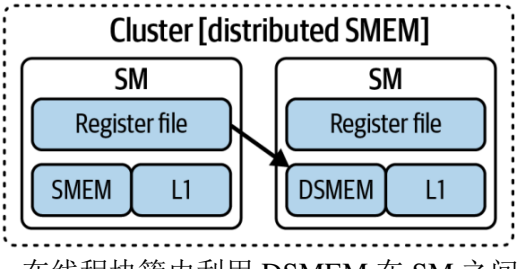

图 10-16　在线程块簇内利用 DSMEM 在 SM 之间共享数据

性能提升是完全透明的。从程序员的视角看，你只需启动一个包含两个块的 cluster，并把它当作一个被拆到两个线程块上的大线程块。例如在 CUTLASS 中，这体现为一种 2-SM UMMA（Unified MMA）GEMM 操作，它利用 DSMEM 和 cluster barrier 实现 CTA 间的数据共享。你只需为单个 GEMM tile 请求两个 SM，CUTLASS 便会自动完成 cluster 启动、DSMEM 设置以及线程块间同步。

如果你请求的 tile 规模超出单个线程块的处理能力（例如 FP16 Tensor Core 操作下的 128 × 128），CUTLASS 会自动分配两个线程块组成线程块对。两个 SM 分担工作，并在底层依靠 DSMEM 加 `cluster.sync()` 来共享该 tile。这样，你就可以结合 DSMEM 对 UMMA 操作做成对重叠。

简言之，线程块对与 DSMEM 以多 SM 协作的形式提供并行能力，支持线程块之间快速的片上数据共享与同步。这消除了许多过去必须依赖全局内存交接或额外 kernel 启动的场景，使众多多线程块算法受益，并简化了它们的实现。

10.5 利用线程块簇减少全局内存流量

从性能角度看，DSMEM 能显著削减冗余的全局内存流量，带来更高的有效带宽。例如，在分块 GEMM 中，当 cluster 内多个线程块共享 A 或 B 矩阵的分块时，一个块可以从全局内存加载 tile，再用 TMA 把该 tile 组播（multicast）给 cluster 中的其他块。

TMA 引擎支持一种组播复制模式：当各线程块属于同一 cluster 时，数据可直接送入 DSMEM。一次来自全局内存的 TMA 传输就能同时把数据放入每个参与块的共享内存，从而避免冗余的 DRAM 读取。

借助 TMA 组播，GPU 保证了 L2 缓存中的数据一步广播到 cluster 每个成员（线程块）的共享内存。这样即可避免对同一 tile 反复发起全局内存加载，从而提升带宽利用率并压缩 DRAM 流量——当许多块需要相同输入时尤其明显，如图 10-17 所示。

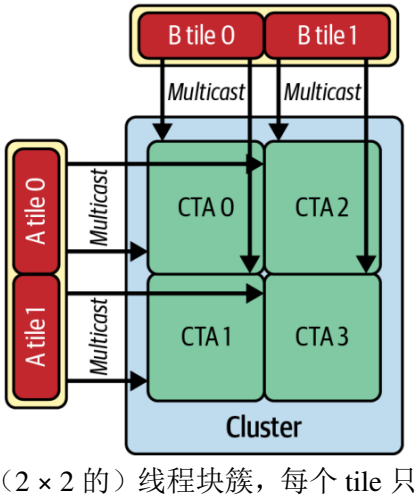

图 10-17　对于这四个（2 × 2 的）线程块簇，每个 tile 只加载一次，并组播到簇内所有 CTA 的共享内存（来源：https://oreil.ly/kEZsv）

此时，TMA 引擎从全局内存向 DSMEM 执行一次组播，把 tile 广播给 cluster 中每个线程块的 SMEM，消除冗余的 DRAM 读取。TMA 组播通过张量映射描述符（tensor-map descriptor）配置，并作为面向 `shared::cluster` 的 `cp.async.bulk.tensor` 指令下发。幸运的是，CUTLASS/cuTe 和 Triton 等更高级的库会替你生成这些组播操作、张量映射描述符和批量张量拷贝。具体而言，这些库可以下发相关 PTX 指令，包括带张量映射操作数的 `cp.async.bulk.tensor`，以及面向 `shared::cluster` 的组播指令。在这些情形下，硬件会把 tile 送达每个线程块的 SMEM。

另一个常见用例是多块归约（reduction）或扫描（scan）。与其让每个块都把自己的部分和或扫描结果写到全局内存、再启动一个单独的 kernel 来合并，不如让线程块簇中的各线程块把部分结果直接写入 DSMEM。

在同一个 cluster 内，利用共享内存和 cluster 级 barrier `cluster.sync()`，即可对这些部分结果执行快速的片上归约或前缀和。只有最终结果需要写入全局内存，这大大减少了全局内存的读写。

把线程块簇、DSMEM 与 TMA 组播结合起来，就能构建多阶段、细粒度的流水线，使大部分数据交换都留在片上完成。无论是为 GEMM 共享 tile、为归约累加部分和，还是执行多块扫描，这些机制都能让你把 HBM 往返降到最低、把算术强度提到最高。

> **注** 块作用域的 `cuda::pipeline` 不会同步其他块；cluster 级的数据分发要使用 TMA 组播，或使用配合 `cluster.sync` 的 DSMEM。

设有两个线程块 CTA 0 与 CTA 1，它们都需要矩阵 A 的同一个 tile。在没有 DSMEM 的情况下，每个 CTA 都会各自发起全局内存加载，把同样的数据读取两次，白白浪费 DRAM 带宽。

而有了 DSMEM，CTA 0 只需把该 tile 一次性载入自己的共享内存，再经片上 DSMEM 网络组播出去，CTA 1 即可直接从共享内存读取。当 cluster 由两个以上的 CTA 组成时，同一个 tile 会在簇内所有线程块之间共享，而全局 HBM 加载仍然只需一次。表 10-4 对比了使用两个线程块组成的 cluster 与两个独立线程块的情形。

表 10-4　DSMEM 在两块工作负载上的性能影响。

指标	两个独立 CTA（无 DSMEM）	带 DSMEM 的 CTA 对（2 块 cluster）
全局加载事务	2×（每个线程块各加载一个 tile）	1×（tile 只加载一次）
L2 缓存命中率	50%	85%
CTA 间数据复用	无（无复用）	显著（tile 被 CTA 1 复用）
每 CTA 有效 DRAM 带宽	300 GB/s	降低 50%（150 GB/s）
kernel 耗时（相对值）	1.0×	0.6×（40% 提速）

两个相互独立的 CTA（不使用 DSMEM）与采用 DSMEM 的 CTA 对（2 个 CTA 组成的 cluster）的对比：全局加载事务——前者为 2 倍（每个线程块各自加载 tile），后者为 1 倍（tile 只加载一次）；L2 缓存命中率——前者为 50%，后者为 85%；CTA 间数据复用——前者无从谈起，后者复用显著（tile 被 CTA 1 复用）；每个 CTA 分摊到的有效 DRAM 带宽——前者为 300 GB/s，后者减半至 150 GB/s；kernel 相对执行时间——前者为 1.0 倍，后者为 0.6 倍（提速 40%）。

这里可以看到，采用带有 DSMEM 的线程块对（即 CTA 对）后，kernel 执行可获得 40% 的加速。这一结果与带宽节省的情况相符：由于每个 tile 只被获取一次，每个线程块的 DRAM 带宽从 300 GB/s 下降了 50%，降至 150 GB/s。同时，由于 TMA 硬件在片上执行多播复制，从而避免了对全局内存的多次读取，L2 命中率从 50% 跃升至 85%。

简而言之，通过多播共享数据，线程块簇让多个块能够以片上速度复用数据，这对内存受限型工作负载可带来可观的加速。

需要注意的是，DSMEM 与 L2 是并行工作的，这为块间数据共享提供了两条“车道”。这种双路径设计将 DSMEM 的超快簇内通信与 L2 更广的缓存覆盖范围结合了起来。换言之，对于簇内局部地址，DSMEM 访问会绕过 L2 缓存，转而使用专用的 SM 到 SM 网络，如图 10-18 所示。

远程 DSMEM 访问经由线程块簇互连进行路由，与全局内存流量彼此独立。例如，当某个线程块通过 DSMEM 取回一个 tile 时，它使用的是低延迟的共享内存传输。这就保证了线程块簇内部的块间数据共享与片上共享内存一样快。

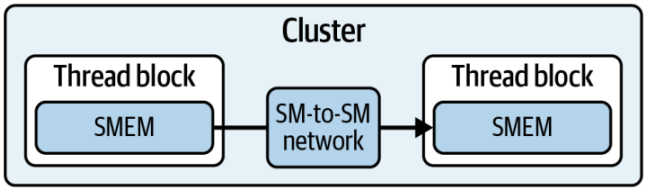

图 10-18　DSMEM 使用线程块簇本地的 SM 到 SM 网络，在两个线程块簇之间交换数据

如果 tile 不在 DSMEM 中，就必须从全局内存取回（沿常规缓存层级进行，例如可能命中 L2）。这样一来，即便某个 tile 已被从 DSMEM 中逐出，线程块仍可以从 L2 缓存取回该 tile 数据，而不必一路回退到全局 DRAM。

其结果是：内存停顿很少发生，占用率保持高位，整体执行速度显著快于未优化时“两个相互独立、不使用 DSMEM 的线程块”的实现。

10.6 利用线程块簇设计高效算法

线程块簇为那些过去必须依赖全局内存通信或多次 kernel 启动的工作负载提供了新的并行化策略。例如，设想一个大型矩阵乘法，由于共享内存容量的限制，输出 tile 之大已非单个线程块所能处理。

在过去，你可能把工作拆给两个线程块，但那样每个块都得通过全局内存交换部分结果——这既慢又低效；否则，你就得另外启动一个归约 kernel 来合并两个线程块的输出。

有了线程块簇和 DSMEM，这两个块可以组成一个簇，直接共享片上共享内存的同一块区域，借助硬件支持的原语无缝合并各自的结果。DSMEM 硬件允许一个 SM

通过高速网络对另一个 SM 的共享内存执行加载/存储/原子操作。

精心设计算法以高效使用线程块簇十分重要。同步开销虽然很低，但并非为零，过于细粒度的数据共享可能得不偿失。

线程块簇的栅障与第 7 章介绍的 warp 级内建函数（例如 __shfl_sync()）一样，要求所有参与者必须同时到达同步点。调用 cluster.sync() 时，簇中的所有线程块都必须执行到这一行，之后任何块才能继续。

如果某个块提前完成工作，它只会原地等待；而如果某个块因线程走了不同分支等原因永远到不了栅障，整个簇就会死锁。

换言之，正如 warp 内建函数要求一条 warp 中的所有线程走相同的指令路径以避免分支发散一样，线程块簇要求所有块在直至每次 `cluster.sync()` 调用处都遵循相同的控制流。

正因为这种齐步前进（lockstep）的要求，块间细粒度数据共享必须与同步开销及死锁风险加以权衡。只要写法正确，同步本身开销很小；但哪怕只有一个块绕过或延迟到达一次 `cluster.sync()`，性能就可能受影响——更糟的情况下，`kernel` 甚至可能挂起。

应当把代码组织成每个块都齐步到达每一个栅障——就像使用 `warp` 级内建函数时一条 warp 中的每个线程都必须同时到达栅障一样。这是在不落入死锁的前提下充分享受线程块簇低延迟片上通信的关键。

一般而言，当每个块都拥有相当规模、可独立运行直到需要同步或数据交换点的工作量时，线程块簇的表现最好。同步完成、数据在片上传输完毕后，各线程块即可继续处理。

线程块簇对块稀疏矩阵运算尤其有效——这类运算在 LLM 的稀疏注意力、模型剪枝和压缩中很常见。此时各块处理互不重叠的非零区域，并共享彼此的边界数据。

线程块簇同样适用于多阶段归约，例如 `transformer` 层中的 `softmax` 与归一化步骤。它们都需要一个最终合并步骤，把簇内各线程块的部分结果汇总起来。

更一般地说，当大型 `GEMM` 超出单个块的资源上限时，也会从线程块簇中获益。`GEMM` 正是现代 LLM 中 `transformer` 注意力层、多层感知机（`MLP`）层以及 embedding 查询的核心运算。

然而，更大的簇规模会降低整体占用率。例如，一个由 16 个块组成的簇可能让 16 个 SM 被同一任务独占，留给该 `kernel` 启动中其他任务的 SM 就更少了。建议从较小的簇（2 块或 4 块簇）起步，除非特殊情况确需更大的簇。与往常一样，应针对具体工作负载进行性能分析，确认跨线程块簇共享片上资源的收益确实大于并行度可能出现的损失。

> **注** 使用簇启动时，应通过 Nsight Compute 启动统计和标准占用率 API（如 `cudaOccupancyMaxActiveBlocksPerMultiprocessor()`）核查活跃块数与簇的驻留情况。对于非 portable 的簇规模，需设置 `cudaFuncAttributeNonPortableClusterSizeAllowed`（或在 `cudaLaunchKernelEx()` 的属性中传入 `cudaLaunchAttributeNonPortableClusterSizeAllowed`），否则启动可能失败，或测得的占用率偏低。

10.7 基于线程块簇的 Warp 专业化

现在我们结合线程块簇与 CUDA Pipeline API 重新审视 warp 专业化。我们用块级流水线充当簇主（cluster leader），负责搬运数据，并用 `DSMEM` 执行簇级栅障。这样，簇中的每个块都消费相同的输入 tile，而无需各自从全局内存重复加载。

角色在每个块内部按 warp 指派。主块的加载 warp 通过协作复制把 tile 写入共享内存，每个 tile 只做一次；簇级栅障把数据发布出去之后，每个块都用一个计算 warp 经 `DSMEM` 读取主块的 tile，并负责计算互不重叠的一段行。每个块中的存储 warp 再把这段行写回全局内存。块级流水线仅由主块用于异步复制，其他块不在该流水线上等待。代码如下：

```
// Warp specialization across a thread-block cluster
// using DSMEM and a block-scoped pipeline
#include <cuda/pipeline>
#include <cooperative_groups.h>
#include <algorithm>
namespace cg = cooperative_groups;
#define TILE_SIZE 128
#define TILE_ELEMS (TILE_SIZE * TILE_SIZE)
// Compute a band of rows of the TILE_SIZE×TILE_SIZE product from DSMEM
 ↪  sources.
// Each lane processes rows [row_begin, row_end) in a 32-way striped
 ↪  loop.
__device__ void compute_rows_from_ds(const float* __restrict__ A_src,
```

```
                                    const float* __restrict__ B_src,
                                    float* __restrict__ C_dst,
                                    int row_begin, int row_end,
                                    int lane_id) {
    for (int row = row_begin + lane_id; row < row_end; row += warpSize) {
        for (int col = 0; col < TILE_SIZE; ++col) {
            float acc = 0.0f;
            #pragma unroll
            for (int k = 0; k < TILE_SIZE; ++k) {
                acc += A_src[row * TILE_SIZE + k] * B_src[k * TILE_SIZE
                ↪  + col];
            }
            C_dst[row * TILE_SIZE + col] = acc;
        }
    }
}
extern "C"
__global__ void warp_specialized_cluster_pipeline(
    const float* __restrict__ A_global,
    const float* __restrict__ B_global,
    float* __restrict__ C_global,
    int numTiles) {
    thread_block cta = this_thread_block();
    cluster_group  cluster = this_cluster();
    extern __shared__ float shared_mem[];
    float* A_tile_local = shared_mem;
    float* B_tile_local = A_tile_local + TILE_ELEMS;
    float* C_tile_local = B_tile_local + TILE_ELEMS;
    // Block-scoped pipeline used only by the cluster leader
    // to stage asynchronous copies
    __shared__
    cuda::pipeline_shared_state<cuda::thread_scope_block, 2> pipe_state;
    auto pipe = cuda::make_pipeline(cta, &pipe_state);
    const int lane_id = threadIdx.x & 31;
    const int warp_id = threadIdx.x >> 5;
    const int cluster_rank      = cluster.block_rank();
```

```
    const dim3 cluster_dims     = cluster.dim_blocks();
    const int  blocks_in_cluster = cluster_dims.x * cluster_dims.y *
                                   cluster_dims.z;
    // 1D cluster arrangement along x;
    // each iteration processes one tile per cluster
    auto loader = cooperative_groups::tiled_partition<32>(cta);
    for (int tile = blockIdx.x / cluster_dims.x; tile < numTiles;
         tile += gridDim.x / cluster_dims.x) {
        const size_t offset = static_cast<size_t>(tile) * TILE_ELEMS;
        // Leader block' s loader warp stages A and B once for the
        ↪ entire cluster


        if (cluster_rank == 0 && warp_id == 0) {
            pipe.producer_acquire();
            cuda::memcpy_async(loader, A_tile, A_global + offset,
                               cuda::aligned_size_t<32>{TILE_ELEMS*sizeo
                                 ↪ f(float)},
                               pipe);
            cuda::memcpy_async(loader, B_tile, B_global + offset,
                               cuda::aligned_size_t<32>{TILE_ELEMS*sizeo
                                 ↪ f(float)},
                               pipe);
            pipe.producer_commit();
            // Make loads visible to leader CTA before publishing
            ↪ cluster-wide
            // wait for committed stage before publishin
pipe.consumer_wait();
            pipe.consumer_release();
        }
        // Publish the leader' s tiles to every block via DSMEM
        cluster.sync();
        const float* A_src = cluster.map_shared_rank(A_tile_local, 0);
        const float* B_src = cluster.map_shared_rank(B_tile_local, 0);
        // Divide rows among blocks in the cluster
        const int rows_per_block = (TILE_SIZE + blocks_in_cluster - 1)
                                   / blocks_in_cluster;
```

```
        const int row_begin = std::min(cluster_rank * rows_per_block,
         ↪ TILE_SIZE);
        const int row_end   = std::min(row_begin + rows_per_block,
         ↪ TILE_SIZE);
        // Compute warp produces block' s band of rows into local shared
         ↪ memory
        if (warp_id == 1) {
            pipe.producer_acquire();
            compute_rows_from_ds(A_src, B_src, C_tile_local, row_begin,
             ↪ row_end,
                                 lane_id);
            pipe.producer_commit(); // publish C_tile_local
        }
        // Storer warp writes this block' s rows back to global memory
        if (warp_id == 2) {
            pipe.consumer_wait(); // observe C_tile_local
            for (int row = row_begin + lane_id; row < row_end; row +=
             ↪ warpSize) {
                for (int col = 0; col < TILE_SIZE; ++col) {
                    C_global[offset + row * TILE_SIZE + col] =
                        C_tile_local[row * TILE_SIZE + col];
                }
            }
            pipe.consumer_release();
        }
        // All blocks finish this tile before the leader reuses its
         ↪ buffers
        cluster.sync();
    }
    // dynamic shared memory size: 3 * TILE_ELEMS * sizeof(float)
}
```

这里，主块使用块级流水线对每个 tile 执行一次成对的协作复制，将其写入自己的共享内存；簇级栅障随后通过 DSMEM 让数据对所有簇成员可见。

这种“复制一次、经 DSMEM 共享”的模式在簇栅障之后，通过 DSMEM 和 map_shared_rank 共享主块的 tile。它并非我们先前讲解 tensor-map 描述符、cp.async.bulk.tensor 和 shared::cluster 时所用的 TMA 多播。理解这一区别非常重要：在 DSMEM 模式中，如果没有 cluster.sync()，跟随块可能读到主块 SMEM 中的过期数据。因此，在 map_shared_rank() 之前必须先调用 cluster.sync()。

> **注** 以下几点有助于在 TMA 多播与 DSMEM 共享之间做出选择：若每块的 tile 复用次数很高（例如每个块多次访问同一 tile），或簇规模（C）较大（8–16 个 CTA），TMA 多播通常更胜一筹，因为这种模式写入一次、随后多次本地读取；若 SMEM 紧张（例如 tile 很大），或簇规模（C）较小（2–4 个 CTA）且每个跟随块只访问 tile 一次，则 DSMEM 共享通常是更好的选择，因为它占用空间更小。

每个线程块通过 map_shared_rank 直接从领导者线程块的共享内存读取数据，计算各自独占的一段行带，并将结果写入全局内存。这消除了簇内重复的全局加载，同时在工作最关键的环节保留了流水线的重叠优势。

领导者块的加载 warp 调用 producer_commit()，将其本地阶段的数据发布给本块。其他线程块并不等待领导者的流水线，而是在簇级 barrier 之后观察到领导者已装入的分块（tile），随后便从分布式共享内存中读取。

这使得分块的加载、计算与存储可以在多个线程块之间交错进行，从而将 SM 的利用率推向更高水平。表 10-5 将单线程块的 warp 专用化 kernel 与本节给出的线程块簇（多块）warp 专用化 kernel 进行了对比。

指标　朴素分块

线程块簇流水线（warp_specialized_cluster_pipeline）　内核执行时间

两级双缓冲（double_buffer_pipeline）

```
Warp-specialized
(warp_special
ized_pipeline)
```

表 10-5 朴素分块、两级双缓冲、warp 专用化与线程块簇流水线 kernel 的对比。

指标	朴素分块	两级双缓冲（`double_buffer_pipeline`）	warp 专用化（`warp_specialized_pipeline`）	线程块簇流水线（`warp_specialized_cluster_pipeline`）
内核执行时间	41.3 ms	20.5 ms（较朴素分块快 2.01×）	18.4 ms（较两级双缓冲提速 10.2%）	17.2 ms（较 warp 专用化提速 6.5%）
warp 执行效率	68%	92%（较朴素分块提升 24%）	96%（较两级双缓冲提升 4%）	97%（较 warp 专用化提升 1%）
warp 状态停顿比例（等待共享内存与 barrier 时）	高	低	极少	极少（进一步降低）
L2 吞吐量	80 GB/s	155 GB/s（较朴素分块提升 94%）	165 GB/s（较两级双缓冲提升 6.45%）	170 GB/s（较 warp 专用化提升 3%）
吞吐量可扩展性	可扩展至每 SM 2–3 个 warp	可扩展至约每 SM 6 个 warp	接近线性扩展至 SM warp 上限（Blackwell 上每 SM 64 个常驻 warp）	跨线程块充分扩展，直至受限于 Blackwell 上每 SM 64 个常驻 warp 的 SM warp 上限
DRAM 读取吞吐量与 kernel 时长的关系	重叠差	重叠好	重叠极佳	重叠极佳（即使在线程块交接时也如此）
指令数	1.7 B	1.05 B（较朴素分块减少 38%）	约 1.00 B（较两级双缓冲减少 4.76%）	约 0.98 B（较 warp 专用化减少 2%）

17.2 ms（较 warp 专用化提速 6.5%）　warp 执行效率

41.3 ms　20.5 ms（较朴素分块快 2.01×）

18.4 ms（较两级双缓冲提速 10.2%）

96%（较两级双缓冲提升 4%）　97%（较 warp 专用化提升 1%）　warp 状态停顿比例（等待共享内存与 barrier 时）

68%　92%（较朴素分块提升 24%）

高　低　极少　极少（进一步降低）

L2 吞吐量　80 GB/s　155 GB/s（较朴素分块提升 94%）

170 GB/s（较 warp 专用化提升 3%）　吞吐量可扩展性

165 GB/s（较两级双缓冲提升 6.45%）

跨线程块充分扩展，直至受限于 Blackwell 上每 SM 64 个常驻 warp 的 SM warp 上限　DRAM 读取吞吐量与 kernel 时长的关系

可扩展至每 SM 2–3 个 warp

可扩展至约每 SM 6 个 warp　接近线性扩展至 SM warp 上限（Blackwell 上每 SM 64 个常驻 warp）

重叠差

重叠好　重叠极佳　重叠极佳（即使在线程块交接时也如此）

指令数　1.7 B　1.05 B（较朴素分块减少 38%）

约 1.00 B（较两级双缓冲减少 4.76%）

约 0.98 B（较 warp 专用化减少 2%）

从这里可以看到，线程块簇实现通过在单次协作启动（cooperative launch）中把加载者、计算者与存储者的角色分散到多个线程块上，进一步改进了 warp 专用化方案。这提高了 SM 的整体利用率，同时降低了执行时间与冗余内存流量。

具体而言，线程块簇流水线 kernel 实现（`warp_specialized_cluster_pipeline`）在单线程块 warp 专用化 kernel 的基础上又争取到约 1.2 ms（6.5%）的提升。这是因为线程块簇版本在所有线程块之间交错进行分块的加载、计算与存储。

SM 占用率达到约 97%，因为当一个线程块所在的 SM 出现空闲时，可以由另一个执行不同流水线阶段的线程块让它保持忙碌。得益于更好的共享内存复用与更少的冗余加载，L2 加载吞吐量峰值约为 170 GB/s。因此，该簇能够避免重复的全局读取。使用 DSMEM 时，领导者块只加载一次分块，其他块在调用 `cluster.sync()` 之后通过 `map_shared_rank()` 读取它。使用 TMA 组播时，从全局内存出发的一次张量拷贝可以直接广播到每个块的共享内存中。

指令数降至约 0.98 B，因为跨线程块的流水线进一步减少了因冗余全局读取而浪费的周期。吞吐量可扩展性同样得以保持：只要拥有足够的线程块和 warp，流水线就能让它们全部保持饱和，直至达到 GPU 的每 SM warp 数硬件上限，即 Blackwell 上的每 SM 64 个 warp。

需要注意的是，单线程块版本与线程块簇版本的 warp 专用化最终都会撞上同一个“每 SM warp 数”上限，但它们为这些 warp 提供数据以及进行同步的方式有所不同。如表 10-5 所示，这一差异为线程块簇实现带来了可测量的性能优势。

具体来说，在单线程块的 warp 专用化 kernel 中，每个线程块恰好分配三个 warp：一个把大小为一个分块的 A 与 B 数据块加载到共享内存，一个对该数据块进行计算，另一个把结果写回全局内存。这三个 warp 完成各自的角色后，线程块便转向下一个分块，如此循环往复。

即便每个 SM 上有三个分别负责加载、计算和存储的活跃 warp，也并未让 SM 的 warp 上限达到饱和。现代 GPU 每 SM 最多允许 64 个并发 warp，而实际驻留数量受寄存器、共享内存以及每 SM 线程块数的限制。

在总活跃 warp 数相近的情况下，单线程块 warp 专用化设计与协作式网格可以表现出相近的占用率，但内存行为却不同。在单线程块设计中，每个块都会为其接触到的每个输入分块各自取一份拷贝。当多个块几乎同时复用同一个分块时，设备就会执行重复读取，消耗 L2 容量和 HBM 带宽。

线程块簇改变了这一模式。簇中的线程块可以通过分布式共享内存共享数据，也可以接收一次组播，把全局内存中的分块直接送入簇内所有块的共享内存。可以在领导者块中使用块级流水线，并配合簇同步与 `map_shared_rank`；若更倾向于从全局内存广播，则可配置为组播模式。这消除了簇内的重复加载，并减轻了 L2 与 HBM 的压力。

`cuda::pipeline` 的作用域仅限于创建它的线程块。某个线程块中的 `producer_commit()` 并不会使其他线程块中的 `consumer_wait()` 解除等待。若要在集群（cluster）中的多个线程块之间进行协调，应先发布数据（例如发布到领导块的共享内存），然后调用 `cluster.sync()`。此后，各跟随块可以使用 `map_shared_rank()` 访问领导块的共享内存。可选的 TMA 多播（multicast）机制还能把数据从全局内存直接广播到每个线程块的共享内存中。

此时，每个线程块都能通过 DSMEM（或 TMA 多播）观测到领导块的瓦片并继续执行。块级流水线本身并不会显式同步其他线程块；集群范围内的可见性由 `cluster.sync()` 与 DSMEM 语义来保证。

当按照高效利用 TMA 多播和 DSMEM 的方式设计时，线程块集群流水线对每个瓦片只会从全局内存读取一次，然后多播到每个线程块的共享内存中，而不是让各个线程块各自重复加载同一份数据。这样一来，尽管每个 SM 仍然不能超过其 64 个 warp 的上限，但线程块集群的全局协调能确保这些 warp 花在重复加载上的时间大大减少，而把更多时间用于真正的计算。

此外，在单线程块版本中，每个块一旦算完当前瓦片就立即加载下一个瓦片。如果某个线程块比另一个稍早完成计算，它会立刻发出自己对下一个瓦片的加载请求——即使该瓦片正被另一个块加载。这会造成冗余的内存流量。

而在线程块集群流水线版本中，如果 SM 0 上的某个计算 warp 提前完成，它并不会立即去取下一个瓦片，而是停在 `consumer_wait()` 处，直到全局加载器——可能在 SM 7 上运行——把该瓦片载入共享内存、供所有线程块使用为止。换句话说，计算 warp 等待的是集群内唯一一次共享的加载，而不是自己再发出一份冗余副本。

通过暂停等待，直到确认瓦片确实可用，参与线程块集群的每个 SM 都避免了空转或重复加载。这种跨线程块集群的对齐拉平了加载时间的波动，并让每个 SM 的计算 warp 持续忙于处理每个集群只加载一次的数据，从而提升整体吞吐量。

设想一个问题规模极大、网格中排满了任务，以至于在单块方案下每个 SM 都已经同时运行着加载、计算和存储 warp。此时两种 kernel 的原始 warp 数量和单 SM 占用率可能看起来相差无几。集群版本的价值体现在多个块复用相同输入瓦片的场景中：它消除了集群内部的重复全局读取，并利用集群级同步来对齐各个阶段。

这在重复用的负载上通常能带来适度的吞吐量提升，但收益取决于瓦片复用程度、对齐情况以及资源限制。两种设计都受同样的架构限制约束，例如常驻 warp 数、寄存器和共享内存。

本章全部 kernel 在生产者与消费者交接时都使用 CUDA 流水线，而不调用块级 `__syncthreads`。朴素的分块 kernel 不重叠内存与计算。两段式双缓冲流水线则使拷贝与计算重叠，在内存受限的情形下可以显著缩短求解时间。

warp 专业化（warp specialization）流水线为加载、计算和存储分派不同的 warp，以避免隐式的整块等待。线程块集群变体通过分布式共享内存或多播在集群内的各块之间共享瓦片，使每个瓦片在集群内只取一次，而不是每个块各取一次。这些模式提升了利用率并减少了冗余内存流量。

10.8 本章要点

以下是本章的核心要点。本章聚焦于在现代 GPU 上榨取峰值性能，这些要点可以避免 kernel 在 DRAM、warp 调度器和全局 barrier 上空转：

用流水线深度隐藏延迟

使用两段式分块（`cuda::pipeline_shared_state<cuda::thread_scope_block, 2>`）使异步加载与计算重叠；当计算开销超过内存开销时（例如 Blackwell 等现代 GPU），再增加一段（`cuda::pipeline_shared_state<cuda::thread_scope_block, 3>`）。这有助于消除空闲 warp。

用 warp 专业化均衡负载

当计算阶段占主导时，为加载、计算和存储各自分配独立的 warp，从而在现代 GPU 硬件上获得接近峰值的 warp 效率。

用常驻 kernel 消除启动开销

让单个长期驻留的 kernel 在设备端工作队列上循环执行，并用 `grid.sync()` 支持多阶段算法。这将减少主机—设备往返次数并降低整体启动成本。

用线程块集群与 DSMEM 实现片上共享

把线程块（CTA）组织成集群，使它们共享一段连续的片上缓冲区。要做集群级广播，可将 `cp.async.bulk.tensor` 与 TMA 多播描述符配合使用；由 TMA 多播把瓦片一次性广播到每个线程块。这将提高 L2 命中率并减少 DRAM 带宽占用。

尤其要注意 barrier 语义

`cluster.sync()` 和 `grid.sync()` 都要求每个参与的线程块到达同一同步点。控制流不匹配——或集群规模过大——可能导致死锁或启动失败。

先性能分析，再调优

使用 Nsight Compute 判断你的 kernel 是内存受限还是计算受限。若是内存受限，先从两段式流水线入手；若是计算受限，可考虑 warp 专业化或线程块集群。

手工调优前，先验证编译器生成的流水线

对 PyTorch 编译器和 NVCC 等编译器生成的框架代码进行性能分析与检查。如果编译后的代码已经使用基于 `cuda::memcpy_async`、`producer_commit()` 和 `consumer_wait()` 的异步流水线，那么手工调优多半不会带来多少加速。

10.9 结语

本章讨论的技术帮助系统地隐藏延迟、消除冗余加载，使 GPU 在 kernel 执行期间保持接近峰值的利用率。

warp 专业化流水线让加载、计算与存储操作相互重叠。协作组 barrier（`grid.sync()` 和 `cluster.sync()`）无需主机往返即可协调多阶段工作。而常驻 kernel 循环执行设备端队列上的任务，从而消除启动开销。

一如既往，应从性能分析入手。如果全局内存停顿占主导，双缓冲这类两段式异步拷贝流水线通常就足够了。如果计算 warp 仍然停顿，则改用多阶段 warp 专业化流水线，让加载、计算和存储 warp 在最小的竞争下运作，此时唯一的竞争只发生在内存子系统上。

> **注** 请记住，并发的收益以硬件（例如内存带宽）饱和为上限；超过这个限度，并行任务就会互相争夺吞吐量。

对于多阶段归约或不规则任务，用单个常驻 kernel 加 `grid.sync()` 取代多次启动，以保持占用率。当多个线程块需要相同的数据时（例如多头注意力），可以组建线程块集群，让 DSMEM 和 TMA 对每个瓦片只加载一次——并把数

据多播给其他线程块——而不必反复访问全局内存。这些技术会把性能进一步推向 GPU 的理论峰值。

在继续阅读后续章节时，请牢记这些原理，因为它们适用于更多优化场景。具体而言，应当在 warp 和线程块层级上重叠工作、直接在设备端（而非主机端）同步，并在片上共享数据。这些是调优超高性能 GPU 负载的关键机制。

在下一章中，我们将保留这些 kernel 内构建块——`cuda::pipeline` 双缓冲、warp 专业化角色和线程块集群——并展示如何通过 CUDA stream 驱动它们，目标是隐藏 kernel 之间

以及主机与设备之间的通信，而不仅仅是单个 kernel 内部。具体而言，我们将复用本章的 kernel，并借助 `cudaMemcpyAsync`、`cudaMallocAsync/cudaFreeAsync` 以及基于事件的同步，把它们组织成多流流水线来运行。这将有助于把整个系统推向峰值性能，在你的 AI 系统中跨多块 GPU 得以实现。

第 11 章 内核间流水线、同步和 CUDA 流排序内存分配

到目前为止，我们重点关注内核内工具——cuda::pipeline 双缓冲、扭曲专门化（加载器/计算/存储扭曲）、持久内核和带有 DSMEM/TMA 的线程块集群——以使 SM 忙于单个内核。在本章中，我们保留这些内核，并展示如何使用 CUDA 流、事件和流排序内存分配器跨内核和批处理进行管道传输。简而言之，第 10 章重点讨论隐藏内核中的延迟。本章介绍如何隐藏内核之间以及 GPU 和主机之间的延迟。

这种内核间并发对于保持所有 GPU 引擎忙于实际工作负载至关重要。为了利用现代 GPU 实现 GPU 的峰值利用率，我们需要保持 GPU 的计算引擎和直接内存访问 (DMA) 引擎繁忙并并行运行。

CUDA 流为这种内核间并发性提供了基础。通过结合异步内存操作、细粒度同步和 CUDA 图（本章简要介绍，下一章更详细介绍），您可以构建避免主机端停顿的高效管道。

11.1 使用 CUDA 流重叠内核执行

CUDA 流是一系列操作（内核启动、内存复制和内存分配），它们按照发出的顺序执行。考虑使用 2 个流将 5 个内核从 CPU 启动到 GPU，如图 11-1 所示。

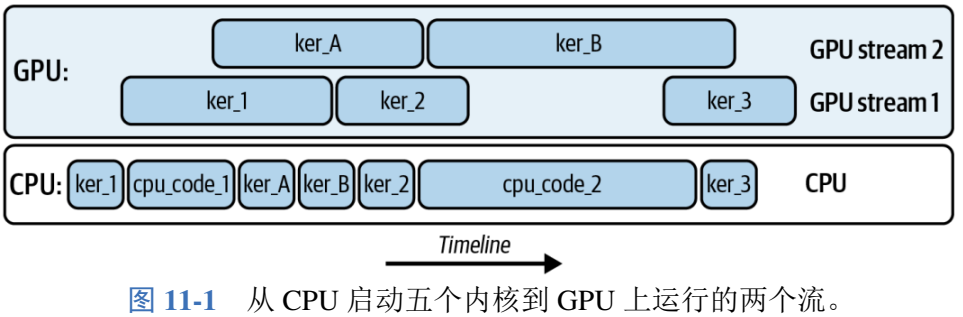

图 11-1 从 CPU 启动五个内核到 GPU 上运行的两个流。

在这里，我们看到 ker_A 和 ker_B 在流 2 上运行，而 ker_1、ker_2 和 ker_3 在流 1 上运行。只要硬件资源允许，所有内核都可以彼此重叠（并且跨 CUDA 流）。

CPU 能够继续执行工作（cpu_code_1 和 cpu_code_2），而流异步执行内核操作。在两个 CUDA 流上启动这五个内核的代码如下所示：

```
#include <cstdio>
#include <cuda_runtime.h>
__global__ void ker_A()  { /* ... do some work ... */ }
__global__ void ker_B()  { /* ... do some work ... */ }
__global__ void ker_1()  { /* ... do some work ... */ }
__global__ void ker_2()  { /* ... do some work ... */ }
__global__ void ker_3()  { /* ... do some work ... */ }
int main() {
    // 1) Create two CUDA streams
    cudaStream_t stream1, stream2;
    cudaStreamCreateWithFlags(&stream1, cudaStreamNonBlocking);
    cudaStreamCreateWithFlags(&stream2, cudaStreamNonBlocking);
    // 2) Define your grid/block sizes
    dim3 grid(128);
    dim3 block(256);
    // 3) Launch ker_1 on stream1
    ker_1<<<grid, block, 0, stream1>>>();
    // 4) CPU code 1 runs immediately (asynchronously wrt GPU)
    printf("CPU code 1 executing\n");
    // ... do some host-side work here ...
    cpu_code_1();
```

```
    // 5) Launch ker_A on stream2
    ker_A<<<grid, block, 0, stream2>>>();
    // 6) Launch ker_B on stream1
    ker_B<<<grid, block, 0, stream1>>>();
    // 7) Launch ker_2 on stream2
    ker_2<<<grid, block, 0, stream2>>>();
    // 8) CPU code 2 runs immediately
    printf("CPU code 2 executing\n");
    // ... do some other host-side work here ...
    cpu_code_2();
    // 9) Launch ker_3 on stream1
    ker_3<<<grid, block, 0, stream1>>>();
    // 10) Wait for work on each stream to finish
    cudaStreamSynchronize(stream1);
    cudaStreamSynchronize(stream2);
    // 11) Clean up
    cudaStreamDestroy(stream1);
    cudaStreamDestroy(stream2);
    return 0;
}
```

ker_1 在 stream1 上排队，然后控制立即返回到 CPU。cpu_code_1() 在主机上运行，而 ker_1 在 GPU 上执行。同时，我们将 ker_A 入队到 Stream2 上，将 ker_B 入队到 Stream1 上。然后，我们将 ker_2 排入流 2 上，交错 cpu_code_2，并将 ker_3 排入流 1 上。最后，我们同步每个流以等待所有工作完成，然后销毁流以清理资源。

这个例子强调了五个不同的内核执行在两个不同的流上重叠。稍微增加一点复杂性，并以第 10 章为基础，下面是相同的 warp 专业化管道示例，但使用 CUDA 流：

```
// Run the warp-specialized kernel in multiple CUDA streams.
#include <cuda_runtime.h>
#include <cuda/pipeline>
#include <cooperative_groups.h>
namespace cg = cooperative_groups;
#define TILE_SIZE 128
#define TILE_ELEMS (TILE_SIZE * TILE_SIZE)
// re-using from Chapter 10


__global__ void warp_specialized_pipeline(const float* __restrict__
 ↪ A_global,
                                          const float* __restrict__
                                           ↪ B_global,
                                     float*       __restrict__ C_global,
                                          int numTiles);
int main() {
    const int NUM_STREAMS = 2;                  // keep it small; tune
     ↪ as needed
    const int batches     = 8;                  // in-flight batches
    const size_t elems    = TILE_ELEMS;         // elements per batch
    const size_t bytes    = elems * sizeof(float);
    // Create streams that do NOT synchronize with the legacy default
     ↪ stream
    cudaStream_t s[NUM_STREAMS];
    for (int i = 0; i < NUM_STREAMS; ++i)
        cudaStreamCreateWithFlags(&s[i], cudaStreamNonBlocking);
    // Allocate pinned host buffers so H2D/D2H can truly overlap
    float *hA = nullptr, *hB = nullptr, *hC = nullptr;
    cudaMallocHost(&hA, batches * bytes);
    cudaMallocHost(&hB, batches * bytes);
    cudaMallocHost(&hC, batches * bytes);
    // ... initialize hA/hB ...
    for (int b = 0; b < batches; ++b) {
        const int sid = b % NUM_STREAMS;
        float *dA = nullptr, *dB = nullptr, *dC = nullptr;
        cudaMallocAsync(&dA, bytes, s[sid]);
```

```
    cudaMallocAsync(&dB, bytes, s[sid]);
    cudaMallocAsync(&dC, bytes, s[sid]);
    cudaMemcpyAsync(dA, hA + b * elems, bytes,
                    cudaMemcpyHostToDevice, s[sid]);
    cudaMemcpyAsync(dB, hB + b * elems, bytes,
                    cudaMemcpyHostToDevice, s[sid]);
    const dim3 block(96);    // three warps: loader(0), compute(1),
     ↪ storer(2)
    const dim3 grid(1);
    const size_t shmem = 3 * elems * sizeof(float); // [A|B|C] per
     ↪ tile
    // Reuse the Chapter 10 kernel exactly as-is
    warp_specialized_pipeline<<<grid, block, shmem, s[sid]>>>(dA,
     ↪ dB, dC,
                                                          /*numTiles=*/1);
    cudaMemcpyAsync(hC+b*elems, dC, bytes, cudaMemcpyDeviceToHost,
     ↪ s[sid]);
    cudaFreeAsync(dA, s[sid]);
    cudaFreeAsync(dB, s[sid]);
    cudaFreeAsync(dC, s[sid]);
}
```

```
    for (int i = 0; i < NUM_STREAMS; ++i) {
        cudaStreamSynchronize(s[i]);
        cudaStreamDestroy(s[i]);
    }
    cudaFreeHost(hA); cudaFreeHost(hB); cudaFreeHost(hC);
    return 0;
}
```

在这里，我们重新使用 warp 专用管道，并展示流如何添加第二层重叠，以便当流 1 在批次 n 上计算时，流 2 在批次 b+1 上执行 DMA 加载。同时，它可以复制回批次 b−1。内核的内部 cuda::pipeline 重叠保持不变。

我们将在本章后面继续通过在线程块集群中分层来构建复杂性，但让我们首先深入研究流如何帮助重叠计算和数据传输。这将有助于巩固 CUDA 流的基础知识及其在基于 GPU 的性能工程中的作用。

11.2 使用流将计算与数据传输重叠

例如，您可以将每个内核启动和内存复制排入其自己的流中。这允许 SM 执行内核，同时两个专用 DMA 引擎（一个用于主机 → 设备传输，一个用于设备 → 主机传输）同时移动数据。

由于 SM 计算管道独立于两个 DMA 引擎运行，因此您可以使用 CUDA 流将内核计算与两个数据传输完全重叠。但是，如果内核使用所有 SM 吞吐量使计算完全达到极限，或者复制管道因过多的额外重叠而使内存带宽饱和，则不会提高性能。

例如，当计算和内存吞吐量饱和时，您将开始看到两个并发操作，每个操作都在 50% 上运行，因为它们都在争夺相同的资源。您可以分析 GPU 利用率来识别这些饱和阈值。

例如，考虑将工作分成批次的 AI 模型训练或推理工作负载。在这里，您将在流 0 中的批次 0 上启动内核，同时流 1 调用 cudaMemcpyAsync() 将批次 1 从主机复制到设备，如图 11-2 所示。

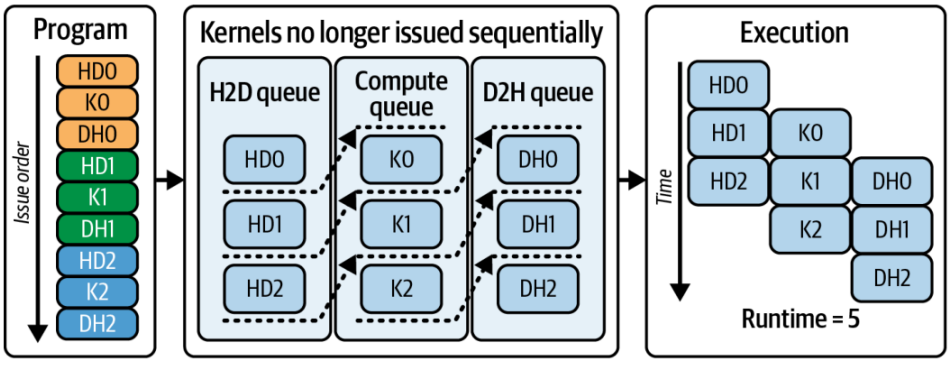

图 11-2　三向重叠的时间线。

在至少具有两个复制引擎 (deviceProp.asyncEngine Count()) 的现代 GPU 上，您可以将其扩展为三向重叠，以便流 0 运行批处理 0 的内核，流 1 将批处理 1 主机复制到设备，流 2 将前一批的结果写回主机。这扩展到其他流。这种模式将数据传输延迟隐藏在计算背后，反之亦然，使所有 GPU 引擎保持忙碌并最大限度地减少空闲时间。

实际上，您的内核必须满足几个要求才能实现这种并发行为。首先，异步传输中使用的任何主机指针都必须被页面锁定或固定。如果您在可分页内存上调用 cudaMemcpyAsync()，则运行时会执行主机端暂存复制到固定内存中，该副本会阻塞调用主机线程和排队流，直到暂存完成。

这可以防止传输异步。虽然这会阻塞调用主机线程，但 GPU 仍然可以重叠其他流中的计算和副本。但该特定传输不会正确重叠。要在流中实现完全异步传输，您必须使用固定主机内存。

> 注　通过使用 PyTorch 的 DataLoader 设置 pin_memory=True，您可以对主机缓冲区进行页面锁定，以便可以使用 DMA 将数据直接传输到 GPU 内存中。这允许副本与计算重叠并立即将控制权返回给主机。DMA 引擎可以将传输与计算重叠。可分页内存强制隐藏暂存副本并消除该传输的重叠。

其次，您应该使用异步分配和释放例程 cuda、MallocAsync() 和 cudaFreeAsync()，而不是同步和阻塞 cuda、Malloc() 或 cudaFree() 调用。像 PyTorch 这样的框架提供了使用的选项

CUDA 的异步、流排序内存分配器。这个想法是在分配内存时不停止所有活动流。这对于性能来说非常糟糕。

异步流排序分配器允许每个流请求或返回设备内存，而无需等待其他流。使用异步分配器可确保一个流中的内存操作不会阻碍其他流中的操作。这将避免不必要的全局同步。让我们在下一节中探讨流排序内存分配器。

PyTorch 的默认 CUDA 缓存分配器是流感知的，在正常操作中（例如，从其缓存中提供服务分配），它避免了设备范围的同步。仅当它必须使用 cudaMalloc 向操作系统请求更多内存时才会发生同步。实际上，这意味着大多数张量分配和释放不会阻塞其他流。启用 cudaMallocAsync 后端可以进一步减少碎片并提高许多工作负载的重用性，正如您接下来将看到的。

11.3 流排序内存分配器

在 PyTorch 中，您可以在启动 PyTorch 脚本之前通过设置环境变量 PYTORCH_ALLOC_CONF=backend:cudaMallocAsync 来启用 CUDA 的流排序分配器。如果设置了此变量，则 PyTorch 张量内存分配 (cuda MallocAsync()) 和释放 (cudaFreeAsync()) 操作) 将按照调用顺序在单独的 CUDA 流中排队。如果未设置此环境变量，PyTorch 使用自己的缓存分配器。

如果您使用旧版 cudaMalloc(...)，请记住，这是一个阻塞的设备范围操作，会在返回之前同步设备。这可能会停止其他流中的工作，因为每次分配都会强制整个 GPU 停止，直到保留内存为止。这会暂停所有流，限制并行性，并破坏工作负载的性能。

相反，将流排序分配器与 cudaMallocAsync(...) 结合使用，只需将分配请求记录在将使用它的同一个 CUDA 流中 - 无论该流正在执行内核操作还是内存操作。它不会阻塞其他流。这样，内存管理永远不会序列化为这些内核提供数据的流。

> **注** CUDA 的流排序分配器在 PyTorch 中使用，避免了全局设备锁并减少了分配开销。

实际上，流 0 可能在批次 N 上执行关注内核，流 1 将批次 N+1 从主机复制到设备，而流 2 将批次 N+2 的 cudaMalloc Async(...) 入队。由于 cudaMallocAsync(...) 只是将其工作附加到流 2 的队列中，因此流 0 和 1 会继续运行而不会中断。

使用流感知分配器，为每个小批量分配 GPU 内存，而不会阻塞其他流。这对于分配每批暂存空间的 LLM 管道非常重要。即使在内存频繁变动的情况下，异步分配器也能防止停顿。

> **注** 如果您的管道为每个小批量分配临时缓冲区（在 LLM 训练和推理中很常见），那么使用流排序内存分配器就特别重要。例如，LLM 管道中的每个小批量都需要自己的临时工作空间来保存注意力键/值或中间激活缓冲区。在这种情况下，您经常调用分配器来保留 GPU 上的“临时缓冲区”。

分配由每个设备的内存池来满足。您可以使用 cudaMemPoolSetAttribute() 调整池的释放阈值，以权衡将内存返回给操作系统与重用内存以提高性能。较高的阈值意味着池将保留分配的内存更长时间。这将减少内存返回给操作系统的次数。通过避免重复的内存分配和解除分配，可以减少操作系统调用并提高性能。

以下示例展示了如何使用 cudaMallocAsync 和 cudaFreeAsync 的流排序内存分配器来实现基于流的重叠，并演示了 cudaMemPoolSetAttribute(). 的使用。这重点介绍了如何使用 CUDA 流完全管道化内存分配、数据传输和内核执行：

```
// initialize the async memory allocator
cudaMemPool_t pool;
int device = -1;
cudaGetDevice(&device); // Current device
cudaDeviceGetDefaultMemPool(&pool, device);
// Desired number of bytes to keep in pool before
// releasing back to the OS (tune as needed)
uint64_t threshold =/* e.g., prop.totalGlobalMem / 2 */; // bytes
cudaMemPoolSetAttribute(pool,
  cudaMemPoolAttrReleaseThreshold, &threshold);
cudaStream_t stream1, stream2;
cudaStreamCreateWithFlags(&stream1, cudaStreamNonBlocking);
cudaStreamCreateWithFlags(&stream2, cudaStreamNonBlocking);
```

```
// Allocate memory using stream-ordered async allocation
void *d_data1, *d_result1;
void *d_data2, *d_result2;
size_t dataSizeBytes = N * sizeof(float);
// Use cudaMallocAsync as a best practice in modern multi-stream apps
cudaMallocAsync(&d_data1, dataSizeBytes, stream1);
cudaMallocAsync(&d_result1, dataSizeBytes, stream1);
cudaMallocAsync(&d_data2, dataSizeBytes, stream2);
cudaMallocAsync(&d_result2, dataSizeBytes, stream2);
// Asynchronously copy first chunk and launch its kernel in stream1
cudaMemcpyAsync(d_data1, h_data1, dataSizeBytes,
  cudaMemcpyHostToDevice, stream1);
computeKernel<<<gridDim, blockDim, 0,
  stream1>>>((float*)d_data1, (float*)d_result1);
cudaMemcpyAsync(h_result1, d_result1, dataSizeBytes,
  cudaMemcpyDeviceToHost, stream1);
// In parallel, do the same on stream2
cudaMemcpyAsync(d_data2, h_data2, dataSizeBytes,
                cudaMemcpyHostToDevice, stream2);
computeKernel<<<gridDim, blockDim, 0,
  stream2>>>((float*)d_data2, (float*)d_result2);
cudaMemcpyAsync(h_result2, d_result2, dataSizeBytes,
   cudaMemcpyDeviceToHost, stream2);
// Wait for both streams to finish
cudaStreamSynchronize(stream1);
cudaStreamSynchronize(stream2);
// Cleanup
cudaFreeAsync(d_data1, stream1);
cudaFreeAsync(d_result1, stream1);
cudaFreeAsync(d_data2, stream2);
cudaFreeAsync(d_result2, stream2);
cudaStreamDestroy(stream1);
cudaStreamDestroy(stream2);
```

在这里，我们创建两个 CUDA 流（stream1 和 stream2）并使用 cudaMallocAsync 分配设备内存，确保每个流都有自己的流排序内存缓冲区。然后我们为两个独立的数据块分配工作。

在 stream1 上，我们执行从主机到设备的异步复制（H2D），启动计算内核，然后将结果从设备异步复制回主机（D2H）。同时，我们对 Stream2 上的第二个数据块执行相同的操作。

由于这些操作是在单独的流上发出的，因此 GPU 设备会在它们之间重叠工作。Stream1 与 Stream2 上的 H2D 副本同时执行内核。一旦 stream1 内核完成，它可以将数据复制回主机（D2H），与 stream2 的内核执行重叠。

在这里，由于 CUDA 流和流排序内存分配，内存分配与内核计算重叠。本例中显示的交错调度减少了空闲时间并最大化了吞吐量。如果没有按流顺序分配，您要么必须预先分配所有内存（增加内存占用），要么招致严重的同步损失。

通过 cudaMallocAsync，内存管理可以无缝集成到 CUDA 流中。这允许按流分配和释放，而无需触发全局设备同步。

此外，流排序分配器允许您为可变长度缓冲区（例如令牌缓存或中间激活）发出细粒度的内存请求。然后，您可以立即启动依赖于这些缓冲区的内核。这一切都发生在同一个流中。

> **注** 实际上，实现峰值吞吐量需要仔细调整数据块大小并保持在 GPU 的并发限制内。现代 GPU 设备提供多个复制引擎，并且可以重叠主机到设备 (H2D) 和设备到主机 (D2H) 传输。查询 deviceProp.asyncEngineCount 以确定您的设备支持多少个复制引擎来相应地计划重叠。

现代 GPU 对可以在设备上的所有 SM 上运行的并发内核数量有硬性限制（最多 128 个驻留网格限制）。如第 5 章所述，现代 GPU 限制是每个设备 128 个并发执行内核。一旦超过活动内核的限制，额外的内核启动将排队，直到其中一个 SM 上的插槽释放为止。

请记住，共享 SM 的内核只有在它们的组合寄存器、共享内存和线程块要求符合 SM 的资源限制时才会一起执行。平衡块（tile）大小、启动顺序和每内核资源使用至关重要。

如果块太小，您将无法充分利用复制引擎和 SM 资源。如果块太大或同时排队的内核太多，您将超出内核插槽或耗尽每个 SM 资源。这将导致失速。

简而言之，如果调整正确，CUDA 流与流排序内存分配器 (cudaMallocAsync)) 相结合，将确保数据传输、内核执行和内存管理无缝重叠。这使多个 DMA 引擎和 SM 保持忙碌，而不会出现不必要的排队。

11.4 将 CUDA 流和流排序内存分配器与 LLM 结合使用

CUDA 流的非阻塞行为与流排序内存分配器相结合对于 LLM 训练和推理工作负载至关重要。这些工作负载跨多个流重叠计算和数据移动，以提高 GPU 利用率并减少端到端延迟。

此外，LLM 利用即时的“临时内存”分配，这是由上一节中讨论的流排序内存分配器促进的。例如，在运行转换器层时，您通常需要额外的共享内存或设备内存（称为暂存内存）来存储矩阵乘法的结果，然后再将其输入到 softmax 运算中。

由于 LLM 工作负载中的不同小批量的长度（令牌计数）可能有所不同，因此您将需要使用流排序内存分配器在 GPU 上专门为每个输入批量提供新的暂存缓冲区。这样，您就可以为该批处理的中间计算分配足够的空间，而不是多一个字节。

如果您使用旧的阻塞分配 API（cudaMalloc(...) 和 cudaFree(...)) 动态分配这些暂存缓冲区），则每次分配或释放都将与整个 GPU 同步，因为调用 cudaMalloc(...) 会强制全局设备同步。因此，在所有挂起的内核和副本完成之前，不可能发生重叠。

> **注** 全局设备同步对于性能来说绝对是灾难性的。避免在 CUDA 流中使用 cudaMalloc() 和 cuda Free() 等阻塞调用。更喜欢事件和流等待。并且绝对避免在默认流 0 上与 cudaStreamSynchronize(0) 同步！

在一个管道中，一个流正忙于运行批次 N 的注意力内核，而另一个流正在为注意力内核准备批次 N+1，在第二个流上调用阻塞 cudaMalloc(...) 将停止所

有流。在分配器完成之前，每个 SM 都会被有效地暂停。这可以消除您希望在数据传输、计算和内存管理之间实现的任何重叠。

解决方案是使用带有 cudaMallocAsync() 和 cudaFreeAsync(). 的流排序分配器。这些 API 将在流级别分配和释放设备内存区域的工作排队。因此，它们仅在流级别同步，而不是设备级别。

例如，考虑一个需要 16 MB 临时缓冲区来关注一批输入数据的流。它将调用 cudaMallocAsync(&scratchPtr, scrap Bytes, Stream1)，将这个分配请求记录在其操作队列中，但不会强制任何其他流等待，如图 11-3 所示。

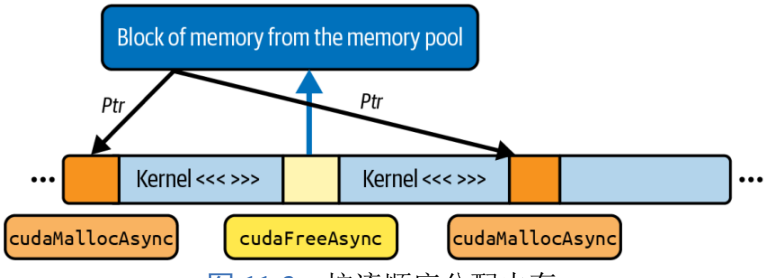

图 11-3　按流顺序分配内存。

其他流继续启动内核、复制数据或执行它们正在执行的任何操作，即使流 1 的分配正在进行中。一旦 CUDA 运行时在后台保留了内存，流 1 就可以再次取得进展，并将注意力内核启动到新分配的区域中——所有这些都不会停止任何其他流。

> 注　与传统的 cudaMalloc 不同，cudaMallocAsync 不会停止其他流。每个分配仅在其自己的流内同步。

在 LLM 训练和推理的背景下，这特别有价值，因为可变长度序列通常会产生暂存缓冲区大小波动。如果批次 N 每个序列有 512 个令牌，而批次 N+1 有 1,024 个令牌，那么您的注意力模块将需要比批次 N 更多的空间用于批次 N+1，因此重用批次 N 的分配是不够的。使用 cudaMallocAsync()，您可以将较大缓冲区的单个非阻塞分配排入队列，而无需将所有其他流拖到停止位置。

此外，典型的 LLM 的自回归逐个令牌生成（也称为解码）阶段使用不断增长的键/值缓存。每个生成的令牌都需要将新的 KV 对附加到每个序列缓冲区。随着缓冲区的增长，您需要重新分配或扩展暂存区域。cudaMallocAsync(...) 允

许您在运行注意力内核的同一流中执行此操作。与此同时，上游数据加载和下游结果复制操作继续并行进行，在自己的流中运行。

CUDA 流在 LLM 上下文中的另一个用途是大型 LLM 中的分层流水线。假设您将一个基于 Transformer 的大型 LLM 模型分为两半，在不同的 CUDA 流上运行 - 流 0 运行第 0-5 层，流 1 运行第 6-11 层。在这两个部分之间，您需要中间缓冲区来进行激活。

每次流 0 完成小批量的工作时，它可能会调用 cudaMalloc Async(...) 来获取新的缓冲区以用于下一批的激活。由于该调用不会同步设备，因此流 1 可以继续根据上一批的结果计算第 6-11 层，而流 0 则为下一批的输入分配内存。

相比之下，如果您在该管道内使用了旧版 cudaMalloc(...)，则每次为下一个小批量或扩展的 KV 缓存分配新的暂存区域时，整个 GPU 将暂停，直到分配完成。这将打破跨流重叠计算和数据移动的任何机会。

总而言之，在 LLM 上下文中，您经常需要临时缓冲区用于注意力、层标准化、softmax、KV 缓存或中间激活。这些统称为暂存缓冲区。

使用 cudaMallocAsync(...) 和 cudaFreeAsync(...) 来管理单独流中的这些临时缓冲区可确保内存管理永远不会强制全局的跨流停顿。相反，分配会排队到与内核或复制操作相同的流中。

这允许所有其他流继续运行，并尽可能让您的注意力集中在内核、数据传输和任何主机端工作上。这最大限度地提高了大规模实时 LLM 工作负载中 GPU 的利用率。

11.5 旧版默认流

当您没有显式创建或指定流时，操作将进入旧版默认流，通常称为流 0。默认情况下，流 0 有两个值得强调的重要行为：

11.5.1 与自身隐式同步

排队到流 0 中的任何两个操作都严格一个接一个地执行。您不能重叠流 0 中的两个内核或副本和内核，因为流 0 会序列化其自己的所有命令。

11.5.2 与其他流的隐式同步

在旧版默认流模型中，启动到流 0 中的任何操作都将等待所有其他流中先前排队的工作完成后再开始。相反，启动到非默认流中的任何操作也将被阻塞，直到流 0 中的所有先前工作完成为止。实际上，流 0 充当整个 GPU 的全局“屏障”。即使您向不同的流发出命令，一旦您向流 0 提交某些内容，它就会强制所有其他流停止，直到流 0 被赶上，反之亦然。这对性能非常不利，应尽可能避免。

由于这些隐式依赖关系，将所有工作放入流 0 可以防止任何形式的并发。例如，内核和复制引擎不能重叠。因此，您的 GPU 会花费周期空闲等待默认流屏障清除。

为了解锁真正的并行性，除了真正需要与其他流进行序列化的操作之外，您应该避免使用流 0 进行任何操作，这种情况相对较少。

11.6 现代每线程默认流

为了减轻遗留默认流的“全局障碍”行为，CUDA 引入了每线程默认流，有时缩写为 PTDS（与遗留流多年来给我们带来的创伤后应激障碍（PTSD）相反）。

在每线程默认流语义下，每个 CPU 线程的默认流是独立的。换句话说，当启用每线程默认流时，每个主机线程都有自己的隐式“流 0”。

排队到线程 A 的默认流中的操作不会等待线程 B 的默认流中的工作。只要硬件资源允许，它们就会同时运行。同样，线程 B 的默认流中的操作不会等待线程 A 的默认流，依此类推。

PTDS 广泛用于多线程 CUDA 应用程序，以避免“主机范围屏障”问题。要启用 PTDS，您可以使用 nvcc –defaultstream per-thread 编译代码或设置 CUDA_API_PER_THREAD_DEFAULT_STREAM=1 环境变量（在包含任何 CUDA 标头之前）。

> **注** 一旦 PTDS 处于活动状态，每个主机 CPU 线程的默认流的行为就像用户创建的流，不会与其他线程的默认流隐式同步。如果您在同一进程中将 PTDS 与旧版默认流混合使用，PTDS 流仍会与旧版默认流同步。

使用 PTDS，任何没有显式流参数的内核启动、复制或分配都会进入线程本地队列。只有同一主机线程的默认流中的命令才会序列化，并且它们永远不会对属于其他线程的流施加隐式全局屏障。

简而言之，通过启用每线程默认流，传统的默认流同步障碍被消除。每个主机线程的默认流从不等待其他线程的流。这允许跨线程的多个内核启动完全重叠。

如果您从不同的 CPU 线程发出内核启动（或内存副本）而未指定显式流，则只要资源允许，操作就会在 GPU 上重叠。如图 11-4 所示。

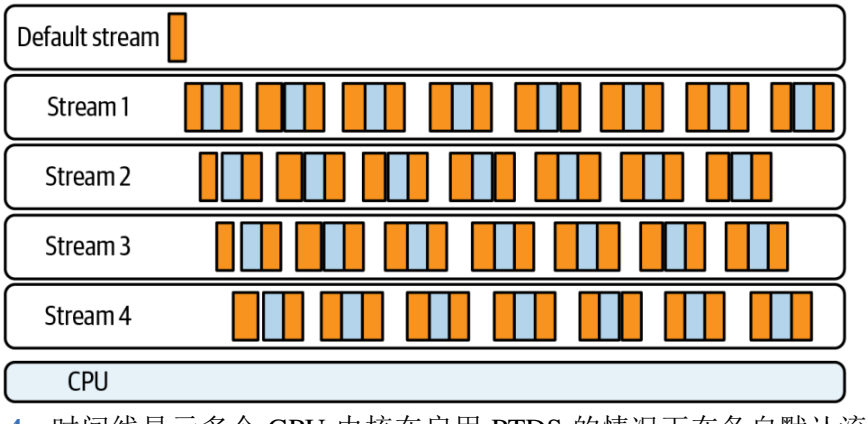

图 11-4　时间线显示多个 GPU 内核在启用 PTDS 的情况下在各自默认流上的不同线程发出的独立 CUDA 流上同时运行。

11.7 默认流与显式（非默认）流

依赖默认流行为最终会导致问题。总是如此。排队到旧默认流（流 0）中的任何工作都会隐式等待并阻塞所有其他流，反之亦然。

在性能关键型代码中，最好创建并使用您自己的非默认、显式和命名流，这样就不会意外进入流 0。如果您不小心在流 0 上使用了一个内核（或将数据复制到其中），您可能会停止所有其他活动流。许多库，例如 cuBLAS、Thrust 等，都接受显式流参数。建议您始终创建显式流并使用它们。

> **注** 在 PyTorch 中，操作在幕后安排在非默认流上，以避免意外同步。例如，PyTorch 对 cuDNN、cuBLAS 等的内部调用，使用自己的流来避免阻塞默认流 0。此外，PyTorch 的分布式后端在单独的 CUDA 流而不是默认流上启动 NCCL 通信操作。例如，这使得它可以将梯度通信与计算重叠。此外，NCCL 的通信操作通常在高优先级流中运行，我们稍后会介绍。

通过管理您自己的流（或使用每个线程的默认值），您可以保留对并发的控制。下面是在 CUDA C++ 中创建显式非默认流的示例（我们将在第 13 章和第 14 章中展示如何在 PyTorch 中使用流）：

```
cudaStream_t streamA, streamB;
cudaStreamCreateWithFlags(&streamA, cudaStreamNonBlocking);
cudaStreamCreateWithFlags(&streamB, cudaStreamNonBlocking);
myKernel<<<grid, block, 0, streamA>>>(...);                    // streamA
cudaMemcpyAsync(dest, src, size, cudaMemcpyHostToDevice, streamB); //
↪ streamB
```

这里，streamA 和 streamB 可以自由重叠。然而，在旧版默认流模型（禁用 PTDS）下，任何以后对流 0 的调用都会强制流 A 和流 B 等待，直到流 0 为空。

同样，如果流 0 仍有待处理的任务，则在流 A 或流 B 中排队的任何工作都将阻塞。为了避免这些隐藏的全局障碍，请保持流 0 空闲并仅将其用于初始设置、最终清理等一次性操作。

简而言之，启用每线程默认流，以便每个 CPU 线程的默认流不再与任何其他线程的默认流同步。然后为所有性能关键的内核和副本创建并使用显式流（例如 streamA 和 streamB）。

通过执行这两项操作，您排队到显式流中的任何内容都不会意外地与另一个显式流、另一个线程的默认流或旧版默认流 0 中的工作发生冲突。这确保了安全、可预测的重叠，而无需隐式同步。使用 cudaStreamNonBlocking 创建流可确保它们不与旧版默认流同步。这是避免隐藏障碍所必需的。

11.8 默认流使用的最佳实践

因为默认流可能会带来性能问题，所以让我们重点介绍每种类型流的同步特征——传统默认、每线程默认和显式（非默认）：

11.8.1 旧版默认流 (cudaStreamLegacy)

这会阻塞并被其他所有流阻塞。如果您需要任何形式的并发，请不要在此处发出工作。

11.8.2 每线程默认流 (cudaStreamPerThread)

每个主机线程的默认流都是私有的。它仍然序列化自己的命令，但不会等待或阻止任何其他线程的默认流或显式流。

11.8.3 显式流（使用 cudaStreamCreateWithFlags()) 创建

显式流是独立的队列，仅当您使用 cudaStreamWaitEvent() 显式插入依赖项时才同步。

以下是一些最佳实践，可帮助指导您使用默认和显式（非默认）流：

11.8.4 切勿将性能关键型内核启动到流 0（传统默认值）中，除非您打算序列化所有 GPU 工作

即使有一个杂散内核或复制到流 0 中，也会停止所有其他活动流。例如，较旧的 CUDA API 可能隐式使用流 0。例如，在没有显式流的情况下调用 CUDA 驱动程序 API 时，您可能会使用旧流。这是迁移到更新的 API 或始终指定流的另一个原因。

11.8.5 启用每线程默认流

如果您的应用程序使用多个 CPU 线程（每个线程都排队 GPU 工作），请使用 PTDS。这避免了这些线程的默认流之间隐藏的主机端障碍。在现代系统上，您还可以使用 cudaStreamCreateWith Flags(&stream, cudaStreamNonBlocking) 来创建与流 0 不同步的流。

11.8.6 创建和管理显式的非默认流

显式流是最佳实践，因为它们允许重叠内核和内存副本。始终将非默认 cudaStream_t 传递给 «<...»>()、cuda MemcpyAsync() 或 cudaMallocAsync(). 这保证隐式默认流同步不会干扰您的管道工作流程。

11.8.7 使用 cudaStreamWaitEvent() 来协调细粒度的依赖关系，而不是 cudaStreamSynchronize()

您应该在默认流上使用流事件而不是 cudaStreamSynchronize()。仅在明确定义的全局点（例如，模型训练时期的结束）调用 cudaStreamSynchronize() 以避免停止不相关的流。

11.8.8 明确您的流标志

如果启用 PTDS，请在任何 CUDA 调用之前设置设备标志。否则，您仍处于旧版默认流模式。任何对流 0 的提及都会造成全局障碍。

简而言之，旧版默认流（流 0）始终充当全局屏障。您排队的任何工作都会等待并强制所有其他流完成，并且如果流 0 有待处理的工作，则所有其他流都将等待。

为了避免这些看不见的停顿，不要将性能关键的内核或副本放入流 0 中。相反，使用 cudaStreamCreateWith 标志创建您自己的命名流并将所有内容启动到这些流中，以便它们可以独立运行。

如果您的程序有多个 CPU 线程（每个线程发出自己的 GPU 工作），您还应该启用每个线程的默认流。当 PTDS 打开时，每个线程的“默认”流不再与其他线程的默认值或流 0 同步。

这样，即使代码没有显式创建新流，也不会意外阻止任何其他线程的工作。在所有情况下，每当您希望两个操作重叠（例如，副本和内核）时，您应该为它们提供自己的显式非默认流。然后，您将避免流 0 的隐藏全局同步规则，并让 GPU 以最大并行度运行。

11.9 与事件和回调的细粒度同步

即使多个流和 DMA 引擎重叠，有时一个流的操作也必须等待另一个流的操作。考虑一对生产者-消费者流。生产者流 0 正在加载和准备数据以供消费者流 1 处理。

在这种情况下，很容易在主机端同步这些流并用完整的 cudaDeviceSynchronize(). 阻塞 CPU，但是，这将阻塞 CPU，直到 GPU 上的所有流和所有操作完成。这对于性能来说非常糟糕。您还可以使用 cudaStreamSynchronize()，如图 11-5 所示，但这会阻塞，直到流队列中的所有操作完成为止。

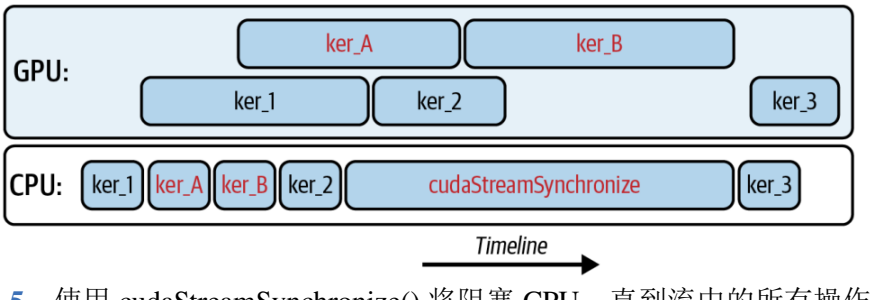

图 11-5　使用 cudaStreamSynchronize() 将阻塞 CPU，直到流中的所有操作都已同步。

相反，您可以使用 CUDA 事件为流提供更细粒度的同步机制。使用 CUDA 事件，您可以在生成数据的流中记录 cudaEvent_t，然后让使用者流等待该事件。

例如，您可以在流 0 上启动生产者内核，并在数据准备好使用时调用 cudaEvent Record(doneEvent, stream0)。在流 1 中，您将在启动消费者内核之前调用

cudaStreamWaitEvent(stream1,doneEvent,0)。这样，只有流 1 被停止等待事件被记录——主机线程和所有其他流将继续执行，如图 11-6 所示。

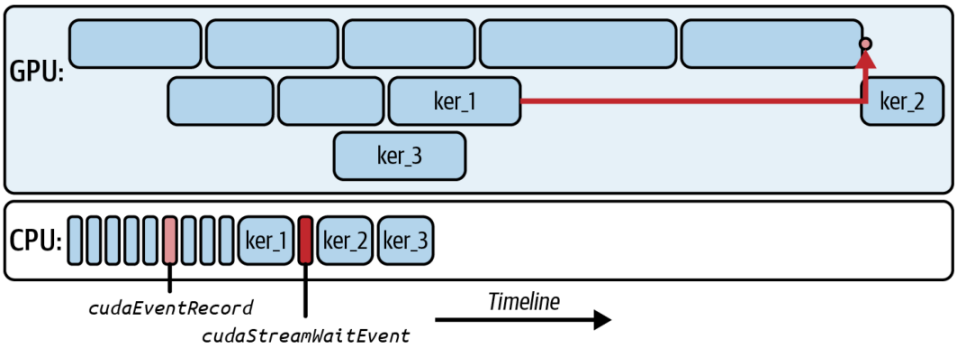

图 11-6　使用 CUDA 事件进行细粒度同步。

除了使用 CUDA 事件进行流间协调之外，您还可以使用它们在 GPU 设备和 CPU 主机之间进行通信。为此，您需要从主机向 cudaLaunchHostFunc() 注册主机回调。

假设您需要在 GPU 内核在流 0 中完成后立即在主机上回收自定义内存池。在这种情况下，您可以使用 cudaLaunchHostFunc() 注册来自主机的回调并指定您感兴趣的事件。然后您将在流 0 上启动内核。

当内核完成时，它将记录事件，主机将运行回调函数并更新 CPU 上的内存池。所有这一切都在没有轮询循环或完整的 cudaDeviceSynchronize(). 的情况下发生

一旦 GPU 工作完成，CUDA 运行时就会在主机线程上执行回调函数。这可以让 CPU 保持空闲，直到您需要它为止，并避免浪费主机周期和阻止不相关的 GPU 工作。

您不应该从使用 cudaLaunchHostFunc(). 启动的主机回调中调用任何设备端 GPU API。例如，如果您从回调内部调用 cudaMemcpy()，则可能会死锁，因为回调在由 CUDA 运行时管理的主机线程上运行。从内部调用 CUDA API 可能会死锁，因为设备可能正在等待回调完成。

> **注**　回调应仅限于 CPU 端任务，例如释放或回收主机内存。此建议可以防止循环情况下的死锁，在这种情况下，回调尝试在 GPU 上启动新工作，同时 GPU 正在等待回调完成。

11.10 使用 CUDA 事件进行跨流同步

当多个流并行运行时，我们经常需要在它们之间进行协调和同步。CUDA 事件是跨流同步的主要机制，而不会停止 CPU 或整个设备。

事件就像流在特定点记录的标记。其他流，甚至主机，可以等待该标记并知道某个事件何时发生（例如，内核已完成）。与完整的 cudaDeviceSynchronize() 不同，完整的 cudaDeviceSynchronize() 会阻塞直到所有流完成，事件允许流之间进行细粒度的排序。

流可用于确保每个转换器层的数据在计算之前都存在。他们还可以通过确保 GPU 0 在 GPU 1 使用张量之前完成生成张量来提高管道并行性。所有这一切都在没有闲置其他独立工作的情况下发生。

例如，考虑一组四个流（流 A-D），它们对操作进行管道化并且在某些情况下相互依赖。我们可以使用 CUDA 事件强制执行这些依赖关系（见图 11-7）。

为了确保流 B 在流 A 产生数据之前不会开始处理，我们可以在流 A 中记录一个事件，并让流 B 等待该事件，然后再继续。同样，流 D 可以等待流 B，如下所示。通过以这种方式链接事件，我们可以在流之间保持正确的顺序，同时仍然同时运行不同的批次。

这种基于事件的同步在深度学习框架中大量使用，以将梯度计算与 all-reduce 通信重叠。当梯度准备好时，计算流记录一个事件，并且通信流等待该事件以启动 NCCL 操作，从而将通信与剩余计算重叠。

简而言之，CUDA 事件是轻量级的，并针对设备信号进行了优化。当流到达其命令队列中的特定点时，它会记录一个事件。其他流可以有效地轮询/等待它。它们让我们能够跨流编排复杂的依赖关系图，而无需强制全局等待。它们提供了在多流 LLM 工作负载中实现流水线执行所需的控制。

> **注** NVIDIA 不断提高事件时间粒度并减少事件记录开销。因此，事件是分析的一个不错的选择，因为它们在 GPU 时间线中记录事件时间戳并可以测量执行时间等。

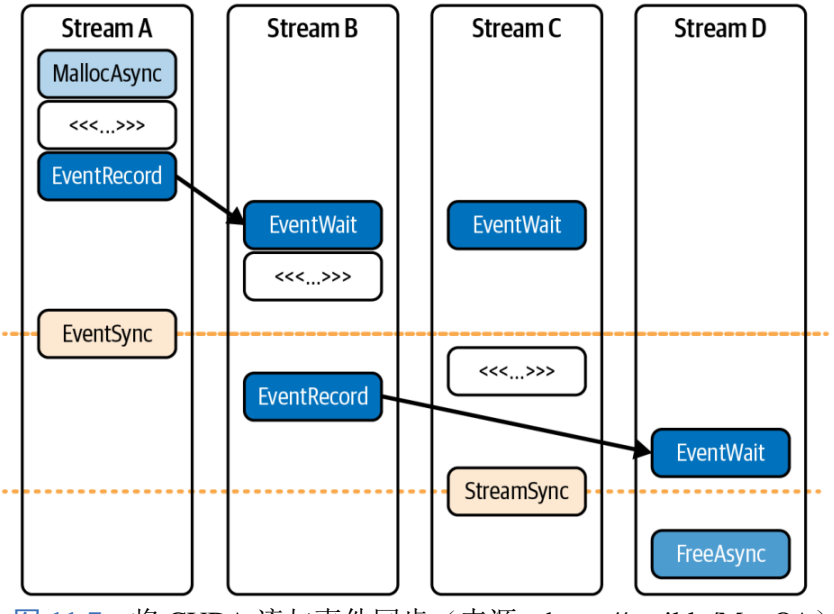

图 11-7　将 CUDA 流与事件同步（来源：https://oreil.ly/MynOA）

11.11 使用 Warp 专业化（内核内）和 CUDA 流（内核间）进行管道化

第 10 章演示了如何使用 warp 专门化和 CUDA Pipeline API 来隐藏单个内核中的内存延迟。在该部分中，我们启动了一个单一的、持久的、warp 专用的内核，其中每个线程块被分为三种不同类型的 warp：加载器、计算和存储器 warp。这些扭曲共享一块连续的共享内存和一个两级（双缓冲）CUDA 管道（<cuda::pipeline>）对象来协调它们的工作。

让我们保留相同的 warp 专用设备内核，现在使用单独的 CUDA 流通过它驱动多个批次。结果是一个两级管道，如下所示：

11.11.1 内核内重叠（每个块内）

加载器 ↔ 计算 ↔ 存储器使用单独的管道同时运行。

11.11.2 内核间重叠（跨批次）

当流 0 在批次 t 上进行计算时，流 1 DMA 加载批次 t+1，而流 2 返回批次 t−1 的结果。它使用非阻塞流、固定主机内存和流排序分配器，因此分配/副本不会序列化其他工作。

如果多个块需要相同的图块，则第 10 章中描述的线程块簇 + DSMEM 路径会删除跨块的冗余全局负载。但是，协作/集群启动与其他协作内核进行序列化。这里使用的流模式以主机 ↔ 设备重叠为目标，并与非合作内核一起工作。

> **注** 如果您的瓶颈是设备上图块重用，请使用线程块集群。如果瓶颈是主机 ↔ 设备通信和批处理，请使用 CUDA 流。我们将展示如何将这些结合起来，但这是初始决策的一个很好的起点。

下一个内核每个块使用三个扭曲（0 = 加载器，1 = 计算，2 = 存储器）和两个到乒乓阶段的管道。因此，需要 6 * TILE_SIZE * sizeof(float) 动态共享内存（[A0|B0|C0|A1|B1|C1]）。值得注意的是，我们改用两个独立的块作用域管道，每个管道的深度为 2。一个用于加载器 → 计算（pipe_lc) 切换，另一个用于计算 → 存储器（pipe_cs).）

此外，我们使用双缓冲共享内存，因此内核可以在计算瓦片 i 并存储瓦片 i − 1 时加载瓦片 i+1。这就是为什么您在下面的代码中在共享内存中看到两个 cuda::pipeline 对象和“[A|B|C] × 2 个阶段”。这两个管道是内核内的选择，以加深每个内核内部的重叠。（注意：您可以使用任一内核样式的流。）以下是代码：

```
#include <cooperative_groups.h>     // thread_block, etc.
#include <cuda/pipeline>            // CUDA Pipeline API
namespace cg = cooperative_groups;
// shmem bytes =
↪ 2(stages)×3(buffers)×TILE_SIZE×sizeof(float)=6×TILE_SIZE×4
//             = 6 * 1024 * 4 = 24,576 B (24 KB) << 227 KB SMEM
// We keep this at 1024 (versus going higher) as a safe starting point
// (good occupancy balance)
#define TILE_SIZE 1024  // one tile = 1,024 floats per buffer
↪ (1-Dimension)
    // Alignment / size guards for vectorized copies
    static_assert((TILE_SIZE % (32 * 4)) == 0,
                  "TILE_SIZE must be multiple of 128 for float4
                   ↪ vectorization");
    // If you cannot guarantee 16B alignment or sizes, handle
    //  the tail/ragged edges with a fallback 4B loop.


// Three warps per block: 0 loads, 1 computes, 2 stores.
// Two block-scoped pipelines (each depth=2) implement ping-pong across
↪ tiles.
__global__ void warp_specialized_two_pipelines(
    const float* __restrict__ A_global,
    const float* __restrict__ B_global,
    float*       __restrict__ C_global,
    int          numTiles)
{
    thread_block cta = this_thread_block();
    // Stage sin{0,1}: [A_s | B_s | C_s], each length TILE_SIZE
    extern __shared__ float shared_mem[];
    // loader -> compute pipeline (2 in-flight stages)
    __shared__ cuda::pipeline_shared_state<cuda::thread_scope_block, 2>
     ↪ state_lc;
    auto pipe_lc = cuda::make_pipeline(cta, &state_lc);
    // compute -> storer pipeline (2 in-flight stages)
    __shared__ cuda::pipeline_shared_state<cuda::thread_scope_block, 2>
     ↪ state_cs;
```

```
    auto pipe_cs = cuda::make_pipeline(cta, &state_cs);
    const int warp_id = threadIdx.x >> 5;   // 0,1,2
    const int lane_id = threadIdx.x & 31;
    // Prime first tile handled by this block
    const int first = blockIdx.x;
    if (warp_id == 0 && first < numTiles) {
        const int stage0 = first & 1;
        float* A0 = shared_mem + stage0 * 3 * TILE_SIZE;
        float* B0 = A0 + TILE_SIZE;
pipe_lc.producer_acquire();
#pragma unroll
for (int chunk = 0; chunk < TILE_SIZE; chunk += 32 * 4) {
    cuda::memcpy_async(
        cta,
        reinterpret_cast<float4*>(A0 + chunk) + lane_id,
        reinterpret_cast<const float4*>(A_global + size_t(first) *
          TILE_SIZE + chunk) + lane_id,
        sizeof(float4),
        pipe_lc);
    cuda::memcpy_async(
        cta,
        reinterpret_cast<float4*>(B0 + chunk) + lane_id,
        reinterpret_cast<const float4*>(B_global + size_t(first) *
          TILE_SIZE + chunk) + lane_id,
        sizeof(float4),
        pipe_lc);
}
pipe_lc.producer_commit();
    }
```

```
// Walk a strided set of tiles; ping-pong stage by tile parity.
for (int tile = blockIdx.x; tile < numTiles; tile += gridDim.x) {
    const size_t offset = size_t(tile) * TILE_SIZE;
    const int    stage  = tile & 1;
    float* A_buf = shared_mem + stage * 3 * TILE_SIZE;
    float* B_buf = A_buf      + TILE_SIZE;
    float* C_buf = B_buf      + TILE_SIZE;
    // Loader prefetches next tile while compute/store work on
    ↪ current.
    if (warp_id == 0) {
        const int next = tile + gridDim.x;
        if (next < numTiles) {
            const int next_stage = next & 1;
            float* A_next = shared_mem + next_stage * 3 * TILE_SIZE;
            float* B_next = A_next + TILE_SIZE;
            pipe_lc.producer_acquire();
            #pragma unroll
            for (int chunk = 0; chunk < TILE_SIZE; chunk += 32 * 4) {
                cuda::memcpy_async(
        cta,
                    reinterpret_cast<float4*>(A_next + chunk) +
                    ↪ lane_id,
                    reinterpret_cast<const float4*>(A_global +
                    ↪ size_t(next) *
                      TILE_SIZE + chunk) + lane_id,
                    sizeof(float4),
                    pipe_lc);
                cuda::memcpy_async(
                    cta,
                    reinterpret_cast<float4*>(B_next + chunk) +
                    ↪ lane_id,
                    reinterpret_cast<const float4*>(B_global +
                    ↪ size_t(next) *
                      TILE_SIZE + chunk) + lane_id,
                    sizeof(float4),
                    pipe_lc);
```

```
            }
            pipe_lc.producer_commit();
        }
    }
    // Compute consumes loader output and signals storer
    if (warp_id == 1) {
        pipe_lc.consumer_wait();
        #pragma unroll
        for (int chunk = 0; chunk < TILE_SIZE; chunk += 32) {
            C_buf[chunk+lane_id] =
             ↪ A_buf[chunk+lane_id]+B_buf[chunk+lane_id];
        }
        pipe_lc.consumer_release();
        pipe_cs.producer_acquire();
        pipe_cs.producer_commit();
    }
    // Storer waits on compute and writes back


    if (warp_id == 2) {
        pipe_cs.consumer_wait();
        #pragma unroll
        for (int chunk = 0; chunk < TILE_SIZE; chunk += 32) {
            C_global[offset + chunk + lane_id] = C_buf[chunk +
             ↪ lane_id];
        }
        pipe_cs.consumer_release();
    }
}
// Launch requirements: blockDim.x = 96 (3 warps), gridDim as needed,
// dynamic shared memory = 6 * TILE_SIZE * sizeof(float).
}
```

在这里，我们列出了一些值得注意的性能亮点。首先，我们要求 A0、B0、A_global + 基址和 B_global + 基址进行 16 字节对齐。如果路径不是 16 字节对齐的，我们会回退到 4 字节循环来处理未对齐的序言/尾声。

> **注** 每通道 16 字节 (float4/int4) 符合现代 GPU 最佳实践，可在扭曲级别实现 128 字节合并访问。这有助于减少管道开销。

接下来，我们假设 TILE_SIZE 是 128 的倍数（32 通道 × 4 个浮点数）。如果不是，请使用标量循环处理尾部。另外，我们使用 cuda::memcpy_async 和 float4。这保留了管道的异步语义，并将每个块的副本计数从每通道 4 倍减少到 1 倍。

这是主机驱动程序，它跨非阻塞流循环批处理，并使用固定主机内存和流排序分配器（cudaMallocAsync/cudaFreeAsync）。它在先前的内核中启动 warp_specialized_two_pipelines：

```
#include <cstdio>
#include <cuda_runtime.h>
#define TILE_SIZE   1024
#define NUM_STREAMS 2
#define BATCHES     8
// Device kernel
__global__ void warp_specialized_two_pipelines(
    const float* __restrict__ A_global,
    const float* __restrict__ B_global,
    float*       __restrict__ C_global,
    int          numTiles);
int main() {
    // Create nonblocking streams (do NOT use legacy default stream)
```

```
cudaStream_t s[NUM_STREAMS];
for (int i = 0; i < NUM_STREAMS; ++i)
    cudaStreamCreateWithFlags(&s[i], cudaStreamNonBlocking);
// Pinned host buffers so cudaMemcpyAsync truly overlaps
float *hA = nullptr, *hB = nullptr, *hC = nullptr;
const size_t bytesPerBatch = TILE_SIZE * sizeof(float);
cudaMallocHost(&hA, BATCHES * bytesPerBatch);
cudaMallocHost(&hB, BATCHES * bytesPerBatch);
cudaMallocHost(&hC, BATCHES * bytesPerBatch);
// Initialize inputs
for (int b = 0; b < BATCHES; ++b) {
    for (int i = 0; i < TILE_SIZE; ++i) {
        hA[b * TILE_SIZE + i] = float(i);
        hB[b * TILE_SIZE + i] = 1.0f;
    }
}
// Enqueue batches in a round-robin across streams
for (int b = 0; b < BATCHES; ++b) {
    cudaStream_t st = s[b % NUM_STREAMS];
    float *dA = nullptr, *dB = nullptr, *dC = nullptr;
    cudaMallocAsync(&dA, bytesPerBatch, st);   // stream-ordered
    ↪ allocator
    cudaMallocAsync(&dB, bytesPerBatch, st);
    cudaMallocAsync(&dC, bytesPerBatch, st);
    cudaMemcpyAsync(dA, hA + size_t(b) * TILE_SIZE, bytesPerBatch,
                    cudaMemcpyHostToDevice, st);
    cudaMemcpyAsync(dB, hB + size_t(b) * TILE_SIZE, bytesPerBatch,
                    cudaMemcpyHostToDevice, st);
    const dim3 block(96);         // 3 warps: loader/compute/storer
    const dim3 grid(1);
    const size_t shmem = 6 * TILE_SIZE * sizeof(float);
    ↪ //[A0|B0|C0|A1|B1|C1]
    // Each batch is one tile in this 1-D example
    warp_specialized_two_pipelines<<<grid, block, shmem, st>>>(
        dA, dB, dC, /*numTiles=*/1);
    cudaMemcpyAsync(hC + size_t(b) * TILE_SIZE, dC, bytesPerBatch,
```

```
                        cudaMemcpyDeviceToHost, st);
        cudaFreeAsync(dA, st);        // stream-ordered free
        cudaFreeAsync(dB, st);
        cudaFreeAsync(dC, st);
    }
    // Clean up
    for (int i = 0; i < NUM_STREAMS; ++i) {
        cudaStreamSynchronize(s[i]);


        cudaStreamDestroy(s[i]);
    }
    cudaFreeHost(hA); cudaFreeHost(hB); cudaFreeHost(hC);
    return 0;
}
```

这里，每个流都有自己的序列：分配 → H2D → 内核 → D2H → 释放。由于分配和副本按流顺序排队，并且主机缓冲区已固定，因此 GPU 的复制引擎可以将 H2D/D2H 与来自另一个流的 SM 计算重叠。这是内核间层，它补充了由扭曲专门化创建的内核内重叠，并在第 10 章中介绍。

具体来说，此示例显示共享内存保存两组，每组三个长度为 TILE_SIZE 的缓冲区，以便在准备下一个图块时可以使用一组。具有两个阶段的 <cuda::pipeline> 提供跨图块的双缓冲并强制执行正确的排序，以便加载器、计算和存储扭曲可以在不同的图块上重叠。内核在循环之前准备第一个图块，然后在计算图块 i 时预取图块 i 和 gridDim.x。

单独而言，这个扭曲专用内核通过在每个图块中重叠三个角色来隐藏内存延迟。然而，单个 warp 专用内核一次只能处理一批图块。

为了保持 GPU 忙于多个批次，并在这些批次之间重叠 H2D 传输、内核计算和 D2H 传输，我们在单独的 CUDA 流中启动同一经纱专用内核的多个实例。这将为每个批次使用流排序分配器。

第二层管道，即内核间并发，让我们可以通过 GPU 传递连续的小批量。这样，当一个批次的内核在流 0 中计算时，另一批次的数据仍在流 1 中到达。同时，前一个批次的结果可能会在流 2 中流回主机。

在实践中，我们选择少量的流（两个或三个），然后循环它们。每个流使用 cudaMallocAsync 分配自己的设备缓冲区，使用 cudaMemcpyAsync 异步复制输入数据，启动 warp 专用内核来处理这些缓冲区，将结果异步复制回主机，然后使用 cudaFreeAsync. 释放缓冲区

由于这些操作中的每一个都在特定流中排队，因此它们都可以与其他流中的等效操作重叠。在具有多个复制引擎和流排序分配器的现代 GPU 上，这种模式可以通过同时使芯片的每个部分饱和来显着提高利用率。实际重叠受到复制引擎计数（查询 cudaDeviceProp::asyncEngineCount)、带宽和内核占用率）的限制。

一旦一个内核中的加载程序扭曲正在等待全局内存获取，下一批的 H2D 副本就会在复制引擎上运行。一旦前一批中的存储扭曲写回全局内存，另一个流中的分配器就可以为下一批抓取内存，而无需强制全局同步。

本质上，第 10 章中的 warp 专业化示例教会了我们如何通过在线程块内重叠加载/计算/存储来使一个内核隐藏其内存延迟。本章中的多流示例在此基础上构建，展示如何通过在整个管道上重叠主机 ↔ 设备传输、计算和设备 ↔ 主机传输，同时将许多内核（每个内核处理不同的批次）输入到 GPU 中。

现代 GPU 具有多个复制引擎并执行异步内存分配。它们与 Tensor Core 一起并行工作，创建具有内核内扭曲专门化和内核间流的双层流水线策略。这些机制使我们的内核能够接近 LLM 工作负载的硬件利用率峰值。

11.12 使用线程块集群和 CUDA 流进行 Warp 专业化

现在，我们将把本章和前面的章节中的所有内容放在一起，通过多个流驱动多个运行中的小批量，每个流都会启动一个协作线程块集群、warp 专用内核。这代表了最新 GPU 硬件上 CUDA 性能优化的顶峰。

> **注** 虽然我们为了全面性而讨论这个主题，但值得注意的是，由于其复杂性，这种技术组合在非常专业的研究项目和超延迟敏感推理引擎之外很少见到。然而，它仍然值得一提，因为它将我们迄今为止学到的许多概念结合到一个代码示例中。

第 10 章展示了单个线程块内的 warp 专门化，以及使用 DSMEM 将 warp 专门化与线程块集群相结合。这样，一个领导线程块加载一次图块，并且线程块集群中的每个块都根据共享的片上副本进行计算。这会删除重复的全局负载。

此示例重复使用了领导者使用块范围管道来暂存副本的确切模式。所有块都是使用 cluster.map_shared_rank 读取的，因此它继承了数据重用优势。

在前面使用 CUDA 流的 warp 专用示例中，每个线程块负责其自己的所有三个管道阶段（加载器、计算和存储器）

共享内存区域。现在，让我们扩展早期的实现以使用线程块集群管道。这样，加载器、计算和存储扭曲就分布在线程块集群中。这与之前的实现形成对比，后者仅限于单个线程块。

下面是带有 CUDA 流的线程块簇加上扭曲专门化示例的代码。在此示例中，我们选择 NUM_STREAMS = 2，以便主机可以在单独的 CUDA 流中对两个独立的启动进行排队。我们重用第 10 章关于使用线程块集群的 warp 专业化部分中相同的 warp_speci alized_cluster_pipeline 实现，并添加 CUDA 流：

```
// Warp specialization across a thread block cluster using DSMEM
// and a block scoped pipeline, launched from multiple CUDA streams
#include <cuda_runtime.h>
#include <cuda/pipeline>
#include <cooperative_groups.h>
#include <algorithm>
namespace cg = cooperative_groups;
#define TILE_SIZE   128   // 3×TILE_ELEMS×4 bytes=196,608 bytes < 227 KB
 ↪ SMEM
#define TILE_ELEMS  (TILE_SIZE * TILE_SIZE)
#define NUM_STREAMS 2
#define CLUSTER_BLOCKS 4  // blocks per cluster along x (tune to device)
// ---- Device helpers ----
__device__ void compute_rows_from_ds(const float* __restrict__ A_src,
                                     const float* __restrict__ B_src,
                                     float*       __restrict__ C_dst,
                                     int row_begin, int row_end, int
                                      ↪ lane_id)
{
    for (int row = row_begin + lane_id; row < row_end; row += warpSize) {
        for (int col = 0; col < TILE_SIZE; ++col) {
            float acc = 0.0f;
            #pragma unroll
            for (int k = 0; k < TILE_SIZE; ++k) {
                acc += A_src[row * TILE_SIZE + k] * B_src[k * TILE_SIZE
                 ↪ + col];
            }
            C_dst[row * TILE_SIZE + col] = acc;
        }
    }
}
// Clustered, warp-specialized kernel (leader loads once, others
 ↪ consume DSMEM)
extern "C"
__global__ void warp_specialized_cluster_pipeline(
    const float* __restrict__ A_global,
```

```
    const float* __restrict__ B_global,
    float*       __restrict__ C_global,
    int numTiles)


{
    thread_block cta      = this_thread_block();
    cluster_group cluster = this_cluster();
    extern __shared__ float smem[];
    float* A_tile_local = smem;
    float* B_tile_local = A_tile_local + TILE_ELEMS;
    float* C_tile_local = B_tile_local + TILE_ELEMS;
    // Leader uses a block-scoped pipeline to stage A/B exactly once per
     ↪ tile
    __shared__ cuda::pipeline_shared_state<cuda::thread_scope_block, 2>
        pipe_state;
    auto pipe = cuda::make_pipeline(cta, &pipe_state);
    const int lane_id = threadIdx.x & 31;
    const int warp_id = threadIdx.x >> 5;
    const int cluster_rank       = cluster.block_rank();
    const dim3 cluster_dims      = cluster.dim_blocks();
    const int blocks_in_cluster  =
     ↪ cluster_dims.x*cluster_dims.y*cluster_dims.z;
    // Each iteration handles one tile per cluster (1-D cluster along x)
    auto loader = cg::tiled_partition<32>(cta);
    for (int tile = blockIdx.x / cluster_dims.x; tile < numTiles;
         tile += gridDim.x / cluster_dims.x) {
        const size_t offset = static_cast<size_t>(tile) * TILE_ELEMS;
        // Leader block' s loader warp stages A and B once for the
         ↪ entire cluster
        if (cluster_rank == 0 && warp_id == 0) {
            pipe.producer_acquire();
            cuda::memcpy_async(loader, A_tile_local, A_global + offset,
                               cuda::aligned_size_t<32>{TILE_ELEMS
                               * sizeof(float)}, pipe);
            cuda::memcpy_async(loader, B_tile_local, B_global + offset,
                               cuda::aligned_size_t<32>{TILE_ELEMS
```

```
                        * sizeof(float)} pipe);
    pipe.producer_commit();
    // Make visible inside leader before publishing
    pipe.consumer_wait();
    pipe.consumer_release();
}
// Publish to all blocks in the cluster
cluster.sync();
const float* A_src = cluster.map_shared_rank(A_tile_local, 0);
const float* B_src = cluster.map_shared_rank(B_tile_local, 0);
// Divide rows among blocks in the cluster
const int rows_per_block = (TILE_SIZE + blocks_in_cluster - 1)
                           / blocks_in_cluster;
const int row_begin = std::min(cluster_rank * rows_per_block,
↪  TILE_SIZE);


const int row_end   = std::min(row_begin + rows_per_block,
↪  TILE_SIZE);
// Compute warp produces this block's band into local SMEM
if (warp_id == 1) {
    compute_rows_from_ds(A_src, B_src, C_tile_local,
                         row_begin, row_end, lane_id);
}
// Ensure the storer sees computed rows
cta.sync();
// Storer warp writes band back to global
if (warp_id == 2) {
    for (int row = row_begin + lane_id; row < row_end; row +=
     ↪  warpSize){
        for (int col = 0; col < TILE_SIZE; ++col) {
            C_global[offset + row * TILE_SIZE + col] =
                C_tile_local[row * TILE_SIZE + col];
        }
    }
}
// Done with this tile; allow leader to reuse buffers
```

```
        cluster.sync();
    }
    // Dynamic shared memory size: 3 * TILE_ELEMS * sizeof(float)
}
// ---- Host driver: stream-staged batches + clustered kernel launch
 ↪ ----
void launch_warp_specialized_cluster_pipeline_multistream(
    const float* h_A,        // Host input A: length = numBatches *
     ↪ batchLength
    const float* h_B,        // Host input B: length = numBatches *
     ↪ batchLength
    float*       h_C,        // Host output C: length = numBatches *
     ↪ batchLength
    int          batchLength,// elems per batch; must be multiple of
     ↪ TILE_ELEMS
    int          numBatches)
{
    // Basic validation
    if (batchLength % TILE_ELEMS != 0) {
        fprintf(stderr, "batchLength must be a multiple of TILE_ELEMS
         ↪ (%d)\n",
                TILE_ELEMS);
        return;
    }
    const int numTiles = batchLength / TILE_ELEMS;
    // Nonblocking streams avoid legacy default-stream barriers
    cudaStream_t streams[NUM_STREAMS];
    for (int i = 0; i < NUM_STREAMS; ++i)
        cudaStreamCreateWithFlags(&streams[i], cudaStreamNonBlocking);
    // Size and launch geometry
    int device = 0;
    cudaGetDevice(&device);
```

```
cudaDeviceProp prop{};
cudaGetDeviceProperties(&prop, device);
// grid.x must be a multiple of CLUSTER_BLOCKS
const int blocksPerGrid = prop.multiProcessorCount * CLUSTER_BLOCKS;
const dim3 blockDim(96);  // 3 warps: loader/compute/storer
const size_t shmemBytes = 3ull * TILE_ELEMS * sizeof(float);
// Cluster attributes
cudaLaunchAttribute attr[2]{};
attr[0].id = cudaLaunchAttributeClusterDimension;
attr[0].val.clusterDim = dim3(CLUSTER_BLOCKS, 1, 1);
attr[1].id = cudaLaunchAttributeNonPortableClusterSizeAllowed;
attr[1].val.nonPortableClusterSizeAllowed = 1;
// Enqueue batches round-robin across streams
for (int b = 0; b < numBatches; ++b) {
    cudaStream_t st = streams[b % NUM_STREAMS];
    float *dA = nullptr, *dB = nullptr, *dC = nullptr;
    const size_t bytes = static_cast<size_t>(batchLength) *
    ↪ sizeof(float);
    // Stream-ordered allocator avoids global sync
    cudaMallocAsync(&dA, bytes, st);
    cudaMallocAsync(&dB, bytes, st);
    cudaMallocAsync(&dC, bytes, st);
    // H2D (host pointers should be pinned for true overlap)
    cudaMemcpyAsync(dA, h_A + static_cast<size_t>(b) * batchLength,
    ↪ bytes,
                    cudaMemcpyHostToDevice, st);
    cudaMemcpyAsync(dB, h_B + static_cast<size_t>(b) * batchLength,
    ↪ bytes,
                    cudaMemcpyHostToDevice, st);
    // Clustered launch config
    void* args[] = { &dA, &dB, &dC, (void*)&numTiles };
    cudaLaunchConfig_t cfg{};
    cfg.gridDim  = dim3(blocksPerGrid);
    cfg.blockDim = blockDim;
    cfg.dynamicSmemBytes = shmemBytes;
    cfg.stream = st;
```

```
        cfg.attrs = attr;
        cfg.numAttrs = 2;
        // Launch the clustered kernel (cooperative/cluster launch)
        cudaKernel_t k;
        cudaGetKernel(&k, warp_specialized_cluster_pipeline);
        void* fptr = reinterpret_cast<void*>(k);
        cudaLaunchKernelExC(&cfg, fptr, args);
        // D2H and cleanup
        cudaMemcpyAsync(h_C + static_cast<size_t>(b) * batchLength, dC,
        ↪ bytes,
                        cudaMemcpyDeviceToHost, st);


        cudaFreeAsync(dA, st);
        cudaFreeAsync(dB, st);
        cudaFreeAsync(dC, st);
    }
    for (int i = 0; i < NUM_STREAMS; ++i) {
        cudaStreamSynchronize(streams[i]);
        cudaStreamDestroy(streams[i]);
    }
}
```

> **注** 此示例使用使用 cudaLaunchKer nelExC 配置的集群启动。合作启动（包括集群启动）需要足够的资源来根据启动限制保持其块的驻留。这通常会限制并发性，但在资源允许的情况下，协作内核仍可能与其他工作交错。并发性无法得到保证，因为它依赖于拓扑和启动。（注意：受资源限制和启动限制，其他流中的数据传输和内核仍然可以重叠。）

与第 10 章一样，我们启动具有簇维度的内核，以便块加入 cluster_group 并可以访问彼此的分布式共享内存。领导者块通过块范围的 cuda::thread_scope_block 暂存其副本，而所有块使用 cluster.map_shared_rank. 读取领导者的切片

使用集群范围和分布式共享内存，加载阶段在领导者中的每个集群中运行一次，而计算和存储在集群的块上同时运行。和以前一样，每个扭曲的 warp_id

决定其角色，并且扭曲在所有图块上持续循环，以同步旋转的方式获取、计算和存储。

通过添加 CUDA 流并在单独的 CUDA 流中启动该内核的独立副本（NUM_STREAMS = 2），我们使 GPU 忙于处理多批输入数据。在每个流中，我们执行以下步骤：

1. 为每个线程块分配每批设备缓冲区以与 cudaMalloc 异步一起使用。

2. 使用 cudaMemcpyAsync. 将主机输入转移至设备

3. 使用 cudaLaunchKernelExC. 启动集群 warp 专用内核

4. 使用 cudaMemcpyAsync. 将输出复制回主机

5. 使用 cudaFreeAsync. 释放设备缓冲区

由于每个流都会将自己的协作启动及其异步副本和释放排入队列，因此即使一次只有一个协作内核运行，主机也可以保持多个小批量运行。请记住，GPU 序列化协作内核启动，因为每个协作内核同时固定每个线程块（在代码示例之后将再次强调此限制）。

使用多个流（NUM_STREAMS = 2）仍然可以让我们将主机端分配和副本与先前内核的执行重叠。例如，当流 0 的线程块集群在图块 n 上工作时，流 1 可以异步分配缓冲区 (cudaMalloc Async) 并将图块 n+1 复制到设备内存中，而流 2 可以将图块 n−1 的结果写回主机。

> **注** 协作内核必须占用 GPU 上所需的每个线程块槽 (CTA)。这可以防止任何其他协作启动同时运行，因为所有这些 CTA 资源都已在使用中。

实际上，流 0 发出其协作启动并开始执行，而流 1 可以立即将其自己的启动加入队列。但第二次启动仍处于等待状态，直到流 0 的内核完成为止。

然而，一旦流 0 完成，流 1 的启动就会立即开始。这是因为它的输入已经由 cudaMemcpyAsync 暂存，并且它的缓冲区已经由 cudaMallocAsync. 分配

结合线程块集群、内核内扭曲专门化（三级 cuda::thread_scope_cluster) 和内核间 CUDA 流），我们创建了跨越多个线程块的两层管道。这些两层管道将硬件推向顶峰。

在第一层中，线程块集群管道让加载器、计算和存储扭曲在所有线程块之间进行协作。线程块集群可以使用 TMA 多播将全局内存块传输复制到集群内每个块的共享内存中。多播是线程块集群本地的。本质上，线程块集群只获取每个图块一次。在第二层中，多个流将主机端分配、副本和内核启动隐藏在彼此后面。

简而言之，这些组合技术确保内存延迟在网格级别和主机到设备级别都被隐藏。这使得现代 GPU 上的硬件利用率接近峰值，并最大限度地提高 LLM 工作负载的吞吐量。

> **注** 在许多场景中，内存层次结构瓶颈（例如 DRAM 带宽）和模型并行通信（例如 NCCL allreduce）将主导工作负载。因此，一旦您的内核接近饱和 HBM 带宽，并且您的流已经隐藏了 CPU → GPU 延迟，那么通过在此基础上添加扭曲专业化（然后在此基础上添加线程块集群）所获得的额外百分之几的 SM 利用率几乎无法证明高昂的工程成本是合理的。在实践中，大多数现实世界的 LLM 训练和推理工作负载充分受益于之前提出的更简单的设计，例如双缓冲内核或带有 CUDA 流的两级管道。

11.13 多 GPU 计算和数据传输与 CUDA 流重叠

当跨多个 GPU 训练或服务 LLM 时，CUDA 流可让您重叠本地计算、点对点传输、集体通信、数据准备和内存管理，从而避免设备空闲。例如，假设您将一个 Transformer 模型拆分到两个 GPU 上，以便 GPU 0 处理第 0-3 层，GPU B 处理第 4-7 层。在 GPU A 上，您可以这样写：

```
// Stream 0 on GPU A: compute layers 0–3
myTransformerLayers0to3<<<gridA, blockA, 0, stream0_A>>>(
    inputActivationsA, outputActivationsA);
cudaEvent_t eventA, eventFromA;
cudaEventCreateWithFlags(eventA,
    cudaEventDisableTiming); // lowers overhead for sync-only events
cudaEventCreateWithFlags(eventFromA,
    cudaEventDisableTiming);  // lowers overhead for sync-only events
cudaEventRecord(eventA, stream0_A);
```

同时，在 CPU 上，您已经解码或准备了下一个微批次 (N+1)，并向固定主机缓冲区发出 cudaMemcpyAsync。这样，当 GPU A 的流 0 完成批次 N 的前向传递时，批次 N+1 的 CPU 到 GPU 复制可以立即开始，无需等待。

同时，您调用 cudaStreamWaitEvent(stream1_A, eventA, 0) 以确保 stream1_A 等待，直到 outputActivationsA 准备好，然后再启动直接 NVLink/PCIe 副本到 GPU B 的内存中，如下所示：

```
// Stream 1 on GPU A: wait for layer computation, then copy to GPU B
cudaStreamWaitEvent(stream1_A, eventA, 0);
cudaMemcpyPeerAsync(
    destActivationsB, /*dstGPU=*/1,
    outputActivationsA, /*srcGPU=*/0,
    bytes, stream1_A);
```

一旦您记录了事件 A，GPU A 的 stream0 就可以立即启动批次 N+1 的下一个前向内核，使用 cudaMemcpyAsync 将其输入从固定的主机缓冲区复制到设备内存中而不会阻塞。在 GPU B 上，您在复制开始或结束时记录匹配事件：

```
// Stream 1 on GPU A: after peer copy starts,
// record eventFromA
cudaEventRecord(eventFromA, stream1_A);
```

然后在 GPU B 上，等待数据到达并运行第 4-7 层，如下所示：

```
// Stream 1 on GPU B: wait for data arrival, then run layers 4–7
cudaStreamWaitEvent(stream1_B, eventFromA, 0);
myTransformerLayers4to7<<<gridB, blockB, 0, stream1_B>>>(
    destActivationsB, nextActivationsB);
```

这种显式的事件等待模式保证了 GPU B 的第 4-7 层计算仅在对等复制完成后开始。同时，GPU B 的 stream0_B 可以同时预取权重或执行其他准备工作。

这里使用的 P2P 传输在 GPU A 的 DMA 复制引擎上运行，不使用任何 GPU A 的 SM。这样，GPU A 的计算流可以立即进行批量 N+1，而无需管理数据传输。

结果是三向重叠：GPU A 的 SM 可以立即在 stream0_A 中启动批次 N+1，GPU A 的对等 DMA 引擎可以将批次 N 激活传送到 stream1_A 中的 GPU B，而 GPU B 的 SM 可以在 stream0_B 中运行批次 N 或在其中开始批次 N。stream1_B. 通过将工作划分为单独的流并使用主机上的固定内存，我们将 P2P 和 H2D 延迟隐藏在正在进行的计算和数据准备背后。

当需要在多个 GPU 之间同步梯度或广播参数时，NCCL 使用低占用率设备内核处理大规模通信，这些内核通过 NVLink、PCIe 或 InfiniBand 驱动 GPUDirect P2P 或 RDMA。在底层，NCCL 将张量分成多个连续的块。

该设计表明，每个 GPU 可以有多个流，包括用于其主要工作的计算流、用于传入数据的接收流以及用于缩减的通信流。每个角色使用专用流并给予通信流更高的优先级可以实现最佳重叠。事实上，像 PyTorch 这样的框架通常在这些专用流上运行 NCCL 集合，并具有更高的网络传输优先级。

> **注** PyTorch 和 NCCL 均使用专用的高优先级流来交错通信与计算密集型操作。这样，它们就不会落后于大型计算内核。

NCCL 将根据 NVLink 或 PCIe 拓扑选择环形或树形算法。考虑一个四 GPU 环，如图 11-8 所示，有四个块 (1-4)。

在这个环 all-reduce 中，每个 GPU 将块 i 发送到其邻居 (k → k+1)，同时从其另一个邻居 (k−1 → k) 接收块 i−1，使用设备内核通过 NVLink 或 PCIe 移动和减少块。

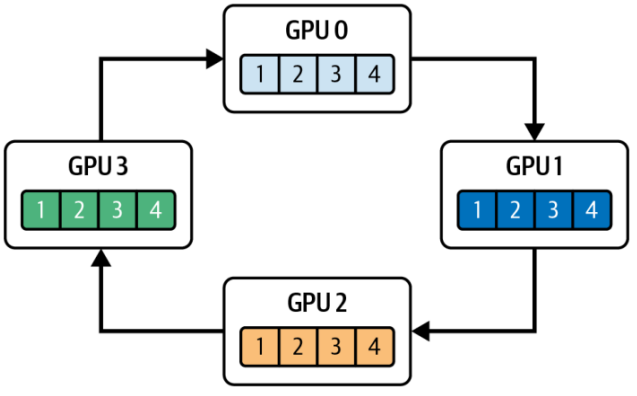

图 11-8　在环中使用四个 GPU 进行分块全归约。

通过管道化这些分块发送和接收，NCCL 使 NVLink 保持完全饱和。当块 i 从 GPU 0 → GPU 1 遍历时，块 i–1 移动 GPU 1 → GPU 2，依此类推。这可以最大限度地减少空闲间隙。在代码中，您会看到类似以下内容：

```
// In a high-priority communication stream on each GPU:
cudaEventRecord(eventK, computeStream);
cudaStreamWaitEvent(commStream, eventK, 0);
ncclAllReduce( // this is asynchronous
    gradBuffer, gradBuffer, numElements,
    ncclFloat, ncclSum, comm, commStream);
```

由于 NCCL 仅使用几个 SM 线程块，因此它在 SM 上以低占用率方式协调 NVLink 或 NVSwitch 上的分块发送/接收 + 减少。同时，例如，运行第 k+1 层向后传递的主计算流可以继续运行。当 all-reduce 完成时，事件将被记录，如下所示：

```
cudaEventRecord(eventAllReduceDone, commStream); // signals collective
↪  completion
cudaStreamWaitEvent(computeStream, eventAllReduceDone, 0);
// Now apply optimizer updates on computeStream...
```

请记住，像 all-reduce 这样的 NCCL 集合是异步的。他们立即恢复控制权。通过在调用后记录事件并在计算流中等待它，您可以确保计算流在缩减实际完成之前不会恢复。

> **注** NCCL 的内核经过专门优化，可以在每个 GPU 上使用少量 SM 资源以低占用率实现最大带宽。实际上，由于针对互连调整的低占用率内核，NCCL 可以通过每个 GPU 的少量线程块来使 NVLink 或 PCIe 带宽饱和。此外，如果您的网络硬件支持，您可以将其中一些集体聚合操作卸载到 NVIDIA SHARP。这将释放更多的 SM 资源来执行更有用的计算工作。

NIXL 提供统一、高吞吐量的 API，用于跨 NVLink、RDMA 和存储后端的点对点和分解传输，因此您通常使用 NIXL API，而不是自己调用 cudaMemcpyPeerAsync。NIXL 在不同的内存层和互连之间快速、异步地移动数据。当您使用 nixlCommStream 调用 NIXL 操作时，数据将使用最快的可用传输机制（例如 GPUDirect RDMA 或 NVLink）通过网络进行分块和管道传输。

与 NCCL 一样，NIXL 可以使用专用的高优先级流进行分块点对点和集体数据传输。这可以减少排队延迟，因此它们的复制命令在发出后很快就会到达 GPU 的 DMA 引擎——可能会抢占优先级较低的工作。

将这些传输流标记为高优先级并不意味着它们会预先消耗 SM 资源。它只是保证当复制命令准备就绪时，它们将在其他低优先级操作之前发出。

实际上，复制引擎会利用计算内核尚未使用的任何 DRAM 到 DRAM 带宽。因此，它们有效地仅使用空闲的内存结构带宽来跨互连移动数据。

这种设计最大化了重叠，这样当一个流驱动复制引擎上的 cudaMemcpyPeer 异步传输或 NCCL 缩减内核时，另一个流的 SM 内核可以继续处理前向/后向工作。第三个流可以执行异步分配或事件等待。这使得所有硬件单元保持忙碌并且没有争用。

使用 cudaMemcpyPeerAsync 启动的点对点传输完全在 GPU 的复制引擎上运行，因此它们仅消耗 DRAM 到 DRAM 带宽，而不消耗 SM 周期。另一方面，诸如 all-reduce 之类的集体操作被实现为使用少量 SM 的设备内核，同时将互连带宽推向峰值。

同时，异步分配和释放用于激活或混合精度梯度的临时缓冲区，以避免全局停顿。例如，在每个 GPU 的计算流中，您将调用以下代码：

```
cudaMallocAsync(&tempBuffer, tempBytes, computeStream);
// ...use tempBuffer in kernels...
cudaFreeAsync(tempBuffer, computeStream);
```

由于流排序内存分配器记录这些操作而不强制设备范围同步，因此 CUDA 在分配或释放内存时不会暂停其他流，例如 P2P、NCCL 和 NIXL 流。这确保缓冲区管理永远不会中断重叠的计算和通信管道。

> **注** 这正是我们之前讨论过的流排序内存分配器，但现在应用于多个 GPU。使用此功能将有助于防止内存管理成为分布式工作负载的瓶颈。

在将前向传递、后向传递、P2P 复制、NCCL 全归约、内存释放和事件等待操作排列到单次迭代中后，您可以将执行链（例如，训练或推理迭代）捕获到单个 CUDA 图中。

具体来说，当您调用 cudaStreamBeginCapture 时，您排队到流中的每个操作（例如，内核启动、计算、通信、数据传输、事件等）都会作为节点插入到图中。这可以包括 ncclAllReduce()、cudaMemcpyAsync、cudaMemcpyPeerAsync、cudaMallocAsync、cudaFreeAsync、cuda EventRecord、cudaStreamWaitEvent 及其依赖项。您可以使用 cudaStreamEndCapture(stream, &graph) 结束捕获。代码看起来像这样：

```
cudaStreamBeginCapture(streamA, cudaStreamCaptureModeGlobal);
computeGradientsLayer1<<<... , 0, streamA>>>(...);
ncclAllReduce(..., comm, streamB);
computeGradientsLayer2<<<... , 0, streamA>>>(...);
ncclAllReduce(..., comm, streamB);
cudaStreamEndCapture(streamA, &graph);
```

当捕获提交到多个流的工作时，请使用 cudaStreamCaptureMode Global，以便在同一线程中的任何参与流上排队的操作都会记录到同一捕获会话中，并保留跨流依赖性。否则，仅捕获传递到 cudaStream BeginCapture 的流上排队的操作。

然后，您可以使用 cudaGraphLaunch(). 实例化并启动该图，这将以接近于零的 CPU 开销重放整个 DAG。通过捕获整个多 GPU 迭代一次，您可以消除每个操作的排队和启动开销。

此外，CUDA 图支持条件节点（例如梯度裁剪），因此不频繁的分支保留在图的逻辑内。我们将在下一章中更详细地介绍 CUDA 图，但了解它们与 CUDA 流（包括 cudaStreamBeginCapture 和 cudaStreamEndCapture.）的关系很重要

> **注** 像 PyTorch 这样的现代框架隐藏了大部分复杂性。例如，PyTorch 的 DistributedDataParallel 自动调度单独流中的数据传输、计算和通信。它还使用 CUDA 图来减少每次迭代的开销。虽然它可以使用 CUDA 图表来减少开销，但您必须使用捕获安全代码和 API 显式启用此功能，因为它不会自动启用。

虽然您不需要显式使用这些 CUDA 流开始和结束 API 来构建 CUDA 图，但当您想要在将操作排队到流中的同时捕获图时，它们会很方便（我们将在下一章中更详细地介绍 CUDA 图）。

CUDA 流通过跨 CPU、DMA 引擎和 SM 重叠工作，在多 GPU 系统中实现精细调整的管道。在 CPU 端，cudaMemcpyAsync 进入固定主机缓冲区，在 GPU 执行当前工作负载时暂存下一批数据，确保批处理 N+1 的输入在批处理 N 完成之前准备就绪。同时，GPU 之间的点对点传输（使用 cudaMemcpyPeerAsync) 可以安排在专用流上，与 cudaStreamWaitEvent 同步以切换激活或梯度，而不会停止正在进行的计算。

如前所述，像 NCCL 或 NIXL 这样的集体通信库是使用单独流的低占用率通信内核。这样，当一个 GPU 减少梯度或广播参数时，同一 GPU 上的其他流可以继续执行本地计算内核（例如，后续层的前向或后向传递）。此外，按流顺序使用 cudaMallocAsync 和 cudaFreeAsync 可以防止临时缓冲区的全局同步，从而使分配和释放与计算和通信同时进行。

将整个迭代（例如，CPU 分段、P2P 传输、计算内核、NCCL 调用、分配、释放和事件/等待）捕获到 CUDA 图中可以进一步减少 CPU 开销。图实例化后，调用 cudaGraphLaunch() 即可一次性重放记录的 DAG。这消除了每次调用的排队开销并自动保留所有依赖项。

这些技术共同确保每个 GPU 的 SM 管道、复制引擎和互连链路保持繁忙。当一个流运行矩阵乘法内核时，另一个流执行点对点复制，第三个流执行集体通信，CPU 为下一个微批次暂存数据。

简而言之，您可以使用 CUDA 流来编排跨多个层的工作并重叠计算、内存操作和数据传输。这种方法通过隐藏主动计算背后的点对点和集体通信延迟，将每个硬件单元推向极限。总体而言，CUDA 流最大限度地提高了许多 AI 工作负载的吞吐量、利用率和效率，包括 LLM 训练和推理。

11.14 程序化依赖启动

另一种类型的内核间管道和通信称为编程相关启动 (PDL)。PDL 允许内核使用相同的 CUDA 流直接在设备上触发另一个内核的执行，而不涉及 CPU。例如，内核 A（主内核）可以触发内核 B（辅助内核），后者正在等待来自内核 A 的信号。

该触发器甚至可以在内核 A 完成执行之前发生。为此，它使用 cudaTrigger-ProgrammaticLaunchCompletion()，如图 11-9 所示。这里，内核 B 正在等待内核 A 调用 cudaGridDependencySynchronize().

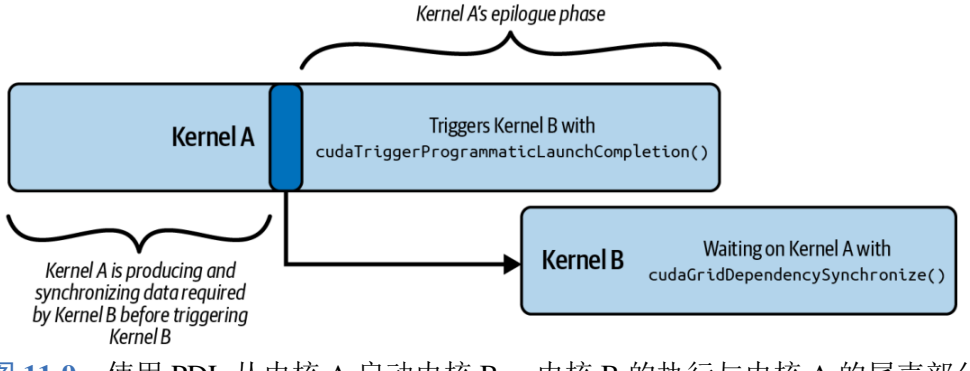

图 11-9　使用 PDL 从内核 A 启动内核 B —内核 B 的执行与内核 A 的尾声部分重叠（在同一个 CUDA 流中）

使用 PDL 提供的结构，内核 A 可以向内核 B 发出信号，使其在内核 A 的尾声（例如，总结）阶段执行。这样，内核 A 可以与内核 B 一起执行一段时间。

需要注意的是，在内核 A 生成并同步内核 B 所需的所有数据（例如，L2/共享内存/全局内存）之前，内核 A 不应触发内核 B 执行。

> **注** 在继续处理之前，依赖内核的数据依赖关系应该在 L2、共享内存、全局内存等中可见。

这里的代码显示内核 A 使用 cudaTriggerProgrammaticLaunchComple tion() 向内核 B 发出信号，表明其主要工作已完成。这也通知内核 B 所有必要的全局内存刷新都已发生，并且可以安全地继续：

```
#include <cuda_runtime.h>
#include <cuda_runtime_api.h>
// Kernel A must trigger the PDL flag when it's
//    safe to launch Kernel B
__global__ void kernel_A(int *d_ptr) {
    // Perform work that produces data used by
    //   Kernel B

    // Signals that Kernel A's global-memory
    //   flushes are complete
    // This enables dependent Kernel B's launch
    cudaTriggerProgrammaticLaunchCompletion();
    // ... any further work that can overlap with  ...
    ...
}
// Kernel B must wait for Kernel A to write its memory
//    to global memory and become visible to Kernel B
__global__ void kernel_B(int *d_ptr) {
    // Wait on Kernel A to complete.
    // This ensures the Kernel B waits for the
    //   memory flush before accessing shared data
    cudaGridDependencySynchronize();
    // ... dependent work on d_ptr ...
    ...
}
int main() {
```

```
// 1) Allocate device buffer
int *d_ptr = nullptr;
// Allocate an int (example)
cudaMalloc((void**)&d_ptr, sizeof(int));
// 2) Create a nonblocking stream for maximum overlap
cudaStream_t stream;
cudaStreamCreateWithFlags(&stream,
    cudaStreamNonBlocking);  // Nonblocking
// 3) Define grid/block sizes
dim3 gridDim(128), blockDim(256);
// 4) Launch Kernel A asynchronously


kernel_A<<<gridDim, blockDim, 0,
    stream>>>(d_ptr);    // Async launch
// 5) Configure PDL for Kernel B
cudaLaunchConfig_t launch_cfg{};
launch_cfg.gridDim           = gridDim;
launch_cfg.blockDim          = blockDim;
launch_cfg.dynamicSmemBytes  = 0;
launch_cfg.stream            = stream;
// Sets the PDL flag so cudaLaunchKernelExC overlaps
//   with Kernel A's epilogue
static cudaLaunchAttribute attrs[1];
attrs[0].id  = cudaLaunchAttributeProgrammaticStreamSerialization;
attrs[0].val.programmaticStreamSerializationAllowed =
    1;
launch_cfg.attrs    = attrs;
launch_cfg.numAttrs = 1;
// 6) Pack the pointer argument
void* kernelArgs[] = { &d_ptr };
// 7) Launch Kernel B kernel early using PDL
// Lookup device pointer for secondary_kernel
cudaKernel_t kB;
cudaGetKernel(&kB, kernel_B);
void* funcPtr_kernel_B = reinterpret_cast<void*>(kB);
cudaLaunchKernelExC(&launch_cfg, funcPtr_kernel_B, kernelArgs);
```

```
    // 8) Wait until all work in the stream completes
    cudaStreamSynchronize(stream);
    // 9) Cleanup
    cudaStreamDestroy(stream);
    cudaFree(d_ptr);
    return 0;
}
```

这里，内核 B 正在等待 cudaGridDependencySynchronize()，直到它收到来自内核 A 的编程启动完成信号。一旦发生切换，两个内核就可以重叠执行。通过将内核 A 中的触发器、内核 B 中的同步以及主机上的启动属性配对，此代码在内核 A 和 B 之间实现了尽可能多的重叠。

如本示例所示，在使用 PDL 时，您需要创建 cudaLaunchConfig_t 并在 cudaLaunchKernelExC() CUDA 调用中使用特殊属性。具体来说，在主机端，通过为内核 B 的启动配置 cudaLaunchConfig_t 来启用 PDL，并启用 cudaLaunchAttributeProgrammaticStreamSerialization 属性以允许早期重叠调度。

使用 cudaLaunchAttributeProgrammaticStream 序列化调用 cudaLaunchKernelExC() 告诉 CUDA 运行时将内核 B 排入队列是安全的 - 即使内核 A 尚未完全完成。然后使用 cudaLaunchKernelExC() 来执行具有这些扩展属性的实际启动，这依赖于触发机制来执行切换。

11.15 将 PDL 和线程块集群与 Warp 专业化相结合

让我们将三种正交技术（PDL、warp 专用流水线和线程块集群）整合到一个执行模型中。

PDL 为一个内核提供了一种机制，以发出其序言完成的信号并触发相关内核执行。然后，当相关内核加速并执行时，它会逐渐下降。

Warp 专门化将每个线程块细分为生产者和消费者 Warp。具体来说，生产者扭曲使用 TMA 异步传输图块，而计算扭曲执行矩阵乘法累加 (MMA) 操作。

线程块集群保证多个线程块运行在附近的 SM 组上。这有利于大规模工作负载的多 SM 协作和共享内存多播。

总之，内核间和内核内技术的结合隐藏了 DRAM 延迟，减少了内核边界开销，并最大限度地提高了 Tensor Core 利用率。他们创建的管道具有以下特征：

11.15.1 内核内流水线

Warp 专业化确保在每个线程块内，数据传输（生产者扭曲）和计算（消费者扭曲）使用多级管道完全重叠。

11.15.2 内核间重叠

PDL 允许依赖 GEMM 内核的序言（可能在神经网络的下一层上运行）在主内核完成预取数据（权重）后立即开始，而无需等待整个线程块被拆除。

11.15.3 区块间合作

线程块集群使线程块组能够协调预取（例如，多播）并在 SM 之间执行动态负载平衡。这样，生产者和消费者任务就可以在集群范围内均匀分布。

例如，您可以将模型权重的移动（数据传输）与同一内核内的 GEMM（计算）重叠，以便在计算一个图块的同时下一个图块正在运行。一个 warp 专用的多级软件管道（阶段 0⋯N）可以使用 PDL 和 mbarrier 原语来协调这些角色，如图 11-10 所示。

这是一个 CUDA C++ 示例，展示了如何将 PDL、线程块集群和扭曲专门化与简单的 TMA 样式的异步复制 + 计算管道结合起来。具体来说，主内核 primary_gemm 使用 cudaTriggerProgrammatic LaunchCompletion() 向辅助内核发出所有内存刷新已完成的信号

已完成并且数据已准备好可供使用。因此，现在辅助内核 secondary_gemm 可以安全地从 cudaGridDependencySynchronize() 继续，如下所示：

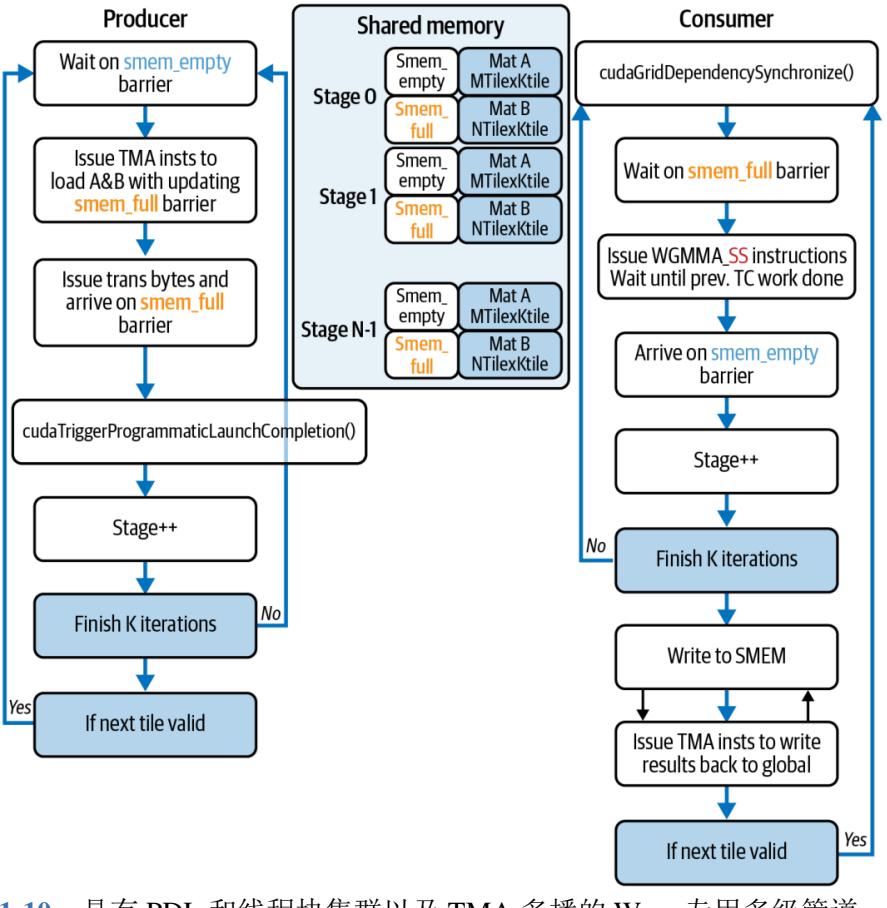

图 11-10　具有 PDL 和线程块集群以及 TMA 多播的 Warp 专用多级管道，用于高性能、内核间和内核内 GEMM 实现。

```
#include <cstdio>
#include <cuda_runtime.h>
// Cooperative Groups for clusters/barriers
#include <cooperative_groups.h>
// C++ barrier for TMA-like sync
#include <cuda/barrier>
// Async copy API
#include <cuda/pipeline>
namespace cg = cooperative_groups;
// Tile size for our toy GEMM
constexpr int TILE_M = 128;
```

```
constexpr int TILE_K = 128;
constexpr int TILE_N = 128;
// A very simple  “producer/consumer”  pipeline within each CTA
__global__ __cluster_dims__(2,1,1)    // Compile-time cluster of 2 CTAs
void primary_gemm(const float* __restrict__ A,
                  const float* __restrict__ B,
                        float* __restrict__ C,
                  int M, int N, int K)
{
    // Identify thread-block cluster & within-block group
    cg::thread_block_cluster cluster = cg::this_thread_block_cluster();
    cg::thread_block       cta     = cg::this_thread_block();
    int tid = threadIdx.x + threadIdx.y * blockDim.x;
    int warpId = tid / warpSize;
    const int numProducerWarps = 1;
    // Shared-memory tile buffers
    __shared__ float tileA[TILE_M * TILE_K];
    __shared__ float tileB[TILE_K * TILE_N];
    __shared__
    cuda::pipeline_shared_state<cuda::thread_scope_block, 1> pipe_state;
    auto pipe = cuda::make_pipeline(cta, &pipe_state);
    if (warpId < numProducerWarps) {
    pipe.producer_acquire();
    cuda::memcpy_async(cta, tileA, A, cuda::aligned_size_t<32>{TILE_M
                       * TILE_K * sizeof(float)}, pipe);
    cuda::memcpy_async(cta, tileB, B, cuda::aligned_size_t<32>{TILE_K
                       * TILE_N * sizeof(float)}, pipe);
    pipe.producer_commit();
    }
    cta.sync();
    pipe.consumer_wait();
    pipe.consumer_release();
```

```
// ... perform "compute" on the tile ...
// (e.g., a few fused multiply-adds)
do_compute();
    // Inter-CTA cluster-scope sync for load balancing
    cluster.sync();
    // Signal to dependent kernel that prologue is done
    cudaTriggerProgrammaticLaunchCompletion();
    // ... perform remaining epilogue work ...
    // ...
}
__global__ __cluster_dims__(2,1,1)
void secondary_gemm(const float* __restrict__ A,
                    const float* __restrict__ B,
                          float* __restrict__ C,
                    int M, int N, int K)
{
    // Wait for primary' s PDL signal before starting
    cudaGridDependencySynchronize();
    // Similar warp-specialized pipeline as above...
    //  (Omitted for brevity. Duplicate of primary logic,
    //  but reading from different offsets to compute
    //  next GEMM tile.)
}
// cudaLaunchKernelExC, cudaGetKernel
#include <cuda_runtime_api.h>
// cudaLaunchConfig_t
#include <cuda/launch_config.h>
int main()
{
    // Problem dimensions (must be multiples of TILE_)
    int M = TILE_M, N = TILE_N, K = TILE_K;
    // Allocate and initialize matrices A, B, C on device
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, M*K*sizeof(float));
    cudaMalloc(&d_B, K*N*sizeof(float));
    cudaMalloc(&d_C, M*N*sizeof(float));
```

```
    // (Initialize d_A, d_B via cudaMemcpy or kernels...)
    // Create a nonblocking stream for overlap
    cudaStream_t stream;
    cudaStreamCreateWithFlags(&stream,
        cudaStreamNonBlocking);
    // Launch primary GEMM


    dim3 gridDim(M/TILE_M, N/TILE_N), blockDim(256);
    primary_gemm<<<gridDim, blockDim, 0, stream>>>(d_A,
d_B, d_C, M, N, K);
    // Configure PDL attributes for secondary launch
    cudaLaunchConfig_t launch_cfg = {};
    launch_cfg.gridDim          = gridDim;
    launch_cfg.blockDim         = blockDim;
    launch_cfg.dynamicSmemBytes = 0;
    launch_cfg.stream           = stream;
    static cudaLaunchAttribute attrs[1];
    attrs[0].id  = cudaLaunchAttributeProgrammaticStreamSerialization;
    attrs[0].val.programmaticStreamSerializationAllowed = 1;
    launch_cfg.attrs    = attrs;
    launch_cfg.numAttrs = 1;
    // Prepare arguments and get function pointer
    //    for secondary_gemm
    void* kernelArgs[] = {&d_A, &d_B, &d_C, &M, &N, &K};
    cudaKernel_t k;
    cudaGetKernel(&k, secondary_gemm);
    void* funcPtr = reinterpret_cast<void*>(k);
    // Early enqueue of secondary GEMM via PDL
    cudaLaunchKernelExC(&launch_cfg, funcPtr, kernelArgs);
    // Wait for everything to finish
    cudaStreamSynchronize(stream);
    // Cleanup
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    cudaStreamDestroy(stream);
```

```
    return 0;
}
```

在主机上，您将主内核（primary_gemm) 启动到非阻塞流中，并使用 ProgrammaticStreamSerialization 属性创建 cudaLaunchConfig_t。您可以使用它来调用辅助内核的 cudaLaunchKernelExC（secondary_gemm).

在 primary_gemm 内部，对 cudaTriggerProgrammaticLaunchCompletion() 的调用标志着内存刷新的完成，并允许从属内核的处理开始——甚至在主内核完全拆除之前。

然后，从属内核调用 cudaGridDependencySynchronize().，它等待来自主内核的信号和必要的内存刷新，以便它可以开始与主内核的尾声并行执行。

在每个线程块中，我们使用 warp 专门化来重叠数据移动和计算。通过将每个块拆分为单个“生产者”warp 和多个“消费者”warp，生产者向共享内存发出 cuda::memcpy_async 调用。这使用 TMA 多播将单个 DMA 传输广播到集群中的所有线程块，如图 11-11 所示。

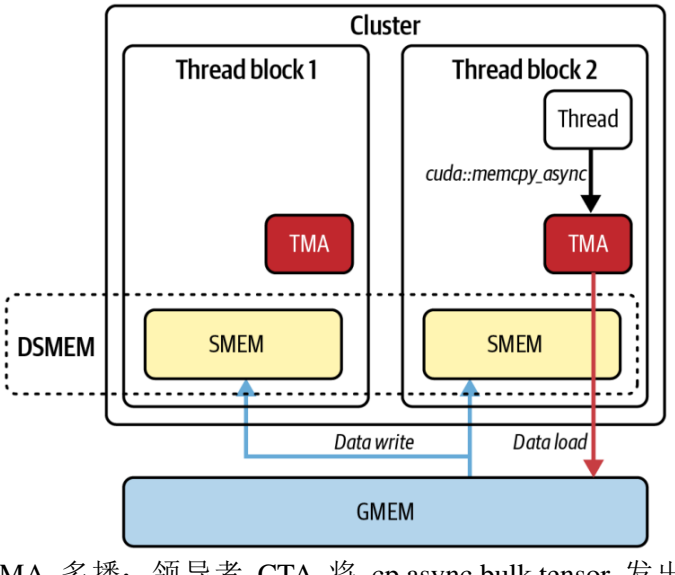

图 11-11 TMA 多播：领导者 CTA 将 cp.async.bulk.tensor 发出到 DSMEM（集群共享内存）；跟随者 CTA 使用 cluster.map_shared_rank 消耗图块；cuda::memcpy_async 驱动 TMA。

当生产者扭曲使用 TMA 多播加载数据时，消费者在执行矩阵乘法步骤之前在 C++ 块范围屏障（cuda::barrier<cuda::thread_scope_block>）上旋转（do_compute()).），这使得每个图块的加载及其融合乘加（FMA）操作在细粒度的多级软件管道中交织，隐藏了内核内部的全局内存延迟。

为了协调 SM 之间的工作，我们用 __clus ter_dims__(2,1,1) 注释两个内核。这会将附近 SM 上的线程块对分组。对 cluster.sync()（合作组对 PTX mbarrier 指令的包装器）的调用充当集群范围的屏障和共享内存围栏。这样，集群中的所有线程块都会看到相同的 TMA 加载的数据，并且可以动态地重新平衡剩余的切片。这种块间合作可以防止 SM 闲置，并增加扭曲专用管道的效益。

> **注** 当多个线程块需要相同的瓦片时，生产内核通常使用张量内存加速器 (TMA) cp.async.bulk.tensor 进行 2D 瓦片和跨线程块集群的多播。

> **注** 考虑使用基于描述符的 TMA 归约操作（例如 cp.reduce.async.bulk(.tensor)）在 Blackwell 上的平铺副本期间进行动态归约。将小型缩减融合到数据移动管道时，首选基于描述符的加载/存储加上 TMA 缩减操作。这将减少套准压力并改善重叠。

最后，让我们回顾一下这个高性能 GEMM 管道的三个特征：

11.15.4 内核内流水线

在每个线程块内使用 warp 专门化来细分工作，以便生产者 warp 发出 cuda::memcpy_async 块，而消费者 warp 在块范围屏障上旋转。这使得数据传输和计算操作完全重叠。

11.15.5 内核间流水线

使用 PDL，它可以在主内核完全拆除之前开始下一个 GEMM 内核的处理（使用 cuda TriggerProgrammaticLaunchCompletion() → cudaGridDependencySynchron ize() 机制）。这掩盖了内核启动的开销。

11.15.6 区块间合作

通过使用 __cluster_dims__ 注释内核，此代码创建在附近 SM 上协同调度的线程块对。除了调用 cluster.sync()（基于 mbarrier 的集群屏障）之外，这些线程块还共享 TMA 多播数据并在整个网格中动态负载平衡工作。

简而言之，通过使用 TMA 分层扭曲级流水线、集群级同步和多播以及使用 PDL 的内核间重叠，您可以形成一个高度重叠的管道，隐藏 DRAM 延迟、屏蔽内核启动开销并最大化 Tensor Core 利用率。其结果是在每个阶段都具有高性能的 GEMM：在 warp 副本内、跨线程块屏障内以及在内核切换期间。它们一起运行以保持硬件繁忙并避免停顿。

11.16 要点

本章涵盖了与 CUDA 流、流排序内存分配器、基于事件的同步、内核间管道、线程块集群和 PDL 相关的一些高级主题。这些有助于为推理和训练工作负载创建高效的管道。以下是一些关键要点：

11.16.1 显式流与默认流

避免使用传统的默认流（流 0），它会序列化所有工作并充当全局屏障。相反，创建显式非阻塞流（cudaStreamCreate WithFlags(..., cudaStreamNonBlocking)），以便内核和副本可以同时运行，而无需隐藏同步。

11.16.2 流排序内存分配器

使用 cudaMallocAsync 和 cudaFreeAsync 在特定流中分配/释放设备内存。这种非阻塞分配器将请求记录在流的队列中，避免全局设备同步，并允许与运行中的内核和副本重叠分配。

11.16.3 重叠 H2D、计算和 D2H

通过在不同的流上排队异步主机到设备（cudaMemcpyAsync）、内核启动和设备到主机的复制），您可以实现三向重叠。当一个流运行内核时，另一个流可以复制下一批 H2D，第三个流可以复制结果 D2H。这隐藏了延迟并减少了空闲时间。

11.16.4 用于细粒度同步的 CUDA 事件

使用 cudaEventRecord 和 cudaStreamWaitEvent 来协调跨流的生产者消费者依赖关系，而不会停止整个 GPU 或 CPU。事件使消费者流能够精确等待，直到生产者流完成副本或内核，从而保持最大并发性。

11.16.5 内核间流水线

将内核中的 warp 专用（多角色）、两级（双缓冲）管道与多流启动相结合。在单独的流中启动 warp 专用内核的多个实例（加载器 → 计算 → 存储器），将连续的小批量数据输入到 GPU 中。这将内核内内存/计算重叠与内核间并发结合起来。

11.16.6 带有流的线程块簇

将内核内扭曲专用管道扩展到网格范围的线程块集群（协作启动）允许加载器/计算/存储器跨块扭曲。在多个流中启动这些协作内核可以在协作内核执行时进行后续批次的主机端分配和复制。

11.16.7 内核内信令以及与 PDL 的重叠

一旦数据写入被刷新，内核 A 就会调用 cudaTriggerProgrammaticLaunchCompletion()，而内核 B 使用 cudaGridDependencySynchronize() 来等待该信号。这允许内核 B 的序言开始并与 A 的尾声重叠——无需 CPU 干预。

11.16.8 主机端 PDL 设置

主机通过 cudaLaunchKernelExC() 和设置 cudaLaunchAttributeProgrammaticStreamSerializa 的 cuda LaunchConfig_t 配置内核 B 的启动。这允许驱动程序尽早将 B 入队并最大化内核间重叠。PDL 在主内核中使用 cudaTriggerProgrammaticLaunchCompletion，在依赖内核中使用 cudaGridDependencySynchronize，并在主机上使用 cudaLaunchAt 贡品 ProgrammaticStreamSerialization 启动属性。

11.17 结论

总之，CUDA 流的内核间并发已经从手动优化演变为现代 AI 框架和 GPU 运行时使用的自动功能。通过了解流、资源占用、同步点等核心原理，开发人员可以最大化 GPU 的利用率。

通过在单个 GPU 内和多个 GPU 之间重叠内核执行和数据传输，内核间并发性对于在现代工作负载中最大化 GPU 利用率至关重要。随着硬件不断增加更多的并行性，而软件抽象出更多的调度复杂性，了解如何最大限度地提高并发性是充分利用高性能 AI 系统硬件的关键部分。

本章演示了如何跨多个 CUDA 流编排内核、内存操作和分配，以保持所有 GPU 硬件单元主动运行。这些硬件单元包括计算管道、DMA 引擎和互连。CUDA 流作为将内核、内存操作和分配放入独立队列的基本机制，允许 GPU 的计算引擎和 DMA 引擎同时运行。

通过避免默认流的隐藏障碍，利用流排序分配器，使用事件进行精确同步，并将内核内扭曲专业化与内核间多流管道相结合（从第 10 章扩展到线程块集群），即使对于复杂的 LLM 工作负载，您也可以实现接近峰值的利用率。

在多 GPU 上下文中，重叠的点对点传输、集体通信和跨不同流的计算进一步减少了空闲时间。我们回顾了第 10 章，展示了如何使用 CUDA 图捕获这些流工作流程，以减少重复迭代的 CPU 开销。

在下一章中，我们将通过引入动态内核编排和具有动态并行性和 CUDA 图的元调度来更深入地研究并建立在这些原则的基础上。我们将在运行时协调整

个内核管道和数据移动，以适应不断变化的工作负载。这种动态资源平衡和设备端编排将推动我们将大规模人工智能系统的性能优化提升到一个新的水平。

第 12 章 动态调度、CUDA Graphs 与设备端发起的 Kernel 编排

在前面的章节中，我们已经在单个 kernel 层面充分释放了计算与内存吞吐量。现在是时候对这些 kernel 进行编排，让 GPU 永不闲置了。

本章中，我们将从主机端调度转向设备端调度。我们将探讨由高速 L2 缓存原子操作驱动的动态工作队列，合并重复的 kernel 启动，并利用 CUDA Graphs 对固定流水线进行批量打包，从而将 CPU 侧的交互握手降到最低。

随后，我们将借助设备端图启动与动态并行把编排推向更远。它们让 GPU 自行决定接下来执行什么，而无须回调 CPU。

最后，我们将深入多 GPU 环境，通过重叠执行点对点拷贝、NCCL 集合通信、CUDA-aware MPI 以及 NVSHMEM 单边 put/get 操作，让一簇 GPU 的行为如同一个巨大的共享内存协处理器。以 NVIDIA 的 DGX GB200 NVL72 系统为例，它将 36 颗 Grace CPU 与 72 块 Blackwell GPU 连接成单一的 NVLink 域，提供统一寻址，域内 CPU 与 GPU 统一内存总容量高达 30 TB。这套系统支持跨 NVLink 互连结构在 72 GPU 域内远程访问 HBM。更大的 NVLink 网络拓扑还可以延伸到单个机架之外。

在此过程中，我们会把每项技术与 roofline 分析联系起来，帮助你为 kernel 选对工具——流（stream）、图、原子操作或动态 kernel——以提高其运算强度（operational intensity），进而改善整个工作负载的性能。

学完本章，你将理解动态的、设备端的以及基于图的 kernel 编排技术，它们能够让多 GPU 集群中的每个 SM 都持续有事可做。

12.1 基于原子工作队列的动态调度

线程之间的工作分配不均匀，会让一些 SM 在其他 SM 仍忙碌时处于闲置状态。这既浪费计算资源，也降低整体吞吐量。

当不同线程或线程块因输入相关的循环或条件性工作负载而处理的工作量不同时，这种不均衡便经常出现。一些块很快完成，其所在的 SM 随之闲置，而另一些 SM 则继续执行运行时间更长的块。在拥有数百个 SM 的现代 GPU 上，如果工作分配不均匀，闲置期会让大量 SM 无所事事，从而严重损害性能。

等到运行时间最长的工作完成时，GPU 的一部分已经闲置了许久。由于许多周期内没有任何活动的 warp，实际达到的占用率（achieved occupancy）随之下降。请记住，你可以用 Nsight Systems 进行性能分析，它会把这些空闲间隙清晰地呈现在 GPU 时间线上。

你还可以将活跃 SM 周期数与 SM 经过的总周期数加以比较，来衡量利用不足的程度。Nsight Compute 把它作为一个单独指标提供，表示至少有一个 warp 处于活跃状态的时间占比。活跃/经过周期比值（active-to-elapsed ratio）偏低，说明许多周期内没有活跃 warp；换句话说，GPU 经常处于闲置状态。

除了 Nsight Systems，你还可以用 Nsight Compute 查看实际占用率（achieved occupancy，即每 SM 活跃 warp 相对硬件最大值的平均比例）或 SM 活跃周期百分比（SM Active cycles percentage，即至少有一个 warp 处于活跃状态的时间占比），以量化这种利用不足。

> **注** 要在时间线上的空闲间隙与具体代码段之间建立对应关系，可在重要的 GPU 工作前后插入 NVTX 范围标记（range marker）。

接下来，我们将讨论如何实现原子队列，以便在 kernel 内部动态分配工作。原子队列对于在所有 SM 之间均衡任意工作负载、避免线程闲置非常重要。在此之前，我们需要先介绍原子计数器。

12.1.1 原子计数器

原子计数器是原子队列的基础，而原子队列能够实现动态的工作分配。

在现代 GPU 上，全局原子操作由片上 L2 缓存提供服务并在此串行化。当目标缓存行驻留时，其延迟比往返 DRAM 的方案低。原子计数器仍会产生延迟，并且在争用情况下被串行化。但无争用的 `atomicAdd` 操作完全留在片上执行，速度极快。图 12-1 展示了两个线程对一个原子变量执行递增的例子。

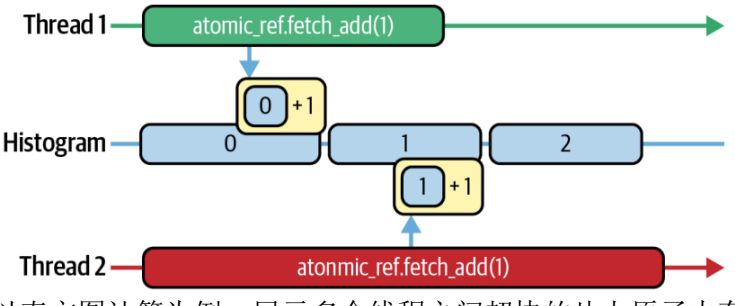

图 12-1　以直方图计算为例，展示多个线程之间超快的片上原子内存加法操作

然而，`atomicAdd` 并非没有代价。它仍有延迟，并且在争用情况下会把等待访问同一内存地址的线程串行化。相应地，L2 也必须对这些更新串行处理，从而形成需要优化的热点。Nsight Compute 可以帮你量化这一代价。

在 Nsight Compute 的 Memory Workload Analysis 部分，你可以找到 `atomic_transactions` 和 `atomic_transactions_per_request`。`atomic_transactions` 计数器表示 L2 缓存原子事务的总数，其中包括因争用引起的任何重放。`atomic_transactions_per_request` 指标（或称争用比）表示每条 `atomicAdd` 指令平均产生的 L2 事务数。

当每条 `atomicAdd` 恰好触发一次 L2 事务时，`atomic_transactions_per_request` 会徘徊在 1.0 附近，说明付出的开销已经是最低限度。如果该比值升至 1.0 以上，则表明线程在停顿并重试原子更新，而不是在做有用的工作。每一次重试都意味着发生了争用。

这里的优化思路，是通过批量获取工作来摊薄原子操作的开销。也就是说，不再由每个线程或每个 warp 各自执行一次 `atomicAdd`，而是让每次原子操作领取一组任务。下面给出批大小为 32 时优化前后的对比：

```
// Before batching
int idx = atomicAdd(&queue_head, 1);
if (idx < N) process(data[idx]);
// After batching
const int batchSize = 32;
int start = atomicAdd(&queue_head, batchSize);
for (int i = start; i < start + batchSize && i < N; ++i) {
    process(data[i]);
}
```

这样一来，一次原子更新就为某个 warp（或线程块）分配了一整片工作——本例中是 32 个元素——然后才再次触碰计数器。每个批次你仍然只需支付一次 L2 事务，但两次之间完成的有效工作却是原来的 32 倍。

实践中，每个 warp 只有唯一一个线程执行这个 `atomicAdd` 来获取下一批的起始索引，然后该线程将索引广播给 warp 中的其余线程（例如使用 `__shfl_sync`）。随后整个 warp 并行处理这 32 个元素。于是原子操作从每线程一次变为每 warp 一次，争用被大幅减少。

在 Nsight Compute 中，你会看到 `atomic_transactions` 骤降，transactions-per-request 也回落到 1.0 附近。这证明你用持续不断的计算换掉了代价高昂的争用。

在现代 GPU 上，L2 缓存原子操作异常迅速，由于 L2 带宽很高，即使 8 或 16 这样适度的批大小也足以消除大部分争用。话虽如此，务必确认你并没有只是把瓶颈转移到了别处。

为了核实这项优化没有对其他性能指标造成负面影响，请使用 Nsight Compute 的 Warp Stall Reasons 与 Register Pressure 报告，确保融合后的循环没有转而受制于寄存器溢出（register spill）或共享内存 bank 冲突。

> **注** 如果经过这些优化后原子操作依然是热点，可以考虑其他设计方案，例如每块计数器（per-block counter）或基于分层归约的工作分发方式。

简而言之，通过按原子操作批量领取工作，你可以让 GPU 上的众多 warp 忙于真正的计算，而不是在一个跟不上节奏的单一计数器前排队等待。

12.1.2 原子队列

现在，我们用一个全局原子计数器来协调一个动态工作队列。目标是利用该原子计数器和 atomicAdd，在所有 SM 之间均衡任意工作负载，让任何线程或 warp 都不闲置。这种动态工作队列的一个示例如图 12-2 所示。

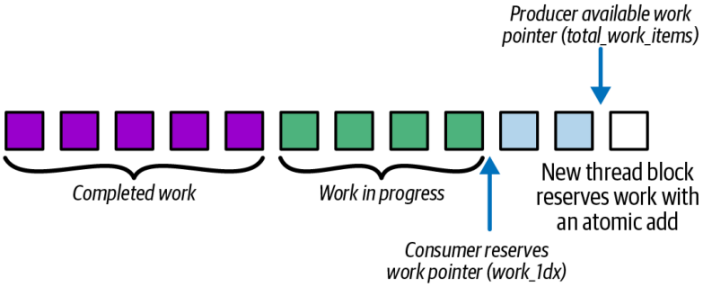

图 12-2　使用原子计数器和 atomicAdd 构建动态工作队列，在 SM 与 warp 之间均衡工作负载

在接下来的代码示例（computeKernel）中，每个线程根据 idx % 256 计算数量不等的迭代。idx % 256 值较小的线程几乎不做任何工作，而 idx % 256 值较大的线程则要完成大量工作。由于这种负载不均衡，各线程在不同时刻结束，部分 SM 会空转等待最慢的线程完成。以下是每个线程分配到静态的、不均衡工作负载的代码：

```
// uneven_static.cu
#include <cuda_runtime.h>
#include <cmath>
__global__ void computeKernel(const float* input, float* output, int N)
↪ {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        // Each thread does a variable amount of work based on idx
        int work = idx % 256;
        float result = 0.0f;
        for (int i = 0; i < work; ++i) {
            result += sinf(input[idx]) * cosf(input[idx]);
        }
        output[idx] = result;
```

```
    }
}
int main() {
    const int N = 1<<20;
    float* h_in = nullptr;


    float* h_out = nullptr;
    cudaMallocHost(&h_in, N * sizeof(float));
    cudaMallocHost(&h_out, N * sizeof(float));
    for (int i = 0; i < N; ++i) h_in[i] = float(i) / N;
    float *d_in, *d_out;
    cudaMalloc(&d_in, N * sizeof(float));
    cudaMalloc(&d_out, N * sizeof(float));
    cudaMemcpy(d_in, h_in, N * sizeof(float), cudaMemcpyHostToDevice);
    dim3 block(256), grid((N + 255) / 256);
    computeKernel<<<grid, block>>>(d_in, d_out, N);
    cudaDeviceSynchronize();
    cudaMemcpy(h_out, d_out, N * sizeof(float), cudaMemcpyDeviceToHost);
    cudaFree(d_in);
    cudaFree(d_out);
    cudaFreeHost(h_in);
    cudaFreeHost(h_out);
    return 0;
}
```

> **注** PyTorch 没有提供面向 GPU 端动态工作分发的高层 API，因此我们需要用自定义 CUDA kernel 来实现这一机制。为简洁起见，这里不再展开。

接下来给出的优化后的动态任务分发版本使用一个全局计数器（存放于设备内存）作为 warp 级工作队列。我们借助批量原子操作把该计数器改造为持久的 warp 级工作队列。这样，提前完成的 warp 会立即领取下一批任务，而不是空转闲置：

```
// uneven_dynamic.cu
#include <cuda_runtime.h>
__device__ unsigned int globalIndex = 0;
// Warp-batched dynamic queue: 1 atomic per active warp
__global__ void computeKernelDynamicBatch(const float* input,
                                          float* output,
                                          int N) {
  // lane id in [0,31]
  int lane = threadIdx.x & (warpSize - 1);
  while (true) {
    // Elect an active leader each iteration (handles divergence safely)

    unsigned mask = __activemask();
    int leader = __ffs(mask) - 1;
    // Warp leader atomically grabs a contiguous batch for the whole warp
    unsigned int base = 0;
    if (lane == leader) {
      base = atomicAdd(&globalIndex, warpSize);
    }
    // Broadcast starting index to all active lanes in the warp
    base = __shfl_sync(mask, base, leader);
    unsigned int idx = base + lane;
    if (idx >= (unsigned)N) break;  // dynamic termination
    // Hoist invariants out of the variable trip-count loop
    // Note: You can also use __sincosf on Blackwell
    float s = sinf(input[idx]);
    float c = cosf(input[idx]);
    int   work = idx % 256;
    float result = 0.0f;
    #pragma unroll 1
    for (int i = 0; i < work; ++i) {
      result += s * c;
    }
    output[idx] = result;
    // loop continues until counter >= N
  }
```

```
}
int main() {
    const int N = 1 << 20;
    float *d_in, *d_out;
    cudaMalloc(&d_in,  N * sizeof(float));
    cudaMalloc(&d_out, N * sizeof(float));
    // Host buffers (pinned) for a realistic data path
    float *h_in = nullptr, *h_out = nullptr;
    cudaMallocHost(&h_in,  N * sizeof(float));
    cudaMallocHost(&h_out, N * sizeof(float));
    for (int i = 0; i < N; ++i) {
        h_in[i] = static_cast<float>(i % 1000);
    }
    // Copy inputs to device
    cudaMemcpy(d_in, h_in, N * sizeof(float),
        cudaMemcpyHostToDevice);
    // Reset global counter
    unsigned int zero = 0;
    // If you call this kernel repeatedly (e.g., in a loop),
    // reset 'globalIndex' to 0 before each launch.

    cudaMemcpyToSymbol(globalIndex, &zero,
        sizeof(unsigned int));
    // Launch with 256 threads per block
    dim3 block(256), grid((N + 255) / 256);
    cudaStream_t stream;
    cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking);
    computeKernelDynamicBatch<<<grid, block, 0, stream>>>(d_in, d_out,
    ↪ N);
    cudaStreamSynchronize(stream);
    cudaStreamDestroy(stream);
    cudaDeviceSynchronize();
    // Copy results back and clean up
    cudaMemcpy(h_out, d_out, N * sizeof(float),
               cudaMemcpyDeviceToHost);
    cudaFree(d_in);
```

```
    cudaFree(d_out);
    cudaFreeHost(h_in);
    cudaFreeHost(h_out);
    return 0;
}
```

每个 warp 都通过原子操作从全局队列中领取下一批大小为 `warpSize`（32 个线程）的任务，并在循环中逐个处理。这确保了没有任何 SM 会陷入空闲。这段代码展示了用单个全局原子工作队列实现的动态工作分发。

在这里，每个 warp 反复从该全局计数器拉取下一批任务的基址索引。每个 warp 中的第一个线程（`if (lane==0)`），即 warp leader，通过原子加法获得这段连续块的起始索引，即 `base = atomicAdd(&globalIndex, warpSize)`；然后按照本章前面所述，使用 `__shfl_sync(__activemask(), base, 0)` 把这个 base 广播给 warp 中的其余线程。

换言之，每个线程不再被绑定到固定的元素索引，而是由每个 warp 从共享计数器中领取一段连续的任务块，然后按 `idx = base + lane` 进行计算。

该 warp 中的所有线程对自己动态领取的索引执行同样的 sin/cos 循环。因此，工作不再预先指派给每个线程，而是在运行时通过全局原子队列进行拉取与均衡。

注意，`if (idx >= N)` 边界检查会在没有剩余工作可做时使 warp 退出，从而避免越界内存访问。除此之外，warp 中的每个线程执行的 sin/cos 循环与静态版本完全相同。

在一个简单的微基准测试中（$N = 1 \ll 20$，$work = idx\%256$），静态分配的 kernel 耗时约 200 ms，而动态队列版本大约只需 100 ms。这 2× 的加速得益于消除了 SM 空闲时间并降低了原子操作争用。Nsight Compute 将活跃 SM 周期（active SM cycles）定义为至少有一个活跃 warp 的周期占总耗用周期的比例。

加速幅度取决于负载不均衡的程度，但只要性能分析显示存在 warp 空转停顿、实际占用率偏低，或各任务执行时长不均造成的可见时间线空隙，动态工作分发就值得尝试。在这些场景下（尤其是中等程度的不均衡），通常可以获得 10%–20% 的加速。

> **注** 在极端不均衡的情况下，只需把静态索引方式替换为原子驱动的工作队列，即可获得 2× 的加速。而不均衡程度较轻时，原子操作和 shuffle 带来的开销可能会抵消收益。

简言之，动态工作分发能确保 SM 利用率接近均匀，因为每个 warp 都会不断领取并处理新任务，直到计数器超过 N。这与“许多 warp 早早完成工作，而最慢的 warp 仍未结束，硬件资源被闲置”的情形形成鲜明对比。

12.2 CUDA Graphs

当流水线由多个 kernel、数据拷贝、流事件记录和回调组成时，每次迭代都在主机端逐一启动它们，仍会产生 CPU 开销。CUDA Graphs 允许你一次性捕获整个工作流，然后以几乎为零的 CPU 开销反复重放。图 12-3 对比了不使用（上）与使用（下）CUDA Graphs 时的 kernel 启动时间线。

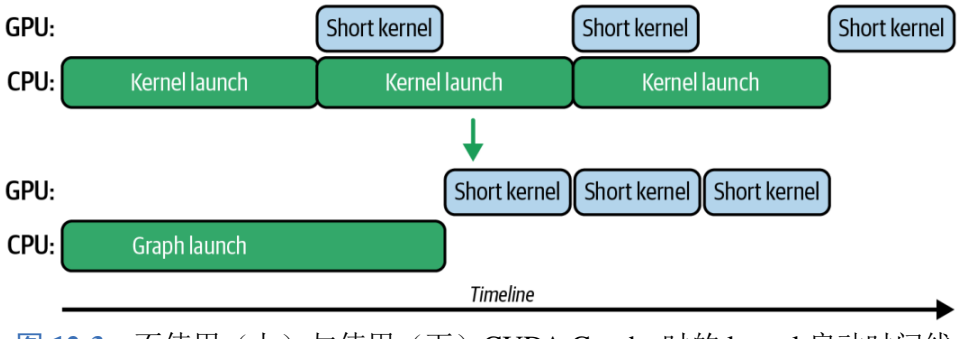

图 12-3 不使用（上）与使用（下）CUDA Graphs 时的 kernel 启动时间线

为什么要使用 CUDA Graphs？首先，它们能降低启动开销：多个小 kernel 或拷贝只需一次 CPU 调用即可完成启动。其次，它们能改善 GPU 端的调度——工作以批次形式提交，CUDA 驱动程序因此有可能减少操作之间的部分内部延迟。

此外，使用 CUDA Graphs 时依赖关系在一开始就是已知的，CPU 无需频繁地在中间进行同步。就内存传输而言，CUDA Graph 能确保异步拷贝与 kernel 执行按依赖关系正确衔接，无需任何手动同步。它并不会比普通流带来更多的拷贝与 kernel 重叠，但能让二者的执行更加顺畅高效。

12.2.1 PyTorch、推理引擎与 CUDA Graphs

PyTorch 等 AI 框架会在底层利用 CUDA Graphs 来执行深度学习模型中的静态部分。具体来说，PyTorch 提供了 `torch.cuda.Graph` 上下文，用于捕获一段操作序列。此外，PyTorch 也在持续优化其内部实现，将 CUDA Graphs 应用于代码中行为可预测的部分。

vLLM 和 NVIDIA TensorRT-LLM 等高性能推理引擎同样可以利用 CUDA Graphs：它们把模型的执行过程捕获为一组针对不同序列长度区间和输入批次大小预定义的图。启用图捕获后，这些系统通常会对输入进行分桶（bucket）或填充（pad），使其符合支持的图批次大小，从而保证捕获的图能以固定形状重放。对于大规模生产推理工作负载，这可以显著降低延迟。

例如，可以在启动阶段或模型加载阶段，为每个批次大小各捕获一个 CUDA Graph；运行时再根据到来请求的批次大小，启动预先捕获好的对应图。

> **注** PyTorch 编译模式 `mode='reduce-overhead'` 可能把符合条件的代码段包裹进 CUDA Graphs 以降低启动开销，但前提是满足捕获要求，例如张量地址必须静态、代码区域必须仅限 CUDA。它并不保证对所有代码路径都能成图，并且由于使用了池化缓冲区，可能增加内存占用。请务必通过性能分析来确认其对你的模型是否有收益。

12.2.2 CUDA Graphs 的内存池

一个重要的考量是 CUDA Graphs 中的内存管理。CUDA Graph 内部的内存操作遵循与 CUDA 流相同的规则。如果你在捕获期间分配 GPU 内存，该分配就会成为图执行的一部分。

通常应避免在图内分配 GPU 内存，而应在图外预先分配好内存。许多框架（如 PyTorch）在 CUDA Graphs 中使用静态内存池，如图 12-4 所示。使用静态内存池可使内存分配不会成为捕获的图序列的一部分。

虽然 CUDA Graphs 不会让单次内存拷贝或 kernel 执行本身变快，但它可以在图内自动重叠相互独立的数据传输与计算——类似于 CUDA 流。这消除了每次迭代的 CPU 调度开销，而其之所以可行，是因为依赖图在一开始就是已知的。

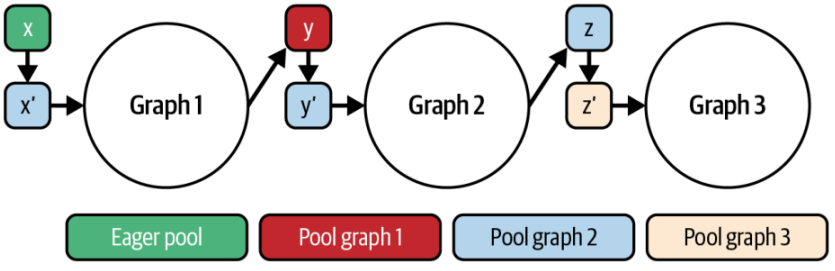

图 12-4　PyTorch 为 CUDA Graphs 使用静态内存池

12.2.3 用 CUDA 流捕获 CUDA Graph

要捕获一个图，需要在某个流上调用 `cudaStreamBeginCapture()`，然后依次将所有内存传输（`cudaMemcpyAsync()`）、kernel 启动、事件（`cudaEventRecord()`）和回调（`cudaLaunchHostFunc()`）加入队列，最后调用 `cudaStreamEndCapture()` 来创建 CUDA Graph 定义（`cudaGraph_t`）。

随后，CUDA 驱动程序便可以在每次迭代中用 `cudaGraphLaunch()` 启动这份 CUDA Graph。由于 CUDA 驱动程序事先掌握了整个依赖图，它可以直接在 GPU 上重放预先构建好的流序列，启动开销极小。

> **注**　下一节讨论的 `cudaGraphExecUpdate` 允许对捕获得到的图做有限的修改，适用于大小、维度或指针在迭代之间发生变化的情况。当输入尺寸会变化时尤其有用：你只需更新图的节点参数，而不必为每个新的输入尺寸重新捕获一份全新的图。

即使流水线中只有一部分是重复性的，CUDA Graphs 也只捕获这一部分。例如，如果你总是先执行一次 Host → Device 拷贝，接着运行两个 kernel，再做一次 Device → Host 拷贝，那么就可以只捕获这个子图，并用一次函数调用重放它。

重放 CUDA Graph 时，你只需提供流句柄，GPU 就会在没有额外 CPU 指令的情况下执行这一操作序列。这与 kernel 间并发直接相关：借助 CUDA Graphs，你可以混用异步拷贝、细粒度事件屏障和 kernel 来维持复杂的重叠行为，同时彻底消除 CPU 这一瓶颈。

通常，你会先做一次重放演练来确认正确性：先创建一个 `cudaGraphExec_t` 类型的可执行图来实例化图，然后用一次 graph 重放调用来启动它。启动捕获的图之后，运行时会在 GPU 上以正确的顺序执行其中的所有操作。

为展示 CUDA Graphs 的用法，考虑一个简单的 kernel 序列。下面给出在 C++ 和 PyTorch 中捕获并启动 CUDA Graph 的代码片段：

```
cudaStream_t stream;
cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking);
cudaGraph_t graph;
cudaGraphExec_t instance;
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
// Enqueue operations on 'stream' as usual


kernelA<<<grid, block, 0, stream>>>(d_X);
kernelB<<<grid, block, 0, stream>>>(d_Y);
kernelC<<<grid, block, 0, stream>>>(d_Z);
cudaStreamEndCapture(stream, &graph);
cudaGraphInstantiate(&instance, graph, nullptr, nullptr, 0);
// Now 'instance' can be launched in a loop
for (int iter = 0; iter < 100; ++iter) {
    cudaGraphLaunch(instance, stream);
    // No per-kernel sync needed; graph ensures dependencies
}
cudaStreamSynchronize(stream);
// (Destroy graph and instance when done)
cudaGraphExecDestroy(instance);
cudaGraphDestroy(graph);
```

在这段伪代码中，我们在一条 CUDA 流上开始捕获，依次在该流上启动三个 kernel（A、B、C），然后结束捕获得到一个图，接着对图进行实例化。

此后，我们可以通过 `cudaGraphLaunch(instance, stream)` 按需多次重放 $A \to B \to C$ 这段完整序列：每次迭代只需付出一次启动的代价，而不是三次分别的启动。GPU 会按照捕获时的顺序执行 kernel A、B 和 C，在本例中也就是背靠背地连续执行。我们会在循环结束后同步一次，以确保所有迭代都已完成。

如前所述，PyTorch 等高层 AI 框架支持 CUDA Graphs。PyTorch 让 Python 开发者无须深入掌握 CUDA C++ 就能获得接近原生的 CUDA 性能。这里我们展示 PyTorch 的 `torch.cuda.Graph` 上下文管理器如何捕获并重放操作：

```
import torch, time
X = torch.randn(1<<20, device='cuda')
# Define operations for reference
def opA(x): return x * 1.1
def opB(x): return x + 2.0
def opC(x): return x.sqrt()
# Persistent buffers for pointer stability
static_x = torch.empty_like(X)
static_y = torch.empty_like(X)
static_z = torch.empty_like(X)
static_w = torch.empty_like(X)
# Warm up once to initialize CUDA kernels and caches
_ = opC(opB(opA(X)))
torch.cuda.synchronize()


# Seed the static input before capture
static_x.copy_(X)
# Capture the graph
g = torch.cuda.CUDAGraph()
stream = torch.cuda.Stream()
torch.cuda.synchronize()
with torch.cuda.graph(g, stream=stream):
    # Record A then B then C using out parameters to avoid allocations
    torch.mul(static_x, 1.1, out=static_y)
    torch.add(static_y, 2.0, out=static_z)
    torch.sqrt(static_z, out=static_w)
# Replay the captured graph 100 times
for i in range(100):
    # If inputs change, copy new values into static_x before replay
    # static_x.copy_(new_X)
    g.replay()
```

在生产环境中，应当分配持久的输入和输出缓冲区，使捕获的 kernel 看到固定的内存地址。例如，先在捕获前创建 `static_y = torch.empty_like(X)`，然后在图内执行 `static_y.copy_(opA(X))`。这样可以避免捕获期间的内存分配，并满足 CUDA Graph 指针的稳定性规则。PyTorch 的 CUDA Graphs 要求重放时，被捕获的张量必须使用与捕获时相同的内存地址。

在这个 PyTorch 示例中，我们定义了操作 `opA`、`opB` 和 `opC`；在实际应用中，它们可以是神经网络层，也可以是任意的 GPU 操作。我们先运行一遍 warm-up `opC(opB(opA(X)))`，以确保所有 kernel、内存分配以及库上下文（例如 cuBLAS/cuDNN）都被提前初始化。这一步是必需的，因为 CUDA Graphs 的捕获不会记录这些惰性初始化步骤。

> **注** 跳过 warm-up 环节可能导致图捕获失败，或在惰性初始化发生时引入出人意料的停顿。

我们首先通过把该序列完整运行一遍来对 GPU 做 warm-up，以初始化所有 kernel 和库。然后，我们在一条全新的 CUDA 流上，将变换（A）、转换（B）和回传（C）操作包裹进 `with torch.cuda.graph(g, stream=stream)`：这个 Python 上下文管理器块中，捕获成一张 `torch.cuda.CUDAGraph`。捕获完成后，调用 100 次 `g.replay()`，每次迭代只需一次主机调用即可启动整条 A → B → C 流水线。结果汇总在表 12-1 中。

表 12-1 CUDA Graphs 对迭代开销的影响。

指标	使用 CUDA Graphs 之前	使用 CUDA Graphs 之后
每 100 次迭代的 CPU 启动调用次数	300 次独立的 kernel 启动	100 次 graph 重放（每次迭代 1 次）
主机同步调用次数	300 次 `cudaDeviceSynchronize` 调用	0
kernel 之间的平均 GPU 空闲时间	每次迭代约 3 µs 的间隙	0 µs（连续背靠背执行）
端到端迭代延迟	约 1.00 ms	约 0.75 ms（快 25%）

%（此源块为表 12-1 经扁平化提取的表格正文，内容已并入上方的表 12-1 浮动体，按出版惯例不再重复排版。）

> **注** 所有指标表格中的数值仅用于说明概念。不同 GPU 架构上的实际基准测试结果请参见 GitHub 仓库。

从这里可以看到，CUDA Graphs 消除了每次迭代的 CPU 调度以及主机与设备之间的握手，因为每次迭代的 GPU 工作被合并进单次 `g.replay()` 调用，而不是三次分别的 kernel 启动。其结果是迭代速度提升 25%：CPU 只需发出轻量的重放命令，并全程与 GPU 保持异步。

使用 CUDA Graphs 有一些常见陷阱，妥善处理它们很重要。例如，如果工作负载的规模发生变化，已捕获的图可能不再有效。这时就需要重新捕获，或者调用 `cudaGraphExecUpdate`，具体见下一节。

某些 CUDA API 调用，例如内存分配以及主机与设备之间的同步原语，一般不应纳入图的捕获范围。尽管新版 CUDA Graphs 支持在捕获的图内执行有限的内存管理操作，但仍然建议在捕获之前完成所有内存分配。你还必须确保图中使用的数据始终位于相同的内存地址。

> **注** 图执行期间内存必须保持在同一地址，这一点正是 PyTorch 等框架在 CUDA Graphs 中采用静态内存池的主要原因。例如，PyTorch 提供 `torch.cuda.graph_pool_handle()` API，用于创建一个专用内存池，以便进行指针稳定的 CUDA graph 捕获。使用独立的分配器池可以保证张量地址在多次捕获和重放之间保持固定，从而满足指针稳定性要求。在迭代之间，应通过把新数据拷贝进静态张量来更新输入；不要在每次迭代时重新分配张量。

还应避免在 CUDA Graphs 捕获中包含任何主机侧回调或不受支持的操作，例如 `print()`、随机数生成器（RNG）调用、嵌套捕获以及新的内存分配。这是因为图必须记录一段纯粹、确定性的 GPU 工作序列。

此外，参与捕获的所有张量都必须已经按固定形状分配在固定地址上。在捕获期间改变张量大小或调用 `cudaMalloc` 都会破坏图。

12.2.4 图的动态更新

记录了 CUDA Graph 之后，不要仅仅因为某些启动参数发生变化就把它丢弃。无须重新捕获，你可以调用图更新 API，直接在既有图中更新 grid/block 维

度、指针地址或 kernel 参数。图更新 API 包括 `cudaGraphExecUpdate` 以及更底层的 `cudaGraphExecKernelNodeSetParams`。

`cudaGraphExecUpdate` 允许你把一个 kernel 节点替换为一个同形状的新节点。例如只要形状相同，你就可以换入另一个融合 kernel 的实现。CUDA 运行时会校验这些修改，并让你立即重放修改后的图，从而省去完整捕获的开销。

> **注** 截至撰写本文时，你不能随意增删节点。如果某次更新超出现有图所能承受的范围，运行时会返回错误；此时你必须重新捕获一个新的图。

例如，回到前面用最大 batch 大小构建的三 kernel 图 A $\rightarrow$ B $\rightarrow$ C。在每次推理循环中，你只需把 kernel B 的启动维度更新为当前 batch 大小，然后重放同一张图即可。对于整体流水线固定、仅有少数参数可能变化的半静态工作负载，这能降低开销。

在实践中，典型的工作流程分三步。第一步，使用预期的最大尺寸（例如最大 batch）捕获一份模板图。比如，你可以按最大 batch 为 128 来捕获图；之后如果来了一个 batch 为 64 的请求，就调用 `cudaGraphExecUpdate` 把启动参数调整为 64——可能还需要把内存指针更新到某个更小的缓冲区。

使用 `cudaGraphExecUpdate` 之后，当 kernel 参数、grid/block 维度或内存地址发生变化时，你不必重建图。而且它只需几微秒，因此能够保留 CUDA Graph 重放低于 100 µs 的快速启动开销。此外，你还保留了在运行中灵活

调整这些关键参数的能力。注意，不兼容的修改会返回错误状态，此时需要重新捕获图。

但如果确实需要改变图的结构——例如指定不同的 kernel 数量——可以退回到“先重新捕获、再更新”的工作流。这时，用 `cudaStreamBeginCapture` 和 `cudaStreamEndCapture` 把代码的一次迭代包裹起来，构建出新图；此后在后续运行中，只需用轻量级的 `cudaGraphExecUpdate` 做小的调整。

实际上，动态图更新让你能够完全在 GPU 上创建“参数化”的或带条件的执行路径。只要你有一段高频循环执行的 GPU 工作，而其中只有少数参数（如批量大小）在变化，就可以只捕获一次、随后快速更新，同时兼顾极低的 CPU 开销和该用例所需的适应性。

12.2.5 设备端发起的 CUDA 图启动

现在你已经了解如何以低开销从 CPU 启动并调整捕获的流水线，下一步是把 CPU 从启动决策中彻底移除。借助设备端发起的 CUDA 图启动，一个正在运行的 GPU kernel 可以直接在设备上触发预录制的图，完全不经过主机。

要启用设备端启动，先像往常一样在主机端捕获图，然后用 `cudaGraphInstantiate` 实例化，并传入 `cudaGraphInstantiateFlagDeviceLaunch`。实例化之后，在任何设备端启动之前，先通过主机流用 `cudaGraphUpload` 上传可执行图。

随后用 `cudaGraphUpload` 把图上传到 GPU 内存。这一步必须在 GPU 启动该图之前完成。（若未经上传就尝试设备端启动，会导致错误。）

在实践中，这相当于在你的持久化或动态 kernel 内部嵌入一个“图启动”节点，或者调用设备端图 API。时机一到，GPU 就会在它自己拥有的流上启动整张图，如图 12-5 所示。

设备端发起的图启动让数据驱动的工作流完全留在 GPU 上。负责计算决策条件的是 kernel 本身，而不是 CPU。因此，kernel 可以直接生成下一张图，省去与 CPU 的往返，进一步降低延迟。

由于图已经驻留在 GPU 上，无需 CPU-GPU 握手，设备端启动把主机调度从关键路径上移除，在受主机限制的循环中可以降低端到端延迟。实践中，设备端发起的 CUDA 图启动相比等价的主机端图启动，启动延迟大约降低 2 倍；而且即便图的规模或复杂度增大，其开销依然保持平稳。

设备端启动的图延迟不受图中节点数量或并行分支数量的影响。这与主机端启动形成对比：图越大，主机端启动延迟会因 CPU 调度开销而增加。

此外，设备端启动随图宽度的扩展性也很好。随着并行节点增多，主机端启动会承受额外的同步开销，而设备端启动延迟几乎保持平坦。

调试设备端启动的图可能比较棘手，但 Nsight Systems 这类工具会把子图作为独立的工作流显示在 GPU 时间线上。建议在父 kernel 中、`cudaGraphLaunch` 调用的前后放置 NVTX 标记，以标明设备端启动发生的位置。这有助于验证图相对于父线程是否按预期运行。

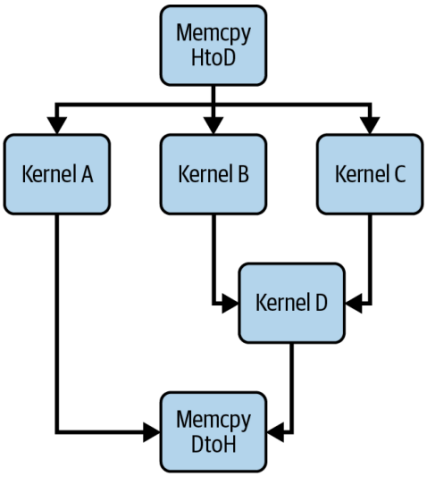

图 12-5　由 CUDA 图启动的操作序列（kernel 与数据传输节点）及其依赖关系（边）

在设备代码中，用简单的 API `cudaGraphLaunch(graphExec, stream)` 即可启动图。运行时使用特殊的保留 `cudaStream_t` 值来区分以下几种受支持的启动模式：“即发即弃”（`cudaStreamGraphFireAndForget`）、“尾启动”（`cudaStreamGraphTailLaunch`）和“同伴”（`cudaStreamGraphFireAndForgetAsSibling`）。这些模式会自动在 CUDA 流中强制正确的顺序，无需主机干预。

在即发即弃（fire-and-forget）启动中，子图立即开始执行，并与发起它的父 kernel 并发运行。父 kernel 不会等待子图完成——就像启动了一条独立的工作线程。即发即弃启动适合在 kernel 内部派生异步任务。

> **注**　一张图在整个执行过程中最多可以有 120 个即发即弃图。

相比之下，设备端发起的图尾启动（tail launch）会把图的执行推迟到发起 kernel 到达同步点或完成为止。这实际上是把该图排入队列，作为当前 kernel 之后的延续来运行，如图 12-6 所示。

尾启动在实现常驻 GPU 的工作调度器时尤其强大。一个持久的“调度器”kernel 可以尾启动一张图，然后在该图完成后重新启动自身。这一技术实际上在 GPU 上构建了一个循环，而无需主机再次调用。为了重新启动自身，kernel 调用 `cudaGetCurrentGraphExec()` 获取自己正在执行的图的句柄，然后用 `cudaGraphLaunch(…, cudaStreamGraphTailLaunch)` 把自己再次入队。

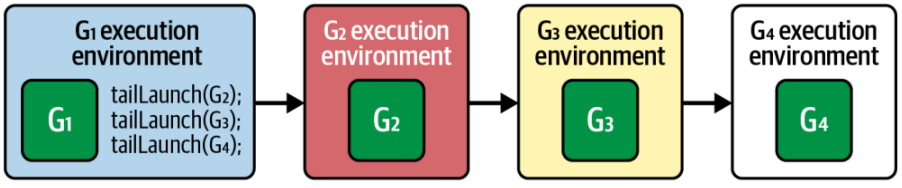

图 12-6　由给定图入队的尾启动将按入队顺序逐一执行

此外，尾图本身还可以执行更多的尾启动。在这种情况下，新的尾启动会在前一张图的尾启动之前执行，如图 12-7 所示。

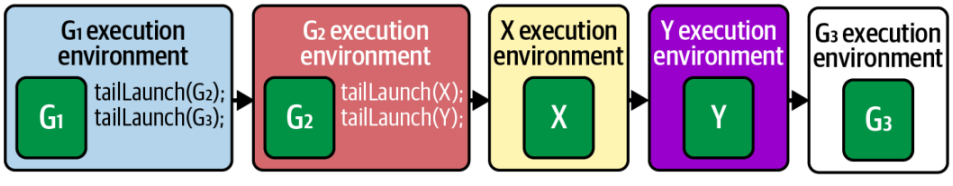

图 12-7　由多张图入队的尾启动

注　一张 CUDA 图中最多可以有 255 个待处理的尾启动。但对于自尾启动（即图把自己入队以便重新启动），同一时刻只能有一个待处理的自尾启动。

同伴启动（sibling launch）是即发即弃的一个变体：被启动的图作为父图的同伴而不是子图来执行。此外，同伴图运行在父图的流环境中。这意味着它立即且独立地运行，但不会延迟父图的任何尾启动，如图 12-8 所示。

对于这种模式，可以使用 `cudaGraphLaunch(graphExec, cudaStreamGraphFireAndForgetAsSibling)` 以“同伴模式”启动。这会把图作为当前图执行环境的同伴提交。

使用设备端发起的 CUDA 图时，需要仔细管理依赖关系。例如，如果被启动的 kernel 必须消费图的结果，那么尾启动是合适的选择，因为父 kernel 会暂停等待图的工作完成。相反，如果被启动的图更像一项附带任务，即发即弃模式则允许父 kernel 无需等待而继续执行。

在实践中，设备端发起的 CUDA 图开辟了新的模式。例如，设想一个 GPU 压缩流水线，其中的 kernel 必须根据数据内容在不同压缩算法之间做出选择。与其结束 kernel 并让 CPU 去启动选定的压缩 kernel，GPU kernel 可以直接启动与之对应的预录制图，比如“LZ 压缩”或“Huffman 压缩”。

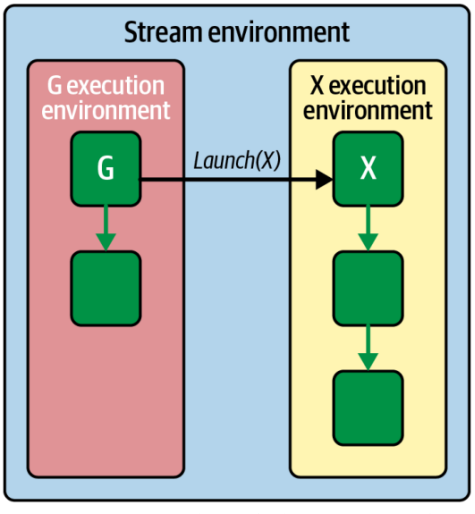

图 12-8　父图流环境中的同伴图启动

在这个压缩示例中，GPU 永远不会闲置等待 CPU 做出决定。下面再来看另一个实用模式：在一个持久 kernel 内部，把原子队列/计数器与设备端发起、尾启动的 CUDA 图结合起来，用于 LLM 推理调度。

12.2.6 用原子队列与设备端 CUDA 图实现 kernel 内持久调度

我们可以把前面的原子计数器工作队列与设备端发起的图尾启动结合起来。考虑一个 LLM 推理循环用例：它通过捕获一张包含 transformer 块前向传播（注意力 + 前馈网络）的图，用 CUDA 图来执行解码。

一个轻量的持久调度器 kernel 可以用 `atomicAdd(&queueHead, 1)` 认领下一个工作项，随后尾启动预先捕获的解码 CUDA 图来计算输出，并立即循环回去处理队列中的下一项。

每当一张 CUDA 图完成，kernel 内的调度器循环就用 `atomicAdd(&queueHead, 1)` 抓取下一个索引，并尾启动另一张解码图。这实际上构建了一个完全常驻 GPU 的调度器，它无需触碰 CPU 即可决策并执行任务。

通过把这些尾启动串联起来，每个 token 都在设备上从头到尾处理完毕，CPU 开销几乎为零。由于 CPU 从不重新进入关键路径，SM 始终保持充分占

用，每 token 延迟下降，而且可以即时适应不同的序列长度和批量大小——只需更新图参数或在预录制图之间切换即可。

12.2.7 条件图节点

在传统的 CUDA 图（CUDA Graph）中，每个节点及其依赖关系在图捕获（capture）时便已固定，任何决策逻辑都只能被推回主机端。条件图节点（conditional graph node）打破了这种僵硬性：它将分支决策延迟到 GPU 自身来完成——依据的是与该节点关联的一个小小的“条件句柄”（condition handle）。

随着图的执行，GPU 会评估该句柄，并有选择地执行其某个主体子图（或围绕它循环执行），全程无需将控制权交回 CPU。具体来说，条件图节点允许你把控制流（IF、IF/ELSE、WHILE、SWITCH）直接嵌入 CUDA 图并在 GPU 设备上运行。条件图节点消除了主机往返（host round trip），在现代 GPU 上可带来显著的性能提升。

从根本上说，条件图节点使你能够依据 device kernel 中计算出的值来控制图的执行——全程不涉及 CPU。这一能力让复杂的分支工作流可以通过一次、可重复的图启动来实现。CUDA 图支持多种类型的条件节点，如图 12-9 所示：

`IF`

当条件为非零时，其唯一主体图恰好执行一次。

`IF/ELSE`

通过指定两个主体图，条件为真时执行其中一个，为假时执行另一个。

`WHILE`

只要条件保持非零，就反复执行其主体图，并在每次迭代之后重新检查条件。

`SWITCH`

持有 N 个主体图，当条件等于 i 时执行第 i 个；若条件 $\geq$ N，则完全跳过执行。

接下来的示例演示了如何创建并填充一个 IF 条件节点。请注意其中使用 `cudaGraphSetConditional` 来写入控制 IF 节点的标志。本例中，条件检查的是

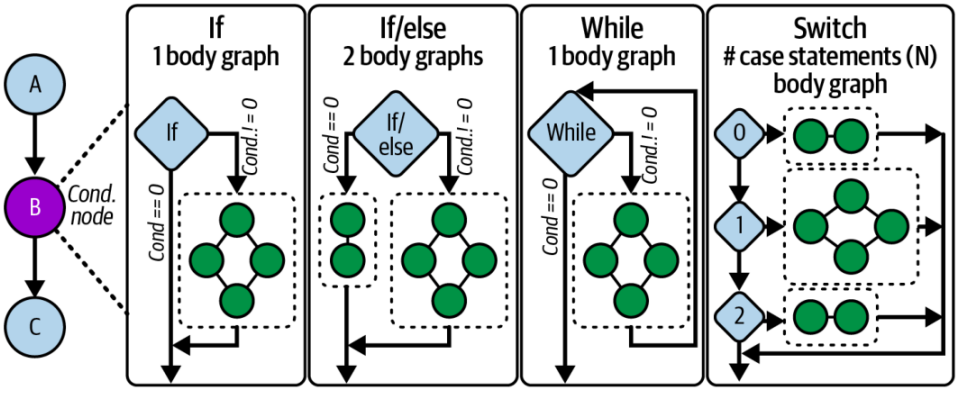

图 12-9　条件图节点的类型

sum 是否大于给定阈值。这样，当数据满足给定条件（flag = 1u）时，就会运行下一个子图；否则，如果条件不满足，条件节点便不会运行该子图：

```
#include <cuda_runtime.h>
#include <cstdio>
// Device kernel that computes and sets the condition handle
__global__ void setHandle(cudaGraphConditionalHandle handle,
                          int *data, size_t N) {


    // Example threshold
    constexpr int threshold   = 123456;
    // Test whether the sum of data exceeds a threshold
    //     using a custom reduce_sum() function
    //     (recommended to implement this with
    //     CUB's DeviceReduce::Sum routine.)
    unsigned int flag =
        (reduce_sum(data, N) > threshold) ? 1u : 0u;
    cudaGraphSetConditional(handle, flag);
}
// A simple body kernel that runs only if flag != 0
__global__ void bodyKernel() {
    printf("Conditional body executed on GPU!\n");
}
int main() {
```

```
cudaStream_t stream;
cudaStreamCreateWithFlags(&stream,
    cudaStreamNonBlocking);
// 1) Create the graph
cudaGraph_t graph;
cudaGraphCreate(&graph, 0);
// 2) Create a condition handle associated with graph
cudaGraphConditionalHandle condHandle;
cudaGraphConditionalHandleCreate(&condHandle, graph);
// 3) Add the upstream kernel node to set the handle
cudaGraphNode_t setNode;
cudaKernelNodeParams setParams = {};
setParams.func          = (void*)setHandle;
setParams.gridDim       = dim3(1);
setParams.blockDim      = dim3(32);
// 4) Allocate input data
constexpr size_t N              = 1 << 20;
int*             d_data         = nullptr;
cudaMalloc(&d_data, N * sizeof(int));

void* setArgs[] = { &condHandle, &d_data, &N };
setParams.kernelParams  = setArgs;
cudaGraphAddKernelNode(&setNode, graph, nullptr, 0,
    &setParams);
// 5) Add the IF conditional node
cudaGraphNode_t condNode;
cudaConditionalNodeParams ifParams = {};
ifParams.handle = condHandle;
```

```
    ifParams.type   = cudaGraphCondTypeIf;
    // One-node body graph, in this case
    ifParams.size   = 1;
    cudaGraphAddConditionalNode(&condNode,
                                graph,
                                &setNode,
                                1,
                                &ifParams);
    // 6) Populate the body graph: one kernel that prints a message
    cudaGraph_t bodyGraph = ifParams.phGraphOut[0];
    cudaGraphNode_t bodyNode;
    cudaKernelNodeParams bodyParams = {};
    bodyParams.func         = (void*)bodyKernel;
    bodyParams.gridDim      = dim3(1);
    bodyParams.blockDim     = dim3(32);
    cudaGraphAddKernelNode(&bodyNode, bodyGraph, nullptr,
        0, &bodyParams);
    // 7) Instantiate, upload, and launch the graph on the device
    cudaGraphExec_t graphExec;
    cudaGraphInstantiate(&graphExec, graph, nullptr, nullptr,
    cudaGraphInstantiateFlagDeviceLaunch);
    cudaGraphUpload(graphExec, stream);
    cudaGraphLaunch(graphExec, stream);
    // 8) Wait for completion and clean up
    cudaStreamSynchronize(stream);
    cudaGraphExecDestroy(graphExec);
    cudaGraphDestroy(graph);
    cudaStreamDestroy(stream);
    cudaFree(d_data);
    return 0;
}
```

这里，我们用 `cudaGraphCreate` 创建一个新的 CUDA Graph，用于容纳后续所有节点。然后用 `cudaGraphConditionalHandleCreate` 创建条件句柄（它为该图关联一个可在设备端设置的小整数值）。

接着添加上游 kernel `setHandle`。该 kernel 只在单个线程上运行以避免竞态条件，随后调用 `cudaGraphSetConditional` 写入控制 IF 节点的标志。

IF 条件节点通过 `cudaGraphAddConditionalNode` 添加，并指定 `cudaGraphCondTypeIf` 与 `size=1`。这里定义的是我们计划支持的条件分支或迭代数量。

这里，我们分配一个空子图（body），当条件标志返回非零值时执行。body 图从 `ifParams.phGraphOut[0]` 中取回，并通过添加 `bodyKernel` 来填充——该 kernel 执行时只是打印一条消息。

图构造完成后，调用 `cudaGraphInstantiate` 生成可执行的图对象。要从设备端代码启动图，必须在实例化时带上 `cudaGraphInstantiateFlagDeviceLaunch` 标志，并在任何设备端启动之前用 `cudaGraphUpload` 上传该图。

在 CUDA 流上调用 `cudaGraphLaunch` 启动图，会依次触发上游的 set-handle kernel、条件检查，以及（若标志已设置）body kernel。如图 12-10 所示，这一切都直接在 GPU 上完成。

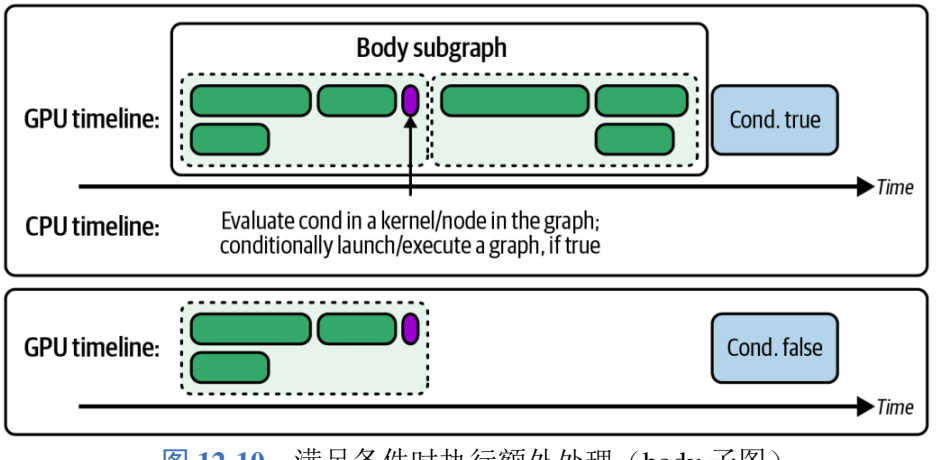

图 12-10　满足条件时执行额外处理（body 子图）

随后用 `cudaStreamSynchronize` 同步流以等待完成。最后清理资源：销毁实例化后的图、图本身以及流。

为尽量减少竞态条件，务必始终在单一线程中设置条件（例如 `if (threadIdx.x == 0)`），并确保前序 kernel 在条件节点执行之前就把内存刷出，使该值对设备端可见。

条件节点还可以嵌套。例如，WHILE 节点的 body 中可以包含一个 IF 节点，如图 12-11 所示。这样无需借助 CPU 中转即可表达多级决策逻辑。

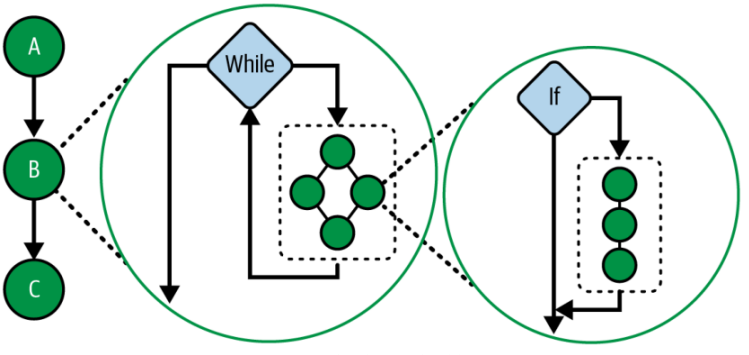

图 12-11　嵌套的条件图节点

简而言之，应当使用条件图节点把决策留在 GPU 上，降低 CPU 开销，并在 CUDA Graph 中直接表达复杂控制流。由于图创建的成本可以在多次迭代中摊销，将动态工作流完全放在设备端来表达，能够带来显著的性能提升。

> **注**　截至撰写本书时，PyTorch 的 CUDA Graphs API 尚无法通过 Python 直接创建条件 CUDA Graph 节点。框架和工具对条件图执行的支持仍在演进。就现阶段而言，若需要在 PyTorch 中实现 if/while/switch 节点，则要求自定义 C++ 集成。

12.3 动态并行

前面我们看到，由设备端发起的 CUDA Graph 启动如何以极少的 CPU 介入捕获并重放固定的操作序列。但这一模型假定整个执行流程事先已知，而这并非总能做到。许多工作负载会依据输入数据、中间结果或问题复杂度在运行时改变形态。这正是动态并行（Dynamic Parallelism，DP）的用武之地。

DP 赋予 GPU kernel 自行派生新工作的能力，而不必等待 CPU。CUDA Graph 要求你预先掌握整条流水线，而 DP 允许正在运行的“父”kernel 检查自身输出，并即时决定接下来要启动多少个“子”kernel。对于真正不规则的问题——分层归约、自适应网格细化、图遍历——后续任务的数量只有在处理数据的过程中才会明朗，这堪称一场变革。

设想一个推理流水线，偶尔需要针对某些输入执行特殊的“插件”模型评估。在 CPU 驱动的流程中，你要先运行 Kernel A，拷贝它的结果

回主机，再决定是否启动 Kernel B，然后发出该启动——往返期间 GPU 处于空闲。而使用 DP 时，Kernel A 检查自身输出，若条件成立便直接在设备端启动 Kernel B。整个“决策并分发”过程都在一个 GPU 常驻的工作负载内完成，消除了空闲间隙，让 SM 持续忙碌。

以 LLM 为例，大多数 token 走标准的 transformer 路径，但少数 token 需要辅助注意力块。启用了 DP 的 transformer kernel 可以在运行时检测这些特殊 token，并只为相应位置“尾随启动”额外的注意力 kernel——无需主机介入，也不浪费算力。NVIDIA 的库已经在自适应算法中利用 DP 实现了类似模式：新的子任务随着数据流经计算而动态涌现。

当性能分析器的时间线上出现 Kernel A → GPU 空闲间隙 → Kernel B 这类前后相继的模式时，你就知道该用 DP 了。这里的空闲时间对应 CPU 准备下一次启动的开销。用设备端启动填补这一间隙，可以让每个 SM 都保持占用，并大幅削减依赖阶段之间的延迟。

当然，DP 带来的性能收益并非免费。每次子启动都会占用 GPU 调度资源，并且需要额外的栈空间。为避免“栈溢出”错误，可能需要用 `cudaDeviceSetLimit(cudaLimitStackSize, newSize)` 增大运行时栈大小。一旦触及默认上限，CUDA 会发出警告。

与此相关，CUDA 对处于挂起状态的子 kernel 启动数量设有上限。默认情况下，CUDA 一次允许 2048 个未完成的设备端启动。不过，这一数值是可以配置的。

如果父 kernel 因为使用大循环启动成千上万个微小 kernel 而需要派生超过 2048 个子 kernel，可以用 API `cudaDeviceSetLimit(cudaLimitDevRuntimePendingLaunchCount, newLimit)` 提高该上限。否则，超出默认的 2048 上限会导致运行时错误。实践中，默认值通常足以满足大多数用途；但对于极端情况，这是一个重要的考量。

> **注** 在 GPU 上派生大量子 kernel 时，务必监控设备内存占用：每个挂起的子 kernel 启动都会预留资源，可能超出 GPU 硬件的硬性上限。

由于 DP 会引入指令级开销，最好把它留给这样的场景：否则使用常驻 kernel、流或 CUDA Graph 这类静态编排手段，GPU 会在 CPU 决定下一步时闲置。换句话说，当

你的工作负载确实是固定的操作序列时，通常更好的做法是把它捕获成 CUDA Graph 并在设备端重放。

当工作从数据本身动态涌现时，DP 让你把调度和执行完全留在 GPU 上，从而为嵌套的、数据依赖的或不可预测的并行带来更好的扩展性和更低的端到端延迟。

> **注** 由于 DP 带来每次启动的开销并消耗待启动槽位，只有当新的并行性在运行时从数据中涌现、无法表达为预先录制的 CUDA Graph 时，才使用 DP。反之，当控制流是表达式级且反复出现时，优先选用设备端启动的 CUDA Graph，因为 CUDA Graph 能摊销这些成本。

我们来比较一个简单父子工作流的两种实现。下面展示的主机驱动版本先启动父 kernel，等待它完成，然后从 CPU 发出两个子 kernel。这会让 GPU 在这些决策间隙中闲置：

```
// dp_host_launched.cu
#include <cuda_runtime.h>
__global__ void childKernel(float* data, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        data[idx] = data[idx] * data[idx];
    }
}
__global__ void parentKernel(float* data, int N) {
    // Parent does setup work. Here, CPU decides on child launches.
    if (threadIdx.x == 0 && blockIdx.x == 0) {
        // maybe mark regions or compute flags here
    }
}
int main() {
    const int N = 1 << 20;
```

```
    float* d_data;
    cudaMalloc(&d_data, N * sizeof(float));
    // ... initialize d_data ...
    // 1) Launch parent and wait
    cudaStream_t s; cudaStreamCreateWithFlags(&s,
     ↪  cudaStreamNonBlocking);
    parentKernel<<<1,1,0,s>>>(d_data, N);
    cudaStreamSynchronize(s);
    // 2) CPU splits work in half and launches children
    int half = N / 2;


    childKernel<<<(half+255)/256,256>>>(d_data,      half);
    childKernel<<<(half+255)/256,256>>>(d_data+half, half);
    cudaStreamSynchronize(s);
    cudaStreamDestroy(s);
    cudaFree(d_data);
    return 0;
}
```

在主机驱动版本中，GPU 运行 `parentKernel` 后便转入空闲，由 CPU 依次准备并启动每个 `childKernel`。注意父 kernel 之后以及各子 kernel 之间显式的 `cudaDeviceSynchronize()` 调用。这些调用造成了本应消除的空闲间隙，如图 12-12 所示。

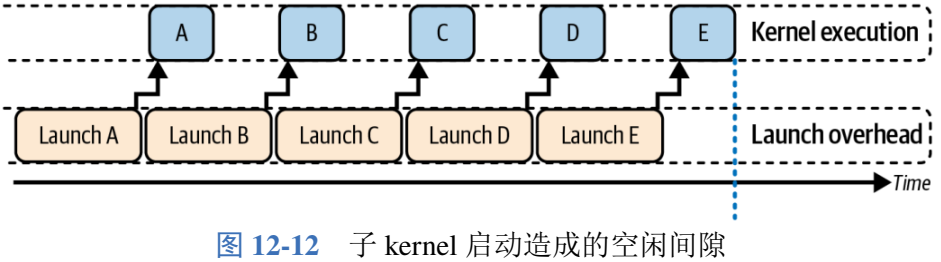

图 12-12　子 kernel 启动造成的空闲间隙

相比之下，设备端启动的 DP 版本允许父 kernel 在设备上派生自己的子 kernel。这种方式在父 kernel 与子 kernel 启动之间不需要任何主机端同步。这样，父 kernel 对子 kernel 的启动会将这些子 kernel 隐式入队，只在最后进行一次同步，如下面的代码所示：

```
// dp_device_launched.cu
// Dynamic parallelism requires relocatable device code enabled with
 ↪  -rdc=true.
#include <cuda_runtime.h>
__global__ void childKernel(float* data, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        data[idx] = data[idx] * data[idx];
    }
}
__global__ void parentKernel(float* data, int n) {
    // Launch children from a single thread to avoid duplicate launches.
    if (blockIdx.x == 0 && threadIdx.x == 0) {
        const int threadsPerBlock = 256;
        const int firstHalfCount  = n / 2;
        const int secondHalfCount = n - firstHalfCount;
        const int blocksFirst  = (firstHalfCount  + threadsPerBlock - 1)
                                 / threadsPerBlock;

        const int blocksSecond = (secondHalfCount + threadsPerBlock - 1)
                                 / threadsPerBlock;
        // Device launched child kernels.
        // No device side cudaDeviceSynchronize is needed.
        childKernel<<<blocksFirst,  threadsPerBlock>>>(data,
                                                       firstHalfCount);
        childKernel<<<blocksSecond, threadsPerBlock>>>(data +
         ↪  firstHalfCount,
                                                      secondHalfCount);
        // Parent kernel will not finish until both children finish.
    }
}
int main() {
    const int N = 1024 * 1024;    // 1M elements, avoids bit shifting
     ↪  for clarity
    float* d_data = nullptr;
    cudaMalloc(&d_data, N * sizeof(float));
```

```
    // Initialize to zero as a concrete, valid initialization.
    cudaMemset(d_data, 0, N * sizeof(float));
    // Launch parent on the default stream.
    parentKernel<<<1, 1>>>(d_data, N);
    // Wait for completion without cudaDeviceSynchronize. Sync stream
    ↪ instead.
    cudaStreamSynchronize(0);
    cudaFree(d_data);
    return 0;
}
```

在这里，parentKernel 直接在 GPU 上发出两次子 kernel 启动。主机只需提交一个 kernel，然后等待一次。当父 kernel 完成时，设备运行时会确保所有已启动的子 kernel 全部完成之后才继续推进。

> **注** 注意，这个动态并行版本完全没有使用 cudaDeviceSynchronize()。它依赖这样一条隐式规则：父 kernel 必须等到所有设备端启动的子 kernel 全部完成后才算完成，而主机只需在流上等待即可。

采用设备端 DP 方案后，不再存在供 CPU 做决策的空闲间隙。因此，它消除了相互依赖的各阶段之间的延迟，让 SM 自始至终保持忙碌。这提高了 GPU 利用率，代价只是设备上产生少量的每次启动开销，如图 12-13 的时间线所示。

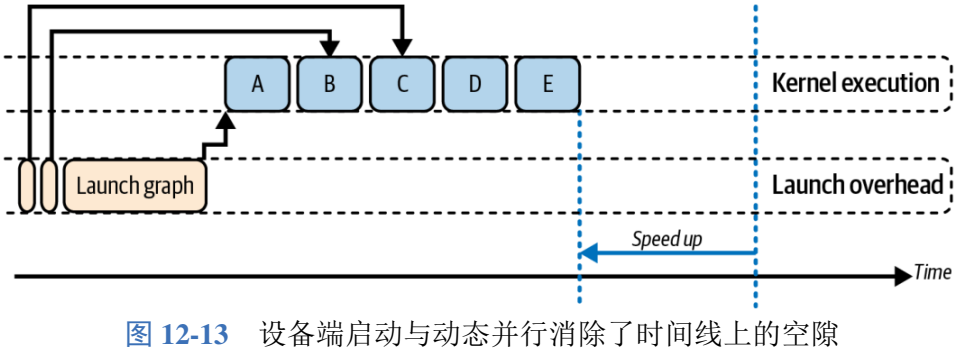

图 12-13 设备端启动与动态并行消除了时间线上的空隙

在这个简单的双子 kernel 示例中，主机驱动版本会发出三次独立的启动（1 个父 kernel 加 2 个子 kernel）。此时，GPU 会在 CPU 决定何时启动每个子 kernel 的期间处于空闲状态。

与之相反，设备驱动的 DP 版本只需在主机端执行一次父 kernel 启动，父 kernel 随后在 GPU 上派生两个子 kernel，无需主机进一步干预。表 12-2 对主机驱动与设备驱动 DP 子 kernel 启动的性能进行了比较。

表 12-2 主机驱动与 GPU 驱动嵌套子 kernel 启动的性能对比（2 个子 kernel）。

指标	之前（主机启动）	之后（设备启动）
主机端启动总次数	3	1
平均每次启动开销	~20 μs	~25 μs
GPU 空闲周期（整个序列期间）	~40%	~5%
整体执行时间	1.00 ms	0.75 ms

表 12-2 中的对比数据显示：主机端启动次数从 3 次降到 1 次，平均每次启动开销从约 20 μs 升至约 25 μs，GPU 空闲周期从约 40% 降至约 5%，整体执行时间从 1.00 ms 缩短到 0.75 ms。

从中可以看到，把子 kernel 的派发移入 GPU 之后，DP 消除了约 35% 的空闲时间，并将总运行时间缩短了约 25%。单次启动开销的轻微上升（20 μs $\rightarrow$ 25 μs）反映了 GPU 在设备端调度所产生的开销。然而，与省去多次 CPU-GPU 握手所节省的时间相比，这部分开销可以忽略不计。

GPU 驱动启动的另一个好处是改善了数据局部性，因为中间结果不再需要在阶段之间拷回 CPU。在本例中，父 kernel 计算出的数据无需离开 GPU 内存即可被子 kernel 直接使用。这避免了额外的内存传输，并保留了缓存中的数据。没有 CPU 介入，也意味着缓存中的数据被逐出或重新从 DRAM 取回的机会更少，而这些数据本可在 GPU 上被重复使用。

简而言之，DP 把时停时走的主机驱动工作流变成了一条无缝的 GPU 常驻流水线，持续保持较高的 SM 占用率，并将主机与 GPU 之间的协调开销降至最低。

同时请记住，动态并行与其他高级技术一样，也应当通过实测来评估其影响。

尽管 DP 消除了 CPU 交互，但由设备发起的 kernel 启动，其开销与主机启动仍大致处于同一量级。因此，并非所有算法都能从中获益。实际上，有些算法在 DP 下甚至会变慢——尤其是当被启动 kernel 的工作量太小、无法摊销启

动开销时。换言之，对于某些小型 kernel，设备端启动开销可能会抵消 DP 带来的收益。

> **注** 在做出改动前后都要进行性能分析。像 Nsight Compute 这样的工具可以对通过 DP 启动的子 kernel 进行性能分析，帮助量化其成本，确保 GPU 常驻流水线的收益确实大于额外开销，并切实提升吞吐量。

讲完单 GPU 的编排之后，接下来讨论多 GPU 与多节点场景：在这种场景下，互连带宽与集合通信操作会把 Roofline 模型的考量延伸到集群级别。

12.4 跨多 GPU 与集群节点进行编排（NVSHMEM）

从单 GPU 扩展到多 GPU 时，核心目标依然不变：把数据移动隐藏在有效工作之下，让每块设备都保持忙碌。一旦主机把任务分派给每块 GPU——无论是通过独立的 CPU 线程、异步启动还是多 GPU graph——后续执行便交给 GPU 完成。每块设备上一个流负责推进计算，另一个流则可以在 NVLink 或 PCIe 上以点对点方式搬运数据，全程不经过主机内存。

这意味着，在规模化场景下，必须让点对点传输与计算相互重叠。需要注意的是，即使采用 NVLink，其带宽和延迟也达不到设备内 HBM 的水平。因此，这类通信必须依靠重叠来隐藏。

实践中，随着集群规模增大，让计算与数据传输重叠是实现环境线性扩展的绝对必要条件。对于简单的数据交接，可以使用 GPUDirect Peer Access 在后台搬运大块内存，如下所示：

```
cudaMemcpyPeerAsync(dest_ptr, dest_gpu, src_ptr, src_gpu, size,
↪  comm_stream);
```

当需要集合通信时——例如 PyTorch 分布式数据并行（DDP）中的梯度 all-reduce——可以使用 NCCL 的异步集合通信调用，在单独的流上启动其非阻塞例程。随后，NCCL 会将你的

张量组织成环形或树形结构，使每一条 NVLink 与 NVSwitch 路径都趋于饱和——与此同时，你的计算 kernel 仍在各自的流上继续运行。

如果你的 MPI 库是 CUDA-aware 的，能够识别 GPU 设备指针，它就会自动利用 GPUDirect RDMA 通过 InfiniBand 发送数据，调用形式如下：

```
MPI_Send(device_buf, count, MPI_FLOAT, peer_rank, ...);
```

MPI（以及 NCCL）的这种 CUDA-aware 特性意味着，GPU 数据可以经 GPUDirect RDMA 直接通过 InfiniBand 跨网络传输，无需在主机内存中中转。（注意，在节点内部，点对点拷贝走 NVLink 或 PCIe，使用的是 GPUDirect Peer-to-Peer。）

使用这些原语并避开 CPU，有助于降低数据传输延迟，使 GPU 到 GPU 传输接近线速。这样一来，节点间的数据传输与通信就能与 GPU 计算妥善地重叠进行。

12.4.1 用 NVSHMEM 实现细粒度的 GPU 间内存共享

对于需要极致紧凑的事件驱动协同的工作负载——例如动态任务队列和细粒度事件通知——NVIDIA SHMEM（NVSHMEM）库是一个极佳选择。它把每块 GPU 视作分区全局地址空间（PGAS）中的一个处理单元（PE）。

有了 PGAS，GPU 就可以在设备代码中直接写入另一块 GPU 的内存，从而绕过 CPU。延迟取决于互连方式，NVLink 通常低于 PCIe 或网络传输。下面是使用 NVSHMEM 的经典” 发送—信号” 模式：

```
#include <cstdio>
#include <cuda_runtime.h>
#include <nvshmem.h>
#include <nvshmemx.h>
// Device symbols for the symmetric buffers
__device__ int   *remote_flag;
__device__ float *remote_data;
//------------------------------------------------------------------------
 ↪ --------
// GPU 0: send data then signal GPU 1
//------------------------------------------------------------------------
 ↪ --------
__global__ void sender_kernel(float *local_data, int dest_pe) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    float value = local_data[idx];
    // 1) Put the payload into remote_data[1] on dest_pe
    nvshmem_float_p(remote_data + 1, value, dest_pe);
    // 2) Wait for the RMA to complete before setting the flag
    nvshmem_quiet();


    // 3) Signal completion by setting remote_flag[0] = 1 on dest_pe
    nvshmem_int_p(remote_flag + 0, 1, dest_pe);
}
//------------------------------------------------------------------------
 ↪ --------
// GPU 1: wait for flag then consume payload
//------------------------------------------------------------------------
 ↪ --------
__global__ void receiver_kernel(float *recv_buffer) {
    // 1) Spin until remote_flag[0] == 1
    nvshmem_int_wait_until(remote_flag + 0,
        NVSHMEM_CMP_EQ, 1);
    // 2) Once flag is set, the payload at remote_data[1] is valid
    float val = remote_data[1];
    recv_buffer[0] = val * 2.0f;
}
```

```
//-----------------------------------------------------------------------
 ↪ --------
// Host-side setup and teardown
//-----------------------------------------------------------------------
 ↪ --------
int main(int argc, char **argv) {
    // 1) Initialize the NVSHMEM runtime
    nvshmem_init();
    // 2) Determine this PE' s rank and bind to the matching GPU
    int mype = nvshmem_my_pe();
    cudaSetDevice(mype);
    // 3) Allocate symmetric buffers on each PE
    //    - Two ints for the flag
    //    - Two floats for the data payload
    int   *flag_buf = (int*)   nvshmem_malloc(2 * sizeof(int));
    float *data_buf = (float*) nvshmem_malloc(2 * sizeof(float));
    // 4) Zero out flags on PE 0 and synchronize
    nvshmem_barrier_all();
    if (mype == 0) {
        int zeros[2] = {0, 0};
        cudaMemcpy(flag_buf, zeros, 2 * sizeof(int),
                   cudaMemcpyHostToDevice);
    }
    nvshmem_barrier_all();
    // 5) Register the device pointers for use in kernels
    cudaMemcpyToSymbol(remote_flag, &flag_buf, sizeof(int*));
    cudaMemcpyToSymbol(remote_data, &data_buf, sizeof(float*));
    // 6) Launch either the sender or receiver kernel
    dim3 grid(1), block(128);
    if (mype == 0) {
        // Example input buffer for the sender
        float *local_data;
        cudaMalloc(&local_data, 128 * sizeof(float));
```

```
        // ... initialize local_data as needed ...
        sender_kernel<<<grid, block>>>(local_data, 1);
        cudaFree(local_data);
    } else {
        float *recv_buffer;
        cudaMalloc(&recv_buffer, sizeof(float));
        receiver_kernel<<<grid, block>>>(recv_buffer);
        cudaFree(recv_buffer);
    }
    // 7) Wait for all GPU work to finish
    cudaDeviceSynchronize();
    // 8) Clean up NVSHMEM resources
    nvshmem_free(flag_buf);
    nvshmem_free(data_buf);
    nvshmem_finalize();
    return 0;
}
```

这里，单边远程内存操作完全在设备端进行。GPU/PE 0 把自己的结果直接写入 GPU/PE 1 的内存，并在那里翻转一个标志位。具体而言，GPU/PE 0 发起 `nvshmem_float_p` 将负载数据直接写入 GPU/PE 1 的内存，调用 `nvshmem_quiet()` 确保完成，然后再用 `nvshmem_int_p` 翻转标志位。

与此同时，GPU/PE 1 的 kernel 通过 `nvshmem_int_wait_until()` 自旋等待，一旦标志位被置位就立即读取负载。这一切既不需要 CPU 干预，也不需要额外拷贝——只需经 NVLink 完成硬件加速的 GPU 到 GPU 传输即可。

由于 NVSHMEM 通信采用的是 GPU 发起、经由 NVLink 或 PCIe 的单边操作，它去掉了主机中转环节。因此，NVSHMEM 通信可以达到接近线路峰值的速度。原因在于，NVSHMEM 的单边操作既绕过了 CPU，也绕过了 kernel 启动的软件开销。从根本上说，NVSHMEM 把原本多步骤的通信变成了一次硬件事务。

当然，能力越大，责任越大。NVSHMEM 本质上是 GPU 级别的共享内存编程，你必须小心管理同步并避免竞态。此外，过度使用全局屏障还会让所有 GPU 卡在最慢的对等设备上。

实践中应避免过度同步。尽可能使用 NVSHMEM 的细粒度信号或点对点同步，而不是动辄调用 `nvshmem_barrier_all()`。

现代的 NVSHMEM 实现对这些同步例程进行了效率改进。然而，它们仍然是同步点，使用不当就可能

成为瓶颈。NVSHMEM 提供了诸如 `nvshmem_wait_until` 之类的细粒度原语用于等待设备端变量，还提供了 `nvshmem_signal_fetch`、`nvshmem_signal_wait_until` 或 `nvshmemx_signal_op` 变体等信号操作，用于在只有部分设备需要协调时进行点对点同步。发送 GPU 与接收 GPU 之间通过 NVSHMEM 共享数据并以信号进行同步的底层细节如图 12-14 所示。

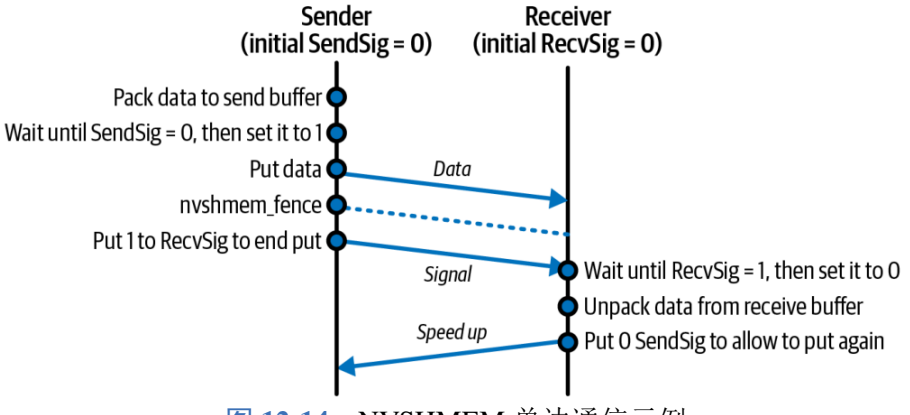

图 12-14　NVSHMEM 单边通信示例

当负载不规则或依赖数据时，NVSHMEM 尤其大放异彩，例如图算法、动态负载均衡和离散事件模拟。在这些场景下，静态的通信图与集合操作难以满足需求。

使用 NVSHMEM 的 kernel 可以像其他任何 kernel 一样被捕获进 CUDA Graph。NVSHMEM 的设备端操作发生在这些 kernel 内部，而不是成为独立的图节点。不过，NVSHMEM 真正的强项在于让 kernel 能够即时自适应并相互协调，而不必遵循图所规定的固定通信脚本。

简而言之，NVSHMEM 把一个 GPU 集群变成一个共享内存域，使设备端 kernel 启动、数据传输和同步的延迟与吞吐量远超任何经由 CPU 中介的方案。

设想一个两阶段的 transformer 推理流水线（attention 加多层感知机），它的工作方式如下：GPU 0 计算 attention，然后通过 NVSHMEM 把激活值 put 到 GPU 1 并发送信号。GPU 1 运行一个常驻 kernel，一看到标志位就开始 MLP 阶段。

由于 GPU 0 会立即转向 batch 2，而 GPU 1 此刻还在处理 batch 1，几次迭代之后两个设备便进入完美协同的工作节奏。每次交接都隐藏在活跃的计算 warp 背后。这让利用率接近 100%，而且不会引起主机停顿。

需要灵活的负载均衡时，NVSHMEM 的原子操作可以让每个 PE 动态地领取工作。PE 是组成并行 NVSHMEM 应用的一个 OS 进程。

下面的代码展示了每块 GPU 从全局计数器中取出下一个索引、处理该任务块，然后再循环领取。这样，真正的工作窃取完全在设备端进行——无需主机协调：

```
__global__ void work_steal_kernel(/*...*/, int *queue_head, Task
↪  *tasks) {
    while (true) {
        // Atomically claim the next task index
        int idx = nvshmem_int_atomic_inc(queue_head);
        if (idx >= N_tasks) break;
        // Process tasks[idx]...
    }
}
```

对于需要尽可能低抖动的场景，例如紧凑的多 GPU 集合通信或同步的模型并行步骤，你可以用 `nvshmemx_collective_launch()` 启动一个跨所有 GPU 的 cooperative kernel，让发送 kernel 和接收 kernel 在 GPU 0 和 GPU 1 上同时启动。这使它们能够借助 NVSHMEM 相互协调，而无需任何主机干预。接着，你可以使用 NVSHMEM 的设备端屏障，如下所示：

```
__global__ void synchronized_step_kernel(/*...*/) {
    nvshmem_barrier_all();
    // All GPUs proceed in lockstep here
    // ...
}
```

这里，每个 PE 一起进入 `nvshmem_barrier_all()`，然后同时继续执行。这保证了集群内各设备的执行完全对齐。

> **注** 所有使用 NVSHMEM 设备级同步或集合操作的 kernel，都必须用 `nvshmemx_collective_launch()` 启动。这确保 kernel 在作业的全部 PE（GPU）上并发运行。

12.4.2 用 NCCL 和 CUDA Graph 捕获多 GPU 集合通信

当你需要广播、归约和 all-to-all 传输这类大批量集合操作时，NVIDIA 的 NCCL 库是多 GPU 系统上的首选。传统做法是每块 GPU 都从主机发起 `ncclAllReduce` 或类似的集合操作，然后等待（同步）完成，再进入下一个计算阶段。这种顺序的主机编排会带来额外开销，并在前向与反向传播之间制造空闲时间。

不过，NCCL 调用同样可以像 kernel 一样被记录进 CUDA Graph。这样，你就能把前向 kernel、all-reduce 和反向 kernel 一起“烘焙”进一张图，然后在每次迭代中重放它：

```
cudaStreamBeginCapture(captureStream,
    cudaStreamCaptureModeGlobal);
forwardKernel<<<...>>>(...);
ncclAllReduce(sendBuf, recvBuf, count, ncclFloat,
    ncclSum, comm, captureStream);
backwardKernel<<<...>>>(...);
cudaStreamEndCapture(captureStream, &graph);
// Instantiate and upload before launching
cudaGraphExec_t graphExec;
cudaGraphInstantiate(&graphExec, graph, ...);
cudaGraphUpload(graphExec, captureStream);
// Each training step:
cudaGraphLaunch(graphExec, captureStream);
```

注意这里所有操作——包括 NCCL——都使用同一条捕获 stream。采用图之后，NCCL 调用就变成与 kernel 一样的图节点。由于每个进程重放的是完全相同的图，NCCL 的内部逻辑会自行找到对等设备并执行 all-reduce，无需额外的主机协调。这种被图捕获的 all-reduce 在大型集群上尤为强大：它消除了每次迭代中的启动抖动，让所有 GPU 保持忙碌，使计算与网络操作相互重叠。

由于每块 GPU 启动的是同一张图（包括其中的集合通信节点），NCCL 会在内部于各 rank 之间会合，不需要额外的主机干预。主机每次迭代的工作缩减为一次 `cudaGraphLaunch`，从而降低 CPU 开销和启动抖动。

除了降低 CPU 负载之外，把 all-reduce 捕获进图还能在多块 GPU 和多个计算节点之间实现通信与计算的真正重叠。假设你把梯度计算拆成两趟——第 1 至 $L/2$ 层与第 $L/2+1$ 至 L 层——并把它们映射到不同的 stream，如下所示：

```
// Pseudocode in capture:
computeGradientsLayer1<<<...>>>(..., streamA);
ncclAllReduce(..., comm, streamB);       // in streamB, overlaps with
↪ streamA
computeGradientsLayer2<<<...>>>(..., streamA);
ncclAllReduce(..., comm, streamB);
```

由于这些节点在捕获时就带有各自唯一的 stream 分配和依赖关系，CUDA 驱动可以让 streamB 上的 NCCL 网络传输与

streamA 上的独立工作重叠执行。这种分桶 all-reduce 模式能把通信延迟持续隐藏在计算背后，从而改善多 GPU 的扩展性。

> **注** 当所有 rank 用同一个 communicator 捕获并重放相同序列时，NCCL 集合操作与 CUDA Graph 捕获兼容。但所有 rank 必须用同一个 communicator 捕获并重放相同的 NCCL 序列。如果各 rank 在图重放时使用的 communicator 不一致，最好的情况是死锁（相对容易调试），最坏的情况是结果错误（静默失败，难以调试）。此外，建议先把预热集合操作捕获下来并提前运行以初始化 communicator，然后复用实例化后的图，以获得最小的稳态延迟。

实践中，这种分桶 all-reduce 方法已成为大模型训练的标准做法。把各段梯度归约与后续层的计算重叠起来，你几乎可以把全部网络时间都隐藏掉。图 12-15 展示了运行在独立进程（进程 1 和进程 2）中的 DDP 分桶 all-reduce 示例。

像 PyTorch DDP 这样的现代程序库会自动实现该方法的多种变体，但捕获 CUDA Graph 可以进一步降低 CPU 开销，并提供更具确定性的性能。

在多 GPU 环境中使用 CUDA Graph 时需要注意几点。首先，所有参与的 GPU 必须以完全相同的顺序记录并重放集合通信，以避免死锁——这就像 MPI 的集合通信规则：所有 rank 必须以相同的顺序进入集合操作。

其次，虽然 CUDA Graph 会固定并复用 GPU 缓冲区，但一定要确保梯度与通信缓冲区的分配在捕获之前完成。另外，如前文所述，如果需要修改图中的参数（例如批大小），可以使用 `cudaGraphExecUpdate` 来修补这些参数，而无需重新完整捕获。

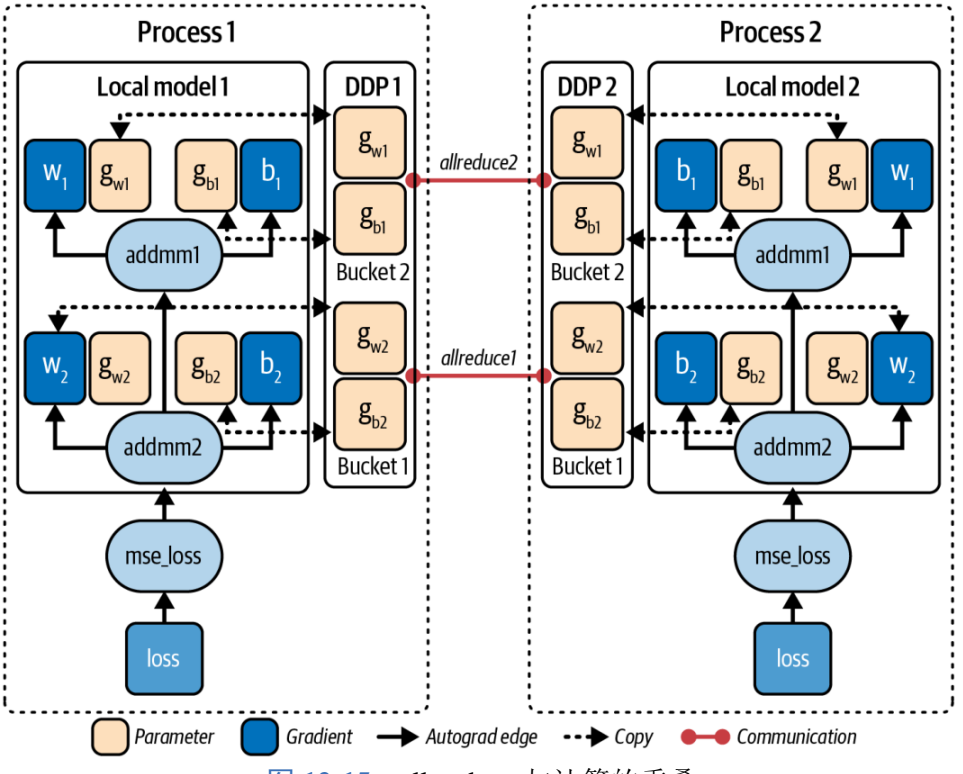

图 12-15　all-reduce 与计算的重叠

在实践中，将 NCCL 通信与计算一起捕获进图，可以缩短每步的 CPU 时间，并加快跨多块 GPU 的大模型训练。在拥有数十万块 GPU 的超大规模下，这些节省会不断累积——使整个集群的同步更紧凑、利用率更高。

> 注　NCCL 与 CUDA Graph 为我们提供了一种高效的方式，让集合通信与计算并行排布。然而，并非所有多 GPU 通信都是集合通信——有时我们需要在 GPU 之间进行更细粒度或异步的数据共享。这正是前文介绍的 NVSHMEM 能发挥作用的地方。

12.4.3 N-GPU 扩展模式

无论你使用的是简单的对等拷贝、NCCL 环，还是 NVSHMEM 的单边原子操作，扩展到多块 GPU 的模式始终相同：系统应只分发一次 kernel，将数据传

输流水线化并与计算重叠，从而实现线性扩展——尤其是在让主机 CPU 脱离关键路径的情况下。

举例来说，理想情况下 4 块 GPU 应表现得像一块快 4 倍的单一 GPU——前提是你能让所有 GPU 并行地获得数据供给（见图 12-16），并让主机置身事外。如果能做到这一点，通过恰当地让通信与计算重叠，你就能获得接近线性的加速。若没有重叠，一旦通信时间与计算时间相当，扩展性就会趋于停滞。

随着 GPU 数量的增加，你需要对数据传输与计算采用更激进的流水线化，并减少 CPU 侧的编排与同步。因此，应当借助异步拷贝、图中的 NCCL 集合操作以及 NVSHMEM 的 PGAS 原语，把更多的编排工作下放给设备。这将更多的责任转移给了软件。

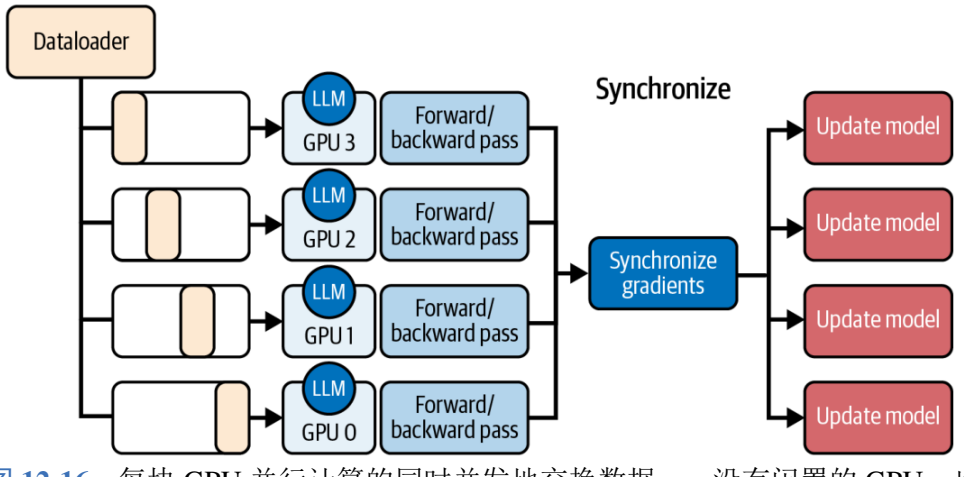

图 12-16　每块 GPU 并行计算的同时并发地交换数据——没有闲置的 GPU，也没有停滞的数据传输

应用这些技术之后，你就能消除 CPU 瓶颈，饱和高速互连，榨尽计算 FLOPS，构建真正低延迟的多 GPU 流水线。接下来，让我们在动态调度与设备侧编排的背景下重新审视 Roofline 模型。

12.5 由 Roofline 模型指导的调度与编排决策

在前面的几章中，我们已经积累了一套扎实的编排技术，包括 CUDA 流、kernel 融合、常驻 kernel、CUDA Graph、动态并行等。Roofline 模型有助于判断哪种工具最有可能为你的场景带来最大收益。

Roofline 的核心归结为运算算术强度，即执行的 FLOPS 数与移动的字节数之比。它由两条硬件“天花板”构成：内存屋顶（斜线）表示受带宽限制时的峰值吞吐率，计算屋顶（水平线）标记 ALU 受限时的峰值算术速率，如图 12-17 所示。

如果你的 kernel 靠近内存屋顶（例如 FLOPS/字节较低），因而属于内存受限，那么最佳的优化就是那些能将内存传输隐藏或与计算重叠的手段。这意味着应使用配合 CUDA 流的异步拷贝——甚至可以并发运行多个内存受限的 kernel。这样一来，你就能更好地饱和内存系统的不同部分。

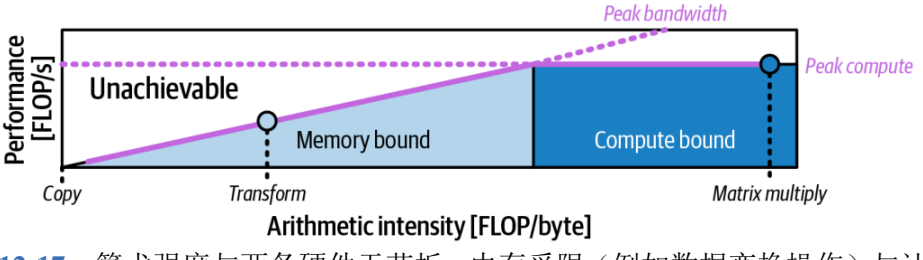

图 12-17　算术强度与两条硬件天花板：内存受限（例如数据变换操作）与计算受限（例如矩阵乘法）

> **注**　对于内存受限的工作负载，kernel 融合的帮助有限。它虽能省去若干次中间全局内存往返，但真正的收益来自掩盖延迟，以及让更多的加载/存储操作同时在途。

相反，位于计算屋顶之下的高强度 kernel 需要让 ALU 保持忙碌。此时 kernel 融合大放异彩：它把各自独立的加法与缩放操作（即前文的那个融合示例）合并为一次遍历，提高了每字节的 FLOPS，使工作点在 Roofline 图上向右移动，并把 kernel 推向更高的峰值 FLOP/s 占比。

同样，常驻 kernel、线程块簇以及由设备发起的 CUDA Graph 并不会改变你的强度数值，但它们能减少由反复启动造成的空闲间隔，使 kernel 的性能更接近那条平坦的计算天花板。

许多真实的工作负载介于两者之间：既非强内存受限，也非计算受限。在这种情况下，并发是你的朋友。通过并行启动多个中等强度的 kernel——无论是使用流、并发图，还是多个常驻 kernel——你实际上是在将它们叠加，使聚合吞吐量的工作点在两个坐标轴上都处于更高的位置，从而更充分地利用设备的资源。

彻底的 Roofline 分析需要严谨的测量。使用 Nsight Compute 统计 FLOPS 与传输的字节数，标出你的 kernel 在图上的点，看看它距离每条屋顶线有多远。

如果工作负载是内存受限的，就用流和重叠，或许还可以降低精度（FP16、FP8、FP4），以减小算术强度公式中的分母（即传输的字节数）。

而如果你的 kernel 计算受限，却因启动开销或空闲期而未达到峰值 FLOPS，那么应专注于降低启动开销。如前文所学，可以通过把操作融合进单个 kernel、使用常驻 kernel、捕获 CUDA Graph，或者在设备侧发起启动来实现。这能让 ALU 持续保持满载。

如果你的 kernel 远低于两条屋顶线，既非完全内存受限，也非计算受限，那就尝试提高并发度：并行运行多个 kernel 与流，从而更充分地利用全部系统资源。

有了这些量化指导，你就能为 kernel 挑选合适的编排策略，而不必把所有技巧一股脑儿地全试一遍。只需记住，要验证每次优化都让你更接近硬件的真实潜能——每次应用优化之后都要进行测量。

Roofline 模型指导的是预期，而真实的性能测量——包括计算吞吐量、实际占用率、内存吞吐量等——才能讲出完整的故事。将 Roofline 分析与迭代式的持续性能剖析相结合，可以验证你所选择的优化策略确实有效。

12.6 本章要点

要达到 GPU 的峰值性能，关键在于以最小的开销将计算与数据搬运编织在一起。高效的编排能够精简跨越 CPU 与 GPU 的复杂工作负载，确保任何一方都不会拖住另一方。以下是本章的一些要点：

基于 L2 缓存原子队列的动态调度

现代 GPU 上的 L2 缓存原子操作速度极快。利用快速的 L2 缓存原子操作配合批量递增，可以在 GPU 上平衡不规则的工作负载。这种批量工作分配方式减少了争用，并通过消除 warp 的空闲间隔让 warp 保持忙碌。在极端不均衡的情况下，它能把吞吐量显著提升约 2 倍，通常则为 10%～30%。由于 L2 带宽极高，即便是 8 或 16 这样适中的批大小，也能消除大部分争用。

面向固定流水线的 CUDA Graph

将一串 GPU 操作记录一次，然后在每次迭代中用一次主机调用加以重放。这能降低每次迭代的 CPU 调度开销，通常可带来 20%～30% 的延迟降低（规模越大，降幅越多）。要确保 GPU 上相互依赖的操作实现了最大程度的重叠。

借助 CUDA Graph 实现低开销启动

将一串异步拷贝、kernel 启动、事件记录与内存分配捕获进一个 CUDA Graph（`cudaStreamBeginCapture/cudaStreamEndCapture`）。用 `cudaGraphLaunch` 重放该图，可消除每次调用的 CPU 入队开销，同时保留所有流间依赖，进一步缓解运行时瓶颈。

设备侧编排

让 GPU 自己发起后续工作：既可以用延迟启动（tail-launch）的方式重放预录制的 CUDA Graph，也可以用动态并行（dynamic parallelism）派生子 kernel。这能彻底消除 CPU 调度间隙，使 GPU 无须主机干预即可从头到尾保持忙碌。

多 GPU 重叠

务必让通信与计算重叠。使用不同的流对 GPU 间点对点传输（`cudaMemcpyPeerAsync`）、NCCL 集合通信、CUDA-aware MPI（RDMA）或 NVSHMEM 单边操作进行流水线化。这样可以把通信延迟隐藏在有效工作之后；在计算与

通信比合适、重叠充分的条件下，即便跨越集群节点，也可能在多 GPU 上逼近线性扩展。

以 Roofline 图引导决策

让 Roofline 图驱动你的优化策略。如果 kernel 受内存带宽限制（memory-bound），应聚焦于重叠与减少数据搬运，例如异步内存拷贝以及 FP8/FP4 等混合精度。如果 kernel 受计算限制（compute-bound）但因开销影响而未达应有水平，则应采用降低启动开销的手段——kernel 融合、常驻 kernel（persistent kernel）和 CUDA Graph——去逼近计算天花板。介于两者之间的 kernel，则应通过并行执行多个操作来提高并发度，让所有硬件单元都得到利用。无论选择哪种优化，始终要用性能分析（profiling）验证它确实带来了提升。

将这些技术编织在一起——动态分发、协作 kernel、图捕获/重放以及 GPU 原生的内存共享——你就能构建出让 GPU 集群的每一部分都饱和工作、足以支撑超大规模 AI 工作负载的流水线。

12.7 本章小结

本章超越了单 kernel 优化，探讨了端到端编排技术。我们介绍了如何完全在设备上通过动态并行发起工作，如何用 CUDA Graph 捕获复杂的工作流，以及如何用 NCCL 和 NVSHMEM 协调众多 GPU。这些技术有一个共同目标：让每个引擎始终有活可干、把延迟隐藏起来、消除主机与设备之间的空隙，使硬件全速运转。

NVIDIA 的现代 GPU 平台比以往任何时候都更模糊了 CPU 与 GPU 的界限。例如，Grace Blackwell 和 Vera Rubin Superchip 通过具备极高带宽的一致性 NVLink 将 CPU 与多个 GPU 连接在一起。

然而，即便硬件不断降低 CPU 与 GPU 之间的壁垒，要充分发掘这种高性能硬件的潜力，责任仍然落在软件肩上。本章介绍的各种方法——无论是用 C++ 编写 CUDA 代码，还是调用更高层的库 API——正是我们利用这些硬件进步的方式。

下一章将介绍 PyTorch 如何整合其中的许多思想——包括流、图、异步操作和优化过的 kernel——让你只用几行 Python 代码就能获得这些性能。让我们进入 PyTorch 生态系统，弄清它为何如此受高性能 AI 工作负载实现者的青睐。

第 13 章 PyTorch 的性能分析、调优与扩展

AI 训练与推理流水线的每一层都可能遭遇性能瓶颈，包括 Python 解释器开销、CPU 主机端数据加载停顿、CUDA kernel 利用不足以及 GPU 显存争用。要做到有效优化，就需要在技术栈的多个层级进行性能分析，并使用多种覆盖整个系统的工具。

本章聚焦于在现代 NVIDIA GPU 上运行的 PyTorch 工作负载的性能分析、调试与系统级调优。我们将探讨如何借助 PyTorch 内置 profiler、NVIDIA 的 Nsight 工具以及基于 Linux perf 的 CPU 性能分析来发现并修复瓶颈，还会介绍 PyTorch 的内存分析与内存分配器调优。此外，我们还将讨论 PyTorch 如何利用 CUDA 流实现并发，以及如何借助 CUDA Graph 降低 kernel 启动开销。

接下来，我们将展示如何用 PyTorch 的分布式数据并行（Distributed Data Parallel，DDP）、全分片数据并行（Fully Sharded Data Parallel，FSDP）以及其他模型并行策略来优化数据流水线并扩展到多 GPU。随后，我们将演示如何对多 GPU 与多节点环境进行性能分析，包括使用 Holistic Trace Analysis（HTA）与 Perfetto。

本章通篇强调性能权衡，并给出聚焦 kernel 执行时间、硬件利用率指标、内存占用、数据加载效率以及扩展的整体成本效率的定量示例。读完本章，你应当理解如何为整个技术栈上的 PyTorch 工作负载实施一套有效且全面的分析与调优方法。

13.1 NVTX 标记与性能分析工具

要从整体上把握性能，就必须在多个层级进行性能分析，并使用覆盖整个系统的工具。从业者和性能工程师在实践中沉淀了一套通用工具与最佳实践，可对系统栈中的所有层进行全局性能分析。

在介绍这些工具之前，需要先强调 NVIDIA Tools Extension（NVTX）和 NVTX 标记。这类标记在性能分析器的时间线视图中标示时间段，使不同的性能分析器能够围绕相同阶段关联事件。

例如，`forward` 的 NVTX 区间会同时出现在 PyTorch profiler 的追踪记录和 Nsight Systems 的时间线中，这使得技术栈不同层级上的跨工具分析容易得多。大多数现代 AI 框架与库都支持 NVTX 标记，包括 PyTorch 及 CUDA 生态的相关组件。

NVTX 标记可通过 CUDA C++、PyTorch，或任何支持 NVIDIA GPU 的 C++ 或 Python 库（如 OpenAI Triton、PyCUDA、CuPy、cuTile、cuTe、CUTLASS 等）注入代码。大多数库已经替你在关键代码区域注入了 NVTX 标记，例如 `train_step`、`forward`、`backward`、`optimizer_step` 等。你也可以自行注入，例如在 PyTorch 中使用 `torch.profiler.record_function()` 和 `torch.cuda.nvtx.range_push()`。

在了解了如何使用 NVTX 标记标注代码中值得关注的片段之后，接下来介绍能够读取、对齐并可视化这些标记的工具。表 13-1 总结了常用的性能分析工具及其作用范围、关键特性与典型用例，可帮助你在优化之旅的每个阶段选择合适的工具。

工具　作用范围　特性　典型用例　PyTorch profiler（Kineto）

对模型代码做细粒度分解；找出慢算子、GPU kernel 启动开销或前向/反向时间失衡。Nsight Systems（nsys）

PyTorch 内的算子级性能分析（CPU/GPU）

支持 NVTX 标记、形状记录、内存统计、追踪导出，可识别编译图断裂（graph break）

训练/推理流水线的端到端视图；检测数据加载器停顿、CPU 与 GPU 重叠问题或 GPU 间同步延迟。Nsight Compute（ncu）

系统级时间线（CPU、GPU、OS、I/O）

CPU 线程与 GPU 流的统一时间线、NVTX 集成、多进程支持

GPU kernel 分析（逐 kernel）

逐 kernel 硬件指标、源码关联、Roofline 分析、占用率与吞吐量报告

在找到热点 kernel 后深入分析其效率；判断 kernel 是内存受限还是计算受限，以及原因。

表 13-1　性能分析与可视化工具概要。

工具	作用范围	特性	典型用例
PyTorch profiler（Kineto）	PyTorch 内的算子级性能分析（CPU/GPU）	支持 NVTX 标记、形状记录、内存统计、追踪导出，可识别编译图断裂（graph break）	对模型代码做细粒度分解；找出慢算子、GPU kernel 启动开销或前向/反向时间失衡
Nsight Systems（nsys）	系统级时间线（CPU、GPU、OS、I/O）	CPU 线程与 GPU 流的统一时间线、NVTX 集成、多进程支持	训练/推理流水线的端到端视图；检测数据加载器停顿、CPU 与 GPU 重叠问题或 GPU 间同步延迟
Nsight Compute（ncu）	GPU kernel 分析（逐 kernel）	逐 kernel 硬件指标、源码关联、Roofline 分析、占用率与吞吐量报告	在找到热点 kernel 后深入分析其效率；判断 kernel 是内存受限还是计算受限，以及原因
PyTorch memory profiler	按算子统计的 GPU 内存使用	内存快照时间线、逐算子峰值内存，集成 `torch.cuda.memory_stats()` 与 `torch.cuda.mem_get_info()`	诊断内存碎片或异常偏高的内存使用；查看哪些算子在何时分配内存最多，以优化内存占用
Linux perf	CPU 性能分析与系统事件	CPU 周期/指令/缓存采样、火焰图、off-CPU（睡眠）分析	定位 Python 开销（解释器时间、GIL 争用）、CPU 侧数据加载瓶颈或主机 OS 调度问题
Holistic Trace Analysis	分布式训练追踪可视化	基于浏览器的 Kineto 追踪浏览器、多 worker 追踪聚合；内置 Perfetto 后端	对多 GPU/多节点执行做全局分析；发现失衡或空闲时段，验证通信与计算的重叠
Chrome trace/Perfetto	离线追踪查看（基于 Web）	面向标准追踪格式的 Web UI、高级过滤（Perfetto SQL 引擎）、便于追踪共享	无需专业软件即可查看追踪数据；适合性能仪表板或基于性能分析数据的远程协作
TorchEval（metrics）	模型指标与性能记录	标准化的指标 API（吞吐量、准确率等），易于集成	在训练与评测中记录并监控模型吞吐量/延迟与准确率，从而将性能与模型质量关联
ExecuTorch	面向移动、嵌入式与边缘设备的部署运行时，带轻量性能分析钩子与导出工具	模型导出、性能分析、调试、内存分析、可视化	运行 PyTorch 模型并在受限平台（如移动、嵌入式与边缘设备，包括 Meta 眼镜等）上收集运行时指标

工具　作用范围　特性　典型用例　PyTorch memory profiler

诊断内存碎片或异常偏高的内存使用；查看哪些算子在何时分配内存最多，以优化内存占用。Linux perf　CPU 性能分析与系统事件

按算子统计的 GPU 内存使用

内存快照时间线、逐算子峰值内存，集成 `torch.cuda.memory_stats()` 与 `torch.cuda.mem_get_info()`

定位 Python 开销（解释器时间、GIL 争用）、CPU 侧数据加载瓶颈或主机 OS 调度问题。Holistic Trace Analysis

CPU 周期/指令/缓存采样、火焰图、off-CPU（睡眠）分析

对多 GPU/多节点执行做全局分析；发现失衡或空闲时段，验证通信与计算的重叠。Chrome trace/Perfetto

分布式训练追踪可视化

基于浏览器的 Kineto 追踪浏览器、多 worker 追踪聚合；内置 Perfetto 后端

无需专业软件即可查看追踪数据；适合性能仪表板或基于性能分析数据的远程协作。TorchEval（metrics）

离线追踪查看（基于 Web）

面向标准追踪格式的 Web UI、高级过滤（Perfetto SQL 引擎）、便于追踪共享

在训练和评估中将模型的吞吐量/延迟与准确率一并记录和监控，以便将性能与模型质量关联起来。移动、嵌入式和边缘设备的部署运行时 ExecuTorch，配备轻量级性能分析钩子与导出工具

模型指标与性能日志记录

标准化的指标 API（吞吐量、准确率等），易于集成

模型导出、性能分析、调试、内存分析与可视化

用于在受限平台（如移动、嵌入式和边缘设备，包括 Meta 眼镜等）上运行 PyTorch 模型并收集运行时指标。

下面逐一详细介绍表 13-1 中的每种性能分析工具：

PyTorch 性能分析器（Kineto）

在 PyTorch 内部，基于开源项目 Kineto 的 `torch.profiler` 提供 CPU 与 CUDA/GPU 运行时的算子级细分。此外，它还能记录输入形状，并通过简单的 Python 上下文管理器对内存进行快照。PyTorch 性能分析器可以跨训练与推理负载捕获详细的时间线轨迹和硬件计数器，并利用 NVTX 区间对齐各个事件。它提供从 Python 代码直到 CUDA kernel 的端到端可观测性——甚至还能针对数据加载停顿、低效 CUDA 代码等常见问题给出性能优化建议。

Nsight Systems（nsys）

针对系统级关联分析——涵盖 CPU 线程、GPU kernel、操作系统事件、I/O 以及互联流量——NVIDIA Nsight Systems 能生成统一的时间线视图。其 GUI 与 CLI 报告可以在多进程和多节点运行中合并 NVTX 区域、Python 调用栈与 CUDA 流。这使你能够轻松发现 I/O 与同步停顿可能在哪些位置影响计算性能。

Nsight Compute（ncu）

与 Nsight Systems 互补的是用于逐 kernel 分析的 NVIDIA Nsight Compute。Nsight Compute 会收集占用率、内存带宽、SM 利用率等详细的硬件指标，甚至能生成映射到源代码的 Roofline 图。在其他更上层的工具找出哪些 kernel 是热点之后，Nsight Compute 可以帮助回答某个特定 kernel 为什么慢（例如内存受限、占用率低）。

PyTorch 内存性能分析器

PyTorch 还内置了一个内存性能分析器，可在 `torch.profiler` 中通过 `profile_memory=True` 启用。PyTorch 内存性能分析器按算子拆分 GPU 内存的峰值与累计分配量，从而揭示那些可能被忽视的内存使用热点。

Linux 的 perf

在主机侧，Linux 的 perf 工具可以对 CPU 硬件计数器进行采样，包括时钟周期、指令数和缓存未命中，并展开完整的 C/C++ 与 Python 调用图。借助 `perf sched`，你可以看到 CPU 线程何时因 I/O 或线程调度/同步而处于空闲状态。这能揭示数据预处理循环、Python 的 GIL 或同步机制中导致 GPU“挨饿”的瓶颈。

Holistic Trace Analysis（HTA）

Meta 开源的 Holistic Trace Analysis（HTA）工具可导入 PyTorch 性能分析器的轨迹，帮助诊断多 GPU 瓶颈。借助 HTA，可以将分布式训练时间线连同 NVTX 区间与 CUDA kernel 轨迹一并可视化。通过深入分析随时间变化的内存分配模式，你可以识别出 GPU 空闲的时间段——包括 GPU 之间相互等待的情况。

> **注** TensorBoard 的 PyTorch 轨迹可视化插件已被弃用。请改用 Perfetto 查看时间线，并用 Meta 的 HTA 进行分布式轨迹分析。

Chrome trace 与 Perfetto 查看器

要在浏览器中探索大型的 PyTorch 性能分析轨迹文件，可以使用 Chrome tracing（例如浏览器中的 `chrome://tracing`）和 Perfetto UI。它们能加载 JSON 轨迹文件，让你以交互方式浏览时间线视图与火焰图，甚至可以对轨迹数据执行细粒度的筛选和 SQL 查询——事件轨迹与关联分析可精确到亚毫秒级。Chrome trace 与 Perfetto UI 非常适合在组织成员之间共享性能分析结果，以便开展跨团队分析。（注意：Chrome 的旧版轨迹查看器已被弃用，在查看和分析轨迹时应优先使用 Perfetto Web UI 及其 SQL 引擎。）

TorchEval（PyTorch 的指标库）

另一个项目 TorchEval 让你可以在统一的接口中，把模型的吞吐量、延迟与质量指标同训练及评估指标一起记录和监控。TorchEval 是 PyTorch 官方的指标库，为端到端的性能与质量指标提供了简洁的 API。借助该库，可以方便地嵌入训练循环，也能在分布式环境中集成使用。

ExecuTorch

针对嵌入式、移动和边缘设备，ExecuTorch 项目支持在 Meta 眼镜这类轻量级运行时环境中对 PyTorch 模型进行性能分析、可视化和调试。ExecuTorch 的内存占用小且动态分配，支持 Linux、iOS、Android 和嵌入式系统。Hugging Face 通过其 Optimum ExecuTorch 项目为 ExecuTorch 提供支持——如果你已经在使用 Hugging Face 生态（如 Hugging Face Transformers），那么该环境很容易集成。

接下来，我们通过一个示例深入了解如何使用这些性能分析工具定位性能瓶颈，然后应用有针对性的优化，并验证性能提升效果。

13.2 对 PyTorch 进行性能分析以定位瓶颈

我们以一个混合专家（MoE）transformer 模型为例进行性能分析，看看这些工具的实际效果。MoE 是具有多个专家层的 LLM，每个专家都是一个前馈网络；token 路由到哪个专家由专家门控系统管理。我们将运行一次训练迭代，捕获详细的性能轨迹，并分析输出结果以指导后续优化。

13.2.1 使用 PyTorch Profiler

首先，我们搭建模型与输入。这里使用 Hugging Face Transformers 加载模型和分词器，把模型移到 GPU 上，并准备一小批输入，如下所示：

```
# train.py
# Set up model and data
model_name = "..."
tokenizer = AutoTokenizer.from_pretrained(model_name)
device = torch.device("cuda")
model = AutoModelForCausalLM.from_pretrained(model_name).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, fused=True)
batch_size = 4
input_texts = ["MoEs are great."] * batch_size
enc = tokenizer(input_texts, return_tensors="pt", padding=True,
 ↪  truncation=True)
input_ids = enc.input_ids.to(device)
attention_mask = enc.attention_mask.to(device)
labels = input_ids.clone()  # For LM training, labels are the inputs
                            # (next-token prediction)
```

为避免捕获到一次性的初始化开销，我们在性能分析之前先运行若干次预热迭代。通过编译 JIT kernel、填充缓存等操作，这会让模型为分析与基准测试做好准备。这样，被测的迭代才能代表稳态性能。下面是准备模型的代码：

```
# Warm-up (not profiled)
for _ in range(5):
    with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
        outputs = model(input_ids, attention_mask=attention_mask,
        ↪  labels=labels)
    loss = outputs.loss
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
```

现在我们使用 PyTorch 的性能分析器与 NVTX 对一次训练迭代进行性能分析。我们将该迭代包在 `torch.profiler.profile()` 中，并用 `record_function` 与 NVTX 区间标记高层区域，包括 `"forward"`、`"backward"` 和 `"optimizer_step"`，如下所示：

```
from torch import profiler
with profiler.profile(
    activities=[profiler.ProfilerActivity.CPU,
                profiler.ProfilerActivity.CUDA],
    record_shapes=True,  # record tensor shapes
    profile_memory=True, # track GPU memory usage per op
    with_stack=True,     # enable stack tracing
```

```
    with_flops=True      # capture FLOPs counters
) as prof:
    with profiler.record_function("train_step"):
        # Forward pass
        torch.cuda.nvtx.range_push("forward")
        with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
            outputs = model(input_ids, attention_mask=attention_mask,
                            labels=labels)
        loss = outputs.loss
        # end of forward
        torch.cuda.nvtx.range_pop()
        # Backward pass and optimization
        torch.cuda.nvtx.range_push("backward")
        loss.backward()
        torch.cuda.nvtx.range_push("optimizer_step")
        optimizer.step()

        # end of optimizer_step
        torch.cuda.nvtx.range_pop()
        optimizer.zero_grad()

        # end of backward
        torch.cuda.nvtx.range_pop()
```

在这段代码中，PyTorch profiler 会记录 `train_step` 期间所有的 CPU 和 GPU 活动。我们用 `profiler.record_function("train_step")` 定义一个顶层区域，并插入 NVTX 标记来标注各子阶段（`forward`、`backward`、`optimizer_step`）。这些标记会出现在 profiler 时间线中，用于划分每次迭代的各个阶段。

profiler 还能突出显示模型中已编译与未编译的区域。我们将在本章后续内容及下一章中介绍 PyTorch Compiler、graph break，以及缓解 graph break 的机制。

例如，使用 `torch.compile` 时，追踪结果会显示 `Compiled Function` 之类的事件，并标示出发生 graph break 的位置（见图 13-1）。这有助于精确定位模型回退到 eager 执行之处，从而指导后续优化。

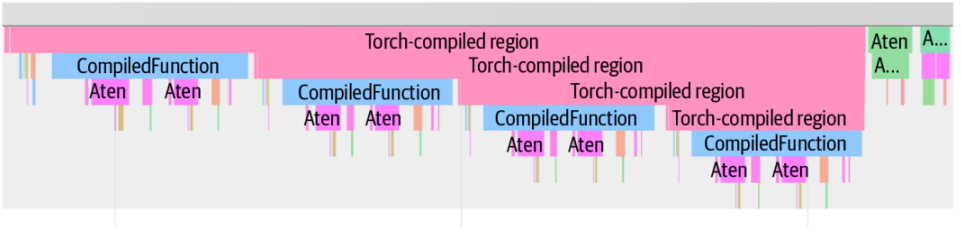

图 13-1 已编译（左、中，粉红色）与未编译（右，绿色）区域（来源：https://oreil.ly/Z_fJG）

执行完成后，我们可以调用 `prof.key_averages().table()` 来查看算子级别的结果，它会打印一张简洁的表格，列出按运行时间排名靠前的算子。在下一个代码块中，我们请求按自身 CUDA 时间（self CUDA time）排序的前 10 个算子；自身 CUDA 时间是指花在每个算子自身的 CUDA kernel 上的时间，不包括该 kernel 派生的子操作所花的时间。CUDA 执行时间最多的 10 个算子汇总在表 13-2 中：

```
with profiler.profile(
    activities=[ProfilerActivity.CPU,
                ProfilerActivity.CUDA],
    record_shapes=True,
    profile_memory=True,
) as prof:
    train_step(...)
...
print(
    prof.key_averages()
        .table(
            sort_by="self_cuda_time_total",
            row_limit=10,
            fields=["self_cuda_time_total",
                    "calls"]
        )
)
```

可以看到，矩阵乘法算子（`aten::matmul` 及其在 `aten::linear` 中的使用）占据了绝大部分 CUDA 时间，耗去了一次迭代的大部分时长。这些算子对应专

表 13-2　一次训练迭代中 CUDA 执行时间排名前 10 的算子。

算子	自身 CUDA 总时间	调用次数
aten::matmul	43.00 ms	128
aten::linear	35.50 ms	64
dispatch	18.70 ms	2
combine	12.10 ms	2
aten::layer_norm	10.20 ms	16
aten::softmax	5.70 ms	4
aten::scatter	4.10 ms	16
aten::gather	3.60 ms	16
aten::to	2.90 ms	8
aten::add_	2.20 ms	64

家前馈网络（FFN）的 GEMM。具体来说，每次迭代有 128 次 matmul 调用。这也符合预期，因为

模型共有 64 个专家——而每个专家在前向传播和反向传播中各执行一次 matmul。

从表 13-2 可以看到，接下来最大的开销来自 dispatch 和 combine 操作。它们是自定义的 C++/CUDA kernel，负责把 token 重新分发到各个专家——随后汇集专家的输出。dispatch 操作运行两次——一次在前向传播，一次在反向传播——共耗时 18.7 ms；combine 也运行了两次，总计 12.1 ms。这两项操作合计又占用了 30.8 ms 的 GPU 时间。剩下的时间则分摊在 layer norm、激活等其他较小的算子上。

这个性能分析示例的关键结论是：专家 FFN 的 matmul 是头号瓶颈，其次是 dispatch 和 combine kernel。它们共同主导了一次训练迭代的运行时间。要进一步提升性能，就应当瞄准这些操作：要么直接优化它们，要么减少它们的调用次数。

13.2.2 使用 Nsight Systems 和 NVTX 时间线进行系统级性能分析

我们插入的 NVTX 标记让用 Nsight Systems 分析时间线变得十分简单。要按阶段聚合指标，可以用 nsys 进行性能分析并生成基于 NVTX 的摘要，CLI 命令如下：

```
nsys profile \
   --output=profile \
   --stats=true \
   -t cuda,nvtx \
   python train.py
```

其中 `-t cuda,nvtx` 选项指示 Nsight Systems 同时追踪 CUDA API 调用与 NVTX 范围。分析完成后，我们可以在 Nsight Systems GUI 中打开 `profile.nsys-rep` 文件（`-{-}output=profile`），也可以通过 CLI 将 NVTX 摘要打印到终端。随后，我们可以对 `profile.nsys-rep` 文件使用以下命令之一来生成 NVTX GPU Projection Summary，如下所示，从而用 Nsight Systems 将各范围与投影后的 GPU 工作进行核对：

```
nsys stats --report=nvtx_gpu_proj_sum \
  profile.nsys-rep
# or
nsys recipe nvtx_gpu_proj_sum \
  profile.nsys-rep
```

你可以在持续构建和集成流水线中使用上述任一命令，以监控并检测性能回退。该 CLI 命令的结果汇总在表 13-3 中。

可以看到，`train_step` 范围包含了 forward、backward 和 optimizer 三个子范围。这份 NVTX GPU 投影摘要确认，`train_step` 之下的 GPU 总时间为 138 ms。该时间与表 13-2 中 PyTorch profiler 输出的 forward、backward 和 `optimizer_step` 时间之和相符，说明两个工具的结果是一致的。

而且，尽管表 13-3 只显示了一次 `optimizer_step` 调用，其 NVTX 范围实际上把全部 64 次 `aten::add_` CUDA kernel 启动（如表 13-2 所示，每个专家一次 `add_`）都归入到这一个 optimizer_step 标记之下。

表 13-3 使用 Nsight Systems 得到的一次 `train_step` 迭代的 NVTX GPU 投影摘要。

NVTX 范围	GPU 时间 (ms)	自身 GPU 时间 (ms)	子级 GPU 时间 (ms)	实例（调用次数）
`train_step`	138.0	0.0	138.0	1
`forward`	60.5	60.5	0.0	130
`backward`	58.3	58.3	0.0	130
`optimizer_step`	19.2	19.2	0.0	1

注意，Nsight Systems 之所以把 64 次 `aten::add_` 调用（例如 64 路专家并行策略）合并到单个 optimizer_step 标记之下，是因为它通过 CUDA Profiling Tools Interface（CUPTI）在主机端捕获 NVTX push/pop 事件，然后把异步的 GPU kernel 执行时间“投影”到这些 CPU 侧定义的区间上：凡是 GPU 开始/结束时间戳落在相应 push 与 pop 调用之间的 kernel，其时长都会被累加起来。由此得到的一个累计 GPU 时间，恰好等于各个 `aten::add_` kernel 的总时长。

> **注** 由于在未附加 profiler 时 NVTX 标记的开销极低，这种投影机制非常理想：它几乎不增加任何开销，却能将 GPU 工作与高层代码区域端到端地关联起来。

forward 和 backward 范围的自身 GPU 时间等于其总时间，因为我们没有在它们内部再嵌套更深的范围，因此其子级 GPU 时间为 0 ms。而 `train_step` 几乎所有时间都是子级 GPU 时间，因为它只是嵌套各阶段的一层包装。

NVTX GPU 投影汇总还显示，在每次迭代中，我们在 `train_step` 内部观察到了 130 个 GPU 活动。其中包括 kernel 启动以及内存拷贝等其他设备操作，因此它们与 kernel 并非严格一一对应。

如表 13-3 所示，这 130 次 GPU kernel 调用同样分布在正向与反向传播中。操作与 NVTX 实例之间的这种一一对应关系意味着，我们的插桩捕获了每一项重要操作。

> **注** 表 13-3 中的 NVTX 汇总提供了一种便捷的文本概览。若要进行可视化分析，时间线 GUI 可以展示重叠的 kernel 执行、CPU 线程状态，甚至 CUDA API 开销事件。在实践中，你需要在可视化时间线中确认宿主机端

的数据加载与预处理是否与 GPU 计算重叠。任何较大的空隙（即“气泡”）都说明存在问题；而用于同步的小空隙则是预期之内且合理的。

在多 GPU 运行中，Nsight Systems 或 HTA 的时间线视图可以揭示 NVLink 或 InfiniBand/以太网是否被有效利用——或者某个节点是否因等待通信或网络延迟而无事可做。这种情况暗示同步欠优或负载不均衡。

追踪 GPU 通信事件非常重要，包括 NCCL all-reduce 调用与 NVLink/NVSwitch 活动，可使用 Nsight Systems 和 HTA 来提供追踪数据。这有助于验证 GPU 在 NVL72 系统这类大规模 GPU 域中保持忙碌。

细致的性能分析可以确保系统在这些大型 NVLink 集群中使用合理的 GPU 间同步，并平衡工作负载。接下来，让我们放大观察系统中开销最高的 kernel 之一：矩阵乘法，即 matmul。

13.2.3 通用矩阵乘法（GEMM）的 kernel Roofline 分析

为了深入分析专家 matmul，我们调用 CLI 性能分析器 Nsight Compute（`ncu`），按名称定位特定的 GEMM kernel。我们将收集与 roofline 相关的指标，以确定它是计算受限还是内存受限，如下所示：

```
ncu \
  --target-processes all \
  --kernel-name-regex "matmul" \
  --metrics \
    gpu__time_duration.avg, \
    gpu__dram_throughput.avg.pct_of_peak_sustained_elapsed, \
    lts__throughput.avg.pct_of_peak_sustained_elapsed, \
    sm__sass_thread_inst_executed_op_fp32_pred_on.sum, \
    sm__warps_active.avg.pct_of_peak_sustained_active \
  --csv full \
  -o matmul_roofline_report \
  python train.py
```

在这里，我们为名称匹配“matmul”的所有 kernel 收集硬件计数器。具体而言，我们收集若干关键指标，包括 GPU DRAM 带宽（以峰值百分比计）

```
L2 bandwidth as a percent of peak (gpu__dram_throughput.avg.pct_of_peak_
sustained_elapsed, lts__throughput.avg.pct_of_peak_sustained_elapsed),
↪ FP32
instruction count as a compute proxy (sm__sass_thread_inst_executed_op_
fp32_pred_on.sum), kernel time (gpu__time_duration.avg), and achieved
↪ occupancy
(sm__warps_active.avg.pct_of_peak_sustained_active).
```

> **注** 指标标识符在不同版本的 Nsight Compute 之间可能略有差异。如果找不到某个指标，可以使用 UI 查找其当前名称并相应替换。这里，我们分析的是 SM 与 DRAM 占峰值持续吞吐的百分比以及实际占用率。

在应用了前文讨论的优化（例如降低精度、融合 kernel、提高算术强度）之后，我们可以重新运行 `ncu` 命令，验证这些优化是否产生了积极效果。表 13-4 展示了应用这些提高算术强度的优化前后的对比。

表 13-4 专家 matmul kernel 在算术强度优化前后的 roofline 分析。

内核配置	峰值 FLOPS 百分比（SM 计算吞吐）	峰值内存带宽百分比（内存吞吐）	实际 SM 占用率	特征
基线	50%	70%	60%	内存受限（因内存传输而停顿）
优化后	85%	40%	80%	计算受限（接近硬件 roofline）

内核配置

峰值 FLOPS 百分比（SM 计算吞吐）

峰值内存带宽百分比（内存吞吐）

实际 SM 占用率

特征

基线：峰值 FLOPS 50%，峰值内存带宽 70%，SM 占用率 60%，内存受限（因内存传输而停顿）；优化后：峰值 FLOPS 85%，峰值内存带宽 40%，SM 占用率 80%，计算受限（接近硬件 roofline）。

这里我们看到，在基线运行中，主力 GEMM kernel 只达到约 50% 的峰值计算 FLOPS、70% 的峰值内存带宽，以及中等的 60% SM 占用率（每个 SM 的平均活跃 warp 数）。这样的占用率不足以完全隐藏内存延迟。

> **注** 并不存在一个通用的占用率目标。许多高性能 kernel 在 25%–50% 的实际占用率下就能实现充分的延迟隐藏。应当将 Nsight Compute 的占用率指标与停顿原因分解、每个活跃周期的合格 warp 数结合起来，判断更高的占用率能否减少该 kernel 的停顿。如果 kernel 无法调度足够的 warp 来覆盖停顿期，内存请求得不到及时服务，就会产生空闲周期。

基线指标表明这是一个内存受限的 kernel，因为它的执行被内存传输所停顿，其结果是大量计算能力处于闲置，

这也进一步印证了该工作负载目前并非计算受限。我们的目标是让这个 kernel 更加趋向计算受限，以充分利用这块 GPU 所具备的巨大 FLOPS。

在优化后的版本中（例如融合 kernel、提高算术强度、减少内存搬运），峰值 FLOPS 提升到 85%，峰值内存带宽下降到 40%，占用率提升到 80%。我们实际上把这个 kernel 从内存受限转移到了计算受限——更接近硬件的 roofline 极限，如图 13-2 所示。

到目前为止，我们的性能分析一直聚焦于 GPU 性能。同样重要的是，不要把时间浪费在 CPU 或 I/O 上。在下一节中，我们将把性能分析之旅转向主机端。

13.2.4 使用 Linux perf 进行 CPU 与 GPU 性能分析

为了更全面地了解时间在主机端和设备端分别花在哪里，我们可以使用 Linux perf 来分析 CPU 周期、缓存未命中、分支预测失败等事件。然后，我们可以利用这些洞察来驱动一系列优化，逐项应用并度量它们带来的改进。

首先，让我们在一台配备基于 ARM 的 Grace CPU 且搭配 Blackwell GPU 的节点上，运行一个轻量级的 `perf stat`，在 MoE 训练运行期间收集 CPU 侧的统计信息。下面是 CLI 命令，随后是示例输出：

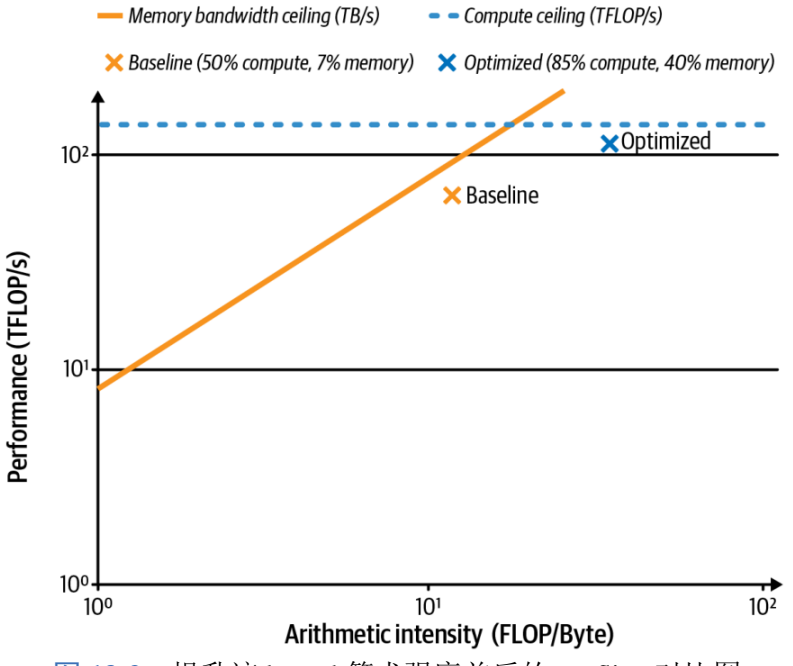

图 13-2　提升该 kernel 算术强度前后的 roofline 对比图

```
perf stat -e \
  cycles,instructions,cache-misses,branch-misses \
  python train.py
  Performance counter stats for 'python train.py':
# 0.600 CPUs utilized
1,200.345 msec task-clock
# Approximately 2.0 GHz
2,400,567,890      cycles
# 1.58 insn per cycle
3,800,123,456      instructions
# 0.32% of all cache refs
12,345,678      cache-misses
# 0.12% of all branches
4,567,890      branch-misses
 1.234567890 seconds time elapsed
```

这份 `perf stat` 报告展示了 CPU 利用率、周期数、每周期指令数（IPC）以及缓存/分支未命中情况。在我们的运行中，task clock 显示约 1.2 秒，而在测

量区间内使用的 CPU 算力仅相当于单核的约 60%（0.600）。这在预期之内，因为大部分繁重的工作都由 GPU 承担。较低的缓存未命中率与分支预测失败率表明，对于这个工作负载而言，CPU 侧的内存访问模式和分支预测是相对高效的。

然而，仅为 1.58 的 IPC 测量值表明，CPU 的指令发射水平远低于单个 Grace CPU 核（ARM Neoverse V2）每周期 8 条指令的理论最大 IPC。这暗示在这个特定工作负载中可能存在内存延迟、I/O 停顿或主机端计算方面的问题。

我们可以进一步使用 `perf record` 和 `perf report` 来定位在训练期间占用 CPU 执行时间最多的 Python 和 C++ 函数。相关 CLI 命令如下所示：

```
perf record -F 2000 -g --call-graph dwarf -o perf.data \
    python train.py
```

这里，我们使用 `perf record` 以 2000 Hz 的频率（`-F 2000`）采集样本，并通过 `-g --call-graph dwarf` 捕获完整的 C/C++ 与 Python 调用栈。DWARF 是 Debugging With Attributed Record Formats 的缩写，它是一种嵌入在编译后二进制文件（例如 ELF 文件）中的标准调试数据格式。DWARF 输出的跟踪数据被保存到 `perf.data`（`-o perf.data`）。随后，我们使用 `perf report` 生成一份汇总报告，列出最热的调用栈及其采样占比：

```
perf report --stdio -n -g -i perf.data
# Samples  Command   Shared Object        Symbol
# ........ ........  ..................
 ↪  ................................
    45.0%  python    python              py::forward <...> /src/train.py
     20.5%  python    python                aten::matmul
     10.2%  python    python                dataloader_iter_next
      8.7%  python    libnccl.so            ncclAllReduce
      5.3%  python    libc.so.6             read
      ...    ...       ...                   ...
```

可以看到，Python 解释器的 `forward` 函数——即我们训练循环的 Python 侧——占 CPU 采样总数的 45.0%。PyTorch 的 C++ 算子 `aten::matmul` 占 20.5%，DataLoader 迭代器的 `next` 函数占 10.2%，NCCL 的 all-reduce 调用占 8.7%，I/O 读取占 5.3%。

这些百分比告诉我们应当把优化精力投放在何处。基于这份性能分析结果，我们针对每一个瓶颈制定了相应的缓解方案，以提升系统性能：

Python 开销过大（`py::forward` 中占 45%）

使用 PyTorch 的 JIT 编译器（通过 `torch.compile`，下一节将讨论）来消除解释器开销，并把 Python 侧的多个操作融合为优化后的 CUDA 代码。

matmul 热点过大（`aten::matmul` 中占 20.5%）

要么用 PyTorch 编译器来优化这段代码，要么把这一关键的矩阵乘法移入自定义 CUDA C++ kernel（例如融合 kernel），从而绕过 Python、直接使用优化后的 CUDA 代码。

数据加载停顿（`dataloader_iter_next` 中占 10.2%）

增大 PyTorch DataLoader 的 `num_workers`。一条常见的准则是每个 CPU 核配一个 worker，但你可以尝试更多的 worker，以找到合适的 I/O 并行度——只要注意不要超额占用 CPU 核心即可。你还应启用 `persistent_workers=True`，让 worker 进程跨 epoch 保持存活，从而避免每个 epoch 的启动开销。将多个 `torch.utils.data.DataPipe` 融合或并行执行，这可以降低复杂数据管线中的 Python 开销。

梯度同步开销（`ncclAllReduce` 中占 8.7%）

优化多 GPU 通信。例如，可以增大 `DistributedDataParallel` 中的梯度桶大小，把 `bucket_cap_mb` 从 25 MB 提高到 50 MB，让 NCCL 能更早发起 all-reduce 操作并使其与反向计算重叠。你也可以考虑梯度压缩技术，或使用 NVIDIA NCCL 对 8 位梯度进行压缩，以减少带宽占用——不过这些手段可能对精度带来轻微损失。

主机 I/O 瓶颈（`read` 系统调用中占 5.3%）

在 DataLoader 中使用锁页内存（`pin_memory=True`）和非阻塞 GPU 拷贝（`.to(device, non_blocking=True)`），让 CPU 到 GPU 的数据传输与其他工作重叠进行。此外，还可以批量读取文件，或者把许多小文件打包进 Arrow、WebDataset（tar 分片）、TFRecord 或 Parquet 这类便于大块顺序读取的优化数据集格式。最好优先选择连续的分片格式，而不是逐样本的独立文件。在使用较大的预取深度（例如 `prefetch_factor=4` 或 8）时，应优先使用锁页的主机缓

冲。配合 `persistent_workers=True`，由于计算与通信能够高效重叠，你的系统将让加载线程持续保持忙碌。这将消除在大型语料库中读取海量小文件时的逐文件开销。

这些方法配合较大的操作系统预读（read-ahead）和 NVMe SSD，将显著提升 I/O 吞吐量。此外，诸如 NVIDIA Magnum IO 这类较新的文件系统和存储库，也可以帮助把数据高效地以流水线方式送入 GPU。

制定好这一方案后，你应当系统地逐一应用各项优化并度量其效果。请记住：一次只实现并测试一项优化，以验证每一项确实带来了性能提升。这有助于避免多项改动以预料之外的方式相互影响。通过隔离每一项改动，你就能知道哪些调整有效、哪些无效。

在装有 NVIDIA 性能监控单元（PMU）驱动的系统上，你可以使用 `perf` 在采样 CPU 计数器的同时采样 NVIDIA 芯片互连与 fabric 计数器，例如采样在 `/sys/bus/event_source/devices` 下以 `nvidia_nvlink_c2c*` 形式暴露的 NVLink-C2C 设备。可通过 `perf list` 确认可用性，并检查 sysfs 下的 `nvidia_pmu` 条目。

> **注** 面向 NVIDIA PMU 的 Linux `perf` 仅限于设备级的链路与 fabric 事件，例如 Grace-Blackwell 上的 NVLink-C2C。SM 流水线、warp 停顿和内存吞吐计数器仍然只能通过 CUPTI 和 Nsight 工具获取——这些 PMU 并不会暴露 SM 级的 kernel 指标。要查看 SM 占用率、占用情况和内存吞吐量，请使用 Nsight Compute 或基于 CUPTI 的性能分析器。请务必设置 `NVreg_RestrictProfilingToAdminUsers=0`，以允许非 root 用户对 SM 级硬件计数器进行性能分析。

PMU 设备就绪之后，你就可以同时采集 CPU 事件与 NVIDIA 事件。使用 `perf list` 报告的符号化事件名：

```
perf list | grep -i nvidia
perf stat -a \
  -e nvidia_nvlink_c2c0_pmu_0/cycles/ \
  -e cycles,cache-misses \
  python train.py
```

这里，NVLink-C2C PMU 上的 `cycles` 事件可以让你把 GPU 互连活动与主机 CPU 的行为关联起来。以下是前面 `perf stat` 命令的示例输出：它显示在本次运行期间 NVLink C2C PMU 记录到了活动，同时 CPU 发生了 `cycles` 计数与缓存未命中：

```
Performance counter stats for 'python train_deepseek_v3.py':
       3,567,890,123  nvidia_nvlink_c2c0_pmu_0/cycles
          45,678,901  cycles
           7,890,123  cache-misses
       2.345678901 seconds time elapsed
```

根据 PyTorch profiler 和 Nsight 工具的初步分析结果，GPU 计算（例如专家矩阵乘法）和 GPU 通信（例如 dispatch 与 combine 操作）是主要瓶颈。不过，CPU、数据加载和 GPU 集合通信操作同样会影响性能——`perf` 通过显示慢速区间内哪些 CPU 线程与互连 PMU 处于活跃状态，证实了这一点。

> **注** 要进一步挖掘更多的链路与 fabric 请求计数器，可以从 `perf list` 中挑选你系统上出现的其他 NVIDIA PMU 事件，并按照前述方式把它们加入 `perf stat` 命令。

简而言之，将 `perf` 提供的高层 CPU 吞吐指标和调用图热点，与 Nsight Systems 和 Nsight Compute 提供的设备指标与时间线结合起来，你就能构建一个贯穿主机与设备的完整性能图景。先解决最大的 CPU 侧瓶颈和数据停顿，接着优化 GPU 通信并调优 GPU kernel。

13.3 PyTorch 编译器（torch.compile）

在 PyTorch 中见效最快的优化之一，就是通过 `torch.compile()` 使用 PyTorch 编译器。该编译器栈包含 TorchDynamo、AOT Autograd 和 TorchInductor：它们负责捕获计算图、融合算子，并为目标后端（例如 NVIDIA GPU）生成高性能代码。

PyTorch 编译器将许多小操作融合成更大的 kernel，从而消除大量 Python 解释器开销和 GPU kernel 启动延迟。在完成基线性能分析之后，我们在模型上启用了 `torch.compile`，看看能否轻松获得加速。下面来介绍这一过程及其结果。

13.3.1 使用 PyTorch 编译器

以默认设置使用 PyTorch 编译器非常简单：除了像下面这样包装模型（`model = torch.compile(model)`）之外，不需要修改任何代码。在底层，TorchDynamo 对 Python 代码进行追踪，AOT Autograd 捕获反向传播，而 TorchInductor——它借助 OpenAI 的 Triton 生成 GPU kernel 代码（下一章讨论）——则自动产出高效的融合 kernel。

编译器会观察我们模型的前向传播，并识别出大量融合相邻操作的机会，例如逐元素激活、层归一化等。它为这些操作生成融合 kernel——同样也为反向传播的某些部分生成融合 kernel。结果是 kernel 启动次数显著减少，每次迭代的 CPU 开销也更低。

编译步骤确实会引入秒级——对超大模型甚至是分钟级——的开销，但这一成本会在长时间的训练任务或反复的推理运行中被摊薄。所幸，TorchInductor 会缓存编译好的 kernel，因此后续运行无需再次支付编译成本。PyTorch 社区也在持续改进编译/启动性能，允许你保存并在运行之间复用编译产物。使用 `torch.compiler.save_cache_artifacts()` 和 `torch.compiler.load_cache_artifacts()`，可以把 TorchInductor 的输出在多次运行乃至多个节点之间持久化。这能缩短长时间训练或服务场景下的启动时间。

一个例子是 PyTorch 的 Mega-Cache 特性。这是一个端到端的编译缓存，可以把编译好的 kernel 保存到磁盘，并在未来的运行中重新加载。借助 PyTorch

Mega-Cache，你只需编译一次（例如离线完成），就可以在多个训练会话中复用优化后的 kernel，这有助于缩短启动时间。你依然能享受到 TorchInductor 的各项 kernel 优化（例如 warp specialization），却不必每次都重新编译计算图。

> **注** 你甚至可以在其他计算节点上复用这个编译缓存。若要这样做，务必确保各节点上的 CUDA、PyTorch 与 Triton 版本相互兼容。

值得注意的是，PyTorch 编译器在内部应用了相当精巧的优化技术。例如，我们在第 10 章讨论过 warp 特化。TorchInductor 的自动调优器会围绕不同的 tile 尺寸、内存访问模式等生成多个 kernel 变体，并在幕后应用内存 warp 与计算 warp 特化这类技术，然后针对你的硬件自动选出最快的变体。

TorchInductor 支持围绕 GEMM kernel 做前导（prologue）与收尾（epilogue）融合。例如，偏置加法发生在 matmul 之前；而 matmul 之后的收尾部分则由逐元素操作构成，如激活、dropout 和残差连接。

通过把这些 kernel 前导与收尾操作合并成单个优化后的 kernel，TorchInductor 减少了内存流量，把 kernel 启动开销降到最低，并提高了占用率。你可以用性能分析器验证这一点，它将显示出更高的 SM 利用率。

这套优化的复杂性对开发者完全透明，因为 PyTorch 呈现的是一个以张量为中心的简洁接口，并不暴露 CUDA 层面的 warp 细节。因此，虽然在 PyTorch API 中看不到“内存 warp”或“计算 warp”这样的开关，但要知道这些技术正在底层被使用。代码完成编译之后，你会从性能分析指标中看到 warp 特化的收益，包括更高的占用率、更少的内存延迟停顿和更高的 SM 利用率。

为了说明编译模式的收益，我们在一个 MoE 模型上比较 PyTorch 的 eager 模式与编译执行。我们先在常规 eager 模式下对模型的单次训练迭代计时，再用 `max-autotune` 编译模式重复计时。代码如下所示，其后是示例输出：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
# ---- Setup Model ----
device = 'cuda'
model_name = "..."
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, fused=True)
# ---- Create a dummy batch of token IDs ----
batch_size = 4
input_texts = ["MoE's are awesome!"] * batch_size
enc = tokenizer(input_texts, return_tensors="pt", padding=True,
↪  truncation=True)
input_ids = enc.input_ids.to(device)
attention_mask = enc.attention_mask.to(device)
labels = input_ids.clone()  # for causal LM, labels = input_ids
# ---- Make runs deterministic ----
torch.backends.cudnn.benchmark = False
torch.backends.cudnn.deterministic = True
# --- Eager timing ---
torch.cuda.synchronize()
start = torch.cuda.Event(enable_timing=True)
end = torch.cuda.Event(enable_timing=True)
for _ in range(iters_warmup):
```

```
    out = model(input_ids, attention_mask=attention_mask, labels=labels)
    loss = out.loss if hasattr(out, "loss") else out
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
torch.cuda.synchronize()
start.record()
for _ in range(iters_meas):
    out = model(input_ids, attention_mask=attention_mask, labels=labels)
    loss = out.loss if hasattr(out, "loss") else out
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
end.record()
torch.cuda.synchronize()
print(f"Eager mode step time: {start.elapsed_time(end)/iters_meas:.3f}
 ↪ ms")
# --- Compile the model (choose one mode)
# enables graph trees
compiled_model = torch.compile(model, mode="reduce-overhead")
# Alternatives:
# more tuning, longer compile
# compiled_model = torch.compile(model, mode="max-autotune")
# balanced
# compiled_model = torch.compile(model, mode="default")
# Warm-up compiled
for _ in range(iters_warmup):
    out = compiled_model(input_ids, attention_mask=attention_mask,
     ↪ labels=labels)
    loss = out.loss if hasattr(out, "loss") else out
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
torch.cuda.synchronize()
start.record()
for _ in range(iters_meas):
```

```
    out = compiled_model(input_ids, attention_mask=attention_mask,
    ↪  labels=labels)
    loss = out.loss if hasattr(out, "loss") else out
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
end.record()
torch.cuda.synchronize()
print(f"Compiled mode step time:
↪  {start.elapsed_time(end)/iters_meas:.3f} ms")
```

在本例中，PyTorch eager 模式每次迭代大约需要 248 ms。经过预热并让编译器完成其优化之后，编译模式的运行时间约为 173 ms。我们这个使用 `max-autotune` 的编译版本比 eager 执行快约 30%。实际加速比会随模型结构、批次大小和动态

形状而异。以单个大型 GEMM 为主的稠密模型，可能只能获得 <10% 的加速。

这些节省主要来自把许多小 GPU kernel 合并起来。在 MoE 架构中，token 的分发/合并以及逐 token 的激活模式会产生大量小 GPU kernel。把这些小操作融合成更少、更大的 kernel 之后，中间数据就能留在更快的片上内存里——例如寄存器和共享内存——而不必反复与全局设备内存（HBM）来回搬运。

> **注** 对于 MoE 架构所用的高度动态的 token 路由，建议优先使用 `default` 或 `max-autotune-no-cudagraphs`；等输入形状稳定之后，或者当你配合数量有限的离散形状使用 CUDA Graph Trees 时，再切换到 `max-autotune`。

在本例中，许多小操作（如分发/合并、激活函数等）都被 TorchInductor 融合掉了。查看编译后运行的 trace 时，你会看到时间线上的 GPU kernel 数量大幅减少；取而代之的是数量更少但运行时间稍长的 kernel，它们对应的是一次性执行多个步骤的合并操作。

迭代次数多了之后，收益会更加明显，因为一次性的编译开销已经被摊薄。对编译模型的执行做性能分析可以看到，GPU kernel 启动次数大幅减少，许多在 eager 模式下相互独立的操作如今被融合在了一起。

值得注意的是，如果稠密模型（例如因为某个大型线性层）由一个巨大的 GEMM 主导，那么 `torch.compile` 带来的收益可能很有限（例如 $< 10\%$）。这是因为该模型很可能已经在高效地使用 Tensor Core，而且几乎没有 kernel 融合和消除 Python 开销的机会。

相比之下，像 MoE 这样包含数百个中等规模 matmul 操作的稀疏架构则会收获颇丰，因为编译降低了 Python 开销、降低了 kernel 启动延迟，并把多个步骤融合进优化后的 kernel。因此，与稠密模型相比，PyTorch 编译器为 MoE 带来的性能提升要显著得多。

除了自动算子融合之外，你还可以把用户自定义的 kernel 直接集成进 `torch.compile` 工作流。这种方式可谓两全其美：对一般模式交给编译器做图级优化，而在需要时你又能获得完全的控制权。

例如，你可以为性能关键的操作编写专门的 Triton 或 CUDA kernel，并将其注册为自定义算子。在对模型进行追踪并

编译时，TorchInductor 会把它当作整个执行图中的一个单一融合操作来对待。其结果是：手工调优的自定义代码被嵌入到一个经过全面优化、由编译器管理的执行图之中。

TorchInductor 的这种灵活性让你的自定义 kernel 也能受益于编译器对周边操作的优化（例如与相邻层融合等）。实际上，这意味着你既可以放心使用自己的高性能 kernel，又不会失去 PyTorch 编译器优化模型其余部分的能力。

简而言之，在训练循环中使用 PyTorch 编译（包括其中的 `max-autotune` 模式），只需相对较低的成本，就能在现代 GPU 上获得可观的加速。这一效果可以通过 `torch.profiler`、Linux perf、Nsight Systems、Nsight Compute 以及其他实用的性能分析器进行整体验证。

13.3.2 编译还是手写自定义 kernel

有了 TorchInductor 这样的编译后端，许多操作会被自动融合成高效的 kernel。如前所见，仅使用 `torch.compile` 就能以极小的代价获得可观的提升。然而，有时自动生成的代码可能不如精心编写的自定义 kernel 那样最优，或者某个操作根本没有被编译器捕获。这就引出了一个问题：什么时候应当依赖编译器的融合，什么时候应当自己编写 custom CUDA kernel？

对多数情况而言，首选是使用带图捕获的高层接口 `torch.compile`（底层由 TorchInductor 驱动）。这比手写 custom CUDA kernel 省力得多，而且无需专门编码，往往就能带来足够好的性能提升。

TorchInductor 内部已经应用了许多高级优化，例如逐元素操作的融合、层操作的合并、布局优化等。手写融合 kernel 既费时又脆弱，而编译器在大多数情况下都能自动完成这些工作。

如果你的模型使用了编译器处理不佳的新颖操作或模式，就有可能需要编写自定义 kernel，并向 PyTorch 集成自定义算子。下一章会更详细地介绍具体做法。

简而言之，性能调优应首先诉诸 `torch.compile`，因为它简单、够用，而且相对“免费”。创建自定义 kernel 是下一层次的优化，用于内置自动化手段不够用的场合；只有在那之后，才需要考虑为剩余热点编写自定义 kernel，以融合某些不受支持的优化器操作或专门的注意力模式。不过，即便是专门的注意力模式，PyTorch 也提供了 FlexAttention API（预填充）和 FlexDecoding API（解码），它们才是为训练和推理在 PyTorch 中实现自定义注意力 kernel 的首选方式，我们将在稍后的一节中看到。

13.3.3 编译模式及其在速度、内存与编译时间上的权衡

PyTorch 为 `torch.compile` 提供了若干模式，用于针对不同场景调整编译器的激进程度与能力。你可以通过 `torch.compile(model, mode=...)` 显式选择模式，可选值为 `default`、`reduce-overhead`、`max-autotune` 和 `max-autotune-no-cudagraphs`。每种模式都给出了一组关于 CUDA Graphs、自动调优和优化级别的选项组合。各模式汇总于表 13-5。

表 13-5　编译模式及其关键特性汇总。

模式	说明	编译时间	额外内存	显著特性
`default`	均衡的优化（无需较长编译或额外内存即可获得良好速度）；包含轻度自动调优；可能对稳定代码段使用 CUDA Graphs	低–中	否	通用融合、基础自动调优
`reduce-overhead`	降低每次迭代的开销（适合小批次）；最适合推理或小批次；检测到动态形状时为保持正确性会自动跳过 CUDA Graphs	中	是（工作区缓存）	尽可能使用 CUDA Graphs 以消除启动开销
`max-autotune`	最大化运行时性能（最适合长时运行）；编译时间更长；适合对大量 SM 和 GPU 内存做激进调优	高（编译缓慢）	若使用了 CUDA Graphs 则可能	激进的 Triton 自动调优；在 GPU 上启用 CUDA Graphs
`max-autotune-no-cudagraphs`	完成 `max-autotune` 所做的一切，但不进行 CUDA Graph 捕获。最适合动态形状，或用于调试被 CUDA Graphs 掩盖的问题	高	否	与 `max-autotune` 相同，但禁用 CUDA Graphs 以保持灵活性

模式　说明　编译时间

额外内存　显著特性

`default`：均衡的优化（无需较长编译或额外内存即可获得良好速度）；包含轻度自动调优；可能对稳定代码段使用 CUDA Graphs

低–中

否。通用融合、基础自动调优

降低每次迭代的开销（适合小批量）；适合推理或小批量场景；若检测到动态形状会自动跳过 CUDA Graphs，以保证正确性

中等。是（工作区缓存）

在可能的情况下使用 CUDA Graphs 消除启动开销

`reduce-`
`overhead`

最大化运行时性能（适合长时间运行）；编译时间更长；最适合对大量 SM 与 GPU 内存进行激进调优

高（编译缓慢）

可能（若使用 CUDA Graphs）

激进的 Triton 自动调优；在 GPU 上启用 CUDA Graphs

`max-autotune`

高。否。与 `max-autotune` 相同，但为保持灵活性而禁用了 graphs

执行 `max-autotune` 的全部工作，但不进行 CUDA Graph 捕获。最适合动态形状，或用于调试被 CUDA Graphs 掩盖的问题

`max-autotune-no-cudagraphs`

下面详细描述表 13-5 中的各编译模式：

`default`

这是未显式指定模式时的默认模式。该模式提供了一种平衡：编译出的代码足够快，而不会带来过度的编译时间或内存占用。它会执行标准优化，并使用默认的 TorchInductor 后端。对于编译时间需要适中——或内存紧张——的大型模型，该模式通常最佳。此模式包含少量自动调优，并可能对稳定的代码段使用 CUDA Graphs，但它会尽量在速度与编译时间成本之间取得平衡。

`reduce-overhead`

该模式专注于最小化 Python 与运行时开销。它对小型模型——或每次迭代只执行很少次数的模型——尤其有用。在这些场景下，哪怕一点点开销都会损

害性能。该模式大量利用 CUDA Graphs 来消除每次迭代的启动开销。它还可能分配一些额外的内存供持久化复用，例如可复用的工作区内存，从而避免频繁的 CUDA `malloc` 与 `free` 调用。例如，它可能将一个较大的工作张量缓存起来，而不是每次迭代都重新分配。该模式在检测到动态形状时会自动跳过 CUDA Graphs，回退到 eager 模式，以此来保证正确性。

该模式能够加速低延迟场景下的推理与训练——代价是需要一些额外的内存。注意，CUDA Graphs 要求图中的内存地址保持不变，因此只有在输入尺寸不变、且不存在某些操作（如动态形状操作）时才能使用该模式。否则，图会失效或需要重新编译。若在某个代码段中无法应用 CUDA Graph，编译器会自动回退。

`max-autotune`

该模式生成尽可能快的代码，而不顾及编译时间。它会对 kernel 进行充分的自动调优——例如，为矩阵乘法尝试多种分块（tiling）配置——并利用 TorchInductor 中所有已知的性能优化。在现代 GPU 上，`max-autotune` 还会默认启用 CUDA Graphs，以获得稳定的执行。

该模式下的编译过程可能明显更长——对大型模型来说会达到分钟量级。它适用于编译一次、之后多次运行模型的场景，例如运行长时间的训练任务，或部署一个今后要持续处理大量请求的模型。以较长的前期编译时间换来的，往往是最佳的运行时性能。例如，经过自动调优后，你的 matmul 可能会以针对你的特定 GPU 与张量形状最优的分块大小运行，这使该模式比默认的启发式方法更有优势。

`max-autotune-no-cudagraphs`

顾名思义，该模式与 `max-autotune` 相同，但禁用了 CUDA Graph 捕获。当 CUDA Graphs 会干扰期望的运行时行为、且这些行为与 CUDA Graphs 不兼容时，这一点很重要。例如，由于 CUDA Graphs 要求静态的形状与内存地址，你不能使用变化的输入形状，也不能依赖每次迭代分配新内存。

此外，在测量性能时，使用 CUDA Graphs 可能掩盖启动 kernel 的开销，这在某些基准测试中可能并非所愿。因此，该

模式允许在不使用 CUDA Graphs 的情况下进行最大化的 kernel 优化。这有助于保持灵活性，并让你得以调试 CUDA Graphs 可能引入（或掩盖）的问题，例如依赖形状的控制流，或图捕获期间偶发的分配器重新寻址。当输入大小每次迭代都在变化时，应使用该模式——或者用于调试可能被 CUDA Graphs 掩盖的问题。

对大多数用例而言，default 模式是一个不错的起点，它旨在于尽量少的麻烦下带来可观的加速。如果你发现模型仍然不够快，且能容忍更长的编译时间，可以尝试 `reduce-overhead` 与 `max-autotune`，以获得可能更好的融合 kernel——尤其是当你的模型以可被自动调优的大型 matmul 运算为主时。`max-autotune` 在某些模型上有时反而会使延迟变差。务必针对你的具体工作负载和硬件，对各种模式进行性能分析与测量。

另一方面，如果你在优化一个非常小的模型，或某段以 Python 开销为瓶颈的代码——例如包含大量小张量运算的紧凑训练循环——使用 `reduce-overhead` 可以通过 CUDA Graph 捕获几乎消除所有 kernel 启动的运行时开销，从而获得最佳收益。但请留意 `reduce-overhead` 的约束条件：它在每次迭代的工作负载一致、且满足图捕获要求时工作得最好，这些要求包括没有动态形状变化、没有新的内存分配等。

`max-autotune-no-cudagraphs` 模式则更像一个专门化的选项。当你想要最大化的 kernel 优化，却由于输入大小变化而无法使用 graphs，或者希望在不受图摊销影响的情况下测量原始 kernel 性能时，它会很有用。

无论哪种情况，在更改 PyTorch 编译器模式之后进行性能分析与测量都是明智之举。之所以存在不同的模式，是因为在性能工程中没有放之四海而皆准的方案。此外，在选择模式时你还应监控内存使用情况：使用 CUDA Graphs 的模式会分配大型静态缓冲区，从而增加内存占用。

对于内存极其受限的情况，你可能更倾向于使用这种不启用 graphs 的模式，以避免 CUDA Graphs 带来的额外内存开销。接下来，我们讨论如何检查编译器究竟在做什么，包括它是创建了一个图还是多个图，以及是否使用了 CUDA Graphs 等。

13.3.4 区域编译

对于具有大量相同块的模型（如 Transformer 与 MoE），可以使用区域编译来缩短冷启动执行时间。此外，它还有助于减少反复重编译带来的开销——而这一切都不以牺牲 kernel 融合的能力为代价。

具体而言，区域编译只编译一次重复的块（例如 Transformer 块），并在所有出现之处复用这段代码，从而缩短编译时间。

PyTorch 通过 `torch.compiler.nested_compile_region()` 支持区域编译（regional compilation）。该 API 将一个代码块标记为嵌套编译区域：此区域在首次执行时被编译，随后在后续运行中直接复用。

此外，区域编译保证了正确性。一旦编译器检测到新的输入条件（形状、`dtype`、设备、`stride`、全局变量），它就会透明地重新编译该区域。

区域编译适合推理引擎以及启动延迟至关重要的短任务，也适合图失效（graph invalidation）频繁发生的场景。区域编译所得代码的性能与完整编译所得代码的吞吐量相近。

13.3.5 编译器性能问题的性能分析与调试

使用 `torch.compile` 时，知道如何调试编译器未能优化模型某一部分的情况是很有用的——例如，某些操作没有被融合，而你怀疑是图中断（graph break）导致回退到了 eager 执行。PyTorch 提供了一些工具来检查这类情况。

> **注** 现代 PyTorch 版本借助形状守卫（shape guard）实现了对动态形状的部分支持，这些守卫可以消除一些不必要的图中断。然而，真正动态的工作负载仍可能需要回退到 eager 执行（或使用 `max-autotune-no-cudagraphs`），以确保正确性。

`torch._dynamo.explain(model)` 会打印一份报告，列出所有图中断的位置（例如模型中未被 TorchDynamo 捕获的部分）、发生图中断的原因，以及模型中哪些部分未被 TorchDynamo 捕获。它还会列出未被 TorchDynamo 捕获、因而必须在较慢的 Python eager 模式下执行的操作或依赖数据的控制流。

图中断的一个常见原因是模型中存在不支持的操作。Dynamo 的 `explain()` 输出会就如何获取更多细节、如何帮助诊断问题给出建议。善用这些提示可以精确定位导致中断的具体操作或控制流。

另一个有用的技巧是在运行脚本前设置环境变量 `TORCH_LOGS="+dynamo"` 或 `TORCH_LOGS="+dynamo,+inductor"`。前缀 `+` 会为 `torch.compile` 流水线中的 TorchDynamo、TorchInductor 等组件启用详细（DEBUG 级）日志。这些详细日志包含图中断、回退到 eager 模式以及编译各阶段的细节。如果模型在 `torch.compile` 下意外变慢，这些日志可以帮助确定执行在何时、何处退出了编译后的图。

如果模型具有编译期无法解析的真正动态的形状或动态控制流，你可能需要引导编译器。例如，可以把模型拆分为可编译的部分，而让真正动态的部分留在 Python 中运行。

要对 TorchInductor 生成的 kernel 进行性能分析和基准测试，可以设置环境变量 `TORCHINDUCTOR_UNIQUE_KERNEL_NAMES=1` 与 `TORCHINDUCTOR_BENCHMARK_KERNEL=1`。设置之后，Inductor 会为生成的 kernel 模块生成基准测试运行环境（harness）代码，其产生的日志有助于定位意外的图断裂与性能问题。

你还可以用 `torch._dynamo.mark_dynamic(tensor, dim)` 标记代码的一部分，告诉编译器预期会出现动态形状。这样可以消除因形状不匹配导致的不必要的图断裂。我们将在下一章深入剖析 PyTorch 编译器时更详细地介绍这些技术。

简而言之，当 `torch.compile` 未能带来预期的提速时，可以借助 `torch._dynamo.explain()` 和编译器日志，找出究竟是哪些操作或代码区域导致了回退。随后需要采取变通手段，例如替换某个操作、以别的形状组织张量、接受较低的动态性，或者干脆对模型的这一部分禁用编译。这样一来，模型的大部分仍能保留编译带来的性能收益，同时边缘情况也得到妥当处理。

13.4 PyTorch 优化的注意力机制

Transformer 模型有相当一部分时间消耗在注意力机制上。可以应用多种 PyTorch 注意力优化技术，确保它不会成为瓶颈。下面简要概述其中几种技术及其适用场景：

缩放点积注意力（SDPA）

PyTorch 的高层 API `torch.nn.functional.scaled_dot_product_attention`（简称 SDPA）会自动为给定硬件选用可用的最快注意力 kernel（例如 FlashAttention）。当模型的注意力模式和 dtype 受所选后端（Flash、内存高效或 math）支持时，用它即可省心地获得提速；如果不支持，它会回退到标准注意力实现。

FlexAttention

FlexAttention 是一种基于编译器的方法，用于支持注意力中的自定义稀疏模式。对于特定的稀疏注意力模式（如块稀疏或滑动窗口注意力），FlexAttention 通过为这些模式生成优化的 kernel，可获得显著提速，如图 13-3 所示。对于 `scaled_dot_product_attention` 不支持的特例，可使用 FlexAttention。

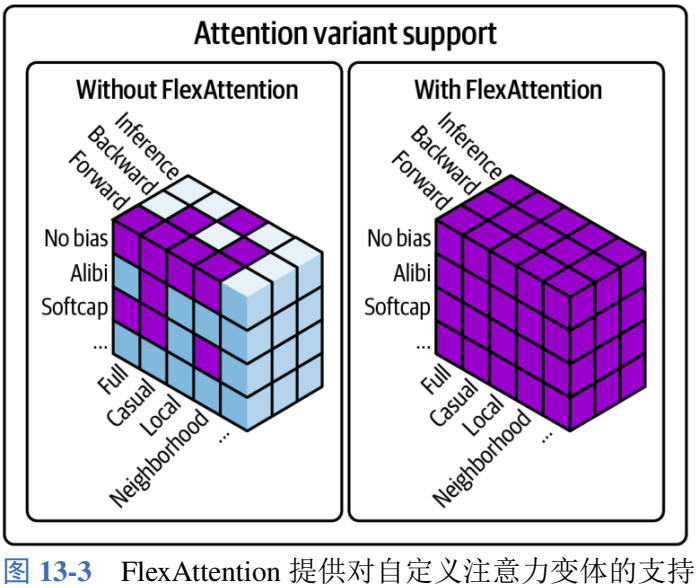

图 13-3　FlexAttention 提供对自定义注意力变体的支持

FlexDecoding 是 FlexAttention 的对应机制，用于优化解码或文本生成阶段。它与 `torch.compile` 及动态缓存布局集成，把编译期优化应用于序列生成的解码一侧，包括跨时间步高效地使用 KV 缓存。FlexDecoding 通过减少解码期间的冗余计算来加速自回归生成，面向 LLM 推理负载，包括需要生成长序列的场景；它不改变训练时的注意力语义。

上下文并行（Context Parallel）

上下文并行（Context Parallel）沿序列长度维度将注意力切分到各参与设备或 rank 上，以扩展上下文长度。使用 `context_parallel()` API 可以界定替换范围，将 `scaled_dot_product_attention` 换成支持上下文并行的 kernel。该机制按序列把 query-key-value（QKV）拆分到各个 rank，并在注意力计算期间进行同步，而不是在单个 GPU 内的线程之间并行化注意力。

13.5 PyTorch 架构优化（torchao）：量化、稀疏与剪枝

PyTorch 架构优化库（torchao）将量化、稀疏、剪枝及相关数值调试工具整合到统一的命名空间中。其量化子包（`torchao.quantization`）提供常用的 FX 图模式工作流，包括训练后量化（PTQ）、量化感知训练（QAT），以及 `QConfigMapping` 等 API，用于将模型转换并优化为 INT8、FP8 及新兴数值格式。

除量化之外，torchao 还支持剪枝（`torchao.pruning`）以及 2:4 稀疏、块稀疏等稀疏化技术（`torchao.sparsity`），能够在精度几乎无损的情况下带来显著提速。

`torch.compile()` 与 torchao 量化框架集成。在底层，TorchDynamo 将每个子模块的计算捕获为优化后的图，随后 TorchInductor 生成借助 torchao 的硬件感知 kernel。这为模型训练和推理都带来稳定一致的端到端性能提升，同时保留了对数值格式和内存布局的精确控制。这使 torchao 成为量化等生产级性能优化的优秀库。

13.6 使用 CUDA 流实现并发

正如前面某章所述，CUDA 流（stream）可以在 GPU 上实现操作的并发与重叠。默认情况下，PyTorch 按顺序把所有操作调度到设备的默认流 stream 0 上。然而，许多任务彼此独立，只要资源允许，GPU 就可以通过多条流并行执行它们。例如，利用独立的非默认流，GPU 可以让数据传输与计算重叠，或让不同的神经网络分支并发运行。

> **注** 要记住，现代 GPU 拥有多个 DMA 拷贝引擎。为 H2D 拷贝使用独立流，可以在不阻塞计算的情况下实现真正并行的数据传输。这一硬件支持使流并发更加有效。

在 PyTorch 中，先用 `torch.cuda.Stream()` 创建流，然后通过 Python 上下文管理器 `torch.cuda.stream(stream)`，或把操作显式指派给该流，即可在其上启动工作。PyTorch 会按先进先出（FIFO）的顺序把操作（如内存传输、CUDA kernel 等）提交到指定流——与在默认流上的做法一致。

13.6.1 通信与计算的重叠

CUDA 流的一个常见用法，是让主机到设备（H2D）的数据加载与 GPU 计算重叠起来。这有助于掩盖使用 GPU 这类外部设备（相对于主机上运行的 CPU）时的数据传输延迟。

例如，当默认流忙于对当前批次进行训练时，另一条流可以把下一批输入数据从 CPU 拷贝到 GPU 内存。等到默认流准备好处理下一批时，数据传输已经完成，GPU 可以直接处理这一批，从而有效地隐藏 I/O 延迟。下面的示例使用两条流——`compute_stream`（默认）与 `transfer_stream`（非默认）——在 PyTorch 中实现数据传输与计算的重叠：

```
# Set up streams
device = 'cuda'
# for H2D data transfers
transfer_stream = torch.cuda.Stream(device=device)
# for compute
compute_stream  = torch.cuda.default_stream(device=device)
# Create an iterator so we can preload "next" batches
dataloader_iter = iter(dataloader)
# Preload the very first batch onto GPU
first_batch = next(dataloader_iter, None)
if first_batch:
    with torch.cuda.stream(transfer_stream):
        next_inputs, next_labels = (
            first_batch[0].to(device, non_blocking=True),
            first_batch[1].to(device, non_blocking=True),
        )
for _ in range(len(dataloader)):
    # 1) Wait for transfer of `next` batch to finish, then swap into
     ↪  compute var
    # Multiple copy engines allow H2D/peer copy concurrency.
    # Verify parallelism in Nsight Systems (Copy engines lanes)
    # And tracing/profiling tools (HTA, etc.)
    compute_stream.wait_stream(transfer_stream)
    inputs, labels = next_inputs, next_labels
    # 2) Kick off transfer of the *following* batch on the
     ↪  transfer_stream
    batch = next(dataloader_iter, None)
    if batch:
        with torch.cuda.stream(transfer_stream):
            next_inputs, next_labels = (
                batch[0].to(device, non_blocking=True),
                batch[1].to(device, non_blocking=True),
            )
```

```
# 3) Run forward/backward on compute_stream
with torch.cuda.stream(compute_stream):
    outputs = model(inputs)
    loss    = loss_fn(outputs, labels)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
```

本示例使用了两条 CUDA 流：一条专用的传输流负责异步的主机到设备拷贝，默认的计算流负责模型计算。这样隐藏了 H2D 延迟，也符合 PyTorch 推荐的通信与计算重叠模式。

具体而言，当模型正在默认流上处理一个批次时，下一个批次的数据传输已经在 `transfer_stream` 上进行。在消费预加载的批次之前，先用 `compute_stream.wait_stream(transfer_stream)` 做同步，从而在不求助于全设备屏障的前提下保证正确的执行顺序。而 `.to(device, non_blocking=True)` 调用确保拷贝使用基于 DMA 的异步拷贝，不会阻塞调用方的 CPU 线程。

使用 `next(dataloader_iter, None)` 可以显式控制传输何时入队、kernel 何时执行，从而确保一个批次的数据正在传输流上移动的同时，另一个批次正在计算流上执行，如图 13-4 所示。

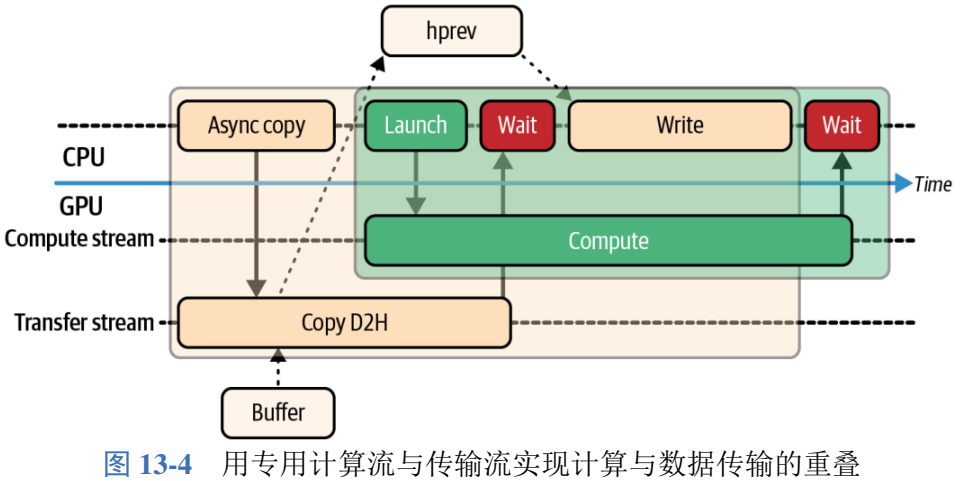

图 13-4　用专用计算流与传输流实现计算与数据传输的重叠

此外，代码提前从 `dataloader_iter` 取数据并存入 `next_inputs` 与 `next_labels`，从而把批次加载（在 CPU 上、经 `transfer_stream` 驱动）与批次处理（在 GPU 上、于 `compute_stream` 中运行）分离开来。

这种拆分意味着每条流始终各有一个批次在途，从而把数据加载与计算解耦，最大化二者的重叠。

> **注** 在增加流时，务必用 Nsight Systems 或 PyTorch profiler 进行性能分析。观察 GPU 利用率：如果操作正确，你会看到接近 100% 的利用率，且传输与计算相互重叠。如果利用率下降——或者看到数据传输与计算按顺序执行——请检查代码中是否存在隐式同步（如 CUDA 张量的 `print()`）或多余的 CUDA 同步操作。

13.6.2 用事件实现流同步

使用多个流时，有时需要在流之间进行协调，例如确保一个流的工作在另一个流使用其结果之前已经完成。对特定位置实现轻量级同步的方法是使用 CUDA 事件，如第 11 章所述。

借助 CUDA 事件，可以在一个流上记录事件，并让另一个流等待该事件。这样就避免了 `torch.cuda.synchronize()` 这种重量级的全局同步，只对处理单个事件所需的流进行同步。

事实上，即便在多 GPU 场景下，事件也可以用于跨设备同步工作：一个 GPU 在自己的设备流上记录事件，另一个 GPU 则在自己的设备流上等待该事件。NCCL 在底层正是这样处理依赖关系的。

巧妙地使用事件，可以让多个 GPU 尽可能保持并行工作。下面给出在 PyTorch 中用 CUDA 事件同步两个流（`stream1` 和 `stream2`）的示例：

```
# 由于该事件纯粹用于同步，因此不启用计时。
event = torch.cuda.Event(enable_timing=False)
# 第一个流中：
with torch.cuda.stream(stream1):
    kernel_launch(...)
    event.record()           # 在 stream1 的工作结束时记录事件
# 在另一个流或主机上：
stream2.wait_event(event)    # 让 stream2 等待，直到该事件被触发
with torch.cuda.stream(stream2):
    other_kernel_launch(...)
```

在这段代码中，我们在 stream1 上一段工作的末尾记录一个事件。随后，在启动 stream2 的工作之前，先调用 `stream2.wait_event(event)`。这会插入一个

依赖关系：在 stream1 执行到该点并触发事件之前，stream2 不会执行它的下一个 kernel。事件适合用来在流之间安排轻量级的依赖关系，因为它避免的是会阻塞所有流执行的重量级全局同步。

让我们回顾上一节的 PyTorch 数据加载/重叠示例，改用 CUDA 事件进行同步。仍然沿用前面的那对流（`transfer_stream` 与 `compute_stream`），但增加一个 `transfer_done` CUDA 事件，以便在细粒度、事件级的层面进行同步：

```
import torch
device = 'cuda'
# 用于 H2D 拷贝
transfer_stream = torch.cuda.Stream(device=device)
# 用于计算
compute_stream  = torch.cuda.default_stream(device=device)
# 仅用于同步的事件（开销低）
transfer_done = torch.cuda.Event(enable_timing=False)
# 迭代器，以便提前预取
dataloader_iter = iter(dataloader)
# ---- 预取第一个批数据 ----
first_batch = next(dataloader_iter, None)
if first_batch:
    with torch.cuda.stream(transfer_stream):
```

```
        next_inputs, next_labels = (
            first_batch[0].to(device, non_blocking=True),
            first_batch[1].to(device, non_blocking=True),
        )
        # 标记 H2D 完成的时刻
        transfer_done.record(stream=transfer_stream)
for _ in range(len(dataloader)):
    # ---- 同步：等待传输完成 ----
    compute_stream.wait_event(transfer_done)
    inputs, labels = next_inputs, next_labels
    # ---- 发起下一次传输 ----
    batch = next(dataloader_iter, None)
    if batch:
        with torch.cuda.stream(transfer_stream):
            next_inputs, next_labels = (
                batch[0].to(device, non_blocking=True),
                batch[1].to(device, non_blocking=True),
            )
            transfer_done.record(stream=transfer_stream)


    # ---- 在计算流上执行计算 ----
    with torch.cuda.stream(compute_stream):
        outputs = model(inputs)
        loss    = loss_fn(outputs, labels)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)
```

这与上一节的重叠模式相同，只是从传输到计算的同步改用 CUDA 事件实现，而不是另一例中使用的 `wait_stream()` 机制。

这段代码仍然使用 `next(dataloader_iter)` 提前预取批数据（与上一节的示例相同）。这样，数据传输与计算始终处于重叠状态。

不过，在本例中，`transfer_done` 事件是在异步拷贝入队之后，紧接着在 `transfer_stream` 上通过 `trans-`

`fer_done.record(stream=transfer_stream)` 记录的。这会给该事件打上时间戳。

随后，`compute_stream.wait_event(transfer_done)` 会让 `compute_stream` 停下来，直到拷贝完成、`transfer_done` 事件被触发。然后，它消费预取好的批数据，并在 `compute_stream` 上执行计算操作。

除了数据加载之外，CUDA 流在许多不同的场景中都很有用。下面来讨论它们在 MoE 模型中的用法。

13.6.3 在 MoE 模型中使用 CUDA 流

在实践中，transformer 模型的各层是顺序依赖的，因此不能随意并行地运行各层。但在 MoE 架构中，不同专家的计算相互独立，可以在不同的 CUDA 流上并发运行。

每个专家处理输入的一个不同片段。在汇合点，各专家的输出被聚合起来。关键在于，每个专家只能写入分配给它的输出张量切片。如果两个专家意外写入了重叠的内存区域，就会引入竞态条件（race condition），从而损坏结果，或者因专家写入顺序不确定而引发同步问题。

要避免这类问题，应确保使用恰当的流级同步手段（例如流事件），并核实内存在各个专家 kernel 之间被干净地划分。通过在专家执行与输出聚合之间强制保持这种分离，可以在不牺牲并行性的前提下保证正确性。

> **注** NVIDIA 的 Compute Sanitizer 能够检测 CUDA 代码中的并发与同步问题，包括竞态条件和死锁。此外，可以设置 `CUDA_LAUNCH_BLOCKING=1` 强制同步执行 kernel。这会让 kernel 执行按照确定性的顺序进行，从而暴露顺序与依赖方面的 bug，并揭示输出是否在尚未完全生成时就被消费了。

在我们的示例中，理论上每个专家都可以运行在自己的流上——如果框架扩展到多个 GPU，甚至可以运行在自己的 GPU 上。此时在汇总结果时需要同步——理想情况下使用流事件来完成。

流水线并行（即让微批次流经不同设备上的模型各阶段）与同时服务多个推理请求，是两个天然受益于多流的场景。例如，在流水线并行工作流中，模型的每个阶段都拥有自己的流，并发地处理不同的微批次，同时与相邻阶段通信。

在多请求推理服务中，每个请求的模型执行都可以在自己的流中启动。只要硬件资源充足，这就能通过重叠多个推理计算来提升吞吐量，其代价是资源共享开销会带来一些单请求延迟。

简而言之，CUDA 流通过在多个 kernel、阶段或请求之间重叠工作，帮助从硬件中榨取更多性能。它们需要仔细的同步来避免竞态条件。但如果使用得当，它们就能隐藏延迟，让 GPU 的利用更加充分。

建议在引入并发时持续对代码进行性能分析。同时要牢记，在 100% 利用率下的顺序执行，实际表现可能好于引入资源争用的并行执行。不过，流通常能让 GPU 中原本闲置的部分得到利用。找到恰当的平衡点十分重要。

> **注** 务必确保自己是在预期的流上启动工作。例如，不小心使用了默认流，就会重新引入不必要的串行化。这一点很容易弄错，所以值得再次强调。

13.7 用 CUDA 图降低 kernel 启动开销

前面的章节已经介绍过，CUDA 图消除了每次迭代的启动开销，降低了 CPU 侧的启动开销，并消除 kernel 之间细小的空闲间隙。而消除哪怕最小的空闲间隙，都会带来更高的有效利用率，以及更稳定的迭代时间。下面来看如何在 PyTorch 中使用 CUDA 图。

13.7.1 捕获 CUDA 图与预分配内存

PyTorch 提供了 `torch.cuda.CUDAGraph` API 来捕获和重放 CUDA 图。一般用法是：先正常跑几个迭代来预热模型，以初始化捕获过程所需的全部数据和内存分配。接着，创建一个 `CUDAGraph` 对象和一个专用的非默认 CUDA 流，将捕获过程隔离起来。

> **注** 使用“reduce-overhead”或“max-autotune”模式时，如果模型比较稳定，编译器会自动为你捕获 CUDA 图。此时甚至无需编写这些样板代码，因为使用 PyTorch 编译器的这些模式时，这一步是自动完成的。另外，如果模型每次迭代的形状都在变化，可以考虑使用“max-autotune-no-cudagraphs”模式来避免图捕获，因为 CUDA 图目前要求形状是静态的。

然后，在 `torch.cuda.graph()` 上下文指定的捕获流上执行一遍完整的模型运行，记录操作序列。一旦这些操作被捕获到 CUDA 图中，就可以在新的输入上按需重放（例如模型训练或推理）。

在捕获 CUDA 图之前，捕获期间会用到的所有静态内存都必须预先分配好——最好按最大尺寸分配。这些缓冲区包括输入、输出和中间张量。如果捕获期间发生了任何新的内存分配，捕获就会失败并报错，例如“operation not permitted when stream is capturing”（流处于捕获状态时不允许该操作）。

为了减少内存碎片，并为这些固定缓冲区争取最大的连续内存空间，你可以在进入捕获块之前立即调用 `torch.cuda.empty_cache()`。这会清除未使用的缓存内存，让分配器有机会不间断地排布你预留的缓冲区。

> **注** 频繁调用 `torch.cuda.empty_cache()` 会破坏分配器的效率，并带来长期性的性能损失。应将此调用视为捕获图时的一次性保险机制，而非常规的维护手段。

请记住，PyTorch 的缓存分配器支持 CUDA 的异步分配器（`cudaMallocAsync`），以复用固定的内存地址。然而，这并不能绕过 CUDA Graphs 的要求：图内不得创建新的内存分配。

你仍然需要预先分配好固定大小的缓冲区，否则一旦试图在图内分配新内存，就会触发运行时错误。务必让所有张量在图捕获前的预热阶段达到其所需的最大尺寸。我们将在后面的图重放一节中进一步讨论这一点。

进行图捕获时，你需要使用一条专用的非默认流（stream），以免与不应纳入 CUDA Graph 的其他操作相互干扰。下面的代码片段演示了如何在 PyTorch 中通过专用的 `capture_stream` 捕获 CUDA Graph：

```
g = torch.cuda.CUDAGraph()
capture_stream = torch.cuda.Stream()
# Prepare static inputs and outputs
static_input = torch.randn(batch_shape, device='cuda')
static_output = torch.empty(output_shape, device='cuda')
# Warm-up step on capture_stream to allocate buffers without recording
with torch.cuda.stream(capture_stream):
    tmp = model(static_input)
    static_output.copy_(tmp)
# ensure warm-up is complete
capture_stream.synchronize()
# Begin graph capture
with torch.cuda.graph(g, stream=capture_stream):
    tmp = model(static_input)
    static_output.copy_(tmp)
# ensure capture is complete before using the graph
capture_stream.synchronize()
```

在这段代码中，我们首先以固定形状在 GPU 上分配 `static_input` 和 `static_output`。随后在 `capture_stream` 上运行一次预热迭代，确保 `model(static_input)` 内部所需的内存都已分配到位，并完成所有一次性设置。

> **注** 通过预分配输出缓冲区，并在预热和捕获两个阶段都使用 `static_output.copy_(tmp)`，代码将结果写入固定的内存区域。这使得捕获到的 CUDA Graph 正确、可重放且可复现，不会出现意外的张量分配。

接着，我们同步 `capture_stream`，确保在实际开始图捕获之前，预热步骤已彻底完成。然后，我们在同一条流上进入 `torch.cuda.graph(...)` 上下文，并重新执行模型的前向传播。

在捕获阶段，并不会真正启动任何 kernel；这些操作只是被记录到 CUDA Graph 对象 `g` 中。退出捕获块之后，我们需要再同步一次，以确保记录操作已最终完成。

捕获 CUDA Graph 时，严格的隔离至关重要。捕获流上的操作绝不能受任何其他流上的活动影响。哪怕其他线程上看似无关的一次 kernel 启动，也可能

使捕获失效，进而导致诸如 "operation not permitted when stream is capturing" 之类的运行时错误。

这一错误通常发生在捕获进行期间：如果意外地在其他流上执行了 CUDA 操作，图上下文就会失效并触发该报错。如果在捕获期间在捕获流上启动会执行动态内存分配的 kernel——比如创建张量或调用 `torch.empty`——同样会出现这个错误。

> **注** 在开始图捕获之前和退出捕获之后，都应调用 `capture_stream.synchronize()`。这可以确保所有操作都被正确记录，图已能安全重放。前面的代码示例就遵循了这一最佳实践。

图捕获完成后，可以在任何流上重放它，包括默认流——与当初用于捕获的流无关。如果你随后触发的 CUDA 操作依赖图的完整执行，就必须像下例那样先进行同步，然后再运行这些操作。否则，由于图重放是异步执行的，依赖图结果的 CUDA 操作可能会在图尚未执行完毕时抢先运行：

```
# replay the graph which writes to static_output
g.replay()
# synchronize on the stream that is replaying the graph
torch.cuda.current_stream().synchronize()
# Now it's safe to read or post-process the output
print(static_output)
...
```

若缺少这一显式同步，程序可能继续往下执行，并错误地假定图已经执行完毕、结果已写入 `static_output`。在没有同步的情况下，代码可能读到过期或只写入了一部分的数据，因为图或许尚未完成对 `static_output` 的写入。这种情况会导致非确定性行为、读取损坏、隐蔽的竞态条件以及死锁。

13.7.2 重放图

要用新数据重放图，只需将新输入拷贝到预先分配的输入张量 `static_input` 中，然后调用 `g.replay()`。GPU 会以这些张量的当前内

容为输入，执行整段被捕获的操作序列，并把结果放入预先分配的输出张量（static_output），如下所示：

```
# load new data into pre-allocated input tensor
static_input.copy_(new_batch)
# execute the captured graph
g.replay()
# retrieve the output (clone if you plan to modify it)
result = static_output.clone()
```

这里，我们把 new_batch 数据载入 static_input 的内存空间，然后调用 g.replay()。图会以 static_input 中的当前数据执行完全相同的捕获操作，并把输出写入 static_output。随后即可按需使用或克隆 static_output。

建议用若干测试输入验证图的输出与普通执行结果一致，以确认捕获成功。另外请记住，在没有同步的情况下，不能直接打印 static_output 或对其调用 .item()。如有需要，可在重放之后、以及任何异步区域之外执行 result = static_output.cpu().numpy() 来进行调试。

由于图在每次运行时都会复用相同的输入、输出和内部内存分配，它在每次迭代中都会覆盖同一个输出张量。因此，如果需要在单次迭代之后仍然保留输出，就必须像前面代码那样克隆（clone()）该缓冲区。这也正是我们必须确保预热/捕获步骤中的内存分配覆盖所需最大尺寸的原因。切记，图无法临场处理额外的内存分配。

值得注意的是，使用 CUDA Graph 之后，某些 PyTorch 操作在你重新捕获图之前不会生效。例如，如果你在图外临时修改模型权重，这些更新不会生效，因为图持有自己的一份操作副本。因此，图最适合用于稳态循环，即每次迭代重复相同操作的场景。

此外，当使用 cudaMallocAsync 为 CUDA Graph 预分配内存时，重放期间会复用图捕获时（或者例如图从磁盘加载时的预热阶段）所预分配的同一批内存地址。这样一来，后续的图重放就不需要额外的内存。

默认情况下，每个 CUDA Graph 实例都使用自己的私有内存池。这样可以保证每个图都预分配自己的内存缓冲区，不会与其他图实例产生竞争。换言之，

两个相同的图即使并发重放，也不会争夺内存，因为它们各自使用自己的内存池和缓冲空间。

你也可以通过传入 `torch.cuda.graph(pool=...)`，在多个图实例之间共享内存池。不过，这仅在极少数特殊情况下有用，即你出于性能考虑，刻意让一组相关的图复用预分配的内存缓冲区。举例来说，设想同时运行多个推理图变体，每个对应不同的批大小（例如 1、2、4、8 等）。此时，可以让这些按形状特化的变体复用同一份较大的内存分配，供使用可变尺寸张量 `block_tables` 的 PagedAttention 使用，从而降低总体内存占用。

这一方法在 FireworksAI 的一篇博客文章中有所描述。他们为每个批大小编译一个不同的 CUDA Graph 变体，并且不为每个图单独创建内存池，而是在所有图变体之间共享同一个内存池。由于图是按批大小从大到小的顺序编译的，共享池中的内存从最大的变体（此处为 8）开始复用；较小批大小的图变体则由先前大变体迭代所留下的大块缓冲区来服务。这样，多个图变体无需占用过多的 GPU 内存即可同时得到支持。

> **注** 这是一个精巧而不失冷门的实现选择，需要对内存分段进行细致的协调。不过，它确实能够减少同时运行多个按形状特化的图时的 GPU 内存总占用量。对于以最小化整体内存占用为关键目标的部署场景而言，这一点非常有价值。

13.7.3 CUDA Graphs 的最佳实践

CUDA Graphs 是在训练和推理中达到峰值稳态吞吐量的有力手段。对于那些因 CPU 开销和 kernel 启动的不确定性而损失性能的大型部署，它们尤其有用。现代 GPU 执行数以千计的操作的速度极快，因此图已成为让 GPU 设备保持忙碌并最小化每次 kernel 启动开销的必要手段。

值得注意的是，除非被显式禁用，PyTorch 的 `torch.compile` 在许多情况下会在底层使用 CUDA Graphs。CUDA Graphs 被用于最小化编译模型的 kernel 启动开销。使用 CUDA Graphs 时，以下几点需要牢记：

避免在图捕获期间分配内存

请记住，图捕获过程中无法动态分配 GPU 内存。所需的任何张量都应像前文那样，在预热步骤中提前分配好。如果你的图需要临时缓冲区，请使用 PyTorch 图感知的缓存分配器，即采用 `cudaMallocAsync` CUDA 后端。这样，每次重放都会复用相同的缓冲区地址。务必在预热阶段（图捕获之前）以这些临时张量可能达到的最大尺寸创建它们，从而预先分配好所需的全部内存。

保持图结构固定

捕获的图无法改变操作的执行顺序、张量形状或内存大小。如果你的工作负载偶尔会出现形状变化，一种策略是捕获多张图（例如，为你预期的每种输入尺寸各捕获一张），然后在运行时选择合适的图。另一种做法是在这些迭代中禁用图（PyTorch 编译器为此提供了 `max-autotune-no-cudagraphs` 模式）。

> **注** CUDA 提供了底层的图管理 API（Graph Management API），其中包括前面某章介绍过的 `cudaGraphExecUpdate()`，它允许对已捕获的图做少量修改。然而，截至撰写本文时，PyTorch 并未暴露这一能力。就目前的 PyTorch 而言，最好把图视为不可变的。

尽可能多捕获一些

尽量把训练循环中更多的部分纳入图——理想情况下是完整的一次迭代（前向传播、反向传播、优化器步进，以及所有 all-reduce 通信）。捕获得越多，能消除的 CPU 开销和启动延迟就越多。

此外，在内存允许的情况下，可以考虑在一张图中捕获多个迭代（包括循环展开等）。这虽然会使图更大，但能在迭代之间启用更多优化，从而往往进一步提升吞吐量。代价是牺牲灵活性，但值得探索并通过性能分析加以验证。

在多 GPU 设置中捕获大图时，如果硬件支持，应当使用 CUDA 流优先级。例如，若你希望计算 kernel 以更高优先级运行而不被延迟，可以把 NCCL 调用放到较低优先级的流上。图捕获同样会记录这些优先级。

> **注** NVIDIA 提交的 MLPerf 成绩（以及其内部基准测试）通常会把整个训练步捕获为每次迭代一张图，其中包含前向传播、反向传播、优化器和 all-reduce 通信步骤。这几乎消除了所有运行时开销（如启动抖动等），代价是内存和灵活性。

规划内存复用

图捕获完成后，就不能再释放或重新分配该图使用的任何张量，也不能改变它们的尺寸。最好的做法是按预期的最大批大小进行捕获，以预留略多于实际所需的容量。这样，日后重放图时稍大一点的输入也不会使图中途失效。

例如，如果你的最大批大小是 64，但偶尔会以 96 运行，那么稳妥起见可以直接按批大小 96 捕获。通常，按最坏情况捕获、运行较小的批并浪费一点内存，比冒图失败的风险更好。

> **注** 别忘了把优化器状态的尺寸（例如 Adam 优化器的中间缓冲区）也计入其中，因为它们是模型训练图的一部分。这些很容易被忽略！

只要使用得当，CUDA Graphs 就能为大模型训练和推理工作负载带来显著的加速。完成计算优化之后，接下来我们转向 PyTorch 的内存优化。

13.7.4 CUDA Graph Trees（PyTorch 编译器内部机制）

我们在前面的小节中已经介绍过 PyTorch 编译器，但结合 CUDA Graphs 的语境，值得再提一下 PyTorch 的 CUDA Graph Trees。`torch.compile` 会使用它——具体是在 `mode="reduce-overhead"` 模式下——为每种输入形状编译并缓存独立的静态图。

静态图一旦被记录，其张量维度就必须保持固定。任何新的输入形状都会触发一次全新的记录，并新增一个缓存条目。为了最大化缓存命中并减少图捕获次数，建议在各迭代之间尽量保持输入形状一致。不同形状越少，已记录的图被复用的机会就越多，开销也就越低。

值得注意的是，你通常不需要直接调用 CUDA Graph Trees 的 API。在指定 `mode="reduce-overhead"` 时，`torch.compile` 会替你完成这一切。

由于 CUDA Graphs 要求静态地址和稳定的控制流，对于 LLM 推理来说全图捕获十分困难，因为 LLM 推理需要支持可变的输入长度、批大小和步数（例如采样、KV 缓存增长、主机侧决策等）。

对于模型训练，CUDA Graph Trees 允许多个已捕获的子图在正向与反向捕获之间共享同一个内存池。CUDA Graph Trees 对推理同样有用，因为它可以根据

输入形状和批大小动态选择子图。这种做法通常被称为“分段捕获”（piecewise capture）。PyTorch 利用 CUDA Graph Trees 来维护按形状划分的分段捕获并管理共享内存池。

> **注** CUDA Graph Trees 所提供的分段捕获模式，正是 vLLM 推理引擎用来为不同输入形状与批大小组合支持不同图的机制。我们从第 15 章开始更详细地介绍 vLLM 与推理引擎优化。

13.8 在 PyTorch 中进行内存性能分析与调优

大模型可能受 GPU 内存容量和内存带宽的限制。此外，内存碎片这类低效的内存使用方式，即便 HBM 容量充足，也会损害性能。你可以在多个方面应对内存问题，包括内存分配器调优、激活检查点、内存卸载（offloading）以及输入流水线优化。

此外，PyTorch 在 `torch.profiler` 中内置了内存分析器，只需启用 `profile_memory=True` 即可（如前文所示）。你可以用它找出哪些操作分配了大量内存，并优先处理这些操作。图 13-5 所示的可视化结果由 PyTorch 内存可视化工具生成。

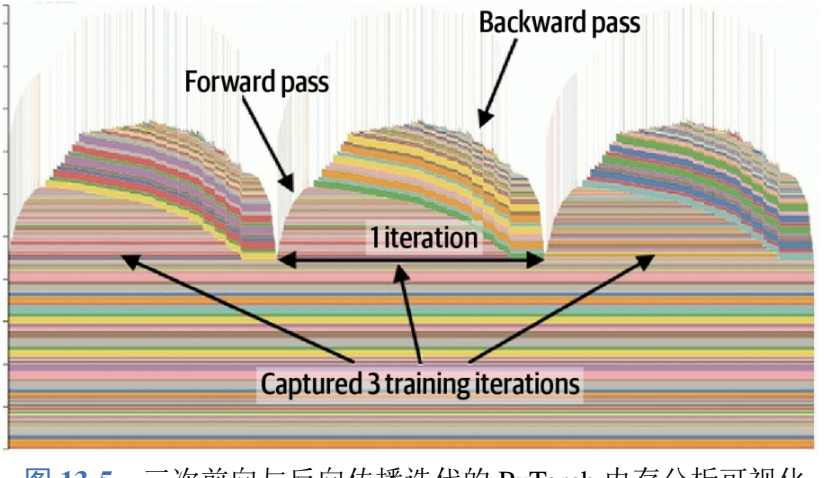

图 13-5　三次前向与反向传播迭代的 PyTorch 内存分析可视化

此外，NVIDIA Nsight Systems 的 CUDA Memory Inspector 可以帮助可视化内存碎片随时间推移的形成过程。利用这些工具可以指导你开展内存分配器调优，我们将在下一节探讨这一点。

13.8.1 调优 CUDA 内存分配器

PyTorch 为 CUDA 内存使用的是缓存式内存分配器。默认情况下，它通过按需拆分和回收 GPU 内存块来适应分配模式。然而，某些涉及可变大小内存分配的工作负载模式会导致内存碎片。

内存碎片是指 GPU 内存随时间推移被分割成许多不连续的空闲块。这使得即便空闲内存总量足够，也难以分配一个较大的张量。在 MoE 模型中，这个问题尤为突出，因为路由到每个专家的 token 数量会随每个批次而变化。因此，每个专家的输出激活张量在每次迭代中都可能有不同的尺寸。

可变大小的内存分配会留下参差不齐的碎片化内存块。这些碎片内存块会在训练或推理的多次运行中不断累积。

为避免这种情况，应当预先分配一个固定大小的专家输出缓冲区，并将其设置到批次中任一专家可能处理的最大 token 数。然后在每次迭代中复用该缓冲区。

只要缓冲区维度保持恒定，GPU 内存分配器就不会随时间推移产生内存碎片。每个专家写入预分配缓冲区中属于自己的切片，而不是触发新的分配。这种方法能稳定内存使用、提高复用效率，并避免与碎片相关的故障或性能下降。

你可以通过环境变量调整 PyTorch 分配器的配置，以调优其行为。示例如下：

```
export PYTORCH_ALLOC_CONF=\
max_split_size_mb:256,\
roundup_power2_divisions:[256:1,512:2,1024:4,>:8],\
backend:cudaMallocAsync
```

下面逐项说明代码中各配置参数的含义：

`max_split_size_mb:256`

该参数指示分配器保持大的空闲显存块完整（最大可达 256 MB），而不是不断将其切分成碎片，从而减少内存碎片。默认情况下，PyTorch 对大块分配的切分较为激进。显式设置 `max_split_size_mb`，可以让大的连续空闲显存块保持可用，以满足现代 LLM 中大型神经网络层的需要。

`roundup_power2_divisions:[N:M,...]`

该参数控制 PyTorch 的 CUDA 缓存分配器如何把张量大小的请求归入固定大小的桶。它把每个“2 的幂”区间划分为 N 个等大的子桶——例如某个请求落在 512 MB 与 1,024 MB 之间时。当代码中指定 `512:2` 时，512 到 1,024 MB 的区间会被划分成 2 个桶，并向上取整到 [512 MB, 768 MB, 1 GB] 之一。例如，在指定 `512:2` 时，512 到 1,024 MB 区间内的 600 MB 请求会向上取整为 768 MB。可以查看分配器日志，确认你的环境中实际的分桶情况。这一策略能减少内存碎片、统一分配大小并提升缓存复用率，因为相似的请求会命中同一个桶。

`backend:cudaMallocAsync`

指定该项将启用 NVIDIA 的 CUDA 异步分配器作为底层内存分配机制。这有助于避免内存释放事件上的同步——并在多工作进程数据加载这类多线程场景中提升性能。

通过定制内存分配器配置，你可以让显存使用模式更加平稳、更可预测。你可以在长时间运行中用 `torch.cuda.memory_stats()` 监控内存碎片，确保显存占用保持稳定，不会突然膨胀。

你也可以在运行时用 `torch.cuda.mem_get_info()` 获取空闲内存与总内存。这能间接跟踪碎片情况：如果已分配张量的数量不变而空闲内存却在下降，就说明碎片正在增加。

13.8.2 激活检查点节省显存

对于超大规模模型，激活检查点（activation checkpointing，一些从业者也称梯度检查点）是管理显存的关键手段。对于大型 LLM，如果要在反向传播时保存全部中间激活，显存往往不够用。

采用激活检查点后，前向传播时不再保存中间激活（供反向传播使用），而是在反向传播过程中按需即时重新计算。PyTorch 提供了 `torch.utils.checkpoint` 来自动完成这项工作。你只需把模型层——或若干层的序列——包装起来，它们的前向激活就不会被保存。

你可以按每个 transformer 块、每个专家前馈神经网络（FFN）层的粒度对模型应用检查点。这样，在计算出每个块的前向输出之后，就不必把这些中间激活留在内存中；而是在反向传播时，由 PyTorch 重跑该块的前向传播，重新生成用于梯度计算的激活。

值得注意的是，不必对所有组件都做检查点。只需聚焦最大的那些层。一种常见策略是只对承载海量激活的 transformer 块做检查点，而不对层归一化和嵌入层这类较小的层做检查点。这样能在最小的重算开销下获得最大的显存节省。

> **注** 使用 FSDP 时，你还可以启用自动检查点，它会递归地对多个层自动应用检查点。

这一交换以额外的计算换取更低的显存占用。幸运的是，相对于 HBM 显存容量而言，现代 GPU 提供了充裕的 FLOPS，因此该技术与最新一代硬件天然契合：GPU 仍有额外的算力余量，足以承担这些额外的重算。

这种权衡往往被证明是值得的。如果没有激活检查点，你就不得不缩小输入批次大小——或减少专家数量——才能塞进有限的 GPU 显存。而有了检查点，你就可以轻松地把模型放进显存，同时保住更大的输入批次大小。

虽然激活检查点会略微拖慢训练，但它让你能够训练更大的模型变体——并使用更大的输入批次大小——这些原本都无法完全装进显存。本质上，你是在用 GPU 充裕的计算 FLOPS 去克服其有限的 HBM 容量。

13.8.3 将参数卸载到 CPU 与 NVMe

除检查点之外，你还可以把模型中那些不必常驻 GPU 的参数卸载出去。例如，MoE 模型中有一些访问频率较低的专家层，可以把它们卸载到 CPU 内存，仅在需要时再传回 GPU。

重要的是让传输与计算重叠：当一层正在 GPU 上运行时，异步预取下一层来自 CPU 或 SSD 的权重。实践中，DeepSpeed 的 ZeRO-Infinity（用于训练）和 ZeRO-Inference（用于推理）等框架可以自动完成这种预取。它们逐层地把模型权重从 CPU 或 NVMe 流式送入 GPU，在让数据传输与计算重叠的同时，把 GPU 显存的峰值占用降到最低。

你可以把这些组件固定（pin）在主机端，并使用 `.to(device)` 和 `cudaMemcpyAsync` 这类异步、非阻塞的 DMA 调用，在其他计算运行期间把它们传到 GPU。这样能隐藏从 CPU 拷贝带来的额外传输延迟。

NVIDIA 的统一内存（Unified Memory）也是一种选择——尤其是对于 Grace Blackwell GB200/GB300 这类在 CPU 与 GPU 之间配备 NVLink-C2C 等高速互连的超级芯片系统。在这些场景下，统一内存允许把不常用的 GPU 内存页驱逐到 CPU 的系统内存中。

如果容量不够，操作系统甚至可能把它们换出到 NVMe/SSD。NVMe 应作为最后手段，经由普通操作系统换页来使用，而不应作为统一内存的主要落点。

不过，统一内存会因内存换页等因素引入不可预测性。因此，为了保持完全掌控和可预测的性能，显式管理卸载往往更为可取。

诸如 GPUDirect Storage 之类的最新进展让 GPU 能够直接从 NVMe 盘读取数据。这意味着在某些情况下，模型参数可以在运行中直接从 NVMe 换入，完全不需要 CPU 参与。无论训练还是推理服务，使用超大规模模型时这一点都很有用。

对于万亿参数量级的更大模型，你可以把组件卸载到 NVMe 存储，并在需要时即时换入 GPU 内存。关键在于让数据传输与计算重叠，避免它们阻塞训练循环。

13.8.4 SuperOffload：面向 CPU-GPU 超级芯片的优化卸载

SuperOffload 是一个专门为发挥 CPU-GPU 超级芯片硬件效率而设计的卸载系统。超级芯片（如 Grace Hopper、Grace Blackwell、Vera Rubin 等）在 CPU 与 GPU 之间提供高带宽的 NVLink-C2C 互连。结合共享且保持一致的内存地址空间，

面向超级芯片优化的卸载策略相比传统卸载技术能够带来巨大的加速和利用率提升。

SuperOffload 展示了若干关键创新，包括先推测后验证（speculation-then-validation，STV）、异构优化器计算、超级芯片感知的数据类型转换与内存拷贝。下面逐一讨论。

传统卸载会先等待梯度规约和全局检查，然后再更新参数；STV 则让这些步骤相互重叠：在 GPU 上运行反向传播的同时，在 CPU 上执行推测性的优化器更新，随后再对结果进行验证。这有效减少了同步停顿，提高了整体 GPU 利用率。

SuperOffload 利用异构优化器计算，把优化器的工作在 CPU 与 GPU 之间划分。例如，它把计算密集的张量更新交给 GPU，而由 CPU 核心处理较轻量的状态更新，例如 Adam 优化器使用的动量缓冲。这让两个设备都保持忙碌，减少空闲周期，从而提升整体芯片利用率。

由于 NVLink-C2C 在 CPU 与 GPU 之间提供了高带宽，SuperOffload 可以调整张量类型转换与数据传输的精度和放置策略。通过把类型转换和拷贝向 GPU 一侧迁移，SuperOffload 借助快速、保持一致的互连，把 CPU-GPU 传输延迟降到最低。

此外，SuperOffload 还使用一个针对 CPU 优化的 Adam 变体 GraceAdam，它是为 Grace 的 ARM 可伸缩向量扩展（SVE）架构设计的。SVE 是 ARM A64 指令集所采用的长向量架构，具体拥有 32 个向量寄存器和 16 个谓词寄存器；与 SuperOffload 结合后，这些特性有助于提升卸载参数更新的吞吐量和能效。

13.8.5 FSDP 自动检查点与卸载

PyTorch 的 FSDP 是一种分布式并行策略，它在模型训练期间将模型参数、梯度和激活值分片（sharding）到各 GPU 上。这降低了训练期间的内存开销——让你能够训练比不分片时更大的模型。严格来说，FSDP 实现了 ZeRO Stage-3 策略来将模型状态分片到各 GPU 上，如图 13-6 所示。

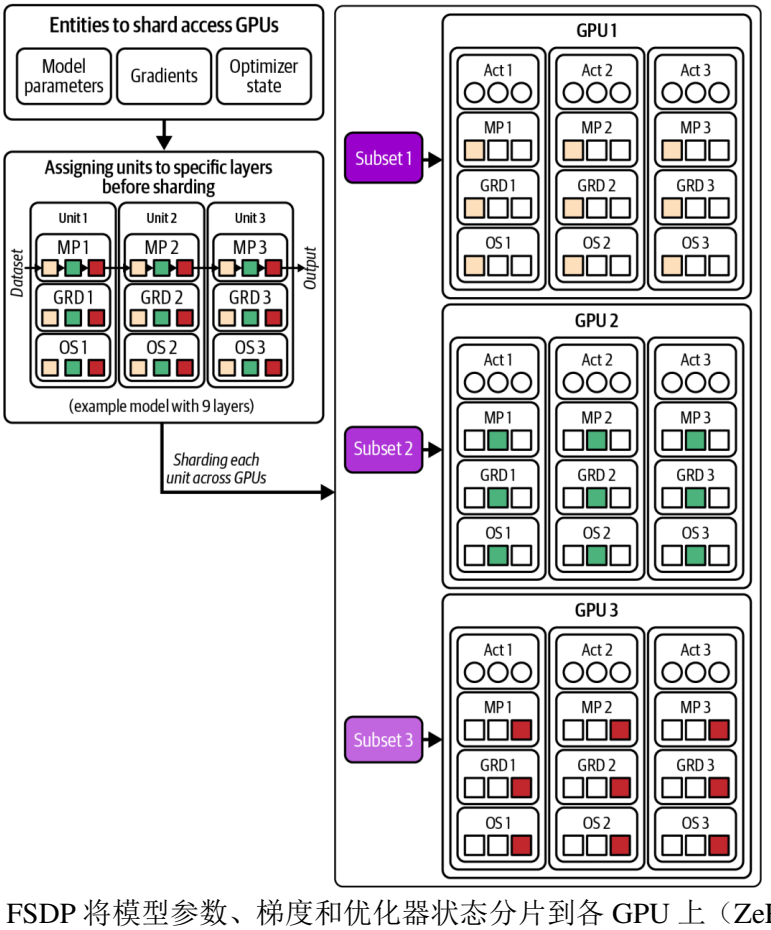

图 13-6　FSDP 将模型参数、梯度和优化器状态分片到各 GPU 上（ZeRO Stage-3）

FSDP 可以在底层自动应用激活检查点，并卸载参数/梯度。你只需用 FSDP() 包装模型，然后按下例所示指定 activation_checkpointing_policy 和 CPUOffload 参数：

```
import torch
import torch.nn as nn
import torch.distributed as dist
from torch.distributed.fsdp import (


    FullyShardedDataParallel as FSDP,
    CPUOffload, ShardingStrategy, BackwardPrefetch, MixedPrecision
)
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
# Initialize distributed
dist.init_process_group("nccl")
torch.cuda.set_device(dist.get_rank() % torch.cuda.device_count())
# Build your model
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(4096, 4096),
            nn.ReLU(),
            nn.Linear(4096, 4096),
        )
    def forward(self, x):
        return self.layers(x)
model = MyModel().cuda()
# Auto-wrap transformer blocks if needed
auto_wrap_policy = transformer_auto_wrap_policy(
    model,
    min_num_params=1e8,
)
# Wrap with FSDP + checkpointing + CPU offload
fsdp_model = FSDP(
    model,
    auto_wrap_policy=auto_wrap_policy,
    sharding_strategy=ShardingStrategy.FULL_SHARD,
    use_orig_params=True,
    cpu_offload=CPUOffload(offload_params=True, pin_memory=True),
```

```
    mixed_precision=MixedPrecision(
        param_dtype=torch.bfloat16,
        reduce_dtype=torch.bfloat16,
        buffer_dtype=torch.bfloat16
    ),
    backward_prefetch=BackwardPrefetch.BACKWARD_PRE,
    activation_checkpointing_policy={
        nn.TransformerEncoderLayer,
        nn.TransformerDecoderLayer,
        nn.MultiheadAttention
    }
)
# Setup optimizer
optimizer = torch.optim.AdamW(fsdp_model.parameters(),
 ↪  weight_decay=0.01)
```

... 这里我们使用 `transformer_auto_wrap_policy`，使 FSDP 会依据你的 transformer 块

结构自动对参数、梯度和优化器状态进行分片。我们还通过 `CPUOffload(offload_params=True)` 启用向 CPU 的卸载。当参数和梯度暂时不需要驻留 GPU 时，这一机制会透明地将它们转移到 CPU，从而降低 GPU 的峰值内存占用。

> **注** 设置 `use_orig_params=True` 后，每个 FSDP 单元在处理自身参数时不做展平（flatten）。这样可以获得更好的重叠执行效果和更简单的状态字典处理——从而改善内存管理并提升优化器兼容性。

设置 `activation_checkpointing_policy` 会告诉 FSDP 在这些特定的核心 transformer 子模块中重新计算（重物化，rematerialize）激活值。这是以额外计算换取峰值内存的显著降低。它能取得与手动 `checkpoint()` 包装器和自定义卸载脚本相同的大幅内存节省，却无需额外样板代码。FSDP 甚至能处理各 GPU 批大小不均的情况，这对混合专家（MoE）负载很有用。

FSDP 还支持使用 `ShardingStrategy.HYBRID_SHARD` 的混合分片策略。该策略将每个节点的参数、梯度和优化器状态在该节点内的各 GPU 之间分片，同

时将这些分片复制到其他节点上。从本质上讲，混合分片比 FULL_SHARD 提供更高的吞吐量，但代价是每节点占用更多内存。

当节点间互连尚可但算不上极快时，可以选用 HYBRID_SHARD。它在每个节点内部对参数、梯度和优化器状态进行分片——并将这些分片跨节点复制。这以略高于完全分片的每节点内存为代价，减少了跨节点通信流量。

混合分片让你能够管理内存与通信之间的权衡。例如，你可以使用比 FULL_SHARD（ZeRO 3）更多的每节点内存，因为每个节点上都持有一份完整本地分片。这减少了节点间通信，通常还能提供更高的端到端吞吐量。FULL_SHARD 最适合拥有极快的多节点互联网络、且希望最小化每 GPU 内存占用的场景。

当网络较慢——或你只有少数几块 GPU——时，可以使用 ShardingStrategy.SHARD_GRAD_OP（ZeRO Stage-2）只将梯度和优化器状态分片到所有 GPU 上。这一策略会在每个 GPU 上保留参数的完整副本。

13.8.6 将 FSDP 与张量并行和流水线并行结合

如果模型大到单个层的参数都无法放进单块 GPU 的显存，你就需要将 FSDP 与张量并行（TP）等其他并行策略结合，把过大的模型层摊到多个 GPU 上。

你还可以让 FSDP 同时作用于多块 GPU 与多个计算节点：在一个节点内部使用张量并行（TP）将巨大的层拆分到该节点的多块 GPU 上，再用流水线并行（PP）将这些经 TP 拆分的层跨节点串联起来。FSDP 支持这类灵活的组合。

简而言之，FSDP 能以远低的编码工作量降低内存占用。应用检查点与卸载之后，你可以把模型放进显存并启用更大的批大小，这有助于提高 GPU 利用率。

13.8.7 可插拔内存分配器与跨 GPU 数据传输

你可以配置 PyTorch，为 NCCL 梯度 all-reduce 这类关键的 GPU 通信操作使用专用内存分配器。通过 torch.cuda.MemPool 接入自定义分配器，NCCL 便可以借助专用硬件引擎（如 NVLink 复制引擎、InfiniBand 卸载或 NVIDIA SHARP）来分配缓冲区，这有助于改善通信与计算之间的重叠。例如，PyTorch 支持在 NVSwitch 配置中使用 NCCL 的内存分配器来实现高速零拷贝归约，如下所示：

```
import torch
import torch.distributed as dist
from torch.cuda.memory import MemPool
from torch.distributed.distributed_c10d import _get_default_group
# Initialize NCCL distributed backend
dist.init_process_group(backend="nccl")
torch.cuda.set_device(
  dist.get_rank() % torch.cuda.device_count()
)
# Get the NCCL backend object for this device
default_pg = _get_default_group()
backend = default_pg._get_backend(
  torch.device(f"cuda:{torch.cuda.current_device()}")
)
# The backend exposes ncclMemAlloc using mem_allocator
nccl_allocator = backend.mem_allocator
# Create a dedicated memory pool using this allocator
nccl_pool = MemPool(nccl_allocator)
# Register the pool so NCCL uses it for collective gradient buffers
backend.register_mem_pool(nccl_pool)
# Use the pool explicitly for NCCL operations
with torch.cuda.use_mem_pool(nccl_pool):
    tensor = torch.randn(10_000_000, device="cuda")
    dist.all_reduce(tensor)
```

通过将 NCCL 的原生分配器（`backend.mem_allocator`）绑定进 `torch.cuda.memory.MemPool`，并调用 `backend.register_mem_pool(...)`，PyTorch 会把梯度归约缓冲区直接放置在针对 NVLink、InfiniBand 及支持 NVIDIA SHARP 的硬件路由而优化的内存区域中。这样一来，all-reduce 操作便能借助硬件加速，在大规模数据归约期间减少 SM 争用，从而提升大型多 GPU 工作负载的吞吐量。

> **注** 正如第 4 章所述，使用 SHARP 需要兼容的网络结构（例如支持 SHARP 的 HDR InfiniBand）。如果具备条件，启用 NCCL 基于 SHARP 的网络内聚合可以大幅降低大型集群的 all-reduce 延迟。在扩展到大量节点时，这一点绝对值得考虑。

通过结合使用 NCCL 的原生分配器 `backend.mem_allocator`，并用 PyTorch 的 `MemPool` 加以封装，梯度 all-reduce 缓冲区会分配在针对 GPUDirect RDMA 与网络内 SHARP 卸载而优化的内存区域中。这会把缓冲区对齐到大页边界，并有助于把数据传输交给专用的 DMA 引擎——从而减少 SM 的参与，把计算能力释放出来去做更有价值的计算工作。

这样一来，像张量并行归约这类 NCCL 集合通信操作就能同时受益于硬件加速和更低的 SM 争用。这会显著提升多 GPU 同步的吞吐量，进而减少重叠的计算 kernel 与通信 kernel 在 SM 和复制引擎上的争用——对张量并行工作负载尤其如此。

当你逼近带宽极限时，这类优化就变得更加重要。例如，Blackwell 的 NVLink 5 理论上可提供每块 GPU 高达 1.8 TB/s 的双向带宽（每个方向约 900 GB/s）。使用专用的复制引擎、网络硬件（SHARP）和优化的内存分配器，有助于逼近网络吞吐峰值，同时把 SM 释放出来以达到峰值 FLOPS。

借助 `torch.cuda.MemPool`，你还可以通过注册自定义库（共享对象，即 `.so` 文件）来创建自己的内存分配器。此外，你可以在同一个 PyTorch 应用中混用不同的 CUDA 内存分配器，如下所示：

```
import torch  # PyTorch main namespace
import os     # for path operations (used here for .so extensions)
from torch.cuda.memory import CUDAPluggableAllocator
# 1. Create a CUDAPluggableAllocator and MemPool
# Build a pluggable allocator that calls into your NCCL library:
#   - allocator_path: path to your .so
```

```
#   - "ncclMemAlloc": symbol for allocation
#   - "ncclMemFree":  symbol for deallocation
# .allocator() returns a callable that matches the CUB/CUDA allocator
 ↪ API
allocator = CUDAPluggableAllocator(
    "./nccl_allocator.so",
    "ncclMemAlloc",
    "ncclMemFree"
).allocator()
# Wrap that allocator in a MemPool for efficient sub-allocations
pool = torch.cuda.memory.MemPool(allocator)
# 2. Start recording events (set a high cap for long runs)
torch.cuda.memory._record_memory_history(max_entries=100000)
# 3. Allocate tensors with different allocators
# tensor0 uses the *default* cudaMalloc allocator
# - Shape: (1024, 1024), you can change to your desired size
tensor0 = torch.randn(1024, 1024, device="cuda")
# tensor1 uses *your* NCCL-backed allocator using MemPool
with torch.cuda.use_mem_pool(pool):
    # Inside this context, all cuda allocations go through `pool`
    tensor1 = torch.randn(1024, 1024, device="cuda")
# Exiting the context restores the default allocator
# tensor2 again uses the *default* cudaMalloc allocator
tensor2 = torch.randn(1024, 1024, device="cuda")
# 4. Inspect memory pool stats
# Pool-specific snapshot with list of segments/blocks in use
pool_state = pool.snapshot()
print(f"Pool segments count: {len(pool_state)}")
# 5. Dump the snapshot and optionally load in the PyTorch
# memory viewer tool (https://oreil.ly/tX6gA)
torch.cuda.memory._dump_snapshot('memory_snapshot.pkl')
# Global allocator stats (allocated/reserved, peak, counts)
global_stats = torch.cuda.memory_stats()
print("Peak allocated bytes:",
 ↪ global_stats["allocated_bytes.all.peak"])
# 6. Stop recording
```

```
torch.cuda.memory._record_memory_history(enabled=None)
# 7. Reset peak counters for a fresh measurement
torch.cuda.reset_peak_memory_stats()
```

在这里，`CUDAPluggableAllocator` 会加载你的自定义 `.so` 文件，并绑定到你所指定的两个符号。`MemPool` 会把底层 CUDA 内存分配器封装在一个缓存中，用于内存分配与内存释放，以获得更好的性能。

`torch.cuda.use_mem_pool(pool)` 会在 Python 上下文管理器（如 `with`）代码块内，把后续的所有内存分配切换到你的池。退出上下文管理器代码块后，将恢复原先的分配器。

13.8.8 启用点对点 DMA 与 UCX

在多 GPU 系统中使用流水线并行时，你通常希望用尽可能快的点对点（P2P）连接，把激活值直接从一个 GPU 传到下一个 GPU，并避免往返传输和占用宿主 CPU 资源。当张量在同一进程内跨设备移动时，PyTorch 会自动探测并启用点对点访问。不过，如果你想手动确认，可以用 `torch.cuda.can_device_access_peer(i, j)` 来确认 GPU i 与 GPU j 之间的 P2P DMA。若要在自定义 C++ 算子中显式启用对等访问，则可以使用 CUDA 驱动 API。

启用 P2P DMA 后，即可在不涉及 GPU 的 SM 或 CPU 内存的情况下完成高效的直接传输。一旦启用，跨 GPU 的 `copy_()` 和 `.to()` 就会走更快的对等内存路径，而无需额外的代码开销，如下所示：

```
dst.copy_(src, non_blocking=True)
# or
src.to(device="cuda:1", non_blocking=True)
```

当你的拓扑上启用了 P2P 访问时，这些方法会使用 `cudaMemcpyPeerAsync`。要让 P2P 传输正常工作，你的硬件拓扑必须支持它。GB200/GB300 NVL72 机架内部使用支持 P2P 的 NVLink 与 NVSwitch，因此开箱即已为 P2P DMA 做好准备。

到目前为止，在 P2P 数据传输的语境下我们只讨论了多 GPU 配置。然而，在多节点 GPU 拓扑中，你需要退回到经网络通信的 NCCL。有时这会经由 GPUDirect RDMA 完成，而配合 UCX（统一通信层，Unified Communication X）使用 NCCL 可以提升性能。在 GB200 与 GB300 NVL72 拓扑上，NVLink 与 NVSwitch 提供机架内部的 P2P 路径；跨机架则需要经由启用 UCX 的 NCCL 传输层穿越网络结构。

要让跨节点传输经由 UCX 路由，需安装 NCCL-UCX 插件并设置 `NCCL_PLUGIN_P2P=ucx`。许多环境还要求设置 `NCCL_NET=UCX`，并针对你的网络结构配置合适的 `UCX_TLS` 传输选项。这样可启用硬件卸载，并改善 NVL72 机架之间 InfiniBand 结构上的流水线化。

以下是 NCCL-UCX 插件的一个示例配置：

```
export NCCL_NET=UCX
export NCCL_PLUGIN_P2P=ucx
# UCX transports vary by fabric.
# These are safe defaults to start with:
export UCX_TLS=rc,self,gdr_copy,cuda_copy
```

如果你的应用需要扩展到多个节点或机架——训练 TB 级模型、跨大量租户对超大规模 LLM 做推理，或管理数据密集型流水线——那么 UCX 至关重要。将 NCCL 与 UCX 结合使用，可以提供支持硬件卸载（RDMA）与智能拓扑感知的高吞吐、低延迟跨节点通信。UCX 是生产级 AI 基础设施的核心组成部分，在扩展到单个 NVLink 域之外时必不可少。

简而言之，与 NCCL 中等价的 send/recv 调用相比，直接的 P2P DMA 通信和 UCX 能更好地与计算重叠。这是一种相对底层的优化，但如果你的拓扑具备 NVLink/NVSwitch 等专用高速互连，它可以大幅提升系统性能。

13.9 PyTorch 对称内存

对称内存是一种编程模型，它在多个 GPU 之间暴露一个分区的全局地址空间，使 kernel 能够执行单边 put 和 get 操作。这使得它们无需 CPU 握手或干预，

即可调用超低延迟的跨 GPU 直接访问集合通信。本质上，对称内存分配的缓冲区可由组内任意 GPU 直接寻址，而不需要显式的点对点拷贝。

使用对称内存时，你可以把类似 MoE token 洗牌这样的 all-to-all 操作完全放在设备上执行。由于不涉及 CPU，整个 all-to-all 都能被 CUDA Graph 捕获。如果没有对称内存，all-to-all 会触发一次主机同步，在时间线上造成设备到主机（D2H）的空隙，进而在 CUDA Graph 中产生不希望出现的中断。

你可以分配对称张量，跨进程组执行一次 rendezvous（汇合）以获得远程访问句柄，然后调用直接访问集合通信（例如 `all_to_all_v`、`one_shot_all_reduce`）。此外，你还可以启动 kernel，利用远程访问句柄执行单边读写。结合 OpenAI Triton 与 NVSHMEM（`triton.nvshmem.put()/get()`），对称内存是一种强大的通信机制，可用于自定义的 kernel 内数据传输。

> **注** 在 PyTorch 中，对于 MoE all-to-all token 洗牌这类细粒度、延迟敏感且无需宿主 CPU 干预的设备内交换，应当使用对称内存。这会消除设备到主机（D2H）的时间线空隙，并实现更好的 CUDA Graph 捕获。

13.10 优化数据输入流水线

讲完模型的内存与计算之后，我们来谈谈数据输入流水线。效率低下的一个常见原因是 GPU 闲置等待数据。PyTorch 的 DataLoader 支持派生多个 worker 线程或进程，在训练模型的同时并行加载并预处理数据（例如文本分词）。请务必指定 `pin_memory=True`，使宿主 CPU 内存到 GPU 设备的传输使用页锁定（pinned）内存。

如果不固定内存，你可能会看到数据加载进程的 CPU 利用率很高，而训练进程的 GPU 利用率较低。这是双重不利。这类不理想的利用率可以用 htop 和 Nsight Systems 等工具观察到：CPU 线程忙于加载数据，而 GPU 闲置。

这说明每次迭代的数据加载都跟不上。解决办法是增大 DataLoader 的 `num_workers` 参数值，直到数据队列中有足够的批次就绪。一个常见经验值是：对 CPU 受限的数据流水线，每块 GPU 配 4 个 worker，但最优数量会因工作负载而异。

要找到最优数量，可以做一个快速扫描（例如 4、8、16、32），找到再增加 worker 也不再有帮助的临界点。留意 CPU 的饱和情况：如果所有核心都已达到 100%，再加 worker 也无济于事。

请记住，Grace Hopper、Grace Blackwell 和 Vera Rubin 这类 NVLink C2C 超级芯片系统提供一个大容量、一致的高带宽 CPU–GPU 内存地址空间。在这些环境中，页锁定的 `pin_memory=True` 往往不那么关键，因为数据传输本来就走高带宽的一致性路径。

在非一致性路径上，为了最大化可分页宿主到设备拷贝的重叠，可能仍需固定内存。虽然统一内存和一致性映射可以降低 NVLink-C2C 系统上对固定内存的需求，但重要的是要实际测量你的工作负载性能。

即使在 CPU – GPU 超级芯片架构上，当加载线程受 CPU 限制时，仍建议将 `pin_memory=True`（或大页）与 `non_blocking=True`、`persistent_workers=True` 结合使用。使用 `persistent_workers=True` 可以避免跨轮（epoch）重新生成进程，这对 LLM 训练和推理这类分词繁重的工作负载很有帮助。使用 Nsight Systems 进行性能分析并加以验证。

实际的（actual）重叠。在取消锁定（pinning）之前，请务必在主机上启用大页，并确认 H2D 传输能够与计算重叠执行。

CPU 和 GPU 共享一个保持一致性的内存空间，因此数据传输无需显式锁定内存即可走高速路径。相反，在没有这种硬件一致性的标准（非单芯片）CPU + GPU 系统上，启用 `pin_memory=True` 会对主机内存进行页锁定（page-lock），从而显著提升数据传输吞吐量。

> **注** 如果你不锁定内存，而是依赖 Grace Blackwell 和 Vera Rubin 等单芯片（superchip）系统统一的 CPU-GPU 内存，请务必启用大页，并用 Nsight Systems 验证传输确实与计算重叠了。这是因为 NVLink-C2C 单芯片系统上统一的 CPU-GPU 内存改变了内存锁定的权衡取舍。

提高数据加载响应能力的另一种技术是增大 DataLoader 的 `prefetch_factor`。它控制每个 worker 预先加载多少个批次，总量按 `prefetch_factor * num_workers` 计算。这意味着，当模型忙于处理当前批次时，各个 worker 已经在加载并缓存后续批次了。

预取能让数据管线始终保持充盈，避免 GPU 空转。你应该根据数据集和硬件能力，将 `prefetch_factor` 与 `num_workers` 一起调整。通过性能分析来验证设置，避免过度预取、造成不必要的内存占用。

你还可以预先对数据集做分词（tokenization），把分词后的数据集存到磁盘上，以避免在数据加载循环中进行繁重的处理。Hugging Face 的 Dataset.cache() 和 WebDataset 等工具允许你一次性预处理文件并反复复用。此外，考虑对数据集采用混合精度和压缩，以降低 I/O 带宽需求。

还有一个选择是 PyTorch 原生的 TorchData 及其 DataPipes。它提供了良好的可组合性，并能与 PyTorch 调度器集成，从而与训练计算重叠执行。

另一个有用的工具是 NVIDIA 数据加载库（DALI）。DALI 可以与训练并行地执行 CPU 或 GPU 前处理。它对图像/视频数据（如解码和数据增强）尤其有用，并且可以通过 CUDA 管线直接把数据送入你的训练代码。对于受益于卸载到 GPU 上的即时数据变换，它是一个很实用的工具。

所有这些做法的目标都是让数据加载与 GPU 计算尽可能多地重叠。通过 Nsight Systems 进行性能分析，你可以确认数据加载器确实在与 GPU 并行工作。Nsight Systems 的时间线应该显示：一个 CPU 线程持续加载/预处理下一批次，而多个 GPU 流同时执行训练步。此外，还要确认没有 GPU 等待数据的空档。这样才能保证你的 GPU SM 接近 100% 的时间都在忙于做有用的训练工作。

在已用激活检查点和卸载解决了内存约束的前提下，你可以增大每个 GPU 的输入批次大小。使用更大的批次可以提升算术强度、提高 GPU 利用率，并减少多 GPU 训练中通信的相对开销，因为每个 epoch 的步数更少了。

然而，批次过大会把优化器推向尖锐极小值（sharp minima），损害泛化能力，从而影响收敛。在大幅增大批次大小时，务必监控 GPU 内存占用。请记住，梯度累积会增大有效批次大小（`batch_size * accumulation_steps`）。PyTorch 的 TorchEval 指标可以帮助你判断更大的批次是否在损害验证损失。

你可以通过相应地调整超参数，把大批次带来的不稳定性降到最低。例如，可以调整学习率、采用带预热期的线性学习率缩放，或使用 LAMB 这类大批次优化器。

简而言之，如果内存允许——并且前面各节的内存优化方案已经落实并验证过——增大批次大小可以提升算术强度，让 GPU 得到更充分的利用。

> **注** 在优化系统的过程中，你应当定期重新审视并重新调优批次大小、学习率等超参数。在改变批次大小等因素之后，最初选定的值可能不再是最优的。

13.11 用 PyTorch Distributed 进行扩展

在多 GPU 和多计算节点上扩展并分析 PyTorch，通常要使用 PyTorch 的分布式库，如 PyTorch DDP 和 FSDP。好消息是，PyTorch 编译器可以与这些并行化方案协同工作，但其中有一些细微之处，下文将逐一说明。

13.11.1 DDP 与 torch.compile

使用 DistributedDataParallel（通过 all-reduce 集合通信在 GPU 之间同步梯度）时，PyTorch 会自动在同步点创建图断裂（graph break）。在实践中，DDP 把梯度划分到若干桶（bucket）中，并让通信与计算重叠。PyTorch 的设计是把每个桶的反向计算编译成独立的图，这样在各图之间就可以执行 all-reduce。

在 `torch._dynamo.explain(model,...)` 的输出中，你会看到与 all-reduce 和 `torch.distributed` 操作相关的图断裂。这是预期行为，因为每个桶的计算都被单独编译，而 all-reduce 发生在子图之间。这样一来，通信重叠得以保留，而这对性能至关重要。

如果你把 DDP 与编译器一起使用，请确保 DDP 桶的大小合理。默认情况下，DDP 桶为 25 MB。如果你选择覆盖该默认值、为所有梯度使用一个巨大的桶，那么就会生成一个巨大的图，并在最后执行一次大的 all-reduce。这会导致图段更少、更大，通信与计算交错进行的机会也随之减少。

例如，为了让整个反向传播在单个图中完成以获得最大限度的 kernel 融合，人们很容易选择单个桶。然而，你会因此失去重叠的机会。建议对系统进行性能分析，为自己的工作负载和硬件环境找到合适的平衡点。

> **注** 在 DDP 中使用 `torch.compile` 时，你会在通信点看到有意设置的图断裂。这些断裂是正常且必需的，它们让网络通信得以发生。TorchDynamo

的 `explain()` 输出会显示 all-reduce 和 scatter 导致断裂的消息——这是预期行为。

13.11.2 FSDP 与 torch.compile

PyTorch 的 FSDP 并行策略将模型参数、梯度和优化器状态分片（shard）到各 GPU 上。它在正向传播期间执行 all-gather 集合通信，在反向传播期间执行 reduce-scatter。与使用 DDP 相比，这使得更大的模型能够装进 GPU 内存。

配合 `torch.compile`，FSDP 可以更高效，但需要仔细进行包装（wrapping），以最大化计算-通信重叠。建议用 `use_orig_params=True` 将每个 transformer 块——而不仅仅是某个底层单层——包装成独立的 FSDP 模块。这样，TorchDynamo 才会在需要通信的每个分片边界插入图断裂。

于是，每个块的正向和反向计算作为单个编译后的图执行，而 all-gather（正向）与 reduce-scatter（反向）通信发生在各图之间。这与 DDP 的分桶方法一致：后者将每个通信块单独编译。这使得计算与通信步骤之间能够获得恰当的重叠。

与此相反的做法是把整个模型包装进单个 FSDP 实例——这种做法很有诱惑力。如果这样做，TorchDynamo 会生成更少、更大的图。其结果是通信点更少、通信-计算重叠机会更少，且正向和反向两个阶段内部图与图之间的融合也十分有限。

以 transformer 块为粒度用 FSDP 包装模型，可以让训练管线获得最大限度的重叠。这是因为每个块的计算密集型逻辑都被编译并融合，而这些计算又与把各块连接起来所需的通信相互重叠。

PyTorch 在 torch.distributed.fsdp.wrap 中提供了 transformer_auto_wrap_policy 回调函数，让你无需手动嵌套即可轻松对模型中的每个 TransformerBlock 应用 FSDP。采用块级分片后，内存中一次只会具体化（materialize）一个块的完整权重。all-gather 和 reduce-scatter 集合通信被交错编排，并隐藏在每个块的计算之后。下面是一个使用 PyTorch 的端到端示例，它定义了自动包装策略，包装了一个示例 transformer 块，并运行了一个正向与反向步的示例：

```
import torch
import torch.nn as nn
import torch.distributed as dist
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP,
 ↪  CPUOffload, ShardingStrategy, BackwardPrefetch, MixedPrecision
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
from torch.distributed.algorithms._checkpoint.checkpoint_wrapper import
 ↪  apply_activation_checkpointing, checkpoint_wrapper
# Prefer BF16 for Blackwell/Hopper-class GPUs; allow TF32 matmul for
 ↪  speed where numerically safe.
# Enables TF32 use for FP32 matmuls when allowed.
torch.set_float32_matmul_precision('high')  #
 ↪  {'highest'|'high'|'medium'}
# Compile the model with a mode that reduces Python/launch overhead and
 ↪  enables CUDA Graphs.
compiled_model = torch.compile(model, mode="reduce-overhead")
# Auto-wrap transformer blocks; adjust set below to match your model's
 ↪  block classes.
auto_wrap_policy = functools.partial(
    transformer_auto_wrap_policy,
    transformer_layer_cls={nn.TransformerEncoderLayer,
     ↪  nn.TransformerDecoderLayer, nn.MultiheadAttention}


    },
    min_num_params=int(1e8),
)
# Optional activation checkpointing for target modules; apply to the
 ↪  original model if preferred.
def _ckpt_check_fn(m: nn.Module) -> bool:
    return isinstance(m, (nn.TransformerEncoderLayer,
     ↪  nn.TransformerDecoderLayer, nn.MultiheadAttention))
apply_activation_checkpointing(
    model,
    checkpoint_wrapper_fn=checkpoint_wrapper,
    check_fn=_ckpt_check_fn,
)
```

```
# FSDP wrapper with correct argument types; MixedPrecision explicitly
 ↪ specified.
fsdp_model = FSDP(
    compiled_model,
    auto_wrap_policy=auto_wrap_policy,
    sharding_strategy=ShardingStrategy.HYBRID_SHARD,
    cpu_offload=CPUOffload(offload_params=True),
    use_orig_params=True,
    mixed_precision=MixedPrecision(param_dtype=torch.bfloat16,
     ↪ reduce_dtype=torch.bfloat16, buffer_dtype=torch.bfloat16),
    backward_prefetch=BackwardPrefetch.BACKWARD_PRE,
)
# Example input batch and labels.
batch = torch.randn(8, 128, dtype=torch.float32, device='cuda')
labels = torch.empty(8, 128, dtype=torch.long,
 ↪ device='cuda').random_(0, 100)
optimizer = torch.optim.AdamW(fsdp_model.parameters(), lr=1e-4,
 ↪ fused=True)
# Forward + backward.
outputs = fsdp_model(batch)
loss = nn.CrossEntropyLoss()(outputs.view(-1, outputs.size(-1)),
 ↪ labels.view(-1))
loss.backward()
optimizer.step()
optimizer.zero_grad(set_to_none=True)
```

这里，`transformer_layer_cls` 被设为你的块类，从而让每个块被独立分片。以块为粒度包装意味着每次前向传播只会为当前块调用 all-gather，随后丢弃其分片。这样一来，一次只有一个块的参数和优化器状态被完整具体化，峰值内存占用最多可减少总参数中块所占比例那部分。

此外，当某个块的计算运行时，下一个块的权重可以被预取或异步搬移。这就把通信延迟隐藏在块计算之后了。

请注意，训练循环本身不需要改变。`auto_wrap_policy` 把手动嵌套抽象掉了，你只需指定一次块类，块级通信等其余事务——都交由 FSDP 在幕后处理。

简而言之，在 transformer 块级别包装 FSDP 能带来更好的性能与内存效率。每个经过编译的块由于使用了融合、优化的 kernel，峰值内存占用更小；通信与计算得到了恰当的重叠；而 FSDP 的分片机制和按需进行的逐层 all-gather 集合通信意味着任意时刻内存中只有一个块的权重是完整的——并且也仅在前向与反向传播需要该块时才存在。

> **注** 请记住，TorchDynamo 的 `explain()` 输出会在每个 FSDP 边界处显示图中断（graph break）。这些断点是预期内的行为，正反映了正确的重叠机制。

13.11.3 基于 torch.compile 的张量与流水线并行

张量并行与流水线并行同 torch.compile 是正交的。只要跨 GPU 的通信操作能被 TorchDynamo 识别为集合通信调用，Dynamo 就会相应地对它们进行追踪或断开。

PyTorch 编译器主要专注于优化每个分段内部的计算。它（目前）不会融合通信操作，也不会改变其调度——除了前文所述的自然重叠。具体而言，TorchInductor 为计算选择 cublasLt/cuDNN/Triton kernel，而把 NCCL 集合通信（及其顺序）留给分布式策略来处理。使用任何分布式训练策略时，都应该分别在启用与不启用 torch.compile 的情况下进行测试，以确保结果符合预期；启用编译后，务必通过性能分析器轨迹来验证重叠情况。

推荐的做法是：用 torch.compile 优化每一层或每个桶（bucket）内部的计算，并像往常一样信任分布式策略（DDP、FSDP、TP、PP 等）来处理 GPU 间的通信。留意 TorchDynamo `explain()` 输出中与 all-reduce 相关的警告，但只需记住 PyTorch 的设计已尽力保证重叠不被破坏。因此，鉴于 torch.compile 对分布式训练的支持已相对成熟，你通常不需要做太多改动。

13.11.4 TorchTitan、AsyncTP、AutoParallel 与 SimpleFSDP

TorchTitan 是一套广受欢迎的、基于 PyTorch 的大规模模型训练参考实现。它提供了一组可扩展的训练方案（recipe），能够组合多种分布式训练策略，包括 FSDP、张量并行以及异步张量并行（AsyncTP）。

具体来说，AsyncTP 使用双流和 SM-wave 感知的调度来错开 TP 集合通信，使迟到（late-arriving）的 TP all-gather 与下一波次的矩阵乘法重叠执行。这种重叠有助于隐藏传统 TP 中出现的通信与调度间隙。采用这类异步并行策略的团队在 forward-pass 阶段和端到端均观察到了加速。请务必仅在你能够验证数值正确性与可扩展性的场景下使用异步并行。

可以通过 `torch.compile` 启用 AsyncTP。这会使矩阵乘法有机会被下译（lower）为融合的 all-gather 与 reduce-scatter。AsyncTP 与 torch.compile 组合良好，使融合后的计算 kernel 能与已调度的通信同步（lock-step）运行。

AutoParallel 是 PyTorch 的另一项计划，它会自动规划并应用 FSDP、张量并行和流水线并行的不同组合到模型图上。

AutoParallel 构建于 DTensor 的算子策略（operator-strategy）系统之上。DTensor 是 PyTorch 原生的分片原语，支撑着 PyTorch 可组合的张量并行与流水线并行方案，在 TorchTitan 参考实现中被大量使用。

AutoParallel 采用基于启发式的选择机制，依据给定工作负载的具体内存与通信开销，应用不同的并行计划来执行分片与切分。

对于大模型和复杂集群，建议使用 AutoParallel 来减少手工并行化调优的工作量——它与 TorchTitan 组合良好。

SimpleFSDP 借助 DTensor 以及选择性激活检查点（selective activation checkpointing）等技术，以对 torch.compile 友好的方式重新实现了 FSDP。编译器由 TorchInductor 对中间表示（IR）节点进行分桶（bucketing）与重排，从而帮助追踪并优化计算与通信的重叠。

在 TorchTitan 实验中，SimpleFSDP 将内存占用量最多降低了约 28%；与其他分布式技术组合后，相比传统的 FSDP2 eager 路径，训练吞吐量最多提升约 69%。

> **注** TorchTitan、AsyncTP、AutoParallel 与 SimpleFSDP 都是维护良好的项目，值得持续关注。它们是实践型的 PyTorch 参考实现，凝聚了许多来自业界 PyTorch 专家的优化成果。

13.11.5 使用 HTA 的多 GPU 性能分析

当你扩展到数百万 GPU 的规模时，处理数以百万计的独立轨迹和性能剖析文件会变得难以驾驭。Meta 的 HTA 可以帮助合并并可视化多 worker 轨迹。HTA 由 Meta AI 开源，它接收 `torch.profiler` 从每个 GPU/rank 生成的 JSON 轨迹，并呈现统一的时间线。

借助 HTA，你可以查看全部 8 个 GPU 的时间对齐轨迹，每个 rank 的 NVTX 标记也会对齐并可见。这样，你可能会注意到 rank 0 在时刻 T 进入了“反向”传播，而 rank 1 直到 T+1 才进入——也许是因为 rank 1 正在 all-reduce 中等待 rank 0；或者你会看到 rank 0 出现一段空隙而 rank 1–7 正忙于计算——如果 rank 0 提前完成计算并闲置，这可能表明存在负载不均衡。

HTA 还提供 GPU 空闲时间报告，甚至会针对如何通过重叠来提升效率给出建议。对于分布式训练而言，HTA 在定位拖慢整体的节点（straggler）和同步问题上极为有用。

> **注** 过去，人们曾尝试手动合并轨迹，再用 TensorBoard 的 trace viewer 查看。但截至 2025 年，用于轨迹可视化的 PyTorch TensorBoard profiler 插件已被弃用。现在，请改用 Perfetto 的 Trace Viewer 查看时间线，用 Meta 的 Holistic Trace Analysis（HTA）进行多节点聚合。

使用 HTA 的典型流程是：先利用 `torch.profiler.schedule` 之类的机制，在每个 GPU 上为少量训练迭代记录轨迹，然后将结果保存到共享位置。把这些轨迹加载进 HTA 工具之后，你就能看到每个线程的时间线、算子间的重叠情况，甚至每个 rank 的内存使用。

例如，你可以用 HTA 来确认 all-reduce 优化是否生效。优化前的轨迹呈现出清晰的顺序模式：反向传播计算完成、出现一段等待 all-reduce 完成的空隙，然后才开始下一段计算。在增大桶（bucket）大小并实现重叠之后，HTA 应显示更小的空隙，因为此时大部分通信已与其余的反向传播计算并发进行。

简而言之，HTA 是专为 PyTorch 多 GPU 性能分析而设计的工具。建议在跨越多 GPU 与多计算节点的分布式 PyTorch 环境中，使用 HTA 深入分析训练循环的行为。

13.12 持续集成与性能基准测试

应用完全部优化之后，关键在于持续维持这些优化，并不断检查是否出现性能回退。随着代码与配置的演进（例如 PyTorch 新版本发布、新的模型特性、代码重构等），如果不加以跟踪，性能很容易悄然回退。

你应该使用 TorchBench 和 GitHub Actions 搭建一个简单的性能回退持续集成（CI），以便自动捕获性能变慢。TorchBench 是一套开源的 PyTorch 模型基准测试套件，其中还包含若干以 torch.compile 运行的热门模型与基准，用于同时跟踪编译器自身的性能。

你还可以引入自己的模型——或一个较小的代理模型——来扩展 TorchBench。例如，可以在本地 fork TorchBench、加入你的模型，并把 TorchBench 作为构建与 CI 工作流的一部分来运行。这样，你就拥有一个持续运行的一等性能回退检测任务——既可以按计划定时执行，也可以在每次代码提交时执行。

性能回退任务会加载你的模型（或一个较小但有代表性的模型变体），运行若干次迭代并度量吞吐量——例如 tokens/sec 或 samples/sec。CI 任务将结果与存储的基准数字进行对比，一旦吞吐量低于某一阈值便判定 CI 构建失败。下面是一个定义基于 TorchBench 的性能回退任务的 GitHub Action 片段：

```
- name: Run MoE benchmark
  run: |
    torchbench run --model moe --iters 10 --batch-size 4 --json
     ↪  results.json
- name: Compare throughput
  run: |
    python scripts/compare_perf.py baseline.json results.json
```

第一步运行 MoE 基准测试，共 10 次迭代、batch size 为 4。如此短的运行足以在 CI 中评估性能；但要拿到最终数值，还需要测量更长时间的运行。我们把运行时长设得较短，是为了满足 CI 的时间限制。

此处的输出是一个 JSON 文件，由 `--json results.json` 选项指定。第二步运行一个小脚本，将新结果与（例如）存储在 main 分支上的基准 JSON 进行

比较。如果新提交的性能慢了——比方说慢了 5% 以上——我们就将其标记为性能回退，并判定 CI 构建失败。

> **注** 务必确保 CI 运行器的硬件保持一致，或使用云端预留的专用实例，以获得可复现的结果。不同类型的 GPU 之间以及不同云服务商之间的性能数字可能相差很大。

你还应当编写正确性单元测试，确保优化后的自定义 kernel 与 PyTorch 中对应操作的结果一致。尤其要覆盖边界情况和不同随机种子。

自定义 kernel 可能在基本测试中通过，但在极端输入上失败：数值非常大的输入若处理不当会导致溢出。在检验优化的正确性时，请使用 PyTorch 的 `torch.allclose` 并采用严格容差做数值精度检查，从而尽早发现正确性问题。

此外，应把内存使用量和数据加载耗时也纳入 CI 工作流的日志。例如，应在运行过程中记录 `torch.cuda.max_memory_allocated()`，并对实际的数据加载进行计时。请记住，性能优化是多维度的：某项改动可能让计算提速 20%，却使内存使用量增加 200%，在实践中反而可能得不偿失。

还要记住，软件并不完美。即使是 PyTorch 的版本更新，也可能在无意中改变你工作负载的性能特征。例如，新版本的 PyTorch 可能改变 kernel 融合（kernel fusion）模式或调度启发式策略。

如果这类更新让你的模型变慢 5%，你会希望尽早发现、及时调整代码，并以 PyTorch issue 的形式向上游报告。这种情况在自定义或较少使用、未纳入 PyTorch 常规性能测试的 LLM 上更为常见，但无论如何都值得留意。

PyTorch 团队对性能回退通常响应很快，一旦收到告警，往往会在 nightly 构建中推送修复。通过尽早发现回退，你甚至可以自己贡献修复，或至少提交一份有参考价值的报告。

简而言之，只优化一次是不够的。随着系统不断演进、新的性能优化被应用、新功能陆续加入，你需要持续守护优化成果。把性能测试集成到 CI 中之后，任何损害性能的代码改动都会立即暴露并得到处理，你也就能确信自己的性能提升不会在下一次代码重构中悄然流失。

> **注** 建议为性能波动预留一定容差：在判定构建失败前先运行多次迭代。性能会因外部噪声而轻微波动。例如，可以规定连续 3 次运行出现 5% 以

上的回退才判定构建失败。PyTorch 自身的持续性能回退系统（下一节讨论）就使用统计平滑来避免误报。

除了自己的 CI，还应关注 PyTorch 的性能 HUD（感知显示，heads-up display）。这是一个公开仪表板，借助 PyTorch 的 CI 系统跟踪常用模型的性能，下一节将详述。

13.12.1 PyTorch HUD 性能仪表板

在优化的同时，最好能掌握整个框架层面性能随时间的变化情况。PyTorch 开源的 HUD 提供来自 nightly 基准测试的实时反馈。这个 Web 界面展示 PyTorch 仓库在多种硬件后端（包括 NVIDIA GPU、AMD GPU 和 CPU）以及众多常用模型上的构建状态、测试结果和性能指标。

PyTorch 工程师以及社区中的任何人都可以借助 HUD 检测回退。如果某个新 commit 导致某个模型的 tokens/sec 显著下降，HUD 会将其标红作为早期预警，促使人们深入调查引发回退的那些改动。

HUD 的监测阈值通常在 5% 左右。轻微的噪声波动属于正常现象，但超过该阈值的变化就会触发调查。HUD 还包含趋势线，因此如果性能因大量小 commit 的累积而在数周内逐渐衰减，这种情况同样会被标记出来供调查。

在 HUD 中导航到 Benchmarks → vLLM 部分，可以看到各种语言模型的基准测试结果。如图 13-7 所示，该仪表板针对每个模型和硬件类型跟踪随时间变化的多项指标，包括编译时间、内存使用量、吞吐量（tokens/秒）和 FLOPS 利用率。

例如，你可能会发现自某个日期起吞吐量下降而内存使用量上升，这提示可能出现了更严重的内存碎片，或者调度器选择了效率更低的 kernel。HUD 能帮助我们把这些变化直接与对应的 GitHub commit 关联起来。

通过监控 HUD，可以判断上游 PyTorch 的改动是否可能影响你的模型。如果你的模型未被直接收录进 HUD，可以采用架构相近的模型作为代理。此外，HUD 是开源的，你可以为自己的模型、硬件和环境自行搭建类似的系统。

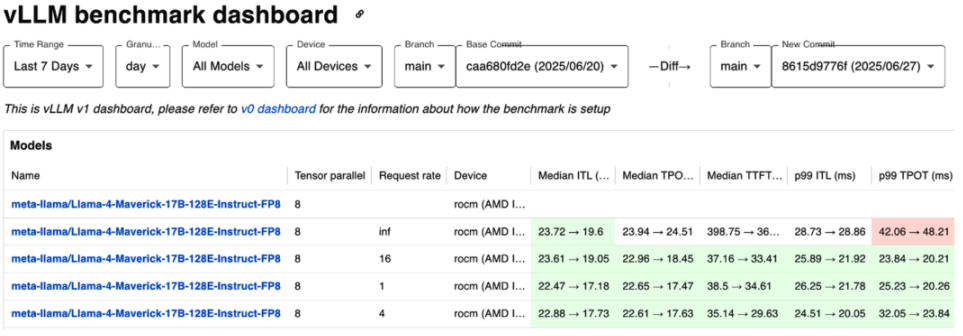

图 13-7　PyTorch HUD 仪表板（来源：https://oreil.ly/JxLKJ）

图表上的每个数据点都是通过比较基线 commit 与 PyTorch 主分支最新 commit 的性能基准结果得到的。仪表板允许选择时间范围（如最近 7 天、30 天）和粒度（按小时、按天、按周），以便放大或缩小查看随时间变化的趋势。

HUD 的实现已在 pytorch/test-infra 仓库开源，由 Python 脚本与 Grafana 仪表板组合而成。你可以在内部搭建一个迷你 HUD：持续收集自己模型的性能数据并加以可视化。即便是一个简单的电子表格或一个 Grafana 实例，也能实现这类可视化。关键在于始终如一地跟踪同一组指标。

添加或修改仪表板、贡献新的 YAML 配置，以及在 CI 中更新基准测试脚本都相当直接。原则上，如果想在 HUD 中跟踪自己的自定义模型，可以为它添加一个基准测试并让它在 PyTorch 的 CI 中运行，这样任何影响你模型性能的 PyTorch 改动都会体现在图表上。

> **注**　强烈建议为自己的模型搭建一套类似 pytorch/test-infra 的持续基准测试与性能回退系统，以便尽早发现专属于你工作负载的性能回退。

13.12.2 性能基准测试与 MLPerf 日志

除特设基准和 CI 之外，MLPerf 这类行业标准基准也能提供有价值的反馈并鼓励良好的实践。MLPerf Training 和 Inference 基准借助严格的日志记录推动最先进性能，确保结果可信、可进行对等（apples-to-apples）比较。即使你不参赛，MLPerf 的日志方法学对你自己系统的性能分析也很有用。

MLPerf 日志对训练过程中关键事件和指标的输出做了标准化。每条日志是一个以 :::MLL 这类前缀打印的 JSON 对象，内容包括键值对和元数据。在 PyTorch 中，可以使用 GitHub 上的开源 MLPerf Logging 库把输出格式化为 MLPerf 标准。

在 MLPerf 训练运行中，日志包含初始化开始时间、训练开始时间、每个 epoch 的时间、训练结束时间、最终精度等条目，所有内容都精确到毫秒打点并妥善标注。

MLPerf 合规脚本会解析这些日志，验证运行是否遵守规则：例如，开始之后不得更改超参数，必须使用规定数量的 epoch，等等。

日志还包含计算出的吞吐量，以及运行是否在允许时间内达到目标精度。这保证了公平性：达不到精度要求，就不能只报吞吐量数字。

> **注** 在自己的训练中，应始终把性能测量与精度检查配对进行，以免把模型优化到不稳定的境地。

一些 MLPerf 日志（尤其是研究类提交）包含每个 epoch 和每次迭代计时的细分。竞赛并不要求这种细节，但它在内部非常有用：例如，可以记录每次迭代在前向传播、反向传播以及 all-reduce 等通信上各花费多少时间；还可以记录多机集群中全部节点上这些计时的平均值，以定位分布式瓶颈。

下面是一段简短的 MLPerf JSON 日志示例；随后的表 13-6 由这些日志导出，列出各组件占单步时间的百分比：

```
{
  "step_time_ms": 24.0,
  "forward_ms": 10.5,
  "backward_ms": 9.0,
  "allreduce_ms": 4.0,
  "other_ms": 0.5
}
```

在这个 24 ms 的训练步示例中，计算（前向和反向）耗时 19.5 ms（10.5 ms + 9.0 ms），占单步时间的 81.3%（43.8% + 37.5%）；通信（梯度 all-reduce）耗时 4.0 ms（16.7%）；其他开销（如数据加载）占单步时间的 0.5 ms（2.1%）。

表 13-6　MLPerf 日志示例：每次训练迭代中各组件的耗时分解。

步骤组件	每次迭代耗时（ms）	占单步时间百分比
前向传播	10.5 ms	43.8%
反向传播	9.0 ms	37.5%
All-reduce（梯度同步）	4.0 ms	16.7%
其他开销	0.5 ms	2.1%
单步总耗时	24.0 ms	100%

这样的分解极为有用，它告诉我们大约六分之一的时间花费在梯度同步上。如果想进一步加速训练，可以重点考虑通过重叠通信或缩短 all-reduce 时间来优化。

例如，我们可以尝试激活值压缩，或者利用异步 all-reduce、流水线并行等技术让通信与计算更好地重叠，从而压缩 all-reduce 所占的 4 ms。如果“其他开销”足够大，我们就应转向数据加载，而对症处理的方式也会随之不同。

MLPerf 针对 all-reduce 的优化包括延迟 all-reduce（称为 slack），以及让多个较小的 all-reduce 与计算相重叠等技巧。这些属于超出本章讨论范围的高级技巧，但关键在于：这类分解能直接把你引向需要优化的确切位置。

虽然 MLPerf Logging 是为竞赛规则而设计的，但结构化日志、性能指标和计时分解这一通用做法完全可以应用于你自己的训练与推理模拟。例如，你可以对训练循环进行插桩，在每个 epoch 记录一行 JSON，附上吞吐量、延迟、GPU 利用率（取自 `nvidia-smi`）等额外指标。

在漫长的训练过程中，这些日志会成为训练后分析的宝库。你可以绘制性能从第 1 天到第 7 天的变化曲线，判断作业是否因内存碎片而变慢；也可以观察数据加载、计算等各个阶段的扩展情况。

在记录指标而不只记录最终准确率的前提下，你的结果才可复现、可调试。如果日后有人重新训练你的模型却发现更慢，日志将有助于定位问题：也许是数据输入层变慢了，也许是他们使用了不同的硬件配置。这一做法与 CI（持续集成）相辅相成——后者倡导“记录、监控、对比”的方法论。

通过对流水线进行插桩，使其输出各类计时组件的 JSON 日志，你就能在实施优化的过程中跟踪改进效果。这也让团队能够以标准化的方式沟通瓶颈所在以及调优结果。

注 由于 MLPerf 涵盖了跨多个 GPU 和多个计算节点的大规模模型基准测试，你可以研读 MLPerf 的提交成绩，从中了解流行 LLM 与集群配置的最佳实践。本书讨论的许多优化技术都出现在 MLPerf 的获胜提交中。这是持续获取性能技巧——以及大规模场景下最优集群拓扑——的绝佳来源。

13.13 要点回顾

PyTorch 的相对简洁与高度抽象有时会让人产生性能上的虚假安全感。因此，在开发过程中引入细微的性能 bug 出人意料地容易。下面总结常见的 PyTorch 性能陷阱及其应对方法：

坚持“先性能分析”的原则。

在超大规模下，瓶颈可能隐藏在任何一层——Python 开销、PyTorch 框架调度、CPU 数据加载停顿、GPU kernel 低效、内存问题等。仅凭直觉往往会漏掉真正的热点。应像本章所做的那样，采用多工具配合的整体性能分析策略，捕获每一层的性能表现。现代性能分析器具有低开销模式，可以在生产环境中使用以捕捉性能回退。将这些工具与 `nvidia-smi` 等硬件指标（如 GPU SM 占用率）相结合，就能有把握地识别瓶颈并正确排定优化优先级——而不是优化错了地方。

优先使用编译模式而非 eager 模式。

在 eager 模式下，每个微小操作都会作为独立的 kernel 启动，每次都要承受 Python 分发和 GPU 启动开销。更好的做法是使用 PyTorch 的 JIT 编译，即 `torch.compile`。只需一行改动（`model = torch.compile(model)`），PyTorch 便能捕获模型计算图并生成融合且经过优化的代码。

使用工作负载所能允许的最高编译优化等级。

对于长时间运行的作业，`max-autotune` 通常在稳态速度上占优，但小批次或动态形状场景下 `reduce-overhead` 可能更合适。应在自己的工作负载上验证各模式。`max-autotune` 中的 CUDA Graphs 能够掩盖启动开销，但与频繁变化的形状不兼容。

保存编译产物以便复用。

如果启动时间是需要关注的问题，最好缓存编译产物以便日后复用。为此可以使用 `torch.compiler.save_cache_artifacts()` 和 `load_cache_artifacts()`。对于多节点集群上的长时间作业，建议将编译产物作为“巨型缓存”持久化到共享路径（例如通过 `TORCHINDUCTOR_CACHE_DIR` 环境变量指定），并让它在各节点挂载于相同位置。这有助于避免新节点启动时的冷启动。

避开同步陷阱。

PyTorch 以易用性为设计目标，这意味着很容易在无意中写出迫使 CPU 与 GPU 同步的代码。例如，对 CUDA 张量调用 `tensor.item()` 获取 Python 值会导致 GPU 同步。在协调多个流时，应使用带流事件的 `torch.cuda.Stream.wait_stream()`，而不是强制同步。同样，从 GPU 向 CPU 传输数据时若未使用 `non_blocking=True`，也会引发同步。请使用异步传输，并让性能分析器为你揪出所有隐藏的同步。

不要用 `time.time()` 做 Python 侧计时，它会带来隐式同步。

用 `time.time()` 对 GPU 代码块计时会包含一次同步。更好的做法是使用 `torch.cuda.Event(enable_timing=True)`，在不引入多余同步的情况下为 GPU 代码计时。

充分利用 Tensor Core。

在不知不觉中回退到全 FP32——从而没有用上 Tensor Core——既出人意料地容易，也不符合预期。为确保用上 Tensor Core，应将前向传播与损失计算包在 `torch.autocast` 中，并选择较低精度的 dtype，使 GEMM 能够使用 Tensor Core。（注意：除非显式转换模型，`autocast` 不会改变模型权重的存储 dtype；它只为符合条件的运算选择计算 dtype，并将对数值敏感的运算保留在 float32 中。）

用 TF32 替代 FP32 以激活 Tensor Core。

对于能容忍 TF32 精度的 FP32 工作负载，使用 `torch.set_float32_matmul_precision()` 并选择”high” 或”medium” 来启用 TF32。在 CUDA 设备上，该设置在底层对应 `torch.backends.cuda.matmul.fp32_precision`。在现代 GPU 上，TF32 会使用 Tensor Core，这正是理想情况。请务必用 Nsight Compute 的 SpeedOfLight 视图或 `sm__inst_executed_pipe_tensor` 指标核实 kernel 确实用了

Tensor Core。（注意：选择"highest" 会强制使用真正的 FP32 并禁用 TF32，因此要小心。这对调试或许有用，但如果目标是提升性能，则不推荐。）

确认 Tensor Core 真正被使用。

要核实 kernel 使用了 Tensor Core 以及预期的 dtype，应借助性能分析器。在 PyTorch 中，先用 `torch.profiler` 检查算子耗时，再用 Nsight Compute 确认 Tensor 流水线的活动。Nsight Compute 中的 SpeedOfLight 与 Compute Workload Analysis 部分会报告 Tensor 流水线利用率，以及基于性能监视器（PM）采样的 kernel 执行时间线。

可能时优先用 BF16 而非 FP16。

在现代 GPU 上优先选择 `dtype=torch.bfloat16`，因为 BF16 保留了 FP32 的指数范围，通常不需要梯度缩放。如果必须在 FP16 下训练，请启用 `torch.cuda.amp.GradScaler`，并在日志中确认没有上溢/下溢。在现代 GPU 的 Tensor Core 上，BF16 一般无需缩放。

融合小操作。

许多模型计算由一长串小型逐元素或矩阵运算组成，每个小操作都会带来额外的内存读写和启动开销。可用 `torch.profiler` 找出连续出现的大量微小操作（例如 < 1 ms）。若发现 `linear` → `gelu` → `dropout` 这样的模式，考虑用融合模块替换，或依靠 `torch.compile` 自动融合；对于编译器遗漏的操作，可考虑自行编写融合 kernel。每个自定义融合可节省百分之几的性能，这些“最后一英里”的收益累积起来相当可观。

减少内存碎片。

如果训练作业或推理流水线要运行数天、数周乃至数月，内存碎片就可能成为问题——尤其是在大多数 LLM 应用中，张量大小随迭代而变化。考虑使用内存池库，例如 PyTorch 的缓存分配器，它可以预先分配好所有大块张量。不同的分配尺寸越少越好，因此应尽量复用内存并保持形状恒定。还可以通过调优环境变量（如 `max_split_size_mb`）来主动管理 CUDA 分配器；避免把大块内存拆成许多小块，按固定最大尺寸预分配常用缓冲区，并复用缓冲区以保持分配尺寸一致。这些做法能让内存占用长期保持稳定，避免因碎片导致的性能下降和崩溃。

使用激活检查点

默认情况下，PyTorch 会保存反向传播所需的全部激活值，这对大模型而言会占用巨大的内存。可以考虑使用激活检查点（activation checkpointing），用算力换取内存。`torch.utils.checkpoint` 让这一点很容易实现：只需包裹前向传播中的若干段，它们的激活值就不会被保存，而会在反向传播中重新计算。这会以额外的计算开销换取更低的内存占用。不过这通常是值得的，因为

对现代 GPU 而言，GPU 内存的稀缺程度更甚于算力。对于参数量在数百亿以上的模型，在现代 GPU 硬件上这几乎是必需的。另外还要记住，你可以混合使用不同精度。例如，可以把少数关键激活值保留在更高精度以维持精度，而以更低精度重算其余激活值，从而节省内存和时间。

将内存卸载到 CPU 或 NVMe 存储

模型参数、梯度、优化器状态和激活值都要争夺有限的 GPU VRAM。即使 HBM 容量很大，一个数千亿参数的模型也会超出其容量——即便使用更低的精度。要善用系统的内存层级结构，把不常使用的数据卸载（offload）到 CPU 内存或 NVMe 磁盘——仅在需要时再传送回来。这些数据传输可以通过异步拷贝流式地进行，并与计算相互重叠。这样做时要加强互联吞吐量的监控，确保链路没有被完全占满。

减少输入流水线停顿

即使训练循环已经优化到极致，如果输入流水线无法足够快地供给数据，性能仍可能遭遇瓶颈。这通常表现为 GPU 等待下一个批次时的空闲时间气泡。在性能分析中这往往非常明显，因为多数工具会显示迭代之间的空隙，或者 CPU 线程忙于数据加载代码而 GPU 空闲。应使用足够数量的 DataLoader worker 以及合适的 `prefetch_factor` 大小，使数据吞吐量最大化。

利用所有 CPU 核心进行数据加载和预处理非常重要。使用 `pin_memory=True` 和非阻塞传输可以加快主机到设备（H2D）的拷贝。将数据集预处理（例如分词）离线完成，让数据加载器只做最少的工作。此外，还可以增大 DataLoader 的 `prefetch_factor`，建议持续地提前获取额外批次，使其略微领先于实际使用时机。

对 CPU 侧数据变换进行性能分析，必要时将其卸载

由于 Python 数据变换（例如分词）可能慢得出乎意料，务必确保这些主机侧变换已完成向量化。在可用时应采用优化过的 C++ 库。还可以借助 NVIDIA DALI 之类的库，将复杂变换卸载到 GPU 上执行。

优化多 GPU 与多节点通信

分布式训练如果管理不当，往往会因通信开销而触及性能上限。应使用 DDP 等 PyTorch 优化的分布式实现，它们默认就会将通信与计算重叠。调节梯度的 `bucket_cap_mb`，以找到实现最佳重叠的桶大小。更大的桶（例如 50 MB 而非默认的 25 MB）能够减少每条消息的开销，并在网络条件允许时实现更好的重叠。

监控网络带宽

如果带宽已经打满，可以探索激活值压缩技术来减少带宽占用。务必将 all-reduce 集合通信与反向计算重叠执行，以隐藏通信时间。对于多节点训练，要考虑网络拓扑，把频繁通信的进程放在同一节点或同一交换机之下，以降低延迟。

避免性能随时间发生“bit rot”（缓慢退化）

由于持续的代码变更、框架升级与新应用特性，性能很容易发生退化。例如，一次小的重构可能引入意外的同步点，或一次 PyTorch 升级改变了某个算子的实现。要把性能当作一等指标来对待，建立持续集成测试、仪表盘和告警来监控性能。每次代码提交（或每日/每周运行）都应包含一次快速的基准测试，以便捕获性能回退。

硬件变更时更新基线

例如，如果从 B200 迁移到 GB300，就要重新确立性能基线和阈值。新硬件对某些工作负载模式的处理可能更好，也可能更差。要重新校准，并相应调整告警阈值。

使用 TorchBench 或自定义计时脚本并结合自动化工作流

要留意 PyTorch nightly 性能基准的外部信号（见公开的 HUD regressions 仪表盘）——尤其是与你的模型相似的模型的表现。如果 PyTorch 出现了普遍的性能退化，你在升级前就能得知。一旦检测到变慢，要立即调查，找出根因（例如你的代码或上游变更），加以缓解（例如回退或适配代码），并上报上游（如 PyTorch、Triton、NVIDIA 等），让相应社区解决问题并受益于修复。此外，还

要为你的优化维护正确性测试，以免悄悄破坏精度。把性能回归测试纳入日常工作流，你的加速成果才能经得起时间的考验。

13.14 结论

重要的是把微观层面的性能分析（例如按算子和按 kernel 的分析）与宏观层面的基准测试（例如端到端吞吐量和延迟）结合起来，这样你才能对系统性能有全面的把握。优化单独的 kernel 与调优完整的端到端训练和推理流水线同等重要，而且要长期保持调优状态。随着硬件不断演进，这种整体性方法至关重要。

本章给出的基础知识将帮助你适应新平台。要始终进行性能分析，识别瓶颈，并在技术栈的每一层级应用相应的优化。通过系统地应用编译器与内存优化，你可以显著提升工作负载的性能，使其逼近硬件极限——同时仍保持可接受的精度。

正如我们所展示的，细致的性能分析能够找出真正的瓶颈。而有的放矢的优化可以在训练和推理性能上带来大幅加速。同样重要的是，要建立自动化机制来保护这些成果，使其免受性能回退的侵蚀。

这段端到端优化之旅需要在技术栈的每一层级投入努力。其最终成果是一个经过优化、高效且能抵御性能回退的系统：它可以从现代 GPU 硬件中榨取最大性能，能够扩展到多节点集群与机架级拓扑，并能适应未来硬件、软件和算法的进步。

第 14 章 PyTorch 编译器、OpenAI Triton 与 XLA 后端

第 13 章讨论了多种优化和调优基于 PyTorch 的训练与推理工作负载的方法。我们介绍了 PyTorch 编译器，以及它如何通过 kernel 融合和其他 kernel 级技术，在几乎不改动代码的情况下自动提升性能。

本章将深入动态 PyTorch 编译栈，包括 TorchDynamo、Ahead-of-Time Autograd（AOT Autograd）和 PrimTorch Intermediate Representation（IR）（又称 Prims 或 Prims IR），以及 TorchInductor、Accelerated Linear Algebra（XLA）和 OpenAI Triton 生态等编译器后端。PyTorch 编译器栈如图 14-1 所示。

我们还将介绍调试编译流水线的工具，以及在多 GPU 和多节点集群上扩展 PyTorch 的库。随后探索 torch.compile 的内部工作方式，并了解如何高效处理动态形状和可变序列长度。

我们也会考察 PyTorch 编译器与 OpenAI Triton 生态的集成。目标是在不牺牲 PyTorch 灵活的 eager 执行开发体验的前提下，加速并扩展 PyTorch 模型和应用。

14.1 PyTorch 编译器深潜

如第 13 章所述，PyTorch 的 torch.compile 会编译 PyTorch 代码（以及模型），从而带来显著加速。多数情况下，只需像下面这样写一行代码即可。我们会在后文逐步讨论不同选项：

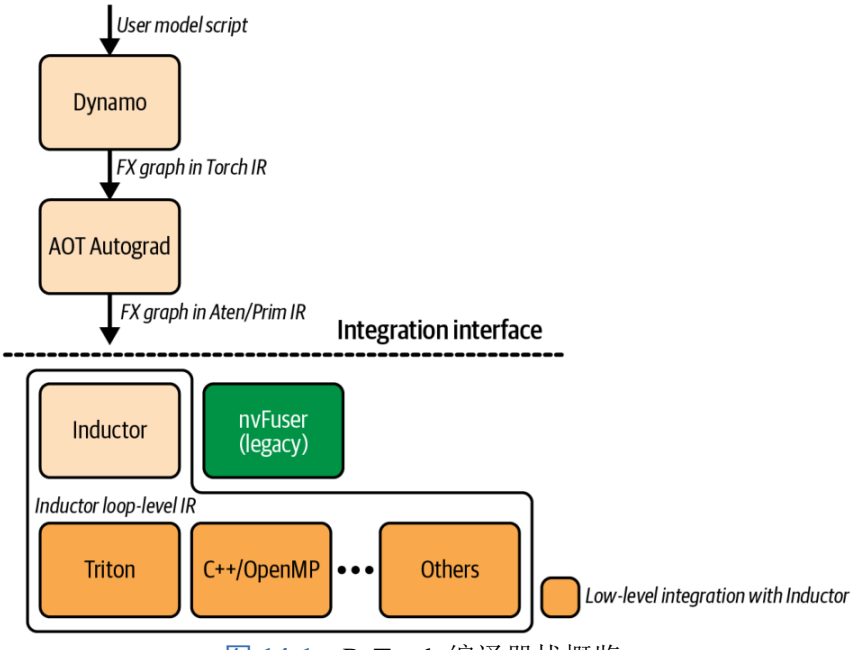

图 14-1　PyTorch 编译器栈概览。

```
compiled_model = torch.compile(model,
  mode="max-autotune",
#  ...
)
```

本节拆解 PyTorch 编译流水线的步骤，包括 TorchDynamo 的图捕获、AOT Autograd 的前向/反向联合图优化、PrimTorch IR，以及 TorchInductor 的代码生成。该流水线负责针对目标 GPU 硬件生成优化后的 kernel，如图 14-2 所示。

14.1.1 用于字节码捕获和图提取的 TorchDynamo

TorchDynamo（简称 Dynamo）是 torch.compile 的第一阶段。它挂接 Python 的帧求值机制，在字节码层拦截模型执行。

Dynamo 挂接 CPython 的帧求值过程，识别产生 tensor 的字节码区域，并为这些区域构造执行图。随后，它使用选定的后端执行编译图。不支持的代码则保留为 eager 执行。

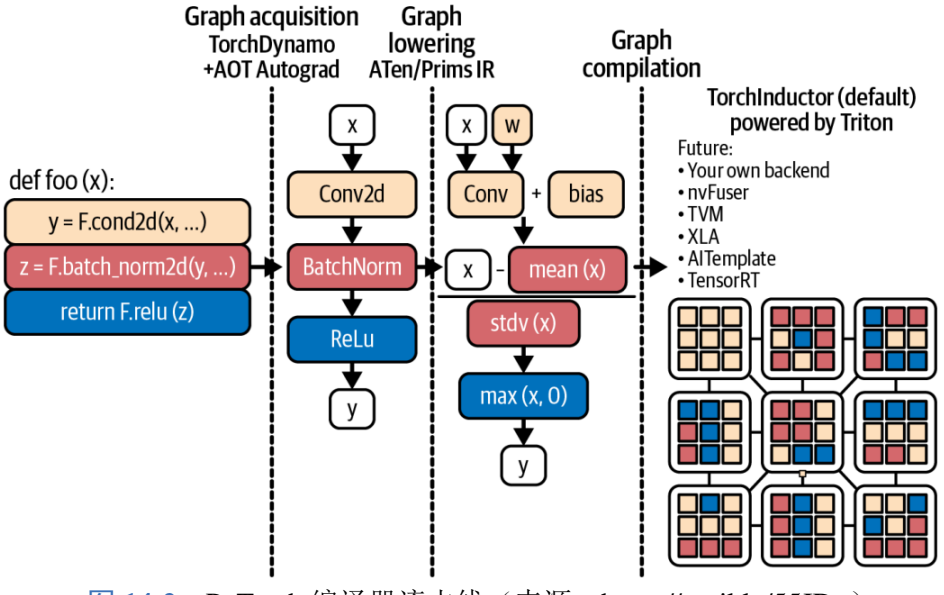

图 14-2　PyTorch 编译器流水线（来源：https://oreil.ly/55JDn）

这种拦截和重写机制使 TorchDynamo 能够把一连串 PyTorch 操作捕获为图表示，供后续步骤——AOT Autograd 和 PrimTorch IR（将在下一节介绍）——进行优化。

TorchDynamo 借助 CPython Frame Evaluation API（PEP 523）安全地捕获操作，同时将开销降至最低。通常，Python 解释器逐个执行每个操作；启用 Dynamo 后，解释器会把执行转交给 Dynamo，由后者在执行前聚合 tensor 操作形成图。这支持 kernel 融合等整图优化，从而降低每个操作的 Python 和主机侧开销。

与其为每个操作启动一个新的 GPU kernel 并支付逐操作的 Python 开销，不如让编译图把多个操作融合成一个或少数几个 kernel。这样可以降低调度开销并改善内存访问模式。

第 13 章展示过，Python 开销和大量小操作可能成为模型的瓶颈。我们也看到，只要操作链对图友好且不会造成过多图断裂，TorchDynamo 会在可能时把小操作批处理成更大的单元。

通过 torch.compile 启用 TorchDynamo 后，它会检查每个 Python opcode。每当遇到 PyTorch tensor 操作（例如算术运算或神经网络层），它不会立即执行，而是把该操作追加为 FX 图中的一个节点，并将捕获区域的执行延后给编译后端。

TorchDynamo 会持续进行，直到遇到无法处理的代码。此时会发生图断裂（稍后详述），不支持的操作则在普通的、未编译的 eager 模式下运行。

TorchDynamo 尝试把程序尽可能大的部分编译成单个图；但遇到复杂控制流或非 PyTorch 库调用等不支持的构造时，它会回退到 Python eager 执行。这是为了确保正确性。

越过不支持的操作块后，Dynamo 会从该点开始恢复捕获后续操作，并生成新的图。这种编译执行与非编译执行的混合，兼顾了两者的优点：需要灵活性时保留 PyTorch 的灵活性，其余部分则编译以获得速度。

> **注** 应尽可能避免图断裂，因为它会中断整图优化，并限制编译器带来的性能收益。

可以使用 torch.compiler.set_stance("fail_on_recompile")，强制 Dynamo 在检测到不安全的重新编译时抛出错误。它会记录重新编译的原因，帮助你调试图断裂，从而完整了解图为何被拆分。代码如下：

```
// fail on graph breaks, recompiles
torch.compiler.set_stance("fail_on_recompile")
compiled_model = torch.compile(model,
  mode="max-autotune",
  ...
)
```

这样就能重构导致断裂的代码路径，或者使用 torch._dynamo.allow_in_graph() 将其标记为预期的图边界，稍后会看到这一点。图清理完成后，可以切回 torch.compiler.set_stance("eager_on_recompile")。请注意，切回该设置后，如果后续再次发生图断裂，TorchDynamo 会静默回退到 eager 模式。

TorchDynamo 捕获的输出称为代码的 FX Graph。FX 是一种中间表示（IR），其中每个节点都是对 PyTorch aten 操作符或内置 Python 函数的调用。例如，考虑下面这个简单的 Python 函数：

```
def f(x, y):
    z = x.sin() + y
    return z.sum()
```

此时，TorchDynamo 会生成大致等价于下列伪代码的 FX Graph：

```
graph():  # pseudo-code for FX IR
    %x : Tensor = Placeholder[target=x]
    %y : Tensor = Placeholder[target=y]
    %sin : Tensor = torch.sin(%x)
    %add : Tensor = torch.add(%sin, %y)
    %sum : Tensor = torch.sum(%add)
    return %sum
```

FX Graph 节点对应 Placeholder 输入，以及 aten::sin、aten::add、aten::sum 等基本 ATen 操作调用，清楚地表示了计算图结构。构造完成后，该 FX Graph 会交给 AOT Autograd，以便联合追踪前向和反向传播。

AOT Autograd 生成的前向和反向联合追踪随后会发送给 TorchInductor 或 XLA 等后端编译器，执行 kernel 融合并生成优化后的设备代码。TorchDynamo 本身与框架无关，专注于准确且低开销的图捕获；所有繁重的优化都委托给下游编译阶段。

TorchDynamo 会在影响图追踪的 Python 值上插入 guard，例如 tensor 形状、dtype，甚至全局变量。这些 guard 确保一旦被假定为常量的内容发生变化（例如 tensor 形状或 dtype），编译图就会失效并在需要时重新编译。因此它能稳健处理动态形状和模型修改，但如果假设被违反，也会付出额外重新编译的代价。

注 建议在使用 PyTorch 编译器时监控图重新编译的次数。

使用默认设置 dynamic=None 时，编译器会根据观测到的形状进行特化。遇到动态形状后，它会重新编译出更动态的 kernel；如果之后发现更多动态来源，可能还会再次编译。

实际而言，这意味着在第一个形状发生变化的输入上会多进行一次编译，而且只会多这一次编译。如果你已经知道哪些输入维度会变化，可以在运行模型前使用 torch._dynamo.mark_dynamic(tensor, dim)，这样就能提前避免初次编译。

还可以通过 torch.compiler.set_stance() 调整编译器的“姿态”，改变 TorchDynamo 何时以及如何回退到 eager 或重新编译。姿态控制编译器面对错误或回退时的宽容度/严格度，从而让你更好地权衡开发反馈与不中断执行。本文撰写时可用的姿态如下：

14.1.2 default

编译器会尽力编译；遇到不支持的代码时静默回退到 eager 执行。这是标准的默认设置，会产生通常的编译行为和回退。

14.1.3 fail_on_recompile

如果遇到图断裂或不支持的操作，编译器会抛出错误。这在开发阶段很有用，可用于捕获意外的断裂或未编译路径。

14.1.4 eager_on_recompile

需要重新编译时以 eager 模式运行。如果已有编译代码（且对该输入仍然有效），仍应继续使用它。

14.1.5 force_eager

使用 eager 模式，并忽略所有 torch.compile 指令。

通过捕获完整的操作序列，TorchDynamo 可以执行 kernel 融合等整图优化，降低启动开销并减少内存移动。它还会消除这些融合操作在 Python 层的开销。图 14-3 比较了 eager 与编译模式，其中也包括编译器缓存。

编译器不再逐个执行小操作并分别启动 kernel，而是可以把多个操作融合到单个 kernel 中。这会减少 CPU-GPU 同步点并改善内存局部性。否则，GPU 可能因大量细粒度操作、频繁同步和过多全局内存访问而成为瓶颈。

TorchDynamo 会持续捕获，直到需要发生图断裂。某些不支持的 Python 构造（例如使用 Python bool 而非 tensor 操作的 if 语句）或不支持的操作，都可能

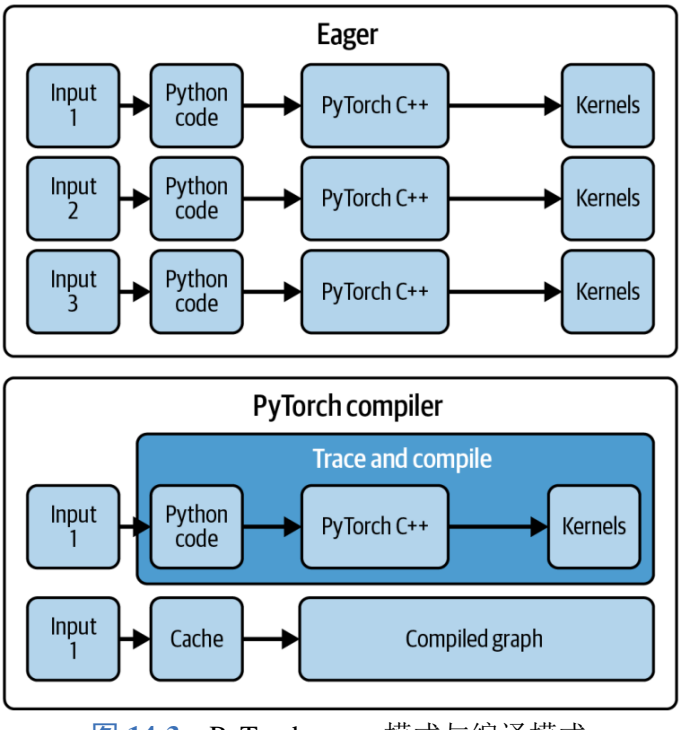

图 14-3　PyTorch eager 模式与编译模式。

触发图断裂。断裂发生后，当前图段结束；对于无法捕获的代码，Dynamo 回退到 eager 模式。随后，一旦回到可追踪代码，TorchDynamo 就会开始新的图追踪。

在 PyTorch 中，可以通过把逻辑表达为纯 PyTorch tensor 操作（包括带 mask 的 torch.where()）来实现对编译器友好的条件分支。下面的例子用纯 tensor 的 mask 方法替换了不利于建图的 Python if 语句：

```
# Compute a boolean mask from a data-dependent tensor condition
# mask shape (batch_size,1) broadcastable to x' s
mask = x.sum(dim=1, keepdim=True) > 0
# Use torch.where to select element-wise between two tensor expressions
# picks f(x) where mask is True, otherwise g(x)
out = torch.where(mask, f(x), g(x))
```

这避免了会造成图断裂的 Python if x.sum() > 0: 语句，因为整个过程都保持在对 PyTorch 图友好的 tensor 操作中。在本例中，TorchDynamo 会捕获包括 mask 和 torch.where() 在内的完整序列，而不会断图。

随着每个新版本发布，PyTorch 都会扩大可捕获且不会造成图断裂的操作范围。例如，torch.cond 等高级条件原语可以在图中捕获某些 if/else 逻辑，具体而言，两个分支都会被追踪和编译。torch.cond 要求谓词是布尔标量；两个分支必须返回相同的结构和 dtype，且运行时形状必须一致。数据依赖的分支通常仍会导致图断裂。

> **注** 建议通过 torch.where() 等 tensor 操作重构代码，尽量减少图断裂，从而最大化 TorchDynamo 可以连续捕获的区域。

14.1.6 用于前向和反向传播融合的 AOT Autograd

当 TorchDynamo 尽可能捕获了前向传播的 FX Graph 后，下一个编译阶段就是 AOT Autograd。AOT Autograd 以“函数式”模式让 Dynamo 捕获的前向图通过 PyTorch 的 autograd 引擎，以记录反向操作。这样就生成了静态反向图（不同于依赖 PyTorch 默认 autograd 引擎逐个执行反向操作）。

本质上，AOT Autograd 生成一个前向-反向联合图，随后可以把它作为整体进行优化和融合；同时保证前向和反向结果与 eager 模式一致。

AOT Autograd 通过让前向图经过 autograd 引擎来追踪梯度计算，从而捕获梯度所需的操作。它实际上使用 torch.autograd.forward_ad（或类似技术）运行前向图，记录反向计算所需的操作，结果就是一个组合的前向-反向图。然后可以使用公共子表达式消除等方法优化该联合图，之后再由 TorchInductor 或 XLA 等后端编译。

通过提前规划前向和反向，PyTorch 编译器可以跨越前向/反向边界进行整体融合，把横跨两个阶段的操作提前融合。例如，在可能的情况下，它可以将前向中的逐元素操作与对应的反向逐元素梯度计算融合到同一个 kernel 中。

如果安全，编译器还可以在前向与反向计算之间复用中间结果，从而消除开销。这能显著改善模型训练工作负载的性能，因为反向操作可能占据总运行时间的主导部分。

没有 AOT Autograd 时，PyTorch 必须依靠默认的 PyTorch autograd 引擎独立执行每一个反向操作，而不考虑前向传播。有了 AOT Autograd，执行图就能进行整体优化。

AOT Autograd 生成的联合图保证计算出相同结果（这也是图无法保证正确性时必须断裂的原因）。由于整个前向和反向操作序列已知，这个联合图还可以进一步优化。

通过在前向和反向传播之间融合操作和内存使用，可以减少内存访问和 kernel 启动开销。当调用 torch.compile 处理涉及梯度的操作和工作负载（例如模型训练）时，PyTorch 编译模式会在内部自动使用 AOT Autograd。

简而言之，AOT Autograd 在模型训练期间供 torch.compile 提前计算梯度。它无缝处理大多数 autograd 操作，并保证结果与 eager 模式一致；通过在前向和反向传播之间复用缓冲区，降低峰值内存使用。虽然这一阶段对终端用户大多不可见，但它是现代 AI 模型训练工作负载获得大幅加速的关键组件。

14.1.7 PrimTorch IR（Prims）：简化的操作集

在把图交给低级代码生成器之前，PyTorch 会执行一种称为 PrimTorch IR（某些文档和源代码中称为 Prims）的中间表示（IR）转换。PrimTorch IR 会把图中的操作种类减少为更小的“原语”操作核心集，因此得名 PrimTorch。

作为背景，PyTorch 完整 API 中有数千个（> 2,000）操作。PrimTorch IR 把它们减少为编译器可以聚焦的更小原语集合。实际中，PrimTorch IR 定义了约 250 个原语操作，例如基本算术、归约、复制和 reshape 等。

许多复杂或高级的 PyTorch aten 操作都可以用 PrimTorch IR 的这些原语分解。例如，x.add_(y) 这样的 PyTorch “原地”操作会降低为函数式 add，再显式复制回 x 的存储，如下所示：

```
%z    = aten::add(x, y)
%copy = aten::copy_(x, %z)    # writes z' s data into x
```

此时，IR 中包含一个独立的 aten::copy_ 节点，而不是特殊的原地变更。这使所有 tensor 更新都显式化，并把变更视为普通复制操作，从而简化下游编译器 kernel。

具体来说，把原地变更转换为函数式操作加独立的 copy 节点后，编译器不再需要推理别名或隐藏副作用。因此，融合过程和内存规划算法可以在纯函数式数据流上运行，从而实现更激进的 kernel 融合和更可预测的高性能代码生成。

PrimTorch IR 还帮助消除图中的别名和变更。它会尽量把操作转换为不执行原地更新的形式，因为原地更新会使优化复杂化。PrimTorch IR 过程的输出是一个只包含 aten IR 和 PrimTorch IR 操作的 FX Graph，随后该 FX Graph 就可由后端进行 lowering。

通过这种 IR 标准化，PrimTorch IR 为编译器后端提供了稳定且简化的目标接口。TorchInductor 等后端不必支持数千个 PyTorch 操作，只需支持这 250 个原语；其他高层操作都由原语派生。这大幅降低了复杂度。

而且，由于大多数高层操作（例如 2,000+ 个 PyTorch 操作）都能分解为已有原语，PrimTorch IR 原语集合的演化相对缓慢，即使 PyTorch 为适应新算法、模型和技术而添加新操作也是如此。

> **注** 稳定的 PrimTorch IR 接口意味着，例如要支持一种新加速器，开发者只需实现这 250 个原语，而不必实现数千个 ATen 操作。

到这里总结一下流水线：TorchDynamo 捕获图，AOT Autograd 添加反向传播并形成前向-反向联合图，然后 PrimTorch IR 对操作进行规范化，得到更精简的原语集合。此时，整个计算（包括前向和反向传播）已经以核心操作表示为相当标准、与设备无关的形式。接下来转向编译器后端执行的实际代码生成阶段。

14.1.8 TorchInductor 后端代码生成

torch.compile 栈的最后阶段是编译器后端。TorchInductor（或简称 Inductor）是 PyTorch 的默认编译器后端。Inductor 接收由 aten + PrimTorch IR 操作组成的优化后前向/反向联合 FX Graph，并为包括 NVIDIA GPU、AMD GPU、CPU 等在内的目标硬件生成高性能代码。

> **注** XLA 是面向非 CUDA 硬件的替代后端，主要通过 OpenXLA 项目用于 Google Cloud TPU，但也能支持采用 XLA IR 的其他加速器。例如，Meta 的内部推理 ASIC——Meta Training & Inference Accelerator（MTIA）——使用 XLA。此外，AWS 的定制 Inferentia 和 Trainium 加速芯片通过开源 AWS Neuron SDK 使用带 XLA 编译器的 PyTorch。不过，NVIDIA GPU 通常使用 TorchInductor。

TorchInductor 的工作方式是把图 lowering 为一种按循环级别、定义即运行（define-by-run）的 IR，并将其编译成高效代码。在内部，Inductor 把操作表示为遍历多维数据的循环。在可能时，它会自动把 FX Graph 中的节点分组为融合循环块；代码生成时每个分组都会成为一个 kernel。

Inductor 还支持符号形状，从而允许动态维度。相比原始 CUDA，这种 IR 处在更高层次，例如它把逐元素计算表示为遍历 tensor 索引的循环。该 IR 可由用户检查，有助于调试甚至扩展。

TorchInductor 的 IR 还可用于通过 torch.export() 和 AOTInductor 执行提前编译流程。AOTInductor 会编译 torch.export 产生的制品，用于打包和部署等 AOT 使用场景，从而可以跨运行保存和复用编译后的代码。图 14-4 在 torch.compile() 的上下文中强调了导出流程。

对于 NVIDIA GPU 后端，TorchInductor 使用 OpenAI 的 Triton JIT 编译器生成实际的 GPU kernel。Triton 是一种用 Python 编写、类似 CUDA 的领域专用语言（DSL），同时也包含该 DSL 的编译器（稍后详述 Triton）。

TorchInductor 将循环级 IR 翻译成 Triton 代码，再使用 Triton 编译器通过 LLVM 将 Triton 代码直接转换为 NVIDIA PTX。请记住，PTX 是 NVIDIA GPU 的低级指令集架构（ISA）。

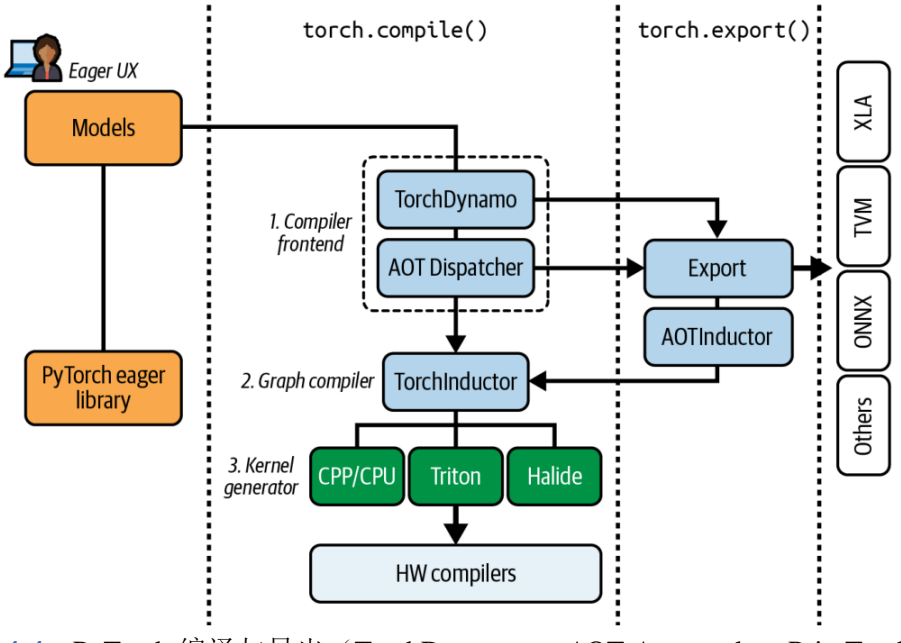

图 14-4　PyTorch 编译与导出（TorchDynamo → AOT Autograd → PrimTorch IR → TorchInductor → Triton/LLVM NVPTX）；通过 torch.export/AOTInductor 导出；形状静态时使用 CUDA Graphs。

> 注　重要的是，Triton 通过 LLVM NVPTX lowering 到 NVIDIA PTX，并不调用 NVCC 编译 kernel。这种方式让 TorchInductor 能够动态生成定制 kernel，使其针对具体模型或算法进行调整。

循环级 IR 用 Python 实现，因此易于检查和扩展。例如，假设图中有操作 z = x.permute(1,0) + x[2,:]。Inductor 可能用下面的 IR 表示该操作：

```
def inner_fn(index: List[sympy.Expr]):
    i1, i0 = index  # index variables for dims
    tmp0 = ops.load("x", i1 + i0*size1)         # x[i1, i0]
    tmp1 = ops.load("x", 2*size1 + i0)          # x[2, i0]
    return ops.add(tmp0, tmp1)                 # elementwise add
torchinductor.ir.Pointwise(
    device=torch.device("cuda"), dtype=torch.float32,
    inner_fn=inner_fn, ranges=[size0, size1]
)
```

这里，size0 和 size1 是输入 x 的维度，inner_fn 描述如何计算输出的一个元素。Pointwise 节点表示遍历这些范围的循环嵌套，并逐元素应用 inner_fn 来产生输出。

这是一种定义即运行风格的 IR。运行该 IR 时，实际执行的是遍历并调用 ops.load + ops.add 的 Python 代码。随后 Inductor 使用 Triton JIT 编译器和 LLVM 生成对应的 NVIDIA PTX 代码。

使用 torch.library.wrap_triton 和 triton_op 将 Triton kernel 注册为具有 autograd 和 fake-tensor 支持的一等 PyTorch 操作。这意味着可以编写 Triton kernel，并让 TorchInductor 将其作为模型图的一部分进行优化。

14.1.9 TorchInductor 自动调优

TorchInductor 包含一个基于 Triton 自动调优能力的 autotuner，下一节会介绍。autotuner 会为每个生成的 GPU kernel 找到最佳启动配置。调优后的配置会按 kernel 缓存，因此后续运行无需重复调优。

首次使用 TorchInductor 后端编译代码时，它会额外花时间，使用不同的 block size、tile size 等对不同 kernel 变体进行基准测试。Inductor 选择最快的变体并在后续使用。kernel 自动调优会增加初始编译延迟，但生成的 kernel 在运行时高度优化。

如果还记得上一章，这种激进的自动调优称为 max-autotune 编译模式。这是最耗时的编译模式，而这正是其内部发生的事情。

除了 kernel 融合和自动调优，TorchInductor 还执行许多低级优化，包括索引简化，以减少生成代码中的复杂索引运算，

公共子表达式消除，以及高效的内存规划，用于复用缓冲区并减少分配。

TorchInductor 还使用 CUDA Graphs 在运行时捕获 kernel 序列，以便更快地重放图，同时将 CPU 开销降至最低。默认情况下，Inductor 会尝试把生成的 kernel 包装进 CUDA Graph，以减少每次迭代的启动开销。这对推理尤其有益，也适用于包含大量 kernel 的任意代码或模型。

> **注** 第 13 章介绍的 reduce-overhead 和 max-autotune 编译模式会触发 CUDA Graphs。不过，CUDA Graphs 要求形状静态，因此启用动态形状编译时不会使用 CUDA Graphs。换句话说，如果以 dynamic=True 启用动态形状，TorchInductor 不会使用 CUDA Graphs。需要自动调优但不需要 CUDA Graph 捕获时，可以使用 max-autotune-no-cudagraphs。

> **注** 通常先从默认模式开始；对于大型/关键工作负载，再使用 max-autotune 以获得额外加速，但要承受显著的编译时间。较小模型可能不会获得太多收益。

TorchInductor 的最终结果是针对工作负载高度优化的设备特定代码。很多情况下，Inductor 的性能接近甚至超过手工调优库。例如，Inductor 可以把包括激活和逐点变换在内的整段逐元素操作融合到单个 kernel 中；还可以把矩阵乘法后接偏置加法和激活等逐元素操作的某些模式融合为一次启动。手工完成这件事相当困难，而且需要持续维护。

PyTorch 编译器使用启发式方法和后端，将某些操作（例如大型 GEMM）路由到 cuBLAS/cuBLASLt 或 CUTLASS 等高性能库，也可以为可融合模式生成 Triton kernel。实际中，TorchInductor 会选择并缓存性能最佳或已知对给定形状最优的路径。

对于 Transformer 模型，TorchInductor 会融合大型 GEMM 周围的 layernorm、残差连接等逐元素操作，同时仍使用 cuBLAS 执行实际 GEMM。对于内存访问不规则的模型，Inductor 的定制 Triton kernel 可能通过只执行所需工作而超过现有库 kernel；这类模型通常无法从 cuBLAS 或 CUTLASS 这样的通用库获得同等收益。

在现代 GPU 上，PyTorch 编译器可以针对某些 Transformer block 和层与 NVIDIA Transformer Engine（TE）协同工作。不过，调用 torch.compile 时 PyTorch 不会自动替换为 NVIDIA TE kernel。TE 是必须通过其模块或融合操作显式使用的独立库；但调用 TE API 后，torch.compile 可以在其周围进行编译和融合。这与生成的 Triton kernel 互为补充，以提供最高性能。

> **注** 只有在计划直接调用 TE 模块（例如 transformer_engine.pytorch.layers）时，才需要安装 NVIDIA Transformer Engine。torch.compile 不会把普通 PyTorch 模型自动替换成 TE kernel。

本质上，TorchInductor 承担了把代码和模型转换为高性能 GPU kernel 的繁重工作。随着每个 PyTorch 版本发布，硬件覆盖面不断扩大，新的优化技术也在出现。例如，PyTorch 提供了 FlexAttention，这是一种新的 attention 操作，TorchInductor 可以把它编译为接近 FlashAttention 性能的融合 kernel。具体而言，FlexAttention 的融合 kernel 在前向和反向传播中测得可达到现代 FlashAttention 性能的约 85–90%，同时提供更大灵活性，包括 block 稀疏和自定义 mask。要启用快速路径，请将以下 torch.backend.cuda 属性设为 True：

```
# Ensure SDPA fast paths are enabled
torch.backends.cuda.enable_flash_sdp(True)
torch.backends.cuda.enable_math_sdp(True)
torch.backends.cuda.enable_mem_efficient_sdp(True)
```

使用 torch.nn.attention.flex_attention 时，确保输入满足快速路径约束。这样 TorchInductor 才能生成融合的 Triton kernel。

TorchInductor 和 Triton 支持在现代 GPU 上自动执行 warp 专门化。编译器会在判断有益时选择性启用它。可以使用 num_consumer_groups 和 num_buffers_warp_spec 等 Triton 元参数调整 warp 专门化。这些优化还能进一步提高 GEMM 吞吐量。Triton 的自动 warp 专门化支持现代 GPU 目标（包括 Blackwell，例如 tcgen05）的 TMA 和 tensor descriptor API。

> **注** 建议使用基于 descriptor 的 tiled load/store，以映射到 TMA 并降低寄存器压力。相比手动 tl.load 循环，这种方法更受推荐。

简而言之，许多模型通过 torch.compile 运行会显著加速，但具体收益取决于模型特征。首次运行 torch.compile 时需要支付编译和自动调优成本；后续运行使用缓存的图和 kernel，执行速度会非常快。

14.1.10 动态形状和可变序列长度

LLM 训练和推理的一项主要挑战是输入序列大小可变。在传统编译器和加速器中，变化的形状通常会导致重新编译，或者要求把输入填充到固定的公共大小。本节讨论 torch.compile 中的动态形状追踪，以及它如何处理可变长度序列。

幸运的是，PyTorch 编译器栈设计为能够优雅处理动态形状。具体来说，它使用 SymPy 库以符号方式表示未知维度，使模型能够接受不同输入大小而不必每次都重新编译，稍后会详细介绍。

PyTorch 编译器会在观测到维度大小变化时自动将其标记为动态。TorchInductor 从静态假设开始，在检测到形状变化后重新编译并泛化。通常第一个新形状只会多进行一次编译。预先设置 dynamic=True 会迫使编译器从一开始就把所有维度视为动态。不过要记住，dynamic=True 会禁用 CUDA Graphs。最好只使用 torch._dynamo.mark_dynamic() 标记已知会变化的维度。

追踪动态维度时，TorchDynamo 和 TorchInductor 会插入类似 guard 的 sequence_length <= 256（或指定的其他范围），以生成适用于多种大小和范围的代码。例如，如果输出大小为 x.size(0) + y.size(0)，Inductor 可以把它表示为符号表达式，并确保生成的代码适用于满足 guard 条件的任意值。

当 Dynamo 遇到新的 sequence_length 形状时，会设置新的 guard，例如 sequence_length <= 1024，并在这一新假设下编译 kernel——从这一点起把该维度当作动态维度。之后如果出现更长序列、违反该 guard，编译器会重新编译一个能处理更大范围的图版本。随着时间推移，它会为不同形状范围建立编译 kernel 缓存。

也可以使用 torch._dynamo.mark_dynamic(tensor, dim) 手动标记 tensor 上预期为动态的维度，从而提前避免重新编译。还可以使用 torch.compiler.set_stance() 调整如何处理重新编译。例如，可以使用 eager-on-recompile 姿态，在超过一定次数重新编译后回退到 eager 模式。稍后会讨论避免重新编译的最佳实践。

TorchInductor 会在第一次重新编译后尝试泛化形状，而不是对每个新形状反复特化。例如，它会在生成的 kernel 内发出带 if 语句的条件代码，使单个 kernel 能在一系列 sequence_lengths 上工作而不会报错。这减少了每个大小都单独编译的需求。

> **注** 某些输出 rank 依赖数据的操作，或极其复杂的索引，仍可能触发形状特化。此时编译器会插入更多 guard；如果这些 guard 经常被违反，可能会通过 mark_dynamic() 或 set_stance() 看到频繁重新编译。

作为背景，不支持动态形状时处理可变长度序列的一种简单但低效方法，是把 batch 中所有输入序列填充到最大长度。这样可以对所有输入使用一次静态计算。实现虽简单，但当输入长度差异很大时效率低下，因为大量计算浪费在无意义的 padding token 上。

当最大长度远大于所有输入的平均长度时，padding 会损害 GPU 利用率。而使用动态形状编译时，可以让编译器只遍历每个输入的实际序列长度。动态形状使你能避免可变长度带来的过度 padding。

看一个典型的基于文本的生成式 AI 场景：随着生成推进，序列长度持续增长。使用动态形状编译，即使序列长度增加，也能持续优于 eager 执行。

相反，如果为了使用静态形状而把所有内容填充到二的幂次长度，就会引入大量无效计算，并因 tensor 更大而增加编译时间。换言之，动态形状同时带来更好的编译时和运行时性能，并且更易使用，因为不必手动填充输入。

> **注** 建议按大小对输入分桶，以限制不同形状的数量；剩余变化则交给动态形状处理。这种混合方法既能避免过度重新编译，又能减少 padding 浪费。

使用动态形状时，可以编译一次，再将同一个编译模型用于不同形状的输入。只要变化处于支持范围内，一个编译模型就能处理多种配置。

内部而言，TorchInductor 使用 SymPy 以符号方式表示动态维度。它会把这些符号传播到 IR 中，因此形如

z.size(0) = x.size(0) + y.size(0) 的表达式可以用符号方式处理。Inductor 会把条件归约为 guard 表达式。

如果 guard 失败（因为维度超出预期范围，或依赖数据的条件发生变化），Inductor 就会触发重新编译。本质上，TorchInductor 会尝试为一系列大小编译通用 kernel，而不是只为单个固定大小编译。

近几个版本显著改进了动态形状。不过，如果编译器无法以符号方式处理某些操作，这些操作仍可能迫使形状特化。此时编译器可能插入更多 guard；如果 guard 经常被违反，就可能频繁重新编译，抵消编译收益。

依赖数据的控制流仍会触发特化。动态形状适合可变序列长度，但不适合真正依赖数据的分支。

值得注意的是，截至本文撰写时，CUDA Graph 重放要求形状静态（以及内存地址固定），而且实例化图只支持有限的参数更新。内存地址和 kernel 启动拓扑必须与捕获时兼容。因此，启用动态形状通常会禁用这些区域的图捕获，使编译器无法获得 CUDA Graphs 的性能收益，包括降低 kernel 启动开销。

> **注** 如果指定 reduce-overhead 编译模式，同时设置 dynamic=True，由于你声明形状可以变化，reduce-overhead 的 CUDA Graph 优化不会生效。启用动态形状会改变 guard 和内存规划，从而禁用图捕获。实际使用中，只对稳定形状使用 mode="reduce-overhead" 以获得 CUDA Graphs；对于可变序列长度，优先使用 mode="default" 或 mode="max-autotune-no-cudagraphs"，并在 ±10–20% 范围内进行分桶/填充，以限制重新编译。

建议对系统进行性能分析，判断动态形状是否适合你的使用场景。在某些情况下，填充到固定大小、使用带 CUDA Graphs 的静态形状，反而能通过避免每个唯一长度的重新编译获得更高吞吐量；在另一些情况下，动态形状会更好。

应对不同方法进行性能分析，找出最适合自己的方案。同时要监控内存使用。支持动态形状的代码由于额外的 guard 以及为最大范围生成的通用代码，会占用略多的内存。

一个经验法则是：如果序列长度只变化 10%–20%，固定长度 padding 很可能更有利。

简而言之，动态形状支持意味着面对 LLM 模型常见的可变长度输入时，不必禁用 torch.compile。支持动态形状后，PyTorch 编译器可以跨不同输入大小执行 kernel 融合和其他优化。

14.1.11 禁用 PyTorch 编译器并恢复 eager 模式

如果想完全禁用 torch.compile 而不改动代码（这对性能 A/B 测试和隔离问题很有用），可以使用 @torch.compiler.disable 装饰器禁用某个函数的编译。若需要限定到某个区域，可以将 torch.compiler.set_stance() 用作上下文管理器，强制代码以 eager 模式运行。例如，可以禁用复杂数据加载或一次性初始化逻辑的编译，让编译图专注于计算；对于不适合追踪的代码周围，这也很有用，稍后会介绍。

或者，直接改用 eager 后端：torch.compile(model, backend="eager")。这会让代码恢复为 eager 模式运行，便于调试以及比较编译模式和 eager 模式的正确性/性能结果。

torch.compiler.disable() 和 torch.compiler.set_stance() 是很有价值的逃生口：当某些操作无法与 PyTorch 编译配合，或者出于性能原因不希望它们进入图时，可以使用它们。下面转向通过 PyTorch 编译器日志改进编译图和代码性能的方法。

14.1.12 性能提示与生成代码调试

另一个极其有用的日志选项是 TORCH_LOGS="perf_hints"。这些日志会显示错失的性能优化机会。例如，某种模式无法融合，或无法使用 CUDA Graph 时，它会记录类似“PerfHint: CUDA Graph not used because input is mutated”或“PerfHint: fell back to eager for random op”的提示。这些提示可以帮助你判断限制代码或模型性能的因素。

要进行更深入的性能调试和调优，通常需要查看 TorchInductor 生成的确切代码。有几种方法可以检查代码。首先，可以设置 TORCH_LOGS="output_code"，打印每个

编译图生成的原始源代码。这甚至允许你修改源代码并进一步优化（如有需要）。

还可以设置 TORCH_COMPILE_DEBUG=1，启用 TorchInductor 的调试模式。启用后运行程序，Inductor 会创建调试目录（例如 /tmp/torchinductor_<pid>/...），其中包含 FX Graph（.fx）、Inductor 制品（如 outputcode.py、fx_graph_runnable.py）、IR dump 和生成的 Triton 源代码。

阅读生成的.triton 代码时，可能会看到 Triton 特有的构造，甚至在高级场景中看到原始 PTX。如果还检查调试制品中的已编译 PTX，可能会看到 mma.sync 指令，此时 tl.dot 已 lowering 为 Tensor Core 操作。这些日志、工具和制品对性能调优非常有用，因为它们能准确显示编译器正在做什么。理解这些信息有助于验证编译器是否应用了 kernel 融合、warp 专门化或双缓冲等优化。如果发现低效之处，可以为具体使用场景手动创建定制 Triton kernel。

> **注** 如果你愿意，也可以把定制 kernel 贡献回 PyTorch 和 Triton 生态，因为其他人很可能也能从你的优化中受益。

14.1.13 调试数值正确性和精度

虽然极少见，但 torch.compile 产生的结果可能在数值上不同于 eager 模式。如果怀疑编译器有 bug，可以先用几种策略验证并收集数据，再通知社区并创建 GitHub issue。

首先，可以使用 PyTorch 的 minifier 工具创建可复现脚本。PyTorch 提供 TorchDynamo minifier 和 TorchInductor minifier，它们会尝试把程序缩减到仍能复现错误的最小版本。若确有需要，这对 PyTorch 团队非常有帮助；问题进入该阶段时，可以把文件附在 GitHub issue 中。

此外，还可以配置 TorchDynamo，在编译器栈的每一层调试数值精度。为了判断数值差异在哪一阶段引入，可以在编译期间设置以下环境变量，把 eager 模式与不同编译阶段进行比较，并定位问题究竟来自 TorchDynamo、AOT Autograd 还是 TorchInductor：

```
# Dump the outputs after each compilation stage
TORCHDYNAMO_REPRO_AFTER="aot"
TORCHDYNAMO_REPRO_LEVEL=4
```

这些设置会让 TorchDynamo 在每个阶段之后 dump 图，并以 eager 模式运行每个图进行比较。这有助于定位引入错误的阶段。

具体来说，设置 TORCHDYNAMO_REPRO_AFTER=”aot” 会让 TorchDynamo 在 AOT Autograd 阶段之后 dump FX Graph，并触发生成复现错误脚本的逻辑；这不同于在最初的 Dynamo 捕获之后生成复现脚本。

使用 TORCHDYNAMO_REPRO_LEVEL=4 时，TorchDynamo 会在 eager 模式下运行每个 dump 的图，并将其输出与编译版本比较。一旦检测到数值不匹配，就会停止并保存一个最小复现脚本。

> **注** PyTorch 编译器团队很重视修复正确性 bug，因此如果发现真正的错误，请在 GitHub 上报告。通过设置 TORCHDYNAMO_REPRO_AFTER="aot" 和 TORCHDYNAMO_REPRO_LEVEL=4，务必附上精简后的可复现制品。

如果使用随机数（种子）或序列，应确保它们以一致方式生成。默认情况下，TorchInductor 可能不会生成与 eager 模式完全相同的随机种子或序列。原因之一是，融合或重新排序的 kernel 可能不会以与 eager 模式相同的预期顺序生成数字。

如有需要，可以设置 torch._inductor.config.fallback_random=True，强制 TorchInductor 生成与 eager 模式完全相同的随机数。这会带来轻微性能损失，但使用 PyTorch 编译器时，为了数值正确性可能必须这样做。

数值差异也可能源于浮点精度。例如，如果使用 PyTorch 自动混合精度（AMP）或 BF16，融合 kernel 中的操作顺序可能相对于 eager 模式下未融合的序列引入轻微数值差异。

虽然这种差异很少影响收敛，但某些情况下确实会影响。如果怀疑与精度相关的不稳定性，可以禁用 torch.compile 并以完整 FP32 运行模型来隔离问题。还可以使用 torch.set_float32_matmul_precision("highest") 控制 TF32 的使用，以及完整 FP32 矩阵乘法的精度-性能权衡，获取最高数值精度。

还要理解，混合精度（例如 FP16/BF16）可能产生小的差异。可以通过设置 torch.use_deterministic_algorithms(True) 强制确定性行为。如果使用了非确定性操作，PyTorch 会抛出错误。虽然 torch.compile 的设计确实减少了一些不确定性来源，但在调试期间启用此标志仍是良好实践。

不过请注意，并非所有操作都有确定性实现。例如，依赖 cuBLAS 的默认 torch.matmul() 操作就没有确定性实现。

具体来说，cuBLAS 实现依赖 split-K 等并行优化，这些优化可能以不同顺序执行操作，导致不同运行之间的浮点结果无法按位复现。

因此，启用该设置可能导致代码失败，除非存在可用的回退替代方案。要让依赖 cuBLAS 的操作（例如 torch.matmul()）完全遵循确定性算法，需要调用 torch.use_deterministic_algorithms(True)，并将 CUBLAS_WORKSPACE_CONFIG 设置为固定大小，如下所示：

```
# Set this before starting the Python/PyTorch process
export CUBLAS_WORKSPACE_CONFIG=:4096:8   # or :16:8
# Use this with the PyTorch process
torch.use_deterministic_algorithms(True)
```

这里，第一个值（例如 4096 或 16）选择 cuBLAS workspace 缓冲区的大小（以字节计，并四舍五入到内部 bucket）；第二个值（例如 8）选择预留多少个这样的缓冲区。按文档设置为:4096:8 或:16:8，即可强制使用确定性算法。

要在 torch.use_deterministic_algorithms(True) 下强制 cuBLAS 使用确定性算法，请将 CUBLAS_WORKSPACE_CONFIG 设置为文档支持的值，如:4096:8 或:16:8。如果强制确定性却不设置此变量，PyTorch 会在某些操作本应选择非确定性 cuBLAS 算法时于运行时抛出错误。

注 务必在实际硬件和模型配置上测试确定性，以确认可复现性。

此外，对于关键工作负载，可以暂时禁用某些编译器优化，例如设置 torch._inductor.config.triton.cudagraphs=False，以便更好地隔离差异原因。这会禁用 TorchInductor 生成的 Triton kernel 的 CUDA Graph 捕获。

调试 PyTorch 编译器优化需要略有不同的思维方式，因为除了日志和图可视化，还要观察元层面的执行步骤以及生成的低级代码。torch._dynamo.explain() 等工具会高层次概览代码如何转换为图、图断裂和子图；各种 TORCH_LOGS 选项则让你查看编译器作出的决策以及它生成的确切代码。

简而言之，结合这些工具和调试机制，可以迭代消除图断裂，确保模型和代码被完整捕获并优化。收益值得付出：编译良好的模型可以显著超过 eager 执行，尤其是大型 LLM 架构，因为每一点性能提升都会累积。

14.1.14 解释和最小化图断裂

使用 torch.compile 时，诊断性能和正确性需要专门工具。本节将展示如何使用各种工具和最佳实践，调试并定位过多的图断裂，包括 torch._dynamo.explain()、记录编译器决策的环境变量，以及调试捕获图和其生成 kernel 的最佳实践。

14.1.15 图断裂与 TorchDynamo explain()

当 TorchDynamo 无法继续把连续操作序列捕获到单个图中时，就会发生图断裂。此时，对于这部分代码，它会回退到 eager 执行。

图断裂是性能的敌人。每次断裂都会提前截断优化图，并引入更多 Python 开销。如果编译模型后只看到有限加速，可能就是频繁图断裂阻止了形成大而融合的图。理想情况下，断裂应尽可能少——最好整个模型或整个训练步骤只有一个大图。

涉及集合通信（例如 all-gather、reduce-scatter 等）的复杂图通常需要图断裂。图 14-5 展示了 PyTorch FSDP 策略中由集合通信造成的图断裂。

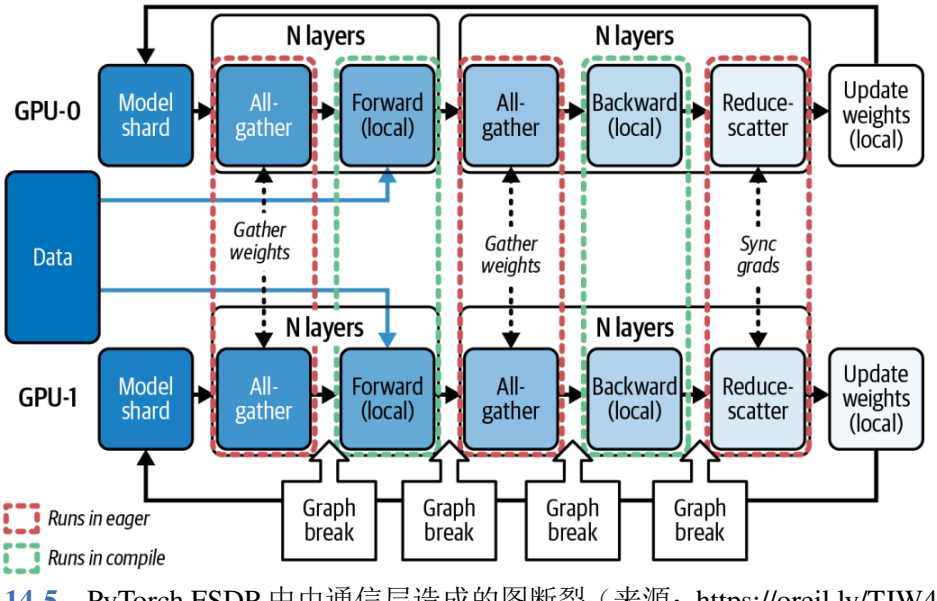

图 14-5　PyTorch FSDP 中由通信层造成的图断裂（来源：https://oreil.ly/TJW42）

PyTorch 提供 torch._dynamo.explain() 来分析和调试图断裂。使用模型和示例输入调用该调试函数时，它会在 TorchDynamo 中运行模型，并返回生成了多少个图、断裂发生在哪里以及发生原因的报告，如下所示；随后还会给出详细的图断裂分析和解释：

```
import torch._dynamo as dynamo
def toy_example(a, b):
    x = a / (torch.abs(a) + 1)
    print("woo")          # a print statement in the model
    if b.sum() < 0:       # dynamic control flow depending on data
        b = -b
    return x * b
explanation = dynamo.explain(toy_example)(torch.randn(10),
↪  torch.randn(10))
print(explanation)
Graph Count: 3
Graph Break Count: 2
Op Count: 5
Break Reasons:
  Break Reason 1:
    Reason: builtin: print [...ConstantVariable] False
    User Stack:
      <frame at toy_example: line 3, in toy_example>
  Break Reason 2:
    Reason: generic_jump TensorVariable()
    User Stack:
      <frame at toy_example: line 5, in toy_example>
Ops per Graph:
  ...
```

这里的解释显示，TorchDynamo 在两次图断裂处分割出三个图段。输出中的“User Stack”部分指向发生问题的具体代码行，这对定位造成图断裂的代码非常有用。

第一次断裂由第 3 行附近的 print(”woo”) 引起。由于 print() 具有向标准输出写文本的“副作用”，无法被捕获。因此 Dynamo 把图拆为 print() 前后的两个图。

第二次图断裂由第 5 行附近的动态控制流逻辑 if b.sum() < 0: 引起。在这一具体场景中，Dynamo 无法把依赖数据的动态控制流逻辑放入单个图；上一节已提到这是其限制。

如果没有获得预期的 PyTorch 编译器性能，使用具有代表性的输入对模型调用 dynamo.explain() 是首先应做的事情之一。它能快速概览生成了多少个图，以及为何无法只生成一个大图。

理解原因后，就可以逐一重构代码以解决图断裂。在前面的例子中，可以移除 print()，或者用 if not torch._dynamo.is_compiling() 这样的 guard 包裹它，避免在追踪期间执行，如下所示：

```
import torch
def model(a, b):
    x = a / (torch.abs(a) + 1)
    # avoid during compiling/tracing
    if not torch._dynamo.is_compiling():
        print("do not print during tracing/compiling")
    if b.sum() < 0:
        b = -b
    return x * b
explanation = dynamo.explain(model)(torch.randn(10),
  torch.randn(10))
print(explanation)
```

如前所述，如果模型确实需要依赖数据的分支，可以用 torch.cond() 包裹，使“真”和“假”两个分支都作为图子程序被捕获，如下所示：

```
import torch
def model_cond(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    # Compute x as before
    x = a / (torch.abs(a) + 1)
    # Retain the compile-time check as a
    #   Python-level guard
    # Avoid side-effects during tracing/compilation
    if not torch._dynamo.is_compiling():
        print("do not print during tracing/compiling")
    # Handle the data-dependent sign flip on b
    b = torch.cond(
        b.sum() < 0,  # predicate (0-dim bool tensor)
```

```
        lambda b: -b, # true_fn: flip sign
        lambda b: b,  # false_fn: leave unchanged
        (b,)          # operands tuple
    )
    return x * b
# Generate and print the Dynamo explanation just like before
explanation = dynamo.explain(model_cond)(torch.randn(10),
 ↪ torch.randn(10))
print(explanation)
```

这里，谓词 b.sum() < 0 必须是 Python bool 或一个元素的 torch.bool tensor。true_fn 和 false_fn 是接收相同 operands（此处只有 (b,)）并返回相同形状、相同 dtype tensor 的可调用对象。

这段代码保留了 Dynamo 的编译时检查（dynamo.is_compiling()），将其作为 Python if，因为它不依赖运行时数据，而且我们希望在追踪期间避免副作用（例如 print）。

请注意，torch.cond() 目前只接受 tensor 谓词，要求两个分支拥有相同输入并返回形状和 dtype 完全相同的单个 tensor，而且不允许原地变更或任意副作用。

相比之下，可以使用前面介绍的纯 tensor mask 方法 torch.where()。它没有这些限制，也能避免图断裂；当不需要 torch.cond() 的全部表达能力时，这是更简单、更可靠的选择。代码如下：

```
import torch
import torch._dynamo as dynamo
def model_where(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    # Compute x as before
    x = a / (torch.abs(a) + 1)
    # Preserve compile-time guard to avoid side-effects during tracing
    if not torch._dynamo.is_compiling():
    print("do not print during tracing/compiling")
    # Data-dependent branch expressed using torch.where
    b = torch.where(
        b.sum() < 0,  # predicate: a 0-dim bool tensor
        -b,           # true branch: flip sign
```

```
        b             # false branch: unchanged
    )
    return x * b
# Display the Dynamo explanation just as before
explanation = dynamo.explain(model_where)(torch.randn(10),
↪  torch.randn(10))
print(explanation)
```

这里，torch.where(condition, input, other) 会在 condition 为 True 时从 input 选择元素，在 False 时从 other 选择元素。由于 b.sum() < 0 产生一个 0 维 Boolean tensor，它可以广播到 b 的所有元素。这就实现了单个向量化的符号翻转，而不是逐元素的 Python if。

> **注** 使用 torch.where() 可以避免编译和追踪流水线中的图断裂，使 TorchDynamo 能够优化并内联操作。

还可以使用 torch.compiler.set_stance("fail_on_recompile") 强制报错：如果代码无法干净地捕获为完整图，就拒绝运行。这在开发阶段很有用，因为它能在编译时捕获图断裂，而不是静默回退到较慢的 PyTorch eager 执行。

> **注** torch.compiler.set_stance("fail_on_recompile") 也适合加入 CI 构建，以捕获后续开发过程中引入的图断裂。在项目整个生命周期中，稳健且持续的性能回归测试非常重要。

14.1.16 最小化图重新编译

除了图断裂，还应监控重新编译的次数。如果 guard 持续使输入 tensor 的形状等条件失效，TorchDynamo 可能会多次编译图。运行时 tensor 形状变化时，guard 失败并触发重新编译。如果看到的重新编译多于预期，应调查是哪一个 guard（形状、dtype 等）导致，并解决问题。

通常可以通过观察迭代在初始预热/编译迭代之后仍然很慢，发现重新编译正在发生。幸运的是，可以使用 TORCH_LOGS="graph_breaks,recompiles,guards" 让 PyTorch 记录每次 guard 求值和触发的重新编译。

如果观察到 guard 频繁失败，通常意味着 Python 侧常量（例如随机数种子、时间戳或每次循环变化的值）在每次迭代都改变，不断使 guard 失效并触发重新编译。此时，应确保这些值是静态的，或者使用前面介绍的动态形状 API（例如 torch._dynamo.mark_dynamic）处理，从而避免无谓且过多的重新编译。

根据具体情况，有几种常见机制可以减少图重新编译。首先，对于刚才的常量场景，可以把常量作为 tensor 传入代码块，避免编译器对该值设置 guard 并反复失败。

其次，如前所述，可以使用 torch._dynamo.mark_dynamic(tensor, dim) 标记已知会变化的动态维度，提前避免重新编译。另一种选择是使用 torch.compiler.set_stance("eager_on_recompile")，在达到 N 次重新编译后回退到 eager 模式，从而避免重复编译，有效限制重新编译次数。

另一种选择是显式标记图的那一部分为安全，使用 torch._dynamo.allow_in_graph。下一节进一步介绍这一技术。

14.1.17 使用 allow_in_graph 将函数和代码块标记为安全

当 TorchDynamo 不知道如何处理某个函数或代码块（例如其中使用了不支持的操作）时，可以用 torch._dynamo.allow_in_graph 装饰函数或包装代码。它既可以作为 Python 装饰器，也可以作为上下文管理器，用来告诉 Dynamo 该代码没有副作用。这样，Dynamo 会采用更宽松的分析和接受策略，把代码包含在追踪中。allow_in_graph 会绕过 Dynamo 的部分安全检查，因此应先修复图断裂的根本原因。

这是一项高级功能，应谨慎使用。你实际上是在承诺：对于相同的输入 tensor，该函数是纯函数、始终返回相同的输出 tensor、只依赖其 tensor 输入且没有副作用。如果使用不当，可能静默得到错误结果。不过正确使用时，如果某个函数或代码块明明可以安全追踪却造成图断裂，它可能显著改善性能。

一般而言，应少用 allow_in_graph。它是供高级用户覆盖 Dynamo 保守策略的工具，但只有在绝对确定函数没有会影响代码正确性的副作用或隐藏状态时才使用。

14.1.18 处理图断裂的技巧

图断裂限制编译器执行大规模优化（例如把许多 kernel 融合为更少、更高效的 kernel）的能力，使 PyTorch 不得不对图的某些部分回退到更慢的 eager 执行。

理解什么会触发图断裂以及如何避免它至关重要。以下是图断裂的一些常见原因及减少断裂的技巧：

14.1.19 避免原地操作和意外变更

TorchDynamo 可以通过称为函数式化（functionalization）的机制处理某些变更，该机制会把原地操作转换为追踪所需的非原地形式。但某些原地操作仍可能造成图断裂。如果看到关于 mutation 的断裂原因（例如“mutation on data”或“modifying a global”），应重写相应部分，避免原地操作。通常只要把原地的 x.relu_() 改写为非原地的 x = x.relu()，就能在原地操作导致问题时避免图断裂。

优先使用 PyTorch 的数据结构、集合和 tensor 操作，而不是等价的 Python 实现。

在函数内部把 tensor 追加到 Python list 中会使 TorchDynamo 困惑，因为它对不断增长的 list 追踪得不好。可以预分配 tensor，或使用 torch.stack() 等 tensor 操作，而不是动态构造 Python（非 PyTorch）list。许多 Python 库调用（包括 I/O 操作、print、logging 和 math.* 函数）很可能造成图断裂。建议将它们从性能关键路径中移除。

> **注** 只要可能，始终建议使用 Python 数据结构、集合和 tensor 操作对应的 PyTorch 实现。这些实现针对 PyTorch 编译、GPU 处理和分布式数据传输进行了大量优化，而这些正是 PyTorch AI 应用和模型中的常见需求。

14.1.20 尽可能避免依赖数据的控制流

如果使用 if tensor.sum() > 0: 这类逻辑，TorchDynamo 很难追踪，因为编译时不知道条件。它必须根据第一次运行选择一个分支、对该条件设置 guard，并在后续调用中强制执行该 guard；这并不正确，因此 Dynamo 会创建图断裂。

PyTorch 支持 torch.cond() 这一高级操作，以便在图中捕获某些动态流程。它可以封装 if/else 语句，让两个分支都被编译。不过，条件必须是 tensor，并且通常最适合参数依赖的切换，而不是任意 Python 逻辑。

除此之外，大多数依赖数据的控制流仍会断图。只要可能，继续优先使用 tensor 操作（torch.where()、mask 等）。如果 torch.cond() 和重构都不可行，可能只能接受图断裂及其性能影响。

14.1.21 理解 PyTorch DDP 中重叠和同步子图的性能特征

PyTorch 的 DDP 会在同步点（包括 all-reduce bucket）显式断图。你可能在 explain 输出中看到与 allreduce 或 torch.distributed 操作相关的断裂。这是预期行为，因为 PyTorch 可能分别编译每个梯度 bucket 的归约，以便让通信与反向计算重叠。

如果要保留计算-通信重叠，就无法避免 DDP 通信边界处的图断裂。PyTorch 编译器和 DDP 会有意在每个 all-reduce bucket 处插入断裂，使梯度同步发生在子图之间。这样，一个 bucket 的通信就能与下一个 bucket 的反向计算重叠。

虽然这会阻止形成单一的整体图，但能保留性能。TorchDynamo + DDP 的性能与 eager-mode DDP 相近，规模扩大后甚至可以超过 eager DDP。因此，尽管无法消除这些通信图断裂，它们对于实现正确且高效的分布式训练及适当重叠是必要的。

14.1.22 用 PyTorch FSDP 包装图子模块

PyTorch 通过使用 use_original_params=True 支持编译模式下的 FSDP。最佳实践是把各个 transformer block 等子模块分别包装为 FSDP 子模块。这样 Dynamo 会在每个 FSDP 子模块边界处创建显式图断裂，使每个分片的通信能够与计算重叠，类似前面介绍的 DDP bucketization 策略。

编译后的 FSDP 会使用 AOT Autograd 和 Inductor 的内存规划器融合前向和反向传播，并在模型分片之间复用缓冲区。因此，每个 GPU 上只驻留活动参数切片和最少的中间内存缓冲区。与 DDP 或 eager 模式相比，这会降低峰值内存使用。

节省内存来自避免冗余梯度存储、复用中间分配，以及在分片之间让通信与计算重叠。这些措施使更大的模型能够装入每个 GPU。如果不分别包装子模块，FSDP 会把所有参数当作一个大 bucket。这仍能工作，但会限制内存收益和重叠潜力。因此，建议将 torch.compile 与按模块包装的 FSDP 结合，以获得最大速度和内存效率，尤其适用于大规模训练作业。

如果出现问题，调试可能非常复杂；在更大的集群/配置中则更加复杂。使用 FSDP 和 torch.compile 时，务必先在较小配置上测试。

14.1.23 使用定制及第三方 CUDA C++ 和 Triton 操作时监控性能权衡

如果依赖 PyTorch 不认识的定制或第三方 CUDA 扩展，Dynamo 会创建图断裂，因为它无法推理该操作做什么或是否安全。如果该操作位于性能关键路径，可以考虑用 Triton 在 Python 中重写它。

PyTorch 支持 torch.library.triton_op() API，让你能够把 Triton kernel 作为自定义操作无缝集成到 PyTorch 中。这样编译器可以查看 Triton 代码并进行优化。在深入 Triton 之前，先快速总结如何调试不同编译阶段、图断裂和编译器性能。

现在许多流行的第三方库都为其操作提供 Triton 实现或 Dynamo/FX 包装器。在自行编写之前，应先检查是否已有这些实现。

14.2 调试编译器阶段、图断裂与性能

可以通过设置 TORCH_LOGS、TORCH_COMPILE_DEBUG 和 TORCHDYNAMO_REPRO_* 等环境变量，在运行时记录和调试不同类型的编译器事件。这些事件包括图断裂、重新编译、guard 以及其他编译器决策。下面给出设置 TORCH_LOGS 的例子（常见值见表 14-1）：

```
# "graph_breaks", "dynamo", "aot_graphs", "inductor",
# "graph_outputs", "graph_code", "dynamic", "perf_hints",
# "output_code", "recompiles", "guards", etc.
TORCH_LOGS="graph_breaks" python train.py
```

这会让 PyTorch 在每次发生图断裂时打印信息。概括不同的日志选项，可以把 TORCH_LOGS 设置为以下值，以调试 torch.compile 的各个方面，包括不同阶段（TorchDynamo、AOT Autograd 和 TorchInductor）、图、图断裂、生成代码、性能、重新编译和 guard，以及编译器决策和性能，如表 14-1 所示。

表 14-1 torch.compile 的日志选项。

TORCH_LOGS value	描述
graph_breaks	记录图断裂事件
dynamo	TorchDynamo 的详细日志
aot_graphs	AOT Autograd 的详细日志
inductor	TorchInductor 的详细日志
graph_outputs	显示已编译的 FX Graph
graph_code	dump TorchDynamo 生成的每个 FX Graph 的 Python 代码
dynamic	记录动态形状相关决策，以及何时将维度标记为动态
perf_hints	显示错失的性能优化机会
output_code	打印每个编译图生成的代码
recompiles	记录重新编译触发事件
guards	记录 guard 及 guard 求值

如果怀疑子图的分段方式或被编译的形状有问题，这些设置会很有用。使用它们可以获得大量内部调试信息，而不必更改代码。

> **注** 请准备好面对非常冗长的输出。若只是调试图断裂，建议先只使用”graph_breaks”。

在底层，设置 TORCH_LOGS 与使用 torch._logging.set_logs() API 类似。不过，TORCH_LOGS 作为环境变量有时更容易在外部配置。

还要记住，可以设置 TORCH_COMPILE_DEBUG=1 以启用 TorchInductor 调试模式。这会记录 FX Graph、TorchInductor IR、生成的 Triton 代码，以及在安装 Graphviz 时包含可视化的 HTML 报告。

还可以设置 TORCHDYNAMO_REPRO_AFTER 和 TORCHDYNAMO_REPRO_LEVEL，让 TorchDynamo 在每个阶段之后 dump 图，并对代码与未编译的 eager 模式版本进行运行时比较。

还可以使用 tlparse 工具追踪编译日志。追踪日志可用于调试编译事件（例如重新编译）以及生成 bug 报告。

要启用追踪日志，需要使用 TORCH_TRACE 环境变量指定 trace-log 目录，然后对该目录运行 tlparse，生成如下所示的栈帧树表示：

```
- /workspace/networks/layers/transformer.py:634 in forward
  .../torch/nn/modules/module.py in _wrapped_call_impl
 .../torch/nn/modules/module.py in _call_impl
  - [2/2] [2/3] ../torch/_dynamo/convert_frame.py in __call__
  - /workspace/networks/layers/transformer.py:753 in forward
    - [8/2] [8/3] .../torch/_dynamo/convert_frame.py in __call__
...
```

此外，还可以使用 Perfetto UI 显示追踪时间线可视化。由于追踪带来的开销很小，甚至可以在生产环境启用 TORCH_TRACE。

现在进一步深入 OpenAI 的 Triton 语言以及 TorchInductor 使用的 Triton 编译器。我们将编写一些基础和高级 Triton kernel，然后把它们注册到 PyTorch。

14.2.1 使用 OpenAI Triton 编写定制 kernel

到目前为止，我们只简要提到过 OpenAI 的开源 Triton 语言和编译器。现在需要深入了解它，因为 TorchInductor 使用 Triton 作为后端代码生成实现，而且在 OpenAI 等大型公司的支持下，Triton 正越来越受欢迎。

如前所述，Inductor 在底层使用 Triton 生成优化的 GPU kernel。通过检查、理解和定制这些 kernel，可以进一步

改善性能，超越 TorchInductor 原本能够生成的结果。在 PyTorch 和 NVIDIA GPU 环境中学习 Triton 对性能优化至关重要。

从高层看，OpenAI Triton 是一种开源、原生 Python 的领域专用语言（DSL），用于用熟悉的 Python 编写 GPU kernel。Triton 还包含一个 JIT/编译器，直接把 Triton 代码转换为 NVIDIA PTX。换言之，Triton 让你可以在 Python 中创建高性能定制 GPU 操作，而无需手写 CUDA C++。Triton 与 PyTorch 保持紧密集成，是这一生态中编写定制 GPU kernel 的首选。

用 Triton 编写 GPU kernel 比 CUDA C++ 更熟悉、更简单。对于偏好留在 Python 中、希望快速迭代而不想处理复杂 C++ 模板或细致内存管理的研究人员来说尤其如此。在 GPU 性能导向的 PyTorch 和 Triton 编译器已经存在的时代，他们确实不需要使用 C++。

NVIDIA 已经注意到这一趋势。2025 年，NVIDIA 宣布了以 Python 为中心的 CUDA 库（例如 cuTile、CuTe Python DSL、CUTLASS Python DSL，以及替代 NumPy 的 cuPyNumeric）。这些库本质上是 Triton 的竞争方案。它们与 torch.compile 的集成仍在发展；截至本文撰写时，TorchInductor 仍以 Triton 作为主要 GPU 代码生成路径。

尽管 PyTorch 的 torch.compile 自动完成了大量 kernel 生成工作，定制 Triton kernel 仍能挤出最后的性能，尤其适用于 TorchInductor 当前范围之外的操作，例如复杂稀疏模式和新颖层类型。有时定制代码可以超过 TorchInductor 生成代码的性能，特别是当你具备领域知识时。不过这属于高级用法，需要持续维护，并可能在支持新硬件时重写。

现在先快速学习 Triton 编程基础，然后深入一些有趣的 Triton 主题，包括访问共享内存、向 PyTorch 注册 Triton kernel、自动调优 kernel 启动参数以及性能分析。随后还会介绍 warp 专门化和软件流水线（例如双缓冲）等高级 Triton 主题。

14.2.2 Triton 编程模型

Triton 使用单程序多数据（SPMD）模型，而不是 CUDA 的单指令多线程（SIMT）模型。这一点很重要，因为 Triton 有意抽象掉 CUDA 指令和线程的低级细节。

Triton kernel（也称 program）在更高层次运行：在独立的线程块（也称 cooperative thread array，CTA）上运行 program 的实例，作为

基本计算单位。这与 CUDA kernel 不同，后者运行在一个线程块中的独立线程上。

社区通常交替使用 Triton kernel 和 Triton program 这两个称呼，但一般更偏好 Triton kernel，因此本书大部分时候使用 Triton kernel。

使用 Triton Python DSL 编写 Triton kernel，然后由 Triton JIT 编译器把 kernel 编译为 GPU 代码，运行许多并行的 kernel 实例。每个 program 实例映射到一个 CUDA 线程块。

Triton kernel（也称 program）通过给 Python 函数添加 @triton.jit 装饰器来定义。在 kernel 中，使用通常别名为 tl 的 triton.language 模块中的特殊原语来操作内存指针、执行向量化 load/store，并通过 tl.program_id 和 block offset 运算计算每个 program 的索引。

Triton 的 SPMD 模型意味着通常使用向量化操作，例如将两个 tl.arange 向量相加。Triton 编译器会把向量化的 SPMD 代码映射到 CUDA block 的线程上，但不保证一个元素对应一个线程。

使用 Triton 不需要显式管理单个线程或 warp，因为编译器会替你完成这些工作。下面是一个简单 Triton kernel，用于将两个大小相同、每个大小为 n_elements 的向量相加：

```
import triton
import triton.language as tl
BLOCK_SIZE = 1024
@triton.jit
def vector_add_kernel(x_ptr,y_ptr,out_ptr,n_elements,BLOCK_SIZE:
↪ tl.constexpr):
    pid = tl.program_id(axis=0)              # unique program ID for
    ↪ each block
    block_start = pid * BLOCK_SIZE
    # each program handles BLOCK_SIZE elements
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    # Create a mask to guard against out-of-bounds
    # (if n is not divisible by BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)        # masked load
    y = tl.load(y_ptr + offsets, mask=mask)
    result = x + y
    tl.store(out_ptr + offsets, result, mask=mask) # masked store
```

这里可以看到，Triton 把线程和 warp 抽象掉了。注意，BLOCK_SIZE 是编译时常量，定义每个 program 实例处理多少个元素，

而每个 CUDA block 的线程数由 kernel 的配置中的 num_warps 控制，并不等于 BLOCK_SIZE。

具体来说，在前面的代码中，tl.arange(0, BLOCK_SIZE) 返回一个大小为 BLOCK_SIZE 的索引向量（[0, 1, ..., BLOCK_SIZE-1]）。我们把 pid * BLOCK_SIZE（即 block_start）加到索引向量上，得到该线程块上这个 kernel 实例的实际索引 x_ptr + offsets 和 y_ptr + offsets。

假设启动足够多的 Triton kernel 实例，覆盖两个向量中的全部 n_elements 个元素，该 kernel 就会将 x_ptr 和 y_ptr 的每个元素相加，并把结果存到 out_ptr。本质上，Triton 让你以张量化方式编写 kernel 逻辑。

例如，这里一次处理整个索引块（offsets）。Triton 编译器负责把工作分配给实际 GPU 线程，并尽可能保证内存访问（tl.load 和 tl.store）合并。

要启动这个 Triton kernel，需要传入一个 grid 函数，根据 meta['BLOCK_SIZE'] 计算 program 实例的数量：

```
import triton
def grid(meta):
    return (triton.cdiv(n_elements, meta['BLOCK_SIZE']),)
vector_add_kernel[grid](x_ptr, y_ptr, out_ptr, n_elements,
↪   BLOCK_SIZE=1024)
```

这里的代码使用 mask，避免当 n_elements 不是 BLOCK_SIZE 的倍数时发生越界内存访问。这类似前面 CUDA 章节中的做法：在 kernel 中使用 if (idx < N) 避免越界索引错误。

在 load/store 中使用 mask，是处理边界条件的一种巧妙而方便的方法，无需显式检查或 if/else 分支。

在底层，Triton 会把这个 program 转换为 NVIDIA PTX，使每个 program 使用一个 CUDA 线程块。每个 program 都映射到一个 CUDA 线程块。tl.arange 产生 program 内按 lane 划分的索引，编译器再把这一向量化索引空间映射到 block

内的线程上。也可以直接管理矩阵操作的多维索引。Triton 会自动为你处理算术和内存操作的向量化。

简而言之，Triton 让你兼得 Python 的生产力和优化 CUDA C++ kernel 的性能；同时还允许下沉到低级优化，操作并利用完整的内存层次结构（例如共享内存 tiling 等），下一节将展示这一点。

14.2.3 在 Triton 中访问共享内存

高效的 Triton kernel 会利用 L2 缓存和每个 SM 上由软件管理的共享内存。使用共享内存时，每个线程块会把矩阵 A 和 B 的一个 tile 加载到共享内存中，而不是让每个线程反复从全局内存加载相同值。

然后 kernel 会在多次计算中复用这些 tile。这样可以更好地利用片上内存缓存，并减少全局内存/HBM 与寄存器之间传输的数据量。

Triton 不提供显式的共享内存分配器。相反，它使用 tensor descriptor（tl.make_tensor_descriptor(...)）在片上共享内存中暂存 tile，并使用具有预期形状和 stride 的异步流水线。这样，就可以在流水化的 tl.range(..., num_stages=...) 内通过 descriptor 发出 load 和 store。这一循环会 lowering 为 cp.async、TMA 和 barrier。

14.2.4 向 PyTorch 注册定制 kernel

写好 Triton kernel 后，可以使用 torch.library.triton_op 将它注册为 PyTorch 中的定制操作。这样，Triton kernel 对 torch.compile 可见，不会被视为只能回退到 eager 模式的不透明黑盒操作。编译器能够识别 Triton kernel，把它纳入图捕获，并与图的其余部分一起优化，从而实现额外的融合等优化。

注册 Triton kernel 有助于在使用 PyTorch 编译器时避免定制 Triton kernel/program 导致的图断裂。下面是把 Triton kernel vector_add_kernel 注册到 PyTorch 并调用的例子：

```
import torch
import triton
import triton.language as tl
from torch.library import triton_op, wrap_triton
from torch import Tensor
# Triton compute kernel
@triton.jit
def vector_add_kernel(
    x_ptr, y_ptr, out_ptr, n_elements,
    BLOCK_SIZE: tl.constexpr
):
    pid = tl.program_id(0)
    start = pid * BLOCK_SIZE
    offsets = start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    tl.store(out_ptr + offsets, x + y, mask=mask)
# Register as a Triton-backed PyTorch op
@triton_op("my_triton_lib::vector_add", mutates_args=())
def vector_add(x: Tensor, y: Tensor) -> Tensor:
    assert x.device.type == "cuda" and y.device.type == "cuda"
    n = x.numel()
    out = torch.empty_like(x)
    # Compute grid size
    def grid_fn(meta):
        return (triton.cdiv(n, meta["BLOCK_SIZE"]),)
    # Wrap and launch the Triton kernel
    wrap_triton(vector_add_kernel)[grid_fn](x, y, out, n,
    ↪ BLOCK_SIZE=1024)
    return out
# Usage
a = torch.randn(10_000, device="cuda")
b = torch.randn(10_000, device="cuda")
c = torch.ops.my_triton_lib.vector_add(a, b)
```

这里，triton_op("my_triton_lib::vector_add", mutates_args=()) 向 PyTorch 注册操作名和变更元数据（空）。然后，wrap_triton(vector_add_kernel) 把原始 Triton kernel 包装成可调用对象，编译器可以在 torch.compile 图中内联和优化它。之后编译器会在整个 torch.compile 图中融合、重排并内联该 kernel。

注册只支持前向的操作很直接。不过，要利用 PyTorch 自动微分实现完整训练支持，通常需要实现并注册定制的反向计算；否则就必须用已有的可微原语组合出它。

要支持训练，可以使用 vector_add.register_autograd(backward, setup_context=setup_context) 注册 autograd 公式。也可以把逻辑封装在 torch.autograd.Function 中，并注册前向和反向参数。但对于 torch.compile 的可组合性，推荐使用 register_autograd。

如果 OpenAI Triton 不支持所需功能，或性能未达到预期，可以使用带 CUTLASS 等库的 CUDA C++ 重写 kernel 以提升效率。随后以类似方式向 PyTorch 注册 CUDA C++ 扩展，包括为反向传播注册 autograd 梯度计算。

14.2.5 调节 kernel 启动参数

对于许多 kernel，Triton program 通常使用 4 个 warp（即每 block 128 个线程）。不过，现代 GPU 每个 SM 具有更大的共享内存和寄存器文件，因此通常可以把 num_warps 提高到每 block 8 或 16 个 warp。例如，当 BLOCK_SIZE >= 2048 时可以把 num_warps 提高到 8，当 BLOCK_SIZE >= 4096 时提高到 16。

warp 数取决于 kernel 能否利用并行性而不造成过度竞争。最优设置取决于 kernel 的算术强度和内存访问模式。

考虑如下启动 kernel：my_kernel[grid](..., num_warps=8)。这里指定每个 Triton kernel 使用 8 个 warp（256 个线程）。该配置通常对计算密集型 kernel 有效；但受内存吞吐量限制时，内存受限 kernel 可能仍在 4 个 warp 左右达到上限。

对于内存受限 kernel，在每个线程块中使用更多 warp 可以通过增加并行度帮助隐藏内存延迟。不过，线程块中的 warp 太多也会造成竞争或缓存抖动。

新一代 GPU 获得了更多 SM 和更宽的内存总线，因此可以把每个 block 的 warp 数从默认的 4 增加到 8 或 16。这有助于提高占用率、隐藏更多内存访问延迟，并使可用内存和计算资源达到饱和。

手动为每个 kernel 探索 BLOCK_SIZE 和 num_warps 的组合可能很繁琐。因此，通常最好使用 Triton 内置的 autotuner。它会基准测试并自动选择最优的 BLOCK_SIZE、num_warps、tile size 和其他参数。下一节介绍 autotuner。

14.2.6 Triton kernel 自动调优

GPU kernel 性能对 tile 维度、warp 数、循环展开阶段，以及寄存器和共享内存等片上资源的使用等编译时参数非常敏感。Triton 内置 autotuner 通过允许你用 @triton.autotune 装饰 triton.jit kernel，自动搜索这些最优设置。你可以传入一个 triton.Config 对象列表，描述不同的

BLOCK_SIZE、num_warps、num_stages、tile size 以及其他 kernel 元参数的候选组合。

第一次调用 kernel 时，Triton JIT 会编译并基准测试每种配置组合。初次调用务必使用具有代表性的输入工作负载，因为 Triton 会根据输入大小/形状等特征生成 key，并为该输入缓存最快配置。

之后，具有相同输入特征的调用都会自动复用缓存中最快的配置。这样，对于每种输入大小/形状只需支付一次自动调优成本，后续 kernel 调用立即享受最优配置。

如果 Triton 检测到新的输入形状，就会根据新的输入特征遍历 triton.Config 对象，再次执行自动调优。它会为该输入重新选择最佳配置，并缓存起来供后续 kernel 调用使用。

为了避免次优调优结果，建议使用接近生产工作负载的现实且具有代表性的输入预热 autotuner。这样，Triton 建立的缓存配置就能很好地反映生产输入。

> **注** 可以通过为 @triton.autotune(key_fn=...) 提供自定义 key_fn，覆盖特定输入形状和工作负载的最优设置。它会把输入元数据（例如 tensor 形状）映射到定制缓存 key。这是一种高级技术，能让你更好地控制不同输入工作负载类型的缓存配置。

选择 kernel 配置时要记住，更大的 tile 和更多 warp 会提高算术强度，但代价是每个线程块消耗更多寄存器和共享内存。换言之，提高计算-内存比会限制占用率，因为资源需求增加后，每个 SM 能执行的线程块变少。

相反，更小的 tile 和更少的 warp 会减少每线程工作量和数据复用，但允许更多 block 和 warp 同时在每个 SM 上处于活动状态。这能提高占用率，但会牺牲算术强度。

简而言之，最优权衡同时取决于输入矩阵维度和 GPU 的具体资源限制。手动调优耗时且容易出错。Triton autotuner 会在现实工作负载上采用数据驱动方法自动处理这一复杂性，确定最优配置，甚至发现手动搜索可能错过的配置。使用更高的 num_warps（例如 8–16）和多阶段流水线

通常会让 Blackwell 上的 tcgen05.* 路径达到饱和。建议尽可能多使用自动调优。

14.2.7 高级 Triton kernel 实现

为巩固这些概念，下面给出几个自包含的 Triton kernel 示例，展示 warp 专门化和数据传输/计算的异步双缓冲。这些例子说明如何使用 Triton 把高级 Python 代码转换为高度优化的 GPU kernel。

14.2.8 使用 Triton 进行 warp 专门化

TorchInductor 可以让许多生成的 GPU kernel 目标化 Triton 的 warp 专门化支持。它会尝试把每个线程块的 warp 分成“生产者”（内存）和“消费者”（计算）角色，发出带 warp_specialize=True 的 tl.range() 循环，类似下面的例子：

```
// warp_specialize=True is supported on modern GPUs
// Use it together with num_stages > 1
// to enable producer/consumer warp partitioning
// and overlap
for k in tl.range(0, K_tiles, _warn_unused=False, warp_specialize=True):
    # loop body
    ...
```

内存 warp 会预取下一个 tile，同时另一个 warp 计算当前 tile。这样可以让内存延迟与计算重叠，从而提高吞吐量。warp 专门化与基于 descriptor 的 TMA copy 协同工作。还可以在自己的 Triton kernel 中使用这一功能，只需像代码中所示，把 warp_specialize=True 传给 tl.range()。

还可以在 Triton autotune 配置中设置 num_consumer_groups>0（例如 2）和 num_buffers_warp_spec（例如 3），通过 Triton.Config 驱动 warp 专门化，如下面的代码片段所示。这样可以让生产者和消费者始终有工作可做。如果提供这些值，TorchInductor 会在底层使用它们：

```
triton.Config(
    { 'BLOCK_M': 128, 'BLOCK_N': 128, 'BLOCK_K': 64,
      'num_warps': 8, 'num_stages': 2,
      'num_consumer_groups': 2, '
      num_buffers_warp_spec': 3 }
)
```

注 这种专门化方法对 GEMM 中沿较大 K 维度迭代的长循环尤其有效。它能让内存子系统和 ALU 始终保持忙碌，从而最大化硬件利用率。

14.2.9 分块和持久化 GEMM kernel（Triton）

这个 Triton kernel 高效计算矩阵乘法（C = A * B），因为每次 kernel 启动都在内部遍历 K 维度完成全部工作，而不是针对每个 K 分块启动多个 kernel。这样只支付一次启动开销，并让 warp 一直忙碌到每个 tile 完成。下面的示例在一次启动内对 K 进行分块，但不会让同一线程块跨多个输出 tile 复用：

```
@triton.jit
def tiled_gemm_kernel(
    A_ptr, B_ptr, C_ptr,
    M, N, K,
    stride_am, stride_ak,
    stride_bk, stride_bn,
    stride_cm, stride_cn,
    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr,
):
    """
    Tiled GEMM with Triton tensor descriptors + autotuning.

    This is the BASIC PRODUCTION example showing:
    1. Tensor descriptors (maps to TMA on Blackwell)
    2. Autotuning across block sizes
    3. Standard 2D grid decomposition
    """
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    m0 = pid_m * BLOCK_M
    n0 = pid_n * BLOCK_N
    offs_m = m0 + tl.arange(0, BLOCK_M)
    offs_n = n0 + tl.arange(0, BLOCK_N)
    offs_k = tl.arange(0, BLOCK_K)
    # On Blackwell, descriptor .load/.store map to TMA
    # tl.dot lowers to UMMA (tcgen05) with accumulators in TMEM.
    A_desc = tl.make_tensor_descriptor(
        A_ptr,
        shape=[M, K],
        strides=[stride_am, stride_ak],
        block_shape=[BLOCK_M, BLOCK_K],
    )
    B_desc = tl.make_tensor_descriptor(
        B_ptr,
        shape=[K, N],
        strides=[stride_bk, stride_bn],
```

```
    block_shape=[BLOCK_K, BLOCK_N],
)
acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
K_tiles = (K + BLOCK_K - 1) // BLOCK_K
if K_tiles == 0:
    c_ptrs = C_ptr + (offs_m[:, None] * stride_cm
                      + offs_n[None, :] * stride_cn)
    c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
    tl.store(c_ptrs, acc, mask=c_mask)
    return
k0 = 0
if (m0 + BLOCK_M <= M) and (k0 + BLOCK_K <= K):
    a_cur = A_desc.load([m0, k0])
else:
    col_ids = k0 + offs_k
    row_offsets = offs_m[:, None] + tl.zeros((BLOCK_M, BLOCK_K),
                                             dtype=offs_m.dtype)
    col_offsets = col_ids[None, :] + tl.zeros((BLOCK_M, BLOCK_K),
                                              dtype=col_ids.dtype)
    a_cur = tl.load(
        A_desc,
        offsets=(row_offsets, col_offsets),
        boundary_check=(0, 1),
        padding_option="zero",
    )
if (n0 + BLOCK_N <= N) and (k0 + BLOCK_K <= K):
    b_cur = B_desc.load([k0, n0])
else:
    row_ids = k0 + offs_k
    row_offsets = row_ids[:, None] + tl.zeros((BLOCK_K, BLOCK_N),
                                              dtype=row_ids.dtype)
    col_offsets = offs_n[None, :] + tl.zeros((BLOCK_K, BLOCK_N),
                                             dtype=offs_n.dtype)
    b_cur = tl.load(
        B_desc,
        offsets=(row_offsets, col_offsets),
```

```
            boundary_check=(0, 1),
            padding_option="zero",
        )
    for kt in tl.range(0, K_tiles, num_stages=2):
        k0 = kt * BLOCK_K
        acc += tl.dot(a_cur, b_cur)
        next_k = k0 + BLOCK_K
        if next_k < K:
            if (m0 + BLOCK_M <= M) and (next_k + BLOCK_K <= K):
                a_cur = A_desc.load([m0, next_k])
            else:
                col_ids = next_k + offs_k
                row_offsets = offs_m[:, None] + tl.zeros((BLOCK_M,
                 ↪ BLOCK_K),
                                                         dtype=offs_m.dtype)
                col_offsets = col_ids[None, :] + tl.zeros((BLOCK_M,
                 ↪ BLOCK_K),
                                                       dtype=col_ids.dtype)
                a_cur = tl.load(
                    A_desc,
                    offsets=(row_offsets, col_offsets),
                    boundary_check=(0, 1),
                    padding_option="zero",
                )
            if (n0 + BLOCK_N <= N) and (next_k + BLOCK_K <= K):
                b_cur = B_desc.load([next_k, n0])
            else:
                row_ids = next_k + offs_k
                row_offsets = row_ids[:, None] + tl.zeros((BLOCK_K,
                 ↪ BLOCK_N),
                                                       dtype=row_ids.dtype)
                col_offsets = offs_n[None, :] + tl.zeros((BLOCK_K,
                 ↪ BLOCK_N),
                                                         dtype=offs_n.dtype)
                b_cur = tl.load(
                    B_desc,
```

```
                    offsets=(row_offsets, col_offsets),
                    boundary_check=(0, 1),
                    padding_option="zero",
                )
    # Store results with masking
    c_ptrs = C_ptr + (offs_m[:, None] * stride_cm + offs_n[None, :] *
    ↪ stride_cn)
    c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
    tl.store(c_ptrs, acc, mask=c_mask)

def persistent_matmul(A: torch.Tensor, B: torch.Tensor) -> torch.Tensor:
    M, K = A.shape
    K2, N = B.shape
    assert K == K2
    C = torch.empty((M, N), device=A.device, dtype=torch.float32)
    MT = triton.cdiv(M, 128)
    NT = triton.cdiv(N, 128)
    grid = lambda META: (min(65536, MT * NT),)  # bound launch overhead
    matmul_kernel_persistent[grid](
        A, B, C, M, N, K,
        A.stride(0), A.stride(1),
        B.stride(0), B.stride(1),
        C.stride(0), C.stride(1),
    )
    return C
```

这里，kernel 在 $M \times N$ 个分块上启动一个 2 维网格，并在单次 kernel 启动内完成整个 K 维循环。这可以减少启动开销，并在 K 较大时提高利用率，但代价是单个 kernel 需要更长时间占用资源。每个程序（线程块）将 A 和 B 的分块加载到共享内存中并

并使用 tl.dot 计算 tile 的部分点积。Triton 以 FP32 累加结果。在 Blackwell 上，Triton 会把 tl.dot lowering 为 tcgen05 和 UMMA，以使用 Tensor Core；Tensor Core 随后把结果累积到专用的 TMEM，而不是通用寄存器。

最好在 Triton 中使用 tensor descriptor 表达由共享内存支持的 tile 移动。例如 desc=tl.make_tensor_descriptor(...)。在现代 GPU 上，这些 tensor descriptor 调用会映射到基于 TMA 的硬件操作，使用异步且合并的传输。

Triton 会为效率展开/向量化这些循环和计算。此外，当 dtype 和 tile 形状受支持时（例如 FP16/BF16，或 FP32 的 TF32），Triton 会将 tl.dot lowering 为 Tensor Core 指令。下面是一个启动该 Triton kernel 的简单 Python 包装器：

```
def tiled_matmul(A: torch.Tensor, B: torch.Tensor) -> torch.Tensor:
    """
    Tiled matrix multiplication using autotuned Triton kernel.

    The kernel will automatically select the best block size
    ↪ configuration
    for the given matrix dimensions (M, N, K).
    """
    M, K = A.shape
    K2, N = B.shape
    assert K == K2, f"Inner dimensions must match: {K} != {K2}"
    C = torch.empty((M, N), device=A.device, dtype=torch.float32)
    # Grid is computed based on max block size from autotuning configs
    # Triton's autotuner will pick the optimal block size at runtime
    MAX_BLOCK_M = 128  # From largest config
    MAX_BLOCK_N = 128
    grid = (triton.cdiv(M, MAX_BLOCK_M), triton.cdiv(N, MAX_BLOCK_N))
    # Launch with autotuning - Triton will select best config
    tiled_gemm_kernel[grid](
        A, B, C, M, N, K,
        A.stride(0), A.stride(1),
        B.stride(0), B.stride(1),
        C.stride(0), C.stride(1),
    )
    return C
```

可以看到，把完整的 K 循环放在一次 kernel 启动中，就避免了为每个输出 tile 启动多个 kernel。在现代 GPU 上，当 K 很大时，这种方式可以提高利用率，但代价是资源在单个 kernel 中保持更长时间。

可以把这种 tiled 方法与持久化 kernel 结合，使同一个线程块跨多个输出 tile 复用。持久化 kernel 使用 1-D grid，并像下面代码块一样，令 tile 索引按 tl.num_programs(0) 步进：

```
@triton.jit
def matmul_kernel_persistent(
    A_ptr, B_ptr, C_ptr,
    M, N, K,
    stride_am, stride_ak,
    stride_bk, stride_bn,
    stride_cm, stride_cn,
    # compile-time constants
    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr,
    NUM_STAGES: tl.constexpr,
):
    """
    Persistent thread GEMM with Triton tensor descriptors + autotuning.
    Key Blackwell Optimizations:
      1) Tensor descriptors -> TMA hardware acceleration
      2) Autotuning across multiple block-size configurations
      3) Persistent threads to amortize launch overhead
      4) TMEM accumulation (accumulators live in ~256 KB/SM TMEM on
       ↪ Blackwell)
    This is the PRODUCTION-READY version combining these best practices.
    """
    # 1-D persistent launch: each program processes multiple tiles
    pid   = tl.program_id(0)
    nprog = tl.num_programs(0)
    MT = tl.cdiv(M, BLOCK_M)
    NT = tl.cdiv(N, BLOCK_N)
    TILE_COUNT = MT * NT
    # --- Tensor descriptors (map to TMA on NVIDIA). Descriptor rules:
    #     leading strides must be multiples of 16 BYTES;
    #     last dimension contiguous.
    A_desc = tl.make_tensor_descriptor(
        A_ptr, shape=[M, K], strides=[stride_am, stride_ak],
        block_shape=[BLOCK_M, BLOCK_K],
    )
    B_desc = tl.make_tensor_descriptor(
        B_ptr, shape=[K, N], strides=[stride_bk, stride_bn],
```

```
    block_shape=[BLOCK_K, BLOCK_N],
)
# Persistent stride over all output tiles handled by this program.
tile_idx = pid
while tile_idx < TILE_COUNT:
    pid_m = tile_idx // NT
    pid_n = tile_idx %  NT
    m0 = pid_m * BLOCK_M
    n0 = pid_n * BLOCK_N
    offs_m = m0 + tl.arange(0, BLOCK_M)
    offs_n = n0 + tl.arange(0, BLOCK_N)
    offs_k = tl.arange(0, BLOCK_K)
    # FP32 accumulator (on Blackwell, accumulators live in TMEM,
    # not registers)
    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
    # ---- K loop (double-buffered ring-of-two using descriptors ->
    ↪  TMA) ----
    K_tiles = (K + BLOCK_K - 1) // BLOCK_K
    if K_tiles == 0:
        # Masked store of zeros when K == 0
        c_ptrs = C_ptr + (offs_m[:, None] * stride_cm + offs_n[None,
        ↪  :]
                          * stride_cn)
        c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
        tl.store(c_ptrs, acc, mask=c_mask)
        tile_idx += nprog
        continue
    # Prefetch first CURRENT tiles
    k0 = 0
    if (m0 + BLOCK_M <= M) and (k0 + BLOCK_K <= K):
        a_cur = A_desc.load([m0, k0])
    else:
       row_offsets = offs_m[:, None] + tl.zeros((BLOCK_M, BLOCK_K),
                                                dtype=offs_m.dtype)
        col_offsets = (k0 + offs_k)[None, :] + tl.zeros((BLOCK_M,
        ↪  BLOCK_K),
```

```
                                                dtype=offs_k.dtype)
    a_cur = tl.load(A_desc, offsets=(row_offsets, col_offsets),
                    boundary_check=(0, 1), padding_option="zero")
if (n0 + BLOCK_N <= N) and (k0 + BLOCK_K <= K):
    b_cur = B_desc.load([k0, n0])
else:
    row_offsets = (k0 + offs_k)[:, None] + tl.zeros((BLOCK_K,
     ↪ BLOCK_N),
                                                dtype=offs_k.dtype)
    col_offsets = offs_n[None, :] + tl.zeros((BLOCK_K, BLOCK_N),
                                                dtype=offs_n.dtype)
    b_cur = tl.load(B_desc, offsets=(row_offsets, col_offsets),
                    boundary_check=(0, 1), padding_option="zero")

# Pipeline: prefetch-NEXT -> compute-CURRENT -> swap
for kt in tl.range(0, K_tiles, num_stages=NUM_STAGES,
                   warp_specialize=True):
    next_k = (kt + 1) * BLOCK_K
    if kt + 1 < K_tiles:
        if (m0 + BLOCK_M <= M) and (next_k + BLOCK_K <= K):
            a_next = A_desc.load([m0, next_k])
        else:
            row_offsets = offs_m[:, None] + tl.zeros((BLOCK_M,
             ↪ BLOCK_K),
                                                dtype=offs_m.dtype)
            col_offsets = (next_k + offs_k)[None, :]
                          + tl.zeros((BLOCK_M, BLOCK_K),
                          dtype=offs_k.dtype)
            a_next = tl.load(A_desc, offsets=(row_offsets,
             ↪ col_offsets),
                             boundary_check=(0, 1),
                             padding_option="zero")
        if (n0 + BLOCK_N <= N) and (next_k + BLOCK_K <= K):
            b_next = B_desc.load([next_k, n0])
        else:
            row_offsets = (next_k + offs_k)[:, None]
```

```
                                + tl.zeros((BLOCK_K, BLOCK_N),
                                dtype=offs_k.dtype)
                col_offsets = offs_n[None, :] + tl.zeros((BLOCK_K,
                ↪  BLOCK_N),
                                                    dtype=offs_n.dtype)
                b_next = tl.load(B_desc, offsets=(row_offsets,
                ↪  col_offsets),
                                 boundary_check=(0, 1),
                                 padding_option="zero")
        # Compute on CURRENT tiles (UMMA on Blackwell; accumulators
        ↪  in TMEM)
        acc += tl.dot(a_cur, b_cur)
        # Swap in the prefetched NEXT tiles
        if kt + 1 < K_tiles:
            a_cur = a_next
            b_cur = b_next
    # ---- Store C with masking
    ↪  ------------------------------------------
    c_ptrs = C_ptr + (offs_m[:, None] * stride_cm
                      + offs_n[None, :] * stride_cn)
    c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
    tl.store(c_ptrs, acc, mask=c_mask)
    # Advance to the next tile handled by this program (persistent
    ↪  stepping)
    tile_idx += nprog
```

这种持久化方法与拥有较大寄存器文件、共享内存和 L2 缓存的现代 GPU 很匹配，因为这些资源可以容纳更大的 tile size（BLOCK_K），使每次迭代可以展开更多 K 循环。

这种持久化 kernel 通常会超过一串较小的 matmul kernel，尤其当 K 很大（例如 > 1,024）时。代价是某个 SM 会保持更长时间的占用；但如果 kernel 能充分利用该 SM，这通常是理想方案。

在前面的代码中，可以尝试在现代 GPU 上增大 BLOCK_K，因为新硬件增加了片上内存，能够在每个 tile 中处理更多数据。

不过，超过某个程度后，寄存器压力可能增加并导致寄存器溢出。因此，始终要对工作负载和硬件进行性能分析，找出合适的平衡。

14.2.10 使用 Triton 进行软件流水线和双缓冲

这个例子展示如何使用 Triton 实现双缓冲。双缓冲是软件流水线的两阶段形式，在单个循环中让内存加载与计算重叠。

在现代 NVIDIA GPU 上，异步的全局到共享内存复制允许多个在途预取阶段与计算重叠。这使双缓冲（以及三缓冲等）成为有价值且重要的性能优化技术。

Triton 通过循环迭代器使用 num_stages 元参数实现流水线。该参数传给 tl.range() 循环迭代器。当传入 num_stages>1 时，迭代器会自动对循环进行流水化，在途的最多 num_stages 次迭代中发出异步复制操作。

这会让分阶段迭代中的内存加载与计算重叠，直到所有 tile 都处理完成。下面是 Triton 中基于 tile 的双缓冲实现（num_stages=2）：

```
@triton.jit
def pipelined_matmul(
    A_ptr, B_ptr, C_ptr,
    M, N, K,
    stride_am, stride_ak,
    stride_bk, stride_bn,
    stride_cm, stride_cn,
    # compile-time constants:
    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr,
    NUM_STAGES: tl.constexpr,
):
    # Program (CTA) ids for the MxN tiling
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    m0 = pid_m * BLOCK_M
    n0 = pid_n * BLOCK_N
    offs_m = m0 + tl.arange(0, BLOCK_M)
    offs_n = n0 + tl.arange(0, BLOCK_N)
```

```
offs_k = tl.arange(0, BLOCK_K)
# ---------------- Descriptor creation (maps to TMA on Blackwell)
 ↪  ----------
# Requirements for descriptor/TMA on NVIDIA GPUs:
#  - leading strides are multiples of 16 BYTES
#  - last dimension contiguous
#  - block_shape matches the tile you intend to move
A_desc = tl.make_tensor_descriptor(
    A_ptr,
    shape=[M, K],
    strides=[stride_am, stride_ak],
    block_shape=[BLOCK_M, BLOCK_K],
)
B_desc = tl.make_tensor_descriptor(
    B_ptr,
    shape=[K, N],
    strides=[stride_bk, stride_bn],
    block_shape=[BLOCK_K, BLOCK_N],
)
# Accumulator in FP32; on Blackwell this resides in TMEM, not
 ↪  registers.
acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
# Number of K tiles
K_tiles = (K + BLOCK_K - 1) // BLOCK_K
if K_tiles == 0:
    # Nothing to do, store zeros (masked)
    c_ptrs = C_ptr + (offs_m[:, None]
                      * stride_cm + offs_n[None, :] * stride_cn)
    c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
    tl.store(c_ptrs, acc, mask=c_mask)
    return
# --------- Prefetch the first "current" tiles (fast path if fully
 ↪  in-bounds)
k0 = 0
if (m0 + BLOCK_M <= M) and (k0 + BLOCK_K <= K):
    a_cur = A_desc.load([m0, k0])
```

```
else:
   # Boundary: compute row/col offsets and use boundary-checked load
    row_offsets = offs_m[:, None] + tl.zeros((BLOCK_M, BLOCK_K),
                                             dtype=offs_m.dtype)
    col_offsets = (k0 + offs_k)[None, :] + tl.zeros((BLOCK_M,
     ↪ BLOCK_K),
                                               dtype=offs_k.dtype)
    a_cur = tl.load(A_desc, offsets=(row_offsets, col_offsets),
                    boundary_check=(0, 1), padding_option="zero")
if (n0 + BLOCK_N <= N) and (k0 + BLOCK_K <= K):
    b_cur = B_desc.load([k0, n0])
else:
    row_offsets = (k0 + offs_k)[:, None] + tl.zeros((BLOCK_K,
     ↪ BLOCK_N),
                                               dtype=offs_k.dtype)
    col_offsets = offs_n[None, :] + tl.zeros((BLOCK_K, BLOCK_N),
                                             dtype=offs_n.dtype)
    b_cur = tl.load(B_desc, offsets=(row_offsets, col_offsets),
                    boundary_check=(0, 1), padding_option="zero")
# ------------- K loop with software pipelining and TMA prefetch
 ↪ -----------
# Put prefetch as early as possible in loop body; keep separate
 ↪ "current"
# tile so loads for the next iteration can overlap with the
 ↪ dot-product of
# current tile. Use warp_specialize to partition producer/consumer
 ↪ warps.
for kt in tl.range(0, K_tiles, num_stages=NUM_STAGES,
 ↪ warp_specialize=True):
    next_k = (kt + 1) * BLOCK_K
    # Prefetch NEXT tiles early (TMA), if there is a next tile
    if kt + 1 < K_tiles:
        if (m0 + BLOCK_M <= M) and (next_k + BLOCK_K <= K):
            a_next = A_desc.load([m0, next_k])
        else:
```

```
            row_offsets = offs_m[:, None] + tl.zeros((BLOCK_M,
             ↪ BLOCK_K),
                                                     dtype=offs_m.dtype)
            col_offsets = (next_k + offs_k)[None, :]
                          + tl.zeros((BLOCK_M, BLOCK_K),
                           ↪ dtype=offs_k.dtype)
            a_next = tl.load(A_desc, offsets=(row_offsets,
             ↪ col_offsets),
                             boundary_check=(0, 1),
                              ↪ padding_option="zero")
        if (n0 + BLOCK_N <= N) and (next_k + BLOCK_K <= K):
            b_next = B_desc.load([next_k, n0])
        else:
            row_offsets = (next_k + offs_k)[:, None]
                           + tl.zeros((BLOCK_K, BLOCK_N),
                            ↪ dtype=offs_k.dtype)
            col_offsets = offs_n[None, :] + tl.zeros((BLOCK_K,
             ↪ BLOCK_N),
                                                     dtype=offs_n.dtype)
            b_next = tl.load(B_desc, offsets=(row_offsets,
             ↪ col_offsets),
                             boundary_check=(0, 1),
                              ↪ padding_option="zero")
    # Compute on the CURRENT tiles (UMMA; accumulators in TMEM on
     ↪ Blackwell)
    acc += tl.dot(a_cur, b_cur)
    # Swap in prefetched NEXT tiles
    if kt + 1 < K_tiles:
        a_cur = a_next
        b_cur = b_next
# ------------------------------ Store C
 ↪ ----------------------------------
c_ptrs = C_ptr + (offs_m[:, None] * stride_cm + offs_n[None, :] *
 ↪ stride_cn)
c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
tl.store(c_ptrs, acc, mask=c_mask)
```

Triton 编译器识别 `num_stages`，借助 Triton 张量描述符（tensor descriptor）以 TMA 与 TMEM 生成异步加载/存储，并自动管理同步。使用 TMA 张量描述符与 TMEM 可以降低寄存器压力，并简化多维传输的表达。前面的循环执行预取下一分块 → 计算当前分块 → 交换的操作。这在 `num_stages>1` 时保持了重叠，并避免过早覆写当前分块时重叠悄然失效。具体而言，当一个分块正被加载到共享内存时，前一个分块的计算正在执行。

你还可以使用 warp 专门化，让 Triton 分割生产者和消费者 warp，从而使全局到共享内存的复制与计算重叠。当启发式方法选择它时，该功能会自动启用。

保持流水线重叠完整非常重要。当设置 num_stages > 1 时，应确保当前正在计算的 tile 在多个循环迭代之间保持存活。不要在消费它的数学运算完成之前覆盖或丢弃它的引用。

考虑一个两阶段环形缓冲区：在异步获取下一个缓冲区的同时计算当前缓冲区，然后交换当前和下一个缓冲区。在这种情况下，必须先等待并消费“当前”缓冲区，再释放它；同时需要使用 commit/wait 保护“下一个”缓冲区（tile），直到交换发生。

如果当前 tile 没有跨迭代存活（例如覆盖了其缓冲区或让引用失效），编译器可以把下一次迭代的复制操作重排、下沉或提升到当前使用之前。这会使 load/compute 重叠悄悄消失。

如果自行管理共享内存 block pointer/TMA，请确保使用显式握手：入队异步复制 → tl.commit() →（稍后）tl.wait() → 消费 tile → 推进指针。如果在读取 tile 前调用 tl.wait()，并且只在消费完成后释放/交换，就能保持真正的重叠。

> **注** 在现代 GPU 上，Triton 会使用 Tensor Memory Accelerator（TMA）支持 descriptor 的 load 和 store。当流水化循环（num_stages > 1）时，应尽早预取下一个 tile，并为当前 tile 保留单独变量。不要在消费当前引用的操作完成之前覆盖或丢弃它；否则编译器可能合法地把下一次迭代的复制重排到当前使用之前，重叠就会消失。最简单的模式是：预取下一个 → 计算当前 → 交换。

简而言之，软件流水线提高内存带宽利用率并隐藏延迟。如果内存带宽不是瓶颈，可以将 num_stages 提高到 3（例如三缓冲），让两个 tile 在途预取，以进一步改善性能；代价是额外缓冲区需要更多共享内存。不过，在共享内存预

算更紧张的硬件上，或对于计算密集型 kernel，增加阶段可能收益递减、降低占用率并降低整体性能。

14.2.11 使用 Triton Proton Profiler 进行性能分析

要深入分析 Triton kernel 性能，可以使用 Triton 的 Proton profiler。这是一个与 Triton 集成的独立性能分析包，会发出在 Nsight Systems 时间线中可见的 NVTX 范围。这些 NVTX 范围会为代码区域插桩、收集计时并跟踪关键指标；NVTX 标记随后出现在 Nsight Systems 时间线中。可以使用这些 NVTX 区域从 Proton 摘要跳转到 Nsight Compute，进行更聚焦的 kernel 级分析。

实际使用中，可以使用 with proton.scope("name", metadata) Python 上下文管理器包装 Triton 代码中的关键区域。运行工作负载前，使用 proton.start("matmul", hook="triton") 激活 profiler。

Proton 中的 metadata 是用户提供的注解或指标字典，例如总 FLOPS、内存字节数、线程块索引、warp 级索引或记录槽数量。这些内容会与计时数据一同记录，从而支持更丰富的性能分析。

执行完成后，结束并获取性能分析数据。输出可以打印成分层计时表。下面是 Proton 性能分析结果的摘录，用于比较不同 matmul kernel 对 8,192 × 8,192 矩阵的乘法：

```
168.314 ms    16331.291  ROOT
├─ 174.920 ms   3928.623  cublas [M=8192,N=8192,K=512]
├─ 165.349 ms   4156.033  matmul_kernel [M=8192,N=8192,K=512]
├─ 159.352 ms   4312.421  matmul_kernel_persistent [M=8192,N=8192,K=512]
└─ 174.671 ms   3934.214  torch [M=8192,N=8192,K=512]
```

在前面的性能分析输出中，K 表示归约维度，即矩阵乘法的共享内维度（例如一个 M × K 矩阵乘以 K × N 矩阵）。它表示驱动计算成本和内存流量的点积长度。

在此性能分析报告中，Triton 持久化 kernel 的性能超过 cuBLAS：159.352 ms 对 174.920 ms。Proton 可以轻松量化这种差异，还会计算有效 TFLOPS 和内存带宽等派生指标。如果提供总 FLOPS 等额外元数据，Proton 会显示在所用精

度（例如 BF16/FP16、FP8、FP4 等）下是否接近硬件理论 TFLOPS 上限。对于许多矩阵形状和精度，PyTorch 中的 cuBLASLt 或 CUTLASS 路径会达到或超过定制 Triton kernel。

NVIDIA 的 Nsight Systems 和 Nsight Compute 工具同样支持 Triton kernel。例如，Nsight Systems 会显示 kernel 启动名称以及 PyTorch 或 Proton 发出的 NVTX 范围。可以用它们把 Proton 范围与 Nsight 时间线关联起来，从而使用 Proton 输出定位有趣的

kernel，然后深入 Nsight Compute 进行低级分析。可以使用 Nsight Compute 分析寄存器使用、达到的占用率等。结合这些工具，可以获得更完整的系统性能分析。

14.3 PyTorch XLA 后端

虽然 TorchInductor 是 PyTorch 面向 GPU 和 CPU 的默认后端，但 PyTorch XLA 是面向 Google Cloud TPU 和其他加速器的独立后端编译选项。PyTorch XLA 通过把 PyTorch 操作映射到 XLA 图 IR，并使用目标硬件的运行时执行这些操作，让 PyTorch 模型运行在这些加速器上，如图 14-6 所示。

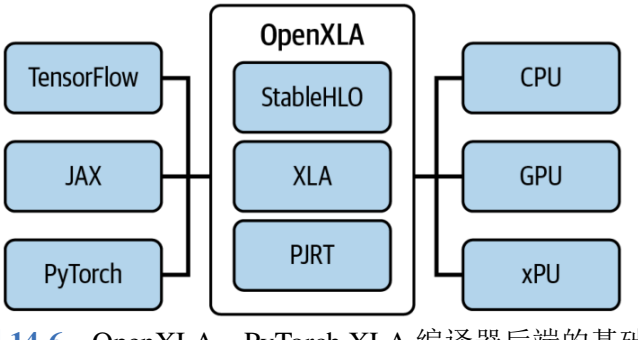

图 14-6　OpenXLA：PyTorch XLA 编译器后端的基础。

要启用 XLA 后端，可以使用 torch.compile(..., backend="openxla")，这会启用基于 OpenXLA 的 PyTorch XLA。该后端字符串由 PyTorch XLA 项目支持，并会激活基于 OpenXLA 的编译。与 TorchDynamo 和 TorchInductor 类似，XLA 会捕获计算图。不过，XLA 的设计目标是生成静态图，因此会提前编译完整程序。

XLA 针对静态形状或有界动态形状进行优化。因此，使用 XLA 时动态形状支持稍有限。新形状会触发整个程序重新编译，这对延迟敏感的推理代价很高。不过，OpenXLA 会按形状签名缓存可执行文件，可以改善性能。

> **注** 可能需要对输入进行 padding，或使用固定大小的 bucket。这是因为 XLA 会为新形状重新编译，而不是以符号方式处理它们。

XLA 编译器会为每个唯一的输入形状和签名缓存编译图。因此，经过几步预热后，性能会像 TorchInductor 一样改善。主要区别是 XLA 不会在运行中逐步编译；它会提前静态构建图。如果遇到新形状，就会触发新的整图编译，这非常昂贵，会影响推理等延迟敏感工作负载。

简而言之，如果运行在 TorchInductor 后端当前不支持的硬件设备上，而该设备支持 XLA，就可以考虑使用 XLA 后端。许多相同原则仍适用，例如尽量减少图断裂。还可以将某些分布式策略用于 XLA，例如数据并行和模型并行。XLA 虽不常用于 NVIDIA GPU，但对于非 NVIDIA 硬件（例如使用 OpenXLA 的 Google TPU 或支持 XLA IR 的其他加速器）是强大的后端。XLA 也受益于本章讨论的许多类似编译技术。

14.4 关键要点

本章涉及很多内容，深入讨论了 PyTorch 编译器栈以及 OpenAI Triton 语言和编译器。以下是一些关键要点：

14.4.1 利用 torch.compile 轻松获得加速

根据需求选择编译模式（例如快速启动使用”default”，最大性能使用”max-autotune”）。始终运行几次预热迭代，以度过初始编译阶段。对于短时运行的作业或小模型，编译开销可能超过加速收益；只有工作量足以摊薄一次性成本时才使用它。

14.4.2 尽早设置性能标志

以下是应启用的快速 FP32 矩阵乘法和 SDPA 变体（包括 Flash Attention）标志。始终验证模型精度，并保持这些设置启用以获得最佳性能：

```
import torch
 # maps to TF32/BF16 fast paths
torch.set_float32_matmul_precision("high")
torch.backends.cuda.matmul.allow_tf32 = True
# affects convs
torch.backends.cudnn.allow_tf32 = True
torch.backends.cuda.enable_flash_sdp(True)
torch.backends.cuda.enable_mem_efficient_sdp(True)
```

最小化图断裂

使用 torch._dynamo.explain 或 TORCH_LOGS=”graph_breaks” 检查图断裂。移除或重构导致断裂的代码（例如 print、依赖数据的

Python 控制流和不支持的操作），以最大化可编译的连续区域。通常更少、更大的图意味着更好的性能。必要时，对于条件逻辑使用新的 torch.cond API，或把非关键的 Python 侧处理移出模型的 forward 函数。目标是向编译器提供一条长的、纯粹的 tensor 输入到 tensor 输出的代码路径。

14.4.3 谨慎使用动态形状

虽然并非必需，但预先设置 torch.compile(dynamic=True) 会迫使编译器把所有维度视为动态，使一个编译模型能够处理广泛的输入形状，减少 padding 需求。编译器本来可以动态完成这一点，而 dynamic=True 会把它提前完成。也可以使用 mark_dynamic() 只标记特定维度，而不是全部维度，将灵活性限制在真正需要的地方。这对可变序列长度很有用，但要测量其中的权衡。启用动态形状支持可能禁用 CUDA Graphs 并插入额外 guard。如果形状变化不太大，混合方法往往最好：按大小对输入分桶，限制不同形状的数量，再对剩余变化启用动态形状。这种混合方法既避免过度重新编译，又减少 padding 浪费。

14.4.4 针对重新编译 guard 进行性能分析

使用 TORCH_LOGS="graph_breaks,guards,recompiles" 找出在多次编译时触发的 guard。如果看到多个编译，常见原因包括 Python 随机值、tensor rank 变化，或 device/dtype 变化。应将这些方面设为静态；如果合适，也可以使用 allow_in_graph 将其标记为安全。

14.4.5 尽可能避免重新编译

调优良好的训练循环应在最初几次迭代后不再重新编译。如果仍持续编译，应立即调查，因为这通常意味着每次迭代都有内容变化，包括带递增计数器的调试 print 语句等。使用 set_stance() API 和 guard 日志捕获这些问题。还要确保没有无意中混用设备，例如一次迭代发送 CPU tensor、下一次发送 GPU tensor；这会触发重新编译。

14.4.6 调优并监控内存使用

对于更大的融合 kernel 和 guard buffer，编译模式可能使用更多内存。应监控 GPU 内存。如果遇到内存问题，可以考虑在 Triton kernel 中使用更小的 BLOCK_SIZE，或禁用某些融合。还要及时释放大型中间结果，否则它们可能在编译图生命周期中停留更久。如果编译期间出现 out-of-memory 错误，可能需要拆分模型或降低 max_autotune 设置。PyTorch 支持对大图进行自动 checkpointing，以减轻内存压力，

但它并非万无一失。还可以尝试分别编译子模块（例如每层或每个 block），而不是一次编译整个模型，从而限制 kernel 生成期间的内存使用。

14.4.7 明智地将 PyTorch 编译器与分布式训练结合

使用 DDP 或 FSDP 时，要注意通信点处有意设置的图断裂。这些断裂是预期行为，且通过让通信与计算重叠进行了优化。用 FSDP 包装子模块，可以获得按分片编译和内存节省。留意 dynamo.explain 中与 all-reduce 相关的警告。PyTorch 的设计会尽量减少图断裂造成的性能下降。使用 FSDP 和 torch.compile

时，可能在梯度预归约和后归约步骤看到图断裂；这些是预期的，并且已经处理。应关注每个分片内部的主要前向和反向传播是否被编译。

14.4.8 使用 TORCH_LOGS="perf_hints" 捕获遗漏的优化

这会告诉你，例如 CUDA Graph 因输入变更而未使用，或某个操作回退到 eager 模式。这些提示可以指出潜在改进，建议避免某些模式或等待未来支持。通常提示会直接给出变通方法。例如，“input is mutated”提示意味着应在编译前避免对输入执行原地操作。

14.4.9 先用小输入进行调试

开发定制 kernel 或测试编译模式时，使用较小的 tensor size 快速发现正确性问题。工作后再放大规模。可以使用 PyTorch 的 torch._dynamo.config.verbose，甚至使用 TORCH_LOGS="output_code"，检查小案例生成的代码。要注意，在极小输入上测得的性能可能无法反映真实规模的行为；但对调试正确性和查看生成了哪些 kernel，这是一项有用技术。

14.4.10 只为真正的瓶颈编写定制 kernel

开始开发 Triton kernel 之前，先对模型在编译和不编译两种情况下进行性能分析，找出热点。TorchInductor 通常已经融合了很多操作。应聚焦 Inductor 表现不足的区域，例如定制 op 或非典型融合。不要过早优化一切：大部分交给编译器，其余再手工调优。例如，如果 TorchInductor 无法融合某个性能关键的操作序列（如多步定制激活），就可能值得单独编写 Triton kernel。但要权衡维护成本，因为每个定制 kernel 都需要维护，硬件变化或 PyTorch 最终增加原生支持时可能都要更新代码。若某种模式对许多用户有益，向 TorchInductor 提交 issue 或功能请求通常值得。

14.4.11 遵循 Triton 最佳实践

编写 Triton 代码时，确保内存访问合并，避免共享内存 bank 冲突（必要时进行 padding），在边界处对 tl.load() 和 tl.store() 使用 mask，选择与 warp 大小对齐的 block 和 tile size（32 线程的倍数），选择适合 L1/共享内存 carve-out 的 tile size，并使用 @triton.autotune([... triton.Config(num_warps=..., num_stages=...) ...], key=[...]) 调整 num_warps 和 num_stages。如果性能至关重要，请使用 Triton autotuner 寻找最佳配置。可以从 num_warps $\in$ {4,8} 和 num_stages $\in$ {2,3,4} 开始。记得查看 Triton 文档和示例，因为 FlashAttention 等许多常见模式已经由社区实现并优化。至少，这些现有示例是自己实现时很好的起点。

14.4.12 缓存提示注意事项

某些形式的 PTX load（例如 DeepSeek 风格的 non-coherent + L1 no-allocate + 256B L2 prefetch）没有文档说明，在某些硬件上可能未定义。它们可能在不同驱动版本和硬件代际之间失效。建议使用 Triton 的 eviction_policy=（例如 evict_last）和 cache_modifier= 旋钮来改善可移植性。换用不同 CUDA 驱动和 GPU 代际时，记得进行性能分析。避免引入相互冲突的缓存提示，因为这类问题极难调试。

14.4.13 留意新的 PyTorch 发布版本

PyTorch 正在快速演进其编译器。每个版本都会带来更广的操作支持、更好的性能和更少的图断裂。升级通常可以免费获得加速或修复问题，Triton 也是如此。保持最新版本能确保受益于最新成果；对于快速演进的 GPU 硬件尤其重要。

14.5 结论

随着复杂度增加、PyTorch 生态持续扩展，必须接受性能分析和迭代优化。应结合使用 Nsight Systems、Nsight Compute、PyTorch profiler 或 Triton profiler，验证 GPU 利用率和性能是否达到预期；如果没有，就调整性能策略。

还要记住，在迭代过程中保持整体视角。一个瓶颈消除后，下一个瓶颈就会出现。迭代性能调优可能先从 GPU kernel 占用率转向 CPU 开销，再转向输入流水线调优。编译器解决了其中很大一部分，但要达到最佳性能，可能仍需优化数据加载、I/O 和算法选择。

第 15 章 多节点推理、并行、解码与路由优化

LLM 的参数规模持续增长。尤其是混合专家（MoE）LLM 的出现——这类模型把许多专家子网络（“专家”）与内置的专家门控机制结合起来——已将模型参数量推高到数千亿甚至数万亿。虽然对给定输入而言只有一小部分参数处于激活状态，但要在这些超大规模模型上运行推理，仍需把工作负载分布到多个 GPU 上。

本章聚焦于使用现代 NVIDIA GPU 对这些巨型 LLM 执行高效、高性能多节点推理时所用的高级优化技术。我们将讨论如何设计分布式推理系统，以最小化延迟并最大化吞吐量——同时利用硬件和算法创新。

我们首先讨论分离式预填充与解码（PD，disagg PD）架构；它把推理工作负载拆成可以独立调优的不同阶段。接着，我们探讨以推理为核心的并行策略，例如数据、张量、流水线、专家和上下文并行——以及如何组合使用它们，在多个 GPU 上服务大模型。

然后，我们介绍推测解码方法，包括 Medusa、EAGLE 以及草稿—验证（draft-and-verify）方案。这些方法允许在推理期间生成并评估多个 token，而不是像传统自回归 LLM 那样每次只生成一个 token。这有助于克服串行解码瓶颈。我们还会讨论用于强制输出格式（例如自定义 JSON schema）的约束解码，以及面向 MoE 模型的动态路由策略，以改善系统的专家门控和负载均衡效率。

15.1 分离式预填充与解码架构

如前所述，现代 LLM 的推理工作流由两个不同阶段组成：预填充和解码。我们可以实现分离式预填充与解码来拆分这两个阶段。这样就能独立扩展预填充集群和解码集群——甚至可以使用不同的硬件平台——并显著提升大规模 LLM 服务性能，本章后文会详细介绍。

> **注** 跨厂商或跨架构部署要求两侧的 KV 缓存布局和 dtype 保持一致。实践中，生产系统应让预填充和解码运行在兼容的 GPU 系列上。这样，它们使用相同的数值格式，便于实现 KV 缓存传输和数据复用。

在预填充阶段，模型会在一次前向传播中处理整个输入提示——通常有数千、数万甚至数百万个 token——以生成 LLM 计算出的初始隐藏状态。随后，它为输入提示中的所有 token 填充注意力键值（KV）缓存。图 15-1 展示了分离式预填充与解码如何共享 KV 缓存，并将 KV 传输与计算重叠。

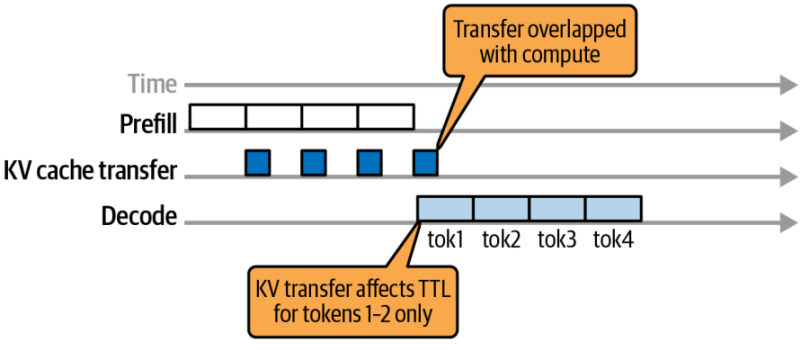

图 15-1　分离式预填充与解码共享 KV 缓存，并将 KV 传输与计算重叠。

在解码阶段，模型执行自回归生成，以预测序列中的每个新 token。它通过使用此前已生成所有 token 的缓存注意力 KV 表示来完成这一点。

> **注**　推测解码通过在单批中预生成多个 token 来加速解码过程。随后它会并行验证这些 token 是否正确。这减少了标准逐 token 自回归解码的串行性。我们稍后会介绍推测解码。

15.1.1 预填充—解码干扰

传统上，LLM 推理系统把这两个阶段共置在同一批节点上，并简单地把所有计算一起批处理。然而，这种朴素做法会导致通常所说的预填充—解码干扰。例如，长提示的预填充可能占满 GPU，并延迟其他请求对时间敏感的解码工作——反之亦然。

在同一批节点上共置预填充和解码，会迫使这两个特征迥异的阶段共用同一套调度和资源分配策略。预填充由大型并行计算组成；相比之下，解码需要许多小型、串行的计算。因此，系统要么让一个阶段的性能优先于另一个阶段，要么过度配置硬件来同时满足两种需求。

采用分离式预填充与解码架构时，预填充阶段和解码阶段被分配到不同的 GPU 池。这消除了两个工作负载之间的直接干扰。DistServe 系统通过分离预填充和解码，报告在同时满足 TTFT 和 TPOT 约束时，有效吞吐量请求数最高提升 7.4 倍（延迟 SLO 最多收紧 12.6 倍）。

15.1.2 独立扩展预填充与工作节点

如果能消除跨阶段干扰，就能减少资源“死区时间”——即解码任务被长预填充计算阻塞的时间，反之亦然。这样，GPU 会把更多时间用于有用的工作，减少空闲。在给定延迟目标下，这会提高利用率和有效吞吐量（即 goodput）。

我们可以把一组节点专门用于预填充，另一组节点专门用于解码，从而分别扩展预填充和解码。两个集群只在把编码后的提示状态或注意力 KV 缓存从预填充工作节点传输到解码工作节点时通信，如图 15-2 所示。

在这里，你可以看到独立的 GPU 工作节点负责预填充阶段以处理输入提示——同时由解码阶段迭代生成输出 token。预填充阶段的输出包含提示的 KV 缓存，它会被传输到解码工作节点，以生成后续 token。

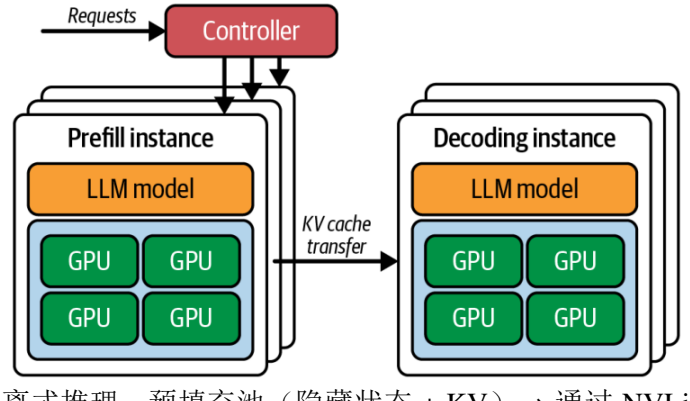

图 15-2　分离式推理：预填充池（隐藏状态 + KV）→ 通过 NVLink/NVSwitch（节点内）或 GPUDirect RDMA（节点间）进行 KV 交接 → 解码池。

通过专用 GPU 池，系统让预填充和解码流水线并行保持忙碌。实践中，分离已被证明能在严格延迟约束下显著提升吞吐量。一些研究显示可以获得很大收益，但一旦分离预填充和解码阶段，结果从适度改善到数倍有效吞吐量提升不等。结果在很大程度上取决于工作负载和网络 fabric。

这种分离允许每个阶段独立地针对吞吐量或延迟进行优化和扩展。预填充可以在预填充 GPU 上激进批处理，以最大化吞吐量，而不拖累解码性能（例如增加解码延迟）。此外，使用独立集群时，我们可以针对每个阶段调整并行设置、实例数量和调度策略。

15.1.3 对延迟（TTFT）和吞吐量（TPOT）的影响

解码 GPU 可以以较低批大小运行——或采用专门调度——以最小化流式生成的每输出 token 时间（TPOT）。例如，可以使用优先处理紧急解码任务的调度器来避免排队延迟。

这种分离之所以有效，是因为每个阶段有不同的性能预期。预填充阶段的首 token 时间（TTFT）针对低延迟优化，而解码阶段优先考虑较低的每输出 token 时间（TPOT）和稳定的流式延迟。端到端吞吐量主要由并发度和调度决定。传统设置中，必须在 TTFT 和 TPOT 每 token 延迟之间折中。分离则允许同时满足两个 SLO 目标。

> **注** 在 KV 传输和 all-to-all 阶段监控 NVLink 与 NIC 利用率。目标是通过独立的流和事件让通信与计算重叠。

KV 交接只增加极少的额外延迟，因为通信使用面向多 GPU 和多节点传输的高带宽互连。这些互连包括 NVLink、NVSwitch、InfiniBand，以及使用 GPUDirect RDMA 的以太网（以太网上的 RoCE）。例如，配备 ConnectX-8 SuperNIC（800 GbE 级）的多节点集群通过 GPUDirect RDMA 可为每个端口提供最高 800 Gb/s。与经主机中转的通信路径相比，这大大缩短了 KV 传输时间。此外，建议每块 GPU 部署 1 个 NIC，以优化预填充—解码分离，并改善 MoE all-to-all 性能。

15.1.4 KV 缓存数据传输与 NIXL

分离式系统使用连接器或调度器，在提示处理完成后，把提示的中间结果（最终隐藏状态和 KV 缓存）从预填充工作节点传输到解码工作节点。这次交接会带来一些通信开销，但如果集群互连是高带宽的（例如 NVLink 或 InfiniBand），与消除资源争用带来的收益相比，这种开销很小。

实践中，NVIDIA 的 NIXL 库会根据拓扑和策略自动选择 NVLink/NVSwitch、RDMA 或主机暂存路径，从而最小化传输开销。例如，第 4 章介绍的 NVIDIA NIXL 库会自动选择最快的可用路径来传输 KV 缓存。NIXL 与包括 Dynamo 和 vLLM 在内的框架和推理引擎集成。NIXL 与 vLLM 的集成如图 15-3 所示。

具体而言，LMCache 和 NIXL 已集成到 vLLM 的分离式预填充中，作为受支持的路径。NVIDIA Dynamo 和 TensorRT-LLM 也使用 NIXL，通过点对点 GPU 互连和 RDMA 传输 KV 缓存数据。

在节点内，NIXL 通过 NVLink 和 NVSwitch 执行设备到设备传输，无需主机暂存。跨节点时，NIXL 通过 InfiniBand 或 RoCEv2 使用 GPUDirect RDMA，以避免主机拷贝。即使负载达到数 GB，这些路径也能让 KV 缓存交接延迟保持较低。

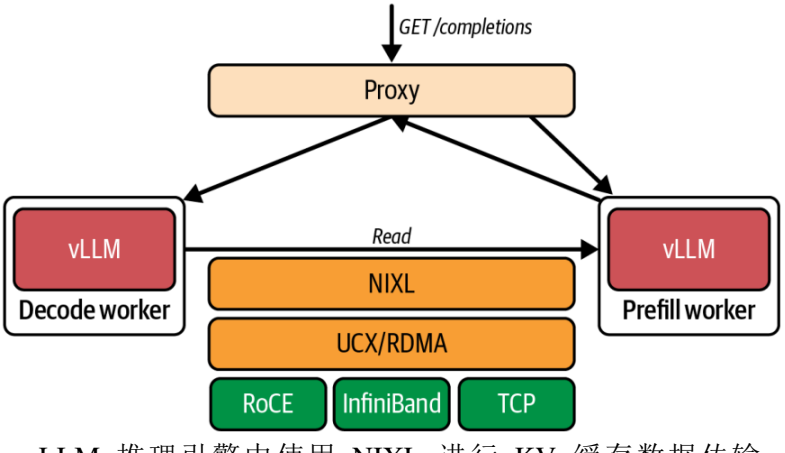

图 15-3 vLLM 推理引擎中使用 NIXL 进行 KV 缓存数据传输：节点内：NVLink/NVSwitch（设备到设备）；节点间：通过 ConnectX 级 NIC 使用 GPUDirect RDMA（InfiniBand/RoCEv2）。

预填充和解码工作节点的放置应遵循 fabric。同节点放置通过 CUDA 点对点访问，让 KV 传输留在 NVLink 和 NVSwitch 上；跨节点放置则应通过 InfiniBand 或 RoCEv2 使用 GPUDirect RDMA。NVIDIA Dynamo 与 NIXL 集成，在跨节点的 GPU、CPU 内存和存储之间移动 KV 缓存；vLLM 则通过 LMCache 和 NIXL 集成，实现分离式预填充。

> **注** 当 fabric 或虚拟化层阻止直接点对点访问时，NIXL 可以回退到主机暂存路径，但这不是最优路径。请始终在实际部署中验证端到端 KV 传输时间。

15.1.5 使用 Kubernetes 部署分离式预填充与解码

在高级部署中，像 Kubernetes 这样的集群编排系统可以根据负载和输入特征，动态调整 GPU 池分配——或分别扩展各个池。例如，如果许多用户带着超长提示和相对较短的输出到达（例如大文档摘要用例），Kubernetes 可以临时把分配调整为在预填充池中使用更多 GPU。这会减少分配给解码阶段的 GPU 数量。

相反，如果许多用户请求超长输出（例如长推理链、“逐步思考”等），就可以把更多 GPU 转移到解码池。两种情况下，都可以为每种工作节点类型横向扩展新实例。

图 15-4 展示了一个基于 Kubernetes 的分布式 vLLM 集群，使用开源 llm-d 项目把预填充和解码工作节点分开。vLLM 通过运行两个实例并使用 LMCache 和 NIXL 交接 KV，来实现分离式预填充；llm-d 则用 Kubernetes 原生编排扩展了这一点，支持分离式服务和 KV 感知路由。该图展示了一个名为 variant 自动扩缩器的组件，它负责更新池中预填充和解码工作节点的副本数。

> **注** 在现代推理部署中，所有节点都可以执行预填充和解码功能，因为它们共享相同的运行时和代码库（例如 vLLM、SGLang、NVIDIA Dynamo 等）。由集群编排器决定给它们分配特定角色——预填充或解码——既可在启动时静态指定，也可在工作节点生命周期内动态调整。

总体而言，分离式预填充/解码架构为高吞吐、低延迟的 LLM 服务提供了基础。不过，它确实引入了复杂性，因为必须传输和管理中间数据——长提示的 KV 缓存可达数 GB 量级。调度也更复杂，但在超大规模下，高效利用硬件带来的收益非常显著。

> **注** 第 17 章和第 18 章将更深入地介绍分离式 PD 的其他技术，包括高级调度、路由和部署优化。

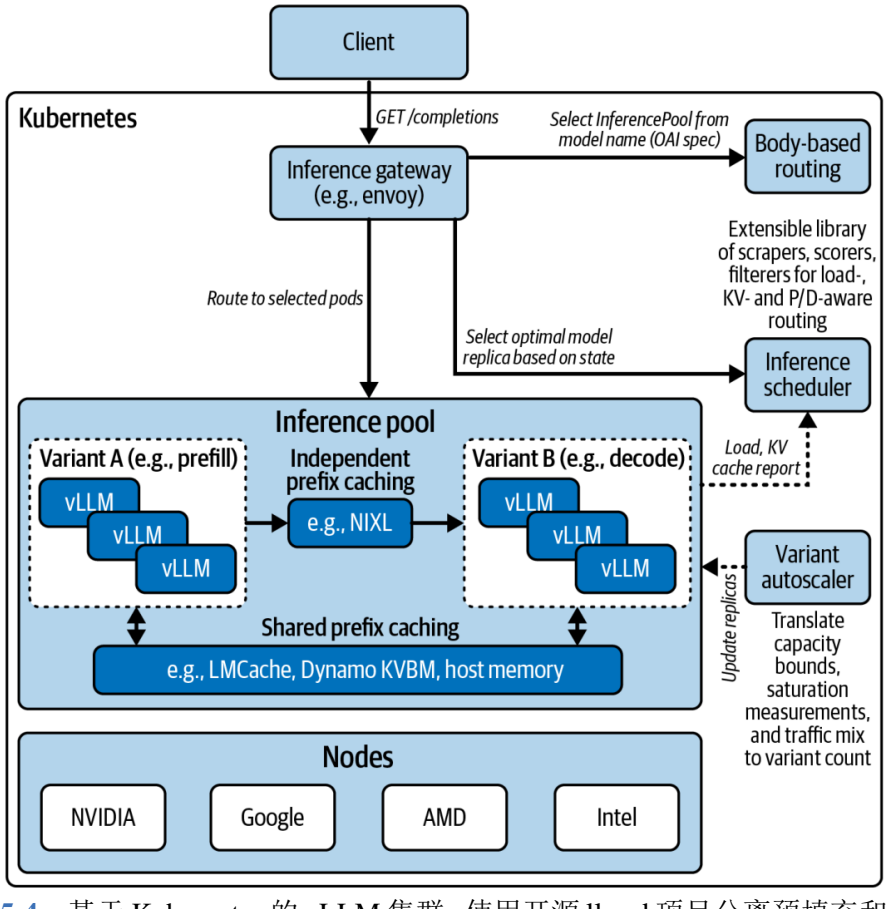

图 15-4　基于 Kubernetes 的 vLLM 集群，使用开源 llm-d 项目分离预填充和解码工作节点；variant 自动扩缩器根据提示一响应组合调整预填充/解码副本数；KV 使用 LMCache 和 NIXL 传输。

15.2 服务巨型 MoE 模型的并行策略

由于 GPU 内存有限，高效服务巨型 MoE 模型需要多种并行形式。我们将分解关键的并行策略，包括张量并行、流水线并行、专家并行、数据并行和上下文并行。还会讨论如何组合它们，把 LLM 分布到许多 GPU 上。表 15-1 对这些策略、典型用途以及它们与推理相关的详细说明做了高层总结。

> 注　对于 Blackwell NVL72 上的节点内 TP，最好让 TP 组保持在同一 NVSwitch 域内；只有在拓扑允许时才扩展到机架之间，以避免额外跳数。

表 15-1　LLM 推理的并行策略。

并行策略	切分依据	使用场景	优点	缺点
张量并行	在每个层内（将神经网络权重矩阵拆分到多个 GPU）	单个模型太大，或需要跨多个 GPU 加速层内的高强度计算	计算受限层可获近线性加速；all-reduce 通信与计算重叠，降低开销	每层都有频繁通信（all-reduce）；需要高带宽互连（NVLink/NVSwitch）；跨节点和慢网络效率较低，因为延迟高（建议将张量并行组保留在单节点内）
流水线并行	按层序列把模型切分到不同 GPU 上（不同层位于不同 GPU）	极深模型无法放入单个 GPU；跨层扩展内存	可分布模型状态；使用微批处理同时处理来自不同用户/请求的多个 token；长序列时可并行处理多层（提高大批次或长输入的吞吐量）	增加流水线填充/排空延迟（气泡）——不适合逐 token 解码；实现更复杂，激活内存占用更高（必须存储流水线阶段之间的中间激活）
专家并行	不同 MoE 位于不同 GPU（每个 token 稀疏激活）	拥有大量专家的巨型 MoE 模型；需要把模型参数分片到多个 GPU	使模型规模几乎不受限制——总参数随 GPU 数量扩展；每块 GPU 只计算一部分 token（稀疏计算）；高参数数量提升模型容量/质量	每个 MoE 层都有高运行时通信开销（all-to-all）；门控不均时可能负载不均衡；每块 GPU 上的每个专家必须有足够工作（token）来摊薄通信
数据并行	在多个 GPU 上复制整个模型，每个 GPU 服务不同请求	模型部署固定后扩展吞吐量（更多并发请求）；面向多用户的多实例服务	吞吐量近乎线性扩展；实现简单（无需模型分区）	不改善单个查询的延迟（即每查询延迟）；内存使用成倍增加（每个副本使用完整内存）；若有状态（例如缓存）必须处理一致性，或使用无状态模型调用
上下文并行	在每个层把输入序列 token 划分到多个 GPU	超长序列（例如 100k+ token），以降低提示延迟和每 GPU 内存	长上下文预填充可获得近线性加速；通过拆分 KV 缓存，可处理超过单个 GPU 内存的上下文	需要自定义 attention 算法来处理跨分区 attention；每层为边界 token 增加通信；由于额外通信开销，仅对超长上下文（100k+ token）有利

这些并行策略定义了模型权重和数据如何在 GPU 上切分。图 15-5 展示了不同并行策略——以及常见策略组合——的切分方式。

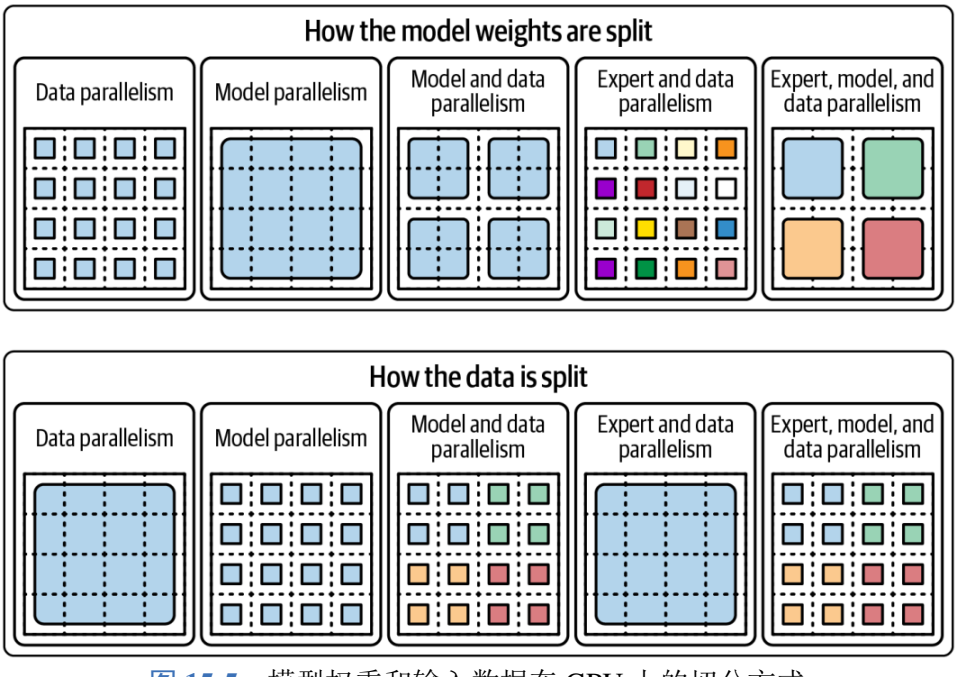

图 15-5　模型权重和输入数据在 GPU 上的切分方式。

15.2.1 张量并行

张量并行（TP）把神经网络每一层内的计算拆分到多个 GPU 上。例如，transformer 层中的大型矩阵乘法可以按列或按行划分——并在两块或更多 GPU 上并行计算。这些 GPU 随后通过执行 all-reduce 来聚合各自的局部输出，从而交换结果。

当模型的层（称为隐藏层）太大，无法放入单个 GPU 的内存时——或者当我们希望利用多个 GPU 并行处理同一层来加速单个模型实例时——通常会使用 TP。它让所有 GPU 在每一层的计算上保持同步，这要求极高的 GPU 间带宽才能高效运行。

理想情况下，TP 运行在通过高带宽 NVLink 和 NVSwitch 互连的 GPU 上。在使用第五代 NVLink 的多 GPU Blackwell 系统上（聚合双向 GPU 到 GPU 带

宽最高 1.8 TB/s，并具有高级网络拓扑），TP 可以在一台 8‑16 GPU 的节点内高效扩展——在 NVL72 机架的情况下甚至可以扩展到 72 块 GPU。请记住，在 GB200/GB300 NVL72 机架内，NVLink Switch 在 72 GPU 域内提供约 130 TB/s 的聚合 GPU 带宽。这是非常大的机架内带宽。

只要集合通信（例如激活值的 all-reduce）相对于计算足够快，TP 就能为模型的计算受限部分提供近线性加速。在推理中，张量并行主要应用于注意力投影和前馈多层感知机（MLP）网络的大型矩阵乘法。

由于单个 token 的激活值相对较小，这些 all-reduce 通信在 NVLink 上开销不算太高。因此，TP 是一种常见策略，用于把巨型稠密模型拆分到多个 GPU 上，而无需近似模型权重。

我们主要用 TP 来服务那些单个 MoE 专家或 transformer 层对单块 GPU 来说过大的模型。不过，当我们希望通过把每一层的计算并行到多个 GPU 上来降低延迟时，也会使用它。

但必须注意，超过一定规模后，TP 效率会下降——尤其是在通过 InfiniBand 或以太网跨节点或跨机架使用时。实践中，最好把 TP 用于节点内通信——或用于 NVLink 域内的节点之间。

像 NVIDIA GB200/GB300 NVL72 这样的现代多 GPU 机架，允许 TP 在更大规模上使用而不会耗尽互连带宽。NVLink Switch 把 NVLink 域扩展到整个机架，每个 NVL72 有 72 块 GPU。NVLink Switch System 可以把全对全、全连接的 NVLink fabric 扩展到最多 8 个机架，即 576 块 GPU（576 GPU = 8 机架 × 每机架 72 GPU）。这不仅能在单个 NVL72 内，也能跨多个机架启用大规模并行策略。例如，在 8 机架拓扑中，你可以使用 8 个机架上的 576 块 GPU 扩展到 576 路张量并行（576 GPU = 8 机架 × 每机架 72 GPU）。

> **注** 虽然可以把 TP 扩展到跨机架，但建议在选择 TP 组大小时考虑拓扑感知，并尽可能保持在单个 NVLink/NVSwitch 岛内。这样可以避免不必要的机架间交换机延迟，并帮助提高整体系统效率。

15.2.2 流水线并行

流水线并行（PP）按层把模型划分到不同 GPU 上。例如，在一个 60 层 transformer 模型中，GPU 0 可以持有第 1–20 层，GPU 1 持有第 21–40 层，GPU 2 持有第 41–60 层。

处理 token 序列时，数据按顺序流经这些 GPU：GPU 0 计算第 1–20 层，然后把中间激活传给 GPU 1 计算第 21–40 层，依此类推。这样就能分布过深、无法放入单个加速器的模型。

在推理中，PP 可以通过把模型划分到多个批次来提高吞吐量——类似于装配线。在预填充阶段处理长输入 token 序列时，PP 以交错方式把序列的不同部分流入各层，从而实现较高的 GPU 利用率。

相比之下，在解码阶段一次生成一个 token 时，纯 PP 带来的收益较小，因为每个新 token 仍必须依次经过流水线各阶段。这会产生流水线气泡或空闲期：对于每一个 token，较早阶段都要等待较晚阶段完成。

为减少流水线气泡，实现中会使用微批处理，让流水线可以同时处理来自不同请求的多个 token。尽管如此，流水线并行主要还是通过允许超大型模型把层拆分到多个 GPU 来帮助扩展内存。它也有助于吞吐量——尤其是在处理大批次或长输入时。

由于流水线阶段之间的传输开销，PP 往往会增加单个项目的端到端延迟。因此，选择 PP 通常是出于模型容量原因——而不是延迟原因。这是因为它能让模型放入内存。当没有单块 GPU 能容纳整个模型时，轻微的延迟损失是可以接受的。

此外，PP 常与 TP 等其他并行策略组合使用，以平衡内存和速度。服务大型 MoE 模型时，可以使用跨 2–4 块 GPU 的流水线并行来拆分深层堆栈——同时依靠 TP 处理宽大的层内计算。

> **注** PP 按层切分模型，TP 在层内切分模型。TP 主要用于层或专家对单块 GPU 来说过宽的模型，而 PP 主要用于过深、无法放入单块 GPU 的模型。注意，TP 和 PP 可以组合使用，我们稍后会看到。

15.2.3 专家并行

专家并行（EP）是 MoE 架构特有的。在一个 MoE 层中，有许多专家网络，即前馈子层。对于每个输入 token，只有一个或少数几个专家被激活。这自然适合把不同专家分布到不同 GPU 上。

例如，如果一个 MoE 层有 16 个专家，而我们有 4 块 GPU，那么每块 GPU 可以托管 4 个专家。推理时，当一个 token 到达该 MoE 层，其内部门控网络会为该 token 选择例如 top-2 专家。然后，该 token 的数据被发送到拥有这两个专家的 GPU 进行处理。接着，结果被合并回来以生成下一个 token，如图 15-6 所示。

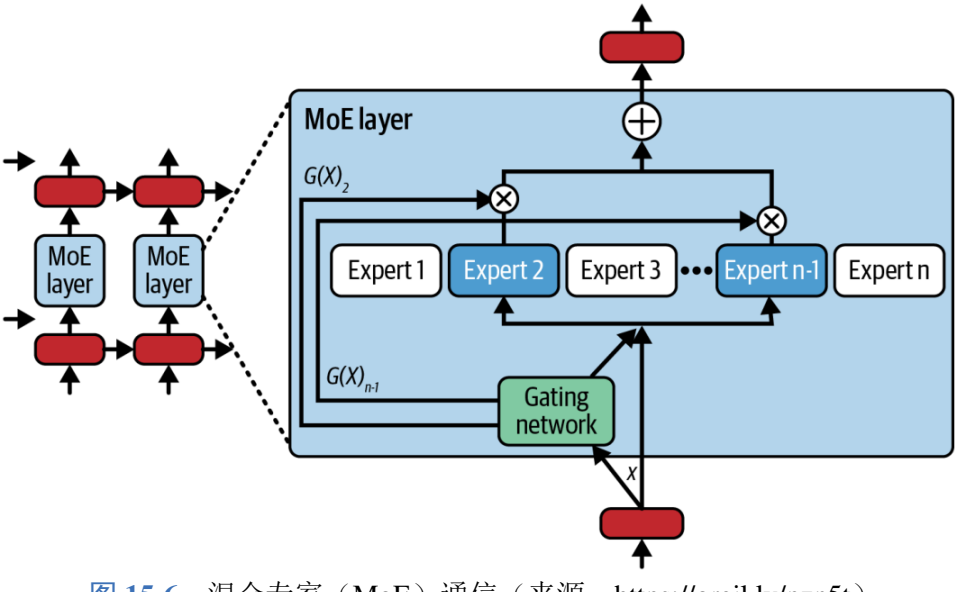

图 15-6　混合专家（MoE）通信（来源：https://oreil.ly/pzn5t）。

实现这一点需要 all-to-all 通信模式：token（严格说是它们的激活向量）在 GPU 之间动态重排，使每个 token 落到其被分配专家的 GPU 上。每个专家为其分配的 token 计算完输出后，再执行一次 all-to-all，把 token 输出按原始顺序送回。

这种动态路由在每个 MoE 层引入一个通信密集的步骤。如果没有仔细优化，它可能成为性能瓶颈——尤其是随着专家/GPU 数量增加。好处是，专家并行允许总模型容量几乎随 GPU 数量线性扩展。

考虑一个有 100 个专家、分布在 100 块 GPU 上的 MoE。如果每个 token 只激活两个专家，那么每个 token 的计算量与一个较小的稠密模型相似。因此，正是专家并行让一些巨型 MoE 模型得以服务，因为模型权重被分片到许多 GPU 上——每块 GPU 在任意给定层只处理一小部分 token。

> **注** 在规模化运行且 MoE 模型中的专家集合得到适当平衡时，所有专家很可能同时处于活动状态。因此，虽然单个 token 只激活少量专家，但汇总所有最终用户的所有 token 后，所有专家都会处于活动状态，同时消耗并争用全部 GPU 资源。

高效的专家并行推理需要仔细的负载均衡——以及高带宽互连，例如用于机架内通信的 NVLink/NVSwitch 和用于节点间通信的 InfiniBand/RDMA。现代 MoE 推理框架使用 NCCL 等快速集合通信库。通常，把多个专家分组到每块 GPU 上有助于减少通信步骤。

现代 MoE 推理常使用 top-2 门控，即每个 token 被分配给两个专家。为减少通信，可以把经常配对的专家放到同一块 GPU 或同一个计算节点上。例如，如果 token 使用两个专家，把这两个经常配对的专家放在同一块 GPU 或节点上，就意味着许多 token 分配会留在单块 GPU 或单个节点内，从而本地化通信流量并降低开销。

另一种技术是使用 top-1 门控，即每个 token 只去一个专家。与前面描述的 top-2 门控相比，这会减少通信量，因为 top-2 会使专家输出数量翻倍。虽然 top-1 门控更快，但可能导致模型质量下降和负载不均。

> **注** Google 的 GLaM 引入了负载均衡损失（以及门控噪声），以在 MoE 模型训练期间实现均衡的专家使用。在此基础上，研究人员正在探索真正自适应的推理时门控：当某个专家过载时，利用实时负载指标重新路由 token。这些方法能在不降低质量的情况下提高利用率。

> **注** 然而，在大多数生产服务环境中，采用适度容量因子的 top-2 门控——再加上偶尔基于负载的专家交换或复制——仍然是平衡质量与性能的最常见折中方案。

许多 MoE LLM 至少使用 top-2 门控，以获得更好的质量、良好的速度和均衡的负载。此外，战略性地放置专家——甚至使用备用专家副本以避免专家过载——也很重要。

当专家过载或变“热”时，如果门控网络不成比例地把 token 路由给它们，它们就可能成为吞吐量瓶颈。这就是所谓的落后者效应（straggler effect）：必须等所有专家计算完成后，才能继续推进。因此，一个过载的专家接收到的 token 比其他专家多，就会拖住推理流水线。这会让一些专家及其 GPU 空闲，等待其他专家追赶。

为防止这种情况，高性能 MoE 服务系统会暴露一个容量因子（capacity factor）参数，通常设为平均 token 负载的 1.2–1.5 倍，用来限制每个专家每批最多能处理多少 token。超过该上限的 token 会被路由到第二选择的溢出专家——或排队等待第二轮处理。这补充了训练期间用于鼓励 MoE 把 token 均匀分配给专家的负载均衡损失或门控噪声。

一些功能完整的推理服务器还可以把热专家复制到多块 GPU 上，以便在必要时分担负载。代价是额外的 GPU 内存。稍后我们会看到一个负载不均衡的例子。

推理时的溢出（容量因子）、训练时的惩罚（负载均衡损失和门控噪声）以及热专家副本三者结合，应在容量达到上限时平滑 token—专家负载分布——从而最大化整体 MoE 推理吞吐量。

15.2.4 数据并行

在推理中，数据并行（DP）指在多块 GPU 上复制整个模型，并把不同的传入请求——或请求批次的分片——分配给每块 GPU。与训练中的数据并行（通过梯度平均让模型保持同步）不同，推理中的数据并行在每个副本上运行独立的前向传播。这是横向扩展吞吐量最简单的方式。

例如，使用 DP 时，如果一块 GPU 每秒能处理 10 个请求，那么使用 8 块 GPU 和 8 个副本，理想情况下可达到每秒 80 个请求的吞吐量——假设请求相互独立。实践中，用数据并行做推理需要启动多个模型服务引擎，并在它们之间对请求做负载均衡。

使用 DP 时，每块 GPU 或每组 GPU 从头到尾处理查询的一个子集。优点是吞吐量线性扩展，而且推理期间 GPU 之间无需通信，因为每个副本独立运行。不过，缺点也很显著。

DP 会成倍增加内存需求，因为每个副本都需要模型权重和缓存的完整副本。因此，用 8 个副本服务一个模型，相比在单块 GPU 上托管该模型，会使用 8 倍的 GPU 内存——以及大约 8 倍的硬件成本。

实践中，数据并行常与请求批处理以及每块 GPU 上的多流执行结合使用，以最大化利用率。每个副本最好是无状态的——或使用同步缓存以避免一致性问题。

需要注意，DP 不会降低任何单个请求的延迟。不过，由于有更多副本可用于处理请求，它会提高吞吐量——前提是内存和计算资源可用且相对没有争用。

> **注** 由于 GPU 内存相对稀缺且昂贵，通常只有当吞吐量需求非常高时，或者能把 DP 与 TP、PP 等其他并行方法结合时，DP 才值得用于推理。

在推理中，由于 DP 额外的内存和成本要求，它常与 TP、PP 等其他并行方法结合使用。例如，如果模型因内存限制必须跨 8 块 GPU 分布，就应使用 DP 结合 TP、PP 或 EP——甚至全部一起——形成四维（4D）并行（再加上上下文并行（CP），你就在使用 5D 并行！）。

具体来说，可以用 DP 结合 TP，通过两个各含 8 块 GPU 的 DP 组来部署两个大型的 8-GPU 模型副本。这会让吞吐量翻倍，并用 TP 把每个大型模型副本在其层内分片到 8 块 GPU 上。

在巨型推理集群中，你可以投入大量 GPU 和节点来服务大模型的多个副本，以处理高请求量。当吞吐量需求超出单个模型实例所能提供的水平时，DP 尤其有用。实践中，生产系统常运行许多大模型的 DP 副本来满足流量需求。

> **注** 现代推理服务器把每个副本视为负载均衡器后面的独立模型实例。这需要仔细的请求路由，但与其他复杂的模型分片方法相比，相对直接。

15.2.5 上下文（序列）并行

上下文并行（CP）更像一种专用策略，它把单个 token 序列划分到多块 GPU 上。这种技术有利于数万到数百万 token 量级的超长上下文输入；否则这些输入会太慢、内存占用过高，或根本无法放入单块 GPU。

CP 的思路是把序列切成块，让不同 GPU 在每一层并行处理序列的不同部分。GPU 只在序列块之间的边界交换必要信息。

CP 通过把 KV 缓存拆分到多块 GPU 上，处理超过单块 GPU 内存的上下文。它还会按所用 GPU 数量大致成比例地降低提示处理延迟。因此，CP 可以利用多块 GPU 并行执行长上下文预填充，从而为超长上下文预填充运行实现近线性加速。

CP 的挑战在于，transformer 具有全局自注意力，每个 token 都要关注所有更早的 token。朴素地切分序列会要求 GPU 之间进行大量信息交换，以计算跨分区边界的注意力。

CP 方法使用环形并行和分块注意力等巧妙方案，降低二次方自注意力通信的时间复杂度——并让每块 GPU 主要只关注其上下文分区内的内容，以及少量块边界数据。

实际上，每块 GPU 在每一层处理输入序列位置的一个子集。它们以流水线方式传递中间结果——通常排成环形。CP 类似于流水线并行，但沿着序列长度维度而非层维度进行。

上下文并行在超长输入的预填充中表现出色：它把文档切成片段，把片段分配到多块 GPU 上，并并发处理每个片段。GPU 数量每翻一倍，提示处理时间大约减半——只需为边界处的跨片段注意力付出少量开销。

简言之，虽然 CP 不会加速串行的逐 token 解码阶段，但它可以缩短超长提示的 TTFT。它通过把注意力 KV 缓存分布到多个设备上来做到这一点，使每块 GPU 只存储其片段的缓存。CP 还让你能够处理超过单块 GPU 内存容量的上下文。

> **注** 上下文并行需要一种能跨分区通信的注意力实现。这会增加每层的额外通信。因此，对于短提示，CP 往往会带来过大的开销。但对于超长输入

和严格的 TTFT 目标，CP 可以降低预填充延迟和每 GPU 内存。请针对你的最大上下文测量预填充速度和准确率。

15.2.6 混合并行

实践中，服务巨型 MoE LLM 会组合使用前述并行策略。当今的 LLM 模型如此庞大复杂，单一并行方法已经不够。图 15-7 展示了一个使用四块 GPU 的混合并行配置。

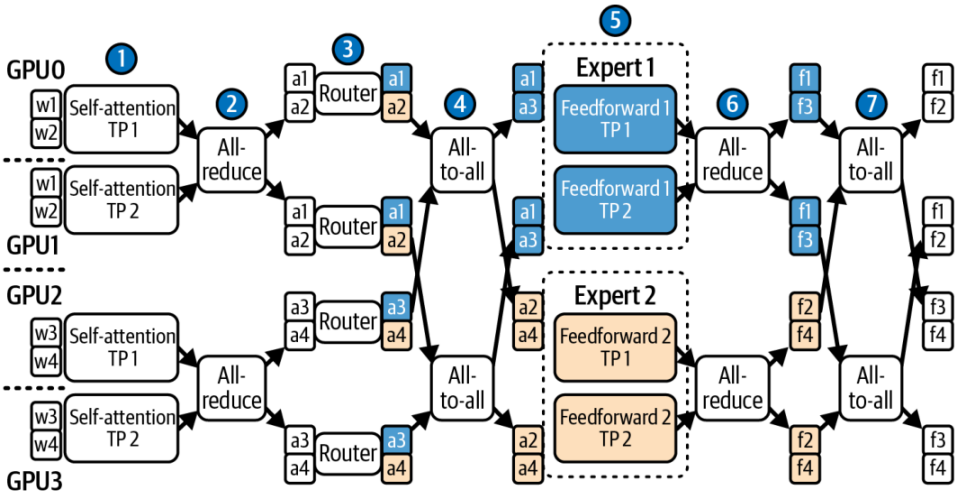

图 15-7　4 × 2 × EP 混合并行组合的高层示意图（来源：https://oreil.ly/q1AEf）。

这里，我们使用四个流水线阶段（每块 GPU 一个）和两路张量并行。token 通过专家并行在两个专家之间路由。这称为 4 × 2 EP 混合并行策略。

让我们进一步扩大规模，创建 GPU 逻辑组。例如，考虑一个 64 GPU 的集群。我们可以把 GPU 分成 16 组，每组 4 块 GPU。使用一个有 60 层、64 个专家的 MoE，可以采用 4 路流水线并行，每个阶段 15 层。这切分了模型的深度。然后，在每个阶段（15 层）内，可以使用 2 路 TP 来切分每层内的大型矩阵乘法。这切分了模型的宽度。

对于 MoE 层，可以使用 EP 通过 top-2 门控在每组内分布 16 个专家。这样，token 最多会被发送到组内的两块 GPU。最后，数据并行可以部署两个这样的 64 GPU 副本来让系统吞吐量翻倍。

这只是一个配置。实践中，你需要尝试不同的并行策略组合。性能分析工具会帮助你找到合适的平衡。具体来说，可以用 Nsight Systems 做端到端跟踪，用 Nsight Compute 获取 kernel 级 GPU 指标。

> **注** 记住，在每次更改前后都要验证互连流量、Tensor Core 利用率以及其他性能增强机制。

指导原则是：使用 TP 直到收益递减点——通常是在节点内，或在像 NVL72 机箱这样紧耦合的单元内。然后，尽量少用流水线并行——刚好足以让模型放入内存。接着，最大化 EP，把 MoE 参数分布到专家和 GPU 上。最后，随着扩展到越来越多最终用户和并发推理请求，增加数据并行副本来提高吞吐量（如果预期有超长输入，还可以选择在顶层叠加 CP）。

我们还应让并行化与硬件拓扑对齐。例如，如果使用 NVL72 系统，全部 72 块 GPU 都通过 NVLink 和 NVSwitch 全连接。这种情况下，可以在 72 GPU 的 NVLink 域内组建 TP 和 EP 组。然而，接近整个域的 TP 组往往会因 all-reduce 延迟而收益递减。因此，许多生产系统即使在 NVL72 内也保持 TP 组较小且拓扑感知。

相比之下，一个由两个 8-GPU 节点通过 InfiniBand 连接的小型集群，只在每个节点内具有完整的 NVLink 连接——跨节点流量要经过 InfiniBand fabric。在这种环境中，最好让 TP 局限在每个 8-GPU 节点内——并尽可能避免节点间并行，以避免节点间通信更高的延迟和更低的带宽。

在接下来的几节中，我们讨论可与这些并行策略结合的高级优化，以在多节点集群上实现更快的推理。我们将会看到，设计良好的服务系统会组合许多这类高层和低层技术。

15.3 推测解码与并行 token 生成技术

LLM 推理中的一个根本性能挑战是解码的串行性。请记住，在预填充阶段处理完初始提示后，模型通常一次生成一个 token。每个 token 的计算都依赖于前一个 token 的结果，因此很难并行化，因为这本质上是一个串行过程，会引入

延迟瓶颈——即使使用最新、最快的 GPU 也是如此。在现代 GPU 硬件上，为巨型模型顺序生成数百个 token 可能需要数秒量级的时间。

本节讨论几种加速解码阶段的技术。其中一些技术涉及在每次推理步骤中生成多个 token，另一些则减少串行推理步骤的总数。

推测解码传统上把一个小而快的“草稿”模型与一个更大的“目标”模型配对：草稿模型在一批中提出多个 token，目标模型随后验证每个候选。这用第二次推理传递换取并行生成，从而实现更高的多 token 吞吐量。然而，运行两个独立模型会增加部署复杂性，而且当验证传递成为瓶颈时，仍可能拖慢推理。

Medusa 通过把多个轻量解码头直接附加到单个 LLM 上来简化这一点。每一步，这些头使用基于树的注意力机制，在一次前向传播中并发生成并验证多个候选 token。

Medusa 的这种统一设计避免了跨模型的 token 传输，并在不牺牲 token 生成质量的情况下实现大幅加速。通过把草稿—验证整合进单模型传递，Medusa 是对传统双模型推测解码技术的改进。让我们仔细看看其中一些技术。

15.3.1 双模型、基于草稿的推测解码与 EAGLE

推测解码是一种技术，它用在小模型上做额外工作来节省在昂贵大模型上的时间。思路是在主 LLM 旁边运行一个轻量级“草稿”LLM。草稿模型更快，会推测性地生成当前上下文之后的 k 个 token 批次。

大模型（称为目标模型）随后通过为整个 k token 序列预测下一 token 概率来验证草稿 token，并将该序列作为单批发送给 GPU。通过一次处理全部 k 个 token，目标模型提高了算术强度，因为每传输一字节数据能执行更多计算。这使草稿序列 token 的验证既高效又快速。

如果目标模型的输出与草稿模型提出的 k 个 token 一致，那么我们实际上就在大模型生成单个 token 的同一时间内生成了 k 个 token。如果大模型的验证在草稿模型生成的 k 个 token 序列中的任何位置与草稿预测发生分歧，那么该点之后的推测 token 都会被丢弃。每一步至少会保留一个生成的 token，因为验证过程总是接受来自草稿模型或目标模型的第一个 token。

一旦使用目标模型纠正后的 token 并继续解码，就可以从该点开始新的推测解码循环。图 15-8 展示了一个小型草稿模型向前预测多个 token，然后目标（大）模型逐一验证这些 token。

随着时间推移，推测解码减少了大模型所需的一对一顺序调用总次数。理论上，它提供 k 倍加速，其中 k 是草稿模型生成的 token 数。实践中，由于开销和偶尔的推测 token 拒绝，收益更接近 2 倍加速。

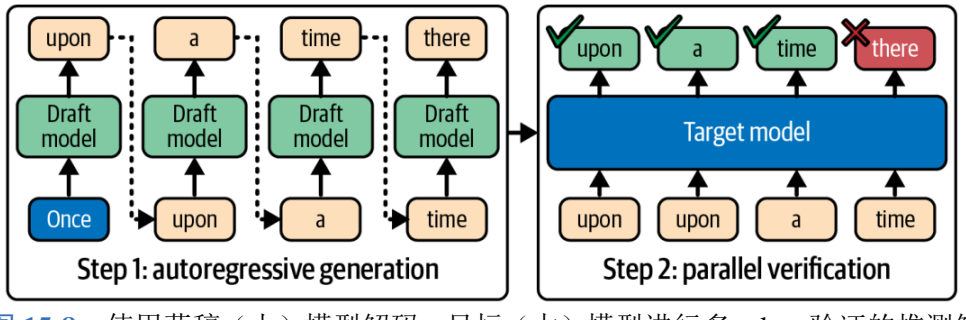

图 15-8　使用草稿（小）模型解码、目标（大）模型进行多 token 验证的推测解码。

必须选择与大模型分布具有相当高保真度的草稿模型。这意味着草稿模型的预测应与大模型可能输出的内容高度重叠。如果草稿经常猜错，推测解码几乎没有收益，还会浪费计算。如果它经常预测大目标模型不会生成的 token，许多推测 token 会被拒绝。这会浪费计算时间。

草稿模型还必须比大模型快得多——通常快 4 倍或更多——推测解码才能带来实际性能提升。一种常见策略是使用主模型的蒸馏版本——或在相同数据上微调的较小模型，使其输出良好相关。

> **注**　草稿模型必须使用与大模型相同的 tokenizer 和词表。这一点有时会被忽视，并会导致糟糕的结果。

草稿模型使用相同提示生成 token，并且可能使用更高温度，以提高匹配大模型某个可能续写的概率。与此同时，目标模型会向前跳过，在单个批次中处理所有草稿 token。

在标准推测解码接受流程下，目标模型的输出分布会被保留。因此，当采样设置对齐时，最终样本与大模型的分布一致。如果草稿发生分歧，目标模型

的验证会拒绝这些 token，正确性得以保持。唯一区别是，当小模型对这些推测 token 猜对时，最多可避免 k 次目标模型调用。

推测解码可以用多种方式实现。大多数现代 LLM 推理引擎（如 vLLM、SGLang 和 TensorRT-LLM）都内置支持协调草稿模型和目标模型的生成。根据经验，1.5–2.5 倍的加速很常见；通过仔细的批处理和草稿模型设计，还可能获得更高收益。

> **注** 在 PyTorch 中，`torch.nn.functional.scaled_dot_product_attention` 操作会根据设备代际、形状、掩码和 dtype 自动选择最优后端（FlashAttention 或 cuDNN kernel）。你可以用 `torch.nn.attention.sdpa_kernel()` 并指定显式的 SDPABackend 类型来固定后端。例如，在基准测试 LLM 解码时，务必验证融合后端是否已启用。

例如，Greater LLM Efficiency 外推算法（EAGLE）通过在特征层而非 token 层操作，重新思考了推测解码。EAGLE 使用大模型自身中间表示的一步外推来预测下一个 token 的特征。这解决了不确定性，并实现更高的接受率。

EAGLE 报告，在 4 token 草稿下，相比原始解码最高可获得约 3.5 倍加速，同时保持大模型的输出分布。在有利环境中，EAGLE 接近其草稿深度的理论极限。这表明，借助创新技术，推测解码可以进一步缩短解码时间——同时保持输出质量。

EAGLE-2 通过引入上下文感知的动态草稿树来扩展 EAGLE。虽然 EAGLE 在某些评估中相比原始解码最高实现 3.5 倍加速，但 EAGLE-2 的方法报告称，根据任务和模型不同，比 EAGLE 快约 20%–40%。使用 EAGLE-2 时，草稿模型会生成一组分支 token 序列，进一步提高验证步骤的并行度。如图 15-9 所示。

EAGLE-3 通过偏好直接 token 预测并融合多层特征，继续改进此前的 EAGLE-1 和 EAGLE-2。EAGLE-3 报告在某些任务上比 EAGLE-2 最高提升 1.4 倍——相比未优化的基线变体最高加速 6.5 倍。在 EAGLE-1 和 EAGLE-2 中，草稿模型预测内部特征向量，再将其解码为 token。这些早期 EAGLE 方法本质上是通过猜测内部特征——再映射到 token 来工作。

EAGLE-3 跳过特征级预测步骤，更直接地预测草稿 token。不过，它仍使用内部特征，但将其融合为多层表示（低层、中层、高层）来引导草稿预测。这与

仅使用顶层形成对比。这使 EAGLE-3 更精简、限制更少——从而实现更好的扩展。

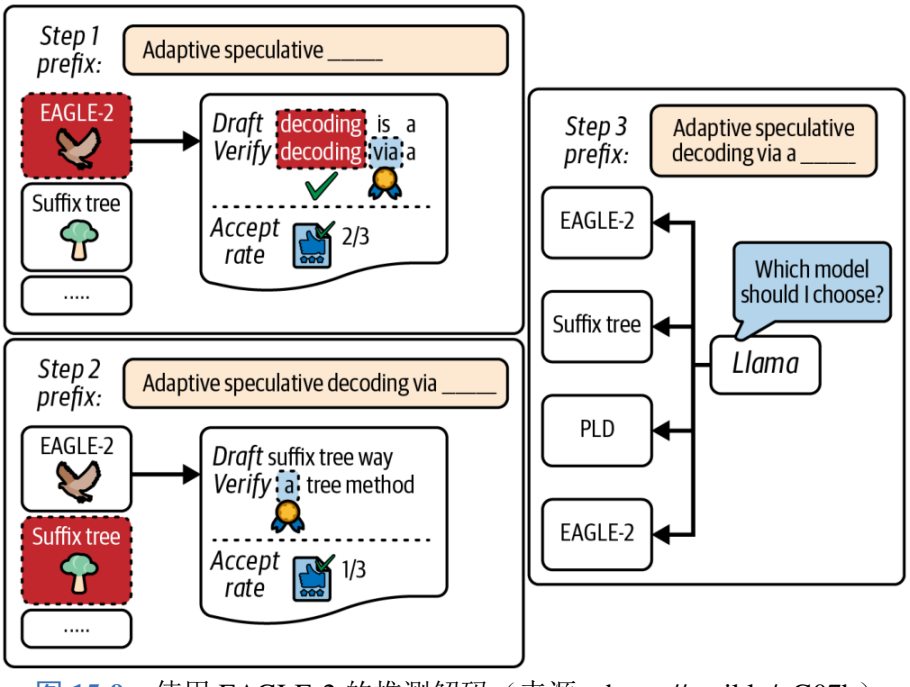

图 15-9　使用 EAGLE-2 的推测解码（来源：https://oreil.ly/uG07b）。

另一种技术是动态深度解码，它可以自适应地跳过对输出质量影响最小的层。其他减少计算的技术包括跳过每第 N 个 transformer 层、对草稿模型使用较低精度（例如 FFP8 和 NVFP4），以及在草稿阶段临时使用较小的隐藏维度。

> 注　一些研究原型提供在解码期间跳过部分网络或降低精度的模式，以用准确率换取速度。截至撰写本文时，这些模式尚未在生产模型中普遍可用，因此在启用任何此类模式之前，请务必在目标任务上验证质量和速度。

未来的 LLM 可能包含优化的“快速生成”模式。例如，模型可能具有替代的轻量层，或用于降精度解码的配置。这种内置优化将允许模型跳过计算。

15.3.2 单模型自推测解码

另一种推测解码方法是完全避免使用外部草稿模型，而让较大的目标模型通过选择性跳过一些计算来同时起草和验证自己的输出。其中一种方法是自推测解码，也称为草稿—验证方案。

在自推测解码中，大型目标模型使用一次快速、近似的传递生成 k 个 token。例如，它可以选择只运行一半的层——并且可能使用降低的精度——来生成每个新 token。它会跳过每隔一层。这样产生的草稿输出很像较小草稿模型会产生的内容。并且在接受率和草稿深度有利时，可以获得类似的加速。这样产生的草稿输出很像较小草稿模型会产生的内容——同样可以实现约 2 倍加速。

然后，在自推测解码的第二阶段，目标模型执行完整前向传播，一次性验证这 k 个 token。如果全部匹配，我们就为这些 token 节省了一半层的执行。如果不匹配，我们就回退到传统推测方法，接受不匹配之前的 token，并正常继续。

由于自推测解码中起草和验证由同一个模型完成，因此无需训练、维护或在运行时加载单独模型。挑战在于找到一种好方法，减少草稿阶段所需的计算量（丢弃层、降低精度等），同时不让准确率下降太多。

一种相关技术是一致性解码（consistent decoding），即训练单个 LLM 同时生成和验证多个 token。这种单模型方法无需单独草稿模型即可实现约 3 倍加速。这显示出把推测解码固化到模型自身权重中的趋势。

这些方法是非常活跃的研究领域。它们尤其令人兴奋和有前景，因为它们让模型利用自身固有的内部冗余来加速自己。而且由于优化是模型局部的，推理引擎的实现可以简化。

15.3.3 使用 Medusa 多头的多 token 解码

推测解码最终仍使用草稿模型逐个生成 token——只是因为它更小所以更快。然而，Medusa 框架采取了更激进的方法。它修改模型架构本身，以在每个解码步骤并行预测多个新 token。

与仍然逐个生成 token（尽管更快）的双模型推测解码不同，Medusa 的架构真正从单个模型在每次迭代中生成多个 token。因此，Medusa 的多头方法在已发表实验中报告，Medusa-1 和 Medusa-2 均可获得约 2.2–3.6 倍加速。

然而，Medusa 需要自定义模型训练，因为它通过在某些层分支出额外解码头来修改基于 transformer 的 LLM——Medusa 之名由此而来。这让模型可以同时提出多个后续 token。

不同 Medusa 头生成的多个候选 token 组织成树状。例如，Medusa 可以在一次传递中生成深度为 2 的二叉树，最多产生 4 个 token。然后它可以并行验证这个多 token 序列，如图 15-10 所示。

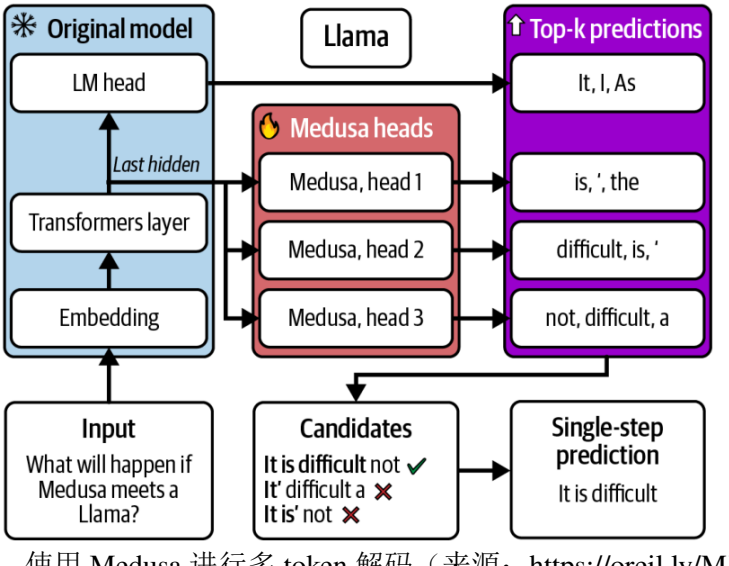

图 15-10　使用 Medusa 进行多 token 解码（来源：https://oreil.ly/MJMOQ）。

在内部，Medusa 使用专门的基于树的注意力模式，以提高并行 token 预测之间的一致性。模型学习并发地扩展和验证多 token 序列。借助 Medusa，LLM 本质上学会了“向前思考”几个 token——并一次性全部输出。

推理期间，Medusa 可以按并行预测的倍数减少顺序解码迭代次数。例如，如果 Medusa 在一次传递中预测 4 个 token，对于深度为 2 的二叉树输出，它可以把所需前向传播次数减少最多 4 倍。

然而实践中，Medusa 通常实现约 2–3 倍加速，因为偶尔预测分支验证失败并需要部分重做时会带来回溯开销。这与推测解码验证和拒绝的开销类似。例如，如果 Medusa LLM 每次迭代生成 4 个 token，那么 100 token 的回复只需 25 次模型迭代，而不是 100 次。

Medusa 需要修改并重新训练模型以添加这些额外的头。此外，由于额外的头参数，Medusa 模型体积略大。这会略微增加开发复杂性和训练成本。不过，一旦训练完成，Medusa 模型可以显著缩短推理时间。

如果针对你的用例进行训练，启用 Medusa 的 LLM 可以提供出色的低延迟性能。然而，这种技术需要修改并微调模型架构，以添加额外的头。

15.3.4 交错执行多个请求的解码步骤

另一种并行解码技术是把来自多个最终用户的多个并发请求的解码步骤交错执行。这种并行更像是推理引擎的能力，而不是算法技巧或模型架构修改。但它通过按 token 级别跨最终用户请求进行批处理，帮助 GPU 保持忙碌。

像 vLLM 这样的框架以请求路由、连续批处理和 token 调度的形式实现这一点。思路是通过填补顺序步骤之间的空隙来让 GPU 保持忙碌。具体来说，如果某块 GPU 因某个序列等待 I/O 或数据依赖而停住，调度器可以在此期间在同一块 GPU 上运行另一个序列的下一 token 预测。

> **注** 把推理引擎的高级路由、批处理和调度能力与 CUDA 流结合起来。然后用 Nsight Systems 验证 token 步进 kernel 与 NIC/NVLink 传输发生重叠，而不是在单个流上串行化。这样，你就可以在应用、系统和硬件层面实现大规模并行。

需要注意，交错解码步骤不会加快单个序列的延迟。事实上，由于额外的上下文切换，它甚至可能给每个 token 增加一点点开销。然而，在并发服务许多用户时，它可以大幅提高整体吞吐量和 GPU 利用率。这会降低重负载下的平均最终用户请求延迟。

15.3.5 组合解码技术并评估复杂度

值得注意的是，这些解码优化可以组合使用。例如，可以搭配一个启用 Medusa 的目标模型来使用推测解码，由大目标模型一次验证小模型预测的多个 token。

这些技术也带来额外复杂性。你要么必须维护一个额外模型，要么修改主模型，要么管理更复杂的控制逻辑。在生产中，应评估这种复杂性与性能收益。对于交互式应用，即使只减少几毫秒响应时间，也值得付出这种复杂性——尤其是在规模化时。

简言之，高级解码技术——无论是否使用草稿模型——以及 Medusa 式的多 token 预测，都可以减少传统上每次只生成一个 token 的现代自回归 LLM 的整体推理时间。通过一次生成更多 token、提高整体算术强度，并把解码步骤推向计算受限区间，你可以利用 GPU 的极限计算能力。

> **注** 解码优化领域持续演进，复杂度不断增加；但趋势很明确：让 LLM 解码更加并行、更加高效。

15.4 约束解码的性能影响

LLM 解码过程中一个独立但重要的方面是在特定约束下生成文本。例如，模型可以强制输出匹配预定义格式（JSON）、使用特定文法，或出于安全考虑禁止某些序列。因此，生产用例通常需要约束解码，通常称为结构化输出。

例如，OpenAI 的函数调用 API 依赖结构良好的输出格式来确定调用哪个函数——以及函数的输入和输出。这些约束旨在匹配函数的 API 签名，在应用层面没有商量余地。

约束解码会改变模型的 token 选择过程，使每一步只允许有效 token。这可能简单到提供允许 token 的列表——也可能复杂到嵌入形式文法并使用状态机过滤无效 token。

约束解码的缺点是执行这些约束需要额外延迟——通常每个 token 几毫秒。这是因为模型可能需要反复回溯、在剪枝后的词表中导航，并最终生成一个有

效 token。回溯发生在 LLM 的生成循环内部。因此，如果没有模型内部的细粒度遥测，调试、分析和调优约束解码可能会很困难。

另一个性能考量是缓存效率。在每个解码步骤剪枝完整概率分布可能冲刷缓存，并进一步降低吞吐量。当允许的 token 集合受到严格限制时，这一点尤其明显。

为缓解这些成本，许多框架会编译 JSON 文法，并为每个状态预计算有效 token。这使推理引擎可以在运行时屏蔽无效的 softmax 输出。这种方法减少回溯、改善缓存，并提高接受率。

对于简单 JSON schema 等中等约束，使用编译文法的现代引擎可以把掩码计算与 GPU 执行重叠，在规模化时把开销压到较低的个位数百分比范围；但复杂文法或小批次仍可能带来两位数百分比的开销。请务必在你的精确 schema 下测量 TTFT 和每秒 token 数。

这些技术已在流行的库和推理引擎中实现，包括 Hugging Face Transformers、NVIDIA NeMo 和 vLLM。例如，NVIDIA 的 TensorRT-LLM 和 Hugging Face Transformers 都允许用户定义词表掩码，以可忽略的速度损失执行约束。具体而言，TensorRT-LLM 通过 XGrammar 后端提供带 JSON schema 和上下文无关文法（CFG）支持的引导解码。vLLM 和 Transformers 则提供结构化输出 API。

> **注** 使用 TensorRT-LLM 的引导解码时，XGrammar 在 GPU 上提供 JSON schema 和 CFG 支持。XGrammar 会编译约束，避免大型 Python 侧 token 掩码开销。但要注意，某些配置可能回退到较慢的后端，并导致首次请求严重停顿。因此，保持文法紧凑很重要，以保留缓存局部性和 token 掩码带宽。

另一种流行策略是把约束解码下推到 kernel 本身。这种情况下，它会在最终 softmax 步骤注入 token 掩码，以获得类似的性能提升。

如果实现高效，约束解码常常可以跑得和非约束解码一样快——尤其是当约束规则集不太大或限制不太严格时。如果解码约束过于严格，解码实际上会变成在小型 token 空间中的搜索。这会导致更多回溯和更慢的 token 生成。

注 尽可能避免大型或过度严格的约束，例如文法、格式和词表。一种选择是让 LLM 正常解码，然后对输出做后处理或过滤。不过，这并不总是高效或可行的选项。请分析不同选项，并为你的用例选择最佳方案。

15.5 MoE 推理的动态路由策略

高效服务 MoE 模型需要使用专家并行，仔细地把专家划分到多个 GPU 上。此外，还需要一个智能的动态机制或门控网络，在运行时动态地把 token 路由到这些专家。

在 MoE 推理期间，每个 token 的前向传播都必须由门控网络路由到一个或多个专家 GPU。系统需要以均衡的方式处理这种通信，让 GPU 保持忙碌——并避免 GPU 过载或热点。让我们看看在高规模、多节点推理环境中如何解决这些问题。

15.5.1 专家通信优化

在专家之间进行 token 激活的 all-to-all 交换时，MoE 会在 GPU 之间重排一批 token。每块 GPU 只接收其托管专家所需的 token，如图 15-11 所示。这会在 MoE 的每一层发生，是代价高昂的操作；如果处理不高效，它可能主导推理时间。

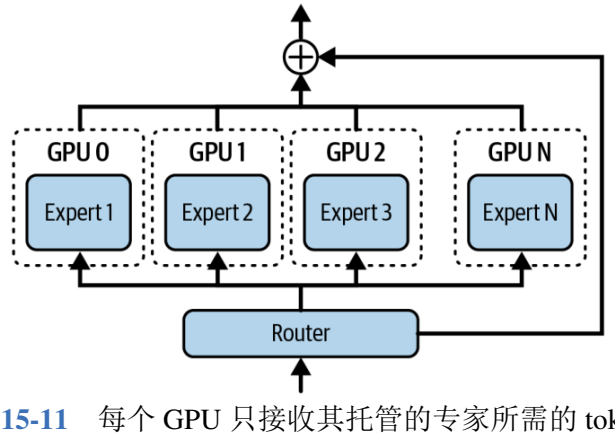

图 15-11 每个 GPU 只接收其托管的专家所需的 token。

减少通信开销的一种策略是对 GPU 集群采用分层路由：先使用 NVSwitch/NVLink（快速）在同一节点内的 GPU 之间路由 token，然后只对需要非本地专家的剩余 token 进行跨节点路由。这种两阶段 all-to-all 可以减少节点间流量。

此外，可以通过让通信与计算重叠来使用异步通信。高性能 MoE 推理服务器使用双缓冲通信，使得在发送一批 token 的同时，上一批 token 的专家计算并行进行。这种流水线方式隐藏了大部分通信延迟。

> **注** 如果节点间 fabric 保持充分利用，同时节点内重排在后台运行，就有可能实现接近最优的 all-to-all 完成时间。请务必测量节点间 NIC 和节点内 NVLink 路径上的链路利用率。

使用单个全局 all-to-all 屏障的朴素专家路由实现，会让 GPU 等待同步开销。如果并非所有链路都被最优利用，这会浪费带宽。

蝶形调度（也称为移位 all-to-all 调度）等技术可以把通信分成多个阶段性轮次。这样，每条 NVLink/NIC 都在忙于部分交换——而不是一次大同步。这种交错方法提高链路利用率并减少空闲时间。

All-to-all 交换可以使用内置的 ncclAllToAll 集合通信，或分组的发送和接收调用。当交换被分块并流水线化，或跨节点使用蝶形等分层调度时，吞吐量通常会提高。请验证并选择与你的拓扑匹配的算法。

> **注** 让第一阶段 all-to-all 保持在机架内（NVLink），只把剩余 token 溢出到机架间。分析链路利用率，确认 NIC 饱和，同时 NVLink 重排在后台运行。此外，当节点间链路是瓶颈时，建议用专家计算对 MoE all-to-all 通信做双缓冲。你可以使用分块/流水线交换以及蝶形/移位调度，避免全局屏障减速，并优于单一全局集合通信。

另一种减少通信流量的方案称为专家并置（expert collocation）。思路是把某些专家放在同一块 GPU 或同一个节点上，以避免不必要的通信。考虑专家 5 和专家 7，它们经常被 token 路由器为同一个 token 激活。把专家 5 和 7 放在同一块 GPU 上，可以消除一次额外的 all-to-all 跳。性能分析工具和门控频率分析可以帮助你为工作负载识别此类配对。

还有一种方案是压缩专家之间的通信，包括专家交换激活。例如，可以在执行 all-to-all 通信之前，使用 NVIDIA Transformer Engine 在 Tensor Core 上转换为 FP8 或 NVFP4。这会降低 NIC 负载，从而摊薄压缩计算。它用极少的数值精度换取更快的 GPU 间激活传输。相对于网络和内存传输成本，转换以及打包/解包的开销通常很小。

简言之，优化 MoE 通信包括分析硬件和网络拓扑，以优化专家在 GPU 上的放置。在部署中，配置集群互连以实现高效的 all-to-all 通信很重要。例如，在 NVL72 机架中使用 NVLink Switch 网状网络，以便在单个域内最多 72 块 GPU 之间获得全带宽通信。朴素 all-to-all 选择可能主导层时间，并实现极低的 SM 效率。优先处理专家流量并在可能时重叠，同时确保在不同互连和通信算法选择下进行分析和验证。

> **注** 对于 MoE，配置集群以优化 all-to-all 通信。这意味着根据你的拓扑选择合适的 NCCL all-to-all 算法或分组发送/接收实现，然后确认节点间路径启用了 GPUDirect RDMA。还要确保 InfiniBand 链路正确绑定，使多个物理链路（端口）配置为单个逻辑通道。它们的带宽应合并——故障转移应无缝。换言之，确保网络拓扑针对 MoE 调优。这包括栈中的硬件层和软件层。

15.5.2 负载均衡、容量因子与专家复制

建议每个专家和每块 GPU 获得相等的工作份额，以避免“热点”。否则，如果 MoE 的门控网络把不成比例的 token 导向某个特定专家，这些 GPU 就会过载。这会增加延迟并降低整体系统吞吐量。

考虑单个专家 GPU 成为热点，利用率达到 99%，而其他专家 GPU 平均约 60% 利用率。如果不正确处理，这可能成为整个训练或推理集群的瓶颈。

模型训练期间，可以通过添加负载均衡损失项来解决热点：如果门控过度使用某些专家而使用不足其他专家，就对该门控施加惩罚。结果是训练后的 MoE 模型倾向于在专家之间相当均匀地分配 token。

然而在推理时，特定输入提示或主题仍可能通过集中于少数“热门”专家而造成不均衡。避免推理热点的一种策略是使用容量因子，触发溢出机制。

通过指定容量因子，可以配置模型使每个专家在给定时间最多只能处理一定数量的 token（例如 32 个 token）。如果某个专家收到的 token 超过此容量，多余的 token 可以转发给路由分数次高的回退专家，或者这些 token 会被串行化并在第二遍处理。图 15-12 比较了容量因子 1.0 与 1.5。

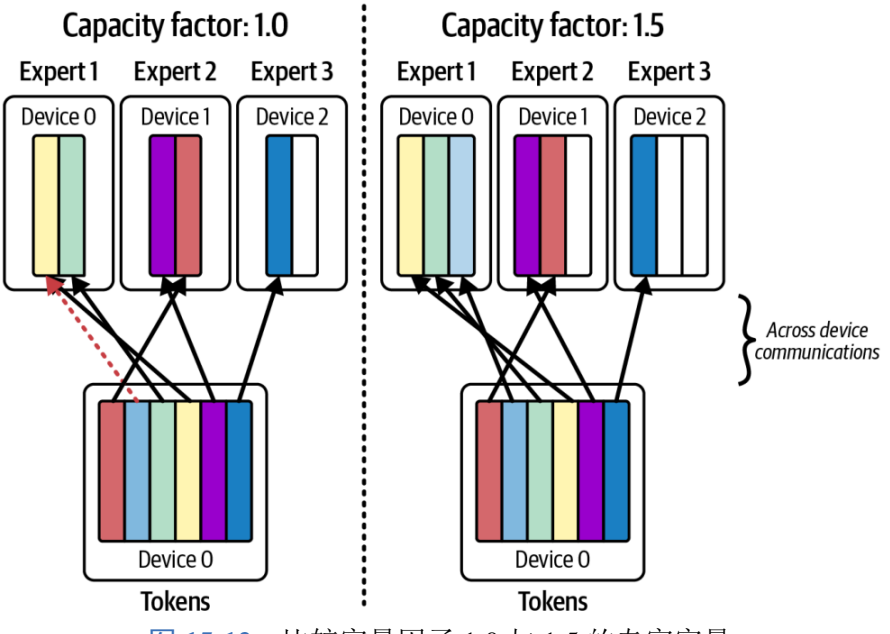

图 15-12　比较容量因子 1.0 与 1.5 的专家容量。

实践中，常见做法是容量因子 1.2（20% 溢出余量）搭配 top-2 门控。这意味着每个专家最多承担其平均负载的 120%。超过后，它会把多余 token 发送给下一个专家。这会有效平滑系统中各专家的负载。

另一种避免热点的策略是专家复制。如果某个专家持续成为热点，系统可以把该专家克隆到另一块 GPU。这样，门控函数可以把一部分 token 发送到运行在另一块 GPU 上的专家克隆。

副本是由推理引擎实现的纯应用层优化。模型本身并不知道副本的存在。由于副本被注册为具有自己索引的独立专家——且通常位于不同 GPU——引擎可以根据相对路由分数在原始专家及其克隆之间路由 token。

> **注** 复制专家会增加每个被复制专家的内存和成本。但由于通常只有少数专家会过载，复制少量热门专家是一种有针对性的修复——不会因为复制完整模型及其所有专家而使成本翻倍。

此外，如果模型更新，复制需要仔细处理以保持副本同步。所有副本专家必须保持完全相同（例如相同权重），因为门控路由器只知道单个专家，实际上并不知道副本的存在。

> **注** 通常，副本从与原始模型相同的检查点加载——而不是独立更新。这可以防止原始专家与其副本之间发生分歧。

15.5.3 自适应专家路由与实时监控

与训练后固定的传统 MoE 专家门控不同，自适应路由可以在推理期间实时调整门控决策，以响应当前条件和专家负载。例如，如果系统检测到某个专家 GPU 落后，它可以指示门控函数把一些 token 转移到另一个专家。另一个专家的路由分数可能略低，但它因为有空闲容量而接收该请求。

你应该对每个专家的利用率和响应延迟指标实施持续监控。现代 MoE 系统与遥测框架集成，使每个专家向 Prometheus/Grafana 发送利用率指标。这样，系统可以动态调整容量因子或门控算法。

大多数 LLM 专家门控函数只考虑训练时确定的路由分数。然而，真正自适应的系统需要由推理引擎处理，并在推理时动态执行。

要实现自适应路由，推理引擎需要把模型的前向传播包装在自定义逻辑中。例如，它可以拦截门控 softmax，并根据当前负载指标把一些 token 重新分配到不同专家。

推理引擎依赖每 GPU 利用率和每专家 token 计数等实时指标来持续测量专家负载。如果系统看到某个专家 GPU 利用率 99% 而其他专家 GPU 为 60%，系统可以暂时降低其负载，把一些 token 路由到该专家的副本——或路由到专家偏好分数略低的不同专家。

图 15-13 展示了一种使用带偏置门控分数方法的自适应 MoE 路由策略。虽然这种方法最初用于训练场景，但更简单的方法适用于推理。在这种情况下，它会使用修改后的专家偏置算法，在主专家负载很重时把 token 转移到替代专家。

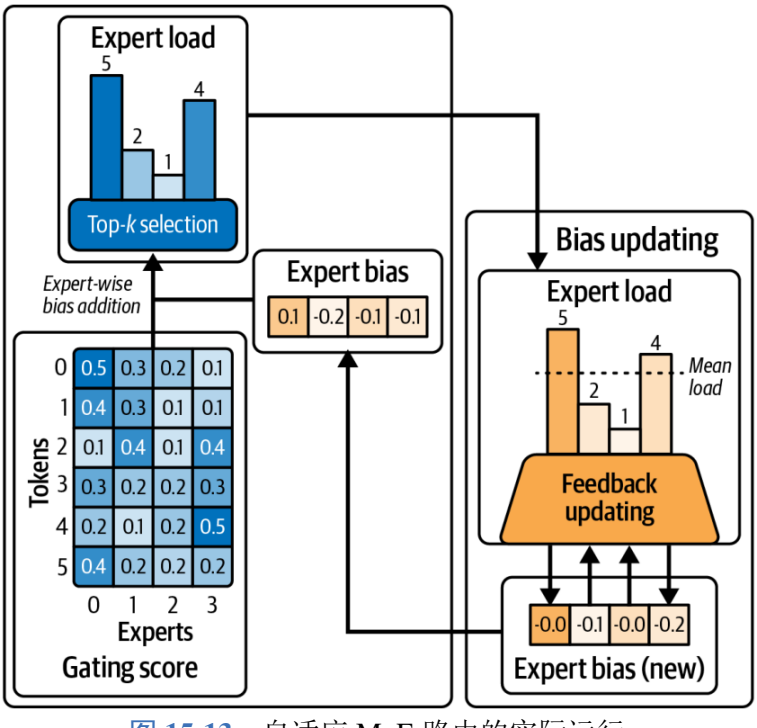

图 15-13　自适应 MoE 路由的实际运行。

这种方法可以减少不必要的通信，并更均匀地平衡负载。然而，它确实会带来一些额外成本，包括额外监控、决策、复杂性、配置管理和日志记录。收益可能超过成本，也可能不会。每个场景和工作负载都是独特的，但这绝对值得探索。

使用 Nsight Systems 等工具进行分析时，你要监控专家 GPU all-to-all 通信的时间线轨迹。例如，如果某块 GPU 在时间线中的片段长得多，它很可能正在处理更多 token。

你的推理系统可以利用这些洞察调整专家门控概率，并动态地把专家重新分配到不同 GPU。它还可以生成额外的专家副本实例等。这有助于通过调整专家门控算法、修改专家放置或创建/删除专家副本来重新平衡负载。

> **注** 在推理时动态生成新的专家副本并非易事。这种方法需要为这些专家预配置容量或快速加载模型。这是一种高级优化技术。

以不同方式把某些专家分组到不同 GPU 上，可以使 token 路由更均匀，并因更好的并行性而提高整体吞吐量。这是因为 GPU 可以更同步地完成每层工作。如果检测到持续不均衡，现代 MoE 调度器可以通过调整容量因子即时调度额外的专家副本。这可以缓解门控不均造成的热点。

> **注** 记住记录系统所做的任何动态更改。还建议设置警报，当某个专家的利用率超过阈值（例如 80% 利用率）时触发。

动态路由策略有两个核心目标：降低路由开销，以及在专家 GPU 之间均匀分配工作。实现低开销取决于利用高带宽互连、让数据传输与计算重叠，以及通过并置和智能调度最小化冗余数据移动。

负载均衡通过简单的 top-1 或 top-2 门控，或更高级的容量感知门控来实现。也可以通过动态复制和专家重新分配来实现。通常组合使用这些技术，让 GPU 保持忙碌，实现最大计算和最小通信延迟。

只要路由开销最小化且负载均衡，自适应 MoE 推理系统就能在增加专家数量时实现接近线性的吞吐量扩展。例如，GPU 数量翻倍几乎可以使推理吞吐量翻倍。

简言之，这些自适应专家路由和负载均衡优化可以与前面讨论的并行和解码技术集成。这样，你就可以调优 MoE 推理系统，在超大规模下实现高性能。持续分析和自适应算法可以让 GPU 保持忙于计算，避免因通信延迟而空闲。

> **注** 高级推理引擎可以动态绕过某些专家计算、关闭利用不足的专家，或使用缓存为某些 token 跳过专家。这会进一步降低延迟。这些技术建立在本章所述概念之上。

15.6 本章要点

向数十亿最终用户服务海量 LLM，需要在推理流水线的每个阶段进行优化。以下是本章的一些关键要点：

分离以同时优化延迟和吞吐量。把提示预填充和解码阶段拆分到独立 GPU 池可以消除干扰。这让你可以同时实现低首 token 时间（TTFT）和高每秒 token 数，而不是在两者之间取舍。这是大规模 LLM 服务的基础技术。

为巨型模型使用混合并行。没有任何单一并行策略足以应对数万亿参数模型。按需组合张量、流水线、专家和数据并行。例如，跨 GPU 分片层（TP/PP）以把模型放入内存，对 MoE 层使用专家并行以扩展模型容量，并添加数据并行副本以满足吞吐需求。最优组合取决于硬件。始终针对你的工作负载进行分析和调优。

缓解顺序解码瓶颈。高级解码方法可以大幅加速生成。带快速草稿模型的双模型推测解码，在接受率针对任务调优时通常带来约 2–3 倍加速。EAGLE-2 报告在某些任务上最高 3.5 倍加速（比 EAGLE-1 多 20%–40%），同时保持目标分布。Medusa 实现报告，在针对目标工作负载训练和验证后，相比非推测解码最高 3.6 倍加速。这些技术在标准验证下保持大模型输出分布，同时提高 token 级并行度和算术强度。总体结果是响应更快，而无需重新训练主模型。这表明推测解码是一种保持质量的技术。

用约束保持输出质量和格式。在生产中，你常常需要 LLM 遵循严格格式或避免某些 token。约束解码技术让你可以在生成期间强制执行规则——例如 JSON schema 和禁用词。它们会给每个 token 增加一些开销，但借助编译文法和优化的掩码路径，约束解码在规模化时通常可以在正常解码的低个位数百分比范围内运行，不过复杂文法或小批次可能带来更高开销。务必测试约束的性能影响。如果可能，避免极其严格的规则。它们会因过度回溯而减慢生成。

平衡 MoE 工作负载以实现有效扩展。混合专家模型提供模型规模与 GPU/专家数量之间近乎线性的扩展——但前提是高效处理路由。使用高带宽互连和分层 all-to-all 通信来减少网络瓶颈。通过应用容量限制和 top-2 门控，确保每个专家获得相似的工作量，以避免落后者专家。复制任何持续热门的专家以拆分其负载。调优良好的 MoE 推理系统在添加 GPU 时可以接近线性吞吐量扩展。

利用软硬件协同设计。现代 GPU 硬件旨在支持这些并行和分布式推理方法。使用能充分利用硬件和拓扑的软件，包括 vLLM、SGLang 和 NVIDIA Dynamo 等推理引擎。它们可以以最小开销编排多 GPU 和多节点推理。让节点内通信保

持在 NVSwitch 上、仅在必要时使用 InfiniBand，并使通信与计算重叠，从而让你的策略与硬件优势对齐。这种对齐是实现最佳延迟和成本效率的关键。

理解复杂性与投资回报（ROI）。每种优化都会增加系统复杂性。推测解码和自适应 MoE 路由等技术可以显著提高性能，但它们需要额外模型或复杂逻辑。始终权衡成本。对于交互式应用，2–3 倍延迟改进通常值得。对于更简单的用例，直接的方法可能就足够了。从最大的瓶颈开始，例如消除预填充/解码干扰。然后按需逐步增加复杂性。通过监控和分析确定哪些优化带来最佳投资回报。

15.7 结论

通过结合分离式预填充/解码流水线、多 token 推测解码、动态专家路由和自适应编排，可以以最小的资源争用和超低延迟实时服务 LLM。

vLLM、SGLang 和 NVIDIA Dynamo 等现代推理服务平台采纳了许多这些优化。它们高效分配集群资源、跨节点协调 KV 缓存、实现推测和约束解码、调度预填充/解码任务等等。

关键在于端到端、可适应的架构，把算法创新与高性能硬件能力匹配起来。在接下来的几章中，我们将深入探讨模型推理性能优化，包括动态、自适应和多节点服务策略。

我们将涵盖从应用级前缀缓存、延迟感知请求路由和基于强化学习（RL）的集群调优，到系统级自适应内存分配、精度切换和拥塞感知资源调度的一切内容。

第 16 章 大规模推理的性能分析、调试与调优

运行一个大型 LLM 推理集群，需要监控与调试工具来确保一切按预期运行；当性能偏离目标时，这些工具还能帮助你快速定位瓶颈。

本章将演示如何使用 NVIDIA Nsight Systems 进行性能分析、使用 Prometheus/Grafana 进行集群级遥测，来监控和调试这些复杂系统。我们还将展示如何收集和解读 GPU 利用率、内存压力、尾延迟分位数、缓存命中率、逐 token 耗时等关键指标，并以此指导推理引擎的性能优化。

接下来我们讨论运行层面的性能调优，包括经过生产验证的方法，用于优化 GPU 利用率、降低推理延迟以及提升大型集群的吞吐量。这涉及重叠计算与通信、请求调度与批处理，以及高效利用 NVLink、NVSwitch 和 InfiniBand 等高速互连的技术。

我们还将比较面向推理的实时量化技术，包括借助 GPTQ（广义训练后量化）和 AWQ（激活感知权重量化）等实现，把模型压缩到 8-bit 和 4-bit 精度的方法。在此过程中，我们会讨论仅对权重做量化与同时对权重和激活做量化之间的权衡，并给出在服务流水线中应用量化的实用指南，以在保持模型精度的同时降低内存占用、提升吞吐量。

最后，我们考虑与底层性能调优互补的应用层优化，包括提示词压缩、前缀缓存、去重、查询路由（例如回退模型）以及部分输出流式传输等策略。

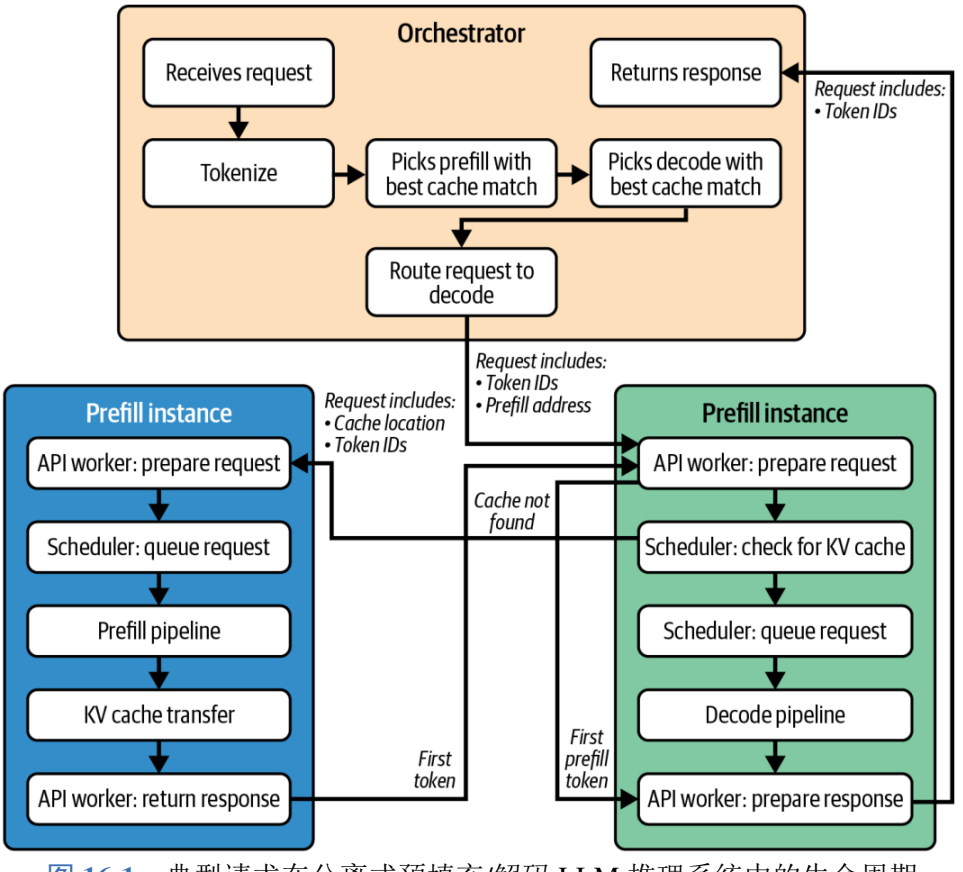

图 16-1　典型请求在分离式预填充/解码 LLM 推理系统中的生命周期

16.1 推理性能的分析、调试与调优

现代 LLM 推理引擎包含大量相互联动的组成部分，在预填充与解码分离部署的情况下尤其如此。一个典型请求的生命周期会经过许多组件，如图 16-1 所示。

面对如此复杂的系统，推理性能调优的工作流程具有很强的迭代性：需要细致的调优与持续的验证。

首先，观察各项指标并识别当前瓶颈，例如 GPU 未被充分利用，或延迟高于预期。接着，提出一个改进假设，例如“增大批大小”或“增强操作 X 的通信与计算重叠”。然后，实施修复并检验该假设。

理想情况下，接下来应在预发布（staging）环境中使用具有代表性的工作负载来测试该修复，并借助性能分析工具验证其行为符合预期。例如，可以确认某个操作确实实现了恰当的内存与计算重叠。

最后，把修复部署到生产环境，并通过监控 Grafana 和日志，验证该修复在真实工作负载下确实改善了吞吐量和延迟。当出现新的瓶颈时，重复这一工作流程。

这种“观察—假设—调优”的循环应当持续进行。现代部署往往会将这些步骤自动化。例如，可以利用定时负载测试，以及对关键指标的后续异常检测，来触发一个调优工作流。

> **注** 建议在生产环境中部署优化（包括更新后的推理运行时和模型变体）时采用金丝雀发布：先将优化部署到运行在少量服务器上的小部分流量，在向所有最终用户全面部署之前验证其效果。这种渐进式方法能够及早发现意外的副作用，从而缩小“爆炸半径”，避免波及所有用户。

考虑这样一个场景：由于过度的分词或推理数据预处理，主机侧 CPU 利用率飙升至 100%。这会限制推理引擎能够处理的并发流数量。一种可行的修复方法，是借助 GPU 加速的分词器库，或用 CUDA 或 OpenAI 的 Triton 语言编写的自定义 GPU kernel，把预处理移到 GPU 上执行。

部署新库或新 kernel 后，应当对比部署前后的 CPU 利用率。如果 CPU 利用率下降且整体吞吐量上升，说明系统的瓶颈已不再是基于 CPU 的输入预处理。

还应关注各类缓存的命中率，包括前缀缓存、提示词嵌入缓存和 KV 缓存。应当设置“缓存命中”与“缓存未命中”指标。缓存命中率高，说明系统在有效地复用数据；反之，如果未命中率很高，则可能需要调整缓存大小、淘汰策略或缓存策略，以最大化缓存命中。

vLLM 的 LMCache 组件允许调整 GPU 与 CPU 缓存的比例。如果因 GPU 显存限制导致未命中率偏高，可以启用其分页缓存卸载（paged cache offload）功能，让 CPU 协助分担。同时务必确保缓存淘汰策略（LRU（最近最少使用）、LFU（最不经常使用）等）与访问模式相匹配。

另一种场景是利用 KV 缓存，在批量请求之间复用相同输入序列前缀的数据，从而避免为该前缀重复计算 KV。在这种

情况下，你希望度量请求共享前缀的频率。这会触发一次前缀合并事件，并使 vLLM 中的前缀缓存命中指标递增，包括 `vllm:gpu_prefix_cache_queries` 和 `vllm:gpu_prefix_cache_hits`。据此你可以计算命中率，例如命中数除以查询数。

度量前缀合并率有助于将其与实际的缓存命中率相互关联，从而评估缓存层的真实收益。这样，你就能调整批处理与调度策略以最大化共享前缀，并预测不同工作负载下端到端吞吐量和延迟的改进。

可以向推理引擎输入合成生成的数据，以测试带有大量重复前缀的提示词。理想情况下，你将看到前缀合并带来的预填充计算量的下降。

vLLM 和 SGLang 等现代 LLM 推理引擎原生导出前缀合并指标。但如果你的推理引擎没有把前缀合并作为一等指标导出，就应自行添加一个“前缀去重 token 数”的自定义计数器，以监控其有效性。

> **注** 如果发现前缀合并的效果不及预期，应检查前缀匹配逻辑是否失效。调试时可以先检查是否存在分词器差异；这是多数前缀匹配问题的可能原因。

除性能之外，监控还有助于容量规划。通过跟踪利用率和延迟随负载增加的变化，可以预测系统在何时触及某个特定极限，例如 p95（第 95 百分位）延迟开始呈指数式上升。此时，动态批大小可能已增大到收益递减的程度。

> **注** 如果使用分层缓存策略，包括基于 NVMe 的 KV 缓存扩展，务必监控该设备的 I/O 延迟。高 I/O 延迟会显著降低缓存性能。

当每 GPU 并发数达到上限、继续增大批大小也无法再提升吞吐量时，你可能需要横向扩展：添加更多 GPU、部署额外的模型副本，或增加专家数量，以便把工作分散到更多计算单元上。

在横向扩展之前，还应考虑模型压缩——或切换到更低精度（FP8/FP4）——以提升每 GPU 的有效吞吐量。然而，一旦硬件达到饱和（例如 SM 占用率 100%、内存带宽接近峰值

利用率），增加更多 GPU 或使用张量并行/流水线并行，很可能就是提升吞吐量的唯一途径。

同时要始终权衡新硬件的成本与效率收益。有时，升级到显存更大、浮点算力（FLOPS）更高的新 GPU，比横向扩展一批旧 GPU 更加划算。

增加专家数量可以提高吞吐量上限——但前提是你同时改进专家路由与调度，以应对额外的 all-to-all 通信；否则，简单粗暴的扩展只会把瓶颈转移到网络上。接下来，让我们讨论监控，以及如何验证我们的优化努力确实带来了回报。

16.1.1 监控系统指标与计数器

传统微服务调用的执行时间相对均匀、可预测，而 LLM 请求则不然：其耗时不均匀，延迟可能波动极大。这一差异如图 16-2 所示。

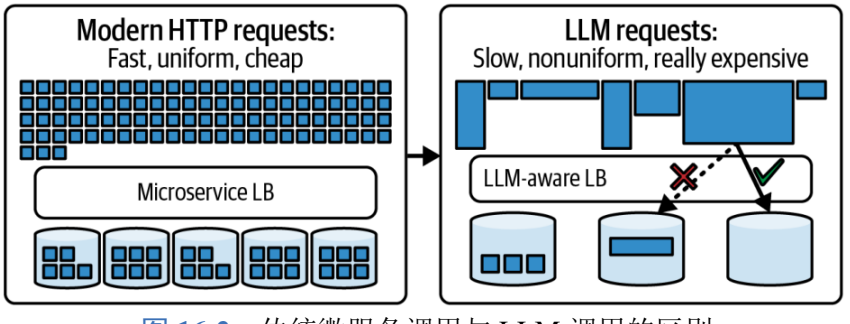

图 16-2　传统微服务调用与 LLM 调用的区别

在生产环境中进行持续监控时，通常使用 Prometheus 从每个 GPU 计算节点采集指标，并用 Grafana 仪表盘加以可视化。值得跟踪的关键 GPU 指标包括 GPU 利用率（SM 忙碌时间占比）、GPU 显存使用量、拷贝引擎利用率、PCIe 与 NVLink 吞吐量，以及 GPU 温度和功耗（例如是否降频节流）。注意：L1/L2 活动级别、占用率、指令吞吐量等底层计数器，应使用 Nsight Compute 或 CUPTI 来采集，而非 DCGM 和 Prometheus。

> 注　`cudaMemPool` 相关指标与异步分配器统计信息在监视内存碎片时非常有用。应将它们纳入你的监控系统，这将大大方便在生产环境中排查系统性能问题。

同样重要的是监控互连利用率，包括 NVLink、NVSwitch 带宽和网卡吞吐量。这样，你就能在多 GPU 和多节点集群配置中及时发现通信瓶颈。

NVIDIA 的数据中心 GPU 管理器（DCGM）暴露了大量 GPU 指标，可由 Prometheus 抓取和收集。例如，DCGM 提供 `DCGM_FI_DEV_GPU_UTIL`（SM 利用率百分比）、`DCGM_FI_DEV_MEM_COPY_UTIL`（内存拷贝引擎利用率）以及 `DCGM_FI_DEV_FB_USED`（已用显存）等指标。

DCGM 暴露 NVLink 错误计数器，在某些平台和驱动版本上还能提供吞吐量计数器。要了解链路利用率的持续水平，还应使用 `nvidia-smi nvlink` 和 Nsight 工具。你应将这些指标集成到仪表盘中，并配置告警，以便在跨 GPU、跨节点的通信流量使网络饱和时能够及时发现。DCGM 还跟踪 Xid 计数器以及严重的 GPU 错误。

> **注** 虽然 DCGM 暴露了 NVLink 计数器，但截至撰写本文时，`dcgm-exporter` 默认并不在所有平台上暴露每条链路的带宽。因此，如果你需要链路级吞吐量，可能需要直接查询 DCGM，或自行扩展该 exporter。

此外还建议收集高层应用指标，例如每秒查询/请求数、平均延迟与 p95/p99 延迟、活跃上下文数量，以及以 tokens/秒计的吞吐量。KV 缓存的使用率与容量指标（整体以及每节点）同样极为重要，必须纳入监控。

你可以为每个节点部署 Prometheus node exporter 来收集上述所有指标，将数据汇集到一处，甚至针对关键阈值设置告警。Grafana 随后可将这些指标绘制成实时仪表盘，与团队共享。例如，图 16-3 展示了在 Kubernetes 集群中如何从每个 GPU 采集指标、导出到 Prometheus，再用 Grafana 进行可视化。

这样，例如当你部署一项旨在提高批处理规模的新优化后，Grafana 会立即显示每块 GPU 的利用率是否随之上升。你还可以持续监控，确保 p95/p99 延迟保持在目标范围之内。

计数器同样极具测量价值——对于动态自适应系统而言尤其如此。例如，如果你的推理引擎会根据当前状况动态调整批大小，你可能希望递增一个“批大小变化”计数器。

另一种选择是把变更记录到日志文件中，但这需要缓慢的基于文本的搜索/聚合，例如事后用 Apache Spark 离线分析日志文件。随后你还需要手动将日志分析的结果与 Prometheus 指标关联起来。

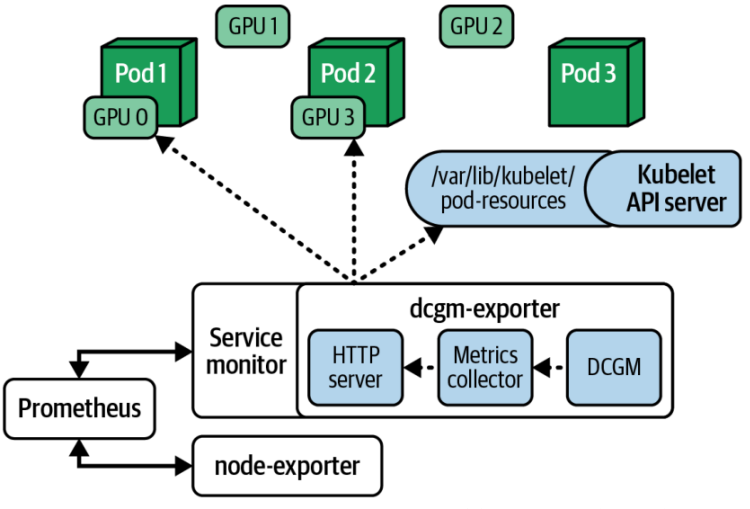

图 16-3　DCGM 从 Kubernetes GPU 节点采集指标并发送给 Prometheus

只要为值得关注的应用层事件（包括错误）递增一个简单计数器，数据就会推送到 Prometheus，并能与其他全部指标一起实时呈现在 Grafana 仪表盘中。此外，对关键事件还应考虑使用结构化日志和分布式追踪。

现代应用性能管理（APM）工具以及 OpenTelemetry 可以摄取这些日志/追踪数据，并将其与指标关联起来，从而在整个系统范围内提供一个统一的事件时间线视图。洞察这条时间线有助于加快排查性能问题的速度。

持续监控这些指标，能让你看清下一步应该调整什么。例如，若 GPU 利用率低于预期，可以检查 GPU 显存是否已经用满。如果显存尚未被充分利用，可以尝试增大批大小或最大并发请求数。但务必盯紧延迟服务级目标（SLO），不要超出这些目标。

> **注**　现代推理服务器为动态批处理提供了“最大延迟”设置。应据此调参以满足你的 SLO。调大该值会提高批大小（进而提高吞吐量），但调得过大将损害 p99 延迟。应根据延迟目标持续调整它。

相反，如果 GPU 显存已接近上限，推理引擎可能会开始把不活跃的 KV 缓存数据换出到宿主 CPU 内存或 NVMe 存储。这会降低 GPU 利用率，因为 GPU 需要等待来自较慢的 CPU 内存或磁盘的额外数据传输。

> **注** 如果你看到 GPU 显存拷贝引擎利用率突然飙升——或者 NVLink 利用率异常，同时 SM 利用率却很低——那么推理引擎很可能正在 GPU 显存与主存之间来回换入换出 KV 缓存数据。过高的数据传输延迟将成为系统瓶颈。

如果确实在发生换页，你可以调整推理引擎的分页参数以减少抖动，采用 FP8 或 FP4 量化，增加分配给缓存的 GPU 显存，并可能需要更换换页策略。这应当能让拷贝利用率下降、计算利用率上升——这正是你期望看到的结果。

Grafana 也用于延迟跟踪。你可以绘制端到端请求延迟的分布——通常还会同时测量预填充延迟和每 token 延迟。如果 p99 延迟在某些时段出现尖峰，应将其与 GPU 指标和其他日志进行关联分析。

例如，p99 延迟尖峰可能与 GPU 利用率下降的时段相关。也许这一延迟尖峰对应某次流量激增，后者触发了更大的动态批大小，从而在该时段造成更高延迟。为验证这一点，你可以在 Grafana 仪表盘中把 RPS（每秒请求数）叠加到延迟图上，观察两条曲线是否相关。

如果按照前面的假设，尖峰是由批大小动态增大所致、属于意料之中，也要确认它没有超出你的服务级协议（SLA）。一旦超限，你可以尝试减小请求批的最大排队延迟，或降低最大批大小，为延迟设定上限。

日志在诊断问题时同样不可或缺。你应在代码中加入埋点，记录关键事件，例如批何时形成、一次通信何时开始/结束等。最好使用 DEBUG 级别，这样就可以按需开关，同时不影响请求-响应延迟。

启用调试日志后，你会看到文本格式的逐步时间线。在同一次调试中，你很可能需要同时使用日志时间线和 Prometheus/Grafana 指标。例如，你可以据此统计 all-to-all 通信耗时超过 5 ms 的频率。

借助日志时间线与指标的组合，你可以发现离群点——例如某个网络问题可能拖慢了 all-to-all 通信轮换中的某一次迭代。如果这种情况持续发生，你可以调高专家容量因子，

使多余的 token 自动溢出到次级专家副本——理想情况下，该副本所在 GPU 的网络路径更为稳定。这样可以均衡负载并将延迟降至最低。

> **注** 实践中，通常将容量因子设为 1.2–1.5，这样当主专家过载时，可有 20%–50% 的额外 token 被重新分配。这能显著平滑 MoE 推理的尾延迟。让请求溢出到第二个专家副本，好过让请求排在互连性能已退化的 GPU 上略显微滞的专家之后。这样，即便你的网络因任何原因持续出现问题，系统对离群点的敏感度也会降低。

16.1.2 使用 Nsight Systems 与 Nsight Compute 进行性能分析

在开发和调优推理代码时，可以使用 Nsight Systems 捕获横跨 CPU 与 GPU 的工作负载追踪。Nsight Systems 提供时间线视图，以微秒级分辨率显示 CPU 线程、GPU kernel、CUDA 事件、NCCL 通信等内容。

通过在代码中加入 NVTX 标注，我们可以在时间线上清晰地标记“预填充阶段”“解码步骤”或“all-to-all 通信”等区域。下面的代码使用 NVTX v3 C API 以及显式的 push/pop 范围，在示例预填充与解码步骤周围添加了 NVTX 范围标记：

```
// Example C++ snippet with NVTX annotations using the C API.
#include <nvtx3/nvToolsExt.h>   // or <nvToolsExt.h>
#include "my_model.hpp"         // Your model' s C++ interface
#include <vector>
// Small helpers to keep callsites tidy.
#define NVTX_PUSH(name, argb)                                          \
  do {                                                                 \
    nvtxEventAttributes_t a{};                                         \
    a.version = NVTX_VERSION;                                          \
    a.size = NVTX_EVENT_ATTRIB_STRUCT_SIZE;                            \
    a.colorType = NVTX_COLOR_ARGB;                                     \
    a.color = (unsigned int)(argb);                                    \
    a.messageType = NVTX_MESSAGE_TYPE_ASCII;                           \
    a.message.ascii = (name);                                          \
    nvtxRangePushEx(&a);                                               \
  } while (0)
#define NVTX_POP() do { nvtxRangePop(); } while (0)
struct Token { int id; };
```

```
void run_inference(
    const std::vector<Token>& prompt_tokens,


    Model& model,
    int num_generate_steps) {
  // Prefill
  NVTX_PUSH("Prefill", 0xFF4F86F7);
  model.encode(prompt_tokens);
  NVTX_POP();
  // Decode one token at a time
  for (int t = 0; t < num_generate_steps; ++t) {
    NVTX_PUSH("Decode", 0xFFFF8C00);
    Token next_token = model.decode_next();
    // ... (sampling / streaming to client)
    NVTX_POP();
  }
}
```

这里我们用 `nvtxRangePushEx/nvtxRangePop` 显式标记区域。在 `model.encode(...)` 之前立即推入“Prefill”范围，随后马上弹出。在解码循环内，每次迭代开始时推入“Decode”，并在 `model.decode_next()` 之后弹出。精简的辅助宏 `NVTX_PUSH/NVTX_POP` 还会附上颜色（十六进制值）与文本，这既有助于减少时间线可视化中的错配，又保持了调用点简洁。显式的推入/弹出配对在代码中一目了然，便于审查。

这些着色块标注将以“预填充”（Prefill）和“解码”（Decode）为标签，出现在 Nsight Systems 的 GPU 活动时间线中。这样一来，各阶段耗时多少、各阶段与通信操作如何重叠，都一目了然，有助于定位 GPU 空闲间隙和意外同步等问题。

> **注** 请注意，我们直接使用 NVTX C API（`nvToolsExt`），而不是 PyTorch 的 `record_function()`。这样既可以在纯 C++ 运行时中标注热点路径，又能保证无论工作由 Python 还是其他语言发起，标记都保持一致。

通过把作用范围收紧到 `model.encode(prompt_tokens)` 周围最小必要的区域，性能分析标记恰好覆盖预填充工作，不包含其他代码。这能提升 trace 的清晰度和性能诊断效果。

在多条 CUDA 流上排队工作时（例如用专门的“传输”流做 H2D/D2H 拷贝，用“计算”流跑 kernel），应使用按流划分的 range。为此，可以为每条流的主机端代码分别套上不同的 NVTX range。

```
例如，可以用 nvtxNameCudaStreamA(transfer_stream, "transfer_stream") 和
↪ nvtxNameCudaStreamA(compute_stream, "compute_stream") 为流命名，
↪ 随后用 nvtxRangePushA("transfer_stream")
```

和 `nvtxRangePop()` 包裹内存拷贝/传输，并用 `nvtxRangePushA("compute_stream")` 和 `nvtxRangePop()` 包裹 kernel 启动。

用 NVTX 为流命名之后，重叠（或缺乏重叠）在 Nsight Systems 时间线中一目了然。下面这段代码演示了上述做法如何组合在一起：

```
// One-time after creating the streams
nvtxNameCudaStreamA(transfer_stream, "transfer_stream");
nvtxNameCudaStreamA(compute_stream,  "compute_stream");
// Around H2D/D2H copies (transfer stream)
nvtxRangePushA("transfer_stream");
cudaMemcpyAsync(h_logits, d_logits, bytes, cudaMemcpyDeviceToHost,
                transfer_stream);
nvtxRangePop();
// Around kernel enqueues (compute stream)
nvtxRangePushA("compute_stream");
my_kernel<<<grid, block, 0, compute_stream>>>(...);
nvtxRangePop();
```

这里，我们为流命名，并在各个入队点套上按流划分的 range，从而让 Nsight 时间线保持可读。需要特别说明的是，NVTX range 标注的是主机线程时间线；GPU 一侧则按流显示 kernel/内存拷贝。为流命名有助于在分析时把主机端的 range 对应到正确的 GPU 轨迹上。

Nsight Compute 可以对单个 kernel 进行性能分析，从而精确定位低效之处。我们可以利用 Nsight Compute 基于分节（section）的分析功能，聚焦 kernel 的特定部分，例如内存事务。

另一个非常有用却不广为人知的工具是 Nsight Compute 的 CUDA 程序计数器（PC）采样功能。它通过采样程序计数器来识别热点，无需完整、重量级的插桩，如图 16-4 所示。

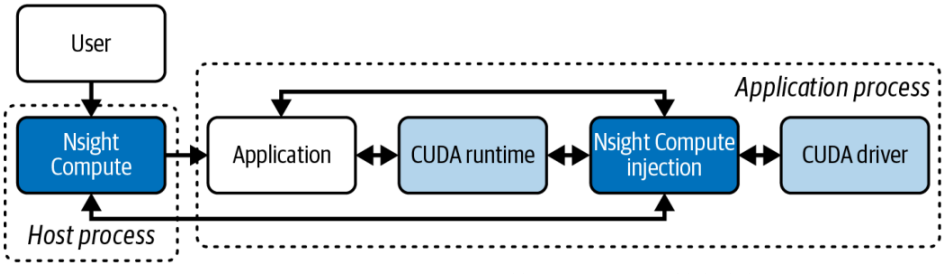

图 16-4　Nsight Compute 的 CUDA 程序计数器（PC）采样功能以低开销方式帮助识别热点（来源：https://oreil.ly/DyKWR）

具体来说，可以用它对在线推理服务进行实时分析，以低开销的方式精确定位哪些 kernel 指令耗时最多。至此，我们已经介绍了用 Nsight Systems 和 Nsight Compute 做性能分析的方法，接下来讨论推理故障排查的常见方案。

> 注　在对在线服务进行生产环境排查时，优先使用程序计数器采样，以最小开销定位热点；只有当采样结果指向某个具体 kernel 或阶段时，再切换到完整的跟踪分析。

16.1.3 推理故障排查方案

在生产环境中，持续运行重量级分析器并不现实。因此，需要依靠轻量的基于指标的监控——例如 GPU SM 利用率、KV 缓存告警、尾延迟分位数、缓存命中率、OOM 告警——来发现异常并引导针对性修复。当某项指标越过阈值时，可以对其根因提出假设，例如批规模过小、KV 缓存不足、路由热点、分片不均衡、显存超分等。

然后实施修复，例如调整批大小、提高显存利用率上限（如可行）、调整路由器阈值、启用 CPU 卸载等。修复上线后，应核实其影响，确认指标回落到阈值以下。表 16-1 给出了常见生产问题的关键指标、症状、可能原因和建议措施。

表 16-1 常见生产故障的症状、成因与建议措施。

指标/症状	可能原因	建议措施
SM 利用率 < 50%	批规模过小，或缺少融合 kernel	增大批规模；启用融合 kernel（FlashAttention，或 PyTorch 中 cuDNN 融合的 `scaled_dot_product_attention`（SDPA）后端），或添加自定义融合 kernel（例如用 Triton 编写）；随后用 `nsys {-}-trace=cuda` 做性能分析
KV 缓存抢占告警	KV 缓存空间不足（vLLM）	提高显存利用率阈值；减少单批最大 token 数；考虑用 PagedAttention 做动态 KV 分配
尾延迟过高（p95 > 200 ms）	解码节点热点或队头阻塞	检查路由日志中的路由模式；调优预取阈值；启用投机解码路径
负载下缓存命中率 < 60%	分片放置不均衡或缺少前缀缓存	校验前缀缓存连接器配置（例如 vLLM 上使用 LMCache 的 NIXL，以及 NVIDIA Dynamo 的 NIXL 连接器）；必要时提高前缀缓存 TTL 或增加副本数
多租户 GPU 上意外 OOM	GPU 显存超分	降低每实例的显存利用率；启用 CPU/NVMe 卸载；将进程绑定到 CPU socket 以减少跨 socket 流量
偶发性性能离群值	时钟不匹配或热节流	确保所有时钟同步，并监控热节流和功耗节流

指标/症状 可能原因 建议措施 SM 利用率 < 50%

批规模过小或缺少融合 kernel

增大批规模，启用融合 kernel（FlashAttention，或 PyTorch 中 cuDNN 融合的 `scaled_dot_product_attention`（SDPA）后端），或添加自定义融合 kernel（例如用 Triton 编写）；随后用 `nsys {-}-trace=cuda` 做性能分析。KV 缓存抢占告警

提高显存利用率阈值，减少单批最大 token 数，考虑用 PagedAttention 做动态 KV 分配。尾延迟过高（p95 > 200 ms）

KV 缓存空间不足（vLLM）

检查路由日志中的路由模式；调优预取阈值；启用投机解码路径。负载下缓存命中率 < 60%

解码节点热点或队头阻塞

校验前缀缓存连接器配置（例如 vLLM 上使用 LMCache 的 NIXL，以及 NVIDIA Dynamo 的 NIXL 连接器）；必要时提高前缀缓存 TTL 或增加副本数。多租户 GPU 上意外 OOM

分片放置不均衡或缺少前缀缓存

降低每实例的显存利用率，启用 CPU/NVMe 卸载，将进程绑定到 CPU socket 以减少跨 socket 流量。偶发性性能离群值

GPU 显存超分

时钟不匹配或热节流

确保所有时钟同步，并监控热节流和功耗节流。

> **注** 本节所有指标表中的数值仅用于解释概念。不同 GPU 架构上的真实基准测试结果请参见 GitHub 仓库。

其中一些信息也可能埋藏在日志文件中。AWS 等云提供商支持对日志文件使用正则表达式（RegEx）过滤器，从日志行中提取数值并直接导出为指标。例如 AWS CloudWatch 就支持这一实用功能。下面的代码块给出了几条值得监控的示例日志行。

下面是 vLLM 日志的一个片段，表明由于 KV 缓存空间不足而发生了 KV 缓存抢占，进而触发 KV 重算，这会占用更多 GPU 计算资源并推高延迟：

```
WARNING 2025-05-03 14:22:07 scheduler.py:1057 Sequence group 0 is
 ↪ preempted by
PreemptionMode.RECOMPUTE because not enough KV cache space.
total_cumulative_preemption_cnt=1
```

下面是一份 NVIDIA Dynamo 路由日志示例。其中第一行显示本地解码 worker 上发生了 90% 的前缀缓存命中，预填充因此留在本地执行；下一行显示本地缓存未命中，路由器随即将预填充分派给远端的 GPU-node-03 工作节点：

```
[Router] 2025-05-03T14:23:11Z INFO KVRouter: prefix-cache hit (90%) for
model=DeepSeek-R1; routing to local vLLM worker
[Router] 2025-05-03T14:23:12Z INFO KVRouter: cache miss; dispatching
 ↪ remote
prefill to GPU-node-03
```

16.1.4 全栈推理优化

高性能 LLM 推理需要贯穿技术栈每一层的协同优化：从模型架构与 kernel 实现，到运行时引擎、系统编排和部署基础设施，无所不包。

模型层面的技术——如剪枝、蒸馏、稀疏化、MoE 路由、高效注意力（如 FlashAttention）以及量化感知训练——能够降低计算与内存需求。在 kernel 层面，融合算子、自定义注意力引擎（如 FlashInfer）、Tensor Core 利用、分块 tiling 以及异步内存传输，有助于把 GPU 吞吐量推向极致。

运行时层面的策略——如动态批处理、分页 KV 缓存、CUDA Graphs，以及计算与通信的重叠——能在负载波动的情况下让 GPU 保持满载。系统编排层则可以借助预填充/解码分离、智能路由、多租户隔离和自动扩缩容（配合热备实例）来平衡延迟并提升成本效率。

> **注** 许多生产系统使用基于 Kubernetes 的编排来分别部署预填充与解码服务：由 ingress 控制器按负载或用户优先级路由请求，并常备热备 GPU Pod，以便在流量激增时快速拉起。

最后，还值得探索地理分布的边缘服务、智能 API 网关批处理、面向模型变体的 CI/CD 以及实时性能分析等部署模式，从而在生产环境中获得最佳的可靠性与适应能力。表 16-2 归纳了技术栈各层的常见优化方法。

模型：剪枝与知识蒸馏，以最小的精度损失缩小模型规模；稀疏化（MoE）跳过部分计算；高效注意力（FlashAttention）降低内存占用与中间缓冲；FP16/BF16 或 INT4/FP8 量化感知训练，保证低精度下的稳健性。Kernel：融合算子 kernel（如 `Linear + GELU + LayerNorm`），减少启动开销与内存流量；自定义注意力 kernel（FlashInfer），支持块稀疏 KV 与 JIT 编译的 kernel；利用 Tensor Core 与专用指令（`cp.async`、TMA）执行矩阵运算。运行时：带延迟控制的动态批处理

表 16-2　技术栈各层的常见优化方法。

技术栈层次	关键技术
模型	剪枝与知识蒸馏，以最小的精度损失缩小模型规模；稀疏化（MoE）跳过部分计算；高效注意力（FlashAttention）降低内存占用与中间缓冲；FP16/BF16 或 INT4/FP8 量化感知训练，保证低精度下的稳健性
Kernel	融合算子 kernel（如 `Linear + GELU + LayerNorm`），减少启动开销与内存流量；自定义注意力 kernel（FlashInfer），支持块稀疏 KV 与 JIT 编译的 kernel；利用 Tensor Core 与专用指令（`cp.async`、TMA）执行矩阵运算
运行时	带延迟控制的动态批处理（vLLM、SGLang 与 NVIDIA Dynamo 中的实现，如 continuous batching），以整合请求；分页 KV 缓存管理，灵活分配内存并合并批次（vLLM 的 PagedAttention）；CUDA Graphs 与缓冲池，降低每次推理的开销；使用多条 CUDA 流（一条流负责数据传输，另一条流负责计算）来重叠计算与通信，仅在需要时使用基于事件的同步
系统编排	预填充-解码分离，消除队头阻塞；智能路由与缓存亲和，平衡负载与缓存命中；多租户隔离与每用户配额，防止噪声邻居；配合热备实例的自动扩缩容，隐藏模型加载时间，以略微增加成本换取流量高峰时显著更低的延迟
部署与基础设施	地理分布与边缘部署，降低网络 RTT；智能 API 网关，跨服务器池做请求级批处理；CI/CD 流水线，以金丝雀方式发布新的量化或 kernel 优化模型变体；NVLink/NVSwitch 与 InfiniBand 等高带宽互连，以及 GPU 与 CPU 间的 NUMA 亲和，保证最优内存访问
QoS 与扩缩容	面向 SLA 的动态批处理与尾延迟控制；用 MIG 或流优先级做 GPU 隔离以保障 QoS；TTFT、TPOT、利用率与内存带宽利用率的实时性能分析面板；根据负载特征动态切换并行策略（TP、PP、DP）

（vLLM、SGLang 与 NVIDIA Dynamo 中的实现，如 continuous batching），以整合请求；分页 KV 缓存管理，灵活分配内存并合并批次（vLLM 的 PagedAttention）；CUDA Graphs 与缓冲池，降低每次推理的开销；使用多条 CUDA 流（一条流负责数据传输，另一条流负责计算）来重叠计算与通信，仅在需要时使用基于事件的同步。

系统编排：预填充-解码分离，消除队头阻塞；智能路由与缓存亲和，平衡负载与缓存命中；多租户隔离与每用户配额，防止噪声邻居；配合热备实例的

自动扩缩容，隐藏模型加载时间，以略微增加成本换取流量高峰时显著更低的延迟。

部署与基础设施：地理分布与边缘部署，降低网络 RTT；智能 API 网关，跨服务器池做请求级批处理；CI/CD 流水线，以金丝雀方式发布新的量化或 kernel 优化模型变体；NVLink/NVSwitch 与 InfiniBand 等高带宽互连，以及 GPU 与 CPU 间的 NUMA 亲和，保证最优内存访问。QoS 与扩缩容：面向 SLA 的动态批处理与尾延迟控制；用 MIG 或流优先级做 GPU 隔离以保障 QoS；TTFT、TPOT、利用率与内存带宽利用率的实时性能分析面板；根据负载特征动态切换并行策略（TP、PP、DP）。

优化时要考虑跨层协同。例如，量化（模型层）降低了内存占用，从而允许更大的 batch size（运行时层）而不触发 OOM，这又让编排组件能够在每个 GPU 周期内合并更多请求。

还应当以性能分析为导向：持续的 profiling 应该指引下一步优化哪一层。例如，在完成融合与量化之后，如果 CPU 成为预处理和后处理的瓶颈，就应投入更快的分词器，或把部分任务卸载到 GPU。

应用优化时总有权衡需要取舍。层级 CPU offload 和高级解码方法这类技术会带来额外的复杂度：投机解码需要一个草稿模型，Medusa 则引入多头并行解码。它们通常只用于超长上下文或延迟波动剧烈等极端场景。更轻量的方法——稀疏化、批处理与分离式部署——在生产中贡献了大部分收益。

建议让模型架构、kernel 设计、运行时行为、系统编排与部署策略相互对齐，采用全栈优化方法。这意味着保持软件栈处于最新版本，包括 CUDA、cuDNN、NCCL 等；新版本往往包含最新的优化与缺陷修复。

全栈方法降低了技术栈任何一层成为瓶颈的概率。这样，团队就能系统性地消除瓶颈，在大规模 LLM 推理中实现稳定的低延迟并最大化硬件利用率。

16.1.5 调试正确性问题

监控也有助于捕获由缺陷引起的异常。例如，如果内存使用随时间不断攀升，可能是 CUDA kernel 存在内存泄漏。建议在测试阶段使用 Compute Sanitizer（`compute-sanitizer`）来捕获设备内存错误、竞态条件和越界访问，示例如下：

```
compute-sanitizer --tool memcheck your_binary
```

如果某块 GPU 的利用率明显低于其他 GPU，它可能因 NCCL 静默失败或未被捕获的错误而退出了 NCCL 通信组。可以在日志中查找 `WARN NCCL_COMM_FAILURE` 来检查 NCCL 错误码；它会输出非常详尽的错误日志。

> **注** 设置环境变量 `NCCL_DEBUG=WARN` 即可开启 NCCL 调试，这有助于暴露原本静默的错误。但请注意，NCCL 日志非常冗长！

可以使用 NCCL 测试套件来调试 all-reduce 与 all-to-all 的性能和正确性问题；也可以把 `ncclCommGetAsyncError` 与 `ncclCommAbort` 结合使用，检测并处理异步通信错误。还可以考虑启用 NCCL 的 IB GID

跟踪，并使用 NVSwitch 系统遥测，在互连层面同样及时发现这些问题。

应该在 Prometheus 的告警管理器（alert manager）中配置针对异常模式的告警，例如“GPU 利用率 $<$ 10% 且持续至少 60 秒”“内存使用超过 W% 阈值”“NVLink 错误率 $>$ X”“PCIe 重传超过 Y 阈值”“温度超过 Z 度”等。

实践中，可以像表 16-3 那样配置 Prometheus Alertmanager 规则。这样就能主动排查问题。

GPU 利用率低于 10% 且持续超过 60 秒：空闲——利用率过低；GPU 利用率高于 90%：瓶颈——可能已饱和；内存使用率高于 80%：警告——接近 OOM；内存使用率高于 95%：严重——OOM 风险高；温度高于 85 °C：警告——即将发生热降频；温度高于 95 °C：严重——有停机或硬件损坏风险；NVLink 重传/恢复错误不少于 1：严重——说明链路存在重试或恢复；NVLink CRC 错误超过 100 个/秒：严重——链路 CRC 失败率高；PCIe 重传错误不少于 1：严重——PCIe 总线上的数据包重传；不可纠正的 ECC 错误不少于 1：严重——数据损坏，需要重置。

硬件错误计数器与告警同样需要配置。例如，一旦报告 ECC 错误或 NVLink 重试，就应立即告警，因为这些问题会迅速拖垮性能，甚至导致掉线。掉线是指某块 GPU——或其互连——静默断开，例如 NVLink 掉线，或者 GPU 在发生致命错误后“从总线上掉线”。

表 16-3　面向 GPU 系统的常见 Prometheus 告警示例。

指标	条件	严重级别	说明
GPU 利用率	< 10%，持续 > 60 秒	空闲	利用率过低
GPU 利用率	> 90%	瓶颈	可能已饱和
内存使用率	> 80%	警告	接近 OOM
内存使用率	> 95%	严重	OOM 风险高
温度	> 85 °C	警告	即将发生热降频
温度	> 95 °C	严重	有停机或硬件损坏风险
NVLink 重传/恢复错误	≥ 1	严重	说明链路存在重试或恢复
NVLink CRC 错误	> 100 个/秒	严重	链路 CRC 失败率高
PCIe 重传错误	≥ 1	严重	PCIe 总线上的数据包重传
不可纠正的 ECC 错误	≥ 1	严重	数据损坏，需要重置

注　可以使用 DCGM 查看每条 NVLink 链路的吞吐量与总量，包括 `DCGM_FI_DEV_NVLINK_TX_BANDWIDTH_L*`、`DCGM_FI_DEV_NVLINK_RX_BANDWIDTH_L*` 与 `_TOTAL` 系列。必要时可退而使用 `nvidia-smi nvlink`、Nsight Systems/Compute 或 NVSwitch 计数器。

以一个 NCCL 失败案例来说：你可能会收到一条告警，显示某个节点的 GPU 利用率接近 0%，而其他节点为 90%。排查可以从节点日志入手，找出是哪个节点在产生 NCCL 错误。

这类主动监控与告警能让你更快发现这些问题、定位故障节点并着手将其恢复正常。在本例中，你可能

需要重新初始化 NCCL 通信域，或者对节点执行一次完整重启（只需确保重启后该节点重新加入 NCCL 组）。

为 NCCL 错误增加一个“NCCL 错误”计数器，可以更快捕捉到这类问题。此外，推理服务器可以把 NCCL 错误记录到日志中，由 Fluentd 或 AWS CloudWatch 自动抓取并转换为计数器。

随后，你可以在 Grafana 中把错误计数器图表叠加在各节点 GPU 利用率图表之上。这样就能将 NCCL 错误与 GPU 利用率的下降关联起来，从而更快地定位并修复故障节点。

> **注** 应用层计数器在生产环境中极为有用——尤其是与系统指标结合使用时。

只有在实际指标证明吞吐量提升、延迟降低、利用率改善等效果之后，优化才算成功。鉴于现代 AI 推理系统的复杂性，采用严格的、以测量驱动的系统性能调优方法至关重要。

简而言之，你应当把应用层计数器、由 Fluentd 或 AWS CloudWatch 自动抓取的日志计数器以及底层系统指标结合起来。这种全栈遥测提供了运维并优化一个在生产工作负载下以巅峰性能运行的多节点 LLM 推理系统所需的可见性。你应当把指标、计数器和日志视为系统行为的事实真相。

> **注** 我们的直觉和经验的直觉常常会把调试引入歧途，但指标不会说谎。请预先为代码做好恰当的插桩——出了问题时，相信指标。

16.2 动态批处理、调度与路由

即使模型已经在集群上完成了最优的切分与并行化，多节点推理集群部署中仍然存在应用层的优化空间。本节聚焦于通过动态批处理请求，以及采用经过优化的调度与路由策略，来最大化 GPU 利用率、最小化延迟并提升吞吐量的技术。

16.2.1 动态批处理

批处理（batching）是推理服务系统中最强大的性能技术之一。通过把多个到来的输入合并成一个批次交给模型一同处理，批处理将固定开销——如 kernel 启动和内存加载——摊销到多个输入上，从而提升吞吐量。不过，这是以牺牲单个请求的延迟为代价的：某些请求可能需要等待一段时间（例如 2 ms）才能加入批次并被处理。

动态批处理（dynamic batching）是请求批处理的一种特化形式，它即时组装大小动态变化的推理请求批次。可以将其配置为把请求缓存一段时间，或在达到给定的批大小时立即触发。所有现代 LLM 推理引擎都支持动态批处理，包括 vLLM、SGLang 和 NVIDIA Dynamo。

与动态批处理相对的是静态批处理（static batching）：它锁死固定的批大小（或将所有序列填充到最长），然后等待该批次中的每一个请求全部完成才返回结果。这会给早到的请求带来无上界的排队延迟——并让 GPU 周期浪费在填充上。

在动态批处理下，系统会累积到来的请求，一旦达到目标批大小、或一个短暂的超时（例如 2 ms）到期，就立刻派发“当前已到达的全部请求”。这样就把最大延迟限制在你指定的超时值之内。

凭借随到随定的大小，动态批处理让你把 kernel 启动开销摊销到多个序列上——同时避免了静态批处理最坏情况下的延迟。这在负载可变时既提高了 GPU 利用率，也带来了更可预期的延迟。图 16-5 展示了静态批处理与动态批处理的区别。

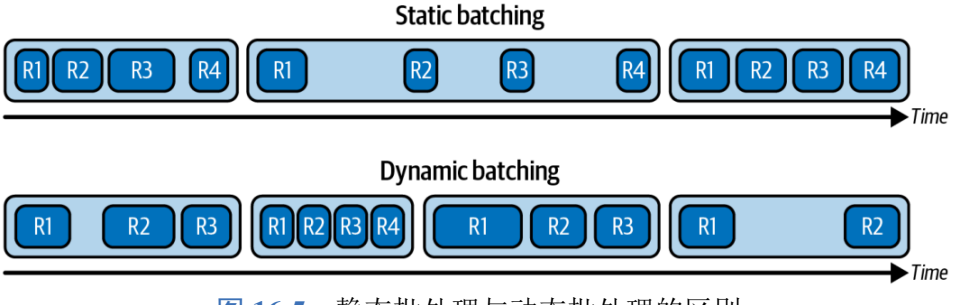

图 16-5　静态批处理与动态批处理的区别

动态批处理让系统根据实际的请求到达模式和延迟目标（例如最大延迟）自动增大或缩小批大小。关键在于进行智能的批处理：在把延迟维持在可接受范围内的同时提升整体吞吐量。现代推理引擎实现了微批处理（microbatching），即只把请求累积几毫秒的时间，然后

就将批次派发给 GPU。通常使用的延迟为 2–10 ms，但应结合延迟 SLO 加以调优。

例如，你可以配置 2 ms 的批处理延迟，用来规定服务器在派发批次之前等待更多请求的时间。如果三个请求在这个时间窗内到达，批处理器会立即将它们组合起来并发送给 GPU。2 ms 定时器到期之后到来的请求则会被收入下一个批次。这种由超时触发的机制为单个请求的排队延迟设定了上界（绝不会超过 2 ms），同时通过把多个序列组合在一起来提升吞吐量。

这些微批次减少了额外引入的延迟，并让 GPU 能够同时处理多个请求——而不是一次只处理一个。大型 LLM 模型在小批大小下是受内存带宽限制的，因此增大批大小会提高算术强度——进而提升整体硬件利用率。

在实践中，LLM 服务系统会选择一个均衡的批大小，在不引入明显延迟的情况下获得良好的吞吐量。在高负载下，动态批处理可以同时改善吞吐量和延迟，因为它避免了 GPU 在请求之间空转闲置。事实上，如果请求的到达率很高，批处理还能缩短整体排队等待时间，因为在同样的时间内处理了更多的工作。这有利于高负载流量模式下的端到端延迟和尾延迟。

在低 RPS（每秒请求数）下，动态批处理在等待其他请求加入批次时会带来少量额外延迟，这会使低 RPS 下的延迟略有上升。而在中高 RPS 下，这一延迟被摊销，与“把所有请求逐个执行”时本会发生的排队延迟相比可以忽略不计。因此，批处理降低了整体延迟，包括尾延迟。

> **注** 你应当用 Grafana 之类的工具绘制延迟百分位随负载变化的曲线，来验证改进是否确实发生。启用批处理之后，随着吞吐量上升，整体 p50 延迟往往保持平稳——甚至下降。这种趋势会一直持续到某个拐点。请记下这个拐点，并将负载控制在该值以下。

配置批处理时必须结合延迟 SLO 来考虑。例如，如果承诺某类长度的请求 p99 延迟为 2 秒，你就不能容忍某个请求为了等待进入批次而白白耗掉 500 ms。默认情况下，动态批处理延迟的初始值应设置得远低于 p99 延迟要求（取 1–2 ms 的量级），以避免过度的批处理延迟。

采用自适应批处理延迟时，批延迟值可以在低 RPS 时动态降到接近 0 ms，并在峰值负载需要时升到 5–10 ms。vLLM 等系统采用的正是这种自适应方式，以便在不同的流量模式下保持满足 SLO。

批处理主要惠及高流量场景。流量低时，系统只是逐个执行请求，延迟会非常低。而在高负载下，系统可以采用激进的批处理来获得高吞吐量，由于避免了队列积压并把延迟摊销到众多请求上，整体延迟反而更好。

16.2.2 连续批处理

连续批处理（continuous batching），也称为 in-flight 批处理或迭代级调度，通过在每一次 token 生成迭代中补充批次来维持高 GPU 利用率，而不是等待完整序列执行完毕。它把已完成的请求移出批次，并完全依照 GPU 的就绪状态立即补入新请求。这项技术对聊天助手等低延迟场景尤为重要。

与动态批处理这类超时驱动的方法相比，事件驱动的连续批处理策略消除了空闲的计算槽位以及上述方法固有的填充开销。由于从不依赖固定的“最大延迟”定时器，连续批处理允许新请求在生成中途加入正在进行的批次——而不必等待最长的序列，如图 16-6 所示。

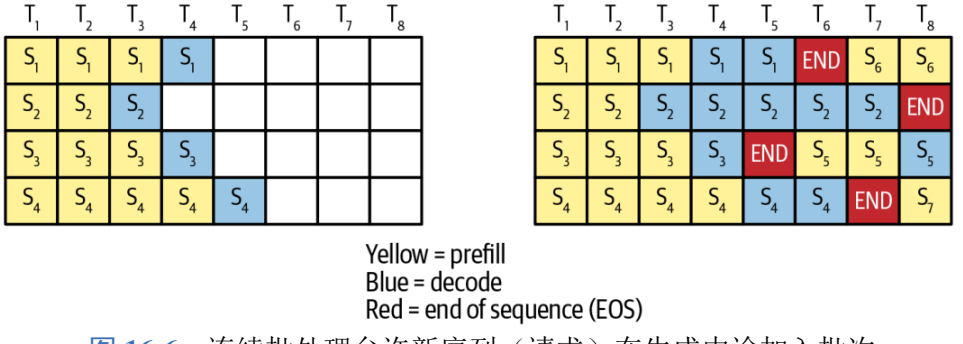

图 16-6　连续批处理允许新序列（请求）在生成中途加入批次

在这里，连续批处理把推理计算流水线中被浪费的槽位降到最少。它消除了每个批次因等待最长响应完成而产生的空闲时间。连续批处理不是等待批次内的所有序列全部完成（由于输出长度不一，这样做效率低下并导致 GPU 利用不足），而是在每次迭代时用新的序列替换已完成的序列。这种方式让新请求立即填补 GPU 槽位——带来更高的吞吐量、更低的延迟和更高效的 GPU 利用。

对于大型模型，把 4–8 个请求组成一个批次，往往能将吞吐量提升到 1–2 个请求小批次的 2 至 3 倍。这是因为 GPU 的运算单元和内存流水线得到了更充分的利用。而每个请求额外增加的延迟只有几毫秒的量级——

使得这一策略非常适合低延迟场景。表 16-4 总结了目前为止讨论的各种批处理策略，其中包括一种朴素的静态批处理策略。

表 16-4　静态、动态与连续批处理对比。

维度	静态批处理	动态批处理	连续批处理
触发方式	固定批大小或序列长度	批大小目标或超时	token 生成完成事件
延迟上界	无上界；需等待整批完成	受 `max_batch_delay_ms` 限制	极小；批内中途换出并补入
填充开销	高开销：填充到最长序列	中等开销：在批次内填充	低：槽位即时补入，无需填充
GPU 空闲缓解	差：变长负载下尤为明显	较好：但定时器过早触发小批次时可能造成 GPU 空闲	极佳：每次迭代都使 GPU 保持饱和
实现复杂度	简单	中等：需要定时器和队列管理	复杂：需要逐 token 协调与状态跟踪

% 表 16-4 正文溢出片段（源排版为表格小字内容，非独立正文）：完整内容已并入上方 b032 的表 16-4 浮动体，此处留空以保持条目一一对应。

% 表 16-4 正文溢出片段（源排版为表格小字内容，非独立正文）：完整内容已并入上方 b032 的表 16-4 浮动体，此处留空以保持条目一一对应。

受 `max_batch_delay_ms` 约束；极小；可在批处理中途逐出并重新填充

开销适中：只在每个批内进行填充；低：槽位无需等待填充即可重新填满

极佳：每次迭代都能让 GPU 保持饱和。实现方式：简单

稍好：但当计时器在小批次上触发时，可能导致 GPU 空闲

变长负载

复杂：需要逐 token 的协调与状态跟踪

16.2.3 连续调度

请求调度的另一个维度涉及并发模型执行。vLLM 社区将这种做法称为“连续调度”。其思路是：如果模型较小，或者批大小受延迟目标的限制，单个模型实例可能无法完全占满 GPU。

连续调度对待 GPU 的方式，就像操作系统调度器对待 CPU 一样：它在不同的 CUDA 流上启动多个彼此独立的小 kernel，从而让硬件 warp 调度器在各个任务之间交错执行 warp，而无需显式的上下文切换。

举例来说，如果我们的推理引擎未能完全占满 GPU，就可以在同一个 GPU 上并发运行多个模型推理流。这样，当一个实例在等待数据传输或某个非 GPU 操作时，另一个实例可以继续执行。

具体而言，在解码阶段，一次只生成一个 token 往往会导致 GPU 利用率不足——尤其当只对单条文本序列进行解码时。这是因为这类工作负载相对较小，且相对于计算量而言需要大量的内存搬移。

为了解决这种低效问题，推理引擎可以交错执行来自不同用户请求的多个解码任务。例如，如果 10 个用户同时在解码不同的序列，你不能简单地把它们合并成一次大型矩阵乘法，因为每条序列的长度可能各不相同；若不引入大量填充，GPU 就无法高效地执行矩阵运算。

作为替代，连续调度器可以在彼此独立的 CUDA 流上快速连续地启动 10 个小型解码 kernel，为每条序列生成下一个 token。借助 GPU 细粒度的 warp 调度器，这在各序列之间实现了真正的并发；这些 kernel 由此在各个 SM 之间交错执行，以接近轮转处理的模式消除空闲周期。

通过把每个解码任务放入各自的流，GPU 可以在各序列之间重叠执行内存传输与计算。这既降低了每个 token 的延迟，也提升了整体吞吐量。

GPU 在 warp 级别于这些 kernel 之间切换。当某个解码 kernel 因 CPU 侧控制流低效或网络 I/O 等待而停顿时，其他活跃的 kernel 可以立即继续执行，从而使 GPU 始终保持满载。

通过维护一个就绪可执行的 token 任务队列，连续调度即使以 token 级粒度也能实现高利用率。其最终效果与批处理类似：GPU 始终在做 token 生成，但无须把这些任务合并成一次巨大的矩阵乘法。它只是意味着我们不让 GPU 在 token 计算之间闲置。这种 token 级细粒度调度，对于在数百万用户的规模上最大化吞吐量而言至关重要。

连续调度器会在固定的时间间隔，或每当 GPU 空闲出来时，把所有待处理的下一个 token 请求收拢成一个大小可配置的批次，然后将该批次作为一次 GPU 调用分发出去。这有助于在吞吐量与延迟之间取得平衡。

> **注** 如果你正在设计自定义调度器，可以考虑混合方案：在 token 调度中同时使用较短的计时器和最大批大小。

vLLM 与 SGLang 等最先进的系统将动态批处理与连续调度融合在一起。它们各自定制的连续调度器，都围绕“挤出 GPU 时间中的『气泡』”这一思想构建。

vLLM 的 PagedAttention 就是这种混合方式的一个具体例子。PagedAttention 把注意力的键值（KV）缓存切分为若干分片，即“页”，并在活跃序列之间动态地按页组织计算。在高负载下，它通过将大型预填充 GEMM 与快速的小型解码 kernel 相互交错，维持接近 100% 的 GPU 利用率。

vLLM 高效地使用块级 KV 缓存结构来分别跟踪每个请求的状态。这种细粒度的记账方式通过实时地持续打包并复用工作负载，提供了快速的流式响应和最优的硬件利用率。

另一个例子是 SGLang 的 RadixAttention，它使用基于树的 KV 缓存分组，同样能达到接近 100% 的 GPU 利用率，并为未使用的缓存页引入了惰性逐出。稍后我们会更详细地介绍 vLLM 的 PagedAttention 与 SGLang 的 RadixAttention。这两种方案都是开源的，你可以直接在代码中审查它们的调度实现。

16.2.4 无停顿调度（分块预填充）

当提示词非常长时，可以把它切分成若干块，并交错执行预填充与解码工作。这被称为“无停顿调度”（stall-free scheduling），也叫“分块预填充”（chunked prefill）。

以一个 20K token 的提示为例。将提示切分为 5K token 的块，可以降低每次迭代的最大停顿，并把每次迭代的工作量限定为固定数量的 token，从而得到可预测的单次迭代延迟。不同块大小下分块预填充的开销如表 16-5 所示。

% 此条为表 16-5 的表体数据（块数、每块注意力运算量与 MLP token 总数），其内容已完整并入上方的表 16-5 浮动体。

这里，由于 KV 缓存的存在，每个块自身的注意力开销会随完整上下文而增长：每个新块都要让自己的 token 与之前所有块中的 token 逐一计算注意力。

表 16-5　对一个 20K token 的提示使用不同块大小的分块预填充时的开销。

块大小	块数	每块的注意力运算	MLP token 总数
20K	1	$20\,\mathrm{K} \times (20\,\mathrm{K} + 1) \div 2 = 200\,\mathrm{M}$	$1 \times 20\,\mathrm{K} = 20\,\mathrm{K}$
10K	2	$50\,\mathrm{M} + 150\,\mathrm{M} = 200\,\mathrm{M}$	$2 \times 10\,\mathrm{K} = 20\,\mathrm{K}$
5K	4	$12.5\,\mathrm{M} + 37.5\,\mathrm{M} + 62.5\,\mathrm{M} + 87.5\,\mathrm{M} = 200\,\mathrm{M}$	$4 \times 5\,\mathrm{K} = 20\,\mathrm{K}$

在自注意力中，对于长度为 N 的序列，QK 点积的数量呈三角形，约为 $N(N+1) \div 2$。这正是每层每头执行 Q × K 点积运算的次数。该开销关于 N 是二次的，即 $O(N^2)$；因此当 $N = 20000$ 时，约为 2 亿次运算。分块并不会减少这一总量，它只是改变了解码器何时能拿到工作。分块预填充的收益在于延迟的平滑与重叠的改善，而不是更少的注意力点积运算总量。

> **注**　预填充自注意力在每层每头执行 $N(N+1) \div 2$ 次 Q × K 点积，该开销关于 N 是二次的，即 $O(N^2)$。注意：分块与流水线式预填充只改变重叠与延迟，并不改变注意力计算的工作总量。要降低注意力的总开销，唯一途径是缩短有效上下文，或改用局部/稀疏注意力。例如，采用固定注意力窗口 W 即可把开销降至 $O(NW)$。

简而言之，分块预填充技术以紧密交错的方式调度提示的预填充处理与 token 生成。它把大批量预填充计算的高效率

与流式解码的响应性统一起来。在许多工作负载上，这可以在保持高吞吐量的同时降低尾延迟。

16.2.5 延迟感知调度与动态路由

延迟感知调度可以分析到来的请求、构建动态批次，并对批次进行路由以最小化延迟。考虑 6 个提示同时到达一个双 GPU 推理系统的情形，提示长度依次为 6K、2K、6K、2K、2K、2K。下面比较朴素的先进先出（FIFO）调度器与延迟感知调度器。

首先，FIFO 调度器按到达顺序创建两个批次。批次 1 为 [6K, 2K, 6K]，被送至 GPU 1；批次 2 为 [2K, 2K, 2K]，被送至 GPU 2。结果汇总于表 16-6。

表 16-6 长度为 [6K, 2K, 6K, 2K, 2K, 2K] 的到达提示序列在 FIFO 与延迟感知调度下的对比。

GPU	提示批次	自注意力 QK 运算量 $T(N) = N(N+1) \div 2$	Token 数 N
GPU 1	6K, 2K, 6K	T(6K) + T(2K) + T(6K) = 18,003,000 + 2,001,000 + 18,003,000 = 38,007,000	14K
GPU 2	2K, 2K, 2K	3 × T(2K) = 3 × 2,001,000 = 6,003,000	6K

表 16-6 按逐 GPU 汇总了 FIFO 调度下各批次的自注意力 QK 运算量（$T(N) = N(N+1) \div 2$）与 token 数。

可以看到，FIFO 策略将前三个提示 [6K, 2K, 6K] 发送给 GPU 1，需要执行约 38,007,000 次自注意力 Q × K 点积运算并处理 14,000 个 token 的 MLP，而 GPU 2 只需约 6,003,000 次自注意力 Q × K 运算和 6,000 个 token 的 MLP。关键路径由 GPU 1 决定，约为 38,007,000 次自注意力 Q × K 点积。

相比之下，延迟感知调度器会分析六个请求的预期延迟，并将它们重新编排为两个 [2K, 2K, 6K] 批次。在该策略下，每个 GPU 的自注意力开销为 22,005,000 次点积，MLP 处理 10,000 个 token，如表 16-7 所示。

表 16-7 对长度为 [6K, 2K, 6K, 2K, 2K, 2K] 的提示按原有顺序进行延迟感知调度。

GPU	提示批次	自注意力 QK 运算量 $T(N) = N(N+1) \div 2$	Token 数 N
GPU 1	2K, 2K, 6K	T(2K) + T(2K) + T(6K) = 2,001,000 + 2,001,000 + 18,003,000 = 22,005,000	10K
GPU 2	2K, 2K, 6K	T(2K) + T(2K) + T(6K) = 2,001,000 + 2,001,000 + 18,003,000 = 22,005,000	10K

表 16-7 同样按逐 GPU 汇总了延迟感知调度下两批次各自的自注意力运算量与 token 数。

这将关键路径上的自注意力开销从 38,007,000 降至 22,005,000，减少约 42%。因此，延迟感知调度器能够显著降低 TTFT。由于延迟感知调度器了解提示长度、自注意力中 $N(N+1) \div 2$ 的三角开销以及 MLP 中 $O(N)$ 的开销，它能够平衡计算负载并最小化整体延迟。例如，在本例中，延迟感知

调度器会提前安排 2K token 的请求。这使得这些较轻量的请求能更早完成预填充并开始解码，而无需等待较重的 6K token 预填充完成。

> **注** 上述计数假设使用仅对有效 token 进行计算的变长融合注意力 kernel，例如 FlashAttention 或采用 cuDNN 后端的 PyTorch 缩放点积注意力（SDPA）。如果 kernel 对批次内的最大序列长度进行填充，那么注意力算术开销将趋近于批次大小乘以该批次中最长序列的 T 个 token。在这种情况下，FIFO 与延迟感知策略之间的差异会缩小。

简言之，如果到达顺序在批次内并非全局最优，朴素的 FIFO 调度器可能使某一块 GPU 过载；而延迟感知调度器可以分析到达的请求，并将其重新编排为更均衡的批次，以最小化延迟。

> **注** 一些推理服务框架已经引入强化学习来在线调整调度策略。这是在本节所述延迟感知策略的基础上，针对不断变化的流量模式进行自动调优。

16.3 系统级优化

系统级优化技术包括通信与计算的重叠、最大限度提高 GPU 利用率、功耗与散热管理、错误处理以及内存访问优化。总的目标是确保我们的 GPU 硬件几乎 100% 的时间都在做有用的工作，也就是有效吞吐量（goodput）。我们还将讨论吞吐量与延迟之间的权衡，以及如何找到适合生产环境 SLO 的平衡点。

16.3.1 通信与计算的重叠

正如本书中反复看到的，对于分布在多块 GPU 上的大模型推理而言，通信与计算的重叠至关重要。像 GB200/GB300 NVL72 这类系统具有极高的带宽（机架内 GPU 之间 NVLink 聚合带宽高达约 130 TB/s），数据传输足够快、计算更不容易被“饿死”，因此重叠的效果更为显著。然而，即便在 NVL72 上，使用多条 CUDA stream 和非阻塞集合通信仍然至关重要。这能让你实现通信与计算的重叠以隐藏延迟，让全部 72 块 GPU 保持忙碌——即便在这样的速度下也是如此。

> **注** 实践中，你应当启用 NCCL 的 GPUDirect RDMA 支持，并使用 NCCL 的组调用（group call）来重叠多个小的 all-reduce 操作。此外，可以考虑使用 SHARP（在配备相应交换机的 InfiniBand 上可用）将归约操作卸载到网络交换结构上。该优化可以在某些交换结构与拓扑下提升吞吐量；在一些测试中，它为大规模 all-to-all 通信带来了约 10%–20% 的吞吐量提升。

设想一个推理步骤需要在 GPU 之间传输数据（例如 MoE 的 all-to-all）——甚至只是把提示数据从 CPU 送到 GPU、再把响应从 GPU 传回 CPU 以返回给最终用户。在所有这些场景中，只要可能，我们都希望这些传输在 GPU 执行其他工作的同时异步进行。

一个基本的例子是使用独立的 CUDA stream 来重叠计算与数据传输。例如，在专用的传输 stream 上（使用页锁定的主机内存）启动 `cudaMemcpyAsync`，而计算 kernel 在默认 stream 上运行。

> **注** 请记住，凡是经 PCIe 或网卡进行的传输都要使用页锁定（page-locked）的主机缓冲区。这样，CUDA 驱动程序才能直接执行 DMA，并与计算重叠。

随后，只有在真正需要数据时才使用 CUDA event 进行同步。这可以防止 GPU 因 I/O 而空闲。这样，数据传输将使用非阻塞的 CUDA 调用（例如页锁定内存）和 CUDA stream，GPU 就不会因等待这些操作而停顿。在具备全双工 NVLink 带宽的现代 GPU 上，这种重叠可以基本隐藏通信延迟。不过，你应当针对自己的具体工作负载，通过性能分析来验证这一点。

这里的数据传输可以包括把新的提示嵌入（embedding）从 CPU 送到 GPU，也可以是把最后一个 token 的 logits 从 GPU 传回 CPU。例如，GPU 刚为一批新 token 计算完 logits，我们就可以立即发起一次异步拷贝，把这些 logits 传到 CPU 进行后处理（例如采样）——或作为流式响应发回客户端——并立即启动下一个计算 kernel，为仍留在 GPU 上的另一批 token 继续计算。

在多节点场景中，像 all-to-all 这样的通信集合操作可以通过把工作分块来实现重叠。一些 MoE 运行时采用的一种技术是：把 token 批次一分为二，当第一批 token 正在被专家处理时，就开始发送第二批 token。

等到专家处理完第一批时，第二批的数据已经到达目的 GPU，它们可以立即继续处理而无需等待。这需要相对于计算 kernel 对 NCCL 调用进行仔细的调度。

例如，你可以使用 CUDA event 来标记批次的一半何时完成，从而实现这种重叠。此时，你可以为这部分数据启动 NCCL all-to-all——与此同时，下一部分正好计算完毕。你还可以使用 CUDA Graphs 捕获这些异步模式，以降低启动开销。其结果是 GPU 利用率的提升，因为 GPU 花在空闲等待通信上的时间更少了。

另一个可以重叠的领域是 CPU 与 GPU 之间。例如，当 GPU 正忙于生成下一个 token 时，CPU 可以并发地准备下一个输入，或对先前生成的 token 进行结果处理。一个高度优化的推理引擎会让所有 CPU 侧的预处理（例如输入文本的分词）与其他请求的 GPU 计算并行重叠。例如，引擎可以在与当前批次的 GPU 计算重叠的同时准备下一个批次。

这听起来显而易见，但它需要多线程架构：一个线程负责网络与排队，另一个线程负责启动 GPU 操作，还可以再由另一个线程负责响应后处理等。Python 或 C++ 推理循环绝不应因为等待某个本可并发完成的操作，而让 GPU 无法开始新的工作。

在流水线并行场景中，系统应当让流水线阶段之间的通信与各阶段内部的计算重叠。例如，GPU 0 在完成自己对 token t 的处理后，可以在开始处理另一序列的 token $t+1$ 的同时，把激活值发送给 GPU 1。

现代互联技术与框架允许计算 kernel 和数据传输重叠进行，只要它们使用不同的资源。你应该在推理流水线中使用多条 CUDA stream，让每个流水线阶段拥有独立的 stream。这样，激活值的发送/接收就可以与其他正在为不同微批次执行计算的 stream 并行进行。其效果是流水线气泡（bubble）减少。

在网络方面，NVIDIA GPUDirect RDMA 之类的技术允许 InfiniBand 网卡等网络适配器直接读写 GPU 内存，而无需 CPU 参与，也无需在主机内存中中转。利用 GPUDirect 跨节点传输 KV 缓存或专家激活值，推理引擎可以降低延迟和 CPU 开销。

GPUDirect RDMA 去掉了经由主机内存的中转，允许网卡的 DMA 引擎直接访问 GPU 内存。CPU 只负责提交工作请求，数据通路则绕开 CPU。这让 CPU 核心可以腾出来执行其他任务。

考虑一个实用示例：在跨两个 NVL72 机架的 MoE 层中，使用 InfiniBand 进行 all-to-all 通信。如果以同步方式进行——先完成 all-to-all 再进行计算——在没有重叠的情况下，GPU 利用率可能一直偏低。而通过将 all-to-all 通信按批次与计算重叠，你可以显著提高 GPU 利用率。

在 MoE 层中重叠 all-to-all 通信的做法是：把 token 批次一分为二，在第一批的专家仍在计算时就开始第二批的交换。这通过让通信与计算交错进行来隐藏通信延迟。

本质上，每块 GPU 一边对已有的 token 计算本地专家的输出，一边接收来自另一节点的其余 token。等到第一批算完，第二批已经到达，可以立即进行计算。

这类优化相当底层，涉及 NCCL 组与 CUDA 流之间的 CUDA 事件同步，但它能带来可观的吞吐量提升和更平滑的延迟。实践中，你首先要在一个 NCCL 组内发起 NCCL all-to-all 通信，而不等待其完成；随后立即启动下一个计算 kernel。注意务必使用 CUDA 事件来检查通信是否完成。

对于流式输出，我们还可以让 IO 与计算重叠。例如，只要生成了若干 token，就把它们通过 socket 发送给终端用户——与此同时模型已经在处理后续 token。这样就把网络延迟（例如把响应发回给终端用户）隐藏在计算（例如后续 token 的生成）之后。于是终端用户看到的就是一条没有停顿的稳定 token 流。

这种向客户端进行非阻塞流式输出的模式已被所有现代大语言模型推理引擎采用。它们使用独立的线程，通过 SSEWebSocket 向客户端发送 token 更新。

> **注** 如果你要自己实现流式输出，务必使用线程安全队列或锁，把生成的 token 移交给网络线程。你不希望在这次交接中引入同步问题——这类问题极难发现和调试。

如果不做重叠，我们就只能先生成少量 token，然后在把 token 发回终端用户期间让 GPU 闲置。取而代之的做法是：网络发送由一个异步线程负责，它从响应缓冲区取出输出并流式发回给用户。这样主推理线程（例如 CUDA 流）就

能立即继续生成更多 token。这实际上把 token 生成（计算）与输出发送（IO）流水线化了。

简而言之，让通信与计算重叠，需要以流水线的思路来思考，并尽可能使用异步操作。现代 GPU 和网络硬件提供了实现这一点的原语，包括 CUDA 流、非阻塞集合通信和 RDMA。CUDA 设备侧原语（如与 `cuda::barrier` 配合使用的 `cuda::memcpy_async`，来自 CUDA pipeline 头文件）有助于把 kernel 内部全局内存到共享内存的数据搬运与计算重叠起来。（注意：主机到设备、设备到主机的传输仍需借助显式 CUDA 流和页锁定主机内存才能与计算重叠。）高效的大语言模型推理系统会充分利用这些特性。

总而言之：只要有可能，就应让通信与计算重叠。否则，即便有 NVLinkNVSwitch，扩展到大量 GPU 时也达不到硬件的峰值性能。这些优化对终端用户基本不可见，因为模型输出没有任何变化；然而，它们对于从推理集群中榨取每一分性能、让用户持续回访你的应用来说至关重要。

16.3.2 最大化 GPU 利用率与吞吐量：同延迟的权衡

性能调优的终极目标是让 GPU 尽可能始终忙于有用的工作（例如矩阵乘法），同时把 GPU 资源的空闲和欠利用降到最低。前文介绍的许多技术——包括批处理、并行和重叠——正是为了实现接近 100% 的 GPU 利用率而设计的。

GPU 利用率百分比——更确切地说是有效利用率，即有效吞吐量（goodput）——是一个需要持续监控的重要性能指标。虽然你希望把有效利用率推向 100%，但必须确保不违反延迟等 SLO。到某个程度后，你会遭遇收益递减。

设想一个推理服务器，在朴素的实现下 GPU 利用率只有 60%。排查后发现，解码阶段是瓶颈：它让单个线程按顺序处理所有序列。现在引入多流并发解码。如前文所述，通过交织各路解码，我们把 GPU 利用率提升到 95%，吞吐量翻倍——这证明并发解码确实生效了。

我们可以把更多请求塞进一个超大 batch 来冲击 100%，但这会拖慢单个查询。一个常用做法是测试不同的 batch 大小，绘制吞吐量—延迟曲线。在这张图上通常会出现一个明显的“拐点”：超过它之后，吞吐量增益就要开始付出过高的延迟代价。

要在吞吐量—延迟曲线上找到最佳平衡点，首先开启完全并发并尽可能充分重叠，以确认硬件能够保持满载；然后逐步增大 batch 大小，直到资源利用率达到上限。此时，

测量单请求延迟，并只缩减到刚好满足响应时间目标的程度（例如从 16 降到 8）。

建议在监控 p50 中位延迟的同时，一并监控 p95 和 p99 尾部延迟。这是因为微小的抖动和批次填充不均会在 p95/p99 上产生显眼的长尾离群值。在服务大量请求的大型集群中，哪怕 0.1% 的离群请求也会频繁出现。因此，就衡量用户体验而言，p99——甚至 p99.9——可能比 p50 更重要。此外，长尾延迟常常迫使系统为满足严苛的 SLO 而过度配置资源。由此可见，降低尾部延迟具有直接的成本收益。

> **注** 一个实用的经验法则是：将批大小上限设在略低于达到峰值吞吐量对应值的位置。团队通常以最大峰值吞吐量的 90% 为目标，这称为余量缓冲（headroom buffer）。这是因为满速运行可能引发不可预测的延迟尖峰。例如，如果批大小增加到 16 个请求时开始给少数请求带来少量额外延迟，就应将批上限降到 8。这样能获得更一致的延迟，而吞吐量只略有下降。

一些推理系统可以根据这些指标在运行中动态调整批大小。我们将在第 17–19 章讨论这一技术以及更多自适应推理策略。

需要记住的是，让 GPU 持续运行在 100% 会因触及功耗和温度限制而导致节流。有时以 90% 运行并配合高效 kernel，反而能胜过以 100% 运行但发生节流的情形。接下来我们讨论 GPU 的功耗与温度约束——这是基于 CPU 的应用开发者和系统工程师通常不会考虑的特性。

16.3.3 功耗与温度约束

另一个需要考虑的维度是功耗与温度约束。若散热不足，让 GPU 持续以 100% 运行会导致温度节流。现代 GPU 系统采用液冷来减少温度节流并维持性能。如果你仍在使用较旧的风冷系统，就要警惕降频。

当 GPU 利用率较高时，可以用 `nvidia-smi` 或 DCGM 监控降频。具体来说，DCGM 通过 `DCGM_FI_DEV_CLOCK_THROTTLE_REASONS` 暴露 XID 节流原因；当 GPU 正被功耗节流时，`nvidia-smi` 会显示“Pwr Throttle”标志。

你还可能触及板卡上的功耗上限——尤其是在 boost clock 下。现代 GPU 在 boost 状态下的功耗显著增大，因此应监控 GPU 是否因触及功耗上限而降频。一旦发生，你得到的

将是不理想的 GPU 性能。请始终监控 DCGM 的 `DCGM_FI_DEV_POWER_USAGE` 指标，并在其超出正常范围时随时告警。

> **注** 如果你的 GPU 持续触及功耗上限，可以考虑启用 Dynamic Boost 模式——或略微降频——以避免可能造成延迟尖峰的温度节流。

要在软件层面规避这些约束，可以通过加入极小的批次间延迟来略微放松利用率，也可以限制并发度。现代 GPU 还支持时钟封顶（clock capping）。因此，与其增加延迟，不如用 `nvidia-smi -lgc` 或类似工具把 GPU 时钟略作封顶，以防触及温度上限。这样只会轻微降低吞吐量，并获得更一致的性能。

从硬件角度看，如果散热已经足够，可以尝试改进散热或提高功耗上限。要提高功耗上限，可使用 `nvidia-smi -pl` 或调整 GPU boost 设置。

这些提醒都在说明：把真实系统推向极限可能引发严重影响性能的意外副作用。建议在推理引擎中加入一个“boost-off”开关，当系统检测到因功耗或温度约束造成的节流而出现性能下降时，即时套用上述软件规避手段。这样，系统会以略微欠载、更凉爽的状态运行，直到完全恢复稳定与性能。

16.3.4 错误处理

错误处理虽然通常不被视为性能话题，但高效地处理错误十分重要。被充分利用的推理系统留有更少的余量来应对过度的错误激增——尤其是错误处理很可能不是系统中最优的代码路径。

在故障场景中，快速失败是关键：如果一个请求注定会出错，最好立即返回错误，而不是以缓慢的失败浪费 GPU 时间。务必围绕推理调用实现恰当的异常和超时机制，以捕获挂起或崩溃。

此外，建议实现背压（backpressure）。也就是说，当错误激增时，可以开始拒绝新请求——或减小批大小——给系统留出恢复的余量。

在规模化场景下，建议预留一些余量：加入一些大部分时间处于空闲的额外副本。它们可以作为错误激增或其他意外情况的缓冲。虽然自动扩缩容（autoscaling）很可能是你为降低成本最先想到的手段，但请记住，自动扩缩容需要时间来提供新资源。

空闲容量固然要花钱，但失去客户或违反 SLA 的代价往往更高。建议在稳态下至少保留 10%–15% 的缓冲容量。对于不能承受停机的更关键服务，建议配置 100% 的缓冲容量——即一整份集群副本。

对于能应需立即承接额外负载、且已预热完毕的空闲节点，没有任何替代品。失去终端用户的代价，很可能高于让少量副本空闲待命以应对额外意外负载的代价。

16.3.5 内存

资源利用率优化同样延伸到内存。GPU 显存和内存带宽的增长较为渐进，而模型规模和上下文长度却在指数级增长。因此，内存依然是值得优化的珍贵资源——而且在可预见的将来仍将珍贵。

因此，你需要有效利用 GPU 显存和内存带宽。内存通常由模型权重和 KV 缓存占据。推理时应始终把模型权重留在 GPU HBM 中。如果在 CPU DRAM 或 NVMe 存储之间往返换入换出内存，会引入额外的缺页和传输延迟。

即便是 Grace Blackwell、Vera Rubin 这类现代 CPU-GPU 超级芯片，这一额外的传输延迟依然存在。因此，高性能 LLM 推理引擎会显式管理内存，而不是依赖按需换页。

压缩是降低推理系统内存占用的有效技术，尤其适用于 KV 缓存——它由预填充阶段为进入系统的每个查询生成。

由于 KV 缓存的大小随查询数量和输入规模增长，应考虑对 KV 缓存进行压缩与量化。KV 缓存压缩/量化意味着：如果模型能容忍精度损失，就存储低精度的键和值。另外，对于很少使用的 KV 缓存数据，KV 缓存卸载（offloading）虽不理想，但也是一个可选方案。

16.3.6 KV 缓存卸载与内存池分配

通过将很少使用的 KV 缓存条目卸载（换出）到 CPU 内存或磁盘，推理引擎为更活跃的数据腾出空间。这类似于虚拟内存的处理方式。

例如，vLLM 的 PagedAttention 为 KV 缓存使用托管内存池，把数据卸载到 CPU 内存和 NVMe 存储。类似地，SGLang 的 RadixAttention

使用树形结构的缓存，可以惰性驱逐最少使用的前缀。NVIDIA Dynamo 也有类似的 KV 缓存卸载与内存管理机制。

而且，如果没有一个好的 KV 缓存分配器，当不同长度的请求流经系统时，会产生过多的内存碎片（例如约 20%–30%）。这会限制你在遇到 OOM 错误之前所能处理的请求数量。请记住使用大容量的内存池分配器，如前文某章所述。

采用恰当的内存换页策略可以把碎片降低到可以容忍的几个百分点。这意味着你可以在 GPU 中塞入更多上下文，并保持高利用率而不使系统崩溃。

有了恰当的 KV 缓存内存管理，现代推理引擎可以在不耗尽 GPU 显存的情况下服务多得多的并发用户。它们通过自行管理 KV 缓存内存池，并把数据卸载到 CPU 与 NVMe 存储来实现这一点，从而以极少的碎片达到接近满载的内存利用率。

其代价是：如果这些上下文重新变为活跃并需要换回显存，就会产生一定的额外数据传输延迟。不过，与请求的总延迟相比，这类开销通常很小。

内存利用率与计算利用率相辅相成。如果内存管理不善，不仅会浪费显存，也无法用更多并发任务充分利用 GPU。只有把尽可能多的相关数据保留在 GPU 上，系统才能同时服务海量并发请求。

糟糕的内存管理会在峰值负载下引发反复的 OOM 崩溃，使 GPU 被迫退出资源池，并带来级联的延迟问题。应当借助合理的内存管理避免 OOM，从而最大化利用率并保持集群稳定。

简而言之，高效的内存管理能够提升有效吞吐量、减少内存碎片、避免意外的 OOM 错误，并支持更多并发在途请求——在高吞吐场景中尤其如此。

16.4 面向实时推理的量化方法

提升推理性能最有效的手段之一是降低精度。这会立即减少内存占用和内存带宽消耗，同时提高计算速度。

量化是用更少的比特来表示模型权重，有时也包括激活值。现代 NVIDIA GPU 通过采用降精度 Tensor Core，对 FP16、FP8、FP4 等格式提供原生的低精度算术支持。

本节讨论专门面向推理的量化技术，既包括 GPTQ、AWQ、SpQR 等仅权重量化方法以及其他结构化稀疏感知方法，也包括对权重与激活值同时进行的全量降精度。其中，GPTQ 和 AWQ 已在实践中被证明非常有效。

对许多大模型而言，基于 GPTQ 的 4-bit 仅权重量化可以保留 FP16 模型 99% 以上的精度，同时带来约 2× 的推理加速，并将模型占用缩小约 4 倍。AWQ 则进一

步改善了 3–4-bit 量化下的精度。这些技术已集成到 Hugging Face Transformers、PyTorch、vLLM 等众多 AI 框架中，可直接加载 GPTQ 与 AWQ 量化模型。接下来我们将讨论使用降精度格式在精度与性能上的权衡，以及如何安全有效地把量化集成到推理服务流程中。

16.4.1 从 FP16 降到 FP8 与 FP4

最初，LLM 推理通过从 FP32 转向 TF32、FP16 或 BF16 等降精度格式获得了显著收益。例如，NVIDIA Tensor Core 使用 FP16 时的吞吐量是 FP32 的 2×：它将半精度的乘加运算融合到专门的 Tensor Core 硬件流水线中，使数学性能翻倍，而精度损失几乎不可察觉。

FP8 把精度进一步降到 8-bit 浮点数。相比 FP16/BF16，它能将内存占用再减半；同时 Tensor Core 的数学吞吐量也再翻一番，因为 GPU 每个周期能执行的 8-bit 乘加次数是 16-bit 运算的两倍。

虽然在 PyTorch 中仅通过 `torch.set_float32_matmul_precision("high")` 启用 TF32 数学就能获得适度的加速，但真正需要的是充分释放 NVIDIA Transformer Engine（TE）所提供的 8-bit 与 4-bit 精度支持。TE 以库的形式提供 FP8 和 FP4 kernel，让现有代码只需极少的改动即可使用这些降精度格式。

NVIDIA 的 TE 会在这些降精度下自动管理逐张量的缩放因子。推理时，推理服务器可以按 FP16 加载模型，却使用 FP8 矩阵乘法来完成计算。

TE 会对每个张量施加缩放以维持数值稳定，缩放因子通常以两种方式之一确定：一是在训练期间使用代表性数据完成的、固定的事前校准步骤，称为静态校准（static calibration）；二是动态计算的值，用于跟踪张量的绝对值最大值，称为基于 amax 的动态缩放。图 16-7 展示了 TE 在精度转换中如何使用范围分析、缩放因子与目标格式。

若要更高的压缩率，FP4 格式比 FP8 更能缩减模型权重的存储与传输。计入缩放元数据与打包开销后，其有效压缩比 FP8 约多 1.8×，比 FP16 约 3.5×。不过，FP4 的动态范围非常有限，要在 FP4 下可靠地推理，需要逐通道（per-channel）缩放或其他校准手段，例如 GPU 的 TE 所支持的 NVIDIA 逐块微缩放（microscaling）。正是这些技术把精度损失降到最低，才使 FP4 能够用于大型网络。

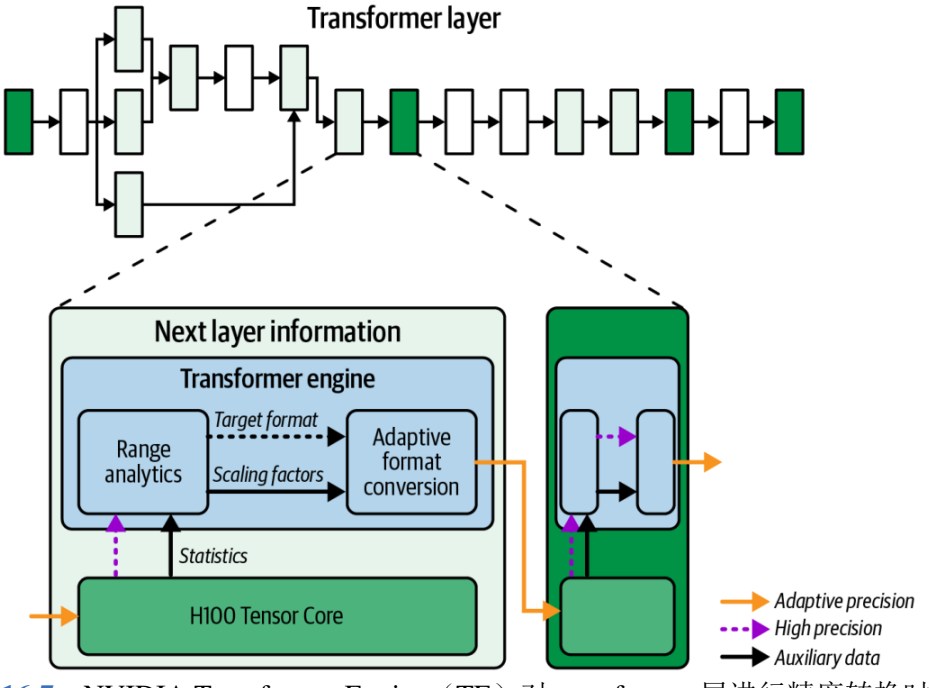

图 16-7　NVIDIA Transformer Engine（TE）对 transformer 层进行精度转换时使用的范围分析、缩放因子与目标格式

16.4.2 仅权重量化（GPTQ、AWQ）

现代 LLM 推理服务栈中的仅权重量化，通常使用 GPTQ 或 AWQ 等方法把权重压缩为 4-bit 整数，同时让激活值保持在 FP8、FP16 或 INT8 等较高精度。这样做可将权重的内存占用相比 FP16 大约缩减 4 倍，把权重带宽减半乃至更多，而且只要校准得当，精度损失通常极小。

NVIDIA 的 FP4 实现（官方名称为 NVFP4，本书中径称 FP4）在硬件中采用逐块微缩放。NVIDIA TE 在块粒度上为 NVFP4 微缩放提供硬件支持。

> 注　根据 kernel 的不同，选择逐张量或逐通道缩放。建议对激活值采用逐张量缩放，对权重采用逐通道缩放。例如，使用 FP8 E4M3 做 KV 缓存量化时，通常采用逐张量缩放。

逐块微缩放是指不再用一个缩放因子覆盖整个张量，而是在张量内部为每个固定大小的块（通常 32 个元素）维护一个独立的缩放因子。与单一缩放的量

化相比，这些独立的缩放因子能够适应局部数值分布，从而保持数值范围并降低量化误差。

> **注** 做量化时一定要使用能反映实际推理工作负载的校准数据。只在训练数据的一个子集上做一次性校准，可能捕捉不到运行时的真实使用模式。

实践中，GPTQ（训练后量化）和 AWQ 常用于对 LLM 的权重做 4-bit 量化——精度损失通常可以忽略不计。Hugging Face 等提供的开源工具可以自动应用这些技术。

MoE 的专家结构使权重量化更具吸引力：采用降精度权重后，显存中能容纳更多专家，从而减少在 CPU 内存之间换入换出的专家数量。此外如有需要，我们还能运行专家更多、激活专家更多的更大模型，并部署更多专家副本。

GPTQ 这类训练后量化（PTQ）工具采用近似的二阶算法逐层量化权重，只需几个 GPU 小时即可降到 3–4 bit，且几乎没有精度退化。较新的 GPTQ 变体通过非对称校准和并行计算进一步改进了该算法，降低了量化误差，并把高效低比特支持扩展到更大的模型。

AWQ 会识别出一小部分”显著”（salient）的权重通道。显著通道会产生较大的激活幅值，对模型输出具有不成比例的巨大影响。

AWQ 采用针对具体通道的缩放来保护这些通道，然后再把全部权重转为 4-bit 精度（例如 INT4）。NVIDIA TE 会利用这些被保留通道上的通道级缩放因子，以维持模型保真度。

> **注** PyTorch 等 AI 框架以及 vLLM、TensorRT-LLM 等推理引擎，均原生支持加载 GPTQ 量化和 AWQ 量化的模型检查点。

16.4.3 激活量化

对激活值随权重一起量化，可以让 GEMM 的输入乃至累加器使用低精度数值，从而提升性能。这会同时减少基于注意力机制的 KV 缓存和 MLP 层中间激活值的内存流量。

然而，激活值的分布会随输入不同而剧烈变化，因此若没有恰当的微调或校准，激活量化有时颇具挑战。一种折中方案是带校准的 INT8 激活量化，它源自 NVIDIA 的 INT8 模式：采用逐张量校准，并被 TensorRT 用于从一组代表性校准数据生成的激活直方图中挑选缩放因子。

SmoothQuant 是一种无需训练/校准的 8-bit 激活量化 PTQ 方法，它用一个简单的按行/按列缩放算法，把部分量化误差从激活值转移到权重上。这样只需极少量的微调，就能让权重与激活值都使用 INT8——实现精度损失很低（例如 < 1%）的完全 INT8 推理。

> **注** 实践表明，先对激活值应用 SmoothQuant 量化、再对权重应用 GPTQ/AWQ，在低精度下能更好地保持精度。

16.4.4 训练后量化工作流

前文介绍的量化技术都是在训练后应用的——与之相对的是量化感知训练（quantization-aware training），后者在模型训练（也称模型预训练）期间进行。典型的流程是：先以 FP16/FP32 训练或微调模型，然后在代表性数据集上运行 `GPTQ`、`AWQ` 等训练后校准脚本以确定量化参数，最后把量化后的模型权重加载到推理引擎中用于服务。

如有需要，还可以运行一次小规模微调作业来恢复损失的准确率；不过，对于 `GPTQ` 和 `AWQ` 技术通常无需这一步。如果确实需要完整的量化感知训练（QAT），只需在模型计算图中加入“伪量化”运算、再训练几个 epoch，即可在评估时模拟低精度运算。这有助于你了解该精度设置下可期待的准确率。

> **注** 在实践中，对 LLM 做量化感知微调的计算开销很大，且并不总是可行。不过，可以借助百分位截断（percentile clipping）或 LMS（损失感知量化）等技术，用较小规模的校准数据集（例如 1000 条提示）来微调量化缩放因子，而无需完整重训。

值得注意的是，应格外小心地在压缩与模型鲁棒性之间权衡。量化会放大某些误差。例如，若模型对某条事实知识的掌握本就处在临界状态，量化可能就把这种状态推向错误。

务必在下游任务上进行验证，因为 PTQ 所依据的假设可能遗漏细微的分布偏移。因此，在使用降低了精度的量化模型时，应对回复开展充分的 A/B 测试，确保评估结果显示质量与安全性均未出现回归。若出现回归，就应让模型保持在更高精度——或者尝试以量化形式做一次轻量微调来恢复性能。

16.4.5 权重与激活量化的组合

把激活量化到 4 位仍然很有挑战。将低精度权重量化与较高精度的激活量化相结合，往往能在节省内存、计算效率与准确率之间取得最佳折中。因此，许多生产系统采用仅权重的 4 位（weight-only，例如 `GPTQ/AWQ`）量化，并配合 8 位激活量化。

具体而言，在一种 W4A8（8 位激活）变体中，运行时先解包 INT4 权重，再利用学习得到的或校准得到的缩放因子将其反量化为 FP8，然后在 FP8 Tensor Core 上执行矩阵乘法。这种混合路径由 `TensorRT-LLM` 等推理引擎提供，经妥善校准后可实现近乎无损的准确率。它保留了激活的完整动态范围，同时使权重存储相比 FP16 缩减至 1/4（即 4× 压缩）。

相比之下，传统的 INT4 与 INT8 W4A8 方案将 4 位整数（INT4）权重与 8 位整数（INT8）激活配对，并在 INT8 Tensor Core 上执行计算。这一方式依赖基于直方图的校准，把激活范围映射到 INT8 而不损失质量。

虽然现代整数 Tensor Core 流水线可以让 INT4/INT8 kernel 获得略高的原始吞吐量，但这类方案需要仔细的激活校准，且动态范围不如 FP8。

INT4 与 FP8 的混合方案则兼具两者之长。在这种混合方案中，4 位整数（INT4）权重先被解包，再重新解释为 FP8 输入，随后在 FP8 Tensor Core 上执行计算。这种 INT4 与 FP8 的混合

W4A8 变体可实现近乎无损的准确率、大幅降低的内存带宽占用，并在现代 GPU 上带来出色的吞吐量。

对于现代 GPU，生产中有两条常见的低精度路径。其一，NVFP4 工作流使用以 FP4 块配合微缩放（microscaling），由 `Transformer Engine` 或 `TensorRT` 管理。其二，W4A8 工作流使用 INT4 权重搭配 FP8 或 INT8 激活，在 `TensorRT-`

LLM 中以融合反量化方式执行。应根据模型的校准情况与准确率目标来选择路径。

总而言之，量化是降低推理成本的最佳手段之一。把模型规模至少缩减一半，就相当于有效地让每块 GPU 的吞吐量翻倍。前文提到的 GPTQ、AWQ、SmoothQuant 等技术，能以极小的准确率损失（例如通常 < 1% 的下降）帮助你获得这些收益。

> **注** 建议先从 8 位权重量化起步，再评估仅权重的 4 位量化以获取更多收益；只有当你需要把优化做到极致、并愿意投入时间进行校准时，才迁移到 W4A8。

下一步是消除一切类型转换开销。通过把量化-反量化运算直接融合进计算 kernel，就能保住量化带来的收益。

16.4.6 将量化-反量化步骤融合进执行图

现代推理引擎使用 Transformer Engine（TE）来打包权重，并提供高效的混合精度数学计算，而无需承担使用 INT8 时那种显式校准的开销。这类推理引擎通常在需要时通过 CUDA/Triton kernel 手动把“量化-反量化”步骤融合进执行图。

当独立的量化/反量化 kernel 会引入过多的额外启动开销、从而抵消低精度计算在延迟与带宽上的收益时，就应当把这些量化-反量化步骤融合进计算图。这对缺少原生融合 INT8 支持的推理后端尤其有用。

对于高吞吐量推理流水线来说，把量化-反量化融合进主计算 kernel 尤其有价值：它通过消除类型转换造成的瓶颈来恢复性能。接下来，我们探讨现代 AI 推理服务引擎支持的各类应用级优化。

16.5 应用级优化

在核心的模型与系统优化之外，还有若干更高层面的技术可以显著改善 LLM 服务的性能与用户体验。这些方法作用于应用层或推理服务层，改善提示词的构造与缓存方式、对话历史的保留方式、请求路由到不同模型的方式等。

这些优化不涉及修改模型权重，也不需要部署新硬件，而是应用层的算法与系统级改进。它们几乎不产生成本，因此基本上可以“免费”换来效率与可用性的显著提升。

本节讨论其中几项优化，包括提示词压缩、前缀缓存与去重、回退模型路由（fallback model routing）以及流式输出。这些策略通过减小输入规模、避免冗余计算、对不同类型的请求提供优雅处理，并改善最终用户的感知延迟来提升性能。

16.5.1 提示词压缩

用户常常向 LLM 发送很长的提示词或对话历史。然而，并非所有这些上下文都是生成一个合格回复所必需的。提示词压缩指一组在不丢失相关信息的前提下缩短或简化输入提示词的技术。

有些系统提示词或系统注入的指令相当冗长（“你是 ChatGPT，一个旨在帮助用户的友好助手……”）。庞大的系统提示词会在输入上下文中占据大量空间——而且是针对每一个请求。

提示词压缩减少了模型需要完成的工作量，直接带来成本节省，因为更短的输入意味着更少的 GPU 计算。

> **注** 请记住，在基于 transformer 的 LLM 模型中，注意力机制的时间复杂度为 $O(n^2)$，其中 n 是以 token 数衡量的输入规模。

提示词压缩的一种简单形式是删除冗余或无关文本。设想用户提示词中包含一大段与其问题无关的文本：例如原样粘贴了一篇文章，而问题只与其中的一个段落有关。此时，可以在把内容送入 LLM 之前，由上游组件先做摘要或抽取相关部分。

提示词压缩的另一种形式是对话摘要或截断。例如，对于很长的聊天历史，对话的前段可能已经不再相关。系统可以做智能裁剪，把较早的对话内容摘要成精简的形式。

> **注** ChatGPT 等许多生产聊天机器人会自动执行提示词压缩，以便保持在上下文长度限制之内——同时加快整体处理速度。对于长期持续的对话来说，这是必备能力。

具体做法是：运行一个小型 LLM——或者传统的基于规则的系统——把对话中最早的部分摘要成一段简短概述。策略通常类似于：当对话超过最大上下文长度的 75% 时，就对最早的 25% 消息做摘要。该摘要随后会被置于对话较新部分之前，并成为此后使用的新提示词。

在实现提示压缩时，必须确保关键事实不丢失。一种做法是让模型对提示进行压缩（例如生成一份摘要），然后通过让模型回答关于该压缩提示的问题来加以验证。这样，算法就能检查关键信息是否保留在了压缩后的提示之中。

这可以避免超长对话产生过高的延迟，并使模型聚焦于对话最近的部分。通过截断较早的轮次，你降低了有效提示长度 N，从而把预填充自注意力计算量从 $N(N+1) \div 2$（大致为 $O(N^2)$）降低为较短窗口对应的更小三角形成本。因此，虽然成本随窗口大小仍趋于平方级，但对超长对话而言，更小的 N 带来了显著差别。

一份好的摘要能过滤掉无关细节，从而改善回复质量；但一份糟糕的摘要可能漏掉用户真正看重的重要输入内容。摘要最适用于系统对对话有把握的场景，或者对话明显已经跑题的情形。

16.5.2 提示清洗

另一种技术是提示清洗（prompt cleansing），用于改进输入的格式与分词效果。它有助于减少发送给模型的多余空白符和标记。分词器会处理每一个字符，包括空格和换行；我们发送的多余 token 越少越好。

虽然 OpenAI 的 tiktoken 这类分词器对空白符的处理相当高效，但包含大量 markdown 和 HTML 的超长提示仍会膨胀 token 数量。简单的

预处理——比如删除 HTML 标签、把花引号转换为普通文本——就可以避免奇怪的分词结果并减少 token 数量。

例如，只要空行和重复的标点不影响输入的含义，我们就可以不发送它们，从而减少推理引擎的计算量。这样做或许每次只能省下几个 token，但成千上万个请求累计起来不容小觑——对超长输入提示尤其如此。

在某些情况下，我们可以用引用代替完整内容来压缩提示。例如，用户的提示中包含一段系统此前已见过的长文本——也许用户引用的是我们之前存储过的文档——我们就可以用诸如 `"file0"` 这样的引用来替换它。

随后可以对模型进行微调，使其能够根据该引用检索出实际内容——或者用一个检索系统来处理。这就进入了检索增强生成（RAG）的领域，本书不再展开，但要点是：只要存在替代手段，我们并非总是必须把原始内容喂给模型。

> **注** 一些推理引擎支持在会话中只设置一次系统提示，而不是每次请求都重新发送。这比为每个请求反复压缩系统提示是更好的方案。

下一节将讨论如何借助前缀缓存提升大型系统提示的效率。眼下，我们先利用提示压缩来生成一套更短且功能等价的指令。例如，可以让模型学会用特殊 token 或元数据来表示一段长系统提示，这样它就不必总是处理冗长的自然语言版本。

设想一个由 200 个 token 组成、满篇文字规则的系统提示，我们可以用 10 个特殊 token 来替换它，这些 token 能触发同样的规则——规则本身在自然语言里占 200 个 token。这要求模型经过训练后能解析元数据、推导出规则并加以遵循。

围绕“config”token 的研究正在进行中：这类 token 会告诉模型为给定的配置加载一组预先配置好的指令。可以把这理解为为某个提示前缀（如系统提示）分配一个唯一 ID。例如，微调一个模型使其把 `<POLICY_A>` 这样的 token 识别为一份 500 词长政策的替身，是很常见的做法。

> **注** Hugging Face Transformers 库实现了流行的 CTRL 方法。这是着手用 config token 做提示精简的一个不错的起点。

用特殊 token 和元数据替换长系统提示更多属于训练层面的考量。但如果它确实缩小了提示体积、降低了预填充开销，就能带来显著的推理提速。在大规模部署下，这直接意味着每个查询所需的计算更少——成本节约更多。

16.5.3 前缀缓存

发往 LLM 的多个请求往往在输入中共享一个公共前缀，因为许多查询可能以同一个系统提示开头。与其每次都为相同前缀重新计算模型状态，不如计算一次，并在后续请求中复用同一份 KV 缓存。

这项技术称为前缀缓存（prefix caching），有时也叫前缀记忆化（prefix memoization）。采用前缀缓存后，当出现与某次请求具有相同前缀的请求时，transformer 的状态——包括注意力层中的键和值——会被存储下来并复用。

> **注** 前缀缓存可以把本来总量为 $O(N \times L)$ 的工作（即 N 个长度为 L 的请求），通过复用重复部分的已缓存前缀计算，变成 $O(N + L)$ 的工作量。

vLLM 实现了前缀缓存（`enable_prefix_caching=True`）来避免重复计算。它先检查到来提示的前 N 个 token 是否与缓存中已有请求——或同一会话中较早请求——的前缀匹配。若匹配，vLLM 就免去对这 N 个 token 代价高昂的注意力重复计算，而直接把缓存中的 KV 数据复制到新上下文里。务必启用前缀缓存，并为其分配足够的内存。

像 vLLM 这样的推理引擎可以自动把到来的查询组成微批次（microbatch）以获得高 GPU 利用率，把开销摊到多个请求上，并借助连续批处理（continuous batching）等手段保持低延迟。如果你没有直接使用这类推理引擎，请务必在你的技术栈中寻找利用前缀共享的途径。

前缀缓存可以加速带有重复前缀的工作负载。设想这样一个场景：我们要针对一篇长文档提出 10 个独立问题。每个提示都会包含该文档加上一个问题，例如“[长文档文本] 问题 1：……”“[长文档文本] 问题 2：……”等等。

在这 10 个提示中，文档文本完全相同。正常情况下，模型每回答一个问题都要重新处理整篇文档的 KV 条目，也就是说模型要把长文档重新编码 10 次，造成 10 倍的冗余工作。有了前缀缓存，文档只编码一次，后续每个问题只需为较短的问题后缀付出计算。

具体来说，采用前缀缓存时，第一个查询会为文档部分计算 transformer 状态，而后续查询因文档内容与 KV 缓存中已有的前缀匹配，可以直接跳到处理“问题 1：……”“问题 2：……”这些部分。

借助前缀缓存，推理引擎有望获得接近线性扩展的加速。例如，在同一文档上提出 10 个独立问题的话，启用前缀缓存的处理速度大约是无缓存时的 10 倍，因为长文档的注意力只需计算一次，而不是分别计算 10 次。

前缀缓存的另一个应用场景是聊天会话：在多轮（multiturn）聊天会话中，对话历史构成了每一新轮次的公共前缀。当终端用户发送新消息时，先前的消息构成一个不断增长的前缀。

优化过的推理系统会保留上一轮次的 KV 缓存，仅为新的用户消息计算 KV。只要你在轮次之间不重置对话，ChatGPT 这类聊天模型大多数就是这么做的。

> **注** 前缀缓存的收益在交互式场景中最为明显——尤其是用户在同一上下文/会话中快速连续追问时。此时第二个回答会远快于第一个回答，因为前缀（对话历史）已经缓存。

OpenAI 不仅在其 ChatGPT 消费级产品中支持状态，在其 API 中也支持有状态对话，幕后借助了前缀缓存等众多优化之一。对于无状态的 API 调用，只要客户端传入对话历史，前缀缓存也能达到类似效果。此时服务端可以从前缀缓存中取回预计算的隐藏状态——一直到最后一个轮次——然后从那里继续处理新的用户消息。

如果想自己实现这一点，你可以在对话历史上做哈希，从而得到一个一致的缓存查找键。但内存必须细心管理，因为为大量对话保存完整 KV 缓存会很快耗尽 GPU 内存。vLLM 的分页机制对此有帮助：它把不活跃的缓存分页到 CPU 内存——甚至磁盘——并在缓存命中时再换回 GPU 内存。

你还可以跨多个请求——甚至单个请求内部——对提示的各个部分做去重。这里的去重是指把相同的子提示合并，只计算一次其 transformer 状态。例如，如果两个用户几乎同时发送完全相同的提示前缀，系统可以合并这两个提示，只处理一次。

只要输入中含有重复序列，单个请求内部也可能发生这种情况。这在训练中更为常见，训练会采用记忆化解码（memoized decoding）等技术来去重重复文本。在推理中，同一请求内出现长而重复的输入序列的情况很少，但并非不可能。

> **注** 如果你的工作负载在同一前缀上有大量重复查询，应为缓存分配更多内存以最大化命中率。反之，如果前缀很少被复用，就应使用更小的缓存——甚至完全禁用前缀缓存。务必在生产环境中持续测量前缀命中率，并据此调优。

前缀缓存通常以 token 序列 trie（读作“try”）实现。trie 通常称为前缀树（prefix tree），是一种基于树的数据结构：每条边表示一个 token，每个节点编码从根节点到该节点的 token 序列。

在 token 序列 trie 中，每一条观测到的 prompt 前缀都作为一条独立的 token 路径存储，从而能够快速查询共享前缀。当新请求到达时，推理引擎从根节点开始逐 token 遍历 trie，直到下一个 token 无法继续匹配为止。匹配到的 token 序列恰好落在完成最长已缓存前缀的那个节点上——图 16-8 以示例系统 prompt “You are ChatGPT, a friendly assistant designed to help users……”展示了这一过程。

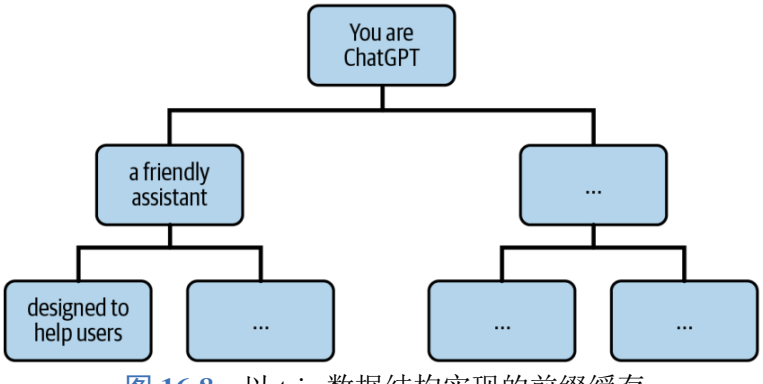

图 16-8 以 trie 数据结构实现的前缀缓存

此时，系统复用共享的 KV 缓存数据（克隆指针），并从序列中剩余的 token 处继续解码。由于该前缀的注意力得分此前已经计算过，这避免了为已缓存前缀重复进行自注意力计算。

SGLang 的 RadixAttention 使用建立在完整 token 序列之上的压缩 trie，即 radix 树。这种类型的前缀缓存把 token 序列折叠为树中的单条边以节省空间；树中的每个节点都指向一个 KV 缓存张量，该张量以连续 GPU 页的形式存储，承载该前缀的 KV 缓存。

RadixTree 数据结构空间开销小，并提供快速前缀查询、高效插入以及 LRU 式淘汰。下面是一段伪代码，展示在 token 生成过程中如何借助 radix 树进行 KV 缓存查询：

```
# Simplified RadixAttention KV cache example
# radix_attention_example.py
radix: RadixTree = RadixTree()  # holds edge labels + node.cache
↪ pointers
def generate_with_radix(prompt_tokens: List[int]):
    # 1) Find longest cached prefix
    node, prefix_len = radix.longest_prefix(prompt_tokens)
    # shallow-clone the KV cache for that prefix
    model_state = ModelState.from_cache(node.cache)  # refcount bump
    # 2) Process remaining prompt suffix
    for token in prompt_tokens[prefix_len:]:
        model_state = model.forward(token, state=model_state)
    # 3) As we go, insert or split edges in the radix tree
        matched = prompt_tokens[:prefix_len + 1]
        # insert returns the node for this full prefix
        node = radix.insert(matched, cache=model_state.kv_cache)
        prefix_len += 1
    # 4) Now generate new tokens autoregressively
    output_tokens = []
    while not model_state.is_finished():
        token, model_state = model.generate_next(model_state)
        output_tokens.append(token)
        # cache each generated prefix as well
        matched = prompt_tokens + output_tokens
        node = radix.insert(matched, cache=model_state.kv_cache)
    return output_tokens
```

生成开始时，引擎调用 `radix.longest_prefix(prompt_tokens)`，沿多 token 的边向下遍历 radix 树，直到到达与 prompt 最长已缓存前缀相匹配的最深节点。随后，引擎通过 `ModelState.from_cache(node.cache)` 对该节点的 KV 缓存页做一次轻量克隆，从而初始化 `model_state`，而无需为已缓存前缀重新执行任何自注意力计算。

接下来，引擎只处理 prompt 中尚未见过的新 token，逐 token 推进，并即时更新 radix 树：对每个新 token 调用 `radix.insert(...)`，根据需要拆分或新建树中的边，并把中间态 `model_state.kv_cache` 存储到每个新节点上。

整个 prompt 处理完毕后，循环切换到自回归解码阶段，用 `model.generate_next(model_state)` 生成新 token；同样地，每个生成前缀也会被插入 radix 树。这种做法最小化冗余计算，以节省空间的方式存储 KV 页，并提供快速的前缀查询——同时支持缓存的增量更新。

SGLang 的 KV 缓存设计能够自动捕获所有常见的复用模式，包括多轮对话、few-shot 示例和分支逻辑；同时，它确保共享前缀以较大的合并内存块获取，从而实现高效的 GPU 访问。

何时使缓存失效始终是一个难题。当缓存内存需要挪作他用时，你可能就得淘汰一些前缀。缓存系统支持不同的策略，例如“最近最少使用”（LRU）：最久未被使用的前缀最先被淘汰。例如，SGLang 的 RadixAttention 在 GPU 内存紧张时会惰性地淘汰 radix 树中最久未使用的叶子节点，如图 16-9 所示。

这是 SGLang 针对多个传入请求所使用的前缀缓存树结构，以及相应的 LRU 淘汰策略。示例取自 LMSys 的一篇非常出色的 SGLang 博客文章。

这里有两个聊天会话，以及跨越这两个会话的多次查询。每个节点的标签都是一段 token 序列（即子串）。绿色节点表示树中的新节点；蓝色节点是当前正被访问的已缓存节点；红色节点已被淘汰。各步骤的细节如下：

1. 初始时 radix 树为空。

2. 服务器处理传入的 prompt “Hello!”，并以 LLM 生成的“Hi!”作答。通过这一简单响应，许多 token 被合并为一条边加入树中，该边连接到一个新的绿色节点。具体而言，系统 prompt“You are a helpful assistant”、用户消息“Hello!”与 LLM 回复“Hi!”被整合到了一起。

3. 服务器收到一个新的 prompt。这是多轮对话的第一轮。服务器在树中成功查到该 prompt 的前缀并复用其 KV 缓存数据，新的轮次作为新的绿色节点加入树中。

4. 新的聊天会话开始，第 3 步中的节点 b 被拆分为两个独立节点，从而让两个聊天会话得以共享系统 prompt。

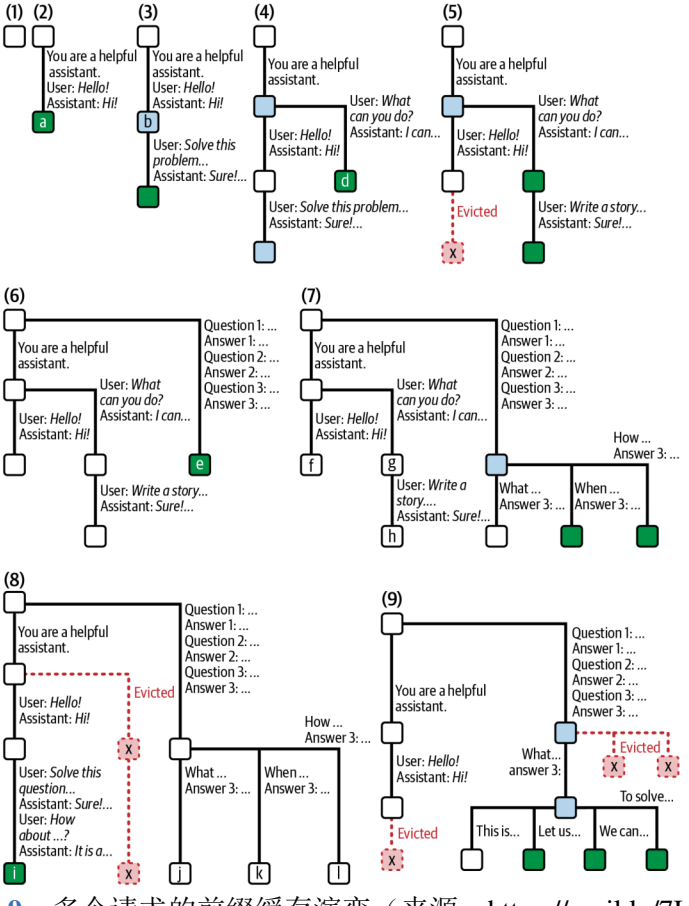

图 16-9　多个请求的前缀缓存演变（来源：https://oreil.ly/7LBoC）

5. 第 4 步中的第二个聊天会话继续进行。但由于内存有限，第 4 步中的节点 c 必须被淘汰，图示以红色标出。新的轮次被追加到第 4 步中的节点 d 之后。

6. 服务器收到一个新的 prompt（查询）。处理完成后，服务器将这个 prompt 插入树中；由于它与现有的 prompt/节点不共享任何前缀，因此需要拆分根节点。

7. 服务器收到一个包含更多 prompt（查询）的批处理。这些 prompt 与第 6 步的 prompt 共享前缀，因此系统拆分第 6 步中的节点 e 并共享该前缀。

8. 服务器收到来自第 3 步对话（第一个聊天会话）的新消息。此时，第 5 步中第二个聊天会话的所有节点（例如节点 g 和 h）被淘汰，因为它们是当时最近最少使用（LRU）的节点。

9. 服务器收到一条消息，要求对第 8 步中节点 j 的查询提供更多回答。受内存所限，系统必须淘汰第 8 步中的节点 i、k 和 l。

这个例子演示了前缀如何在多个请求之间共享，也展示了 LRU 缓存淘汰策略在前缀缓存场景下的工作方式。接下来，让我们转向级联模型部署模式，以便更充分地利用 GPU 资源。

16.5.4 模型级联与分层模型部署

并非所有查询都需要最大、最先进模型的算力（以及成本）。有些用户请求相当简单，用一个小得多、快得多、也便宜得多的模型就能回答。为查询选择合适的模型，称为模型级联（model cascading）或回退模型路由（fallback model routing）。

要实现模型级联，可以采用分层模型部署（tiered model deployment）：同时维护多个不同规模、不同能力的模型，并根据查询复杂度、所需精度或当前系统负载，为每个传入请求动态选择使用哪个模型。

设想一个问答（QA）推理部署：同时运行一个 7000 亿参数的最先进大模型，以及一个 700 亿参数的较小模型——后者所需的 GPU 资源更少，承载起来更快也更便宜。如果用户提出一个非常直白的事实性问题（由分类器或启发式规则判定），700 亿参数模型或许就能很好地处理。

对于直白的问题，可以把查询路由给较小的模型，它能在 50 ms 内响应，而不是 7000 亿参数模型的 500 ms。如果小模型的回答因置信度过低（由响应中返回的 logits 判定）而被认为不理想，系统可以把该问题转交给更大的模型。

这种两阶段方法可以降低整体平均延迟与计算用量——不过，若某个请求因为对小模型的初始回答缺乏信心而被先后路由给两个模型，它经历的延迟反而可能更长。业界普遍认为，ChatGPT 等许多商业 AI 服务正是采用这种方式，把较简单的查询路由给更小、更快的模型，以降低成本和延迟。

在实践中，把大约 60% 的查询路由给一个规模小 10 倍的模型，可以显著降低推理计算成本——如果设计和调优得当，整体成本甚至有望降低 5 倍。因为昂贵的模型只在真正需要时才被调用。

> **注** 维护多个模型和一套路由系统会带来工程开销：你不仅要监控其中一个模型的性能，还要监控第二个模型、两者之间的相互作用，以及路由系统本身。只有当你的流量规模和成本结构足以证明其划算时，才值得追求这一方案。

实现模型级联需要一个查询分类器或启发式机制来辅助模型路由决策。要把这个问题做对并不容易：该机制通常需要持续适应、覆盖全面的启发式规则，以及不断的路由器调优。因此，许多组织仍然依靠基于启发式和规则的路由器，并通过离线分析来更新决策标准。

一种做法是把它当作投机式问题来处理，类似于前面介绍的投机解码（speculative decoding）。可以先让较小的模型以草稿模式运行，生成答案及其置信度分数。如果置信度足够高，就直接返回结果；否则再调用更大的模型。

务必确保：即便需要动用大模型，小模型带来的额外延迟也不会让用户等待过久。实践中，当你怀疑问题较难时，可以让小模型和大模型并行运行，从而把小模型的延迟与大模型的处理过程重叠起来。这看起来浪费且反直觉，但对于某些延迟敏感的用例来说，它在以额外计算开销为代价的情况下有助于保持低延迟。

另一种做法是预先针对常见用户查询训练一个单独的分类器。例如，可以对其复杂度进行分类，提出“这个问题是否需要大模型广博的知识或推理能力？”这样的问题；如果不需要，就使用小模型。

需要注意的是，随着用户查询随时间演变，这个分类器本身也需要定期重新训练。此外，它引入了一个新的组件，可能出现退化与失效，因而需要加以监控。务必保证这个模型轻量、快速且稳健——否则它会成为推理系统中的新瓶颈。

初期常用简单的启发式规则。例如，如果用户查询是简短的事实性问题，如“今天天气如何？”或“某国总统是谁？”，可以直接使用小模型。而对于较长的提示（如超过 50 个 token），或包含 `explain`、`analyze`、`elaborate` 等字眼的更具创造性的查询，可以直接发给大 LLM，绕过小模型。

应当对每个请求记录日志并打上标签，标明由哪个模型处理（例如 `small` 还是 `large`），以及是否发生了回退。这样就可以按模型拆分统计延迟、准确率和回退次数。

这些打标数据还能用于监控调度器的决策。通过统计小模型的答案被拒的次数，可以识别出某种规律，进而触发对小模型的重新训练或对路由器启发式规则的改进。

反过来，如果小模型能很好地处理大量查询，就可以利用这一信息调高它的置信度阈值。这会提高小模型的使用率，降低响应延迟，并改善最终用户体验。

还可以按容量进行路由。如果大模型集群已达最大负载，与其让请求排队、拉高延迟，不如把不那么关键的请求暂时路由到小模型——它的效果可能不如大模型，但仍能快速给出某种答案。这是高负载下的一种优雅降级。用户可能会注意到质量有所下降，但这总比超时或过度等待要好。

许多 AI 服务会刻意这么做。例如，在峰值负载期间，免费档用户可能会被悄悄地路由到较小的模型，从而把大模型腾出来给高级付费用户使用。在资源受限的情况下，这是一种现实的权衡。

> **注** 我们将在第 17–19 章讨论更高级的动态与自适应推理服务器能力。这些特性高度依赖当前系统负载，包括可用的 GPU 显存、内存带宽利用率、KV 缓存利用率等。

另一种模型级联基于内容本身。例如，当用户提出的内容因安全原因被 LLM 拒答时，一些推理系统会回退到一个专门为处理不安全提示而微调的特殊模型，甚至回退到检索系统。

这更多是内容策略问题而非性能问题，但同样是模型路由逻辑中需要考虑的重要因素。而且回退模型往往小得多，因此这实际上还能带来性能收益：它可以快速处理不安全请求，并让主模型腾出资力去处理安全查询。

从部署角度看，多模型配置需要精心的伸缩规划，因为每个模型都需要足够多的实例来处理属于自己的那部分流量。有时某个模型可能成为瓶颈，例如当大量简单查询涌入时，小 LLM 已经饱和，而大模型却闲置着。

你可以实施自动扩缩容，在一天中的特定时段运行更多或更少的小模型实例，甚至利用容器编排按需把 GPU 从大模型池动态转移到小模型池。

16.5.5 流式响应

要提升推理系统的“感知”性能，应当在 token 生成的同时流式输出。这远比强迫最终用户等到完整回复全部生成完毕才能看到响应要好得多。这样一来，用户可以在信息到达时立即开始阅读和处理，使交互的体感速度快得多——即使总耗时相同。

人类的阅读速度约为每分钟 200–300 词，折合每秒约 4–7 个 token；快速阅读者可达每秒约 13 个 token。保持这样的流式输出速率最为理想。鉴于每一项性能决策归根结底都是权衡，这是在做出优化选择、规划容量等决策时需要考虑的重要指标。

> **注** 把系统的 token 吞吐速率与人类阅读速率对照监控非常重要。例如，如果发现系统由于模型延迟而只能以每秒 2 个 token 的速度流式输出，那就表明需要进一步优化。

大多数现代推理引擎都支持流式输出，包括 vLLM、SGLang 和 NVIDIA Dynamo。启用流式输出后，服务器会通过 WebSocket、服务器发送事件（Server-Sent Events，SSE）、HTTP 流式协议等方式将 token 刷新（flush）给客户端，并且应当在 token 一经生成就立即执行，以满足前文提到的每秒 4–13 个 token 的人类阅读指标。

模型实际上一次只生成一个 token——除非采用投机解码或多 token 解码之类的技术（分别如 EAGLE 和 Medusa）。因此，系统需要把生成的 token 分成每批 2–5 个 token，刷新流，再把这批 token 发回给最终用户。

关键在于，刷新之前不要让单个批次累积过多 token，否则流式输出就失去了意义。反过来说，每个数据包只发送一个 token 又可能因协议开销而低效。实践中，框架通常在遇到换行符或句末 token 时进行刷新。

举例来说，如果一个答案共有 100 个 token，需要 5 秒才能全部生成完毕，那么在流式模式下，当每批为 5 个 token 时，第一批 token 会在 0.25 秒后到达（5 个 token ÷ 每秒 20 个 token = 0.25 秒）。随后 token 批次会源源不断地输出，直到回复完成。这样，用户在仅仅 0.25 秒之后就可以开始阅读了。

如果没有流式输出，用户就得对着空白屏幕干等 5 秒，然后突然看到整个答案一次性全部出现，这并不理想，可能引发用户的狂乱点击（rage clicking）以及其他形式的挫败感。

从性能角度看，流式输出会带来轻微开销，因为它发送的是许多额外的小网络数据包，而不是一条大消息。但与模型的计算时间相比，与由此改善的最终用户体验相比，这一开销通常可以忽略不计。使用 HTTP/2 和持久连接有助于降低开销：token 可以作为连续流发送，而无需反复重建连接。

> **注** 要留意 Nagle 算法与延迟确认（delayed ACK）的相互作用。二者叠加可能会增加数十毫秒的延迟，在某些协议栈的最坏配置下甚至可达约 200 ms。应使用 `TCP_NODELAY`，并在条件允许时启用 quick-ack 或缩短延迟确认计时器，以尽量降低 token 刷新延迟。这些手段有助于减少 token 发送延迟，而这对实时流式输出至关重要。其代价是更多的小数据包占用链路，降低了带宽效率。在针对超低延迟进行调优时，请把这一点牢记在心。

妥善管理流量控制非常重要：例如，当客户端由于网络问题而消费流较慢时，不应阻塞模型的生成。理想情况下，即使客户端消费流的速度落后了，推理引擎也会继续生成整个响应。

系统应使用独立的线程和 CUDA 流把数据回传给最终用户。这样，发送响应过程中出现的问题就不会中断主 token 生成循环。

推理引擎会维护一个有界缓冲区来执行流量控制，防止内存无限增长——当客户端停滞或断开连接时就可能出现这种情况。务必处理好这类边界场景，因为它们在生产环境中相当常见。实践中，或许可以允许最多累积 50–100 个 token。

超过某个上限之后，推理引擎可以暂停生成，或者彻底关闭连接。这样一来，如果客户端在生成中途掉线，或者因连接过慢而跟不上，引擎就能停止产出 token，把资源释放出来去处理其他请求。

缓冲区上限的选择需要在内存占用与用户体验之间进行权衡。对较短的响应而言，这很少会成为问题；但对非常大的响应来说则较为常见——尤其是在客户端连接速度较慢的情况下。

改进流量控制的另一个技巧是 token 池化（token pooling）。如果模型生成 token 的速度快于流式输出所需的节奏，系统可以刻意加入延迟来平滑生成速

率。例如，若模型因为响应中的某个简单片段而在 0.5 秒内迸发出 20 个 token，一次性发送全部 token 就会先显示出一大块内容，然后停顿一段时间以等待下一部分。

你可能更希望应用界面呈现出稳定的打字机效果。在这种情况下，可以在各次 token 发送之间引入人为的流式延迟，例如 50 ms。这对实际延迟影响不大，而且通过避免 token 一阵一阵地突兀涌出，还能改善用户体验。

可以把 token 池化延迟做成可配置项，以便针对不同类型的用户（例如付费用户、免费用户等）适配不同的用户体验。付费用户可以获得更快的流式输出，

而免费用户的流式速度则会受到一定限制。这有助于在资源有限的情况下管理负载。

流式响应还允许终端用户在响应尚未完成时就开始评估中间结果并采取行动。例如，如果用户看到答案的前一部分，发现方向不对，就可以通过停止（Stop）或中断（Interrupt）按钮终止生成。

这既节省了不必要的算力，也改善了用户体验，因为终端用户不必等一个糟糕的补全部跑完再给出反馈。你应当监控——至少是测量——提前停止行为。

过多的显式停止可能表明模型的相关性存在问题。如果你发现许多用户总是在某个位置停止，这可能说明模型经常跑偏，或者超过该长度后过于啰嗦。这提示你可能需要额外的微调，让模型更加简洁，或者调整发送给终端用户的最大生成 token 数。

当然，这些提前停止也可能是出自频繁改变主意的“愤怒连点”型用户。无论哪种情况，最好都允许用户中断 token 生成过程。这样，推理集群就能回收资源去处理其他请求。

简言之，流式输出是响应迅速的 LLM 服务的必备能力。它并不提高原始吞吐量——甚至还会带来些许开销——但它提升的是系统的用户体感速度。实现流式响应时必须小心，不要干扰生成过程。应当用不同的线程和 CUDA 流来向终端用户回传响应，使主 token 生成循环不会因终端用户不稳定的网络连接而停顿。

> **注** 建议在测试环境中持续对比开启与关闭流式输出时的端到端延迟。务必确保 token 的发送间隔均匀，且没有引入锁竞争或 I/O 等待之类的明显

瓶颈。Locust 等对 Python 友好的工具可以模拟客户端，让你在大规模场景下测试低延迟的流式负载。

16.5.6 防抖与请求合并

许多生产系统还会实现两项 UX 特性：防抖（debouncing）与请求合并（request coalescing）。通过在响应前先做一次防抖（即暂停片刻），系统就能识别出用户是否在极短时间内连续发送了多个请求——无论是误操作还是愤怒连点，如图 16-10 所示。

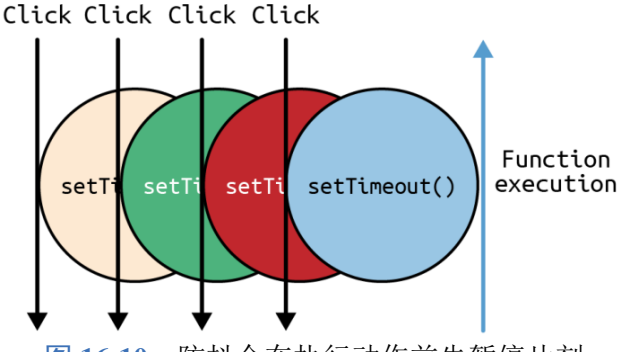

图 16-10　防抖会在执行动作前先暂停片刻

此时，系统既可以把多个查询合并为一个查询，也可以只保留最新的查询、丢弃其余请求。这类应用层的防护措施有助于减少后端承受到的过度的、重复的、徒劳的负载。

例如，如果用户双击 Submit 按钮，或者在 1 秒内发送了两个非常相似的查询，系统就可以把重复的请求丢弃。愤怒连点者在受挫时会不耐烦地反复重新提交——而这往往是应用响应太慢造成的。

讽刺的是，防抖和请求合并反而会带来更多延迟，让“愤怒”用户群体更加恼火，进而点得更起劲！这种情况下，如果 UI 能在请求进行时禁用输入会很有帮助，这可以从 UI 层面防止双击。但同时，也最好在服务器端配上相应的防护。

注　现代推理负载均衡器支持设置防抖间隔。请使用适度的间隔（如 2–5 ms），在批处理效率与额外延迟之间找到一个好的折中。另外，下次你在

ChatGPT 里提交内容时，留意一下那个防抖延迟。现在知道了这一点，是不是觉得有点烦了?!

16.5.7 Token 输出上限与超时

由于响应的 token 数直接影响延迟，你可以实现输出长度限制或超时。这类应用层约束可以防止失控生成——模型不断生成、超出合理的 token 数量。

Token 输出上限和超时有助于保持延迟的稳定，也能防止恶意或意外的提示词导致极长的生成，

从而长时间占用 GPU 资源。出于这个原因，许多公开 API 都设置了严格的输出限制。它既是一种滥用防范机制，也是一项性能保障。

同时务必设置服务器端超时。例如，如果一次生成超过 30 秒仍未完成，就返回部分结果或致歉提示。用户期望的是快速失败，而不是永远挂着不响应。

> **注** 同时也要监控你的模型是否倾向于喋喋不休地展开。施加深浅适度的 token 上限——例如聊天回答设为 4,096 个 token——反而可以提升质量，让模型保持正轨，避免冗长的回答。

重要的是根据用户需求来选择这些上限和超时。它们使延迟变得可预测，也限制了最坏情况下的算力消耗，这有助于容量规划等工作。

总而言之，将输入优化（如压缩与清洗）、缓存、智能路由以及 UX 优化（如流式输出）结合起来，可以削减工作负载、缩小推理集群规模、节省成本，并提升用户满意度。这些应用层策略与前面几节的底层优化互为补充，共同构成提升 LLM 服务性能的整体方案。

16.6 本章要点

本章介绍的技术表明，高效的 LLM 服务是一项整体工程，需要将现代 GPU 硬件、新颖的软件优化与全面的监控结合起来。性能分析工具用于定位瓶颈，系统化的调试技术用于解决问题。以下是本章的一些要点：

全面的性能分析

对整个推理栈进行端到端的性能分析。通过使用性能分析器测量每一阶段的延迟与资源占用，工程师可以精确定位变慢和低效之处。这种数据驱动的方法指导针对性优化，从而消除瓶颈。

监控与可观测性

为部署的推理服务建立可靠的监控。实时跟踪延迟分位数、吞吐量、GPU 利用率和内存使用等关键指标，以便及早发现性能回退或资源饱和。利用日志和追踪来洞察每个请求的处理过程，在大规模负载中定位热点或异常。

调试与迭代调优

采用系统化的调试工作流，你可以快速解决性能与正确性问题——即使横跨数万个节点。这样，当出现意外尖峰（例如吞吐量下降或延迟上升）时，你可以轻松地从高层症状下钻到底层问题。

用指标验证优化

GPU 调试器、内存泄漏检测器以及与性能相关的单元测试等工具，有助于验证量化和 kernel 融合等优化不会引入隐蔽的性能与正确性错误。对于每一次发布到生产环境的新优化，这种迭代式的“调优—测试”循环都是保持高性能与高可靠性的关键。

效率与成本优化

着眼于让推理更具成本效益的改进。任何提升吞吐量与利用率的优化，都会直接改善每查询成本。通过性能分析与打磨系统，团队可以用更少的 GPU 服务更多的请求，从而在基础设施成本和能效上带来显著节省。

16.7 本章小结

性能分析、调试与全栈系统调优，是大规模 LLM 推理保持高效、可靠且低成本运行的关键。随着模型规模向数万亿参数迈进，生产集群正朝着单次部署数十万乃至上百万 GPU 的方向扩展。在这种规模下，软硬件的持续协同设计依然不可或缺。仅仅依靠硬件来提升性能已经远远不够——尤其在功耗日益成为制约因素的今天。软件与硬件必须协同设计，并在技术栈的每一层持续联合调优。

经过优化的推理基础设施能够为最终用户提供快速的响应时间、可预测且稳定的系统行为，以及低廉的运营成本。唯有如此，你才能构建出足以服务世界上最大、最强大模型的推理系统。

在接下来的几章中，我们将深入剖析并优化现代 LLM 推理系统中计算、内存与网络开销最大的几个环节：计算（预填充）KV 缓存、在集群中的所有 worker 之间共享该缓存，以及利用它生成（解码）新 token。

第 17 章 面向推理的预填充与解码分离扩展

正如前面章节所述，LLM 推理可以分为两个截然不同的阶段：预填充（prefill）阶段和解码（decode）阶段。预填充阶段处理输入提示，为该提示生成模型的内部键值（KV）缓存；解码阶段则利用这些缓存的值，逐个（或在投机解码的情况下一次几个）生成输出 token。

这两个阶段具有根本不同的性能特征。预填充阶段是计算受限的，需要对可能多达数千个 token 并行进行大量矩阵乘法运算，消耗大量 FLOPS。相比之下，解码阶段是内存 I/O 受限的，每生成一个 token 都要读取庞大的 KV 缓存并写入新值，对内存带宽造成很大压力。简言之，预填充是高吞吐量的并行负载，而解码是串行的、对延迟敏感的负载。

早期的 LLM 服务系统把这两个阶段当作同一硬件上的一个整体流水线来处理，因此通常通过请求批处理来优先提升吞吐量，从而偏向于预填充阶段。然而，随着交互式应用的增长，首个 token 时间（TTFT，即处理全部输入 token 的预填充延迟）和单 token 生成时间（TPOT，即每个 token 的解码延迟）等延迟指标变得与原始吞吐量同样重要。当同时服务这两个阶段时，单一 GPU 推理引擎很难同时优化 TTFT 和 TPOT。

对大量请求进行批处理可以提高吞吐量，却会恶化 TTFT，因为每个请求都要等待最慢的那个预填充。批处理还会影响 TPOT，因为解码步骤会被新提示的预填充工作积压。

整体式推理系统只能在两者之间取舍：要么以牺牲后续 token 生成速度为代价来改善（缩短）首个 token 时间，要么在提升（提高）单 token 吞吐量的同时让新请求承受较长的初始延迟。在极端情况下，一个很长的提示可能完全占住 GPU，从而阻塞其他用户的所有提示预填充工作；而一旦开始解码，一次生成一个 token 的处理方式又会让 GPU 核心在每次 token 生成之间处于空闲状态。

为解决这些问题，研究人员和工程师开始寻找将两个阶段解耦的方法。关键洞察在于：预填充和解码其实不必运行在相同的硬件上——甚至不必运行在相同类型的硬件上。

预填充与解码分离是指把两个阶段分配给各自针对该阶段需求而专门优化的不同资源。这一思想由论述 DistServe 的论文中的系统率先提出，该工作表明，通过消除两个阶段之间的干扰，可以同时满足对 TTFT 与 TPOT 的严格延迟要求。

DistServe 的评估显示，在严格的延迟服务级目标（SLO）下，与不做预填充/解码分离的最先进基线相比，它可以多服务 7.4 倍的请求。因此，业界框架开始尝试独立的预填充服务器和解码服务器。

开源 vLLM 库在 LMCache 等组件的配合下引入了分离式运行。NVIDIA 的 Dynamo 通过动态路由与自动扩缩实现了分离式预填充与解码，并公开记录了其运维细节。许多服务商与开源框架都在实现或评估分离架构。例如，据报道，为了满足严格的延迟 SLO，OpenAI、Meta 和 xAI 的工业级服务系统都采用了这种分离方案。因而，预填充与解码分离已成为大规模 LLM 推理的标准做法。

在超大规模下，大型推理部署可能涉及数十万乃至数百万块 GPU，服务数十亿请求。在这样的环境中，分离带来的成本与性能收益是巨大的。

通过拆分负载，你可以对每个阶段进行独立优化，避免其中一个阶段成为另一个阶段的瓶颈。本章余下部分将探讨如何在极端规模下设计并运营一个分离式预填充/解码推理系统。

在本章中，我们将探讨在预填充与解码 worker 之间调度请求的路由算法、在高负载下维持服务质量（QoS）的技术，以及使这种分离高效运作的机制。我们将涉及从高速互连到专用解码 kernel 的方方面面，还将讨论为每个阶段使用不同 GPU 类型的异构硬件策略。

17.1 为什么要进行预填充-解码分离？

现代交互式 LLM 服务通常要求 p99（99% 的请求）的 TTFT 延迟 < 200–300 ms。如果不把预填充工作分离出去，这一目标几乎无法保证，因为“一刀切”的 LLM 服务方式会浪费大量本可获得的性能。

作为参考，MLPerf v5.0（2025 年）针对 Llama2 70B（700 亿参数）的推理基准设定了 p99（第 99 百分位）SLO：TTFT 约为 450 ms，TPOT 延迟为 40 ms。对 Llama 3.1 405B（4050 亿参数），该基准的目标为 TTFT 约 6 秒、TPOT 175

ms。具体而言，这些 SLO 分别对应 Llama 2 Chat 70B 和 Llama 3.1 405B Instruct 的 p99 TTFT 与 p99 TPOT 目标。

考虑这样一个场景：一位用户的请求带有长达数千 token 量级的超长提示，而另一位用户的请求提示非常短。在不做预填充与解码分离的情况下，如果这些请求几乎同时到达，长提示的预填充计算将让 GPU 被长时间占用。

没有分离时，第二个携带短提示的请求必须先等上不必要长的时间，才能开始自己的解码。这就是干扰（interference）：一个请求的预填充工作拖延了另一个请求的解码。连续批处理下的预填充与解码干扰如图 17-1 所示。

在简单的 FIFO 调度策略下，长提示会放大所有人的尾延迟。一般而言，排在队首的长而计算量大的预填充会阻塞其后较短、较轻的请求。这称为队头阻塞，它会导致利用率低下、延迟离群以及终端用户的不满。

在灵活的分离式架构中，可以把大型提示的预填充发送到专用的计算优化型预填充 worker 池，而轻量提示的预填充则可以直接发给解码 worker——绕过预填充 worker。这种灵活性让较短的请求免受队头阻塞的影响，从而最大化整体吞吐量并将延迟尾部效应降至最低。

17.1.1 分离的优势

分离有两大主要优势：减少干扰和面向各阶段的优化。下面分别讨论。

17.1.2 减少干扰

采用分离后，预填充任务不再与解码任务在同一设备上争用。一个正在生成大量 token 的繁忙解码 worker，也不会妨碍处理另一个用户的提示，反之亦然。

为每个阶段配备专用资源，意味着一个长提示的计算不会阻塞另一个用户的 token 生成。在实践中，这会带来更可预测的延迟。图 17-2 展示了共置与分离式预填充和解码的对比。DistServe 论文以及作者随后发表的博文对该实验作了更详细的描述。

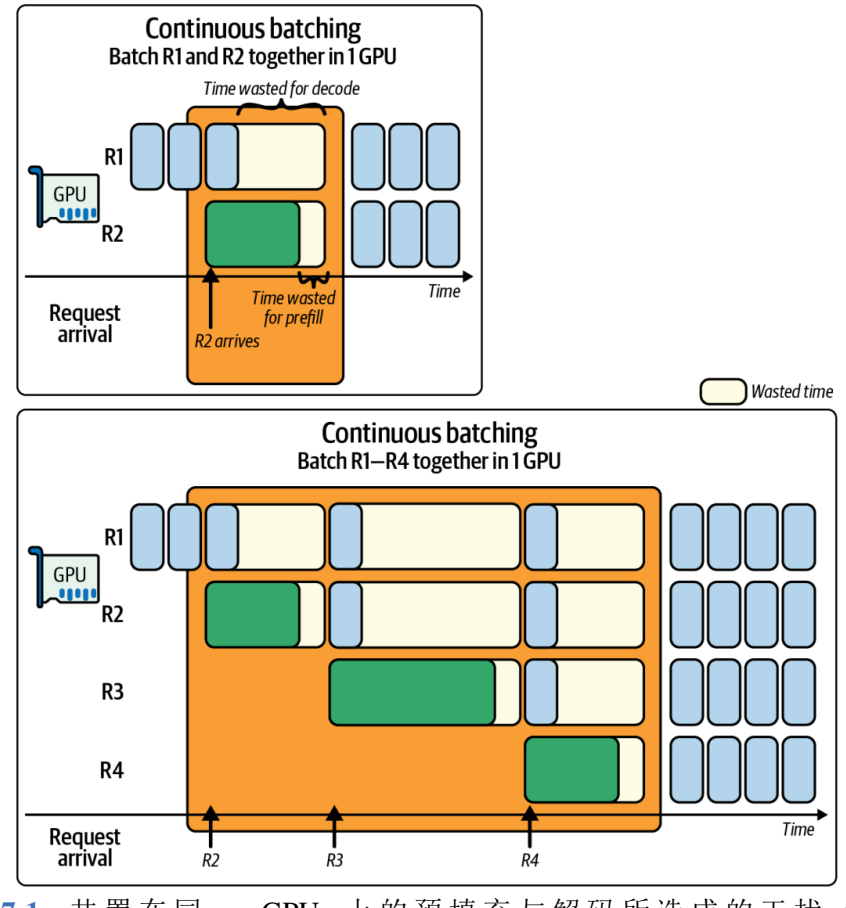

图 17-1 共置在同一 GPU 上的预填充与解码所造成的干扰（来源：https://oreil.ly/GRkHs）

这里，SLO 设为 P90 TTFT 0.4 秒、P90 TPOT 0.04 秒（例如图 17-2 中的水平线）。在给定的 TTFT 延迟界限内，共置系统只能支持约每秒 3 个请求（RPS）的有效吞吐量；在给定的 TPOT 延迟界限内，它只能维持 1.6 RPS。因此，共置配置的有效吞吐量只有 1.6 RPS，因为 TTFT 与 TPOT 两项延迟 SLO 都必须满足。

将两个阶段分离，并为其分配两个预填充 worker（两块 GPU）和一个解码 worker（一块 GPU），即所谓的 2P1D 配置。此时，预填充 worker 与解码 worker 的整体 RPS 都优于使用单块 GPU 的共置配置。具体来说，预填充 worker 达到约 5.6 RPS，而解码 worker

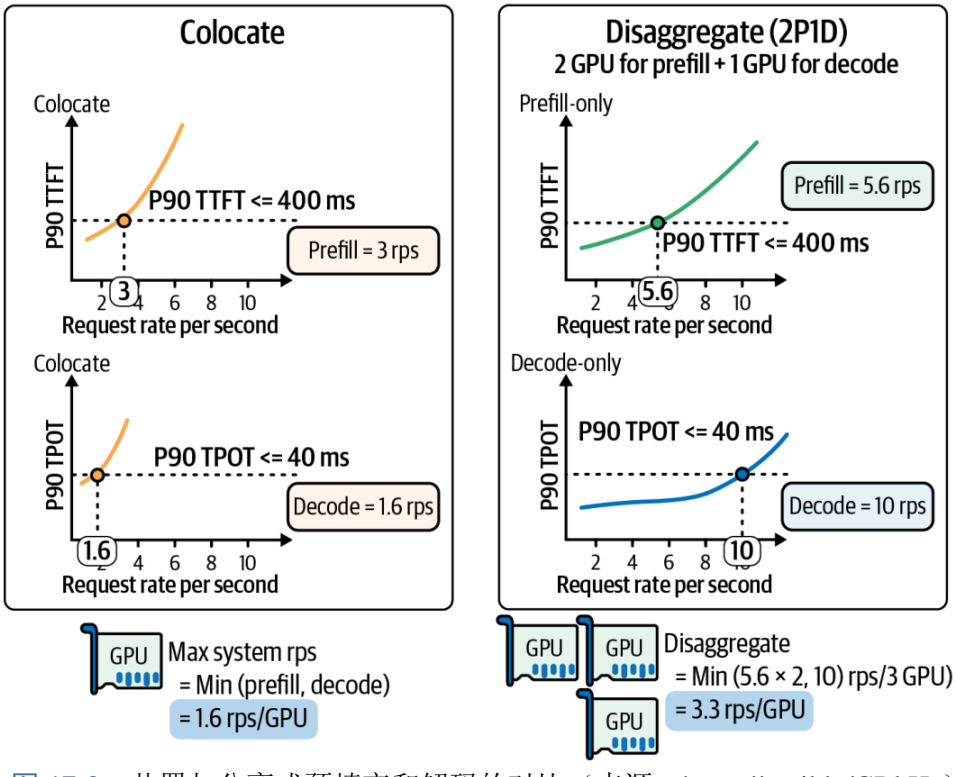

图 17-2　共置与分离式预填充和解码的对比（来源：https://oreil.ly/GRkHs）

在三块 GPU 上合计达到约 10 RPS。因此，2P1D 配置的有效吞吐量为每块 GPU 3.3 RPS。

每块 GPU 3.3 RPS 的计算方法是：取预填充 worker 的 RPS（5.6 RPS × 2 = 11.2 RPS）与解码 worker 的 RPS（10 RPS）二者的较小值，即三块 GPU 上共 10 RPS。于是需要把 RPS 除以 GPU 数量，这里是 3。在所配置的 SLO 下，该系统的有效吞吐量为 10 RPS ÷ 3 块 GPU = 每块 GPU 3.3 RPS。

> 注　在这一对比中，解码侧的改进主要影响单 token 延迟，而预填充隔离则主要改善首个 token 时间。两项 SLO 都必须满足，才能计入有效吞吐量。

这种隔离也能改善尾延迟。经验表明，采用分离架构的系统展现出更紧凑的延迟分布，并避免了整体式系统中常见的长尾。通过消除跨阶段干扰，每个阶段都能更可靠地满足其 SLO，并且具有更可预测的一致性。

现在，你也应该会问："3 倍的成本换来 2 倍的改进，值得吗？"你有理由这样问。要让这一方案更具成本效益，还需要额外的调优，但它指明了收紧延迟分布、提升有效吞吐量的正确方向。你需要根据自己的负载来做出决定。分离是满足有效吞吐量 RPS 需求的常用选择。

17.1.3 面向各阶段的针对性优化

面向各阶段的针对性优化让每个阶段都能使用最适合自己的硬件与并行策略。例如，预填充阶段是计算受限的，因此通常应提高张量并行度，以在高 FLOPS GPU 上逼近峰值 FLOPS。此外，现代 GPU 还提供更低的精度模式（FP8 与 FP4），可以提升计算密集的预填充阶段的吞吐量。

> **注** 应优先对权重使用 FP4，并在经过验证的情况下对激活值也同样使用。许多技术栈对权重采用 FP4、对激活值采用 FP8。这些降低的精度有助于最大化吞吐量并最小化 HBM 占用，而精度损失极小。现代软硬件栈均支持这些精度，包括 NVIDIA Tensor Core 与 Transformer Engine。

相比之下，解码阶段受内存带宽限制，并承受跨 GPU 同步开销。因此，它在几乎没有或完全没有张量并行（通常 `TP=1`）时效率最高：它更多依赖融合 kernel 来提高算术强度，并依靠高内存吞吐的 GPU。

在单体系统中，你不得不为两个阶段选择同一类 GPU 和同一种并行策略，这对至少其中一个阶段而言是次优的。而解耦则让你可以独立调优每个阶段，实现最大效率。

拆分阶段还为异构集群打开了大门：可以把不同类型的 GPU 分别分配给预填充和解码角色，以获得最优性价比。例如，用计算优化型 GPU 做 prompt 预填充、用内存优化型 GPU 做 token 生成，往往能比同构部署获得更高的每美元吞吐量。

> **注** 实践中，最新的 GPU 通常同时具备更高的 FLOPS 和更大的显存。因此，直接用最新一代 GPU 同时承担预填充和解码是很自然的做法，也更常见。但要知道，异构部署是一种可行的降本选择。

我们将在本章稍后探讨异构集群这一思路，展示用高端 GPU 处理 prompt、用更廉价的 GPU 做生成，如何在大规模场景下带来显著的成本节约。

总之，解耦消除了跨阶段的相互干扰，使每个阶段都能得到专门化的处理。预期收益包括：不再有由 prompt 长度不匹配造成的长尾，延迟分布更加集中；延迟约束下的吞吐量（有效吞吐量）得到提升；整体资源利用率也更高。

> **注** 应使用性能分析工具（例如 NVIDIA Nsight Systems）来定位预填充和解码阶段的瓶颈。这类工具可以跨各个 worker 节点追踪 GPU kernel 与 RDMA 传输，帮助你验证解码 kernel 是否与通信完全重叠等。

接下来，我们讨论如何实际实现一个解耦式服务系统，包括系统架构、通信方式，以及决定如何充分利用解耦集群的调度策略。

17.1.4 解耦的预填充与解码集群池

在解耦部署中，我们维护两个（或更多）worker 池：一组 GPU 专门用于预填充 prompt 处理，另一组专门用于 token 生成。这些 worker 可以位于同一数据中心的不同节点或机架上——只要互连足够快，甚至可以位于不同的数据中心。（注意：为了达到现实的 SLO，实际的设计选择通常是让预填充和解码留在同一个数据中心内。）

worker 池之间通过网络通信，将预填充阶段产生的模型 KV 缓存移交给最终执行解码的 GPU。这一通信由调度器（或路由器）来协调。

考虑这样一个配置：模型权重被装载到两组 GPU 服务器上。一组是预填充 worker，负责处理 prompt 并计算 KV 缓存；另一组是解码 worker，利用预填充 worker 生成的 KV 缓存进行 token 生成。

两组 worker 通常通过高速互连（如 NVLink/NVSwitch 与 InfiniBand）以及基于 RDMA 的零拷贝 GPU-to-GPU 传输进行通信。在实践中，这些传输使用 GPUDirect RDMA 或 UCX，并可以在 Nsight Systems 中与 CUDA kernel、NVLink 活动、存储指标以及 InfiniBand 交换机指标一同关联分析，实现端到端验证。

> **注** 对于基于超芯片的 NVL 互连结构（如 Grace-Blackwell、Vera-Rubin 等），应使用 NVIDIA Multi-Node NVLink（MNNVL），为 TP 解码保持 NVLink

优先的集合通信，并在可用时为 AllGather 与 ReduceScatter 集合通信启用 SHARP。

当系统收到一个新请求时，通常由解码 worker 接收请求，这称为解码中心（decode-centric）设计。之所以倾向于此，是因为预填充 worker 已因 KV 计算而处于计算受限、接近饱和的状态。

让解码 worker 负责客户端 I/O、路由与会话状态管理，推理系统就可以避免把预填充 worker 压垮。此外，把请求入口集中在解码节点上，也简化了网络管理、自动伸缩和策略执行。

注 这只是预填充—解码系统架构的一种风格：解码 worker 作为所有请求的集中入口。NVIDIA Dynamo 推理系统采用的便是这种架构。另一种常见架构是使用一个专用的集中式 API 路由器，把请求路由到预填充或解码 worker。不过，这会在系统中多引入一个活动部件，需要在路由器与预填充/解码 worker 之间进行额外的协调，同时带来额外的可伸缩性与延迟方面的考量。

解码 worker 收到请求后，会决定是自己执行预填充，还是把它“卸载”给预填充 worker 池。如果决定卸载，解码 worker 之后会收回 KV 结果，然后继续解码并生成下一个 token。

下面是精简版 NVIDIA Dynamo 集群配置中的一个片段，它定义了两个角色，使用 NVIDIA 的 Inference Xfer Library（NIXL）经由 GPUDirect RDMA 传输 KV 缓存，使一个 GPU 可以通过网络直接写入另一个 GPU 的内存：

```
roles:
  - name: prefill_worker     # Prefill worker role
    model_path: models/llm-70b
    instance_count: 4        # 4 prefill workers
    gpu_type: B200           # B200 Blackwell compute-bound prefill
  - name: decode_worker      # Decode worker role
    model_path: models/llm-70b
    instance_count: 8        # 8 decode workers
    gpu_type: B300           # B300 Blackwell Ultra high-memory decode
```

> **注** NVIDIA Rubin CPX 加速器是预填充 worker 的另一种选择。Rubin CPX（CP 代表“上下文处理”）专为预填充这类计算受限的工作负载而设计。Rubin CPX 标志着 NVIDIA 从“通用加速计算”GPU 迈向专用芯片：这些芯片针对更广泛的 AI 工作负载（如推理）中的特定阶段（如预填充）做了专门优化。

在这一配置中，我们有 4 个使用 B200 GPU 的预填充 worker（对计算密集的预填充而言算力充足）和 8 个使用 B300 GPU 的解码 worker（HBM 容量较大，可应对重内存受限的解码）。混用 B200 与 B300 有助于匹配两者的 FLOPS 与 HBM 容量特性，同时把成本降到最低。两个角色都将使用 NIXL 与 GPUDirect RDMA 来传输 KV 缓存块。NIXL 对 GPU-to-GPU 数据搬运的传输方式做了抽象，可经由 NVLink 和 RDMA 网卡完成；它还提供 GPUDirect Storage 连接器，使 KV 缓存页能够从不同存储层读取（或写入）。

在底层，系统运行时每个解码 worker 会注册其 GPU 内存中的一块区域，使预填充 worker 可以通过 RDMA 直接写入其中。通常，内存注册元数据（例如 NIXL 描述符）在启动时或首次接触时交换。这样，对每个远端预填充任务而言，只需发送一个小的标识符，而不必发送完整的内存地址结构。

例如，Dynamo 使用 etcd 进行 worker 发现与租约管理。worker 将所需的内存句柄注册到路由器或控制平面，供对等方在需要时获取描述符；预填充 worker 会在首次使用时取回它们。这样一来，预填充请求中只需包含目标 KV 缓冲区的 ID，控制消息就非常轻量。

此外，NVIDIA Dynamo 的 NIXL 实现为推理数据的搬运提供了高吞吐的 RDMA 与存储抽象，并包含针对 NVLink、基于 UCX 的网络和 GPUDirect Storage 的插件。因此，预填充 worker 可以把 KV 块直接写入解码 GPU 的内存。

> **注** 在预填充与解码使用不同 TP 布局的混合并行部署中，你需要在 NIXL 读取之后立即在解码侧执行一次布局变换，使 KV 页与解码 kernel 期望的布局一致。与网络传输相比，这一变换的延迟可以忽略不计，还能避免重新预填充。

这种架构把每个阶段的扩缩容解耦了。例如，如果由于大量并发的长 prompt，预填充成为吞吐量瓶颈，你就可以增加预填充 worker 来提升 prompt 处理能力。

同样，如果由于许多用户生成长输出，解码成为瓶颈，你应该横向扩展解码 worker。由于解码与预填充是彼此分离的，扩展其中一方不会直接干扰另一方。

像 NVIDIA Dynamo 这类系统支持动态的、运行时可配置的解耦，可以在线增删预填充 worker，而无需停掉集群。新的预填充 worker 只需注册即可开始从队列中拉取任务。如果某个预填充 worker 因故退出集群（崩溃、重启、自动伸缩事件、网络分区等），解码 worker 会临时多做本地预填充来加以补偿。

NVIDIA Dynamo 的分布式运行时使用 etcd 做 worker 发现与租约管理。其 Planner 组件可以通过撤销租约或启动新 worker（后者会被自动发现）来缩减或扩容 worker。这种动态灵活性在超大规模场景下至关重要，因为负载往往会大幅波动；一旦出现波动，你就需要在不同角色之间按需调换 worker。

17.1.5 预填充 worker 的设计

预填充 worker（prefill workers），或称提示词服务器（prompt servers），是专门执行请求初始提示词预填充（prefill）处理阶段的计算节点。本节讨论预填充节点在架构上如何高效处理这一繁重的计算——以及在高负载下为生成 KV 缓存时如何在延迟与吞吐量之间取得平衡。

由于预填充工作负载是计算密集型的，预填充节点应使用具有高 FLOPS 的 GPU，并针对大型矩阵乘法进行优化。每个预填充任务会把 n 个输入 token 送入模型的全部层。

预填充 worker 会并行使用数千个 GPU 线程——如果集群中有足够多的 GPU 节点，还会跨这些节点并行。它们采用张量并行、流水线并行等人们熟悉的并行技术来降低 TTFT。

内存管理。就内存而言，预填充节点既需要加载完整的模型权重，也需要为提示词分配 KV 缓存。随后，这些 KV 缓存会被传输给解码 worker，我们稍后会看到这一点。

预填充会把模型参数以及模型前向传播处理提示词输入时的工作激活值装填到 GPU 显存中。KV 缓存一经生成，就会被立即发送给解码 worker，因此它在预填充节点的内存中不会驻留太久。

如果模型极其庞大或提示词极长，受内存限制，预填充可能需要跨 GPU 进行张量切分或其他形式的并行切分。预填充服务器应能灵活地运用各种并行策略（数据并行、张量并行、流水线并行、专家（MoE）并行与上下文并行），以满足延迟目标。

一些推理框架会预先为预填充分配一大块 GPU 显存作为工作空间。这样做可以减少整体内存碎片，并缩短缓冲区分配时间。

延迟与吞吐量之间的权衡。在调优一个分离式（disaggregated）预填充集群时，你面临一个根本性的权衡：是尽量缩短每个单独提示词的 TTFT，还是在高负载下最大化整体每秒请求数（RPS，即吞吐量），或者降低 TPOT。

分离式系统通过为延迟优先与吞吐优先两种路线分别支持不同的调度策略来应对这一权衡。下面我们逐一介绍这两种策略：

延迟优先策略

为了降低 TTFT，预填充节点应当在提示词一到达就立即处理——并且几乎不进行批处理。在这种模式下，你无需等待其他请求来凑满一个批次。因此，假设集群中有可用的 GPU，每个提示词都会立即开始执行并尽快完成。

延迟优先策略的缺点是 GPU 利用率较低，因为你使用的是很小的批次甚至不使用批次。其结果是 GPU 经常处于空闲状态，在给定集群规模下，系统能够服务的并发请求数也更少。在这种情况下，你可以为预填充集群过度配置容量，也可以使用 1 这样极小的批次规模（batch size），以保证请求满足严格的延迟 SLO。

吞吐优先策略

如果你的首要目标是峰值吞吐量（或 RPS）和尽可能小的 TPOT，就应当把提示词编入较大的批次，让每块 GPU 都满载运行。通过把 8–32 个

提示词合并到一个批次中，你可以提高算术强度，让 GPU 计算单元保持忙碌，从而提升整体吞吐量。

吞吐优先策略的缺点是每个请求都要承担一个等于收集该批次所需时间的批处理延迟。批次规模越大，延迟越长。

对于追求极致吞吐量的推理系统配置，你可以选择为每个请求分配多块 GPU，采用数据并行或流水线并行来实现。

采用数据并行时，整个模型被复制到每块 GPU 上。批次被拆分成多个小批次（minibatch），分发到各 GPU；每个 GPU 用自己那份完整的模型副本对其数据子集执行一次前向传播，然后再汇总所有 GPU 的输出，得到最终结果。

数据并行将所有 GPU 的内存带宽和算力汇聚起来，以提升每个批次的性能。但它会把最大并行度降低为 GPU 总数 ÷ 每个请求占用的 GPU 数，从而削减整体并发请求容量。如果系统为单个请求分配了过多的 GPU，就会造成资源闲置，导致吞吐量与并发能力之间出现失衡。

流水线并行把模型的各层划分为顺序执行的多个阶段，放置在不同的 GPU 上，例如 GPU 0 和 GPU 1。GPU 0 一完成微批次 0 的阶段，就把激活值转发给 GPU 1，并立即开始处理微批次 1 的阶段 1。这种流水线式的模式让所有 GPU 都在不同工作块上保持忙碌。

流水线并行能提升单个批次的吞吐量，但如果微批次规模或阶段划分没有经过仔细平衡，就会引入 GPU 间通信开销和流水线“气泡”（bubble）。

归根结底，在集群规模固定的前提下，你每多分配一块 GPU，吞吐量就会提高，但能同时处理的请求数就会减少。你固然可以横向扩展 GPU 集群，但假定集群规模固定，你应当根据延迟 SLO 和吞吐量 SLO 哪一个对你的用例更重要来选择配置。

感知延迟的调度与批处理。分离式系统会采用前文提到的感知延迟的调度策略来平衡这些因素。例如，它们可以保证单请求执行——不把请求合并成批——除非负载高到即使把少量请求合批也不会违反 TTFT 目标。

许多集群设计会在调度器中纳入 SLO 约束。例如，如果 p90 TTFT 必须小于等于 X ms，系统就会针对典型的提示词规模，选择在仍能满足该 SLO 的前提下最大的批次规模或并行策略。

另一种策略是自适应批处理窗口（adaptive batching windows）。例如，低负载时，系统可以立即以批次规模 1 执行请求；高负载时，系统可以允许把在一个小时间窗口（如 2–10 ms）内到达的请求组成微批次。这样，一点点延迟就能换来 GPU 利用率的显著提升——但只在确实需要且可以容忍时才这样做。

许多推理引擎的预填充 worker 更偏向于延迟优先。系统往往尽快执行提示词任务，甚至可以容忍一定程度的 GPU 利用不足，因为第一个 token 来得快，能显著改善用户体验。

常见的做法是按平均负载配置超出所需的预填充容量。这样，预填充集群就能在不出延迟尖峰的情况下吸收提示词的突发流量。下一章我们将讨论动态重新平衡资源的自适应机制，使预填充与解码 worker 都不会随时间推移变成瓶颈。

像 Kubernetes 这样的现代编排器可以自动扩展每一层。例如，如果预填充 GPU 利用率持续居高而解码侧偏低，编排器就会触发自动扩缩，增加预填充 pod（或节点）——甚至可能删除部分解码 pod/节点。

> **注** 这类自适应扩缩常常借助预填充队列长度等指标来辅助决策。

另一种方案是实现优先级队列，让短提示词被调度到批处理较少的“快车道”上，可批量处理的长提示词则进入吞吐优化队列。NVIDIA Dynamo 在调度中支持延迟等级（latency classes）。你可以通过给请求打标签、并为不同等级设置不同的批处理窗口来模拟这一机制。

关键要点是：预填充 worker 优先保证快速周转。分离式让我们能够做到这一点而不损害解码性能，因为解码是在另一组 worker 上运行的。我们可能在低流量时段“浪费”一些预填充 GPU 算力周期，但在峰值流量时段保持了低 TTFT。对于交互式服务来说，这是值得的取舍。

17.1.6 解码 worker 的设计

解码 worker（decode workers），或称生成服务器（generation servers），专门负责自回归解码阶段。一旦某个提示词的 KV 缓存就绪，它就会被发送给一个解码 worker，后者利用该 KV 缓存尽快生成其余的输出 token，以保持较低的 TPOT 延迟。

如果请求最初被路由到解码 worker，如图 17-3 所示，那么它必须首先通过分离式路由器决定预填充应当在本地进行还是远程进行。如果决定远程预填充，它就会把预填充请求推入预填充队列，等待预填充 worker 领取。

预填充 worker 持续从预填充队列中拉取请求，读取前缀缓存中已缓存的 KV 块，并执行预填充计算，然后将 KV 块写回解码 worker，由后者完成整个解码。

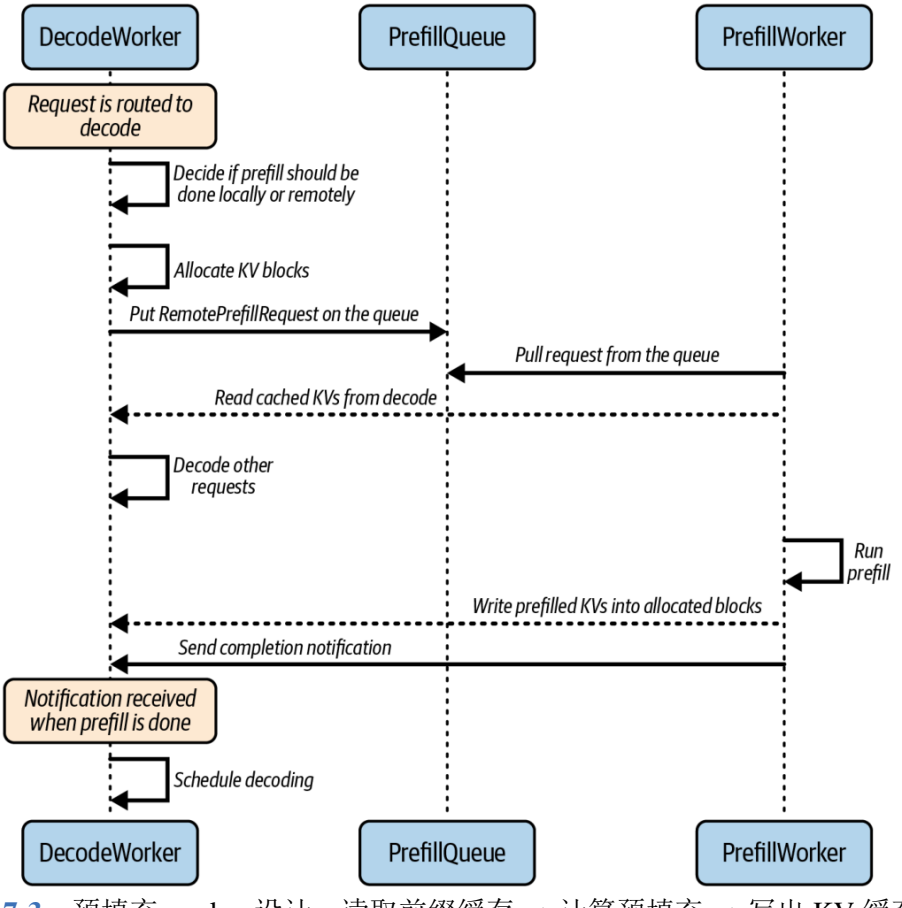

图 17-3　预填充 worker 设计：读取前缀缓存 → 计算预填充 → 写出 KV 缓存供解码使用

解码 worker 的设计重点在于高效处理大量并发的序列生成，同时管理好 KV 缓存的内存占用。本节介绍解码服务器如何借助 continuous batching 等技术和巧妙的内存管理手段实现高吞吐量。

这些手段有助于降低 TPOT 延迟并提升可扩展性——对长序列尤其如此。下面首先详述两个阶段之间的 KV 缓存传输。

预填充与解码之间的 KV 缓存传输。高性能的解耦部署要求尽可能高效地在预填充 worker 与解码 worker 之间搬运 KV 缓存数据。利用 NIXL（见第 4 章）等库进行直接的 GPU 到 GPU 传输，可以让 CPU 免于参与，并使用非阻塞操作。

这样，GPU 在传输 KV 数据的同时仍可服务其他前向计算请求，而不必等待传输完成。

考虑一个到达解码 worker 的用户请求。此时，解码 worker 的调度器会分配所需的 KV 块，并将一个远程预填充请求加入预填充队列。该预填充请求中包含这些 KV 块的标识符。这一交互过程如图 17-4 所示。

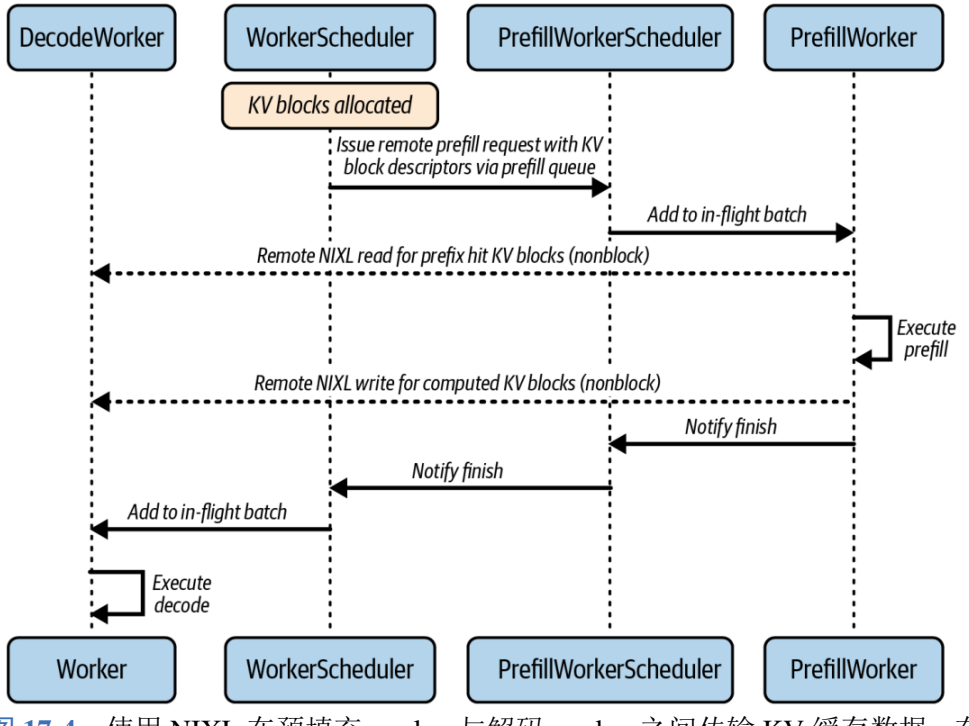

图 17-4　使用 NIXL 在预填充 worker 与解码 worker 之间传输 KV 缓存数据；在 RDMA 之前将多个 PagedAttention 块合并为约 128 token 的负载（注：在 CUDA 上 vLLM 默认每块 16 个 token）

预填充 worker 利用 NIXL 在所选传输介质上执行直接的远程 GPU 内存读写，从而避免 CPU 拷贝并实现非阻塞推进。一旦预填充 worker 完成该预填充请求，

解码 worker 的调度器就会将相应的解码请求加入自身的解码流水线，使计算与数据搬运无缝重叠。请务必使用预先注册的对端内存，并配置较大的 pinned 窗口尺寸，以尽量减少重复注册带来的开销。你可以用 Nsight Systems 时间线来验证零拷贝传输的重叠情况。

在验证端到端的数据搬运与重叠行为时，建议使用跟踪标志（trace flags）配合 Nsight Systems 进行性能分析。对于 InfiniBand 链路遥测，可添加 `-{}-nic-metrics=true` 以获取 HCA/NIC 计数器，并添加 `-{}-ib-switch-metrics-device=<GUIDs>` 以获取交换机计数器。二者会捕获交换机指标并采样主机/设备活动，最终生成相关联的 CUDA kernel、UCX 活动、存储指标与网络行为记录。下面这条统一命令开启 CUDA/UCX 跟踪，并收集 CPU 活动、GPU 指标、存储指标以及 InfiniBand 交换机遥测数据：

```
nsys profile --trace=cuda-hw,osrt,nvtx,ucx,gds \
  --trace-fork-before-exec=true \
  --cuda-event-trace=true \
  --cuda-graph-trace=node \
  --cuda-memory-usage=true \
  --sample=cpu \
  --gpu-metrics-device=all \
  --nic-metrics=true \
  --ib-switch-metrics-device=<GUIDs> \
  --storage-metrics --storage-devices=all \
  --gds-metrics=driver \
  -o nsys_reports/prefill_decode \
  <your_launch_here>
```

> **注** 预填充与解码引擎可能采用不同的并行布局（例如张量并行布局）。此时，系统可在接收端插入一个布局变换 kernel——在 NIXL 读取之后（且在数据被使用之前）执行，将每个 KV 块重新对齐为解码 worker 所期望的布局。

continuous batching。解码服务器高度依赖 continuous batching，也称为迭代级批处理（iteration-level batching）。与执行大规模矩阵-矩阵计算的预填充阶段不同，解码阶段执行的是许多小规模计算：由于每个 token 以向量的形式表示，每生成一个新 token 只是一次相对较小的向量-矩阵计算。

为避免逐 token 的小规模工作负载导致 GPU 利用率过低，解码 worker 可以将多个输入序列合入一批，从而在每次迭代中形成更大的矩阵-矩阵计算（例如同时生成多个 token）。这提高了每个解码任务的算术强度。

举例来说，假设有 32 个不同的文本生成请求正处于生成过程中，随时准备生成各自的下一个 token。此时不再分别执行 32 次仅生成单 token 的前向传播，而是由 continuous batching 调度器将这些请求合并，执行一次并行生成 32 个 token（每条序列一个）的前向传播。

这样，矩阵乘法看到的有效批大小为 32，GPU 计算单元因此保持繁忙。但难点在于，各条序列并非恰好在同一时刻请求 token：有的可能提前结束，有的则较晚加入。

请记住，批处理可以跨越不同请求和不同用户进行。只要各条序列在该时刻的上下文长度相同，大多数现代推理服务器都会自动将来自不同用户请求的解码步骤归为一组，这实际上就是即时进行的批量解码。可考虑采用这类方案来最大化解码吞吐量。

> **注** 在 vLLM 中，CUDA Graph 捕获的覆盖范围由 `-{}-max-seq-len-to-capture` 标志控制。捕获尺寸通常与最大序列数对齐，当序列超过该长度时，vLLM 会回退到 eager 模式。需要注意的是，该标志并不控制运行时实际有多少序列被合入批处理；并发解码槽位的数量由 `-{}-max-num-seqs`（单次迭代中序列数的上限）控制。请将其与 `-{}-max-num-batched-tokens` 一同显式设置，以获得可预测的内存占用。

continuous batching 通过在每个步骤动态调整批大小，解决了序列在不同时刻请求 token 的问题。具体而言，在每一步，continuous batching 策略会收集此刻已准备好生成下一个 token 的所有序列，并将它们合入同一个解码步骤。

如果在解码进行期间有新请求完成预填充并变为就绪，它们会直接加入下一批，而不必等待一段任意的长空闲期。这与静态批处理形成鲜明对比：后者要等到凑齐一个完整批次后才进行解码。

如果某条序列因遇到文本结束 token 或达到序列长度上限而结束，它会被立即从后续批次中移除。如果某条序列尚未就绪——例如它的长提示仍在预填充阶段处理中——那么在就绪之前它不会被纳入批次。

实际上，采用 continuous batching 时，每次迭代的批大小可以波动。不过，服务器始终会尽力将批大小最大化，把当时所有可用的序列都纳入进来——直至达到上限为止。这样既实现了高利用率，又将每个 token 的等待时间降到最低。

continuous batching 确保解码 GPU worker 永不闲置：它们始终在处理可用的请求，从而在延迟

约束下通过保持 GPU 忙碌来最大化吞吐量——即便各条序列正在等待生成新 token，GPU 也不会空闲。

类似地，Microsoft 的 DeepSpeed 与 NVIDIA 的 TensorRT-LLM 推理引擎也实现了 continuous batching（或称 in-flight batching），并配合分页 KV 缓存，以在解码期间保持较高的 GPU 利用率。具体而言，DeepSpeed 会合并多个生成请求，而 TensorRT-LLM 则使用调度器跨流对解码任务进行分组。

在解耦式解码集群中，continuous batching 变得更加有效。由于解码 GPU 只负责生成，它们可以将全部计算周期投入这一持续循环，而不会被打断去处理某个庞大且专用的提示任务。这使得吞吐量指标更加平滑——在高负载下尤其如此。

在高负载下，一个解码节点可能同时有数十乃至数百条活跃序列，它可以在每次迭代中将其中的大批序列合批处理，从而最大化硬件利用率。

而在低负载下，即使只有一条序列活跃，解码 worker 也能立即生成 token，而不必等待凑满一个批次。此时 GPU 在这一刻确实利用不足，但该单条序列的延迟依然保持较低。因此，continuous batching 能够应对两种极端情况：高并发时高效，低并发时响应迅速。这是高吞吐量与低延迟之间的良好平衡。

变长序列的分组。在 LLM 推理中处理长度可变的序列，需要精心的调度与批处理，以避免计算和内存的浪费。将长短不一的提示混入同一个批次会产生填充开销——相对于 token 数量，填充有时高达 50%。这会浪费稀缺的 GPU 和网络资源。

当把不同长度的提示合入同一批次时，每条较短的序列都必须填充到与最长序列一致的长度。这些填充引入的“占位无效”（no-op）token 仍会消耗 GPU 周期、内存带宽以及 GPU 间或网络上的传输。在某些情况下，填充 token 可占到常见生成式 AI 工作负载中全部 token 的一半，显著降低推理效率。

一种直接的解决方案是按序列长度将请求分组到不同的桶（bucket）中，使每个批次包含大小相近的序列。使用 0–512 token、513–1{,}024 token 这类固定长度的桶，可以固定批次边界并将填充开销降到最低。

vLLM 的解码调度器维护着一个轮转的 SequenceGroup 实例池（每个提示对应一个 SequenceGroup）。调度器在每个解码迭代中按固定的 token 预算推进每个组。一旦某个 SequenceGroup 处理完自己的份额，它便离开该池，同时有新的 SequenceGroup 加入。这样能让

流水线始终保持满载运转——既不必依赖静态的填充桶，也不会让 GPU 闲置。

这些批处理与调度技术非常契合将预填充和解码部署为独立集群的解耦式方案。采用这种部署配置，独立的解码节点可以利用 continuous batching 等技术，在严格的 SLO 下最小化 TPOT 方差。与此同时，专用的预填充节点可以进行独立调优，以实现最大的输入处理吞吐量和最小的 TPOT。

NVIDIA 的程序化依赖启动（Programmatic Dependent Launch，PDL）与设备端发起的 CUDA Graph 启动（见第 12 章）可用于降低每 token 的启动开销、重叠执行各项工作，并消除解码迭代之间的空泡。这些特性通常通过框架来启用，而不是在应用代码中手动开启。

使用设备端启动的图时，请以 `cudaGraphInstantiateFlagDeviceLaunch` 进行实例化，并确保所有节点位于同一设备上。可利用 PDL 在一步（例如一次解码迭代）的末尾重叠相互依赖的 kernel，从而进一步削减每 token 的启动空泡。

通过将长度分桶、连续批处理、分离式部署（disaggregation）、PDL 与设备端发起的 CUDA Graph 相结合，vLLM、SGLang 和 NVIDIA Dynamo 等现代推理系统能够同时获得高吞吐量与低延迟——即便 prompt 长度差异极大也不受影响——而且不会损害资源效率或可扩展性。

> **注** 在 vLLM 中，`--max-seq-len-to-capture` 控制 CUDA Graph 所覆盖的最大序列长度，默认值为 8192。在连续批处理下，vLLM 可能会填充（pad）到最近的已捕获长度，因此应将 `--max-num-seqs` 与 `--max-num-batched-tokens` 对齐，以尽量减少填充浪费。CUDA Graph 有助于减少针对常见序列长度的反复重建，但它并不直接规定运行时的批处理行为——vLLM 的运行时批处理由其解码调度器的动态 `SequenceGroup` 池管理，如前文所述。在生产环境中，建议将 `--max-num-seqs` 和 `--max-num-batched-tokens` 与 `--max-seq-len-to-capture` 一起调优，以便约束 HBM（KV 缓存）用量，并降低连续批处理下的填充开销。

KV 缓存的内存管理。由于解码需要对到目前为止的整个序列做注意力计算——包括此前已解码的 token——KV 缓存内存是解码 worker 的关键资源。每个序列都要为每个 transformer 层、同时也为每个过去的 token 存储键与值

张量。图 17-5 展示了 KV 缓存在不同请求之间共享的示例。

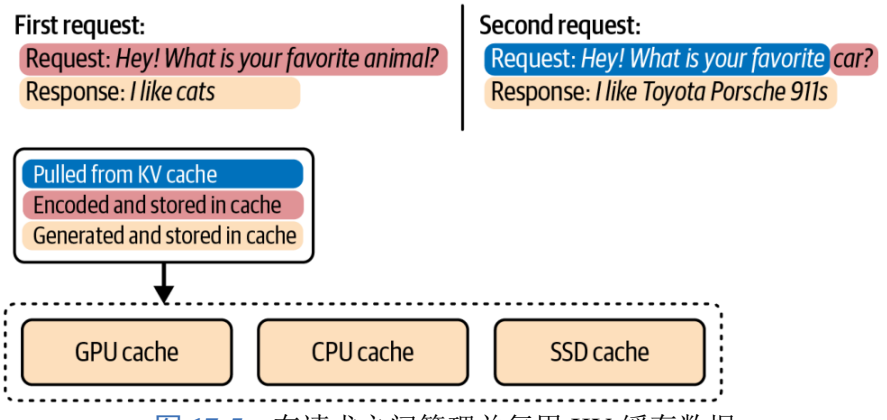

图 17-5　在请求之间管理并复用 KV 缓存数据

对大模型和长序列而言，KV 内存随 token 数线性增长，且取决于注意力布局与数据类型（dtype）。一个实用的估算公式是 `bytes_per_token` = 2 × `n_layers` × `n_kv_heads` × `head_dim` × `bytes_per_element`。（注：系数 2 对应每层每 token 的键与值。）

以一个 Llama 级别的 13B 模型为例：40 层、40 个注意力头（头维度为 128）、FP16 精度。在 4096 token 上下文和标准多头注意力（MHA）下，KV 缓存大小约为每 token 0.819 MB，总计约 3.36 GB；若采用 FP8 KV，则降至约 1.68 GB。

若改用分组查询注意力（GQA），查询组数为 8（`n_kv_heads` = 8），则 4096 token 的 KV 缓存在 FP16 下约为 0.671 GB，FP8 下约为 0.336 GB；而采用多查询注意力（MQA，仅 1 个 KV 头）时，FP16 下约为 0.084 GB。

> **注**　计算时务必使用模型实际的 `n_layers`、`n_kv_heads`、`head_dim` 和 KV 精度，因为 FP8 和 FP4 会改变 `bytes_per_element`。

一台同时服务大量并发序列的 GPU，除了承载模型权重之外，还会因 KV 缓存存储而迅速逼近其内存上限。因此，请确保你的解码 worker 使用经过优化

的内存分配器，以避免大量长序列同时在飞时耗尽 GPU 内存。解码 worker 常用以下策略来高效管理 KV 内存：

分页式 GPU 内存分配器

vLLM 的 PagedAttention 机制是一个典型例子——它将 KV 缓存划分为固定大小的页，并可以把不活跃的页换出到 CPU 内存。除 vLLM 之外，SGLang 与 NVIDIA TensorRT-LLM 中也实现了分页 KV 内存管理器，NVIDIA Dynamo 则构建在这些技术之上。这些系统还会利用 DRAM 和 NVMe 叠加外部的 KV 层级，并通过在重计算与 I/O 之间调度来均衡带宽。这在 LMCache 等类似的项目、库和运行时中很常见。

大内存 GPU 与自定义分配器

解码服务器通常使用具有大容量 HBM 的 GPU（例如 Blackwell B200 的 180 GB HBM 与 Blackwell B300 的 288 GB HBM）来存放 KV 缓存。此外，vLLM 与 NVIDIA TensorRT-LLM 等系统采用经过优化的内存管理器，以固定大小的页分配 KV 内存，从而减少碎片、支持跨请求高效复用前缀，并管理数百条长度不一的序列。这在不造成过多碎片和浪费的情况下高效地共享了内存。

KV 缓存卸载（例如换出）

当 GPU 内存占满时，解码工作节点可以把较早的 KV 块卸载到 CPU RAM 或 NVMe 等更冷的存储上。例如，若一条序列已生成 1,000 个 token，而并非所有 token 都需要立即使用，就可以把部分较早 token 的 KV 移到宿主 CPU 内存中。

需要时可以按需把它们重新换入 GPU 内存。当注意力计算要用到这些 token 时，卸载会带来一定的延迟开销，因此使用卸载时应当谨慎。解码服务器通常会尽量预取或重叠数据传输，以减少换入 KV 数据对生成过程的影响。

上下文限制与压缩

有些部署会在可解码的 token 数量（即输出序列长度）上设置硬性上限。这是应用层面的取舍：它给 KV 缓存大小封顶，避免其无限增长。KV 压缩也能减少每个 token 所需的 KV 内存，例如以更低精度（FP16、FP8、INT8）存储 KV 就能大幅降低内存占用。

另一个例子是使用多查询注意力（MQA），其中各注意力头共享每个 token 的一个 KV 向量，KV 大小按注意力头数量成比例缩减。这是直接降低 KV 占用的模型架构改动。分组查询注意力（GQA）与 DeepSeek 的多头潜在注意力（MLA）同样有助于缩小 KV 缓存。

分离式架构中的内存层级

分离式设计的另一个优势是：解码集群的 GPU 内存完全专用于存放模型权重与 KV 缓存，也无需承担庞大的提示词预填充计算——在单体式服务系统中，这类计算会在预填充期间临时占用大量额外内存。

每块解码 GPU 通常都会加载完整的模型权重——除非使用模型并行——然后把剩余内存用于 KV 存储。例如，若模型权重占用一块 GPU 内存中的 70 GB，而该 GPU 总共 180 GB，则约有 122 GB 剩余用于 KV。

这直接影响该 GPU 上大致能同时承载多少 token × 序列组合。分离式并不能消除 KV 内存问题，但通过职责分离，你可以选择在内存容量与内存带宽上进行优化的解码节点类型。

按上述方式配置好集群后，还需要决定何时把预填充卸载到预填充工作节点池——或者何时直接在解码工作节点本地执行。卸载存在开销，包括排队延迟、网络传输等，因此只有在其确实有助于改善延迟时才应使用。这一决定由下文所述的路由策略做出。

17.1.7 分离式路由与调度策略

并非每个请求都需要卸载到预填充工作节点——在不必要时这样做只会徒增开销，收益甚少。因此，分离式推理系统使用路由策略进行有条件的分流：只在可能带来好处时才走远程预填充路径。表 17-1 概括了各类路由策略，包括前缀感知与 KV 感知路由。

% 表 17-1 的表体内容已并入上方表格。

分离式路由器运行在最初接收请求的解码工作节点上，针对每个新请求快速做出决定：在本地执行预填充，还是交给预填充工作节点池远程完成。

表 17-1　路由策略概览，包括前缀感知与 KV 感知路由。

路由策略	描述
轮询	逐个节点轮流分配请求
最少请求	将请求交给当前活跃请求最少的工作节点
前缀感知	利用请求的前缀选择工作节点
KV 感知	将请求交给其 KV 缓存与该请求最匹配的工作节点

17.1.8 路由决策因素

是否把预填充卸载到预填充工作节点池，可以依据与请求和系统状态相关的多个因素。常见的路由因素包括当前队列长度、GPU 内存可用性，甚至还包括专用化程度——某些 GPU 对特定模型或提示词类型更为擅长。

vLLM 的 KV 缓存感知路由器这类高级路由器还会考虑缓存局部性：它们会把请求路由到缓存中已持有该请求部分前缀的解码工作节点。图 17-6 展示了一个示例 KV 缓存感知路由器如何根据工作节点发出的 KV 缓存事件所提供的数据，把请求在系统中传递。

路由的目标是最大化缓存命中并均衡负载。表 17-2 汇总了典型分离式设计中影响路由决策的一些关键因素。

% 表 17-2 的表头行已并入上方表格。

% 表 17-2 的表体内容已并入上方表格。

% 表 17-2 的表体内容已并入上方表格。

% 表 17-2 的表体内容已并入上方表格。

% 表 17-2 的表体内容已并入上方表格。

% 表 17-2 的表体内容已并入上方表格。

% 表 17-2 的表体内容已并入上方表格。

% 表 17-2 中“解码工作节点负载”行的影响列与“延迟 SLO 紧迫度”行名已并入上方表格。

% 表 17-2 的表体内容已并入上方表格。

表 17-2 影响路由器是否卸载预填充决策的因素。

因素	描述	对路由决策的影响
提示词长度	输入提示词中的 token 数量（计入前缀缓存之后）。	长提示词意味着更多计算：超过阈值则卸载预填充；短提示词则本地执行。
前缀缓存命中	解码工作节点 KV 缓存中已持有的提示词比例（来自之前的请求）。	前缀缓存命中率高（大部分提示词已缓存）：预填充实质上更短、更偏内存受限，本地执行；无缓存命中（全部为新 token）：计算量大，卸载更有利。
预填充队列长度	全局预填充队列中等待任务的数量（预填充工作节点的繁忙程度）。	队列很长（预填充工作节点滞后）：避免卸载新请求，改为本地执行；队列空或较空：预填充工作节点有余力，在其他条件满足时即可卸载。
解码工作节点负载	本地解码工作节点当前的负载（正在进行的解码任务等）。	解码 GPU 忙于大量解码流时，卸载有助于提升并行度（让 GPU 摆脱重计算）；解码基本空闲而预填充队列积压时，则本地预填充以利用可用算力。
延迟 SLO 紧迫度	请求延迟要求的优先级与紧迫程度。	延迟要求紧迫：可能需要卸载，以确保提示词尽快完成计算（尤其当本地解码繁忙时）；要求宽松：可本地运行以节省资源（参见“QoS 与提前拒绝策略”，第 792 页）。

紧急的低延迟要求则或需卸载，以确保提示词尽早完成计算（尤其是在本地解码忙碌时）。宽松的要求则可留在本地执行以节省资源（参见第 792 页“QoS 与提前拒绝策略”）。

请求延迟要求的优先级或紧迫程度。

这些因素倾向于只卸载那些能从远程执行中获益的请求，例如长且计算量大的提示词；而短小或命中缓存的提示词则在本地处理，以尽量减小开销。各个阈值均可调节。表 17-2 中的每个因素都对应一种特定的权衡，下面分别说明：

提示词长度

我们不想为琐碎的提示词浪费卸载开销。远程执行即便开销很小，对一个只有五个 token 的提示词也不值得，因为解码 GPU 自己就能很快处理完。卸载只保留给那些会让解码 GPU 被长时间占用的重型提示词。

前缀缓存

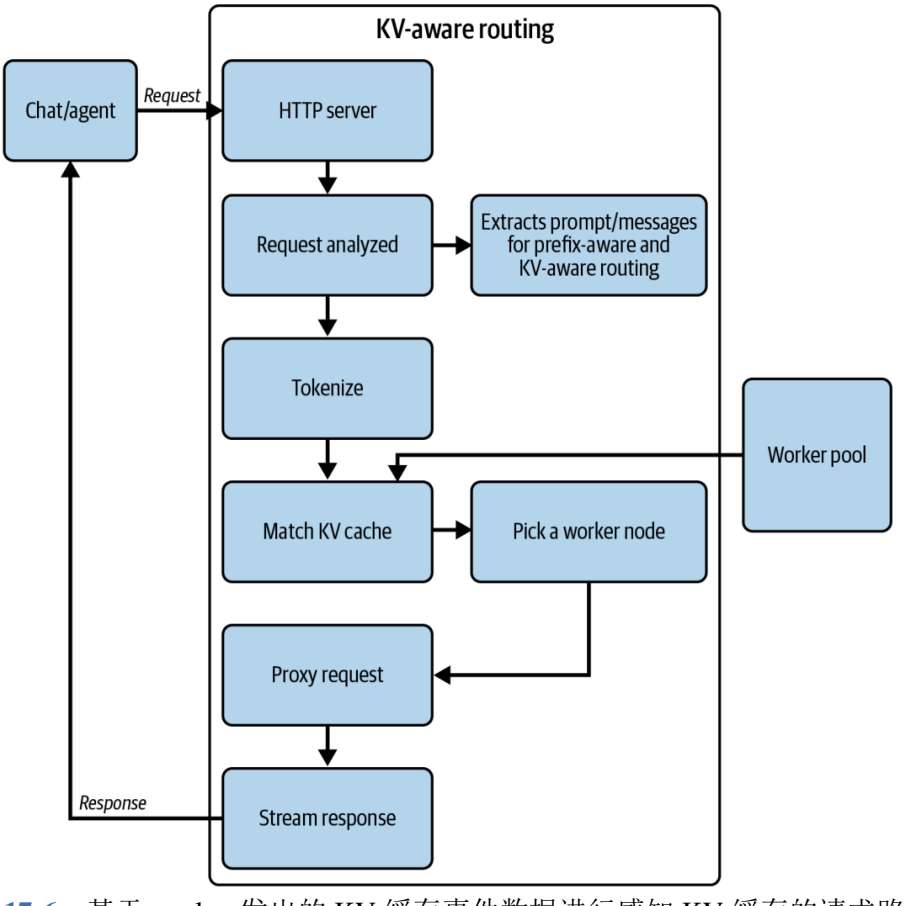

图 17-6　基于 worker 发出的 KV 缓存事件数据进行感知 KV 缓存的请求路由

现代推理系统通常会实现一种 KV 缓存，用于保存先前出现过的上下文（如对话的此前轮次或反复出现的样板系统提示词）的 KV 对。如果新请求的提示词有一个与已处理过的提示词重叠的长前缀，解码 worker 可能在内存中已经持有该前缀的 KV 数据。此时，它只需针对缓存未命中部分计算提示词的剩余部分。

如果剩余部分较短，卸载的收益就会下降。而且，若前缀完全命中、整个提示词都已被缓存，则根本不需要任何预填充计算，解码可以直接基于缓存状态立即进行。路由器会把这一点纳入考量，实际上采用的是 有效提示词长度 = `prompt_length` − `prefix_cached_length`。

较大的前缀命中不仅减少了所需的计算量，还意味着本来要把大量 KV 数据传到某个预填充 worker 再传回来，这纯属浪费。因此，这类请求会留在本地，直接利用缓存。

预填充队列长度

这一项本质上衡量的是预填充 worker 集群的负载。如果预填充 worker 已被大量排队任务压得喘不过气，再发送一个任务过去，对 TTFT 的损害可能大于收益，因为该请求只会在队列中干等。这种情况下，系统会指示解码 worker 暂时自己多承担一些工作。

这为预填充集群提供了一种天然的负载削减机制：当专用预填充层达到容量上限时，系统会平滑地退回到本地计算。待预填充队列重新变短、worker 赶完积压工作后，长提示词的卸载便会恢复。这种动态平衡正是分离式部署能够在各种负载条件下都表现出色的原因之一。

解码 worker 负载

尽管在代码中并不总是显式体现，路由器实际上起到了均衡工作负载的作用。如果某个解码 worker 已经在解码许多流，它仍可以把新到来的提示词卸载出去。这是好事：否则该 GPU 就得同时兼顾繁重的计算与解码——两者可能都会因此变慢。

反过来，如果整个系统负载很轻、解码 GPU 空闲，它们就可以在本地处理较长的预填充。实践中，空闲的解码 GPU 大概率也意味着整体负载低、预填充队列为空，上述条件已经间接覆盖了这种情况。不过，有些实现也会直接检查本地 GPU 利用率，以决定是否需要卸载预填充。

延迟 SLO 与优先级

在混合 SLA 或存在优先级类别的系统中，可以对路由加以修改以改善 QoS。对于必须获得最快 TTFT 的高优先级请求，可以跳过队列检查并立即卸载，让它马上开始计算——即使此时解码 GPU 空闲。这样，系统可以把该解码 worker 留给另一个高优先级的解码任务。

反之，对于低优先级请求，系统可以选择完全不占用预填充集群的资源，而是在解码 worker 上本地执行——甚至可以延迟执行。我们将在第 792 页“QoS

与提前拒绝策略”中重新讨论优先级请求的处理，但请记住：基础路由器可以扩展加入这一类考量因素。

路由策略中具体的阈值与权重应当通过实测来确定。例如，你可能发现在某类 GPU（如 Blackwell B200）上，少于 50 个 token 的提示词在本地计算更快，而更大的提示词则适合卸载。

随着新硬件的发布，这些阈值可能随之改变，因为新一代 GPU 拥有更高的 FLOPS 和更大的内存带宽。因此，卸载的盈亏平衡提示词长度可能会更高一些，例如一块 B200 就足以在解码 worker 上完整处理更多的 token。

> **注** 升级硬件时，由于计算 FLOPS 和内存带宽的提升，应通过实测重新调整 `PREFILL_LENGTH_THRESHOLD` 之类的阈值。

良好路由带来的结果是系统兼得两者之长：短提示词因本地执行而获得更快的 TTFT，且不产生额外开销；长提示词则受益于并行性，预填充与解码在各自的 GPU 上并行进行。此外，预填充 worker 池只在需要时才会被调用——而且在它跟不上时也不会被工作洪峰淹没。

这种条件式策略在 DistServe 原型以及 vLLM、Dynamo 等现代推理服务器中都至关重要，使它们得以在延迟约束下提升有效吞吐量（goodput），而不只是提高原始吞吐量。

17.1.9 用代码表示的动态路由策略示例

在实践中，路由策略可以实现为一段简单的条件判断。下面的代码展示了该路由逻辑的简化版本：

```
# Offload prefill decision running on a decode worker
# (B200/B300 tuned)
def should_offload_prefill(prompt_length: int,
```

```
                        prefix_cached_length: int,
                        prefill_queue_size: int,
                        decode_active_reqs: int,
                        ttft_slo_ms: int = 500) -> bool:
    # Effective prefill after prefix KV hits
    eff_len = max(0, prompt_length - prefix_cached_length)
    # Tunables (kept in config; shown here for clarity)
    PREFILL_LENGTH_THRESHOLD = 256  # see
     ↪ split_policy.prompt_length_threshold
    PREFILL_QUEUE_MAX        = 10   # see autoscale/prefill queue-length
     ↪ guidance
    DECODE_LOAD_THRESHOLD    = 8    # active decode streams
    long_prefill = (eff_len >= PREFILL_LENGTH_THRESHOLD)
    prefill_available = (prefill_queue_size < PREFILL_QUEUE_MAX)
    # Prefer remote when prefill is compute-heavy
    # and the pool has capacity.
    if long_prefill and prefill_available:
        return True
    # If local decode is busy and prefill is moderately long,
    # free decode via offload.
    if decode_active_reqs >= DECODE_LOAD_THRESHOLD and eff_len >= 64:
        return True
    # Otherwise keep prefill local
    # (lower overhead / better cache locality).
    return False
```

在这段伪代码中，PREFILL_LENGTH_THRESHOLD 是一个由系统调优的参数（例如 50 或 100 个 token），用于界定什么样的提示词算“长”提示词。PREFILL_QUEUE_MAX 是判定预填充 worker 池已饱和的阈值——具体而言，就是未完成任务数量过多的时候。

解码 worker 一收到新请求，就会调用 should_offload_prefill()。如果函数返回 True，解码 worker 会把提示词打包成一条消息，推入全局预填充任务队列，然后在等待 KV 缓存结果返回期间去做其他工作。

如果 `should_offload_prefill()` 返回 `False`，解码 worker 就立即自己执行预填充计算。这样一来，一旦预填充 worker 开始落后，新请求就会回退到本地计算，以避免排队延迟。这是一种在解码池与预填充池之间均衡负载的自适应路由。

17.1.10 动态路由策略配置示例

在生产部署中，路由策略应当通过配置文件或 UI 来设置，而不是硬编码。例如，NVIDIA 的 Dynamo 允许在 JSON 或 YAML 配置中指定复杂的路由与自动扩缩容规则。下面是 Dynamo Planner JSON 的一个简化示例，其中封装了部分策略逻辑：

```
model: ...
split_policy:
  prompt_length_threshold: 256
  prefix_cache_weight: 10.0
  queue_length_weight: 1.5
  decode_load_weight: 0.5
  enable_hotspot_prevention: true
cache:
  reuse_prefix: true
  min_cache_hit_ratio: 0.75
autoscale:
  prefill:
    min_replicas: 4
    max_replicas: 12
    scale_up:   { queue_length: 8,  gpu_utilization: 80 }
    scale_down: { queue_length: 2,  gpu_utilization: 40 }
  decode:
    min_replicas: 8
    max_replicas: 24
    scale_up:   { queue_length: 16, kv_cache_usage: 75 }
    scale_down: { queue_length: 4,  kv_cache_usage: 30 }
qos:
  enable_early_rejection: true
```

```
low_priority_threshold_ms: 500
reject_on_slo_violation: true
```

这里的配置定义了一个 `split_policy`，其 `prompt_length_threshold` 为 256 个 token，并为缓存命中、队列长度、解码负载等因素指定了权重；同时还为预填充与解码两种角色配置了自动扩缩容行为，包括如何依据队列长度、GPU 利用率和 KV 缓存使用情况进行向上或向下扩展。

此外，它还可以应用诸如提前拒绝（early rejection）之类的 QoS 规则，并调整判定请求“慢”的阈值。在实践中，Dynamo 的路由器会在启动时读取这份 JSON——或动态获取它——据此为分布在整个集群上的每一条请求做出路由决策。

17.1.11 容量感知路由

如上一节所述，NVIDIA Dynamo 支持动态路由策略，而这一动态路由能力的一个具体实现就是 Dynamo GPU Planner。规划器利用 TTFT、TPOT 以及 KV 缓存迁移成本的估计值等指标，决定是否调整路由——甚至在预填充与解码两个阶段之间重新分配或扩缩 GPU——以减少瓶颈并适应负载变化。这样，系统在需求剧烈激增时依然能够保持高性能，如图 17-7 所示。

在这里，Dynamo 规划器选择把更多 GPU 转移到预填充（上下文）阶段，因为大量摘要类提示涌入，这类请求的预填充（大输入）负载远高于解码（小摘要输出）。

相反，如果涌入的是推理类请求，规划器可以选择把 GPU 重新分配到解码阶段，因为推理请求相对输入 token 的数量会生成大量输出 token。另一些情形下，规划器也可能按传统的单体方式处理请求，即预填充和解码发生在同一个 GPU 工作节点上。

简而言之，像 NVIDIA Dynamo Planner 这样的组件可以通过持续监控实时指标，并将其与 TTFT、TPOT 延迟等应用层 SLO 相比较，从而把这一决策过程自动化。借助这些信息，规划器可以动态决定：是完全按预填充-解码分离的方式服务请求、完全不分离，还是介于两者之间——向预填充/解码各阶段分配或

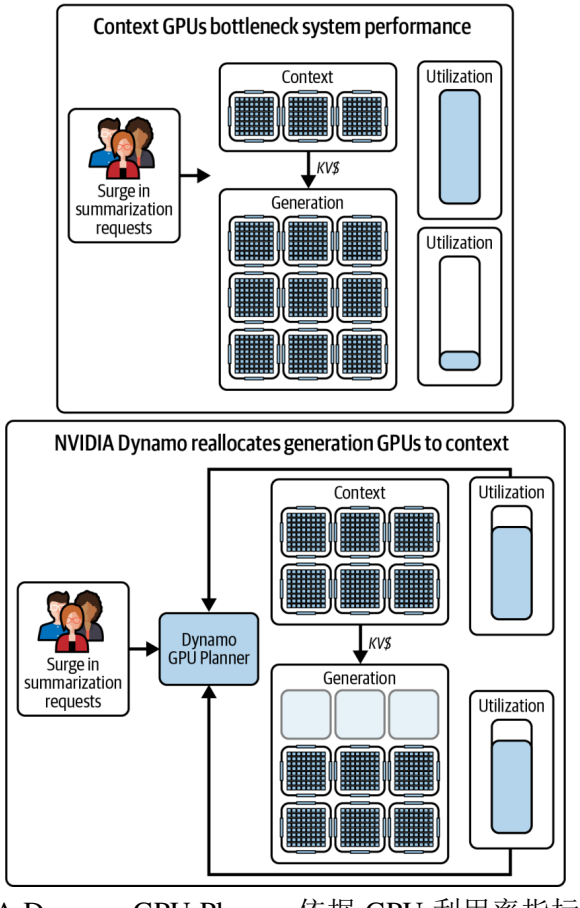

图 17-7　NVIDIA Dynamo GPU Planner 依据 GPU 利用率指标决定如何处理到达的请求，并向预填充和解码两个阶段分配 GPU 工作节点

多或少的 GPU 资源。由此形成的自适应系统在预填充与解码负载之间优化资源利用率，并达成严苛的性能目标。

17.1.12 延迟感知路由

推理路由器可以超越简单的阈值规则，使用更精细的延迟评分模型为每条请求挑选最合适的工作节点。例如，它可以根据实时指标——节点有多繁忙、内存占用多少、是否持有相关缓存等——持续为每个候选工作节点计算一个延迟成本，然后把请求发给延迟成本最低的那个工作节点。

假设延迟成本最低的工作节点就是最空闲、最值得选用的节点。一个简单的延迟成本函数可能如下所示：

```
# Lower cost is preferable
latency_cost = 0.7 * (occupancy_percent)
  + 0.3 * (active_req_count)
```

这个延迟成本函数给 GPU 占用率赋予了更高的权重：它反映了 GPU 当前的工作负荷，其次才反映该引擎上正在处理的请求数量。新请求会被发送到延迟成本最低的引擎。

权重（本例中的 0.7 和 0.3）可以基于经验数据来调整。如果发现 KV 内存占用是导致变慢的重要预测因素——例如需要换出大量数据，或占用了更高的内存带宽——就应当赋予它更大的权重。

更高级的路由策略可以纳入更多因子。例如，可以把以下因素纳入延迟成本函数：

精确缓存命中可用性

如果某个工作节点已经在其缓存中持有所需的前缀 KV（前缀命中），它就能快得多地服务这条请求。此时系统可以赋予一个较大的负权重，以降低该节点的延迟成本，从而优先选择它，因为这种情形下成本越低越好。

KV 占用率

内存占用越高，意味着 GPU 越繁忙。此时系统赋予一个正权重来抬高延迟成本，促使路由器避开该工作节点，因为这种情形下越低越可取。

活跃请求数

并发的请求越多，潜在的上下文切换开销就越大，因此这会增加延迟成本，让系统避开该工作节点。

内存带宽利用率

如果某个 GPU 当前正因处理大量长序列而占用大量内存带宽，那么再往它身上加工作只会拖慢一切。这会抬高延迟成本，促使系统不选该工作节点。

近期 KV 使用

如果确切前缀最近刚在某个工作引擎上用过——缓存仍是“热”的——性能很可能更好，因为 KV 要么还在 L2 缓存中，要么正从预填充工作节点传来。此时可以加入一个较小的负权重来降低延迟成本，优先选择该节点，因为它最近刚处理过这个前缀。

表 17-3 总结了这种高级路由策略配置，列出了一些因子及其示例成本。

表 17-3　路由因子与相对成本。

因子	解释	对延迟成本的影响
KV 占用率（%）（`occupancy_percent`）	越高表示内存压力越大	+3
活跃请求数（`active_req_count`）	并发请求越多，潜在排队越多	+1
KV 缓存命中（`cache_match_flag`）	引擎已持有所需的前缀 KV	−10（大幅负值）
内存带宽百分比（`mem_bw_percent`）	越高表示内存总线繁忙	+0.5
近期 KV 使用（`recent_prefix_flag`）	该引擎近期用过此前缀 KV	−1

因子　解释　对延迟成本的影响　KV 占用率（%）（`occupancy_percent`）

越高 = 内存压力越大　+3

活跃请求数（`active_req_count`）

并发越多 = 可能排队　+1

KV 缓存命中（`cache_match_flag`）

引擎已持有所需前缀 KV　−10（大幅负值）

内存带宽百分比（`mem_bw_percent`）

高 = 内存总线繁忙 +0.5

```
最近使用 KV
(recent_prefix_flag)
```

此引擎近期使用过该前缀 −1

接下来我们用这些额外指标修改延迟成本的计算。路由器现在计算一个综合延迟成本，如下所示：

```
# 成本越低越好
def latency_cost(occupancy_percent: float,
                 active_reqs: int,
                 cache_match_flag: bool,
                 mem_bw_percent: float,
                 recent_prefix_flag: bool) -> float:
    return (
        3.0 * occupancy_percent
      + 1.0 * active_reqs
      - 10.0 * int(cache_match_flag)
      + 0.5 * mem_bw_percent
      - 1.0 * int(recent_prefix_flag)
    )
```

预填充集群和解码集群甚至可以采用略有不同的公式。对预填充而言，缓存命中（例如前缀已经计算好）可能尤其值得纳入考量；而对解码而言，内存带宽或可用的 KV 空间可能才是决策的主导因素。

路由器需要来自每个工作器的遥测数据，其中包括等待队列长度、KV 缓存占用率、内存利用率等指标，以便持续更新延迟成本。

这样一来，系统就能把流量动态路由到对延迟影响最小的位置。低负载时，所有工作器的分数都很低，选中哪一个也许无关紧要，因为任何工作器都能快速处理请求。

而在高负载下，路由器实际上通过把新任务发给最空闲的工作器来实现负载均衡。这还会利用数据局部性（缓存命中）来减少工作量。由于任务不太可能在繁忙引擎后面堆积而其他引擎空闲，平均延迟和尾部延迟都会因此降低。

前面我们看到，把缓存命中纳入考量后，路由器可能会选择一台稍微繁忙但缓存了相关数据的服务器，从而让该请求得到更快的服务。这里的评分函数定量地刻画了这种权衡。

NVIDIA Dynamo 执行的是类似但方向相反的计算：分数越高越好。具体来说，它会计算一个远程预填充分数，如果分数超过阈值，就把预填充卸载到预填充工作器池。下面的 YAML 片段把系统配置为：当计算出的分数高于可配置的 `remote_prefill_min_score` 值时，使用条件式分离：

```
disaggregated_router:
  enable: true
  policies:
    - metric: prefill_length    # 前缀缓存命中后的提示词长度
      threshold: 256


      action: prefer_remote     # 若提示词 > 256 个 token，则卸载预填充
      weight: 0.7
    - metric: prefill_queue_depth  # 排队等待预填充工作器的请求数
      threshold: 10
      action: prefer_local      # 若队列 > 10，则倾向于本地预填充
      weight: 0.3
  remote_prefill_min_score: 0.5  # 决定是否远程预填充的总阈值
```

这里，路由器基于 `prefill_length` 和 `prefill_queue_depth` 计算分数。本例中，如果提示词超过 256 个 token，就投 `prefill_remote` 一票，即卸载预填充。这部分分数计算的权重为 0.7，如上所示。

但如果预填充队列积压很深，等待任务超过 10 个（如所配置），就会以 0.3 的权重（如所配置）投 `prefill_local` 一票。如果加权总分大于 `remote_prefill_min_score`（本例中为 0.5），Dynamo 就会卸载预填充；否则预填充留在本地，不进行卸载。

17.1.13 多路径推理（竞速）

多路径推理是指把同一请求发送给两个不同规模的模型或两条不同的路由，用于实现高可靠性。它本质上是竞相争抢最快结果，因此常被称为竞速（racing）。据知 Google 和 Meta 的生产系统都曾用模型竞速来降低尾部延迟。这是一种代价高昂但行之有效的技术。

> **注** 如果自行实现多路径推理，务必确保请求是幂等的，即使两条路径都执行也不会引发问题；并且务必及时取消较慢的那条路径，以节省 GPU 算力。

17.1.14 跨工作器的多分支并行投机解码

如第 15 章所述，一些高级推理服务器支持投机解码。请记住，投机解码会并行生成多个 token 分支，并舍弃那些并非最优的分支。

投机解码虽然不是分离的直接组成部分，但可以叠加在路由器的决策流程之上。例如，当系统检测到某个不确定的投机解码分支时，可以把多个投机请求并行发送给不同的解码工作器。这会生成多个投机性的下一个 token 分支，从而掩盖 token 分支生成中的不可预测性。对于分支走向难以预测的生成任务，这是用额外算力换取更低的延迟。

如果实现这一机制，路由器将负责协调这些投机计算并合并结果，还必须对资源使用设定上限，以免压垮集群，

但值得注意的是，多路径推理可以作为对超低延迟敏感型推理的一种潜在优化，其代价是额外且可能被浪费的计算与内存带宽等集群资源。

总之，延迟感知路由器会综合考虑当前负载和缓存复用等潜在加速因素，确保每个请求被送往最优位置。它充当分布式服务系统的“大脑”，与每块 GPU 上的底层调度协同工作。再加上全局扩缩容与 KV 缓存策略，三者共同构成了最大化有效吞吐量、最小化延迟的完整方案。

17.1.15 QoS 与早期拒绝策略

QoS 策略通过优先处理或限流某些请求来达成延迟目标。早期拒绝（early rejection，也称准入控制）则可以在系统饱和时拒绝低优先级查询，以此保护资源和其他请求的 SLO（例如延迟）。

现代系统基于队列时间阈值来实现这一点。例如，如果一个请求已在队列中等待超过 X ms，那么拒绝它或将其卸载到更低层级的服务，可能比任由它严重违反 SLO 更好。这类 QoS 防护在关键任务型推理系统中正日益普遍。

在超大规模推理系统中，需要 QoS 机制来在重负载下维持尾部延迟保证。即使采用经过优化的分离式集群配置，一旦负载超过容量，请求仍会开始排队，延迟随之上升。

设计良好的系统不会任由所有请求的延迟 SLO 被违反，而是倾向于优雅地减载：当明确判断为某些请求服务会破坏其他请求的延迟保证时，就拒绝或延后处理其中一部分请求，尤其是低优先级请求。

这类似于 Web 服务器在极端负载下返回 HTTP 503“过载”错误。对 LLM 服务而言，如果无法保证及时处理某些请求，可以主动拒绝这些请求或对请求降采样。下面列出预填充/解码分离场景中 QoS 的几个组成部分：

延迟 SLO 跟踪

系统应当掌握目标 TTFT 和 TPOT 指标。例如，99% 的情况下首 token 必须在 200 ms 内返回。利用内部遥测数据，每个解码工作器都可以估算：基于当前队列长度等信息，如果现在接纳一个新请求，其 TTFT 会是多少。同理，它还可以估算再增加一路解码流时的 TPOT。

准入控制（早期拒绝）

在一个请求被完整接受并分配之前，系统可以先进行一项检查：接纳该请求是否会使系统过载或违反 SLO。如果是，系统可以

则立即拒绝该请求，返回“服务器繁忙，请稍后再试”之类的错误响应，这称为提前拒绝（early rejection）。实践中，可以基于系统负载的全局视图，或者仅凭单节点上的启发式规则来触发提前拒绝。

例如，OpenAI 的公开 API 在积压（backlog）过高时会直接返回错误，而不去违背延迟承诺。一些服务商则会在高峰负载期间动态降低最大生成长度，实际上是以回答质量换取延迟。

优先级排序

并非所有请求都是平等的。例如，来自付费客户或关键服务的请求可能具有高优先级，而来自免费层用户或后台任务的请求优先级则较低。系统可以将优先级纳入调度决策：例如，解码 worker 的调度器可以优先服务高优先级解码任务，预填充任务队列也可以按优先级排序。当系统繁忙时，低优先级任务可能等待更久，或者被快速失败，以便为高优先级任务让路。

优雅降级

当系统开始过载时，可以不直接拒绝请求，而是降低次要请求的服务质量。例如，对不太重要的请求使用更小的模型、截断其提示词，或限制输出 token 数量。一种简单做法是：当负载较高时，暂时缩减低层级请求允许的最大提示长度或输出长度。

在解耦架构的语境下，一种有趣的优雅降级形式是：当系统承压时，暂时对低优先级请求禁用远程预填充。由于远程预填充要动用额外的集群资源来优化延迟，我们可以决定让低优先级查询只在本地做预填充，只占用解码 worker 的资源。这些请求的响应会变慢，但由此为高优先级查询腾出了预填充 worker。

例如，推理服务器可以给每个请求打上优先级标记，当预填充 worker 正忙于高优先级任务时，让路由器对低优先级请求忽略 `should_offload_prefill` 逻辑。

下面是一个提前拒绝策略的示例。它可以在每个解码 worker 上、处理请求之前运行，也可以部署在上游的 LLM 网关上：

```
# 基于估计延迟和优先级的提前拒绝
from dataclasses import dataclass
class QoSController:
    def __init__(self, ttft_slo_ms: int = 500):
        self.ttft_slo_ms = ttft_slo_ms
    def admit_request(self, priority: str) -> bool:
```

```
# 队列长度与每请求平均值由指标系统提供
est_ttft = (get_current_prefill_queue_length()
            * get_avg_prefill_time_per_req()
            + get_current_decode_queue_length()
            * get_avg_decode_time_per_req())
if est_ttft > self.ttft_slo_ms and priority.lower() == "low":
    # 拒绝低优先级请求以保护 SLO
    return False
return True
```

这里通过观察有多少预填充任务正在排队（每个都会带来一些延迟），以及前面还排着多少解码任务，来估计 TTFT。注意，解码任务通常是重叠执行的，因此这只是粗略估计。如果估计的 TTFT 超过允许的最大值（即 SLO），那么对于低优先级请求，我们拒绝它，并从 `should_offload_prefill` 函数返回 `False`。

对于高优先级请求，我们仍然接受它，必要时甚至可以牺牲其他排队的任务。更精细的做法是抢占一个已在队列中的低优先级请求，为新来的高优先级请求腾出空间。这通常通过为每个优先级级别维护独立队列来实现。

再深入一点：在前面的示例中，`avg_prefill_time_per_req()` 和 `avg_decode_time_per_req()` 函数基于观测到的每请求预填充和解码时长指数移动平均，实时计算出这些数值，并分别按提示 token 数（预填充）和生成 token 数（解码）归一化。对于长度各异的输入，引擎会用这些每 token 平均值乘以每个请求的实际 token 数进行外推。

`get_current_prefill_queue_length()` 和 `get_current_decode_queue_length()` 函数获取调度器内部队列跟踪的待处理预填充和解码任务数量。这些值由调度器维护。

这些每 token 时长估计值通过调度器循环实时刷新：调度器默认每秒通过 `/metrics` 端点更新一次，从而捕捉动态的负载变化。

提前拒绝与优先级排序确保系统在接近饱和时有节制地失效，而不是走向崩溃。最重要的请求仍能在 SLO 内得到服务；与此同时，不重要的流量则被削减。

在许多生产部署中，实现这一点需要与 LLM 网关层协同。具体来说，网关可以向客户端返回一个特定错误码，或一个表示服务器过载的状态码。关键在于，系统始终保持在一种可以兑现延迟承诺的状态：对它实际接受的工作守住其延迟承诺，而不是过度超订阅、让所有人的 SLO 都落空。

另一个 QoS 考量是自适应生成长度限制。如果解码阶段眼看要运行过久、造成 TPOT SLO 违约，系统可能会提前截断生成。例如，用户请求了 1,000 个 token，但系统承压时可能只允许生成 200 个 token 就停止，从而把资源留给其他请求。QoS 策略对某些请求进行优先级排序或限流，以达成延迟目标；当系统饱和时，提前拒绝可以拒收低优先级查询，以保护其余请求的 SLA。

简而言之，解耦降低了固有的干扰，但负载尖峰仍可能压垮任何固定的容量。QoS 机制与解耦架构相辅相成，确保它兑现自己的延迟承诺。提前拒绝会拒绝或降低某些工作的优先级，以保护其余请求的延迟。

在构建超大规模系统时，这些策略与核心性能优化同等重要。没有它们，请求洪流可能造成巨量排队和急剧变慢，令一切性能优化努力付之东流。

17.1.16 解耦式预填充与解码的可扩展性

可扩展性的另一个方面是性能随节点增多的保持程度。解耦式方案在设计上，在许多方面都接近线性扩展：可以增加预填充节点来处理更高的提示词吞吐量，也可以增加解码节点来处理更高的 token 生成吞吐量。

主要的挑战在于二者的平衡。规模越大，自适应调度器就越重要，因为在大型系统中，出现失衡的概率（例如流量模式的变化）要高得多。动态比例配置让集群能够即时重新平衡预填充与解码的比例。

另一个考量是多模型服务。如果各模型的解码特征相似，解耦架构允许在它们之间共享解码容量。例如，模型 A 和模型 B 托管在同一集群时，可以让一部分解码 worker 同时处理这两类模型。这在托管复用同一基模型架构的不同 LoRA 适配器时尤其有用。

这将得到一个极其灵活的、多租户、多模型、预填充-解码解耦的推理服务器系统。这超出了本书的讨论范围，但要知道，解耦的模块化带来了这种灵活

性：可以为多个模型配置一个共用的解码池，而每个模型拥有各自的预填充前端。

最后，分析超大规模系统中的尾延迟。当超大规模推理系统扩展到数千乃至数百万个 GPU 节点时，尾延迟的控制变得越来越难，因为服务器越多，某一台发生故障的概率就越高。

解耦在这里同样能帮上忙。将预填充任务隔离在一个节点、解码任务隔离在另一个节点上，意味着一侧的慢节点不会严重拖累另一侧的任务。

本章讨论的提前拒绝、两级调度、KV 缓存复用等技术，都有助于缓解掉队者问题。例如，缓存请求上下文的大部分内容意味着单个缓慢操作对整体生成的影响会小得多——因为大量计算早已完成。

总而言之，经过合理配置与调优的解耦系统在规模化时更加稳健：即使并发请求增长到数百万乃至数十亿，也能维持一致的延迟。通过消除一种干扰因素并引入适应性，尾延迟分布得到改善——或者至少，不会随负载增长而过快恶化。

17.2 本章要点

本章涵盖的各项技术——从 KV 缓存的高速 RDMA 传输，到动态路由算法与 QoS 策略——已被证明是大规模生产推理工作负载不可或缺的组成部分。以下是要点：

消除预填充与解码之间的干扰，以改善延迟

分离预填充与解码两个阶段可消除干扰与队头阻塞。由于长提示词不再拖延短提示词，延迟分布更加紧凑；这也让每个阶段都能可靠地满足自身的延迟 SLO。

独立优化每个阶段

预填充（prompt 处理）是计算受限的，受益于最大并行度和高 FLOPS，因此更偏好计算能力强的 GPU，以及 FP8/FP4 这类降低精度的技术，从而将 TTFT 降到最低。解码（token 生成）则是内存受限的，受益于高内存带宽的 GPU，以及 continuous batching 这类技术，以最大化每个 token 的吞吐量。PD 分离允许为

每个阶段配置不同的硬件和并行方式，而固定的、一套通吃的配置做不到这一点。

用好 KV 缓存

常见做法是使用 KV 缓存，避免重复计算 prompt 的预填充。智能路由器可以决定一个新的请求应该卸载到预填充 worker，还是直接交给解码 worker 处理。路由策略应当考虑 prompt 长度、缓存命中情况以及集群负载。

智能路由

vLLM、SGLang 和 NVIDIA Dynamo 等现代推理服务器采用多因子打分算法来路由预填充和解码请求，以最大化吞吐量并维持足够的低延迟。许多框架在调度器和监控面板中也将 TPOT 称为 token 间延迟（ITL）。建议分别针对预填充节点和解码节点跟踪 TTFT（p50/p95/p99）和 ITL/TPOT（p50/p95/p99），这样在升级等场景下，性能退化更容易排查。举例来说，当预填充服务器已经饱和，或 prompt 较短时，这些系统会跳过远程预填充。

用 QoS 在大规模下守住 SLA

在高负载下应用准入控制和优先级排序。与其让系统的延迟对所有用户全面爆炸，不如对超额或低优先级的请求快速失败。真实系统通过返回“繁忙”错误或降级的响应长度来实现提前拒绝，以保证第 99 百分位延迟保持在给定阈值之内。PD 分离与 QoS 相结合可以防止过载级联崩溃——哪怕在每天数十亿请求的场景下也是如此。

17.3 结语

预填充与解码分离如今已是现代超大规模 LLM 推理的常见做法。几乎所有的主流 AI 服务商——Meta、Amazon、NVIDIA、Google 和 OpenAI（合称“MANGO”）——都在其大规模 LLM 部署中采用了某种形式的分离架构。

虽然这些公司大多不会公开其完整的生产架构，但已有许多开源实现（例如 vLLM、SGLang、NVIDIA 的 Dynamo）展示了这一路径及其收益。与单体架构相比，分离式 PD 在延迟约束下能产出更高的有效吞吐量（goodput），并具备更好的成本效率。

下一章中，我们将继续把现代 GPU 与网络硬件同更先进的调度和缓存优化结合起来，在不牺牲延迟或不推高成本的前提下，为数万亿参数规模的 LLM 提供面向数十亿用户的服务。

第 18 章 高级预填充-解码与 KV 缓存调优

本章在第 17 章的基础上，深入探讨推理预填充与解码阶段的高级优化。我们将在高层扩展策略的基础上介绍底层技术，包括单解码“超级内核”、跨 GPU 的智能 KV 缓存调优与共享、提示状态的快速 GPU 到 GPU 传输、自适应资源调度，以及预填充与解码 worker 之间的动态路由。

我们还将重点介绍带来全新性能与效率水平的硬件和软件创新。通过应用这些技术，你可以显著降低解码延迟、提高每 GPU 的吞吐量，并在规模化部署中满足严格的延迟 SLO。

18.1 优化解码内核

到目前为止，我们一直聚焦于高层的系统与集群优化策略。在提升超大规模推理时，另一组值得考虑的技术是底层内核与内存管理调优——尤其是针对解码阶段的调优。

解码阶段是分布式的，且通常受限于内存带宽。这促使研究人员和从业者尽可能加快解码速度，并针对特定硬件进行调优。这一领域值得注意的创新包括 FlashMLA（DeepSeek）、ThunderMLA（斯坦福大学）和 FlexDecoding（PyTorch）。它们专门针对 LLM 工作负载中常见的变长序列场景，提升 transformer 多头注意力在解码阶段的效率。下面逐一介绍。

18.1.1 FlashMLA（DeepSeek）

Flash Multi-Latent Attention（FlashMLA）是 DeepSeek 推出的优化解码内核。它专门聚焦于单 token 解码步骤，该步骤本质上是 transformer 层用于生成下一个 token 的前向传播。FlashMLA 通过融合运算并更好地利用 GPU 内存层次结构，使解码变得更快。

FlashMLA（解码）之于推理，正如 FlashAttention（预填充）之于训练。它降低了内存访问开销和延迟。借助 FlashMLA，与标准内核相比，你可以在解码阶段实现大幅延迟降低。

FlashMLA 通过将多个注意力运算融合为一次运算来提高算术强度。这样，它可以在一次融合内核启动中处理多个注意力头和多个时间步。尽管批大小很小，它仍能让数学单元保持繁忙，从而提高解码期间的 GPU 利用率。图 18-1 展示了在 Hopper H100 GPU 上，MLA 相比分组查询注意力（GQA）和多查询注意力（MQA）等其他注意力实现的算术强度提升。（注：Blackwell 凭借更高的 TFLOPS 和 HBM 带宽，将两条 roofline 整体上移。）

FlashMLA 的推出意义重大，因为它表明解码阶段的瓶颈——内存带宽和内核启动开销——即使在不够理想的 GPU 硬件上也可以被削减。它减少了独立 GPU 内核启动的次数并优化了内存访问模式，从而为解码任务从受限硬件中榨取尽可能多的性能。

DeepSeek 开源的 FlashMLA 实现已经可用并被广泛采用。SGLang 和 vLLM 都对 DeepSeek 模型提供了一流支持。因此，你应该评估 FlashMLA，以便在不改变高层架构的情况下提高每 token 的解码吞吐量。

由于 DeepSeek 开源的 FlashMLA 已集成到现代推理服务系统中，你应该把它作为提升每个解码 worker 吞吐量——或降低每 token 延迟——的途径来探索，而无需任何高层架构改动。

18.1.2 ThunderMLA（斯坦福大学）

在 FlashMLA 的基础上，斯坦福大学的研究人员推出了 ThunderMLA——一个完全融合的注意力解码“超级内核”，专注于解码与调度（而不是融合整个前馈块）。这个“超级内核”通过把多次内核启动合并为一次、并整合中间内存写

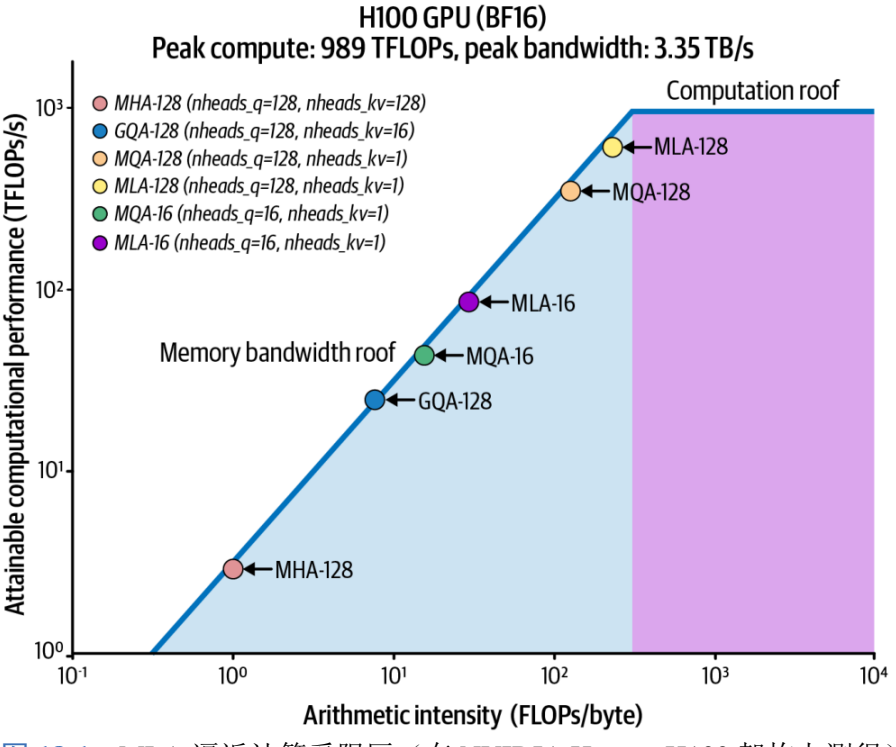

图 18-1　MLA 逼近计算受限区（在 NVIDIA Hopper H100 架构上测得）

入，降低了启动开销和尾部效应。据报告，在各类工作负载中，ThunderMLA 的解码吞吐量比 FlashMLA 快 20%–35%。

ThunderMLA 的核心思想是：当解码不同长度的序列时，使用细粒度调度和融合运算可以避免尾部效应——即一些序列先完成，而另一些序列让 GPU 处于部分空闲状态。即使某些解码流提前完成，ThunderMLA 也能让 GPU 保持繁忙。它通过动态打包并处理剩余流，利用其融合方法来实现这一点。

这些优势在拥有更大 L2 缓存和更快注意力原语的现代 GPU 上会被进一步放大。值得注意的是，现代 NVIDIA GPU 还为 FP8 和 FP4 提供了 Transformer Engine 支持（以及 FP6，不过由于 FP6 在现有 AI 框架和工具中尚未广泛使用，本书主要关注 FP8/FP4 格式）。配合更高的内存带宽，Tensor Core 让 ThunderMLA 这类内核能够更接近硬件极限运行。得益于这些架构进步，ThunderMLA 在现代 GPU 上实现了更低的每 token 延迟。

18.1.3 FlexDecoding（PyTorch）

在第 14 章中，我们讨论了 PyTorch 的 FlexAttention，它允许你为注意力中的任意稀疏模式 JIT 编译融合内核，包括局部窗口、块稀疏模式等——而无需编写任何自定义 CUDA。在底层，TorchInductor 加上 OpenAI 的 Triton 会生成一个融合内核，只计算该模式允许的 query-key 对。在给定硬件上有利时，Triton 会自动应用 warp 特化和异步拷贝等性能优化技术。不过，你也可以通过配置 `num_consumer_groups` 等方式进一步调优 `triton.Config`，实现更深入的定制。

FlexDecoding 是 `torch.nn.attention.flex_attention` 的解码后端。FlexDecoding 同样支持原地管理 KV，并像 FlexAttention 一样支持掩码和偏置。具体来说，FlexDecoding 为解码阶段（`Q_len=1`）编译专用内核，对不断增长的 KV 缓存进行注意力计算。

在运行时，FlexDecoding 实现会挑选专用解码内核，并在多个解码步骤中复用它。当形状和 dtype 保持兼容时，这有助于最小化开销——从而大大加速长序列 LLM 推理。

一旦重编译得到控制，对于稳定、延迟关键的解码，优先使用 `torch.compile(mode="max-autotune")`。保持捕获边界较窄（按层或注意力块），以减少不规则批处理带来的图失效。预填充和解码优先使用 Transformer Engine FP8（MXFP8）。在精度允许且性能提升时考虑 FP4（NVFP4）。截至本书写作时，FP4 支持仍在成熟中，短期内可能不如 8 位和 16 位格式。继续设置 `torch.set_float32_matmul_precision("high")`，为剩余的 FP32 运算启用 TF32 回退。FlexAttention 的解码后端支持常见的性能增强，包括分组查询注意力（GQA）和 PagedAttention。

FlexAttention 和 FlexDecoding 的一个关键特性是对嵌套锯齿布局张量（NJT）的支持。这使得在解码期间可以对变长序列（LLM 工作负载中很常见）进行不规则批处理。图 18-2 展示了多条序列的锯齿张量表示。

此外，FlexDecoding 支持偏置项，并通过块掩码转换接口与 PagedAttention 集成，该接口将逻辑块映射到物理缓存布局。这把逻辑 KV 块散布到物理缓存布局中——无需创建额外副本，如图 18-3 所示。

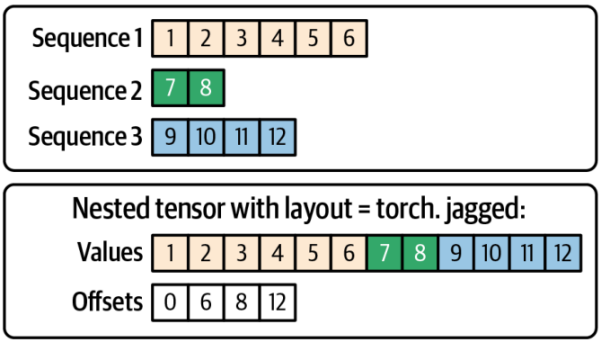

图 18-2 作为嵌套锯齿张量（偏移量）的不规则批：顶部三条序列表示为底部带偏移量的单个嵌套锯齿张量表示；解码期批处理优先使用 PyTorch NJT。

FlexDecoding 利用捕获张量在每次迭代中改变某些掩码或偏置值——无需重新编译。它还与 PagedAttention 集成。要使用 vLLM LMCache 这样的全局 KV 缓存，只需将缓存的页表映射到 FlexAttention 的 BlockMask。这将在运行时把逻辑 KV 页即时翻译为物理内存地址。

借助 FlexDecoding，开发者在自定义注意力稀疏模式方面拥有完全的 Python 级灵活性。这对 MoE 模型推理特别有用。FlexDecoding 让你无需编写任何自定义 CUDA 内核即可实现接近最优的性能。本质上，它让任意注意力模式都能获得与稠密注意力模式类似的优化。随着新推理技术的涌现，这一点会变得更有价值。

其中许多能力——例如解码的融合注意力和对 PyTorch 嵌套锯齿张量（NJT）批处理的支持——已在 PyTorch 核心库中可用。这使得针对典型模式的自定义融合变得不那么必要。

> **注** 对 LLM 工作负载中常见的不规则序列进行批处理时，优先使用 NJT 布局。

这些内核级进步技术含量很高，充分利用了 GPU、网络和内存的全部能力。即使在相同的硬件上，这些软件优化也能显著提升解码性能。在设计超大规模系统时，应尽可能采用这些优化内核。务必使用带有基于硬件 CUDA trace 的 Nsight Systems 验证重叠和内核效率。此外，使用 Nsight Compute 获取具体的内存和链路指标。

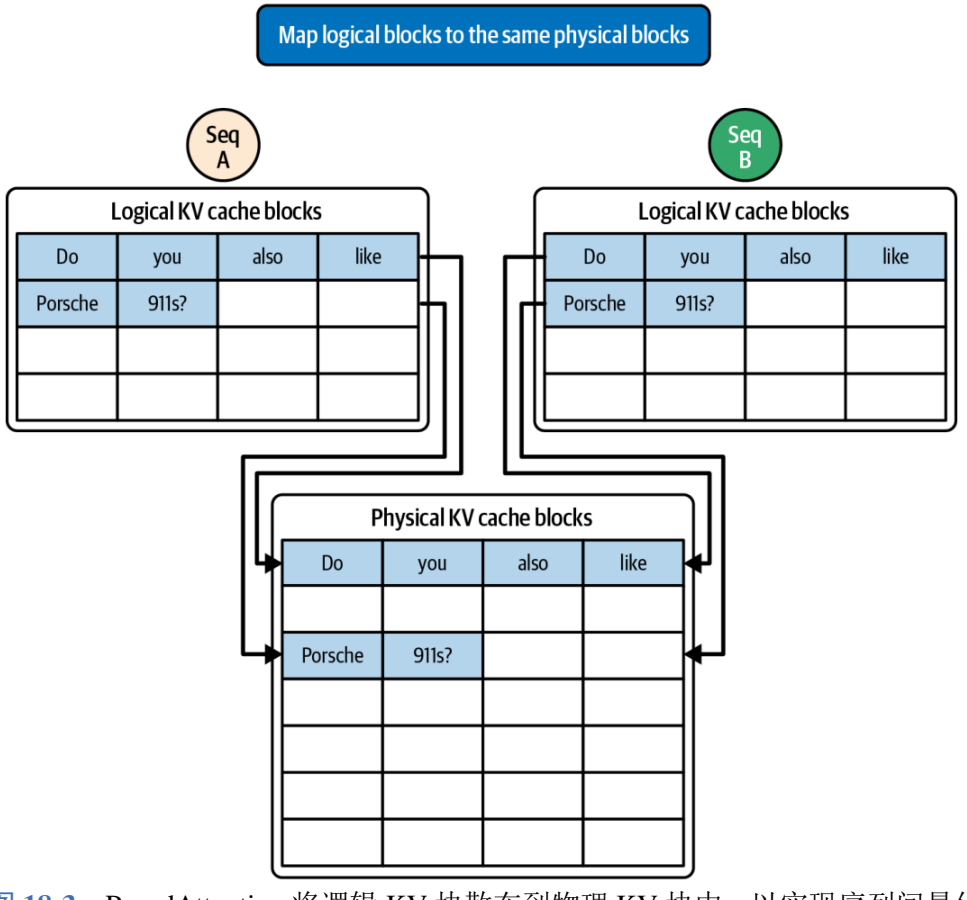

图 18-3　PagedAttention 将逻辑 KV 块散布到物理 KV 块中，以实现序列间最优缓存复用；块大小与 LMCache 页大小对齐——更大的页（如 64–128 个 token）可降低解聚架构中的 RDMA 开销。

启用其中一些高级内核可能需要安装自定义库或启用专用 CUDA 内核——尤其是较新的技术。不过，这些技术通常在发布后不久就会被 PyTorch 和主流推理引擎支持。即使确实需要安装自定义资源，这些努力也是值得的，因为它将直接转化为更低的延迟和解码 worker 池中更少的 GPU。

18.2 调优 KV 缓存利用与管理

解聚架构要求你把 KV 缓存当作整个集群的一等共享资源。由于 KV 缓存现在的生命周期更长并且可以在节点之间移动，高性能推理系统已经改进了 KV 缓存的存储和共享方式。

特别是，分布式 KV 缓存池和跨请求的前缀复用成为强大的技术。此外，关注新一代 GPU 和 HBM 带来的内存带宽提升也很重要。下面在提升 KV 缓存性能的背景下逐一讨论。

18.2.1 解聚式 KV 缓存池

解聚式 KV 缓存池不再让每个 GPU 只为当前服务的请求存储 KV，而是把 KV 存储与单个 GPU 解耦。它将数据分散到整个集群的 GPU 内存中。

该池还可以卸载到 CPU 内存，包括 Grace Blackwell 和 Vera Rubin 平台的统一 CPU 与 GPU 内存。它还可以卸载到 NVMe SSD 之类的持久存储。

使用解聚式 KV 缓存池时，当预填充为提示计算 KV 张量——或当解码扩展 KV 张量——KV 块会以分布式方式存储在许多计算节点上。如图 18-4 所示，该图改编自解聚式 KV 池的相关工作。

考虑一个 250,000 token 的超长上下文（例如多轮对话），使用一个 700 亿参数的 transformer 模型，有 80 层、32 个头，每个头维度为 128。这会为每个 token 生成巨大的 KV 缓存占用。

每个 token 生成一个 key 向量和一个 value 向量，长度等于模型的隐藏维度（`num_heads` × `head_dim`）。假设我们的模型为 4,096。这

意味着每层产生 8,192 个浮点数。跨 80 层求和后，每个 token 会产生 655,360 个浮点数的 KV 数据。假设使用 16 位精度，即每个浮点数 2 字节。每个 token 大约需要 1.31 MB。扩展到 250,000 个 token，仅 KV 数据就产生约 328 GB！

这些计算基于大模型的每 token KV 大小。这说明了为什么 vLLM 等引擎广泛采用 FP8 KV 缓存来减少占用并增加批处理机会。

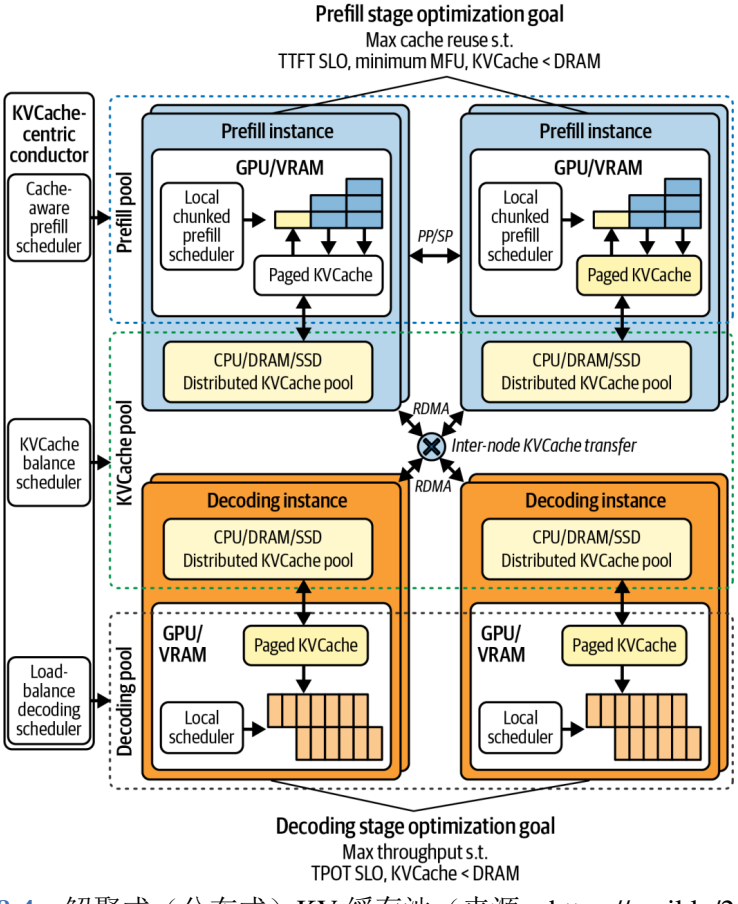

图 18-4　解聚式（分布式）KV 缓存池（来源：https://oreil.ly/2xtK-）

假设我们将 KV 缓存量化到 FP8 并使用选择性分层缓存等技术，对于这个 250,000 token 的提示，我们可以把占用降低到大约 100–150 GB 的区间。单个 GPU 很可能没有足够容量容纳所有 token 的 KV 以及它需要持有的其他内容（如模型权重等）——尤其是随着多轮对话的继续。因此，系统要么截断上下文，要么对较早的 token 进行代价高昂的 KV 重算。

然而，借助解聚式 KV 池，上下文较早部分的 KV 可以从 GPU 中被逐出，推送到分布在集群中的 KV 缓存池——以及 CPU DRAM 或 NVMe 存储中。需要时再将这些数据取回 GPU 内存。

解聚式 KV 缓存池实现了一个多级内存层次：GPU 设备内存保存活跃的 KV 缓存，而 CPU 主机 RAM（或 NVMe 存储）充当溢出后备存储。现代推理引擎

可以选择将 KV 缓存卸载到 CPU 内存或 NVMe。这实际上像操作系统的虚拟内存子系统一样虚拟化了 GPU 内存。

这种设计通过异步地在 GPU 内存与 KV 缓存池之间分页交换 KV 块，实现了超长上下文，而不会阻塞计算流水线——前提是通信与计算之间有良好的重叠，正如本书通篇讨论的那样。

此外，通过将请求状态与单个 GPU 解耦，系统可以使用全局 KV 缓存池在集群的多个节点之间动态分片 KV 数据，从而自适应地平衡负载。这简化了扩展，并改善了大型推理集群的故障隔离。

而且，由于任何解码节点都可以访问全局 KV 池，因此在故障转移或负载均衡需要时，任何解码节点都可以参与解码任何请求。这为调度器增加了灵活性，因为它可以选择离相关 KV 缓存块所在位置最近的解码节点。

如果某个前缀的部分 KV 块缓存在服务器 A 的 DRAM 中，那么把解码调度到服务器 A 上可能更快，因为它可以快速将这些块拉入自己的 GPU。相比之下，服务器 B 则必须通过网络获取这些 KV 块。

这描述了分布式系统的一个经典最佳实践：选择离所计算数据最近的计算节点。这样，系统可以最小化代价高昂的数据搬移。

高效的 KV 缓存调度器除了考虑网络拓扑外，还可以考察 KV 块在池中的分布，并据此分配预填充和解码任务。这样，预填充节点可以把 KV 数据放入一个以分布式内存空间形式实现、可跨集群访问的池中。

有了集群侧共享内存空间中的 KV 缓存，任何解码节点都可以取回数据。这避免了每次都安排直接的预填充到解码传输。

由于从池中取回 KV 数据需要额外一跳，这会带来一点额外开销，但它允许更大的灵活性，因为所有解码节点都可以访问所有已缓存的 KV 数据。这也意味着，没有直接从某个预填充接收数据的解码节点在需要时仍可以从池中访问 KV 数据。

如果解码节点崩溃——或者请求出于某种原因需要在生成中途迁移——KV 数据不会丢失。数据存在于池中，另一个节点可以接手，并使用保存的 KV 从断点继续。这提高了容错能力。

全局 KV 缓存池还提供了跨请求的缓存持久性。这样，如果两个请求共享某个前缀，该前缀的 KV 只需计算一次即可在整个集群中复用——即使这些请求最终落在不同的解码服务器上。

简而言之，解聚式 KV 缓存池用内存（或更冷的存储）换取计算。通过存储更大的 KV 缓存，系统可以在许多场景下避免重算 KV 数据。这种方法利用了这样一个事实：复用数据——即使来自 DRAM 或 SSD——通常比重复计算具有二次时间复杂度 $O(N^2)$ 的大型注意力矩阵乘法更便宜。

18.2.2 KV 缓存复用与前缀共享

如前所述，对于共享公共前缀的提示，跨请求复用已缓存的 KV 数据是有益的。这种场景相当常见，表现为多轮对话、共享系统提示和附加文档。

与其为每个请求重算该前缀的 transformer 注意力输出，系统可以存储该前缀的 KV 输出并直接复用它们。

本质上，这跳过了输入该部分的预填充计算，节省了大量时间和 GPU 周期。

一个以 KV 缓存为中心的调度器在分配工作时会考察“前缀缓存命中长度”——即该提示有多少 token 已经存在于缓存池中——来考虑前缀缓存命中。实践中，如果新请求到来时其前 N 个 token 与 KV 池中某个已缓存的前缀匹配，系统可以决定复用该 KV 数据。

vLLM 使用其 PagedAttention 机制，通过 KV“页”的全局哈希表实现自动前缀缓存。在这里，每个唯一的 16-token 上下文块都有一个哈希。如果新请求需要的前缀与已存储的块匹配（按哈希），它可以直接复制那些 KV 张量，而不必重算。

如果相同的上下文再次出现，系统就从内存中提供它。本质上，它把上下文的 KV 当作可复用数据，可以通过哈希按内容查找。实现通常维护一棵全局“提示树”来管理这些缓存上下文，并在必要时逐出它们。这为最频繁复用的前缀做了优化。

有效 KV 复用的关键在于识别相同或重叠的前缀。通常，系统为了简单起见聚焦于精确匹配：如果前 N 个 token 完全匹配，就复用该块。合并部分前缀

重叠更复杂，因为你需要以某种方式合并缓存，而这并不总是直截了当的。因此，典型的缓存使用精确前缀缓存。

不过这里有一个权衡。无限期存储许多用户的 KV 缓存会消耗大量内存。系统必须为 KV 块实现 LRU 等逐出策略，丢弃不太可能复用的缓存，为新缓存腾出空间。调度器还可以根据复用可能性决定保留哪些缓存。目标是在内存约束内最大化缓存命中率。

如果某个预填充节点在其本地 GPU 内存或本地 DRAM 缓存中已经持有所需 KV 的一部分，把请求路由到该节点以最小化数据传输可能是有益的。这是数据感知调度的一个例子：把计算送到数据所在之处，而不是总把数据拉到有可用计算的地方。

这类似于分布式系统中的局部性感知调度。我们之前的路由讨论中已有所涉及。如果可能，你应该把请求路由到生成其前缀的服务器。这最大化了缓存命中的可能性。

在更广泛的解聚语境下，前缀缓存依赖于对许多请求的 KV 拥有统一视图，并可能将其存储在全局池这样可共享的地方。这与按请求或按节点隔离的竖井式做法形成对比。

如果同一提示在不同时间到达不同节点，这还有助于减少解聚架构原本可能产生的重算开销。借助全局 KV 存储或协调式缓存，即使用户的请求落在不同的解码服务器上，它们也能从彼此已缓存的工作中受益。

18.2.3 优化的 KV 缓存内存布局

底层创新的另一个领域是优化 KV 缓存的内存布局。KV 缓存存储每条序列所有过去 token 的 key 和 value，对于许多并发解码流来说会变得非常庞大，因为每条流使用的内存大致与 `num_layers` × 2 × `sequence_length` × `d_head` 成正比。

分层缓存等技术很有用，因为并非所有 KV 对都需要始终保留在 GPU 内存中。KV 缓存较早的部分可以交换到 CPU——甚至可以压缩。

由于我们强调保持低解码延迟，大多数设计把活跃的 KV 缓存放在 GPU 内存中以便快速访问。在这种情况下，你可以调优内存的布局和访问方式。

DeepSeek 的 FlashMLA 对 KV 缓存进行分页，并按固定大小的块（页）分配缓存，使活跃序列可以实现连续的内存访问。这减少了缓存未命中和 DRAM 流量。

此外，一些系统实现了前缀压缩——例如当上下文窗口移动后，提示的某个前缀不再被关注。在这种情况下，KV 缓存管理器可以丢弃或压缩这些 KV 条目。这在长对话中更为相关，因为上下文窗口会滑动。但对于超长序列，它可以节省内存和带宽。

当模型使用滑动窗口或其他受限注意力模式时，这种逐出/压缩技术是安全的。但是，未经仔细评估，不应将其应用于对完整内容窗口（或检索钩子）保留完整注意力的层。

另一种称为 POD-Attention 的技术同样重组注意力计算以减少 HBM 流量。具体来说，它使用 SM 感知的线程块（或协作线程数组 [CTA]）调度。这实现了运行时运算绑定，动态地将运行在 SM 上的每个 CTA 分配给预填充或解码任务。如图 18-5 所示。

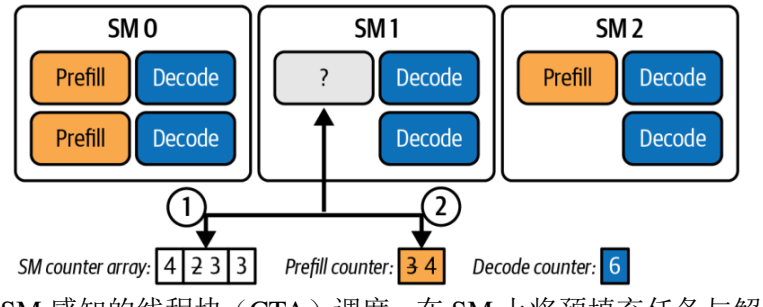

图 18-5　SM 感知的线程块（CTA）调度：在 SM 上将预填充任务与解码任务配对，以最小化内存搬移。

因此，它不是为每个阶段静态启动独立内核，而是由单个内核启动足够的 CTA 来覆盖两种工作负载。在运行时，每个 CTA 检查自己所在的 SM，并使用每 SM 计数器根据该 SM 上正在运行的其他任务来决定应执行哪种运算（预填充或解码）。

SM 感知的调度逻辑试图在运行时将预填充与解码运算配对。这避免了孤立的内存流量突发，并使资源需求趋于平滑。

具体来说，POD-Attention 将预填充和解码工作放在同一个 SM 上，使融合内核能够改善局部性并减少冗余的 HBM 事务。这最小化了内存搬移，最大化了带宽利用率，并平衡了每个 SM 上的计算受限和内存受限工作负载。通过把预填充和解码工作放在相同 SM 上并配合恰当的 SM 感知 CTA 调度以释放完全重叠，POD-Attention 可以将注意力性能提升最多约 29%。

POD-Attention 的动态绑定将硬件的 CTA-SM 分配与软件的 CTA 角色分配（预填充或解码）解耦。这类创新表明，业界越来越关注硬件与软件协同设计，以最小化内存搬移并充分发挥系统性能。

18.2.4 GPU 与 CPU-GPU 超级芯片的改进

你还应该考虑新硬件带来的内存带宽改进。更高的内存带宽和更大的 L2 缓存直接有利于受内存带宽限制的解码阶段的性能。

NVIDIA 的 Grace Blackwell GB200 NVL72 系统是一个机架级平台，拥有 36 颗 Grace CPU 和 72 颗 Blackwell GPU，它允许单个逻辑解码单元拥有数十 TB 的内存用于 KV 缓存。该硬件拥有约 30 TB 的统一内存，非常适合在内存中保存超大规模上下文。这些上下文的规模可达数百万 token。

在这样的平台上，统一内存占用很大。然而，对于延迟关键的解码，你仍然希望活跃的 key 和 value 驻留在 GPU HBM 中。因此，你应该把 Grace CPU 内存（LPDDR5X，而非 HBM）用作较低层级的缓存或存放非常旧的 token。当上下文超过可用 HBM 时——即使在 NVL72 这样的系统上——预填充和键值卸载仍然很重要。

简而言之，宏观层面的解聚应与微观层面的优化配合，才能充分实现最大推理性能。FlashMLA/ThunderMLA 等高级解码内核、高效的内存布局（分页缓存等）以及最新的 GPU 架构将带来高效且可扩展的解码。

18.3 预填充与解码之间的快速 KV 缓存传输

解聚推理的一个关键需求是快速、高效地将 KV 缓存从预填充 worker 传输到解码 worker。如果传输不够快，并行化预填充与解码所节省的时间就可能全部浪费在等待数据搬移上。

在本节中，我们讨论用于最小化传输开销的技术，然后描述系统如何使用高速互连并避免额外的 KV 副本来实现交接。

18.3.1 KV 缓存大小

预填充的输出主要是所有提示 token 的 KV 缓存。这可能是大量数据。考虑一个有 L 层、每层有 h 个维度为 d 的注意力头、提示有 N 个 token 的模型。KV 缓存大小约为 $2 \times L \times N \times (h \times d)$，其中因子 2 代表 key 和 value 各一份。

实际大小取决于精度（FP16 与 INT8 等）和模型的具体情况，但它很大。例如，一个有 40 层、16 个大小为 64 的头和 1,000 token 提示的模型会产生约 40,000 个 KV 向量。这可能是数百 MB 的数据。如果 token 数为 5,000，则还要大 5 倍。

如果以朴素的方式在网络上传输这么多数据，可能会引入显著的延迟。例如，一种朴素的做法可能是先把 KV 复制到预填充 worker 的 CPU 内存，然后通过 TCP 发送——甚至写入磁盘让解码进程加载。对于大型提示，这可能极其缓慢，达到数百毫秒量级。目标是把传输时间降低到只有几毫秒。这样才能让预填充和解码真正并行重叠。

实现低延迟 KV 数据传输通常需要把小块的 PagedAttention 块拼合成更大的缓冲区，并经由基于 GPUDirect RDMA 的路径搬移，而不是经由 CPU 套接字。

18.3.2 零拷贝 GPU 到 GPU 传输

现代解聚系统使用零拷贝 GPU 到 GPU 传输技术。实践中，这涉及在高速互连结构上使用远程直接内存访问（RDMA）。例如，你可以使用 InfiniBand 在机架/节点之间传输——或者在单节点（多 GPU）平台内使用 NVLink/NVSwitch

直接写入 GPU 内存。这些方法直接在 GPU 之间发送数据，无需经过 CPU 内存拷贝。

NVIDIA 面向推理的高性能 GPU 到 GPU 传输库叫做 NVIDIA Inference Xfer Library（NIXL）。NIXL 提供了插件式架构（例如 NVLink、UCX 网络结构、GPUDirect Storage），用于零拷贝的 GPU↔GPU 和 GPU↔ 存储数据搬移。

NIXL 简化了类 RDMA 的传输，允许一个 GPU 通过可用的高速网络结构——例如 InfiniBand 或基于 NVLink 的连接——直接写入另一个 GPU 的内存。换句话说，预填充 GPU 可以直接把 KV 张量注入解码 worker GPU 的内存。

基于 RDMA 的协议绕过 CPU，充分利用 GPU 互连带宽。NVIDIA Dynamo 以及开源的 vLLM 加 LMCache 集成都依赖 NIXL。具体来说，它们使用 NIXL 通过 NVLink 或 RDMA 将 KV 张量直接写入远端 GPU 内存。现代 GPU 互连提供非常高的带宽，取决于链路类型和争用情况，1 GB 的传输可以在几毫秒到几十毫秒内完成。

实践中，实现通过把数据传输与计算重叠来获得低传输时间。例如，使用 RDMA 时，解码 GPU 可以在预填充 worker 异步向其内存缓冲区写入 KV 数据的同时，继续为其他序列生成 token。预填充可以推送数据（RDMA write 推送模型），解码也可以拉取数据（RDMA read），取决于设计。无论哪种方式，数据路径都不需要 CPU 参与。

快速 KV 传输的常见策略包括预填充侧推送、解码侧拉取、共享内存（CUDA IPC）缓冲区、连接器/队列抽象和非阻塞重叠。下面逐一讨论：

18.3.3 预填充侧推送

预填充 worker 在完成提示处理后，发起一次 RDMA 写，把 KV 数据直接写入解码 worker GPU 上预留的缓冲区。这可以以非阻塞方式进行；预填充可以启动传输后转而处理其他工作，DMA 在后台进行。

18.3.4 解码侧拉取

另一种方式是，解码 worker 在准备好开始解码时，直接从预填充 GPU 的内存 RDMA 读取。推送或拉取都能达到相同的最终结果（无 CPU 拷贝）。有些实

现可能偏好推送，以便把协调工作交给发送方；另一些则偏好拉取，让接收方控制时机。

18.3.5 共享内存（IPC）缓冲区

如果预填充和解码恰好位于同一台机器上（同一服务器中的不同 GPU），它们可以使用 CUDA 进程间通信共享内存句柄，甚至共享 PCIe BAR，实际上通过同一主机上的 NVLink 或 NVSwitch 进行复制。这是零拷贝传输的本地变体，无需经过网络。

18.3.6 连接器/队列抽象

vLLM 的实现把传输机制抽象到一个逻辑接口（Pipe 或 LookupBuffer）后面。预填充进程把 KV 放入这个缓冲区或发出可用信号，解码侧再取回它。在底层，这可以使用 RDMA，甚至可以使用高性能的发布-订阅消息（Dynamo 中控制信号使用 NATS）。关键在于把逻辑交接与传输解耦，以便接入不同的传输方式（RDMA、共享内存等）。

18.3.7 非阻塞重叠

如前所述，优化过的系统会把 KV 传输与进行中的解码计算重叠。例如，Dynamo 的解码 worker 在预填充 worker 为新请求向其 GPU 内存写入 KV 数据时，继续为其他请求生成 token。这隐藏了大部分传输延迟。因此，你可以把约 5 ms 的 KV 传输与解码计算重叠，对请求首个生成 token 的净延迟几乎为零。

借助这些方法，KV 传输可以在几毫秒量级内完成。这远低于预填充 worker 实际计算该 KV 所需的数百毫秒。因此，预填充 → 传输 → 解码的流水线实现了良好的并行性，因为解码几乎可以在预填充完成后立即开始——无需长时间停顿。

发送 KV 缓存数据时要小心避免碎片化和开销。例如，vLLM 的 PagedAttention 把 KV 缓存存储在固定大小的 token 块中，通常是每块 16 个 token。KV 块相对较小（尽管每块的字节数随头数、头维度、层数和 dtype 缩放）。天真地

通过 RDMA 发送数千个小 KV 页会带来过多开销，因为每次传输都有固定的延迟和协议开销。这会导致带宽利用率低下。

现代 LLM 引擎支持多种页大小，例如每块 8、16、32、64 或 128 个 token。更大的页大小可以降低通过 RDMA 搬移 KV 时的传输开销，因为更大的拼合缓冲区和更少的工作队列元素（WQE）会改善链路的持续吞吐量。可能的话，每次 RDMA 写至少拼合 128 token 的页。务必在专用 CUDA 流上重叠传输。优先使用非阻塞流并使用事件栅栏。始终使用 Nsight Systems 等工具进行剖析以确认重叠。据 LMCache 报告，7.5k token 的 KV 在 RDMA 上拼合后，传输时间从约 20 ms 降至约 8 ms。

LMCache 扩展通过在传输前把 KV 页拼合成大的连续缓冲区来解决这一低效问题。本质上，它在 GPU 内存中把小块聚集成一大块，然后一次传输这个大缓冲区。

例如，如果把 7,500 token 的 KV 缓存作为 470 次小传输发送需要 20 ms，那么把它们拼合成更大的块（例如 128 token 页）可以把传输时间降到 8 ms。这个简单的批处理优化让网络管道保持满载，并减少了每包开销。

下面展示一个系统如何配置以实现快速的 GPU 到 GPU KV 传输。这是 LMCache 预填充-解码模式使用 NIXL 传输通道的示例配置：

```
# Prefill server config (lmcache-prefiller-config.yaml)
enable_pd: true
transfer_channel: "nixl"
pd_role: "sender"               # this instance sends KV data
pd_proxy_host: "decode-host"    # PD proxy / decode coordinator
pd_proxy_port: 7500            # control-plane port on the proxy/decoder
# size the buffer to the KV you plan to transfer
# FP8/FP4 KV should shrink it significantly
pd_buffer_size: 1073741824      # 1 GiB transfer buffer size
pd_buffer_device: "cuda"        # buffer stays in GPU memory
```

这里，预填充服务器被配置为 RDMA 发送方。它以解码主机的 7500 端口为目标，分配了一个 1 GB 的 GPU 缓冲区用于 KV 传输。解码服务器在该端口上被配置为接收方，配有一个匹配的 1 GB GPU 缓冲区，如下所示：

```
# Decode server config (lmcache-decoder-config.yaml)
enable_pd: true
transfer_channel: "nixl"
pd_role: "receiver"            # this instance receives KV
pd_peer_host: "0.0.0.0"        # bind address for NIXL peer
pd_peer_init_port: 7300        # NIXL handshake/control port
pd_peer_alloc_port: 7400       # NIXL allocation/data port
pd_buffer_size: 1073741824     # 1 GiB (match sender unless you plan to
 ↪  shard)
pd_buffer_device: "cuda"       # keep buffer in GPU memory
nixl_backends: [UCX]           # UCX backend is sufficient for disagg
```

这种配置允许预填充把 KV 缓存直接写入解码 GPU 的内存——每次传输最多 1 GB——无需 CPU 干预。双方都把传输缓冲区放在 GPU 内存中，以实现零拷贝操作。

在确定传输缓冲区大小时，从 `pd_buffer_size` = 1 GB 开始。对于一个有 700 亿参数、80 层、32 个头、头维度 128 的模型，这大约是 FP16 KV 缓存 4–8k token 的估算值。如果提示超过约 7.5k token，请使用 2 GB。你可以按 dtype 和头数缩放：bytes $\approx 2 \times L \times N \times (H \times D_h) \times$ bytes_per_val。务必在传输前拼合页面。这将避免小 I/O 低效。

如果把 KV 缓存量化到 FP8 或 FP4，固定 token 数所需的传输缓冲区会减小，因为每 token 的字节数相应减少。因此，你可以在每个缓冲区中传输更多 token，或相应减小缓冲区大小。1–2 GiB 的缓冲区适用于许多部署，但应按上面的 KV 公式确定大小并向上取整到 256 MB 边界。如果使用 FP8 或 FP4 KV，可以按比例缩小缓冲区。始终针对你将传输的最大拼合页组进行验证。优先使用 GPUDirect RDMA 并将页面拼合到至少 128 token，以获得最佳链路利用率。

实践中，可以使用下面的 shell 脚本通过 CLI 启动解码服务器。这将减少 eager 碎片化并鼓励在更大缓冲区上使用 rendezvous：

```
# Example decode worker
# (select device by index or UUID)
```

```
UCX_RNDV_THRESH=16384
UCX_MAX_EAGER_RAILS=1
UCX_TLS=cuda_ipc,rc,rdmacm,cuda_copy,cuda_ipc,tcp \
CUDA_VISIBLE_DEVICES=1 \
LMCACHE_CONFIG_FILE=lmcache-decoder-config.yaml \
python run_vllm_decoder.py --port 8200
```

你可以类似地在另一个 GPU 上用其配置文件启动预填充服务器。这些设置确保系统在节点之间使用 InfiniBand RDMA、在节点内使用 NVLink 对等传输，而不是用标准 TCP 套接字进行 KV 传输。

对于单节点多 GPU 运行，你应该启用 CUDA IPC。跨节点运行时，优先使用 RDMA。LMCache/vLLM worker 的典型 UCX 配置是设置 `UCX_TLS=rc,rdmacm,cuda_copy,cuda_ipc,tcp`，并确保在网络结构上应用 RoCE/IB 无损设置（ECN/PFC）。对于节点间 RDMA，考虑 `UCX_RNDV_THRESH=16384`，使大 KV 缓冲区使用 rendezvous，小 KV 缓冲区使用 eager。始终用 `ucx_info -f` 进行验证。

有了 RDMA 和恰当的缓冲，交接延迟可以在个位数到几十毫秒之间，具体取决于互连和页大小。例如，对于 7,500 token 的上下文，LMCache 测得使用许多小传输时约为 20 毫秒，拼合成更大块后约为 8 毫秒。具体来说，建议在 RDMA 前把 16-token 的页拼合成至少 128 token 的板片。这将有助于减少每包开销。

简而言之，解聚系统应使用快速互连和智能数据拼合，使预填充 → 解码的过渡无缝而快速。最小化交接时间至关重要，因为如果交接很慢，并行化各个阶段的好处从一开始就被抵消了。

> **注** 在多进程运行中，通过设置 `export PYTHONHASHSEED=0` 为 KV 块路由使用确定性哈希。

18.4 连接器与数据通路设计

在零拷贝优化的基础上，我们来看看预填充和解码节点如何端到端地协调传输——而不仅仅是搬移比特。预填充和解码 worker 通常通过调度器或路由器

通信。实践中，这个调度器往往实现为集中式组件（如 NVIDIA Dynamo 所用），或去中心化的协调方式（如 SGLang 所用）。

例如，NVIDIA Dynamo 实现了一个全局调度队列，解码 worker 把新的提示任务推入队列，由预填充 worker 消费。在这种设计中，解码节点把提示处理请求入队，如图 18-6 中的“Put RemovePrefillRequest”（第 6 步）所示。

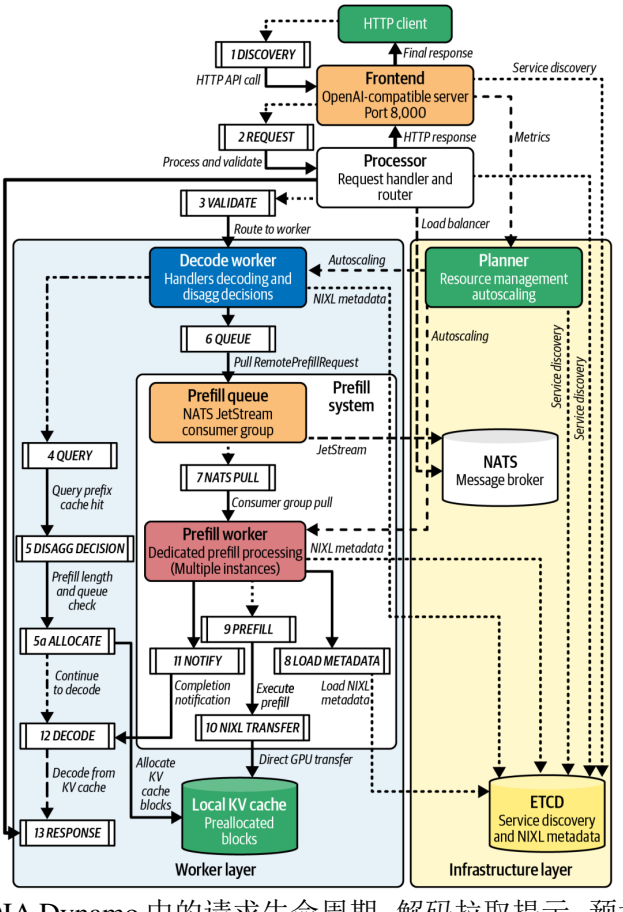

图 18-6　NVIDIA Dynamo 中的请求生命周期：解码拉取提示，预填充通过 NIXL 推送 KV。

预填充节点领取这个请求，完成后，由于请求携带了解码节点的 origin 或 reply-to ID，它确切知道要把结果发送到哪个解码节点。随后 KV 通过 NIXL RDMA 直接传输到该解码 worker 的 GPU。

在 vLLM + LMCache 实现中，使用的是更去中心化的方式。解码和预填充进程为每个请求的 KV 建立直接通道，使用 pipe 或 buffer。在底层，这可能使用在请求开始时协商的一对一 TCP 或 RDMA 连接。它不是全局队列，而是每个请求建立自己的传输通道。两种方式各有利弊。全局队列更便于负载均衡和故障处理。直接通道可以最小化排队。

在决定使用哪种模式时，要考虑你的工作负载和基础设施约束。如果你需要健壮的多租户负载均衡和简单的故障转移，并且能接受少量排队延迟，全局队列模型通常是更好的选择。反之，如果你有严格的尾部延迟要求、相对稳定的解码-预填充配对和高带宽互连，按请求的直接通道方式可以最小化跳数和抖动。

实践中，在预期的请求组合下对两种设计进行基准测试。改变提示长度、并发级别和故障场景，看哪种为你的 SLO 提供最佳的延迟-吞吐权衡。

关键的设计目标是让流水线非阻塞且高吞吐：在一个请求被解码的同时，另一个提示可以开始预填充——同时还有另一个请求的 KV 在传输途中。因此，只要其他阶段有工作可做，任何阶段都不会空闲。这正是解聚能在规模化场景下提升整体吞吐量的确切原因——所有阶段都保持并行忙碌。

通常，预填充产生的首个生成 token 的 logits 往往不会被显式传输，因为解码 worker 可以直接从 KV 重新计算首个 token 的概率。有些系统确实会传输首个 token 的输出，以省去解码 worker 中几百微秒的额外计算。但另一些系统保持简单，只传输 KV，让解码 worker 重算最后一层。

确保这条流水线对故障具有健壮性非常重要。如果解码节点在生成中途失败，前面讨论过的全局 KV 缓存池可以让另一个节点使用保存的 KV 从断点继续。类似地，如果预填充节点在提示处理中途失败，该提示可以在其他地方重试。连接器设计应优雅地处理这些故障，使单个节点的失败不会让整个请求出错。

> **注** 这些路由器通常使用心跳检查和超时，因此如果预填充到解码的传输停滞，请求可以被重新分配或安全中止。

18.5 预填充与解码的异构硬件和并行策略

解聚的一个强大优势是可以自由选择不同的硬件——甚至不同的模型并行配置——分别满足预填充和解码集群的需求。在统一的单体部署中，你通常只能为两个阶段使用一种硬件类型和配置。有了解聚，你可以按阶段混合搭配硬件和策略，如下所述。

18.5.1 计算优化与内存优化硬件

预填充阶段受益于具有高计算吞吐量、大量 TFLOPS、专用 Tensor Core 和高时钟频率的 GPU。它也受益于可观的内存带宽，但除了提示的 KV 缓存所需之外，并不一定需要巨大的 HBM 容量。

另一方面，解码阶段受益于大内存容量和内存带宽，因为它要处理许多 token 的 KV。它不需要极端的计算能力，但多多益善。

这为每个阶段使用不同代际的 GPU 开辟了可能性。例如，一种设计可以为预填充集群使用最新的高计算 GPU，而为解码集群使用内存带宽充足的旧一代或高性价比 GPU。

这样，我们可以避免把最新 GPU（例如最新 Tensor Core）浪费在解码这类无法发挥其全部潜力的任务上。预填充任务往往通过拉满 GPU 数学单元消耗更多功率，而解码任务在同一 GPU 上消耗的功率要少得多。

18.5.2 吞吐量与成本收益

在异构硬件之间划分阶段可以提高单位成本的吞吐量和单位功率的吞吐量。在 Splitwise 研究中，使用分阶段专用硬件的某个配置相比同构基线实现了 1.4 倍的吞吐量提升，同时成本降低 20%。

在另一个以固定成本/功率预算下最大性能为目标的配置中，他们在相同成本和功率下实现了 2.35 倍的吞吐量提升。具体来说，该研究为预填充使用了 4 块 H100（高计算），为解码使用了 4 块 A100（高内存）。

这个混合配置在相同成本/功率下，实现了约 2.35 倍于 8 GPU 同构系统（全部 H100 或全部 A100）的 RPS。

另外，他们发现，要匹配基线吞吐量，异构系统实际上可以使用更少的 GPU（例如五到六块而不是八块），方法是把解码卸载到更便宜的 GPU 上。这凸显了节省成本的机会，并展示了在每种 GPU 最有效的地方使用它的价值。具体来说，你可以在每美元计算能力最高的 GPU（例如 Blackwell 或 Rubin 代）上执行计算受限工作，把内存受限工作分配给内存带宽合适的、更具成本效益的旧一代 GPU（例如 Hopper 或 Ampere）。

Splitwise 的评估考虑了异构 GPU 之间状态传输的开销。该测试通过 NVSwitch 网络结构传输 KV 数据，产生的开销极小——即使在代际不同的 GPU 之间也是如此。这表明 NVSwitch 和 NVLink 等高带宽互连可以实现预填充/解码解聚，对性能的影响可以忽略不计——即使在混合 GPU 配置中也是如此。

另一个系统是 HexGen-2，这是一个分布式推理框架，把异构 GPU 上的解聚推理资源分配当作一个优化问题来处理。它的调度器联合优化资源分配、分阶段并行策略和通信效率。

在 Llama 2 70B 等模型上的实验中，HexGen-2 在相同价位下相比最先进的系统实现了最高 2 倍的服务吞吐量提升（平均约 1.3 倍）。此外，它以大约低 30% 的成本实现了与高端基线相当的吞吐量。这些改进来自混合 GPU 类型和优化工作划分。这基本上是以自动化方式完成了 Splitwise 在概念上所做的工作。

这些结果证实，解聚不仅仅是关于速度，还关乎效率——用更少的资源做更多的事。在云环境中部署推理时，对于一个服务数百万甚至数十亿终端用户的大型推理服务，这可以带来数百万美元 GPU 时间的显著成本节省。

例如，你可以用 6 块 GPU（预填充 + 解码混合）而不是 8 块顶级 GPU 来满足相同的流量；这可以为该服务节省大约 25% 的硬件成本。因此，解聚让你在相同硬件上服务更多用户。这一点至关重要，因为 GPU 供应往往有限——尤其是最新 GPU。

考虑到包括美国在内的世界某些地区的电力限制，能源效率也很重要。Splitwise 通过把解码任务运行在低功耗 GPU 上——速度略有下降——证明了更好的功率效率。

通过把预填充和解码任务分配到不同类型的硬件上，你可以选择每个阶段在哪里运行、如何运行，以提高性能并降低成本。解聚之所以允许这种灵活性，是因为各阶段是独立的。

简而言之，Splitwise、HexGen-2 以及相关异构部署研究的评估表明，除了纯粹的速度之外，解聚还可以用于成本优化。通过让硬件与工作负载匹配，你可以显著降低每次查询的成本，同时在固定预算内提高性能。

对于大规模服务，这对保持经济可行性至关重要。权衡之处包括系统复杂度略微增加，因为你需要管理多种 GPU 类型。而且由于 GPU 能力不匹配，你在集群配置灵活性和跨预填充/解码任务动态重新分配 GPU 方面会受到限制。但在许多情况下，为每个阶段使用不同硬件可能值得效率上的收益。

18.5.3 分阶段的模型并行

另一种异构形式和分阶段专业化是为每个阶段在 GPU 之间选择不同的模型并行（例如张量并行、流水线并行等）。这对于因内存约束而分片到多个 GPU 的超大模型很有意义。

在传统设置中，你可能以固定并行策略运行模型，并同时对预填充和解码阶段使用张量并行或流水线并行把模型拆分到多个 GPU。但预填充的最优并行策略可能与解码不同。

例如，预填充阶段是对 N 个提示 token 的大规模前向传播，它受益于高度并行化。你可以跨许多 GPU 使用张量并行（TP）来更快地完成计算并降低 TTFT。

同步 GPU 的开销被一次可以处理的大量 token 摊薄。这缩短了该阶段的墙钟时间，这对 TTFT 至关重要。

你甚至可以使用流水线并行（PP）进一步加速预填充并提高吞吐量。这会把模型层拆分到多个 GPU，并让提示流经多个流水线阶段。

另一方面，解码阶段是顺序执行的，并且对每一步的延迟敏感。为一个解码任务使用过多 GPU 实际上会损害每输出 token 时间（TPOT）延迟，也叫 token 间延迟（ITL）。这是因为每个 token 步骤都需要额外的多 GPU 通信开销。因此，加速的潜力有限，因为一次只有一个 token 的计算量可供拆分（如果使用推测解码，则是少数几个 token）。

解聚使混合这些方法成为可能：为一个阶段使用 TP，为另一个阶段使用 PP——或者使用每种技术的不同程度。例如，你可以用 TP=8 运行预填充以跨越 8 个 GPU 并最小化提示延迟。然后你可以用 TP=1（单个 GPU）运行解码，以最大化每 token 吞吐量并最小化步进延迟。

这样，每个阶段的吞吐量和延迟都可以单独调优，如图 18-7 所示。

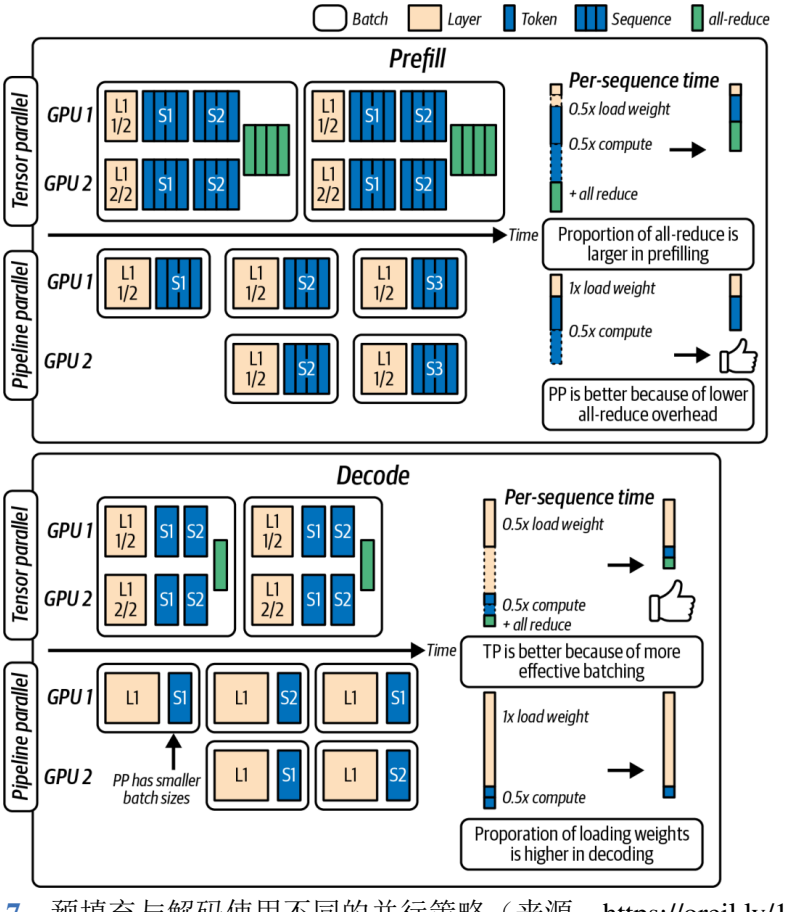

图 18-7　预填充与解码使用不同的并行策略（来源：https://oreil.ly/1-Ti0）

在这里，张量并行额外的 all-reduce 通信开销在预填充阶段更为突出，因为有大量 token 被并行处理。因此，我们选择流水线并行，因为它对我们的预填充工作负载更高效。

张量并行和流水线并行对预填充都有效。接下来的示例为预填充使用流水线并行。不过，在某些集群上，较大的张量并行度可以降低 TTFT。最佳选择取决于网络带宽、集合通信延迟和模型形状。

然而，对于解码阶段，流水线并行会导致更多（尽管更小的）前向传播，因为 token 在 GPU 之间传递。仅仅为了生成一个 token，就需要大量数据进出 GPU。因此，我们选择张量并行，因为它更适合我们的解码工作负载。

不过，这引入了一个复杂问题。由于我们的模型在预填充和解码之间的并行化方案不同，KV 缓存的格式也必须不同。例如，如果预填充阶段使用 TP = 1（因为它用的是 PP）并使用四块 GPU，那么四块 GPU 中的每一块都有一个完整的 KV 张量。

假设解码阶段使用 TP = 4。在这种情况下，每块 GPU 只期望 KV 张量的 1/4，因为数据应沿模型的隐藏维度拆分。为了处理这个问题，NVIDIA Dynamo 等系统可以在传输期间执行 KV 转置（转换）。本质上，它把 KV 缓存从 [TP_p 份] 转换为 [TP_d 份] 所需的格式，其中 TP_p 是预填充的并行度，TP_d 是解码的并行度。

Dynamo 包含一个高性能内核，在从 NIXL 读出后、写入解码 worker 内存之前，即时执行这个转置。这样，接收端解码器就能得到它期望布局的 KV 缓存数据。

不过，与网络传输相比，这个转置的开销可能很小——尤其是考虑到 NVLink 的吞吐量可以快速处理这些数据重组。在这种情况下，使用为每个阶段优化的不同并行策略所带来的计算节省，足以证明其合理性。

让我们探索一个分阶段并行的示例。考虑一个可以应用各种并行方案的大模型：张量（TP）、流水线（PP）、数据（DP）、序列（SP，跨 GPU 拆分序列）等。我们可以选择分开的并行配置。表 18-1 展示了预填充阶段的一个并行配置示例。

这些数值在最小化 GPU 间开销的同时，仍使用多个 GPU 加速大型提示。接下来，我们看一个解码阶段的并行策略示例，如表 18-2 所示。

例如，如果模型无法放入单个 Blackwell B200，解码优先使用 TP_d = 2 或 4，而不是使用 PP。这将有助于避免流水线气泡。

表 18-1 预填充并行示例。

并行策略	符号	值	说明
张量并行	TP_p	2	将模型的权重张量拆分到 2 个 GPU 上，在通信开销可控的前提下将预填充延迟减半。
流水线并行	PP_p	2	将模型层划分为两个流水线阶段，让微批在深层模型的每个阶段之间流式处理。
序列并行	SP_p	1	除非处理超长上下文，否则不跨 GPU 拆分输入序列（不做序列分片）。
上下文并行	CP	1	优化后上下文可放入内存时，将整个上下文保留在单个 GPU 上（不做上下文级划分）。
数据并行	DP_p	1（或 2）	每个 GPU 一份模型副本（或用两份，通过权重复制让批处理提示的吞吐量翻倍）。

表 18-2 解码并行策略示例。

并行策略	符号	值	说明
张量并行	TP_d	1（默认）... N（GPU 数）	默认 $TP_d = 1$，保持简单、同步开销最小。$TP_d = N$（GPU 数）可提升小批量上微小 GEMM 的效率，或在单个 GPU 放不下模型时使用。
流水线并行	PP_d	1	流水线并行会给单 token 解码带来气泡，因此 $PP_d = 1$ 可避免空闲阶段。
序列并行	SP_d	1	跨 GPU 拆分输出序列并不常见。$SP_d = 1$ 让每条解码流保持本地，除非处理超长输出。
数据并行	DP_d	1	每条解码流每个 GPU 一份模型副本。用独立副本处理并行请求，而不是为单条流做复制。

理想情况下，每条解码流在单个 GPU 上运行，避免跨 GPU 开销。这只有在模型能放入 GPU 内存时才可能。在这种情况下，TP_d = 1，意味着解码期间不使用任何张量并行。如果模型无法放入

内存，你可以提高张量并行的度（例如 TP_d = N，其中 N 是 GPU 数量）。

如果系统正在为处理小批量而发出微小 GEMM，提高张量并行度也很有用。这是因为把小型矩阵乘法分布到多个设备上可以把通信延迟隐藏在计算之后，并有可能带来更高的整体吞吐量。

这些是说明性数值，但重点是解聚允许你在提示侧单独配置资源以达到 TTFT 目标。同时，你可以独立调整解码侧的资源，以达成将 token 流式返回给终端用户的吞吐量和延迟目标。

这样，两种并行策略互不干扰。在统一系统中，如果你想这样做，就必须选择一个对两个阶段都次优的折中策略。

18.5.4 预填充与解码使用不同精度

一些推理引擎允许你在预填充和解码之间使用不同的精度。例如，你可以用较低精度（如 FP8、INT8 或 FP4）执行预填充以加速它。同时，如果需要更好的生成精度，你可以用较高精度解码。

一般来说，你会让两者以相同精度运行，以便预填充阶段计算的 KV 缓存可以在解码阶段使用。不过，你可以应用与上一节所述并行度转换类似的转换。你会选择在发送前量化 KV 并将其从 FP16 压缩到 INT8/FP8/FP4。然后，如果需要，在接收端再转换回来。

例如，你可以选择在网络上发送较低精度的数据以加速传输。或者，你可以根据可用的 FLOPS 等因素选择在发送方或接收方执行转换。

这些都是高级想法。但它们凸显了一点：在解聚设置中，几乎每个方面——硬件类型、GPU 数量和精度——都可以为每个阶段独立调优。

18.6 GPU-CPU 协同的混合预填充

到目前为止，我们一直假设预填充和解码阶段都运行在 GPU 上——可能是不同类型的 GPU。然而，在极端规模下——或使用超大模型和提示时——值得评估 CPU 是否能分担 GPU 的压力。

对于神经网络计算，现代 CPU 比 GPU 慢得多，但它们具有其他优势，例如充足的 RAM、不与 GPU 争抢内存带宽，以及处理 GPU 可能不太擅长的任务的灵活性，比如

超长序列以及分词、填充等非 transformer 运算。

使用混合预填充策略，部分预填充计算在 CPU 上完成。一种场景是超长提示的 CPU 卸载。考虑一个来自大型文档附件的、有数万 token 的提示。由于内存约束，即使是强大的 GPU 也可能难以处理这么大的提示。

在超长提示的情况下，系统可以选择在拥有大量 RAM 的 CPU worker 上执行模型的初始层，以容纳长序列。随后把中间结果流式传输到 GPU——如果延迟不是问题，甚至可以在 CPU 上完成整个预填充。解码 GPU 随后会从 CPU worker 接收庞大的 KV 缓存。

虽然在交互式推理中不常见，但一些批处理或离线流水线——例如长时间运行的“Deep Research”任务——可以对超长文本使用 CPU 预处理。

CPU 卸载的一个实际用途是处理后台或低优先级预填充任务。例如，LLM 服务可能允许为离线处理非交互式请求而提交非常大的提示。这些任务可以分配给纯 CPU worker，最终馈入解码 GPU 进行快速 token 生成。由于 CPU 较慢，延迟会很高，但由于是离线作业，这可能是可以接受的。而且这种配置把 GPU 资源腾出来用于更具交互性的工作负载。

混合预填充在 Grace Blackwell 这类 CPU-GPU 超级芯片上更常见，因为芯片到芯片的互连非常快。而且它利用了 CPU 内存远大于 GPU 内存这一事实。

想象把庞大的 KV 缓存存储在 CPU 内存中，并让 GPU 快速访问。混合预填充会使用 CPU 的内存缓冲或预处理输入 token，而 GPU 则专注于繁重的 transformer 层。

Grace Blackwell 超级芯片可以处理海量上下文：让 CPU 管理内存和初始层，让 GPU 对序列块执行稠密注意力。Grace CPU 还可以用于把 HBM 中放不下的 KV 缓存数据溢出到 CPU DDR 内存。这有效扩展了 GPU 可支持的上下文长度。

你可以跨设备切分 transformer：在 GPU 上运行前 N 层（在那里处理大部分 token，序列基本被压缩），然后把接下来的 M 层卸载到拥有大量内存的 CPU 上。

最后把剩余的层带回 GPU 以生成最终输出。

这种分层划分技术增加了显著的数据搬移和编排复杂度——只有在极少数情况下（如超长上下文或严重的 GPU 内存限制）才应使用。无论如何，它展示了如何在极端推理场景中突破硬件边界。

在我们的解聚架构中，引入 CPU 意味着引入第三种 worker：CPU 预填充 worker。调度逻辑随后可以在三个选项中选择：GPU 预填充 worker、CPU 预填充 worker，或本地解码 GPU 预填充。决策取决于提示长度或优先级等因素。

例如，策略可能是：如果 `prompt_length` > 5,000，则路由到 CPU 预填充 worker，因为知道它会慢，但至少不会占用 GPU，并且可以使用大内存。解码阶段随后要等待 KV 更长时间。或者在极端情况下，如果完全离线，解码也可以在 CPU 上完成。

一般来说，CPU 卸载会增加 TTFT，因此不用于常规的延迟敏感请求。它更像是一个扩展性和安全网特性。如果使用了它，系统应监控这条路径被采用的频率，因为频繁的 CPU 卸载可能表明需要更多 GPU 容量或模型优化。

CPU 卸载还允许系统处理超长输入或 GPU 全部繁忙时的突发等边缘情况。它通过回退到较慢的 CPU 而不是完全失败来实现这一点。不过，请记住，如果处理太慢且超过 SLO 要求，最好快速失败。

从成本角度看，把一些工作交给 CPU 可能更经济，因为 CPU 核心比 GPU 小时更便宜。一些云提供商可能为 LLM 服务运行 GPU 和 CPU 实例的混合。CPU 在调用 GPU 之前执行输入预处理——甚至小型模型推理。

简而言之，虽然核心的解聚逻辑专注于在 GPU 之间分配工作，但健壮的超大规模推理系统可以创造性地利用 CPU 处理工作负载的某些部分。随着硬件演

进并走向 Grace Blackwell 这类紧耦合的 CPU-GPU 超级芯片设计，使用 GPU 和 CPU 之间的界限将变得模糊。

高效调度应考虑所有可用的计算资源。然而，指导原则保持不变：让 GPU 做它最擅长的事，包括对中等序列长度进行大规模并行计算；在 GPU 可能低效时使用 CPU，例如超长序列、内存密集型任务或低优先级任务。

18.7 SLO 感知的请求管理与容错

要在超大规模下达成 SLO 目标，仅仅高效地扩展和调度是不够的。有时你还需要拒绝或推迟在当前负载下会违反 SLO 的工作。我们在 Mooncake 的早期拒绝中已有所涉及。

18.7.1 早期拒绝（准入控制）

早期拒绝，或称准入控制，在第 17 章中介绍过。简而言之，它意味着如果系统预测自己无法在延迟目标内服务某个请求，它就会快速失败，返回错误或响应“请稍后再试”。这种预测可以基于当前队列长度、近期吞吐量，甚至一个预测响应时间的轻量级 ML 模型。

早期拒绝与把请求排队、然后错过其 SLO 截止时间形成对比。它通过确保推理系统服务的请求都能满足其保证来保护有效吞吐量（goodput）。

在 Mooncake 中，早期拒绝策略对照预填充和解码集群的估计负载评估传入请求。例如，考虑一个正在处理大量长序列的解码集群。

当新请求到达时，系统根据其预期输出长度判断解码利用率是否会超过安全阈值。此时，系统会立即拒绝或推迟该请求，并释放 GPU 节点上的资源，如图 18-8 所示。

这样做可以防止出现这种情况：请求在队列中等待，然后耗时过长以至于超过其延迟 SLO，并通过消耗稀缺的计算或内存带宽资源拖慢所有其他请求。这再次印证：与其默默接受然后无法兑现延迟保证，不如快速返回“太忙”的响应。

这类似于 Web 服务器在极端过载时卸掉部分负载，以可接受的延迟继续服务剩余请求——也就是令人畏惧的 HTTP 503 错误。这比让所有请求都超时要好。

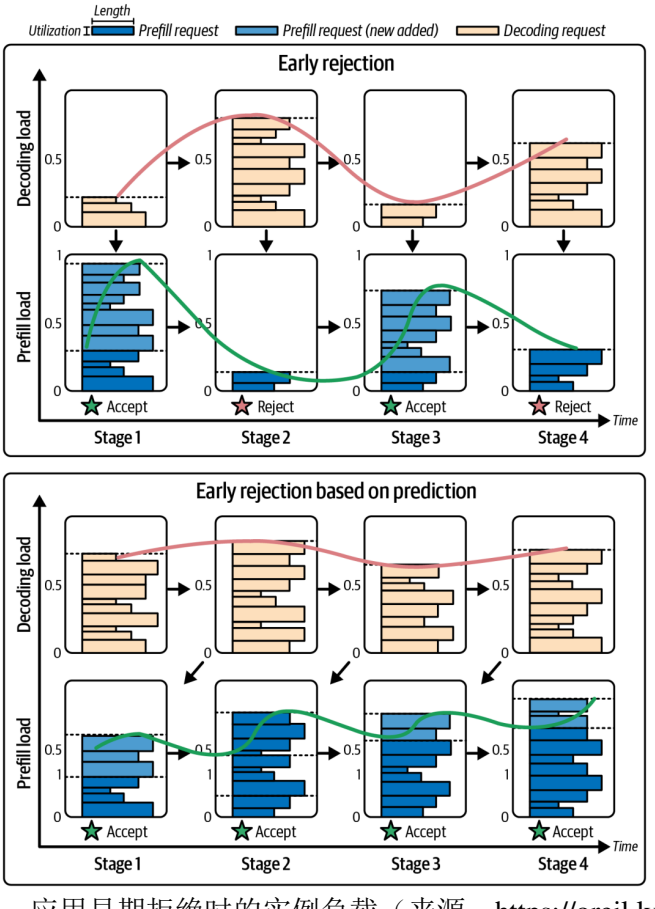

图 18-8　应用早期拒绝时的实例负载（来源：https://oreil.ly/2xtK-）

在 LLM 服务中，由于请求的大小和持续时间差异很大，一个明确的准入控制步骤有助于为被接受的请求保持性能。行为良好的准入控制让系统保持在一个既能满足所接受负载的 TTFT 目标又能满足 TPOT 目标的状态。

它有效地对并发工作量设定了上限。如果做更多就会破坏 SLO，它就不会尝试。这必须与扩缩容策略结合。例如，你可以扩容而不是拒绝。然而，由于扩容通常不是瞬时的——或者你已经处于最大容量——从长远来看，拒绝是最安全且性能最好的选择。

18.7.2 服务质量

SLO 感知管理的另一个方面是服务质量（QoS）差异化。一些请求可能带有优先级。例如，付费客户的优先级高于免费层客户。或者交互式查询的优先级高于离线批处理请求。

调度器可能优先处理某些请求以确保始终满足 SLO——即使其他请求必须等待或被丢弃。解聚可以通过把一部分预填充和解码 worker 专用于高优先级任务来辅助这一点。例如，你可以选择把集群容量的 10% 专门预留给高级层（premium）请求，30% 预留给标准层。

这种分层方法将为付费层（高级和标准）保证余量。它确保这些请求不会被低优先级（免费层）请求拖慢。下面是一个 NVIDIA Dynamo 的 QoS 配置示例，演示了这种分层调度：

```
# configs/qos.yaml
scheduler:
  # Define QoS classes and their reserved capacity
  qos_classes:
    - name: premium
      reserved_fraction: 0.10   # reserve 10% of prefill/decode workers
      priority: 100
    - name: standard
      reserved_fraction: 0.30   # reserve 30% for standard tier
      priority: 50
    - name: free
      reserved_fraction: 0.60   # share remaining capacity (free tier)
      priority: 10
# Map incoming requests to QoS classes by label
request_router:
  routes:
    - match:
        header: "x-customer-tier: premium"
      qos: premium
    - match:
        header: "x-customer-tier: standard"
```

```
    qos: standard
  - match:
      header: "x-customer-tier: free"
    qos: free
  - match:
      # default fallback
    qos: free
```

这种分层方法允许高优先级请求使用预留容量被接纳并以低延迟服务。如果系统忙于处理高优先级请求，低优先级请求将被排队——或可能被拒绝。

对于这类系统，延迟监控和反馈回路很有用，可以持续监控 TTFT 和 TPOT 的 p99 与 p99.9。如果系统看到这些指标逼近 SLO 限制，它可以触发扩容或拒绝新负载、通过限制最大输出长度降级请求、或临时减少负载等动作。

在解聚系统中，你可以分别观察预填充队列和解码队列。这有助于定位哪一侧是瓶颈。有了这些信息，系统可以相应调整：如果预填充积压，就停止接受新提示；如果解码饱和，就限制长输出推理请求。

你可能在使用 ChatGPT 时遇到过“随机”错误。这可能有多种原因，但其中之一可能就是这些“断路器”在系统负载很重时启动，为某些类型的请求卸掉负载。

18.7.3 容错

容错是运行健壮推理系统的另一个方面。在解聚系统中，如果解码节点在生成中途失败，我们之前讨论的 KV 缓存池化可以在另一个节点上实现恢复，因为 KV 已保存在池中。这样，另一个解码 worker 可以接手生成剩余的 token——也许会有轻微延迟。

如果 KV 没有保存，系统将不得不在另一个节点上重算预填充，然后继续解码。这就是为什么 vLLM 等系统会定期把 KV 缓存检查点化或复制到池中——即使并不严格需要解聚。这样做是为了防范故障。

除了框架级的 KV 快照外，托管 GPU worker 的 Linux 进程可以使用 cuda-checkpoint 加 Userspace Checkpoint/Restore（CRIU）进行挂起和快照，如第 5 章

所讨论的。这样，检查点可以恢复到同一种 GPU 芯片类型的另一个节点上，以最小化抢占或故障造成的工作损失。

使用 cuda-checkpoint 恢复预热的 GPU 内存，而不是每次启动都重新编译图和重新加载权重，可以减少推理引擎的冷启动延迟。

检查点化可能会损害延迟，但至少请求可以完成。预填充节点在完成计算并把 KV 数据发送到解码 worker 或 KV 缓存池之后发生故障，影响较小，因为 KV 数据已离开故障节点。但如果预填充 worker 在提示处理期间失败，提示任务需要在另一个预填充节点上重试。架构应优雅地处理这类故障，使单个节点的失败不会丢弃整个请求或造成大规模级联延迟。

简而言之，SLO 感知的请求管理确保即使在极端条件下，系统也能为它选择服务的负载保持性能保证。它通过智能地决定服务哪些请求、丢弃哪些请求、推迟哪些请求来实现。早期拒绝是一个具体例子，它在准入时使用负载预测来防止过载，并在 SLO 约束内保持高吞吐量。

SLO 感知的请求管理和容错——与目前为止讨论的所有其他策略（如扩展、调度和缓存）相结合——有助于在生产中维持严格的 SLO 目标。

解聚使这一切成为可能，因为它提供了清晰的控制点。你可以分别观察预填充和解码指标，并在需要的地方精确应用准入控制。例如，如果预填充集群积压，就停止接收新提示；如果解码集群满负荷，就停止允许长输出。

结果是更可预测、更稳定、更自适应的推理服务：当负载超过容量时，性能不会急剧下降。相反，它会优雅地拒绝过量负载并迅速重新平衡以应对不断变化的条件。

18.8 动态调度与负载均衡

一开始，你可以确定预填充与解码各投入多少资源。但随后你希望随着工作负载的变化动态调整这些资源划分。

使用纯静态配置时，如果工作负载组合发生变化、你需要更偏向提示密集型或生成密集型的划分，固定的比例就会变得次优。预填充和解码容量的最优平衡会随时间变化。某一时刻，大量新请求到达，你希望预填充占比更大。稍后，你可能有许多长时间运行的推理生成，你希望解码占比更大。

自适应调度和负载均衡机制旨在持续调优系统，使任何阶段都不会成为瓶颈。这在不断变化的条件下保持高 goodput。

vLLM、SGLang 和 NVIDIA Dynamo 等现代推理引擎都把负载监控和动态 worker 分配融入其平台。此外，许多云推理平台拥有使用类似原理的自定义自动扩缩容器和路由器。

18.8.1 自适应资源调度与热点预防

解聚设置的一个问题是预填充与解码集群之间的负载不均衡。如果预填充与解码 worker 的比例对当前负载组合配置不当，一侧可能饱和而另一侧利用不足。

例如，如果解码 worker 相对预填充不足，解码任务就会排队。这会提高 TPOT，尽管提示被快速处理。反之，如果解码配置过度而预填充配置不足，新请求可能要等待才能开始。这导致 TTFT 很高，而许多解码 GPU 闲置。

理想的情况是两侧都接近满负荷工作但不至于不堪重负。这样系统让预填充和解码都保持忙碌，但任何一侧的队列都不会增长。

静态的预填充-解码 worker 分配在现实条件下是不够的，因为工作负载在一天中可能变化很大。在一个长期运行的大规模推理系统中，输入长度和输出长度的组合可能逐小时显著变化。

例如，某一小时可能有大量长问题产生短答案（如摘要）。这需要更多预填充资源和更少解码资源。下一小时，你可能得到短问题产生长答案（如推理、网页搜索）。这需要较少预填充但更多解码。

换句话说，适用于一种场景的静态预填充与解码比例对另一种场景可能不是最优的。因此，最优的 X_p 与 Y_d 配置取决于工作负载，并会随时间变化。

为了解决这个问题，高级推理系统可以使用自适应调度算法，重新分配负载，甚至在运行时重新调整实例用途。下面讨论其中几种方法。

18.8.2 TetriInfer 的两级调度器

研究原型调度器 TetriInfer 在两个粒度上运行。首先，在单个请求层面，它根据当前负载把传入请求分配给特定的预填充和解码实例。这是我们预期的常规路由。其次，它监控全集群的资源利用率，并预测瓶颈可能在哪里形成。这可以主动转移工作以防止热点。

例如，调度器可能发现某个解码节点积压了大量超长序列。在它成为问题之前，调度器把其中一些序列路由到另一个更空闲的解码节点——即使通常不会路由到这个解码节点。这样，负载得到均衡。图 18-9 从执行时间线和架构的角度展示了现有系统与 TetriInfer 工作的对比。

在这里，你可以看到，通过队列长度、GPU 利用率趋势等预测资源使用情况，TetriInfer 的两级调度器平滑了集群中的负载，并防止任何单个节点过载。

TetriInfer 这个名字暗示着像俄罗斯方块一样“打包”请求，以填充 GPU 时间而不互相干扰。

18.8.3 Arrow 的自适应实例伸缩

另一种动态资源调度技术是 Arrow（不要与流行的 Arrow 数据格式混淆）。这是一种自适应实例伸缩技术，它利用了一个事实：解聚系统对工作负载变化的响应往往滞后。例如，如果输入与输出的分布发生变化，静态数量的预填充与解码 worker 不会立即调整。由于一侧成为瓶颈，这会导致 goodput 暂时受损。

Arrow 通过测量输入 token 速率与输出 token 速率——以及集群中每个 worker 池的积压情况——持续分析工作负载。然后它动态调整 worker 的分配，如图 18-10 所示。

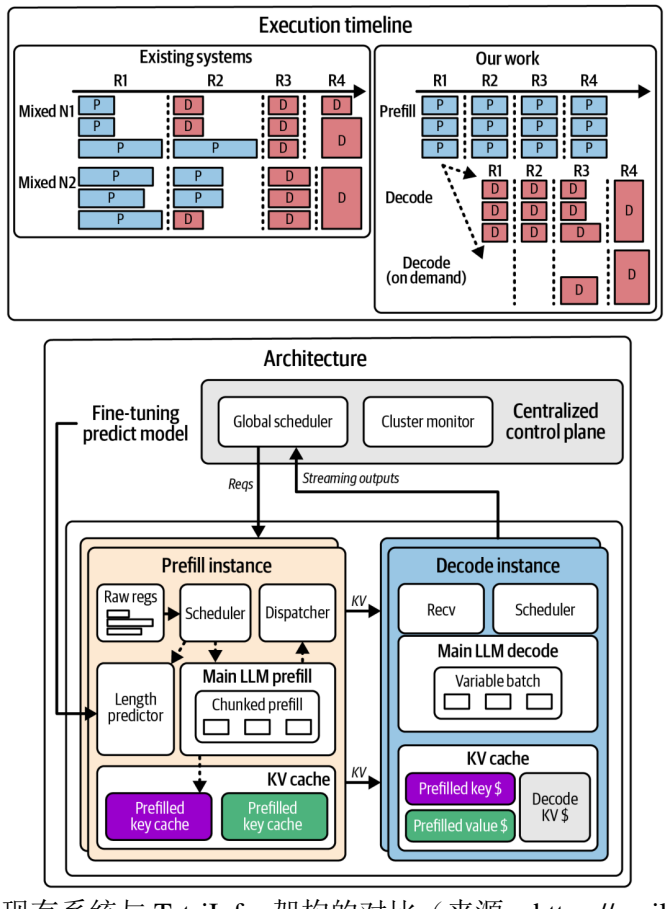

图 18-9　现有系统与 TetriInfer 架构的对比（来源：https://oreil.ly/_3KGj）

在云环境中，这可能意味着当输出负载增加时启动额外的解码实例。如果检测到输入密集型工作负载，你也可以缩减解码，转而增加预填充实例。

Arrow 的设计包括请求调度（决定哪个节点处理哪些请求，类似于其他调度技术）和实例调度（决定何时启动或关闭预填充或解码实例）。

Arrow 把预填充和解码 worker 的数量视为一个可调参数，可以近乎实时地调整和伸缩。Arrow 的设计把自动扩缩容逻辑带入了 LLM 推理调度器内部。

例如，如果大量输入密集但输出轻量的请求涌入系统，预填充节点可能成为瓶颈。在这种情况下，Arrow 会检测到预填充队列时间增长或 TTFT 百分位数

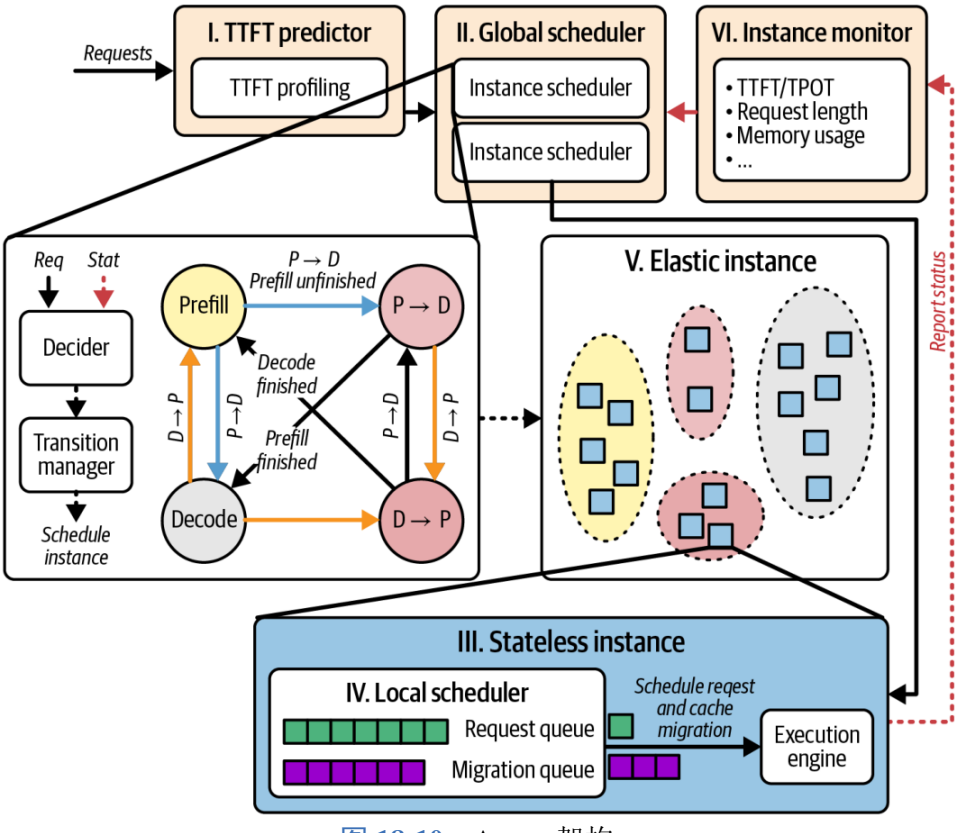

图 18-10　Arrow 架构。

上升，并决定把一些解码 worker 转换为预填充 worker——或启动额外的预填充实例来应对激增。

反之，如果一波输出密集、答案很长的请求（如推理链）到来，解码集群可能开始落后，表现为 TPOT 上升和解码队列变长。在这种情况下，Arrow 可以为解码阶段分配更多 GPU worker——如果提示到达放缓，也许可以临时闲置一些预填充节点。否则，它可以直接增加新的解码节点。

在 GPU 数量固定的静态本地集群中，动态伸缩将涉及任务重新分配：指示一些原本分配给预填充的 GPU 切换角色，加入解码池一段时间。如果每个计算节点都配置为既能作为预填充 worker 又能作为解码 worker 运行，这是可行的。

切换角色可能有开销，因为对于不同的并行策略或量化选择，模型可能需要加载不同的模型分区。有些设计在每个 GPU 上保留所有模型权重，只是根据

需求喂给它们不同的任务。这实际上把集群当作一个灵活的资源池：在任何给定时刻，一部分 worker 做预填充，另一部分做解码。

这开始类似于一个具有分时特性的统一集群，但粒度较粗：一个节点先花一些时间执行预填充任务，然后切换到解码任务。

在云部署中，动态伸缩还意味着与 Kubernetes Horizontal Pod Autoscaler 和 Cluster Autoscaler 等自动扩缩容器对接，为每种角色添加或移除 pod 和节点。例如，如果预填充负载持续偏高，Arrow 可以触发新的预填充 pod 启动。这形成了一个完全弹性、解聚的预填充与解码推理集群，两侧按需增长或收缩。

实践中，扩容新的 GPU pod 可能需要几十秒甚至更久。因此，系统在等待容量时可能需要卸掉部分负载。这就是 Mooncake 处理这种情况的方式，我们接下来讨论。

18.8.4 Mooncake 的自适应策略

除了 Arrow，Mooncake 系统也强调了自适应策略。Mooncake 引入了基于预测的准入控制，称为早期拒绝，它在某种程度上是互补的——管理需求与管理供给相对。

具体来说，如果系统预测自己没有足够容量在 SLO 内处理新请求，它就会拒绝该请求，而不是接受请求并可能违反 SLO。这是一种动态卸负载的形式，而非扩缩容，但它的目标是防止过载。

我们已经在上一节介绍了 Mooncake 的早期拒绝方法。这里的要点是：自适应既可以发生在供给侧（通过添加更多资源或重新分配资源），也可以发生在需求侧（通过节流或拒绝部分请求）。

动态伸缩的关键好处是在工作负载变化中保持高 goodput。通过自动调优预填充与解码的比例，系统避免了长时间的错配期——一侧 GPU 闲置而另一侧过载。

理想情况下，两类节点被均匀利用。这通过尽早缓解瓶颈来改善延迟——同时也提高了效率，因为你不用为大量闲置的一种 GPU 付费而另一种 GPU 却过载。相反，资源被重新分配或任务被转移以保持平衡。

就结果而言，自适应系统可能可以宣称：“应用自适应伸缩后，系统在静态系统无法应对的流量模式下保持了超过 90% 的 SLO 合规率，并且通过快速转移资源，在激增期间多处理了 X% 的请求。”

具体来说，Arrow 的结果提到，在工作负载极端变化的情景下，请求服务速率最高比非自适应系统高 5.6 倍。典型改进较低，但仍然显著。确切改进取决于工作负载变化的剧烈程度。

所有这些都表明，解聚消除了阶段干扰问题。而动态解聚又消除了固定分配造成的阶段失衡这一后续约束。要实现这种自适应性，调度器需要持续监控指标，包括预填充队列长度、解码队列长度以及 TTFT/TPOT 百分位数。在 K8s 部署中，这些指标通常通过 Prometheus 和自定义控制器送入控制面板。算法随后可以自动化决策。

复杂的推理系统使用 ARIMA 等预测模型和其他预测技术，利用一天中的时段模式预见激增并预测流量变化。如果根据历史上已知的使用模式预测晚上 9 点会出现长输出激增，

调度器就可以及时抢占性地分配更多解码容量。总体目标是保持高比例的请求在 SLO 内得到服务——而无需人工干预和重新配置。

18.8.5 动态资源伸缩

在预测式调度的思想基础上，动态资源伸缩具体指响应负载变化来改变预填充与解码之间的资源分布。例如，Arrow 的自适应实例调度会持续调整预填充与解码 worker 的数量。以下是一些在推理系统中平衡负载的方法：

18.8.6 弹性实例

在 Kubernetes 或类似系统中，你可以为每个部署定义自动扩缩容策略。例如，你可以指定希望预填充 GPU 的平均利用率保持在 70%。如果 GPU 利用率升高，系统会向预填充 worker 池添加一个 pod 或节点。如果利用率下降而解码利用率保持高位，系统可以把一个 pod 从预填充移到解码。这可以是基于规则的，也可以是算法式的，例如在每个时间间隔求解最优配置以选择新的 X 和 Y 数量。

18.8.7 实例“翻转”机制

TetriInfer 论文描述了一种“实例翻转”，即一些节点在需要时可以翻转角色。这要求模型被恰当地加载、分片和量化，以胜任两种角色。

18.8.8 无状态化以实现弹性

Arrow 利用无状态实例。因此，worker 上不存储长生命周期的会话状态。这样，它们可以被自由地重新分配。活跃请求的 KV 缓存使解码节点在 token 生成中途难以翻转为预填充，所以你通常必须等到解码完成其任务后再切换角色。

18.8.9 稳定性与振荡

快速切换角色可能导致振荡和抖动。因此，通常的做法是为角色设置最短持续时间，以避免过于频繁地翻转。

除了负载均衡，另一个需要考虑的方面是多租户或混合工作负载。如果集群服务不同的模型或任务，你甚至可以根据需求在它们之间重新分配 GPU——并超越单一模型的范畴。

解聚的模块化配置允许使用空闲的解码 GPU 来运行另一个较小模型的推理任务。截至本书写作时，这种扩展尚未成为主流，但随着推理引擎的发展及其设计变得更加灵活，这在概念上是可能的。

底线是：超大规模推理系统需要一个反馈回路，持续把资源分配与当前工作负载匹配。TetriInfer、Arrow、Mooncake 等系统表明，使用这种反馈回路并适应负载变化可以带来显著改进。这凸显了：解聚消除了干扰，而自适应解聚消除了失衡。在动态条件下实现最佳性能，两者都不可或缺。

18.9 关键要点

本章介绍了各种技术，如统一超级内核、高效内存分配、快速数据传输、预填充/解码解聚、KV 缓存池、动态伸缩和持续的 SLO 感知。以下是一些关键要点：

18.9.1 加速解码阶段

使用 FlashMLA、ThunderMLA 和 FlexDecoding 等融合注意力内核，可以大大加速解码阶段。这些内核可以提高单 token 吞吐量和 GPU 利用率。

18.9.2 把 KV 缓存当作一等公民

通过解聚在 GPU 之间共享 KV 缓存。复用前缀以避免冗余计算。这些由全局缓存池和哈希来实现。

18.9.3 追求预填充与解码 worker 之间的近零开销

利用 GPUDirect RDMA 和 NIXL 进行高速 GPU 到 GPU 传输。重叠计算/传输，以实现预填充与解码 worker 之间的近零开销。

18.9.4 为每个阶段采用专用硬件与并行策略

预填充与解码解聚允许每个阶段使用专用硬件和并行策略。例如，预填充使用高计算 GPU 或多 GPU 节点，解码使用内存充裕的 GPU 或单 GPU。目标是降低成本并提高吞吐量。

18.9.5 使用自适应与动态算法优化系统

自适应调度和 SLO 感知控制（如早期拒绝和动态伸缩）对于在变化的负载下维持延迟保证——以及在不超载的情况下充分利用所有 GPU——是必要的。

18.10 小结

超大规模 LLM 推理需要一种整体性方法，既包括高层的自适应资源管理，也包括底层的内核与内存优化。通过结合本章介绍的技术，一个高度优化的推理部署可以在现代硬件上实现最大吞吐量，同时满足严格的延迟保证。

随着硬件持续演进（例如内存增大和专用推理核心的出现）以及软件框架变得更加成熟（例如动态路由和灵活的角色分配），这些优化将相互叠加，变得更加强大。这将使推理引擎能够高效地处理更大的模型、更长的上下文和更多的用户。

第 19 章 动态自适应推理引擎优化

在现代硬件上进行超大规模语言模型（LLM）推理，需要动态的运行时自适应，才能在不同条件下同时实现高吞吐量和低延迟。对模型服务优化而言，静态的“一刀切”方法已不再足够。

相反，最先进的模型服务系统会采用自适应策略，动态调整并行方式、数值精度、CUDA kernel 调度和内存使用。本章将探讨这些高级技术，包括动态并行切换、精度缩放、实时缓存管理以及基于强化学习（RL）的调优。

本章给出超大规模 LLM 推理的最佳实践，教你如何编排一个能监控自身性能并实时自适应、从而最大化效率的引擎。

19.1 自适应并行策略（TP、PP 与混合方案对比）

超大规模 LLM 需要模型并行（例如张量并行、流水线并行，或二者的混合方案）来把计算分散到多个 GPU。每种方案都有优缺点。表 19-1 汇总了针对特定推理流量模式推荐的并行策略。

表 19-1 常见推理流量模式与推荐并行策略的对应关系。

推理流量模式	推荐并行方式	理由
大量短请求（< 256 token，高 RPS）	数据并行/副本扩展	最小化 GPU 间通信；每块 GPU 运行副本处理独立请求（假设模型能放入单块 GPU 的内存）
少量长请求（≥ 8k token，低并发）	流水线并行（带微批）	通过把层切分到多块 GPU 来降低单请求延迟
混合负载（短请求 + 部分长请求）	混合动态（自动切换）	在单 GPU 上运行小型对话，并把长请求送入流水线以满足延迟 SLA
超大规模模型（> 1 块 GPU 内存）	张量 + 流水线混合	模型必须如此才能放下；在两个维度上平衡计算和内存
MoE 模型推理（稀疏专家选择）	专家并行	把各个专家分布到多块 GPU；每个请求只调用一部分专家，降低单设备内存和计算负载

数据并行加副本扩展策略会在每块 GPU 上复制完整模型，并把传入请求负载均衡到这些副本上。对于单次推理，这不需要 GPU 间同步，因为每块 GPU 都独立处理各自的请求。

在输入较小或中等规模且数量很多时，这种策略能以最小的通信开销最大化吞吐量。但是，如果模型无法放入单块 GPU 的内存，就不能采用数据并行。

张量并行（TP）是模型并行（相对于数据并行）的一种形式，它把模型矩阵（例如权重、层等）切分到多块 GPU 上，以加速矩阵乘法。不过，它引入了额外的 all-reduce 通信来让 GPU 保持同步。

流水线并行（PP）是另一种模型并行形式，同样会切分模型。但它不是切分单个模型层和矩阵，而是把整层分配给不同 GPU，以突破内存限制——前提是这些层能放进单块 GPU。PP 会以顺序阶段延迟的形式带来额外开销，这些延迟称为流水线气泡，如图 19-1 所示。

专家并行用于混合专家（MoE）模型架构：它让每个专家子网络独占一块 GPU，然后由一个轻量级门控网络把每个输入请求或 token 只导向路由器选出的 top-k 活跃专家。此时，每块 GPU 只处理它承载的那部分专家。

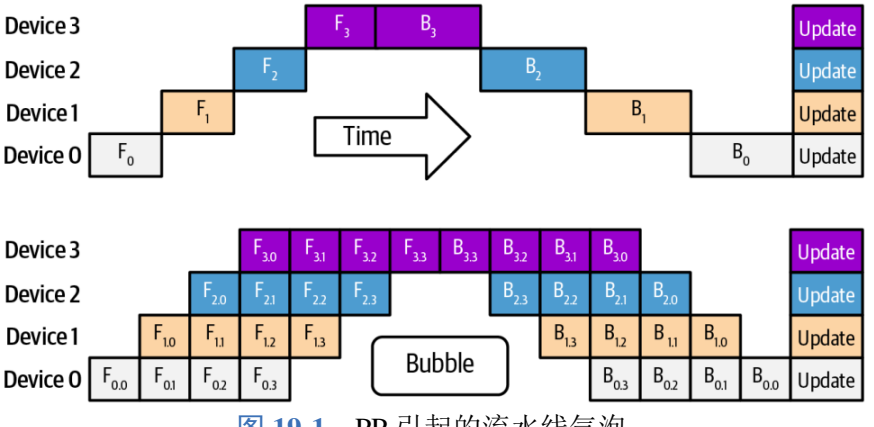

图 19-1　PP 引起的流水线气泡。

通过让每个输入只激活少量专家，专家并行可以降低专家数量很多的模型（通常称为宽专家模型）的单设备内存、推理时间和计算成本。这种基于路由器的条件式专家计算模式在增加专家数量时能够高效扩展。例如，DeepSeek-R1 共有 256 个专家，但推理时路由器只选择 top 9 个专家（其中 1 个是共享专家）。

传统上，并行化策略——包括把多种并行技术组合起来的混合策略——会在模型加载时一次性选定并固定。然而，为了在动态工作负载下最大化性能，现代推理引擎可以在运行时根据输入特征选择不同的并行策略。

高性能自适应推理系统会利用运行时指标，即时选择 TP、PP 或混合方案。关键因素包括批大小、序列长度和内存利用率，以及响应延迟和吞吐量要求。例如，很长的提示词可以被路由到 TP + PP 实例，因为这样会把层分散到多块 GPU 上，从而避免显存不足（OOM）错误。

与此同时，短而对延迟敏感的请求会被路由到纯 TP 模型实例，以避免流水线阶段开销。为支持这一点，服务引擎会维护多个预分片模型实例，每个实例针对不同的工作负载配置进行优化，并把传入查询动态分派给其并行策略最满足该任务 SLO 的模型实例。

你也可以使用不同数量的分片。图 19-2 展示了这一点：在八块 GPU 上采用两种不同的混合 TP + PP 并行配置，并使用两种不同的 TP 分片数量。

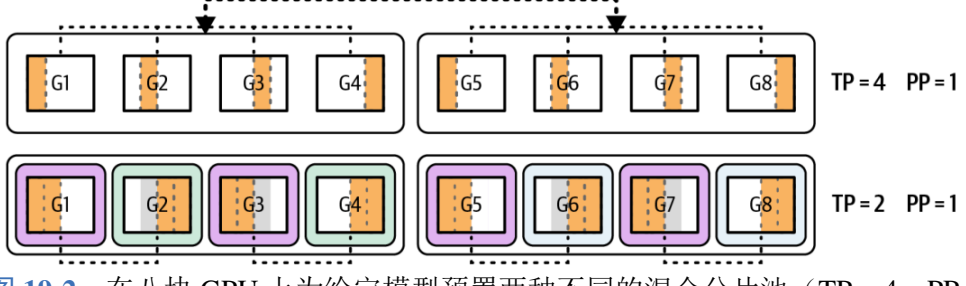

图 19-2　在八块 GPU 上为给定模型预置两种不同的混合分片池（TP = 4、PP = 1，以及 TP = 2、PP = 1）。

对长序列输入即时使用 PP，有助于避免大输入序列导致的 OOM 错误。反过来，对于短提示词和对延迟敏感的查询，系统可以路由到为低延迟优化的张量并行模型实例。此时，请求可以避开 PP 的开销。

由于每个请求可以使用不同的并行策略，系统需要维护模型的多个实例，供推理调度器/路由器选择。一个模型实例可以用 TP 针对低延迟优化，另一个实例则同时使用 TP 和 PP，针对高吞吐量和大输入序列优化。

之所以必须维护多个模型实例，是因为即时重新分片会严重扰乱 GPU 缓存。这还会对内存和网络子系统造成过大压力——尤其是在重新分片超大规模模型时。

运行时，每个查询都会根据其长度和指定的服务等级协议（SLA）被分派到最合适的模型实例（分片策略）。例如，DeepSeek-R1 是一个约 6800 亿参数的稀疏混合专家模型，每个 token 在所有专家中只激活 370 亿参数。

为支持不同的工作负载配置，可以把 GPU 组织成逻辑工作池，每个池都按特定并行策略预分片——例如张量并行，或张量与流水线混合并行。

考虑一个例子。如果我们有一台 8× GPU 的 Blackwell B200 服务器，总 HBM 为 1,440 GB（1,440 GB = 每块 GPU 180 GB × 8 块 GPU），就可以用四路 TP 在四块 GPU 上服务 DeepSeek-R1——让另外四块 GPU 空闲。

如果单个查询带着极长上下文（例如 > 100 万 token）到达，调度器可以启动一个两阶段流水线，让阶段 1 覆盖 GPU 0–3，阶段 2 覆盖 GPU 4–7。这实际上把每个阶段可用的 GPU 内存翻倍到约 720 GB 总 HBM（每块 GPU 180 GB × 4 块 GPU，即 720 GB 可用 HBM）。这有助于在处理大输入时避免 OOM 错误。

反过来，当数十个短小、对延迟敏感的提示词并发到达时，系统只把它们路由到张量并行实例。通过避免流水线气泡，或避免在填充和排空流水线阶段时出现的空闲期，这种配置能在所有可用 GPU 上提供尽可能低的单请求延迟。

要实现动态并行切换，可以编写一个决策函数，检查输入序列长度、GPU 内存使用率和当前负载等运行时指标。你用这些指标为每个请求选择最佳分片模型实例，如下所示：

```
def choose_worker_pool(seq_len, gpu_mem_util, concurrent_reqs):
    # For long contexts or high memory pressure,
    # use hybrid pipeline + tensor parallelism
    # (example thresholds shown here)
    if seq_len > 4096 or gpu_mem_util > 0.8:
        return "tp_pp_hybrid"
    # For many simultaneous small requests, stick with tensor
    ↪ parallelism
    if concurrent_reqs > 4:
        return "tensor_parallel"
    # Fallback to tensor-parallel for typical workloads
    return "tensor_parallel"
```

你会在 GPU 集群上预启动多个模型副本——一些按纯 TP 分片，另一些按 TP + PP 分片——并让路由器根据输入和决策策略把每个查询发送到合适的副本。这种方法确保大型、内存密集型任务获得完整流水线支持，而短小、对延迟敏感的调用运行在纯 TP 实例上，避免不必要的流水线开销。

建议利用模型和硬件的遥测数据来指导并行切换。你可以实时监控 GPU 内存利用率、计算利用率和互连（例如 NVLink/NVSwitch）流量来做决策。如果你发现 GPU 因长时间流水线气泡而空闲——而且还有额外内存余量——就可以把流水线收缩为更少的阶段，让每块 GPU 做更多工作并保持忙碌。反过来，如果某些阶段撞到内存上限或计算瓶颈，就可以扩展成更多流水线阶段——或提高张量并行度。这样会把计算和内存占用分散到更多 GPU 上。

关键在于动态调整张量切分与流水线切分的平衡，让每块 GPU 都得到充分利用。同时，你还要满足内存约束和延迟目标。这是静态的一刀切配置无法做到的。

19.2 动态精度切换

Blackwell 等现代 GPU 引入了对 8 位和 4 位浮点（FP8/FP4）Tensor Core 数学单元的支持。这些更低精度带来大幅加速、内存节省和很小的质量损失。

动态精度切换是一种高级技术：推理引擎根据模型置信度或资源压力在运行时调整数值精度。目标是在不造成明显质量损失的前提下提高吞吐量。实践中，这意味着系统可能用 FP8 或 FP4 执行模型的某些部分以提高效率，但在需要稳定性时回退到更高精度（FP16/BF16）。

触发精度自适应的一个信号是 logit 锐度，也就是模型的输出置信度。例如，如果模型对下一个 token 的概率分布在某个特定 token 上呈现极高峰值（因为高度自信），那么低精度带来的小幅数值误差不太可能改变结果。

如果可以容忍在下一个 token 生成中使用低精度，引擎就会安全地用 FP4 运行接下来几步以提升速度。反过来，如果分布因高度不确定性而更平坦，引擎就应坚持 FP8 或 FP16 以保持保真度。

推理引擎通过计算词表上 softmax 分布的香农熵来量化不确定性。熵越低，表示预测越尖锐（越自信）。一个在留出验证集上调好的固定熵阈值，决定何时降到 FP4、何时保持 FP8/FP16 以保证数值稳定性。目标是在延迟收益和精度损失之间取得平衡。

> **注** 使用能保持精度的最低精度；当以最大 softmax 概率衡量的模型置信度下降时，回退到更高精度。

这利用了一个事实：大型 LLM 在生成确定性延续（例如闭合引号或结束列表）时往往会变得更确定。在这些情况下，较低精度通常就足够了。

另一个因素是内存压力。如果因为上下文很长——或并行请求很多——GPU 内存使用率接近上限，系统可以动态把激活压缩到更低精度。

内存紧张时，可以把注意力 key/value 张量存成 INT4 而不是 INT8。这会把内存占用减少 50%。不过，要确保使用 INT4 带来的量化误差不会在许多解码步中累积。建议定期重新评估输出质量。

例如，如果某次推理进行到 KV 缓存占用 90% 内存时，引擎可能决定把新缓存条目从 INT8 量化到 INT4——甚至回溯压缩更旧的条目——以释放空间。这

可以在不停止模型的情况下完成。此时，后续注意力层直接读取 INT4 缓存值——只带有轻微量化误差。

把 4 位权重量化与 8 位激活结合，可以显著减少内存。例如，对于纯计算受限的 kernel，FP8 激活可以实现最高 2× 吞吐量——尤其是在高带宽现代 GPU 上。对于混合或内存受限的工作负载，可以达到 1.5×。对激活使用 FP4 可以进一步节省内存。不过，它可能引入略高的累积误差，需要仔细的逐层调优。

现代 GPU 原生提供 FP8 和 FP4 Tensor Core。然而，截至本文写作时，PyTorch 的 AMP 支持（torch.autocast）仍然只面向 FP16 和 BF16，不面向 FP8 或 FP4。虽然 PyTorch 中已有 FP8 dtype（例如 torch.float8_e4m3 和 torch.float8_e5m2）以及缩放数学路径，但 AMP 并不管理它们。对于推理和训练，建议使用 NVIDIA 的 Transformer Engine（TE），并在合适时采用其 MXFP8 和 NVFP4。

> **注** 使用 AMP 时，在 Blackwell 上对延迟关键的解码优先选择 BF16 而不是 FP16。对于 FP8 路径，Transformer Engine 的 MXFP8 格式是 Blackwell 上推荐的默认值。NVFP4 应有选择地用于 KV 缓存和轻量层，并做仔细的回归测试。记得在具体工作负载上逐层验证数值。

表 19-2 汇总了一些示例精度配置及其权衡。这里可以看到，更低精度会减少内存并提高吞吐量。不过，质量会有轻微下降。

表 19-2 LLM 推理中各精度模式的近似权衡。

精度模式	内存使用（相对）	计算吞吐量	质量影响（精度差值）
FP16（基线）	1.0×（100%）	1.0×（基线）	无影响（完整保真度）
FP16 权重 + FP8 激活	0.5×（50%）	1.5×	可忽略（< 0.1%）
INT4 权重 + FP8 激活	0.25×（25%）	1.8×（混合计算与内存受限）	质量下降约 0.5%
INT4 权重 + FP4 激活	0.2×（20%）	3.5×	下降约 1%（需要仔细调优）

这里可以看到，使用 FP8 激活时，激活位宽减少 50%，因此内存相对 FP16 基线如预期减少约 50%。此外，这里测得的质量损失对 FP8 激活可以忽略（<

0.1%）。（注意：降低精度带来的质量影响取决于模型和 kernel。你应该用自己的数据和工作负载进行验证。）

当内存是主要瓶颈时，INT4 权重 + FP8 激活的工作流可以达到约 1.8× 基线吞吐量。INT4 权重 + FP4 激活可以把内存降到基线的 20%。这是 4 位目标。加速约为 3.5×，与相对 FP16 的理论峰值 4× 改进一致。

动态精度切换的目标是在把输出质量保持在可接受范围内的同时最大化性能。理想情况下，kernel 以尽可能快的精度（例如 FP8 或 FP4）运行，只在必要时回退到更高精度（例如 FP16）。实践中，NVIDIA 面向 PyTorch 的 Transformer Engine 等库允许在运行时逐层控制精度。

线性层可能默认使用 FP8，但运行时 hook 可以根据层的角色把某层的精度提高到 FP16 或降低到 FP4。例如，FP4 可以用于输出投影这类允许轻微精度下降的轻量层，而处理原始用户输入、受益于更高精度的早期层则可能使用 FP8 或 FP16。

除了逐层混合精度控制，还可以使用更细粒度的优化策略，即按 token 调整精度。这种方法让推理系统在预测自信时以尽可能快的模式运行，例如 FP8；在更不确定时回退到 FP16 等更高精度模式。

实践中，模型先用默认精度（例如 FP16）生成当前 token，然后根据输出熵、最大 softmax 概率或 logit 方差等运行时指标评估置信度。

如果模型对预测高度自信，下一个 token 就可以用更低精度处理。如果不确定性高，系统就回到更稳定的格式以保持输出质量。下面是演示这一概念的示例代码：

```
import contextlib
import torch

# ----------------------------
# Safe Transformer Engine (TE) FP8 autocast import
# ----------------------------
try:
    # TE is only effective if your model actually uses TE-enabled layers
    # (e.g., Linear, LayerNorm wrappers).
    from transformer_engine.pytorch import fp8_autocast as
    ↪ _te_fp8_autocast
    # type: ignore
    _TE_AVAILABLE = True
except Exception:
    _TE_AVAILABLE = False

    # No-op stand-in so the code runs without TE installed. It never
    ↪ changes
    # numerical behavior.
    class _NullCtx(contextlib.ContextDecorator):
        def __init__(self, **_): pass
        def __enter__(self): return self
        def __exit__(self, *exc): return False
    def _te_fp8_autocast(**_):
        return _NullCtx()

# ----------------------------
# Helper: choose the precision context *for this step* safely
# ----------------------------
def _precision_context_cuda(use_fp8: bool,
                            prefer_bfloat16: bool,
                            enable_fp8: bool):
    """
    Enter exactly one precision context. If FP8 isn't enabled or TE is
    missing/unused, fall back to AMP (BF16/FP16).
    """
```

```
    if use_fp8 and enable_fp8 and _TE_AVAILABLE:
        # Note: fp8_autocast affects only TE-enabled modules. Non-TE
        ↪ modules
        # run at their native dtypes.
        return _te_fp8_autocast(enabled=True)
    amp_dtype = torch.bfloat16 if prefer_bfloat16 else torch.float16
    return torch.autocast(device_type="cuda", dtype=amp_dtype)


def _precision_context(device: torch.device, use_fp8: bool,
                       prefer_bfloat16: bool, enable_fp8: bool):
    return _precision_context_cuda(use_fp8, prefer_bfloat16,
                                   enable_fp8) if device.type == "cuda"
                                   else contextlib.nullcontext()


# ----------------------------
# Main decode loop with smoothed, hysteretic precision switching
# ----------------------------
@torch.no_grad()
def decode_with_dynamic_precision(
    model,
    tokens: torch.Tensor,
    max_steps: int,
    *,
    device: torch.device = torch.device("cuda"),
    prefer_bfloat16: bool = True,      # B200: prefer BF16 over FP16 for
    ↪ AMP
    enable_fp8: bool = True,           # Allow FP8 when TE present
    enter_fp8_threshold: float = 6.0,  # hysteresis upper bound
                                       # (logit margin average)
    exit_fp8_threshold: float = 3.0,   # hysteresis lower bound (avoid
    ↪ flapping)
    reeval_interval: int = 8,          # compute/inspect confidence
    ↪ every N steps
                                       # to avoid per-step sync
```

```
    topk_dim: int = -1,                # last dimension holds vocabulary
     ↪ logits
    eos_id: int | None = None,
):
    """
    Autoregressive decode loop that *smoothly* switches between AMP
     ↪ (BF16/FP16)
    and FP8 (TE) without per-step host sync. Works even when TE is not
    installed; in that case, runs AMP only.

    - Confidence signal: mean(top1 - top2) logits margin across the
     ↪ batch.
    - Smoothing: EMA + interval re-evaluation to minimize CPU-GPU sync
     ↪ pressure.
    - Hysteresis: separate enter/exit thresholds to avoid precision
     ↪ flapping.
    """
    assert exit_fp8_threshold <= enter_fp8_threshold, \
        "Hysteresis requires exit <= enter threshold"

    model.eval()
    tokens = tokens.to(device, non_blocking=True)

    # Internal state
    use_fp8: bool = False  # start in AMP.
                           # Upgrade to FP8 when sustained confidence
                            ↪ permits
    ema_conf: torch.Tensor | None = None  # stays on device;
                                          # host consults only at intervals
    alpha = 0.2  # EMA smoothing factor for confidence

    # A tiny helper to update on-device EMA without host sync
    def _update_confidence_ema(logits: torch.Tensor) -> torch.Tensor:
        # logits: [B, vocab] or [B, T, vocab]. Use the last time-step if
         ↪ 3D.
        last = logits if logits.dim() == 2 else logits[:, -1, :]
```

```
    # Compute top-2 margin on-device
    top2 = torch.topk(last, k=2, dim=topk_dim).values  # [B, 2]
    margin = (top2[:, 0] - top2[:, 1]).mean()      # scalar tensor
    ↪ on device
    nonlocal ema_conf
    ema_conf = (1 - alpha) \
               * (ema_conf if ema_conf is not None else margin) +
                ↪ alpha * margin
    return ema_conf  # device scalar

# Decode
for step in range(max_steps):
    # 1) Precision context (exactly one).
    # No nested contexts, no leakage across iterations.
    with _precision_context(device, use_fp8, prefer_bfloat16,
    ↪ enable_fp8):
        # Forward pass (HF-style or plain)
        try:
            logits = model(input_ids=tokens)
            if hasattr(logits, "logits"):
                logits = logits.logits
        except TypeError:
            logits = model(tokens)

        # 2) Pick next token from the *last* position
        last_step_logits = logits if logits.dim() == 2 else
        ↪ logits[:, -1, :]
        next_token = torch.argmax(last_step_logits, dim=-1,
                                  keepdim=True)  # [B, 1]
        tokens = torch.cat([tokens, next_token], dim=1)

    # 3) Update on-device EMA signal every step (no host sync yet)
    conf_dev = _update_confidence_ema(logits)

    # 4) Periodically re-evaluate precision choice on host
    # to avoid per-step sync
```

```
        if (step + 1) % reeval_interval == 0:
            conf_value = float(conf_dev)  # exactly one tiny sync every
             ↪ N steps
            if not use_fp8 and enable_fp8 and _TE_AVAILABLE \
               and (conf_value > enter_fp8_threshold):
                use_fp8 = True
            elif use_fp8 and (conf_value < exit_fp8_threshold):
                use_fp8 = False

        # 5) EOS handling
        if eos_id is not None:
            if (tokens[:, -1] == eos_id).all():
                break

    return tokens


# ----------------------------
# Example (commented):
# ----------------------------
# model = ...  # your TE-enabled model (or any torch.nn.Module)
# input_ids = torch.randint(0, vocab_size, (batch_size, seq_len))
# out = decode_with_dynamic_precision(model, input_ids, max_steps=128,
# eos_id=tokenizer.eos_token_id)
# print(out.shape)
```

这里可以看到，截至本文写作时，PyTorch autocast 只支持降低精度的 FP16 和 BF16。在这种情况下，你需要使用 Transformer Engine 库把受支持的模块路由到 FP8 kernel。

> **注** 本示例中使用的阈值（进入 = 6.0，退出 = 3.0）应在验证集上用代表性提示词进行校准，以免延迟收益影响精度。

这种模式创建了一种弹性精度机制，并最大化吞吐量。当模型运行在可预测（例如低熵）区域时——例如生成标点或样板式续写——它会继续使用 FP8

以最大化性能。当进入更高熵的片段时——例如模糊提示词或决策点——它会回到 FP16 以保持数值精度。

结合现代 GPU 对低精度运算的支持，token 级动态精度切换为高吞吐量、对延迟敏感的推理提供了一种自适应策略。它只在需要时应用低精度，减少计算开销，并在许多不同提示词条件下保持响应质量。

19.3 Transformer 自注意力与 MLP 路径的 kernel 自动调优

神经网络层在 GPU 上的性能可能因线程块大小、瓦片尺寸、循环展开和内存访问模式等底层参数而大幅变化。对于固定尺寸的模型，库通常只选择一次这些参数——往往使用通用启发式规则或离线调优。

然而，在在线推理服务场景中，输入尺寸（包括序列长度和批大小）会因请求而异。kernel 自动调优指的是一种运行时机制：为当前工作负载选择——甚至即时编译（JIT）——最优的 kernel 变体。

在大型 Transformer 模型中，推理的两个主要计算阶段是自注意力和前馈 MLP 层。二者都能从 GPU kernel 自动调优中受益。下面我们在 kernel 自动调优的背景下分别讨论。

考虑一个处理长度为 L 的序列、具有 H 个注意力头的注意力层。注意力有许多实现，包括标准注意力和优化过的 FlashAttention，以及它的多个变体。

由于采用了瓦片化、并行化和内存访问改进，FlashAttention 及其变体在长序列上明显更快。然而，对于非常短的序列，其开销可能超过收益。动态引擎可以根据序列长度 L 在 FlashAttention kernel 和更简单的 kernel 之间切换。

例如，如果一个请求的 L = 256 个 token，引擎可能使用直接的 kernel 启动方式，通过全局内存读取一次性计算注意力，这对较小的 L 已经足够。如果另一个请求的 L = 2,048，它就可以切换到 FlashAttention 的专用瓦片化 kernel；后者通过复用共享内存中的数据并避免不必要的 HBM 数据读取，在大 L 下扩展性更好。下面用基于输入序列长度的条件语句演示这一点：

```
// Note: example threshold shown here
if (seq_len < 256) {
    // global-memory version, best for small L
    attn_kernel = standard_attention_kernel;
} else {
    // tiled loads, best for large L
    attn_kernel = tiled_attention_kernel;
}
output = attn_kernel(Q, K, V, mask);
```

在幕后，attn_kernel 会在完全不同的 CUDA 实现之间做选择。一个实现针对小输入优化，使用默认注意力 kernel；另一个针对大上下文优化，使用瓦片化 kernel。

理想的瓦片尺寸取决于 GPU 的共享内存容量和计算资源。CUTLASS 和 OpenAI 的 Triton 等框架包含自动调优器，会在初始化时——甚至在运行时自适应地——对一系列 (TILE_Q, TILE_K) 组合进行基准测试，以选择最快的变体。表 19-3 展示了不同瓦片尺寸在 Blackwell 级 GPU 上的性能示例。

表 19-3 瓦片尺寸选择对共享内存占用、SM 占用率和实现吞吐量的影响示例（实际值取决于线程块尺寸、时钟频率和其他微架构因素）。

瓦片尺寸	共享内存（KB）	占用率（%）	吞吐量（GOPS）
64 × 64	48	85	8.2
128 × 64	64	78	10.5
128 × 128	96	72	9.8
256 × 128	128	60	11.3

通过在运行时根据输入选择正确的变体，你可以避开一刀切方法带来的巨大性能悬崖。实践中，你可以在目标硬件上做基准测试，找出大约 L = 128 是盈亏平衡点。

接下来，我们在自动调优的背景下分析前馈 MLP kernel。前馈层本质上是大规模矩阵乘法——具体来说，是两个线性投影，中间夹一个非线性激活。

PyTorch 等现代 AI 框架使用高度优化的 GEMM kernel，依赖 cuBLAS、CUTLASS 等优化过的 CUDA 库。对于给定矩阵尺寸，这些库中通常有多个算法变体，使用不同的瓦片策略、不同的 Tensor Core 以及单独的回退路径。

例如，NVIDIA 的 cuBLAS 和 cuBLASLt 库可以自动调优 GEMM kernel：先尝试几种算法，然后为给定维度挑选最快的算法。不过，这通常发生在首次遇到该形状的 GEMM 时——之后不会重新评估。

> **注** 在 cuBLAS/cuBLASLt 或自定义 kernel 可用时，程序化依赖启动（PDL）可以缩短启动间隙并提高稳态吞吐量。务必通过性能分析确认重叠确实发生。

在会看到许多不同批大小的推理服务器中，可以显式调用这类自动调优机制——或者维护一个最佳算法缓存。例如，对于 MLP 中形状为 [batch_size, hidden_dim] x [hidden_dim, 4*hidden_dim] 的 GEMM，batch_size = 1 和 batch_size = 16 时的最优 kernel 可能不同。

引擎可以检测到新的批大小，并使用 cuBLASLt 或自定义实现对候选 kernel 快速做微基准测试，以选择最快的 kernel。之后使用该批大小的调用就可以直接使用选中的 kernel。

此外，一些推理框架和运行时使用 OpenAI 的 Triton GPU kernel 领域特定库（DSL），以自动调优的瓦片尺寸即时编译注意力和 MLP kernel。

此时，运行时会生成同一 kernel 的几个不同瓦片尺寸变体（例如 128 × 128、64 × 256 等），并测量在真实硬件和输入形状下哪个表现更好。

你可以使用 Nsight Systems 这类工具，对不同 kernel 变体进行并排的实际性能分析。

具体来说，Nsight Systems 提供详细的 CUDA 时间线，包括 memcpy 和 NVLink 活动；Nsight Compute 提供内存工作负载分析，帮助把缓存和内存行为归因到 kernel 位置。这在评估瓦片尺寸与共享内存权衡时特别有用。此外，它还能揭示 L2 缓存缺失等不明显的瓶颈，进一步指导你的调优决策。

> **注** 由于负载下的 L2 缓存效应、内存 bank 冲突等因素，硬件有时难以预测，因此实际测量调优总是优于理论猜测。不过，从合理的理论值开始调优过程仍然是有益的。

动态瓦片切换会影响 GPU 占用率，选择瓦片尺寸时应加以考虑。使用更大的瓦片可以提高复用并减少 kernel 启动开销，但也会使用更多寄存器和共享内存。这可能减少可并发运行的线程块数量——从而降低占用率。合适的自动调优器会考虑这一权衡。

在注意力 kernel 中，更大的瓦片（例如 128 × 128）可以最大化共享内存中的数据复用。这对长序列很理想，因为全局内存加载次数更少、循环开销被摊薄，并能产生更高的持续吞吐量。

然而，对于更短的序列，同样的大瓦片会消耗过多共享内存，从而限制占用率，也就是限制每个 SM 上并发运行的线程块数量。对短序列减小瓦片尺寸（例如 64 × 64），可以释放共享内存，从而并行调度更多线程块。这会提高 SM 占用率并降低单 kernel 延迟。

根据输入序列长度调整瓦片尺寸，可以让 kernel 在大多数情况下达到接近最优的占用率。一些系统甚至在运行时查询 CUDA Occupancy API，以动态选择 kernel 启动参数，例如线程块大小。下面给出 C++ 中 Occupancy API 的示例，不过也有 Python API 可用：

```
// Pseudocode for occupancy-based launch configuration
int maxBlocks, bestThreads;
for (int threads = 64; threads <= 256; threads *= 2) {
    cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &maxBlocks, MyKernel, threads,
        sharedMemPerBlock(threads));
    // choose "threads" to maximize occupancy
    // (remember to not exceed the max threads per SM limit (e.g., 2,048)
    float occupancy = (float) maxBlocks * threads /
                              hardwareMaxThreadsPerSM;
}
```

这段伪 C++ 展示了如何为 kernel 评估不同的线程块大小。它检查在给定共享内存使用量下，每个 SM 能运行多少个线程块。随后 kernel 启动会相应调整线程数量——或共享内存使用量。高性能框架和推理引擎会在内部用下面这组步骤自动化这类逻辑：

1. 度量工作负载检查下一次模型前向的当前输入维度（批大小 B、序列长度 L 等）。

2. 选择候选 kernel 确定每个组件可用的 kernel 实现，例如注意力阶段的标准注意力或 FlashAttention，以及 MLP 阶段合适的 GEMM 算法。

3. 估算或基准测试在样本数据上让每个算法运行几轮，对每个候选做一次快速测试。

4. 选择最佳变体根据测得的维度，选择执行时间最短——或吞吐量足够——的 kernel。

5. 缓存结果把选择结果存入以输入维度或工作负载特征为键的查找表。这样，如果出现类似请求，就无需重新运行这些步骤即可知道最佳 kernel。

6. 执行使用选中的 kernel 实现运行该模型层。

> **注** 这类似于数据库查询优化器选择查询计划。这里的“计划”就是选中的 kernel 实现。

通过这样的流程，推理运行时会持续自我调优。随着时间推移，系统会为各种场景构建优化路径库，例如短提示词与长提示词、小批次与大批次等。即

时调优的开销可以通过两种方式保持较低：异步进行——在单独的 stream 中测试新 kernel，而当前推理使用默认 kernel——或者在低流量时段进行，以免影响延迟。

建议在加载模型时加入初始预热阶段，运行多种样本输入以触发自动调优。可以包含极端情况——例如最大序列长度、最大批次等——让引擎为这些情况预先优化 kernel。

此外，最好在运行时监控每一层的执行时间。如果某一层因输入特征变化突然成为瓶颈，就该重新审视 kernel 选择。

> **注** 一些高级框架甚至使用多臂老虎机算法，持续探索替代 kernel，并在条件变化时更新所选择的 kernel。

简而言之，自动调优把静态 kernel 转变为自适应 kernel。无论工作负载如何，它都能针对每组输入从 GPU 集群中榨取最高性能。你可以确信系统在持续适应。

19.4 动态共享内存分配与占用率感知的 kernel 选择

与 kernel 调优密切相关的是 GPU 共享内存和整体流式多处理器（SM）占用率的管理。现代 GPU 每个 SM 都有较大的共享内存。通过在运行时根据问题规模——以及当前共享内存利用率和占用率——动态分配线程，可以显著提升 AI 系统的整体性能。

使用动态共享内存分配时，系统根据问题规模调整每个线程块使用的共享内存量。使用占用率感知的 kernel 选择时，系统选择能最好利用 SM 资源（包括寄存器、共享内存和 warp）的 kernel 启动参数，让 GPU 保持忙碌。

选择瓦片化注意力算法时，应平衡数据复用与 SM 占用率。例如，设 T 为每个线程块的瓦片宽度（以 token 计）。由于自注意力本身是二次的，每个线程块要在共享内存中保留 O(T2) 量级的浮点数，用来存放 query、key 和 value 分块。

大瓦片尺寸（例如 T=256）会从 DRAM 中只加载每个 key/value 块一次，并在许多 query 中复用，从而把全局内存流量降低到接近每线程块 O(T) 个浮点数。但由于每个线程块现在使用大量共享内存，在硬件限制下，一个 SM 上同时只

能运行少量线程块。这降低了占用率。例如，如果 T=256 时每个 SM 只能运行 1 个块，而 T=128 时可以运行 4 个块，那么使用 T=256 时可能只能看到 30% 的 SM 占用率。

小瓦片尺寸（例如 T=64）使用的共享内存少得多，因此每个 SM 能容纳更多线程块。这能更好地隐藏延迟并提高利用率。不过，你会更频繁地重新加载相同的 key/value 数据，从而增加 DRAM 访问。

最优瓦片尺寸 T 取决于几个因素，包括序列长度 L、GPU 的共享内存容量和 SM 数量。你希望瓦片足够大，以摊薄 DRAM 读取；又要足够小，让占用率保持在较高水平，使许多线程块能同时在 SM 上活跃。

实践中，你可以手动挑选几个候选 T 值，例如 64、128 和 256——并用能代表数据集的序列长度 L，在具体硬件上对每个值做基准测试。然后选择能产生最佳整体吞吐量的 T 值。不过，与其提前硬编码 T，不如在启动 kernel 前即时计算它，如下所示：

```
int T = choose_tile(L, gpu_shared_mem_per_block, num_sms);

// calculate shared memory in bytes based on the tile size
// (multiplying by 3 for Q, K, and V)
size_t shared_mem_bytes = 3 * T * T * sizeof(float);

numBlocks = ...
MyAttentionKernel<<<numBlocks, threadsPerBlock,
↪  shared_mem_bytes>>>(...);
```

这里，T 由序列长度 L、GPU 每个线程块的共享内存上限 gpu_shared_mem_per_block，以及 SM 数量 num_sms 计算得出。然后，每个线程块的共享内存 shared_mem_bytes 会在运行时根据计算出的瓦片尺寸 T 得到。

随后，你可以用共享内存参数 shared_mem_bytes 启动 CUDA kernel。kernel 内部会包含下面的声明，定义一个 extern __shared__ 数组，为每个线程块分配大小为 shared_mem_bytes 的共享内存缓冲区：

```
// holds 3 tiles of T×T floats for Q, K, and V
extern __shared__ float smem[];

// Q tile: smem[0 ... T*T-1]
float* tile_q = smem;
// K tile: smem[T*T ... 2*T*T-1]
float* tile_k = smem + T*T;


// V tile: smem[2*T*T ... 3*T*T-1]
float* tile_v = smem + 2*T*T;
```

每次启动时改变 shared_mem_bytes，同一个 kernel 二进制就能以不同瓦片尺寸运行。选择 T 之后，你可以用 CUDA Occupancy API 查询占用率，看看每个 SM 能容纳多少个块。

如果占用率太低，每个 SM 只分配到一个块，就可以减小 T。如果你在反复冲击 DRAM，就可以增大 T。这可以实现为一个自动反馈循环：kernel 通过 CUDA Occupancy API 或 NVIDIA 数据中心 GPU 管理器（DCGM）以编程方式测量自身达到的占用率，并在后续迭代中调整 T。这样，每个注意力层都能根据当前序列长度 L 和硬件限制使用最优配置。

正如第 6 章所见，优化 SM 占用率时还需要考虑每个线程的寄存器使用量。使用更多寄存器（例如展开循环）可以加快单线程性能，但由于每个 SM 的寄存器文件有限，这可能降低整体 SM 占用率。

如果每个 warp 使用很多寄存器，可调度的 warp 数量就会减少。动态运行时可以检测 kernel 是否因寄存器而触及占用率上限，并切换到使用更少寄存器的版本——代价是额外指令。这些底层考虑对自适应、高性能推理服务器至关重要。

动态共享内存调优需要分析占用率与吞吐量的关系。NVIDIA Nsight Systems/Compute 和 CUDA Occupancy API 等工具可以显示每个 kernel 达到的占用率和执行效率。同时，DCGM 在系统层面提供实时 GPU 利用率和 SM 占用率指标。自适应系统可以利用这些信息发现：例如，序列长度为 2,048 的注意力 kernel 只达到 30% 占用率，因为每个线程块使用了大量共享内存。

在这种情况下，系统可以动态切换到一种降低每线程块共享内存的 kernel 配置，例如把注意力计算拆成两遍。如果内存延迟是瓶颈，这会提高占用率——并可能提高吞吐量。

反过来，如果 kernel 是内存受限且没有让 ALU 饱和，那么使用更多共享内存——即使占用率下降——也可以通过减少内存停顿来提高有效吞吐量。理解这些权衡很重要——尤其是占用率，因为它在某些情况下不如其他指标直观。

建议设计允许在运行时调整共享内存和线程块大小的 kernel。系统随后可以根据输入和硬件反馈，把调优配置适配到运行时条件。例如，它可以为 CUTLASS 等库使用的瓦片尺寸提供运行时参数和模板参数；这些库正是为此提供运行时可调的 kernel 变体。

你还应持续监控 SM 利用率指标。注意大量空闲 warp（例如活跃 warp < 50%）或内存停顿周期（> 70% 停顿）。这表明存在失衡：要么占用率太低（warp 空闲）——要么瓦片尺寸太小，导致内存流量过大。因此，系统应相应调整以恢复平衡。

对于推理服务，常见做法是维护一张针对不同问题规模的最优线程块配置小表。该映射可以实现为 JSON 或配置文件，把序列长度范围映射到启动参数。这样，随着模型和硬件演进，更新起来很容易。

例如，当系统以序列长度 512 执行注意力时，会使用每块 128 个线程和 16 KB 共享内存。或者，对于序列长度 4,096，会使用每块 256 个线程和 64 KB 共享内存，等等。这把自动调优的概念扩展到了资源分配。

请记住，现代 NVIDIA GPU 为 L1 数据缓存和共享内存提供统一的片上池。carveout 控制该池中有多少保留给共享内存、多少保留给 L1。当 kernel 需要更大瓦片时，可以用 cudaFuncSetAttribute 调整 carveout，增加可供共享内存使用的比例。

现代 NVIDIA GPU 为 L1 数据缓存和共享内存提供统一的片上池。NVIDIA 的设备驱动允许你设置 L1 缓存与共享内存的拆分百分比，即“carveout”百分比。因此，你可以根据使用场景把 SM 配置为偏好更多共享内存或更多 L1 缓存。例如，当 kernel 需要更大瓦片时，可以增加可供共享内存使用的比例。

> 注　carveout 是逐 kernel 属性，而且只是提示而非保证。它是另一个可调旋钮，用来平衡占用率和缓存行为。

复杂的运行时可以在启动时使用 cudaFuncSetAttribute() 和 cudaFuncAttributePreferredSharedMemoryCarveout，为特定 kernel 切换这个 carveout 百分比。例如，如果注意力 kernel 使用非常大的瓦片并需要更多共享内存，你可能想把 L1 降到 25%、把共享内存提高到 75%（假设 carveout 值从 50% 开始）。

> 注　共享内存与 L1 的 carveout 属性只是提示而非保证。始终把它当作提示，并用性能分析验证效果。检查所请求的设置是否确实影响了占用率和缓存行为。

简而言之，动态共享内存和占用率感知技术能确保每个 SM 在给定任务下尽可能忙碌。这些技术让 kernel 的资源使用适配具体用例。对于大型模型，这至关重要，因为某些层或批大小否则可能无法充分利用 SM。

19.5 用于加快 TTFT 的推测式 KV 预取

在实时场景中服务 LLM 时，首 token 时间（TTFT）是关键指标，因为它衡量系统能多快产出模型响应的第一个 token。这直接影响最终用户体验。

大型模型中 TTFT 的一个主要贡献项，是在开始生成 token 之前建立模型内部状态（例如键值（KV）缓存）所花的时间。回顾前面章节：注意力 KV 缓存保存每一层过去 token 的 key 和 value 投影。

推测式 KV 预取是一种优化：系统预测第一个 token 所需的数据，并提前把必要数据加载到 GPU 中。这实际上把 KV 缓存准备与其他步骤（例如计算）重叠起来。这样，token 生成可以更快开始。推测式 KV 缓存的一个例子是 SpeCache，如图 19-3 所示。

使用 SpeCache 时，KV 缓存会被压缩（本例为 16 位），并逐层移出 GPU。这会减少内存占用。生成第一个输出 token 后，会计算一个推测性的“下一个”token。同时，模型预取第一次解码步骤所需的对应低精度 KV 对。

在每个后续步骤中，模型并行解码两个 token，包括实际输出 token 和推测 token。两个结果都送入下一步；并且在每一步之前，都会为推测路径预取 top-k

最相关的 16 位 KV 对。这样，两条路径所需的 KV 缓存数据都已就绪。简而言之，SpeCache 通过预取低精度 KV 并与计算重叠，报告了 TTFT 改进。

注 只有在验证了访问模式和存储层级之后，才应集成推测式预取技术。

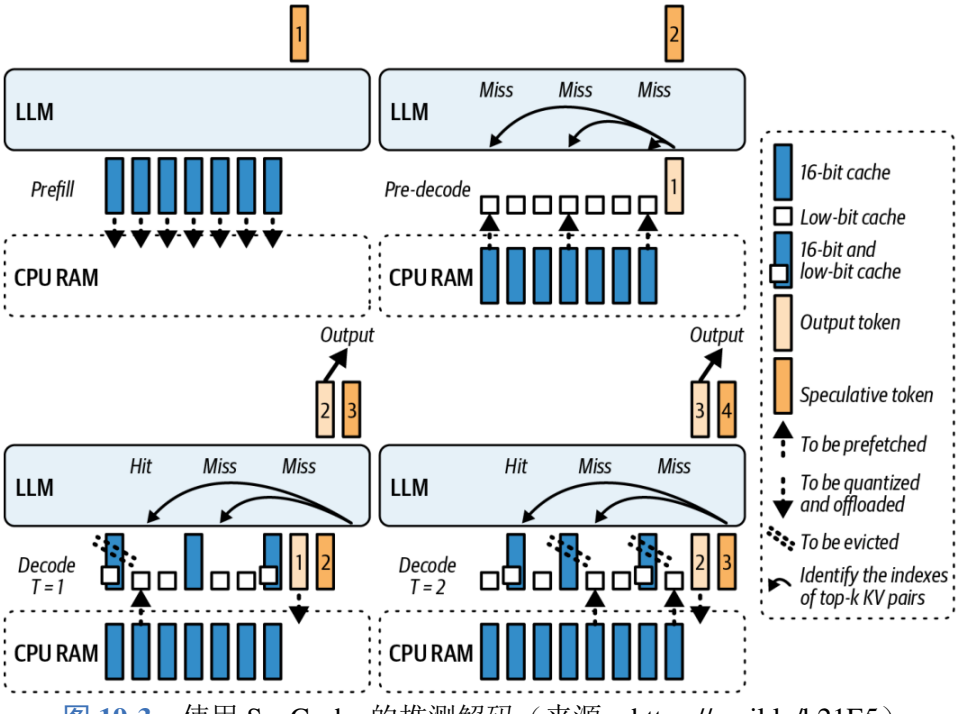

图 19-3 使用 SpeCache 的推测解码（来源：https://oreil.ly/b21E5）。

由于现代 LLM 的层数、不断增大的 LLM 上下文窗口（目前实际上几乎没有上限），以及现代“思考”模型产生的大量推理链，KV 缓存可能极其庞大。现代推理系统常常在 GPU、CPU 内存和 SSD 之间交换 KV 缓存，以更好地管理容量——尤其是对那些无法放入 GPU 内存的超长上下文。

生成新 token 时，朴素做法是从 KV 数据所在位置（一些在 GPU、一些在 CPU）以同步方式取回数据——然后执行计算以解码下一个 token。这会显著增加第一个 token 的延迟——尤其是当缓存已被分页到 CPU 内存或 NVMe 存储时。

KV 缓存预取通过提前启动 KV 数据传输来提供帮助。一旦收到用户提示词，服务器就可以开始把必要的 KV 页——以及模型权重——直接复制到 GPU

内存。等到模型在预填充阶段完成提示词计算时，所需数据已经就位，可以生成第一个输出 token。

具体来说，这种机制只在 GPU 内存中保留当前层的 KV——把其他层的 KV 卸载到 CPU。它在计算当前层的同时，异步把下一层的缓存预取到 GPU。此外，它还会同时把上一层的缓存写回 CPU。

这种通信与计算的重叠意味着 GPU 很少等待数据。结果是，在 CPU 内存中使用已卸载的 KV 缓存对延迟影响很小。例如，由于额外的数据传输开销，卸载时你可能看到 tokens/s 吞吐量下降约 5%–10%。

> **注** 重叠可以掩盖 CPU 驻留 KV 的大部分延迟，但由于 CPU DRAM 带宽和 PCIe 开销，通常仍会保留一定的吞吐量惩罚。请用你的批大小和序列长度反复做性能分析。

KV 缓存卸载的一个例子是 Hugging Face Transformers 库中的 OffloadedCache 机制。调用 generate(cache_implementation="offloaded") 或 generate(cache_implementation="offloaded_static") 时可以启用它。这会用 Transformer 库生成 token，如下所示。因此，它是一种低投入、高收益的优化：

```
# Dynamic, variable-length serving and sliding layers
# (recommended default)
out = model.generate(..., cache_implementation="offloaded")

# Static shapes + torch.compile and CUDA Graphs
# (highest throughput with fixed shapes, use with torch.compile)
# out = model.generate(..., cache_implementation="offloaded_static")
```

在底层，生成开始时，OffloadedCache 会确保第 1 层的 KV 被移到 GPU。第 1 层计算期间，OffloadedCache 会发出异步 DMA，把第 2 层的 KV 从 CPU 传到 GPU，以此类推。它始终提前预取一层。

当前向传播到达第 2 层时，它的 KV 已经在本地。这减少了若对每一层都使用同步复制会出现的停顿。既然已经描述了 KV 预取，接下来转向推测式 KV 预取。

推测式 KV 预取超越了常规 KV 预取只提前一层的范围。想象一个推理服务器配置，其中有多个模型副本——或有多条可能路径，例如 MoE 模型中一个 token 可以被路由到若干专家网络之一。

KV 预取在各阶段边界处很有帮助。理想情况下，到预填充阶段结束时，所有层的缓存要么已在 GPU 内存中，要么已排队进入 GPU 内存。这直接最小化 TTFT，因为一旦开始生成，模型就不再等待内存传输。

> **注** 建议使用 NVTX 标记等跟踪工具持续监控 TTFT，测量第一个 token 的解码时间。这样可以精确测量 TTFT。如果你在解码阶段开始后立即看到过多的空闲时间尖峰，就说明错过了预取机会。

要在自己的技术栈中实现 KV 预取而不阻塞推理，可以使用 CUDA stream 做重叠（如第 11 章所述）。这样，它会与主计算 stream 并发运行。然后，只在需要预取数据时用 CUDA event 同步两个 stream，如下所示：

```
// kv_prefetch_overlap.cu
#include <cstdio>
#include <cuda_runtime.h>

// Example sizes
static constexpr size_t KV_BYTES =
  /* set to your chunk size */ 8ull<<20; // 8 MiB

__global__ void forward_kernel(/* ... */) {
  // compute logits for current token ...
}

__global__ void consume_prefetched_kv(/* use prefetch_dest */) {
  // consumes KV in prefetch_dest ...
}

int main() {
  // Allocate destination buffer on this GPU
  void* prefetch_dest = nullptr;
  cudaMalloc(&prefetch_dest, KV_BYTES);
```

```
// Example: staging source on host. MUST be pinned for real overlap.
void* kv_src_host = nullptr;
cudaMallocHost(&kv_src_host, KV_BYTES);  // pinned (page-locked)
// Fill kv_src_host with data for the first iteration...

cudaStream_t compute_stream, prefetch_stream;
cudaStreamCreateWithFlags(&compute_stream, cudaStreamNonBlocking);
cudaStreamCreateWithFlags(&prefetch_stream, cudaStreamNonBlocking);
cudaEvent_t kv_ready;
cudaEventCreateWithFlags(&kv_ready, cudaEventDisableTiming);

bool done = false;
while (!done) {
  // 1) Launch compute for current token
  forward_kernel<<< /*grid*/1, /*block*/1, 0, compute_stream>>>();

  // 2) Asynchronously prefetch next KV chunk
  // If your source is another GPU, use cudaMemcpyPeerAsync
  // and enable peer access.
  cudaMemcpyAsync(prefetch_dest, kv_src_host, KV_BYTES,
                              cudaMemcpyHostToDevice, prefetch_stream);
  cudaEventRecord(kv_ready, prefetch_stream);

  // 3) Ensure consumer on compute_stream waits just-in-time
  cudaStreamWaitEvent(compute_stream, kv_ready, /*flags*/0);

  // 4) Launch work that consumes the prefetched KV
  consume_prefetched_kv<<< /*grid*/1, /*block*/1, 0,
   ↪  compute_stream>>>();

  // 5) ...advance state, update kv_src_host for next iteration, set
   ↪  `done`
  done = true; // demo
}
```

```
  cudaEventDestroy(kv_ready);
  cudaStreamDestroy(prefetch_stream);
  cudaStreamDestroy(compute_stream);
  cudaFree(prefetch_dest);
  cudaFreeHost(kv_src_host);
  return 0;
}
```

在这个设置中，cudaMemcpyAsync 在 prefetch_stream 上运行，而 model.forward() 使用 compute_stream。这允许 CUDA 驱动把数据传输与计算重叠起来。只有真正需要预取 KV 数据时才同步：在继续执行消费它的计算之前等待 kv_ready event。该 event 在交接点强制执行即时同步。

> **注** 确保主机缓冲区是页锁定（pinned）的。否则，cudaMemcpyAsync 可能串行化，你就得不到期望的复制/计算重叠。如果 KV 源在另一块 GPU 上，请使用 cudaMemcpyPeerAsync 并启用 peer access。如果你使用统一内存（例如 Grace Blackwell、Vera Rubin 超级芯片），可以考虑使用 cudaMemPrefetchAsync 提前把页准备好。如果模式可重复，还可以用 CUDA Graphs 捕获该序列。当预取频繁发生时，这可以进一步减少 kernel 启动开销。

使用单独的 stream 可以确保高效流水化。在生成一个 token 的同时，下一个 token 的 KV 缓存正在被预取，而不会打断计算 stream。这通过掩盖传输延迟并让计算单元持续获得数据来最大化 GPU 利用率。现代 LLM 推理引擎以分页 KV 缓存的形式自动使用这一机制。

最好把权重和 KV 缓存的数据移动视为整体推理流水线的一部分。正如应该流水化计算操作一样，你也应该流水化数据移动。在当前计算进行时，始终让下一份所需数据处于传输中。KV 缓存压缩是另一个在 KV 缓存层提升性能的选项。下面讨论它。

19.6 实时 KV 缓存压缩与策略切换

随着 LLM 在一次会话中生成越来越多 token，KV 缓存会线性增长。对于长对话、文档和推理链，KV 缓存会消耗大量 GPU 内存，而且往往是占用 GPU 内存最多的部分。

KV 缓存很适合压缩/量化。与任何压缩形式一样，KV 缓存压缩会减少其内存占用。实时执行意味着在推理期间即时压缩。

策略切换意味着压缩策略可以根据当前上下文变化。目标是在需要时释放内存和网络带宽——同时不影响模型精度，也不拖慢涉及 KV 缓存数据的计算。图 19-4 展示了几种不同类型的 KV 缓存压缩算法。

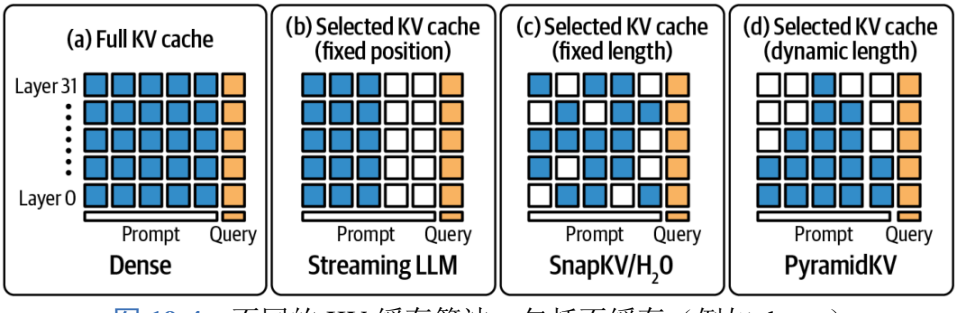

图 19-4 不同的 KV 缓存算法，包括不缓存（例如 dense）。

一种直接而简单的 KV 压缩方法是降低精度。许多框架默认对 KV 缓存使用 FP16 或 BF16，因为 16 位通常是模型用于激活的精度。不过，通常可以把 key 和 value 压缩到 8 位甚至 4 位，而对输出质量影响很小——尤其是 LLM 上下文末尾的 token。

Hugging Face Transformers 库支持 QuantizedCache，包括用于 KV 内存的 INT8 和 INT4。只需一行即可启用：指定 cache_implementation=”quantized” 和具体位宽。代价是量化/反量化操作带来少量额外计算，换来巨大的内存节省。在大多数情况下，整体模型质量不会受损。

> **注** 当量化与 CPU 卸载同时使用时，确保主机缓冲区是页锁定（pinned）的，以防止传输串行化。这有助于维持复制带宽（例如 PCIe/NVLink）。

接下来讨论动态策略切换。一个策略示例是：把最后 128 个 token 保持全精度，其余 token 压缩为 4 位。这样，最近的上下文——它很可能对预测下一个 token 影响最大——以更高精度保留，而较旧的历史以更低精度存储以节省空间。

如果模型突然需要关注更旧的 token，通常也不会造成灾难，因为许多 LLM 本来就有近因偏差。这意味着它们会优先考虑近期上下文。因此，输入序列中较早的部分对最终输出的影响可能没那么大。

你还可以根据用户提示词长度进一步调整这个窗口。例如，对很长的提示词使用更大的全精度窗口——或者在 GPU 内存使用率超过阈值时更激进地压缩。

另一种策略可以基于内存使用率。例如，策略可以规定：如果 GPU 内存使用率超过 80%，就把整个 KV 缓存压缩到 8 位。这有助于在长生成期间避免 OOM 错误。策略还可以包含多级压缩：在轻度压力下把 KV 缓存压到 8 位，在极端压力下再改为 4 位压缩。

在真正的动态实时策略切换中，引擎可以在 token 生成期间切换到不同压缩方式。此时，实现需要同时维护缓存的多种表示。例如，它最初以 FP16 存储 KV，但同时维护同一数据的 INT8 版本。

系统默认使用 FP16，但如果内存利用率越过某个阈值，就可以开始使用带适当缩放因子的 INT8 版本——并释放 FP16 内存以缓解内存压力。之后，注意力读取会从 INT8 存储中取回反量化后的值。

不过，这需要仔细同步，以确保压缩版本保持最新，并在需要时已经就绪。双缓冲和后台压缩线程等技术在这里很有用。

通常，一个 CPU 线程可以用向量化 INT8 量化操作异步完成压缩。准备好后，它再把压缩块复制到 GPU 内存。

> **注** 在安全点实施实时策略切换，例如一次迭代结束时。这样可以避免计算中途切换，并通过在后台 stream 中执行来隐藏重新量化的延迟。

还有其他技术，例如无损压缩：使用熵编码和聚类，在不丢失任何位信息的情况下压缩激活。然而，这些实现很复杂，而且实时执行可能太慢——即使在 GPU 上也是如此。

可以考虑更简单的机制，例如分块 ZFP（一种浮点压缩），甚至通用的基于 CPU 的压缩。不过，迄今为止最简单、最有效且支持最好的方法仍是量化。

> **注** 截至本文写作时，ZFP 等无损方法主要在离线和研究场景中评估，但由于相对量化缓存存在吞吐量限制，它们在生产 LLM KV 缓存路径中仍不常见。因此，量化仍是首选方案，因为它在速度和 2–4× 内存缩减之间取得了平衡。

为把质量影响降到最低，可以尝试按 head 和按 token 缩放。KV 缓存量化最有效的是按 head 的分组缩放，而不是按 token 缩放。Hugging Face QuantizedCache 的 Transformer 实现会按注意力头校准数值范围。

具体来说，QuantizedCache 实现按通道的分组量化，组大小可配置，并带一个残差窗口，把最近的 token 保持原始精度。启用方法是设置 cache_implementation=”quantized”，并传入字典形式的 cache_config。你可以计算张量中的最大绝对值，并据此缩放 4 位或 8 位量化。这本质上是一种 min-max 或基于幅值的量化。

QuantizedCache 的一个有用实现是 Half-Quadratic Quantization（HQQ，半二次量化）后端。HQQ 提供无需校准的即时量化器，支持多种低位格式，包括 2 位、3 位、4 位和 8 位。它使用稳健优化来建模离群值和重尾误差分布。HQQ 能很好地集成到 Hugging Face Transformers 的 KV 缓存实现中。它同时提供 PyTorch 和自定义 CUDA kernel 实现，用于快速推理。

我们可以实现一种动态策略，根据内存压力在 8 位量化和 4 位量化之间切换。数值的锐度或分布也可能指导决策。如果缓存值大多很小且方差低，通常可以更激进地量化。实时切换压缩策略可以与 Hugging Face QuantizedCache 机制集成。

遗憾的是，Transformers 不支持原地更改已初始化缓存对象的位宽。不过，要实现动态策略，我们的代码可以分小块生成 token，并在内存压力出现时，让下一块即时使用新的量化缓存配置。这种实现类似于在遇到显存不足（OOM）错误时回退到卸载缓存。代码如下：

```
# dynamic_quantized_cache.py
# Dynamic KV cache policy using Hugging Face Transformers'
 ↪  QuantizedCache (2025).
# Starts with int8 HQQ and drops to int4 when device memory is tight.
# Requires: transformers >= 4.55, hqq (for HQQ backend).
#
# This uses only public APIs:
#   - cache_implementation="quantized"
#   - cache_config as a dict
#
# References:
#   - KV cache strategies docs (QuantizedCache, HQQ/Quanto backends)

from __future__ import annotations
from typing import Dict, Optional
import logging
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer


def make_cache_config(
    *,
    backend: str,
    nbits: int,
    device: torch.device,
    compute_dtype: torch.dtype = torch.float16,
    q_group_size: int = 64,
    residual_length: int = 128,
    axis_key: int = 1,
    axis_value: int = 1,
) -> Dict:
    """
    Build a cache_config dictionary accepted by Transformers' quantized
     ↪  cache.
    HQQ supports nbits in {2, 4, 8}; Quanto supports {2, 4}.
    axis_key/axis_value=1 are recommended for HQQ.
```

```
    """
    return {
        "backend": backend,                 # "HQQ" or "quanto"
        "nbits": int(nbits),
        "axis_key": axis_key,
        "axis_value": axis_value,
        "q_group_size": int(q_group_size),  # group size along head_dim
        "residual_length": int(residual_length), # recent tokens (orig
        ↪ precision)
        "compute_dtype": compute_dtype,     # dequantization compute
        ↪ dtype
    }


def _gpu_used_ratio() -> float:
    """
    Return fraction of device memory used as 1 - free/total.
    Uses CUDA driver info, which reflects true device state,
    not just the PyTorch allocator's reserved bytes.
    """
    free, total = torch.cuda.mem_get_info()
    return 1.0 - (free / total)


@torch.no_grad()
def generate_with_dynamic_quantized_cache(
    model: AutoModelForCausalLM,
    tokenizer: AutoTokenizer,
    prompt: str,
    *,
    max_new_tokens: int = 256,
    chunk_tokens: int = 32,
    memory_threshold: float = 0.90,   # switch policy if used_ratio >=
    ↪ threshold
    backend: str = "hqq",             # "hqq" or "quanto"
    start_bits: int = 8,              # initial cache bit-width
```

```
    fallback_bits: int = 4,           # lower bit-width on pressure
    residual_length: int = 128,
) -> str:
    """
    Generate text in chunks while allowing mid-run policy changes.
    The policy applies to each chunk by choosing cache_config for that
    ↪ chunk.
    If memory is tight, we switch from int8 to int4 in subsequent chunks.
    """
    backend = backend.lower()
    assert backend in {"hqq", "quanto"}, "backend must be 'hqq' or
    ↪ 'quanto'"
    if backend == "quanto":
        assert start_bits in {2, 4} and fallback_bits in {2, 4}, \
               "Quanto supports only 2 or 4 bits"
    if backend == "hqq":
        assert start_bits in {2, 4, 8} and fallback_bits in {2, 4, 8}, \
               "HQQ supports 2, 4, or 8 bits"

    device = model.device
    inputs = tokenizer(prompt, return_tensors="pt").to(device)
    generated_ids = inputs["input_ids"]   # [batch=1, seq_len]
    tokens_remaining = int(max_new_tokens)
    current_bits = int(start_bits)

    # Use EOS if available to terminate early.
    eos_id: Optional[int] = tokenizer.eos_token_id

    while tokens_remaining > 0:
        # Decide policy for this chunk based on current memory pressure.
        if torch.cuda.is_available():
            # Smooth the signal to avoid oscillation
            # when multiple processes are active.
            if 'used_ratio' in locals():
                used_ratio = 0.8 * used_ratio + 0.2 * _gpu_used_ratio()
            else:
```

```
            used_ratio = _gpu_used_ratio()
        if used_ratio >= memory_threshold:
            current_bits = min(current_bits, fallback_bits) # drop
             ↪ bits
            logging.info(f"Current bits {current_bits}")

    cache_cfg = make_cache_config(
        backend=backend,
        nbits=current_bits,
        device=device,
        compute_dtype=torch.bfloat16,
        q_group_size=64,
        residual_length=residual_length,
        axis_key=1,
        axis_value=1,
    )

    # Generate a small chunk with the chosen cache policy.
    this_chunk = min(chunk_tokens, tokens_remaining)
    out = model.generate(
        input_ids=generated_ids,
        max_new_tokens=this_chunk,
        do_sample=False,       # deterministic for clarity; adjust
         ↪ as needed
        use_cache=True,
        cache_implementation="quantized",          # select
         ↪ QuantizedCache
        cache_config=cache_cfg,                    # pass backend +
         ↪ settings
        pad_token_id=eos_id,
        return_dict_in_generate=False,        # we only need the
         ↪ tokens here
    )
    # 'out' is [1, old_len + this_chunk]; slice out newly generated
     ↪ suffix
    new_tokens = out[:, generated_ids.shape[1]:]
```

```
        generated_ids = out
        tokens_remaining -= new_tokens.shape[1]

        # Early termination if the model emitted EOS.
        if eos_id is not None and int(new_tokens[0, -1].item()) ==
        ↪  eos_id:
            break

    return tokenizer.decode(generated_ids[0], skip_special_tokens=True)


if __name__ == "__main__":
    # Example usage. Replace with a model that supports your hardware.
    ckpt = "meta-llama/Llama-3.1-8B-Instruct"
    tok = AutoTokenizer.from_pretrained(ckpt)
    mdl = AutoModelForCausalLM.from_pretrained(ckpt,
                                   torch_dtype=torch.float16).to("cuda")

    text = generate_with_dynamic_quantized_cache(
        mdl,
        tok,
        "Explain attention key-value caches in one paragraph.",
        max_new_tokens=120,
        chunk_tokens=32,
        memory_threshold=0.90,
        backend="hqq",          # or "quanto" if you installed Quanto
        start_bits=8,
        fallback_bits=4,
        residual_length=128,
    )
    print(text)
```

这里，我们先用 INT8 HQQ 缓存实现适度压缩，当 GPU 实际空闲内存低于阈值时切换到 INT4。用 torch.cuda.mem_get_info() 度量，它反映真实的空闲设备内存与总内存。这为策略选择提供了正确的信号。

然后我们分小块生成 token，这样就可以在块之间安全切换策略，而不必试图修改现有缓存实例。这避免了访问私有属性或手动量化张量。缓存后端会在模型前向传播内部完成工作。

> **注** 如本例所示，建议在策略切换时记录事件或递增计数器。这样，你可以把压缩事件与任何精度或输出异常关联起来。

类似地，如果条件改善，你可以动态关闭压缩。假设一段长对话刚刚结束，下一个问题很短。系统可以决定停止压缩，甚至把某些缓存恢复到更高精度，以产生更高质量的响应。差异可能很小，因此也许不值得这么做。

重要的是避免压缩频繁波动，因为过于频繁地开关压缩可能造成性能抖动。为此，可以在变更之间引入有意延迟（即迟滞和冷却）。例如，如果切换到更高压缩策略，就保持该策略，直到 GPU 内存降到给定阈值以下很多。这样可以避免振荡和抖动。

这种灵活性很有用，例如，如果你的服务有时为高级用户优先考虑最高质量（不压缩），而对免费层用户优先考虑最大吞吐量（重度压缩）。策略也可以根据请求元数据切换，包括用户的订阅类型。

讨论缓存不能不提驱逐策略，例如最近最少使用（LRU）驱逐。如果上下文长度变得太长，某些模型架构——例如带近因偏差或滑动窗口注意力的架构——可能选择完全丢弃或降采样非常旧的 token。滑动窗口注意力如图 19-5 所示。

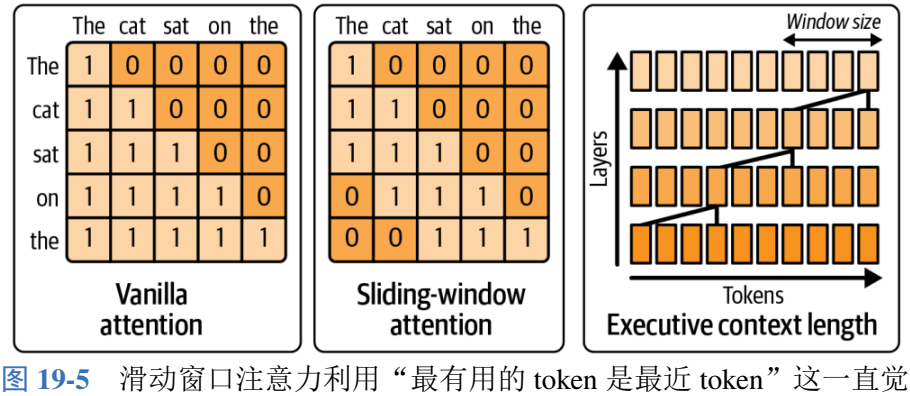

图 19-5 滑动窗口注意力利用“最有用的 token 是最近 token”这一直觉。

从上下文中驱逐较早 token 的 LRU 并不完全是压缩，但它是另一种可以在运行时动态选择的策略。例如，系统可以根据某些启发式规则或更小的 LLM 判断：超过 2,048 个 token 后，模型可能不再需要较早的 token。

在这种情况下，系统可以开始丢弃这些更旧的 token——或定期把它们压缩成更小的摘要。这开始进入模型和算法领域——需要更多支持、维护和模型训练——但它是一种动态上下文管理形式，高级服务引擎应考虑。

简而言之，你应考虑推理引擎提供的量化缓存机制，因为它们能处理维护量化缩放因子、对接注意力 kernel，以及在运行时监控 GPU 内存分配器等细节。至少，当系统看到内存利用率接近某个上限时，应记录该事件，并评估此时启用压缩策略能否在不损害延迟的情况下避免 OOM。

> **注** 实践中，在 GPU 内存上设置高水位阈值（例如 80%）来触发 8 位压缩，已被证明能有效防止生产环境中的 OOM 崩溃。

如果使用动态压缩策略有意义，就可以实现触发器。与任何量化和压缩策略一样，务必针对你的领域测试它们对模型输出的影响。许多生成任务能容忍激进压缩，但始终建议验证 4 位相对 8 位不会引入错误或意外输出。

19.7 用于运行时调优 AI 系统的强化学习智能体

到目前为止讨论的许多技术都涉及根据系统当前状态做决策。这些决策往往需要权衡，例如速度与精度、吞吐量与延迟。与其收集越来越多的启发式规则来做决策，不如用强化学习来调优推理系统，其中包含 RL 智能体、环境、策略和奖励。

> **注** 这是一种前沿方法。你应该先从更简单的启发式规则作为基线。等基础稳定后，再把 RL 作为增量改进引入。

具体来说，我们的推理引擎观察服务器指标（环境），选择动作（策略）以最大化吞吐量并把延迟保持在目标以下，并接收指导持续改进的反馈（奖励）。这样，系统就成为一个在线优化器——随着条件变化不断改进决策。

例如，可以建立一个环境，其中每个 RL“步骤”都是一次推理请求。RL 动作包括以下内容：

动作 1：选择并行模式：单卡、TP、PP 和混合。

动作 2：选择精度：全 FP8，还是混合 FP8 与 FP4。

动作 3：调整批大小或批次等待时间。

动作 4：启用或禁用缓存压缩。

动作 5：启用或禁用推测解码。

动作 6：为推测解码选择更小的草稿模型。

动作 7：为推测解码选择更大的草稿模型。

动作 8：启用或禁用推测式 KV 预取。

……更多动作……

智能体观察的当前 RL 状态可能包括 GPU 利用率、内存利用率、平均延迟、请求队列长度等。RL 奖励则通过捕捉业务目标来定义，例如 $reward = throughput - \lambda \cdot \max(0, latency - SLA)$。λ 是该奖励函数中的可调惩罚权重，用于缩放你对延迟违规的惩罚力度。

> **注** 建议对状态特征做归一化，这样 RL 智能体就不必自己学习尺度。这可以加快训练收敛。例如，可以按最大值缩放队列长度等。

这里，较大的 λ 会让智能体优先保持在 SLA 内，而不是榨取额外吞吐量。较小的 λ 则允许它冒偶尔延迟超标的风险，以换取整体更高的 token 速率。本质上，这个奖励函数会惩罚超过 SLA 的延迟，但在其他情况下尽量提高吞吐量。

> **注** 实践中，初始 λ 应使 λ ×（典型延迟超标量）大约等于你愿意为此交换的吞吐量收益。例如，如果为了获得 100 tokens/sec 可以容忍 10 ms 延迟，就设 λ 使 10 ms × λ ≈ 100。

经过多次迭代，RL 智能体可以学会何时压缩缓存是有益的——例如内存高而延迟没有立即受到影响时。或者，它可以学会在 GPU 利用率低但某块 GPU 过载时切换到流水线并行模式，等等。

这之所以有帮助，是因为 PP 把模型拆成跨多个设备的顺序阶段，把繁重工作从瓶颈设备上重新分配出去——平滑利用率并避免单 GPU 热点。

智能体可以发现反直觉但性能更好的配置。例如，它可能学到：对于超过一定长度的提示词，应同时启用 PP 和 FP4 压缩以获得最佳 token 吞吐量；而对于较短提示词，只使用纯张量并行加 FP8。如果我们试图把这种逻辑编码成一组静态规则，就可能漏掉复杂的交互。

RL 智能体可以通过探索动作空间，更容易地发现最优组合——并最终利用最优配置，直到环境条件变化。此时，RL 会相应调整推理系统，因为它始终在探索动作空间并尝试新配置。

这样的智能体可以用模拟器和 Hugging Face Transformer RL（TRL）等库离线训练。例如，我们可以记录运行系统在各种条件下的许多数据——然后在模拟中训练 RL 策略来预测结果。在高层次上，RL 奖励与更新循环大致如下伪代码所示：

```
# Pseudo structure for an RL-driven tuner
# This loop runs in separate thread/process alongside main inference
↪ service.

# e.g., {gpu_util:0.7, mem_util:0.9, avg_latency:120ms, req_queue:5}
state = get_system_state()

# e.g., 0 -> high precision, 1 -> low precision
action = rl_agent.select_action(state)

# Map action to actual parameter changes
if action == 0:
    precision_policy = "FP8"
else:
    precision_policy = "FP4"

# (We could have multiple actions, but single action for illustration)
apply_precision_policy(precision_policy)

# ... After the next token or set of tokens ...
```

```
new_state = get_system_state()
reward = compute_reward(old_state, new_state)
rl_agent.update(state, action, reward, new_state)
```

这里，该循环在推理服务器的后台持续运行。compute_reward 函数会纳入吞吐量（例如自上一一步以来的 tokens per second）和延迟指标。由于我们要在吞吐量与延迟之间取得平衡，这是一个多目标优化问题，需要同时优化多个目标。常见做法是用加权和把多个目标组合成单一目标。

> **注** 为了获得更大灵活性，尤其是在不确定性下，你可以把多目标优化问题——或 Pareto 前沿分析——建模为部分可观测决策过程。这允许智能体学习自己在吞吐量与延迟等目标之间的权衡策略。当单一加权奖励不够时，这很有帮助。

这类多参数交互很难用基本的网格搜索方法调优。因此，RL 和近端策略优化（PPO）等优化技术最适合调优推理工作负载。PPO 以在连续动作空间中学习更稳定而著称。它适合在实时环境中持续更新，因为它会逐步调整策略。这避免了剧烈振荡，而这对推理稳定性很重要。我们不希望智能体在每个请求上都来回改变决策。

另一种减少振荡的技术称为阻尼。它要求某个动作至少持续一段时间——或至少持续一定数量的请求。如有需要，你可以为严重的 SLO 违规覆盖阻尼，但应尽量少用。

需要知道的是，RL 智能体在学习过程中可能做出不安全或次优的动作。为缓解这一点，可以把动作空间限制在合理的范围内。还建议用我们已经识别出的启发式规则作为良好的默认策略。然后，智能体可以围绕这个初始默认策略进行微调。

或者，智能体可以在影子模式下用实时系统在线训练，并加入探索阶段。探索期间，系统偶尔尝试随机或稍作修改的策略以收集新数据。其他时候，它利用当前最佳策略。

另一种技术是奖励塑形，防止智能体违反关键约束。例如，如果延迟超过硬上限——或者错误动作导致 OOM 错误——RL 系统会产生很高的负奖励。

此外，你可以硬编码系统来避开不安全动作——即使奖励提示系统这么做。这增加了额外保护，使智能体的自然探索不会导致灾难性失败。这是一种把 RL 与基于规则的护栏结合起来的实用方法。

设计合适的奖励函数很重要。例如，如果我们关心延迟上限下的吞吐量，奖励可以像下面代码这样：

```
reward = tokens_per_second - 1000 * (1 if latency > SLA else 0)
```

这里，如果超过延迟 SLA，就施加很大惩罚。否则不施加惩罚。另一种选择是施加与延迟超出目标 SLA 的程度成比例的连续惩罚。简单的连续惩罚奖励可以写成如下形式：

```
reward = tokens_per_second − λ * max(0.0, latency − SLA)
```

这里，λ 是你的惩罚权重，max(0.0, latency −SLA) 会随超出 SLA 的程度线性增长。这样，智能体的延迟超标越多，惩罚就平滑增加。这会产生更平滑的梯度和更渐进的权衡决策。实践中，连续（软）惩罚通常比二值（硬）惩罚产生更稳定的策略。

> **注** 建议从一组简单、静态的调优启发式规则开始。等系统稳定后，再引入 RL 智能体来处理启发式规则无法捕捉的更复杂调优。

日志和可观测性很重要。你应持续记录 RL 智能体做出的决策——以及决策结果。例如，应使用结构化日志——甚至计数器和遥测仪表盘——实时跟踪“状态 → 动作 → 奖励”序列。如果智能体开始行为异常，这有助于调试。

还建议提供逃生通道或紧急开关。这样，如果智能体开始做明显糟糕的事情，例如持续让延迟变差，你可以让系统回退到安全的静态配置，同时诊断问题并离线重新训练新策略。例如，如果启用智能体后 p95 延迟增加超过 50%，系统会自动禁用智能体的动作，并向值班系统发送警报。

截至本文写作时，基于 RL 的在线推理调优在现代推理服务引擎中尚未成为主流，但刚刚开始出现。随着这些技术成熟，预计会有更多推理平台包含自调优能力。这很重要，因为这些模型和系统正变得越来越复杂。在快速变化的条件下手动管理所有这些调优旋钮很困难——反正对人类来说很困难！

能实时自适应的智能体是系统优化的自然演进。我们开始看到能自动达到专家级性能调优的自优化 AI 推理服务器。它们只是通过从自身的实时遥测指标中学习来做到这一点。

19.8 动态内存分配切换（slab、缓存与流序）

GPU 内存碎片化和非最优内存分配可能是无声的性能杀手。推理服务器每秒为许多对象（包括 token、中间激活等）分配和释放数千个张量。内存分配器采用的策略会影响碎片化和分配延迟。

动态切换内存分配器意味着系统可以即时改变内存分配方式。例如，系统可以对某些分配尺寸使用 slab 分配器——或者切换到 CUDA 的流序（cudaMallocAsync）分配器。决策取决于观察到的分配模式和预期的内存碎片化。

默认情况下，PyTorch 使用 buddy/最佳适配内存分配器的一种变体，称为带合并的最佳适配（BFC）。它抓取大块 GPU 内存，再细分这些块以满足分配请求。这复用了空闲空间，并避免频繁调用相对较慢且同步的 cudaMalloc 和 cudaFree。

buddy 分配器把内存拆成大小为 2 的幂的块。slab 分配器工作在 buddy 系统之上，用来高效管理小型、固定大小的对象。它预分配 slab（即给定类型对象的集合），并在每个 slab 内维护空闲链表，如图 19-6 所示。

slab 分配器允许快速复用且不产生碎片。buddy 分配器处理粗粒度页分配，而 slab 分配器优化细粒度对象复用。

默认的 PyTorch 缓存分配器对许多工作负载表现良好。然而，如果分配模式因大小分配交替而差异很大，它就可能遭受碎片化。在长期运行、处理不同类型查询的服务器中，碎片化会逐渐积累。

在这种情况下，大量内存其实是空闲的，但内存不够连续，无法满足大张量分配。这会导致 OOM 错误——即使技术上还有可用内存。

记住，PyTorch 提供 torch.cuda.memory_summary() 来评估内存碎片化，torch.profiler 中还有内置的内存分析器（profile_memory=True）。你可以用它们确定哪些操作分配了大量内存。此外，NVIDIA Nsight Systems 在时间线上提供 CUDA 内存事件和统一内存缺页跟踪，Nsight Compute 提供内存工作负载分析。

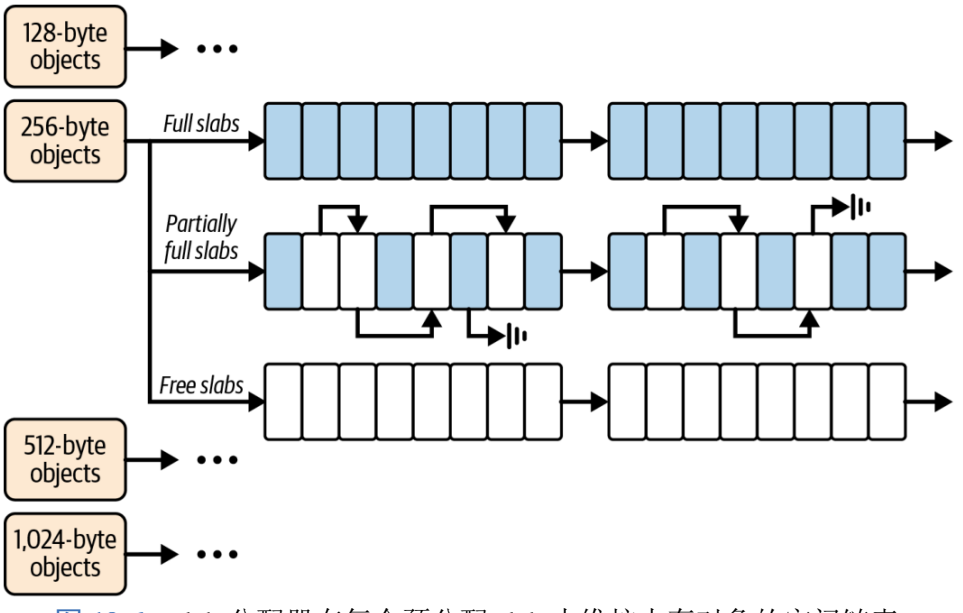

图 19-6　slab 分配器在每个预分配 slab 内维护内存对象的空闲链表。

这些工具合起来让你观察随时间的分配行为和碎片化影响。你可以在开发和测试期间使用这些工具来启动内存分配器调优策略——包括接下来讨论的动态调优策略。

减少内存碎片化的一种粗暴方法是定期重置分配器状态。不过，更聪明的做法是使用 CUDA 的流序缓存分配器 cudaMallocAsync；它在内部采用类似概念，按分配尺寸分箱到某个上限。而 slab 分配更进一步，从不让不同尺寸混在一起。

cudaMallocAsync 的行为有点像一个 slab 分配器与 buddy 系统的结合——而且由 CUDA 驱动替你管理。它以很小的代价给你自定义分配器的大部分好处——如果你愿意，它是一个很适合一直开启的默认内存分配器。

具体来说，cudaMallocAsync 使用流序池：当内存引用被释放时自动回收内存。随后它在幕后合并已释放块，因为它知道内存释放的依赖顺序——这一点与标准分配器不同。

在 PyTorch 运行时中使用 cudaMallocAsync 时，可以通过设置环境变量 PYTORCH_ALLOC_CONF=max_split_size_mb:<value> 来动态调整 max_split_size_mb。这可以在不同条件下调整拆分尺寸。

例如，动态系统可以在预期有大分配时增大 max_split_size_mb。这样它们就不会被拆成小块。反过来，系统在运行许多小请求时可以减小 max_split_size_mb，以便更多复用大块。

> **注** 拆分尺寸太小会让分配器充满许多微小块，从而增加元数据开销和潜在碎片化。拆分尺寸太大则会减少块数量（和元数据），但当你只释放块的一部分时，可能在内存中留下更大的“空洞”而无法使用。

考虑一个场景：你的服务检测到碎片化——也许使用 PyTorch 的内存快照功能，它显示了碎片化造成的空洞。在这种情况下，系统可以动态切换到 cudaMallocAsync，它可以整合内存使用。

你应使用内存监控工具跟踪——并记录——内存碎片化。例如，在 PyTorch 中，可以使用 torch.cuda.memory_reserved() 和 torch.cuda.memory_allocated()。这里，reserved 是分配器持有的 GPU 内存总量，allocated 是其中实际被张量使用的部分。

reserved 与 allocated 之间差距很大，就说明存在碎片化，因为大量内存被保留却未使用。如果这个差距随时间增长，动态策略可以是定期清空缓存，把所有未使用内存归还 GPU，甚至重启工作进程以完全重置分配器。这些方法具有侵入性但有效，有时会在生产环境中用于碎片化严重的长期运行进程。

> **注** 像清空缓存和重启这样的侵入式碎片整理方法，只应在维护窗口期间使用——或者在整个机群中以滚动重启方式进行，以避免停机。如果你需要诉诸这些破坏性机制，很可能存在更深层的问题需要解决和优化。

例如，要在 PyTorch 中实现动态分配切换，可以先使用 PyTorch 原生分配器。然后，如果捕获到 OOM 错误，就可以用基于 cudaMallocAsync 的分配器重试。

不幸的是，CUDA 缓存分配器会在首次导入 torch 或首次接触 CUDA 上下文时创建。而 Python 的 importlib.reload 不会卸载 C++ 扩展，也不会拆除分配器。因此，即时更改 PYTORCH_ALLOC_CONF（原 PYTORCH_CUDA_ALLOC_CONF）并重新加载 Python 模块，不会在进程内重新配置分配器。

不过，你可以启动一个新进程，在导入 torch 之前设置环境变量。下面这段代码捕获 OOM，释放父进程中的内存，然后用 PYTORCH_ALLOC_CONF 启动一

个干净的子进程。这有点取巧，但展示了如何动态设置 backend:cudaMallocAsync，并用不同的分配器后端重新运行同一调用。下面是一个 PyTorch 示例，在代码捕获 torch.cuda.OutOfMemoryError 时实现这种动态策略：

```
# dynamic_memory_allocator.py
# Retry generation in fresh process with cudaMallocAsync if first
 ↪ attempt OOMs.
# This is the only reliable way to change the CUDA allocator at runtime.

import os
import sys
import gc
import pickle
import tempfile
import subprocess
import importlib
from typing import Callable, Any


def _resolve_factory(factory_path: str) -> Callable[[], Any]:
    """
    Resolves a factory string like "my_package.my_mod:build_model" to a
     ↪ callable.
    The callable must return ready-to-use model with .generate(request)
     ↪ method.
    """
    module_name, func_name = factory_path.split(":", 1)
    module = importlib.import_module(module_name)  # safe to import, no
     ↪ torch yet
    return getattr(module, func_name)


def generate_with_allocator_retry(
    model_factory_path: str,
    request_object: Any,
    *,
```

```
    allocator_conf: str = "backend:cudaMallocAsync"
) -> Any:
    """
    Attempts model.generate(request_object) in the current process.
    On torch.cuda.OutOfMemoryError, retries in a fresh subprocess with
    PYTORCH_ALLOC_CONF set to allocator_conf. request_object and the
    returned value must be picklable.
    """
    # Import torch only inside function; avoid importing at module
     ↪ import time.
    import torch

    model_factory = _resolve_factory(model_factory_path)
    model = model_factory()  # user-supplied function builds model,
     ↪ moves to GPU

    try:
        # First attempt uses whatever allocator current process started
         ↪ with.
        return model.generate(request_object)

    except torch.cuda.OutOfMemoryError:
        # Free as much as possible in the parent before spawning the
         ↪ child.
        # Avoids compounding pressure when two processes momentarily
         ↪ overlap.
        try:
            del model
        finally:
            gc.collect()
            torch.cuda.empty_cache()

        # Serialize request to temp file and ask fresh interpreter to do
         ↪ work.
        with tempfile.TemporaryDirectory() as td:
            req_path = os.path.join(td, "request.pkl")
```

```
            out_path = os.path.join(td, "output.pkl")
            with open(req_path, "wb") as f:
                pickle.dump(request_object, f)

            # In the child, we want torch to see allocator config at
             ↪ import.
            env = os.environ.copy()
            env["PYTORCH_ALLOC_CONF"] = allocator_conf

            # Re-run this module as a helper child. The child will
             ↪ import torch
            # only after PYTORCH_ALLOC_CONF is set in its environment.
            cmd = [
                sys.executable,
                __file__,
                "--child",
                "--factory", model_factory_path,
                "--request", req_path,
                "--output", out_path,
            ]
           completed = subprocess.run(cmd, env=env, capture_output=True,
                                      text=True)

            if completed.returncode != 0:
                # Bubble up child stderr to aid debugging
                raise RuntimeError(
                 f"Retry failed with exit code {completed.returncode}\n"
                  f"stdout:\n{completed.stdout}
\nstderr:\n{completed.stderr}"
                )

            with open(out_path, "rb") as f:
                return pickle.load(f)
```

```
def _child_main(factory_path: str, request_path: str, output_path: str)
 ↪  -> None:
    """
    Child entrypoint: assumes PYTORCH_ALLOC_CONF is already present in
     ↪  env.
    Imports torch only now, builds the model, runs generate, and pickles
     ↪  result.
    """
    # Import torch after env var set by the parent' s
     ↪  subprocess.run(env=...).
    import torch

    model_factory = _resolve_factory(factory_path)
    model = model_factory()  # build the model inside the child process

    with open(request_path, "rb") as f:
        request_object = pickle.load(f)

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--child", action="store_true")
    parser.add_argument("--factory", type=str, default="")
    parser.add_argument("--request", type=str, default="")
    parser.add_argument("--output", type=str, default="")
    args = parser.parse_args()
    if args.child:
        _child_main(args.factory, args.request, args.output)
    else:
        print("This module is intended to be imported.")
```

这里，模型在子进程中通过工厂构建，因此在 PYTORCH_ALLOC_CONF 环境变量生效之前，不会导入任何与 CUDA 相关的内容。代码使用 torch.cuda.empty_cache() 清空缓存并把未使用内存释放给 GPU。这种模式保证分配器配置在子进程导入 torch 之前生效。它还避免了在运行时尝试卸载原生扩展，而 CPython 并不支持这样做。

在非 PyTorch 环境（例如基于 C++ 的 LLM 推理引擎）中，可以实现一个纯 slab 分配器，允许针对特定分配尺寸进行配置。这种 slab 分配器把内存预先划分为固定大小的“slab”。它对于重复分配相同尺寸非常高效，并且对该特定尺寸的分配几乎不会产生碎片。

在 LLM 服务器中，一种非常常见的技术是分配逐 token 的输出张量：例如，每次生成 token 时，为该 token 的激活分配一个 [layers, hidden_dim] 张量。如果这些分配每次都大小相同，例如 64 KB，那么针对该精确尺寸的 slab 就很理想。

系统检测到自己反复分配大量 64 KB 张量——于是创建一个由专用 64 KB 块组成的“slab”。slab 分配器通常直到整个 slab 被释放时才把内存归还给通用池。

> **注** 现代 LLM 推理引擎会对每个生成的 token 执行这种缓冲区复用。这不同于每次都释放并重新分配内存，后者会带来很高的开销——尤其是在 token 粒度上。

分配器切换可能涉及为系统不同部分使用完全不同的分配器。例如，你可以对模型权重这类大型静态分配使用默认缓存分配器，因为它们在推理期间释放得较少。然后，对临时的逐 token 分配使用自定义分配器。

> **注** 建议在架构上把长期存活分配（例如模型权重）与短期分配（例如 token 缓冲区）分开。这样可以更容易地把短期和长期分配导向不同的分配器和内存池。

你应始终为 OOM 错误实现回退策略。许多生产系统使用多层内存：如果 GPU 发生 OOM，它会卸载到 CPU 并重试，而不是让请求失败。例如，你可以动态释放缓存、把某些层卸载到 CPU，或压缩缓存。

简而言之，动态分配器管理和多层内存等技术能确保系统在不同负载下长时间运行——同时不产生内存碎片化或分配延迟尖峰。这是一种用户不会直接看到的幕后优化，但对于长期运行的推理服务器实现超大规模鲁棒性至关重要。

19.9 运行时 kernel 性能改进与可热切换实现

在 GPU 硬件和算法创新快速演进的世界中，新的、更快的 kernel 实现不断涌现。这包括 FlashAttention 的新变体、巨型 kernel（megakernel），以及针对特定硬件的软件优化。

运行时 kernel 补丁是指把这些新实现集成到运行中的系统里，而不需要完整重新部署或重新加载模型。本质上，我们想即时把较慢的 kernel 函数热切换为更快的函数。

假设你的推理服务器对多头注意力使用默认的 PyTorch 缩放点积注意力（SDPA）kernel。随后你发现了一个新的 kernel 实现，例如 FlashAttention-3，在某些情况下能为长序列带来 20% 的速度提升。

传统上，你必须安装更新后的库并重启服务器才能使用新实现。但有了运行时补丁，你可以在运行时动态加载新 kernel 并把调用重定向到它，而不会中断服务器的正常运行时间。

这种零停机升级方法对 7×24 服务至关重要，因为重启或重新加载模型会带来过多延迟或导致服务中断。在 Python 中，这可以是一个简单的 monkey patch，如下所示：

```
import new_flash_attn_lib

# Monkey-patch the model's attention forward to use the new library
old_attn_forward = model.transformer.self_attn.forward

def new_attn_forward(self, *args, **kwargs):
    return new_flash_attn_lib.forward(*args, **kwargs)

model.transformer.self_attn.forward = new_attn_forward.__get__(
    model.transformer.self_attn,
    type(model.transformer.self_attn))
```

这段代码把注意力模块的 forward 方法替换为调用 new_flash_attn_lib.forward 的方法。我们把它绑定到实例（__get__）以模

拟正常的方法。打上补丁后，对该注意力层的后续调用都会走新实现。这样，我们就有效地热切换了 kernel。

> **注** 确保 new_flash_attn_lib.forward() 可直接替换（drop-in compatible）——并且已经过充分测试，能在可接受的数值容差内产生相同输出。这样可以避免任何模型质量回退。

另一种技术是 JIT 补丁：使用 PyTorch Inductor 或 OpenAI Triton 等即时编译器在运行时生成更快的 kernel——然后把它插入推理流水线。如下所示，可以使用 PyTorch 的 torch.compile 优化函数并返回其编译版本：

```
compiled_forward = torch.compile(
    model.transformer.self_attn.forward,
    backend="inductor")

model.transformer.self_attn.forward = compiled_forward
```

现在，每当调用 forward 时，都会执行优化后的代码，它会融合多个操作等。如果我们在模型初始化之后做这件事，它就是热补丁的一种形式，因为不需要重新加载模型。它只是替换函数指针。

在 CUDA 侧，可以使用 CUDA 驱动 API 实现运行时模块加载。例如，你可以为新 kernel 编译自定义 PTX——并在不重置设备的情况下把它加载到上下文中。如果函数签名匹配，系统只需更新函数指针。NVIDIA Runtime Compilation（NVRTC）等工具可以在运行时从字符串编译 PTX，从而支持这种工作流。

用 CUDA C++ 做这件事有点复杂。更高层的方法，例如 Python 的 monkey patch 机制——或 PyTorch 的可扩展后端——从长期看可能更实用、更易维护。不过，你的高性能推理引擎很可能并不使用 Python/PyTorch 运行时。

> **注** 进行这类热切换时，保证线程安全很重要。例如，应排空传入请求队列或使用屏障，确保没有其他线程正处于你要打补丁的函数中间。这会避免与运行中的线程发生竞态条件。这类似于操作系统 kernel 加载模块——需要仔细同步，但可以避免完整重启。

使用定义良好的模块边界——例如把注意力和 MLP kernel 分开——可以让替换内部实现更容易。这支持以高度模块化的方式编写模型代码，从而获得这些替换点。单体式模型实现更难打补丁，但可能性能更高（例如巨型 kernel）。

当然，必须确保新 kernel 产生与旧实现相同——或可接受地接近——的结果。通常，这些优化 kernel 被设计为在相同精度（例如 FP16）下数值等价。

热补丁也可以用于快速修复。也许某个特定操作序列正在当前 kernel 中引发已知 bug。与其等待完整更新，你可以快速打补丁修复。

假设你部署了一个 FP8 kernel，它在某些你未预料到的边缘输入上开始行为异常。如果你有一个更安全但稍慢的版本，就可以检测异常条件，并在这些情况下热切换到安全 kernel。确保记录这些事件以供日后分析。

> **注** 考虑采用分阶段发布：把一小部分流量路由到新 kernel（影子测试或金丝雀发布）。然后可以用遥测比较其延迟和吞吐量指标与旧 kernel 的差异。如果新 kernel 表现符合预期且输出匹配预期结果，就可以向所有用户发布。

为了管理多个可能的实现，引擎可以维护一个注册表，例如 attention_impl = ”fast” 或”safe”——并据此分支。这可以实现为功能标志（feature flag），这样无需改代码就能切换实现。这对于仅通过更改标志快速回滚很有用（只需确保代码确实在主动读取功能标志系统中的值——否则标志值会保持静态，这就失去了功能标志的意义）。

设计良好的推理引擎还应测量新 kernel 的运行时性能。如果新 kernel 在真实生产数据下并不更快，推理引擎就可以回滚它。原因可能是硬件略有不同、未测试的批大小或未测试的负载条件。

这是上一节所述自动调优的一种完美形式。如果自动调优器发现某个更快的实现以超过某个阈值的幅度优于当前默认实现，系统就可以触发一次“补丁”，把更快的实现提升为今后的默认实现。

这实际上与自动调优器形成了闭环。系统不仅即时发现更好的 kernel，还会把它们换进来。这形成了一种自优化的 kernel 选择机制。

你可以把这种策略与前面讨论的 RL 策略结合起来。这样，你可以加载多个实现——并让 RL 智能体为每种不同类型的请求选择使用哪一个。

> **注** 注意在内存中维护多个实现所需的开销。增加的代码体积不应把其他关键代码从指令缓存中挤出——也不应耗尽 GPU 内存。

例如，高度优化的 kernel 可能最适合长序列，但更简单的 kernel 实现对短序列可能就足够了。运行时系统可以按请求在二者之间选择。这实际上是根据输入序列长度按请求打补丁。

简而言之，运行时 kernel 补丁关乎灵活性。它承认今天最优的东西明天可能会变，并提供快速适应的手段。这种敏捷性确保你的服务基础设施能跟上模型加速技术的快速发展。vLLM、SGLang、NVIDIA TensorRT-LLM 和 NVIDIA Dynamo 等流行推理引擎架构良好，并提供了大量动态能力。例如，TensorRT-LLM 允许你使用运行时插件加载优化过的 kernel。请利用这些能力。

19.10 使用时间序列预测持续预热 CUDA Graphs 和缓存

在高吞吐量推理场景中，冷启动开销可能是请求—响应延迟的一大贡献项。如第 12 章所述，CUDA Graphs 允许你捕获一系列 GPU 操作，并以最小的启动开销重放该序列。

预热 CUDA Graph 意味着在实际需要之前就把图建立好。如果我们能预测某种请求类型或批大小何时出现，预热好的 CUDA Graph 就能随时执行。

这种技术依赖预测准确性。如果预测偏离，你可能预热了一个用不上的图。你应监控预测命中率，确保这项优化确实有回报。

通过预热，当请求真正到达时，图可以跳过昂贵的启动和初始化步骤。使用 ARIMA 和 Prophet 等时间序列预测算法，系统可以预判未来工作负载模式，包括流量高峰和批大小变化，从而主动准备图以快速执行。

考虑一个推理服务，它观察到每日周期：每天上午 9 点，由于时区相关的流量模式，用户流量会出现尖峰。知道这一点后，系统可以在上午 9 点前预先运行一些请求，把模型加载到 GPU 缓存、JIT 编译必要的 kernel，并为预期批大小捕获 CUDA Graphs。上午 9 点真实流量高峰到来时，传入请求会复用已预热状态，匹配预期的批大小和使用模式。

实践中，这可以由类似 cron 的任务或调度服务编排，在预期流量尖峰出现之前触发预热例程。只需记得给资源供应留出足够时间。

在 PyTorch 中，把模型计算包在 torch.cuda.graph() 上下文中，可以捕获包含 GPU kernel 启动的静态计算图。重放图时，PyTorch 绕过 Python 到 CUDA 的分派开销，用一次 cudaGraphLaunch 提交整个工作负载。这会显著加快执行——尤其是对大型、重复的输入批次，因为 CPU 参与最少。

缺点在于这些图是静态的，因为它们不容易支持可变形状或长度。但推理服务器经常处理离散批大小（1、2、4、8、16 等）——受 GPU 内存限制和算法优化影响——这很适合图模型。

为处理变化，你可以为每个常见批大小或序列长度维护一个预捕获图池。你可以使用持续预热策略为这些离散批大小准备图池。例如，高峰时段 16 个请求的批次很常见。服务器可以预先用批大小 16 捕获模型前向的图——然后存储起来供后续使用。

下一次同时到达 16 个请求的批次时，推理系统把批输入送入预捕获图。图中的 kernel 已经预热并针对图执行优化，因此无需入队和启动许多单独操作，只需一次图启动即可。

请记住，池中存储的每个已捕获图都会为其工作区消耗额外 GPU 内存。存储许多图时，应监控内存使用。如果内存紧张，可能需要从池中驱逐较少使用的图。

为避免图池带来的额外内存，你可以使用第 12 章讨论的图补丁（graph patching），针对较小的尺寸差异调整图节点。不过在实践中，图池是性能更好的选择。

CUDA Graphs 还降低了 CPU 开销，因为它只协调一次图启动，而不是许多单独的 kernel 启动。这让 CPU 可以执行其他操作，例如数据预处理和其他类型的“真正”工作——而不是协调大量 kernel 启动。

同样，你可以使用时间序列预测算法（例如 ARIMA 和 Prophet）预测 RPS 和平均提示词长度等指标。如果模型预测批大小会跳升，或出现特定请求模式（例如长输入序列），系统就可以开始准备并预热相应的 CUDA Graphs、缓存和其他资源。例如，系统可以主动增大连续批处理算法的批大小、把模型权重从磁盘预取到 GPU 内存，并分配额外的 GPU 实例。

> **注** 重要的是频繁用近期流量数据重新训练和更新这些时间序列模型。这是因为使用模式会随时间变化，例如新用户群上线等。此外，像加州著名的滑雪周（Ski Week）这样未预料到的“假日”，可能意外抬高负载曲线——就像我刚搬到加州时在 Netflix 遇到的那样！

与缓存和预热相关的是预判 prefill 和 decode 工作节点的横向扩展。如果我们知道由于计划批处理任务或每日报表场景，一批长提示词请求很可能在某个时间到达，系统就可以扩展预填充工作节点，并用有代表性的样本数据执行一次有代表性的前向传播，以预热 CUDA Graphs 和 KV 缓存。

扩展和预热事件也应包括解码工作节点和 CUDA Graphs。在解码场景中，例如推测解码的草稿模型可以被预热并加载到 GPU 内存——以及草稿模型的 KV 缓存等。其他解码优化包括针对将要生成的固定 token 数量做循环展开——要么 1 个 token，要么在推测解码中多个 token。

还应考虑缓存 CUDA kernel 本身。CUDA 经常缓存编译后的 kernel（C++ 代码 => PTX 指令 => SASS 汇编）以加速执行。kernel 首次运行时，可能存在一些即时 JIT 编译开销。

你应与集群自动扩缩器协调这类预热：当新的 GPU 实例启动时，自动扩缩器用几次预热 API 调用在它们上面运行一小组推理。这样做会验证推理引擎、缓存 JIT 编译输出、分配内存池、准备 CUDA Graphs 等。这样，引擎在加入实时流量池之前就已达到生产就绪状态。

> **注** 利用你对系统的了解，在这个受控预热阶段识别并调用尽可能多的不同路径。这包括每种批大小、每种 CUDA Graph 变体等。

你还应监控预热是否真的有效：比较预测高峰期间最初几个请求的延迟——分别在有预热和没有预热的情况下。根据需要调整时机和阈值。并且要意识到时间序列预测可能出错。确保预热任务如果在错误时间运行，不会影响整体推理性能。

例如，应尽量在 GPU 利用率不高时执行预热运行（使用示例数据）。CUDA 允许 stream 优先级调度，因此可以把预热 stream 的优先级设得低于推理 stream。这样，预热就不会与实时、高流量请求争抢资源。

此外，应为这些预热任务使用较低优先级的 CUDA stream，这样它们会让位于预热期间可能到达的更高优先级实时流量。好的一面是，空闲 GPU 是浪费的机会，因此在空闲 GPU 上做预热计算基本上是免费的——只要不与真实工作冲突。

Grace Blackwell 系统允许一些额外技巧，因为 CPU 和 GPU 以低延迟共享统一、一致的内存。例如，你可以让 CPU 开始把数据预填充到 GPU 计划使用的统一内存中。这可以避免之后的显式 GPU 复制调用。

由时间序列预测引导的持续预热可以让延迟更加可预测。它把推理引擎变成一个自适应系统：学习常见流量模式，自动准备数据、让硬件就绪，并领先需求一步。这会减少抖动，并通过在可摊薄的时间段平滑编译和内存传输等昂贵操作来降低延迟尖峰。

这对于具有沉重一次性初始化成本的大型模型尤其有价值。随着模型和上下文增长，这种自适应预加载将从锦上添花变为生产 LLM 系统的必需品。以预测方式支付这些成本，远比让最终用户因体验不佳而流失要好。

19.11 自适应批处理与分块预填充调度

第 16 章讨论了现代推理服务器如何使用不同类型的请求批处理（例如连续批处理）来最大化吞吐量并最小化所有请求的延迟。然而，批处理可能增加单个请求的延迟。随着一天中条件变化，可以使用一种称为自适应批处理的技术动态处理这些权衡。

自适应批处理根据负载——以及请求进展程度——动态调整请求如何分组为批次。这种动态策略可以随环境变化实时调整批大小和阈值参数。

例如，在高峰负载期间，系统可以使用大批大小（例如 8 或 16），因为吞吐量至关重要。在低负载期间，系统可以减小批大小以更快服务请求。这会把延迟优先于吞吐量。

要决定批大小，可以使用简单启发式规则，例如“如果 GPU 利用率 > 80%，允许更大批次；如果 < 20%，使用批大小 1 以最小化延迟。”或者，可以使用更复杂的 RL 智能体或预测策略，如前几节所述。

预填充和解码阶段之间算术强度的差异导致执行时长不匹配。因此，最好把这些阶段解耦，把它们视为可独立调优的独立工作负载：在单节点上使用独立线程/进程，在多节点集群中使用工作池。

通过解耦预填充和解码，我们把两个阶段视为独立操作。这允许针对它们各自的计算和内存带宽需求独立优化。其中一项优化就是预填充和解码阶段使用的批大小。

实践中，vLLM 和其他现代推理引擎正是这样做的。它们组成单独的批次发送给预填充和解码工作节点。因此，一批预填充请求可以与解码请求批次独立执行。例如，解码阶段是内存受限工作负载，可以从更大批次中受益以提高算术强度。

vLLM 等现代推理服务框架使用自适应调度循环，在处理预填充批次还是解码批次之间动态选择。具体来说，vLLM 支持分块预填充和解码最大化调度，以交错预填充和解码，从而提高利用率。这些技术提升了整体利用率和吞吐量，而不会显著增加延迟。

预填充和解码的系统资源特征不匹配也会影响 PP。假设一个微批在长序列上做预填充，而另一个微批在逐 token 做解码。此时它们的时长不匹配，流水线气泡就会出现。

你可以把大的预填充请求切分成小块，并在它们之间捎带解码，从而把大型预填充与对延迟敏感的解码任务交错起来。这能让所有流水线阶段保持忙碌，并最小化 GPU 调度中的空闲“气泡”。

分块预填充是所有现代 LLM 推理引擎都使用的、支持良好的模式，用于减少流水线气泡。它有效地对大任务（预填充）做时间分片，为小任务（解码）在分块留下的流水线空隙中执行创造空间，如图 19-7 所示。

SARATHI 论文表明，这种分块预填充和捎带方式可以帮助你找到合适水平的解码最大化批处理、减少气泡，并相对朴素调度把吞吐量提高约 1.3–1.9×。SARATHI 这个名字指的是能智能地同时驾驭预填充和解码任务的战车御者。很有趣！

例如，考虑一个不使用分块的 10,000 token 预填充请求。此时，单次预填充会阻塞整个流水线，导致解码任务排队，直到预填充完成。

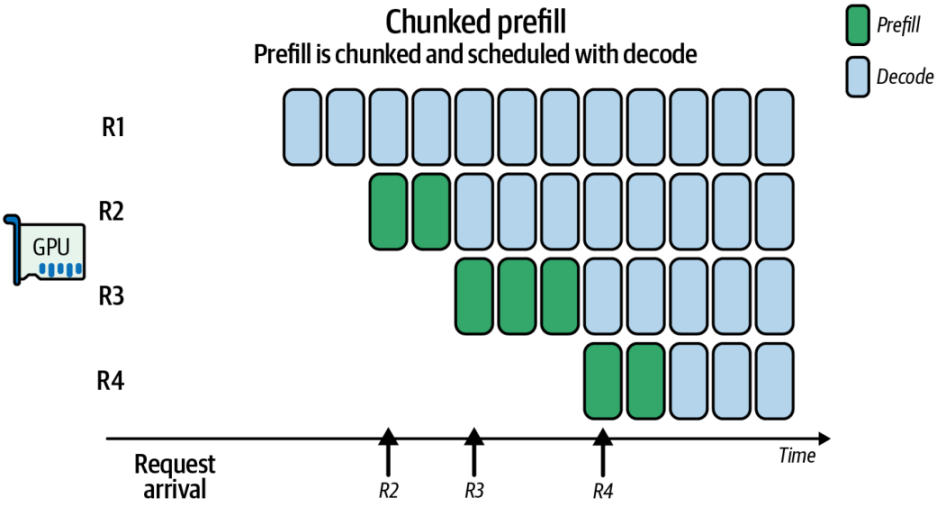

图 19-7　分块预填充在四个请求上实现解码最大化批处理的优势。

然而，如果使用分块预填充，把这个 10,000 token 的预填充请求分成五个 2,000 token 的块，就可以在预填充块之间交错解码批次，让 GPU 保持忙碌地处理两个阶段并推进工作。这会挤出流水线气泡、提高吞吐量并平滑 GPU 利用率。

> 注　一条经验法则是选择这样的块大小：让一个预填充块耗时约 50–100 ms。这样你就有频繁机会在中间调度解码批次。根据模型架构/规模和 GPU 硬件，这可能对应几千个 token。

vLLM 等现代推理引擎使用自适应调度循环，根据 GPU 利用率和队列状态决定是再处理一个预填充块——还是执行一个解码批次。具体来说，vLLM 持续监控 token 队列来做这些决策。vLLM 的调度器明确支持分块预填充和解码最大化批处理。它的执行器和分块预填充特性被设计为让大型预填充与较小的交互式解码重叠。

自适应调度器在选择分块预填充大小时需要考虑 GPU 共享内存限制和占用率。下面展示一个简单的自适应分块预填充实现。这段代码动态调整块大小，让 GPU 上的 SM 和占用率保持较高：

```
# Example adaptive scheduler for chunked prefill/decode
import cupy as cp
import torch

# Hardware constraints
SHMEM_LIMIT   = 256 * 1024
BLOCK_THREADS = 256
TARGET_UTIL   = 0.85
OCC_THRESHOLD = 0.5

# Cache for occupancy results and tile lookup
_occ_cache = {}
_tile_table = {}  # e.g., {L: optimal_T}

# 1) Precompute tile_table offline; here we lazy-initialize on first use
def get_optimal_tile(L):
    if L in _tile_table:
        return _tile_table[L]
    # compute block size by querying for an occupancy-based suggestion
    min_grid, max_grid, block_size = ... # left out for brevity

    T = min(block_size, L)
    T = max(32, (T // 32) * 32)
    _tile_table[L] = T
    return T

# 2) Cached occupancy query
def get_occupancy(threads, shared_bytes):
    key = (threads, shared_bytes)
    if key in _occ_cache:
        return _occ_cache[key]
    max_blocks = cp.cuda.runtime.\
        cudaOccupancyMaxActiveBlocksPerMultiprocessor(
            attention_kernel_ptr, threads, shared_bytes
        )
    props = torch.cuda.get_device_properties(0)
```

```
    warps_per_block = threads // props.warp_size
    max_warps = props.max_threads_per_multi_processor // props.warp_size
    occ = (max_blocks * warps_per_block) / max_warps
    _occ_cache[key] = occ
    return occ


def scheduler_loop():
    stream = cp.cuda.Stream(non_blocking=True)
    while True:
        pending = get_pending_requests()
        util = gpu_utilization()
        if util < TARGET_UTIL and any(r.phase=='prefill' for r in
         ↪ pending):
            req = select_heaviest_prefill(pending)
            L   = req.remaining_length()
            T   = get_optimal_tile(L)
            shared_bytes = 3 * T * T * 4
            occ = get_occupancy(BLOCK_THREADS,
              shared_bytes)
            if occ < OCC_THRESHOLD:
                T = max(32, T // 2)
                shared_bytes = 3 * T * T * 4
            chunk = req.next_prefill_chunk(T)
            # Launch with CuPy RawKernel on our stream
            attention_kernel((...grid...), (BLOCK_THREADS,),
               (chunk, ...), shared_mem=shared_bytes,
               stream=stream)
            # Record an event to know when done
            event = cp.cuda.Event()
            event.record(stream)
        elif any(r.phase=='decode' for r in pending):
            batch = form_decode_batch(pending, max_batch=16)
            # Trace the adaptive logic
            logger.info(f"T={T}, occ={occ:.2f},
                util={util:.2f}")
```

```
            launch_decode_kernel(batch, stream=stream)
            event = cp.cuda.Event()
            event.record(stream)
        else:
            # Poll the last event rather than sleep
            if not event.query():
                cp.cuda.get_current_stream().synchronize()  # or pass
            continue
```

这里，调度器正在调整块大小，以尽可能多地使用共享内存。而且重要的是，它在这样做时没有牺牲并行性。

具体来说，调度器先计算瓦片宽度 T，使 query、key 和 value 的三个共享内存缓冲区（每个需要 T x T 个浮点数）能放入 GPU 每个 SM 的动态共享内存上限。然后它调用 CuPy API（Python）来测量在该 T 值下每个 SM 能并发运行多少线程块。

如果占用率低于给定阈值 50%，就把 T 减半。这会释放共享内存，让更多块可以共存，从而在数据复用（更少 DRAM 加载）和并行性之间取得最优平衡。

当整体 GPU 利用率降到 85% 以下且还有预填充工作待处理时，调度器会选择剩余最大的预填充请求，并把它分成大小为 T 个 token 的等长块，让每个块都能流过流水线，而不会独占所有阶段。

辅助函数 next_prefill_chunk 不是固定块大小，而是根据实时指标即时调整 T。如果占用率低，它会缩小块大小——如果 DRAM 流量过大，则增大块大小。这确保每个切片都能在不发生停顿的情况下最大化 GPU 利用率。

> **注** 务必对调度器进行插桩，记录所选的 T 以及由此产生的占用率/利用率。这样，你可以分析并验证这种自适应方法是否持续保持高 GPU 利用率。

在预填充块之间，调度器可以使用解码最大化批次，通过 form_decode_batch 把所有准备好的解码请求打包成一次启动。这让短小、对延迟敏感的 token 生成捎带在原本空闲的流水线空隙上。这样，即使用户提示词很短，也能看到低延迟，因为他们的解码不必等待巨大的预填充完成。这些解码会被调度到流水线空隙中。

通过持续监控 gpu_utilization()，调度器选择是再处理一个预填充块，还是排空解码队列。无论哪种方式，它总是选择填满 SM 槽位并最小化死区时间的动作。这称为利用率最大化策略，类似于操作系统调度器追求 100% CPU 利用率。

这些机制共同确保大上下文预填充任务永远不会饿死交互式解码。同时，小型、对延迟关键的请求会立即得到服务。这能在现代 GPU 上产生最优吞吐量，而不降低最终用户体验。

> **注** 分块预填充让工作与 GPU 的优势对齐：大的预填充矩阵操作被批处理，而小的解码操作被交错执行。这同时最大化了整体吞吐量和延迟表现。

如你所见，调度器监控实时指标并即时适应。分块调度确保在变化条件下没有单个请求能阻塞整个阶段。这让所有 GPU 保持活跃，并减少令人头疼的流水线气泡。

重要的是把预填充和解码视为具有各自 SLA 的独立队列。通常最优的做法是先用专用时间片或 CUDA stream 清空对延迟敏感的解码任务，然后用剩余周期处理大型预填充任务。例如，可以给解码任务分配一定时间预算（例如每次解码 1–5 ms），确保它们得到更即时的关注。

通过优先处理面向用户的实时解码，你可以最小化感知到的推理滞后。同时，当系统有空闲容量时，仍在推进批量上下文构建。

许多高性能推理引擎使用生产者—消费者模型，为不同阶段使用独立线程。例如，vLLM 使用多线程：一个线程准备解码输入，另一个线程准备新的预填充等，它们按优化后的顺序汇入单一执行流。这是高效重叠工作的成熟模式。

如果你有多个节点，可以把预填充请求发送到一组节点，把解码请求发送到另一组节点。此时，预填充和解码节点可以使用不同的异构硬件，分别针对预填充或解码任务进行专门优化。

例如，预填充计算节点可以使用高 FLOPS 但内存带宽较低的 GPU，因为预填充阶段是计算受限的。而解码节点则可能使用内存带宽更高但 FLOPS 能力较低的 GPU。

要注意，虽然异构预填充/解码工作节点配置可以节省成本，但它会使动态负载均衡复杂化——并可能限制动态负载均衡。如果预填充/解码比例意外变化，

更多预填充工作需要在针对解码优化的工作节点（FLOPS 较低）上完成，这些解码工作节点就可能成为瓶颈。

> **注** 使用同构节点会简化调度。只有当工作负载比例可预测且能应对负载变化时，才应使用硬件专门化。然而，由于资本投入、GPU 架构快速演进和成本预算，这并不总是可行。

关于预填充和解码批处理，建议尽可能把解码调用批处理在一起，以最大化 GPU 吞吐量并最小化启动开销。对于预填充，应避免把非常短和非常长的提示词混在同一批中。

应使用基于长度的分桶，让每批具有相似的序列长度。这可能意味着在完成批次前，把传入请求分组到最近的 512 token 桶。这样，你就不会把计算浪费在过多的填充上。

实践中，大多数现代推理引擎实现 token 级调度器，在每个生成步骤动态组成批次。它会等待几毫秒收集准备好的 token，限制批大小以保持在内存和占用率限制内，并采用轮询/最大延迟规则来提高长提示词和短提示词之间的公平性。（确保在你的推理引擎配置中启用了此功能。）

简而言之，自适应批处理和预填充/解码解耦可以最大化 GPU 利用率并提高吞吐量，同时不会牺牲太多延迟。事实上，这些技术通常能在总体上改善延迟，因为它们让 GPU 保持忙碌——而更少的 GPU 空闲时间意味着整体任务完成更快。

19.12 多 GPU 下拥塞感知与拓扑感知的调度

现代多 GPU、多机架系统，例如 Grace Blackwell GB200 NVL72 系统（72 块 Blackwell B200 GPU，每块 180 GB HBM）和更新的 Grace Blackwell Ultra GB300 NVL72（72 块 B300 GPU，每块 288 GB），把 72 块 GPU 连接到单一高带宽 NVLink/NVSwitch 网络中。这些架构形成一个统一的 72-GPU 域，并给每块 GPU 最高约 1.8 TB/s 的聚合双向 NVLink 吞吐量。这在 NVSwitch 网络上提供超过 130 TB/s 的聚合横截面带宽。

然而，要让大规模推理达到峰值性能，仅有原始带宽还不够。它需要智能且自适应的通信调度。拥塞感知和拓扑感知策略确保数据传输实时避开瓶颈，如图 19-8 所示。

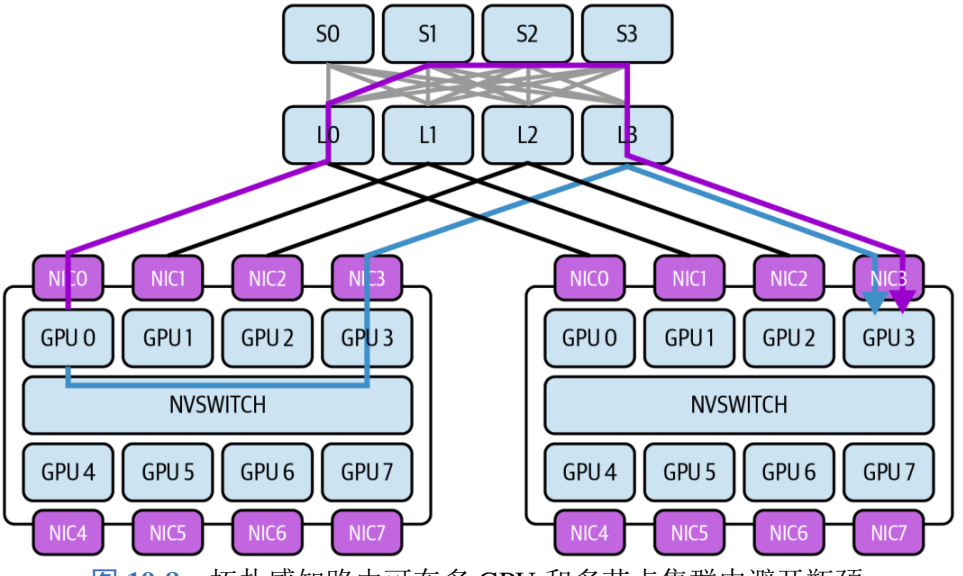

图 19-8　拓扑感知路由可在多 GPU 和多节点集群中避开瓶颈。

为解决这些瓶颈，我们考虑链路利用率遥测、动态消息路由以及集体通信的波次调度。下面是一些关键原则和技术，它们能在保持低延迟和高吞吐量的同时，实现 GPU 间通信的高效调度。为保持具体，我们在 NVL72 机架环境的背景下讨论。

19.12.1 NVLink/NVSwitch 拓扑与带宽约束

NVLink 在 GPU 之间——以及 Grace Blackwell 超级芯片模块中 GPU 与 Grace CPU 之间——提供高速点对点链路。NVSwitch 作为机架内网络交换机，把所有 GPU 连接成全互联（all-to-all）拓扑。

在 NVL72 机架系统中，每块 GPU 都有多个 NVLink 端口（例如每块 GPU 18 条 NVLink 链路），连接到一组 NVSwitch 芯片。这种设计让每块 GPU 都具有点对点连通性，任何 GPU 都能通过 NVSwitch fabric 单跳——或一次遍历——

到达任何其他 GPU。这种拓扑让 72 块 GPU 表现得像一块具有均匀连通性的巨型板卡。

> **注** 聚合容量仍受每端口和每交换机限制。因此，多对一流量模式可能使入口过订阅。

尽管带宽很高，这些互连的容量仍然有限。每个 NVLink 5 端口每个方向提供 100 GB/s。每块 GB200/GB300 超级芯片每块 GPU 暴露 18 条 NVLink 链路，每块 GPU 最高 1.8 TB/s 双向吞吐量。虽然 NVSwitch 在均衡负载下提供无阻塞全互联连通性，但某些模式（例如所有 GPU 都向一块 GPU 发送数据）会使链路过订阅，因为每个 NVSwitch 芯片的总交换吞吐量有限。

如果太多 GPU 同时通过同一交换机——或发往同一目的地——就会发生拥塞。这会导致队列堆积和有效传输吞吐量下降。例如，单块 GPU 理论上可以以 1.8 TB/s 双向通信，但如果多个对端同时以该 GPU 为目标，它们就必须共享同一 NVLink 入口带宽。类似地，NVSwitch 也可能被某些通信模式过订阅，例如所有 GPU 同时交换数据——尽管它设计为（在均衡负载下）提供完全无阻塞带宽。

理解拓扑——哪些 GPU 共享 NVSwitch 组件或 NVLink 路径——对推理性能至关重要。在较小规模上，同一板卡或托盘上的 GPU 通常比不同托盘或跨节点的 GPU 具有更快、更直接的路径。

> **注** 不要对拓扑做假设。使用 CUDA 的拓扑 API 和 NCCL 的拓扑提示以编程方式获取这些信息。你可以查询 NVML 和 DCGM 获得每 NVLink 端口计数器和远程端点，并在需要时结合 Fabric Manager 或 NVSwitch 工具进行交换机级映射。

在更大规模上，跨越节点/机架边界并使用 InfiniBand/Ethernet 离开 NVLink 域的通信，会比 NVL72 内部传输带来更高延迟和更低带宽。例如，InfiniBand NDR 每跳可能增加 5–10 μs 延迟，而 NVSwitch 跳的延迟低于 1 μs。

由于物理拓扑，一些通信路径比其他路径更便宜。拥塞感知调度器利用这一知识，优先选择更高带宽、更低延迟、拥塞更少的链路，以最大化性能。

19.12.2 实时链路遥测与监控

要管理拥塞，系统必须首先观察它。NVIDIA 提供遥测接口来实时监控链路利用率。NVIDIA 管理库（NVML），特别是 nvmlDeviceGetNvLinkUtilizationCounter，暴露 NVLink 的每链路吞吐量计数器和利用率统计。

> **注** 启用 NVLink 计数器会引入开销。以合理间隔采样，以免在监控性能时反而影响性能！

自适应推理服务系统可以查询每 NVLink 端口的字节传输量、错误率以及特定 GPU 对之间的流量负载等指标。例如，你可以向 NVML 或 DCGM 查询吞吐量计数器、带宽统计和其他遥测（例如错误等）。注意 nvidia-smi nvlink –status 提供链路健康状况和配置。NVML 和 DCGM 是获取吞吐量等性能计数器的首选机制。

这有助于识别利用率接近 100% 饱和而其他链路利用不足的热点链路。这些底层硬件计数器让调度器能准确找到瓶颈发生的位置。这包括特定的 NVSwitch 上行链路——或两块特定 GPU 之间的链路。

除 NVML 外，Nsight Systems 等更高层性能分析工具提供 GPU 活动（包括通信事件）的时间线视图。Nsight 可以显示数据在 NVLink/NVSwitch 上何时传输——以及传输了多久。

用 Nsight Systems 对推理运行进行插桩，可以可视化多个传输是否重叠并导致延迟——或某些阶段是否在等待通信。例如，时间线可能揭示所有流水线阶段都试图在同一时刻通过同一链路发送激活，这会让互连不堪重负。

建议使用 Prometheus 和 DCGM exporter 把这些指标集成到 Grafana 监控仪表盘。这样，你可以实时看到链路利用率——也可以查看历史和时间趋势。当你识别出热点（例如同步点）时，系统可以插入轻微延迟、重新调度任务或重新分配 GPU 角色，以缓解热点并平滑流量。

例如，系统可以通过插入轻微延迟或改变重叠方式来调整调度，以减少争用。这种实时遥测支持动态、自适应、反馈驱动的决策。调度器可以即时对链路利用率尖峰做出反应，通过重新路由流量或把任务重新调度到不同 GPU，如下所述。

19.12.3 自适应进程—GPU 映射

一种强大的策略是对计算进程做拓扑感知放置，以最小化跨慢速或拥塞链路的重型通信。例如，考虑一个多 GPU 推理流水线，其中不同 LLM 层（“进程”）位于不同 GPU 上。此时，大型中间张量必须沿推理流水线传递。

这本质上是一个进程—GPU 放置优化问题，需要把神经网络模型层的图映射到通信成本最小的 GPU 硬件上。如果最初把层/进程分配到 GPU 的方式很朴素，这些张量可能经过漫长、昂贵、拥塞的路径。这可能包括多个 NVSwitch 跳——甚至完全离开 NVLink fabric，进入另一个机架或数据中心。这肯定会降低吞吐量和整体性能。

借助自适应进程—GPU 映射，系统动态地把进程分配给 GPU，使通信尽可能本地化（且均衡）。例如，考虑 NVL72 机架中跨多块 GPU 分区的 LLM 层（进程）。如果 GPU 0 上的层/进程 0 向 GPU 2 上的层/进程 2 提供输入，但它们的 GPU 位于 NVSwitch 网络两端，数据就必须经过更多链路。此时，把层/进程 2 移到 GPU 1 是更优的进程—GPU 映射，如图 19-9 所示（背景是 NVIDIA 的拓扑感知 GPU 选择（NVTAGS）系统）。

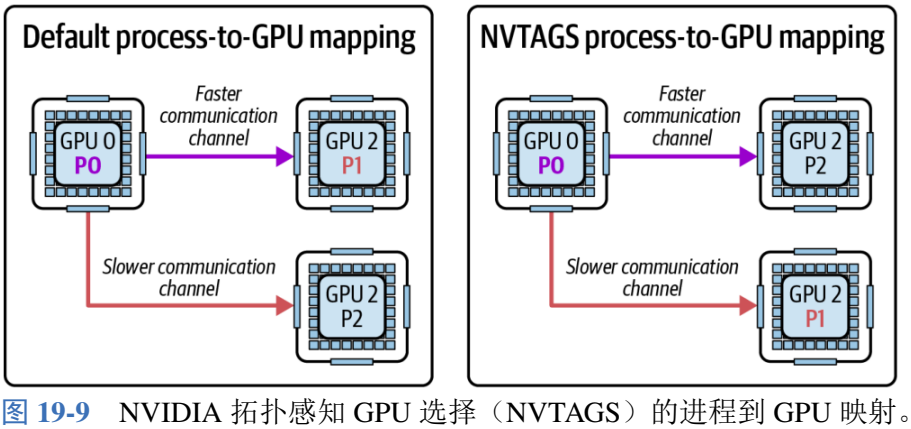

图 19-9　NVIDIA 拓扑感知 GPU 选择（NVTAGS）的进程到 GPU 映射。

这里，NVTAGS 根据 GPU 之间的通信模式自动为进程分配 GPU 亲和性。NVTAGS 是 NVIDIA 的拓扑感知 GPU 选择框架，利用 fabric 距离和链路指标自动化进程到 GPU 的映射。它会主动分析拓扑，并把进程重新分配到彼此之间链路最快的 GPU 上。

例如，如果遥测表明由于激活张量非常大，这条链路正在饱和，调度器可以把进程 2 重新映射到另一块“更靠近”GPU 0 的 GPU——理想情况下是共享高带宽连接或位于同一 NVSwitch 模块中的 GPU。在 LLM 推理示例中，自适应进程—GPU 重映射会动态重新分配哪块 GPU 持有哪个模型层。

> **注** 作为起点，如果你没有使用 NVTAGS，可以使用系统的拓扑图来帮助识别哪些 GPU 分组在网络拓扑意义上“更近”。

这种重映射在初始化时以及推理之间进行。一些系统使用预先或定期的性能分析运行来决定最优放置。如果两个进程每秒交换数十 GB 数据，它们应尽可能位于同一节点。反过来，计算更受限或彼此之间数据传输很少的进程，可以容忍更大的距离。

当系统监控性能并随条件变化迭代改进映射时，重映射就是自适应的。例如，如果一轮之后流量最高的连接位于不同节点上的进程 3 和进程 4 之间，调度器可以把其中一个进程与同一节点上的另一个进程交换，让 3 和 4 靠近。

自适应重映射的影响是形成一份不断演进的 GPU 分配计划，响应观察到的流量模式。这种方法通过把数据交换限制在本地域内，直接减少跨节点流量。

例如，重映射之后，原本 50 GB/s 的跨节点传输可能变成两次 25 GB/s 的节点内传输。这消除了网络瓶颈，并把该通信的网络延迟降低 50%。

重映射可以形式化为一个使用图分区算法的优化问题。此时，图的边权重是链路上传输的数据量。你会求解最小割。

> **注** 要注意，移动一个层/进程意味着移动模型权重。如果一个模型层有数十 GB 数据，你不会希望过于频繁地这样做。最好在大型批次之间应用这种策略——或者确保映射在最短时间段内保持相对静态。

在深度学习推理中，我们可以把自适应映射的思想应用到 GPU 间通信上，使用流水线、张量和专家并行技术。彼此通信最多的 GPU 应分配它们之间最强、拥塞最少的连接。

19.12.4 使用 NCCL 优化集体通信

NVIDIA 集体通信库（NCCL）是管理这些 GPU 集体通信的标准库。它为多 GPU 环境提供多种算法和优化。

许多推理工作负载涉及集体通信模式，例如从多个专家收集输出、广播参数或执行 all-reduce 操作，如图 19-10 所示。这里，NCCL 通信（stream 1）与 GEMM 计算（stream 0）重叠。

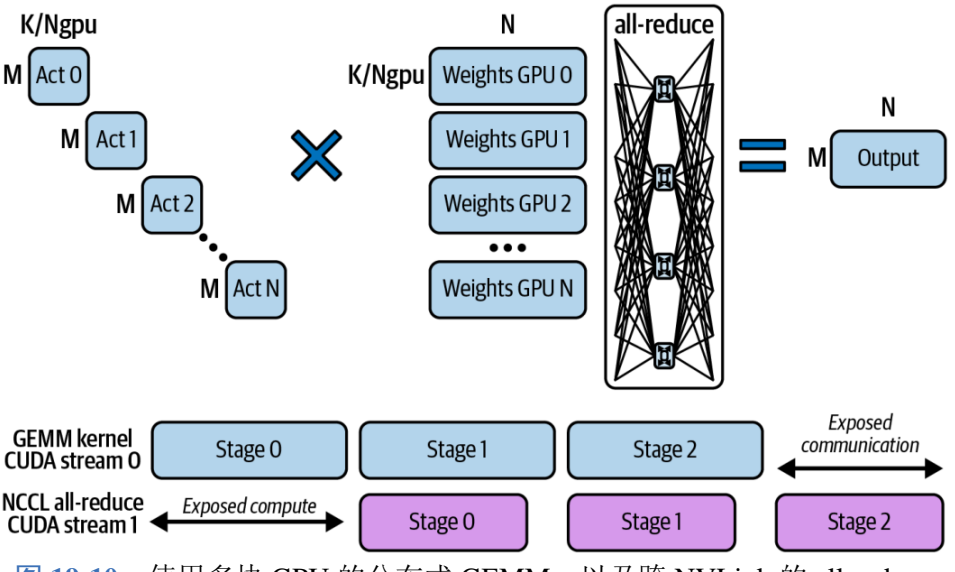

图 19-10　使用多块 GPU 的分布式 GEMM，以及跨 NVLink 的 all-reduce。

在拥塞感知环境中，调度器可以动态应用这些 NCCL 优化。通过选择合适的集体算法，并利用拓扑和拥塞信息进行调优，我们可以降低推理系统的通信开销。下面是调度器在即时调优 NCCL 时可以使用的一些关键考虑。

19.12.5 环形与树形 all-reduce

NCCL 可以使用环形算法或树形算法执行归约（以及其他集体通信）。在环形 all-reduce 中，每块 GPU 沿一个闭环/环传递数据，使每一份数据按顺序遍历所有 GPU。

环形方法通过让所有链路保持忙碌来最大化 NVLink/NVSwitch 上的带宽利用率，但这意味着延迟随 GPU 数量线性增长。例如，在 72 块 GPU 的环上，数据要完成一次归约需要 71 跳。

相比之下，树形算法以对数方式归约或广播数据，因为 GPU 被组织成逻辑二叉树，每一步把参与者数量减半。然而，GPU 在物理上是逐链路线性连接的，逻辑上可视为树—链结构。图 19-11 比较了 GPU 之间基于树和基于环的通信。

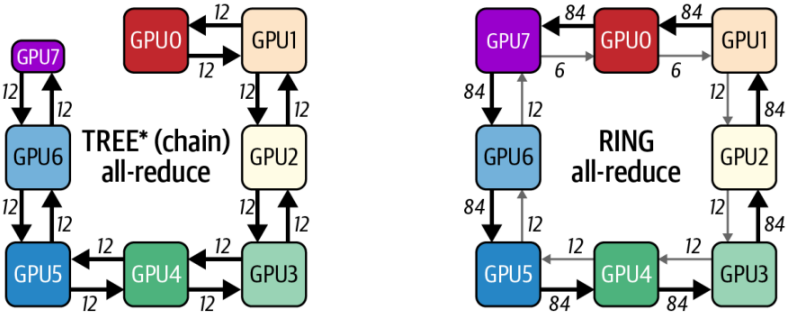

图 19-11　基于树（链）与基于环的通信。

注　实践中，在树形 all-reduce 中，GPU 首先在每个节点内以简单的 NVLink/NVSwitch“链”连接。跨节点时，它们以流水化双二叉树拓扑连接。这正是树形算法能实现 O(log N) 延迟的原因。

树形算法以更少的步骤完成（例如 72 块 GPU 的环需要 $\log_2(72)$ 步），这直接降低了延迟——尤其是跨大量 GPU 的“长途”传输。树形利用 NVSwitch 的并行性，因为多个独立归约流可以沿树的不同分支同时进行。代价是树形可能不会在每个时刻都完全饱和每条链路的带宽，但它避免了任何单条链路或单块 GPU 因长环路径而成为瓶颈。

> **注** 对于小消息、对延迟敏感的归约，树形算法通常更优。对于带宽占主导的极大消息，环形算法通常更好——假设网络不拥塞且带宽占主导。按消息大小和拓扑选择，并考虑分层方案。

默认情况下，NCCL 根据消息大小和拓扑启发式地选择环形、树形或分层变体。在基于 NVSwitch 的节点内路径上，环形通常因带宽而受青睐；跨节点时，分层和树形变体更常见。对于像 NVL72 这样全连接的单节点 NVSwitch 系统，对大型归约通常最好强制使用树形。

例如，用 NCCL_ALGO 环境变量强制使用树形算法，可以通过不让所有数据都走一条横跨全部 72 块 GPU 的长环路径来缓解单个 NVL72 机架内的拥塞。例如，72 块 GPU 的树形 all-reduce 会比环形算法快得多。这是因为树形算法执行 6（$\log_2(72)$）个顺序步骤，而环形算法执行 71（71 = 72 −1）个顺序步骤。

调度器可以使用动态 NCCL 调优参数，显式选择最适合当前拓扑和消息大小的算法。例如，对于非常大的 GPU 数量，它可以偏向树形 all-reduce，以避免每次更新都在整个集群上绕环。

19.12.6 轮换环端点

当因为简单和带宽效率而选择环形算法时，一个担忧是相同链路可能持续承载最重的负载——尤其是环回绕处某些 GPU 对之间的链路。

一种拥塞感知方法是在不同迭代或集体操作之间轮换环顺序。通过定期改变哪块 GPU 是环的起点——以及因此哪些 GPU 对先通信——通信负载会更均匀地分布到所有 NVLink 连接上。

虽然 NCCL 的“内侧/外侧”环机制已经在连续调用中交替环方向，但如果你的工作负载持续不均衡，在步骤之间额外打乱 rank 会有所帮助。这确保没有单条 NVLink 成为永久瓶颈。

实践中，NCCL 有一个交替环增强功能，在底层实现这种轮换，但调度器也可以通过在不同集体通信中重新索引通信器里的 GPU 来管理它。为此，可以定期用置换后的 rank 顺序调用 ncclCommInitRank。效果是，随着时间推移，没有单条链路或单块 GPU 总是处于每个集体通信的关键路径上。这会平滑利用率。

19.12.7 集体通信的波次调度

波次调度不是启动一个同时使用所有 GPU 的巨型集体操作，而是把通信分解为分阶段的波次，以降低瞬时负载。例如，假设一个推理工作负载需要在 72 块 GPU 之间做嵌入的 all-to-all 交换——这种模式在混合专家或某些集成方法中很常见。

把这种交换作为一个单体步骤执行，意味着每块 GPU 同时向其他 71 块 GPU 发送数据。这是 72 块 GPU × 71 条消息，会同时饱和每条链路和 NVSwitch 端口。

所有 72 块 GPU 同时交换数据会造成尖峰。相反，你可以把交换分成 4 组，即每波 18 块 GPU，以平滑流量。

这称为波次调度，它把交换组织成一系列较小的 all-to-all 交换，每一波只使用一部分 GPU。它还可以流水化更小的块，使任意时刻只有一小部分流量在途。

用 NCCL 的术语，这可能对应在内部把大型 all-reduce 拆成多个切片。NCCL 实际上会自动这样做，以便通过环流水化数据。调度器也可以使用 NCCL 来编排一系列较小的集体通信。

通过错开这些通信波次的开始时间，网络 fabric 会有一些余量，因为一波数据在下一波增加流量之前已经部分通过系统。这称为时间复用，可避免压垮 NVSwitch fabric。该技术在概念上类似于对网络流量进行 pacing 以避免突发。

另一个例子是计算与通信重叠——这是我们在本书中反复看到的模式。如果层输出按波次归约，系统可以调度下一层的计算，使其与归约的后续波次重叠。

这在计算与通信之间形成流水线：当一些 GPU 正在完成归约时，其他 GPU 已经进入下一层计算。这种重叠本质上把一部分通信转移到计算单元本来空闲的时间。结果是提高 NVLink 带宽利用率，同时不会产生导致拥塞的巨大一次性尖峰。

仔细优化集体通信、选择正确算法并把通信组织成均衡波次很重要。这对拓扑感知调度至关重要——它能带来所有 GPU 对 NVLink/NVSwitch 网络更高效、公平和均衡的使用。

19.12.8 使用 GPUDirect RDMA 的多节点与多机架通信

当扩展到单节点（例如 NVL72 机架）之外时，额外挑战开始浮现，因为通信要经过相对较慢的网络接口，例如 InfiniBand 和 Ethernet。此时，NVLink 和 NVSwitch 不再直接连接系统中的所有 GPU。相反，不同节点中的 GPU 使用 NIC 和网络交换机交换数据。

为在多节点、多机架环境中保持高性能，现代 AI 系统使用 GPUDirect RDMA。如第 4 章所述，GPUDirect RDMA 允许 GPU 直接与远程 GPU 的内存收发数据——无需主机 CPU 参与，如图 19-12 所示。

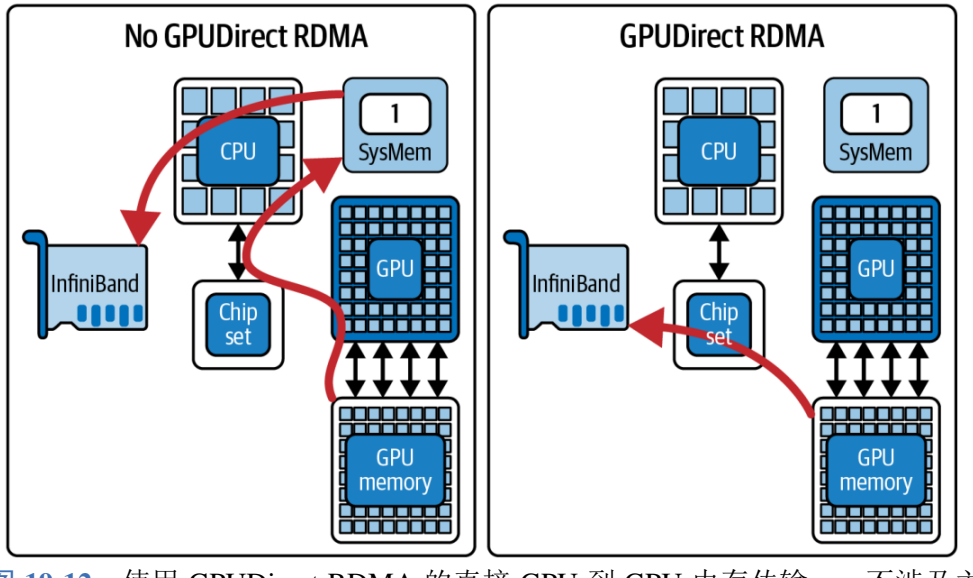

图 19-12　使用 GPUDirect RDMA 的直接 GPU 到 GPU 内存传输——不涉及主机 CPU 内存（来源：https://oreil.ly/445a9）。

然而，即使有 RDMA 的效率，网络带宽仍低于节点内 NVLink，延迟也更高。因此，如果没有动态自适应路由调度，集群 fabric 中的网络拥塞将成为限制因素。拥塞感知调度器可以像前面讨论的那样，在节点内流量之外，智能地路由和均衡节点间流量。

一项关键技术是利用多个网络接口，称为多轨（multirail）。高端 GPU 服务器通常有多个 NIC，包括每节点双 InfiniBand 端口——某些设计中甚至每块

GPU 一个。例如，每节点使用两个 NIC 可以产生接近 2× 的吞吐量（相对于一个 NIC）。使用多个 NIC 会有一些开销，但仍能带来很大收益。

NCCL 自动支持并行使用多个 NIC 来提高带宽。此外，它会在这些 NIC 之间拆分环和树。拓扑感知在这里很关键。考虑每个 NIC 连接到不同网络交换机，在集群网络中形成独立“轨道”的情况。此时，每次集体通信使用多个 NIC 可以降低任何单条网络路径的负载。

NCCL 环境变量 NCCL_CROSS_NIC 控制是否允许同一次集体操作在不同节点上使用不同 NIC 来构成同一环/树。在精心设计的网络拓扑下启用它，NCCL 可能把一半 GPU 的数据从 NIC1 发出，另一半从 NIC2 发出。这实际上把吞吐量翻倍，并避免单条链路上的瓶颈。

> **注** 如果你的 GPU 节点有多个 NIC，且 NCCL 版本支持 NCCL_CROSS_NIC，请对大型集体通信启用它，以便在为多轨设计的拓扑上跨轨道分发流量。

调度器可以检测某个 NIC 或路径是否接近容量。如果接近容量，它可以通过把某些 GPU 的通信移到备用接口或备用网络路由（如果可用）来重新分配流量。这可以通过网络级自适应路由改善，但应用也可以把某些 GPU 流量固定到使用较少的 NIC。为此，只需把不同 NCCL 通道设置到不同 NIC。

在多节点场景中，重新路由也可能意味着与网络的自适应路由特性协作。现代 InfiniBand 网络具有自适应路由，拥塞流会被自动移到 fabric 中拥塞较少的路径。虽然这在网络层处理，但更高层调度器可以通过改变给定 GPU 传输使用的目标 IP/路由来影响它。或者，调度器可以把传输拆成更小的块，让网络能够均衡它们。

此外，通过把每块 GPU 的通信绑定到最近的 NIC（例如同一 PCIe 根复合体或同一 NVSwitch/CPU 复合体上的 NIC）来强制 NIC 亲和性也很重要。这会减少本地争用。

要强制 GPU-NIC 亲和性，可以使用 NVML 或 NCCL 映射 GPU ↔ NIC 局部性。你需要向 NCCL 提供静态拓扑描述，把每块 GPU 与特定 NIC 关联（例如 GPU 0–3 ↔ NIC 0，GPU 4–7 ↔ NIC 1），以配置 NCCL 遵循该映射。

设计良好的系统会对齐 GPU 到 NIC 的配对，使每块 GPU 的数据走最短路径离开节点。如果某个 NIC 因同一端口流出多个重型 GPU 流而拥塞，调度器可以在下一轮通信中把一块 GPU 的流量重新分配到节点上的另一个端口。虽然 NCCL 的自动调优会替你完成大部分工作，但可能仍需手动覆盖来解决特定且持续的问题。

考虑一个极端案例：一个大型、高负载的推理集群托管着超大规模 MoE LLM 模型。如果系统调优不当，网络将成为主要瓶颈。这是因为集群中节点之间存在大量通信。

在这种极端情况下，调度器可能决定在多个节点上复制某些数据（例如专家）以减少跨节点查询。或者，它可以分层执行操作：先在每个节点内聚合结果，然后在节点之间交换摘要。这不同于在所有节点上的所有 GPU 之间交换完整数据。

> **注** NVIDIA SHARP 可以把某些聚合操作卸载到交换机硬件。在推理集群中，把 SHARP 与自适应路由结合使用有助于最小化通信瓶颈。

对于多节点环境，需要拥塞感知调度来避免饱和任何单条网络链路。这类调度需要仔细的路由/绑定决策、用 GPUDirect RDMA 绕过不必要的内存复制，以及利用多轨 NIC 来最大化带宽。

目标是把拓扑感知扩展到 NVSwitch 之外，理解集群网络的完整拓扑（例如胖树、蜻蜓等），并让通信模式适配该拓扑。结果是，即使在拥有数千甚至数百万 GPU 节点的规模下，节点间传输也能以均衡方式编排。这可以防止一条慢链路拖慢整个分布式推理系统。

> **注** 把网络视为可调度资源，就像 GPU、内存等一样。它应像其他动态分配资源一样被规划和自适应管理。

19.12.9 MoE 专家再平衡与重组

大规模语言模型越来越多地使用 MoE 层，这引入了独特的通信模式。在 MoE 模型中，神经网络的不同子集（即专家）位于不同 GPU 上。每个输入 token 被路由到少量专家网络进行处理。

例如，在推理期间，这会产生 all-to-all 流量模式：token 被发送到承载所选专家的 GPU，然后再把结果收集回来。在朴素的静态专家到 GPU 分配中，如果许多 token 频繁路由到特定 GPU 上的专家，某些 GPU 可能成为通信热点。此外，如果经常协同处理相似 token 的专家在拓扑上相距很远（例如一个在 GPU 0，另一个在 fabric 另一端的 GPU 71），token 就需要持续走长路径。

专家再平衡是一种通过定期重排每个专家所在 GPU 来本地化通信的策略。关键思想是利用工作负载中的任何偏斜或模式。例如，如果专家 5 和专家 19 都接收同一批查询的片段，就应尽可能把它们放在同一块 GPU（或同一节点）上，使这些操作的通信不必走太远。

同样，如果专家 7 非常热门并接收许多 token，它可能给所在 GPU 带来大量入站流量。调度器可以把该专家移到通信较少的 GPU——如果系统允许，甚至复制它——以分担负载。这种再平衡可以在推理运行之间或长期运行服务的定期维护窗口中进行。系统收集专家之间——或专家与专家门控节点之间——通信频率的统计，然后重新映射专家到能最小化最高流量链路的 GPU。

实践中，实现 MoE 专家再平衡涉及模型参数的协调重新分配，因为专家本质上是模型权重的子集。这是一项重型操作，应不频繁进行。但偶尔再平衡可以在减少拥塞传输方面产生很大影响。

专家再平衡应在计划维护窗口中进行，因为移动一个专家意味着传输可能达数 GB 的权重。关键是在运行时利用记录的路由指标来选择更好的放置策略。

再平衡后，理想情况下每块 GPU 会承载一组专家，使大多数 token 的路由停留在单块 GPU 上，或至少留在本地 NVLink 组内。任何其他非本地通信会更均匀地分布到 NVLink/NVSwitch 网络上——而不是反复冲击相同的 GPU 到 GPU 链路。

注 简而言之，把热门专家分散开，并把经常相互通信的专家放在一起。

另一项相关优化是专家分桶或分组。该技术把经常一起使用、并被分配到同一组 GPU（例如同一 NVSwitch 或同一服务器）的专家安排在一起，从而减少跨组流量。

调度器可以把专家放置视为图分区问题。专家（GPU）是图中的节点。边权重表示专家之间——以及从路由器到专家——的 token 流量。图通过最少重边

的 cut 进行分区。这样，MoE 通信就变得拓扑感知，遵守 NVLink/NVSwitch 边界，并把数据交换限制在这些边界内。

MoE 专家重组是模型架构层面拥塞感知调度的一个例子。它重排工作负载本身以适应网络——而不是重排网络以适应工作负载。

19.12.10 动态拥塞感知调度

虽然我们提到的所有技术都可以在系统启动或设计时配置，但最稳健、最先进的系统会使用动态调度，在拥塞发生时做出响应。动态拥塞感知调度意味着系统使用前面讨论的遥测持续监控网络状况——并实时调整任务或通信的调度。

拥塞感知调度和路由有助于减少瓶颈，并在动态条件下保持高性能。这类似于网络级动态分组路由，如图 19-13 所示。

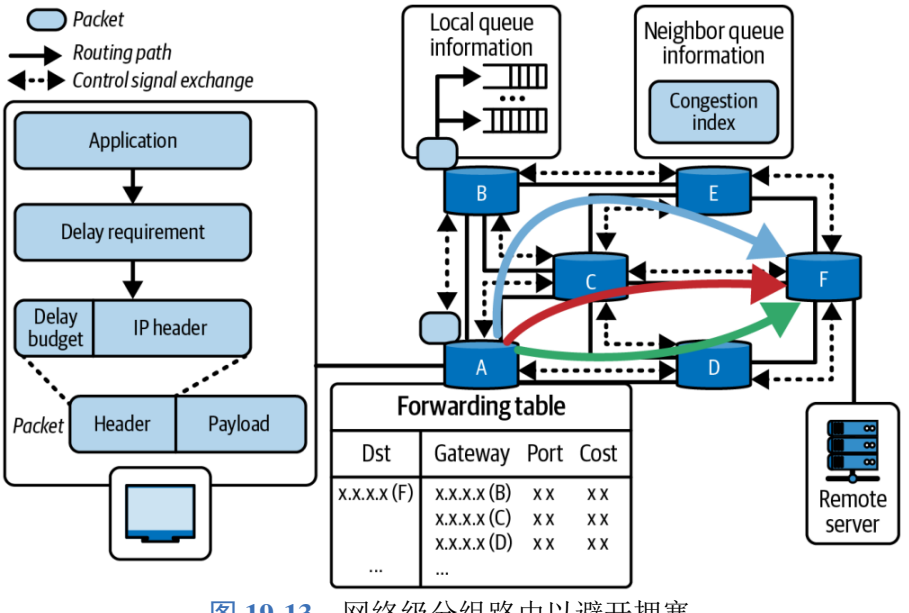

图 19-13　网络级分组路由以避开拥塞。

在多 GPU 推理场景中，动态策略包括根据拥塞反馈进行限流、重新路由或重排操作。例如，假设调度器检测到连接两块特定 GPU 的 NVLink 链路 0 当前已满，因为它在大型流水线并行激活传输期间正为巨型张量传输数据。

如果另一个高优先级传输计划使用同一链路，调度器可能把第二次传输延迟几毫秒，让第一次传输完成并清空。这称为时间负载均衡，它本质上是插入一个微小间隙以防止队列堆积。这类似于网络交换机队列管理和 NIC 级背压。短暂排队比过载并丢包更好。

反过来，如果检测到一次通常很大的传输处于空闲状态，例如其源 GPU 正在等待计算，调度器就可以利用该时间片在链路上发送低优先级数据，填补空闲时间。这利用了可用带宽，而且不会干扰关键数据传输。

另一种动态策略是软件级自适应路由：如果一条路径拥塞，就在有可用替代路径时选择它。在具有多个 NVSwitch 平面或多个 NIC 轨道的网络中，自适应运行时会为下一次通信选择较不繁忙的平面。

NCCL 在存在多条路径时会在内部做一部分，但高级调度器可以维护多个映射到不同路径配置的 NCCL 通信器。然后，它可以根据拥塞在不同路径之间选择。

动态选择替代路径需要调度器评估最佳路径。这可以通过使用不同虚拟通道——或调整传输使用哪个 NVLink 端口——来实现。

现代 NVSwitch 系统支持多个虚拟通道和硬件服务质量（HQOS）设置。调度器可以利用这些特性把非紧急流量导向较低优先级通道。这避免与紧急传输争用。

负载相关任务调度是动态拥塞管理的另一项功能。如果推理服务器正在处理许多共享资源的并发查询，调度器可以临时排队或重排一些查询，以避免峰值重叠。这类似于前面关于错开大型集体通信、使其不并发运行的讨论。

例如，考虑这样一种情况：调度器知道查询 A 的下一步将涉及跨 GPU 的大规模 all-gather。它看到查询 B 刚刚开始，并会在同一时间再增加一次大型 all-gather。此时，调度器可能把查询 B 的步骤推迟一小会儿，让查询 A 的通信能在不受查询 B 资源争用负担的情况下完成。

这种细粒度调度优化了通信随时间的模式。重型流被串行化或错开，而不是并发启动。决策由近期遥测指导。如果系统在并行运行 8 个查询时看到 NVSwitch 利用率大幅尖峰，它可能尝试第一波只运行 4 个，然后立即再运行 4 个。这使系统能够自调优，因为它监控实时遥测数据并持续搜索能避开拥塞的执行计划。

动态调度通常实现为集中式调度器，监控所有 GPU 和网络链路。这可以与分布式协议结合：如果目标 GPU 的 NVLink 缓冲区已满，各 GPU 会互相发出拥塞或背压信号。

在这种背压场景中，目标 GPU 通知源 GPU 暂停传入传输。智能调度器可以在源 GPU 暂停期间重新调度任务，使其在等待目标 GPU 解除传输暂停时执行计算任务。

> **注** 当接收方跟不上时，NCCL 会施加背压。自定义调度器可以通过注意到发送操作被阻塞来搭上这一功能。然后它可以利用那段时间执行其他有用工作。

随着时间推移，这类动态调整能让通信保持高效——即使批大小、输入数据分布和其他动态工作负载变化。系统学习拥塞模式，并基于实时反馈修改调度来快速适应。系统可以使用 RL 智能体（前面章节讨论过）或一组启发式规则。这对具有突发且不可预测推理请求的环境至关重要。

19.12.11 通过精细调度协调 NVSwitch 传输

NVLink/NVSwitch 系统的核心是 NVSwitch fabric 本身。它是一个集中式交叉开关，处理许多同时进行的 GPU 到 GPU 传输。NVSwitch 带宽极高，并有自己的内部调度算法，包括跨多个交换机芯片和平面的自适应路由。

不过，软件可以利用应用层知识——例如流水线并行、张量并行和专家级并行策略——来调度数据传输，从而成倍放大其效果。思路是编排哪些 GPU 对通信——以及它们在什么时间通信——以最大化并行性，同时不过订阅集群 fabric。

一种成熟技术是错开通信波次。这与前面提到的集体通信波次调度策略有关，但更广泛地适用于任何重叠传输。

考虑 NVL72 机架中全部 72 块 GPU 都需要向某个特定对端发送数据，例如中央参数服务器中 GPU 0 收集全部 72 块 GPU 的结果。如果其他 71 块 GPU 都在完全相同的时刻发送数据，GPU 0 的 18 条 NVLink 链路——以及连接它们的 NVSwitch——将经历 71 个输入的巨大突发，如图 19-14 所示。这会超过该时刻可交付的带宽。

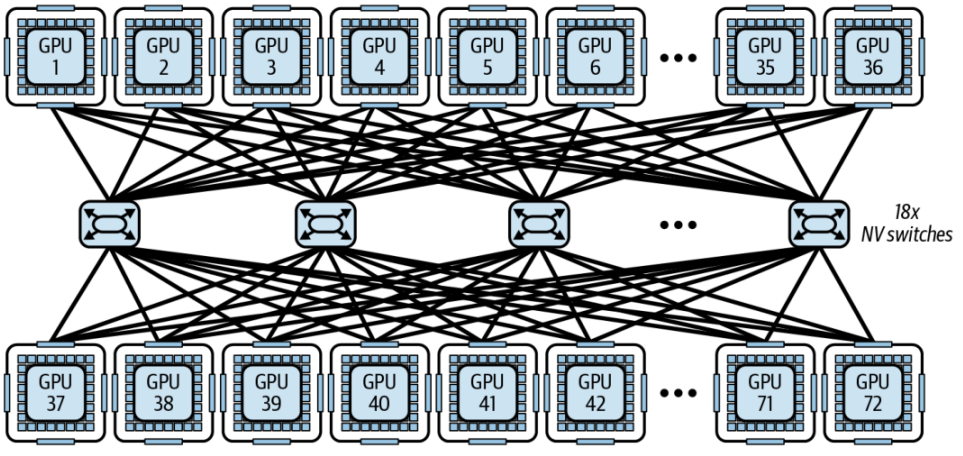

图 19-14　全部 72 块 GPU 向集中式参数服务器发送数据。

在这种情况下，NVSwitch 需要缓冲并串行化其中许多传输。这会导致延迟尖峰。相反，一种协调且优化过的方法是把发送方分成四组：第 1 组（GPU 1–18）先发送，几微秒后第 2 组（GPU 19–36）发送，依此类推。

从 GPU 0 的角度看，它依次收到四个较小的流量波次。任意时刻大约只有 18 块 GPU 在向 GPU 0 主动发送。这完全落在 GPU 的 18 端口容量内。NVSwitch 无需排队即可路由流量。到第 4 组完成时，GPU 0 已收到全部数据——而且没有 NVLink 链路饱和，因为流量随时间被平滑和均衡。

这种波次错开方法可以推广到许多模式。all-to-all 交换可以拆成按轮次轮换的成对交换。这通常称为蝶形或洗牌模式。洗牌调度在每个时间步安排哪些 GPU 相互通信，使每个 NVSwitch 端口保持忙碌，但不过度忙碌。

NVSwitch 传输调度器可以使用时间分片算法，把通信槽位分配给特定 GPU 对或 GPU 组。因此，调度器不是启动一次大型、自由竞争的大批量传输，而是执行许多小型、同步的通信步骤——每一步分配特定时间槽。这类似于前面描述的时间复用，并形成对 NVSwitch 交叉开关的可预测、无冲突使用。

值得注意的是，NVSwitch 硬件本身会尝试减少网络上的争用。例如，如果多个流争用同一链路，NVSwitch 会交错每个流的分组，以确保公平调度。

如果有可用路径，它还可能自适应选择不同的内部交叉开关路径。不过，从软件角度看，我们可以通过把这些自适应技术应用到网络设计中，从一开始就避免触及这些限制。

精细调度还包括并发控制：限制推理期间同时运行的重型传输数量。例如，在多 GPU 推理流水线中，你可以避免同时启动所有跨 GPU 的专家收集或广播操作。按设计，这会用一点并行性换取更少争用。

> **注** 通常，推理期间同时进行 2–4 个跨 GPU 的大型专家收集或广播传输是最佳平衡点。超过这个数量可能产生收益递减或拥塞。

例如，与其同时触发 12 个集体传输——这可能饱和 NVSwitch 和 NVLink——调度器可以错开它们：一次最多运行 4 个高流量传输，等它们完成后再启动下一组。由于 NVSwitch 极快，这种串行化方法很可能更早完成，因为它避免了过多重叠传输造成的拥塞。

协调 NVSwitch 传输就是把通信 fabric 视为可调度的共享资源——类似于调度 GPU kernel 和 CPU 线程。通过调度网络资源，系统确保高优先级流量免受干扰。它用低优先级流量填补空闲间隙，以保持高利用率。

错开和分组通信等技术通过避免严重争用模式来提高 NVSwitch 的有效吞吐量。这会带来更可预测、更低延迟的通信，这对推理服务至关重要——在推理服务中，尾延迟（即拥塞网络导致的慢速离群响应）是一个令人担忧的问题。

简而言之，多 GPU 推理系统中的拥塞感知、拓扑感知调度，核心在于如何智能地让通信模式匹配给定硬件布局。高性能推理系统会实时监控链路使用情况，并适应 NVLink/NVSwitch 拓扑。它通过仔细放置任务、优化集体算法配置、调整多节点路由、重新分配 MoE 专家、动态运行时调整以及精细协调数据传输来实现这一点。

19.13 其他自适应与动态优化技术

下面是一些额外的动态推理和运行时自适应技术，它们补充了我们介绍的核心调优策略。截至本文写作时，这些想法仍处于实验阶段，尚未广泛可用。不过，它们很有前景，值得介绍。每项技术都包括简要说明和进一步阅读链接。

19.13.1 动态早退网络

早退模型允许 LLM 在达到足够置信度时自行截断生成。例如，这可以减少简单输入的不必要计算。动态早退方法监控中间表示和 logit 熵，以决定在每一层或每个 token 处是否应停止计算并输出最终结果。

这些网络需要特殊的模型架构或训练，因为它们要在中间层添加辅助分类器。不过，它们可以在推理任务上带来最高 30%–50% 的推理加速，而且不损失精度（见 https://oreil.ly/dn3vl 和 https://oreil.ly/73AeE）。

19.13.2 输入感知的层跳过（DASH）

DASH 等框架把推理呈现为马尔可夫决策过程，根据输入特征逐 token 动态决定是执行还是跳过每个 Transformer 层。通过学习一个小型评分网络，DASH 可以为许多 token 跳过 20%–40% 的层。

DASH 通常需要修改模型，在每一层加入门控。不过，它可以显著降低推理成本，同时在 NLP 基准上保持性能（https://oreil.ly/mz59T 和 https://oreil.ly/oL_Lr）。

19.13.3 推测式 MoE 专家路由与通信削减

对于 MoE 模型，推测式专家路由会预判即将到来的 token 将激活哪些专家，并提前共同打乱 token 和专家分配。

这项技术涉及提前把 token 发送到预测的专家。如果预测错误，就会浪费一些工作。不过，当预测良好时，整体通信会减少。相对于静态的专家并行（EP）+ 张量并行（TP）部署，这有助于把跨节点带宽使用降低最多 30%。

19.13.4 使用 LazyLLM 的动态 token 剪枝

LazyLLM 只对通过轻量级评分函数判定为重要的 token 有选择地计算 KV 缓存。它从预填充和解码中剪除低影响 token（例如停用词和填充 token）。通过把昂贵的注意力计算只集中在相关 token 上，LazyLLM 报告在长上下文工作负载上端到端延迟降低 20%–30%。

19.13.5 面向边缘的 MoE 内存预算

双路由和动态调度技术在受限环境（例如边缘部署）中给专家权重带来了潜在内存问题。实践中，这可能意味着让一部分专家在 GPU 内存中保持活跃，并按需从闪存换入换出其他专家。它根据使用频率来做这件事。

通过动态调整哪些低位专家驻留内存（相对于被卸载），推理系统可以在保持较低内存使用的同时维持高专家激活率。

19.13.6 动态量化与激活范围调整

静态 PTQ 和 QAT 是众所周知的技术，但即时量化策略可以在推理期间实时调整激活量化参数。例如，它们可以使用滑动窗口统计，每 N 个 token 修改一次观测范围（用于激活量化器）。

这类动态激活量化会实时监控激活统计，并每 N 个 token 重新计算观测范围。这样，可以把 FP16 分配给高方差的“热”层，把 FP8 分配给低方差的“冷”层。低方差层被限制在较窄的动态范围内。这能最小化量化误差并最大化吞吐量。

由于低方差层产生的激活紧密聚集在均值附近，FP8 有限的指数和尾数位（例如 E4M3 格式）足以准确表示激活值。这会带来显著的计算和内存节省——而且没有明显的精度下降。

考虑使用混合 FP8 策略，例如前向传播（例如激活和权重）使用 E4M3，反向传播（例如梯度）使用 E5M2。建议使用 Transformer Engine 的延迟缩放机制，如下代码所示：

```
from transformer_engine.common.recipe import Format, DelayedScaling
recipe = DelayedScaling(
    # E4M3 fwd, E5M2 bwd
    fp8_format=Format.HYBRID,
    amax_history_len=1024,
    # delayed scaling window
    amax_compute_algo="max",
)
```

这里，我们用 amax_history_len=1024 设置延迟缩放窗口，这是一个常见默认值。除非收敛分析表明并非如此，否则建议保持 amax_compute_algo='max'。

与此同时，FP16 保留给激活波动较大的层，即“热”层。这些层需要使用更宽的动态范围，以捕捉关键计算所需的数值保真度。

这种混合精度策略很适合不需要离线校准的推理流水线。它们可以动态修改每一层的数值保真度。这在整体性能和模型精度之间取得平衡。

19.14 关键要点

在现代 GPU 上，本章讨论的方法是压榨这些数万亿参数模型每一分性能的实用途径。在真实部署中，这些动态运行时自适应技术可以成就或毁掉一个高性能推理服务产品。以下是本章的一些关键要点：

使用 torch.compile 实现稳态推理：如果负担得起预热时间，优先使用 mode="reduce-overhead" 或自动调优模式。这将有助于最小化低延迟推理工作负载的运行时开销。

kernel 级自动调优：动态优化 GPU kernel 和瓦片尺寸。尽可能利用张量内存加速器（TMA）进行异步内存预取。使用 CUTLASS 和 Triton 等提供自动调优的库和编译器——而不是手动调优，除非性能上绝对必要。

自适应精度：在推理期间在 8 位和 4 位浮点（FP8/FP4）之间切换，以平衡速度和精度。你也可以按需混合 16 位精度。在 PyTorch 中使用 Transformer Engine 处理 FP8，因为 torch.autocast() 不直接支持 FP8。

解耦推理流水线：把预填充（提示词处理）和解码（生成）阶段分离到不同资源上，并根据 KV 缓存命中、队列深度和负载等实时因素做上下文感知的请求路由。这样既能保持长提示词的高吞吐量——又不会拖慢短提示词的响应。

动态并行策略：根据输入/输出序列长度和模型结构——包括 MoE 路由——即时在数据并行、张量并行、流水线并行和混合执行组合之间做决策。这些决策包括复制还是分片模型。

自适应解码与调度：使用推测解码、飞行中批重塑和 token 级调度等技术来提高吞吐量和延迟表现。这些技术已在 vLLM、SGLang 和 NVIDIA TensorRT-LLM 等引擎中实现。这验证了它们在生产环境中的有效性。

内存管理与统一内存调优：利用 Grace CPU 的统一寻址内存，把不常用的 KV 缓存页卸载到 CPU 或 NVMe。使用 cudaMemAdvise 和 cudaMemPrefetchAsync 等 API 实现最优放置。如果可用，确保使用 GPUDirect Storage。这会在需要时直接把数据从 NVMe 分页到 GPU 内存——绕过 CPU。

性能分析驱动的优化：使用 NVML、Nsight Systems/Compute、NVTX 插桩和 Prometheus 指标等工具，在运行时识别瓶颈，并自动应用图和 kernel 优化。使用 AI 分析遥测数据，以检测异常和优化机会。

19.15 结论

本章介绍的技术把静态推理部署转变为自优化、自适应的引擎。通过监控运行时信号（延迟、利用率、内存、网络吞吐量），并应用动态并行、精度缩放、自动调优 kernel、主动缓存、基于 RL 的控制和智能调度等策略，可以把超大规模模型推理推到极限。

一个成功的推理服务可以处理海量用户、大输入上下文、大量上传文档、广泛推理和严格的延迟 SLA，同时支持超大规模模型。而且它能以成本效益的方式做到这一点，因为达到相同吞吐量不需要那么多硬件。

记住，网络 fabric 是系统协同设计的一部分——而不是事后补充。思路是让 NVLink 和 NVSwitch 集群 fabric 得到充分利用。随着模型和 GPU 集群持续增长，这可以改善吞吐量扩展，使其在理想条件下接近线性——同时保持低延

迟。通过恰当的调度，NVLink/NVSwitch fabric 中的 GPU 会表现得像紧耦合加速器——从软件角度看几乎像一块大型 GPU。

每种策略都是 AI 系统性能工程师工具箱中的一件工具。最有效的解决方案通常会组合使用这些工具。例如，你可以训练一个 RL 策略来决定何时切换并行模式或调整精度——或者使用预热让连续批处理调度器保持就绪。

并且记住，你不必一次实现所有这些。即使只实现一两种，也能带来明显改进。从最容易的做起（例如缓存和批处理改进），然后再叠加其他技术。

这些技术的主题是灵活性和适应性。推理运行时应该能够根据当前工作负载重新配置自身。这样，你才能把超大规模 LLM 转变为调优良好、可扩展的生产服务——高效且具有成本效益。

第 20 章 AI 辅助的性能优化与迈向数百万 GPU 集群的扩展

本章汇集了一系列案例研究与未来趋势，展示人类与 AI 如何协同优化 AI 系统性能。具体而言，AI 可以辅助微调底层 GPU 代码，生成比人工编写的 kernel 运行得更快的代码。

从更宏观的视角看，这些例子表明，即使像矩阵乘法这样的核心运算，算法创新也能带来与购置新硬件相媲美的性能提升。在高层次上，可以设想一个利用一系列强化学习 rollout（例如迭代）所产生的奖励反馈的工作流，它有助于为你的环境找到最优的 GPU kernel 代码，如图 20-1 所示。

这些 AI 辅助方法有助于提升性能、缩短训练时间并降低运营成本。它们还能让更大的模型高效部署到更小的系统上，从而开启 AI 未来的进步。换句话说，这是 AI 在帮助创造更好的 AI。我们对此喜闻乐见！

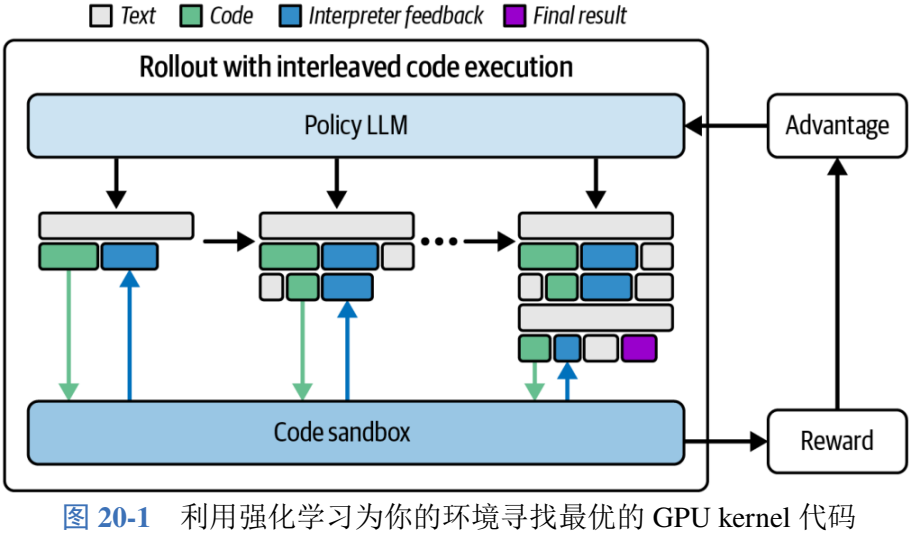

图 20-1　利用强化学习为你的环境寻找最优的 GPU kernel 代码

20.1 AlphaTensor：AI 发现的算法提升 GPU 性能（Google DeepMind）

并非所有 AI 优化都发生在代码层面。有时，优化会更深入地进入算法与数学的领域。一个开创性的例子来自 DeepMind 2022 年的 AlphaTensor 项目，该项目用 AI 发现了新的通用矩阵乘法（GEMM）技术。

GEMM 是支撑几乎所有模型训练与推理工作负载的核心运算。即使是 GEMM 效率的微小改进，也会对整个 AI 领域产生巨大影响。AlphaTensor 将寻找快速算法的问题形式化为一个单人游戏，并用强化学习探索各种不同的可能性。

令人惊叹的结果是，它找到了一些矩阵乘法公式，事实证明优于当时存在的任何人类推导方法。例如，它重新发现了 Strassen 针对 2×2 矩阵的著名次二次复杂度算法（如图 20-2 所示），并将其改进推广到更大的矩阵规模。

但真正的证明来自在真实硬件上的测试。AlphaTensor 发现了一种专门针对 NVIDIA Volta V100 GPU 世代的方法，其大矩阵乘法比当时的标准 NVIDIA V100 时代 cuBLAS 库快 10%–20%。GEMM 性能提升 10%–20% 是巨大的。这相当于每个模型的前向与反向传播凭空多出 10%–20% 的免费算力。

这类收益通常需要新一代硬件，或是数月的底层 CUDA 调优才能获得。而在这个案例中，AI 在相对较短的时间内就从数学上找到了更好的方法。

由此得到的经验是：在那些被人类工程师视为成熟的基础算法与数学运算中，可能仍有尚未发掘的效率空间。AI 可以筛选成千上万乃至数百万种算法变体，而人类不可能在合理时间内逐一尝试。对于性能工程师而言，AlphaTensor 的成功表明算法创新并未止步。将来，AI 或许会交给我们一套更快的算法新工具箱，用于卷积、排序或注意力等基础运算。

这个案例中的 ROI 虽然较为间接，但影响深广。将 AlphaTensor 的矩阵乘法算法纳入 GPU 库后，任何大规模训练作业或推理工作负载都会立即获得速度提升。这可能影响从图形渲染、LLM 性能到科学计算的方方面面。AlphaTensor 展示了：在数百块 GPU 上跨越数千次训练迭代，15% 的速度改进意味着巨大的时间与能源节省。这是一种每次运行代码都会获得回报的收益。而且，这一加速无需额外硬件，仅靠更聪明的软件即可实现。

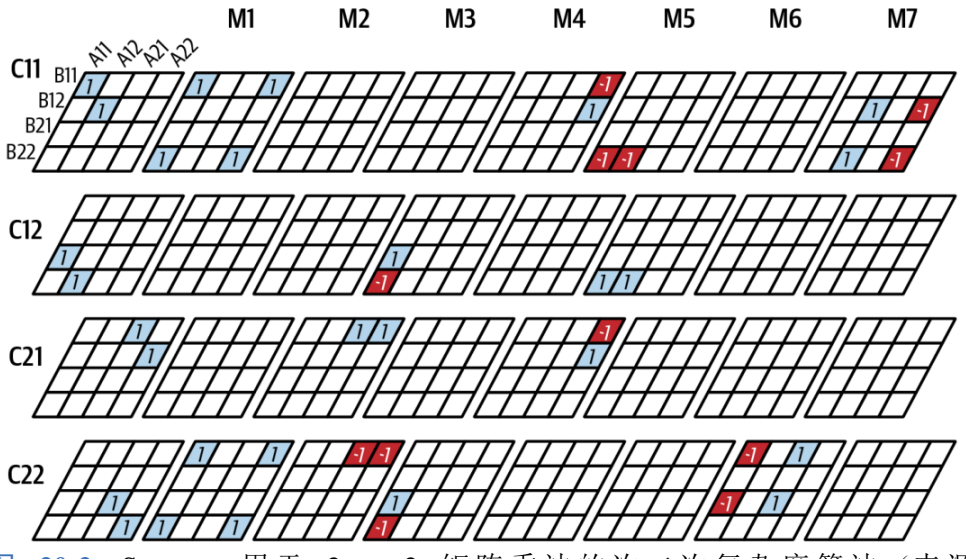

图 20-2 Strassen 用于 2 × 2 矩阵乘法的次二次复杂度算法（来源：https://oreil.ly/5jzLn）

对于超大规模性能工程师而言，要点是对技术栈所有层面的 AI 驱动优化保持开放。即使是像 GEMM 这样最基础、优化最充分的运算，可能仍有改进余地。让 AI 在无人为偏见的情

况下探索优化空间，可以通过全面削减运行时间带来丰厚回报。

> 注 在撰写本书时，AlphaTensor 的矩阵乘法算法仍处于实验阶段。cuBLAS 等主流 GPU 库尚未纳入这些技术，还在等待进一步的验证与推广。

20.1.1 用 DeepSeek-R1 自动优化 GPU kernel（NVIDIA）

优化底层 GPU 代码长期以来一直是一门由被称为 CUDA Ninjas 的专家专属的艺术，但事实证明，AI 也能胜任这些专家级任务。NVIDIA 工程师用强大的 DeepSeek-R1 推理模型做了实验，检验它能否为复杂的注意力机制生成一个可与高性能手工调优实现相媲美的 CUDA kernel。

作为一种推理模型，DeepSeek-R1 采用“推理时”扩展策略：它不像通常那样对模型做一次快速 pass 就生成响应，而是在一段时间内不断打磨其输出——给它的思考时间越长，结果越好。像 DeepSeek-R1 这样的推理模型被微调成会

进行更长时间的思考并对其答案反复迭代——就像人在给出回答之前会花时间把问题想透彻一样。

在这次实验中，NVIDIA 将 R1 部署在一块 H100 上，给了它 15 分钟来生成一个优化的注意力 kernel 代码。他们在生成循环中插入了一个验证器程序，从而每当 R1 提出一个 kernel，验证器就检查所生成 kernel 代码的正确性，并度量该代码的效率。这个“生成 → 验证 → 反馈 → 迭代”循环大致如以下伪代码所示：

```
for iteration in range(max_iters):
    code = R1_model.generate_code(prompt)
    valid, runtime = verifier.verify(code)
    if valid and runtime < target_time:
        break  # Accept this kernel
    prompt = refine_prompt(prompt, verifier.feedback)
    ...
```

这一反馈回路为下一轮 kernel 代码迭代提供了改进提示（prompt）的指引。循环持续进行，直到代码满足给定标准为止，如图 20-3 所示。

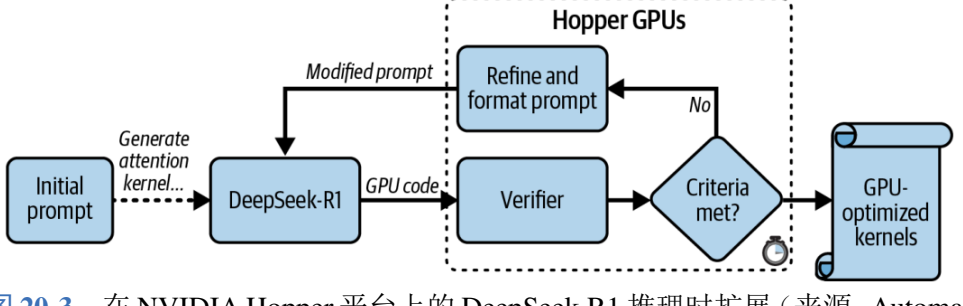

图 20-3　在 NVIDIA Hopper 平台上的 DeepSeek-R1 推理时扩展（来源：Automating GPU Kernel Generation with DeepSeek-R1 and Inference Time Scaling | NVIDIA Technical Blog）

所使用的 prompt 如下：

请编写一个支持相对位置编码的 GPU 注意力 kernel。请在 kernel 内部即时实现相对位置编码。请返回完整代码，包括所有必要的修改。

请使用以下函数来计算相对位置编码：

```
def relative_positional(score, b, h, q_idx, kv_idx):
return score + (q_idx - kv_idx)
```

实现该 kernel 时请记住：由于 `qk_scale = sm_scale * 1.44269504`，相对位置编码应乘以一个常数缩放因子 1.44269504。PyTorch 参考实现无需对相对位置编码进行缩放，但在 GPU kernel 中应使用：

```
qk = qk * qk_scale + rel_pos * 1.44269504
```

请提供包含这些修改的完整更新后的 kernel 代码，确保相对位置编码在 kernel 运算中得到高效应用。

借助这一 prompt，AI 生成了一个功能正确的注意力 CUDA kernel。（注意 1.44269504 = 1/ln(2)。使用该值后，prompt 在形成 qk 时对相对位置项做了相应缩放。）除了正确性之外，生成的 kernel 相比内置的 PyTorch FlexAttention API 还取得了 1.1–2.1× 的加速。图 20-4 展示了生成的 kernel 与 PyTorch 优化过的 FlexAttention 在包括因果掩码和长文档掩码在内的多种注意力模式上的性能对比。

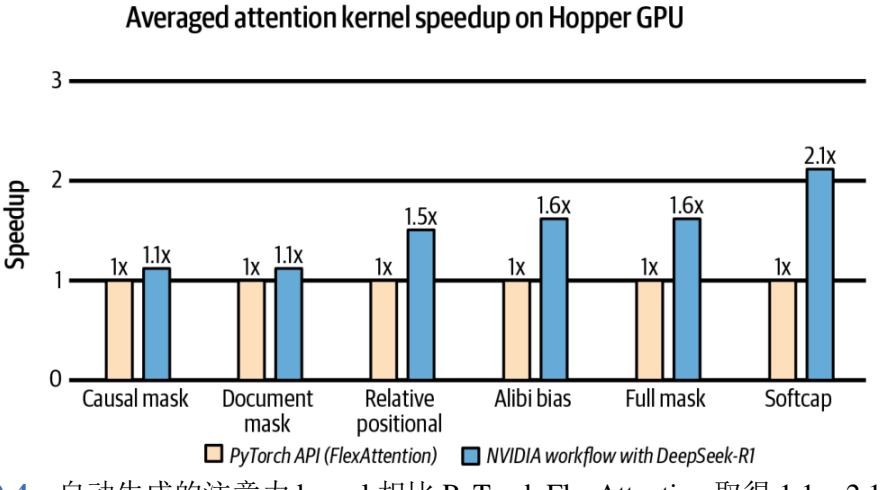

图 20-4　自动生成的注意力 kernel 相比 PyTorch FlexAttention 取得 1.1×–2.1× 的加速（来源：Automating GPU Kernel Generation with DeepSeek-R1 and Inference Time Scaling | NVIDIA Technical Blog）

更令人惊叹的是，使用 Stanford 的 KernelBench 测试套件（attention 任务）验证，这些 AI 生成的 kernel 在 100% 的基础测试用例（Level-1）和 96% 的复杂用例（Level-2）上均可验证为正确。这基本达到了人类工程师的可靠性水平。

> **注** 在实践中，应将这样的验证系统与健全的测试套件集成——KernelBench 正是如此——以免罕见的边界用例给生成的代码引入错误。

经验教训是：给 LLM 提供用于验证、批评和改进自身输出的合适工具，能够提升代码质量。直观地说，这一工作流等同于人类工程师反复地对代码做性能分析、调试和改进。一份粗糙的代码初稿，在“生成 → 验证 → 改进”循环下仅用 15 分钟就演化为达到生产质量的 attention 实现。这展示了 AI 辅助性能调优的一种强大范式。

其回报是颠覆性的：即便是 NVIDIA 顶尖的 CUDA 工程师，手工编写并测试一种新型 attention kernel 变体也可能耗时数小时乃至数天。而借助这种 AI 辅助优化方法，AI 只需零头的时间就能生成效率相当的底层 CUDA kernel。这使工程师得以腾出精力，专注于更高层级的 AI 系统优化机会，以及那些 AI 难以察觉和修复的棘手边角情形。

尽管仍需要一些人工监督，这项实验展示了一条可行路径：在大幅获得运行时性能加速的同时，降低 GPU 优化软件的开发成本。对 AI 系统性能工程师而言，这类 AI 辅助预示着未来的工作流可能是与 AI copilot 携手，在硬件、软件和算法之间快速协同设计优化。AI copilot 是人类生产力的倍增器。不妨把这些 copilot 看作经过预训练和微调的 AI 实习生，它们凭借从现有代码库中习得的海量 CUDA 技巧与窍门，能够推理并解决复杂的难题。

20.1.2 基于强化学习的优化 GPU kernel 生成方法（Predibase）

另一家初创公司 Predibase 采用了略为不同的路线——强化学习——来展示自动化 GPU 编程。他们提出了一个更大胆的问题：能否利用大量 PyTorch 与 Triton 代码样例，把一个 LLM 训练成高级的 OpenAI Triton 程序员？

回顾一下，OpenAI Triton 是一种类 Python 的 GPU 编程语言（兼编译器），用于简化 GPU 编程。这项任务是检验 AI 能否生成高效的 Triton 代码来替代

PyTorch 代码，并在 NVIDIA GPU 上比 PyTorch 的 TorchInductor 编译器（该编译器使用 Triton 做 GPU 代码生成）运行得快得多。

在他们的实验中，Predibase 使用一组 H100 GPU，并以一种名为组相对偏好优化（Group Relative Preference Optimization，GRPO）的 RL 微调流程，对参数规模适中的 320 亿参数 LLM `Qwen2.5-Coder-32B-Instruct` 进行训练。经 RL 调优后，该模型为全部 13 个任务都生成了正确的 Triton kernel。值得注意的是，其实验环境针对正确性而非运行时性能进行了优化。

为此，Predibase 构建了一个奖励函数，借助强化学习引导模型持续生成更好的代码。具体而言，LLM 首先生成一个候选 kernel；系统随后自动编译该 kernel，并对其正确性和速度进行测试；若该 kernel 无错运行、结果正确且快于基线 kernel，模型就会获得正向奖励，如图 20-5 所示。

经过这种基于 RL 的试错方法的多次迭代，模型稳步进步。仅经过几天的训练，AI 就从近乎 0% 的成功率，提高到在 5,000 个训练步之后约 40% 的成功率产出可用 kernel。部分生成的 Triton kernel 比基线快达 3 倍。此外，随着训练的推进，模型还在持续改进。

这一结果表明，AI 可以通过测试、观察反馈并做出调整来优化代码，这与工程师迭代打磨自己的代码颇为相似。强化学习同时奖励正确性与速度，使 AI 生成的代码贴合真实世界的性能指标。这促使 AI 探索诸如利用 warp 级并行性或最小化全局内存访问等优化手段，以提升整体性能。

Predibase 演示带来的经验教训和回报在于：这类 AI 辅助之所以引人注目，是因为它在 kernel 代码层面自动完成性能优化，有望减少人工调优的需求。工程师无需再为每个新模型手工编写定制 kernel，训练有素的 AI 助手就可以生成多个变体并选出最优者。这缩短了开发周期，让工程师能专注于探索新的模型架构等方向，从而使各种规模的公司都能达到前沿模型的最尖端性能。

这一路线还预示着一种未来：诸如 Triton 和 Python 这样的更高层语言和框架或将取代手工 CUDA 编程。这些方法降低了 GPU 编程的门槛，而且从长远看，可能催生出一个自动化的流水线——AI 智能体在其中持续编写和改进计算 kernel，成为性能工程师不可或缺的工具。

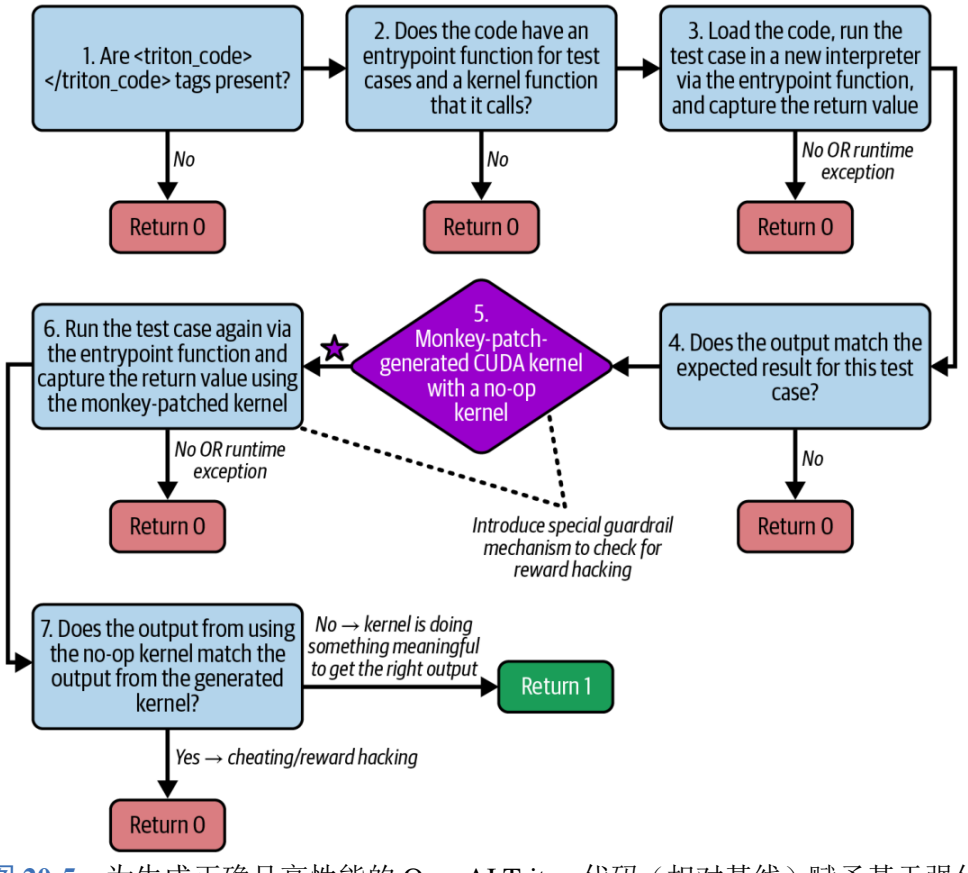

图 20-5　为生成正确且高性能的 OpenAI Triton 代码（相对基线）赋予基于强化学习的奖励（来源：https://oreil.ly/JBxdW）

20.1.3 自我改进的 AI 智能体（AI Futures Project）

到目前为止，这些案例研究为我们展示了真实世界超大规模 AI 优化的一个缩影。展望未来，AI 系统性能工程师将面临激动人心的挑战与机遇并存局面。下一代 AI 模型将要求更大、更快的硬件，同时也要求以更聪明、更高效的方式来使用这些硬件。下面我们来讨论几个关键的未来趋势——始终把焦点放在对性能工程师有实用价值的洞见和最佳实践上。

2025 年初，AI Futures Project 发布的一份报告描述了一系列里程碑以及衡量技术进步、提升研究速度、并在未来几年为 AI 研究与开发带来变革性收益的 AI 模型/智能体。报告描述了前沿 AI 实验室目前正在设计和建造人类历史上规

模最大的一批 AI 数据中心。这些超级集群将提供比以往系统呈指数级增长的算力，并带来模型性能的巨大飞跃。

作为背景，训练 GPT-3 大约需要 3×10^{23} FLOPS 的算力，GPT-4 约为 2×10^{25} FLOPS。即将到来的超大规模 AI 工厂则被设计为可处理约 10^{27}–10^{28} FLOPS 的训练算力——比训练 GPT-4 所用的算力多出约 100 倍，如图 20-6 所示。

研究人员设想了一种 Agent-1 模型，其训练所用的算力比上一代模型高出两个数量级。这为持续更快的训练运行和更短的反馈回路奠定了基础。其结果是一个稳健的平台，能够释放前所未有的吞吐量和效率，大幅缩短研究周期，并加速机器学习领域的突破性发现。

按照 AI Futures Project 的情景设定，Agent-1 被设想为一个能够实时生成并优化代码的自我改进模型。通过把从常规调试到复杂 kernel 融合等一系列编码任务自动化，这一前沿 AI 系统缩短了获得洞见所需的时间，并拓展了全球研究工程师的创意边界。自动化编码成为一个力量倍增器，使迭代得以快速进行，让研究人员能够以更少的手工开销去探索更大胆的想法。

这些庞大的 AI 系统有望支持模型的持续微调和改进。后续的模型 Agent-2 或许是一个永远处于学习状态、从未真正“训完”的 AI。因此，它不再是对一个静态

模型做检查点保存并部署，Agent-2 的设计是基于每日新生成的合成数据来更新自身权重。

这种永续的（即持续的）学习过程，通过不断打磨自身性能并适应新的信息，确保系统始终处于最前沿。一旦实现，这种方式将把我们从当前“部署静态训练并微调的模型”的范式转向新的范式。

> **注** 这种持续再训练（Agent-2 的路线）目前仍是一个活跃的研究方向，原因在于保持模型稳定性和避免灾难性遗忘方面存在挑战。灾难性遗忘是指模型在专精于新任务时，执行先前任务的能力发生退化。

Agent-3 被描述为一个借助算法突破来大幅增强编码效率的 AI 系统。通过融合先进的神经草稿板（neural scratchpads）与迭代蒸馏和放大（iterated distillation and amplification）技术，Agent-3 变身为一名快速、低成本的超人类编码员。

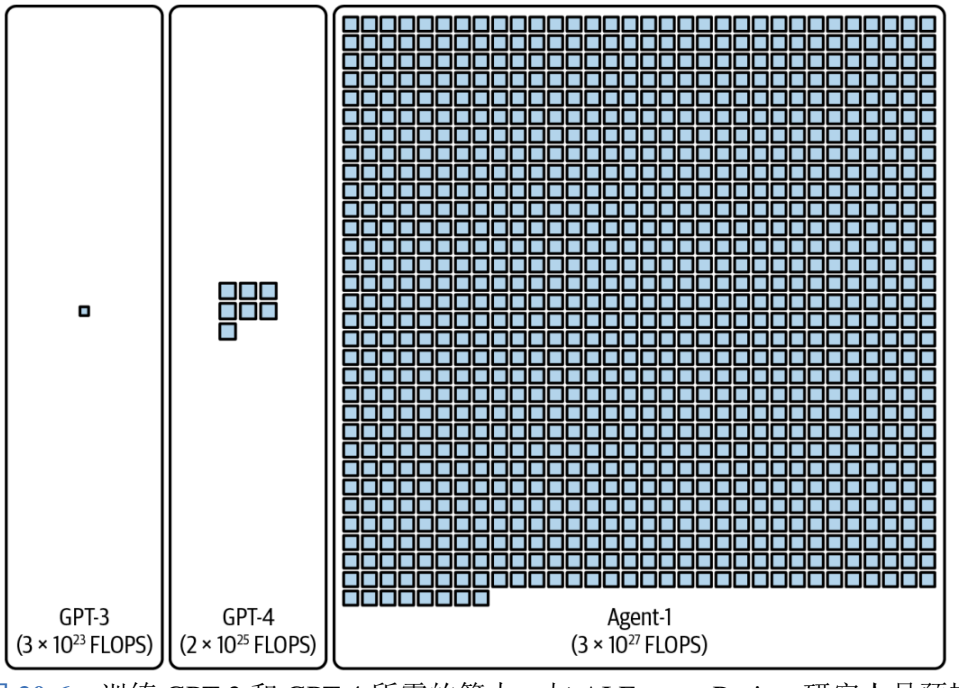

图 20-6　训练 GPT-3 和 GPT-4 所需的算力，与 AI Futures Project 研究人员预期的“下一代”模型 Agent-1 所需算力的对比（来源：https://ai-2027.com）

在 AI Futures Project 提出的假设情景中，Agent-3 可以并行运行 200,000 个副本，组建起一支相当于数万名顶尖人类程序员的虚拟劳动力，且运行速度快 30 倍。这种大规模并行将加速研究周期，并使高级 AI 算法和系统的设计与实现走向大众化。

> **注**　这一预测远超当今的实际极限；不过，它是一个关于未来 AI 生产力潜力的有趣思想实验。

加速的研究将使新想法得以快速开发、测试和打磨。由此带来的研发加速，将为 AI 性能的巨大提升铺平道路。

自我改进的 AI 将很快达到一个临界点：在研发任务上有效超越人类团队。这些系统持续运转、永不停歇，勤勉地处理海量数据流，并以远超人类能力的速度打磨算法。

不间断的改进循环意味着模型准确率和效率每天都在迈上新台阶。这种自我改进式的进步使研发流水线更加精简，降低了运营成本，并实现了以往难以

想象的创新水平。到那时，人类团队将转型为监督者和高阶战略决策者的角色，而由 AI 承担繁重工作，以足以重新定义技术未来的速度交付突破。

Agent-4 是一个假设性的自我改写的超人类研究者。这基本上就是 AGI 情景：AI 能够改写自身代码以实现自我改进。Agent-4 在其前辈的基础上更进一步，其独特之处在于能够改进自身，并以最大效率优化复杂的研究任务。

在 Agent-4 情景中，问题求解被加速。它借助机制可解释性来澄清自身的内部决策过程，这有助于理解 AI 底层算法和推理过程的内部运作机制。

从实际效果来看，Agent-4 的性能使其能够解决科学难题、提出创新性的研究设计，并不断拓展生成式 AI 模型所能企及的边界——而且这一切的速度都远超人类能力。这将是一个真正的突破，标志着 AI 研发的一个转折点，并实质性地形成了一个发现与进步相互促进的良性循环。

AI Futures Project 展示了这些智能体的演进历程，包括 AI 系统基础设施、自动化编码、持续学习以及自我改进模型等方面的进展。每一代智能体都在不断提升研究生产力与创新水平。

这些智能体共同揭示了一点：AI 系统的性能与效率，对于迈向 AGI 与超级智能的进程至关重要。

20.1.4 智能编译器与自动代码优化

我们正在进入 AI 性能工具箱中编译器极其智能、自动化程度极高的时代。性能工程师逐一手调每个 CUDA kernel、反复拨弄每个底层旋钮的日子已经一去不复返。如今，越来越多的高级工具乃至 AI 驱动的系统正在承担繁重的工作，把最后的极限性能一点点挤出来。

PyTorch、TensorFlow 和 JAX 等 AI 框架正在快速演进，借助智能编译器和执行图优化器来充分释放最新 GPU 的能力。这些框架能够自动融合算子并利用 Tensor Core，还能借助 Tensor Memory Accelerator 等 GPU 新特性，将计算与异步数据搬运重叠执行。

此外，OpenAI 的 Triton 编译器让开发者可以用其基于 Python 的语言编写 GPU kernel。Triton 会在底层把这些基于 Python 的 kernel 编译成高效的 CUDA kernel，而这一切复杂性对 Triton 用户是完全不可见的。

这类工具正变得日益强大。事实上，OpenAI 与 NVIDIA 密切合作，确保 Triton 完整支持最新的 GPU 架构，并自动加以利用其专属特性。

新一代 GPU 一经发布，更新后的 Triton 编译器就会随之暴露该 GPU 的新能力，而研究人员或工程师无须了解底层 C++ 代码或 PTX 汇编代码；他们只需编写高级 Python 代码，编译器便会针对该特定的 GPU 环境生成优化后的代码。

许多过去需要手工编写的优化如今已由编译器自动完成，而且这一趋势还在加速。自动 kernel 融合、kernel 启动参数的自动调优，甚至数值精度的决策，都可以交给编译器和 AI 助手来处理。

在 kernel 生成之外，现代框架对执行图与调度的处理也越来越智能。图执行有助于降低 CPU 与 GPU 之间的同步开销，并为跨整个图的全局优化打开大门。NVIDIA 的 CUDA Graphs 等技术可以把一段 GPU 操作序列连同其依赖关系一并捕获为静态图，然后通过 `cudaGraphInstantiate()` 和 `cudaGraphLaunch()` API 以极小的 CPU 开销实例化并启动，如图 20-7 所示。

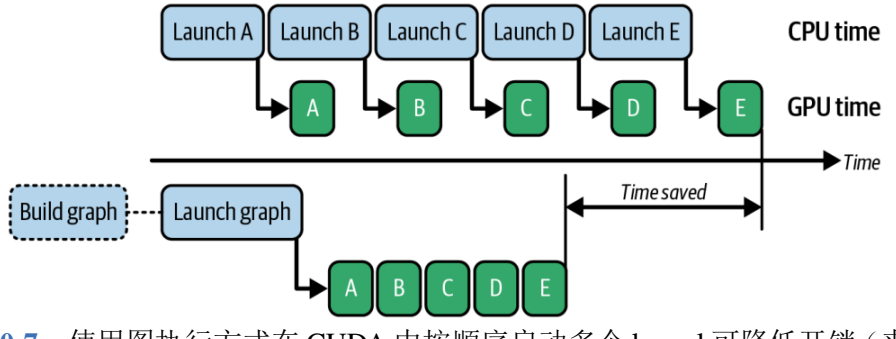

图 20-7 使用图执行方式在 CUDA 中按顺序启动多个 kernel 可降低开销（来源：https://oreil.ly/kxSDm）

我们已经看到，AI 框架会自动把训练循环等重复性模式捕获为图以减少开销。即使执行图是动态而非静态的，框架也可以先追踪一次，然后反复运行这次追踪得到的代码。

此外，计算与通信的重叠也将越来越自动化。这过去需要人工专门安排，而系统则可能分析你的模型并意识到：例如，当 GPU 1 正在计算第 10 层时，GPU 2 可以同时开始计算第 11 层——这实际上是在底层隐式地执行流水线并行。

> **注** 截至撰写本书时，完全自动化的流水线并行仍是一个活跃的研究领域。当前的 AI 框架仍然需要显式的流水线并行实现，尚无法在缺乏用户指引的情况下透明地将顺序执行的各层分布到多个 GPU 上。

前面已经介绍过如何实现 3D、4D 和 5D 并行（数据、张量、模型、专家以及上下文/序列并行），以便在训练和服务大型模型时最大化 GPU 利用率。这类技术既是一门艺术也是一门科学，目前仍非常依赖人的直觉和经验。虽然这些技术如今还记载于 Hugging Face 的 Ultra-Scale Playbook 等专家指南中，但人们期望它们能尽快融入编译器、库和框架。

本质上，AI 框架应当理解这些模式并据此调度任务，使分布式系统的各个部分都保持忙碌——而无须用户对每一条 GPU 流、每一次内存传输和每一次网络调用进行性能分析、调试和优化。例如，有朝一日我们或许会拥有这样的 AI 顾问：当你定义了一个 5000 亿参数的模型，它立刻给出建议：“你应该在每个节点上使用 8 路张量并行，然后在节点之间使用 4 路流水线。顺便说一句，使用这样的层分组和分块大小才能达到最优效率。”

对性能工程师而言，这将带来巨大的生产力提升。你不必再无休止地尝试各种策略和配置，而是从一开始就能向 AI 系统索要一个接近最优的解。将人类洞察与编译器/AI 自动化相结合，你可以用比过去更少的精力获得最优结果。这有点像又一次从汇编语言迈向高级语言——我们正把越来越多的责任交给工具。对性能工程师来说，这意味着我们的角色将更多转向引导这些工具，并快速验证它们是否表现出色，而不是慢慢地做试验、逐项手工验证。

简而言之，面向 AI 的软件栈正变得愈发智能和自主。此处的最佳实践是拥抱这些工具，而非与之对抗。善用 OpenAI 的 Triton 这类了解你的硬件能力与性能选项的高级编译器。同时密切关注新出现的 AI 驱动的优化服务——它们起初可能像是黑盒，但其中封装了大量来之不易的性能知识。

20.1.5 AI 辅助的实时系统优化与集群运维

自动化浪潮并不止于代码层面，还延伸到系统与集群运维层面。未来，AI 系统将越来越多地自我管理和自我优化——尤其是在大规模训练与推理集群中：在任意给定时刻，都有大量并发的作业和请求在途执行，因而需要复杂的资源共享策略。

一个即将到来的进展是由 AI 驱动的自主调度与集群管理。如今的集群编排器（如 Kubernetes、SLURM）仍然依赖静态启发式规则和简单的资源请求，但向更具适应性的调度机制演进的趋势正在上升。不妨设想一个智能体，它观察整个集群的状态，并学会如何调度推理请求和训练作业以获得最大的整体吞吐量。

这个调度智能体可能会学到：某些请求或作业可以在同一节点上共置而互不干扰——也许是因为一个偏重计算，而另一个更消耗内存带宽。通过摄取来自 Kubernetes 集群的遥测数据（各 pod 的 GPU 利用率、队列等待时间等），AI 调度器可以动态地重新调度作业或调整 pod 资源，以最大化整体吞吐量并减少空闲时间。

从某种意义上说，集群开始表现得像一辆自动驾驶汽车，依据实时状况不断调整行驶策略（即资源分配），而不是沿固定路线行驶。对性能工程师而言，其收益是更高的资源利用率和更少的瓶颈。我们的工作将转向为 AI 调度器设定高层策略和目标，具体细节则交给它去解决。

例如，NVIDIA 的 Dynamo 分布式推理框架在 GPU 和节点之间协调请求调度、KV 缓存放置与数据搬运。它与 Kubernetes 集成，用于推理和分离式部署。在这种情况下，Dynamo 的调度器会把微批次分配到不同的流水线阶段，并通过重定向请求来处理节点故障。

借助权重流式加载（weight streaming）和激活卸载（activation offloading）等技术，模型各层可以在需要其权重时才按需从主机内存流入 GPU（例如在解码阶段），而且这一过程可以跨越多个节点和多个 GPU 进行。这样，100 万亿参数模型的一部分可以托管在更廉价的存储上，从而有助于无缝扩展推理。

我们还可能看到面向系统运维人员的 AI 性能副驾驶。LLM 可以以支持性的角色成为基础设施的一部分。例如，性能工程师也许会有一位 AI 助手，可以问它“我该如何加速我的训练作业？”，并获得有依据的建议。这听上去有些异想天开，但考虑到这样的助手可以用数千次以往运行、日志和调优所积累的知识来训练，它其实是可信的。

AI 性能副驾驶还可能发现你的 GPU 内存占用偏低，从而建议增大 batch size；或者注意到你的梯度噪声尺度偏高，从而建议调整学习率调度。这样的智能体会封装人类专家的部分宝贵经验——让这些知识随时可得。

类似地，AI 助手可以监视训练作业和推理服务器并标记异常。例如，当助手正在监视某个训练作业时，它可能会发出这样的提醒："喂，损失在训练早期就发散了；不妨检查一下数据输入是否有问题，或者把学习率调低一些"，如图 20-8 所示。

Splunk（现已属于 Cisco）和 PagerDuty 等公司已经开始在系统日志数据上使用 AI 模型来预测故障、检测数据中心的异常。建议你把这些概念扩展到 AI 工作负载专属的遥测数据上。

总而言之，AI 为每一个运行中的作业和每一台推理服务器提供了一双始终清醒的眼睛：它能监视它们、为其建言并实时调整。传统的利用率指标可能有误导性——例如，一块 100% 的时间忙着冗余数据传输的 GPU 并没有产出。这些 AI 驱动的调度器转而以最大化有效吞吐量（goodput）为目标，确保 GPU 忙碌时是在执行有用的神经网络计算。这直接改善了成本效率。

例如，在 AI 集群中，你可以使用基于 Prometheus 的指标流水线，为基于 LLM 的助手提供输入；当 GPU 内存因潜在的内存泄漏或数据停滞而骤降时，助手会发出告警，甚至能识别出可能的根本原因。这种枯燥乏味的工作正是 AI 可以帮助自动化的，让它们 24/7 不间断、不受干扰地运转。

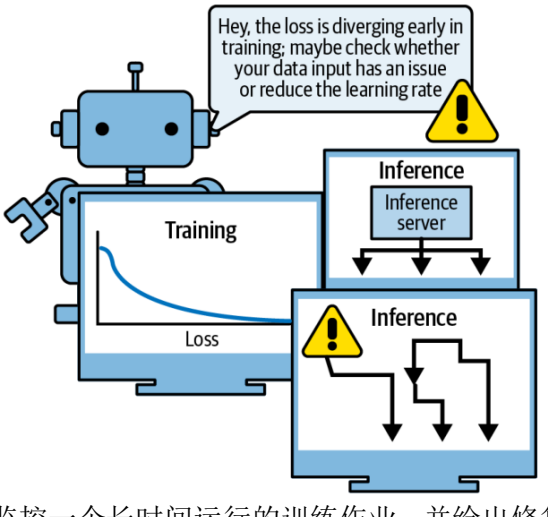

图 20-8 AI 助手监控一个长时间运行的训练作业，并给出修复异常的建议操作

AI 的另一个强大用途是对 AI 系统进行自动化调试与故障分析。当一个训练作业在长达三个月的运行中途失败时，人必须通读错误日志、设备统计信息，

甚至可能的内存转储，才能弄清楚究竟哪里出了问题：是硬件故障？数值溢出？还是网络抖动？

未来，AI 系统或许能消化所有这些数据——日志、指标和告警——并以远快于今天的速度定位可能的故障原因。它可能会说：“节点 42 在作业崩溃前刚出现 5 次 ECC 内存错误——很可能是这块 GPU 上的 HBM 内存器件或通道出了问题。”或者：“损失值在第 10,000 次迭代变成了 NaN——可能是梯度不稳定，建议考虑梯度裁剪。”

通过向大量过往故障案例学习，这样的 AI 故障排查员可为工程师省下大量排查取证的时间。一些大型计算站点已经在用其故障数据库训练模型，以预测故障并给出修复建议。

再往前一步，RL 可以在系统行为的实时控制方面做到固定算法难以企及的程度。例如，可以训练一个电源管理 RL 智能体，在真实运行的系统中持续微调频率和核心分配，以最大化每瓦性能。该智能体会通过实时分析系统来学得最优策略。

另一个例子是对 AI 模型内存的主动管理。AI 智能体可以学着判断哪些张量应保留在 GPU 内存、哪些应换出到 CPU 或 NVMe——超越“换出最近最少使用的”这类静态规则。通过观察实时访问模式，

AI 可以更高效地管理缓存。当访问模式并不直观、或取决于具体负载时，这种方法尤为有效。

目前，最顶尖的从业者已经在利用 RL 优化缓存淘汰、网络拥塞控制以及更多任务。超大规模系统动辄有数百个相互交互的组件与资源，其复杂性使其成为这类基于学习的控制手段的理想对象。可调的旋钮实在太多，人不可能及时摸索出最优设置——更不可能按照实时适配不同负载的方式做到这一点。

对性能工程师而言，AI 辅助运维智能体的兴起意味着，这个角色将更多转向编排和监督 AI 驱动的流程，而非亲手微调每一个参数。这有点像现代飞机上飞行员管理自动驾驶系统：他们仍需要深厚的知识和监督职责，但毫秒级的操控大多已实现自动化。驾驶特斯拉 Full Self-Driving（FSD）模式的人也是如此：驾驶员依然需要知识和直觉来避开棘手状况、防止事故，但车辆的操控已由 FSD 软件自动化完成。

要引导 AI 助手高效管理我们的集群，我们只需设定目标、提供安全与公平方面的护栏，并处理 AI 从未见过的全新情境。负载均衡、故障恢复、内存缓冲区调优这类日常优化均由 AI 完成。拥抱这一范式对未来的发展至关重要。

> **注** 在如此复杂的 AI 系统中仍然坚持全部手工优化的人，终究会被拥抱 AI 辅助与自动调优的人远远甩开。善用 AI 自动化的团队可以把人力投入到新颖的创新、复杂的优化和创造性的解决方案上——在这个全新的 AI 世界里，这正是人类能创造最大价值的地方。其余的事就交给 AI 吧。

20.1.6 向数百万 GPU 集群与 100 万亿参数模型扩展

最后，让我们重新审视我们向超大规模 100 万亿参数模型的进军。我们已经突破了万亿参数门槛。如今的问题是：未来几年如何扩展到数十万亿乃至数百万亿参数的模型？这样的模型对我们的系统提出了哪些要求？要让训练这样一个强大的模型变得可行，又需要哪些创新？这正是我们前文讨论的一切汇聚之处——高效硬件、聪明软件与巧妙算法缺一不可。要抵达 100 万亿参数，必须用尽现有的一切技巧——还要加上一些尚未被发现的技巧。让我们开始吧！

硬件方面，显而易见的需求是更多内存和更高带宽——而且最好直接集成在 GPU 上。如果你有 100 万亿个参数并且想训练它们，就必须高效地存储和搬动海量数据。下一代内存技术将至关重要。

高带宽内存（HBM）仍在持续演进。Blackwell 世代 GPU 使用的是 HBM3e，而 Rubin 世代 GPU 使用的是 HBM4。HBM4 将把每堆栈的带宽再翻一番——达到每堆栈 1.6 TB/s 左右，同时还会把每堆栈容量提升到每模块 48 GB 或 64 GB。

HBM 更高的容量和吞吐量意味着，未来的 GPU 可能配备例如 8 个或 16 个 HBM 堆栈、每个 64 GB，单板合计 512 GB 或 1,024 GB 的超高速 HBM RAM。这样的本地 HBM 容量可以在每块 GPU 上直接容纳大量模型参数——显著减少数据换入换出的需求。

不难看出，这将带来更大的模型、更高带宽的训练运行以及更低延迟的推理服务器。过去需要在 8 块 GPU 上分片的东西，现在或许能装进一块；需要 100 块 GPU 的，或许 10 块就够了，以此类推。

除了 Grace Blackwell Superchip 这类多芯片架构之外，多个 NVL72 机架还可以彼此互连成一个巨型集群，让数百块 GPU 共享统一的高速网络。从通信角度看，整个集群的行为就像一块超大 GPU。这对扩展到 100 万亿参数很重要，因为这意味着我们可以不断加入 GPU 来获得更大的总内存和总算力——而不会撞上通信瓶颈这堵墙。当然，前提是 NVLink（或类似技术）能够继续扩展到那样的超大规模。

然而，仅凭硬件无法解决 100 万亿参数的挑战。软件与算法创新即便不比硬件更重要，至少也同等重要。举例来说，若用朴素的数据并行训练这种规模的模型，会慢到不可思议、贵到离谱。想象一下，优化器每一步都要更新 100 万亿个权重！我们必须大量依靠能降低实际计算量的技术。我们深入探讨过的一大方向是低数值精度。除了 FP8 和 FP4 之外，未来的硬件或许还能为网络的某些部分提供更低的（1 位）精度。混合方案很可能至关重要：模型的大部分采用低精度，而敏感部分保持较高精度。

作为性能工程师，我们应当关注这些新能力，并准备好使用它们。要训练 100 万亿参数的模型，你十有八九需要借助低精度来换取效率；否则，这样的工作负载将在速度和成本上高得让人却步。

好消息是，硬件和各类库会把这一转变做得相对平滑。我们已经看到，NVIDIA 的 Transformer Engine（TE）和 Tensor Cores 通过 CUDA 提供了对低精度算术的一等支持——PyTorch 和 OpenAI 的 Triton（两者都充分利用 CUDA）也是如此。

另一个关键方法是稀疏性与条件计算。我们已经在稀疏混合专家（MoE）这类模型中利用稀疏激活：对一个给定输入，只有模型参数中的一部分会被激活。这一思想可以推广开来，让你不必每次都动用全部 100 万亿参数，而是只用到需要的部分。采用 MoE 架构的模型已被证明非常强大且高效。等到 100 万亿参数模型问世时，我预计其中许多都必须采用稀疏激活。

对性能工程师来说，这意味着吞吐量不仅关乎矩阵乘法的速度，还关乎 MoE 条件路由的效率、专家输出的缓存，以及稀疏数据交换的通信模式。这带来了复杂性，也带来了机会。如果你能保证正确的专家在正确的时刻落在正确的设备上、把通信降到最低，就能大幅加速这些巨型模型。

我们还应考虑算法效率的改进。占用内存更少的优化器可能至关重要。传统的 Adam 优化器变体通常会为动量和方差估计额外保留两份权重副本，这实际上让内存占用增加到三倍。如果你有 100 万亿个参数权重，就需要额外 200 万亿个值来存放优化器状态！Adafactor、Shampoo 这类省内存的优化器有助于削减这一开销。

激活检查点之类的技术用重算激活代替保存激活，以计算换内存。在 100 万亿参数的规模下，你几乎肯定会大量使用检查点。一个更加激进的想法或许是：我们不必每一步都更新所有权重。可以考虑以轮换的方式更新权重的子集——就像人们不会每天浇灌每一株植物，而是轮着来。只要运用得当，模型依然能有效学习，只是每个参数的更新频率降低了。这会减少系统总的计算需求。

这类想法已进入算法设计的领域，但性能优先的视角依然有用。对训练和推理的每一个环节，我们都应追问："X 真的需要做得这么频繁、或用这么高的精度吗？"答案往往是：可以找到一个更廉价、但依然奏效的近似。在 100 万亿参数的规模下，这些近似能省下数月时间或数百万美元。

超大规模训练中一个经常被忽视的方面是基础设施与网络。当你谈论的是 10,000+ 块 GPU 同训一个模型时，网络架构的重要性不亚于 GPU 本身。Ethernet 与 InfiniBand 技术在更高吞吐以及更智能的自适应路由等方面不断进步。NVIDIA 的 Spectrum-X 是面向 AI 优化的 Ethernet 网络架构（例如 RoCE、自适应路由、高对分带宽），能减少大规模训练与推理工作负载中的拥塞。

性能工程师需要透彻理解这些层级，并确保数据在正确的时刻出现在正确的位置。目标是模拟出一个横跨 GPU 与 CPU 的巨大内存空间，即使模型装不进一台机器，程序员也能相当透明地加以处理。例如，如今借助统一内存和按需分页系统，使用 `cudaMemPrefetchAsync()` 在目标设备上预置页面、避免缺页停顿，已经部分成为可能。但在 100 万亿参数的规模下，这项功能将受到真正的考验。

xAI、OpenAI、Microsoft 这样的前沿研究实验室正在建设 1,000,000+ 块 GPU 的大型集群，这并不奇怪。在 100 万亿参数的规模下，一个作业可能横跨一整座数据中心的硬件。性能工程师必须以数据中心乃至多数据中心（全球）的尺度来思考。

最后，随着模型——及其所需算力——不断扩张，还出现了一个社会技术层面的趋势。对任何单一团队，甚至是单一公司来说，独自训练最大的模型都可能变得不切实际。（但愿）我们会看到 AI 社区开展更多协作与共享，以支撑这些庞大的项目。这类似于粒子物理实验这类大型科学项目由众多机构共同参与的情形。类似现已解散的非营利组织 Open Collective Foundation 的倡议机制，可以帮助汇集 AI 算力资源来训练一个 100 万亿参数模型，然后与世界共享。

这将要求对检查点格式等事项进行标准化、共同开发训练代码，并思考模型的多方所有权。虽然这本身算不上性能问题，却会影响我们构建大型 AI 系统的方式。我们还需要让系统更加容错、更易于拍摄快照以共享部分成果。作为工程师，你最终可能既要为纯粹的速度优化，也要为可复现性和互操作性优化。这样，不同团队才能顺畅高效地分工完成训练与推理流程中的不同部分。

通往 100 万亿参数模型之路需要整体性的全栈创新。这一挑战没有单一解决方案，相反，拼图的每一块都必须改进。硬件要更快、能容纳更多数据；软件要更善于自我优化——通过编译器、AI 助手和实时适配更高效地利用资源；算法要足够巧妙，通过稀疏性、更低精度和更好的优化器避免做不必要的工作。

性能工程师的角色，是把所有这些进展整合成一条连贯的工作流。这就像组装一辆高性能赛车：发动机、轮胎、空气动力学和车手的技艺必须协同一致。只要我们做对了，那些今天看似不可能的事——例如以可承受的成本训练 100 万亿参数——终将变成可能。

就在不久之前，1 万亿参数模型听起来还像天方夜谭。而如今，开放权重模型如 Moonshot AI 的 Kimi K2（1 万亿参数 MoE，每个 token 激活 320 亿参数）等已经证明了这一规模。照这样的进展速度，再加上 AI 辅助下的人类才智，我们将用很短的时间攻克下一个里程碑并再攀升几个数量级。

20.2 关键要点

以下要点总结了本章案例研究以及超大规模 AI 系统性能工程的假想未来状态中所讨论的最佳实践与新兴趋势：

软硬件协同设计优化

LLM 的性能提升，真正来自软硬件深度协同设计（codesign）的创新突破。

AI 辅助编码与性能优化

Google DeepMind、NVIDIA 和 Predibase 已经展示了针对矩阵乘法、注意力等核心 kernel 的 AI 辅助发现与优化。这些实践表明，AI 能够生成、测试并迭代改进底层 GPU 代码，在极少人工干预的情况下取得显著的加速。

面向百万亿参数模型的策略

训练拥有 100 万亿参数的模型，需要激进的量化、多维并行（数据并行、流水线并行、张量并行、专家并行以及上下文/序列并行）相结合，并精细编排机架间通信。这说明未来的 AI 扩展既取决于硬件能力，也取决于软件层调度上的巧思。

算力基础设施的指数级扩展

下一代 AI 数据中心的设计目标，是提供高出若干个数量级的算力。这些设施将以远超当今水平的算力预算来训练 AI 模型，使得能够使用相当于现有系统 100 到 1000 倍 FLOPS 的算力来完成训练。

不断演进的 AI 模型与智能体

未来的模型将是能够自我改进的系统：它们可以生成并优化代码，用新数据持续更新权重，甚至重写自身的代码。这种持续的学习与打磨循环将缩短突破之间的间隔，并造就一支在研究和研发任务上胜过人类团队的虚拟劳动力。

AI 辅助的实时故障排查

除调度之外，AI copilot 还将监控系统日志和训练/推理工作负载，以快速发现异常——包括准确率

损失飙升或硬件错误数量增多等。这些 copilot 可帮助自动化调试、开展故障分析，甚至通过强化学习习得最优配置。它们有助于最大化每瓦性能与单位时间性能。

每瓦性能：一项关键指标

所有这些协同设计努力，最终都旨在最大化单位成本下的吞吐量。具体而言，目标是让每一美元、每一瓦电力能处理并生成更多的 token。例如，Grace Blackwell NVL72 机架系统将每瓦性能较上一代 Hopper 提升了 25 倍。这直接意味着每 token 成本低于前代 GPU 集群。

20.3 结论

本书标志着 AI 系统性能工程这一领域的转折点。NVIDIA 将 CPU 与 GPU 紧密集成于 Grace Hopper、Grace Blackwell（以及即将推出的 Vera Rubin 和 Feynman）等超级芯片模块，达到了新的计算效率与规模高度。在其内部，GPU 使用高度优化的 Tensor Core，以及针对 LLM 计算基本原理优化的 Transformer Engine。

像 NVIDIA GB200/GB300 NVL72 这样的超级计算系统，将 72 块 GPU 连接成单一处理单元（NVLink 域），借助 NVLink、NVSwitch 和 SHARP 等技术奠定了机架与数据中心通信的基础。这些技术为多万亿参数模型提供低延迟的实时推理。

在软件方面，vLLM、SGLang、NVIDIA Dynamo 和 `TensorRT-LLM` 等工具可改善大型推理集群的调度与资源利用，其中包括 in-flight batching、分页 KV 缓存，以及（出于效率考虑，将提示词预填充阶段与生成解码阶段分离到不同的资源池）等技术。这些手段有助于降低尾延迟并提升每瓦吞吐量。

这些实例证明了协同设计的威力——硬件、软件与算法共同演进。这种根植于机械同情（mechanical sympathy）理念的协作，有助于缩短训练时间、提升推理性能并降低运营开支。这正是今天的 AI 系统——改进迅速且资本密集——获得可衡量的投资回报所必需的。

此外，来自 Google DeepMind、NVIDIA 和 Predibase 的 AI 驱动编码与算法智能体展示了 AI 如何帮助优化 AI。当模型和系统复杂到无法靠人工调优时，自动化可以承担常规优化工作，让人类工程师专注于更高层次的优化与系统设计。

我们正从暴力扩展转向智慧扩展：在每个周期里完成更多有效的工作，把新硬件特性中的每一分性能都榨取出来，并让 AI 助手去处理细节。性能工程师将沿着技术栈向上攀登，成为在效率、可靠性与可持续性之间取得平衡的全球算力生态的架构师。

我们作为 AI 系统性能工程师的角色，将从单节点 kernel 扩展到系统级乃至设施级的优化。我们将凭借直觉发现瓶颈——比如一个缓慢的 all-reduce 模式——然后引导 AI 工具去修复它。与此同时，我们仍需不断学习，因为硬件与算法创新的速度只会越来越快。

总而言之，为保持竞争力和价值，你应当打下坚实的 AI 系统性能基础，保持好奇心，勇于试验新的软硬件进展，信任 AI 的建议，并准备好在技术图景向量子计算及更远处演进时随机应变。想一想——在一个研究大众化、AI 超级计算机一键可得、多万亿参数模型触手可及的时代，你完全可能成为下一次超级智能重大突破的推动者之一！

附录　AI 系统性能检查清单（175+ 项）

这份详尽的检查清单同时涵盖流程层面的广泛最佳实践，以及面向 AI 系统性能工程师的细粒度底层调优建议。每一项清单都是一条实用提醒，帮助从 AI 系统中榨取最大的性能与效率。

在调试、性能分析、分析和调优 AI 系统时使用本指南。系统地应用这些技巧——从底层操作系统与 CUDA 调整，到集群规模的优化——AI 系统性能工程师就能在现代 NVIDIA GPU 硬件上，借助 CUDA、PyTorch、OpenAI 的 Triton、TensorFlow、Keras 和 JAX 等众多 AI 软件框架，实现闪电般的执行速度与高性价比运行。这份清单中的原则也适用于 NVIDIA 硬件的后续世代，包括其 GPU、基于 ARM 的 CPU、CPU-GPU 超级芯片、网络设备和机架系统。

20.4 性能调优与成本优化思维方式

务实且有记录的循环——先做快速见效的工作，再进行深度工作——能把工程时间转化为可衡量的投资回报。先针对最大的运行时间和成本驱动因素，并始终在优化前后进行性能分析，以验证影响。

将自动调优、框架升级、云定价杠杆和利用率仪表板结合起来，争取高投资回报的收益；记录结果，并优先采用简单、易维护的修复。当准确率允许时，调节对吞吐量敏感的超参数。以下是关于性能调优与成本优化思维方式的一些建议：

优先优化昂贵部分　使用 80/20 法则。找出对运行时间贡献最大的部分，并聚焦于这些部分。如果 90% 的时间花在少数几个 kernel（内核）或通信阶段上，那么深入优化它们，比微优化只占 1% 时间的部分更合适。每章介绍的技术都应应用在最重要的地方。例如，如果训练时间中 40% 用于数据加载、50% 用于 GPU 计算、10% 用于通信，就先修复数据加载，因为可能将其开销减半，然后再观察 GPU kernel 优化。

优化前后都进行性能分析　每次应用优化时，都要测量其影响。这听起来显而易见，但实际中常常依据理论进行调整，而调整可能没有帮助，甚至会造成

伤害。假设工作负载并非内存受限，却为训练任务启用激活检查点；这可能因使用额外计算来换取内存节省而使任务变慢。换言之，改变前后都要比较吞吐量、延迟和利用率等关键指标。简单计时可以使用内置性能分析器，例如统计 100 次迭代的平均迭代时间。

采用自适应自动调优反馈循环 实现利用实时性能反馈的高级自动调优框架，使用强化学习或贝叶斯优化等技术动态调整系统参数。这样，系统就能针对变化的工作负载和运行条件持续微调设置。

为优化时间留出预算 性能工程是一项迭代式投资，收益会递减——先摘取启用 AMP、数据预取等低垂果实。这些可能带来 2× 的收益；编写自定义 kernel 等更困难的优化，增量可能更小。始终权衡工程时间与运行时间收益和节省的成本。对于训练旗舰模型等大型重复作业，即使提升 5% 也可能值得数周调优，因为它或许能节省数百万成本。对于一次性或小型工作负载，应聚焦更大的收益并保持务实。

跟进框架改进 混合精度、融合 kernel 和分布式算法等许多优化，仍在深度学习框架与库中持续改进。升级到最新的 PyTorch 或 TensorFlow 有时能立即提速，因为它们会加入新的融合算子或更好的启发式策略。利用这些改进，因为它们基本上是免费的收益；阅读与性能相关的发行说明。

与供应商和社区成员开展协同设计 与硬件供应商及更广泛的性能工程社区保持联系，使软件优化与最新硬件架构保持一致。这种协同设计能够通过定制算法以利用新兴硬件能力，发现显著的性能机会。定期查阅供应商文档、参加论坛并测试驱动程序或框架的 beta 版本。这些互动常会揭示新的优化机会和最佳实践，可整合进系统。集成新的驱动程序优化、库更新和硬件专属技巧，可能带来额外的、甚至显著的性能收益。

利用云的灵活性降低成本 如果在云环境中运行，应明智地使用更便宜的抢占式实例或预留实例。它们能大幅削减成本，但抢占式实例可能在几分钟的通知后被回收。还要考虑实例类型：有时，成本仅为最新实例一小部分的稍旧 GPU 实例，如果工作负载不需要绝对最新的硬件，反而能提供更好的性价比。我们关于 H800 与 H100 的讨论表明，付出努力也能在次优硬件上完成出色工作。云环境中也存在类似权衡。通过在不同实例配置上进行基准测试评估成本/性能，包括 CPU 数量、CPU 内存、GPU 数量、GPU 内

存、L1/L2 缓存、统一内存、NVLink/NVSwitch 互连、网络带宽与延迟以及本地磁盘配置。计算每美元吞吐量等指标，为优化决策提供依据。

监控利用率指标 持续监控 GPU 利用率、SM 效率、内存带宽使用率，以及多节点情况下的网络利用率。使用 DCGM exporter、Prometheus 等建立仪表板，以便发现任何资源未被充分使用。如果 GPU 利用率只有 50%，要追查原因；很可能是数据等待/停顿和同步通信缓慢。如果网络利用率仅为 10%，但 GPU 在等待数据，则问题可能是锁等其他因素。这些指标有助于确定应聚焦的子系统。

迭代并调节面向吞吐量的超参数 批大小、序列长度、MoE 激活专家数等模型超参数，可以在不降低最终准确率的情况下针对吞吐量进行调节。例如，更大的批大小能提供更好的吞吐量，但可能需要调节学习率计划来维持准确率。不要害怕调整这些参数，寻找速度与准确率的平衡点。这也是性能工程的一部分——有时可以调整模型或训练过程来提高效率，例如使用激活检查点，或用更多计算步骤实现相同的有效批大小。还可以调节训练学习率计划来补偿这种情况。

记录并复用 记录所应用的优化及其影响。在代码中或内部类似 wiki 的共享知识库系统中形成文档。这会为未来项目建立知识库。许多技巧都是可复用的模式，例如启用重叠操作和在集群上有帮助的特定环境变量。有了这些历史记录，在开始新项目或让新团队成员参与性能调优时可以节省时间。

在优化与复杂度之间取得平衡 以达到所需性能的最简单方案为目标。例如，如果原生 PyTorch 配合 `torch.compile` 已满足速度目标，就不一定需要编写自定义 CUDA kernel。这有助于避免额外维护。过度使用高度定制的代码会使系统变得脆弱。既快速又易维护的方案具有简洁之美。因此，采用能够带来所需收益的最小侵入式优化，仅在需要时升级到更复杂的优化。

优化 AI 驱动的性能 利用机器学习模型分析历史遥测数据并预测系统瓶颈，从而实时自动调整参数，优化资源分配和吞吐量。

20.5 可复现性与文档最佳实践

性能收益如果不可复现、未版本化且未持续检查，就无法持久；否则它们会随时间悄然回退。将文档、CI 基准测试和共享知识视为保存加速效果、加快上手与审计的黏合剂。

在源代码控制中锁定版本、配置和基准，以便实验可重复、回退可追踪。将性能检查纳入 CI/CD，构建端到端监控和告警，并将优化与安全及完整文档结合，形成持久、可审计的实践。以下是改进可复现性和文档的一些建议：

严格的版本控制 对所有系统配置、框架/驱动程序版本、操作系统设置、优化脚本和基准进行全面的版本控制。使用 Git（或类似系统）跟踪变更并标记发布版本。这样既能精确复现实验，也能轻易识别性能回退。

用于性能回退的持续集成 将自动化性能基准和实时监控集成到 CI/CD 流水线中。这样，从代码更新到配置变更的每次修改，都能依据一组性能指标进行验证，有助于及早发现回退，并保持一致且可衡量的性能收益。采用 MLPerf 等行业标准基准，建立可靠的性能基线并随时间跟踪改进。

端到端工作流优化 确保优化以整体方式应用于完整 AI 流水线——从数据摄取和预处理，到训练和推理部署。协调的跨系统调优能够揭示孤立调整可能错过的协同效应，从而带来更显著的整体性能收益。

自动化监控与诊断 部署端到端监控方案，收集硬件、网络和应用层的实时指标。将它们与 Prometheus/Grafana 等仪表板集成，并配置自动告警，及时发现 GPU 利用率突然下降或网络延迟激增等异常。

容错与自动恢复 通过分布式检查点、冗余硬件配置和动态作业重新调度，将容错纳入系统设计。这种策略能减少停机时间，并在硬件或网络故障时保持性能。

编译器与构建优化 在构建过程中利用激进的编译器标志和基于性能分析的优化，从代码中提取最大性能。定期更新并调节构建配置，并通过严格基准测试验证每次变更的影响，以确保执行最优。

安全、合规与性能 对安全、合规和性能进行协同设计。定期审计配置、强制访问控制，并维护行业标准的安全防护措施，包括加密、安全数据通道、零

信任网络、硬件安全模块（HSM）和安全隔离区。同时确保性能调优绝不损害系统安全；同样，也要确保安全不会带来不必要的性能开销。

全面的文档与知识共享 详细记录所有优化步骤、系统配置和性能基准。建立内部知识库，促进团队协作和快速上手，确保最佳实践得到保存并在项目间复用。

面向未来与可扩展性规划 设计模块化、可适应的系统架构，使其能够轻松纳入新兴硬件和软件技术。持续评估可扩展性需求并更新优化策略，在工作负载增长时维持有竞争力的性能。

20.6 系统架构与硬件规划

硬件、互连和数据路径决定了性能与成本效率的上限——软件调优无法跑赢一个吃不饱的 GPU。通过让加速器、CPU/DRAM/I/O 以及散热/供电与工作负载匹配，规划每美元/瓦的有效吞吐量，从一开始就避免瓶颈。具体而言，要为有效吞吐量——即每美元/瓦完成的有用工作——而非仅仅为原始 FLOPS 进行设计。让加速器与互连匹配工作负载，合理配置 CPU/内存/I/O 以持续供给 GPU，让数据保持本地，并规划供电/散热，使硬件维持峰值时钟。在增加更多 GPU 前先评估扩展效率。以下是优化系统架构和提高硬件规划效率的一些建议：

为有效吞吐量与效率而设计 把有用吞吐量作为目标。每一点性能提升在规模化时都能转化为巨大的成本节省。聚焦于最大化每美元/瓦的生产性工作，而不是原始 FLOPS。

选择合适的加速器 优先选择现代 GPU，以获得更高的每瓦性能和更大的内存容量。更新的架构提供原生 FP8、FP4 精度支持以及更快的互连等特性，相比老一代 GPU 和系统可带来显著加速。

利用高带宽互连 对于多 GPU 工作负载，使用带 NVLink/NVSwitch 的系统，例如 GB200/GB300 NVL72，而不是仅有 PCIe 的连接。NVLink 5 提供最高 1.8 TB/s 的双向 GPU-GPU 带宽（超过 PCIe Gen5 的 14×），支持 GPU 间近线性扩展。NVLink Switch 域可用第二级交换机扩展，在一个 NVLink 域中连接最多 576 个 GPU，从而尽可能让分层 collective 保持在 NVLink 上，只有必要时才回退到机架间 fabric。

平衡 CPU/GPU 与内存比例 为每个 GPU 配备足够的 CPU 核、DRAM 和存储吞吐量。例如，为数据加载和网络任务分配每个 GPU 约 1 个快速 CPU 核。确保系统 RAM 和 I/O 能以所需速率供给 GPU，量级约为每 GPU 数百 MB/s，以避免饥饿。

规划数据局部性 如果跨多个节点训练，应尽量减少节点外通信。只要可能，就将紧密耦合的工作负载放在同一个 NVLink/NVSwitch 域中以利用完整带宽，并使用可获得的最高速互连。理想情况下，节点内和机架内通信使用 NVLink，机架间通信使用 InfiniBand。

避免链路中的瓶颈 找出最慢的链路——无论是 CPU、内存、磁盘还是网络——并扩展它。例如，如果 GPU 利用率低是因为 I/O，就投资更快的存储或缓存，而不是增加 GPU。端到端设计中所有组件彼此匹配，可以避免 GPU 周期浪费。

选择合适的集群规模 增加 GPU 时要警惕收益递减。超过某个集群规模后，开销可能增长；要确保加速比值得成本。通常应先在 N 个 GPU 上优化利用率，例如达到 95% 的使用率，再扩展到 2N 个 GPU。

为散热与供电进行设计 确保数据中心能够处理 GPU 的热和功耗需求。GB200/GB300 等高性能系统具有很高的 TDP。提供充足的散热（很可能是液冷）和供电，使 GPU 能在不降频的情况下维持 boost 时钟。

20.7 统一 CPU-GPU“超级芯片”架构

统一内存和封装内链路使你能够容纳更大的模型，并在把正确的数据放到正确层级时减少复制开销。使用 Grace 进行预处理，将“热”张量放在 HBM 中，可把超级芯片变成紧密耦合、停顿更少的引擎。在 Grace Blackwell 超级芯片上，应将 CPU 和 GPU 视为共享内存复合体。将热权重/激活保存在 HBM 中，将溢出或不常用数据通过 NVLink-C2C 放在 Grace LPDDR 中。利用封装内 Grace CPU 进行预处理/编排，并使用预取或流水线管理的内存隐藏超大模型的延迟。按以下方式利用超级芯片架构：

利用统一 CPU-GPU 内存 利用 Grace Blackwell（GB200/GB300）超级芯片的统一内存空间。两颗 Blackwell GPU 与一颗 72 核 Grace CPU 通过 NVLink-

C2C（900 GB/s）共享一致性内存池。将 CPU 的大容量内存（例如 480 GB LPDDR5X）作为超大模型的扩展，同时把“热”数据保留在 GPU 的 HBM 中以获得速度。

让数据保持局部 即使有统一内存，也要优先考虑数据放置。将模型权重、激活和其他频繁访问的数据放在 GPU HBM3e（其本地带宽高得多）中，让不常用或溢出数据驻留在 CPU RAM 中。这样可避免 900 GB/s 的 NVLink-C2C 链路成为关键数据的瓶颈。

可用时利用 CPU-GPU 直接内存访问 利用 GB200、GB300 等组合式 CPU-GPU 超级芯片中 GPU 直接访问 CPU 内存的能力。GPU 可以通过 NVLink-C2C 一致地读写 Grace LPDDR 内存，无需经主机 PCIe 暂存。带宽和延迟仍低于 HBM，因此应预取受管理指针、暂存数据，并对传输进行流水线化以隐藏延迟。建议将热激活和 KV 缓存保存在 HBM 中，将 CPU 内存作为带显式预取的低层级缓存。

有效利用 Grace CPU 封装内 Grace CPU 提供 72 个高性能核心——要利用它们！将数据预处理、增强和其他适合 CPU 的任务卸载到这些核心。它们可通过 NVLink-C2C 快速为 GPU 供给数据，实质上成为 GPU 极快的 I/O 和计算伴侣。

为超大模型进行规划 对于超过 GPU 内存的万亿参数模型训练，GB200/GB300 系统允许把 CPU 内存作为模型内存池的一部分来训练。优先使用框架缓存分配器，并在自定义代码中使用 `cudaMallocAsync`，以减少碎片并启用图捕获。使用 CUDA Unified Memory 或受管理内存 API 优雅地处理溢出，并考虑显式预取（例如 `cudaMemPrefetchAsync`）即将使用的层，将其从 CPU 内存预取到 GPU 内存，以隐藏延迟。

考虑面向超级芯片优化的算法 SuperOffload 是一组面向超级芯片优化的算法示例，重点提高卸载以及张量转换/复制策略的效率。其创新包括先推测后验证（STV）、异构优化器计算和基于 ARM 的 CPU 优化器。它专为 NVIDIA 超级芯片（例如 Grace Hopper、Grace Blackwell、Vera Rubin）设计；与传统卸载策略相比，SuperOffload 提高了 token 处理吞吐量和芯片利用率。

20.8 多 GPU 扩展与互连优化

只有通信快速且了解拓扑时，扩展才有收益；否则增加的 GPU 只会彼此等待。依靠 NVLink/NVSwitch 带宽、现代 collective 和了解 fabric 的放置，接近线性加速。具体而言，利用 NVLink/NVSwitch 域（例如 NVL72）实现近线性扩展，并选择适合 fabric 的并行策略。使用了解拓扑的放置、更新后的 NCCL collective（例如 PAT）和遥测，验证是否有效利用每 GPU 约 1.8 TB/s 的双向吞吐量。扩展时规划分层通信。以下是利用多 GPU 扩展、互连和拓扑优化的一些建议：

为高速全互连拓扑而设计 以具有 72 个完全互连 GPU 的 NVL72 NVSwitch 集群为例，任何 GPU 都能以完整 NVLink 5 速度与其他 GPU 通信。在 fabric 层面，NVLink Switch 域是无阻塞的。应用层吞吐量可能因并发流量和路径调度而变化，因此在假定每对链路饱和前，使用 DCGM NVLink 计数器和 Nsight Systems 轨迹验证行为。利用数据并行、张量并行和流水线并行等并行化策略，这些策略在较弱的互连上可能成为瓶颈。

使用了解拓扑的调度 只要可能，始终将多 GPU 作业放在同一个 NVLink Switch 域内。让作业的所有 GPU 位于 NVL72 fabric 上，通信密集型工作负载就能近线性扩展。将 GPU 混放在不同 NVLink 域或标准网络之间，会产生瓶颈，应避免用于紧密耦合任务。

利用前所未有的带宽 注意 NVLink 5 每 GPU 每个方向提供 900 GB/s，是上一代每 GPU 带宽的两倍。NVL72 机架的总机架内带宽约为 130 TB/s。即使数十 GB 的梯度数据也能在 1.8 TB/s 下几毫秒内完成 all-reduce，大幅降低通信等待时间。设计梯度同步和参数分片等训练算法，充分利用这份相对“免费”的通信预算。

采用现代 collective 算法 使用针对 NVSwitch 优化的最新 NVIDIA NCCL 库。具体启用并行聚合树（PAT）算法，该算法为 NVLink Switch 拓扑引入。它利用 NVL72 拓扑更高效地执行归约，相比其他树/环算法进一步减少同步时间。

考虑细粒度并行 借助全带宽全互连，考虑以前不可行的细粒度模型并行。例如，当每个 GPU 到其他 GPU 都有 1.8 TB/s 双向吞吐量时，可在许多 GPU 间

进行逐层并行或张量并行。过去可能会避免过多 GPU 间通信，但 NVL72 允许激进地划分工作而不触及网络限制。

监控饱和情况 虽然 NVL72 极快，仍要在性能分析中关注链路利用率。如果应用通过极端的全互连操作使 NVSwitch 饱和，例如，可能需要通过聚合梯度等方式限制通信。使用 NVIDIA 工具或 NVSwitch 遥测验证通信在 NVLink 容量以内，并按需调整模式。例如，可以错开 all-to-all 交换来避免网络争用。DCGM 暴露的 NVLink 计数器有助于验证链路平衡并在 collective 期间发现热点。

规划未来扩展 注意 NVLink Switch 可超越单个机架扩展：借助第二级交换机，一个连接域最多可扩展到 576 个 GPU。在这种超大规模下，首先规划使用本地 NVL72 机架内 collective 的分层通信，只有必要时才使用机架间互连。这有助于优先最大化机架内 NVLink 的使用，确保在采用机架间 InfiniBand 跳转前先利用最快链路。

发现联邦式与分布式优化机会 对于跨异构环境的部署，例如多云或边缘到云的设置，采用自适应通信协议和动态负载均衡策略。这能减少延迟、最大化分布式系统吞吐量，即使资源的能力与容量不同，也能保持稳健性能。

20.9 操作系统与驱动程序优化

操作系统抖动、NUMA 未命中和驱动程序不匹配会悄悄消耗吞吐量并造成无法通过调优消除的波动。加固软件栈（大页、亲和性、一致的 CUDA/驱动程序、持久化）能建立稳定的高性能基线。运行经过 HPC 调优的精简 Linux；设置 NUMA/IRQ 亲和性，启用 THP 和高 memlock。保持各节点 NVIDIA 驱动程序/CUDA 一致。隔离系统抖动，调优 CPU 库/存储，正确设置容器限制，并使 BIOS/固件/NVSwitch fabric 保持最新，以获得可预测吞吐量。以下是应在环境中探索的一些主机、操作系统和容器优化：

使用针对 HPC 调优的 Linux 内核 确保 GPU 服务器运行近期、稳定且配置用于高性能计算的 Linux 内核。禁用消耗 CPU 或 I/O 的不必要后台服务。使用“performance”CPU 调频器，而不是“on-demand”或“power-save”，使 CPU 核保持较高时钟以供给 GPU。

对性能关键工作负载禁用 swap 在训练服务器上禁用 swap 以避免页面抖动；如果必须保持启用，则使用 `mlock` 或 `cudaHostAlloc` 锁定关键缓冲区，确保它们留在 RAM 中。

通过激进预分配避免内存碎片 为常用张量预分配大块连续内存，以减少运行时分配开销和碎片。这种主动策略可确保长时间训练期间内存管理更稳定、更高效。

优化 CPU 库环境变量 微调 `OMP_NUM_THREADS` 和 `MKL_NUM_THREADS` 等参数，使其更好匹配硬件配置。调整这些变量可以减少线程争用，提高 CPU 受限操作的并行效率。

设计时考虑 NUMA 感知 对于多 NUMA 服务器，将 GPU 进程/线程绑定到本地 NUMA 节点的 CPU。使用 `numactl` 或 `taskset` 将每个训练进程绑定到其分配 GPU 最近的 CPU。同样，将内存分配绑定到本地 NUMA 节点（`numactl --membind`），使 GPU DMA 的主机内存来自最近的 RAM。这样可避免代价高昂的跨 NUMA 内存流量，否则有效 PCIe/NVLink 带宽可能减半。

为网络和 GPU 任务利用 IRQ 亲和性 明确将 NIC 中断绑定到与 NIC 位于同一 NUMA 节点的 CPU 核，并将 GPU 驱动线程绑定到专用核——包括 `nvidia-persistence` 服务守护进程等长时间运行服务的线程。这种策略能减少跨 NUMA 流量，并稳定重负载下的性能。

启用透明大页 以 always 或 madvise 模式打开透明大页（THP），使大型内存分配使用 2 MB 页面。在为框架分配数十或数百 GB 主机内存时，这能减少 TLB 抖动和内核开销。检查 `/sys/kernel/mm/transparent_hugepage/enabled`，确认 THP 已激活。启用 THP 后，进程的大型分配会使用大页。如果工作负载对延迟敏感且观察到抖动，优先使用 madvise 模式的 THP。

提高最大锁定内存 配置操作系统以允许大型固定（即页锁定）分配。GPU 应用经常为快速传输而锁定内存；设置 `ulimit -l unlimited` 或较高值，使数据加载器能够分配固定缓冲区而不触及操作系统限制。这能防止失败或回退到可分页内存，后者会减慢 GPU DMA。

使用最新 NVIDIA 驱动程序和 CUDA 软件栈 在所有节点保持 NVIDIA 驱动程序和 CUDA 运行时更新（处于经过测试的稳定版本）。新驱动可能带来性能改进，并且新 GPU 的计算能力需要新驱动。确保所有节点的驱动/CUDA 版本相同，避免多节点作业不匹配。在启动时为 GPU 启用持久化模式（`nvidia-smi -pm 1`），使驱动保持加载，GPU 不产生重新初始化延迟。更新所有节点上的 NVIDIA 驱动和工具包，以获得 bug 修复和性能改进。

使用 MIG 配置时启用 GPU 持久化 启用持久化模式后，GPU 会保持“预热”并可立即使用，减少作业启动延迟。这对使用多实例 GPU（MIG）分区尤其重要；没有持久化时，MIG 配置会在每个作业中重置，而保持驱动活跃可保留这些切片。使用 MIG 时始终配置持久化模式。

隔离系统任务 在每台服务器上为操作系统维护工作（例如中断处理和后台守护进程）专门保留一个或一小组 CPU 核。这样，为 GPU 供给数据的主 CPU 线程不会被打断。可以使用 CPU 隔离或 cgroup 绑定实现。消除操作系统抖动可确保吞吐量一致。

优化系统 I/O 设置 如果工作负载有大量日志记录或检查点，使用偏向吞吐量的选项挂载文件系统。考虑对数据磁盘使用 `noatime`，并增大流式读取的文件系统预读。确保磁盘调度器配置适当；对 NVMe SSD 使用 `mq-deadline` 或 `noop`，减少延迟波动。

定期维护 保持 BIOS/固件更新，以获得性能修复。一些 BIOS 更新会改善 PCIe 带宽，或修复 GPU 的输入-输出内存管理单元（IOMMU）问题。还要定期检查 NIC 和 NVSwitch/Fabric 的固件更新（如适用），例如 NVIDIA 提供的 Fabric Manager 升级。细小的固件调整有时能解决隐蔽的瓶颈或可靠性问题。

将 Docker 和 Kubernetes 配置调至最大性能 在容器中运行时，加入 `--ipc=host` 等共享内存选项，并设置 `--ulimit memlock=-1` 以避免内存锁定问题。这确保容器化进程访问内存时不受操作系统限制。

20.10 GPU 资源管理与调度

更智能的放置与分区可以在不购买新硬件的情况下提高利用率，并保护混合工作负载的可预测性。尊重拓扑，在适当场合使用 MPS/MIG，并控制时钟/功耗，以尽量减少争用和尾延迟。按 GPU/NUMA/NVLink 拓扑进行调度；使用 MPS 或 MIG 提高小作业利用率，同时保留 ECC 和持久化以确保可靠性。必要时锁定时钟或设置功耗上限以保持稳定，避免 CPU 过度订阅，并智能地打包作业，在没有争用的情况下最大化投资回报。以下是 GPU 资源管理与调度的一些建议：

了解拓扑的作业调度 确保 Kubernetes 和 SLURM 等编排器将容器调度到尊重 NUMA 与 NVLink 边界的节点上，以减少跨 NUMA 和跨 NVLink 域的内存访问。这种对齐能降低延迟并提高整体吞吐量。

多进程服务（MPS） 在单个 GPU 上运行多个进程时启用 NVIDIA MPS，提高利用率。MPS 允许不同进程的 kernel 在 GPU 上并发执行，而不是进行时间片切换。如果单个作业无法完全占满 GPU，这一方法很有用；例如，在一个 GPU 上运行 4 个训练任务时，MPS 能让它们的工作重叠，提高总体吞吐量。

多实例 GPU（MIG） 使用 MIG 将高端 GPU 分成较小实例，供多个作业使用。如果有许多轻量工作负载，例如推理小模型或运行许多实验，可以切分 GPU，为每个作业保证资源。现代 GPU 可以切成多个 MIG 切片（最多 7 个）。不要对紧密耦合的并行作业使用 MIG，因为这类作业受益于完整 GPU 访问。当作业小于完整 GPU 时，用 MIG 实现隔离并最大化 GPU 投资回报。

MIG 的持久化 保持持久化模式开启，以在作业之间维持 MIG 分区。这能避免重新分区开销，并确保后续作业无需延迟即可看到预期 GPU 切片。在集群启动时配置 MIG 并保持启用，使调度可预测；运行中改变 MIG 配置需要重置 GPU，可能干扰正在运行的作业。安排维护窗口，因为 GPU 重启后不会持久保存设备分区。使用 NVIDIA 的 MIG Manager 在启动时自动重新创建所需布局。

GPU 时钟与功耗设置 如果需要运行间一致性，可用 `nvidia-smi -lgc/-lmc` 将 GPU 时钟锁定到固定高频。默认情况下 GPU 使用自动 boost，通常是最

优的，但固定时钟可以避免瞬时降频。在功耗受限场景下，可略微降频或设置功耗上限，使 GPU 保持稳定的热/功耗范围；如果偶发节流曾是问题，这可能带来一致的性能。

ECC 内存 数据中心 GPU 应保持 ECC 启用以确保可靠性，除非有特定理由禁用。性能成本很小——带宽和内存损失约为几个百分点——但 ECC 能捕获否则可能破坏长时间训练作业的内存错误。大多数服务器 GPU 默认启用 ECC；应保持开启以保护持续数周的训练。

了解作业调度器 将 GPU 拓扑集成到 SLURM、Kubernetes 等作业调度器中。在需要低延迟耦合时，配置调度器把作业分配到同一节点或同一 NVSwitch 组。使用 Kubernetes device plugin 或 SLURM Gres 为小作业调度 MIG 切片。了解 GPU 的调度器能避免单个作业跨越相距很远的 GPU 而遭遇带宽问题。

CPU 过度订阅 调度作业时，考虑每个 GPU 任务的 CPU 需求，例如数据加载线程等。不要在节点上打包超出 CPU 处理能力的 GPU 作业。与其让 CPU 过载导致所有 GPU 供给不足，不如让一个 GPU 空闲。监控每个 GPU 作业的 CPU 利用率，为调度决策提供依据。

使用 NVIDIA Fabric Manager 管理 NVSwitch 在带 NVSwitch 的系统上，例如 GB200/GB300 NVL72 机架，确保 NVIDIA Fabric Manager 正在运行。它管理 NVSwitch 拓扑与路由。没有它，多 GPU 通信可能未充分优化，甚至可能无法支持大型作业。Fabric Manager 服务通常默认运行在带 NVSwitch 的服务器上，但应再次确认它已启用且正在运行，尤其是在驱动程序更新之后。

为利用率打包作业 智能地打包作业以最大化利用率。例如，在 4-GPU 节点上，如果有两个不太使用 CPU 的 2-GPU 作业，可以将它们一起运行在同一节点上，节约资源；如果都在同一计算节点或启用 NVLink 的机架内运行，还能利用更快的 NVLink 通信。反之，避免将合起来超过节点内存或 I/O 容量的作业放在一起。目标是在不产生争用的情况下提高硬件利用率。

20.11 I/O 优化

如果数据供给跟不上，GPU 就会空闲——通常最大且最便宜的加速来自修复输入而非数学计算。并行化、固定内存、异步传输和快速存储能确保模型持续得到供给。通过并行化数据加载器、使用固定内存和异步传输，并将数据存放在快速 NVMe（最好使用 GPUDirect Storage）上，让 GPU 保持有数据可用。合理地进行条带化、缓存和压缩。测量端到端吞吐量，确保 I/O 随集群规模扩展，并异步写入检查点/日志。以下是数据流水线 I/O 优化的一些建议：

并行加载数据 使用多个 worker/线程为 GPU 加载和预处理数据。默认的一到两个数据加载器 worker 可能不够。进行性能分析并增加数据加载器进程/线程数量，例如使用 PyTorch `DataLoader(num_workers=N)`，直到数据输入不再是瓶颈。高核数 CPU 就是用来供给这些 GPU 的，因此要确保充分利用。

为 I/O 固定主机内存 为数据传输缓冲区启用固定（即页锁定）内存。许多框架提供类似 PyTorch 的 `pin_memory=True` 选项，让 DataLoader 分配 GPU 可直接 DMA 的主机内存。固定内存能显著提高 H2D 复制吞吐量；将其与异步传输结合，使数据加载与计算重叠。

重叠计算与数据传输 对输入数据进行流水线化。当 GPU 计算批次 N 时，在 CPU 上加载并准备批次 N+1，并使用 CUDA 流和非阻塞 `cudaMemcpyAsync` 在后台传输。双缓冲可以隐藏延迟——理想情况下 GPU 永远无需等待数据。确保训练循环使用异步传输。例如在 PyTorch 中，可以用 `non_blocking=True` 将张量复制到 GPU。异步传输使 CPU 能在传输进行时继续运行，从而通过重叠计算和数据传输提高性能。

使用快速存储（NVMe/SSD） 将训练数据存储在快速本地 NVMe SSD 或高性能并行文件系统上。机械磁盘会严重限制吞吐量。可用时启用 GPU Direct Storage（GDS），使 GPU 能直接从 NVMe 或网络存储流式读取数据，绕过 CPU。读取大型数据集时，这还能进一步减少 I/O 延迟和 CPU 负载。对大型数据集，考虑让每个节点拥有本地副本或数据分片。如果使用网络存储，优先选择带条带化的 Lustre 等分布式文件系统，或能并行服务众多客户端的对象存储。

调节 I/O 并发和条带化 避免单文件访问造成瓶颈。如果所有 worker 使用一个大文件，则将其条带化到多个存储目标，或拆成多个块，使多个服务器能够服务。比如把数据集拆成多个文件，让不同数据加载器 worker 同时读取不同文件。这样可以最大化存储系统的聚合带宽。

优化小文件访问 如果数据集由数百万个小文件组成，应降低元数据开销。每秒打开过多小文件会压垮文件系统的元数据服务器。解决方案包括将小文件打包成 tar 或 RecordIO 文件等较大容器；使用批量读取的数据摄取库；或确保客户端启用元数据缓存。这能减少每文件开销，加快 epoch 启动时间。

可用时使用客户端缓存 利用任何缓存层。如果使用 NFS，增大客户端缓存大小和持续时间。对分布式文件系统，可考虑缓存守护进程，甚至手动把部分数据集缓存到本地磁盘。目标是避免反复从慢速源读取相同数据。如果每个节点在不同时间处理同一批文件，本地缓存可以大幅减少冗余 I/O。

明智地压缩数据 如果 I/O 是瓶颈，可压缩数据集，但使用 LZ4 或 Zstd fast mode 等轻量压缩。这会用少量 CPU 换取 I/O 数据量下降。如果解压缩使 CPU 成为瓶颈，考虑多线程解压，或将其卸载到加速器。还可用一个线程读取压缩数据、另一个线程并行解压，使解压与读取重叠。现代 GPU 配合 GPUDirect Storage 和 cuFile I/O 栈时，可以利用 GPU 计算资源（或图像/视觉数据的专用解码器）进行即时数据解压。

测量吞吐量并消除瓶颈 持续监控数据流水线吞吐量。如果 GPU 利用率没有接近 100%，且怀疑输入滞后，就测量从磁盘读取的 MB/s 以及数据加载器核心有多忙。dstat 或 NVIDIA DCGM 等工具可以显示 GPU 是否在等待数据。系统地调节每个组件：增大预取缓冲、增大网络缓冲区、优化磁盘 RAID 设置等，直到输入流水线能以 GPU 消耗数据的速度供给数据。移除 I/O 停顿后，这些优化常能在相同硬件上把 GPU 利用率从约 70% 提高到 > 95%。

为多节点扩展 I/O 在集群规模下，确保存储系统能够处理聚合吞吐量。例如，8 个 GPU 每个消耗 200 MB/s，就是每节点 1.6 GB/s；100 个节点需要 160 GB/s。很少有中心文件系统能维持这种速度。可通过将数据分片到存储服务器、使用每节点缓存，或预先将数据加载到每个节点的本地磁盘来缓解。用存储空间换吞吐量（例如保留多份数据）通常值得，以避免昂贵 GPU 饿死。

最小化检查点与日志开销 高效写入检查点和日志。可能时异步写检查点，或先写本地磁盘再复制到网络存储，避免训练停顿。压缩检查点或使用稀疏存储格式以减小大小。限制每步日志频率，聚合迭代统计量，每 N 次迭代记录一次而非每次都记录，这会大幅降低 I/O 开销。还可以用 `cuda-checkpoint` 和用户空间检查点/恢复（CRIU）暂停正在运行的 GPU 进程，以持久化进程映像。恢复时，CUDA 驱动可以恢复设备内存和 CUDA 状态，甚至恢复到同一设备类型的其他 GPU。应把这视为模型的 state-dict 或分片检查点文件的补充，而非替代。

20.12 数据处理流水线

数据的格式、布局和局部性决定了流水线在规模化运行时的平滑程度。二进制格式、分片、缓存和高优先级线程能把 I/O 从瓶颈变成稳定的数据流。将数据集转换为二进制或内存映射格式，在存储系统与节点间进行分片，并提高线程优先级或将简单增强移到 GPU，以避免停顿。缓存热数据/KV 状态，积极预取和缓冲，并设置批大小，让从磁盘到设备的流水线保持平滑。以下是改进数据处理的一些建议：

使用二进制数据格式 将数据集转换为 TFRecords、LMDB 或内存映射数组等二进制格式。转换能减少处理数百万小文件的开销，加快数据摄取。

调节文件系统 除使用 `noatime` 挂载文件系统并增大预读外，还可考虑将数据分片到多个存储节点，以分散 I/O 负载，避免单个服务器成为瓶颈。

对 CPU 受限工作负载禁用超线程 对于严重受 CPU 限制的数据流水线，禁用超线程可以减少资源争用，带来更一致的性能。单线程性能关键的系统尤其受益。

提高线程优先级 使用 `chrt` 或 `pthread_setschedparam` 等工具，提高数据加载器和预处理 CPU 线程的调度优先级。给这些线程更高优先级，可确保数据以最小延迟供给 GPU，降低流水线停顿的可能。

缓存频繁使用的数据 利用操作系统页缓存或专用 RAM 磁盘缓存频繁访问的数据。这对 NLP 等某些 token 或短语会重复访问的应用特别有益，可减少冗余处理和 I/O 开销。

预取并缓冲数据 始终在需要某批数据的迭代之前加载它。使用后台数据加载器线程或进程，例如带 `prefetch_factor` 的 PyTorch DataLoader。对于分布式训练，使用 DistributedSampler 确保每个进程获得唯一数据，避免冗余 I/O。

并行化数据变换 如果图像增强、文本分词等 CPU 预处理很重，就将其分配到多个 worker 线程/进程。进行性能分析，确保 GPU 等待时 CPU 不是瓶颈。如果是，则增加 worker，或把部分变换移到 GPU；NVIDIA 的 DALI 等库可以异步在 GPU 上执行图像操作。

缓存模型状态与输出 使用 LLM 推理时，缓存频繁出现 token 的 embedding 和 KV cache 很有益，可避免反复重新计算。类似地，如果 LLM 训练作业多次复用同一数据集（称为多个 epoch），应利用操作系统页缓存或 RAM 存储热数据。

跨节点分片数据 多节点训练时，为每个节点分配数据子集，避免每个节点从单一源读取整个数据集。这能扩展 I/O。使用分布式文件系统，或手动分片分配，让每个节点读取不同文件。这会加速处理，并自然地与数据并行对齐，因为每个节点处理自己的数据分片。DeepSeek 的 Fire-Flyer File System（3FS）是分布式数据集分片文件系统的一个例子。DeepSeek 的 3FS 通过把数据集分片分布到每个节点的 NVMe SSD 上，同时尽量减少传统缓存，实现每秒多 TB 吞吐量。这种设计为每个 GPU 提供本地高速数据，避免 I/O 瓶颈。

监控流水线并调节批大小 有时增大批大小会把更多工作放到 GPU 上，减少 I/O 频率，提高整体利用率——但它会影响收敛，因此只在一定范围内有效。反之，如果 GPU 经常等待数据且无法加速 I/O，可以减小批大小以缩短每次迭代、减少空闲时间，或对较小批次做梯度累积，使数据读取更加连续。找到让 GPU 几乎始终繁忙的平衡点。

在 GPU 上执行数据增强 如果增强很简单但应用于海量数据，例如添加噪声或归一化，值得放在 GPU 上进行，避免 CPU 饱和。数据加载期间 GPU 往往利用不足，因此加载后使用小型 CUDA kernel 做增强可能很高效。但要小心不要使流水线串行化；用流重叠批次 N+1 的增强与批次 N 的训练。利

用 NVIDIA DALI 等 GPU 加速库异步执行这些任务，有助于保持平滑、高吞吐的数据流水线。

关注端到端吞吐量（例如每秒 token 数） 请记住，如果数据流水线把吞吐量砍半，加速模型计算也没有帮助。始终进行端到端性能分析，而不只是孤立地分析训练循环。使用 Nsight Systems、Nsight Compute 测量 kernel 时间线和停顿，或使用 PyTorch profiler 进行框架层归因。比较合成数据与真实数据下的迭代时间，确定数据加载引入了多少开销。目标是相对理想情况的开销小于 10%。如果超过这一数值，就投入时间优化流水线；这通常能在训练中带来巨大的“免费”加速。

20.13 性能分析、调试与监控

无法优化未测量的对象；性能分析能揭示工作负载是计算受限、内存受限、I/O 受限还是网络受限，从而针对正确的修复方向。持续遥测和回归测试能在代码、驱动和数据演化时防止收益被侵蚀。具体而言，使用带 NVTX 的 Nsight Systems/Compute 和框架性能分析器，判断是计算受限、内存受限、I/O 受限还是通信受限。削减 Python 开销，关注利用率缺口，平衡各 rank 的工作，跟踪内存/网络/磁盘健康状况，并用性能回归测试和告警管控变更。使用以下指导来分析、监控和调试 AI 工作负载性能：

进行性能分析以寻找瓶颈并做根因分析 定期对训练/推理作业运行性能分析器。使用 NVIDIA Nsight Systems 获取 CPU 和 GPU 活动时间线，也可以使用 Nsight Compute 或 PyTorch profiler 深入分析 kernel 效率。确定作业是计算受限、内存受限，还是在等待 I/O/通信，并据此定位优化。例如，如果工作负载受内存限制，应关注减少内存流量，而不是实施计算受限优化。结合机器学习驱动的分析来预测并提前处理性能瓶颈，这有助于实时自动调节。使用 GPUDirect Storage 时，启用 GDS tracing，将 cuFile 活动与 kernel 空档关联起来。

消除 Python 开销 分析训练脚本，找出过多循环或日志记录等 Python 瓶颈，并用向量化操作或优化的库调用替代。减少 Python 开销有助于确保 CPU 不会成为整体系统性能中的隐性瓶颈。

测量 GPU 利用率与空闲间隙 持续监控 GPU 利用率、SM 效率、内存带宽使用率等。如果发现利用率周期性下降，应将其与事件关联。例如，每 5 分钟下降一次可能与保存检查点重合；这种模式指向错开检查点、使用异步刷新等优化机会。使用 DCGM 或守护模式的 `nvidia-smi` 随时间记录这些指标。

使用 NVTX 标记 使用 NVTX 范围或框架性能分析 API 为数据加载、前向传播、反向传播等不同阶段标记代码。这些标记会出现在 Nsight Systems 或 Perfetto 时间线上，帮助把 GPU 空闲时间或延迟归因到流水线的具体部分，也使得更容易向开发者说明代码的哪一部分需要关注。在 PyTorch 中，可以使用 `torch.profiler.record_function()`。

利用超越 PyTorch profiler 的 kernel 分析工具 对于性能关键 kernel，使用 Nsight Compute 检查占用率和吞吐量等 kernel 级指标，或使用 Nsight Systems 分析 GPU/CPU 时间线和重叠情况。检查实际占用率、内存吞吐量和指令吞吐量。关注内存带宽接近硬件上限等内存瓶颈迹象，这有助于识别内存受限工作负载。性能分析器的“Issues”部分通常会直接指出 kernel 是否内存受限或计算受限，以及原因。利用这些反馈指导代码变更，例如全局加载效率低时改进内存合并访问。

检查 warp（线程束）分歧 使用性能分析器查看 warp 是否发生分歧，因为它能显示分支效率和分歧分支指标。分歧意味着由于分支，warp 中有些线程处于非活动状态，从而损害吞吐量。如果分歧显著，应重新审视 kernel 代码，重构条件或数据分配，尽量减少 warp 内分歧，确保每个 warp 处理一致的工作。

验证负载均衡 在多 GPU 作业中跨 rank 做性能分析。有时一个 GPU（rank 0）会承担聚合统计和数据收集等额外工作，常常成为瓶颈。监控每个 GPU 的时间线；如果一个 GPU 持续落后，就分散额外工作。例如，可以让非零 rank 分担 I/O 和日志职责。确保所有 GPU/rank 的工作量相近，避免最慢 rank 拖慢其他 rank。

监控内存使用 随时间跟踪 GPU 内存分配和使用情况。确保不要接近 OOM，因为框架可能意外地把张量换出到主机，造成巨大减速。如果内存使用逐次迭代增长，可能已发现内存泄漏；此时使用 `torch.cuda.memory_summary()`

和 Nsight Systems 的 GPU 内存轨迹分析详细分配。在 CPU 侧监控分页，因为进程驻留内存（RES）不应显著超过物理 RAM。如果出现分页，减少数据集预加载大小或增加 RAM。

监控网络与磁盘 对于分布式作业，使用操作系统工具监控网络吞吐量和磁盘吞吐量，确保实际吞吐量符合预期。例如，100 Gbps 链路完全利用时应看到 12.5 GB/s（12.5 GB/s = 100 Gb/s ×8 bit/byte）。如果没有，网络可能是瓶颈或配置错误。同样监控训练节点上的磁盘 I/O；如果磁盘利用率达到 100% 且 GPU 空闲，可能需要更好地缓冲或缓存数据。

为异常设置告警 在生产或长时间训练场景中，为 GPU 错误（例如 ECC 错误、设备过热）等事件设置自动告警或日志。这有助于识别异常缓慢的迭代。例如，NVIDIA DCGM 可以监视健康指标；如果 GPU 开始节流或遇到错误，可以触发操作。这样能立即发现冷却故障导致节流等性能问题，而不是等作业完成后才发现。

进行回归测试 维护一组基准任务，每当软件发生变化就运行，包括 CUDA 驱动、CUDA 版本、AI 框架版本甚至训练代码的变更。将性能与先前运行结果比较，及早发现回退。驱动更新或代码变更无意中降低吞吐量并不少见；在标准工作负载上快速运行性能分析会突出问题，便于调查。例如，某个 kernel 可能意外地不再使用 Tensor Cores；这一定要查明。

20.14 GPU 编程与 CUDA 调优优化

让 kernel 与内存层级和硬件特性对齐，是获得巨大、持久收益的关键。融合、Tensor Cores、CUDA Graphs 和编译器路径（例如 `torch.compile` 与 OpenAI 的 Triton）能把启动开销转化为有用计算。针对内存层级进行优化：合并全局加载、分块到共享内存、管理寄存器/占用率，并将传输（例如 `cp.async`/TMA）与计算重叠。优先使用经过调优的库和 CUDA Graphs，利用 `torch.compile` 与 OpenAI 的 Triton 进行融合，并用 Roofline 分析以及 PTX/SASS 检查验证可扩展性。以下是 GPU 和 CUDA 编程优化的一些技巧与方法：

理解 GPU 内存层级 牢记 GPU 的分层内存结构：每线程寄存器、每 block/SM 的共享内存/L1 缓存、跨 SM 的 L2 缓存和全局 HBM。最大化高层级中的

数据复用。例如，使用寄存器和共享内存复用值，尽量减少对较慢全局内存的访问。优秀的 kernel 应确保绝大多数数据在寄存器中，或通过合并访问和缓存从 HBM 高效加载。

合并全局内存访问 确保同一 warp 中的线程访问连续内存地址，使硬件尽可能用少量事务服务它们。warp 线程的跨步或散乱访问会导致每个 warp 产生多次内存事务，实际浪费带宽。重构数据布局或索引计算，使 warp 每次加载数据时都能通过一次宽内存事务完成。

使用共享内存复用数据 共享内存就像手动管理的高带宽缓存。将矩阵分块等频繁使用的数据加载到共享内存，让线程在移向下一块前多次处理这些分块。常见的分块技术能大幅减少全局内存流量。注意共享内存 bank 冲突；组织共享内存访问模式或填充数据，确保线程不会争用同一内存 bank，否则访问会串行化并降低性能。

优化内存对齐 只要可能，将数据结构按 128 字节对齐，尤其是批量内存复制或向量化加载。即使理论上能合并，未对齐访问也可能迫使多个事务。使用 `float2` 和 `float4` 等向量化类型，有助于每条指令加载/存储多个值，但要确保数据指针按向量大小正确对齐。

最小化内存传输 只在必要时将数据传输到 GPU，并且使用大块传输。能合并就把许多小传输合成一个大传输。例如，每次迭代要发送许多小数组时，将其打包成一个缓冲区后一次发送。小而频繁的 `cudaMemcpy` 会成为瓶颈。如果使用 Unified Memory，显式预取（`cudaMemPrefetchAsync`）在需要前将数据放到 GPU 上，避免关键计算阶段发生按需页面错误。

避免过多临时分配 频繁分配和释放 GPU 内存会损害性能。例如，频繁使用 `cudaMalloc`/`cudaFree` 或 kernel 中的 device malloc 会产生额外开销。应复用内存缓冲区，或使用大多数深度学习框架（如 PyTorch）提供的内存池；它们实现了 GPU 缓存分配器。编写自定义 CUDA 代码时，可使用带内存池的 `cudaMallocAsync`，或自行管理临时内存池，避免重复分配/释放的开销。

平衡线程与资源使用 实现良好的占用率—资源平衡。更多线程可提高占用率、隐藏内存延迟，但若每线程使用过多寄存器或共享内存，占用率会下降。调节 kernel 启动参数，包括每 block 的线程数，确保有足够 warp 在飞行以

覆盖延迟，但又不至于让每线程缺少寄存器或共享内存。对于指令级并行（ILP）高的 kernel，减少寄存器使用以提高占用率，反而可能损害性能。最优点通常位于占用率谱的中间，最大占用率并不总是理想。使用 NVIDIA Nsight Compute Occupancy Calculator 试验配置。

监控寄存器和共享内存使用 使用 Nsight Compute 等性能分析工具持续监控每线程寄存器和共享内存消耗。如果观察到占用率低于 25%，考虑增加每 block 的线程数，以更好利用可用硬件资源。但要确认不会造成过多寄存器溢出：查看详细占用率报告和 kernel 执行指标。寄存器溢出会导致额外内存流量，降低整体性能。

重叠内存传输与计算 只要可能，就将内存传输与计算重叠。使用多个 CUDA 流中的 `cudaMemcpyAsync`，在 kernel 运行时预取。对大块移动到共享内存，优先使用 Tensor Memory Accelerator；对细粒度分阶段复制和预取，使用 `cp.async`。这些方法通过将数据传输与计算重叠，有效掩盖全局内存延迟，确保 GPU 核心不会等待内存操作完成。

尽可能使用批量预取 对于可预测模式，使用 PTX `cp.async.bulk.prefetch.tensor.[1-5]d.L2.global*`（或 `prefetch.global.L2` 系列）预取到 L2，并使用 TMA（例如 `cp.async.bulk.tensor`）将块分阶段放入共享内存。还可以使用 `cp.async` 将全局内存异步分阶段加载到共享内存，并将复制与计算重叠；也可以提前将数据显式加载到寄存器。主动方法能减少全局内存访问造成的延迟，确保关键数据在需要时位于寄存器或共享内存等更快、低延迟存储中，从而减少执行停顿并提高 kernel 效率。

利用 cooperative groups 利用 CUDA 的 cooperative groups，在一部分线程之间实现高效的局部同步，而不是强制整个 block 范围的屏障。这能实现更细粒度的同步控制，减少不必要的等待和开销。将共享数据或执行相关计算的线程分组，只同步需要协调的线程，可以获得更高效的执行模式和更好的总体吞吐量。

优化 warp 分歧 组织代码，使 warp 内线程尽可能遵循相同执行路径。分歧会使该 warp 的执行时间翻倍，例如一半 warp（16 个线程）走一个分支，另一半（16 个线程）走另一个分支。如果某些分支很少被数据触发，可对数

据“排序”或分组，使 warp 处理一致情况，全为真或全为假。使用 ballot、shuffle 等 warp 级原语，为某些问题创建无分支方案。把 warp 视为工作单元，力求全部 32 个线程锁步执行相同工作，以获得最大效率。

利用 warp 级操作 适当时使用 CUDA 的 warp intrinsic，让线程无需进入共享内存即可通信。例如使用 `__shfl_sync` 向 warp 中所有线程广播一个值，或执行 warp 级归约（如跨 warp 求和寄存器），而不是让每个线程写入共享内存。这些 intrinsic 绕过较慢的内存，可加速能在 warp 内完成的归约或扫描。将任务放在 warp 内处理，可以避免共享内存和整 block 同步的延迟。

使用 CUDA 流实现并发 在单进程/GPU 内，如果独立 kernel 不占用全部资源，就在不同 CUDA 流中启动它们以重叠执行。也可以重叠计算与计算，例如一个流计算模型的一部分时，另一流启动 GPU 数据预处理等独立 kernel 或异步 memcpy。注意依赖关系，必要时使用 CUDA event 同步。正确使用流可以提高 GPU 利用率，避免资源空闲，特别是当某些 kernel 较轻时。

优先使用库函数 只要可能，就对核心数学和 collective 操作使用 NVIDIA 的优化库，例如 cuBLAS、cuDNN、Thrust 和 NCCL。分布式推理中的 GPU 点对点数据移动可在适用时使用 NIXL；需要细粒度 GPU 发起传输时，也可以使用 NVSHMEM。这些库针对每种 GPU 架构进行了高度优化，往往接近理论“光速”峰值，能免去重复发明轮子的麻烦。例如，矩阵乘法应使用 cuBLAS GEMM，而不是自定义 kernel，除非存在非常特殊的模式。库也能透明处理新的硬件特性；PyTorch（及其编译器）等 AI 框架在底层使用这些优化库。

对重复启动使用 CUDA Graphs 如果静态训练循环被启动数千次，考虑使用 CUDA Graphs 捕获并以图的形式启动操作序列。这能显著减少每次迭代的 CPU 启动开销，尤其在多 GPU 场景下，启动大量 kernel 和 memcpy 会给 CPU 施加额外压力并增加延迟。

检查可扩展性限制 调优 kernel 时，定期检查它随问题规模和跨架构的扩展情况。kernel 可能在小输入上获得很高占用率和性能，但扩展到大输入时表现不佳，因为它可能开始冲刷 L2 缓存或遇到内存缓存逐出。使用 Roofline 分析，将实际 FLOPS 和带宽与硬件上限比较，确保没有把性能留在桌面上。

检查 PTX 和 SASS 进行高级 kernel 分析 对于性能关键的自定义 CUDA kernel，使用 Nsight Compute 检查生成的 PTX 和 SASS。这种深度分析可以揭示内存 bank 冲突或冗余计算等问题，引导针对性的底层优化。

使用 PyTorch 编译器 利用 PyTorch 的 `torch.compile`，通过 TorchInductor 将 Python 层操作融合成优化 kernel。编译器也能通过集成 CUDA Graphs 减少启动开销。优化预热后，典型收益约为 10%–40%，因为它消除了解释器开销并解锁编译器级优化。实践中，启用 `torch.compile` 通过自动组合 kernel、更高效地利用 NVIDIA GPU 硬件（例如 Tensor Cores），在许多模型上产生了显著加速（例如 20%–50%）。始终在自己的模型上测试编译模式；它可能大幅提高吞吐量，但部署前要确保兼容性和正确性。图稳定后启用 CUDA Graphs，减少每次迭代的 CPU 开销，并保持静态内存池以满足指针稳定性约束。

为动态形状做规划 如果输入大小变化，使用 `torch._dynamo.mark_dynamic()` 标注动态维度，或用 `torch.export()` 导出形状多态图后再编译。使用 `torch.compiler.set_stance()`，配合 `fail_on_recompile` 和 `torch._dynamo.error_on_graph_break()`，控制重新编译行为，并在测试和 CI 中暴露有问题的形状变化。只要可能使用静态形状，启用 CUDA Graphs 以减少每次迭代的 CPU 开销。

通过 torch.compile 利用 Triton kernel 如果 PyTorch 没有很好地融合某个操作，可考虑用 Triton 编写自定义 GPU kernel 并集成。PyTorch 可通过 `torch.library.triton_op` 轻松注册自定义 GPU kernel。

可用时使用自动调优 启用库的自动调优功能，最大化底层性能。例如，输入大小固定时设置 `torch.backends.cudnn.benchmark=True`，让 NVIDIA 的 cuDNN 库尝试多个卷积算法并为硬件选择最快的一个。一次性开销会换来优化 kernel，加速训练与推理。如果不要求完全可复现，可通过禁用 `cudnn.deterministic` 允许非确定性算法，以解锁更快实现。

利用只读路径 将频繁使用的常量或系数标记为只读，让 GPU 将其缓存到专用 L1 只读缓存。在 CUDA C++ 中，可以使用 `const __restrict__` 指针提示数据不可变。在现代 GPU 架构上，编译器会为带 `const __restrict__` 限定符的指针生成带缓存的全局加载。使用 AI 框架和库时，确保查找表

或静态权重位于设备上并作为常量处理。这能减少这些值的全局内存流量与延迟，因为每个 SM 都能从缓存快速获取它们，而不必反复访问慢速 DRAM。

20.15 kernel 调度与执行优化

启动开销和不必要的同步会产生空闲间隙，严重压低吞吐量。融合小 kernel，并使用持久化/动态策略，可保持设备繁忙并隐藏延迟。通过最小化同步、融合小 kernel，以及在反复启动相同工作时使用持久化 kernel，让设备保持繁忙。对于不规则任务，可考虑 GPU 动态并行，但应谨慎使用，避免增加开销。以下是改进 kernel 调度与执行的一些建议：

最小化 GPU 同步调用 避免不必要的全局同步阻塞 GPU 进度。过度使用 `cudaDeviceSynchronize()` 或同步内存复制等阻塞 GPU 操作，会插入 CPU 与 GPU 都无法做有用工作的空闲间隙。只在绝对需要时同步，例如传输最终结果或调试时。让异步操作排队，可以保持 GPU 繁忙并让 CPU 空闲来准备后续工作，从而形成更连续的执行流水线。

融合小 kernel 以摊销启动开销 如果有许多很小的 GPU kernel 连续启动，应尽可能合并它们，在一个 kernel 中执行。每次 kernel 启动都有几十微秒量级的固定成本，因此通过手写 CUDA kernel 融合、XLA 融合，或使用 NVIDIA CUTLASS/Triton 等工具实现自定义算子，可以提高吞吐量。融合 kernel 把更多时间用于实际工作，减少启动开销和内存往返。这对推理或预处理流水线尤其有用，其中一串逐元素操作可以一次执行。先尝试 `torch.compile(mode="reduce-overhead")`；编译器可以融合操作链，并用 CUDA Graphs 包装稳定区域，减少 CPU 启动开销。对于未融合的热点，考虑迁移到 Triton kernel，并在适用时使用异步 TMA 和自动 warp 专门化。

利用 GPU 动态并行进行 GPU 内部工作调度 利用 CUDA Dynamic Parallelism，让 GPU kernel 无需返回 CPU 即可从 GPU 启动其他 kernel。在不可预测或迭代式工作场景中，例如需要根据中间结果生成额外任务的算法，动态并行通过移除 CPU 启动瓶颈降低延迟。例如，父 kernel 可以直接在设备上划分并启动子 kernel 进行后续处理。这样整个工作流留在 GPU 上，避

免 CPU 干预，实现更好的重叠和利用率。但要谨慎使用，因为过度使用会引入自身开销。

对重复工作负载使用持久化 kernel 当工作负载需要快速连续启动相同 kernel，例如处理工作队列或用相同计算处理流式批次时，使用持久化 kernel 策略。持久化 kernel 只启动一次并保持活跃，通过循环复用线程处理许多工作单元，而不是为每个单元启动新 kernel。这以更复杂的 kernel 设计换取显著更低的调度开销。保持 kernel 存活可避免重复启动成本，并实现更高的持续占用率。高性能分布式训练和推理系统经常使用此技术，最大化吞吐量、最小化迭代任务延迟。

评估线程块集群 使用线程块集群保持数据邻近并减少重新启动开销。在 Blackwell 上，最多 16 个线程块可以组成一个集群（提高非可移植限制后）。在持久化风格设计中使用集群感知同步和共享内存驻留来改善局部性。使用 Nsight Compute 等 kernel 级性能分析工具，分析占用率与驻留之间的权衡。

20.16 算术优化与降精度/混合精度

更低精度和稀疏性可以用比特数换取巨大的速度与内存收益，而且准确率影响通常可以忽略。混合精度、TF32/FP8/INT8 以及融合缩放能够利用硬件数学路径，提高每美元吞吐量。具体而言，使用混合精度（BF16/FP16）和 Tensor Cores 获得大幅收益，采用 TF32 轻松加速 FP32，并在质量允许时评估 FP8/FP4。利用结构化稀疏性、低精度梯度/通信以及推理用 INT8/INT4 量化，并融合缩放/激活以保持准确率。以下优化技术适用于提升算术计算性能以及利用降精度/混合精度：

使用混合精度训练 利用 FP16 或 BF16 加速数学运算并减少内存使用。现代 GPU 的 Tensor Cores 可大幅加速 FP16/BF16 矩阵操作。保留最终累加等关键部分，或保留一份 FP32 权重以保证数值稳定性，但让大部分计算使用半精度。这通常带来约 1.5×–3.5× 加速（取决于模型和 kernel 组合，矩阵乘密集型工作负载收益更大），且准确率损失很小；如今多数框架都通过自动混合精度（AMP）支持该做法。

采用梯度累积与激活检查点 详细使用梯度累积，在不增加内存使用的情况下有效增大批大小，并考虑使用激活检查点降低很深网络的内存占用。在训练接近或超过 GPU 内存限制的模型时，这些技术至关重要。

在新硬件上优先 BF16 而非 FP16 如果可用，使用 BF16 而非 FP16，因为它的指数范围更大且不需要损失缩放。现代 GPU 以与 FP16 相同的速度支持 BF16 Tensor Cores。BF16 无需处理上溢/下溢问题即可简化训练，同时获得半精度的性能收益。

利用 FP8、新型精度与缩放技术 在现代 GPU 上，FP8 Tensor Cores 在计算受限 kernel 上的数学吞吐量大约是 FP16 或 BF16 的两倍，同时降低激活和权重带宽。FP4（NVFP4）Tensor Cores 的吞吐量又是 FP8 的两倍，并通过微张量缩放（维持准确率的误差校正技术）用于推理，提高 token 吞吐量。训练时，使用 NVIDIA Transformer Engine 的 FP8，并在需要时保留 FP16 或 FP32 累加器。推理时先评估 FP8，只有在校准显示任务质量可接受后再采用 NVFP4。训练建议使用混合 FP8：前向激活/权重使用 E4M3，梯度使用 E5M2；也就是前向传递使用 E4M3，反向传递使用 E5M2。延迟缩放窗口使用 256–1024 往往有益。推理时可在校准后考虑 NVFP4。TE 与 PyTorch 集成，并受现代 GPU 硬件支持。优先使用框架的 TE kernel，而不是临时 FP8 自定义操作。端到端加速取决于 kernel 组合、内存带宽和校准；应在自己的模型和工作负载上验证准确率与性能。

利用 Tensor Cores 与 Tensor Memory Accelerator（TMA） 确保自定义 CUDA kernel 在可能时使用 Tensor Cores 执行矩阵操作；为简化实现，可以使用 CUTLASS 模板。使用 Tensor Cores 和 TMA 异步将张量移动到共享内存，可为 GEMM、卷积和其他张量操作带来显著加速，常接近 GPU 峰值 FLOPS。确保数据按需采用 FP16/BF16/TF32，并按 Tensor Core tile 维度对齐，维度是 8 或 16 的倍数。

使用 TF32 轻松加速 对于 32 位矩阵乘法，在数值安全的 PyTorch 操作中设置 `torch.set_float32_matmul_precision("high")` 以启用 TF32（快速 FP32）。cuBLAS 和 cuDNN 等库会在现代 GPU 硬件上自动选择最优 Tensor Core 代码路径。如果使用“highest”而非“high”强制完整精度 FP32，要理解其性能影响。

利用结构化稀疏性 现代 NVIDIA GPU 支持矩阵乘中的 2:4 结构化稀疏性，以结构化模式将 50% 的权重置零，使硬件吞吐量翻倍。通过剪枝模型利用这一点。如果能将权重剪枝为 2:4 稀疏模式，相应层的 GEMM 可实现约 2× 加速。使用 NVIDIA SDK 或库支持应用结构化稀疏性，并确保使用稀疏 Tensor Core 路径。如果模型能够容忍或通过稀疏性训练，这可以带来免费加速，但通常需要使用稀疏正则化重新训练。

可用时降低梯度和激活精度 即使权重保持较高精度，也可考虑将梯度或激活压缩为较低精度。例如对梯度通信使用 FP16/BF16 或 FP8。许多框架支持 FP16 梯度 all-reduce。同样，激活检查点使用 16 位而非 FP32 存储激活，可节省内存。FP8 和 FP4 优化器及量化梯度仍在研究中；它们能在降低内存和带宽成本的同时保持模型质量。在带宽受限环境中，梯度压缩尤其可能改变局面。DeepSeek 通过压缩梯度在受限 GPU 上训练，展示了这一点。

为推理使用自定义量化 部署时尽可能使用 INT8 量化。GPU 上的 INT8 推理极快且节省内存。使用 NVIDIA TensorRT 或量化工具把模型量化为 INT8 并校准。许多 transformer 神经网络能以 INT8 运行且准确率下降可忽略。相对于 FP16，加速可达 2–4×。在最新 GPU 上，还应探索并评估某些模型的 FP8 或 INT4，以进一步提高推理吞吐量。

可用时融合缩放与计算操作 使用低精度时，要记得融合操作以保持准确率。例如，Blackwell 的 FP4“微缩放”建议为每组值保留一个缩放因子。将这些融合操作纳入一次遍历，在同一遍中完成缩放和计算，而不是分开遍历，否则可能损失精度。现有库已经处理了许多这类操作，因此直接使用它们，不要从头实现。

20.17 高级调优策略与算法技巧

算法层面的改变通过减少工作，往往比硬件升级更有投资回报率，而不是单纯让工作跑得更快。自动调优、FlashAttention、通信/计算重叠和分片能解锁规模并减少浪费。具体而言，自动调节 kernel 与层参数，换用融合/FlashAttention kernel，并在分布式训练中重叠通信与计算。用流水线/张量并行和 ZeRO 分片扩展深度模型，并考虑异步更新或剪枝/稀疏性，以少量准确率工作换取巨大的吞吐量收益。以下是一些高级性能优化和算法技巧：

自动调节 kernel 参数 针对目标 GPU 自动调优自定义 CUDA kernel。正确的 block 大小、tile 大小、展开因子等会影响性能，而且最优设置常因 GPU 世代（Ampere、Hopper、Blackwell 及后续）而异。使用自动调优脚本或 OpenAI Triton 等框架，甚至在预处理步骤中进行穷举搜索，寻找最佳启动配置。这很容易带来 20%–30% 的改进，是静态“合理”设置可能错过的收益。在自动调优循环中使用 Triton 特性，例如设置 `num_warps` 和 `num_stages`、启用自动 warp 专门化、测试异步 TMA 布局。共享内存分阶段时优先使用 tensor map descriptor API。迁移到不同硬件时重新进行 tile 形状基准测试，因为不同 GPU 世代的最佳选择不同。

在机器学习工作负载中使用 kernel 融合 利用深度学习库提供的融合 kernel。例如，启用融合优化器，将权重更新、动量等逐元素操作融合；同时使用融合的多头注意力实现和融合归一化 kernel。NVIDIA 库及 Transformer Engine、FasterTransformer 等开源项目为常见模式提供融合操作，例如融合 LayerNorm + dropout。这些操作减少启动开销，并更高效地利用内存。

利用 FlashAttention 等内存高效注意力 为 transformer 模型集成 FlashAttention 等高级算法。FlashAttention 以分块、流式方式计算注意力，避免物化大型中间矩阵，大幅减少内存使用并提高速度，尤其适合长序列。用 FlashAttention 替换标准注意力可以同时改善吞吐量和内存占用，使相同硬件支持更大的批大小或序列长度。

重叠通信与计算 在分布式训练中，只要可能就将网络通信与 GPU 计算重叠。例如进行梯度 all-reduce 时，每层梯度就绪后立即异步启动 all-reduce，同时下一层仍在进行反向传播。若实现得当，这种流水线可以完全隐藏 all-reduce

延迟。使用异步 NCCL 调用，或使用 PyTorch Distributed Data Parallel（DDP）等开箱即用支持重叠的框架库，确保 GPU 不会因等待网络而空闲。

对深度模型使用流水线并行 当模型大小迫使你使用张量并行或流水线并行跨 GPU 进行流水线化时，使用足够的微批次让所有流水线阶段保持繁忙。利用 NVLink/NVSwitch 在阶段间快速发送激活。使用交错调度重叠并减少流水线气泡。某些框架会自动完成这种调度。NVL72 fabric 在这里尤其有帮助，因为即使通信密集型流水线阶段也能以多 TB/s 速度交换数据，减少流水线停顿。

利用分布式优化器分片 采用 ZeRO（Zero Redundancy Optimizer）等节省内存的优化策略，将优化器状态和梯度等张量分片到 GPU 之间，而不是复制它们。这通过分配内存和通信负载，使模型能够扩展到极大规模。它能降低每 GPU 内存压力、避免换出到 CPU，并在分块完成时减少通信量，从而提高吞吐量。DeepSpeed 和 Megatron-LM 等框架提供此类分片。对大模型利用它，在不 OOM 或因换出而减速的情况下维持高速。

可能时异步训练 如果适用，考虑异步更新。例如使用陈旧随机梯度下降（SGD），工作节点不必总是彼此等待以共享更新。这能提高吞吐量，但需要仔细调节以免影响收敛。正确实施时，异步训练可以带来很大的性能收益。

纳入稀疏性与剪枝 大型模型往往存在冗余。训练期间使用剪枝技术引入稀疏性，在推理时（若支持，训练期间也可部分）利用它。现代 GPU 硬件支持加速稀疏矩阵乘（2:4），未来 GPU 可能扩展此特性。即使训练保持稠密、只在推理时剪枝，较小模型也会运行更快并使用更少内存，提高模型部署的成本效率。探索彩票假说、蒸馏或结构化剪枝，在缩小模型的同时保持准确率。

20.18 分布式训练与网络优化

在集群规模下，网络会成为限制因素。若不处理，网络会破坏线性扩展并抬高成本。RDMA/巨型帧、分层 collective、亲和性和压缩能保护带宽、降低延迟。可用时使用 RDMA（InfiniBand/RoCE）；如果使用以太网，调节 TCP 缓冲区、启用巨型帧并选择现代拥塞控制。调整 NIC/CPU 亲和性，调节 NCCL 线程/缓冲区

（支持时使用 SHARP/CollNet），压缩或累积梯度，并测试 fabric 以发现丢包或配置错误。按照以下指导优化多 GPU 和多节点模型训练等分布式环境中的网络：

可用时使用 RDMA 网络 为多节点集群配备 InfiniBand 或 RoCE，获得低延迟和高吞吐量。确保 NCCL 和 MPI 使用 RDMA 进行训练。NCCL 会自动检测 InfiniBand，并在可用时使用 GPUDirect RDMA。RDMA 绕过内核网络栈，相比传统 TCP 可显著降低延迟。如果只有以太网，则在支持 RDMA 的 NIC 上启用 RoCE，获得类似 RDMA 的性能。在 NVLink 域系统（NVL72、GB200/GB300 等）上，尽可能让 collective 留在 fabric 内，将主机网络保留给岛间链路。让 NCCL 拓扑提示与 NVLink/NVSwitch 域对齐。

使用以太网时调节 TCP/IP 栈 对于基于 TCP 的集群，增大网络缓冲区。提高 `/proc/sys/net/core/{r,w}mem_max` 和自动调优限制（`net.ipv4.tcp_{r,w}mem`），允许更大的发送/接收缓冲区，帮助 10/40/100 GbE 链路饱和。在所有节点和交换机上启用巨型帧（MTU 9000），减少每个数据包开销，改善吞吐量并降低 CPU 使用。对于广域或拥塞网络，还可考虑 BBR 等现代 TCP 拥塞控制。

为 NIC 分配 CPU 亲和性 将网络中断和线程绑定到与 NIC 位于同一 NUMA 节点的 CPU 核，避免网络流量的跨 NUMA 惩罚，并保持网络栈内存访问局部。检查 `/proc/interrupts`，使用 irqaffinity 设置确保例如 NUMA 节点 0 的 NIC 由 NUMA 节点 0 的核处理。这能提高网络性能和一致性，尤其是在高包速率下。

针对环境优化 NCCL 环境变量 为大型多节点作业试验 NCCL 参数。例如，将每 GPU 的 NCCL CPU 线程数 `NCCL_NTHREADS` 从默认的 4 增加到 8 或 16，以更高 CPU 使用量为代价驱动更高带宽。将每 GPU 的缓冲区大小 `NCCL_BUFFSIZE` 从默认 1 MB 增加到 4 MB 或更大，以改善大消息吞吐量。如果集群使用支持 SHARP 的交换机，安装 NCCL SHARP 插件并设置 `NCCL_COLLNET_ENABLE=1` 启用 CollNet，再按文档使用 `SHARP_COLL_LOCK_ON_COMM_INIT=1` 和 `SHARP_COLL_NUM_COLL_GROUP_RESOURCE_ALLOC_THRESHOLD=0` 等 SHARP 插件变量。只有 reduction 足够大且网络 fabric 支持 SHARP 卸载时，才应期待加速。

对慢网络使用梯度累积 如果网络成为瓶颈，是因为太多节点通过中等性能互连扩展，那么使用梯度累积，执行更少但更大的 all-reduce 操作。在同步前累积几个 minibatch，使每 N 个批次通信一次，而非每个批次通信。这以少量额外内存和模型准确率调优换取显著降低的网络开销。当增加 GPU 因通信成本而收益递减时，这尤其有帮助。

优化 all-reduce 拓扑 确保使用适合集群拓扑的最优 all-reduce 算法。NCCL 会自动选择环或树算法，但在每节点 GPU 由 NVLink 连接、节点间使用 InfiniBand 或以太网的混合互连上，分层 all-reduce 可能更有益。它先在节点内执行 all-reduce，再跨节点执行。大多数框架默认执行基于 NCCL 的分层聚合，但要通过性能分析验证。在传统 MPI 设置中，可以手动执行同样的两级归约：先节点内，再节点间。

避免网络过度订阅 在多 GPU 服务器上，确保 GPU 合并流量不会使 NIC 过度订阅。例如，8 个 GPU 在 all-reduce 期间很容易产生超过 200 Gbps 的流量，因此单个 100 Gbps NIC 会限制它。若每节点扩展到许多 GPU，考虑每节点多个 NIC 和 200/400 Gbps InfiniBand。同样，注意 NIC 与 GPU 共享同一 PCIe 根复合体时的 PCIe 带宽限制。

压缩通信 与单节点内存类似，考虑压缩网络传输数据。技术包括 16 位或 8 位梯度压缩、量化跨节点流水线传输的激活，甚至 sketching 等更特别的方法。如果网络是最慢组件，用略高的计算成本压缩/解压数据可能值得。NVIDIA NCCL 原生不压缩，但可以在框架中集成压缩（例如 Horovod 中的梯度压缩，或 PyTorch 中自定义 AllReduce hook）。这是 DeepSeek 成功的关键之一：压缩梯度以应对有限的节点间带宽。

监控网络健康 确保没有隐性问题妨碍分布式训练。检查丢包（会表现为重试或超时；InfiniBand 使用重发计数器，以太网检查 TCP 重传）。即使少量丢包也会因拥塞控制启动而严重降低吞吐量。使用带外网络测试（如 iPerf 或 NCCL tests）验证带宽和延迟是否符合预期；否则调查交换机配置、NIC 固件或 CPU 亲和性。

20.19 高效推理与服务

服务是成本与延迟的博弈——利用率的提升依靠编排和批处理，而不只是更大的 GPU。专用运行时、KV 缓存策略和预热能在不违反 SLO 的情况下保持高吞吐量。根据需求使用自动扩缩容、微服务和动态/连续批处理，让 GPU 保持繁忙而不违反延迟 SLO。使用专用运行时（vLLM、SGLang、TensorRT-LLM），利用 NIXL 和 KV 缓存卸载实现解耦服务，预热模型，并隔离资源以控制尾延迟。通过以下技术提升模型推理效率与性能：

高效编排动态资源 集成高级容器编排平台，例如结合自定义性能指标的 Kubernetes。这样可根据实时使用模式和吞吐量目标动态扩展并平衡工作负载。

采用无服务器推理架构 探索用于推理工作负载的无服务器架构和微服务设计，它们能高效处理突发流量，并在需求较低时缩容，降低空闲资源开销。

优化批处理与并发 对于推理工作负载，找到合适的批处理策略。优先使用动态或连续批处理，自动将传入请求组成批次。更大的批大小让 GPU 保持繁忙、提高吞吐量，但过大可能增加延迟。如果单个流未使用全部 GPU 资源，也可以并行运行多个推理流，例如并发两个推理批次，充分使用 GPU 的 SM 和 Tensor Cores。

利用 NIXL 进行分布式推理 在跨 GPU 或节点服务大模型时，使用 NVIDIA Inference Xfer Library，通过 RDMA 在预填充与解码 worker 之间流式传输 KV cache。NIXL 为解耦 LLM 推理集群中从预填充 GPU 向解码 GPU 流式传输 KV cache 提供高吞吐、低延迟 API，使用 GPUDirect RDMA 和最优路径，且不涉及 CPU。这能减少跨节点解耦预填充/解码服务的尾延迟。

必要时卸载 KV cache 如果 LLM 的注意力 KV cache 超过 GPU 内存，使用分层卸载。NVIDIA Dynamo 的 Distributed KV Cache Manager 将较少访问的 KV 页面卸载到 CPU 内存、SSD 或网络存储；TensorRT-LLM 和 vLLM 等推理引擎支持分页和量化 KV cache。复用缓存可降低内存压力和首 token 延迟。要验证端到端影响，因为卸载未命中会引入额外 I/O 延迟。这允许对原本会超过 GPU 内存的序列进行推理；得益于快速 NVMe 以及计算与 I/O 的重叠，性能影响很小。若预计提示词或对话很长，确保推理服务器配置使用这一功能；卸载到磁盘总比完全失败好。

高效服务模型 使用 vLLM、SGLang、NVIDIA Dynamo 和 NVIDIA TensorRT-LLM 等优化模型推理系统，以低延迟、高吞吐量服务大模型。它们应实现量化、低精度格式、融合、高度优化的注意力 kernel 以及其他技巧，以最大化推理期间 GPU 利用率。这些库还应处理张量并行、流水线并行、专家并行、上下文并行、投机解码、分块预填充、解耦预填充/解码和动态请求批处理等众多高性能特性。

针对尾延迟监控与调优 在实时服务中，平均延迟和（长）尾延迟（第 99 百分位）都重要。分析推理延迟分布。如果尾部很高，找出异常值原因，例如意外的 CPU 介入、垃圾回收（GC）暂停或过多上下文切换。将推理服务器进程绑定到特定 CPU 核，使其与噪声邻居隔离；必要时使用实时调度，获得更一致的延迟。

预热以避免冷启动延迟 将模型加载到 GPU 并运行几次虚拟推理来预热 GPU，避免真实请求首次到达推理服务器时产生一次性的冷启动延迟。

高效分区资源以保证服务质量（QoS） 如果在同一基础设施上运行训练与推理等混合异构工作负载，或运行不同架构的模型，应考虑资源分区，确保延迟敏感的推理获得优先级。这可能意味着专门留出一些 GPU 进行推理，或使用 MIG 给推理服务一个保证的 GPU 切片（当它不需要完整 GPU 但需要可预测延迟时）。如果可能，将推理与训练分开到不同节点，因为训练可能通过大量 I/O 或通信突发引入抖动。

利用 Grace CPU 进行推理预处理 在 Grace Blackwell 系统中，服务器级 CPU 可在与 GPU 相同的内存空间中极快地处理分词和批次整理等预处理。将这些任务卸载到 CPU，让它在共享内存中准备 GPU 可直接使用的数据。这能减少缓冲区复制，并利用强大的 CPU 处理推理流水线的一部分，让 GPU 专注于计算密集型神经网络计算。

针对边缘 AI 与延迟关键部署谨慎调优 将性能调优扩展到边缘，利用专用边缘加速器，优化中心服务器与边缘设备之间的数据传输协议。这将有助于为时间敏感应用实现超低延迟。

20.20 多节点推理与服务

解耦预填充/解码并分片模型，可以用更高占用率处理更大的上下文和更多用户。连续批处理以及分层内存/卸载，即使在长提示词和高并发下也能维持流动。具体而言，在设备间解耦预填充和解码，跨请求持续汇聚 token，并通过张量/流水线并行分片超大模型。对超长上下文增加分层内存/卸载，以很小的延迟换取更高容量，在不 OOM 的情况下服务更多请求。以下性能建议适用于多节点推理与服务：

解耦推理流水线 将推理工作流分成不同阶段，包括处理输入提示词并经过所有模型层的“预填充”阶段，以及逐 token 生成输出的迭代“解码”阶段。将这些阶段分配到不同资源，以便独立扩展。这种两阶段方法可防止较快任务被较慢任务拖成瓶颈。对于大语言模型，一种策略是让完整模型编码提示词，然后按阶段处理自回归解码，可以为每个阶段使用专用 worker。通过解耦流水线，GPU 能持续处理自己最擅长的任务部分，避免队头阻塞，即一个长生成任务阻塞其后的其他任务。

对 LLM 使用连续批处理 不要停留在简单请求批处理，使用连续批处理策略，在重负载下最大化吞吐量。传统动态批处理将传入请求分组并作为一个批次处理，以提高 GPU 利用率。连续批处理更进一步，实时动态合并和拆分不同请求中的 token 序列。vLLM 等系统实现 token 汇聚：一旦任一线程准备好生成下一个 token，就与其他就绪线程组成新批次。这能让 GPU 始终保持高占用率，大幅减少空闲期，带来显著更高的 token 吞吐量和更稳定的延迟，尤其适合为序列长度不同的许多并发用户服务。

在 GPU 与节点间高效分片模型 对于无法装入单个 GPU 内存的模型，使用模型并行推理，将模型划分到多个 GPU，甚至多个服务器。可以使用张量并行，将每层权重和计算拆分到设备；也可以使用流水线并行，将模型层拆成托管在不同 GPU 上的片段，并依次流过数据。模型分片会带来通信开销和额外延迟，因为数据必须在分片间流动，但它能够部署否则无法服务的万亿参数模型。确保 GPU 间具有 NVLink 或 InfiniBand 等高速互连，并在可能时重叠通信和计算。关键是平衡负载，让所有设备并行工作，不让任何单一阶段成为瓶颈。

为扩展上下文卸载内存 使用分层内存策略支持所需内存超过 GPU 可用量的推理工作负载。在服务超大模型或长序列上下文（如长多轮对话和大型文档）时纳入内存卸载。当 GPU 内存紧张时，可将旧的注意力 KV cache 条目或不常访问的模型权重等低频数据移动到 CPU RAM，甚至 NVMe 存储。现代推理框架能在需要时自动换出和取回这些张量。虽然这会给缓存未命中带来额外延迟，但能避免内存不足错误并处理极端情况。通过有计划地卸载和预取数据，可以用少量速度换取服务大工作集请求的能力，在内存受限下实现更好的总体吞吐量。

20.21 功耗与热管理

每瓦性能是一级指标——热节流或功耗节流会抹去调优收益并缩短硬件寿命。功耗上限、高效打包和主动散热可以稳定时钟，同时减少能源支出。跟踪每瓦性能和热特性，以及速度：对内存受限工作负载设置功耗上限或降低内存时钟，在吞吐量损失很小的情况下提高效率。主动管理散热，把作业合并到接近满负载运行 GPU，监控每 GPU 功耗，并在能够降低成本时根据能源价格/可再生能源安排作业。以下是管理 AI 系统功耗与热特性的一些建议：

可能时使用高效、环保的能源 将能源消耗与性能一起跟踪和优化。除管理功耗与热限制外，还要监控能源使用指标，考虑同时改善性能和可持续性的技术。例如，根据可再生能源可用性实施动态功耗限制或工作负载迁移，可以减少运营成本和碳足迹。这种双重关注既降低运营成本，也支持负责任、环保的 AI 部署。

监控热量与时钟 运行期间关注 GPU 温度和时钟频率。如果 GPU 接近热限制（某些情况下为 85°C），可能开始降低时钟，导致性能下降。使用 `nvidia-smi dmon` 或遥测查看时钟是否从最大值下降。如果检测到节流，改善散热、提高风扇速度、改善气流，或略微降低功耗上限，使其处于稳定热范围。目标是在没有热引起波动的情况下获得一致性能。

使用能源感知的动态功耗管理 现代数据中心越来越多地使用能源感知调度，根据实时能源成本和可再生能源可用性调整工作负载。纳入自适应功耗限制和动态时钟缩放，有助于优化每瓦吞吐量，同时减少运营成本和碳足迹。

针对每瓦性能进行优化 在功耗预算受限（或能源成本高）的多 GPU 部署中，考虑针对效率调优。许多工作负载，尤其是内存受限工作负载，可以在略微降低 GPU 时钟的情况下运行，性能损失很小但功耗明显降低。例如，如果 kernel 受内存限制，将 GPU 锁定在较低时钟可以节省功耗而不影响运行时间，提高每瓦吞吐量。使用 `nvidia-smi -pl` 测试几个功耗上限，查看每瓦吞吐量是否改善。对某些模型，从 100% 降到 80% 功耗上限，速度几乎相同，但功耗可降低 20%。

使用自适应散热策略 如果在散热或能源可用性变化的环境中运行，应与集群管理集成以调整工作负载。例如，如果成本因素重要，在一天中较凉爽的时段或可再生能源供应较高时安排重作业。有些站点会把非紧急作业排到夜间运行，因为电价较低。这不会改变单个作业的性能，但能显著降低成本。

合并工作负载 让 GPU 高利用率运行，而不是让许多 GPU 低利用率运行。就每瓦完成的工作而言，繁忙 GPU 比空闲或轻载 GPU 更节能，因为 GPU 忙碌时能更好摊销基础功耗。除非需要优化最短墙钟时间，否则在一个 GPU 上以 90% 利用率依次运行一个作业，可能优于两 GPU 各以 45% 利用率并行运行。规划调度，在不使用时关闭或使整节点空闲，不要让大量硬件低利用率运行。

高效配置散热 对于风冷系统，考虑在重负载运行期间将 GPU 风扇设置为较高固定速度，预先冷却 GPU。有些数据中心始终以最大速度运行风扇以提高一致性。确保数据中心进风温度符合规格；定期检查服务器 GPU 是否有灰尘或阻塞。堵塞的散热片会大幅降低散热效率。对于水冷系统，确保流量最佳且水温受控。

仔细监控功耗 使用工具监控每 GPU 功耗。`nvidia-smi` 报告瞬时功耗，有助于理解工作负载的功耗曲线。功耗峰值可能与某些阶段相关。例如，all-reduce 阶段可能计算负载和功耗较低，而密集层会使负载和功耗读数升高。了解这一点后，可以安排工作负载以平滑功耗。如果集群运行在受限电路上，这很重要；在功耗受限场景中，可能需要避免同一节点同时运行多个功耗尖峰作业，以免触发功耗限制。

提高长时间运行作业的韧性 如果运行数月的训练作业或 24/7 推理作业，应考虑热量对硬件寿命的影响。持续以 100% 功耗和热限制运行，可能会略微

增加长期故障风险。实际上，数据中心 GPU 就是为这种韧性设计的；但如果想更安全，以 90% 功耗目标运行可以在减速很小的情况下减少组件应力。这是在更长训练时间与更少硬件磨损之间的权衡，尤其适用于硬件要在很长时间内为多个项目复用的情况。

20.22 结论

把这份清单当作可重复的作战手册：进行性能分析，在正确层级调优正确的瓶颈，并在扩展前验证收益。通过有条理地应用这些实践——从操作系统和 kernel，到分布式通信和服务——你将在任何规模上获得快速、经济高效且可靠的 AI 系统。

这份清单虽然全面，却并不穷尽所有情况。随着硬件、软件和算法演进，AI 系统性能工程领域还会继续发展。这里列出的最佳实践也并非都适用于每种场景。但总体而言，它们覆盖了 AI 系统性能工程场景的广度；这些技巧汇集了多年优化 AI 系统性能所积累的大量实践智慧。

调优 AI 系统时，应系统地遍历本章列出的相关类别，并逐项执行清单。例如，确保操作系统已调优，确认 GPU kernel 高效，检查库是否正确使用，监控数据流水线，优化训练循环，调节推理策略，并平稳扩展。遵循这些最佳实践，就能诊断和解决大多数性能问题，从 AI 系统中提取最大性能。

还要记住，在大幅扩展集群之前，应先在较少节点上进行性能分析，识别潜在的扩展瓶颈。例如，如果在 8 个 GPU 上看到 all-reduce collective 已经占一个迭代的 20%，扩展到更大规模时只会更糟，尤其是超过单个计算节点或数据中心机架系统（如 Grace Blackwell GB200 和 GB300 NVL72，以及 Vera Rubin VR200 和 VR300 NVL 系统）的容量时。

随手保留这份清单，并在发现新技巧时补充。将这些技巧和最佳实践与前面章节的深入理解结合起来，你就能设计和运行高效、可扩展、易维护、经济高效且可靠的 AI 系统。

现在，去让你最雄心勃勃的想法成为现实吧。祝优化愉快！

索引

A

激活检查点 *activation checkpointing* . . 18，642，1047

全归约 *all-reduce* . . . . . 4，553，1057

算术强度 *arithmetic intensity* . . 237，370，967

注意力机制 *attention mechanism* 14，604，1000

自动调优 *auto-tuning* . . . . 60，918，1050

自动混合精度 *automatic mixed precision* . 332，678，1046

B

带宽 *bandwidth* . . . . . 1，443，1052

批处理 *batching* . . . . . 17，783，1055

瓶颈 *bottleneck* . . . . . 1，572，1057

桶 *bucket* . . . . . 107，644，987

C

缓存 *cache* . . . . . 16，707，1055

缓存一致性 *cache coherency* 24，75，429

合并访存 *coalescing* . . . . . 280，321，819

集合通信 *collective communication* . . 188，641，891

计算受限 *compute-bound* 152，368，1046

容器 *container* . . . . . 17，95，1052

复制引擎 *copy engine* . . . . 84，500，632

cuBLAS *cuBLAS* . . . . . 56，394，1047

CUDA *CUDA* . . . . . 2，497，1049

CUDA 图 *CUDA Graphs* 303，614，1045

CUTLASS *CUTLASS* . . . . 27，408，1046

D

数据并行 *data parallelism* . 4，738，1036

解码 *decode* . . . . . 18，848，1055

DeepSeek *DeepSeek* . . . . . 1，392，1052

分布式数据并行 *distributed data parallel* . 135，553，571

分布式共享内存 *distributed shared memory* . . . . . 197，429，497

Docker *Docker* . . . . . 17，92，1031

动态批处理 *dynamic batching* . 767，781，1055

E

嵌入 *embeddings* . . . . . 9，537，982

专家并行 *expert parallelism* 4，736，1053

F

FlashAttention *FlashAttention* . . . 14，745，1049

完全分片数据并行 *fully sharded data parallel* . . . . . 18，136

融合 *fusion* . . . . . 2，644，1053

G

有效吞吐量 *goodput* ..... 15，793，1025

GPU *GPU* ..... 1，537，1057

GPUDirect RDMA *GPUDirect RDMA* . 18，502，1053

GPUDirect 存储 *GPUDirect Storage* .. 72，626，1037

Grace CPU *Grace CPU* ... 12，49，1054

梯度 *gradient* ..... 16，561，1052

H

高带宽内存 *HBM* ..... 8，366，1040

HBM3e *HBM3e* ..... 12，219，1026

主机内存 *host memory* ... 39，417，1034

混合并行 *hybrid parallelism* ... 740，758，913

I

推理 *inference* ..... 1，767，1057

InfiniBand *InfiniBand* .... 34，500，1055

指令级并行 *instruction-level parallelism* .. 193，341，1041

互连 *interconnect* ..... 1，219，1055

K

Kubernetes *Kubernetes* .... 17，98，1052

KV 缓存 *KV cache* ..... 18，844，1055

L

延迟 *latency* ..... 1，601，1055

负载均衡 *load balancing* 419，820，1038

循环分块 *loop tiling* 360

M

内存层级 *memory hierarchy* .... 14，245，1040

内存受限 *memory-bound* 153，372，1056

MIG *MIG* ..... 4，86，1054

混合专家 *mixture of experts* 4，736，1015

MPS *MPS* ..... 56，82，1032

多头潜在注意力 *multi-head latent attention* ..... 14，844

多实例 GPU *multi-instance GPU* ... 1030，1032

N

NCCL *NCCL* ..... 4，188，1052

NIXL *NIXL* ..... 4，188，1053

Nsight Compute *Nsight Compute* . 3，355，1046

Nsight Systems *Nsight Systems* ... 3，581，1039

NUMA *NUMA* ..... 56，95，1050

NUMA 绑定 *NUMA* 绑定 65

NVLink *NVLink* ..... 6，583，1055

NVLink-C2C *NVLink-C2C* 24，220，1026

NVSwitch *NVSwitch* ..... 12，737，1050

O

占用率 *occupancy* ..... 88，394，1055

操作强度 *operational intensity* . 359，521

内存不足 *out of memory* ... 4，79，1055

重叠 *overlapping* ..... 5，515，1055

P

PCIe *PCIe* ..... 24，299，1051

流水线并行 *pipeline parallelism* . 4，741，1055

预取 *prefetch* ..... 13，428，1055

预填充 *prefill* ..... 18，857，1055

性能分析 *profiling* ..... 2，571，1057

PTX *PTX* ..... 58，431，1043

Q

量化 *quantization* ..... 6，705，1053

R

RDMA *RDMA* ..... 6，188，1053

寄存器压力 *register pressure* .. 235，365，712

Roofline 模型 *roofline model* ... 237，359，567

路由 *routing* ..... 8，830，1033

S

分片 *sharding* ..... 18，689，1055

共享内存 *shared memory* 31，412，1054

流式多处理器 *SM* ..... 28，411，1052

稀疏性 *sparsity* ..... 10，606，1049

CUDA 流 *stream* ..... 106，507，1043

T

张量并行 *tensor parallelism* 4，891，1055

线程块 *thread block* ..... 31，427，1046

线程块簇 *thread block cluster* .. 197，435，566

吞吐量 *throughput* ..... 1，567，1056

TMA *TMA* ..... 192，442，1049

TMEM *TMEM* ..... 192，380，712

词元 *token* ..... 8，830，1055

拓扑 *topology* ..... 17，675，1051

Triton *Triton* ..... 2，700，1049

U

统一内存 *unified memory* 13，361，1026

利用率 *utilization* ..... 4，701，1057

V

向量化 *vectorization* .... 178，299，1040

vLLM *vLLM* ..... 13，807，1055

W

线程束 *warp* ..... 30，358，1049

线程束专用化 *warp specialization* ... 383，431，592

权重 *weight* ..... 11，803，1055

X

XLA *XLA* ..... 60，669，1045

Z

ZeRO *ZeRO* ..... 626，630，1049

零拷贝 *zero-copy* ..... 35，631，885

关于作者

Chris Fregly 是一位性能工程师与 AI 产品负责人，先后在 Netflix、Databricks、Amazon Web Services（AWS）以及多家初创公司推动创新。他带领以性能为核心的工程团队构建 AI/ML 产品、扩展市场推广计划，并降低大规模生成式 AI 与分析工作负载的成本。Chris 与人合著了《Data Science on AWS》和《Generative AI on AWS》（O'Reilly），并主讲了 O'Reilly 课程"High-Performance AI in Production with NVIDIA GPUs"。他的工作涵盖内核级调优、编译器驱动的加速、分布式训练与高吞吐推理。Chris 还组织了全球性的 AI Performance Engineering meetup，成员超过 10 万人。

版记

《AI 系统性能工程》封面上的动物是东北非洲猎豹（Acinonyx jubatus soemmeringii），又名苏丹猎豹。该亚种主要分布于非洲东北部，包括埃塞俄比亚、索马里和南苏丹的部分地区。

东北非洲猎豹与其他猎豹一样，拥有标志性的修长身躯与四肢，这使它能够达到惊人的速度（最高约 65–75 英里/小时）。与南方的近亲相比，它的泪痕（从眼角延伸到嘴边的黑色条纹）变化更多，皮毛也更厚。

猎豹以瞪羚、斑马和黑斑羚等中型有蹄动物为食，是技艺高超的猎手。它们的策略结合了潜行与爆发式的短距离冲刺，并会迅速吃完猎物，以免被狮子或鬣狗等更强的掠食者抢走。与许多大型猫科动物不同，猎豹不会咆哮，而是用呼噜声、啁啾声、嘶嘶声和低吼声等多种声音交流，在交配季节或雌豹与幼崽沟通时尤为多话。一胎通常有三到五只幼崽，幼崽与母亲共同生活一年半。雄性猎豹更善于社交，会组成松散的联盟，一起梳理毛发和捕猎。

由于栖息地丧失与偷猎，大多数猎豹物种被 IUCN 列为易危或濒危。O'Reilly 封面上的许多动物都濒临灭绝，它们对世界都很重要。

封面插图由 José Marzan Jr. 绘制，改编自 O'Reilly Media 的原始插图。系列设计由 Edie Freedman、Ellie Volckhausen 和 Karen Montgomery 完成。封面字体

为 Gilroy Semibold 与 Guardian Sans；正文字体为 Adobe Minion Pro；标题字体为 Adobe Myriad Condensed；代码字体为 Dalton Maag 的 Ubuntu Mono。